Bijou

Documentation Book

Design records, runtime contracts,
terminal surfaces and product evidence.

426 source documents / 32 TikZ diagram conversions

42 actual DOGFOOD component specimens

Jacquard 24 / IBM Plex Mono

20 October 2026

Preface

A terminal is a demanding place to build an application. Space is measured in cells. Input belongs to whichever surface has focus. A resize can change the useful shape of a page, and a different language can change the space its controls need. Colour, motion and cursor position help some readers; other readers encounter the same program through plain output, a screen reader, a script or an automated tool.

Bijou grows out of taking those conditions seriously. Its ambition is to make terminal software whose behaviour, structure and meaning remain understandable across those settings. The visible interface and the facts behind it should agree. Layout should own the geometry used for interaction. State changes and side effects should have clear owners. Host services should sit behind explicit boundaries. These principles shape both the code and the way the project explains its decisions.

This book brings that explanation together. It contains practical guides, architectural and design-language principles, records of proposed and completed changes, release and migration notes, retrospectives, audits, and rendered component specimens. The collection began as working repository documents. The editorial introductions provide a route through that collection while preserving the source records and their provenance.

An order for reading

Part I, Philosophy and architecture, establishes the ideas behind the system: runtime truth, explicit boundaries, humane interaction, graceful lowering, localization and inspectability. These chapters explain the constraints against which later choices should be judged.

Part II, Using Bijou, turns those ideas towards application work. Begin with the general Guide and the terminal-scenarios chapter, then follow the reference material needed for a particular task. The advanced guide, rendering material, package-development notes and component-family reference support deeper work. DOGFOOD is the documentation application; BlockLab is the standalone component workbench. Their presence throughout the book reflects the project's use of its own tools as a proving ground.

Part III, Design records, preserves the reasoning behind individual changes. Its introduction explains the recurring document format before the first record appears. The records are grouped by concern so a reader can follow a line of work across the documentation app, developer experience, runtime, design language, localization and delivery tooling.

Part IV, Releases and migration, follows changes into release-facing claims and upgrade guidance. Part V, Retrospectives and historical planning, records what happened during the work. Part VI, Audits and archive records, preserves

assessments, experiments and discarded directions. Together they let a reader compare an intention, an implementation claim and its later consequences.

Part VII, Rendered components and provenance, closes the book with actual component specimens and the inventory of original sources. It connects the written contracts to visible terminal surfaces and gives each source record a traceable identity.

Why the records have a recurring shape

A design document is useful when it makes a decision easier to examine. The recurring fields in this collection ask who needs a change, what useful outcome it should produce, where its boundaries lie, and what observations would establish success. Questions about accessibility, language, directionality and automated inspection bring different ways of using the same feature into that discussion early.

The resulting documents can look formal. Their value comes from the questions they expose: a named outcome can be challenged, a non-goal limits a promise, an invariant constrains an implementation, and a proposed test gives a reviewer something concrete to inspect. The introduction to the design-record part explains these fields in detail, including the distinction between tests proposed before implementation and evidence recorded after it.

The collection spans changes in the project’s working practices. Terms such as sponsor, hill, playback, cycle, lane and legend belong to that history. They help explain the records in which they appear. They do not impose the retired METHOD process on new work. Current operating agreements and the live issue tracker govern current execution.

Reading with evidence in mind

Each source chapter retains its original path and text. Its status fields and future-tense promises describe the record at the time it was written. A design can contain a sound argument without proving that the feature shipped; a release note can state a result without establishing that the result still holds in today’s code. Use linked issues, pull requests, commits and recorded checks to follow a claim into implementation. Read current tracker state from the tracker.

When building an application, use this book to understand the contracts and choices, then verify the exact API against the version being used. When reviewing a change, read its design, closeout evidence and subsequent history together. When looking for a subject such as Theme Lab or BlockLab, follow its named records and the part introductions instead of treating every mention as a complete operating guide.

The aim is a useful relationship between explanation and execution. A reader should be able to understand a choice, find the evidence behind it, and decide what still needs to be checked.

About this edition

This edition contains 426 original source documents: 196 design-directory records and 230 guides, references, strategy notes, invariants, release documents, audits and historical records. New editorial material consists of the preface, part introductions and reading guidance. Source chapter bodies remain preserved; their arrangement and displayed chapter numbers follow the book's reading order.

All 32 Mermaid source diagrams are rendered as native TikZ. The component atlas contains 42 vector projections generated from actual DOGFOOD Surface cells at fixed times, viewports, variants and a shipped theme. The atlas documents those captures; the original sources and their hashes are listed in the final appendix.

Jacquard 24 is used for titles and headings, with IBM Plex Mono for prose and code. Both are distributed by Google Fonts; their SIL Open Font Licenses accompany the book sources. DejaVu Sans supplies technical symbols absent from the primary fonts. The book's palette is anchored on #F7685F; component artwork retains its actual application colours.

The source corpus is a frozen record of the retired Markdown documents. Selected emoji markers have readable text labels in the typeset edition, and workstation paths are normalized where applicable. Source paths, hashes and historical links provide provenance. They do not substitute for current tracker state or a fresh verification of runtime behaviour.

Contents

Preface i

About this edition iii

I Philosophy and architecture 1

Introduction: The ideas behind the system 2

1 VISION 3

2 Bijou Vision 5

3 BIJOU UX DOCTRINE 12

4 ARCHITECTURE 14

5 ARCHITECTURE 17

6 System-Style JavaScript 18

7 Invariants 25

8 Strategy Notes 27

9 Accessibility and Assistive Modes 29

10 AI Explainability Standard 34

11 The Buffer Holds Facts 38

12 Codecs Are Not Domain Models 40

13 Commands Change State, Effects Do Not 41

14 ComponentStory v0 42

15 Content Guide 48

16 Docs Are the Demo 53

17 DOGFOOD 54

18 Focus Owns Input 63

19 Focus Owns Input 64

20 Graceful Lowering Preserves Meaning 72

21 Host APIs Stay Behind Adapters 73

22 The Humane Shell 74

23 Layout And Viewport Rules 87

24 Layout Localization Pipeline 91

25 Layout Owns Interaction Geometry 96

26 Localization and Bidirectionality 97

27 Low-Allocation Renderer 103

28 Notification History Belongs to the Shell 110

29 Preference Lists Belong to Bijou 119

30 Runtime Truth Wins 125

31 Schemas Live At Boundaries 126

32 Settings Belong to the Shell 127

33 Shell Owns Shell Concerns 135

34 State Machine and View Stack Are Distinct 136

35 Tests Are the Spec 137

36 Topmost Layer Dismisses First 138

37 Visible Controls Are a Promise 139

II Using Bijou 140

Introduction: From principles to an application 141

38 Guide — Bijou 142

39 End-To-End Terminal Scenarios 144

40 DOGFOOD 164

41 Package Development Guide 168

42 CLI 171

43 MCP 173

44 Advanced Guide — Bijou 176

45 Documentation Standards 181

46 Secondary Example Map 191

47 Migrating To Bijou v4.0.0 193

48 Bijou Specifacts 199

49 Render Pipeline Guide 203

Design system and reference 208

50 Bijou Design System 209

51 Blocks 213

52 Component Families 218

53 Data Visualization Policy 255

54 Foundations 260

55 Validate And Adapt Packed Bijou Cell Receipts 267

56 Patterns 271

57 Pointer and Mouse Policy 292

58 Inspect Profunctor Page Artifacts 295

59 Theme Authoring Guide 299

60 Theme Rule Recipes 306

61 Theme Token Vocabulary 309

III Design records 320

Introduction: How to read a design record 321

DOGFOOD design records 323

62 DF-001 — Show DOGFOOD Coverage Progress 324

63 DF-003 — Enforce DOGFOOD Coverage Floor 328

64 DF-004 — Raise DOGFOOD Coverage Floor to the Next 5-Point Target 332

65 DF-005 — Raise DOGFOOD Coverage Floor to 19% 337

66 DF-006 — Raise DOGFOOD Coverage Floor to 24% 342

67 DF-007 — Raise DOGFOOD Coverage Floor to 29% 347

68 DF-008 — Raise DOGFOOD Coverage Floor to 34% 352

69 DF-009 — Raise DOGFOOD Coverage Floor to 39% 357

70 DF-010 — Raise DOGFOOD Coverage Floor to 44% 362

71 DF-011 — Raise DOGFOOD Coverage Floor to 49% 367

72 DF-012 — Raise DOGFOOD Coverage Floor to 54% 372

73 DF-013 — Raise DOGFOOD Coverage Floor to 59% 377

74 DF-014 — Raise DOGFOOD Coverage Floor to 64% 382

75 DF-015 — Raise DOGFOOD Coverage Floor to 69% 385

76 DF-016 — Raise DOGFOOD Coverage Floor to 74% 388

77 DF-017 — Raise DOGFOOD Coverage Floor to 79% 391

78 DF-018 — Raise DOGFOOD Coverage Floor to 84% 394

79 DF-019 — Raise DOGFOOD Coverage Floor to 100% 397

80 DF-021 — Shape DOGFOOD As Terminal Docs System 400

81 DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav 407

82 DF-023 — Publish Repo, Package, And Release Guides In DOGFOOD 411

83 DF-024 — Publish Philosophy, Architecture, And Doctrine Guides In DOGFOOD 415

84 DF-025 — Make DOGFOOD The Only Human-Facing Docs Surface 418

85 DF-026 — Demote Examples To Secondary Reference Status 424

86 DF-027 — Storybook-Style Tool for Bijou 427

87 DF-028 Story Capture Matrix 431

88 DF-029 Fixture-to-Docs Promotion Path 434

89 DF-030 Dogfood Docs Surface Block 437

90 DF-031 Status and feedback standard Blocks 443

91 DF-036 Audit loading placeholders family across real surfaces 449

92 DF-039 Component-family standard Blocks 452

93 DF-041 Audit inspector panels family across real surfaces 458

94 DF-046 Choice and navigation standard Blocks 461

95 DF-049 Audit multi-field and staged forms family across real surfaces 467

96 DF-053 Audit motion and shader effects family across real surfaces 470

97 DF-054 Late-family standard Blocks 474

98 DF-060 Audit viewport masking and scrollable inspection panes family across real surfaces 480

99 DF-060 V7 DOGFOOD Release Title Screen And Closeout Six-Pack 483

100 DF-061 Audit overlay primitives family across real surfaces 489

101 DF-062 Audit notification system family across real surfaces 492

102 DF-063 Audit app shell family across real surfaces 496

103 DF-064 Audit keybinding help and shell hints family across real surfaces 499

104 DF-065 Audit workspace layout family across real surfaces 502

105 DF-066 Audit data visualization family across real surfaces 505

106 DF-067 Prove Responsive DOGFOOD Layout Variants 508

107 DF-069 - Block-Authored DOGFOOD 511

108 DF-070 - DOGFOOD Block Product Polish 515

109 DF-071 - DOGFOOD Block-Authored Surfaces 518

110 DF-072 DOGFOOD Release Title Artifact Gallery 521

111 DF-073 Raster-to-Glyph Title Art 526

112 DF-074 Fluid Triangle Title Screen 533

113 DF-075 DOGFOOD Post-Release Polish 541

114 DF-076 - BlockLab Demo Readiness Navigation 545

115 DF-077: DOGFOOD App Landing Extraction 547

116 DF-078 DOGFOOD Release Story Surfaces 551

117 DF-150 - DOGFOOD Terminal Guardrails 556

Developer experience design records 558

118 DX-001 — Type Framed App Messages and Updates End-to-End 559

119 DX-002 — Reconcile Cmd Typing With Cleanup and Effect Patterns 561

120 DX-022 Layout Inspector Overlay 563

121 DX-023 Input Routing Inspector 566

122 DX-024 Surface Diff Viewer 569

123 DX-025 Component Metadata Contract 572

124 DX-026 Mode-Lowering Linter 575

125 DX-030 Add boundary-aware pointer selection and copy 578

126 DX-031 Standard Bijou Blocks 586

127 DX-031F Block Tree Rendering 645

128 DX-032 Create TUI app skeleton renders consumer pages 650

129 DX-033 Remove showcase layout sludge 654

130 DX-034 Declarative View Data Binding 657

131 DX-036 Responsive Table Layout and Variants 678

132 DX-038 - BlockLab Rename And Migration 686

133 DX-039 - BlockLab Product Parity 691

134 DX-040 V7 Post-Merge Review Regression Fixes 699

135 DX-041 Image-to-Glyph Side App 702

136 DX-042 Shared UI Scene IR And Bijou Render Target 706

137 DX-043 Portable Bijou Blocks And Multi-Endpoint IR 719

138 DX-044 GraphQL Authored Bijou Block IR Proof 731

139 DX-045 GraphQL Block Groups And Debug Facts 738

140 DX-046 GraphQL Authored DOGFOOD Block Fixture 744

141 DX-047 - Blocks App Binding Snippets 750

142 DX-048 V8 Runtime Graph And Scene IR Contract 755

143 DX-049 VISOR Artifact Bundle Proof 761

144 DX-050-profunctor-page-inspection 768

145 DX-052 NO_COLOR Is Not a Capability 773

146 DX-125 - Day Zero Onboarding Bridge 778

Runtime engine design records 781

147 RE-001 — Define Runtime Engine Architecture 782

148 RE-002 — Promote First-Class State Machine and View Stack 787

149 RE-003 — Retain Layout Trees and Layout Invalidation 792

150 RE-004 — Route Input Through Layouts and Layer Bubbling 797

151 RE-005 — Buffer Commands and Effects Separately 802

152 RE-006 — Formalize Component Layout and Interaction Contracts 807

153 RE-007 — Migrate Framed Shell Onto Runtime Engine Seams 812

154 RE-008 — Rendering Performance Findings 821

155 RE-024 Surface Budget Warnings 825

156 RE-030 — Average High-Resolution Canvas Colors 828

157 RE-031 — Raytrace Shader Kernel 832

158 RE-032 — Fit Cell Glyphs From Coverage 836

159 RE-033 Stable Spring Pulse Integration 840

160 RE-034 Glyph-Fit Canvas Resolution 843

161 RE-035 Mandatory Layout Envelope And Constraint Negotiation 846

162 RE-036 Packed Bijou Cells Surface Adapter 863

163 RE-041 v7.2 Framework Input Stabilization 873

164 RE-127 - Command And Render Runtime Guardrails 877

165 RE-142 - Node Bootstrap Failure Contract 880

Design language design records 882

166 DL-001 — Capture Design Language Doctrine 883

167 DL-002 — Canonicalize Patterns and Blocks 888

168 DL-003 — Prove Canonical Patterns in Shared Surfaces 893

169 DL-004 — Prove Drawer Rhythm and Notice Rows 898

170 DL-005 — Prove Inspector and Guided Flow Rhythm 902

171 DL-006 — Prove Inspector Panel Rhythm 907

172 DL-007 — Promote Inspector Panel Block 912

173 DL-008 — Promote Guided Flow Block 917

174 DL-011 Theme Contrast Doctor 922

175 DL-012 Separate focus gutter chrome from scrollbar UI tokens 925

176 DL-013 Design token and theme builder API 928

177 DL-014 Theme Inspector And Lab 939

178 DL-015 Theme safe pairs and contrast matrices 944

179 DL-016 Mode-aware runtime theme adapter 949

180 DL-017 - DOGFOOD Light Theme Readiness 954

181 DL-018 - First-Party Theme Variant Coverage 959

182 DL-019 - Theme Rule Selection And Inspection 964

183 DL-020 - Dogfood Rule-Authored First-Party Themes 972

184 DL-021 - Dogfood Theme Token Usage Proof 976

185 DL-022 - Theme Lab Editor Graph UX 979

Localization design records 984

186 LX-001 — bijou-i18n Runtime Package 985

187 LX-002 — bijou-i18n Tools Package 990

188 LX-003 — Spreadsheet Adapters and Catalog Exchange Workflows 996

189 LX-004 — Provider Adapters for Workbook and Bundle Exchange 1002

190 LX-005 — Rich Spreadsheet and Filesystem Adapters 1007

191 LX-006 — XLSX Localization Adapters 1012

192 LX-008 — Localize Shell Chrome and DOGFOOD 1017

193 LX-019 — Localization Port Contract 1023

194 LX-020 - DOGFOOD Locale Demo Readiness 1028

195 LX-021 DOGFOOD String Debt Ratchet 1033

196 LX-022 DOGFOOD App Main Ratchet 1037

197 LX-023: DOGFOOD Blocks And Capture Ratchet 1041

Workflow and delivery design records 1045

198 WF-001 — Adopt the Legends/Cycles Workflow 1046

199 WF-003 — Replace smoke:examples:* With smoke:dogfood 1050

200 WF-004 — Shape The Next Release 1055

201 WF-005 — Close 4.1.0 i18n Publish-Surface Gap 1062

202 WF-006 — Cut Clean 4.1.0 Release Boundary 1067

203 WF-009 - Keep Release PRs Under Automated Review File Limits 1072

204 WF-122 - Cross-Platform Contributor Path 1075

205 WF-123 - Issues-First Method Intake 1078

206 WF-125 Draft-first cycle start workflow 1081

207 WF-127 V7 Issue-Complete Tracker Sync 1084

208 WF-128 Safe GitHub Comments And Milestone Item Mirrors 1087

209 WF-129 Path-Gate DOGFOOD CI 1091

210 WF-130 Roadmap Goalpost Policy 1094

211 WF-131 Roadmap Forward-Looking Refresh 1099

212 WF-132 - Release Readiness And DOGFOOD I18n Gates 1104

213 WF-133 - Esbuild Security Patch 1108

214 WF-134 Code Dojo ESLint Ratchet 1 1111

215 WF-135 Respecting the Dojo 1000-Count Burndown 1115

216 WF-136 Respecting the Dojo Ratchet To 2018 1119

217 WF-137 Respecting the Dojo Ratchet To 1965 1123

218 WF-138 Respecting the Dojo Ratchet To 1857 1127

219 WF-139 Respecting the Dojo Ratchet To 1796 1131

220 WF-140 Respecting the Dojo Ratchet To 1729 1135

221 WF-141 Respecting the Dojo Ratchet To 1679 1138

222 WF-142 Respecting The Dojo Ratchet To 1621 1141

223 WF-143 Respecting the Dojo Ratchet To 1564 1144

224 WF-144 Respecting the Dojo Ratchet To 1514 1147

225 WF-145 Respecting the Dojo Ratchet To 1461 1150

226 WF-146 Respecting the Dojo Ratchet To 1393 1153

227 WF-147 Respecting the Dojo Ratchet To 1319 1157

228 WF-148 Respecting the Dojo Ratchet To 1268 1160

229 WF-149 Respecting the Dojo Ratchet To 1216 1163

230 WF-150 Respecting the Dojo Ratchet To 1165 1167

231 WF-151 Respecting the Dojo Ratchet To 1110 1171

232 WF-152 Respecting the Dojo Ratchet To 1055 1175

233 WF-153 Respecting the Dojo Ratchet To 1004 1179

234 WF-154 Respecting the Dojo Ratchet To 953 1183

235 WF-155 Respecting the Dojo Ratchet To 624 1187

236 WF-156 Respecting the Dojo Ratchet To 573 1192

237 WF-157 Respecting the Dojo Ratchet To 520 1197

238 WF-158 Respecting the Dojo Ratchet To 468 1202

239 WF-159 Respecting the Dojo Ratchet To 415 1207

240 WF-160 Respecting the Dojo Ratchet To 364 1212

241 WF-161 Respecting the Dojo Ratchet 262 1217

242 WF-162 Respecting the Dojo Ratchet 162 1220

243 WF-163 Respecting the Dojo Ratchet 112 1224

244 WF-164 Respecting the Dojo Ratchet 62 1229

245 WF-165 Respecting the Dojo Ratchet 12 1234

246 WF-166 - V8 Dependency Security Closeout 1244

247 WF-167 Roadmap Milestone Authority 1249

Other design records 1260

248 RE-008 — Byte-Packed Surface Representation 1261

249 ASAP-001 — Fix MCP DAG Type Error 1264

250 AUDIT-001 — Mentor Audit of examples/showcase 1267

251 BE-001 Project Big Extraction: DOGFOOD Blocks Breakout 1270

252 HT-002 — Layered Focus and Interaction 1276

253 HT-003 — Implement Layer Stack and Input Map Routing 1282

254 HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection 1286

255 HT-007 Focus Map Surface 1290

256 HT-008 — Surface-native text bypasses the terminal sanitization boundary 1293

257 Design Records 1296

IV Releases and migration 1297

Introduction: From implementation to a release claim 1298

258 Bijou Workflow 1299

259 Bijou 7.1.0 Release Evidence 1301

260 Bijou 7.2.0 Release Evidence 1309

261 Bijou 8.0.0 Release Evidence 1319

262 Release Docs 1330

263 Migration Guide: Bijou v4.0.0 To v4.1.0 1332

264 Migrating to Bijou 4.2.0 1339

265 Migrating to Bijou 4.3.0 1341

266 Migrating to Bijou 4.4.0 1343

267 Migrating to Bijou 4.4.1 1344

268 Migrating to Bijou 5.0.0 1346

269 Migrating to Bijou 7.0.0 1348

270 Migrating to Bijou 7.1.0 1350

271 Migrating to Bijou 7.2.0 1352

272 Migrating to Bijou 8.0.0-rc.1 1354

273 Release 1357

274 What’s New In Bijou v4.1.0 1370

275 What’s New in Bijou 4.2.0 1376

276 What’s New in Bijou 4.3.0 1379

277 What’s New in Bijou 4.4.0 1381

278 What’s New in Bijou 4.4.1 1384

279 What’s New in Bijou 5.0.0 1386

280 What’s New in Bijou 7.0.0 1389

281 What’s New in Bijou 7.1.0 1392

282 What’s New in Bijou 7.2.0 1395

283 What’s New in Bijou 8.0.0-rc.1 1397

V Retrospectives and historical planning 1400

Introduction: Learning from the work 1401

284 CI-001 — mermaidSurface() Component 1402

285 CI-002 — Deterministic Time-Travel Debugger 1403

286 CI-003 — Surface Shaders (CRT / Scanlines) 1404

287 CI-003 — Surface Shaders (CRT / Scanlines) 1405

288 DAG — Layer-Skipping Edges Silently Occluded By Intermediate Nodes 1407

289 DF-002 — Expand DOGFOOD Component Family Coverage 1410

290 DF-020 — Deepen DOGFOOD Story Depth and Variant Quality 1411

291 DF-020 — Deepen DOGFOOD Story Depth and Variant Quality 1412

292 DF-002 — Expand DOGFOOD Component Family Coverage 1414

293 MCP interactive documentation for all Bijou components 1416

294 DL-008 — Promote Guided Flow Block 1418

295 DL-009 — Formalize Layout and Viewport Rules 1419

296 DL-009 — Formalize Layout and Viewport Rules 1421

297 DL-009 — Strictly Typed App Interface Examples 1422

298 DL-009 — Strictly Typed App Interface Examples 1424

299 DL-010 — Strictly Typed App Interface Examples 1425

300 DL-008 — Promote Guided Flow Block 1426

301 DX-003 — Rationalize Table APIs and Public Table Types 1428

302 DX-004 — Smooth Surface and String Composition Seams 1430

303 DX-005 — Polish Small Component and Import Ergonomics 1432

304 DX-006 — Debug Performance Overlay Component 1434

305 DX-007 — Decompose app-frame.ts Mega-Closure 1436

306 DX-007 — Decompose app-frame.ts Mega-Closure 1438

307 DX-007 — Project-Wide Numeric Input Sanitization 1440

308 DX-007 — Project-Wide Numeric Input Sanitization 1441

309 DX-008 — PerformanceObserver('gc') Is Unreliable For Our Measurement Needs 1443

310 DX-008 — Render Pipeline Documentation Guide 1446

311 DX-008 — Sanitize Wrapper for User-Facing Numeric Inputs 1447

312 DX-008 — Sanitize Wrapper for User-Facing Numeric Inputs 1448

313 DX-009 — Auto-Init Context and Actionable Error Messages 1450

314 DX-010 — Typed Sub-App Adapter Factory 1451

315 DX-011 — Key Collision Warnings at Registration Time 1452

316 DX-012 — Document Render-Path Naming Convention 1453

317 DX-013 — Motion Key Lifecycle Warning 1454

318 DX-014 — Fix Root README Interactive Example 1455

319 DX-015 — Perf Gradient as CI Benchmark 1456

320 DX-016 — Error Surface for Crash Post-Mortems 1458

321 DX-016 — Error Surface for Crash Post-Mortems 1460

322 DX-016 — Scenario Provenance Tags For Intelligent Bench Filtering 1461

323 DX-017 — Agent-First Bench Output Format 1464

324 DX-017 — One-Call Node Host Entry Point 1467

325 DX-017 — One-Call Node Host Entry Point 1468

326 DX-018 — Runtime Test Harness for TEA Apps 1470

327 DX-018 — Runtime Test Harness for TEA Apps 1471

328 DX-019 — Third-Party Theme Reactivity Hook 1473

329 DX-019 — Third-Party Theme Reactivity Hook 1474

330 DX-020 — Layout Localization Pipeline Guide 1476

331 DX-020 — Layout Localization Pipeline Guide 1477

332 DX-021 — Error Surface for Crash Post-Mortems 1479

333 DX-028 — Align front-door docs with repo truth 1480

334 DX-003 — Rationalize Table APIs and Public Table Types 1482

335 DX-004 — Smooth Surface and String Composition Seams 1484

336 DX-005 — Polish Small Component and Import Ergonomics 1486

337 DX-006 — Debug Performance Overlay Component 1488

338 DX-007 — Decompose app-frame.ts Mega-Closure 1491

339 DX-008 — Render Pipeline Documentation Guide 1493

340 DX-009 — Auto-Init Context and Actionable Error Messages 1495

341 DX-010 — Typed Sub-App Adapter Factory 1497

342 DX-011 — Key Collision Warnings at Registration Time 1499

343 DX-012 — Document Render-Path Naming Convention 1501

344 DX-013 — Motion Key Lifecycle Warning 1503

345 DX-014 — Fix Root README Interactive Example 1505

346 DX-015 — Perf Gradient as CI Benchmark 1507

347 HT-001 — Notification Center Polish and Discoverability 1509

348 HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection 1510

349 HT-006 — Allow pages to push notifications in createFramedApp 1511

350 HT-001 — Notification Center Polish and Discoverability 1513

351 HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection 1515

352 Upstream edge-overlay scrollbars and BCSS overflow handling 1517

353 Upstream edge-overlay scrollbars and BCSS overflow handling 1518

354 LX-007 — Service-Oriented Localization Adapters 1519

355 LX-009 — Localize Shell Help, Notification, and Directional Surfaces 1520

356 LX-010 — Built-in i18n Catalog Loader 1521

357 LX-010 — Built-in i18n Catalog Loader 1523

358 LX-012 — DOGFOOD i18n Debt Inventory 1525

359 LX-007 — Service-Oriented Localization Adapters 1527

360 LX-009 — Localize Shell Help, Notification, and Directional Surfaces 1529

361 RE-011 — Eliminate Per-Frame Allocation in Render Pipeline 1531

362 RE-012 — Pipeline Observability Hooks 1532

363 RE-013 — Memoize Workspace Layout Tree 1533

364 RE-014 — Design Doctrine 1534

365 RE-017 — Frame does not fill terminal background with surface.primary.bg 1535

366 RE-018 — Hex Color Strings As The Canonical fg/bg Representation 1537

367 RE-019 — Surface dirtyWords Bitmap Serves The Wrong Master 1540

368 RE-019 — Pre-Parse Theme Token Colors Into Bytes 1543

369 RE-020 — Typed Color Representation (ColorRef Branded Type) 1546

370 RE-020 — Typed Color Representation (ColorRef Branded Type) 1549

371 RE-021 — Frame Owns the Pump 1551

372 RE-021 — Frame Owns the Pump 1554

373 RE-022 — Public PackedSurface Type Guard 1557

374 RE-022 — Public PackedSurface Type Guard 1558

375 RE-022 — Unified Layout Calculation Engine 1559

376 RE-022 — Unified Layout Calculation Engine 1560

377 RE-023 — Stabilize Render Scheduling in runtime.ts 1562

378 RE-023 — Stabilize Render Scheduling in runtime.ts 1563

379 RE-024 — Terminal Content Sanitization Boundary 1565

380 RE-024 — Terminal Content Sanitization Boundary 1566

381 RE-025 — Scoped Node IO Adapter 1568

382 RE-025 — Scoped Node IO Adapter 1569

383 RE-026 — Drain runtime commands before shutdown 1571

384 Graveyard 1573

385 Legacy Backlog Archive 1574

386 Legacy ASAP Archive 1576

387 Legacy Bad-Code Archive 1577

388 Legacy Cool Ideas Archive 1578

389 Legacy Inbox Archive 1579

390 Legacy Up Next Archive 1580

391 v4.5.0 — Legacy Release Lane Snapshot 1581

392 v5.0.0 — Legacy Release Lane Snapshot 1582

393 Retro 1583

394 Retro 1584

395 RE-007 — Migrate Framed Shell Onto Runtime Engine Seams 1585

396 RE-011 — Eliminate Per-Frame Allocation in Render Pipeline 1587

397 RE-012 — Pipeline Observability Hooks 1589

398 RE-013 — Memoize Workspace Layout Tree 1591

399 RE-014 — Design Doctrine 1593

400 RE-016: DAG renderer should handle cycles gracefully 1594

401 RE-016 — graphemeWidth Miscounts Emoji-Like Icon Characters 1596

402 WritePort byte-level API for zero-alloc ANSI output 1598

403 WF-002 — Migrate Legacy Planning Artifacts 1599

404 WF-007 — Align Local Validation With CI typecheck:test Gate 1600

405 WF-008 — METHOD backlog helper generates malformed note filenames 1602

406 WF-122 - Day Zero Audit Fixes 1604

407 WF-002 — Migrate Legacy Planning Artifacts 1607

408 WF-007 — Align Local Validation With CI typecheck:test Gate 1609

409 Background fill regression in v4.4.0 1611

VI Audits and archive records 1613

Introduction: Reading assessments in context 1614

410 Codebase Health Assessment — 2026-04-05 1615

411 Documentation & README Audit — 2026-04-05 1622

412 AUDIT: CODE QUALITY (2026-04-11) 1626

413 AUDIT: DOCUMENTATION QUALITY (2026-04-11) 1629

414 AUDIT: READY-TO-SHIP ASSESSMENT (2026-04-11) 1631

415 Code Quality Audit 1633

416 Documentation Quality Audit 1639

417 Ready-to-Ship Assessment 1643

418 Graveyard 1650

419 I-0d — Per-stage render pipeline instrumentation (buried 2026-04-09) 1651

420 I-3 — Custom baseline JSON format + compare tool (buried 2026-04-09) 1653

421 RE-017 — Byte-Pipeline Recovery 1654

422 Archive 1683

423 Graveyard 1684

424 Bijou Component System Audit 1685

425 diag() — ASCII DAG Renderer for Bijou 1695

426 Feature Request: Canvas Primitive & Box Width Override 1705

VII Rendered components and provenance 1709

Introduction: Connecting written contracts to surfaces 1710

A The component cell atlas 1711

B Source inventory 1734

I

Philosophy and architecture

The ideas behind the system

Bijou's architecture starts with the conditions under which terminal software is used. A person may be navigating with a keyboard, inspecting a dense table, reading through assistive output or using a narrow viewport. An automated consumer may need the same information as structured facts. The system's design has to account for those encounters with the application.

The vision and doctrine chapters make the project's commitments explicit. Runtime truth gives priority to what the program actually does. Hexagonal boundaries separate application meaning from host services. The state-update-view model gives changes and effects identifiable owners. The layout and interaction principles connect rendered geometry to focus, selection and input routing.

How the principles fit together

Several themes recur. Graceful lowering asks how a rich surface preserves meaning in simpler output modes. Localization treats language and direction as inputs to layout and interaction. Inspectability asks what a person or an agent can learn about a surface without guessing from its appearance. The design language gives these concerns visible form through shared patterns, tokens and component families.

DOGFOOD supplies a place to exercise those commitments in a real application. A principle becomes easier to assess when a reader can follow it into a shell, a component preview, a settings surface or a rendering test. This relationship between doctrine and a working surface explains the repeated emphasis on evidence throughout the later parts.

How to use these chapters

Start with the vision, then the architecture and the invariants relevant to the work at hand. Treat an invariant as a constraint to investigate in the implementation. The prose states what the project is trying to make reliable; source code and focused checks establish how a particular revision meets that commitment.

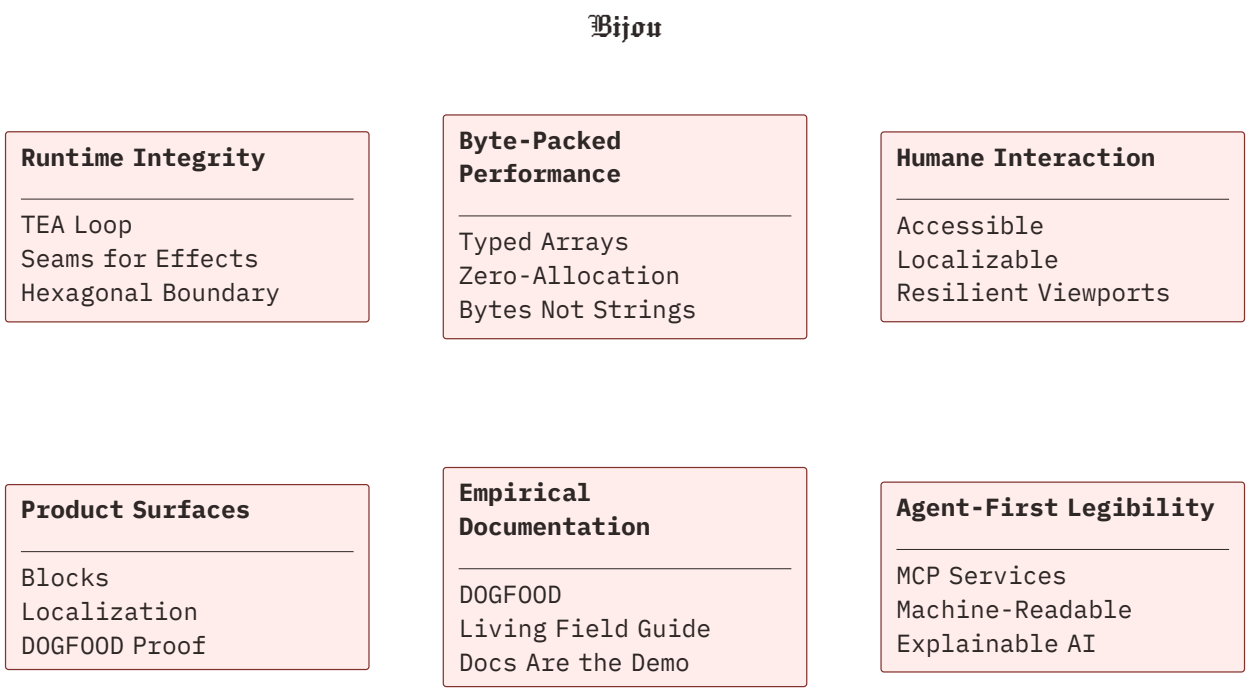
The strategy notes also preserve aspirations and transitions. Their wording belongs to their original context. When a later design changes a boundary or revises a policy, read the two records together and follow the linked integration evidence. This part provides the vocabulary for that reading; the design records show the vocabulary being put under pressure.

1

VISION

docs/VISION.md

Bijou is an industrial-grade TypeScript engine for terminal software where runtime truth, design language, and documentation are unified.



Core Tenets

1. Runtime Integrity

A deterministic state-update-view loop (TEA) with explicit seams for effects and commands. Logic is isolated from platform IO via a strict hexagonal boundary.

2. Byte-Packed Performance

A rendering foundation built on typed arrays and zero-allocation hot paths. The surface layer speaks bytes, not strings, ensuring high-fidelity output at scale.

3. Humane Interaction

A shell model that remains truthful under the pressures of keyboard, mouse, compact viewports, and localization. Accessibility and internationalization are substrate properties, not afterthoughts.

4. Product Surfaces

Bijou's architecture becomes credible only when it survives visible product surfaces. Blocks and localization are the current pressure points: Blocks must move from contracts into real reusable product assemblies, and localization must make DOGFOOD visibly usable without hiding missing translations behind copied English fallback data.

5. Empirical Documentation

DOGFOOD is the canonical proving ground. Every component, Block, localization flow, and pattern is verified in a living product surface within the repository. Documentation should not merely describe the system; it should expose the system working under real product pressure.

6. Agent-First Legibility

The system is designed to be codable and inspectable by both humans and AI. MCP rendering services and interactive documentation provide a machine-readable interface to the toolkit.

The goal is not more terminal widgets. It is the geometric lawfulness of the terminal as a professional application bedrock.

2

Bijou Vision

docs/strategy/bijou-vision.md

> Historical note: the canonical project-identity signpost is now docs/VISION.md. This file remains as expanded background and provenance, not as the active queue surface.

Living statement of what Bijou is for, how it should feel, and where it should be used

What Bijou is

Bijou is a framework for building **surface-native terminal applications** that are:

- styled
- stateful
- testable
- humane
- honest across different output modes

Bijou is not trying to pretend the terminal is a browser.

It is trying to take the terminal seriously as its own medium:

- dense, but not cramped
- expressive, but not decorative for its own sake
- interactive, but still degradable
- structured, inspectable, and testable

The long-term goal is not only to ship components.

The goal is to make it straightforward to build terminal software that feels:

- calm
- explicit
- reliable
- accessible
- governable

What Bijou is not

Bijou is not primarily:

- a string-templating helper for ad hoc CLI output
- a browser-emulation layer in a terminal window
- a pile of isolated widgets without shell doctrine
- a framework that treats docs, accessibility, and graceful degradation as afterthoughts

It should stay willing to say “this belongs to the shell,” “this belongs to layout,” or “this should lower differently in a non-rich mode” instead of flattening every concern into raw component calls.

Product vision

Bijou should become the framework people reach for when they want to build serious terminal software with:

- real application state
- real interaction structure
- real layout ownership
- real testing and replayability
- real design standards

The ideal Bijou app should be explainable in a few plain statements:

- where state lives
- what views are visible
- what currently owns input
- how layout is being constrained
- what commands can change state
- what effects may happen as a consequence

That clarity is part of the product.

Design philosophy

1. Hexagonal at the core

The core should own:

- state
- view stack
- layout
- routing
- commands

- effects

Adapters should own:

- terminal protocols
- Node boundaries
- clocks, files, and platform APIs
- transport and environment details

If a behavior is important enough to test, inspect, or lower accessibly, it should probably exist inside the hexagon.

2. Graceful lowering is part of the contract

Rich mode is not the only reality.

Bijou should preserve meaning across:

- rich interactive mode
- static mode
- pipe mode
- accessible mode

Visual parity is not the requirement. Semantic honesty is.

3. Layout is a first-class concern

Components should not only know how to render.

They should also have explicit answers for:

- how they size
- how they stretch or shrink
- how they align and anchor
- how they overflow
- when they need a viewport

If layout ownership is vague, interaction ownership becomes vague too.

4. Input belongs to the active interaction model

Input should flow through explicit ownership:

- topmost active view/layer first
- layout-defined interaction geometry
- component-local handling
- commands and effects emitted intentionally

This is how modals block lower layers, how shells avoid stale controls, and how input remains explainable instead of branch-order magic.

5. Commands and effects are not the same thing

Commands mean:

- “app, change something meaningful”

Effects mean:

- “runtime, perform this consequence”

Bijou should keep that distinction clean.

It is not just type hygiene. It is how stateful behavior stays understandable.

6. The shell is a product surface, not glue

Search, help, settings, notifications, quit flows, and framing chrome are part of the user experience.

They should not feel like bolted-on conveniences around “the real app.”

7. The docs are the proving ground

DOGFOOD is important because it forces Bijou to live with its own decisions.

The docs should not only describe the system. They should exercise it.

8. Human and agent users both matter

Agents are first-class users of the framework and its artifacts.

A good Bijou system should be understandable enough that:

- humans can reason about it
- agents can inspect it
- tests can prove it
- docs can teach it

That means explicit contracts beat folklore.

Design values

Calm over noise

Density is good. Chaos is not.

Explicit over magical

The framework should prefer clear ownership and named seams over hidden cleverness.

Honest over flashy

A simpler truthful fallback is better than a rich presentation that lies about capability.

Structured over stringly

Structured surfaces, layout nodes, and runtime objects are preferable where behavior needs to be inspectable or composable.

Testable by design

Important behavior should be expressible in deterministic tests instead of only in screenshots or subjective demos.

Humane by default

The framework should make it easier to do the right thing for:

- accessibility
- localization
- explainability
- reduced motion
- low cognitive load

Recommended use cases

Bijou is a strong fit for:

Operator and internal tools

Apps where people live in terminals and need:

- dashboards
- workflow tools
- review surfaces
- deployment or environment controls
- data inspection
- incident response interfaces

Developer tools

Apps where state, logs, structured output, and keyboard-driven flows matter:

- local dev control planes
- test/replay tools

- code review helpers
- repo/workspace tools
- build and release interfaces

Guided and explainable interfaces

Apps that need to teach, justify, or walk users through decisions:

- AI-assisted flows
- operator handoff tools
- approvals
- audits
- onboarding/tutorial surfaces

Dense information browsing

Apps that need to browse or compare structured information:

- tables
- trees
- timelines
- layered detail panes
- searchable docs and knowledge systems

Documentation and field guides

Apps where the documentation itself should be interactive, testable, and mode-aware.

DOGFOOD is the canonical example of this.

Less ideal use cases

Bijou is probably a weaker fit when the primary requirement is:

- pixel-perfect graphical presentation
- highly freeform mouse-first visual design
- document editing that depends on browser/editor ecosystems Bijou does not aim to replace
- experiences where the terminal itself is already the wrong medium

That is not a failure. It is a boundary.

What good Bijou apps should feel like

A good Bijou app should answer these questions quickly:

- Where am I?
- What can I do right now?
- What currently owns input?
- What changes if I activate this?
- How do I dismiss or back out safely?

It should also make these answers legible to the builder:

- Which state is active?
- Which views are stacked?
- Which layout owns this space?
- Which component owns this interaction?
- Which command changes state?
- Which effect leaves the core?

Relationship to other docs

This document sits above more specific doctrine and design notes:

- Bijou UX Doctrine
- The Humane Shell
- Accessibility and Assistive Modes
- Localization and Bidirectionality
- AI Explainability Standard
- Content Guide
- Architecture — Bijou

Living-document note

This document should evolve when Bijou's self-conception changes:

- if the architecture center of gravity changes
- if the product promise changes
- if the recommended use cases change
- if the framework starts aiming at a different class of software

If those things change and this file stays stale, the repo will start lying about what Bijou is for.

3

BIJOU UX DOCTRINE

docs/strategy/bijou-ux-doctrine.md

Bijou is a high-fidelity terminal UI engine with an explicit design stance on runtime truth, shell honesty, and human-centered interaction.

Core Tenets

1. Focus Owns Input

The active region must be unambiguous and possess the primary control surface. The user should never wonder whether a keypress affects the current pane, a hidden modal, or a stale footer hint.

2. Visible Controls Are a Promise

If a control hint is visible, it must function for the active layer. Advertising inactive or covered controls is a failure of shell integrity.

3. Structural Selection

Selection should be communicated through background fill, structural emphasis, or positional glyphs. Color may reinforce selection but must not be the sole differentiator.

4. Density Requires Rhythm

TUI density is an industrial strength, not a license for visual noise. Gutters, margins, and consistent rhythm between labels and values must be preserved.

5. The Shell is a Product

The app frame—header, footer, help, search, settings, and notifications—is a unified human-facing product. It exists to orient the operator, centralize common behavior, and reduce anxiety.

6. Graceful Lowering is an Obligation

Rich interactive mode is one profile. Bijou preserves semantic truth across s tatic, pipe, and accessible modes. Lower profiles do not need visual parity; they require semantic honesty.

7. Accessibility is a Substrate Property

Bijou considers reading order, low-vision, color-blindness, and cognitive load at the engine level. An application is not accessible merely because a component can lower to text.

8. Localization is Geometric

Locale affects wording, wrapping, and reading direction. Bijou avoids hardcoded assumptions that prioritize English or LTR layouts.

9. AI Surfaces Must Explain

AI-mediated output must be explicit about its nature, evidence, and confidence. It should guide the operator toward the expected next action.

Interaction Patterns

- **Keyboard-First, Mouse-Enhanced**: Everything important remains reachable by keyboard. In practice, the shell stays keyboard-first, mouse-enhanced, and mouse input enhances this truthful model through spatial interaction.
- **Topmost Layer Dismisses First**: Layered TUIs must be predictable. Layers should dismiss from the top, and the top dismissible layer is the first to be dismissed by Esc.
- **Top Layer Owns Visible Controls**: The layer on top owns the visible prompts, controls, and escape affordances until it yields or dismisses; in plain terms, the thing on top owns visible controls.
- **DOGFOOD is the Proving Ground**: The documentation itself must exercise the shell, search, and accessibility posture truthfully.
- **Doctrine Before Blocks**: Blocks come after doctrine. Reusable blocks come after interaction doctrine, not before it; policy should shape components instead of the other way around.

The goal is to move the terminal from a collection of widgets to a professional application bedrock.

4

ARCHITECTURE

ARCHITECTURE.md

Bijou is an industrial-grade TypeScript engine organized around a strict Hexagonal (Ports and Adapters) architecture.

Core Boundary

The @flyingrobots/bijou core is pure TypeScript with zero runtime dependencies. All platform-specific concerns are isolated behind three primary ports:

Port	Responsibility	Representative Methods
RuntimePort	Environment and terminal facts	env(), columns, rows, stdoutIsTTY
IOPort	Input, output, timers, and files	write(), rawInput(), setInterval(), readFile()
StylePort	Color and styling decisions	styled(), rgb(), hex(), bold()

Adapters (e.g., nodeRuntime, nodeIO, chalkStyle) live in @flyingrobots/bijou-node or test-specific modules.

Package Stack

1. @flyingrobots/bijou (The Foundation)

- Zero-dependency primitives: Surface, LayoutNode, and BijouContext.
- Core component families and interactive prompts.
- Output-mode detection and graceful lowering.

2. @flyingrobots/bijou-tui (The Engine)

- The Elm Architecture (TEA) runtime: App, Cmd, and Msg.
- High-fidelity rendering pipeline: layout, diffing, and ANSI emission.
- Declarative motion (springs/tweens) and transition shaders.
- Standardized shell chrome and overlay composition.

3. @flyingrobots/bijou-node (The Host)

- Official Node.js implementations of all core ports.
- Worker-thread runtime for offloading heavy TEA logic.
- Native surface recording for documentation and debugging.

4. @flyingrobots/bijou-i18n (The Voice)

- In-memory localization runtime.
- Bidirectional support (LTR/RTL) and catalog management.

Rendering Pipeline

Static Path (CLI/Pipes)

```
data -> core component -> mode-aware render -> string/surface result -> stdou
t
```

Interactive Path (TUI)

```
input -> TEA update -> view() -> Surface | LayoutNode -> localize root -> pai
nt -> Diff -> terminal
```

For the LayoutNode branch specifically, the runtime rebases the tree into a local non-negative root before paint. The detailed ownership model is captured in docs/strategy/layout-localization-pipeline.md. The user-facing layout and overflow rules that sit on top of that pipeline are captured in docs/strategy/layout-and-viewport-rules.md.

Buffered Protocols

When Bijou buffers commands, events, or cross-boundary actions, those buffers should carry plain facts rather than executable behavior. The execution owner stays outside the buffer, which keeps queued state inspectable, replayable, and testable.

Use The Buffer Holds Facts as the canonical invariant for that rule.

Interaction Profiles

Bijou adapts its rendering behavior based on the detected output mode: - **interactive**: Full TEA loop, motion, and shell chrome. - **static**: Single-frame render (e.g., CI logs). - **pipe**: Plain-text fallback; no colors or complex layout. - **accessible**: Linearized output for screen-readers.

DOGFOOD serves as the architectural pressure point, where all packages are verified in a unified, professional application surface.

5

ARCHITECTURE

docs/ARCHITECTURE.md

This is the short architecture signpost for the docs surface.

Bijou is a nine-package monorepo for terminal software in TypeScript. The root ARCHITECTURE.md is the main entrypoint, but DOGFOOD and the docs lane still need a repo-owned summary here.

Structure

- @flyingrobots/bijou owns the pure core: prompts, themes, components, and surfaces
- @flyingrobots/bijou-tui owns the fullscreen runtime, layout, motion, overlays, and framed shell
- @flyingrobots/bijou-node owns the terminal/process boundary, worker runtime, and recorder
- the i18n packages own runtime catalogs plus localization authoring/exchange workflows

Read Next

- root architecture signpost
- runtime architecture
- node architecture
- core architecture

6

System-Style JavaScript

docs/system-style-javascript.md

How to write JavaScript infrastructure that lasts.

This is a repo-wide engineering doctrine for Bijou and other flyingrobots codebases. It is not a formatting style guide. It is a doctrine for writing infrastructure that remains honest under execution, replay, migration, debugging, replication, failure, and time.

Bijou Adaptation

Bijou is currently implemented largely in TypeScript. That does not weaken this doctrine. It sharpens how TypeScript should be used here.

In this repo:

- TypeScript is a useful dialect, not the final authority
- runtime-backed modeling outranks type-only confidence
- boundary parsing, explicit adapters, and tests outrank editor comfort
- strong claims about architecture should be anchored in runtime code, invariants, and executable proof

This doctrine therefore does **not** mean "rewrite Bijou into plain .js tomorrow." It means Bijou should treat runtime truth as architectural authority even when the implementation language is TypeScript.

Rule 0: Runtime Truth Wins

When the program is running, one question matters above all others:

What is actually true right now, in memory, under execution?

If the answer depends on comments, conventions, vanished types, wishful thinking, or editor vibes, the code is lying.

Trusted domain values must be created through runtime construction, parsing, or validation that establishes their invariants. Once established, those invariants must be preserved for as long as the value remains trusted.

This rule outranks documentation, build steps, editor hints, static overlays, compile-time tooling, team folklore, and "but the linter said it was fine."

What This Means In Practice

Infrastructure cannot afford fake contracts:

- a type that vanishes at runtime is not an authoritative contract
- a comment describing a shape is not an authoritative contract
- a plain object that “should” have valid fields is not an authoritative contract
- an IDE tooltip is not an authoritative contract
- a compile step is not an authoritative contract

These tools can be useful. None of them outrank the runtime.

Why It Matters Here

Bijou is not just view glue. The repo touches:

- runtime state and event flow
- host boundaries and adapters
- deterministic tests and replayability
- localization catalogs and exchange formats
- long-lived public APIs
- shell behavior that needs to stay explainable to both humans and agents

Mushy assumptions here turn into bugs with long half-lives.

The Hierarchy Of Truth

When layers disagree, authority flows in this order:

1. **Runtime domain model**: constructors, invariants, methods, error types
2. **Boundary schemas and parsers**: Zod, decoders, protocol validators
3. **Tests**: the executable specification
4. **JSDoc and design docs**: human-facing explanations of the runtime model
5. **IDE and static tooling**: navigation, refactoring support
6. **TypeScript**: useful dialect, not final authority

Scope

This standard is optimized for:

- infrastructure code with strong invariants
- long-lived systems with explicit boundaries
- direct execution workflows portable across hosts
- browser-capable cores
- JavaScript-first or TypeScript-as-dialect repositories

- code that must be teachable, legible, and publishable

It is not a claim that every JavaScript project should look like this. It **is** a claim that, for this family of repositories, runtime-backed domain modeling beats soft shape trust.

Language Policy

JavaScript First, TypeScript Subordinate

JavaScript remains the conceptual base language for this doctrine:

- fast to write and change
- direct to execute
- portable
- expressive enough for serious infrastructure
- readable enough to teach from

Bijou currently uses TypeScript heavily, but the rule is the same: TypeScript may improve refactoring, editor support, and consumer ergonomics. It does **not** replace runtime validation, preserve invariants by itself, or excuse weak modeling.

Use TypeScript where it helps. Never confuse it with the source of truth.

Escape Hatch: Rust Via WebAssembly

When JavaScript is insufficient for tight CPU work, hostile binary parsing, cryptographic kernels, or memory-sensitive hot loops, use Rust.

Preferred split:

Layer	Language	Role
Core domain logic	JavaScript / TypeScript with runtime-backed modeling	Default, portable, browser-capable
Performance-critical kernels	Rust -> Wasm	Safety and speed where justified
Host adapters	JavaScript / TypeScript	Node, browser, worker boundaries
Orchestration	JavaScript / TypeScript	Glue between cores and hosts

Architecture

Browser-First Portability

The browser is the most universal deployment platform and the ultimate portability test. Core logic should prefer web-platform-friendly primitives.

```
// Portable
const bytes = new TextEncoder().encode(text);
const arr = new Uint8Array(buffer);
```

```
const url = new URL(path, base);

// Node-only; belongs in adapters
const buf = Buffer.from(text, 'utf8');
const resolved = require('path').resolve(p);
```

Hexagonal Architecture Is Mandatory

Core domain logic must not depend directly on Node globals, filesystem APIs, process, Buffer, or host-specific calls. Those belong behind adapter ports.

Core rule: core logic should not know that Node exists.

In Bijou terms:

- @flyingrobots/bijou defines ports and pure helpers
- @flyingrobots/bijou-node owns Node integration
- runtime, i18n, and tooling seams should stay explicit rather than leaking host assumptions inward

Object Model

System-style JavaScript organizes code around runtime-backed objects:

- **Value objects** for meaningful values with invariants
- **Entities** for identity and lifecycle
- **Results / outcomes** as runtime-dispatchable objects, not soft tagged bags
- **Errors** as first-class domain failures, not stringly typed message parsing

Classes are often a good tool here, but the actual rule is stronger than “use classes”: important concepts need runtime-backed forms with preserved invariants.

Principles

P1: Domain Concepts Require Runtime-Backed Forms

If a concept has invariants, identity, or behavior, it needs a runtime-backed representation. A typedef or plain object is not enough.

P2: Validation Happens At Boundaries And Construction Points

Untrusted input becomes trusted data only through constructors or dedicated parse methods. Constructors establish invariants and do not perform I/O.

P3: Behavior Belongs On The Type That Owns It

Avoid switching on kind or type tags when runtime dispatch will do. Put behavior on the owning type.

P4: Schemas Belong At Boundaries, Not In The Core

Use schemas to reject malformed input at the edge. Domain types own behavior and invariants inside the boundary.

P5: Serialization Is The Codec's Problem

The byte layer stays separate from the meaning layer. Domain types do not need to know how they are encoded.

P6: Single Source Of Truth

Do not duplicate the same contract across JSDoc, TypeScript, and validators. Define the runtime model first. Everything else derives from or documents it.

P7: Runtime Dispatch Over Tag Switching

Inside a coherent runtime, instanceof is often correct. When values cross realms or duplicate module boundaries, use branding instead of pretending nominal identity still holds.

Practices

- **any is banished; unknown is quarantined**: unknown is allowed only at raw edges and should be eliminated immediately through parsing
- **trusted values must preserve integrity**: use Object.freeze(), private state, or defensive copying where needed
- **error type is primary; codes are optional metadata**: never branch on err.message
- **parameter objects must add semantic value**: do not normalize option sludge into public API design
- **raw objects may carry bytes, not meaning**: plain objects are for decoded payloads or logging, not trusted domain semantics
- **magic numbers and strings are banished**: give semantic constants real names
- **boolean trap parameters are banished**: prefer named policies or separate methods
- **structured data stays structured**: do not make machines parse prose to recover data
- **module scope is the first privacy boundary**: if it is not exported, it is private
- **JSDoc documents the runtime model; it does not replace it**

Tooling Discipline

Lint is law.

- lint errors fail CI
- suppressions require a documented reason
- the hardest enforcement should land on unsafe coercion, floating promises, raw Error use, and host-specific leakage into core code

When TypeScript is used:

- it remains subordinate to runtime validation
- it must not substitute for domain modeling
- any is banned
- unknown belongs only at raw edges and should disappear quickly

The Anti-Shape-Soup Doctrine

Most bad infrastructure comes from weak modeling. The discipline is:

1. name the concept
2. construct the concept with validated invariants
3. protect the invariant
4. attach the behavior to the owning type
5. guard the boundary with schemas and parsers
6. separate the codec from the domain
7. isolate the host behind adapters
8. document the runtime that actually exists
9. test the truth

Review Checklist

Before merging, ask:

- is this a real domain concept, and where is its runtime-backed form?
- where is unknown eliminated?
- does construction establish trust?
- does behavior live on the type that owns it?
- is anyone parsing err.message like a raccoon in a dumpster?
- are there magic numbers or strings?
- could this logic run in a browser-capable core?
- is tooling fiction being mistaken for architecture?

Derived Invariants

This doctrine is reflected in these repo invariants:

- Runtime Truth Wins
- Schemas Live at Boundaries

- Host APIs Stay Behind Adapters
- Codecs Are Not Domain Models
- Tests Are the Spec

This is infrastructure. Code cannot rely on costumes or pretend that comments are contracts. Runtime truth beats phantom certainty every time.

7

Invariants

docs/invariants/README.md

These are project-wide truths that design and implementation should treat as non-negotiable unless deliberately changed.

When a design doc or legend references an invariant, it should link to the specific invariant file.

Every invariant should also name at least one legend that protects it. If an invariant has no protecting legend, the repo has a governance gap.

Current invariants:

- Tests Are the Spec
- Runtime Truth Wins
- Schemas Live At Boundaries
- Host APIs Stay Behind Adapters
- Codecs Are Not Domain Models
- Focus Owns Input
- Topmost Layer Dismisses First
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Shell Owns Shell Concerns
- Docs Are the Demo
- Commands Change State, Effects Do Not
- The Buffer Holds Facts
- Layout Owns Interaction Geometry
- State Machine and View Stack Are Distinct

Protection Map

Invariant	Protecting legend(s)
Tests Are the Spec	WF, RE, DF, DL
Runtime Truth Wins	RE, DX, LX
Schemas Live At Boundaries	RE, DX, LX
Host APIs Stay Behind Adapters	RE
Codecs Are Not Domain Models	RE, LX
Focus Owns Input	HT, RE, DL

Invariant	Protecting legend(s)
Topmost Layer Dismisses First	HT, RE
Visible Controls Are a Promise	HT, DF, DL
Graceful Lowering Preserves Meaning	DL, LX
Shell Owns Shell Concerns	HT, DF
Docs Are the Demo	DF, WF, DL
Commands Change State, Effects Do Not	RE, DX
The Buffer Holds Facts	RE, DX
Layout Owns Interaction Geometry	RE, DL
State Machine and View Stack Are Distinct	RE

8

Strategy Notes

docs/strategy/README.md

This directory mixes living doctrine with older planning notes.

Do **not** start new delivery work here. Current queue truth lives in:

- GitHub issues
- GitHub milestones
- Linear execution items once connected; see workflow

Use strategy notes for doctrine, rationale, and historical reference.

Living Doctrine

These notes are still current reference surfaces:

- accessibility-and-assistive-modes.md
- ai-explainability-standard.md
- bijou-ux-doctrine.md
- content-guide.md
- focus-owns-input.md
- humane-shell.md
- layout-and-viewport-rules.md
- layout-localization-pipeline.md
- localization-and-bidirectionality.md
- low-allocation-renderer.md
- notification-history-belongs-to-the-shell.md
- preference-lists-belong-to-bijou.md
- settings-belong-to-the-shell.md

Historical Notes

These remain useful as provenance or superseded design context, but they are not the current signpost for active work:

- bijou-vision.md — historical expansion; use docs/VISION.md for the canonical project identity signpost

- component-story-v0.md — early DOGFOOD substrate design note
- dogfood.md — historical milestone framing; use docs/DOGFOOD.md for the current docs-surface signpost

9

Accessibility and Assistive Modes

docs/strategy/accessibility-and-assistive-modes.md

Design note for accessible Bijou experiences beyond visual rich-mode parity

Why this exists

Bijou already has an accessible lowering target for many individual components.

That is useful, but it is not enough.

A component can lower honestly while the overall app still fails people who need:

- screen readers
- low-vision support
- color-blind-safe signaling
- reduced motion
- lower cognitive load
- alternate input or assistive interaction

This note exists to make one thing explicit:

accessible output is necessary, but it is not the same thing as an accessible product.

Current reality

Today Bijou is relatively strong at:

- giving many core components an explicit accessible rendering path
- avoiding purely decorative borders/backgrounds in accessible mode
- encouraging explicit text like Warning: message instead of ornamental styling

Today Bijou is not yet strong enough at:

- making a complex framed app like DOGFOOD truly usable in accessible mode

- representing interactive structure in a semantic layer above pure visual layout
- addressing low-vision and color-blind needs as first-class design inputs

Product stance

Bijou should treat accessibility as a design system concern, not a fallback string concern.

That means:

- a view may need to lower to a different experience, not merely a flatter rendering
- linear reading order matters more than preserving spatial cleverness
- visual semantics must become explicit text when required
- assistive users should not be abandoned simply because a surface was originally designed for rich TUI interaction

Design principles

1. Linear truth beats spatial fidelity

When an interface becomes hard to perceive spatially, Bijou should prefer a linear, explicit representation over a compressed imitation of the rich layout.

For example:

- a three-pane docs shell should probably lower to a guided sequential reading model
- not a cramped textual copy of the same spatial arrangement

2. Important meaning must survive without color

State should not rely on hue alone.

Selection, severity, status, and affordance should also be carried by:

- text
- labels
- structure
- glyphs
- grouping

3. Reduced motion is a first-class mode

Animation can enrich a TUI, but it should never be required for comprehension.

Bijou should support:

- reduced-motion profiles
- low-motion shell behavior
- motion-free explanations of dynamic state

4. Accessible mode is not a dump

Plain text should still be:

- structured
- oriented
- navigable
- intentional

Accessible lowering should not mean “unstyled but still confusing.”

5. Cognitive load is part of accessibility

Accessibility includes how much the user has to remember, infer, and hold in working memory.

We should minimize:

- hidden modes
- overloaded hint bars
- unexplained jargon
- surfaces with too many equally weighted actions

Assistive posture

Screen readers and terminal reality

Terminal screen-reader behavior varies and may not provide a rich widget model comparable to the web.

Bijou should therefore not assume:

- semantic terminal accessibility metadata will rescue a visually dense TUI
- a screen reader can infer pane relationships or control ownership from layout alone

Instead, Bijou should own semantic clarity in the content it emits.

Voice and alternate interaction

Bijou should eventually grow toward a semantic model that can describe:

- role
- label

- description
- current state
- available actions
- focus order

That model could support:

- accessible lowering
- docs
- testing
- explainability
- future assistive adapters

This should be treated as a semantic shadow model rather than a visual clone of rich-mode layout.

Low-vision design posture

Bijou should eventually support and audit for:

- high-contrast themes
- monochrome-safe or low-hue themes
- reduced visual clutter
- larger spacing or simplified shell modes where appropriate

This does not mean every app must ship all of these immediately.

It does mean the framework should make them possible and should not fight them.

Color-blindness posture

The framework should help builders avoid:

- meaning carried only by red/green or similar hue pairings
- state changes visible only through chroma
- token combinations that collapse under common color-vision deficiencies

Future tooling should include:

- contrast checks
- token-pair warnings
- color-blind simulation or heuristics

DOGFOOD implication

DOGFOOD is the proving surface, which means Bijou should eventually support an accessible DOGFOOD experience that is genuinely usable.

The likely direction is:

- search-first
- linear navigation
- explicit “you are here” copy
- explicit action lists
- no dependence on three-pane spatial inference

That should be treated as a different truthful experience, not a cramped copy of the rich shell.

That is a different experience, not just a visually flattened copy of the rich shell.

Backlog direction

Recommended next-layer backlog:

- high-contrast / low-motion shell profiles
- token contrast and color-blind auditing tools
- semantic shadow-model exploration for interactive surfaces
- accessibility review of DOGFOOD as a full app, not just component stories
- cognitive-load heuristics for shell and component surfaces

Relationship to other doctrine

This note should be read alongside:

- Bijou UX Doctrine
- The Humane Shell
- Localization and Bidirectionality
- Content Guide

Acceptance bar

This direction is working when:

- Bijou stops equating accessible mode with “flat text fallback”
- rich-shell surfaces are evaluated as experiences, not just as render outputs
- low-vision, motion, and color-blind needs influence design decisions early
- the framework begins to grow explicit semantic and audit tooling in support of that goal

10

AI Explainability Standard

docs/strategy/ai-explainability-standard.md

Design note for explicit, governed AI-mediated UX in Bijou surfaces

Why this exists

If Bijou is going to support AI-mediated interfaces, suggestions, assistants, or explainability surfaces, it needs a product standard that is stricter than "show some generated text."

The xyph TUI already demonstrates a strong pattern:

- visible [AI] labeling
- explicit explainability sections
- rationale and evidence
- expected next action
- confidence-aware posture

Bijou should adopt that seriousness as doctrine.

AI output should never feel like:

- hidden automation
- unexplained authority
- decorative speculation

Product stance

AI-mediated UI in Bijou should be:

- explicit
- attributable
- explainable
- actionable
- governed

AI should surface as advisory or assistive unless the app explicitly states otherwise.

Users should be able to answer:

- what is AI-generated here?
- why is it being shown?
- what evidence or basis does it rely on?
- what am I expected to do with it?
- how confident is it?

Core principles

1. Mark AI visibly

AI-mediated content should use an explicit [AI] marker or equivalent canonical treatment.

The user should not have to infer that a suggestion, explanation, or summary came from an agent.

That marker should be part of a canonical explainability component/pattern, not a one-off badge.

DL-005 establishes that first concrete seam through explainability().

2. Explain why

AI output should include or make available:

- why it was suggested
- what it is based on
- what decision or workflow it is trying to support

3. Separate evidence from recommendation

Recommendation should not masquerade as fact.

Bijou should encourage surfaces that distinguish:

- observation
- evidence
- recommendation
- next action

4. Confidence must be interpretable

If confidence is shown, it should be attached to explainable reasoning rather than used as mystical authority.

Confidence without context increases user anxiety and misuse.

5. AI should lead to governed next actions

A useful AI surface should make it clear what comes next:

- review
- accept
- reject
- inspect more evidence
- do nothing

6. AI should reduce ambiguity, not add it

An explainability surface is successful when it lowers uncertainty.

It is unsuccessful when it adds more jargon or produces an attractive wall of text without operational value.

Canonical explainability fields

Bijou should gradually standardize around explainability structures like:

- label: [AI]
- artifact kind
- source
- source mode
- rationale / why
- evidence basis
- confidence
- expected next action
- governance or review note

This does not have to be one monolithic component immediately.

It does need to become a consistent standard.

DOGFOOD and docs implication

DOGFOOD should eventually teach AI-mediated surfaces the same way it teaches other component families:

- labeled clearly
- explained honestly
- shown with rationale and next-action posture

This is especially important if Bijou later grows:

- AI-assisted blocks
- AI-authored settings or recommendations
- design/debug assistants

Relationship to XYPH

This direction is influenced by the stronger explainability posture already present in xyph, including:

- explicit [AI] labeling in TUI views
- structured explainability bodies
- rationale and evidence sections
- confidence-aware governance posture

Bijou should learn from that standard rather than reinventing a weaker one.

Relationship to other doctrine

This note should be read alongside:

- Bijou UX Doctrine
- Content Guide
- Accessibility and Assistive Modes

Acceptance bar

This direction is working when:

- AI-mediated surfaces in Bijou are visibly marked
- explainability becomes a product standard rather than ad hoc copy
- users can understand why an AI artifact is present and what they should do next
- the framework encourages governed, attributable AI experiences instead of mysterious ones

11

The Buffer Holds Facts

docs/invariants/buffer-holds-facts.md

Buffers, queues, and message channels should carry facts about what happened or what should happen next. They should not carry executable behavior.

Meaning

Good buffered values look like:

- { type: 'quit-requested' }
- { type: 'notification-added', level: 'info', text: 'Saved' }
- { type: 'frame-action', action: 'push-notification', notification }

Bad buffered values look like:

- class instances with apply() or run() methods
- closures that capture ambient state and perform side effects later
- "smart" objects whose meaning only exists when executed

The rule is simple:

> the buffer should hold facts, not code

Interpretation belongs at the boundary that owns side effects.

Why

Fact-shaped buffered values are:

- serializable
- inspectable
- loggable
- replayable
- testable without executing host effects

Behavior-shaped buffered values make the system harder to audit because the meaning of the queue is hidden inside arbitrary executable code.

Applies to

This invariant applies to:

- command buffers
- event channels
- queued frame actions
- cross-thread messages
- persisted or replayable runtime logs

It is especially important anywhere we may later want:

- diagnostics
- crash replay
- deterministic tests
- worker transport
- post-mortem inspection

Does not mean

This invariant does **not** mean every internal helper must become data.

It does not apply to:

- direct hot-path loops where no queue/buffer boundary exists
- local strategy tables used only as implementation detail
- pure helper functions that never become buffered values

The rule is about what crosses a buffered or queued boundary, not about banning functions from the codebase.

Review questions

When adding a queue or buffered protocol, ask:

1. Could I print every buffered item as plain data and understand the system?
2. Could I serialize and replay this later?
3. Is the side-effect owner explicit and centralized?
4. Would a new engineer have to execute the object to learn what it means?

If the answer to the last question is yes, the design is probably violating this invariant.

Protected by

- RE — Runtime Engine
- DX — Developer Experience

12

Codecs Are Not Domain Models

docs/invariants/codecs-are-not-domain-models.md

Protected by legends

- RE — Runtime Engine
- LX — Localization and Bidirectionality

Serialization formats and decoded payload objects are not the same thing as trusted domain values.

Implications:

- wire shapes should stay separate from runtime meaning
- encode/decode code should live in codecs, not be smeared across domain types
- decoded bytes or JSON should be turned into runtime-backed values before business logic trusts them
- a stable wire format does not excuse weak domain modeling

13

Commands Change State, Effects Do Not

docs/invariants/commands-change-state-effects-do-not.md

Protected by legends

- RE — Runtime Engine
- DX — Developer Experience

Commands are requests for app-owned stateful action. Effects are non-stateful runtime consequences.

Implications:

- input events are not commands
- effects such as sound, telemetry, or announcements should not be treated as state transitions
- a single input may emit multiple commands and multiple effects
- command and effect buffers should stay distinct so the runtime can apply each intentionally

14

ComponentStory v0

docs/strategy/component-story-v0.md

> Historical note: this is an early DOGFOOD substrate design note. Use docs/DOGFOOD.md for the current docs-surface signpost and METHOD backlog/design docs for active queue work.

Design note for the first DOGFOOD substrate

Why this exists

Bijou currently has several near-miss representations of "a component in context":

- design-system family docs
- package/example README prose
- showcase ComponentEntry
- smoke and canary examples

Those artifacts overlap, but they do not share one canonical contract. ComponentStory v0 is the first attempt to fix that.

The goal is not to design the forever-schema. The goal is to create the smallest structured record that can prove this loop:

1. a component family can be documented from structured fields
2. the same record can render a live demo
3. the same record can show graceful lowering across profiles
4. the same record can later grow selectors and scenarios without being thrown away

The problem with ComponentEntry

Today's showcase contract in examples/showcase/types.ts is useful, but too compressed:

- description is one markdown blob
- render(width, ctx) is one opaque demo function

- graceful lowering is mostly described in prose, not structured
- “when to use / when not to use” is not first-class data
- selectors, scenarios, source snippets, and variant controls have nowhere to live

That means it is good enough for a showcase, but not strong enough to become the substrate for docs, tests, and tooling.

Design goals

ComponentStory v0 should:

1. **Be structured enough for docs rendering** The docs surface should not have to parse one giant markdown blob to find summary, use, avoid, and lowering guidance.
2. **Stay small enough to author by hand** If the schema is too heavy, maintainers will bypass it and drift returns immediately.
3. **Support profile-aware demos directly** A story should be able to render itself in rich, static, pipe, and accessible.
4. **Model variants explicitly** Component families often have more than one canonical configuration worth teaching.
5. **Leave room for selectors and scenarios later** v0 should not depend on a selector model or scenario harness, but it should not block them either.

Non-goals for v0

ComponentStory v0 is not trying to solve:

- every possible interactive scenario
- full PatternStory design
- visual editor compatibility
- complete replay/assertion semantics
- an exhaustive control/knob system
- every component in the repo at once

Proposed shape

```
type StoryPackage = 'bijou' | 'bijou-tui' | 'bijou-tui-app';
type StoryMode = 'interactive' | 'static' | 'pipe' | 'accessible';

type StoryPreview = string | Surface;

interface StoryLowering {
  readonly interactive: string;
  readonly static: string;
  readonly pipe: string;
  readonly accessible: string;
}

interface StoryDocs {
  readonly summary: string;
```

```
  readonly useWhen: readonly string[];
  readonly avoidWhen: readonly string[];
  readonly relatedFamilies: readonly string[];
  readonly gracefulLowering: StoryLowering;
}

interface StoryVariant<State = void> {
  readonly id: string;
  readonly label: string;
  readonly description?: string;
  readonly initialState?: State;
  readonly render: (input: {
    readonly width: number;
    readonly ctx: BijouContext;
    readonly state: State;
  }) => StoryPreview;
}

interface StoryProfilePreset {
  readonly id: string;
  readonly label: string;
  readonly mode: StoryMode;
  readonly width: number;
}

interface StorySource {
  readonly examplePath?: string;
  readonly snippetLabel?: string;
}

interface ComponentStory<State = void> {
  readonly kind: 'component';
  readonly id: string;
  readonly family: string;
  readonly title: string;
  readonly package: StoryPackage;
  readonly docs: StoryDocs;
  readonly profilePresets: readonly StoryProfilePreset[];
  readonly variants: readonly StoryVariant<State>[];
  readonly source?: StorySource;
  readonly tags?: readonly string[];
}
```

Why this shape

docs

This is the biggest difference from ComponentEntry.

Instead of one markdown blob, the story has:

- summary
- useWhen
- avoidWhen
- relatedFamilies
- gracefulLowering

That makes the docs surface renderable without brittle parsing and keeps the design-system doctrine aligned with the story itself.

profilePresets

This makes profile comparison explicit and small.

v0 should not model every possible runtime knob. It only needs enough to render canonical comparisons:

- one rich preset
- one static preset
- one pipe preset
- one accessible preset

Width is included because graceful lowering is often width-sensitive in terminal UIs.

variants

Variants are first-class because many families are not taught well with one demo:

- alert() needs more than one tone
- modal() may need a simple and structured-body variant
- viewportSurface() may need one text case and one structured case

v0 keeps this minimal:

- label
- optional description
- optional initial state
- render function

source

This stays intentionally light in v0.

The docs app only needs a way to say:

- where the canonical example lives
- or what snippet label should be shown

Anything more ambitious can come later with story tooling.

Migration from ComponentEntry

The intended mapping is:

ComponentEntry	ComponentStory v0
id	id
name	title
subtitle	docs.summary
description	split into docs.* fields
pkg	package
render(width, ctx)	variants[0].render(...)
tier	dropped from v0 story core; this is showcase/workbench metadata, not story truth

The important design choice here is dropping tier from the canonical story model.

Why:

- tier is about embedding/showcase ergonomics
- it is not a core truth about the component family itself
- if a future workbench needs it, it can layer that on top instead of contaminating the story substrate

Recommended first stories

The first v0 stories should cover three different teaching problems:

1. alert() / note()

Why:

- low interaction complexity
- clear graceful lowering
- easy to render side-by-side

What they validate:

- structured docs fields
- simple profile comparison
- variant support

2. modal() / drawer()

Why:

- structured overlay content
- clearer “use vs avoid” doctrine
- richer TUI-only behavior without requiring a full app shell

What they validate:

- surface-native story previews
- overlay-family doctrine in structured form
- more than one meaningful variant

3. viewportSurface() or navigableTableSurface()

Why:

- tests a component whose value is not just appearance but behavior and composition

What they validate:

- structured TUI docs can still describe behavior honestly
- the docs surface can present width-sensitive behavior
- later selector/scenario integration has a real target

Open questions

These are intentionally deferred, but should stay visible:

1. Should ComponentStory carry source snippets directly, or only references to example files?
2. Should family be a free string in v0, or a stricter id shared with the design-system docs?
3. Should profile presets live on the story, or should the docs app own the canonical presets globally?
4. Should PatternStory share a common base with ComponentStory, or be defined independently later?

Recommended implementation sequence

1. Define the v0 type in code without promising permanence.
2. Author three canonical stories by hand.
3. Build the first docs surface around those stories.
4. Only after that, revisit selectors and scenarios.

That order matters because the immediate question is not “what is the perfect schema?” It is “can one structured story record actually replace duplicated docs/demo truth?”

15

Content Guide

docs/strategy/content-guide.md

Voice, tone, and copy doctrine for Bijou interfaces

Why this exists

In Bijou, words are part of the product.

This is especially true in terminal interfaces, where:

- labels are compact
- hints compete for space
- accessible and pipe modes rely heavily on text
- empty states and confirmations must do real orientation work

This guide exists to keep Bijou copy:

- calm
- clear
- localizable
- accessible
- operationally honest

Voice

Bijou copy should sound:

- direct
- calm
- precise
- unpretentious

This applies to both human-facing and agent-facing surfaces.

Avoid copy that feels:

- hyped
- mysterious

- overly cute
- apologetic
- passive when action is needed

Tone by surface

Shell chrome

Shell copy should be terse and operational.

Examples:

- ? Help
- / Search
- F2 Settings

Chrome should not carry essay-length explanation.

Empty states

Empty states should orient.

They should answer:

- what this place is
- why it is empty
- what to do next

They should not default to:

- “Nothing here”
- “No data”
- “Oops”

Confirmations

Confirmations should state consequence, not merely ask a generic yes/no question.

Good:

- Quit Bijou Docs?
- Discard unsaved filter changes?

Better body copy:

- what will happen
- what will not happen
- what the safe choice is when needed

Errors

Errors should try to provide:

- what happened
- basis if known
- next action if possible

Avoid vague panic language.

Notifications

Notifications should be short, factual, and scannable.

They should confirm:

- what changed
- what succeeded
- what needs attention

without pretending to be a full explanation surface when they are not.

Explainability surfaces

Explainability copy should identify:

- source
- rationale
- evidence
- confidence posture
- next action

Writing rules

1. Prefer explicit over clever

Especially in:

- accessible mode
- onboarding copy
- help
- confirmations

2. Use verb-first action labels

Examples:

- Open settings
- Search components

- Dismiss notification

3. Avoid unexplained jargon in first-use paths

Terms like:

- lane
- pane
- profile
- lowering

should be introduced with care when the user is new.

4. Treat hint bars as scarce space

Hints should be:

- short
- truthful
- current

They should not attempt to explain the whole app.

5. Write for localization

Avoid:

- English-specific jokes
- brittle idioms
- copy that only works at one exact length

6. Write for accessible linear reading

When a surface lowers to accessible mode, the copy should still make sense when read top-to-bottom with no spatial context.

7. Write for agents as first-class users

Agent-facing copy, labels, and explanations should be:

- explicit
- inspectable
- deterministic
- low on hidden implication

AI-specific copy posture

When AI is present:

- mark it
- explain why it is present
- explain what the user should do next
- avoid anthropomorphic bluffing

See AI Explainability Standard.

Relationship to other doctrine

This note should be read alongside:

- Bijou UX Doctrine
- Accessibility and Assistive Modes
- Localization and Bidirectionality
- AI Explainability Standard

Acceptance bar

This guide is working when:

- shell copy is consistent and calm
- empty states teach instead of merely reporting absence
- confirmations explain consequence
- accessible and localized surfaces have usable prose
- Bijou's tone feels intentional across apps and docs

16

Docs Are the Demo

docs/invariants/docs-are-the-demo.md

Protected by legends

- DF — DOGFOOD Field Guide
- WF — Workflow and Delivery
- DL — Design Language

DOGFOOD is not just reference material. It is a proving surface.

Implications:

- the docs should strive to use canonical Bijou componentry
- shell and component polish discovered in DOGFOOD should be promoted, not trapped
- the documentation should exercise the same runtime and design system it describes

17

DOGFOOD

docs/strategy/dogfood.md

> Historical note: this milestone framing is kept for provenance. The current docs-surface signpost is docs/DOGFOOD.md, and new delivery work should start in METHOD backlog/design surfaces instead of here.

Documentation Of Good Foundational Onboarding and Discovery

Milestone strategy for the post-v4 living docs platform

Why this milestone exists

After v4.0.0, the next problem is not "what capability can Bijou add next?" It is:

- how does someone understand Bijou quickly?
- how do docs, demos, examples, and tests stop drifting apart?
- how do we show graceful lowering instead of merely describing it?
- how do we make the system easy to explore without multiplying one-off example apps?

This milestone is the answer. It is the first post-v4 product milestone because it improves understanding, trust, and development speed at the same time.

DOGFOOD is a better name than "Milestone 1" because it reflects the actual philosophy:

- docs should be experiential, not only descriptive
- components should be explored in the medium they are built for
- users should be able to compare states, modes, and patterns directly
- the same source of truth should drive teaching, testing, and tooling

Design-thinking frame

Primary user

The primary user is the **curious builder**:

- a developer evaluating Bijou for a real terminal application
- a contributor trying to understand a component family or pattern
- a returning maintainer trying to remember what is canonical and what is not

They are not asking for infinite flexibility first. They are asking:

- “What is this thing?”
- “When do I use it?”
- “What does it look like in real terminal conditions?”
- “How does it behave in rich, static, pipe, and accessible?”
- “Can I trust that the example, docs, and tests all describe the same thing?”

Secondary user

The secondary user is the **component author / framework maintainer**:

- they need a safe place to evolve a component family
- they need stories and scenarios that can double as docs and regressions
- they need a way to inspect behavior without hand-rolling a bespoke demo for every new idea

Core jobs to be done

1. **Help me understand Bijou without hunting across the repo.** I want one place where the doctrine, the live behavior, and the example source agree.
2. **Help me compare interaction profiles and states quickly.** I want to see the same component in rich, static, pipe, and accessible without writing four custom demos.
3. **Help me trust what I am seeing.** I want docs, demos, and regression scenarios to come from one structured source of truth instead of diverging copies.
4. **Help me evolve the system safely.** I want to add or refine a component family once and have that work propagate into docs, previews, and test surfaces.

The current problem

Today, Bijou is much healthier than it was pre-v4, but the learning surface is still fragmented:

- doctrine lives in docs/design-system/*
- public teaching lives in package READMEs and example READMEs
- interactive demonstrations live in example programs
- regression coverage lives in smoke harnesses, PTY scripts, and targeted tests

Each of those serves a purpose, but they do not share one canonical representation of “a component family in context.” That fragmentation

increases maintenance cost and makes the product harder to learn than it needs to be.

Principles

This milestone should follow these principles:

1. **One source of truth** Stories should be structured records that can feed docs, demos, previews, and tests.
2. **Show behavior, not only prose** Lowering, width sensitivity, and interaction changes should be visible side-by-side.
3. **Docs are product** The docs experience is not a sidecar artifact. It is part of Bijou's value proposition.
4. **Terminal-native first** Bijou should document itself in its own medium where possible.
5. **Regression surfaces should emerge from teaching surfaces** If a story is canonical enough to teach, it should be close to canonical enough to test.

Scope of DVGFVVD

Current validation slice

The first implementation slice now lives in examples/docs.

It is intentionally small:

- three canonical component stories
- one framed docs shell
- one docs pane derived from structured story fields
- one live demo pane driven by the same story record
- profile switching across rich, static, pipe, and accessible

That slice is not the final docs product. It is the first proof that the story substrate can drive teaching and live previews without re-encoding the doctrine by hand.

It also exposed a broader shell need: when a docs lane is visually focused, that lane should own its local controls. The resulting shell/input design is captured in Focus Owns Input and Focused Pane Input.

The docs shell also now has two obvious shell-level proving surfaces:

- component search should feel like a standard shell affordance, not a bespoke docs shortcut
- settings should open as a shell-owned drawer rather than as a custom page overlay

The resulting settings direction is captured in Settings Belong to the Shell and Frame Settings Drawer.

As the settings drawer matured, another design truth became clear: if DOGFOOD is serious about proving Bijou, polished preference-row presentation should graduate into canonical Bijou componentry instead of remaining trapped inside shell-local drawing code. That direction is now captured in Preference Lists Belong to Bijou and Preference List Surface.

As the docs app grew, that broadened into a larger shell-quality question: the title screen, header, footer, search, help, settings, notifications, confirmation flows, and quit semantics all need to feel like one intentional product. That direction is now captured in The Humane Shell.

The next shell-feedback layer under that umbrella is notification review: transient notices need a standard home once they disappear. That direction is now captured in Notification History Belongs to the Shell and Shell Notification Center.

1. Story Protocol

This is the highest-leverage foundation.

Reference documents:

- ComponentStory v0 design note
- Story Protocol spec

What it is:

- a structured ComponentStory / PatternStory model
- enough metadata to drive:
  - docs pages
  - live demos
  - examples/showcase entries
  - scenario tests
  - replay artifacts

What it should capture:

- title and summary
- family / pattern identity
- use / avoid guidance
- graceful lowering descriptions
- demo inputs / states
- width and profile presets
- optional source snippet references
- optional selectors / semantics
- optional scenario hooks

Why it matters:

- without this, every later investment duplicates logic
- with it, the docs TUI, scenario harness, lab mode, and replay tooling all share the same substrate

Primary assumption to test:

- one story format can serve both human-facing docs and machine-facing regression/tooling needs without becoming too heavy

2. Bijou Docs UI

This is the flagship user-facing product surface.

What it is:

- a Bijou-native docs application
- left navigation, content pane, demo pane
- width and interaction-profile controls
- source and guidance next to live render output

Why it matters:

- it lets Bijou document itself in the medium it is designed for
- it makes the design system feel real instead of abstract
- it provides a natural home for component families, patterns, and canonical demos

Primary assumption to test:

- developers learn Bijou materially faster from a terminal-native docs surface than from disconnected README prose

3. Interaction Profile Simulator

This makes graceful lowering concrete.

What it is:

- a shared control surface for rich, static, pipe, accessible
- likely paired with width presets, reduced motion, and no-color modes

Why it matters:

- graceful lowering is one of Bijou's clearest differentiators
- right now it is described better than it is demonstrated

Primary assumption to test:

- showing profile shifts directly will reduce both documentation ambiguity and component misuse

4. Surface Selector Model

This makes stories and scenarios targetable in a stable way.

What it is:

- semantic targeting over surfaces and layout output

- ids, classes, scoped regions, and text-based hooks where appropriate

Why it matters:

- docs and tests should not depend on brittle coordinates
- selector semantics are the bridge between human-facing demos and machine-driven assertions

Primary assumption to test:

- a small selector model can cover most docs/demo/test needs without turning into a fake DOM

5. Surface Scenario Test Harness

This turns stories into interactive truth surfaces.

What it is:

- a Playwright-like harness for Bijou apps and surface stories
- scripted input, semantic selection, frame assertions, deterministic flows

Why it matters:

- it can make the same stories useful for docs demos and regression checks
- it reduces the cost of verifying behavior that is currently trapped inside bespoke examples

Primary assumption to test:

- once stories and selectors exist, scenarios can become the shared layer for docs, bug repros, and regression testing

6. Component Lab Mode

This is the interactive workbench inside the docs system, not a separate competing product.

What it is:

- a knob-driven or control-driven exploration surface for stories
- state toggles, width toggles, profile toggles, maybe timing or pointer toggles

Why it matters:

- it helps authors and users poke at a component family without writing another one-off demo
- but it should live inside the docs/story system, not alongside it as another disconnected example app

Primary assumption to test:

- a lab mode is most valuable when it is powered by stories, not by custom per-example controls

The next pass

The next pass should not try to build all six pieces at once. It should validate the smallest coherent loop:

1. **Story Protocol v0**
2. **Docs TUI vertical slice**
3. **Interaction Profile Simulator**
4. **Surface Selector Model**
5. **Surface Scenario Test Harness**
6. **Component Lab Mode**
7. **Shell-owned settings drawer**

Why this order

1. Story Protocol v0

Build first because everything else gets cheaper and truer once stories exist.

The goal is not to design the perfect eternal schema. The goal is to prove that one story record can power:

- one docs page
- one live demo
- one source snippet
- one profile comparison

Recommended first families:

- alert() / note() for simple lowering
- modal() / drawer() for richer structured overlays
- navigableTableSurface() or viewportSurface() for structured TUI behavior

2. Docs TUI vertical slice

Once the story shape exists, build a narrow docs app that renders a few canonical stories.

The first slice should include:

- left nav
- content pane
- demo pane
- one or two width presets
- one or two family pages

Do not try to build the full docs universe first.

3. Interaction Profile Simulator

Add the control that most directly demonstrates Bijou's difference.

The first simulator should support:

- rich
- static
- pipe
- accessible

Optional next toggles:

- reduced motion
- narrow width
- no color

4. Surface Selector Model

Once stories and docs exist, define the semantic targeting needed to make them testable and inspectable.

Keep the first version minimal:

- id
- class
- scoped region / pane identity
- text contains

5. Surface Scenario Test Harness

After stories and selectors exist, use them to prove the docs surface can also be exercised mechanically.

The first version should be able to:

- open a story
- toggle profile or width
- assert on semantic targets
- capture a frame sequence if needed

6. Component Lab Mode

Only after the above loop exists should the team add richer controls.

That keeps the lab from becoming another disconnected example swamp.

Success criteria

DOGFOOD is successful when:

- a new contributor can learn a component family from one canonical surface
- docs, demos, and regressions are visibly converging instead of drifting

- at least one family can be documented, previewed, profile-switched, and scenario-tested from the same story record
- the docs app feels like a real product surface, not a pile of examples behind a menu

Non-goals for the first iteration

- replacing every example immediately
- building the full visual studio
- perfecting every replay or selector capability before the docs TUI exists
- inventing a giant story schema before proving a narrow vertical slice

Recommended first slice

If the team wants the most leverage with the least risk, start here:

1. define ComponentStory v0
2. author three canonical stories:
    - alert() / note()
    - modal() / drawer()
    - viewportSurface() or navigableTableSurface()
3. build a docs TUI shell that renders those stories
4. add profile switching to those pages

That is the smallest slice that proves the philosophy instead of merely describing it.

18

Focus Owns Input

docs/invariants/focus-owns-input.md

Protected by legends

- HT — Humane Terminal
- RE — Runtime Engine
- DL — Design Language

The active thing should own the controls.

Implications:

- inactive panes should not respond to active-pane controls
- shell overlays and modal layers preempt underlying content
- visible focus styling should correspond to real input ownership
- help and footer hints should reflect the active input region

19

Focus Owns Input

docs/strategy/focus-owns-input.md

Design note for pane-scoped input in createFramedApp()

Why this exists

The first DOGFOOD docs shell exposed a real shell problem, not just a docs-app bug:

- visual focus can move to a different pane
- the active pane can change in shell state
- but page-wide bindings still handle keys as if nothing changed

That makes the app feel dishonest. If the right pane is focused and the left pane still reacts to arrow keys, the shell is lying about where input goes.

This is not only a docs concern. Any multi-pane framed app will eventually want the same thing:

- the active area should own local controls
- inactive areas should stop responding
- shell overlays and modal layers should still preempt lower layers
- mouse focus and keyboard focus should follow the same model

The right place to solve that is the standard app frame.

Design-thinking frame

Primary user

The primary user is the **workspace builder**:

- a developer building a multi-pane TUI with createFramedApp()
- someone who expects pane focus to mean something operational, not just visual
- someone who does not want to cram every local interaction into one page-global keyMap

They are asking:

- “Which area owns my keys right now?”
- “If focus moves, do the controls move with it?”
- “Will the shell still keep command palette, help, and tab management coherent?”

Secondary user

The secondary user is the **framework maintainer / docs author**:

- they need the shell to express local controls honestly
- they need the help surface to describe the currently active area
- they need mouse and keyboard behavior to align instead of drifting apart

Jobs to be done

1. **Help me trust focus.** When the shell highlights a pane as active, local input should target that pane and not a hidden sibling.
2. **Help me compose local interactions safely.** I want pane-local key and mouse behavior without rewriting the shell or creating one enormous page keymap.
3. **Help me keep shell behavior coherent.** Global shell actions like help, palette, tab switching, and modal capture should still work consistently.
4. **Help me explain the app truthfully.** The help surface should reflect the active area’s local bindings, not stale page-wide assumptions.

Hills

Hill 1 — Focus means ownership

When a user moves focus inside a framed app, the active area owns the next local interaction. Inactive panes do not keep responding as if they were still active.

Hill 2 — Shell layers still win

Help, command palette, and modal capture continue to preempt lower layers. Local pane input should feel powerful, not anarchic.

Hill 3 — Authors declare areas, not routing spaghetti

App authors should be able to declare pane-scoped input near the pane definition instead of encoding a whole workspace’s behavior inside one page-global keymap.

The current problem

Today createFramedApp() already tracks:

- the active page
- the focused pane per page
- pane-local scroll positions
- shell overlays like help and command palette

But it still routes most custom keyboard input through:

- FramePage.keyMap
- FramePage.modalKeyMap
- CreateFramedAppOptions.globalKeys

That means:

- focus affects scrolling, not input ownership
- page-level bindings can keep mutating the wrong pane
- help can only talk about shell/global/page bindings, not the active pane
- mouse focus and keyboard routing are only partially aligned

The DOGFOOD docs shell is just the first visible symptom:

- focus can move to the variants pane
- but the family lane still responds to arrow keys because the page keymap is still global

Principles

1. **Focus should imply local input ownership** If the shell says a pane is active, pane-local controls should route there.
2. **Layering should stay explicit** Shell overlays and modals must remain above pane input.
3. **Keyboard and mouse should share one focus model** Click-to-focus and key-based focus should activate the same local area.
4. **The frame should own the stack** Apps should declare local input sources. The frame should synthesize the routing model internally.
5. **Tests are the spec** The first implementation step is failing app-frame tests that describe routing truthfully.

Proposed concept

The philosophy is:

> **Focus owns input.**

In practical shell terms:

- the focused pane contributes a local input layer
- when focus moves, that layer is replaced
- inactive panes no longer receive local keys
- shell overlays/modals still sit above pane-local behavior

The frame should manage this as an internal input stack, even if the first public API stays simple and declarative.

Proposed vo API shape

The first public step should stay pane-scoped and explicit:

```
interface FrameInputArea<PageModel, Msg> {
  readonly paneId: string;
  readonly keyMap?: KeyMap<Msg>;
  readonly helpSource?: BindingSource;
  readonly mouse?: (args: {
    readonly msg: MouseMsg;
    readonly model: PageModel;
    readonly rect: LayoutRect;
  }) => Msg | undefined;
}

interface FramePage<PageModel, Msg> {
  // existing fields...
  readonly inputAreas?: (model: PageModel) => readonly FrameInputArea<PageModel, Msg>[];
}
```

Why this shape:

- it stays close to the existing paneId-based frame layout model
- it does not force apps to manually manage an InputStack
- it leaves room for a richer future stack model without throwing away the first API

Internal routing model

createFramedApp() should internally synthesize a layered routing model from its existing shell state plus any active input areas.

Conceptual layers

From highest priority to lowest:

1. **Shell modal layers**
   - help
   - command palette
   - future shell-owned overlays that capture input
2. **Page modal layer**
   - existing modalKeyMap
3. **Frame shell layer**

- tab switching
- pane cycling
- shell help/palette open
- shell scroll actions that intentionally target the focused pane

4. **Focused pane layer**
   - the inputAreas entry whose paneId matches the currently focused pane
5. **Page/global fallback layers**
   - existing keyMap
   - existing globalKeys

The exact ordering of page/global/frame conflicts should preserve the spirit of the existing keyPriority option:

- keyPriority: 'frame-first' means shell bindings still win over page-derived bindings
- keyPriority: 'page-first' means page-derived bindings win before shell fallbacks

For v0, the focused pane should count as **page-derived**, not as a shell layer.

Mouse model

Mouse should follow the same ownership concept.

v0 expectations

- clicking inside a focusable pane focuses it first
- after focus is updated, pane-local mouse handling can run for that pane
- wheel scrolling over a pane focuses it and scrolls the hovered/focused pane
- shell chrome and shell-owned overlays still intercept first

This keeps mouse support additive and consistent with the keyboard model instead of becoming a parallel routing system.

Help and discoverability

The help surface should become more truthful once pane-local input exists.

For v0:

- the short help strip should continue to show shell-level controls first
- the full help view should merge the active pane's helpSource into the current binding view
- inactive pane bindings should not be shown as if they are currently usable

This is a real product value of the feature, not just a nice-to-have.

Scope

In scope for v0

- pane-scoped key handling in createFramedApp()
- pane-scoped mouse handling in createFramedApp()
- shell-managed focus-to-input ownership
- help integration for the active pane
- migration of the DOGFOOD docs shell as a proving surface

Out of scope for v0

- arbitrary nested input graphs beyond framed panes
- full public InputStack authoring inside frame apps
- generalized interactive-component protocol
- animation work on the DOGFOOD landing hero
- redesigning every shell binding or every scroll primitive

Tests are the spec

The first implementation step should be failing tests in app-frame.test.ts and the docs preview tests.

Minimum acceptance tests

1. **Focused pane keymaps only fire for the active pane**
   - If the variants pane is focused, local arrow bindings in the family pane do not fire.
2. **Changing focus replaces the active pane layer**
   - Tabbing or clicking into a different pane immediately changes which pane-local bindings are active.
3. **Page modal capture still blocks pane input**
   - Existing modalKeyMap behavior remains stronger than pane-local bindings.
4. **Shell modal capture still blocks pane input**
   - Help and command palette continue to preempt lower layers.
5. **Shell bindings remain coherent**
   - Frame-level commands like pane cycling or help toggle still work while pane-local input exists.
6. **Pane-local mouse handlers only run for the focused/targeted pane**
   - A click in one pane does not invoke another pane’s local handler.
7. **Help reflects the active pane**

- The full help surface shows the active pane's bindings instead of stale page-global guidance.

8. **DOGFOOD regression**
   - In the docs shell, focusing the variants pane stops arrow keys from moving the family accordion.

Recommended implementation loop

1. **Write the shell tests first**
   - Add focused-pane keyboard routing tests to app-frame.test.ts
   - Add a docs-preview regression that encodes the current broken behavior
2. **Add pane-scoped input areas to FramePage**
   - Start with declarative inputAreas(model)
   - Keep all existing page/global APIs working
3. **Synthesize the active layer in createFramedApp()**
   - Resolve the focused pane
   - Insert the active area into the internal routing order
4. **Wire help to the active pane**
   - Merge helpSource from the focused area into the shell help surface
5. **Add mouse support to the same model**
   - Click-to-focus
   - wheel-to-focus-and-scroll
   - pane-local click dispatch after shell interception
6. **Migrate DOGFOOD**
   - Move its lane controls out of one page-global keymap and into pane-scoped areas
7. **Only then evaluate a richer public stack model**
   - If the abstraction proves out, consider whether framed apps should later expose a more direct layer/stack contract

Open questions

1. Should inputAreas eventually support non-pane regions inside a pane, or is pane scope enough for the first public step?
2. Should the active pane's bindings appear in the short help strip, or only in the full help overlay?
3. Should a pane-local keymap be able to override shell scroll keys like j/k, or should those remain shell-reserved in v0?
4. Should the internal routing eventually use the existing public createInputStack() implementation directly, or keep a lighter frame-specific dispatcher?

Recommendation

Do **not** implement this as a docs-only fix.

The docs shell merely surfaced the problem. The right solution is a frame-shell feature with the docs app as the first proving ground.

20

Graceful Lowering Preserves Meaning

docs/invariants/graceful-lowering-preserves-meaning.md

Protected by legends

- DL — Design Language
- LX — Localization and Bidirectionality

Rich mode is not the only reality.

Lowering does not need visual parity, but it must preserve truthful meaning across:

- rich / interactive
- static
- pipe
- accessible

Implications:

- lower modes should remain explicit and usable
- color, motion, and layout should not be the only carriers of meaning
- inaccessible “TUI-only” experiences should be treated as debt, not victory

21

Host APIs Stay Behind Adapters

docs/invariants/host-apis-stay-behind-adapters.md

Protected by legends

- RE — Runtime Engine

Node, browser, worker, filesystem, terminal, and environment specifics belong behind explicit adapter seams.

Implications:

- core packages should not reach directly for process, Buffer, or filesystem APIs
- host integration code should stay in adapter packages and modules
- tests should be able to swap host behavior through ports and mocks
- portability claims are stronger when the core does not know the host

22

The Humane Shell

docs/strategy/humane-shell.md

Design note for a coherent human-centered app shell in createFramedApp()

Why this exists

Bijou now has many of the right shell primitives:

- framed pages
- focused panes
- command palette
- help overlay
- settings drawer
- notifications
- confirmation flows

But having the primitives is not the same thing as having a product.

The next quality bar is not “can the shell do many things?” It is:

- does the shell feel calm and intentional?
- can a first-time user understand how to search, get help, adjust preferences, and quit safely within a few seconds?
- do header, footer, palette, settings, help, notifications, and confirmations feel like one family?
- does the shell tell the truth about what is active and what input goes where?

This note exists to define that product direction before more one-off shell tweaks accumulate.

Design-thinking frame

Primary user

The primary user is the **workspace operator**:

- someone inside a real framed TUI
- someone who wants to navigate, search, change settings, and recover safely without memorizing repo trivia
- someone who expects the shell to make the application feel trustworthy

They are asking:

- “Where do I go to search?”
- “How do I change preferences?”
- “What does this focused area mean?”
- “How do I get help without losing my place?”
- “How do I leave safely?”

Secondary user

The secondary user is the **workspace builder**:

- a developer building on createFramedApp()
- someone who wants a good shell without rebuilding shell UX from scratch
- someone who wants to declare page content and settings, not invent new chrome rules every time

They are asking:

- “What should a high-quality Bijou shell look like by default?”
- “Which concerns belong to the shell versus the page?”
- “How do I get consistency across apps without a giant pile of bespoke conditionals?”

Jobs to be done

1. **Help me get oriented immediately.** A user should understand the shell’s major entry points quickly: search, help, settings, and quit.
2. **Help me trust focus and ownership.** When the shell highlights a pane or opens a layer, the active area should own input honestly.
3. **Help me adjust preferences without leaving context.** Settings should feel like a shell sidecar, not a page detour.
4. **Help me recover and leave safely.** Global quit behavior and confirmation flows should be predictable across framed apps.
5. **Help me keep feedback proportional.** Temporary information should not feel as heavy as persistent or reviewable information.

Hills

Hill 1 — First-use confidence

A new user can enter a framed Bijou app and discover search, help, settings, and quit without reading external documentation.

Hill 2 — One shell, many surfaces

Header, footer, command palette, help, settings, notifications, and confirmations feel like one coherent shell family instead of unrelated overlays.

Hill 3 — The shell is calm

The shell favors clear orientation and layered disclosure over dumping the full control scheme into the chrome at all times.

Hill 4 — Safety is consistent

Global quit, confirmation, and notification behavior is stable enough that users can trust it across framed apps.

Shell promise

The shell should make the app feel:

- calm
- legible
- safe
- consistent
- honest about focus and control

The shell should not feel:

- noisy
- tutorialized all the time
- full of spilled key legends
- visually clever but operationally unclear

Product principles

1. **Search is a front door** Users should be able to open a framed app and immediately search for what they want.
2. **Help is discoverable, not omnipresent** The shell should advertise that help exists, then reveal the deeper control map on demand.
3. **Footer is operational truth** The footer should carry the most important current-status and quick-control information.
4. **Visible controls are a promise** If the shell or page shows a control hint, that control should work for the active layer or focused region right now. Dead hints erode trust faster than missing hints.
5. **Header is identity and context** The header should identify where the user is, not attempt to explain every possible action.

6. **Settings are shell state** Preferences should live in a shell-owned drawer and behave the same way across framed apps.
7. **Settings should explain themselves** A setting row should make its control type, current value, and purpose legible without requiring users to infer them from a terse label.
8. **Shell polish should graduate into canonical componentry** If the shell discovers a reusable presentation pattern, that pattern should become a Bijou component family instead of staying trapped inside shell-local rendering.
9. **Quit is safe by default** Interactive framed apps should treat Esc, q, and Ctrl+C as quit requests routed through the shell.
10. **Notifications scale by weight** Short-lived notices, persistent notifications, and reviewable history should be separate layers, not one overloaded mechanism.
11. **Layers dismiss from the top** Esc should dismiss the topmost dismissible layer before it ever falls through to shell quit behavior.
12. **Input maps should drive truth** The same input map that governs the active layer should also govern the visible controls the shell advertises.
13. **Layers should be explicit objects** A shell layer should own its title, visible controls, and richer help metadata instead of forcing the shell to rediscover them in separate branches.

Layer stack doctrine

The shell should treat interactive surfaces as a stack of layers, not as a pile of unrelated branch checks.

Candidate layers include:

- workspace base layer
- pane-local modal layer
- search / command palette layer
- help layer
- settings layer
- notification-center layer
- quit-confirm layer

The key rule is:

- only the topmost layer owns input

That rule should govern:

- routing
- Esc dismiss order
- footer hints
- help/control summaries
- future agent-visible shell semantics

Dismiss order

Esc should follow one humane rule:

1. dismiss the topmost dismissible layer
2. restore the next layer beneath it
3. only when no dismissible layer remains, interpret Esc as a quit request

This prevents the shell from closing one layer and accidentally acting on the thing beneath it in the same keystroke.

Input maps as the source of truth

Each layer should eventually own an input map that describes:

- what keys it handles
- what dismissal behavior it supports
- what controls may be shown in the footer/help
- whether it blocks, reviews, or augments underlying content

This is the cleanest seam for keeping:

- control hints truthful
- agents able to reason about the active layer
- focus semantics aligned with actual runtime routing

Proposed shell anatomy

1. Landing screen

The landing screen is allowed to be expressive, but it should still obey shell rules.

Purpose:

- establish brand and atmosphere
- provide one obvious next action
- give a minimal escape hatch

Recommended structure:

- fullscreen art / shader treatment
- no heavy header
- one reserved footer line

Footer zones on landing:

- **left**: brief controls
  - Esc/q quit
  - Enter continue or any key continue

- **center**: optional telemetry / mode
  - e.g. 54 fps • auto/full
- **right**: version
  - e.g. v4.0.0

The key design decision:

- telemetry belongs in the footer, not floating in the canvas, because it is shell information, not part of the art

2. Header

The header should be terse.

Its job is:

- identity
- current page / workspace context
- lightweight discoverability cues

It should not try to be a full control legend.

Recommended header shape:

- left: app title and current page/title
- right: short shell affordances
  - ? Help
  - / Search
  - F2 Settings

Optional contextual detail:

- active pane name
- current mode/profile
- route breadcrumb if actually helpful

Avoid:

- long scrolling control strings
- repeated legends that compete with page content
- dense status dumps in the top chrome

3. Footer

The footer should become the primary operational lane.

Recommended footer zones:

- **left**: core controls
  - Esc/q quit
  - Tab next pane
  - current route-specific primary action if needed

- **center**: current context
  - focused pane
  - landing FPS/mode
  - selected profile / variant / state
- **right**: status summary
  - version
  - notification count
  - connection / runtime status if relevant

This is the shell surface that should carry fast-truth information. The header should not have to do that job.

The design rule is:

- if a focused pane owns special controls, the footer may advertise them
- if that pane loses focus or a shell layer opens, those controls should disappear from visible chrome
- the shell should never keep showing controls that the current layer cannot actually honor

4. Workspace gutters and pane geometry

The shell should keep a one-cell breathing gutter between major structures:

- header and workspace
- workspace and footer
- left rail, center content, and right rail

This matters because otherwise the shell feels cropped or visually accidental.

Pane focus should remain obvious, but focus styling should not overpower the content.

5. Command palette

The command palette should be treated as a universal front door, not a power-user afterthought.

It should support:

- search by component/page name
- shell actions
- settings entry point
- navigation
- optional recent-history or notification recall later

Design direction:

- / and Ctrl+P both open it in interactive framed apps

- palette search should be the first thing users reach for when they know what they want

6. Help

Help should be layered:

- **short help affordance** in shell chrome
- **full help surface** on demand

The full help surface should be:

- shell-owned
- scrollable
- truthful to the active context

Recommended help structure:

1. shell/global controls
2. focused pane controls
3. active overlay controls
4. current page-specific shortcuts

Long-term direction:

- help should become searchable or filterable once the command palette/search substrate is mature enough

7. Settings drawer

The settings drawer should remain a left-edge shell sidecar.

It should feel like:

- a stable shell surface
- sectioned preferences
- compact values on the trailing edge

The shell should own:

- open/close behavior
- scrolling
- row focus
- activation
- input capture

The app should provide:

- sections
- rows
- values
- actions

The shell should keep owning drawer behavior, but the row treatment inside it should be allowed to graduate into canonical Bijou componentry once it proves itself. That follow-on direction is captured in Preference Lists Belong to Bijou.

8. Notifications and notification history

The shell should differentiate between:

- **toast**
  - ephemeral
  - low-weight
  - auto-dismiss
- **notification**
  - persistent enough to matter
  - shell-managed
  - should be reviewable
- **notification history**
  - archive / inbox / center
  - where missed or important notices can be recovered

Design direction:

- toasts should stay lightweight and not try to be a history mechanism
- a future notification-history surface should be reachable from shell affordances or the command palette
- the footer is a good place for a subtle notification count or inbox cue

That more detailed direction is now captured in Notification History Belongs to the Shell and the companion Shell Notification Center spec.

9. Confirmation flows

Confirmation should be treated as a shell/system pattern, not ad hoc prose.

Use confirmations for:

- destructive actions
- irreversible actions
- quitting when work could be lost or when the shell wants a consistent safe exit policy

Avoid confirmations for:

- trivial reversible actions
- low-consequence navigation
- “are you sure?” spam

Standard confirmation behavior should include:

- clear primary question
- short consequence line
- stable affordances:
  - y
  - n
  - Enter
  - Esc

10. Global quit behavior

Interactive framed apps should have a standard shell-level quit policy.

Recommended default:

- Esc
- q
- Ctrl+C

all map to **request quit**

Narrow exception:

- shell-owned text-entry surfaces may keep printable characters like q available for input
- Esc and Ctrl+C should still request quit from those surfaces

The shell then applies a policy:

- confirm by default for interactive framed apps
- immediate for non-interactive/piped contexts
- app-specific override only when truly justified

Why:

- this makes exit semantics predictable
- it prevents every app from inventing its own “sometimes q quits, sometimes it does nothing, sometimes Esc closes a random pane” rules

The shell should expose this as a real policy, not just a hardcoded DOGFOOD behavior.

Recommended default shell policy

Header

- minimal and contextual
- no long legends

Footer

- always present in framed interactive apps
- reserved for controls, state, and operational truth

Search

- / and Ctrl+P
- palette visible in chrome by name

Settings

- F2 and palette command
- shell-owned drawer

Help

- ?
- shell-owned overlay
- active-pane aware

Quit

- Esc, q, Ctrl+C
- shell requestQuit policy

Notifications

- toast for ephemeral updates
- persistent notifications for important updates
- planned history surface for recovery

Out of scope for this note

- exact BCSS token palette for the shell
- final landing-screen motion style
- every possible notification-center information architecture detail
- full tutorial/onboarding flow design

Those are related, but this note is about the shell contract first.

Acceptance tests this design implies

1. **A framed app shows a stable shell affordance set** Header and footer expose search, help, settings, and quit consistently.
2. **Header is concise** The shell does not regress into long truncated legends in the top chrome.
3. **Footer carries operational truth** Landing telemetry, version, and quit hints render in the reserved footer line instead of floating inside content.
4. **Global quit is shell-owned** Esc, q, and Ctrl+C route through one standard quit policy in interactive framed apps.
5. **Settings are shell-owned** Preferences open in the standard drawer and underlying page input is blocked while it is open.
6. **Help is layered and contextual** The full help surface includes shell controls and active-pane controls without implying that inactive panes still own input.
7. **Notifications are recoverable by weight** Toasts stay ephemeral, while persistent shell notifications can eventually be reviewed later.

Proving surface

DOGFOOD should be the first proving surface for this design.

Why:

- it already uses the standard frame shell
- it already has a landing screen
- it already exposes search, help, settings, notifications, and quit behavior
- it is visible enough that shell drift shows up quickly

The rule should be:

> if the shell feels confusing in DOGFOOD, the shell contract is not ready yet.

Recommended implementation order

1. **Footer-first shell cleanup**
   - move landing telemetry into the footer permanently
   - simplify the docs/footer roles until header and footer read cleanly
2. **Global quit policy**
   - formalize shell requestQuit
   - route Esc, q, and Ctrl+C through that policy
3. **Search/help/settings discoverability pass**
   - make shell entry points visible and calm in the chrome
4. **Notification history concept**
   - add the first shell-level recovery path for persistent notifications

5. **FTUI onboarding primitives**
    - once the shell itself is stable enough to teach honestly

23

Layout And Viewport Rules

docs/strategy/layout-and-viewport-rules.md

Strategy note for Bijou's canonical layout ownership and overflow behavior

Why this exists

Bijou already has strong layout instincts, but too many of them have lived as local bug fixes or component folklore.

This note turns those instincts into explicit rules:

- who owns whitespace
- when a child fills versus centers within a parent
- what resize is allowed to change
- when overflow becomes a viewport instead of a paint trick
- how scrollbars, focus, hit-testing, and scrolling must stay aligned

Read this alongside layout localization pipeline. That note explains how a LayoutNode tree becomes a local paint root; this one explains the user-facing layout and overflow rules that tree should obey.

Rule 1: Parents own placement, children own content

A child surface or layout node owns its own content. A parent owns where that child sits inside available space.

That means:

- children do not silently self-center inside a larger parent rect
- children do not invent extra outer padding unless the component contract says they do
- parents decide stack order, alignment, and unused slack

If a surface appears "floating" inside a larger region, that should be because the parent layout expressed that intention explicitly.

Rule 2: Slack is a layout decision, not an accident

Unused width or height must be explainable.

Legitimate causes:

- the parent intentionally centers or aligns a child
- the parent reserves gutter, rail, or chrome space
- the child is intrinsically smaller and the parent deliberately does not fill

Illegitimate causes:

- a child paints less than the geometry routing thinks it owns
- a container leaves dead area because no rule says who should fill it
- a scrollbar or overlay rail steals width without owning that contraction

If there is visible slack, someone should clearly own it.

Rule 3: Resize changes geometry, not semantics

Terminal resize may change:

- split ratios translated into actual rect widths/heights
- grid track sizes
- wrapping points
- viewport capacity

Resize must not silently change:

- focus ownership
- which node conceptually owns a region
- whether content is now “different” instead of merely reflowed

When a resize forces a more compact rendering, the semantic structure should stay the same even if line breaks, truncation, or viewport extent change.

Rule 4: Overflow belongs to a viewport

Overflow is not just “paint past the edge and hope clipping works.”

If content needs to move independently of its container, that region should be owned by a viewport or another explicit scroll-capable primitive.

Use a viewport when:

- keyboard navigation needs a stable scroll state
- mouse wheel input should move content
- a scrollbar is shown
- clipping and visible extent are part of the contract

Do not fake viewport behavior by:

- clipping a child while storing scroll state somewhere unrelated
- painting a scrollbar for content that has no canonical scroll owner
- letting hit-testing or focus believe content is visible when the viewport has clipped it away

Rule 5: Scrollbars are promises, not decoration

If a scrollbar is visible, it promises:

- there is a real scroll owner
- the thumb reflects actual visible extent
- keyboard and mouse scrolling target that same owner
- hit-testing and focus use the same geometry the rail implies

A scrollbar must never be merely a painted affordance with separate hidden scroll truth elsewhere.

Rule 6: Layout geometry drives interaction geometry

Pointer hit-testing, wheel routing, focus movement, and scroll targeting should reuse the same retained geometry that layout/render already established.

That means:

- input routing should not rebuild a different mental model of the workspace
- scroll regions should not guess where panes are from visual pixels alone
- retained layout caches are legitimate when they preserve layout truth

This rule is the bridge between the design-language posture here and the runtime invariant Layout Owns Interaction Geometry.

Rule 7: Wrap, clip, and viewport are distinct choices

Overflow handling should be explicit:

- **wrap** when the content should remain fully readable inside flowing text
- **clip** when the content is intentionally bounded and off-edge detail is not currently navigable
- **viewport** when the content remains conceptually larger than its visible window and users need to move through it

Do not blur these:

- clipping is not scrolling
- wrapping is not resizing the model

- viewporting is not just clipping plus a painted thumb

Rule 8: Shell chrome must not lie about owned width

Headers, footers, gutters, split dividers, notification rails, and scrollbar overlays all participate in layout ownership. They must either:

- consume width/height explicitly, or
- overlay without changing the body's owned geometry

The body region should never be measured as if chrome were absent and then painted as if chrome had stolen width after the fact.

Practical consequences

When touching layout code, ask:

1. Who owns this whitespace?
2. If content overflows, where does scroll state live?
3. If a scrollbar is visible, which viewport does it speak for?
4. Does input routing use the same geometry that render used?
5. Would a resize reflow the same semantic layout, or accidentally change it?

If those answers are not explicit, the layout contract is still too vague.

Read this with

- layout localization pipeline
- Layout Owns Interaction Geometry
- packages/bijou-tui/ARCHITECTURE.md
- docs/design-system/patterns.md

24

Layout Localization Pipeline

docs/strategy/layout-localization-pipeline.md

Strategy note for how Bijou turns LayoutNode trees into local render roots

Why this exists

The LayoutNode path in @flyingrobots/bijou-tui is easy to use from the app side, but the runtime ownership model is easy to misunderstand.

The important distinction is:

- **layout localization** decides what coordinate space the tree owns
- **paint** only blits already-positioned surfaces into a target surface

If those two stages get blurred together, it becomes difficult to reason about:

- negative coordinates and off-origin roots
- where width and height are measured
- when layout is recomputed
- where retained-layout reuse should fit later

This note exists to make that path explicit.

The current runtime path

For a TUI render driven by run(...), the flow is:

1. app.view(model) returns either a Surface or a LayoutNode.
2. wrapViewOutputAsLayoutRoot(...) converts the view output into a runtime lay outRoot.
3. If the output was a LayoutNode, localizeLayoutNode(...) shifts the whole tree into a local non-negative coordinate space.
4. Layout-stage middleware such as motion and BCSS operate on that localized root.
5. The paint middleware blits node.surface values into the target surface at the node's already-localized rect.x / rect.y.
6. Diff and output stages serialize the target surface to terminal bytes.

The relevant seams today are:

- packages/bijou-tui/src/view-output.ts
- packages/bijou-tui/src/layout-node-surface.ts
- packages/bijou-tui/src/runtime.ts
- packages/bijou-tui/src/pipeline/middleware/paint.ts

Coordinate spaces

There are three distinct coordinate spaces in play.

1. Author space

This is the coordinate space returned by app.view(model).

In author space:

- a root node may start at negative x or y
- siblings may be positioned relative to a parent origin that is not (0, 0)
- the bounds of the full tree may extend outside the root node's own rect

That is legal input. The runtime does not require app authors to hand-normalize it.

2. Local root space

This is the coordinate space owned by the runtime pipeline.

localizeLayout(...) recursively measures the tree bounds and computes:

- width = maxX - minX
- height = maxY - minY
- dx = -minX
- dy = -minY

localizeLayoutNode(...) then translates the whole tree by (dx, dy) so the localized root and every descendant sit in a non-negative local coordinate space.

The important rule is:

> Paint should never have to rediscover where the root begins.

By the time the pipeline reaches paint, the root already owns a local origin.

3. Target-surface space

This is the actual framebuffer for the current render.

The target surface may be larger than the localized tree because the runtime normalizes to at least the current viewport size:

- width = max(viewport.width, localized.width)
- height = max(viewport.height, localized.height)

That means localization decides the tree's own bounds, while normalization decides the backing framebuffer size.

Localization is not paint

Localization and paint are separate responsibilities.

Localization does:

- recursive bounds measurement
- ownership of root translation
- conversion from author coordinates to local runtime coordinates
- reporting the minimal localized width and height

Paint does:

- walk the already-localized tree
- blit node.surface into the target surface
- trust the rects it is given

The paint middleware in packages/bijou-tui/src/pipeline/middleware/paint.ts is intentionally simple because localization already happened upstream.

If paint ever needs to reason about negative roots, accumulated offsets, or tree rebasing, the ownership boundary has already been violated.

The recursion model

measureLayoutBounds(...) and paintLayoutNodeWithOffset(...) are both recursive, but they solve different problems.

Bounds recursion

measureLayoutBounds(...) walks the whole tree and accumulates:

- smallest x
- smallest y
- largest x + width
- largest y + height

This is the phase that determines the translation and final localized size.

Paint recursion

paintLayoutNodeWithOffset(...) or the paint middleware then walks the tree again and blits node surfaces into the already-sized target surface.

That recursion is intentionally dumb:

- no geometry solving
- no bounds discovery
- no re-localization

It is only applying the results of earlier layout ownership.

When layout is recomputed today

In the base run(...) runtime, layout localization is recomputed on every render.

That happens because the Layout stage does this every frame:

1. call app.view(model)
2. call wrapViewOutputAsLayoutRoot(...)
3. store the result on state.layoutRoot

So today there is no shared retained-layout cache inside the generic runtime render path.

Other paths follow the same basic rule:

- normalizeViewOutput(...) recomputes localization when a scripted driver or test harness needs a Surface
- wrapViewOutputAsLayoutRoot(...) recomputes localization when the pipeline needs a runtime-owned root tree

That is correct today, but it is not the final word on performance.

Where retained layouts fit

Bijou already has retained-layout primitives in runtime-engine.ts, and the framed shell uses retained trees for input routing and hit testing.

The important current truth is:

- **retained layouts exist**
- **the generic render pipeline does not yet retain and reuse layoutRoot**

So retained layout work should not be described as if the base runtime already paints from a persistent layout cache.

The likely future fit is:

1. view(model) produces a logical tree in author space
2. localization establishes a runtime-owned local root

3. that localized root becomes the retained artifact
4. invalidation determines when it must be rebuilt
5. paint and hit testing both consume the same retained localized tree

The key design pressure is consistency. A future retained-layout system should not make paint use one coordinate contract while hit testing uses another.

Rule of thumb

If you are working in this area:

- treat localization as the handoff from author coordinates to runtime coordinates
- keep paint dumb
- keep target-surface sizing separate from tree localization
- describe retained layouts as a future reuse seam unless you are explicitly in the shell/runtime-engine retained-layout code

Read this with

- packages/bijou-tui/src/view-output.ts
- packages/bijou-tui/src/layout-node-surface.ts
- packages/bijou-tui/src/runtime.ts
- packages/bijou-tui/src/pipeline/middleware/paint.ts
- packages/bijou-tui/src/runtime-engine.ts
- packages/bijou-tui/ARCHITECTURE.md
- packages/bijou-tui/ADVANCED_GUIDE.md

25

Layout Owns Interaction Geometry

docs/invariants/layout-owns-interaction-geometry.md

Protected by legends

- RE — Runtime Engine
- DL — Design Language

The authoritative geometry for interaction comes from retained layout, not from ad hoc handler registration or render-time guessing.

Implications:

- hit testing should use layout rects
- scroll ownership should align with layout-owned viewport geometry
- when layout changes, interaction geometry changes with it
- components should not keep stale global registrations after moving, resizing, hiding, or disabling

26

Localization and Bidirectionality

docs/strategy/localization-and-bidirectionality.md

Design note for locale-aware Bijou shells, content, and directional layout

Why this exists

Localization in Bijou should not mean:

- translating a few strings later
- assuming English layout forever
- bolting RTL support onto already hardcoded left/right shell behavior

Locale affects:

- copy
- wrapping
- value placement
- date/time/number formatting
- help and hint text
- reading direction
- layout mirroring

This note exists so Bijou can treat localization and bidirectionality as framework concerns early, before more LTR and English assumptions harden.

Concrete follow-on package work is currently tracked as backlog under:

- LX-001 — bijou-i18n Runtime and Tooling Packages

Product stance

Bijou should provide:

- localization-friendly APIs
- catalog-based localization runtime
- generic resource and data lookup, not only strings
- shell support for localizable message descriptors
- locale-aware formatting seams

- logical layout direction

Bijou should not assume:

- filesystem storage
- a specific translation backend
- a specific persistence mechanism
- English as the default spatial budget

By default, the framework can remain in-memory and adapter-driven.

Design principles

1. Localized text must reflow honestly

A layout should not depend on one English string length.

When localized text grows, Bijou should prefer:

- stacking
- wrapping
- secondary-line fallbacks

over:

- clipping
- collision
- ambiguous truncation

2. Use logical direction, not only physical direction

Framework APIs should increasingly think in:

- start
- end
- leading
- trailing

instead of only:

- left
- right

Physical left/right should remain available as explicit escape hatches, but logical direction should become the default mental model.

3. Text direction and layout direction are not identical

A locale may be RTL while still containing many LTR fragments:

- file paths

- commands
- IDs
- URLs
- code
- numbers

Bijou should not mirror or reorder these blindly.

4. Shell chrome must be localizable

The shell should not hardcode English strings for:

- help labels
- search labels
- settings labels
- notification labels
- quit-confirm copy
- empty states

Those surfaces should be designed to accept localized message descriptors later, even if today they are still powered by in-memory English defaults.

5. Translation workflow reality should be designed in

Real localization workflows involve:

- catalogs
- changed-source detection
- stale translations
- spreadsheet exchange
- translator-facing export and import
- shared references to controlled terms or structured data

Bijou should treat that as normal rather than pretending localization is always a tiny in-code dictionary.

6. Overflow policy should be explicit

For localized text, Bijou should prefer:

- wrap
- stack
- reflow

before:

- clipping
- collision
- motion

marquee can exist, but only as an explicit overflow policy for compact single-line affordances and not as the default localization fix.

7. Bidirectionality should be intentional

Not everything should mirror.

Bijou should define what is:

- logical and mirrored
- physical and fixed
- content and direction-sensitive

RTL / BiDi posture

What should generally mirror

- shell drawer default side when it is conceptually “start” or “end”
- side-rail ordering in standard shells
- breadcrumb flow
- disclosure and directional affordances
- pane sequencing where order is semantic rather than physical

What should generally not blindly mirror

- code
- IDs
- paths
- commands
- URLs
- raw tabular data unless explicitly directional

Why this matters early

If we hardcode left/right everywhere now, later RTL support becomes invasive across:

- app frame
- split panes
- drawers
- notifications
- help
- settings
- breadcrumbs

That is avoidable if we start naming and thinking logically now.

Framework direction

Longer term, Bijou should likely carry directional state in context or equivalent render configuration, something like:

- direction: 'ltr' | 'rtl' | 'auto'

That would allow shell, components, and later blocks to make consistent decisions without local hacks.

Localization tooling direction

Future framework/tooling support should consider:

- message-descriptor APIs
- catalogs for message, resource, and data entries
- locale-aware formatting helpers
- pseudo-localization mode
- spreadsheet import/export workflows
- stale-translation detection
- long-string stress testing
- layout overflow warnings for localized shells
- audits for hardcoded user-facing strings in shell/component layers
- dedicated runtime and tooling package boundaries for localization work

DOGFOOD implication

DOGFOOD should eventually prove:

- localized shell chrome
- direction-aware shell layout
- profile/help/search/settings copy that remains legible under localization pressure

Full RTL DOGFOOD does not need to be the first implementation target, but DOGFOOD should not keep teaching LTR-only assumptions forever.

Relationship to other doctrine

This note should be read alongside:

- Bijou UX Doctrine
- Accessibility and Assistive Modes
- Content Guide

Acceptance bar

This direction is working when:

- Bijou stops introducing new unnecessary physical left/right assumptions
- shell/content copy is treated as localizable framework surface
- long localized strings have honest layout behavior
- RTL is designed as a first-class future capability instead of a retrofit fantasy

27

Low-Allocation Renderer

docs/strategy/low-allocation-renderer.md

Design note for reusable framebuffers and a low-garbage hot render path

Why this exists

The DOGFOOD title screen made a real renderer truth visible:

- app-side title-screen work was slow enough to optimize
- terminal paint cost still dominates at very large window sizes
- even after the title-screen perf pass, the core runtime and surface stack still generate a lot of temporary garbage every frame

That means the next performance step is not another one-off screen optimization. It is renderer architecture.

The question is not:

> “Can we make this one shader faster?”

The real question is:

> “Can the Bijou runtime render rich full-screen surfaces without allocating a storm of temporary objects every frame?”

The answer is:

- **we can get much closer**
- **but not with the current runtime/surface hot path**

Evidence

DOGFOOD title-screen benchmark

At 220x58, the optimized DOGFOOD title screen now benchmarks at:

- 3.39 ms/frame for update + view + normalize
- 3.98 ms/frame for update + view + normalize + diff/write into test IO

That is a major improvement over the earlier path:

- 12.27 ms/frame before for update + view + normalize
- 11.81 ms/frame before for update + view + normalize + diff/write

So there was real waste in the title screen itself, and it was worth removing.

Allocation profile

The remaining problem is not retained memory. It is temporary churn.

Measured over 600 DOGFOOD landing frames at 220x58:

- transient heap delta: about 6.6 MB
- retained heap delta after forced GC: about 213 KB

So this is **not a leak**. It is **temporary garbage production**.

GC behavior

The trace confirms that the runtime is feeding the collector frequently:

- repeated minor scavenges
- at least one old-space mark-compact pause around 24 ms

That is enough to matter for animation smoothness and pacing.

The current bottlenecks

1. Surface is object-heavy by design

The current Surface API is ergonomic, but not cheap:

- get() returns masked cell copies
- set() writes through applyMask(...), which returns a fresh Cell
- clear() replaces every cell object
- clone() deep-copies the whole surface
- getRow() allocates both arrays and masked cell copies

This is fine for a friendly public API. It is not fine for a hot renderer.

2. The runtime allocates a fresh target surface every frame

Today the runtime:

- allocates a new targetSurface for every render
- diffs it against currentSurface
- then clones the target into currentSurface

That means the frame loop is paying for:

- one full-surface allocation
- one full-surface clone

on every frame, even before terminal paint cost is considered.

3. The diff path allocates while reading cells

renderDiff(...) currently walks the grid through surface.get(...).

That means the diff hot loop allocates just to inspect cells. It also:

- builds temporary token objects
- builds batchText strings
- builds a growing output string

Some string allocation is unavoidable in an ANSI renderer. The per-cell object churn is not.

4. The style layer is string-oriented

The Node style adapter ultimately produces styled strings. That is expected, but it means the final bridge to the terminal will never be literally zero-allocation.

So the goal should be:

- **near-zero-allocation in the hot render loop**
- not a fantasy of zero allocation everywhere

The decision

Bijou should pursue a **low-allocation renderer**, not promise a literally zero-allocation runtime.

The design decision is:

1. keep the current Surface API as the ergonomic public composition model
2. stop treating that API as the final hot-path framebuffer contract
3. introduce reusable framebuffers and direct-cell diffing in the runtime
4. move toward a dedicated internal mutable render buffer if the first stage proves worthwhile

In short:

> Surface remains the authoring/composition type.
> The runtime grows a lower-allocation framebuffer path underneath it.

Non-goals

This effort does **not** mean:

- rewriting the whole design system around typed arrays immediately
- removing all object allocation from every part of Bijou
- breaking the public Surface API in the first slice
- optimizing away terminal-emulator paint costs that live outside Bijou

Those would blur the work and make it much riskier than it needs to be.

Principles

1. **Fix the hot path first** We should remove runtime/framebuffer churn before inventing a whole new render substrate.
2. **Preserve public ergonomics** App authors should not need to think in framebuffers just to build normal components.
3. **Make each step benchmarkable** Every stage should have before/after benchmarks and GC observations.
4. **Tests are the spec** Runtime, diff, and resize behavior must stay covered while the internals change.
5. **Avoid premature big-bang rewrites** The first slice should be a contained runtime improvement, not a wholesale new rendering engine.

Proposed architecture

Stage 1 — Runtime framebuffer reuse

Replace the current per-frame target allocation and post-diff clone with a double-buffer model:

- allocate frontBuffer once per viewport size
- allocate backBuffer once per viewport size
- render into the back buffer
- diff front vs back
- swap references

This removes the largest guaranteed per-frame allocations without changing public component APIs.

Stage 2 — Direct-cell diff fast path

Teach renderDiff(...) to read cells directly from surface.cells in the hot loop instead of calling surface.get(...).

That should eliminate:

- masked read allocations

- extra per-cell object copying during diff

It also creates a clearer path to style-run caching later.

Stage 3 — Mutable/internal render buffer

If stages 1 and 2 are not enough, add an internal runtime type such as:

- FrameBuffer
- MutableSurface
- or RenderSurface

This type should:

- mutate cells in place
- support clear/fill/blit without per-cell object replacement
- expose direct array access for diffing

This is the likely bridge between the public Surface model and a truly lean renderer.

Stage 4 — Packed style and glyph data

Longer-term, the renderer can move toward:

- compact style ids
- bitflags for modifiers
- glyph/string lookup tables
- packed arrays instead of per-cell object shapes

This is where the renderer starts behaving more like a real framebuffer instead of a grid of JS objects.

Stage 5 — ANSI emission optimization

Once the buffer model is leaner, improve the terminal bridge itself:

- cache style runs
- reduce repeated SGR construction
- avoid unnecessary string concatenation churn
- write larger coherent chunks

That still will not be literal zero-allocation, but it will be much closer to the practical lower bound for ANSI output.

Scope for the next implementation slice

In scope

- reusable front/back surfaces in the runtime
- no per-frame targetSurface = createSurface(...)
- no per-frame currentSurface = targetSurface.clone()
- direct-buffer swap on successful output
- benchmarks and runtime tests proving equivalence

Out of scope

- a new public Surface API
- packed typed-array cells
- changing component authoring patterns
- redesigning the style port
- terminal-specific output throttling or adaptive quality logic

Acceptance criteria

The first slice should be considered successful if:

1. runtime behavior is unchanged from the app author's point of view
2. framebuffers are reused across steady-state renders
3. the per-frame allocation profile improves materially
4. render benchmarks and smoke suites stay green
5. resize behavior still replaces buffers correctly when dimensions change

Open questions

1. Should the reusable back buffer be fully cleared each frame, or should some middleware rely on an already-empty surface contract that needs stronger enforcement?
2. Should the diff fast path be added to the existing Surface implementation, or should it be introduced only after a dedicated internal framebuffer exists?
3. How far can we push runtime reuse before the Surface object model itself becomes the dominant remaining cost?

Recommendation

Proceed immediately with **Stage 1: runtime framebuffer reuse**.

It is the highest-leverage improvement because:

- it removes unavoidable per-frame allocations already visible in the runtime
- it does not require a public API break

- it sets up every later optimization
- it is easy to validate with benchmarks and tests

After that, take **Stage 2: direct-cell diff fast path** before deciding whether Stage 3 needs a new internal buffer type.

28

Notification History Belongs to the Shell

docs/strategy/notification-history-belongs-to-the-shell.md

Design note for a shell-owned notification center in createFramedApp()

Why this exists

Bijou already has the underlying notification substrate:

- stacked live notifications
- actionable notifications
- archived notification history
- runtime notification routing in the frame shell

That means the next problem is not “can Bijou archive notifications?” It is:

- where does a human go when they miss one?
- how do they review important notices without reopening a one-off demo or reading logs?
- how do notifications feel like part of the shell instead of a bolt-on overlay system?

If every framed app invents its own history modal, drift is guaranteed:

- different entry points
- different close behavior
- different filters
- different scroll semantics
- different footer/status cues

That is the same kind of product drift that justified shell-owned settings and shell-owned quit policy. Notification history should follow the same path.

Design-thinking frame

Primary user

The primary user is the **workspace operator**:

- they are in the middle of real work
- a toast or warning may have appeared while they were reading somewhere else
- they need a calm place to recover missed information without losing context

They are asking:

- “What just happened?”
- “Did I miss something important?”
- “Where do I go to review notices after they disappear?”
- “Can I trust that important shell feedback has a durable home?”

Secondary user

The secondary user is the **workspace builder**:

- a developer using createFramedApp()
- someone who wants a standard review surface for shell/app notifications
- someone who does not want to rebuild archive modals, footer counters, filters, and keyboard behavior for every app

They are asking:

- “Can the shell give me a standard notification center?”
- “Can I surface important notices without building a custom inbox UI?”
- “Will notification review stay coherent with help, settings, search, and quit?”

Jobs to be done

1. **Help me recover missed feedback.** If a transient notice mattered, I need a stable place to review it later.
2. **Help me distinguish feedback by weight.** Not every message deserves the same treatment. Toasts, persistent notifications, and reviewable history should feel intentionally different.
3. **Help me review without losing my place.** I should be able to inspect notifications while keeping the current workspace visible behind the shell surface.
4. **Help me implement durable feedback once.** App authors should supply notification state and optional actions, not reinvent archive UI and shell entry points.

Hills

Hill 1 — Missed notices are recoverable

A user can reliably open one shell-owned place to review current and recent notifications.

Hill 2 — Feedback feels proportional

Low-weight toasts stay lightweight. Important notices remain reviewable. The shell does not force every event to become a blocking modal.

Hill 3 — Notification review feels shell-owned

The center behaves like a sibling of help, search, settings, and quit, not like a page-specific hack.

Hill 4 — Context remains visible

The notification center should feel like a sidecar review surface, not a full app detour.

Product direction

The shell should provide a **right-edge notification center**.

The mental model is:

> toasts are what just happened; the notification center is where I go to review what mattered.

Why a right-edge drawer:

- settings already occupy the left edge as the “change preferences” sidecar
- notifications are naturally review-oriented rather than configuration-oriented
- a right-edge surface lets the current workspace remain visible
- it gives the shell a balanced left/right sidecar model:
  - left: settings
  - right: notification review

This should be treated as a shell surface, not as a special page.

Feedback ladder

The shell should make these layers feel distinct:

Toast

- ephemeral
- low-weight
- auto-dismiss
- no expectation of deep recall by itself

Notification

- meaningful enough to matter
- shell-managed timing/stacking/routing
- still visible in the active workspace context

Notification center

- durable review surface
- archive/inbox/history semantics
- filterable
- reachable through shell affordances and command search

This is the important design rule:

> **Toasts are not inboxes.**

If an event may matter after it disappears, the shell needs a review surface.

Relationship to the existing substrate

Bijou should build this feature on the notification model that already exists in notification.ts:

- NotificationState.items
- NotificationState.overflowExits
- NotificationState.history

That means:

- no second archive store
- no parallel “notification center state” that drifts from the notification stack
- no bespoke demo-only history format

The shell should own:

- container geometry
- entry points
- footer cues
- focus/scroll/input ownership

- review affordances

The app should own:

- notification meaning
- optional archived-item actions
- optional filter/value policy when custom notification state is supplied

Visual model

The intended feel is a calm right-edge inbox drawer, not a giant modal wall of prose.

Rough structure:

```
+--------------------------------------+
| Notifications                   All |
+--------------------------------------+
| Live: 2 • Archived: 14               |
|--------------------------------------|
| ERROR    2m ago                      |
| Deploy blocked                       |
| Runtime failed to boot candidate     |
| Retry deploy                         |
|--------------------------------------|
| WARNING  4m ago                      |
| Queue pressure rising                |
| Latency trending upward              |
|--------------------------------------|
| INFO     8m ago                      |
| Background sync ready                |
+--------------------------------------+
```

Important properties:

- right-edge shell drawer
- short header with title and active filter
- compact status line summarizing live vs archived counts
- vertically scrollable entries
- tone, time, title, message snippet, and optional action cue
- no attempt to turn the drawer into a full log viewer

Principles

1. **Toasts are not the archive** The stack is for current interruption; the center is for review.
2. **The shell owns recovery** Notification review should be as standard as help, search, settings, and quit.
3. **The current workspace stays visible** This is a sidecar, not a route transition.
4. **Footer tells the truth** The footer should surface a subtle notification/inbox cue so users know the center exists.
5. **Filters should stay simple** Start with pragmatic filters that match the existing notification substrate: All, Actionable, Error, Warning, Success, Info.

6. **Archived actions must be explicit** If an archived notification exposes a follow-up action, the shell should label that clearly as a deliberate review-time action, not pretend the toast never expired.
7. **Tests are the spec** The next implementation slice should be driven by failing app-frame tests and one proving surface, not by adding another app-specific history modal.

Proposed vo API shape

The frame should own the container. Apps should provide notification content declaratively.

```
interface FrameNotificationCenter<Msg> {
  readonly title?: string;
  readonly state: NotificationState<Msg>;
  readonly filters?: readonly NotificationHistoryFilter[];
  readonly activeFilter?: NotificationHistoryFilter;
  readonly onFilterChange?: (filter: NotificationHistoryFilter) => Msg | undefined;
  readonly onActivateItem?: (item: NotificationRecord<Msg>) => Msg | undefined;
  readonly emptyLabel?: string;
}

interface CreateFramedAppOptions<PageModel, Msg> {
  // existing fields...
  readonly notificationCenter?: (args: {
    readonly model: FrameModel<PageModel>;
    readonly activePage: FramePage<PageModel, Msg>;
    readonly pageModel: PageModel;
    readonly runtimeNotifications: NotificationState<never>;
  }) => FrameNotificationCenter<Msg> | undefined;
}
```

Why this shape:

- the shell still owns drawer layout and behavior
- apps can expose their own notification state when they already have one
- the frame can still provide a fallback center for frame-managed runtime notifications
- filters and archived-item activation stay opt-in and explicit

Standard shell behavior

Entry points

The shell should expose a stable way to open notification review:

- a standard shell binding such as Shift+N
- a command-palette entry like Open notifications
- a footer cue that implies review is available

The exact footer copy can evolve, but it should remain small and truthful.

Footer cue

The footer should not dump archive counts constantly.

It should prioritize:

- live notification count when there are active notices
- a subtle inbox/history affordance when the archive is non-empty

The footer cue is discoverability, not a second notification center.

Geometry

- anchored to the right edge
- bounded width with compact-terminal fallback
- keeps the active page visible behind it
- owns focus/input while open

Scroll and focus

- wheel scrolls the drawer when hovered
- arrow keys / j / k move through entries
- d / u, g / G page and jump
- the underlying workspace does not keep reacting while the drawer is open

Filters

v0 filters should stay aligned with the notification model already in the repo:

- All
- Actionable
- Error
- Warning
- Success
- Info

Cycling filters with one key is acceptable in v0. A richer row/segmented filter control can come later if it proves necessary.

Archived-item activation

Archived items should only expose a primary action when the app provides one explicitly.

That action should be framed honestly:

- “Run again”
- “Open details”
- “Retry”

not as if the archived notification were still a live toast waiting for immediate response.

vo proving surfaces

The first proving surfaces should be:

1. the framed notifications lab in examples/notifications/main.ts
2. frame-managed runtime notifications in createFramedApp()

DOGFOOD should adopt the shell cues later, but it should not be forced to invent fake notifications just to validate the shell container.

Out of scope for vo

The first slice should **not** try to solve everything:

- unread/read persistence
- cross-session archive storage
- grouped threads or notification threads
- a full log viewer
- snooze/remind-later workflows
- per-notification custom drawer layouts
- replacing structured app logs

Those can come later if the shell-owned review surface proves valuable.

What success looks like

When this is done well:

- a user sees a toast, misses it, and still knows where to go later
- a framed app gets durable notification review without a custom history modal
- the footer hints that review exists without becoming noisy
- notification review feels like part of the shell family instead of another isolated overlay

Recommended implementation order

1. Add failing app-frame tests for a shell-owned notification center:
    - open/close
    - scroll ownership
    - filter changes
    - footer cue

    - command-palette entry
2. Add one focused proving surface in the framed notifications lab.
3. Implement the right-edge drawer and shell routing in createFramedApp().
4. Reuse the existing notification archive model instead of introducing new storage.
5. Only after that, consider archived-item actions and richer inbox semantics.

29

Preference Lists Belong to Bijou

docs/strategy/preference-lists-belong-to-bijou.md

Design note for promoting shell settings presentation into canonical Bijou componentry

Why this exists

The shell settings drawer now looks materially better than it did at first:

- focused rows feel selected instead of merely pointed at
- descriptions make preferences legible
- narrow drawers can still explain what a row does
- choice and toggle rows read differently

That quality is good, but too much of it still lives inside shell-specific rendering code.

That is the wrong long-term direction for DOGFOOD.

If DOGFOOD is serious about “the docs are the demo,” then the docs shell should strive to use canonical Bijou componentry wherever possible. The title screen can remain a custom proving surface. The settings drawer should not.

The shell should own:

- placement
- drawer chrome
- input routing
- open / close behavior
- footer/help discoverability

But the rows themselves should be a standard Bijou visual language, not a pile of bespoke shell drawing rules.

Design-thinking frame

Primary user

The primary user is the **workspace operator**:

- someone navigating settings in a real framed app
- someone who expects toggles, choices, and descriptions to read clearly
- someone who should not have to mentally decode custom row layouts every time a shell drawer opens

They are asking:

- “What kind of setting is this?”
- “What is currently selected?”
- “What happens if I activate this row?”
- “Why does this row look selected?”

Secondary user

The secondary user is the **framework builder**:

- someone building on createFramedApp()
- someone who wants polished settings UI without copying DOGFOOD-only row presentation code
- someone who wants one canonical vocabulary for settings, preferences, onboarding lists, and maybe later notification review or inspectors

They are asking:

- “Can I get this settings-row quality without hand-drawing it myself?”
- “Can the shell depend on stable preference-list components rather than shell-local rendering?”
- “Can the same row family be reused outside settings?”

Problem statement

Right now the shell settings drawer is conceptually correct but architecturally inverted:

- the shell owns the right behavior
- but the shell also owns too much bespoke row rendering

That makes the shell harder to evolve and weakens the DOGFOOD promise:

- the docs are not proving a canonical preference-list family
- shell polish risks becoming trapped inside app-frame.ts
- later surfaces may reinvent nearly identical “selected preference rows” instead of reusing a standard component

Product direction

Bijou should promote the current settings-row presentation into a canonical component family.

Working name:

- preferenceListSurface()
- preferenceRowSurface()
- optional row helpers for:
  - toggle
  - choice
  - info
  - action

The shell settings drawer should then become:

- shell-owned container behavior
- canonical Bijou preference-list rendering

That split keeps the shell strong without letting it become a custom UI toolkit inside the UI toolkit.

Principles

1. **DOGFOOD should use canonical componentry** If a shell surface becomes polished and reusable, it should graduate into standard Bijou presentation instead of staying bespoke.
2. **The shell owns behavior, not every pixel** Drawer placement, input ownership, dismissal, and discoverability belong to the shell. The row family should belong to Bijou componentry.
3. **Preference rows should be semantically legible** A user should be able to distinguish:
   - selected row
   - row label
   - current value
   - description
   - toggle vs choice vs info
4. **Compact-width truth matters** If the value will collide with the label, the component should adapt honestly by stacking or wrapping rather than truncating into ambiguity.
5. **Persistence is not a default framework concern** The component family and shell settings experience should work entirely in memory. Persistence, if added later, should come through an explicit port/adapter, not a hidden framework assumption.

Proposed component-family scope

In scope

- canonical preference row rendering
- selected-row treatment

- value-label placement rules
- stacked-value fallback when inline space is too tight
- row descriptions
- row-kind affordances:
  - toggle
  - choice
  - info
  - action
- token-aware theming for:
  - selected state
  - value emphasis
  - muted descriptions

Out of scope

- persistence of preferences across sessions
- arbitrary form controls embedded inside rows
- editable text fields inside the first preference-list family
- a full application settings persistence framework
- replacing every shell surface with a generic list in the same slice

Proposed API direction

The exact names can change, but the family should roughly support:

```
interface PreferenceRow {
  readonly id: string;
  readonly label: string;
  readonly description?: string;
  readonly valueLabel?: string;
  readonly kind?: 'toggle' | 'choice' | 'info' | 'action';
  readonly selected?: boolean;
  readonly checked?: boolean;
  readonly enabled?: boolean;
}

interface PreferenceSection {
  readonly id: string;
  readonly title: string;
  readonly rows: readonly PreferenceRow[];
}
```

The shell can still keep its current higher-level FrameSettings model and adapt that model into the canonical row family.

That preserves shell ergonomics while standardizing the actual row rendering.

Why this matters beyond settings

Once the preference-row family exists, it has clear follow-on value:

- shell settings drawers
- onboarding/tutorial step lists
- notification review rows
- future inspectors or property panels

So this is not just a shell cleanup. It is a design-system promotion step.

Relationship to DOGFOOD

DOGFOOD should remain the first proving surface:

- the shell settings drawer is where the current row treatment became credible
- DOGFOOD should be the first user of the canonical component family
- once that family exists, DOGFOOD stops "special-casing a nice settings menu" and starts proving the actual product vocabulary

That is exactly the kind of migration DOGFOOD should encourage.

Relationship to settings persistence

Persistence is intentionally not part of this slice.

Framework stance for now:

- settings can live entirely in memory
- the shell can read and write them inside the page/app model
- if persistence is needed later, it should come through a port/adapter boundary
- Bijou should not assume filesystem, database, browser storage, or any other capability by default

That keeps the framework honest and composable.

Acceptance bar

This direction is successful when:

- the shell settings drawer no longer hand-draws its own special row treatment
- DOGFOOD uses the canonical preference-list family
- the selected-row behavior, value placement, and description rhythm remain at least as good as the current bespoke version
- persistence is still absent by default and explicitly out of scope

Recommended implementation sequence

1. Write failing design-system / shell tests for canonical preference-row rendering.
2. Build the smallest reusable preference-list surface family.
3. Migrate the shell settings drawer to use it.
4. Keep DOGFOOD as the first proving surface.
5. Only after the row family is stable, consider whether other shell surfaces should adopt it.

30

Runtime Truth Wins

docs/invariants/runtime-truth-wins.md

Protected by legends

- RE — Runtime Engine
- DX — Developer Experience
- LX — Localization and Bidirectionality

When runtime facts disagree with comments, type-only claims, or editor comfort, runtime truth wins.

Implications:

- trusted values must be established through construction, parsing, or validation
- a type annotation alone does not make data safe
- docs and JSDoc must describe runtime behavior that actually exists
- if the runtime and the types disagree, fix the mismatch instead of hand-waving it away

31

Schemas Live At Boundaries

docs/invariants/schemas-live-at-boundaries.md

Protected by legends

- RE — Runtime Engine
- DX — Developer Experience
- LX — Localization and Bidirectionality

Schemas and parsers should reject malformed input at system edges, then hand trusted values to runtime-backed domain forms.

Implications:

- unknown belongs at the edge, not deep in the core
- decoding and validation should happen before domain logic starts
- core behavior should not keep re-validating already trusted values
- plain decoded objects are not the final domain model

32

Settings Belong to the Shell

docs/strategy/settings-belong-to-the-shell.md

Design note for a frame-owned settings drawer in createFramedApp()

Why this exists

The DOGFOOD docs app exposed a product gap that is larger than the docs app itself:

- users want global preferences quickly
- they expect to reach them from the shell, not by navigating into a one-off page
- those preferences often affect the whole surface, not just the current page

Examples:

- theme selection
- animation or reduced-motion preferences
- input hints and help visibility
- output/profile defaults for a docs or workbench surface

If each framed app invents its own settings overlay, the result will drift:

- different entry points
- different keyboard behavior
- different scroll behavior
- different focus and dismissal rules
- different mouse expectations

That is exactly the kind of shell concern the standard app frame should own.

Design-thinking frame

Primary user

The primary user is the **workspace user**:

- they are already inside a framed app
- they want to adjust preferences without leaving their place
- they expect settings to feel global, stable, and discoverable

They are asking:

- “Where do I change app-wide preferences?”
- “Can I do that without leaving the current page?”
- “Will the shell behave the same way in every framed app?”

Secondary user

The secondary user is the **workspace builder**:

- a developer using createFramedApp()
- someone who wants settings UX without re-implementing drawers, key routing, or scrolling preferences

They are asking:

- “Can the frame give me a standard settings surface?”
- “Can I declare preferences, sections, and actions instead of rebuilding infrastructure?”
- “Will this stay coherent with help, palette, and modal routing?”

Jobs to be done

1. **Help me adjust global preferences in place.** I want to change shell- or app-level behavior without leaving the screen I am on.
2. **Help me trust the shell.** Settings should feel like a first-class shell affordance, not a special page pretending to be one.
3. **Help me keep settings readable and tappable.** Preferences should be easy to scan, scroll, and toggle in a narrow drawer.
4. **Help me implement preferences once.** App authors should declare settings data and effects, not rebuild list rendering, drawer motion, and input plumbing.

Hills

Hill 1 — Settings feel shell-owned

In a framed app, settings open from the shell and behave like the shell. They do not feel like a page hack or a random modal.

Hill 2 — Preferences are readable in a narrow lane

The drawer should support a vertically scrolling, sectioned preference list that works in compact widths and still feels deliberate.

Hill 3 — Input remains honest

When the settings drawer is open, it owns local input. The underlying page does not keep reacting as if the drawer were not there. Mouse, keyboard, and help should all tell the same truth.

Product direction

The shell should provide a **left-edge settings drawer** with a structured, scrollable preference list.

The mental model is:

> settings are a shell-owned sidecar, not a page transition.

That means:

- the current page remains visible behind it
- the drawer opens from the left edge
- the drawer can scroll independently
- the drawer owns input while open
- the rest of the shell remains visible but non-interactive underneath

The visual model

The intended feel is closer to an iOS-style settings list than to a long markdown help page.

Rough structure:

```
+-----------------------------------------+
| Settings                        [info]  |
+-----------------------------------------+
| Appearance                              |
|  Theme                          Paper   |
|  Motion                           On    |
|-----------------------------------------|
| Docs                                    |
|  Landing animation                  On  |
|  Profile default                Static  |
|-----------------------------------------|
| Shell                                   |
|  Show hints                       On    |
|  Mouse support                    On    |
+-----------------------------------------+
```

Important properties:

- section headings
- rows that read like preferences, not generic commands
- a compact trailing value or toggle affordance
- enough spacing to scan quickly
- scroll support when the list exceeds the drawer height

Principles

1. **Settings are shell state first** The frame should own open/close, focus, scroll, and entry affordances.
2. **Page content stays visible** This is a drawer, not a full route transition.
3. **Preferences should be declarative** App authors should define sections and rows, not low-level drawer plumbing.
4. **Mouse and keyboard should agree** Click, wheel, arrow keys, Enter, and Escape should all follow the same ownership model.
5. **Tests are the spec** The next implementation step should be failing shell tests, not one-off docs hacks.

Proposed vo API shape

The first public step should keep the shell responsible for the container while apps declare the content model.

```
interface FrameSettingRow<PageModel, Msg> {
  readonly id: string;
  readonly label: string;
  readonly description?: string;
  readonly valueLabel?: string;
  readonly action?: Msg;
  readonly kind?: 'action' | 'toggle' | 'choice' | 'info';
  readonly enabled?: boolean;
}

interface FrameSettingSection<PageModel, Msg> {
  readonly id: string;
  readonly title: string;
  readonly rows: readonly FrameSettingRow<PageModel, Msg>[];
}

interface FrameSettings<PageModel, Msg> {
  readonly title?: string;
  readonly sections: readonly FrameSettingSection<PageModel, Msg>[];
}

interface CreateFramedAppOptions<PageModel, Msg> {
  // existing fields...
  readonly settings?: (args: {
    readonly model: FrameModel<PageModel>;
    readonly activePage: FramePage<PageModel, Msg>;
    readonly pageModel: PageModel;
  }) => FrameSettings<PageModel, Msg> | undefined;
}
```

Why this shape:

- the shell still owns layout, drawer chrome, and interaction rules
- the app only supplies structured preference content
- rows can dispatch messages back into the page/app model without exposing shell internals
- the model leaves room for future persistence without forcing it in v0

As the first proving surface matured, the row treatment itself proved valuable enough to deserve promotion into canonical Bijou componentry. That follow-on direction is captured in Preference Lists Belong to Bijou and Preference List Surface.

Standard shell behavior

Entry

The shell should expose a standard way to open settings:

- a frame action and binding such as ctrl+,
- a command-palette entry like Open settings

This should be part of the shell, not left to every app.

Drawer placement

- anchored to the left edge
- consistent width policy, likely a bounded percentage of screen width
- standard header row with title and optional lightweight info glyph

Scroll behavior

- the settings list scrolls independently
- wheel scrolling over the drawer scrolls the drawer
- arrow keys, j/k, d/u, g/G can scroll the list when it is focused/open
- the underlying page does not scroll while the drawer is active

Selection behavior

For v0, the simplest honest behavior is:

- one active row
- arrow keys move between rows
- Enter triggers the row action
- Space can toggle boolean rows
- mouse click activates a row

The shell should provide enough behavior that the list feels native even before richer controls exist.

Dismissal

- Esc closes the drawer
- shell help and command palette remain able to preempt it if intentionally opened
- clicking outside the drawer should close it only if we decide the shell uses outside-click dismissal consistently; otherwise v0 should avoid surprise dismissal

Input layering

This feature depends on the Focus Owns Input direction.

Conceptually, the order should be:

1. shell modal layers
2. settings drawer layer
3. page modal layer
4. frame shell layer
5. focused pane layer
6. page/global fallback

The important behavior is:

- when settings are open, the settings drawer owns local navigation and scroll
- the underlying page does not continue responding
- the shell can still intentionally open help or palette above settings

This is why settings should be built into the frame, not bolted onto one page.

Mouse model

Mouse should be first-class here.

vo expectations

- click inside a row focuses and activates/selects it
- wheel scroll inside the drawer scrolls the settings list
- click inside the drawer does not leak through to the page
- click in shell chrome still hits shell chrome if it is intentionally above the drawer layer

If we do not build mouse support into this, the drawer will feel unfinished immediately.

Help model

When settings are open:

- the short help strip should mention settings navigation and dismissal
- the full help view should reflect the settings layer rather than only the current page

That keeps the shell honest and discoverable.

Scope

In scope for v0

- shell-owned settings drawer in createFramedApp()
- declarative settings sections and rows
- scrollable settings list
- keyboard navigation and selection
- mouse click and wheel support
- command-palette entry to open settings
- docs app as a proving surface

Out of scope for v0

- settings persistence across sessions
- arbitrarily rich embedded controls inside rows
- editable text fields inside settings
- page-specific custom drawer layouts
- a full generalized preferences framework for every component in the repo

Tests are the spec

The next implementation loop should start with failing tests in app-frame.test.ts and docs-preview.test.ts.

Minimum acceptance tests

1. **The frame can open a standard settings drawer**
   - A standard shell action opens settings without the page building a one-off overlay.
2. **The settings drawer owns input while open**
   - Arrow keys and scroll actions move within settings instead of mutating the underlying page.
3. **Mouse input does not leak through the settings drawer**
   - Clicks and wheel events inside the drawer are consumed by settings behavior.
4. **The settings list scrolls when content exceeds drawer height**
   - Long preference lists remain usable in compact terminals.
5. **The command palette can open settings**
   - The shell exposes settings as a discoverable shell command, not just a hidden keybinding.
6. **The docs app can expose preferences without custom drawer plumbing**

- DOGFOOD becomes a proving surface, not a special-case shell fork.

Recommended first proving surface

The DOGFOOD docs app is the right v0 proving surface because it already has obvious shell-level preferences:

- landing theme preset
- default profile
- animation on/off
- hint visibility

That is enough to validate:

- drawer layout
- shell input ownership
- mouse routing
- command-palette entry
- scrollable settings sections

without overdesigning persistence or form controls too early.

Recommended next implementation loop

1. Write failing app-frame tests for opening, scrolling, and owning input.
2. Add a failing docs-preview test that opens settings and changes one visible preference.
3. Implement the shell-owned drawer and basic settings list renderer.
4. Prove the docs app can adopt it with minimal custom code.

The key discipline is the same as the rest of the post-v4 work:

> design first, tests second, implementation third.

33

Shell Owns Shell Concerns

docs/invariants/shell-owns-shell-concerns.md

Protected by legends

- HT — Humane Terminal
- DF — DOGFOOD Field Guide

Global shell behavior should belong to the shell, not be reimplemented by each page.

Examples:

- search
- help
- settings
- notifications
- quit and confirmation flows

Implications:

- pages declare content and page-local behavior
- shell layers own shell routing, chrome, and discoverability
- polished shell patterns should graduate into canonical componentry when reusable

34

State Machine and View Stack Are Distinct

docs/invariants/state-machine-and-view-stack-are-distinct.md

Protected by legends

- RE — Runtime Engine

Application state and visible views are related, but they do not mean the same thing.

Implications:

- a state may outlive a particular view or layer
- a view may persist while underlying state changes
- state transitions may replace or clear the active view stack
- view-stack operations should not be used as a synonym for application-state transitions

35

Tests Are the Spec

docs/invariants/tests-are-the-spec.md

Protected by legends

- WF — Workflow and Delivery
- RE — Runtime Engine
- DF — DOGFOOD Field Guide
- DL — Design Language

If behavior matters, it must be proved in tests.

Implications:

- design docs should identify the tests to write first
- golden-path tests are not enough by themselves
- edge cases, failure cases, and stress/fuzz cases matter when warranted
- implementation does not get to quietly overrule tested behavior

36

Topmost Layer Dismisses First

docs/invariants/topmost-layer-dismisses-first.md

Protected by legends

- HT — Humane Terminal
- RE — Runtime Engine

When multiple interactive layers are open, dismiss actions should target the topmost dismissible layer before they are interpreted as broader shell actions.

Implications:

- overlays, drawers, palettes, and modals should dismiss in top-down order
- Esc should not both dismiss one layer and trigger quit or another action beneath it
- the topmost layer owns the active controls and the visible footer/help hints
- underlying panes and regions remain inert until the covering layer is dismissed

37

Visible Controls Are a Promise

docs/invariants/visible-controls-are-a-promise.md

Protected by legends

- HT — Humane Terminal
- DF — DOGFOOD Field Guide
- DL — Design Language

If a control hint is shown, it should work right now for the active layer or region.

Implications:

- stale legends are a bug
- hidden or covered regions should not keep advertising dead controls
- footer and help surfaces should prefer accuracy over verbosity

II

Asing Bijou

From principles to an application

This part gathers the material used while building with Bijou. It begins with orientation and end-to-end terminal scenarios, moves through the documentation application and development tools, and then provides the design-system and API references needed for more detailed work.

The package boundaries matter. Core components and surfaces express terminal content. The TUI runtime supplies the application loop, input and layout machinery. Node adapters provide terminal and host integration. The framed application shell assembles common application behaviour. Keeping those roles distinct helps a small command-line tool remain small and helps a larger application put each responsibility in the appropriate place.

Choose a route through the material

For a first application, read the Guide and terminal-scenarios chapter before the advanced rendering material. For a specific capability, follow the package, component-family or reference chapter that owns its contract. Read the render-pipeline and advanced-guide chapters when frame production, effects or performance become the question. Migration chapters should be read with their release version in view.

DOGFOOD and BlockLab provide two useful routes into the project. DOGFOOD presents the documentation and component explorer inside a full application shell. BlockLab concentrates on the story catalog, previews, variants and profiles. Theme Lab is an application surface for exploring theme editing and relationships. Their design records appear later in the book; the practical notes here explain how these surfaces fit into development and validation.

Examples and reference material

Examples illustrate a contract within a particular context. Check which package version, output mode and host an example assumes before adapting it. A component that can be displayed in several modes still needs a caller to choose the correct runtime and output path.

The reference chapters provide denser material: token rules, family inventories, schemas, diagnostics and exact boundary behaviour. They support lookup during implementation. The first part explains why those boundaries matter; the design-record part explains how individual boundaries were introduced or changed. Together they provide a path from an immediate task to the reasoning behind its constraints.

38

Guide — Bijou

GUIDE.md

This is the developer-level operator guide for Bijou. Use it for orientation, the productive-fast path, and to understand how the monorepo packages orchestrate the terminal surface.

For deep-track doctrine, render pipeline internals, and repository-wide engineering standards, use ADVANCED_GUIDE.md.

Choose Your Lane

1. Build a CLI or Script

Use the core toolkit for prompts, structured output, and environment detection. - **Read**: @flyingrobots/bijou Guide - **Host**: @flyingrobots/bijou-node Guide

2. Build an Interactive TUI

Use the TEA runtime for full-screen applications with layout, motion, and overlays. - **Read**: @flyingrobots/bijou-tui Guide - **Host**: @flyingrobots/bijou-node Guide

3. Build a Framed App

Use the compact framed app tutorial when the counter is too small but DOGFOOD is too large to learn from. - **Run**: npm run app-frame - **Read**: Framed app tutorial

4. Learn the Monorepo

Understand the project's identity, work doctrine, and architectural boundaries. - **Read**: docs/README.md (The Documentation Map) - **Read**: docs/VISION.md (Core Tenets) - **Read**: Contributor workflow

5. Proving Ground

See Bijou in action by running the documentation app. - **Run**: npm run dogfood - **Read**: docs/DOGFOOD.md

Big Picture: Package Orchestration

Bijou is a tiered engine. You choose your depth based on the task:

1. **@flyingrobots/bijou (Foundation)**: Pure TypeScript. No I/O. Logic for boxes, tables, trees, and prompts. This layer produces Surface or LayoutNode objects.
2. **@flyingrobots/bijou-tui (Engine)**: The TEA loop. It orchestrates those surfaces, manages focus, drives animations, and calculates the minimal ANSI delta to update the screen.
3. **@flyingrobots/bijou-node (Host)**: The physical bridge. It provides the terminal dimensions, handles stdin events, and emits the final ANSI bytes to stdout.

Orientation Checklist

- ☐ **I am building a standalone tool**: Start with packages/bijou/GUIDE.md.
- ☐ **I am building a full-screen dashboard**: Start with packages/bijou-tui/GUIDE.md.
- ☐ **I need to localize my app**: Start with packages/bijou-i18n/README.md.
- ☐ **I am debugging the render loop**: Start with ADVANCED_GUIDE.md.
- ☐ **I am contributing to the repository**: Start with docs/WORKFLOW.md and the GitHub issue tracker.

Rule of Thumb

For a comprehensive inventory, use docs/README.md.

For current work and planned releases, use GitHub issues and milestones.

New to the project? Start with the README.md and the package guides linked above.

The goal is to move the terminal from a collection of widgets to a professional application bedrock.

39

End-To-End Terminal Scenarios

docs/guides/terminal-scenarios.md

This guide walks through Bijou from small one-shot terminal output to full-screen TUI apps and shader-driven visual renderers. It is scenario-first: pick the thing you want to build, then follow the package boundary and control flow that fits the job.

Use this guide when you are asking one of these questions:

- I want to print some colors.
- I want to make an interactive form.
- I want to make a basic TUI app.
- I want to make a Braille ray trace renderer.

For a live tour of the repository product surface, run:

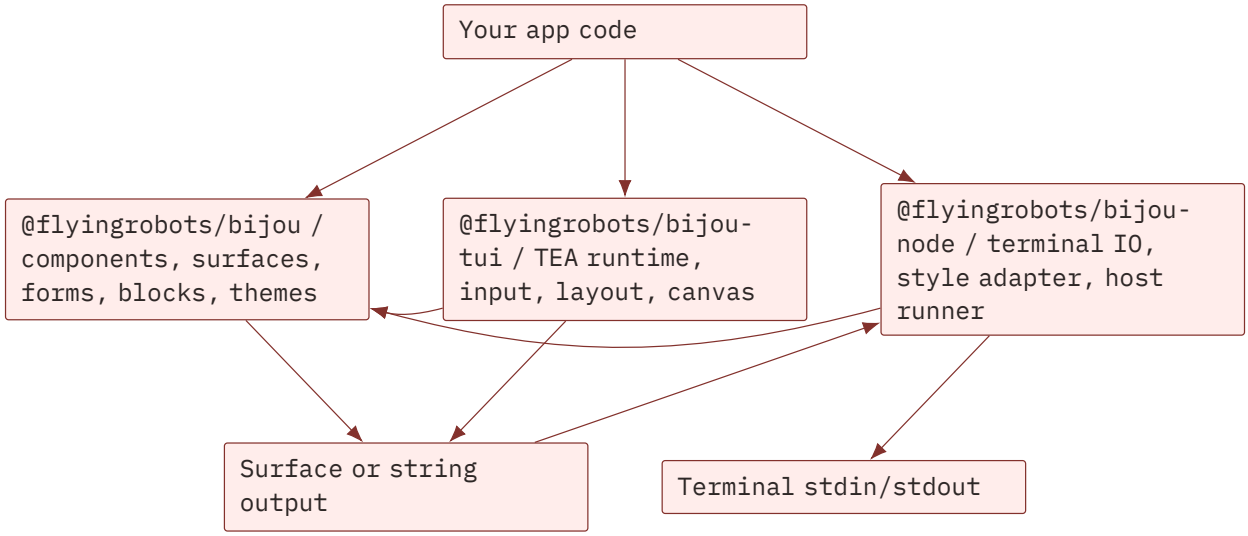
```
npm run dogfood
```

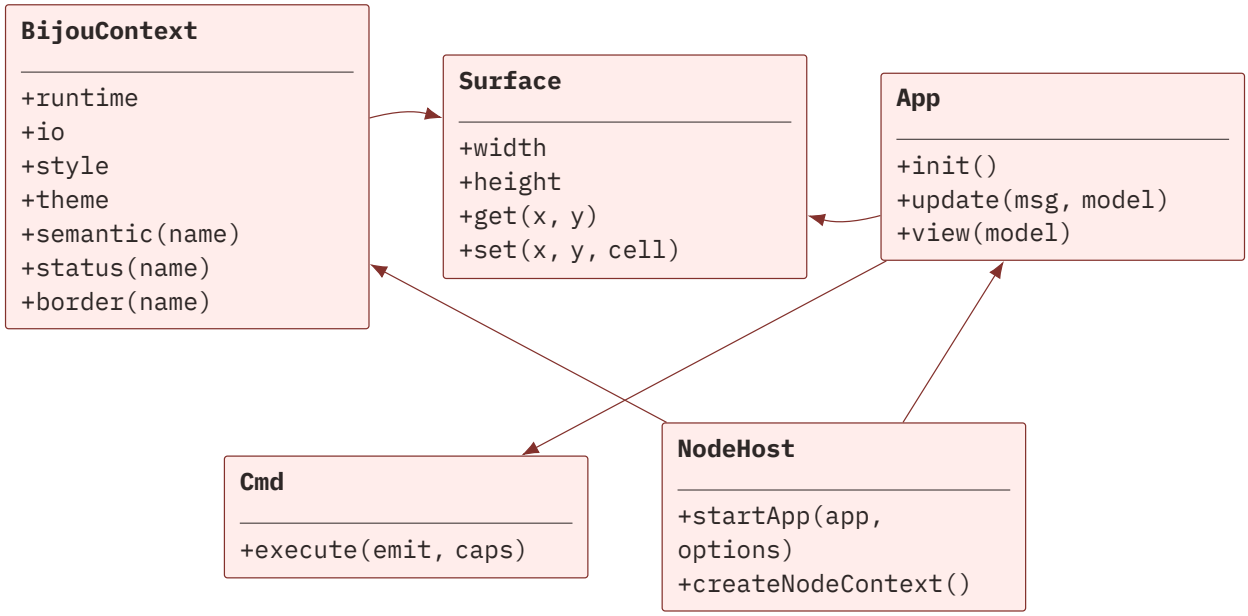
For focused component and preview work, run:

```
npm run blocklab
```

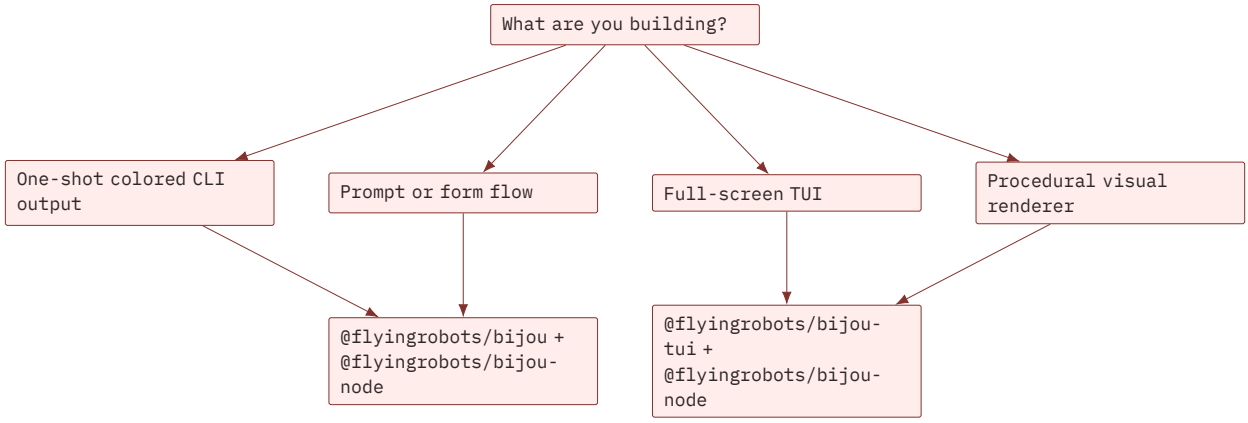
Package Stack

Bijou is intentionally layered. Core rendering and contracts are portable; interactive runtime behavior lives in the TUI package; host I/O lives behind Node adapters.





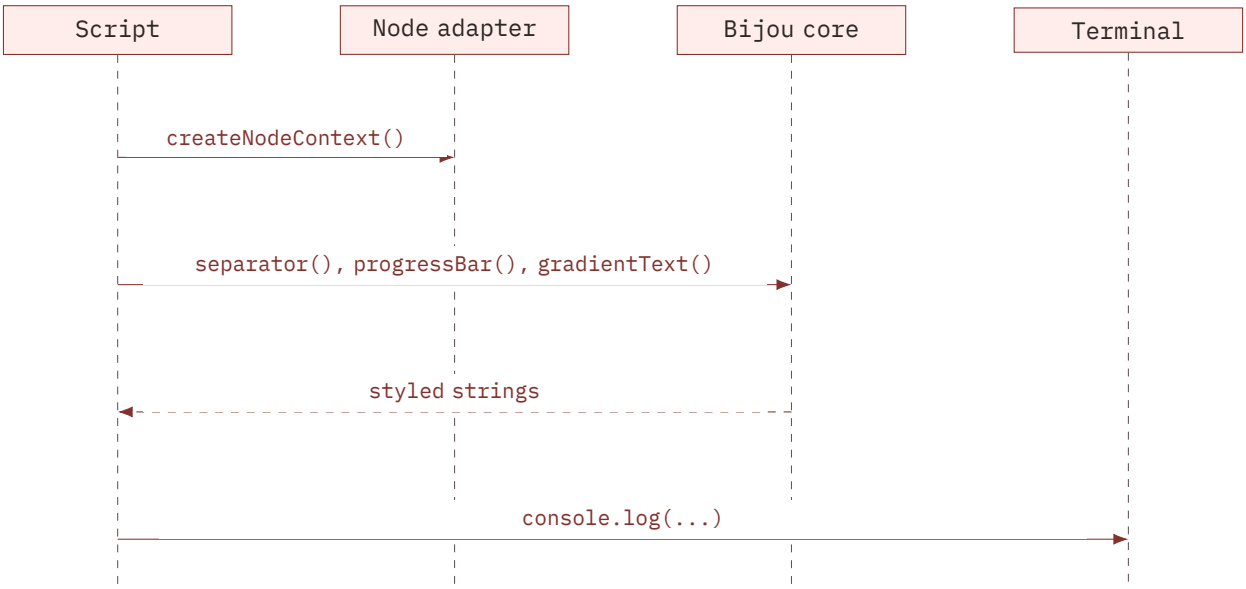
Choose A Scenario

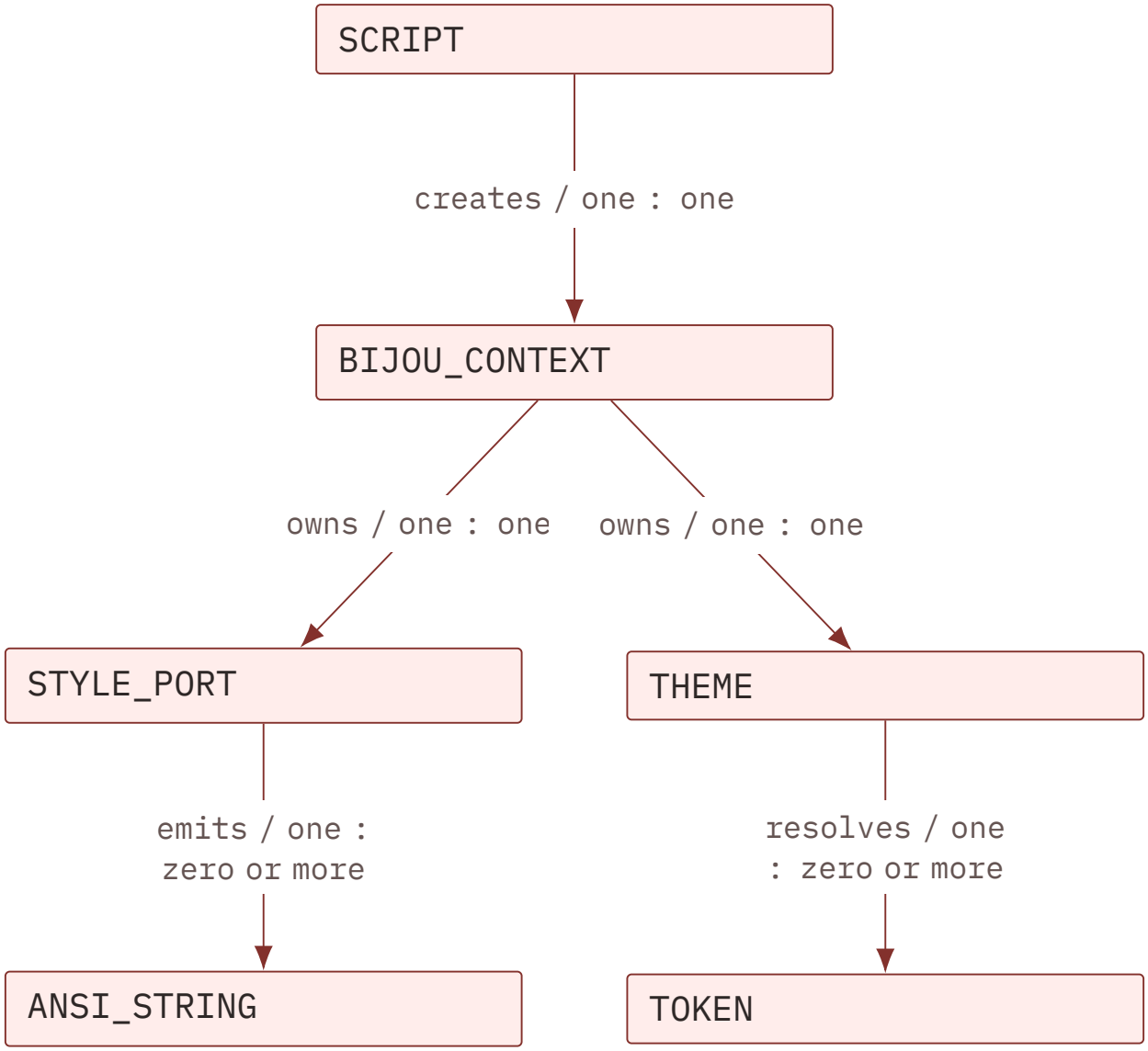


Scenario	Primary packages	Main output
Print colors	@flyingrobots/bijou, @flyingrobots/bijou-node	Strings with host styling
Interactive form	@flyingrobots/bijou, @flyingrobots/bijou-node	Prompt results
Basic TUI app	@flyingrobots/bijou-tui, @flyingrobots/bijou-node	Surface frames
Braille ray trace renderer	@flyingrobots/bijou-tui, @flyingrobots/bijou-node	Shader-rendered Surface

Scenario 1: Print Some Colors

Use core components and a Node context when the program writes a result and exits. This is the right lane for CLIs, scripts, release notes, status output, and other non-full-screen tools.





Code

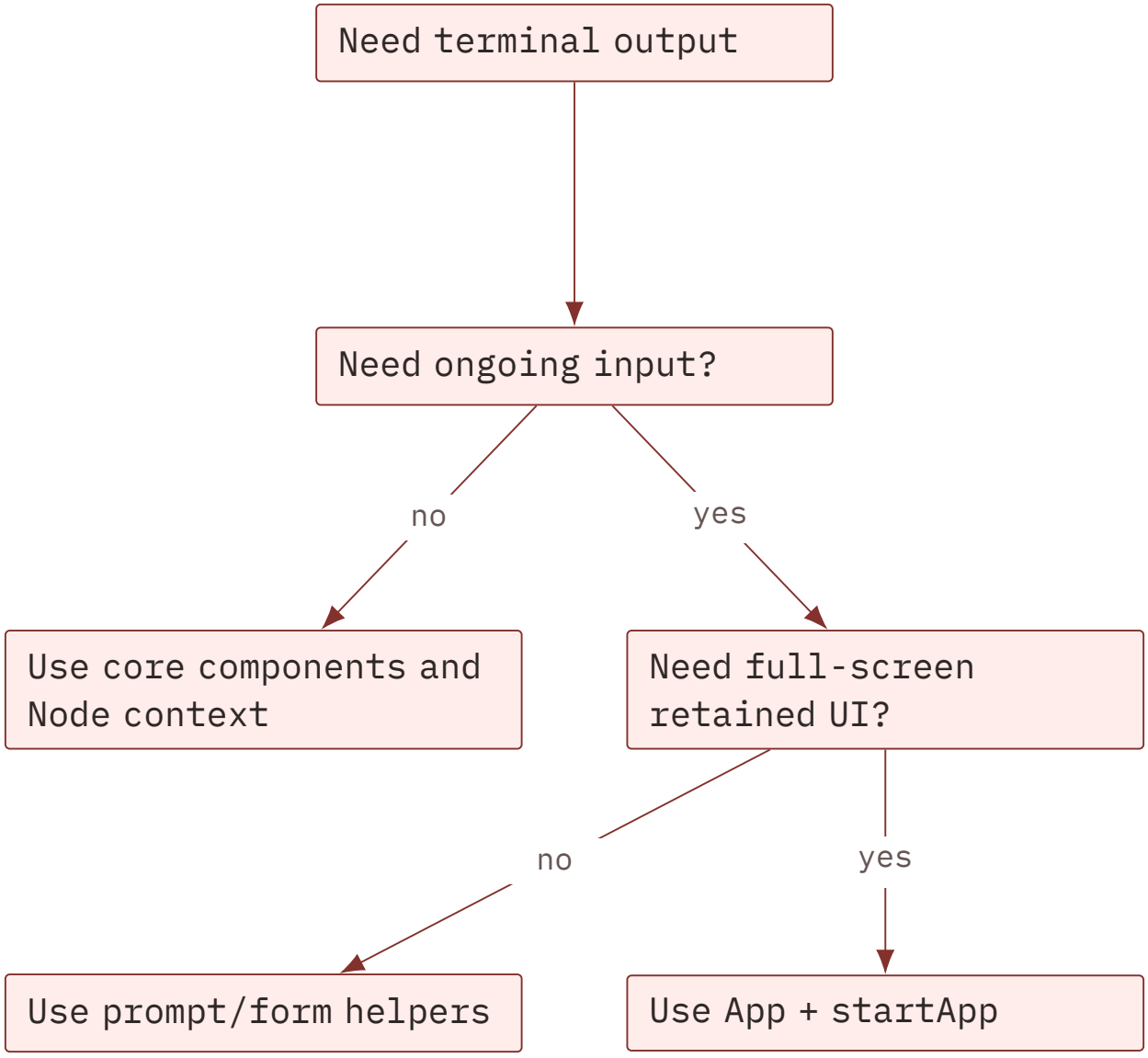
```
import {
  alert,
  box,
  gradientText,
  progressBar,
  separator,
} from '@flyingrobots/bijou';
import { createNodeContext } from '@flyingrobots/bijou-node';

const ctx = createNodeContext();
const brandGradient = ctx.theme.theme.gradient.brand;

console.log(separator({ label: 'deploy', width: 64, ctx }));
console.log(ctx.style.styled(ctx.status('success'), 'OK'));
console.log(gradientText('Bijou terminal output', brandGradient, {
  style: ctx.style,
  noColor: ctx.theme.noColor,
}));
console.log(progressBar(72, { width: 32, showPercent: true, ctx }));
console.log(alert('Production deploy is ready.', { variant: 'success', ctx }));
console.log(box('Use core components for one-shot terminal output.', { ctx }));
```

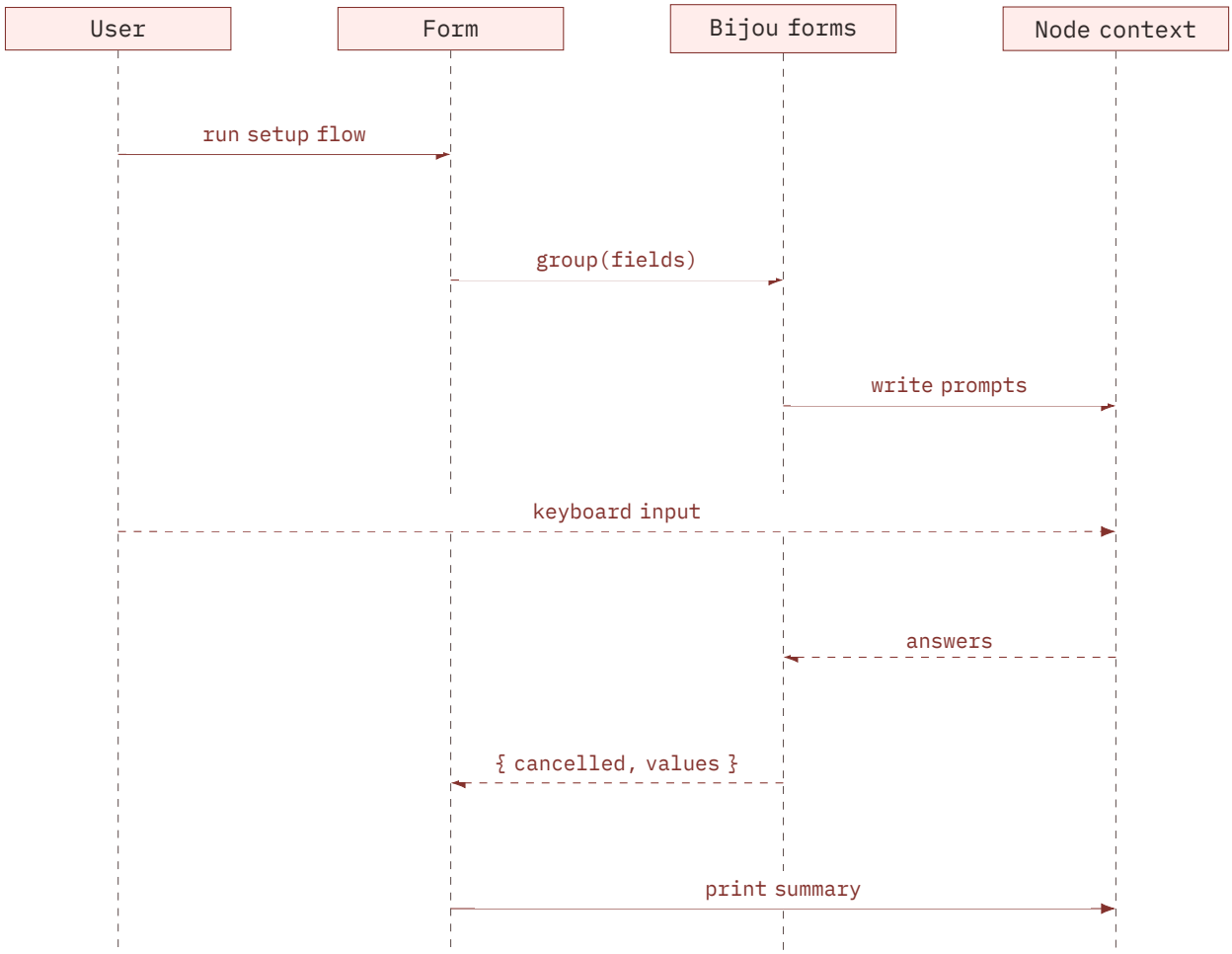
Rule Of Thumb

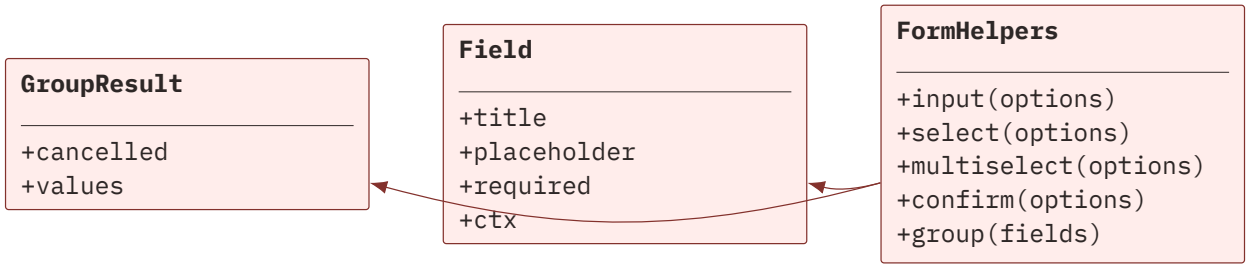
Use string-returning components for command output. Do not start the TUI runtime unless you need retained state, input routing, animation, or full-screen redraw.



Scenario 2: Make An Interactive Form

Use forms when the program should ask a bounded sequence of questions and then return to normal terminal output. Forms write through the context I/O port and return structured values.





Code

```
import { box, confirm, group, input, multiselect, select } from '@flyingrobots/bijou';
import { initDefaultContext } from '@flyingrobots/bijou-node';

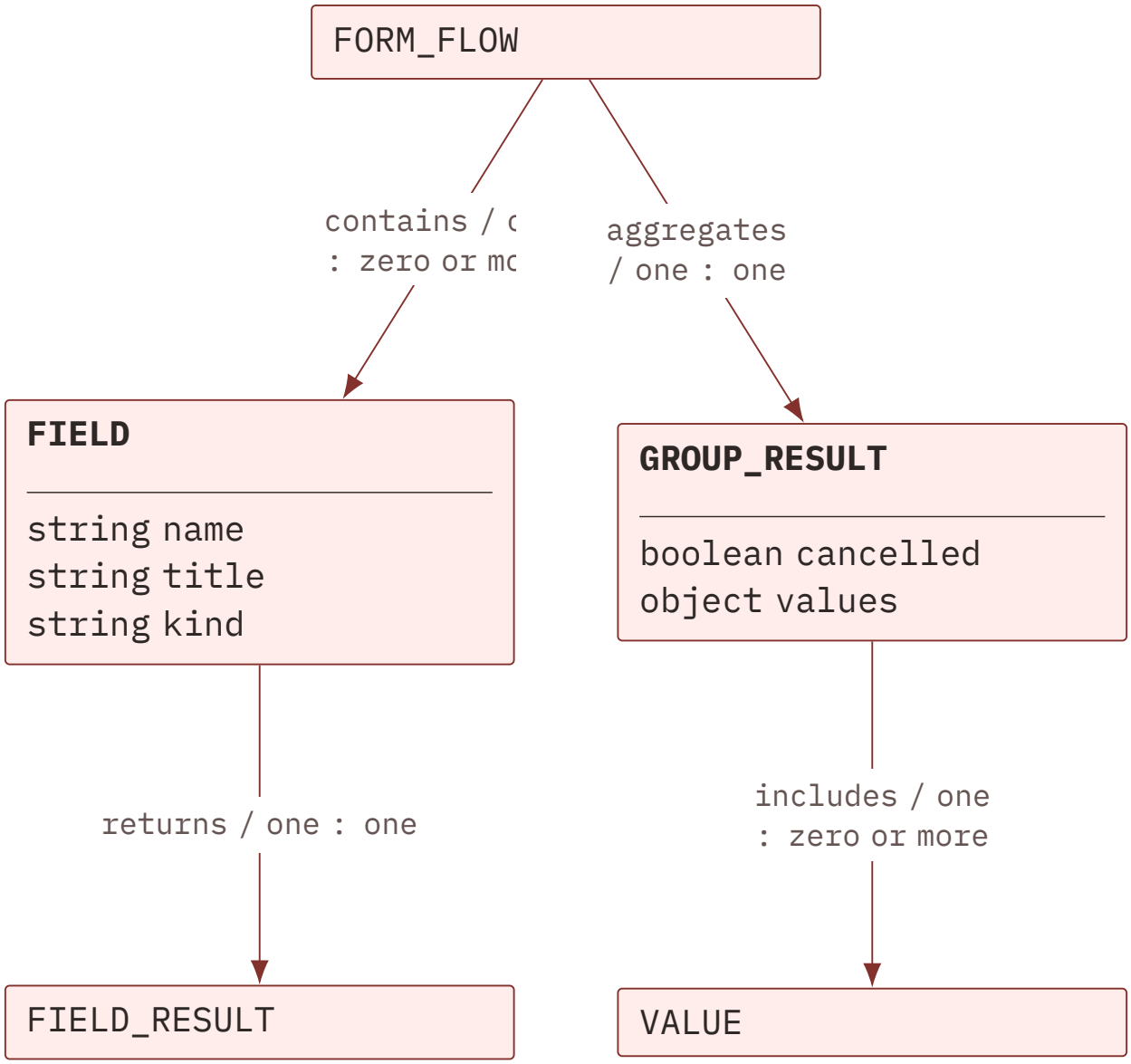
const ctx = initDefaultContext();

const result = await group({
  name: () => input({
    title: 'Project name:',
    placeholder: 'my-app',
    required: true,
    ctx,
  }),
  framework: () => select({
    title: 'Framework:',
    options: [
      { label: 'Express', value: 'express' },
      { label: 'Fastify', value: 'fastify' },
```

```
      { label: 'Hono', value: 'hono' },
      { label: 'None', value: 'none' },
    ],
    ctx,
  }),
  features: () => multiselect({
    title: 'Features:',
    options: [
      { label: 'TypeScript', value: 'typescript' },
      { label: 'ESLint', value: 'eslint' },
      { label: 'Docker', value: 'docker' },
    ],
    ctx,
  }),
  deploy: () => confirm({
    title: 'Set up deployment?',
    defaultValue: true,
    ctx,
  }),
});

if (result.cancelled) {
  console.log('Setup cancelled.');
} else {
  console.log(box([
    `Name: ${result.values.name}`,
    `Framework: ${result.values.framework}`,
    `Features: ${result.values.features.join(', ') || 'none'}`,
    `Deploy: ${result.values.deploy ? 'yes' : 'no'}`,
  ].join('\n'), { ctx }));
}
```

Data Shape

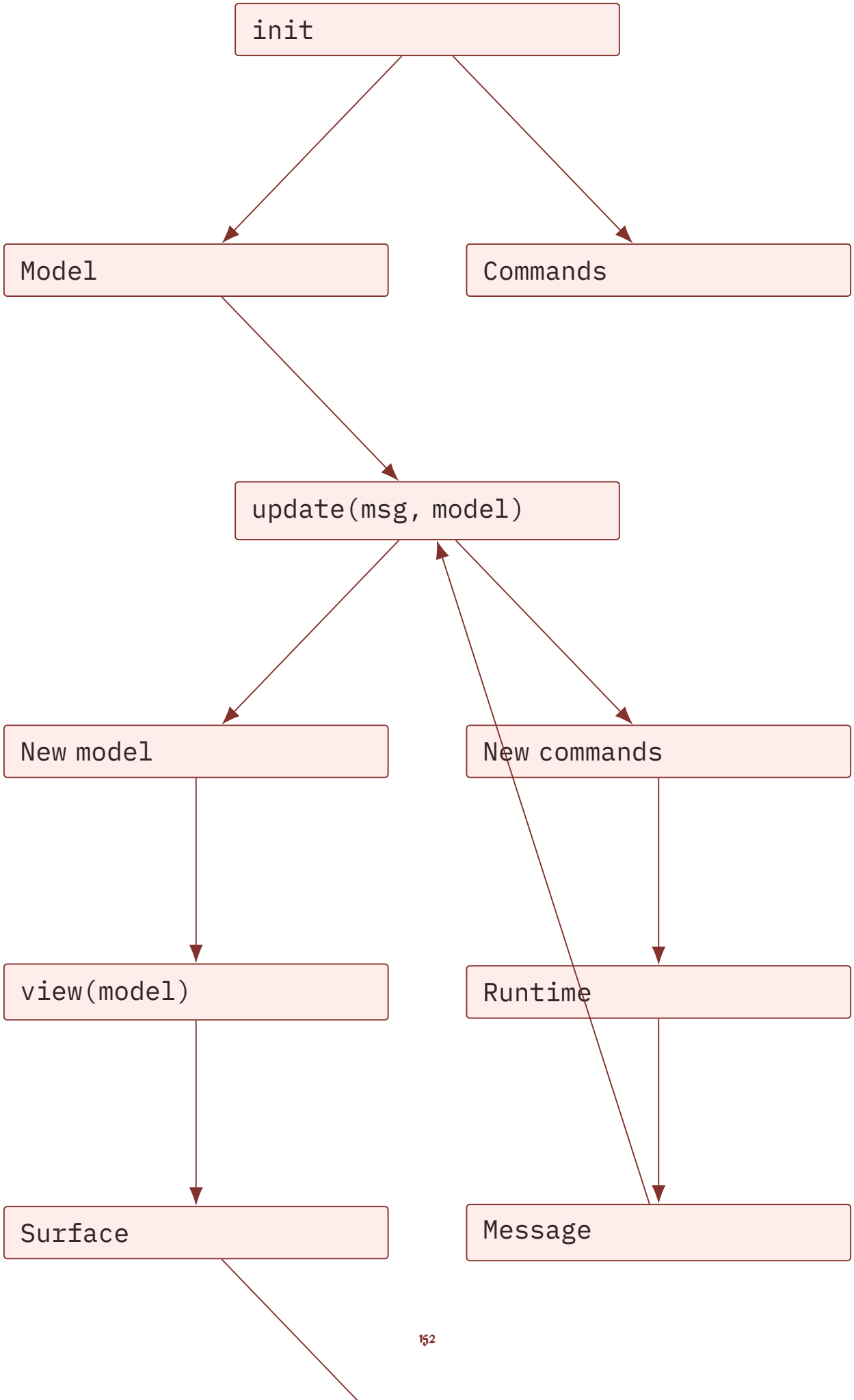


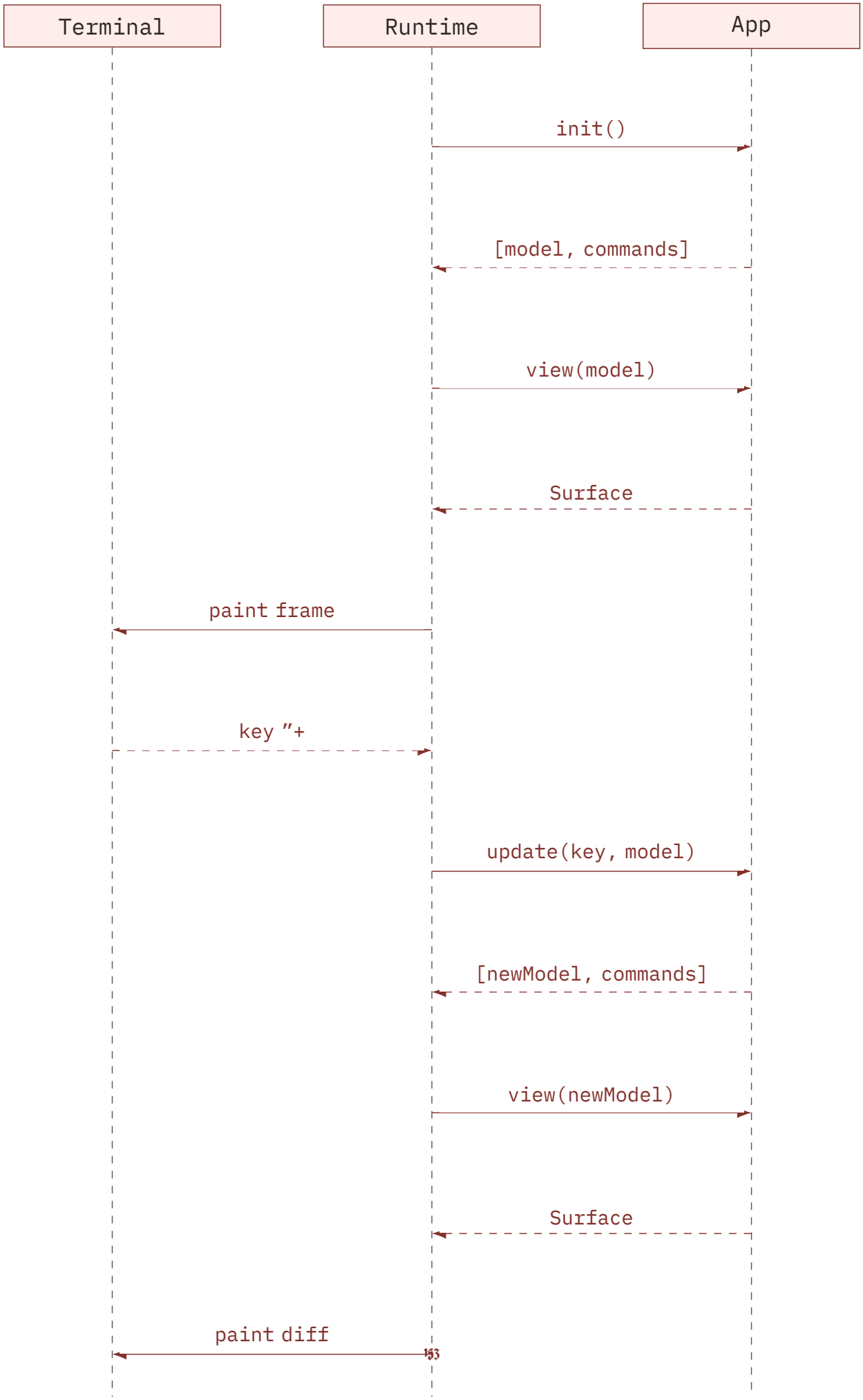
Scenario 3: Make A Basic TUI App

Use the TEA runtime when the app needs a retained model, input handling, commands, animation, or repeated frames.

The loop is:

1. init() creates the first model and commands.
2. Runtime delivers messages.
3. update() returns a new model and commands.
4. view() lowers the model to a Surface.
5. The host writes the minimal terminal diff.


init
Model
Commands
update(msg, model)
New model
New commands
view(model)
Runtime
Surface
Message


Terminal
Runtime
App
init()
[model, commands]
view(model)
Surface
paint frame
key "+
update(key, model)
[newModel, commands]
view(newModel)
Surface
paint diff

Code

```
import { box, kbd, stringToSurface } from '@flyingrobots/bijou';
import { startApp } from '@flyingrobots/bijou-node';
import { isKeyMsg, quit, type App } from '@flyingrobots/bijou-tui';

interface Model {
  readonly count: number;
}

type Msg = { type: 'increment' } | { type: 'decrement' } | { type: 'quit' };

const app: App<Model, Msg> = {
  init: () => [{ count: 0 }, []],

  update: (msg, model) => {
    if (isKeyMsg(msg)) {
      if (msg.key === '+' || msg.key === 'up') {
        return [{ count: model.count + 1 }, []];
      }
      if (msg.key === '-' || msg.key === 'down') {
        return [{ count: model.count - 1 }, []];
      }
      if (msg.key === 'q' || (msg.ctrl && msg.key === 'c')) {
        return [model, [quit()]];
      }
    }

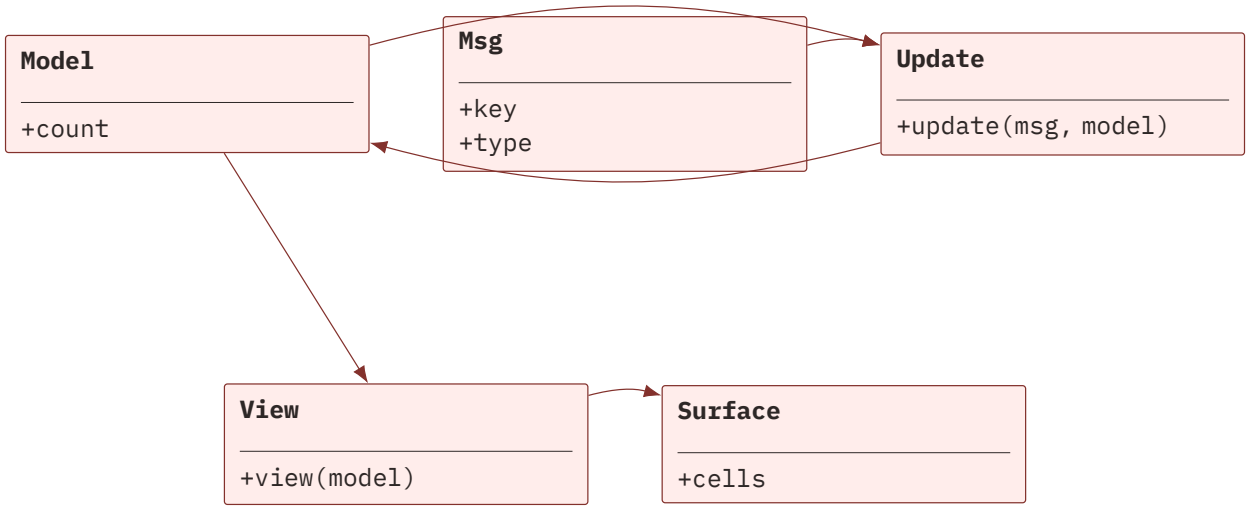
    return [model, []];
  },

  view: (model) => {
    const text = [
      box(`Count: ${model.count}`),
      '',
      `${kbd('+')} increment`,
      `${kbd('-')} decrement`,
      `${kbd('q')} quit`,
    ].join('\n');

    return stringToSurface(text, 32, 8);
  },
};

await startApp(app);
```

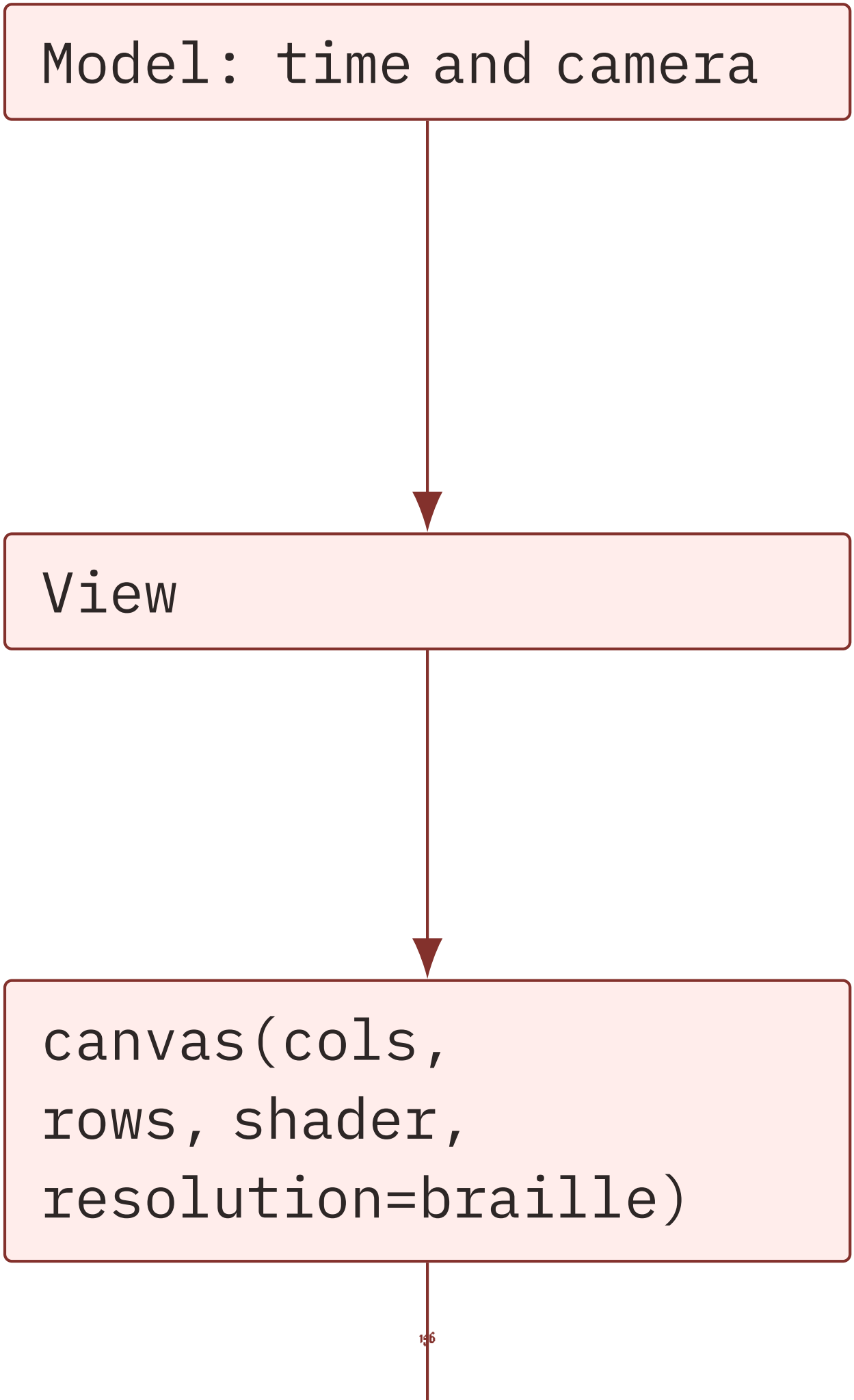
Ownership Boundaries

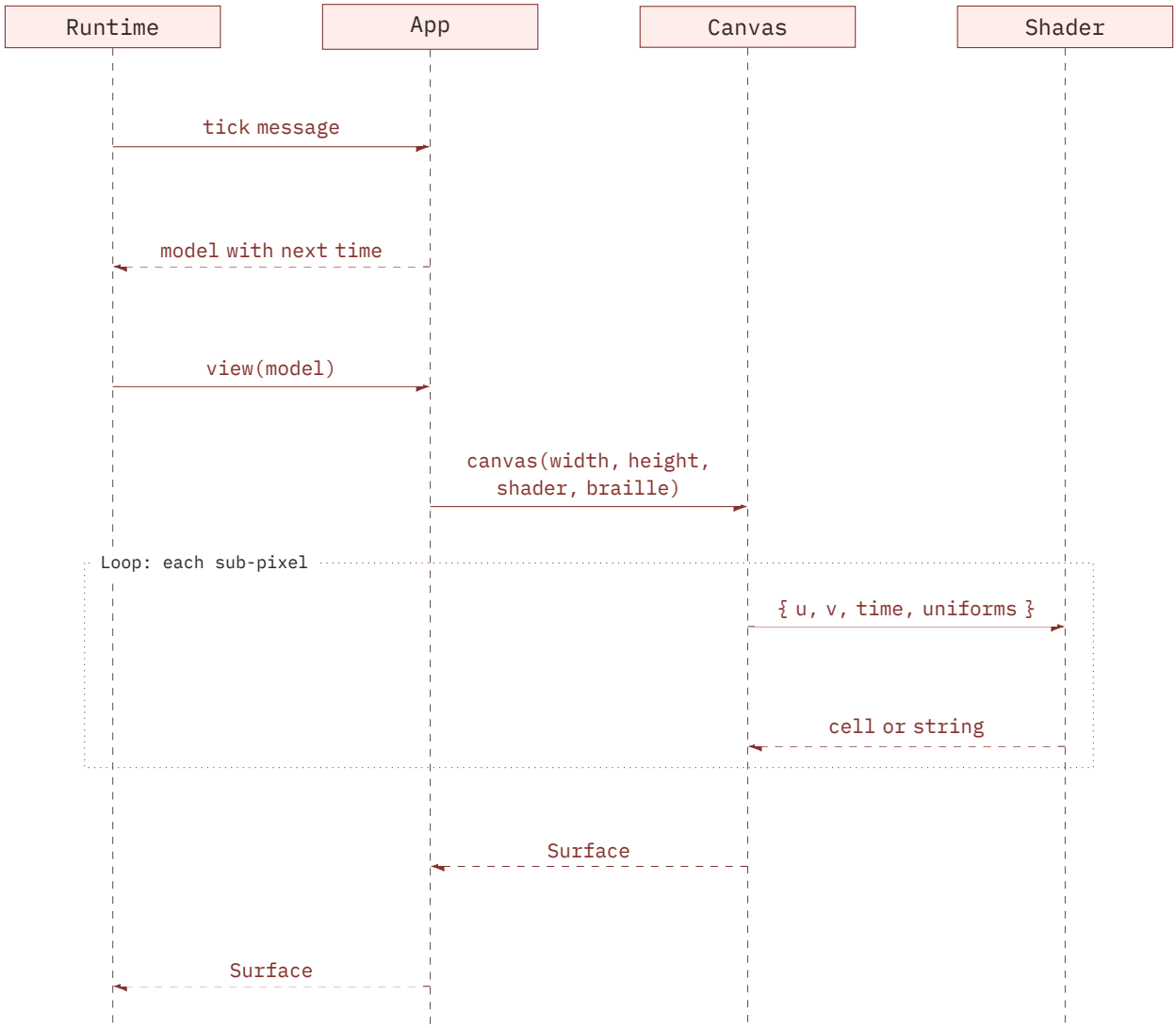


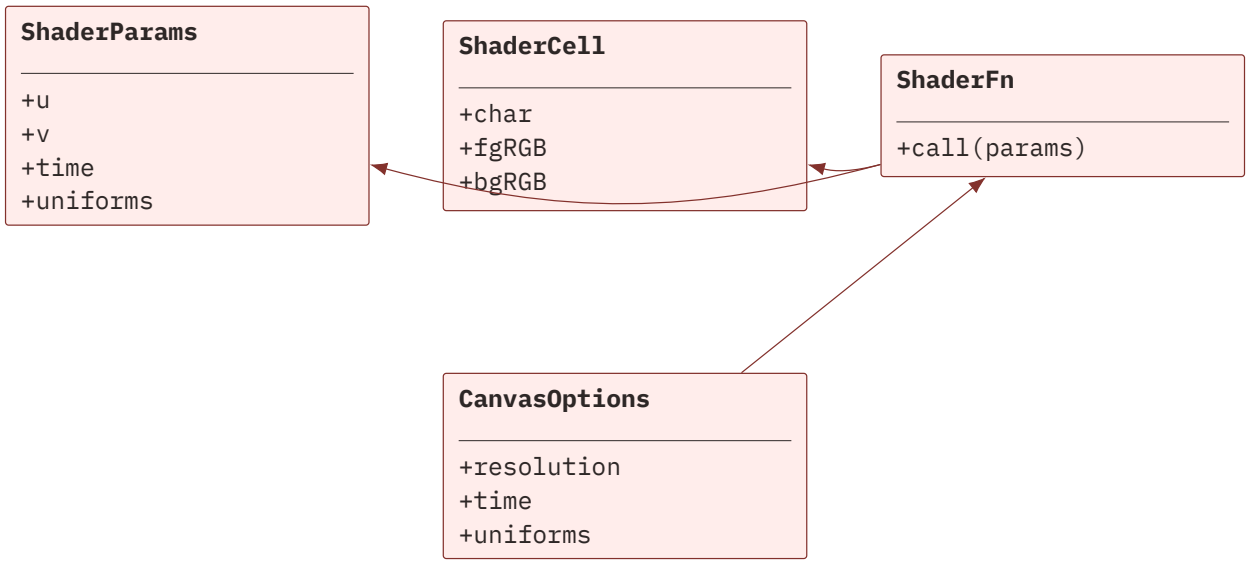
Do not mutate the model from inside view(). Views render. Updates own state change. Commands express runtime effects such as quit or delayed messages.

Scenario 4: Make A Braille Ray Trace Renderer

Use the TUI canvas() renderer when you want procedural character graphics. The Braille resolution samples each terminal cell as a 2 by 4 sub-pixel grid, so a small terminal surface can carry more visual detail than one character per cell.


Model: time and camera
View
canvas(cols,
rows, shader,
resolution=braille)


Runtime
App
Canvas
Shader
tick message
model with next time
view(model)
canvas(width, height, shader, braille)
Loop: each sub-pixel
{ u, v, time, uniforms }
cell or string
Surface
Surface


ShaderParams
+u
+v
+time
+uniforms
ShaderCell
+char
+fgRGB
+bgRGB
ShaderFn
+call(params)
CanvasOptions
+resolution
+time
+uniforms

Code

```
import { startApp } from '@flyingrobots/bijou-node';
import {
  canvas,
  isKeyMsg,
  quit,
  tick,
  type App,
  type ShaderFn,
} from '@flyingrobots/bijou-tui';

interface Model {
  readonly time: number;
}

type Msg = { type: 'tick' };

function sphereDepth(x: number, y: number, radius: number): number | null {
  const d2 = x * x + y * y;
  if (d2 > radius * radius) return null;
  return Math.sqrt(radius * radius - d2);
}

const shader: ShaderFn = ({ u, v, time }) => {
  const x = (u - 0.5) * 2.4;
  const y = (v - 0.5) * 1.8;
  const z = sphereDepth(x, y, 0.82);

  if (z === null) {
    return { char: ' ', bgRGB: [8, 10, 22] };
  }

  const lightX = Math.cos(time * 0.8) * 0.55;
  const lightY = -0.35;
  const lightZ = 0.9;
  const lightLength = Math.hypot(lightX, lightY, lightZ);
  const shade = Math.max(0, (x * lightX + y * lightY + z * lightZ) / lightLength);
  const rim = Math.max(0, 1 - z);

  return {
    char: shade + rim * 0.2 > 0.28 ? 'x' : ' ',
    fgRGB: [
      Math.round(80 + shade * 150),
      Math.round(110 + shade * 120),
      Math.round(180 + shade * 60),
    ],
    bgRGB: [8, 10, 22],
  };
};

const app: App<Model, Msg> = {
  init: () => [{ time: 0 }, [tick(33, { type: 'tick' })]],

  update: (msg, model) => {
    if (isKeyMsg(msg) && (msg.key === 'q' || (msg.ctrl && msg.key === 'c'))) {
      return [model, [quit()]];
    }

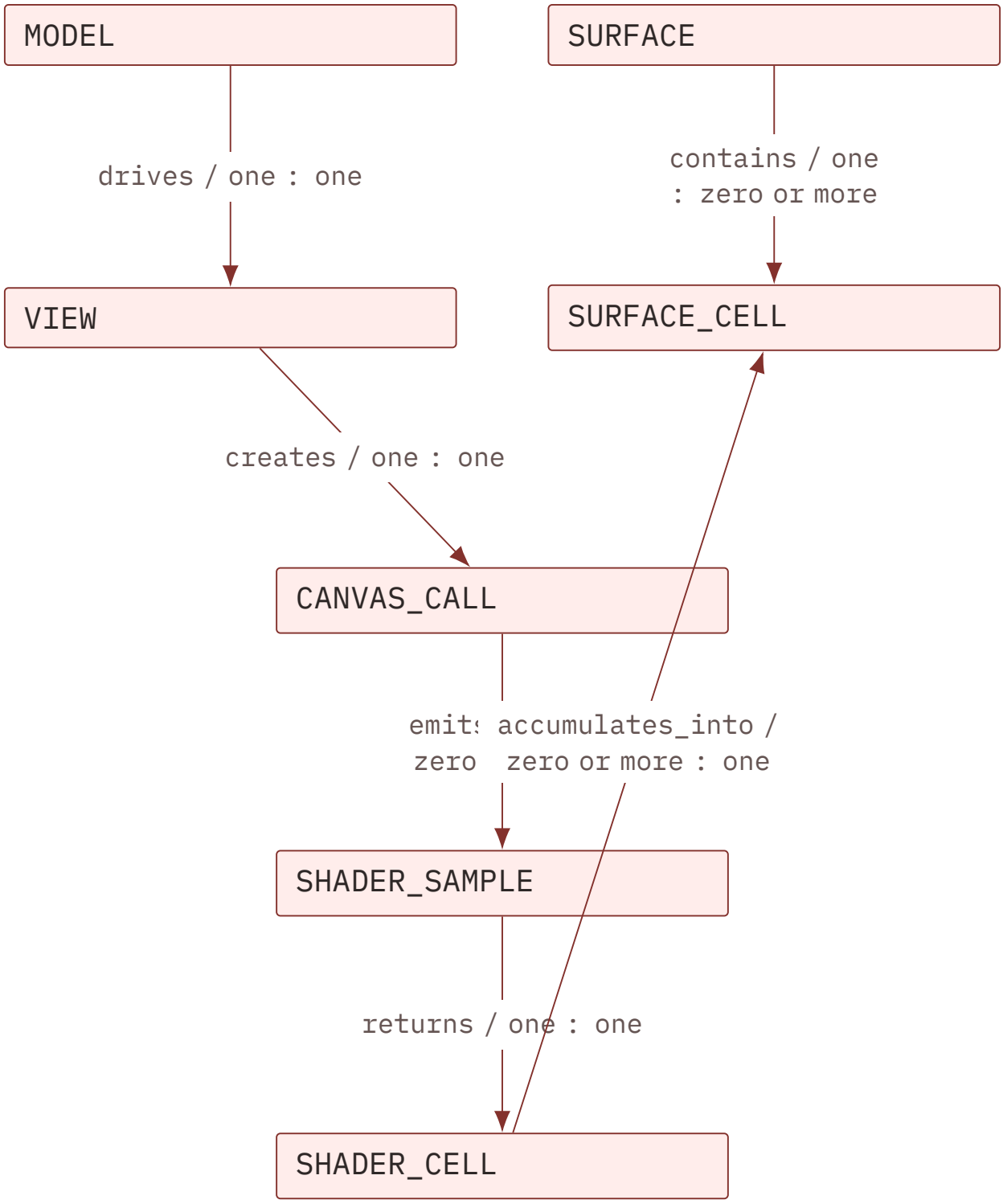
    if ('type' in msg && msg.type === 'tick') {
      return [{ time: model.time + 0.033 }, [tick(33, { type: 'tick' })]];
    }

    return [model, []];
  },

  view: (model) =>
    canvas(72, 28, shader, {
      resolution: 'braille',
      time: model.time,
    }),
};

await startApp(app);
```

Renderer Data Flow



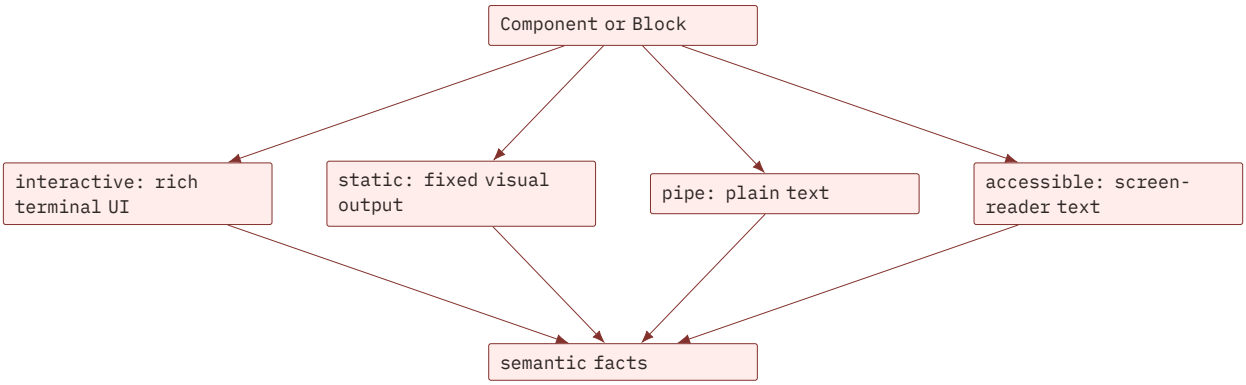
Performance Notes

- Keep shader math deterministic and bounded.
- Keep allocations out of the inner shader path.
- Use uniforms for static scene values instead of closing over mutable runtime objects.

- Prefer resolution: 'braille' for high-density binary coverage and resolution: 'glyph' when you want glyph fitting from coverage.
- Use npm run bench and npm run soak when changing renderer internals.

Mode Lowering

Bijou components should degrade intentionally when output is not an interactive terminal.

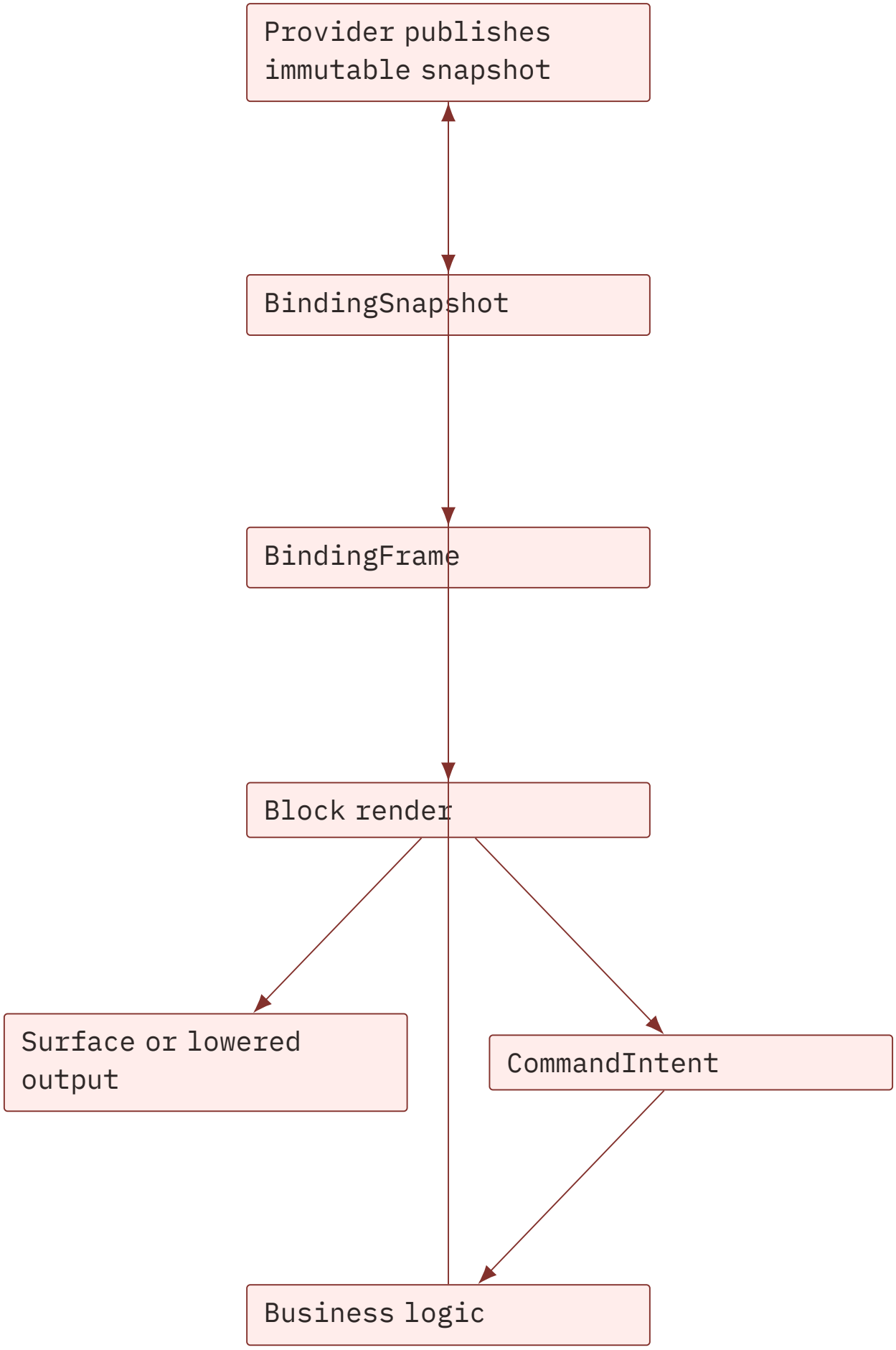


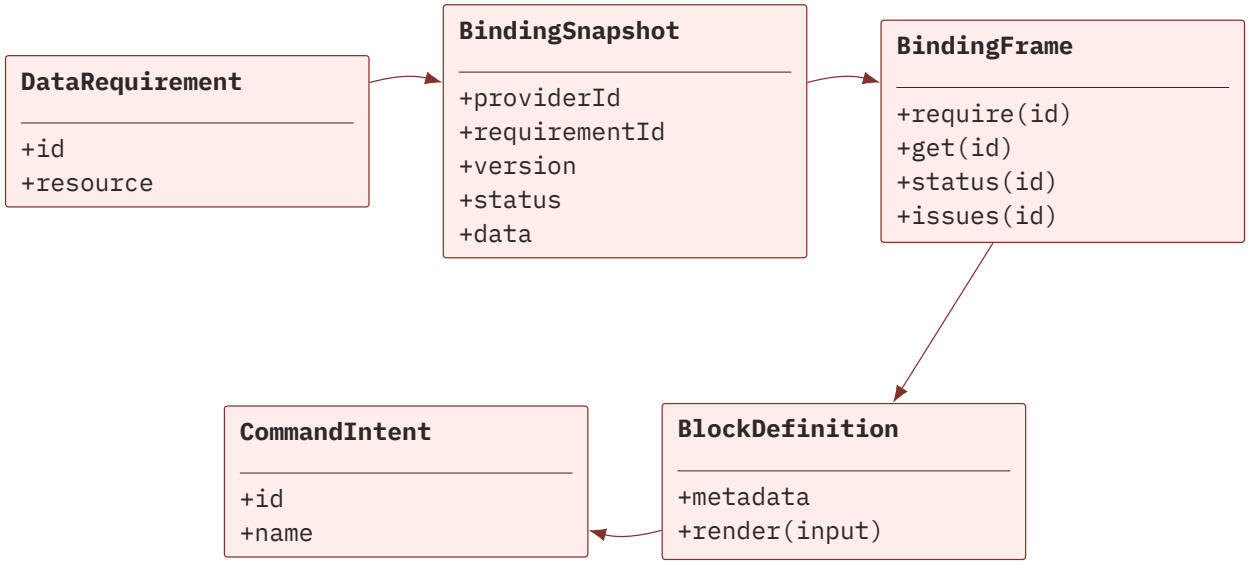
Use this rule:

- Interactive and static modes may draw boxes, colors, charts, or Braille surfaces.
- Pipe mode should emit useful plain text.
- Accessible mode should prioritize meaningful spoken order.
- Semantic facts should survive lowering when tooling or agents need to inspect the output.

Blocks And Data Binding

For reusable application surfaces, use Blocks. Blocks declare metadata, variants, slots, data requirements, command intents, and mode-lowering posture. They do not pull mutable state directly.

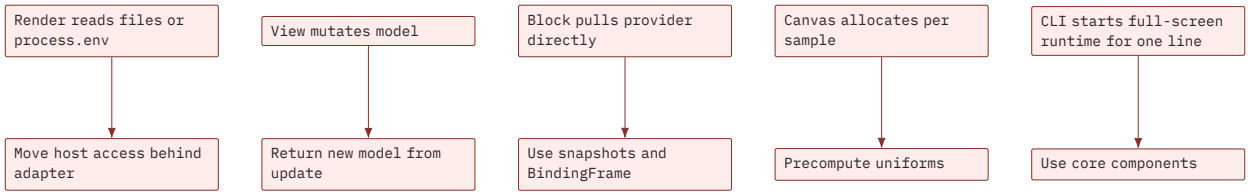

Provider publishes immutable snapshot
BindingSnapshot
BindingFrame
Block render
Surface or lowered output
CommandIntent
Business logic



That loop is unidirectional:

```
business logic/providers
  -> immutable snapshots
  -> render frame
  -> views/blocks render
  -> command intents leave as user intent
  -> business logic owns change
```

Common Mistakes



Mistake	Better path
Parsing stdin inside a component	Let host/runtime produce messages or form answers.
Calling provider refresh from render	Emit a command intent; business logic owns change.
Duplicating English fallback strings in translated catalogs	Load English fallback separately.
Using a full-screen TUI for simple output	Use core components and a Node context.
Treating terminal graphics as raw strings everywhere	Promote them to Surface when layout and diffing matter.

Validation Checklist

Before shipping a terminal scenario, run the narrow command for the surface you changed, then the shared gates:

```
npm run lint
npm run typecheck:test
npm test
npm run docs:inventory
npm run verify:interactive-examples
```

For user-facing product proof:

```
npm run dogfood
npm run blocklab
```

For rendering or long-running runtime changes:

```
npm run bench
npm run soak
```

40

DOGFOOD

docs/DOGFOOD.md

Documentation Of Good Foundational Onboarding and Discovery

DOGFOOD is the canonical human-facing docs surface and proving ground for the Bijou engine.

Run

```
npm run dogfood
```

DOGFOOD initializes its language through the host locale adapter and then lets users change the preferred language from the Settings drawer (F2). The docs app consumes this through a locale port rather than reading process state from view code, so localization remains behind the same hexagonal boundary as the rest of the host integration.

To inspect and enforce the current source-level localization debt:

```
npm run dogfood:i18n:debt
```

That report counts remaining localizable DOGFOOD UI strings by source surface and missing DOGFOOD Markdown localizations by locale. It is a source inventory, not a rendered-output scraper, so ids, paths, and catalog-backed fallback calls stay separate from visible raw copy.

DOGFOOD Markdown files can declare localization metadata in YAML frontmatter:

```
---
dogfood:
  localization:
    sourceLocale: en
    locales: [fr, es, de]
    localized:
      fr: ./guide.fr.md
      es:
        path: ./es/guide.md
---
```

If a DOGFOOD Markdown file omits that frontmatter, the ratchet assumes the source locale is English and expects every supported non-English locale. It looks for conventional sibling files such as guide.fr.md and locale-directory files such as fr/guide.md. localized.<locale> paths are resolved relative to

the source Markdown file before those conventional fallbacks are checked. The DOGFOOD Markdown loader strips frontmatter before rendering prose, so metadata can live beside the source document without leaking into the reader.

DOGFOOD's catalog source is a committed CSV string table:

```
examples/docs/i18n/source/dogfood-strings.csv
```

Build the selected-locale runtime JSON catalog files from that table with:

```
npm run dogfood:i18n:build
npm run dogfood:i18n:check
npm run dogfood:i18n:complete
```

The generated runtime files live under:

```
examples/docs/i18n/catalogs/<locale>/<namespace>.json
```

DOGFOOD loads the generated English catalog as an explicit production fallback, then loads only the selected locale directory at runtime. The generated catalog for fr, for example, carries French values only. English source values remain in the source table and the generated English catalog, not every translated payload.

In non-production builds, missing selected-locale strings render as a bright missing-localization marker instead of quietly falling back to English. That keeps untranslated UI visible while development is running.

New DOGFOOD localization keys, changed source strings, changed translated values, and removed keys must be handled semantically across every supported locale: en, fr, es, and de. npm run dogfood:i18n:complete enforces that policy for localization keys changed relative to origin/main by default, ratchets the full missing-translation count, and accepts -- --base <ref> for CI comparison refs or -- --all for a full-table audit. The pre-push hook and CI both run the completeness gate plus npm run dogfood:i18n:check, so a new string cannot merge with missing translations or stale generated catalogs.

The raw-string debt gate is stricter than a global ratchet. If a branch touches a DOGFOOD TypeScript source file that still has raw visible-copy debt, npm run dogfood:i18n:debt fails until that file is cleaned or the strings are proved non-localizable by scanner policy. This prevents old localization debt from surviving in files that are actively being edited.

Numbered extraction modules such as release-title.part01.ts retain the canonical release-title.ts source identity for touched-file comparisons. The gate sums debt across the numbered parts and requires that total to fall below the merge-base root count; splitting a file cannot reset or merely redistribute its debt.

The same source table can also be exported through the i18n workbook adapters instead of requiring translators to edit the docs app source directly:

```
npm run dogfood:i18n:export -- --locale fr --format csv
npm run dogfood:i18n:export -- --locale fr --format tsv --out /tmp/dogfood-fr
npm run dogfood:i18n:export -- --format json --bundle /tmp/dogfood-catalog.json
npm run dogfood:i18n:coverage
```

The CSV/TSV and JSON conversions are produced by @flyingrobots/bijou-i18n-tools; DOGFOOD still loads generated runtime catalog JSON rather than parsing spreadsheet files during rendering.

For the standalone BlockLab development and testing workbench:

```
npm run blocklab
```

For the deterministic text-first story index and capture matrix:

```
npm run blocklab:index
```

Purpose

DOGFOOD is where Bijou proves its architectural integrity by building itself. It is not an example; it is the repository's living docs application.

It gathers the package guides for the published workspace, release guidance, and the doctrine, architecture, invariants, and design-system guidance that operators need when learning or validating Bijou.

DOGFOOD also publishes its own docs experience as DogfoodDocsSurfaceBlock, the canonical app-level Block for navigation, reader content, search state, and proof artifacts. That contract gives humans and agents one inspectable surface identity for the docs app instead of reconstructing product truth from adjacent rendering code.

Today's Proof Points

- **Shell Integrity**: Testing the framed shell, tabs, and pane focus in a production-grade surface.
- **Canonical Docs Surface Block**: DogfoodDocsSurfaceBlock owns the DOGFOOD docs surface contract across navigation, reader, search, proof artifacts, command intents, and lower-mode facts.
- **Documentation Jump Search**: / opens a global documentation search that can jump across sections, activate the right page, and select the matching guide or component story.
- **Component Explorer**: A live, interactive field guide to every component family.
- **Blocks Section**: A first-class Blocks lane covering block purpose, authoring, first-party blocks, live surface-backed previews, and lowering posture.
- **Locale Preference**: The Settings drawer can switch DOGFOOD's preferred language after startup, while startup itself uses the host locale adapter.
- **Localization Debt Ratchet**: npm run dogfood:i18n:debt counts remaining localizable source strings by DOGFOOD surface across examples/docs/**/*.ts modules, keeps tooling-only exclusions documented, counts missing Markdown localizations by locale, fails when either baseline increases, and blocks touched DOGFOOD TypeScript files that still contain raw visible-copy debt.
- **Localization Completeness Gate**: npm run dogfood:i18n:complete requires every changed DOGFOOD localization key to carry current values for all

supported locales, and ratchets the full missing-translation count before pre-push or CI can pass.
- **BlockLab Workstation**: A standalone interactive story browser plus deterministic story index and matrix capture path over the same DOGFOOD story catalog.
- **Graceful Lowering**: Verifying that documentation renders correctly across rich, static, pipe, and accessible modes.
- **Responsive Product Layout**: Proving that resize is not enough by selecting wide, standard, narrow, and tiny docs layouts that keep constrained terminals useful.
- **Motion and Shader Surfaces**: Exercising canvas, glyph-fit raytracing, transitions, springs, and timelines from the docs app itself.
- **Design Language**: Defining and enforcing the project's visual and interactive standards.

Relationship to Examples

The examples/ tree contains isolated reference material and API proofs. **DOGFOOD** is the primary entry point for understanding the system's unified behavior.

DOGFOOD implementation lives in examples/docs/.

41

Package Development Guide

PACKAGE_DEVELOPMENT.md

This guide describes the minimum steps to add a new package to the Bijou monorepo without breaking workspace conventions.

Why the packages are split

Bijou keeps package boundaries narrow so apps can adopt only the layer they need:

- @flyingrobots/bijou is pure TypeScript and owns surfaces, components, blocks, themes, and port contracts.
- @flyingrobots/bijou-node adapts those ports to Node.js, terminal IO, filesystem access, and host environment detection.
- @flyingrobots/bijou-tui owns the interactive TEA runtime, key/input routing, layout, animation, and render pipeline.
- @flyingrobots/bijou-tui-app owns the batteries-included framed shell.
- create-bijou-tui-app gives app authors a project scaffold instead of asking them to assemble the shell manually.
- i18n packages are split between runtime lookup and authoring/workflow tools so apps do not import CSV, TSV, XLSX, or filesystem tooling at runtime.

That split adds workspace setup cost, but it keeps pure rendering independent from host IO and lets small CLIs avoid full TUI shell dependencies.

1) Create package scaffold

Create a package directory under packages/<name>/ with:

- package.json
- tsconfig.json with composite references
- source folder (usually src/)
- optional test folder and build artifacts

A minimal package.json follows this shape:

```
{
  "name": "@flyingrobots/bijou-<package>",
```

```
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "exports": {
    ".": "./dist/index.js"
  },
  "scripts": {
    "build": "tsc -p tsconfig.json",
    "test": "vitest --run --config ../../vitest.config.ts"
  },
  "dependencies": {
    "@flyingrobots/bijou": "workspace:*"
  }
}
```

2) Register in the workspace

The root workspace uses globs (packages/*), so no explicit package list change is required for package discovery if you keep the package under packag es/.

3) Wire TypeScript project references

For workspace tsc -b execution to compile your package correctly:

1. Ensure package tsconfig.json has composite: true and a valid references array.
2. Add a package-level tsconfig.tsbuildinfo entry under tsconfig conventions as needed.
3. Keep rootDir/outDir under src and dist.

4) Connect ports and context where relevant

Packages that contribute runtime behavior should pass through the port and context abstractions:

- **Runtime side**: export RuntimePort implementations (@flyingrobots/bijou).
- **IO side**: keep terminal/file/timer interactions on IOPort.
- **Style side**: keep color and formatting behind StylePort.

Never import process or fs in core packages. If you need host access, move it into an adapter package and inject through ports.

5) Export surfaces and contracts in a public API file

Use a package-level src/index.ts to aggregate public exports and keep import paths stable for consumers.

```
export * from './runtime.js';
export * from './types.js';
```

6) Add tests and docs

Every added public function should include at least:

- Unit test coverage.
- A short fixture or usage example.
- Package README section describing setup and API shape.

7) Add to user-facing docs

For discoverability:

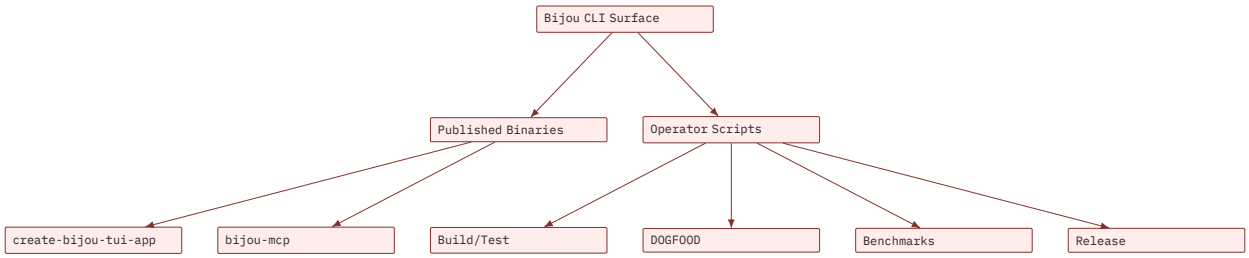
- Add/update the root package map entry if the package is user-facing.
- Add the package README excerpt to README.md Package Map when needed.
- Add any new API docs under docs/ and cross-link from the appropriate architecture or GUIDE file.

42

CLI

docs/CLI.md

The Bijou command surface is a composite of published binaries and repo-local operator scripts.



Published Binaries

create-bijou-tui-app

Scaffold a new high-fidelity TUI workspace.

```
npm create bijou-tui-app@latest my-app
```

bijou-mcp

Run the MCP server exposing Bijou rendering tools over stdio.

```
node node_modules/@flyingrobots/bijou-mcp/bin/bijou-mcp.js
```

Operator Scripts

Execute these from the monorepo root.

Build & Test

- npm run build: Compile the workspace.
- npm run test: Execute the test suite.
- npm run lint: Run linting and type-checks.

DOGFOOD & Proving

- npm run dogfood: Launch the documentation TUI.
- npm run smoke:dogfood: Run the DOGFOOD smoke tests.
- npm run smoke:canaries: Execute scaffolding canary tests.

Performance & Benchmarks

- npm run perf: Launch the interactive visual performance demo.
- npm run bench: Execute the benchmark suite.
- npm run bench:ci:gradient: Run the fixed gradient stress lane used in CI.
- npm run bench:compare: Compare results against a baseline.
- npm run soak: Run long-duration soak tests.

npm run bench -- run now supports: - --tag=... for scenario provenance filtering - --format=json for nested bench.v2 output - --format=jsonl / --format=flat for agent-first flat metric records

---

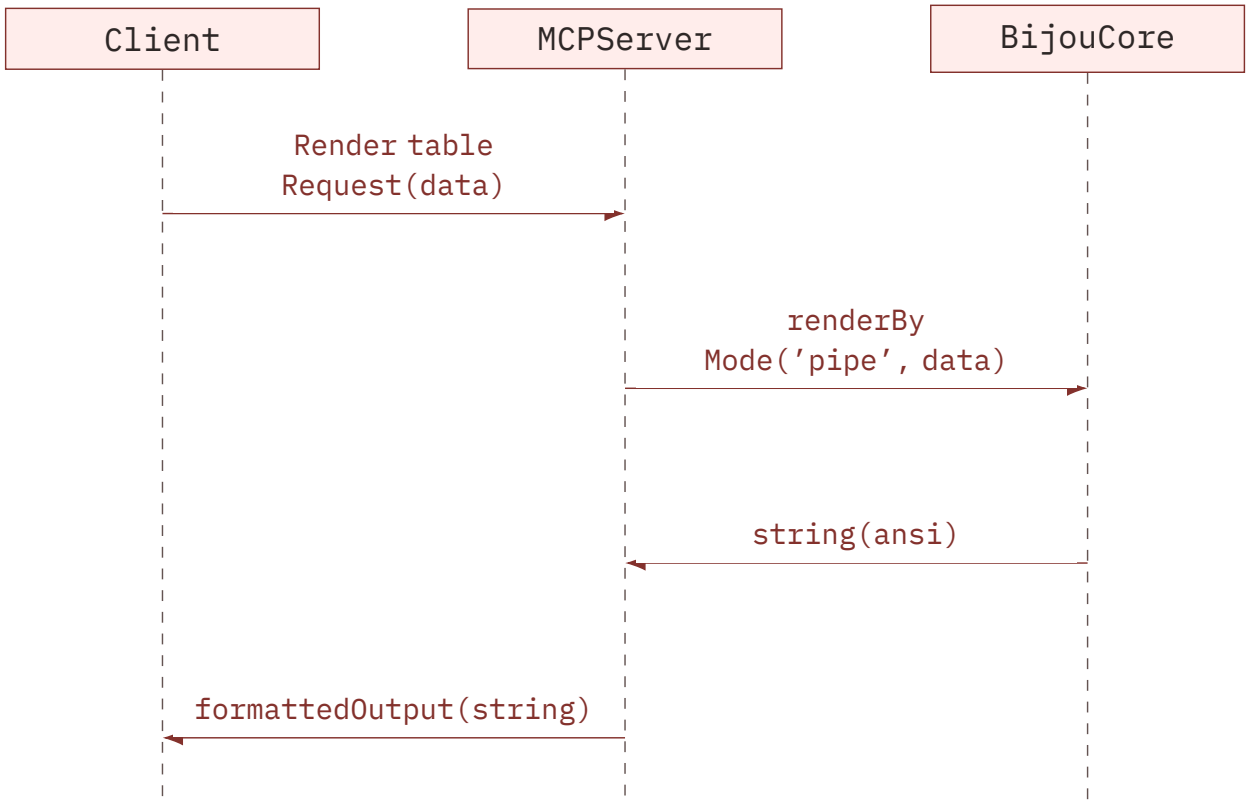
The release truth is maintained in docs/release.md.

43

MCP

docs/MCP.md

This signpost covers Bijou's current MCP surface.



What Ships Today

Bijou currently ships one MCP package:

- `@flyingrobots/bijou-mcp`

It exposes Bijou terminal components as rendering tools over stdio so an MCP client can ask for structured terminal output without needing a live TTY.

What It Is Good For

Use the MCP server when you want:

- box-drawn or structured terminal output inside a chat or agent context
- Unicode-rich output without ANSI escape codes
- component rendering as a tool surface rather than as a fullscreen runtime

What It Is Not

The MCP package is not the fullscreen app runtime.

It does not own:

- createFramedApp()
- shell behavior, pane focus, help, settings, notifications, or quit flow
- animation, motion, diffed terminal rendering, or interactive input routing

For those, use the normal package/runtime docs instead:

- packages/bijou-tui/GUIDE.md
- packages/bijou-tui/ADVANCED_GUIDE.md

Configure It

Typical MCP config:

```
{
  "mcpServers": {
    "bijou": {
      "command": "node",
      "args": ["node_modules/@flyingrobots/bijou-mcp/bin/bijou-mcp.js"]
    }
  }
}
```

The package README has the fuller install/config reference:

- packages/bijou-mcp/README.md

Tool Families

The current tool surface is grouped around:

- data and structure: tables, trees, DAGs, enumerated lists
- containers and layout: boxes, header boxes, separators, constrain
- feedback and status: alerts, progress, stepper, timeline, logs, badges
- navigation: tabs, breadcrumb, paginator
- rich panels: explainability, inspector, accordion
- utility: keyboard indicators, hyperlinks, skeletons

For the exact tool list and examples, use the package README.

Read Next

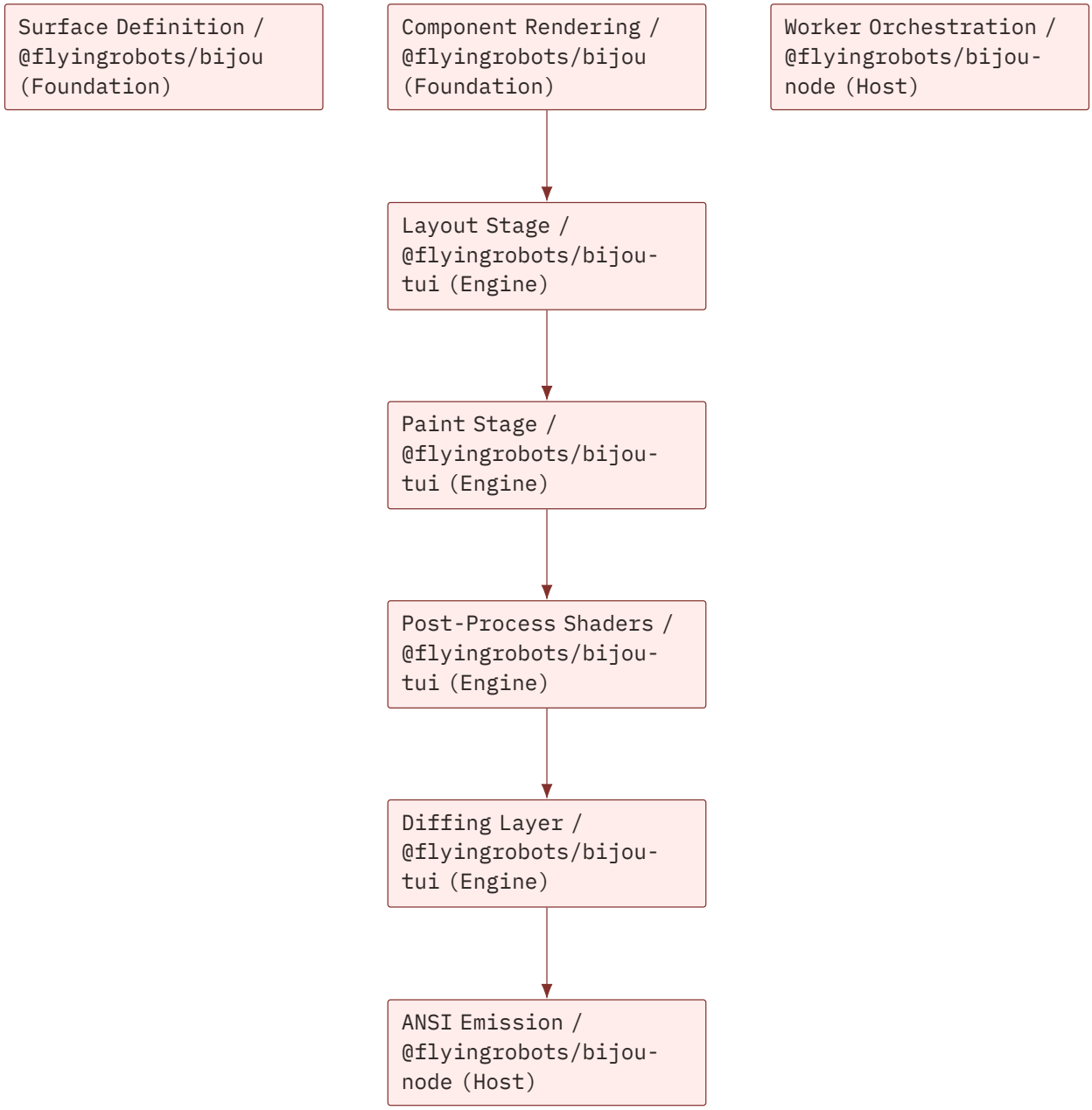
- exact tool catalog and example output: packages/bijou-mcp/README.md
- repo command surfaces: docs/CLI.md
- architecture and boundaries: ARCHITECTURE.md

44

Advanced Guide — Bijou

ADVANCED_GUIDE.md

This is the second-track manual for Bijou.



Use the root README and the package GUIDE.md files when you want the productive-fast path. Use this guide when you need the deeper doctrine, the proving surfaces, or the repo workflows that sit behind the public quick starts.

What Belongs Here

This guide is for topics like:

- bytes-only internal rendering rules and explicit string boundaries
- shell doctrine and createFramedApp() ownership
- programmable pipeline, shaders, paint, transforms, and middleware seams
- DOGFOOD as the canonical proving surface
- performance and soak workflows
- localization workflows and bidirectionality posture
- worker/runtime boundaries and scripted capture
- smoke, PTY, canary, and release-readiness expectations

Reading Order

Use this sequence when you need repo-truth beyond the quick path:

1. Read the Root README for the package map and public posture.
2. Read the package guide you are actively using.
3. Read the matching package ADVANCED_GUIDE.md.
4. Use DOGFOOD, the performance docs, and the strategy/spec docs as the proving surfaces for the subsystem you are touching.

Package Lanes

- Core advanced work: packages/bijou/ADVANCED_GUIDE.md
- Runtime advanced work: packages/bijou-tui/ADVANCED_GUIDE.md
- Node/runtime-boundary work: packages/bijou-node/ADVANCED_GUIDE.md
- Localization runtime: packages/bijou-i18n/README.md
- Localization tooling: packages/bijou-i18n-tools/README.md
- Filesystem exchange tooling: packages/bijou-i18n-tools-node/README.md
- Spreadsheet exchange tooling: packages/bijou-i18n-tools-xlsx/README.md

Rendering Doctrine

The core rendering posture is simple:

- structured terminal UI should stay on the Surface path
- strings are explicit boundary output, not the internal runtime model

- themes, backgrounds, and shell chrome should resolve to bytes before the hot render loop, not by reparsing strings every frame
- lowering back to text is allowed, but it should happen deliberately at the boundary

Start here when that doctrine matters:

- packages/bijou/ADVANCED_GUIDE.md
- packages/bijou-tui/ADVANCED_GUIDE.md
- Byte-Pipeline Recovery
- Low-Allocation Renderer
- Rendering Pipeline Spec

Shells, DOGFOOD, And Proving Apps

DOGFOOD is not a side example. It is the canonical human-facing docs and shell proving surface for the repo.

Use these docs when you are shaping shell behavior, shell-owned overlays, settings, notification review, or documentation posture:

- DOGFOOD
- DOGFOOD strategy note
- Humane Shell
- Settings Belong To The Shell
- Notification History Belongs To The Shell
- packages/bijou-tui/ADVANCED_GUIDE.md

Useful proving surfaces:

- examples/docs/app.ts
- examples/release-workbench/main.ts
- scripts/docs-preview.test.ts
- tests/cycles/RE-017/frame-shell-theme-dogfood-demo.test.ts

Pipeline, Shaders, Painting, And Transforms

If you are touching frame-to-frame rendering, post-process effects, or shader behavior, do not stop at the API entrypoints. Read the doctrine and the implementation seams together.

Start here:

- packages/bijou-tui/ADVANCED_GUIDE.md
- Layout Localization Pipeline
- packages/bijou-tui/src/pipeline/pipeline.ts
- paint middleware
- motion middleware
- css middleware

- grayscale middleware
- transition shaders
- Shader V2 Spec
- Render Pipeline guide backlog note

Performance And Soak Work

Bijou treats performance as a product surface, not as a cleanup pass.

The current performance and soak lane is spread across:

- examples/perf-gradient/main.ts
- perf overlay
- Byte-Pipeline Recovery
- 4.4.0 release notes
- Debug Performance Overlay Component

That is the lane to read when you are investigating hot-loop cost, proving a new optimization, or deciding whether a feature belongs in the runtime, middleware, or shell.

Localization And I18n Workflows

Localization is not only string substitution. The repo posture includes directionality, shell copy, proving surfaces, and exchange workflows.

Start here:

- Localization and Bidirectionality
- Layout Localization Pipeline
- packages/bijou-i18n/README.md
- packages/bijou-i18n-tools/README.md
- packages/bijou-i18n-tools-node/README.md
- packages/bijou-i18n-tools-xlsx/README.md
- LX-001 runtime package design
- LX-002 tooling package design

When validating localization work, also look at the proving surfaces:

- examples/docs/app.ts
- scripts/docs-preview.test.ts
- tests/cycles/LX-008/localized-shell-and-dogfood.test.ts

Runtime Boundaries, Workers, And Capture

If the question is about Node ownership, worker threads, recorder flows, or how Bijou crosses the runtime boundary, read the Node lane directly:

- packages/bijou-node/ADVANCED_GUIDE.md
- worker runtime spec
- examples/v3-worker/README.md
- examples/v3-worker/main.ts
- examples/v3-worker/record.ts

Testing, PTY, And Release Proof

The advanced bar in Bijou is not “unit tests only.” Real proof usually spans:

- deterministic package tests
- DOGFOOD/docs preview tests
- PTY or canary smoke coverage
- release-readiness and release-preflight
- targeted performance probes when render-path work changes

Useful anchors:

- scripts/smoke-canaries.ts
- scripts/smoke-dogfood.test.ts
- scripts/docs-preview.test.ts
- docs/release.md
- docs/method/release-runbook.md

Rule Of Thumb

If the public GUIDE.md got you productive, stay there.

If you are asking “how does the shell own this,” “where should this render-path decision live,” “how do we prove this in DOGFOOD,” or “what workflow keeps this truthful across runtime modes,” you are in ADVANCED_GUIDE territory.

45

Documentation Standards

docs/DOCUMENTATION_STANDARDS.md

Status: Current project policy for new and substantially changed documentation.

Applies to: User, DOGFOOD, component, CLI, MCP, runtime, IR, release, workflow, and contributor documentation in Bijou.

Normative terms: **MUST**, **SHOULD**, and **MAY** indicate requirement strength.

This standard adapts reader-task documentation discipline to Bijou's existing contract and DOGFOOD practices. It does not require a mass rewrite of existing pages. Apply it when creating documentation, changing behavior, or touching a page enough that leaving it below this bar would create new debt.

1. Purpose

Documentation is part of the product contract. A Bijou page should help a specific reader do one of these jobs:

- learn the product through a guided first success;
- complete a real task in their own environment;
- look up exact facts while working;
- understand a concept, invariant, boundary, or design choice;
- troubleshoot an observable failure;
- change the implementation safely and verify the result;
- inspect a release, cycle, or architecture decision and its evidence.

A page MUST have one primary job. Do not force a README, design document, topic page, or release packet to behave as a tutorial, reference manual, roadmap, and architecture guide at the same time.

2. Corpus Map

Bijou keeps durable truth in a small set of known places.

Location	Primary job
README.md	Public front door: what Bijou is, package roles, quick start, current release posture, and links to deeper documentation.
GUIDE.md	Repository and package orientation for readers moving beyond the quick start.
docs/README.md	Documentation spine and intent-based routing index. New durable pages MUST be linked here.
docs/DOGFOOD.md	Contract for the canonical human-facing documentation application and proving surface.
docs/guides/	Tutorials and how-to guides for supported user and contributor workflows.
docs/design-system/	Living design-system reference: foundations, tokens, components, families, and patterns.
docs/invariants/	Short, living explanations of architectural laws that current implementation and tests uphold.
docs/CLI.md	Exact command and operator-script reference.
docs/MCP.md	Exact MCP server and tool reference.
docs/ARCHITECTURE.md	Current structural explanation for packages, ports, adapters, and major runtime relationships.
docs/design/	Cycle design records. Active records define proposed contracts; landed records preserve why a change was made.
tests/cycles/	Executable cycle evidence. Tests are the specification when prose and executable behavior disagree.
docs/WORKFLOW.md	Concise operator workflow and recurring safety practices.
docs/CHANGELOG.md	Release-visible historical ledger.
docs/archive/, docs/audit/, docs/audits/, and docs/method/retro/	Historical snapshots. They explain prior state and MUST NOT pose as current truth.

GitHub issues and milestones own roadmap and release scope. Linear is the chosen execution tracker; migration is pending connection and workspace/team selection. Until verified Linear links exist, source GitHub issues remain live. Repository documentation explains contracts, decisions, and evidence, not mirrored queue state.

Do not create empty placeholder directories. Add a page when it has a real reader job.

3. Page Types

3.1 Tutorial

A tutorial is a guided learning path. Use one when a newcomer needs a controlled first success, such as rendering a first component or building a small framed application.

A tutorial MUST:

- state prerequisites and starting state;
- use a known-good supported path;
- provide actions in tested order;
- show expected intermediate and final results;
- explain how to verify success;

- end with what the reader learned and where to go next.

3.2 How-To Guide

A how-to guide helps a competent reader complete a real task, such as adding a component to DOGFOOD, recording a fixture, or running a release gate.

A how-to guide MUST:

- be titled as a goal, preferably starting with a verb;
- state the expected result;
- identify blocking prerequisites;
- give the shortest safe route to the result;
- include exact commands, settings, controls, or API calls;
- explain how to verify success;
- link to reference or explanation instead of reproducing it.

3.3 Reference

Reference pages support exact lookup. Public reference includes:

- package exports and component options;
- CLI commands, arguments, exit behavior, and output;
- MCP tools, inputs, results, and errors;
- IR, artifact, source-map, receipt, and validation contracts;
- theme tokens, component families, and lower-mode behavior;
- configuration, environment variables, and repository scripts.

Reference MUST state exact names, syntax, fields, defaults, constraints, compatibility behavior, output, errors, and examples. When the underlying surface is machine-readable, reference SHOULD be generated or coverage checked.

Reference describes behavior that exists. Proposed behavior belongs in a design record, issue, or roadmap.

3.4 Explanation And Invariant

Explanation develops a mental model: why Bijou separates Components from Blocks, why layout owns interaction geometry, or why graceful lowering preserves meaning.

Explanation SHOULD describe mechanisms, relationships, tradeoffs, alternatives, and limits. It MUST NOT become an unstructured source-file tour.

An invariant page states one durable law. It MUST name:

- the law;
- the boundary it protects;
- observable consequences;

- executable evidence or the current evidence gap.

3.5 Design Record

A design record shapes one bounded change. It MUST identify:

- the decision summary and hill;
- current truth;
- scope and non-goals;
- public, runtime, artifact, or workflow contract;
- tests to write first;
- acceptance criteria;
- playback questions;
- tracker ownership.

Proposed work is not evidence. After implementation, the record SHOULD include the measured result or link to its witness.

3.6 Test Plan And Executable Evidence

A test plan is the contract ledger for behavior. It MUST identify:

- stable requirement or cycle identities where the repository uses them;
- planned or implemented cases;
- the exact behavior or invariant under test;
- the oracle;
- the evidence type;
- the status;
- the concrete test, fixture, script, or witness when implemented.

A gap MUST be marked as a gap. Planned work MUST NOT be described as passing evidence.

3.7 Troubleshooting

Troubleshooting starts with a symptom a user or operator can observe, such as:

- terminal output is missing color;
- input reaches the wrong layer;
- DOGFOOD does not render;
- an artifact fails validation;
- a release or Code Dojo gate fails.

A troubleshooting page MUST list discriminating checks first, map signals to likely causes, give concrete recovery actions, and show how to verify the fix.

3.8 Workflow Reference

A workflow reference describes a recurring contributor or maintainer operation, such as preparing a release, processing review feedback, or updating a ratchet.

A workflow reference MUST:

- describe only the current operational contract;
- identify authoritative scripts and runbooks;
- link to workflow verification or witness evidence;
- place warnings before destructive, privileged, or externally visible steps;
- avoid duplicating product reference, release notes, or roadmap promises.

3.9 Release And Historical Record

A release packet records a bounded release decision, its scope, gates, and publication evidence. A changelog records notable user- or operator-visible change.

Historical records MUST remain clearly historical. Do not silently rewrite a past audit, retro, or shipped release record to match current implementation. Add a current reference or a dated correction instead.

4. Documentation Upkeep Loop

For a meaningful behavior, contract, or workflow change:

1. Update design or rationale when the change needs design discussion.
2. Update the relevant test plan or cycle contract before implementation.
3. Add the smallest deterministic executable evidence that fails for the missing behavior.
4. Implement the behavior.
5. Update living reference after the behavior exists.
6. Mark planned cases implemented and record the actual evidence.
7. Update README.md, docs/README.md, docs/CHANGELOG.md, the external tracker when public surface, documentation routing, release status, or project posture changes.
8. Run documentation upkeep before final self-review and before requesting final pull-request review.

Small fixes may scale this down. They still need an honest claim, executable evidence when behavior changes, and living documentation that does not contradict the result.

5. Examples And Executable Truth

Examples are part of the contract.

User-facing examples MUST:

- be syntactically valid;
- use supported behavior;
- include enough context to run or interpret them;
- use least-privileged and safe defaults;
- identify destructive or privileged actions before they appear;
- show an observable result when one exists.

Examples SHOULD be extracted from tested files or executed in CI when practical.

5.1 Runnable, Illustrative, And Abridged Examples

A runnable example uses supported behavior and includes required context. Test or execute it automatically when practical.

An illustrative example may omit setup or nonessential detail, but it MUST be labeled illustrative and MUST NOT be presented as directly runnable.

An abridged example may shorten large input or output, but it MUST identify the omitted material and preserve the behavior relevant to the explanation.

5.2 Code Blocks And Terminal Examples

Every fenced block SHOULD declare its language or content type:

- bash or sh for copyable shell commands;
- typescript, json, yaml, or the relevant language for structured input;
- text for expected output;
- console only for a deliberate transcript containing prompts and output.

Do not include $ or > prompts in a block intended for copy and paste. Present commands and output separately.

Run:

```
npm run code-dojo:debt
```

Expected output shape:

```
Code Dojo debt: N violations
```

When output is nondeterministic, say which parts vary. Label output as exact, representative, or abridged when the distinction matters. Never fabricate output to make an example appear complete.

5.3 Placeholders

Use clearly fictional and context-safe values.

Context	Preferred placeholder
Copyable shell command	sample.ts, example-target, or $BIJOU_EXAMPLE
Configuration value	"sample" or "example-theme"
Hostname	example.com
Formal syntax notation	<artifact>
Secret or credential	an explicitly fake value such as test_token_example

Do not use <your-file> inside a copyable shell command because angle brackets have shell meaning.

5.4 Dangerous Commands

For destructive, privileged, costly, irreversible, or externally visible actions:

1. Place the warning before the command.
2. State the exact consequence and scope.
3. Provide a dry run or safer alternative when available.
4. State required permissions.
5. Include backup or rollback guidance when applicable.
6. Explain how to verify the result.

6. Visuals And Accessibility

Visual products should be shown visually when the visual answers a reader question. Bijou documentation MAY use terminal captures, DOGFOOD fixtures, screenshots, annotated examples, or short recordings.

Every nontrivial visual MUST:

- answer a stated or obvious reader question;
- have meaningful labels or adjacent explanation;
- include alt text or a concise textual equivalent;
- distinguish conceptual simplification from exact implementation when needed;
- omit secrets, personal data, production identifiers, and sensitive details.

Informative images MUST have alt text or a textual equivalent. Decorative images SHOULD have empty alt text where the publishing surface supports it. Complex diagrams SHOULD have adjacent explanatory prose instead of relying on a long alt attribute.

Visuals MUST NOT rely on color, position, animation, or shape alone to communicate essential meaning. Screenshots and recordings MUST NOT be the only place where essential instructions, code, or errors appear.

Terminal examples MUST preserve accessible and pipe-mode meaning when the document claims those modes are supported.

7. Writing, Style, And Terminology

Write like a competent teammate: direct, precise, and approachable.

- Use you for actions the reader performs.
- Use Bijou, the package name, command, script, or component name for actions the system performs.
- Use imperative verbs for procedures.
- Prefer active voice when it clarifies responsibility.
- Use present tense for current behavior.
- Avoid hype, vague reassurance, unnecessary apology, and excessive exclamation.
- Avoid we unless referring to an explicit project decision or policy.

Put the result, decision, warning, or essential condition first. Give each sentence one main job. Use numbered lists for ordered procedures and bullets for parallel options, requirements, or checks.

7.1 Markdown And Typography

Use formatting to communicate type, not to manufacture emphasis.

- Use bold for exact visible labels and genuine warnings.
- Use inline code for commands, options, filenames, paths, configuration keys, fields, literal values, error identities, and code symbols.
- Use exact casing for products, packages, commands, fields, and errors.
- Use <kbd> for keyboard keys where the publishing surface renders it accessibly; otherwise use inline code.
- Use descriptive link text that states what the destination provides.
- Do not use here, this link, or a bare filename as the entire link label.
- Declare a language or content type on fenced blocks.

Use tables only for genuinely two-dimensional lookup, comparison, or structured facts. Do not use them for long narrative passages or procedures.

7.2 Canonical Terminology

Use one canonical term for each concept. Define unfamiliar terms at first use. Mention an alias once only when it materially improves search or recognition.

Use exact Bijou distinctions:

- A **Component** is leaf rendering vocabulary.
- A **Block** owns product semantics, data contracts, and lowering facts.
- A **Surface** is structured terminal-cell output.
- ui-scene-ir/1, bijou-block/1, and other versioned artifacts are explicit contracts.

- Interactive, static, pipe, and accessible are output modes with distinct claims.
- A command requests change; an effect interacts with the outside world.

7.3 Inclusive And Accessible Language

Use literal, neutral language that describes the technical condition directly.

- Use gender-neutral language when gender is irrelevant.
- Avoid identity-based or stigmatizing metaphors.
- Prefer unavailable, hidden, degraded, unresponsive, excluded, or blocked when those are the actual conditions.
- Avoid culturally specific idioms when they make instructions harder to understand or translate.

7.4 Notes, Cautions, And Warnings

Use callouts consistently:

- **Note** — useful context that does not affect safety or correctness.
- **Important** — information required to complete the task correctly.
- **Caution** — an action may cause an undesirable or costly result.
- **Warning** — an action may cause data loss, a security problem, or an irreversible change.

Do not use a warning merely to make ordinary text look important.

8. Checks And Enforcement

Documentation quality requires deterministic checks and human judgment.

Run the current repository documentation gates for documentation changes:

```
npm run docs:inventory
git diff --check
```

When design-system documentation changes, also run:

```
npm run docs:design-system:preflight
```

When workflow files change, run an available pinned workflow validator. If the repository does not provide one, record that absence rather than claiming the workflow was validated.

The repository contains .markdownlint.json, but no pinned Markdown-lint runner or internal-link checker is currently part of package.json. Until those tools land, Markdown style and internal links remain required review checks rather than falsely reported automated gates.

CI SHOULD block on facts it can determine reliably:

- malformed Markdown when a pinned checker exists;
- broken internal paths and explicit anchors when an offline checker exists;
- references to repository files that do not exist;
- failed examples or tutorials declared executable;
- undocumented public commands, options, settings, fields, statuses, or errors when coverage is required;
- informative images without alt text or a textual equivalent;
- changed contract behavior without updated evidence or a documented no-documentation-impact decision;
- destructive examples without a preceding warning;
- examples containing real credentials or forbidden production identifiers.

The following SHOULD normally remain advisory:

- page, sentence, or paragraph length;
- passive voice and jargon density;
- number of bullets;
- suspected missing diagrams;
- tone and template-like phrasing;
- table complexity;
- external-link health;
- screenshot age;
- pull-request size.

These signals help editors. They are poor universal merge gates.

9. Review Checklist

Before calling a documentation change complete, check:

- The page has one primary reader job.
- Living references describe the behavior proposed for the merge target.
- Planned work lives in a design record, test plan, roadmap, issue, or PR.
- Examples use supported behavior and show observable results when practical.
- Public commands, options, settings, fields, statuses, and errors have or link to reference coverage.
- User-facing terminal behavior has a capture, fixture, or textual equivalent when a visual materially helps.
- New durable pages are linked from docs/README.md.
- Release-visible changes update docs/CHANGELOG.md.
- Direction or release-order changes update GitHub issues and milestones, with linked Linear execution items once connected.
- npm run docs:inventory and git diff --check pass.

The objective is not a perfectly uniform library. The objective is a documentation corpus where readers, reviewers, tests, and agents can find the right authoritative page at the moment they need it.

46

Secondary Example Map

docs/EXAMPLES.md

This file is no longer a giant per-example status ledger.

It is a maintainer-facing and agent-facing reference map, not a public docs hub.

DOGFOOD is the primary living-docs and proving surface. In v4.4.0, 31 examples were retired because DOGFOOD now covers their component families directly. The remaining examples exist for runtime seam references, shell proving, and leaf API demos that DOGFOOD does not yet teach.

If you are trying to learn Bijou, start with DOGFOOD, not this file.

For the full secondary/internal inventory, use ../examples/README.md.

Keep

These examples still have clear secondary/reference value.

Runtime References

Keep these as the main runtime reference set for the v4.0.0 line:

- examples/v3-demo
- examples/v3-css
- examples/v3-motion
- examples/v3-subapp
- examples/v3-worker
- examples/v3-pipeline

They keep the historical v3-* names, but they remain the clearest runtime seam references on this branch.

DOGFOOD And Shell Proving Surfaces

Keep these because they prove product-level behavior rather than isolated leaf rendering:

- examples/docs

    - the living docs app and current proving surface
- examples/release-workbench
    - canonical multi-view control-room shell
- examples/notifications
    - notification stack, routing, history, and shell review behavior
- examples/app-frame
    - framed shell reference

Remaining Leaf Examples

Keep leaf examples when they are the clearest isolated explanation of a family or API seam not yet covered by DOGFOOD stories. The remaining set in this file is the current reference for example retirements that still depend on deeper DOGFOOD teaching. The historical depth follow-on is DF-020.

Cut Later

An example becomes a retirement candidate only when all of the following are true:

1. DOGFOOD or another canonical scenario now teaches the same decision clearly.
2. No migration guide or package README still points to the example.
3. It is not carrying unique smoke or regression value.
4. It is not the clearest isolated proof of a public API seam.

Relationship To Release Smoke

Release smoke now runs through smoke:dogfood.

Tracked closure:

- WF-003 — Replace smoke:examples:* With smoke:dogfood

That means examples are no longer the release-facing smoke contract even if some internal reference tooling around example scenarios still exists.

47

Migrating To Bijou v4.0.0

docs/MIGRATING_TO_V4.md

This guide is for existing Bijou apps moving onto the 4.0.0 release line.

The short version:

- @flyingrobots/bijou is still the degradation-first toolkit for CLIs, prompts, logs, and composable terminal output.
- @flyingrobots/bijou-tui is the high-fidelity runtime for fullscreen and app-like terminal experiences.
- @flyingrobots/bijou-tui-app is the framed shell built on top of that runtime.
- @flyingrobots/bijou-node provides the Node adapters plus the worker runtime and native demo recorder.

Upgrade Checklist

1. Upgrade all Bijou packages together to 4.0.0.
2. Decide whether each part of your app is a core string-first flow or a V4 runtime flow.
3. Treat Surface-returning helpers as real render values, not implicit strings.
4. Keep the fullscreen runtime pure: App.view and framed pane renderers return Surface or LayoutNode, never raw strings.
5. Prefer the *Surface companion helpers for common runtime chrome, and cross string boundaries explicitly with surfaceToString(..., ctx.style) only when you truly need a string endpoint.
6. If you use nested TEA apps, prefer initSubApp() and updateSubApp() for lifecycle wiring, with mount() for rendering.
7. If you adopt BCSS, stay within the documented supported scope for the current release line.
8. If you author custom transition shaders, use the v4 override names: overrideChar, overrideCell, and overrideRole.

Package Roles In v4

@flyingrobots/bijou

Use this for:

- prompts and forms
- CLI output and logs
- boxes, tables, trees, badges, alerts, progress bars
- themes, output-mode detection, and test adapters

@flyingrobots/bijou remains the stable, pure, degradation-first package. It contains both string-oriented and surface-oriented primitives. 4.0.0 does not require every core helper to return Surface.

@flyingrobots/bijou-tui

Use this for:

- fullscreen TEA apps
- layout trees and surface composition
- motion, overlays, and shell-style interaction
- programmable pipeline features

The runtime now treats ViewOutput as the public render contract.

```
type ViewOutput = Surface | LayoutNode;
```

@flyingrobots/bijou-tui-app

Use this when you want:

- framed shell chrome
- tabs, footer/help chrome, overlays, drawer/modal flows
- a batteries-included app skeleton

The shell accepts pane renderers that return ViewOutput, not just string blocks.

@flyingrobots/bijou-node

Use this for:

- createNodeContext() / initDefaultContext()
- raw input, resize events, styling, file access
- runInWorker() / startWorkerApp()
- recordDemoGif() and related native recorder helpers

The Biggest Behavioral Change: Pure Runtime Boundaries

Some runtime-first helpers now return Surface. The most visible example is badge().

That means this old pattern is no longer safe everywhere:

```
console.log(badge('OK'));
```

If the surrounding API expects a string, convert explicitly:

```
import { badge, surfaceToString } from '@flyingrobots/bijou';

const ctx = initDefaultContext();
const text = surfaceToString(badge('OK', { ctx }), ctx.style);
console.log(text);
```

The same rule applies when you feed badge() or other surface output into:

- console.log(...)
- template strings
- Array.join(...)
- string-first helpers like table() cells

For common app-shell composition, prefer the surface-native companions instead of converting back to strings:

```
import { badge, separatorSurface, tableSurface } from '@flyingrobots/bijou';

const table = tableSurface({
  columns: [{ header: 'Service' }, { header: 'Status' }],
  rows: [['api', badge('LIVE', { variant: 'success', ctx })]],
  ctx,
});
```

The preferred surface-native companions are:

- boxSurface()
- headerBoxSurface()
- separatorSurface()
- alertSurface()
- tableSurface()
- vstackSurface()
- hstackSurface()

Transition-era aliases were removed from the runtime-facing API surface:

- use boxSurface(), not boxV3()
- use vstackSurface(), not vstackV3()
- use hstackSurface(), not hstackV3()

surfaceToString() Now Needs A StylePort

In 4.0.0, surfaceToString() requires a StylePort.

```
surfaceToString(surface, ctx.style);
```

If you previously called surfaceToString(surface) with one argument, update those call sites.

App.view And Framed Pane Renderers

The runtime now documents the truth:

```
interface App<Model, Msg> {
  view(model: Model): Surface | LayoutNode;
}
```

Framed pane renderers follow the same rule. Return:

- Surface for direct rendering
- LayoutNode for layout-tree composition

If you still have string-returning runtime views, convert them explicitly before they cross the runtime boundary.

Sub-Apps: Prefer Lifecycle Helpers

For nested TEA apps, the release-ready pattern is:

- initSubApp() for child startup
- updateSubApp() for child updates and command mapping
- mount() for rendering the child view into the parent
- mapCmds() when you need direct command mapping

mount() alone is not the full lifecycle story.

BCSS Scope In 4.0.0

BCSS is real in 4.0.0, but intentionally scoped.

Supported:

- Type, .class, and #id selectors
- var(token.path) lookups
- @media (width ...) and @media (height ...)
- supported text-style properties on documented V3 surface components
- frame shell regions that expose BCSS identity

Not promised in 4.0.0:

- a global CSS cascade across arbitrary layout nodes
- blanket styling of every component in the repository

If you need a good template, start from examples/v3-css.

Recommended Migration Flow

1. Upgrade dependencies together

```
npm install @flyingrobots/bijou@4.0.0 @flyingrobots/bijou-node@4.0.0
```

Add the runtime packages if the app uses them:

```
npm install @flyingrobots/bijou-tui@4.0.0 @flyingrobots/bijou-tui-app@4.0.0
```

2. Fix string/surface seams first

Look for:

- console.log(badge(...))
- `${badge(...)}`
- arrays of mixed strings and Surface
- string-only helper APIs receiving Surface

3. Update framed panes to ViewOutput

If your app uses createFramedApp(), return Surface or LayoutNode directly. Remove any string-returning pane shims.

4. Update nested app wiring

If you built manual child init/update plumbing, consider moving to initSubApp() and updateSubApp().

5. Add smoke coverage

At minimum, run:

```
npm run build
npm test
npm run typecheck:test
npm run smoke:dogfood -- --skip-build
```

Canonical Runtime References

These directories keep their historical v3-* names, but they are still the canonical runtime references for the 4.0.0 line on this branch.

- Runtime starter: examples/v3-demo
- BCSS: examples/v3-css
- Motion: examples/v3-motion
- Sub-apps: examples/v3-subapp
- Workers: examples/v3-worker

- Pipeline extension: examples/v3-pipeline

48

Bijou Specifacts

docs/specs/README.md

This directory is **legacy** planning surface and is retained for historical context. Do **not** start new work here.

This directory contains JSON acceptance artifacts that describe cross-cutting product expectations for rendering, layout, worker runtime, and component behavior.

The repository currently treats these files as stable historical references plus living acceptance contracts for large features. When you add new behavior, prefer docs/strategy, docs/design, and docs/method for planning and execution, then reflect the final contract here if it helps future maintainers.

Purpose

Each *.spec.json file captures:

- a concise feature name and design intent,
- requirements and user story,
- acceptance criteria (what can be tested or observed),
- scope boundaries,
- a minimal test plan.

They are useful for:

- onboarding new contributors to historical design decisions,
- translating high-level design docs into concrete test scenarios,
- aligning future implementation work with explicit pass/fail criteria.

Current Directory Shape

- docs/specs/animation-heartbeat.spec.json
- docs/specs/bijou-css.spec.json
- docs/specs/bijou-web.spec.json
- docs/specs/buffered-surfaces.spec.json

- docs/specs/cell-diffing.spec.json
- docs/specs/component-system.spec.json
- docs/specs/continuum-bridge.spec.json
- docs/specs/data-viz.spec.json
- docs/specs/decoupled-layout.spec.json
- docs/specs/devtools-inspector.spec.json
- docs/specs/effect-handlers.spec.json
- docs/specs/eventbus-middleware.spec.json
- docs/specs/focused-pane-input.spec.json
- docs/specs/frame-settings-drawer.spec.json
- docs/specs/motion-api.spec.json
- docs/specs/preference-list-surface.spec.json
- docs/specs/rendering-pipeline.spec.json
- docs/specs/shader-v2.spec.json
- docs/specs/shell-notification-center.spec.json
- docs/specs/shell-quit-policy.spec.json
- docs/specs/snapshot-harness.spec.json
- docs/specs/story-protocol.spec.json
- docs/specs/sub-app-composition.spec.json
- docs/specs/syntax-textarea.spec.json
- docs/specs/token-graph.spec.json
- docs/specs/worker-runtime.spec.json

JSON Contract (Observed Shape)

A spec file generally follows this structure:

```
{
  "name": "Feature Name",
  "designPattern": "Component / Pattern Name",
  "requirements": [
    "Requirement 1",
    "Requirement 2"
  ],
  "userStory": {
    "asA": "Persona",
    "iWant": "Desired behavior",
    "soThat": "Resulting benefit"
  },
  "acceptanceCriteria": [
    "Observable result that indicates success"
  ],
  "backwardCompatibility": "Compatibility notes or migration story",
  "scope": {
    "inScope": [
      "Area 1",
      "Area 2"
    ],
    "outScope": [
      "Not covered in this slice"
    ]
  },
  "testPlan": {
    "goldenPath": "Recommended manual/automated check",
    "edgeCases": ["Edge case scenario"],
    "fuzzStress": "Stress or scale scenario",
```

```
    "failureCases": ["Known failure to catch"]
  },
  "priority": "P1",
  "estimatedComplexity": {
    "linesOfCode": "~200",
    "timeComplexity": "O(...)",
    "workEstimate": "Medium"
  }
}
```

All fields are plain JSON strings, arrays, or small objects. The schema is intentionally simple so specs can be edited and reviewed without generated tooling.

validateSpec() Utility

At present, there is no globally required build-time validator for these files. For local consistency, follow this pattern before merging a spec change:

1. Parse JSON.
2. Validate that required top-level keys exist (name, requirements, acceptance Criteria, userStory, testPlan, scope, priority).
3. Validate value shapes (string, array, and nested object forms).
4. Optionally add a unit test around the parse rule for your feature area.

If you add or refactor a script, use a function named validateSpec() with this contract:

```
type SpecValidationError = { path: string; message: string };

type SpecValidationResult = { ok: boolean; errors: readonly SpecValidationError[] };

function validateSpec(value: unknown): SpecValidationResult {
  // parse and validate contract, return shape errors only.
}
```

This keeps spec maintenance local and transparent, and lets future tooling consume the same contract without hard coupling to a single process.

How to Add a New Spec File

1. Create docs/specs/<name>.spec.json and keep it narrowly focused.
2. Use one user story and explicit acceptance criteria.
3. Distinguish in-scope and out-of-scope decisions.
4. Add at least one test plan section (goldenPath, edgeCases, failureCases).
5. Update any related design docs and add a short link from the spec if relevant.
6. Run a JSON parse check (manual cat + node -p) before commit.

Example quick-check:

```
node -e "const fs=require('node:fs');
↪  JSON.parse(fs.readFileSync('docs/specs/rendering-pipeline.spec.json', 'utf8'));"
```

Operational Notes

The spec directory is not a replacement for executable tests. It is a contract surface that should be supported by:

- unit tests,
- integration tests,
- DOGFOOD coverage checks,
- release smoke and regression gates.

If a spec is no longer accurate, archive it in place and create an updated one rather than silently mutating historical evidence.

49

Render Pipeline Guide

docs/guides/render-pipeline.md

The render pipeline in @flyingrobots/bijou-tui is a deterministic, stage-ordered execution chain.

It runs as:

1. Layout
2. Paint
3. PostProcess
4. Diff
5. Output

Default Runtime Wiring

run() installs middleware in this fixed stage order:

1. Layout: call app.view(model) and produce the layout output.
2. Layout: apply motion reconciliation.
3. Layout: apply BCSS middleware when options.css is present.
4. Paint: paint the resolved layout tree into targetSurface.
5. Diff: compute terminal diff and fill outBuf.
6. Output: emit diff and swap framebuffers.
7. configurePipeline(pipeline) if provided by the caller.

Custom middleware runs after the built-ins in the same stage and cannot change stage ordering.

Default and Custom Stage Contract

Use configurePipeline() when your middleware belongs to one of these five stages:

- Layout: inspect or rewrite LayoutNode trees.
- Paint: mutate targetSurface by painting extra marks directly.
- PostProcess: apply surface-wide shaders or instrumentation before diff.
- Diff: adjust output encoding just before emit (advanced usage).

- Output: run end-of-frame side-effects after terminal write and swap.

Inter-Stage Data Contracts

A RenderState instance is shared across all middleware for a single frame. The key shared state channels are:

- model: current model value for this frame.
- layoutMap: geometry metadata for resolved nodes.
- data: unstructured scratchbag intended for middleware communication.

layoutRoot via RenderState.data

In current runtime code, the Layout stage places the resolved layout tree in state.data['__bijou:layoutRoot'].

That object is then consumed by motionMiddleware() and paintMiddleware() in later stages.

Recommended access pattern for custom middleware:

- Read and write your own namespaced keys, for example:
  - state.data['my-app:focusState']
  - state.data['my-app:trace:enabled']

This avoids private runtime keys and avoids coupling across feature layers.

The built-in layout keys are intentionally not part of your public app contract.

LayoutMap Semantics

layoutMap is currently populated with keyed geometry for internal systems such as a11y overlays, layout debugging, and event routing.

If you need layout geometry from custom middleware, guard key lookups and treat missing values as an expected pre-layout state.

Framebuffer Flip Model (Current Surface Truth)

Diff compares:

- state.currentSurface: the previous visible surface
- state.targetSurface: the next surface under construction

After Output runs, the runtime flips the internal buffers.

The conceptual model is:

```
before Output:   currentSurface is frame N, targetSurface is frame N+1
after Output:    currentSurface is frame N+1, targetSurface becomes previous frame N
```

That means middleware appended to Output after swap must not assume that stat e.currentSurface still points at the old frame unless it reads it before invoking default output semantics.

In practical terms:

- Use currentSurface for diff inputs and visibility of the previous frame.
- Use targetSurface as the paint target for the frame currently being built.
- Do not retain references across frames outside Layout/Paint middleware.

Why the Pipeline Is Synchronous

The pipeline is synchronous by design to keep frame cost measurable and predictable.

Bijou's TEA runtime calls view(model) for a frame. That full-model render is intentional: state is explicit, the frame can be reproduced from one model snapshot, and tests can ask "what would this model render?" without replaying hidden mutations.

The tradeoff is that expensive view work is paid on every rendered frame. Keep view() pure and bounded:

- store durable app facts in the model,
- derive only the visible slice needed for the current frame,
- cache expensive non-visual preparation outside view() when the model changes,
- use commands for IO, timers, and other async work.

If each stage can run with bounded synchronous cost, the frame time budget is:

```
frameCost = layout + paint + postProcess + diff + output + middlewareOverhead
```

If an asynchronous stage is inserted without careful partitioning, total frame cost becomes:

```
frameCost = layout + paint + postProcess + await(someAsyncIO) + diff + output
```

That removes a strict per-frame budget and can stall rendering on unrelated IO.

The architectural rule is therefore:

- Keep rendering synchronous and deterministic.
- Move async work into commands.
- Receive async results as messages, then update model and re-render.

Runtime diagnostics enforce that posture. If a render middleware returns a Promise, Bijou writes a pipeline diagnostic, keeps the current frame moving, and ignores any late next() call for stage ordering. A rejected async middleware Promise is also reported through the pipeline error channel. The diagnostic is not a supported async render mode; it is a guardrail for finding accidental async middleware before it turns into frame latency or out-of-order painting.

Worked Example

The runtime pipeline supports middleware insertion points that preserve the fixed stage ordering.

```
import { run } from '@flyingrobots/bijou-tui';

run(app, {
  configurePipeline(pipeline) {
    pipeline.use('PostProcess', (state, next) => {
      state.targetSurface.set(0, 0, { char: 'X', empty: false });
      next();
    });
  },
});
```

Because this middleware is registered before app execution, it runs on every frame after all built-ins for PostProcess and before default diffing.

Timing Hooks

You can observe per-stage timings from middleware.

```
import { getRenderStageTimings, run } from '@flyingrobots/bijou-tui';

run(app, {
  configurePipeline(pipeline) {
    pipeline.onStageComplete((stage, durationMs, state) => {
      console.log(stage, durationMs);
      console.log(getRenderStageTimings(state));
    });
  },
});
```

Notes:

- onStageComplete(...) fires once per stage, in fixed stage order.
- getRenderStageTimings(state) returns timings captured for that frame.
- The timings array is replaced on every pipeline.execute(state) call.
- Empty stages are represented in timing output.

RenderState Reference

The public shape is:

- model: read by convention in rendering.
- ctx: active BijouContext.
- dt: wall time delta in seconds.
- currentSurface: previous frame surface, normally treated as read-only.
- targetSurface: current frame paint surface.
- outBuf: optional UTF-8 output buffer for default diff stage.
- layoutMap: geometry metadata map.
- data: middleware coordination bag.

data is also where layout root and per-frame timing metadata are held.

Stage Failure Semantics

A middleware exception does not kill the app immediately.

- The exception is routed through the same diagnostics path as other runtime faults.
- The frame may render partially or degrade for that tick.
- Rendering continues so the app remains recoverable and operational.

If your custom middleware must fail fast, emit a command or message to transition into an intentional error state during update().

Read Next

- Layout Localization Pipeline
- packages/bijou-tui/ADVANCED_GUIDE.md

Design system and reference

The following chapters gather the shared visual and structural vocabulary used while composing an application: foundations, patterns, component families, theme rules and exact interchange contracts. Use the family inventory to locate a capability, then follow its owning contract. These source documents retain their original scope; the live application and package version provide the context for applying them.

50

Bijou Design System

docs/design-system/README.md

Bijou now has enough surface area that “here are the exports” is no longer a good documentation strategy.

This section defines the design language behind the library:

- the foundations that shape the UI
- the token vocabulary that turns color doctrine into API rules
- the interaction and layout patterns that recur across apps
- the component families and their intended jobs
- the policy for visualizing data instead of guessing ad hoc each time

This is deliberately closer to Carbon than to an API reference.

What this section is for

Use these docs when you need to answer:

- What problem does this component family solve?
- Which variation should I use?
- Which token family should this component use?
- When should I choose one component over another?
- When should I avoid this component entirely?
- Does this belong in core @flyingrobots/bijou or in @flyingrobots/bijou-tui?
- How should this degrade across interactive, static, pipe, and accessible runs?
- What is the terminal-first equivalent of a desktop/web design-system pattern?

Read in this order

1. Foundations
2. Theme Token Vocabulary
3. Theme Authoring Guide
4. Theme Rule Recipes

5. Patterns
6. Blocks
7. Pointer and Mouse Policy
8. Component Families
9. Data Visualization Policy

For the raw inventory and taxonomy work behind these docs, see ../archive/component-system-audit.md.

Core system ideas

Bijou is not just a component library

Bijou is made of five layers:

1. **Foundations**
   - color, spacing rhythm, text hierarchy, overflow, motion, focus, interruption
2. **Theme vocabulary**
   - canonical token families and first-party usage rules
3. **Patterns**
   - status, selection, browsing, overlays, forms, shell composition
4. **Component families**
   - specific primitives that implement those patterns
5. **Blocks**
   - reusable product-level assemblies such as app frames, settings flows, and guided workflows

Components and blocks should be downstream of the first three layers. If the design language is weak, the component catalog and block library both become noisy no matter how many exports exist.

Core versus TUI ownership

- @flyingrobots/bijou owns semantics that survive degradation.
- @flyingrobots/bijou-tui owns focus, motion, overlays, shell behavior, scrolling, history, and other lifecycle-heavy interactions.

That means:

- alert() belongs in core.
- toast() belongs in TUI.
- notifications belong in TUI because they are a system, not just a rendering.

Variations are not all equal

Bijou uses three kinds of variation:

- **semantic variants**
  - for example status tone or severity
- **interaction-layer variants**
  - for example accordion() versus interactiveAccordion()
- **render-path variants**
  - for example box() versus boxSurface()

Only the first kind is a “visual variant” in the usual design-system sense. The other two exist because Bijou has to survive different interaction and rendering layers.

Graceful lowering is a first-class design constraint

Every family should answer:

- What does this look like in a rich interactive terminal?
- What remains true in static CI output?
- What remains true in pipe mode?
- What becomes simpler in accessible mode?

If a component cannot answer that question, it probably belongs in @flyingrobots/bijou-tui, not in core.

Carbon-style documentation standard

Every mature component-family page in Bijou should include:

- What it is
- Variations
- When to use
- When not to use
- Content guidance
- Interaction ownership
- Graceful lowering path
- Related families
- Closest Carbon analogue

The component-family guide now meets this structural baseline and its structure is checked by a repo preflight script. That preflight is intentionally structural: it verifies required family sections and graceful-lowering mode coverage inside the family guide. Semantic accuracy, package/example doc drift, and touched-component doc completeness are still review responsibilities, not something the current script can prove automatically.

Component-change gate

From the v4 pure-surface cleanup onward, any component-family change should be evaluated against the docs in the same slice.

For every component or component family touched, verify:

- documentation exists
- documented variants are current
- When to use guidance is present and still accurate
- When not to use guidance is present and still accurate
- core-versus-TUI ownership is explicit
- graceful lowering is documented across rich, static, pipe, and accessible when applicable
- related families are listed

The CI preflight enforces the structural minimum for the family guide. If any of the broader checks below are missing or materially wrong, the docs are still incomplete and that gap should be fixed in the same slice or explicitly logged as backlog debt before the work is considered done.

Current gaps this section is meant to close

Bijou is already strong on breadth and runtime architecture. The weaker parts are:

- explicit usage guidance
- family-level documentation
- documentation completeness against the standard defined above
- pattern-level decision support
- data-visualization doctrine
- mouse and multi-input policy

These docs exist to correct that.

51

Blocks

docs/design-system/blocks.md

This document defines what Bijou means by a reusable block.

Blocks are larger, opinionated assemblies built from canonical patterns and component families.

What a block is

A block is not:

- a one-off demo composition
- a single primitive component
- a shell-specific hack that only one proving surface understands

A block is:

- a reusable product-level assembly
- composed from canonical patterns
- built from existing component families and TUI ownership rules
- documented with clear jobs, ownership, and lowering expectations

Blocks come after doctrine, patterns, and components.

That ordering matters:

1. doctrine explains what good Bijou UX should feel like
2. patterns explain recurring interaction language
3. component families provide reusable primitives
4. blocks compose those primitives into repeatable product structures

If a block appears before the first three layers are clear, it usually hardcodes accidental taste instead of reusable language.

What blocks should standardize

Blocks should standardize:

- composition

- structure
- copy rhythm
- focus ownership
- common shell or workflow affordances

Blocks should not hide:

- important interaction ownership
- lowering behavior
- accessibility semantics
- localization or directional assumptions

Current public contract

Bijou now exposes a metadata-first block authoring contract from @flyingrobots/bijou:

- BlockMetadata declares a block's package, name, family, scale, modes, slots, variants, config options, composed components, semantic facts, story ids, and docs.
- BlockDefinition pairs that metadata with an explicit render function.
- BlockPackageManifest declares the exported blocks, package version, Bijou peer range, docs, and tags for ordinary NPM distribution.
- defineBlock() and defineBlockPackage() validate definitions before authors publish or index them.
- defineBlock() can attach DX-034 view data contracts and command intents for discovery without provider handles or command callbacks.
- appShellBlock, readerSurfaceBlock, inspectorPanelBlock, inlineStatusBlock, inFlowStatusBlock, transientOverlayBlock, activityStreamBlock, shortcutCueBlock, progressIndicatorBlock, the DOGFOOD component-family block exports, standardBlocks, standardBlockStories, and standardBlockPackageManifest expose the first-party block definitions: AppShell, ReaderSurface, InspectorPanel, inline status, in-flow status, transient overlay feedback, activity stream, shortcut cues, progress, grouping, explainability, documents, links, dividers, text entry, choices, navigation, disclosure, path progress, brand emphasis, mode-aware primitives, dense comparisons, hierarchies, exploration lists, and temporal dependency views. Their render functions now produce deterministic first-proof output across interactive, static, pipe, and accessible modes while keeping provider lifecycle, command dispatch, and production AppShell behavior out of the block definition layer.
- defineAppShellComposition() groups runtime-backed blocks into logical navigation, content, inspector, status, and overlay slots without rendering.
- blockRenderNode() and renderBlockTree() explicitly render nested block declarations when a host or preview wants child block output inside parent slots. Ordinary block.render() remains a local single-block render contract.

- defineBlockSchemaAdapter(), defineSchemaBlock(), and bindSchemaBlockInput() validate unknown boundary data before producing block render input and binding facts.
- validateBlockMetadata(), blockMetadataSummary(), and report helpers give tests, docs, DOGFOOD, MCP payloads, and agents a deterministic way to inspect block facts without rendering a terminal surface.

Blocks are described before they render, package authors avoid hidden global registration, and tooling can discover slots and mode support directly. Schema-bound blocks validate unknown boundary data without making Zod, GraphQL, or database schemas part of the core block dependency surface. Provider subscriptions, active hierarchy traversal, rendered AppShell, and DOGFOOD block multi-mode captures remain follow-on work.

First credible block candidates

App frame

The framed shell is already one of Bijou's strongest recurring structures.

As a future block, App frame should standardize:

- pane rhythm
- active-region treatment
- shell header/footer ownership
- shell overlays such as settings and help

Settings drawer

The settings drawer is a credible future block because it already has:

- canonical preference rows
- shell-owned dismissal and discovery behavior
- stable section rhythm
- truthful active-row treatment

As a block, it should be reusable without depending on DOGFOOD-specific code.

Notification center

The notification center is a likely future block once Humane Terminal work resumes.

It should eventually standardize:

- notice rows
- archive/history review
- filter rhythm
- shell-owned interrupt versus review behavior

Guided flow

Guided flow is the general block family for:

- onboarding
- setup sequences
- explainability walkthroughs
- multi-step assisted flows

This should compose:

- preference rows
- alerts and notes
- stepper or timeline language
- explicit next actions

DL-005 proves the first canonical seam in this family through explainability( ), which turns the AI explainability doctrine into a structured guided-flow surface instead of leaving it as prose only. DL-008 promotes the reusable block seam through guidedFlow(), which standardizes calmer multi-step assistance, section rhythm, and next-action treatment without forcing every guided surface to masquerade as AI explainability.

Inspector panel

An inspector-style side panel is another strong candidate because it naturally combines:

- titled sections
- supporting copy
- current-value emphasis
- compact but calm spacing

DL-006 now proves the first canonical seam in this family through inspector( ), which turns inspector-panel rhythm into a reusable core component instead of leaving it trapped in a bespoke DOGFOOD detail box. DL-007 promotes the first block-level seam in this family through inspectorDrawer(), which standardizes drawer-attached inspector composition in TUI shells instead of leaving existing inspector drawers to hand-compose strings.

Block acceptance bar

A structure is ready to become a canonical block when:

- it recurs across more than one surface or product need
- its interaction ownership is stable
- its patterns are already canonical elsewhere
- its lowering story is explicit
- it improves reuse instead of trapping more quality inside one shell

What not to do

- do not promote every nice-looking demo into a block
- do not let blocks bypass canonical patterns
- do not let blocks become storage, service, or app-domain assumptions
- do not treat blocks as the first place where accessibility or localization is considered

Relationship to the current cycle

DL-002 does not implement a blocks runtime.

This cycle exists to make the block layer explicit so future cycles can promote real recurring structures deliberately instead of accidentally.

52

Component Families

docs/design-system/component-families.md

This document is the working semantic map for the shipped Bijou component families.

The goal is to stop thinking in terms of “every export is its own component.”

How to read this

- **Family** means one semantic job.
- **Variants** may be semantic, interaction-layer, or render-path variants.
- **Core** means it belongs in @flyingrobots/bijou.
- **TUI** means it belongs in @flyingrobots/bijou-tui.

Documentation completeness status

This page is a living component-family reference.

The stated design-system standard requires each mature family doc to include:

- what it is
- variations
- when to use
- when not to use
- content guidance
- interaction ownership
- graceful lowering across rich, static, pipe, and accessible
- related families
- closest Carbon analogue

This page now meets the structural documentation standard for shipped families: every family below includes variants, usage guidance, content guidance, ownership, graceful lowering, related families, and a Carbon analogue. The remaining work is about depth and sharper examples, not missing required sections.

Component slice checklist

When a component family is changed in code, the matching family docs should be checked in the same slice.

Minimum verification:

- the family is present in this guide
- the listed variants still match reality
- Use when and Avoid when still match the implementation and examples
- graceful lowering still matches the runtime behavior
- related families still make sense after the code change

If those checks fail, the component work is not doctrinally complete yet.

Status and feedback families

Inline status

- Family: badge()
- Variants:
  - semantic/status variants
- Use when:
  - status is compact and local to another object
- Avoid when:
  - message must stand alone or explain itself
- Content guidance:
  - keep badge labels terse, stable, and noun-like or state-like
  - avoid embedding full sentences or multi-step guidance in badges
- Ownership:
  - core
- Graceful lowering:
  - rich/static: keep compact label styling inline with nearby content
  - pipe: lower to plain inline text label without relying on color
  - accessible: speak the status in plain words next to the owning object
- Related families:
  - alert()
  - note()
  - notification system
- Carbon analogue:
  - tag

In-flow status block

- Family: alert()
- Variants:
  - semantic tones
  - render-path companion: alertSurface()
- Use when:
  - status should remain part of the page/document
- Avoid when:
  - lifecycle/history/stacking is required
- Content guidance:
  - include the state, cause, and next useful action in the body when possible
  - keep the headline short and let supporting text carry detail
- Ownership:
  - core
- Graceful lowering:
  - rich/static: remain a boxed or otherwise clearly separated in-flow message
  - pipe: lower to emphasized plain text that stays in document order
  - accessible: keep message text and severity explicit, with no dependence on borders or color
- Related families:
  - badge()
  - note()
  - toast()
  - notification system
- Carbon analogue:
  - inline notification

Low-level transient overlay

- Family: toast()
- Variants:
  - status variants
  - anchor/placement
- Use when:
  - the app needs a one-off transient overlay and is composing overlays directly
  - placement matters, but lifecycle/history does not
- Avoid when:
  - stacking, routing, actions, or recall matter

    - the content should remain in page flow
- Content guidance:
    - keep the message short enough to scan without stopping the task
    - avoid multiline operational detail that really belongs in notifications or a drawer
- Ownership:
    - TUI
- Graceful lowering:
    - rich: anchored transient overlay with placement and timing
    - static: lower to visible in-flow status or last-frame summary rather than hidden motion
    - pipe: lower to one plain event line or app-owned log entry
    - accessible: prefer explicit announcement text over spatial anchoring
- Related families:
    - alert()
    - modal()
    - notification system
- Carbon analogue:
    - toast notification primitive

Transient app notifications

- Family: notification system
- Variants:
    - TOAST
    - INLINE
    - ACTIONABLE
    - tones
    - placements
    - history/archive view
- Use when:
    - the app owns transient messaging and may need stacking, routing, actions, or recall
- Avoid when:
    - the content is primary page content
    - one local overlay is enough and app-wide lifecycle is unnecessary
- Content guidance:
    - titles should summarize the event, while body text should explain consequence or next step
    - actionable notifications should expose one clear primary action instead of a menu of choices
- Ownership:

    - TUI
- Graceful lowering:
    - rich: stacked overlays with placement, action buttons, animation, and archive/history
    - static: visible notices plus accessible history review surface
    - pipe: lower to ordered event lines and routed warnings/errors
    - accessible: simplify to explicit chronological reviewable notices with actions described in text
- Related families:
    - toast()
    - alert()
    - log()
    - modal()
- Carbon analogue:
    - toast notification, actionable notification, notification panel/center

Activity stream

- Family: log()
- Variants:
    - log levels
- Use when:
    - order and accumulation matter
- Avoid when:
    - the message should interrupt instead of accumulate
- Content guidance:
    - log lines should remain chronological, specific, and attributable
    - avoid decorative phrasing that makes scanning and filtering harder
- Ownership:
    - core
- Graceful lowering:
    - rich/static: retain ordered styled lines with level cues
    - pipe: already natural plain sequential output
    - accessible: keep timestamps/levels explicit and avoid decorative formatting
- Related families:
    - notification system
    - alert()
    - badge()
- Carbon analogue:
    - activity log / event stream pattern

Loading placeholders

- Family: skeleton()
- Variants:
  - single-line, multiline, and region-shaped placeholders
- Use when:
  - the expected content shape is known but the content is not available yet
  - a short-lived loading interval would otherwise cause distracting layout pop
- Avoid when:
  - honest partial content can already be shown
  - the delay is long enough that a clearer loading state, progress indicator, or retry path is needed
  - the placeholder would outlive the user’s trust and start to look like fake content
- Content guidance:
  - skeletons should roughly match the final information density instead of becoming ornamental gray bars everywhere
  - use them for short-lived uncertainty, then yield to partial or explicit loading states as soon as real content exists
  - keep labels or surrounding context explicit so the user knows what is loading
- Ownership:
  - core
- Graceful lowering:
  - rich/static: preserve the placeholder shape while loading is genuinely transient
  - pipe: lower to explicit loading text or field labels instead of decorative placeholder bars
  - accessible: announce loading state and the affected region explicitly rather than relying on visual shimmer or shape
- Related families:
  - spinner()
  - progressBar()
  - note()
- Carbon analogue:
  - skeleton state

Inline shortcut cues

- Family: kbd()
- Variants:
  - single key, chord, and adjacent grouped key cues

- Use when:
  - a local action or instruction needs an inline shortcut hint
  - the shortcut belongs next to the thing it affects
- Avoid when:
  - the user needs a broader command reference or grouped shortcut map
  - the chip would become the primary content instead of a supporting cue
- Content guidance:
  - show only the keys the user actually needs at that moment
  - keep shortcut chips close to the related action label instead of collecting them into decorative clutter
  - use shell help views for broader keybinding discovery; keep kbd() for inline, local hints
- Ownership:
  - core
- Graceful lowering:
  - rich/static: preserve compact key-chip treatment inline with related content
  - pipe: lower to explicit key names in plain text without decorative framing being the only cue
  - accessible: speak the shortcut and related action in one readable phrase
- Related families:
  - helpView()
  - commandPalette()
  - note()
- Carbon analogue:
  - keyboard key / shortcut hint

Progress indicators

- Family:
  - progressBar()
  - createProgressBar()
  - createAnimatedProgressBar()
  - spinnerFrame()
  - createSpinner()
- Variants:
  - determinate bar
  - indeterminate spinner
  - live controller helpers
  - animated progress interpolation
- Use when:
  - the user needs honest feedback that work is ongoing

    - determinate versus indeterminate progress is actually known
- Avoid when:
    - loading is so brief that the indicator would flicker more than it helps
    - the state should be explained as a durable message or result instead of transient activity
- Content guidance:
    - use progressBar() when completion can be estimated honestly
    - use spinnerFrame() / createSpinner() when activity is real but percent-complete is not
    - switch from placeholder to progress indicator only when process feedback matters more than preserving layout shape
    - pair progress indicators with clear labels so users know what is happening, not just that something is moving
- Ownership:
    - core
- Graceful lowering:
    - rich/static: preserve determinate or indeterminate activity cues honestly
    - pipe: lower to explicit text progress or repeated status lines without fake animation semantics
    - accessible: announce task state, progress, and completion explicitly in text
- Related families:
    - skeleton()
    - badge()
    - notification system
- Carbon analogue:
    - progress bar / loading spinner

Structural families

Framed grouping

- Family: box()
- Variants:
    - titled via headerBox()
    - render-path companions via boxSurface() and headerBoxSurface()
- Use when:
    - grouping, containment, or local region identity matter
    - peer panels need to read as separate working areas
    - a compact title plus detail helps orient a panel without consuming a full heading block

- Avoid when:
  - urgency or interruption is the primary job
  - every subsection would get its own border purely for decoration
  - a separator, heading, or whitespace would communicate structure more honestly
- Content guidance:
  - use box() for grouped content and headerBox() when the region needs a clear title plus compact secondary detail
  - titles should describe the region's job, not restate surrounding context
  - headerBox() detail text should carry terse metadata such as environment, path, version, or scope rather than a second sentence
  - nested boxes should be rare and should explain real hierarchy or comparison, not just add more chrome
  - keep dense prose out of narrow boxes unless wrapping materially helps comprehension
- Ownership:
  - core
- Graceful lowering:
  - rich/static: keep bordered or titled containment when space allows
  - pipe: lower to plain grouped text with spacing and optional title
  - accessible: preserve title and content order without decorative framing
- Related families:
  - separator()
  - alert()
  - grid()
  - splitPane()
- Carbon analogue:
  - tile / contained section

Explainability walkthroughs

- Family: explainability()
- Variants:
  - visible provenance label
  - rationale, evidence, next-action, and governance sections
  - confidence and source metadata
- Use when:
  - AI-mediated or machine-assisted output needs explicit explanation instead of decorative summary
  - the user needs one calm guided-flow surface that separates recommendation from supporting evidence
  - the next action matters more than conversational prose
- Avoid when:

    - the content is just a generic status or note with no recommendation/evidence structure
    - the app needs a non-AI guided flow or a full multi-step wizard instead of one explainability surface; prefer guidedFlow() for the calmer general case
    - the app is hiding service latency or speculative output behind an authoritative-looking card
- Content guidance:
    - keep the title as the recommendation or explanation itself, not a vague marketing headline
    - make [AI] or equivalent provenance visible
    - separate rationale, evidence, and next action into distinct sections instead of mixing them together
    - governance text should clarify review posture without becoming the dominant content
- Ownership:
    - core
- Graceful lowering:
    - rich/static: preserve section rhythm and explicit provenance inside one calm grouped surface
    - pipe: lower to labeled plain-text sections with one obvious next action
    - accessible: linearize the same fields explicitly with no dependence on borders, color, or layout
- Related families:
    - guidedFlow()
    - note()
    - alert()
    - stepper()
    - timeline()
- Carbon analogue:
    - no exact analogue; closest to an explainability panel or guided recommendation card

Inspector panels

- Family: inspector()
- Variants:
    - current-selection emphasis
    - compact titled sections
    - muted supporting-copy sections
- Use when:
    - a side panel needs to summarize the currently selected thing without taking over the primary task

    - supporting context should stay structured and calmer than the main content
    - the panel needs one obvious current value plus compact supporting sections
- Avoid when:
    - the content is a full guided recommendation with evidence and next-action structure; prefer explainability()
    - the surface is really just an alert, note, or one-line status
    - the panel needs its own complex navigation or multistep interaction model
- Content guidance:
    - make the current selection more obvious than supporting copy
    - keep section titles short and concrete
    - let supporting details look supporting; do not let descriptions overpower the active value
    - stack explicit sections before inventing another decorative box inside the panel
- Ownership:
    - core
- Graceful lowering:
    - rich/static: preserve titled containment, current-selection emphasis, and compact section rhythm
    - pipe: lower to explicit field labels with one obvious current selection
    - accessible: linearize the same fields in plain language without borders or color
- Related families:
    - box()
    - explainability()
    - preferenceListSurface()
    - notification system
    - inspectorDrawer()
- Carbon analogue:
    - side panel / contextual detail panel

Formatted documents and prose

- Family: markdown()
- Variants:
    - headings, emphasis, lists, blockquotes, links, GFM-style pipe tables, code fences, and other supported markdown structures with mode-aware lowering
- Use when:

    - help, reference, release notes, readme-like content, or bounded prose needs lightweight structure
    - the same content should remain honest across rich, pipe, and accessible output modes
- Avoid when:
    - the content needs deep document navigation rather than one rendered block
    - the app is really composing interface layout, forms, or command surfaces rather than prose
    - browser-grade markdown fidelity or arbitrary user-authored rich documents are expected
- Content guidance:
    - headings should break the content into scannable chunks instead of reproducing the entire document hierarchy from another medium
    - keep fenced code blocks and quotes short enough that the surrounding task still matters more than ornamental formatting
    - links should remain explicit and self-describing, since terminals vary in clickable-link support
    - use markdown for documents and reference prose, not as a substitute layout engine for application chrome
    - when markdown appears in overlays or panes, keep the scope intentionally bounded and move to pager or navigation patterns if the content becomes a document-reading task
- Ownership:
    - core
- Graceful lowering:
    - rich/static: preserve supported markdown structure and emphasis honestly within terminal constraints
    - pipe: lower to plain text with heading/list/code semantics still understandable without styling
    - accessible: linearize headings, lists, links, and code cues explicitly in reading order
- Related families:
    - note()
    - box()
    - pager()
    - helpView()
- Carbon analogue:
    - structured text / markdown content block

Linked destinations

- Family: hyperlink()
- Variants:

    - OSC 8 clickable link
    - explicit fallback text or URL modes
- Use when:
    - the destination matters and should remain part of the rendered output
    - terminals with hyperlink support should get a direct clickable affordance without hiding the actual destination semantics
- Avoid when:
    - the destination is untrusted, ambiguous, or should not be activated casually
    - link text is the only place where critical meaning or safety context appears
    - the user needs an app-owned action rather than an external destination
- Content guidance:
    - link labels should describe the destination or resource, not generic phrases like “click here”
    - when trust matters, keep the host or destination obvious in the surrounding text or fallback behavior
    - use fallback modes intentionally so non-supporting terminals still preserve the destination honestly
- Ownership:
    - core
- Graceful lowering:
    - rich/static: use OSC 8 when supported while keeping meaningful visible link text
    - pipe: lower to explicit text plus destination URL when needed
    - accessible: preserve label and destination clearly in reading order without assuming clickability
- Related families:
    - markdown()
    - note()
    - helpView()
- Carbon analogue:
    - link

Dividers

- Family: separator()
- Variants:
    - labeled, unlabeled
    - render-path companion: separatorSurface()
- Use when:
    - a section boundary is needed without full containment

- Avoid when:
  - the content needs its own grouped region
- Content guidance:
  - labels should name the next section or state, not repeat a page title
  - use dividers sparingly so they mark real boundaries rather than visual noise
- Ownership:
  - core
- Graceful lowering:
  - rich/static: render as visual divider or labeled rule
  - pipe: lower to simple text divider or heading-like label
  - accessible: preserve section naming without decorative characters being the only cue
- Related families:
  - box()
  - tabs()
  - breadcrumb()
- Carbon analogue:
  - divider

Choice and form families

Text entry

- Family:
  - input()
  - textarea()
- Variants:
  - short-form versus long-form entry
- Use when:
  - the user is providing text rather than choosing from a fixed set
- Avoid when:
  - choices are already known and enumerable
- Content guidance:
  - prompts should describe the expected input shape and any critical constraints
  - placeholders and defaults should clarify intent, not substitute for the label
- Ownership:
  - core

- Graceful lowering:
  - rich: interactive field/editor treatment with validation feedback
  - static: render current value or prompt snapshot honestly
  - pipe: fall back to line-buffered prompt/input flow
  - accessible: keep prompt, validation, and submitted value explicit in text
- Related families:
  - select()
  - filter()
  - group()
  - wizard()
- Carbon analogue:
  - text input / text area

Single choice

- Family:
  - select()
  - filter()
- Variants:
  - regular choice
  - searchable choice
- Use when:
  - the result is stored value state
- Avoid when:
  - selecting should trigger an app command rather than persist a value
  - the option set is large enough that search, ranking, or narrowing is the real task
- Content guidance:
  - option labels should be distinct, parallel, and easy to scan
  - searchable choice should use concise matching text, not long descriptive paragraphs
  - if search is the real job, prefer filter() semantics over stuffing too many options into plain select()
- Ownership:
  - core
- Graceful lowering:
  - rich: keyboard choice list with focus and optional search narrowing
  - static: show current selection or available options snapshot
  - pipe: lower to numbered or searchable textual selection flow
  - accessible: keep option labels, current selection, and prompt semantics explicit

- Related families:
  - multiselect()
  - commandPalette()
  - input()
- Carbon analogue:
  - dropdown / combo box

Multiple choice

- Family: multiselect()
- Variants:
  - default selections, checkbox set behavior
- Use when:
  - the user is building a set
- Avoid when:
  - the choice is singular or immediate action-oriented
  - the user is really firing commands rather than collecting lasting state
- Content guidance:
  - options should read like members of one set, not unrelated commands
  - summary text should make selection state obvious without requiring visual checkboxes
- Ownership:
  - core
- Graceful lowering:
  - rich: checkbox-style set selection with keyboard toggling
  - static: show selected values snapshot honestly
  - pipe: lower to comma-separated or numbered textual selection flow
  - accessible: describe the current set in plain text rather than relying on checkmarks alone
- Related families:
  - select()
  - filter()
  - group()
- Carbon analogue:
  - multi-select / checkbox group

Binary decision

- Family: confirm()
- Variants:
  - default yes/no orientation

- Use when:
  - the decision is genuinely binary
- Avoid when:
  - nuance, comparison, or explanation is needed
  - the prompt is really a multi-option choice disguised as yes/no
- Content guidance:
  - questions should be explicit about the consequence of yes versus no
  - destructive confirmations should name the thing being changed or removed
- Ownership:
  - core
- Graceful lowering:
  - rich: yes/no confirmation prompt or focused confirm surface
  - static: show the confirmation question and current default honestly
  - pipe: lower naturally to textual yes/no input
  - accessible: preserve the binary choice and default state explicitly
- Related families:
  - modal()
  - alert()
  - wizard()
- Carbon analogue:
  - confirmation dialog pattern

Multi-field and staged forms

- Family:
  - group()
  - wizard()
- Variants:
  - single-step grouped form
  - staged or branching flow
- Use when:
  - the user is progressing through related inputs
- Avoid when:
  - the fields are unrelated or the result is a command
- Content guidance:
  - each step or group should have a clear goal and avoid mixing unrelated concepts
  - summaries and progress text should orient the user without duplicating every field label
- Ownership:

    - core
- Graceful lowering:
    - rich: grouped or staged prompts with progress and validation
    - static: show the current step/group snapshot honestly
    - pipe: lower to sequential prompts while preserving branching meaning
    - accessible: keep step names, requirements, and progress explicit in text
- Related families:
    - input()
    - select()
    - confirm()
    - stepper()
- Carbon analogue:
    - form group / multi-step flow

Content organization families

Peer navigation

- Family: tabs()
- Variants:
    - tab labels, badges, active state
- Use when:
    - views are peers and only one is active
- Avoid when:
    - order or hierarchy is the main idea
- Content guidance:
    - tab labels should be short destination names, not action phrases
    - unread counts or compact status belong in secondary adornments, not the core label
- Ownership:
    - core
- Graceful lowering:
    - rich/static: keep one active peer view visibly selected
    - pipe: lower to labeled current section plus sibling list if needed
    - accessible: preserve peer destinations and active state explicitly in text
- Related families:
    - stepper()
    - breadcrumb()
    - createFramedApp()

- Carbon analogue:
  - tabs

Progressive disclosure

- Family:
  - accordion()
  - interactiveAccordion()
- Variants:
  - static disclosure
  - keyboard-owned TUI inspection
- Use when:
  - detail is secondary to summary
- Avoid when:
  - sections are peers that deserve first-class navigation
- Content guidance:
  - section headers should summarize the concealed content well enough to support scanning
  - disclosed content should stay tightly related to its summary row
- Ownership:
  - core plus TUI interaction layer
- Graceful lowering:
  - rich: collapsible sections with optional keyboard-owned inspection
  - static: show expanded/collapsed state honestly without requiring motion
  - pipe: lower to section headings with visible disclosed content
  - accessible: keep section labels and disclosure state explicit in text
- Related families:
  - tabs()
  - box()
  - interactiveAccordion()
- Carbon analogue:
  - accordion

Path and progress

- Family:
  - breadcrumb()
  - stepper()
  - paginator()
- Variants:
  - location, process progress, page count

- Use when:
  - the UI needs wayfinding or progress communication
- Avoid when:
  - tabs or lists express the job more clearly
- Content guidance:
  - path and progress labels should emphasize where the user is, not every possible destination
  - step titles should be stable and action-oriented enough to support review and recovery
  - use breadcrumb() when path context helps orient the current location, not as a decorative header flourish
  - use paginator() when compact position-in-sequence feedback is enough and the user does not need a richer navigation control
  - use stepper() when the current stage and remaining path matter more than peer switching
- Ownership:
  - core
- Graceful lowering:
  - rich/static: retain location, step, or page state with simple styling
  - pipe: lower to plain path/progress/page summaries
  - accessible: preserve order and active/current state explicitly
- Related families:
  - tabs()
  - wizard()
  - statusBar()
- Carbon analogue:
  - breadcrumb / progress indicator / pagination

Motion and shader effects

- Family:
  - canvas()
  - transition shaders
  - animate()
  - timeline()
- Variants:
  - decorative visual field
  - transition reinforcement
  - motion choreography
  - custom shader override
- Use when:

    - motion or shader output clarifies state change, transition, or atmosphere
- Avoid when:
    - the effect is only decorative noise
    - readability, scanning, or task focus would be harmed
    - the same meaning would be clearer as stable content or a simpler status cue
- Content guidance:
    - canvas() should be reserved for deliberate visual moments such as splash surfaces, atmospheric backgrounds, or specialized visual regions, not routine productivity chrome
    - transition shaders should reinforce page or workspace change, not compete with the content being changed
    - animation timing should support comprehension and must have an honest reduced-motion or non-interactive fallback
    - shader overrides should preserve semantic content where possible and avoid turning ordinary task flows into spectacle
- Ownership:
    - TUI
- Graceful lowering:
    - rich: preserve the motion or visual effect when it materially helps
    - static: lower to truthful final-state snapshots without pretending the transition still exists
    - pipe: drop decorative effects and keep only the meaning-bearing content
    - accessible: preserve state change meaning explicitly without requiring visual motion
- Related families:
    - createFramedApp()
    - statusBar()
    - notification system
    - timeline()
- Carbon analogue:
    - no exact analogue; closest to expressive motion and illustration guidance rather than one component family

Expressive branding and decorative emphasis

- Family:
    - loadRandomLogo()
    - gradientText()
- Variants:
    - ASCII logo sizes
    - single-line or multiline gradient emphasis

- Use when:
  - the interface needs a deliberate branded or celebratory moment
  - splash surfaces, docs, demos, and launch moments benefit from expressive treatment
- Avoid when:
  - routine application chrome or task-critical labels need to stay plain and scannable
  - color, decoration, or branding would compete with the actual work
  - the same text must remain equally legible in constrained or no-color environments
- Content guidance:
  - treat logos and gradients as rare emphasis, not the default voice of the interface
  - never make critical state, instructions, or navigation depend on gradient color alone
  - use expressive branding to open, celebrate, or orient, then get out of the user's way
- Ownership:
  - core
- Graceful lowering:
  - rich/static: preserve expressive branding where it adds real atmosphere without obscuring content
  - pipe: lower to plain ASCII/text without pretending the gradient or brand treatment still carries meaning
  - accessible: keep the textual content explicit and avoid decorative output that adds noise to the reading order
- Related families:
  - canvas()
  - box()
  - markdown()
- Carbon analogue:
  - closest to illustration, expressive color, and brand guidance rather than one component family

Authoring families

Mode-aware custom primitives

- Family:
  - renderByMode()
- Variants:
  - interactive/rich branch

    - pipe branch
    - accessible branch
    - app-specific authored primitive
- Use when:
    - the app needs a domain-specific primitive that does not belong in the shared component library
    - the same semantic thing must render honestly across multiple output modes
- Avoid when:
    - an existing Bijou family already matches the job
    - the branching only exists to chase cosmetics rather than preserve semantic truth
- Content guidance:
    - author one semantic primitive first, then define how it lowers by mode
    - keep all branches truthful to the same underlying meaning instead of inventing different features per mode
    - prefer app-owned names and domain language over generic helper wrappers
- Ownership:
    - core authoring helper for app-defined primitives
- Graceful lowering:
    - rich/static: use the richest honest rendering for the environment
    - pipe: lower to plain, explicit text without hidden styling assumptions
    - accessible: preserve the same semantic meaning in a reading-order-friendly form
- Related families:
    - note()
    - badge()
    - markdown()
- Carbon analogue:
    - closest to an internal component authoring pattern rather than a shipped Carbon family

Data and browsing families

Dense comparison

- Family:
    - table()
    - tableSurface()
    - navigableTable()
    - navigableTableSurface()
- Variants:

    - table() for passive comparison in core string output
    - table({ variant: "box" }) for the default Unicode boxed table
    - table({ variant: "ruled" }) for borderless report output with a heavy header rule and light row separators
    - table({ variant: "header-rule" }) for compact reports with only a header separator
    - table({ variant: "plain" }) for quiet space-aligned columns
    - table({ variant: "ascii-grid" }) for portable +---+ output
    - table({ variant: "markdown" }) for GitHub-flavored Markdown table text
    - table({ variant: "definition" }) for field/value reports
    - table({ variant: "expanded" }) for psql-style record inspection
    - tableSurface() for passive comparison in V3 surface-first output
    - navigableTable() for keyboard-owned inspection in the TUI layer
    - navigableTableSurface() for keyboard-owned inspection on the structured surface path
- Use when:
    - row/column comparison is the main task
- Avoid when:
    - hierarchy or dependency structure dominates
    - items are primarily one-dimensional and should read as a list instead
- Content guidance:
    - column headers should describe comparable attributes, not narrative explanations
    - cell text should stay compact enough that wrapping does not obscure the comparison task
    - table() fits human output to the available terminal width by default; use layout: "intrinsic" when content-sized output is explicitly needed
    - keep pipe/data serialization explicit with pipeFormat; TSV remains the default pipe lowering, while CSV, Markdown, and ASCII grid are opt-in
    - use navigableTableSurface() when the table lives inside a rich TUI surface and the row-aware inspection model should remain structured
    - unlike list/picker/palette families, navigable tables should keep row-aware scrolling semantics rather than generic line clipping whenever wrapped rows materially affect comparison
    - if rows wrap so heavily that column comparison is no longer readable, the table should usually collapse to a summary row plus a focused drill-down region instead of pretending dense comparison still works
    - choose navigableTable() / navigableTableSurface() only when keyboard traversal is actually part of the job; otherwise keep the table passive
- Ownership:
    - core plus TUI interaction layer
- Graceful lowering:
    - rich: passive table or focused navigable inspection depending on variant
    - static: retain tabular comparison when width allows, otherwise favor honest wrapped rows

    - pipe: lower to textual row/column output without hidden clipping
    - accessible: preserve headers, row labels, and comparison semantics explicitly
- Related families:
    - browsableList()
    - tree()
    - navigableTable()
- Carbon analogue:
    - data table

Hierarchy

- Family:
    - tree()
    - filePicker()
    - filePickerSurface()
- Variants:
    - static hierarchy
    - interactive filesystem browser
- Use when:
    - parent/child nesting is the mental model
- Avoid when:
    - multiple parents or graph dependencies matter
    - the interaction is really filesystem traversal, where filePicker() / filePickerSurface() are more honest
    - dense comparison matters more than nesting
- Content guidance:
    - labels should make nesting meaningful even when indentation is flattened or linearized
    - path context should be explicit when many siblings have similar names
    - use filePickerSurface() when the picker lives inside a rich TUI pane and should inherit shared viewport masking semantics; keep filePicker() as the explicit text-lowering path
    - directories should remain visually distinct from files, and moving to a parent or child path should be explicit rather than hidden behind overloaded actions
    - tree() should stay focused on passive hierarchy display; if arbitrary hierarchy needs keyboard-owned non-filesystem inspection later, that should become an explicit interactive tree family instead of overloading tree() itself
- Ownership:
    - core plus TUI interaction layer
- Graceful lowering:

    - rich: render hierarchy with optional interactive file/navigation layer
    - static: keep visible nesting and path structure honestly
    - pipe: lower naturally to textual indented hierarchy/path lists
    - accessible: preserve parent/child relationships in linear text
- Related families:
    - browsableList()
    - dag()
    - filePicker()
- Carbon analogue:
    - tree view / file browser pattern

Lists for exploration

- Family:
    - enumeratedList()
    - browsableList()
    - browsableListSurface()
- Variants:
    - passive list
    - interactive browsable list
- Use when:
    - one-dimensional items are being scanned or explored
- Avoid when:
    - columns or hierarchy carry the meaning
- Content guidance:
    - list rows should begin with the most distinguishing label or identifier
    - descriptions should help the scan task, not turn each row into a paragraph
    - use browsableListSurface() when the list belongs inside a rich TUI region and should share viewport masking semantics with pagers and focus panes; keep browsableList() as the explicit text-lowering path
    - if the next user action is “run a command” rather than “inspect a record,” move up to commandPalette() instead of making the list pretend to be an action launcher
- Ownership:
    - core plus TUI interaction layer
- Graceful lowering:
    - rich: passive or keyboard-browsable list with optional descriptions
    - static: keep readable ordered list snapshot
    - pipe: lower naturally to plain item list text
    - accessible: preserve item ordering, active selection, and descriptions explicitly

- Related families:
  - table()
  - tree()
  - commandPalette()
- Carbon analogue:
  - ordered/unordered list / selectable list

Temporal or dependency views

- Family:
  - timeline()
  - dag()
  - dagSlice()
  - dagStats()
  - dagPane()
- Variants:
  - chronological
  - dependency graph
  - focused fragment
  - metrics
  - interactive graph inspection
- Use when:
  - time or dependency is the actual structure
  - readers need to answer “what happened next?” or “what depends on this?”
- Avoid when:
  - a plain table or tree is enough
  - the graph is mostly decorative architecture wallpaper
  - a summary metric or local fragment would answer the question more honestly than the whole structure
- Content guidance:
  - events and nodes should expose the causal or temporal relationship, not just labels
  - annotations should stay lightweight enough that the structure remains legible
  - use timeline() when chronology is the actual reading path, not as a decorative status list
  - if timeline rows become so dense that the user is comparing attributes instead of following sequence, collapse to summary milestones plus drill-down or move to a table/log view
  - use dag() / dagPane() when causality, dependency, or multi-parent structure matters more than order
  - use dagPane() when graph inspection is an active task and keyboard-owned navigation, focus, and scrolling are part of the job; keep plain dag() for passive explanation

    - use dagSlice() when the honest scope is a neighborhood, ancestor chain, or descendant chain and the full graph would exceed the reader’s cognitive budget
    - use dagStats() when graph health, breadth, depth, or duplication risk matters more than visual shape
    - if most nodes have only one parent and the reader still consumes them in sequence, timeline() or tree() is usually clearer than a DAG
    - audit-trail timelines should preserve event ordering and consequence clearly, but avoid stuffing each row with enough metadata that the temporal story disappears
- Ownership:
    - core plus TUI interaction layer
- Graceful lowering:
    - rich: preserve chronology or graph shape with optional focused inspection
    - static: keep time/dependency order honestly even if the spatial rendering simplifies
    - pipe: lower to ordered event lists or textual dependency traces
    - accessible: linearize the structure while preserving temporal/dependency meaning explicitly
- Related families:
    - table()
    - tree()
    - log()
    - dagPane()
- Carbon analogue:
    - timeline has a close analogue; DAG does not and should be treated as a specialized Bijou family

Viewport masking and scrollable inspection panes

- Family:
    - viewport()
    - viewportSurface()
    - pager()
    - pagerSurface()
    - focusArea()
    - focusAreaSurface()
- Variants:
    - pure scroll mask
    - linear pager
    - focused pane with gutter
    - text-lowering path
    - surface-native path

- Use when:
  - a bounded pane is for reading, reviewing, or scrolling through content rather than navigating a richer table, tree, or graph structure
- Avoid when:
  - the content needs row/column comparison, hierarchy, or domain-specific navigation semantics
- Ownership:
  - TUI
- Content guidance:
  - use viewportSurface() as the base masking primitive when any existing Su rface or layout-backed region simply needs bounded overflow scrolling
  - keep viewport() as the explicit text-lowering path for string-first output, not the default teaching path for rich TUI composition
  - use pager() / pagerSurface() for long linear text where the status line and current line position matter
  - use focusArea() / focusAreaSurface() when the pane participates in a larger workspace and needs explicit focus ownership
  - prefer the surface-native path when the pane body is already composed from Surface content; keep the string path as explicit lowering, not the default teaching path
  - gutter chrome should communicate focus and workspace ownership, not carry primary content meaning
  - browsableListSurface(), filePickerSurface(), and commandPaletteSurface() now converge on viewport semantics; navigableTable() remains the row-aware holdout because wrapped-row comparison still needs a more specialized scroll model
  - viewport masking is the wrong abstraction when the user thinks in semantic rows rather than rendered lines; in that case keep row-aware scroll ownership local to the domain-specific component
- Graceful lowering:
  - rich: scrollable pane with honest status/focus chrome and bounded viewport behavior
  - static: visible excerpt or current window with enough status context to explain where the user is
  - pipe: lower to sequential text without pretending the hidden region is still interactively present
  - accessible: linearize the pane content while preserving scroll position or focus context explicitly
- Related families:
  - viewport()
  - viewportSurface()
  - dagPane()
  - createFramedApp()
  - browsableList()
- Carbon analogue:

    - closest to a scrollable content region or code viewer; not a direct Carbon family

Overlay and shell families

Overlay primitives

- Family:
    - tooltip()
    - drawer()
    - modal()
    - toast()
- Variants:
    - explanatory
    - supplemental
    - blocking
    - transient
- Use when:
    - the app needs layered interaction surfaces
- Avoid when:
    - the content should just live in the document or shell
- Ownership:
    - TUI
- Content guidance:
    - tooltip() content should be short, local, and explanatory, not actionable or scrollable
    - drawer() content should support supplemental work, inspection, or side-by-side context without stealing the whole task
    - modal() content should justify blocking the user and should end in a clear decision, confirmation, or review step
    - toast() content should be short-lived and self-contained, not a surrogate notification center
    - if the user may need recall, stacking, or routing, stop composing ad hoc toast() overlays and move up to the notification system
    - if the user still needs the main task visible and interactive, prefer dr awer() over modal()
    - when rich TUI structure matters, overlays should accept or compose structured Surface content instead of flattening component content into strings first
- Graceful lowering:
    - rich: layered overlay surfaces with blocking or non-blocking behavior depending on variant

    - static: lower to visible in-flow or shell-level snapshots rather than hidden z-order
    - pipe: lower to plain text event or prompt surfaces appropriate to interruption level
    - accessible: linearize overlay content with explicit relationship to the blocked or supplemented context
- Related families:
    - notification system
    - alert()
    - createFramedApp()
- Carbon analogue:
    - tooltip / side panel / modal / toast notification

Notification system

- Family:
    - renderNotificationStack()
    - renderNotificationHistory()
    - pushNotification()
    - dismissNotification()
    - tickNotifications()
    - framed runtime notification routing
- Variants:
    - ACTIONABLE
    - INLINE
    - TOAST
    - info, success, warning, error tone variants
    - per-notification placement and duration
    - history/archive view
    - frame-routed runtime notifications
- Use when:
    - the app owns transient messaging as a system instead of rendering one ad hoc overlay at a time
    - stacking, placement, routing, or history matter
    - warnings and errors should be reviewable after the moment they first appear
- Avoid when:
    - the message should remain in normal page flow as part of the document
    - a single local transient overlay is enough
    - the user must stop and decide before continuing
- Ownership:
    - TUI
- Content guidance:

    - use notifications for events and follow-up prompts, not durable documents or long-form explanation
    - actionable notifications should expose one obvious next step, not a miniature workflow or form
    - if users may need to revisit prior notices, the archive/history surface is part of the feature, not an optional add-on
    - route app-owned warnings and errors here when recall matters, while still keeping stderr/log output honest where appropriate
    - use placement and duration to control interruption level, not to smuggle unrelated importance into every message
- Graceful lowering:
    - rich: stacked notifications with actions, placement, history, and explicit dismissal
    - static: visible current notifications or a truthful archived summary without pretending transient timing still exists
    - pipe: lower to sequential event text or explicit warning/error records
    - accessible: linearize current and archived notices with tone, action, and dismissal state made explicit
- Related families:
    - toast()
    - alert()
    - modal()
    - drawer()
    - createFramedApp()
- Carbon analogue:
    - notification / toast notification center pattern

App shell

- Family:
    - createFramedApp()
    - statusBar()
    - statusBarSurface()
    - commandPalette()
    - commandPaletteSurface()
- Variants:
    - shell container, status rail, action discovery
- Use when:
    - the app has multiple views, overlays, navigation, and shell chrome
- Avoid when:
    - the app is really one screen or one prompt
- Ownership:
    - TUI

- Content guidance:
  - the shell should frame destinations and workspace state, not become a dumping ground for unrelated metadata
  - use statusBarSurface() when shell chrome already lives on the structured Surface path; keep statusBar() for explicit text output or lowering
  - use commandPaletteSurface() when the palette lives inside a rich TUI shell or overlay and should share viewport masking semantics with other panes; keep commandPalette() for explicit text output or lowering
  - status lines should carry concise global context, not replace in-page guidance
  - command palette entries should prefer actions, navigation targets, and recent destinations over field-style data entry
  - group palette items by user intent when possible, and avoid turning the palette into a browseable record list
  - tabs should represent peer destinations or work areas, not disguised command buttons
  - use notifications for events and follow-up, not the status rail
  - use help hints for shortcut discovery and shell scope, not for action execution or event messaging
- Graceful lowering:
  - rich: full shell chrome with tabs, palette, overlays, notifications, and workspace regions
  - static: retain the active page and essential shell context without pretending background interactivity exists
  - pipe: lower to current page content plus minimal status/context framing
  - accessible: linearize active shell state, navigation context, and current overlays into one readable flow
- Related families:
  - statusBar()
  - helpShortSurface()
  - tabs()
  - commandPalette()
  - notification system
- Carbon analogue:
  - shell, header/footer, command palette

Keybinding help and shell hints

- Family:
  - createKeyMap()
  - helpView()
  - helpViewSurface()
  - helpShort()
  - helpShortSurface()
  - helpFor()

    - helpForSurface()
- Variants:
    - grouped reference, single-line hint, filtered subset, explicit text output, structured-surface output
- Use when:
    - the app is keyboard-owned and the user needs discoverable shortcuts or a grouped command reference
- Avoid when:
    - the controls are already obvious from context and the help text would just restate visible labels
- Ownership:
    - TUI
- Content guidance:
    - use helpShortSurface() for shell hints that should stay on the structured Surface path; keep helpShort() for explicit text output or lowering
    - use helpViewSurface() when the full keybinding reference is embedded inside a rich TUI surface; keep helpView() for plain text and pipe-oriented help
    - group names should describe jobs (Navigation, Actions, Selection) rather than raw input mechanics
    - help text should clarify behavior and scope, not repeat page prose or become a substitute command palette
    - use the command palette for discoverable actions and navigation, and keep help focused on shortcut explanation
- Graceful lowering:
    - rich: concise shell hints plus an optional grouped help view or modal
    - static: retain the active hint and the relevant grouped reference without pretending background input is live
    - pipe: lower to plain text shortcut summaries and grouped help blocks
    - accessible: linearize help content into readable sections with explicit scope and action labels
- Related families:
    - createFramedApp()
    - statusBarSurface()
    - commandPalette()
- Carbon analogue:
    - keyboard shortcuts help, command reference, hotkey cheat sheet

Workspace layout

- Family:
    - splitPane()

    - splitPaneSurface()
    - grid()
    - gridSurface()
    - flex()
    - vstack()
    - hstack()
    - place()
- Variants:
    - user-resizable, named-area, compositional layout primitives with structured-surface and explicit text-lowering paths
- Use when:
    - spatial arrangement materially helps the task
- Avoid when:
    - a sequential flow would be simpler and more legible
- Ownership:
    - TUI
- Content guidance:
    - use splitPaneSurface() when the user benefits from explicit primary-versus-secondary context and the panes are already rich TUI surfaces; keep splitPane() for explicit text composition or lowering
    - use gridSurface() when multiple stable regions deserve simultaneous visibility and the regions are already rich TUI surfaces; keep grid() for explicit text composition or lowering
    - use flex() / vstackSurface() / hstackSurface() / placeSurface() to keep rich TUI composition structured; keep vstack() / hstack() / place() for explicit text composition or lowering
    - region titles and borders should explain job and hierarchy, not merely expose geometry
- Graceful lowering:
    - rich: keep spatial relationships and resizable or placed regions where they materially help
    - static: retain simplified spatial grouping when possible without fake interactivity
    - pipe: lower to sensible sequential content order
    - accessible: linearize regions in a predictable reading order with section labels
- Related families:
    - box()
    - createFramedApp()
    - focusArea()
- Carbon analogue:
    - layout grid / split layout / stack primitives

Data visualization

- Family:
  - sparkline()
  - brailleChartSurface()
  - statsPanelSurface()
  - perfOverlaySurface()
- Variants:
  - inline trend (sparkline string), high-density area chart (braille Surface), labeled metric panel (boxed key-value Surface), prebuilt performance dashboard (composed Surface)
- Use when:
  - the user needs to see numeric trends at a glance without leaving the terminal
  - a time-series or rolling metric is the primary decision input
  - you need a compact performance or status dashboard overlay
- Avoid when:
  - exact numeric precision matters more than shape — use table() instead
  - the data is categorical rather than continuous — use table() or tree()
  - the visualization would be decorative rather than informative
- Ownership:
  - Core
- Content guidance:
  - use sparkline() for inline trend hints next to a label; keep it short (8–20 chars)
  - use brailleChartSurface() when the data deserves area-chart density and sub-pixel smoothness
  - use statsPanelSurface() when multiple labeled metrics belong in one titled panel
  - use perfOverlaySurface() as the ready-made FPS + memory overlay; prefer it over assembling stats + chart manually
  - always supply min/max when comparing across views so axes stay stable
- Graceful lowering:
  - rich: Unicode block or Braille glyphs with semantic color tokens
  - static: same glyph rendering without animation
  - pipe: numeric summary (min/max/mean) or CSV-style output
  - accessible: spoken trend summary (e.g. “rising from 2 to 8 over 10 samples”)
- Related families:
  - table()
  - progressBar()
  - timeline()
- Carbon analogue:

- sparkline / area chart / metric card

Families that still need stronger doctrine

These are shipped, but the guidance is still thinner than it should be:

- separator() label discipline in dense shells and long documents
- tabs() overflow/truncation policy when peer destinations outgrow the available width
- wizard() / group() content architecture for long, interruptible setup flows

53

Data Visualization Policy

docs/design-system/data-visualization.md

Bijou already ships several data-display families:

- table()
- tree()
- timeline()
- dag()
- dagSlice()
- dagStats()
- interactive siblings such as navigableTable() and dagPane()

This document defines when to use each one and when not to visualize at all.

1. Start with the question, not the component

Before choosing a view, answer:

1. Is the user comparing records?
2. Is the user following hierarchy?
3. Is the user following chronology?
4. Is the user following dependencies or flow?
5. Is the user inspecting, or just reading?

If you do not know the question, the visualization choice will be decorative rather than informative.

2. Default selection ladder

Use table() when comparison is primary

Choose a table when:

- the same attributes matter across many rows
- the user needs side-by-side comparison
- scanability matters more than storytelling

Avoid table() when:

- hierarchy is primary
- chronology is primary
- the dependency structure itself is the meaning

Use tree() when parent/child hierarchy is primary

Choose a tree when:

- nesting is the message
- branches matter more than attributes
- the user needs to understand containment or lineage

Avoid tree() when:

- nodes have multiple parents
- the user is comparing many scalar attributes across peers

Use timeline() when chronology is primary

Choose a timeline when:

- sequence over time is the point
- events build a story
- state changes need a temporal narrative

Avoid timeline() when:

- side-by-side comparison matters more than order
- timestamps are metadata rather than the central axis

Use dag() when dependency or flow is primary

Choose a DAG when:

- nodes depend on one another
- parallel branches matter
- multiple parents or non-tree edges matter

Avoid dag() when:

- a plain tree or table already tells the truth
- the graph is so large that the shape becomes noise

3. Passive versus active inspection

Passive reading

Use:

- table()
- tree()
- timeline()
- dag()

when the user is reading or scanning.

Active inspection

Use:

- navigableTable()
- browsableList()
- dagPane()
- future interactive tree/file-browser patterns

when the user needs focus, selection, keyboard traversal, or drill-in behavior.

4. Density policy

Terminals tolerate dense information, but the wrong kind of density becomes illegible quickly.

Good density

- aligned columns
- consistent row shape
- compact but meaningful labels
- restrained use of borders and status color

Bad density

- too many competing tones
- wide wrapped columns inside already dense tables
- graphs with too many labels to read
- timelines with so much metadata that order disappears

5. Narrow terminal policy

Visualization must degrade honestly in narrow spaces.

Preferred behavior

- reflow to fewer columns

- wrap supporting text
- reduce metadata before reducing meaning
- collapse to summary plus drill-in path where appropriate

Avoid

- silently truncating the very field the user is trying to inspect
- forcing a graph into a width where it no longer communicates structure

6. When not to visualize

Do not force a data structure into a component just because one exists.

Avoid visualizing when:

- a short sentence says it more clearly
- a count or badge communicates enough
- the user only needs one record
- the structure is incidental rather than informative

Examples:

- one build status: use badge() or alert(), not table()
- one warning with context: use alert() or notification, not timeline()
- one path: use breadcrumb() or plain text, not tree()

7. Data visualization and graceful lowering

These families should preserve meaning across modes:

- table() should still compare
- tree() should still show nesting
- timeline() should still show order
- dag() should still show dependency structure, or degrade to a truthful summary if a full graph is not legible

If the only good version of a view is the rich interactive one, then the family needs:

- a TUI-only interaction layer
- a simplified lower-fidelity companion
- or a different canonical representation

8. Future chart policy

When Bijou adds chart families such as sparklines, bars, or scatter plots, the default rules should be:

- use charts for trend, shape, or distribution
- use tables for exact values
- pair chart and summary where the chart alone cannot carry the meaning
- do not make color the only way categories are distinguished
- narrow terminals should prefer simpler marks over denser ones

9. Practical recommendations for today

- Prefer table() for operational comparison.
- Prefer timeline() for histories and release/event storytelling.
- Prefer tree() for nested structure and navigation lineage.
- Prefer dag() for pipelines, dependencies, and workflow execution reasoning.
- Prefer interactive siblings only when the user must inspect, not just read.

54

Foundations

docs/design-system/foundations.md

This document defines the shared design language behind Bijou.

It is not an API document. It is the policy layer that should shape API design, examples, and component usage.

1. Color

Purpose

Color in Bijou should communicate:

- status
- hierarchy
- separation
- focus
- interruption level

Color should not carry meaning alone. Every important state must still survive noColor, pipe, and accessible runs.

Color roles

Bijou should treat tokens as belonging to a small set of roles:

- **foreground**
  - primary text, secondary text, subdued text
- **surface**
  - canvas, elevated region, overlay, inset region
- **status**
  - success, warning, error, info, neutral
- **accent**
  - active/focused selection, primary affordance
- **border/separator**

- grouping and structural division

The exact API-level mapping from these roles to Bijou's shipped token families is defined in Theme Token Vocabulary.

Color rules

- Use status color to explain state, not to decorate.
- Use accent color for focus and active affordances, not for arbitrary emphasis.
- Use background fills sparingly and deliberately. A background should indicate containment, elevation, or interruption.
- Avoid using more than one strong status tone in the same local region unless comparing states is the point.
- In noColor, every state must still read correctly from words, symbols, order, or borders.

Terminal-specific guidance

- Strong color contrast is expensive in a dense terminal. Use saturated status tones for small focal areas, not for entire screens.
- Large filled regions should be reserved for overlays, selected/focused states, or major grouping moves.
- Bright foreground on dark background is the default mental model in many terminals, but docs should not assume dark mode semantics.

2. Spacing and rhythm

Carbon uses a 2x grid. Bijou needs a terminal-first equivalent.

Bijou spacing model

- The atomic unit is **1 cell**.
- The default rhythm unit is **2 cells**.
- Dense internal padding can use 1-cell increments.
- Structural gaps between sections, panes, and overlay edges should prefer the 2-cell rhythm.

Default guidance

- inline gap: 1
- local group gap: 1 or 2
- section gap: 2
- pane or card padding: usually 1 vertically, 2 horizontally
- modal margin from viewport edge: minimum 1, preferred 2
- stacked transient gap: 1

Why this matters

Without a declared rhythm, terminal UIs become visually noisy very fast. Bijou should feel intentionally packed, not cramped.

3. Text hierarchy and emphasis

Terminal typography is mostly:

- placement
- spacing
- boldness
- token choice
- borders/dividers
- line length

not font families or font sizes.

Hierarchy levels

- **page/shell title**
  - strongest label in the local frame
- **section title**
  - container or region label
- **body**
  - default content
- **supporting text**
  - help text, metadata, timestamps, hints

Emphasis rules

- Prefer structure before decoration.
  - use spacing, grouping, and labeling first
- Bold should mark titles, active labels, or short key facts.
- Dim/subdued text should carry metadata, not primary instructions.
- Avoid combining too many modifiers at once.
  - bold + color + underline + inverse is usually design panic, not design clarity

Content density

- Tight density is acceptable in terminals, but meaning must still scan.
- Long prose should wrap and breathe.

- Dense data views should prefer alignment and scanability over decorative framing.

4. Overflow

Bijou should default to **wrap for prose-like content** and **truncate or clip only where layout semantics demand it**.

Default policy

- prose, supporting text, and descriptions: wrap
- labels inside constrained shell chrome: clip or truncate
- tabular cells: wrap when the row can grow, truncate when preserving scanability is more important
- activity streams and logs: do not silently destroy ordering to preserve prettiness

Why

Truncation is often a hidden data-loss bug disguised as layout discipline.

Required controls

Families that render text in constrained spaces should expose overflow choices where practical:

- wrap
- truncate
- clip

Not every component needs every mode, but the library should speak one consistent overflow language.

5. Motion

Motion should clarify:

- entry
- exit
- relationship between before and after
- focus shifts
- interruption level

Motion should not exist just because the runtime can animate.

Motion rules

- Transient surfaces may animate in and out.
- Page or pane transitions should reinforce spatial or logical movement.
- Fast repetitive interactions should avoid theatrical motion.
- Any animation must have a deterministic non-animated truth underneath it.

Practical policy

- notifications: short edge-based slide or fade motion
- shell/page transitions: modest, directional, and optional
- loading states: subtle looping motion
- structural resize: prefer stable layout over decorative flourish

6. Elevation and interruption

Terminals do not have shadows in the usual sense. Elevation must be conveyed through:

- border treatment
- fill treatment
- separation from background
- reserved space
- z-order and animation

Interruption ladder

From least interruptive to most interruptive:

1. inline help or note
2. badge or subtle status
3. alert
4. toast / notification
5. drawer
6. modal

This ladder should show up consistently in docs and examples.

7. Focus and active state

Bijou is keyboard-first.

The user should always be able to tell:

- what is focused
- what is active

- what is selected
- what is merely present

Focus cues

Use some combination of:

- accent color
- border or gutter treatment
- active label treatment
- position within a structured list

Rules

- Do not rely on color alone for focus.
- Focus cues should be local and unmistakable.
- Shell-level focus and component-level focus should not compete visually at the same strength.

8. Input modality policy

Keyboard first

Bijou should assume keyboard interaction is the baseline.

Every important interactive flow should work without a mouse.

Mouse as enhancement

Mouse input is valuable, but it should be:

- additive
- discoverable
- consistent
- never the only path

Pointer policy

- Click should usually mirror an existing keyboard action.
- Hover-only meaning should be minimal.
- Drag should be reserved for clearly spatial interactions, such as split dividers.
- Overlays should consume mouse events before background content.

9. Package ownership

Core @flyingrobots/bijou

Use core for semantics that survive graceful degradation:

- status and structural components
- forms and prompts
- prose and log output
- data views that still make sense as text

@flyingrobots/bijou-tui

Use TUI for:

- shells
- overlays
- scrolling
- focus-heavy inspection
- animation
- notification systems
- pointer-heavy or lifecycle-heavy behavior

10. What this means for future library work

New component work should start by answering:

1. Which pattern is this serving?
2. Is it semantic, interaction-layer, or render-path variation?
3. Does it belong in core or TUI?
4. What is the default overflow behavior?
5. How does it lower through non-interactive modes?
6. What is the Carbon-style “when to use / when not to use” story?

55

Validate And Adapt Packed Bijou Cell Receipts

docs/reference/packed-bijou-cells.md

Use parsePackedBijouCellsReceipt() to validate an unknown packed-bijou-cells/1 value. Use adaptPackedBijouCellsToSurface() when the validated bytes must become a Bijou terminal Surface.

The adapter preserves the receipt's dimensions, ten-byte cell records, and grapheme side-table order. It does not normalize malformed input, reconstruct cells, render a terminal frame, or validate the complete source scene.

Public API

The package exports:

Export	Contract
parsePackedBijouCellsReceipt()	Validate unknown input and return a fresh receipt
adaptPackedBijouCellsToSurface()	Validate unknown input and copy it into a packed Surface
PackedBijouCellsValidationError	Typed failure with code, path, and detail
PACKED_BIJOU_CELLS_VERSION	packed-bijou-cells/1
PACKED_BIJOU_CELL_FORMAT	bijou-packed-cell-u8x10-le/1
PACKED_BIJOU_CELL_STRIDE	10
PACKED_BIJOU_GLYPH_POLICY	unicode-grapheme-side-table/1
PACKED_BIJOU_SCENE_VERSION	ui-scene-ir/1
MAX_PACKED_BIJOU_CELLS	100,000
MAX_PACKED_BIJOU_DIMENSION	9,999 cells per axis
MAX_PACKED_BIJOU_GLYPH_CODE_UNITS	256 per glyph
MAX_PACKED_BIJOU_GLYPH_UTF8_BYTES	32 per glyph
MAX_PACKED_BIJOU_SIDE_TABLE_ENTRIES	4,096
PACKED_BIJOU_CELLS_VALIDATION_CODES	Ordered validation-code inventory
PackedBijouCellsReceipt	Complete receipt type
PackedBijouCellsScene	Scene identity and per-cell ownership
PackedBijouCellsFocus	Focused and focusable scene identities
PackedBijouCellsChroma	Exact accepted chroma profile
PackedBijouCellsValidationCode	Validation-code union

Import the functions and receipt type from @flyingrobots/bijou:

```
import {
  adaptPackedBijouCellsToSurface,
  parsePackedBijouCellsReceipt,
  type PackedBijouCellsReceipt,
} from '@flyingrobots/bijou';
```

Validate unknown input without constructing a Surface:

```
export function validateReceipt(
  input: unknown,
): PackedBijouCellsReceipt {
  return parsePackedBijouCellsReceipt(input);
}
```

Adapt unknown input after validation:

```
export function adaptReceipt(input: unknown) {
  return adaptPackedBijouCellsToSurface(input);
}
```

Both functions reject invalid input with PackedBijouCellsValidationError. Inspect its code, path, and detail fields instead of parsing the message.

Receipt Shape

Every field is required. Unknown or non-enumerable fields, accessors, symbols, sparse arrays, and non-JSON-shaped objects are rejected.

```
interface PackedBijouCellsReceipt {
  readonly receiptVersion: 'packed-bijou-cells/1';
  readonly widthCells: number;
  readonly heightCells: number;
  readonly cellFormatId: 'bijou-packed-cell-u8x10-le/1';
  readonly glyphPolicyId: 'unicode-grapheme-side-table/1';
  readonly bytes: readonly number[];
  readonly sideTable: readonly string[];
  readonly scene: {
    readonly sceneVersion: 'ui-scene-ir/1';
    readonly sceneHash: string;
    readonly nodeIds: readonly string[];
    readonly cellNodeIds: readonly string[];
  };
  readonly focus: {
    readonly focusedNodeId: string | null;
    readonly focusableNodeIds: readonly string[];
  };
  readonly chroma: {
    readonly colorSpace: 'srgb';
    readonly channelEncoding: 'uint8';
    readonly alphaEncoding: 'uint6';
    readonly terminalDefaultEncoding: 'presence-bits';
  };
}
```

The maximum accepted receipt contains 100,000 cells, with no axis longer than 9,999 cells. The byte payload must contain exactly ten unsigned bytes per cell. Negative zero is not a canonical unsigned byte and is rejected.

Cell And Glyph Rules

cellFormatId fixes each cell at ten bytes:

Offsets	Meaning
0..1	Unsigned 16-bit little-endian character value

Offsets	Meaning
2..4	Foreground red, green, and blue channels
5..7	Background red, green, and blue channels
8	Modifier and empty-cell flags
9	Six-bit opacity plus foreground/background presence bits

Character values below 0xF000 are direct Basic Multilingual Plane values. Direct values must be terminal-control safe and have terminal width 1; UTF-16 surrogate values and the NUL sentinel are rejected. An empty cell uses a space glyph plus the empty-cell flag. Values at or above 0xF000 select sideTable[characterValue - 0xF000].

Each side-table entry must be unique, terminal-control safe, and exactly one grapheme with terminal width 1. An entry contains at most 256 UTF-16 code units and encodes to at most 32 UTF-8 bytes. Format-control, mark-only, and default-ignorable-only graphemes are rejected. Every entry must be referenced by at least one cell. A directly encodable value below 0xF000 is rejected from the side table. Side-table order is authoritative.

When a foreground or background presence bit is unset, the corresponding three color channels must be zero. An absent color means terminal default, not black. A dashed flag is valid only with the dotted/dashed underline bits.

Scene, Focus, And Chroma Rules

scene.sceneHash is a lowercase SHA-256 hexadecimal digest. scene.nodeIds is a unique non-empty inventory, and scene.cellNodeIds contains one declared owner for every cell.

Every focusable node must occur in scene.nodeIds. focusedNodeId is either null or a member of focusableNodeIds.

Version 1 accepts only this chroma profile:

```
colorSpace: srgb
channelEncoding: uint8
alphaEncoding: uint6
terminalDefaultEncoding: presence-bits
```

No color-space or channel conversion occurs.

Diagnostics

The deterministic message format is:

```
packed-bijou-cells/1 <code> at <path>: <detail>
```

Paths escape control and default-ignorable characters as Unicode escape sequences so diagnostics remain visually stable.

Code	Meaning
invalid-shape	The value is not the required exact JSON shape
unknown-field	An object or array has an unowned field
wrong-literal	A version or policy identifier is unsupported
invalid-dimension	Dimensions are non-positive, unsafe, excessive, or overflow
invalid-byte	A payload value is not an integer from 0 through 255
byte-length-mismatch	The payload is not exactly ten bytes per cell
invalid-glyph	A direct glyph, side-table entry, or reference is invalid
invalid-color	Terminal-default color channels are nonzero
invalid-modifier	Modifier bits describe an invalid combination
invalid-scene	Scene identity or per-cell ownership is invalid
invalid-focus	Focus identity is duplicate, unknown, or inconsistent
invalid-chroma	The chroma profile is unsupported

Paths identify the rejected location, for example $.bytes[17] or $.focus.focusedNodeId.

Adapter Result

adaptPackedBijouCellsToSurface() validates the complete receipt before it calls createSurface(). On success:

```
surface.width === receipt.widthCells
surface.height === receipt.heightCells
Array.from(surface.buffer) deep-equals receipt.bytes
surface.sideTable deep-equals receipt.sideTable
```

All surface cells are marked dirty for their first render. Scene, focus, and chroma metadata remain receipt evidence; they are not stored inside Surface.

Fixture And Verification

The checked fixture exercises direct glyphs, side-table glyphs, explicit and terminal-default colors, scene ownership, and focus:

- Packed Bijou cell fixture
- RE-036 design record

Run the focused contract evidence:

```
npx vitest run tests/cycles/RE-036
```

56

Patterns

docs/design-system/patterns.md

This document defines the recurring product patterns that Bijou should support consistently.

These are the decision frameworks behind the component families.

0. Selection, focus, and rhythm

The pattern

Bijou should make it obvious:

- which item is currently selected
- which region currently owns input
- which copy is primary versus supporting
- how dense content is meant to breathe

These decisions should not be improvised per surface.

Canonical rules

Background fill means current selection

Use background fill to indicate the current selected row or item inside a collection.

Why:

- it reads immediately in dense TUIs
- it does not depend on one tiny pointer glyph
- it survives better than hue-only emphasis when terminals differ

Selection may also use supporting cues such as:

- a leading chevron
- bold value text
- a stronger border inside the selected row

But those are supporting cues. The canonical selected-item treatment is still background fill.

Structural accent means active region

Use a structural accent such as a gutter, border emphasis, or region frame to show the active region.

Why:

- region focus and row selection are not the same thing
- a pane can be active while its selected row changes
- a selected row can exist inside an inactive region without stealing shell ownership

The accent should describe ownership of input, not merely visual importance.

Visible controls are a promise

Follow Visible Controls Are a Promise.

If a region is not active, its controls should not be advertised as though they will work.

When the shell owns focus, shell controls replace pane-local cues.

One cell of padding matters

Dense TUIs still need rhythm.

Canonical defaults:

- one cell of padding between content and region edges
- one blank line between titled sections
- one blank line between preference rows or similar stacked interactive rows when the surface can afford it

Do not cram labels, values, and descriptions together when one cell of space would make the surface easier to parse.

Stack beneath the label before truncating

When labels and values compete for the same horizontal space:

- wrap or stack first
- reflow second
- truncate only when the surface truly demands single-line identity

For settings and similar rows, values should stack beneath the label before truncating into ambiguity.

Marquee is an explicit overflow behavior for compact, focused single-line surfaces. It is not the default solution.

Supporting copy should look supporting

Descriptions, helper copy, and secondary explanation should look visibly quieter than primary actions and current values.

Use:

- muted tokens
- dim modifiers when needed
- more breathing room around primary actions than around support prose

Do not make supporting copy compete with what the user is supposed to act on now.

Layers should behave like a stack

When interactive layers overlap, they should behave in top-down order.

Canonical rules:

- the topmost layer owns input
- the topmost layer owns the visible control hints
- dismiss actions target the topmost dismissible layer first
- underlying regions do not keep advertising controls while covered

This means a palette, drawer, or modal should not merely look on top. It should also be the source of truth for routing and footer/help copy.

1. Status and feedback

The pattern

Not all feedback should interrupt the user equally. Choose the least disruptive surface that still preserves the needed meaning.

Escalation ladder

badge()

Use when:

- the status is compact
- it belongs inline with another object
- the surrounding content already provides context

Avoid when:

- the user must act on the information
- the message needs explanation
- the status must persist as its own block

note()

Use when:

- the user needs supporting explanation
- the content is informational, not urgent
- the message belongs inside a form or guided flow

Avoid when:

- urgency or danger must be communicated
- the message should interrupt

alert()

Use when:

- the state should remain in the document flow
- the message is important enough to deserve its own block
- the user should see it while reading or editing nearby content

Avoid when:

- the event is brief and transient
- it belongs in an activity stream
- the app should manage stack/history/lifecycle

toast()

Use when:

- the event is transient
- the app is composing a one-off overlay directly
- custom placement matters, but full notification lifecycle does not

Avoid when:

- the message must remain in the page content
- the app needs stacking, history, routing, or recall
- the user needs to review a set of related events later

Content guidance:

- keep the message short and self-contained
- do not hide follow-up obligations in a toast body
- prefer one clear state statement over mixed status plus instructions plus history

notifications

Use when:

- the app owns transient messaging as a system
- stacking, placement, actions, routing, or history matter

- the user may need to reopen or review prior notices

Avoid when:

- the message should remain in page content
- a single local overlay would do
- the user must stop and decide before continuing

Content guidance:

- use notifications for events, not for durable documents
- route warnings and errors here when the app owns runtime messaging as a system
- actionable notifications should expose one obvious next step, not a mini workflow
- if users may need recall, the history/archive surface is part of the feature, not an optional afterthought

modal()

Use when:

- the user must stop and decide
- the app needs an explicit review or confirmation surface
- background actions should be blocked

Avoid when:

- a lighter inline or overlay treatment would do

Content guidance:

- the title should state the decision or review task clearly
- the body should explain the consequence, not repeat generic warning copy
- the modal should end in a clear next action or dismissal path
- avoid turning a modal into a drawer with extra blocking chrome

skeleton()

Use when:

- the content shape is known but a short-lived loading gap would otherwise cause layout pop
- the user benefits from seeing where content will land

Avoid when:

- honest partial content is already available
- the loading state is long-lived enough that explicit progress or retry messaging would be more honest
- the placeholder would imply certainty the app does not actually have yet

progressBar() / createAnimatedProgressBar()

Use when:

- percent-complete is actually known or can be estimated honestly
- the user benefits from determinate progress instead of generic activity

Avoid when:

- completion is unknown and the percentage would be fake
- the work is so brief that the bar would only flicker

Loading-screen rule:

- a loading screen should show at most one progress bar
- that bar must represent actual loading progress
- do not use a progress bar on a title or splash screen unless the screen is truly a loading surface

spinnerFrame() / createSpinner()

Use when:

- the task is active but indeterminate
- the user needs reassurance that work is still happening

Avoid when:

- completion can be expressed honestly as a determinate bar
- animation would distract more than it reassures

2. Selection versus action

This is one of the easiest places for a component library to become fuzzy.

select()

Use when:

- the user is choosing one value
- the set is short and stable
- the result becomes stored state

Avoid when:

- the list is large enough that search is the real task
- the outcome should trigger an immediate app command instead of persisting a value

multiselect()

Use when:

- the user is building a set
- multiple simultaneous selections are meaningful

Avoid when:

- the user should choose exactly one option
- the user is really firing commands one at a time instead of building a lasting set

filter()

Use when:

- the user is choosing one value from a larger set
- search/narrowing is the main task

Avoid when:

- the option set is already short and stable enough for select()
- the result is a command or navigation action rather than stored value state

commandPalette() / commandPaletteSurface()

Use when:

- the result is an action, navigation, or command
- the app wants one global action-discovery surface
- the user may benefit from mixed command groups such as actions, navigation, or recent destinations

Content guidance:

- palette entries should read like things the app can do, not field values the app stores
- group commands by user intent (Navigation, Actions, Recent) instead of raw subsystem names when possible
- keep descriptions short enough to scan without turning the palette into a browseable document

Avoid commandPalette() / commandPaletteSurface() when:

- the user is filling out a field value
- the result should behave like stored form data
- the content is really a browseable record list rather than an action list

3. Comparison versus browsing versus inspection

table()

Use when:

- the user needs comparison across columns
- the same attributes matter across many rows
- the table should fit the available terminal width and wrap long cells instead of forcing callers to pre-compute column widths

Avoid when:

- hierarchy is the main point
- the user needs focused keyboard inspection rather than passive comparison
- the item set is effectively one-dimensional and would read more honestly as a list

Variants:

- variant: "box" keeps the default Unicode boxed table
- variant: "ascii-grid" renders a portable +---+ grid
- variant: "ruled" renders a borderless table with a heavy header rule and light row separators
- variant: "header-rule" keeps only the header separator
- variant: "plain" renders aligned columns with spaces and no rules
- variant: "markdown" renders a GitHub-flavored Markdown pipe table
- variant: "definition" renders two-column field/value reports
- variant: "expanded" renders psql-style record inspection

Examples:

> box
> ┌──────┬────────┐
> │ Name │ State │
> ├──────┼────────┤
> │ Alice │ active │
> └──────┴────────┘
>
> ruled
> Name State
> ━━━━━ ━━━━━━
> Alice active
>
> header-rule
> Name State
> ----- ------
> Alice active
>
> plain
> Name State
> Alice active
>
> markdown
> | Name | State |
> |-------|--------|
> | Alice | active |
>
> definition
> Field Value
> ━━━━━ ━━━━━━
> State active
>
> expanded

```
-[ RECORD 1 ]------------
Name  | Alice
State | active
```

Lowering:

- pipe mode still defaults to TSV
- callers can opt into pipeFormat: "csv", "markdown", or "ascii-grid"
- visual aligned-column variants use spaces, not tabs; tabs are reserved for TSV serialization

tableSurface()

Use when:

- the user needs the same passive comparison semantics as table()
- the surrounding V3 runtime is already composing Surface output
- cells may contain surface-native labels or badges

Avoid when:

- direct string output is the final destination
- the app needs keyboard-owned row/cell focus semantics

navigableTable() / navigableTableSurface()

Use when:

- table data must be actively traversed
- the app wants row/cell focus semantics
- comparison across stable columns is still the job even while navigating

Avoid when:

- passive comparison is enough
- the output must remain portable through core-only string rendering
- the user is really selecting among one-dimensional records instead of comparing attributes

browsableList() / browsableListSurface()

Use when:

- items are one-dimensional
- descriptions matter
- the user is exploring or choosing among records

Avoid when:

- actions/commands are the result rather than records
- column comparison is the point
- hierarchy/path structure is the point

filePicker() / filePickerSurface()

Use when:

- the user is traversing filesystem hierarchy
- the current path and directory transitions are part of the job

Avoid when:

- the content is generic hierarchy rather than actual file/path navigation
- actions/commands are the result rather than a selected file or directory

3.5 Viewport masking and scroll ownership

Governing rules

- viewportSurface() is the canonical masking primitive for bounded overflow scrolling in rich TUI composition.
- viewport() is the explicit text-lowering path when the content is intentionally string-first.
- pagerSurface() and focusAreaSurface() are specialization layers on top of the viewport primitive, not separate scroll systems.
- If a component only needs a clipped window plus scroll state, it should wrap viewport semantics rather than invent a bespoke scrollY slicer.
- navigableTableSurface() is the important exception: wrapped comparison rows keep row-aware scroll semantics, so generic line clipping is the wrong abstraction there.
- Scroll ownership must stay unified. If keyboard focus, mouse hit-testing, and rendering disagree about the visible window height, the component has a layout bug.
- Scrollbars belong to the viewport layer. Components should not fake overflow by moving rows around without giving the user a truthful masked window.

Layout rules

- Components should either fill the rectangle their parent assigns them or be intentionally placed within it by a parent-owned layout helper such as placeSurface().
- Parents own outer whitespace and anchoring. Child components should not silently shrink and leave accidental slack against a pane edge.
- When a pane has a fixed header or separator row, the remaining rows should belong to one honest body region rather than an ad hoc mix of body plus invisible slack.
- Overflow handling must be explicit: wrap, stack, clip, or viewport. Scrolling content without a matching viewport contract is not an acceptable fallback.

viewportSurface()

Use when:

- an existing Surface or layout-backed region just needs a scrollable mask
- the app already owns the surrounding chrome and only needs honest overflow scrolling
- the component should stay on the structured Surface path

Avoid when:

- the content is intentionally string-first and the text-lowering path is the actual destination
- the component needs domain-specific focus chrome or status semantics that belong in a higher-level wrapper

3.6 Containment and formatted documents

box() / headerBox()

Use when:

- a region needs visible containment or a local title
- peer panels need to feel like separate working areas
- compact metadata belongs in a header rather than in the surrounding prose

Avoid when:

- every subsection would get a border just for visual decoration
- the real job is status escalation, not containment
- a divider, heading, or simple whitespace would communicate structure more honestly

markdown()

Use when:

- the content is help, reference, release notes, or bounded prose
- lightweight structure such as headings, lists, quotes, links, and code fences materially helps comprehension
- the same content should lower honestly across rich, pipe, and accessible modes

Avoid when:

- the content is so large that it really needs document navigation instead of one rendered block
- the prose is user-authored rich content that expects full browser-grade markdown semantics

- the app is really laying out UI regions or controls and is trying to use markdown as a layout engine
- dense comparison, forms, or command discovery are the real jobs

hyperlink()

Use when:

- the destination itself should remain part of the rendered output
- terminals with link support can benefit from clickability without hiding where the link goes

Avoid when:

- the label is vague or generic
- trust depends on seeing the actual destination, but the surrounding content hides it
- the interaction is really an app-owned action instead of an external destination

kbd()

Use when:

- a local action needs an inline shortcut cue
- the key hint belongs right next to the control, field, or instruction it affects

Avoid when:

- the user needs a grouped command reference or shell-level keymap
- shortcut chips would start competing with the real content for attention

pagerSurface() / focusAreaSurface()

Use when:

- the scrollable region needs explicit line status, current-position framing, or focus gutter semantics
- the pane is part of a larger workspace that needs visible ownership/focus signals

Avoid when:

- a plain masked region is enough and extra chrome would distract from the task

Choosing among browseable structures

- Choose browsableListSurface() when rows are essentially labels plus brief descriptions.

- Choose commandPaletteSurface() when the outcome is an action or navigation command.
- Choose filePickerSurface() when path traversal is the mental model.
- Choose navigableTableSurface() when comparison across columns remains the job even while navigating.
- Choose viewportSurface() directly when the content already exists as a region and just needs bounded overflow masking.

tree()

Use when:

- the primary relationship is parent/child
- the user needs to understand nesting or containment

Avoid when:

- multiple parents or dependency edges matter
- the user is really traversing filesystem paths, where filePickerSurface() is more honest
- dense row/column comparison would communicate the content more clearly than nesting

dag() / dagPane()

Use when:

- the primary relationship is dependency or flow
- multiple parents or graph edges matter
- the user needs to answer questions like “what does this depend on?” or “what does this unblock?”

Avoid when:

- a tree, timeline, or table would answer the question more directly
- the graph would mostly act as architecture wallpaper instead of supporting a concrete task
- the user really needs a metrics summary or focused fragment instead of the whole graph

timeline()

Use when:

- chronology is the main structure
- events should read as a sequence, milestone trail, or audit-oriented progression

Avoid when:

- dependency or causality matters more than ordering

- the content is so dense that a table, log, or summary-plus-detail drill-down would be more readable
- the user is really comparing attributes across many events rather than following a temporal story

3.7 Expressive branding and decorative emphasis

loadRandomLogo() / gradientText()

Use when:

- the product needs a deliberate branded or celebratory moment
- splash surfaces, docs, or demos benefit from a little atmosphere

Avoid when:

- ordinary workspace chrome, navigation, status, or instructions need to stay neutral and legible
- the emphasis would compete with the actual task
- the color treatment is being asked to carry meaning that should live in text

5. App-authored primitives

renderByMode()

Use when:

- the app needs a domain-specific primitive that the shared component library should not own
- the same semantic concept must lower honestly across rich, pipe, and accessible output

Avoid when:

- a shipped Bijou family already matches the job
- the branching only exists to chase visual novelty instead of preserving meaning

4. Disclosure and progressive complexity

accordion()

Use when:

- content is secondary or optional
- one page can hold both summary and detail

Avoid when:

- the sections are peer destinations that deserve explicit navigation

tabs()

Use when:

- views are peers
- only one view should be foregrounded at a time

Avoid when:

- the user is progressing through steps
- hierarchy/path context matters more than peer switching

stepper()

Use when:

- the user is moving through a process
- order matters
- the current stage should be understandable at a glance

Avoid when:

- the views are non-sequential
- the user is really switching among peer destinations instead of progressing through a flow

breadcrumb()

Use when:

- the user needs path context
- hierarchy or nesting should stay visible while reading the current location

Avoid when:

- peer destinations are the real model
- the path is so deep or unstable that a simpler label plus local navigation would be clearer

paginator()

Use when:

- the current page or position within a bounded sequence matters
- page count should be visible without turning into a full navigation menu

Avoid when:

- the user needs rich peer navigation rather than compact progress
- the content is an infinite feed or unbounded stream where page counts are misleading

5. Overlay escalation

Choose overlays by how much they should interrupt and how much context they should retain.

Governing rules

- Overlays are for interruption, review, or supplemental context. They are not substitutes for normal page structure.
- Pick the least interruptive overlay that still preserves the meaning and action path.
- If users need history, stacking, routing, or recall, move up from toast() to the notification system.
- If background input must stop, use modal() rather than trying to make a toast or drawer behave like one.
- If the content is long-lived and belongs in the reading flow, use alert() or a normal page region instead of any overlay.
- Use structured Surface content inside overlays when the interrupting layer needs real rows, embedded components, or richer layout. Flatten to plain text only at explicit lowering boundaries.

tooltip()

- for tiny local explanation attached to nearby content
- prefer one or two short lines that clarify a control, label, or state
- not for decisions
- not for content that needs scrolling, recall, commands, or workflow ownership

drawer()

- for supplemental context, inspectors, side work
- keep the main surface visible
- prefer when the user needs reference material while still working in the main task
- use panel-scoped drawers when the supplemental context belongs to one workspace region instead of the whole frame
- do not turn a drawer into a modal by stuffing it with blocking confirmation copy

modal()

- for blocking review, choice, or confirmation
- background shortcuts and pointer actions should be blocked
- use sparingly; interruption must be justified
- prefer short review-and-decide tasks over deep browsing or long-lived side work

toast()

- for one transient overlay the app is composing directly
- not for stack/history/routing problems
- if the same class of event happens repeatedly, stop composing ad hoc toasts and move up to notifications

notifications

- for events, not documents
- use history/archive when recall matters
- prefer over repeated ad hoc toasts once app-level messaging becomes systemic
- route app-owned warnings and errors here when users may need to revisit them after the initial interruption
- keep actions singular and obvious; do not hide multi-step workflows inside notification cards

6. Forms and progression

group()

Use when:

- the form fits one mental step
- fields are directly related

wizard()

Use when:

- the flow has stages
- branching or transformation matters
- the user benefits from progressive disclosure

confirm()

Use when:

- the decision is truly binary

Avoid when:

- the user needs to compare options or consequences in detail

7. App shell composition

Governing rules

- The shell exists to frame destinations, workspace state, and global actions. It should not compete with page content for narrative space.
- Use shell chrome for orientation, status, and cross-cutting actions. Put explanation and task detail back into the page.
- Prefer the simplest workspace that preserves the mental model. Do not use split panes or grids just because the runtime can.
- If a region is not meaningfully inspectable, comparable, or supplemental, it probably should not be its own pane.
- Keep shell roles distinct:
  - status rails communicate concise global state
  - help hints teach shortcuts and scope
  - command palette surfaces action discovery
  - notifications surface events and follow-up
- If one shell element starts doing another shell element's job, the design is drifting.

createFramedApp()

Use when:

- the app has multiple views or work areas
- shell chrome, palette, overlays, and notifications should be standardized
- the frame should own the default hosted runtime path via app.run(...) or ru nFramedApp(...)

Avoid when:

- the app is really just one screen or one prompt loop

Content guidance:

- tabs should represent peer destinations, not unrelated commands
- shell notifications and overlays should remain subordinate to the active page task
- help text should clarify shell behavior, not restate page content
- command palette entries should prefer actions and navigation targets, not field-style data entry or explanatory prose
- use drawers for supplemental workspace context and notifications for event messaging; do not overload the status rail with either

- prefer the self-running framed-shell path before dropping down to raw run(app)

statusBar() and statusBarSurface()

Use when:

- global state, mode, or shortcut context belongs at the shell edge

Avoid when:

- the content is explanatory or multi-line

Content guidance:

- keep status text short, global, and low-drama
- use statusBarSurface() when shell chrome is already being composed as a Surface
- keep statusBar() for explicit text output or lowering
- reserve strong emphasis for mode changes, faults, or focus-critical state
- do not turn the status bar into a secondary page body
- do not route transient warnings or task-specific explanation here when notifications or in-page content would be more honest

helpShort() / helpView() and surface companions

Use when:

- keyboard ownership is real and shortcut discovery needs a concise hint or grouped reference

Avoid when:

- the text would only restate visible controls without clarifying behavior or scope

Content guidance:

- use helpShortSurface() for shell hints that stay on the structured Surface path
- use helpViewSurface() for grouped shortcut references embedded inside a rich TUI surface or modal
- keep the plain string helpers for explicit lowering, pipe output, or textual docs/examples
- group by user intent (Navigation, Actions, Selection) rather than by key shape
- help is for shortcut discovery and scope clarification, not for executing commands or replacing a command palette

splitPane() and grid()

Use when:

- spatial composition helps the user reason about the workspace

Avoid when:

- the layout would be simpler and more legible as a sequential flow

Content guidance:

- use split panes for comparison, side-by-side reference, or inspector-style secondary context
- use grids when the user is reading across stable regions, not when one long column would be clearer
- label regions by job, not by visual position alone

8. Mouse and multimodal interaction

Mouse support in Bijou should follow these rules:

- keyboard is baseline
- mouse is enhancement
- every click target should mirror an existing keyboard path
- overlays consume pointer input before background content
- drag is reserved for spatial interactions such as pane resizing

Good early mouse targets:

- tabs
- split dividers
- notification actions and dismiss buttons
- list/table row focus

Bad early mouse targets:

- hover-only critical meaning
- interactions that have no keyboard parity

9. Lowering across modes

Every pattern should have a graceful lowering story.

Examples

10. Motion and shader effects

Motion and shader effects are supporting patterns, not the product itself.

Use them when:

- they clarify transition, progress, focus, or spatial change

- the user benefits from seeing continuity rather than a sudden visual jump
- the effect reinforces the information architecture instead of competing with it

Avoid them when:

- they are only decorative noise
- they make dense work harder to read
- the same information would be clearer as stable content, a status change, or a simpler transition

Specific guidance:

- use transition shaders to reinforce page or workspace change, not to show off that the renderer can animate
- use canvas() for purposeful visual surfaces such as splash, background atmosphere, or dedicated visual moments, not as a default wrapper around ordinary productivity screens
- reduced-motion and non-interactive modes must still tell the truth even when the effect disappears entirely
- notifications may lower to logs or in-flow alerts
- command palette may lower to documented shortcuts or static command lists
- navigable inspection views may lower to passive read-only views
- modals may lower to step-by-step prompts or inline confirmation blocks

The docs for each family should make that lowering explicit instead of assuming an interactive full-screen terminal.

57

Pointer and Mouse Policy

docs/design-system/pointer.md

Bijou is keyboard-first, not keyboard-only.

Mouse support should improve fluency without creating a second product that only works for pointer users.

What mouse is for

Use mouse in Bijou to:

- switch visible context quickly
- dismiss or activate transient overlays
- inspect and select already-visible objects
- adjust obviously spatial controls such as split dividers

Do not use mouse as the only path to:

- submit forms
- dismiss critical errors
- navigate core app flows
- reveal essential state through hover alone

Routing order

Pointer events should resolve from most interruptive layer to least:

1. active modal or command surface
2. transient overlays and notifications
3. shell chrome
4. page content

Background content should not keep receiving pointer actions while a true modal surface is open.

For shell and overlay families specifically:

- modal() should consume pointer input ahead of everything behind it

- notifications and ad hoc overlays should consume clicks before shell chrome or page content
- shell tabs, palette rows, and pane-scoped overlay controls should behave like pointer shortcuts for already-available keyboard actions, not as pointer-exclusive product paths

Click policy

Clicks should usually mirror an existing keyboard action:

- clicking a tab should do what tab-switching already does
- clicking a notification action should do what pressing enter on the focused notification already does
- clicking dismiss should do what the dismiss shortcut already does

If there is no keyboard-parity story, the pointer behavior is probably under-designed.

Hover policy

Hover is optional and should never be the only carrier of important meaning.

Terminals vary widely in pointer fidelity, update rate, and user expectation. Hover can be useful for polish, but it is a weak foundation for essential workflows.

Spatial policy

Use drag only for controls that are already spatial by nature:

- split-pane dividers
- canvas or graph inspection surfaces
- future resizable inspectors or drawers

Avoid inventing drag gestures for ordinary list, table, or form interactions.

Current library baseline

The current first-class mouse baseline in @flyingrobots/bijou-tui should be:

- shell tabs are clickable
- notification cards support click-to-dismiss and click-to-activate
- mouse requires explicit runtime opt-in with run(app, { mouse: true })
- modal-style overlays block background pointer interaction
- shell/pane pointer routing should remain subordinate to overlay routing when interruptive surfaces are open

Further component adoption should follow the same rule:

- add mouse where it accelerates an existing interaction
- keep keyboard parity
- route overlays before background content

58

Inspect Profunctor Page Artifacts

docs/reference/profunctor-page-inspection.md

Use lowerProfunctorPageArtifacts() to validate and inspect one canonical Profunctor Page artifact family through Bijou's terminal target. The function returns a deterministic ui-scene-ir/1 scene, Surface, target map, receipt, and text witness. It does not mutate the input artifacts.

This reference covers the bounded bijou-terminal-project-page/1 target. It is not a general Profunctor Page renderer.

Public API

Import the function, error type, and input or output types from @flyingrobots/bijou:

```
import {
  lowerProfunctorPageArtifacts,
  ProfunctorPageTargetError,
  type ProfunctorArtifactInputs,
  type ProfunctorPageInspectionMode,
  type ProfunctorPageTargetProof,
} from '@flyingrobots/bijou';
```

The call accepts three exact UTF-8 JSON sources:

```
const inputs: ProfunctorArtifactInputs = {
  page: {
    filename: 'page.profunctor.json',
    source: pageSource,
  },
  sourceMap: {
    filename: 'page.profunctor.map.json',
    source: sourceMapSource,
  },
  buildManifest: {
    filename: 'page.profunctor.build.json',
    source: buildManifestSource,
  },
};

const proof: ProfunctorPageTargetProof =
  lowerProfunctorPageArtifacts(inputs, { mode: 'source-refs' });
```

The example is illustrative. pageSource, sourceMapSource, and buildManifestSource must contain the complete canonical artifact bytes.

Inputs

Input	Required artifact version	Bounded target requirement
page	profunctor-page/0	ProjectPage composition using only the supported block definitions
sourceMap	profunctor-page-source-map/0	Structured source occurrences with repository-relative paths
buildManifest	profunctor-build-manifest/0	One route, no sources or obstructions, and a digest matching the page bytes

Each source must be canonical JSON with either no trailing newline or exactly one trailing newline. The parser rejects unknown fields, missing fields, duplicate identities, malformed identity prefixes, host-local paths, digest drift, and cross-artifact disagreement.

The page, source map, and build manifest must agree on the page identity. The manifest entity must agree with the page entity, its sole route must equal the page route, and its dependencies must equal the page dependency digests in the same order. Every source-map source and record digest, node provenance source digest, and build entity digest must occur in that dependency list. The page root must be a visible block:page whose route agrees with the page route. It must appear first in reading order, and every page node must be reachable from it through the declared slots.

Supported Blocks

The bounded target supports these visible block definitions:

- block:page
- block:project-hero
- block:project-facts
- block:project-documentation
- block:project-related

A visible unsupported block fails with BIJOU_PAGE_BLOCK_UNSUPPORTED. A hidden unsupported block becomes an explicit hidden-unsupported-block residual. Content that cannot fit the bounded 100-column by 28-row terminal profile also fails instead of being clipped.

Semantic-document source-map entries, build-manifest document sources, and application islands are outside this target. They fail explicitly instead of being approximated.

Inspection Modes

Omit mode to use normal.

Mode	Witness content
normal	Project headings, facts, documentation destinations, and related-project actions
node-ids	Page-node identities
source-refs	Structured source-occurrence or provenance references
token-refs	Design-token references
composition	Composition, template-node, and block-definition identities
obstructions	Required capabilities, target dispositions, and residual explanations

Modes change the terminal witness. They do not change the input identities or claim browser semantics. Unsupported modes and unowned option fields fail with BIJOU_PAGE_INPUT_REFERENCE_INVALID. Multi-value diagnostic facts use one bounded terminal row per value so the witness preserves every fact.

Outputs

ProfunctorPageTargetProof contains:

Field	Contract
scene	Bijou-owned ui-scene-ir/1 value
surface	Lowered terminal Surface
terminalProof	Existing Bijou terminal-lowering proof and cell source map
targetMap	bijou-profunctor-page-map/1 identities, regions, actions, tokens, source occurrences, reading order, outline, landmarks, and capability outcomes
receipt	bijou-profunctor-page-receipt/1 input/output digests and explicit upstream-claim posture
witness	Deterministic plain-text terminal witness
artifacts	Canonical scene, map, receipt, and witness file names with exact output bytes

The target preserves page, composition, override, template, content, source-occurrence, token, action, and target-render identities. Every visible page node receives one render identity. A hidden unsupported node receives one explicit residual.

Native links become Bijou action facts. Heading and landmark semantics become structural facts. Semantic HTML remains an explicit residual; the target does not inherit the build manifest’s browser claims.

Diagnostics

Failures throw ProfunctorPageTargetError. Inspect code, path, and detail instead of parsing the message.

Code	Meaning
BIJOU_PAGE_INPUT_JSON_INVALID	JSON is malformed or not canonical
BIJOU_PAGE_INPUT_VERSION_UNSUPPORTED	An input artifact version is not supported
BIJOU_PAGE_INPUT_DIGEST_MISMATCH	The manifest page digest does not match the exact page bytes
BIJOU_PAGE_INPUT_IDENTITY_MISMATCH	Related artifacts or owned source occurrences disagree
BIJOU_PAGE_INPUT_REFERENCE_INVALID	A required field, identity, graph edge, path, token, capability, action, or source shape is invalid
BIJOU_PAGE_BLOCK_UNSUPPORTED	The bounded target cannot honestly lower the requested block or source family
BIJOU_PAGE_OUTPUT_INVALID	An unexpected output construction or lowering invariant failed

Canonical Fixture And Verification

The checked Keep fixture pins the website-owned input bytes and the Bijou-owned outputs:

- Keep Profunctor Page conformance fixture
- DX-050 design record

Regenerate the output artifacts:

```
npm run generate:profunctor-page-target
```

Verify that the checked artifacts match the current lowerer:

```
npm run check:profunctor-page-target
```

Run the focused contract evidence:

```
npx vitest run \
  tests/cycles/DX-050/profunctor-page-target.part01.test.ts \
  tests/cycles/DX-050/profunctor-page-target.part02.test.ts \
  tests/cycles/DX-050/profunctor-page-target.part03.test.ts \
  tests/cycles/DX-050/profunctor-page-target.part04.test.ts \
  tests/cycles/DX-050/profunctor-page-target.part05.test.ts
```

59

Theme Authoring Guide

docs/design-system/theme-authoring.md

This guide explains how to write your own Bijou theme, how to choose the right token family, and how to load a custom theme into a real app.

Use this together with:

- Theme Token Vocabulary for what each token family means
- packages/bijou/src/core/theme/tokens.ts for the raw type contract
- packages/bijou/src/core/theme/presets.ts for concrete built-in examples

What a Bijou theme contains

Every Bijou theme has these top-level groups:

- name
- status
- semantic
- gradient
- border
- ui
- surface

The built-in required keys inside those groups are:

- status: success, error, warning, info, pending, active, muted
- semantic: success, error, warning, info, accent, muted, primary
- border: primary, secondary, success, warning, error, muted
- ui: cursor, focusGutter, scrollThumb, scrollTrack, sectionHeader, logo, tableHeader, trackEmpty
- surface: primary, secondary, elevated, overlay, muted
- gradient: brand, progress

If a built-in key is missing in a DTCG JSON theme, Bijou will still load the document, but that missing token resolves to a black/default placeholder. In practice, that means an incomplete theme is considered malformed even if it parses successfully.

Choose tokens by job, not by color

Before picking colors, decide what each token is responsible for:

- semantic.* is for foreground meaning and emphasis
- surface.* is for fills, containment, elevation, and interruption level
- border.* is for structure and boundary emphasis
- ui.* is for reusable shell/runtime chrome
- status.* is for actual state labels and indicators
- gradient.* is for rare decorative or continuous-range emphasis

If you need the rulebook for those meanings, read Theme Token Vocabulary first. This guide is about authoring and loading; that guide is about semantics.

Token value shape

A color token can be either:

- a simple foreground hex string, like "#88c0d0"
- or an object with:
  - hex
  - optional bg
  - optional modifiers

Supported modifiers are:

- bold
- dim
- strikethrough
- inverse
- underline
- curly-underline
- dotted-underline
- dashed-underline

Example token values:

```
"primary": "#eceff4"
```

```
"primary": {
  "hex": "#eceff4",
  "bg": "#2e3440",
  "modifiers": ["bold"]
}
```

The shortest working custom theme

If you want a file you can start from immediately, this is the smallest honest shape to copy and edit.

```
{
  "name": { "$value": "my-theme" },
  "status": {
    "success": { "$type": "color", "$value": "#22c55e" },
    "error": { "$type": "color", "$value": "#ef4444" },
    "warning": { "$type": "color", "$value": "#f59e0b" },
    "info": { "$type": "color", "$value": "#38bdf8" },
    "pending": { "$type": "color", "$value": { "hex": "#64748b", "modifiers": ["dim"] } },
    "active": { "$type": "color", "$value": "#38bdf8" },
    "muted": { "$type": "color", "$value": { "hex": "#64748b", "modifiers": ["dim",
    ↪  "strikethrough"] } }
  },
  "semantic": {
    "success": { "$type": "color", "$value": "#22c55e" },
    "error": { "$type": "color", "$value": "#ef4444" },
    "warning": { "$type": "color", "$value": "#f59e0b" },
    "info": { "$type": "color", "$value": "#38bdf8" },
    "accent": { "$type": "color", "$value": "#c084fc" },
    "muted": { "$type": "color", "$value": { "hex": "#94a3b8", "modifiers": ["dim"] } },
    "primary": { "$type": "color", "$value": { "hex": "#f8fafc", "modifiers": ["bold"] } }
  },
  "border": {
    "primary": { "$type": "color", "$value": "#38bdf8" },
    "secondary": { "$type": "color", "$value": "#c084fc" },
    "success": { "$type": "color", "$value": "#22c55e" },
    "warning": { "$type": "color", "$value": "#f59e0b" },
    "error": { "$type": "color", "$value": "#ef4444" },
    "muted": { "$type": "color", "$value": "#64748b" }
  },
  "ui": {
    "cursor": { "$type": "color", "$value": "#38bdf8" },
    "focusGutter": { "$type": "color", "$value": "#c084fc" },
    "scrollThumb": { "$type": "color", "$value": "#38bdf8" },
    "scrollTrack": { "$type": "color", "$value": "#334155" },
    "sectionHeader": { "$type": "color", "$value": { "hex": "#f8fafc", "modifiers": ["bold"]
    ↪  } },
    "logo": { "$type": "color", "$value": "#38bdf8" },
    "tableHeader": { "$type": "color", "$value": "#f8fafc" },
    "trackEmpty": { "$type": "color", "$value": "#1e293b" }
  },
  "surface": {
    "primary": { "$type": "color", "$value": { "hex": "#f8fafc", "bg": "#0f172a" } },
    "secondary": { "$type": "color", "$value": { "hex": "#e2e8f0", "bg": "#1e293b" } },
    "elevated": { "$type": "color", "$value": { "hex": "#f8fafc", "bg": "#334155" } },
    "overlay": { "$type": "color", "$value": { "hex": "#f8fafc", "bg": "#0f172a" } },
    "muted": { "$type": "color", "$value": { "hex": "#94a3b8", "bg": "#020617" } }
  },
  "gradient": {
    "brand": {
      "$type": "gradient",
      "$value": [
        { "pos": 0, "color": "#38bdf8" },
        { "pos": 1, "color": "#c084fc" }
      ]
    },
    "progress": {
      "$type": "gradient",
      "$value": [
        { "pos": 0, "color": "#22c55e" },
        { "pos": 1, "color": "#38bdf8" }
      ]
    }
  }
}
```

Save that as something like themes/my-theme.json.

Loading a custom theme

For Node-hosted apps, the fastest path is BIJOU_THEME:

```
BIJOU_THEME=./themes/my-theme.json npm run dogfood
```

Applying a theme programmatically

If you already have a Theme object in code, the easiest hosted path is:

```
import { startApp } from '@flyingrobots/bijou-node';

await startApp(app, { theme: MY_THEME });
```

That one call creates the Node host context with your theme and uses it for the run. If the app relies on ambient ctx resolution, that same themed context is also registered as the default for the hosted run.

If you want explicit ownership of the context object first, use:

```
import { createNodeContext } from '@flyingrobots/bijou-node';

const ctx = createNodeContext({ theme: MY_THEME });
await startApp(app, { ctx });
```

Automatic light/dark theme sets

If you want more than one theme, @flyingrobots/bijou-node now supports a named theme set at the host layer:

```
await startApp(app, {
  themes: [
    { id: 'light', theme: LIGHT_THEME },
    { id: 'dark', theme: DARK_THEME },
  ],
  themeMode: 'auto',
});
```

themeMode accepts:

- auto
- light
- dark

When themeMode is auto, the host uses Bijou's terminal color-scheme detection to choose the initial entry.

If your ids are not literally light and dark, add explicit scheme hints:

```
await startApp(app, {
  themes: [
    { id: 'sunrise', scheme: 'light', theme: LIGHT_THEME },
    { id: 'midnight', scheme: 'dark', theme: DARK_THEME },
  ],
  themeMode: 'auto',
});
```

User override policy

There are three practical override layers:

- themeOverride for an app-owned persisted user choice
- BIJOU_THEME for host/env-driven selection
- themeMode: 'auto' for light/dark automation when nothing more explicit is chosen

Example:

```
await startApp(app, {
  themes: [
    { id: 'light', theme: LIGHT_THEME },
    { id: 'dark', theme: DARK_THEME },
  ],
  themeMode: 'auto',
  themeOverride: savedThemeId,
});
```

Behavior:

- themeOverride wins when it matches one of the provided entries
- otherwise BIJOU_THEME can still select a named entry by id
- otherwise, if BIJOU_THEME is set but does not resolve to a known preset, JSON theme, or named theme entry, the explicit theme fallback wins when provided
- otherwise themeMode picks the initial theme
- otherwise the first entry becomes the fallback

That same pattern works for any hosted app that goes through @flyingrobots/bijou-node theme resolution.

If BIJOU_THEME points at a .json file, Bijou treats it as a DTCG document. If it points at a known preset name, Bijou loads the preset instead.

Examples:

```
BIJOU_THEME=cyan-magenta npm run dogfood
BIJOU_THEME=teal-orange-pink npm run dogfood
BIJOU_THEME=./themes/my-theme.json npm run dogfood
```

Authoring in TypeScript

If you own the app code and want to construct a theme in code instead of a JSON file, match the Theme type directly:

```
import type { Theme } from '@flyingrobots/bijou';

export const MY_THEME: Theme = {
  name: 'my-theme',
  status: {
    success: { hex: '#22c55e' },
    error: { hex: '#ef4444' },
    warning: { hex: '#f59e0b' },
    info: { hex: '#38bdf8' },
    pending: { hex: '#64748b', modifiers: ['dim'] },
    active: { hex: '#38bdf8' },
    muted: { hex: '#64748b', modifiers: ['dim', 'strikethrough'] },
  },
  semantic: {
    success: { hex: '#22c55e' },
    error: { hex: '#ef4444' },
    warning: { hex: '#f59e0b' },
```

```
    info: { hex: '#38bdf8' },
    accent: { hex: '#c084fc' },
    muted: { hex: '#94a3b8', modifiers: ['dim'] },
    primary: { hex: '#f8fafc', modifiers: ['bold'] },
  },
  border: {
    primary: { hex: '#38bdf8' },
    secondary: { hex: '#c084fc' },
    success: { hex: '#22c55e' },
    warning: { hex: '#f59e0b' },
    error: { hex: '#ef4444' },
    muted: { hex: '#64748b' },
  },
  ui: {
    cursor: { hex: '#38bdf8' },
    focusGutter: { hex: '#c084fc' },
    scrollThumb: { hex: '#38bdf8' },
    scrollTrack: { hex: '#334155' },
    sectionHeader: { hex: '#f8fafc', modifiers: ['bold'] },
    logo: { hex: '#38bdf8' },
    tableHeader: { hex: '#f8fafc' },
    trackEmpty: { hex: '#1e293b' },
  },
  surface: {
    primary: { hex: '#f8fafc', bg: '#0f172a' },
    secondary: { hex: '#e2e8f0', bg: '#1e293b' },
    elevated: { hex: '#f8fafc', bg: '#334155' },
    overlay: { hex: '#f8fafc', bg: '#0f172a' },
    muted: { hex: '#94a3b8', bg: '#020617' },
  },
  gradient: {
    brand: [
      { pos: 0, color: [56, 189, 248] },
      { pos: 1, color: [192, 132, 252] },
    ],
    progress: [
      { pos: 0, color: [34, 197, 94] },
      { pos: 1, color: [56, 189, 248] },
    ],
  },
};
```

What you may extend

The public generic Theme<S, U, G> contract allows app-level extension of:

- status
- ui
- gradient

That means app code can add stable extra keys when the domain really needs them.

Good examples:

- extra status values for a real workflow state machine
- extra ui tokens for recurring shell chrome
- extra gradient keys for a stable branded effect

Bad examples:

- adding one-off ad hoc token keys because a single component needs a special case
- using status.* for focus or selection

- using surface.* as a generic text-emphasis bucket

Practical authoring rules

- Start from a built-in preset and edit semantically, not from a blank page.
- Keep foreground/background contrast honest on every surface.* token.
- Use defineThemeSafePairs() plus doctorTheme() when an app or component needs a reusable contrast contract across multiple foreground/background roles.
- Test the theme in interactive, static, pipe, and accessible modes.
- Do not rely on color alone; labels and structure still need to make sense in noColor, pipe, and accessible lowering.
- If the theme adds recurring framework chrome, prefer extending ui instead of hard-coding raw colors in shell code.

Where to look next

- Theme Token Vocabulary
- packages/bijou/GUIDE.md
- packages/bijou/ADVANCED_GUIDE.md
- packages/bijou-node/GUIDE.md

60

Theme Rule Recipes

docs/design-system/theme-rule-recipes.md

Theme rules let Bijou themes describe how semantic colors are selected instead of only storing final hex values.

The runtime contract is unchanged:

```
primitive colors
  -> selector rules
    -> TokenGraph
      -> ThemeAccessors
        -> terminal rendering
```

Rules live in @flyingrobots/bijou. They do not depend on external design token packages and they compile to the same TokenValue objects the renderer already consumes.

Bijou dogfoods this flow for the owned bijou-dark and bijou-light presets: their primitive ink, brand, and surface palettes compile through TokenGraph rules into the same public Theme objects existing consumers already use. Legacy vivid themes and third-party palette fixtures remain literal compatibility presets.

When To Use Rules

Use a rule when a token should follow a stable decision:

- readable foreground on a known surface
- vivid accent that must not reuse success or error
- muted candidate that still clears a contrast floor
- closest palette color to a target
- ordered ramp position such as the middle or final stop

Do not use rules to hide one-off taste decisions. If the product needs exactly one named color, write the token directly.

Primitive To Semantic Flow

Start with primitive colors, then derive product roles from them:

```
import {
  bestContrastWith,
  createTokenGraph,
  mostVivid,
  scope,
} from '@flyingrobots/bijou';

const graph = createTokenGraph({
  palette: {
    ink: '#0b1020',
    paper: '#f8fafc',
    blue: '#3399ff',
    green: '#22c55e',
    red: '#ef4444',
  },
  surface: {
    primary: { fg: '#f8fafc', bg: { ref: 'palette.ink' } },
  },
  semantic: {
    primary: bestContrastWith({ ref: 'surface.primary.bg' }, scope('palette')),
    accent: mostVivid(scope('palette'), {
      against: { ref: 'surface.primary.bg' },
      minContrast: 4.5,
      not: ['palette.green', 'palette.red'],
    }),
  },
});
```

semantic.primary is now “the highest-contrast palette color on the primary surface.” semantic.accent is “the most vivid readable palette color that is not reserved for success or error.”

Inspect The Decision

Rules are useful only when they can explain themselves:

```
const fact = graph.inspect('semantic.accent');
```

The inspection fact includes:

- the rule id
- the selected candidate
- candidate paths and resolved colors
- excluded candidates
- contrast failures
- dependency paths

This is the data future Theme Inspector and Theme Lab surfaces should render. They should not infer token ownership from RGB equality or screenshots.

Selector Vocabulary

bestContrastWith(target, candidates) chooses the highest-contrast candidate.

minContrastWith(target, candidates, { ratio }) chooses the first candidate that reaches the requested contrast ratio.

mostVivid(candidates, options) chooses the highest-chroma candidate; it is readable only when against and minContrast are provided.

leastVivid(candidates, options) chooses the lowest-chroma candidate; it uses the same readability rule.

closestColor(target, candidates) chooses the closest RGB candidate.

nthColor(candidates, index) chooses an ordered candidate by integer or relative index.

Candidate scopes are deterministic. scope('palette') reads direct children of the palette scope in definition order.

Lower-Mode Rule

Rules improve color choice. They do not make color the only source of meaning.

Every UI that consumes these tokens still needs labels, symbols, structure, or state text that survives NO_COLOR, pipe output, and accessible lowering.

61

Theme Token Vocabulary

docs/design-system/theme-tokens.md

This document locks down the canonical token vocabulary for Bijou.

The Theme type in packages/bijou/src/core/theme/tokens.ts is the API truth. This page is the doctrine truth: it defines what each token family is for, what first-party components should prefer, and what counts as misuse.

Why this exists

Bijou already ships a typed token model:

- semantic
- surface
- border
- ui
- status
- gradient

What was missing was the rulebook. Without one, component authors end up choosing tokens by "what looks right in this preset" instead of by semantic job, and the design system drifts into an untyped color bucket.

Core rule

Choose tokens by meaning, not by color.

If a component needs a green foreground because it means "success", use a success token. Do not reach for whichever family currently contains a green hex value in the active preset.

Canonical families

semantic

Use semantic for general foreground meaning and emphasis inside a component.

This is the default family for:

- primary text emphasis
- supporting or muted text
- active or focused emphasis inside a component
- generic semantic cues that are not domain statuses

Built-in keys:

- primary
- muted
- accent
- success
- error
- warning
- info

House rules:

- prefer semantic.primary for deliberate foreground emphasis
- prefer semantic.muted for metadata, hints, and supporting text
- prefer semantic.accent for local active/focused affordances inside a component
- do not use semantic.* as a generic background-fill family

surface

Use surface for containment, fill, elevation, and interruption level.

This is the default family for:

- panel and card backgrounds
- sidebars and secondary rails
- elevated regions
- overlays, drawers, and modal fills
- muted or disabled regions

Built-in keys:

- primary
- secondary
- elevated
- overlay
- muted

House rules:

- treat surface.primary as the default content-region fill
- treat surface.secondary as the secondary-pane or sidebar fill
- treat surface.elevated as the default fill for bounded contained objects
- treat surface.overlay as the interruption or overlay fill

- treat surface.muted as the disabled, inset, or de-emphasized fill
- do not use surface choice to smuggle status meaning

border

Use border for structure, grouping, and boundary emphasis.

This is the default family for:

- box outlines
- separators that need tokenized color
- severity-bearing frames when the frame itself carries meaning

Built-in keys:

- primary
- secondary
- success
- warning
- error
- muted

House rules:

- prefer border.muted for ordinary structure
- use border.primary or border.secondary for stronger non-status grouping
- use status-colored borders only when the border itself is part of the state
- do not use warning/error borders as decoration

ui

Use ui for runtime chrome and reusable system affordances that do not belong to domain data.

This is the default family for:

- cursor or selection chrome owned by the shell/runtime
- focused pane gutters owned by reusable shell frames
- scrollbars and tracks
- section headers
- logos or branded shell marks
- table headers when they are part of reusable framework chrome
- empty tracks and similar UI scaffolding

Built-in keys:

- cursor
- focusGutter
- scrollThumb
- scrollTrack
- sectionHeader

- logo
- tableHeader
- trackEmpty

House rules:

- reserve ui.* for framework/runtime chrome, not business-domain statuses
- prefer a dedicated ui token when the same affordance recurs across shells
- use ui.focusGutter for focused shell pane gutters; use semantic.accent for local active emphasis inside a component
- do not use ui.* as a back door for arbitrary one-off component colors

status

Use status for explicit state labels and indicators attached to an object, process, or record.

This is the default family for:

- badges
- timeline states
- health/readiness indicators
- process states such as pending or active

Built-in keys:

- success
- error
- warning
- info
- pending
- active
- muted

House rules:

- use status.* when the UI is naming or signaling a real status
- prefer status.pending and status.active over reusing accent/focus tokens
- do not use status.* for focus, hover, or generic “selected” styling

gradient

Use gradient for rare multi-stop emphasis that is intentionally decorative or specialized.

Built-in keys:

- brand
- progress

House rules:

- gradients are optional emphasis, not required meaning
- never make critical status, navigation, or instructions depend on a gradient
- prefer gradient.progress only when the component is actually showing a progress fill or similar continuous range
- keep logos and brand treatments rare

Per-Token Library Reference

Every token below is a role, not a paint name. The examples are intentionally strict so components remain readable when a theme changes.

Semantic Tokens

Token	Use when	Do not use when
semantic.primary	Main readable text, selected document body text, and deliberate foreground emphasis.	The text is supporting metadata, a status label, or decorative brand copy.
semantic.muted	Secondary copy, timestamps, helper copy, inactive labels, and low-emphasis hints that must still be readable.	The content is disabled, destructive, or critical to the task.
semantic.accent	Local emphasis inside a component, selected inline text, command affordances, and non-status callouts.	The meaning is focus owned by the shell, a process status, or a brand logo.
semantic.success	Human-readable success copy, positive validation messages, and resolved inline facts.	The UI is only drawing a status dot, badge, or border. Use status.success or border.success.
semantic.error	Error copy, destructive inline warning text, and validation failure prose.	The role is merely an error-colored frame or status chip.
semantic.warning	Cautionary copy where a user may need to slow down before acting.	Ordinary emphasis, progress, or decorative warm color.
semantic.info	Informational copy, neutral notices, and non-blocking explanatory text.	Active selection, focus, or branded chrome.

Surface Tokens

Token	Use when	Do not use when
surface.primary	The default document, editor, or main content background.	Sidebars, modal interruption, disabled wells, or one-off accent fills.
surface.secondary	Navigation rails, sidebars, metadata panes, and secondary work regions.	Main reading surfaces or raised controls that need elevation.
surface.elevated	Cards, popovers, field groups, panels, and bounded objects that sit above the base surface.	Full-screen content regions or modal scrims.

Token	Use when	Do not use when
surface.overlay	Modal dialogs, drawers, command palettes, and interruption surfaces.	Decorative tinting, status meaning, or ordinary sidebars.
surface.muted	Disabled regions, inset wells, skeleton tracks, and deliberately de-emphasized containers.	Muted foreground text. Use semantic.muted.

Border Tokens

Token	Use when	Do not use when
border.primary	Strong structural grouping, active panel outlines, and primary separators.	Local text emphasis or generic accent foreground.
border.secondary	Secondary grouping, alternate section outlines, and non-primary divider emphasis.	Warning, error, or selected state.
border.success	A boundary itself communicates a successful state.	A success badge or success prose would be clearer.
border.warning	A boundary itself communicates caution or review-needed state.	Decorative warm framing or hover/focus styling.
border.error	A boundary itself communicates invalid, destructive, or failed state.	Non-error emphasis or ordinary grouping.
border.muted	Default boxes, separators, table rules, and low-emphasis structure.	Disabled text or muted copy.

UI Tokens

Token	Use when	Do not use when
ui.cursor	Runtime cursor, selected shell row chrome, and reusable selection affordances.	Business-domain active state or selected text inside one component.
ui.focusGutter	Focus indicators owned by the app frame, pane gutter, or reusable shell.	Inline focus styling inside a component.
ui.scrollThumb	Scrollbar thumbs and viewport-position indicators.	General progress bars or sliders.
ui.scrollTrack	Scrollbar tracks and scroll wells.	Disabled containers or progress-empty tracks.
ui.sectionHeader	Reusable shell headers, section rails, and framework-level headings.	Document body headings with product semantics.
ui.logo	Product mark, brand lockup, title-screen brand treatment, or recurring shell identity mark.	Generic accent text or status emphasis.
ui.tableHeader	Reusable table header chrome that is part of the framework.	Arbitrary table cell foreground or selected row styling.
ui.trackEmpty	Empty progress tracks, meters, and fixed scaffold tracks.	Scroll tracks, disabled fills, or muted text.

Status Tokens

Token	Use when	Do not use when
status.success	A badge, chip, timeline item, or indicator says the object succeeded.	Success prose or border-only success.
status.error	A badge, chip, timeline item, or indicator says the object failed or is invalid.	Error prose or destructive button text.
status.warning	A badge, chip, timeline item, or indicator says caution or review is required.	Warm decorative emphasis.
status.info	A badge, chip, timeline item, or indicator says informational or neutral status.	General informational prose.
status.pending	A state indicator says queued, waiting, loading, or not yet resolved.	Disabled content or muted metadata.
status.active	A state indicator says running, live, current, or in progress.	Keyboard focus, selected row, or cursor chrome.
status.muted	A state indicator says archived, inactive, skipped, or intentionally quiet.	Ordinary secondary copy.

Gradient Tokens

Token	Use when	Do not use when
gradient.brand	A rare brand treatment, title art, or identity moment benefits from multi-stop color.	The user must infer meaning from it.
gradient.progress	A meter, progress fill, or continuous range benefits from directional color.	Static decoration, status badges, or body text.

Default Dark/Light UX Audit

The legacy CYAN_MAGENTA preset is useful as a vivid terminal-native preset, but it is not ideal as the model for dense product surfaces. It uses saturated terminal colors for many unrelated jobs: cyan appears as info, active state, cursor, scrollbar, logo, and primary border; magenta appears as accent, secondary border, and focus gutter. That makes the app energetic, but it also makes many regions compete at the same visual priority.

Bijou's first-party defaults now use a calmer shell posture:

- neutral surfaces carry most of the screen
- one accent family carries local emphasis and shell selection
- status colors remain distinct from focus and brand
- muted text remains contrast-safe instead of merely dim
- dark and light themes both satisfy readable foreground/background pairs

The BIJOU_DARK preset uses dark neutral surfaces, readable cool/warm accents, and separate success/warning/error/info colors. The BIJOU_LIGHT preset uses light neutral surfaces with darker semantic colors instead of pastel foregrounds. DOGFOOD consumes those defaults directly as one DOGFOOD shell theme family with Dark and Light modes, so the docs app proves the same palette family that first-party components should treat as the baseline. Both modes target at least 4.5:1 contrast for surface text and common semantic foregrounds on every shell surface.

Compared with large design systems, Bijou is intentionally compact today. The runtime vocabulary has about thirty direct color-bearing slots plus two gradient roles. Material-style systems expose more paired foreground/background roles. Carbon and Atlassian-style systems go further with text hierarchy, interaction states, icon roles, and component-specific aliases. Bijou should grow only when a missing role causes repeated ambiguity in real components.

Safe-Pair Contrast Contract

Theme contrast is declared through safe pairs rather than buried inside one-off test loops. A safe pair names a foreground token path and a background token path that a theme promises will remain readable.

Safe-pair background paths may target token background slots:

```
semantic.primary on surface.primary.bg
semantic.muted on surface.secondary.bg
status.active on surface.overlay.bg
ui.cursor on surface.elevated.bg
```

Use the defineThemeSafePairs() builder when a first-party app or component needs a reusable contrast matrix:

```
const pairs = defineThemeSafePairs()
  .readable('semantic.primary', 'surface.primary.bg')
  .status('status.error', 'surface.overlay.bg')
  .chrome('ui.cursor', 'surface.secondary.bg')
  .build();
```

Then pass those pairs to doctorTheme():

```
const report = doctorTheme(theme, { contrastPairs: pairs });
```

Safe-pair kinds communicate intent:

- readable: ordinary text or component foregrounds that must stay legible
- status: process, health, timeline, or object-state colors
- chrome: shell/runtime affordances such as cursor and selection chrome

Do not use safe pairs as a way to bless every possible token combination. Declare pairs only when a component or product surface expects that combination to work.

Theme Debugger And Lab

DOGFOOD now has two inspectability surfaces:

- F10 opens a bounded Theme Inspector drawer in the docs shell.
- The Themes page publishes a Theme Lab with default palettes, shell-gallery facts, token swatches, and safe-pair diagnostics.

Current UX:

- F10 opens the drawer on the active DOGFOOD shell theme.
- F10 or Esc closes the drawer.
- The drawer shows token swatches grouped by semantic, surface, border, ui, s tatus, and gradient.
- Each swatch renders the token path and uses the token value as the swatch fill or foreground depending on token family.
- The drawer reports the safe-pair pass count for the active theme.
- The Theme Lab page lists the first-party dark/light defaults, DOGFOOD shell themes, landing palette themes, and token swatches.

Still deferred:

- dark/light preview without committing the app theme
- element-level token overlays
- renderer provenance that can report whether a cell came from a token, raw color, or lowered/no-color fallback

The current implementation inspects theme tokens only. Element-level token overlays should wait until renderer provenance is available, because guessing token ownership from final RGB values would be misleading.

First-Party Defaults

When first-party components need a default token and do not have stronger domain-specific semantics, prefer these defaults:

- default supporting text: semantic.muted
- explicit primary emphasis: semantic.primary
- local active or focused affordance inside a component: semantic.accent
- default structural border: border.muted
- default contained-region fill: surface.elevated
- default secondary-pane fill: surface.secondary
- default overlay fill: surface.overlay
- default disabled or inset fill: surface.muted
- reusable shell cursor or selected-shell chrome: ui.cursor
- reusable focused shell gutter chrome: ui.focusGutter
- reusable scrollbar chrome: ui.scrollThumb and ui.scrollTrack
- table header chrome: ui.tableHeader
- progress empty track: ui.trackEmpty

These defaults are conventions, not excuses to skip judgment. If a component has a stronger semantic job, use the family that matches the job.

First-Party Prohibitions

First-party components should follow these rules:

- do not read raw token paths from ctx.theme.theme.* when an accessor exists
- do not invent new first-party token families ad hoc
- do not use status.* when the real meaning is focus or selection
- do not use surface.* as a generic foreground-emphasis bucket
- do not use gradients as a substitute for semantic color
- do not rely on color alone; meaning must survive noColor, pipe, and accessible

If the current token families are not expressive enough, change the theme contract and docs in the same slice instead of sneaking around the vocabulary.

Extension Rules

Third-party and app-level code may extend:

- status
- ui
- gradient

That is already part of the generic Theme<S, U, G> contract.

Rules for extension:

- extend when the new key represents a stable semantic role, not one component's temporary styling whim
- keep first-party components on the built-in shared vocabulary unless the contract is intentionally widened
- prefer wrapping a component with explicit token options over hard-coding raw token-path lookups
- when theme changes must invalidate cached component work, prefer the public observeTheme(ctx, handler) seam over direct tokenGraph.on(...)

Access Pattern

Component code should prefer the accessor layer:

- ctx.semantic(...)
- ctx.surface(...)
- ctx.border(...)

- ctx.status(...)
- ctx.ui(...)
- ctx.gradient(...)

That keeps component code coupled to the public vocabulary instead of the raw object shape or token-graph internals.

Relationship To Foundations

The design-system foundations describe the high-level visual roles:

- foreground
- surface
- status
- accent
- border or separator

This page maps those roles onto the shipped API vocabulary:

- foreground -> semantic
- accent -> semantic.accent
- surface -> surface
- border or separator -> border
- reusable framework chrome -> ui
- explicit named state -> status
- rare multi-stop emphasis -> gradient

Examples

Examples of the intended split:

- alert() should use semantic(variant) plus a matching border, not a gradient
- badge() should use status.* when it is carrying a real status
- kbd() should use muted structure tokens, not a status token
- tableSurface() should use ui.tableHeader for reusable header chrome
- shell cursor, focused gutters, scrollbars, and track fills should use ui.*, not business statuses
- overlays and drawers should communicate interruption primarily through surface.overlay, not by tinting everything as a warning

III

Design records

How to read a design record

The records in this part were working agreements about changes to Bijou. A useful record makes a proposed outcome, its limits and the evidence needed to judge it visible before those choices become scattered through code. Some records also carry implementation notes and closeout evidence added as the work progressed.

The recurring format reflects that purpose. It gives a reviewer several ways to question a change: whether it serves a concrete user, whether its boundaries are clear, whether it preserves existing contracts, and whether success can be observed. The fields are a way of exposing decisions to inspection. Their presence alone does not establish the quality or completion of the work.

Sponsorship and the hill

Sponsored Human and **Sponsored Agent**, sometimes written as sponsor fields, identify the people and automated consumers whose needs give the work its purpose. Human descriptions often name a task or a situation. Agent descriptions often emphasize deterministic data, provenance, inspection and reproducible checks. A named participant can also identify authorship or responsibility; the name itself is not evidence that a review occurred.

The **Hill** states the useful outcome being pursued: who should be able to do what, and why it matters. It gives the implementation an outcome to answer to. Read it alongside the scope and non-goals to understand the size of the promise.

Playback questions and boundaries

Playback Questions turn the outcome into observations. Can the intended person complete the task? Does the right pane own a key? Does a change appear in the same frame? Can an automated consumer inspect the result? These questions connect a technical plan to a demonstration or check that another reader could assess.

Scope and **Non-Goals** describe what the change owns and what remains outside it. **Prerequisites**, linked work and implementation outlines explain which contracts or earlier changes the proposal relies on. A dependency should be read as a claim about what must already be correct, not inferred merely because two records discuss the same topic.

The different postures of one feature

Accessibility and assistive-reading sections ask how the feature communicates when its visual presentation is unavailable or unsuitable. Localization and directionality sections ask how language, text expansion and reading direction affect geometry and interaction. Agent-inspectability sections ask which stable facts, identities, diagnostics and outputs can be examined without reconstructing meaning from a terminal image.

These questions arise from Bijou's philosophy: the useful meaning of a feature has to survive more than one way of encountering it. Their explicit place in the record makes it possible to notice an omitted mode or an unsupported interaction before implementation hides the omission behind a convincing default view.

Invariants, tests and closeout

Linked Invariants connect the proposal to contracts that the system is expected to preserve. The architecture or implementation outline identifies the seams expected to carry the change. **Tests To Write First** describes proposed evidence. A listed test is a plan until a recorded run or reproducible result establishes what was actually checked.

Closeout and retrospective sections can record what landed, which checks ran and what remains unresolved. Follow their issue, pull-request and commit references when assessing a completion claim. A source-text assertion, a focused runtime test, a visual capture and a full validation run establish different things; retain the scope of the evidence being cited.

Reading across the collection

The records are grouped into DOGFOOD, developer experience, runtime, design language, localization, workflow and other design concerns. Identifiers such as DF, DX, RE, DL, LX and WF are historical family labels. Templates and naming changed over time, and some identifiers were reused. Use the full source path, title and linked work to identify a record reliably.

Status fields, dates, lane names and future-tense sections retain their original context. Historical references to cycles, legends and METHOD explain how a record was organized. They do not reinstate that process as a requirement for current work. GitHub issues and milestones own current scope, and current operating agreements govern how a new change is carried out.

Read a record as an argument with evidence to follow. Its strongest contribution is often the precise question it allows the next reader to ask of the code.

DOGFOOD design records

These records follow the documentation application as a product and a proving ground: coverage, navigation, stories, blocks, responsive layouts and release-facing surfaces. Early coverage increments record the route towards a fuller component catalog; later records put shared contracts under the pressure of a complete shell. Read individual coverage claims with their measurement date and definition in view.

62

DF-001 — Show DOGFOOD Coverage Progress

docs/design/DF-001-show-dogfood-coverage-progress.md

Legend: DF — DOGFOOD Field Guide

Why this cycle exists

We want to release Bijou without pretending DOGFOOD is already a complete reference.

The honest answer is not to block release forever. It is to make DOGFOOD visibly truthful about its current documentation coverage.

This cycle exists to put that truth directly into the initial docs experience:

- a progress indicator
- an explicit ratio
- a defensible denominator tied to the canonical component-family reference

Scope of this cycle

This cycle intentionally covers:

- a canonical DOGFOOD coverage metric based on documented component families
- mapping current DOGFOOD stories onto that coverage model
- a docs-pane progress bar and ratio
- tests that prove the landing surface reflects real coverage instead of a hand-wavy claim

It does **not** include:

- expanding story coverage itself
- claiming feature completeness
- a broader release-management dashboard
- automatic release gating

Human users

Primary human user

A user opening DOGFOOD to decide how complete and trustworthy the field guide currently is.

They need:

- a clear signal that DOGFOOD is still in progress
- an honest coverage ratio
- enough context to understand that the metric is about documented component families

Human hill

A user can open DOGFOOD and immediately see that it is a living field guide with explicit progress, not a silently incomplete reference pretending to be finished.

Agent users

Primary agent user

An agent maintaining DOGFOOD story coverage over time.

It needs:

- a stable coverage model
- a defensible denominator
- a way to add stories later and have the landing progress move automatically

Agent hill

An agent can add or remove DOGFOOD story coverage and see the docs-pane progress update from real source data rather than manual copy edits.

Human playback

1. A user launches DOGFOOD.
2. The landing screen shows the normal hero treatment.
3. The initial docs content pane shows a progress bar and ratio for current DOGFOOD documentation coverage.
4. The user understands that DOGFOOD is in progress and can see how much of the canonical family map is currently covered.

Agent playback

1. An agent opens the DF-001 cycle doc.
2. It sees that the denominator is the canonical component-family reference, not a vague export count.
3. It runs the cycle and app tests.
4. It can verify that the docs-pane copy and progress bar are driven from real story coverage data.
5. It can point to the next backlog item for actually expanding coverage.

Linked invariants

- Tests Are the Spec
- Visible Controls Are a Promise
- Docs Are the Demo

Invariants for this cycle

- DOGFOOD must not imply fuller coverage than it actually has
- the coverage denominator must be explicit and stable
- adding story coverage later should update the progress metric automatically
- the title screen must not present documentation coverage as though it were a loading state
- the docs-pane coverage UI must remain secondary to the primary onboarding content

Implementation outline

1. Create a DOGFOOD legend and coverage-focused cycle doc.
2. Define the canonical coverage denominator from the design-system component-family reference.
3. Map current DOGFOOD stories onto that family coverage model.
4. Write failing cycle and app tests for the new landing progress treatment.
5. Render a progress bar and ratio in the initial docs content pane.
6. Close the cycle and spawn the next DOGFOOD coverage backlog item.

Tests to write first

Under tests/cycles/DF-001/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves from canonical family reference data

- the initial docs content surface shows the coverage ratio and percent
- the next DOGFOOD backlog item exists

Risks

- choosing a denominator that feels arbitrary or magical
- making the title screen feel like a loading screen when it is actually a title screen
- hardcoding a number that will drift the moment a new story lands

Out of scope

- story authoring for additional families
- docs completeness gating in CI
- release automation

Retrospective

What landed

- a new DOGFOOD legend
- a real coverage model based on canonical component families
- story-to-family coverage mapping
- a docs-pane progress bar and ratio that update from the current story set

Drift from ideal

This cycle does not improve DOGFOOD coverage itself.

That is intentional. The goal here is honesty first, not pretending that a progress indicator is the same thing as documentation work.

Debt spawned

Spawned:

- DF-002 — Expand DOGFOOD Component Family Coverage

63

DF-003 — Enforce DOGFOOD Coverage Floor

docs/design/DF-003-enforce-dogfood-coverage-floor.md

Cycle for turning DOGFOOD coverage honesty into a CI-enforced release gate instead of a passive progress display

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-001 — Show DOGFOOD Coverage Progress

Why this cycle exists

DF-001 made DOGFOOD honest about how incomplete it currently is.

That honesty should not remain a purely visual promise. The repo should also enforce a minimum documentation floor so DOGFOOD cannot quietly regress or ship while pretending its progress bar means something.

This cycle exists to:

- make the minimum acceptable DOGFOOD coverage explicit
- enforce that minimum in CI and release-readiness flows
- document the ratchet policy for future DF cycles

Human users / jobs / hills

Primary human users

- maintainers shipping Bijou releases
- contributors expanding or touching DOGFOOD docs coverage
- reviewers deciding whether a cycle improved or regressed documentation completeness

Human jobs

1. Ship Bijou without accidentally letting DOGFOOD coverage slip below the current honest bar.
2. See the current floor and next expected target without reverse-engineering old commits.
3. Treat docs completeness as a real quality gate, not a vague aspiration.

Human hill

DOGFOOD coverage progress is not just displayed in the app; it is enforced in the repo and raised deliberately over time.

Agent users / jobs / hills

Primary agent users

- agents expanding DOGFOOD stories
- agents preparing release candidates
- agents auditing whether documentation completeness is meeting the current floor

Agent jobs

1. Compute the current DOGFOOD family-coverage ratio from source data.
2. Compare that ratio against an explicit floor.
3. Fail deterministically when the floor is not met.
4. Surface the next required target so future cycles know the next ratchet point.

Agent hill

An agent can determine from source-controlled policy whether the current DOGFOOD story set is acceptable for CI and release without heuristics or manual judgment.

Human playback

1. A maintainer opens the repo after a coverage-expansion cycle.
2. The DOGFOOD coverage floor is explicit in source control.
3. CI checks the current real story coverage against that floor.
4. If coverage drops below the floor, CI fails before merge or release.
5. The maintainer can also see the next target to raise by in the next dedicated DOGFOOD coverage cycle.

Agent playback

1. An agent computes DOGFOOD coverage from stories.ts and the canonical component-family reference.
2. The agent reads the explicit floor and next target from the shared policy module.
3. The agent runs the coverage gate script.
4. If coverage is below the floor, the script fails with the measured percent, floor, and next target.
5. If coverage meets the floor, CI proceeds and the next ratchet remains visible for future cycles.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Implementation outline

1. Add a shared DOGFOOD coverage-policy module with:
   - explicit floor percent
   - explicit +5-point ratchet increment
   - explicit next target
   - helpers for evaluating/asserting the floor
2. Add a dogfood:coverage:gate script that computes real coverage and fails when the floor is not met.
3. Add focused tests for the policy and gate behavior.
4. Wire the gate into CI and release-readiness flows.
5. Update DF legend/changelog and spawn the next backlog item for actually raising the floor.

Tests to write first

- cycle doc regression for DF-003 and its follow-on backlog item
- policy/gate tests proving:
  - current coverage meets the current floor
  - the next target is exactly floor + 5
  - the gate fails when the floor is set above current coverage
- release-readiness regression proving the new gate runs as part of the plan

Risks / unknowns

- CI can enforce the current floor, but it cannot infer from git history whether maintainers actually raised the floor every time they should have
- the initial floor needs to be set to the current honest value, not an aspirational one that would immediately fail the repo
- the meaningful follow-on is expanding DOGFOOD coverage itself, not only adding more policy

Retrospective

What landed:

- explicit DOGFOOD coverage floor policy in source control
- a reusable coverage gate script
- CI/release hooks enforcing the current floor
- an explicit next target to make the next +5-point ratchet visible

What did not land:

- actual new DOGFOOD component-family stories
- automatic historical enforcement of “every cycle raised by 5” beyond the explicit next-target ratchet

Follow-on:

- DF-004 — Raise DOGFOOD Coverage Floor to the Next 5-Point Target

64

DF-004 — Raise DOGFOOD Coverage Floor to the Next 5-Point Target

docs/design/DF-004-raise-dogfood-coverage-floor-to-next-target.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-003 — Enforce DOGFOOD Coverage Floor
- DF-002 — Expand DOGFOOD Component Family Coverage

Why this cycle exists

DF-003 made the current DOGFOOD coverage floor executable.

That was necessary, but it still left the honest next step untouched:

- DOGFOOD only documents 3 of 35 component families
- the enforced floor is 9%
- the next declared target is 14%

This cycle exists to earn that next target honestly by documenting enough additional component families that the floor can be raised without gaming the metric.

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 3 to 5 families
- raising the enforced floor from 9% to 14%
- raising the next declared target from 14% to 19%
- proving the result through cycle tests and the existing gate script

It does **not** include:

- pretending every component now has full docs coverage
- changing the canonical family denominator
- inventing placeholder stories with no real use/avoid/lowering guidance
- broad DOGFOOD shell or navigation redesign

Human users

Primary human user

A builder evaluating whether DOGFOOD is becoming meaningfully more complete over time.

They need:

- the progress bar and ratio to move for a real reason
- newly documented families to be legitimate, not filler
- the CI gate to reflect actual content growth

Human hill

A user can see that DOGFOOD is more complete than before, and the reported progress corresponds to real newly documented families they can browse.

Agent users

Primary agent user

An agent tasked with growing DOGFOOD coverage while preserving honesty.

It needs:

- an explicit target to hit
- a clear smallest honest route to that target
- tests that prove the new stories count toward canonical family coverage

Agent hill

An agent can raise the DOGFOOD floor to the next target only by adding real story coverage that the gate and docs app both recognize.

Human playback

1. A user enters DOGFOOD.
2. The docs pane reports a higher coverage percentage than before.
3. The higher value corresponds to newly browseable component stories instead of a policy-only change.

4. The user can open those new stories and see real usage guidance, variants, and lowering notes.

Agent playback

1. An agent opens the DF-004 cycle doc.
2. It can see that the target is 14%, which requires 5 covered families out of the current 35.
3. It adds real stories for the smallest credible uncovered families.
4. The cycle tests and gate script both confirm that coverage is now 14% and the floor can be raised safely.
5. The next 5-point target is recorded for the following cycle.

Linked invariants

- Tests Are the Spec
- Visible Controls Are a Promise
- Docs Are the Demo

Invariants for this cycle

- the floor must only move after real coverage moves
- every newly counted family must have an actual DOGFOOD story with real docs guidance
- the gate must reflect the new floor and the next declared target
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-004 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:
   - 14% coverage
   - 5 documented families out of 35
   - two newly covered family ids
   - a raised floor of 14 with a next target of 19
3. Add the smallest honest new DOGFOOD stories needed to hit the target.
4. Update the coverage floor policy and keep the gate green.
5. Close the cycle and spawn the next ratchet backlog item.

Tests to write first

Under tests/cycles/DF-004/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 5 of 35 families and 14%
- the story catalog now covers Inline status and Inline shortcut cues
- the enforced floor is 14 and the next target is 19
- the next DOGFOOD backlog item exists

Risks

- adding superficial stories that technically count but do not teach
- bumping the floor before the content really exists
- choosing families that distort the docs app navigation without improving the field guide much

Out of scope

- complete DOGFOOD component coverage
- localization work
- shell interaction redesign
- changing the coverage metric itself

Retrospective

What landed

- new DOGFOOD stories for badge() and kbd()
- canonical family coverage increased from 3/35 to 5/35
- DOGFOOD docs coverage now resolves to 14%
- the enforced floor moved from 9% to 14%
- the next declared target moved from 14% to 19%

Drift from ideal

No material drift.

This cycle stayed intentionally narrow: two honest families, one ratchet step, and no fake breadth.

Debt spawned

Spawned:

- DF-005 — Raise DOGFOOD Coverage Floor to 19%

65

DF-005 — Raise DOGFOOD Coverage Floor to 19%

docs/design/DF-005-raise-dogfood-coverage-floor-to-19-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-004 — Raise DOGFOOD Coverage Floor to the Next 5-Point Target

Why this cycle exists

DF-004 proved the coverage ratchet can move honestly.

The next promised step is now explicit:

- DOGFOOD currently documents 5 of 35 component families
- the enforced floor is 14%
- the next declared target is 19%

This cycle exists to earn that next target with real component stories, not by moving the policy first.

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 5 to 7 families
- documenting progress and loading surfaces with honest static and animated previews
- raising the enforced floor from 14% to 19%
- raising the next declared target from 19% to 24%

It does **not** include:

- broad DOGFOOD navigation redesign
- a full story-runtime state system for every component preview
- pretending the progress/loading families are now exhaustively documented

- changing the canonical component-family denominator

Human users

Primary human user

A builder evaluating whether DOGFOOD explains Bijou's loading and progress affordances honestly.

They need:

- a static determinate progress preview they can scan quickly
- a looping animated progress preview that proves the component is alive, not just described
- explicit loading-placeholder guidance instead of only shell-level status chrome

Human hill

A user can open DOGFOOD and see both static and animated progress examples plus real loading-placeholder guidance, and the documented coverage percentage moves for that real added content.

Agent users

Primary agent user

An agent tasked with expanding DOGFOOD coverage while preserving honesty and keeping previews deterministic.

It needs:

- a clear smallest honest route from 14% to 19%
- a preview seam that can animate without inventing a second runtime
- tests that prove both the docs coverage and the animated preview behavior

Agent hill

An agent can raise DOGFOOD to the 19% floor only by adding real stories for the Progress indicators and Loading placeholders families and proving that the progress preview actually animates on pulse.

Human playback

1. A user enters DOGFOOD and opens the progress story.

2. They can see a static determinate progress example and a looping animated example.
3. They can open the loading-placeholder story and see realistic skeleton usage guidance.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-005 cycle doc.
2. It can see that the target is 19%, which requires 7 covered families out of 35.
3. It adds real stories for Progress indicators and Loading placeholders.
4. It threads a minimal preview clock into DOGFOOD so the animated progress preview changes on pulse without inventing general mutable story state.
5. The cycle tests and gate confirm that coverage is now 20%, which safely clears the 19% floor, and the next target is recorded as 24%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- the progress story must include both a static and a looping animated preview
- the animated preview must be driven by deterministic pulse time, not real wall-clock side effects
- loading placeholders must be documented through a real skeleton() story before they count toward coverage
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-005 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:
   - 7 of 35 documented families and 20% real coverage
   - coverage of Progress indicators and Loading placeholders

    - an enforced floor of 19 with a next target of 24
    - a looping progress preview that visibly changes on pulse
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Thread a preview clock into the docs story renderer for deterministic animation.
5. Raise the floor policy and spawn the next ratchet backlog item.

Tests to write first

Under tests/cycles/DF-005/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 7 of 35 families and 20%
- the story catalog now covers Progress indicators and Loading placeholders
- the enforced floor is 19 and the next target is 24
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the static progress variant remains visually stable across a pulse
- the animated progress variant changes across a pulse

Risks

- adding an animation seam that is broader than this cycle actually needs
- counting a family before the story is useful enough to teach with
- letting the animated preview become a special-case runtime instead of a small deterministic rendering input

Out of scope

- complete DOGFOOD coverage
- shell help/localization work
- service-backed localization tooling
- a full generalized animation framework for stories

Retrospective

What landed

- new progressBar() DOGFOOD story with both a static determinate preview and a looping pulse-driven rollout preview
- new skeleton() DOGFOOD story covering short-lived loading placeholders

- a minimal preview clock in the docs explorer so stories can animate deterministically on pulse without a separate story runtime
- canonical family coverage increased from 5/35 to 7/35
- DOGFOOD docs coverage now resolves to 20%, which safely clears the 19% floor
- the enforced floor moved from 14% to 19%
- the next declared target moved from 19% to 24%

Drift from ideal

No material drift.

The only architectural addition beyond new stories was a tiny preview-time input for story rendering. That stayed appropriately narrow and did not turn into a generalized mutable story-state system.

Debt spawned

Spawned:

- DF-006 — Raise DOGFOOD Coverage Floor to 24%

66

DF-006 — Raise DOGFOOD Coverage Floor to 24%

docs/design/DF-006-raise-dogfood-coverage-floor-to-24-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-005 — Raise DOGFOOD Coverage Floor to 19%

Why this cycle exists

DF-005 proved the coverage ratchet can keep moving without padding the docs with fake content.

The next promised step is now explicit:

- DOGFOOD currently documents 7 of 35 component families
- the enforced floor is 19%
- the next declared target is 24%

This cycle exists to earn that next target with real field-guide coverage for structural grouping and inspector surfaces.

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 7 to 9 families
- documenting box() as the canonical containment/grouping family
- documenting inspector() as the canonical side-panel inspection family
- raising the enforced floor from 19% to 24%
- raising the next declared target from 24% to 29%

It does **not** include:

- broad DOGFOOD navigation redesign
- generalized docs-side inspector tooling beyond real component stories

- pretending either family is exhaustively documented after one cycle
- changing the canonical component-family denominator

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach the structural surfaces that shape real Bijou apps instead of only status, loading, and overlays.

They need:

- one story that makes box() and headerBox() feel like honest grouping primitives instead of generic chrome
- one story that makes inspector() feel like the right side-panel summary surface rather than another arbitrary box
- coverage progress that rises only because those new families are genuinely documented

Human hill

A user can open DOGFOOD, find real box() and inspector() stories, and see the documentation-coverage percentage rise because those two families are now honestly represented.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without gaming the number or introducing shallow filler stories.

It needs:

- a clear smallest honest route from 19% to 24%
- tests that prove exactly which two new families were added
- a content slice that uses canonical componentry instead of bespoke DOGFOOD-only display hacks

Agent hill

An agent can raise DOGFOOD to the 24% floor only by adding real stories for F ramed grouping and Inspector panels, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new structural grouping story.
2. They can see both plain containment and titled panel usage through box() / headerBox().
3. They open the new inspector story and see a calm side-panel summary with one obvious current selection and structured supporting sections.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-006 cycle doc.
2. It can see that the target is 24%, which requires 9 covered families out of 35.
3. It adds real stories for Framed grouping and Inspector panels.
4. It updates the coverage policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 26%, which safely clears the 24% floor, and the next target is recorded as 29%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- box() coverage must teach grouping/containment, not just decorative borders
- inspector() coverage must teach current-selection emphasis plus calmer supporting sections
- the two new families must be backed by real stories, not only design docs or app usage elsewhere
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-006 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:

    - 9 of 35 documented families and 26% real coverage
    - coverage of Framed grouping and Inspector panels
    - an enforced floor of 24 with a next target of 29
    - real box() and inspector() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-006/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 9 of 35 families and 26%
- the story catalog now covers Framed grouping and Inspector panels
- the enforced floor is 24 and the next target is 29
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new stories are reachable in the docs app and render recognizable box() / inspector() content

Risks

- adding a grouping story that looks decorative instead of teaching real containment choices
- making inspector() feel redundant with the existing docs-side variant panel instead of explaining the family clearly
- moving the policy first and treating the content as catch-up instead of earned progress

Out of scope

- complete DOGFOOD coverage
- shell-layer registry work
- localization follow-on work
- guided-flow block promotion

Retrospective

What landed

- new real box() DOGFOOD story teaching grouped containment and titled detail panels
- new real inspector() DOGFOOD story teaching calm side-panel summary rhythm
- canonical family coverage increased from 7/35 to 9/35
- DOGFOOD docs coverage now resolves to 26%, which safely clears the 24% floor
- the enforced floor moved from 19% to 24%
- the next declared target moved from 24% to 29%

Drift from ideal

No material drift.

The cycle stayed intentionally narrow: two real structural stories, one ratchet update, and one docs-preview integration proof that the new inspector story is reachable through DOGFOOD itself.

Debt spawned

Spawned:

- DF-007 — Raise DOGFOOD Coverage Floor to 29%

67

DF-007 — Raise DOGFOOD Coverage Floor to 29%

docs/design/DF-007-raise-dogfood-coverage-floor-to-29-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-006 — Raise DOGFOOD Coverage Floor to 24%

Why this cycle exists

DF-006 proved DOGFOOD can keep ratcheting upward without cheating and while still documenting real, important families.

The next promised step is now explicit:

- DOGFOOD currently documents 9 of 35 component families
- the enforced floor is 24%
- the next declared target is 29%

This cycle exists to earn that next target through real feedback-history coverage: one low-level transient overlay family and one activity-stream family.

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 9 to 11 families
- documenting toast() as the low-level transient overlay primitive
- documenting log() as the canonical activity stream primitive
- raising the enforced floor from 24% to 29%
- raising the next declared target from 29% to 34%

It does **not** include:

- notification-center redesign

- shell-level history workflow changes
- pretending DOGFOOD is now complete
- changing the canonical component-family denominator

Human users

Primary human user

A builder evaluating whether DOGFOOD now teaches transient feedback and chronological status output clearly enough to choose the right tool.

They need:

- one story that shows toast() as a low-level directly composed overlay rather than a full notification system
- one story that shows log() as an accumulating activity stream rather than a one-off status message
- coverage progress that rises only because those families are now genuinely documented

Human hill

A user can open DOGFOOD and understand the difference between toast() and log() as real, documented families, and the coverage number rises because that guidance is actually present.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without inflating the number through thin or redundant stories.

It needs:

- a clear smallest honest route from 24% to 29%
- tests that prove which feedback/history families were added
- integration checks that the new stories are reachable in the docs app

Agent hill

An agent can raise DOGFOOD to the 29% floor only by adding real toast() and log() stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new toast story.
2. They can see that toast() is a directly composed transient overlay, not a full notification center.
3. They open the new log story and see chronological, accumulating event output with level cues.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-007 cycle doc.
2. It can see that the target floor is 29%, which can be cleared by adding at least one more family, but the cycle intentionally documents two related families.
3. It adds real toast() and log() stories.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 31%, which safely clears the 29% floor, and the next target is recorded as 34%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- toast() coverage must teach direct overlay composition and avoid pretending it is the notification center
- log() coverage must teach chronological accumulation rather than one-off status chrome
- the new families must be backed by real stories in DOGFOOD itself
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-007 into the active cycle area and enrich it with playbacks and invariants.

2. Write failing cycle tests that demand:
    - 11 of 35 documented families and 31% real coverage
    - coverage of Low-level transient overlay and Activity stream
    - an enforced floor of 29 with a next target of 34
    - real toast() and log() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-007/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 11 of 35 families and 31%
- the story catalog now covers Low-level transient overlay and Activity strea m
- the enforced floor is 29 and the next target is 34
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new toast story is reachable in component search and renders recognizable transient-overlay content

Risks

- making toast() look like a notification-center substitute instead of a low-level overlay primitive
- making log() feel like a decorative box rather than an activity stream
- over-documenting shell notification behavior in a component-family cycle instead of keeping the scope narrow

Out of scope

- notification-center polish cycles
- localization follow-on work
- new shell-layer behaviors
- complete DOGFOOD coverage

Retrospective

What landed

- new real toast() DOGFOOD story teaching low-level transient overlay composition
- new real log() DOGFOOD story teaching chronological activity-stream output
- canonical family coverage increased from 9/35 to 11/35
- DOGFOOD docs coverage now resolves to 31%, which safely clears the 29% floor
- the enforced floor moved from 24% to 29%
- the next declared target moved from 29% to 34%

Drift from ideal

No material drift.

The cycle intentionally over-earned the floor by documenting two related families instead of stopping at the minimum one-family bump required by rounding. That makes the field guide stronger without changing the ratchet contract itself.

Debt spawned

Spawned:

- DF-008 — Raise DOGFOOD Coverage Floor to 34%

68

DF-008 — Raise DOGFOOD Coverage Floor to 34%

docs/design/DF-008-raise-dogfood-coverage-floor-to-34-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-007 — Raise DOGFOOD Coverage Floor to 29%

Why this cycle exists

DF-007 proved the coverage ratchet can keep moving while staying focused on real product value.

The next promised step is now explicit:

- DOGFOOD currently documents 11 of 35 component families
- the enforced floor is 29%
- the next declared target is 34%

This cycle exists to earn that next target through the documentation-oriented families that make DOGFOOD feel more like a real field guide: bounded markdown prose and explicit linked destinations.

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 11 to 13 families
- documenting markdown() as the bounded formatted-document family
- documenting hyperlink() as the explicit link/destination family
- raising the enforced floor from 29% to 34%
- raising the next declared target from 34% to 39%

It does **not** include:

- broad docs-shell redesign

- full reference-doc navigation or pager workflows
- introducing new component families into the canonical denominator
- pretending DOGFOOD is now complete

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach reference-style prose and explicit links as real Bijou surfaces instead of only showcasing status, overlays, and inspection panes.

They need:

- one story that shows markdown() as bounded structured prose, not a substitute layout engine
- one story that shows hyperlink() as an honest terminal destination primitive with fallback behavior
- coverage progress that rises only because those documentation-oriented families are genuinely present

Human hill

A user can open DOGFOOD, learn when to use markdown() and hyperlink(), and see the coverage number rise because the field guide now actually covers those reference surfaces.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without turning it into shallow catalog filler.

It needs:

- a clear smallest honest route from 29% to 34%
- tests that prove which documentation-oriented families were added
- one integration check that the new story is reachable through the docs app itself

Agent hill

An agent can raise DOGFOOD to the 34% floor only by adding real markdown() and hyperlink() stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new markdown story.
2. They can see bounded reference prose with headings, lists, and links rendered honestly.
3. They open the new hyperlink story and see explicit destination labels and fallback behavior instead of vague “click here” copy.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-008 cycle doc.
2. It can see that the target floor is 34%, which can be cleared by adding one family, but the cycle intentionally documents two related documentation families.
3. It adds real markdown() and hyperlink() stories.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 37%, which safely clears the 34% floor, and the next target is recorded as 39%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- markdown() coverage must teach bounded structured prose, not application chrome masquerading as markdown
- hyperlink() coverage must keep destinations explicit and meaningful instead of relying on generic labels
- the new families must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-008 into the active cycle area and enrich it with playbacks and invariants.

2. Write failing cycle tests that demand:
    - 13 of 35 documented families and 37% real coverage
    - coverage of Formatted documents and prose and Linked destinations
    - an enforced floor of 34 with a next target of 39
    - real markdown() and hyperlink() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-008/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 13 of 35 families and 37%
- the story catalog now covers Formatted documents and prose and Linked destinations
- the enforced floor is 34 and the next target is 39
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new markdown story is reachable in component search and renders recognizable bounded document content

Risks

- making the markdown story look like a generic blob of prose instead of a bounded structured document
- making the hyperlink story feel redundant with markdown links instead of explaining explicit destination handling
- overloading DOGFOOD with long prose blocks that crowd the existing three-pane shell

Out of scope

- full docs-reader navigation
- pager/localization follow-on work
- shell help/localized directional surfaces
- complete DOGFOOD coverage

Retrospective

What landed

- new real markdown() DOGFOOD story teaching bounded structured prose
- new real hyperlink() DOGFOOD story teaching explicit linked destinations and fallback behavior
- canonical family coverage increased from 11/35 to 13/35
- DOGFOOD docs coverage now resolves to 37%, which safely clears the 34% floor
- the enforced floor moved from 29% to 34%
- the next declared target moved from 34% to 39%

Drift from ideal

No material drift.

The cycle stayed focused on documentation-oriented surfaces instead of adding another status primitive, which makes DOGFOOD more credible as a field guide rather than only a component catalog.

Debt spawned

Spawned:

- DF-009 — Raise DOGFOOD Coverage Floor to 39%

69

DF-009 — Raise DOGFOOD Coverage Floor to 39%

docs/design/DF-009-raise-dogfood-coverage-floor-to-39-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-008 — Raise DOGFOOD Coverage Floor to 34%

Why this cycle exists

DF-008 made DOGFOOD feel more like a field guide by documenting bounded prose and explicit linked destinations.

The next promised step is now explicit:

- DOGFOOD currently documents 13 of 35 component families
- the enforced floor is 34%
- the next declared target is 39%

This cycle exists to earn that next target through two core form families that make DOGFOOD more credible as a guide to real decisions and set-building flows: confirm() and multiselect().

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 13 to 15 families
- documenting confirm() as the binary-decision family
- documenting multiselect() as the multiple-choice family
- raising the enforced floor from 34% to 39%
- raising the next declared target from 39% to 44%

It does **not** include:

- full form-group or wizard coverage

- replacing the docs app with live interactive form execution
- changing the canonical denominator
- pretending DOGFOOD now covers the whole form system

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach real input and decision primitives instead of only display, feedback, and docs-reference surfaces.

They need:

- one story that shows confirm() as a real binary-decision prompt with honest defaults and consequences
- one story that shows multiselect() as a real set-building flow with keyboard toggling and fallback semantics
- coverage progress that rises only because those families are genuinely present

Human hill

A user can open DOGFOOD, learn when to use confirm() and multiselect(), and see the coverage number rise because the field guide now actually covers those decision/input families.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without adding shallow filler stories.

It needs:

- a clear smallest honest route from 34% to 39%
- tests that prove which decision/input families were added
- one integration check that the new form stories are reachable through the docs app itself

Agent hill

An agent can raise DOGFOOD to the 39% floor only by adding real confirm() and multiselect() stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new confirm() story.
2. They can see a real binary-decision prompt with an explicit default and concrete yes/no consequences.
3. They open the new multiselect() story and see a keyboard-owned set-selection flow with toggled options and truthful fallback guidance.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-009 cycle doc.
2. It can see that the target floor is 39% and that the intended route is the two uncovered form families confirm() and multiselect().
3. It adds real DOGFOOD stories for those families.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 43%, which safely clears the 39% floor, and the next target is recorded as 44%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- confirm() coverage must stay a genuine binary-decision story instead of becoming a hidden multi-option flow
- multiselect() coverage must stay about set-building, not command dispatch disguised as checkbox rows
- the new families must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-009 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:

    - 15 of 35 documented families and 43% real coverage
    - coverage of Multiple choice and Binary decision
    - an enforced floor of 39 with a next target of 44
    - real confirm() and multiselect() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-009/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 15 of 35 families and 43%
- the story catalog now covers Multiple choice and Binary decision
- the enforced floor is 39 and the next target is 44
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new confirm() story is reachable in component search and renders recognizable binary-decision content

Risks

- making the stories look like fake diagrams instead of truthful prompt snapshots
- making the multiselect story read like a generic checklist instead of a real set-building flow
- documenting forms in a way that overpromises interactivity that DOGFOOD does not actually execute inside the preview pane

Out of scope

- group() and wizard()
- live embedded form execution inside DOGFOOD
- shell redesign
- complete DOGFOOD coverage

Retrospective

What landed

- new real confirm() DOGFOOD story teaching explicit binary decisions with honest defaults
- new real multiselect() DOGFOOD story teaching set-building prompts with keyboard toggling and truthful textual fallbacks
- canonical family coverage increased from 13/35 to 15/35
- DOGFOOD docs coverage now resolves to 43%, which safely clears the 39% floor
- the enforced floor moved from 34% to 39%
- the next declared target moved from 39% to 44%

Drift from ideal

No material drift.

The cycle stayed focused on genuine form and decision families instead of adding another passive display surface, which makes DOGFOOD more credible as a guide to actual interaction patterns.

Debt spawned

Spawned:

- DF-010 — Raise DOGFOOD Coverage Floor to 44%

70

DF-010 — Raise DOGFOOD Coverage Floor to 44%

docs/design/DF-010-raise-dogfood-coverage-floor-to-44-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-009 — Raise DOGFOOD Coverage Floor to 39%

Why this cycle exists

DF-009 made DOGFOOD more credible as a guide to real decision flows by documenting confirm() and multiselect().

The next promised step is now explicit:

- DOGFOOD currently documents 15 of 35 component families
- the enforced floor is 39%
- the next declared target is 44%

This cycle exists to earn that next target through one more honest input family and one more honest content-organization family:

- select() / filter() for **Single choice**
- tabs() for **Peer navigation**

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 15 to 17 families
- documenting select() / filter() as the **Single choice** family
- documenting tabs() as the **Peer navigation** family
- raising the enforced floor from 39% to 44%
- raising the next declared target from 44% to 49%

It does **not** include:

- full text-entry coverage
- stepper/breadcrumb/paginator/path doctrine work
- live embedded form execution inside DOGFOOD
- changing the canonical denominator

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach real one-choice prompts and peer navigation instead of stopping at multi-choice forms and passive display surfaces.

They need:

- one story that explains when to use select() versus filter() without pretending they are different family headings
- one story that explains tabs() as peer navigation instead of progress or hierarchy
- coverage progress that rises only because those family headings are genuinely represented

Human hill

A user can open DOGFOOD, learn when to use select() / filter() and tabs(), and trust the higher coverage percentage because those new families are actually present in the field guide.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without gaming the denominator or adding shallow filler.

It needs:

- the smallest honest route from 39% to 44%
- tests that prove which new family headings were added
- a clear rule that one family story can represent multiple related primitives when the canonical family heading is shared

Agent hill

An agent can raise DOGFOOD to the 44% floor only by adding real select() / fi lter() and tabs() stories, and the cycle tests prove the coverage increase, the new family ids, and the next target.

Human playback

1. A user enters DOGFOOD and opens the new select() / filter() story.
2. They can see one variant where a visible list resolves to one selected value and another where search narrows a larger one-choice set.
3. They open the new tabs() story and see peer sections that share one workspace without implying progress or hierarchy.
4. The docs coverage card reports a higher percentage that corresponds to those real new family headings.

Agent playback

1. An agent opens the DF-010 cycle doc.
2. It can see that the target floor is 44% and that the intended route is sing le-choice plus peer-navigation.
3. It adds real DOGFOOD stories for those families.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 49%, which safely clears the 44% floor, and the next target is recorded as 49%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- select() / filter() coverage must stay about choosing one durable value, not command dispatch
- tabs() coverage must stay about peer navigation, not progress or hierarchy
- the new family headings must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-010 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:
   - 17 of 35 documented families and 49% real coverage

    - coverage of Single choice and Peer navigation
    - an enforced floor of 44 with a next target of 49
    - real select() / filter() and tabs() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-010/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 17 of 35 families and 49%
- the story catalog now covers Single choice and Peer navigation
- the enforced floor is 44 and the next target is 49
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new tabs() story is reachable in component search and renders recognizable peer-navigation content

Risks

- making the select() / filter() story feel like a fake embedded terminal instead of an honest reference snapshot
- making tabs() read like steps or breadcrumbs instead of sibling section switching
- raising the floor without making the new family headings actually teach anything useful

Out of scope

- input()
- wizard() and group()
- stepper/paginator/breadcrumb coverage
- complete DOGFOOD coverage

Retrospective

What landed

- a new real select() / filter() DOGFOOD story teaching visible-list and searchable single-choice prompts under one honest family heading

- a new real tabs() DOGFOOD story teaching peer navigation without confusing it with progress or hierarchy
- canonical family coverage increased from 15/35 to 17/35
- DOGFOOD docs coverage now resolves to 49%, which safely clears the 44% floor
- the enforced floor moved from 39% to 44%
- the next declared target moved from 44% to 49%

Drift from ideal

No material drift.

The cycle stayed disciplined about family honesty: select() and filter() share one family heading, so the field guide teaches them together instead of pretending they each justify a separate ratchet increment.

Debt spawned

Spawned:

- DF-011 — Raise DOGFOOD Coverage Floor to 49%

71

DF-011 — Raise DOGFOOD Coverage Floor to 49%

docs/design/DF-011-raise-dogfood-coverage-floor-to-49-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-010 — Raise DOGFOOD Coverage Floor to 44%

Why this cycle exists

DF-010 made DOGFOOD more credible around one-choice prompts and peer navigation.

The next promised step is now explicit:

- DOGFOOD currently documents 17 of 35 component families
- the enforced floor is 44%
- the next declared target is 49%

This cycle exists to earn that next target through two deeper form families that move DOGFOOD closer to real application-building guidance instead of stopping at one-off prompts:

- input() / textarea() for **Text entry**
- group() / wizard() for **Multi-field and staged forms**

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 17 to 19 families
- documenting input() / textarea() as the **Text entry** family
- documenting group() / wizard() as the **Multi-field and staged forms** family
- raising the enforced floor from 44% to 49%
- raising the next declared target from 49% to 54%

It does **not** include:

- live embedded text-entry execution inside DOGFOOD
- every remaining form-adjacent family like breadcrumbs, steppers, or progress trackers
- changing the canonical denominator
- pretending DOGFOOD now covers the whole input stack

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach richer data-entry flows instead of stopping at choice prompts and passive documentation surfaces.

They need:

- one story that shows input() and textarea() as honest short-form versus multiline text-entry work
- one story that shows group() and wizard() as related-input flows with grouping or step progression
- coverage progress that rises only because those deeper form families are genuinely present

Human hill

A user can open DOGFOOD, learn when to use input() / textarea() and group() / wizard(), and trust the higher coverage number because those families are actually represented in the field guide.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without gaming the denominator or filling the guide with shallow visual-only surfaces.

It needs:

- a clear smallest honest route from 44% to 49%
- tests that prove which new form families were added
- at least one docs-app integration check that the new stories are reachable through component search

Agent hill

An agent can raise DOGFOOD to the 49% floor only by adding real text-entry and staged-form stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new input() / textarea() story.
2. They can see one short single-line entry example and one multiline text-entry example, with prompt and helper text staying explicit.
3. They open the new group() / wizard() story and can see both a grouped form snapshot and a staged step with explicit progress labeling.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-011 cycle doc.
2. It can see that the target floor is 49% and that the intended route is text-entry plus multi-field-and-staged-forms.
3. It adds real DOGFOOD stories for those families.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 54%, which safely clears the 49% floor, and the next target is recorded as 54%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- input() / textarea() coverage must stay about text entry, not fake static prose dressed up as a field
- group() / wizard() coverage must keep grouping and step intent explicit instead of collapsing into one unstructured box
- the new families must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-011 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:
   - 19 of 35 documented families and 54% real coverage
   - coverage of Text entry and Multi-field and staged forms
   - an enforced floor of 49 with a next target of 54
   - real input() / textarea() and group() / wizard() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-011/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 19 of 35 families and 54%
- the story catalog now covers Text entry and Multi-field and staged forms
- the enforced floor is 49 and the next target is 54
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new group() / wizard() story is reachable in component search and renders recognizable staged-form content

Risks

- making the text-entry story feel like a decorative text card instead of an honest field snapshot
- making the staged-form story look like a progress widget instead of a real grouped or stepwise input flow
- raising the floor without enough actual teaching value for builders evaluating form primitives

Out of scope

- live embedded editors inside DOGFOOD
- stepper, breadcrumb, or paginator coverage
- complete DOGFOOD coverage

Retrospective

What landed

- a new real input() / textarea() DOGFOOD story teaching single-line and multiline text entry under one honest family heading
- a new real group() / wizard() DOGFOOD story teaching grouped and staged form flows
- canonical family coverage increased from 17/35 to 19/35
- DOGFOOD docs coverage now resolves to 54%, which safely clears the 49% floor
- the enforced floor moved from 44% to 49%
- the next declared target moved from 49% to 54%

Drift from ideal

No material drift.

The cycle stayed disciplined about family honesty: the field guide teaches paired primitives under their shared family headings instead of pretending every export deserves its own ratchet increment.

Debt spawned

Spawned:

- DF-012 — Raise DOGFOOD Coverage Floor to 54%

72

DF-012 — Raise DOGFOOD Coverage Floor to 54%

docs/design/DF-012-raise-dogfood-coverage-floor-to-54-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-011 — Raise DOGFOOD Coverage Floor to 49%

Why this cycle exists

DF-011 made DOGFOOD more credible around text entry plus grouped and staged form flows.

The next promised step is now explicit:

- DOGFOOD currently documents 19 of 35 component families
- the enforced floor is 49%
- the next declared target is 54%

This cycle exists to earn that next target through two calmer structural teaching surfaces that make the field guide feel broader and more production-like instead of only deeper on forms:

- explainability() for **Explainability walkthroughs**
- separator() / separatorSurface() for **Dividers**

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 19 to 21 families
- documenting explainability() as the **Explainability walkthroughs** family
- documenting separator() / separatorSurface() as the **Dividers** family
- raising the enforced floor from 49% to 54%
- raising the next declared target from 54% to 59%

It does **not** include:

- a full AI-docs or assistant workflow inside DOGFOOD
- every remaining structure/navigation family like breadcrumbs, timelines, or disclosure stacks
- changing the canonical denominator
- pretending DOGFOOD now covers the full shell and block language

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach calmer structural and guided-flow surfaces instead of leaning too heavily on prompts, forms, and feedback widgets.

They need:

- one story that shows explainability() as a real recommendation surface with visible provenance, evidence, and next-action structure
- one story that shows dividers as real section rhythm rather than decorative lines
- coverage progress that rises only because those structural families are genuinely present

Human hill

A user can open DOGFOOD, learn when to use explainability() and when to use s eparator() / separatorSurface(), and trust the higher coverage number because those families are actually represented in the field guide.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without turning the field guide into a one-note catalog of form primitives.

It needs:

- a clear smallest honest route from 49% to 54%
- tests that prove which new structural families were added
- at least one docs-app integration check that the new stories are reachable through component search

Agent hill

An agent can raise DOGFOOD to the 54% floor only by adding real explainability and divider stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new explainability() story.
2. They can see one calm guided recommendation surface with visible provenance, evidence, next action, and governance text.
3. They open the new separator() story and can see both labeled and unlabeled section rhythm that separates real sections without over-boxing everything.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-012 cycle doc.
2. It can see that the target floor is 54% and that the intended route is expl ainability-walkthroughs plus dividers.
3. It adds real DOGFOOD stories for those families.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 60%, which safely clears the 54% floor, and the next target is recorded as 59%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- explainability() coverage must show provenance, evidence, and next-action structure instead of a vague summary card
- divider coverage must show real section rhythm, not decorative line spam
- the new families must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-012 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:
   - 21 of 35 documented families and 60% real coverage
   - coverage of Explainability walkthroughs and Dividers
   - an enforced floor of 54 with a next target of 59
   - real explainability() and separator() / separatorSurface() stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-012/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 21 of 35 families and 60%
- the story catalog now covers Explainability walkthroughs and Dividers
- the enforced floor is 54 and the next target is 59
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new explainability() story is reachable in component search and renders recognizable provenance and evidence content

Risks

- making the explainability story feel like generic marketing prose instead of a recommendation surface
- making the divider story feel like visual filler instead of honest section rhythm
- raising the floor without enough actual teaching value for builders evaluating calmer structural patterns

Out of scope

- full AI workflow or assistant tooling inside DOGFOOD
- breadcrumbs, timelines, or disclosure-tree coverage
- complete DOGFOOD coverage

Retrospective

What landed

- a new real explainability() DOGFOOD story teaching visible provenance, evidence, governance, and next-action structure
- a new real separator() / separatorSurface() DOGFOOD story teaching labeled and unlabeled section rhythm
- a dedicated examples/explainability/main.ts example so the story points at real source instead of an invented snippet path
- canonical family coverage increased from 19/35 to 21/35
- DOGFOOD docs coverage now resolves to 60%, which safely clears the 54% floor
- the enforced floor moved from 49% to 54%
- the next declared target moved from 54% to 59%

Drift from ideal

No material drift.

The cycle stayed disciplined about broadening DOGFOOD's teaching surface instead of just adding more form primitives: the field guide now covers guided recommendation cards and calmer section boundaries without inventing a new denominator or overclaiming completeness.

Debt spawned

Spawned:

- DF-013 — Raise DOGFOOD Coverage Floor to 59%

73

DF-013 — Raise DOGFOOD Coverage Floor to 59%

docs/design/DF-013-raise-dogfood-coverage-floor-to-59-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-012 — Raise DOGFOOD Coverage Floor to 54%

Why this cycle exists

DF-012 broadened DOGFOOD with calmer structural teaching surfaces through explainability and dividers.

The next promised step is now explicit:

- DOGFOOD currently documents 21 of 35 component families
- the enforced floor is 54%
- the next declared target is 59%

This cycle exists to earn that next target through two shell-facing TUI families that make the field guide feel more like a whole application environment instead of only a catalog of component primitives:

- helpView() / helpShortSurface() for **Keybinding help and shell hints**
- renderNotificationStack() / renderNotificationHistorySurface() for **Notification system**

Scope of this cycle

This cycle intentionally covers:

- adding enough real DOGFOOD component stories to move coverage from 21 to 23 families
- documenting keybinding help and shell hints through the real help-view APIs
- documenting the notification system through the real stack and history APIs
- raising the enforced floor from 54% to 59%
- raising the next declared target from 59% to 64%

It does **not** include:

- covering the whole app shell or workspace-layout families yet
- a full interactive notification lab inside DOGFOOD
- changing the canonical denominator
- pretending DOGFOOD now fully documents every shell surface

Human users

Primary human user

A builder evaluating whether DOGFOOD can now teach richer shell-owned TUI patterns instead of stopping at individual widgets and forms.

They need:

- one story that shows full grouped help plus compact shell hints through the actual help APIs
- one story that shows transient notification stacking alongside archived review/history through the actual notification-system APIs
- coverage progress that rises only because those shell-facing families are genuinely present

Human hill

A user can open DOGFOOD, learn when to use the help-view family and when to use the notification system family, and trust the higher coverage number because those families are actually represented in the field guide.

Agent users

Primary agent user

An agent tasked with continuing the DOGFOOD ratchet without turning the field guide into a one-note list of static display primitives.

It needs:

- a clear smallest honest route from 54% to 59%
- tests that prove which shell-facing families were added
- at least one docs-app integration check that the new stories are reachable through component search

Agent hill

An agent can raise DOGFOOD to the 59% floor only by adding real help and notification-system stories, and the cycle tests prove both the coverage increase and the new floor policy.

Human playback

1. A user enters DOGFOOD and opens the new help-view story.
2. They can see one compact hint strip and one grouped keybinding reference that clearly teach scope and grouping.
3. They open the new notification-system story and can see both live stacked notifications and an archived notification-history review surface.
4. The docs coverage card reports a higher percentage that corresponds to those real new stories.

Agent playback

1. An agent opens the DF-013 cycle doc.
2. It can see that the target floor is 59% and that the intended route is keyb inding-help-and-shell-hints plus notification-system.
3. It adds real DOGFOOD stories for those families.
4. It updates the floor policy only after those stories exist.
5. The cycle tests and gate confirm that coverage is now 66%, which safely clears the 59% floor, and the next target is recorded as 64%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Invariants for this cycle

- the floor must move only after real coverage moves
- help coverage must use the actual help APIs and keep grouped reference distinct from short shell hints
- notification-system coverage must show real system behavior like stack/history, not just a single toast primitive
- the new families must be backed by real DOGFOOD stories
- the denominator must still come from the canonical component-family reference

Implementation outline

1. Move DF-013 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that demand:

    - 23 of 35 documented families and 66% real coverage
    - coverage of Keybinding help and shell hints and Notification system
    - an enforced floor of 59 with a next target of 64
    - real help and notification-system stories in DOGFOOD
3. Add the smallest honest DOGFOOD stories needed to hit the target.
4. Raise the floor policy and spawn the next ratchet backlog item.
5. Close the cycle with retrospective notes describing what actually landed.

Tests to write first

Under tests/cycles/DF-013/:

- the active cycle doc includes the required workflow sections
- DOGFOOD coverage resolves to 23 of 35 families and 66%
- the story catalog now covers Keybinding help and shell hints and Notificati on system
- the enforced floor is 59 and the next target is 64
- the next DOGFOOD backlog item exists

And in the DOGFOOD preview integration bar:

- the new help story is reachable in component search and renders recognizable grouped-help content

Risks

- making the help story feel like a generic text card instead of a keyboard-owned reference surface
- making the notification story show only one toast instead of a real system with stack/history semantics
- raising the floor without enough actual shell-teaching value for builders evaluating TUI application behavior

Out of scope

- full app-shell coverage
- workspace-layout coverage
- complete DOGFOOD coverage

Retrospective

What landed

- a new real help story teaching helpView() and helpShortSurface() under the Keybinding help and shell hints family
- a new real notification-system story teaching stacked live notifications and archived notification history through the actual TUI notification APIs
- canonical family coverage increased from 21/35 to 23/35
- DOGFOOD docs coverage now resolves to 66%, which safely clears the 59% floor
- the enforced floor moved from 54% to 59%
- the next declared target moved from 59% to 64%

Drift from ideal

No material drift.

The cycle stayed focused on shell-facing teaching value rather than simply adding more visual primitives. DOGFOOD now explains both shortcut discovery and recallable transient messaging through the same real APIs builders would use in product code.

Debt spawned

Spawned:

- DF-014 — Raise DOGFOOD Coverage Floor to 64%

74

DF-014 — Raise DOGFOOD Coverage Floor to 64%

docs/design/DF-014-raise-dogfood-coverage-floor-to-64-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-013 — Raise DOGFOOD Coverage Floor to 59%

Why this cycle exists

DF-013 proved that DOGFOOD could teach shell help and the notification system honestly.

The next promised step was to keep the ratchet moving by documenting the shell frame itself instead of only shell-adjacent surfaces:

- createFramedApp() / statusBarSurface() / commandPaletteSurface() for **App shell**
- splitPaneSurface() / gridSurface() for **Workspace layout**

Scope of this cycle

This cycle intentionally covers:

- adding the app-shell family to DOGFOOD
- adding the workspace-layout family to DOGFOOD
- moving the field guide from 23/35 families to at least 25/35
- clearing the declared 64% floor

It does **not** include:

- progressive disclosure or path/progress teaching
- data-browsing families
- final DOGFOOD completion

Human users

Primary human user

A builder deciding whether Bijou can teach whole application framing and spatial composition, not only individual widgets.

Human hill

A user can open DOGFOOD and understand when to use shell chrome versus page content, and when split/grid layout is honest instead of decorative geometry.

Agent users

Primary agent user

An agent continuing the coverage ratchet without padding the numerator with cosmetic stories.

Agent hill

An agent can only satisfy DF-014 by adding real app-shell and workspace-layout stories and by proving the floor moved because those families are now actually present.

Human playback

1. A user opens a new shell story and sees framed app chrome plus command discovery.
2. They open a workspace-layout story and see split and grid composition with clearly named regions.
3. The DOGFOOD coverage card rises because those two families are now real.

Agent playback

1. An agent reads DF-014 and sees the target pair: app shell plus workspace layout.
2. It adds real stories using the shipped shell and layout APIs.
3. The coverage tests confirm that DOGFOOD now clears the 64% floor.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Implementation outline

1. Move DF-014 into the active cycle area.
2. Add real app-shell and workspace-layout stories.
3. Raise the ratchet only after those families exist.
4. Spawn the next DOGFOOD cycle.

Tests to write first

Under tests/cycles/DF-014/:

- the cycle doc has the required workflow sections
- DOGFOOD preserves app-shell and workspace-layout coverage
- the floor is at least 64 with the next target at least 69
- the next DOGFOOD cycle exists

Retrospective

What landed

- DOGFOOD now teaches createFramedApp() / statusBarSurface() / commandPaletteSurface() under the **App shell** family
- DOGFOOD now teaches splitPaneSurface() / gridSurface() under the **Workspace layout** family
- later cycles pushed the floor higher, but this slice remains one honest ratchet step in that larger climb

Drift from ideal

No material drift.

Debt spawned

Spawned:

- DF-015 — Raise DOGFOOD Coverage Floor to 69%

75

DF-015 — Raise DOGFOOD Coverage Floor to 69%

docs/design/DF-015-raise-dogfood-coverage-floor-to-69-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-014 — Raise DOGFOOD Coverage Floor to 64%

Why this cycle exists

After shell and layout coverage, the next missing gap was navigation structure:

- accordion() / interactiveAccordion() for **Progressive disclosure**
- breadcrumb() / stepper() / paginator() for **Path and progress**

Scope of this cycle

This cycle intentionally covers:

- progressive disclosure
- path and progress
- moving DOGFOOD from 25/35 to at least 27/35 families
- clearing the declared 69% floor

It does **not** include data-browsing or authoring families yet.

Human users

Primary human user

A builder choosing between peers, steps, disclosure, and wayfinding patterns inside a real terminal product.

Human hill

A user can see both summary-first disclosure and explicit path/progress patterns in DOGFOOD without inferring them from prose alone.

Agent users

Primary agent user

An agent continuing the ratchet through real navigation-oriented stories.

Agent hill

An agent can only satisfy DF-015 by adding real progressive-disclosure and path/progress coverage, not by relabeling an existing story.

Human playback

1. A user opens a disclosure story and sees summary-first sections plus the keyboard-owned accordion path.
2. They open a path/progress story and see breadcrumbs, steppers, and pagination patterns in one bounded field-guide surface.

Agent playback

1. An agent reads the target families for DF-015.
2. It adds those stories and raises the floor only after they exist.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo

Implementation outline

1. Add real navigation-organization stories.
2. Prove the coverage increase in tests.
3. Spawn the next cycle.

Tests to write first

Under tests/cycles/DF-015/:

- the cycle doc has the required workflow sections
- the story catalog preserves progressive-disclosure and path/progress coverage
- the floor is at least 69 with the next target at least 74
- the next cycle exists

Retrospective

What landed

- DOGFOOD now teaches progressive disclosure through accordion() and interactiveAccordion()
- DOGFOOD now teaches path and progress through breadcrumb(), stepper(), and paginator()

Drift from ideal

No material drift.

Debt spawned

Spawned:

- DF-016 — Raise DOGFOOD Coverage Floor to 74%

76

DF-016 — Raise DOGFOOD Coverage Floor to 74%

docs/design/DF-016-raise-dogfood-coverage-floor-to-74-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-015 — Raise DOGFOOD Coverage Floor to 69%

Why this cycle exists

Once navigation patterns were covered, the next missing field-guide gap was straightforward browsing:

- table() / navigableTableSurface() for **Dense comparison**
- enumeratedList() / browsableListSurface() for **Lists for exploration**

Scope of this cycle

This cycle intentionally covers:

- dense comparison
- lists for exploration
- moving DOGFOOD from 27/35 to at least 29/35 families
- clearing the declared 74% floor

Human users

Primary human user

A builder deciding whether a record should be a table, an exploration list, or something more structural.

Human hill

A user can see the difference between comparison-oriented tables and scan-first exploration lists by opening the stories directly.

Agent users

Primary agent user

An agent continuing the coverage ratchet through record-browsing surfaces.

Agent hill

An agent can only satisfy DF-016 by adding real dense-comparison and exploration-list stories backed by the shipped APIs.

Human playback

1. A user opens the dense-comparison story and sees both passive and focused table inspection.
2. They open the list-exploration story and see passive list reading plus browsable record inspection.

Agent playback

1. An agent reads DF-016 and sees the target families.
2. It adds real stories for table/list browsing.
3. The coverage floor clears the 74% threshold.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo

Implementation outline

1. Add real data-browsing stories.
2. Prove the new family ids in tests.
3. Spawn the next cycle.

Tests to write first

Under tests/cycles/DF-016/:

- the cycle doc has the required workflow sections
- DOGFOOD preserves dense-comparison and exploration-list coverage
- the floor is at least 74 with the next target at least 79
- the next cycle exists

Retrospective

What landed

- DOGFOOD now teaches dense comparison through table() and navigableTableSurface()
- DOGFOOD now teaches lists for exploration through enumeratedList() and browsableListSurface()

Drift from ideal

No material drift.

Debt spawned

Spawned:

- DF-017 — Raise DOGFOOD Coverage Floor to 79%

77

DF-017 — Raise DOGFOOD Coverage Floor to 79%

docs/design/DF-017-raise-dogfood-coverage-floor-to-79-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-016 — Raise DOGFOOD Coverage Floor to 74%

Why this cycle exists

The field guide still lacked structural browsing families:

- tree() / filePickerSurface() for **Hierarchy**
- timeline() / dag() for **Temporal or dependency views**

Scope of this cycle

This cycle intentionally covers:

- hierarchy
- temporal or dependency views
- moving DOGFOOD from 29/35 to at least 31/35 families
- clearing the declared 79% floor

Human users

Primary human user

A builder trying to decide when to show a tree, a file browser, a timeline, or a dependency graph.

Human hill

A user can open DOGFOOD and directly compare hierarchy-oriented and time/dependency-oriented teaching surfaces.

Agent users

Primary agent user

An agent broadening DOGFOOD without flattening every remaining family into one generic “advanced views” card.

Agent hill

An agent can only satisfy DF-017 by adding real hierarchy and temporal/dependency stories that use the shipped primitives.

Human playback

1. A user opens the hierarchy story and sees both passive tree structure and a file-browser snapshot.
2. They open the temporal/dependency story and see both chronology and causal structure.

Agent playback

1. An agent reads the target families for DF-017.
2. It adds real hierarchy and temporal/dependency stories.
3. The coverage floor clears 79%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo

Implementation outline

1. Add hierarchy and temporal/dependency stories.
2. Prove the new families in tests.
3. Spawn the next cycle.

Tests to write first

Under tests/cycles/DF-017/:

- the cycle doc has the required workflow sections
- DOGFOOD preserves hierarchy and temporal/dependency coverage
- the floor is at least 79 with the next target at least 84
- the next cycle exists

Retrospective

What landed

- DOGFOOD now teaches tree() / filePickerSurface() under **Hierarchy**
- DOGFOOD now teaches timeline() / dag() under **Temporal or dependency views**

Drift from ideal

No material drift.

Debt spawned

Spawned:

- DF-018 — Raise DOGFOOD Coverage Floor to 84%

78

DF-018 — Raise DOGFOOD Coverage Floor to 84%

docs/design/DF-018-raise-dogfood-coverage-floor-to-84-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-017 — Raise DOGFOOD Coverage Floor to 79%

Why this cycle exists

With the major browsing families in place, DOGFOOD still needed to prove Bijou's expressive edge:

- loadRandomLogo() / gradientText() for **Expressive branding and decorative emphasis**
- canvas() for **Motion and shader effects**

Scope of this cycle

This cycle intentionally covers:

- expressive branding
- motion and shader effects
- moving DOGFOOD from 31/35 to at least 33/35 families
- clearing the declared 84% floor

Human users

Primary human user

A builder evaluating whether Bijou can teach expressive and animated surfaces honestly instead of treating them as decorative afterthoughts.

Human hill

A user can see how branding and motion belong in Bijou without confusing those surfaces for everyday productivity chrome.

Agent users

Primary agent user

An agent filling remaining coverage gaps without overusing spectacle or inventing fake browser-only effects.

Agent hill

An agent can only satisfy DF-018 by adding real branding and shader stories with honest non-motion lowerings.

Human playback

1. A user opens the branding story and sees deliberate but bounded emphasis.
2. They open the motion story and see shader-driven visual output with clear reduced-fidelity lowerings.

Agent playback

1. An agent reads DF-018 and sees the two expressive families still missing.
2. It adds those stories.
3. The coverage floor clears 84%.

Linked invariants

- Tests Are the Spec
- Docs Are the Demo

Implementation outline

1. Add the branding and motion stories.
2. Verify the families are present.
3. Spawn the final DOGFOOD completion cycle.

Tests to write first

Under tests/cycles/DF-018/:

- the cycle doc has the required workflow sections
- DOGFOOD preserves expressive-branding and motion/shader coverage
- the floor is at least 84 with the next target at least 89
- the next cycle exists

Retrospective

What landed

- DOGFOOD now teaches rare brand emphasis through loadRandomLogo() and gradientText()
- DOGFOOD now teaches deliberate shader/motion surfaces through canvas()

Drift from ideal

No material drift.

Debt spawned

Spawned:

- DF-019 — Raise DOGFOOD Coverage Floor to 100%

79

DF-019 — Raise DOGFOOD Coverage Floor to 100%

docs/design/DF-019-raise-dogfood-coverage-floor-to-100-percent.md

Legend:

- DF — DOGFOOD Field Guide

Depends on:

- DF-018 — Raise DOGFOOD Coverage Floor to 84%

Why this cycle exists

The final remaining DOGFOOD gap was the last two authoring/feedback families:

- pushNotification() / renderNotificationStack() for **Transient app notifications**
- renderByMode() for **Mode-aware custom primitives**

Landing those two families completes the current component-family map:

- 35/35 families documented
- 100% component-family coverage
- the coverage floor saturates at 100% because there is no higher honest target

Scope of this cycle

This cycle intentionally covers:

- transient app notifications
- mode-aware custom primitives
- full DOGFOOD family coverage
- saturating the coverage floor and next target at 100%

It does **not** promise that every family already has maximum depth or every imaginable variant story.

Human users

Primary human user

A builder evaluating whether DOGFOOD has crossed from “credible partial field guide” to “complete component-family map.”

Human hill

A user can browse DOGFOOD and trust that every currently documented component family in the canonical reference is represented by a real story.

Agent users

Primary agent user

An agent closing out the coverage ratchet honestly, with no placeholder or denominator games.

Agent hill

An agent can only satisfy DF-019 by making resolveDogfoodDocsCoverage(COMPONENT_STORIES) return 35/35 and by proving the final two families are present.

Human playback

1. A user opens the transient-app-notifications story and sees live app-owned transient notices distinct from the broader notification-system history story.
2. They open the mode-aware custom primitive story and see one semantic primitive lowering honestly across modes.
3. The DOGFOOD coverage card reports full family coverage.

Agent playback

1. An agent reads DF-019 and sees the final two missing families.
2. It adds those stories.
3. The coverage gate saturates at 100%.

Linked invariants

- Tests Are the Spec

- Docs Are the Demo
- Graceful Lowering Preserves Meaning

Implementation outline

1. Add the final two missing stories.
2. Prove DOGFOOD now covers every component family in the canonical reference.
3. Saturate the floor and next target at 100%.
4. Spawn a non-ratchet follow-on backlog item for deepening quality and variants.

Tests to write first

Under tests/cycles/DF-019/:

- the cycle doc has the required workflow sections
- DOGFOOD resolves to 35/35 documented families and 100%
- the final two family ids are present
- the floor and next target are both 100%
- the next non-ratchet DOGFOOD backlog item exists

Retrospective

What landed

- DOGFOOD now teaches app-owned transient notification behavior distinctly from the broader notification-system history surface
- DOGFOOD now teaches mode-aware custom primitives through renderByMode()
- the story catalog resolves to 35/35 families and 100% coverage
- the coverage floor and next target both saturate at 100%

Drift from ideal

No material drift.

DOGFOOD now covers the whole component-family map. The remaining work is depth, sharper examples, and better variant richness rather than missing families.

Debt spawned

Spawned:

- DF-020 — Deepen DOGFOOD Story Depth and Variant Quality

80

DF-021 — Shape DOGFOOD As Terminal Docs System

docs/design/DF-021-shape-dogfood-as-terminal-docs-system.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-019 — Raise DOGFOOD Coverage Floor to 100%
- WF-004 — Shape The Next Release

Sponsor human

A builder or maintainer who expects DOGFOOD to behave like Bijou's terminal-native documentation site, not only a component-story browser.

Sponsor agent

An agent that must answer, from repo artifacts alone:

- what DOGFOOD is supposed to document
- which parts of the repo must be reachable inside DOGFOOD before 4.1.0 ships
- which missing docs surfaces are now release blockers

Hill

DOGFOOD is shaped as a terminal documentation system for Bijou, with the current component-library explorer treated as one important section inside a broader docs site rather than the whole product.

Why this cycle exists

DOGFOOD has crossed an important threshold:

- all 35/35 canonical component families are now represented
- the shell and landing surfaces are strong enough to feel like a real product
- the docs app is already the proving surface for Bijou's shell, design-language, and component work

But the scope is still too narrow for what the repo now claims DOGFOOD to be.

Today, DOGFOOD is still mostly a **components docs view**:

- it has a landing screen
- it has a component-family explorer
- it can show component story docs and variants
- it does **not** yet act like a terminal-native documentation site for the repo as a whole

That gap is now release-shaping:

- there is no real viewer for prose docs from docs/
- there is no DOGFOOD section for repo/package documentation
- there is no DOGFOOD section for Bijou's philosophy, architecture, and doctrine docs
- the current shell structure still assumes "components" is the whole docs product

If 4.1.0 ships while DOGFOOD remains component-only, Bijou will still have a strong component field guide, but it will not yet have the terminal docs surface that the repo increasingly implies exists.

Human users

Primary human user

A builder evaluating Bijou as a framework, not just as a pile of widgets.

Human hill

A user can open DOGFOOD and browse:

- what Bijou is
- how it is organized
- what each package is for
- how the component library works
- what philosophical and architectural ideas shape the repo

without leaving the terminal to read the main docs corpus.

Agent users

Primary agent user

An agent tasked with extending DOGFOOD, importing docs, or explaining Bijou's public surface to users.

Agent hill

An agent can recover the expected DOGFOOD information architecture from repo docs and can see exactly which missing docs surfaces block 4.1.0.

Playback questions

1. If a user asks "where do I read real Bijou docs in the terminal?", does DOGFOOD have a clear answer beyond component stories?
2. Can a user browse both the component library and the non-component docs corpus without the app pretending they are the same thing?
3. Can a user find package-level explanations for @flyingrobots/bijou, @flying robots/bijou-tui, the i18n packages, and the app template?
4. Can a user read Bijou's philosophy, architecture, and doctrine docs from inside DOGFOOD?
5. Can an agent point to explicit 4.1.0 blocker items for those gaps?

Accessibility / assistive reading posture

This cycle is prose-heavy. The docs shell must preserve legible reading in plain text, keyboard navigation, and scrollable surfaces that do not depend on screenshots, pointer-only interaction, or color alone to communicate structure.

Localization / directionality posture

The docs shell should stay localization-ready, but this cycle does not block 4.1.0 on translating the entire imported prose corpus. The important requirement is that section metadata and reader structure do not hard-code LTR-only assumptions.

Agent inspectability / explainability posture

The DOGFOOD docs-site shape must be explicit in repo artifacts:

- this cycle doc
- release blocker backlog files in docs/method/graveyard/legacy-backlog/v4.1.0/
- signposts such as PLAN.md, BEARING.md, and release.md

Agents should not have to infer from chat memory that DOGFOOD is supposed to grow beyond the component explorer.

Non-goals

- importing every markdown file in the repo into DOGFOOD for 4.1.0
- full website parity with static docs tools
- full-text search across every prose page
- a general GSAP-like text-effects API for the entire framework
- eliminating every piece of bespoke DOGFOOD chrome before 4.1.0

The immediate requirement is a real docs-site shape and a non-embarrassing minimum corpus, not infinite scope growth.

Evidence

Current DOGFOOD shape

- DOGFOOD already has a strong landing screen and framed docs shell
- the current explorer is fundamentally centered on component families, component docs, and variants
- the existing app still treats that component explorer as the main docs product

Current repo docs shape

Bijou's actual docs corpus already exists outside DOGFOOD:

- docs/ARCHITECTURE.md
- docs/system-style-javascript.md
- docs/METHOD.md
- docs/VISION.md
- docs/release.md
- docs/releases/4.1.0/
- package READMEs and repo-facing top-level docs

So the missing work is not "write every thought from scratch." The missing work is exposing the right corpus and navigation structure inside DOGFOOD itself.

Decision

DOGFOOD's 4.1.0 role

For 4.1.0, DOGFOOD should be treated as a **terminal documentation site for Bijou**, not only a component gallery.

Expected top-level sections

At minimum, DOGFOOD should visibly distinguish these sections:

- **Start Here / Guides**
  - what Bijou is
  - how to get oriented
  - how to start building
- **Components**
  - the current family/story/variant explorer
- **Packages**
  - what each shipped package is for
  - when to reach for each one
- **Philosophy / Architecture**
  - design doctrine, architecture, and repo philosophy
- **Release / Migration**
  - what changed in the current line
  - how to move between major versions

The current components view remains important, but it becomes one section of the docs system instead of the whole app.

4.1.0 blocker stance

These gaps are now **release blockers** for 4.1.0:

- no prose-doc reader plus top-level docs navigation
- no repo/package docs inside DOGFOOD
- no philosophy/architecture/doctrine docs inside DOGFOOD

If those remain absent, the release would still ship a strong component field guide, but not the terminal docs system Bijou increasingly claims to have.

Deliverables from this shaping cycle

This cycle should spawn three explicit 4.1.0 blockers:

- DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav
- DF-023 — Publish Repo, Package, And Release Guides In DOGFOOD
- DF-024 — Publish Philosophy, Architecture, And Doctrine Guides In DOGFOOD

Implementation outline

1. Shape a top-level DOGFOOD docs-site IA instead of treating “components” as the only section.
2. Add a prose-doc reader path so markdown- and guide-like pages can be read inside the app.
3. Expose a minimum set of repo/package/release pages in that reader.
4. Expose a minimum set of philosophy/architecture/doctrine pages in that reader.
5. Keep component stories as the “Components” section rather than letting them keep masquerading as the whole docs product.

Tests to write first

Under tests/cycles/DF-021/:

- the cycle doc exists and names the missing docs-site sections
- a docs/method/graveyard/legacy-backlog/v4.1.0/ lane exists
- the three blocker items exist in that lane
- PLAN.md, BEARING.md, and release.md acknowledge reopened 4.1.0 blockers around DOGFOOD scope

Risks

- treating DOGFOOD’s component explorer as “good enough docs” and shipping a narrower product than the repo’s own story implies
- reopening the 4.1.0 boundary without making the new blockers explicit in METHOD signposts
- trying to solve search, docs import, architecture writing, and motion APIs all at once instead of defining a credible minimum docs-site contract

Retrospective

What landed

- DOGFOOD’s next shape is now explicit: terminal docs system, not only component gallery
- the missing 4.1.0 docs-site surfaces are named as concrete blockers
- the release boundary is narrowed around a more honest docs promise

Debt spawned

Follow-on work such as generalized text-motion APIs, richer search, and deeper bespoke-chrome cleanup may still matter, but they should not hide the more important docs-site gaps that now block 4.1.0.

81

DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav

docs/design/DF-022-build-prose-docs-reader-and-top-level-dogfood-nav.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-021 — Shape DOGFOOD As Terminal Docs System

Sponsor human

A builder or maintainer who expects DOGFOOD to behave like a real docs site shell instead of only a component-family browser.

Sponsor agent

An agent that must inspect DOGFOOD and answer:

- where the top-level docs sections live
- whether DOGFOOD can read prose docs inside the app
- whether the component explorer is still being mistaken for the whole docs product

Hill

DOGFOOD visibly behaves like a terminal-native docs shell with top-level section tabs and a real prose-doc reader, while keeping the component explorer as one important section rather than the whole app.

Why this cycle exists

DF-021 reframed the release honestly: DOGFOOD had enough shell and component breadth to feel substantial, but it still behaved like a components-only product.

That meant 4.1.0 still lacked a visible docs-site shell:

- there was no top-level section navigation
- there was no prose-doc reader inside the app
- users could not tell where non-component docs were supposed to live

This cycle closes that shell-shape gap without pretending the full repo, package, release, doctrine, and architecture corpus has already landed.

Playback questions

1. Does DOGFOOD now expose top-level docs navigation instead of a single component-only page?
2. Is the default docs route a prose-reader-oriented section rather than dropping directly into the component explorer?
3. Can a user visibly switch between Guides, Components, Packages, Philosophy, and Release?
4. Does the component browser still work as one section inside the docs shell?
5. Is the remaining 4.1.0 blocker work now clearly about publishing the real corpus rather than inventing the shell shape?

Accessibility / assistive reading posture

The new reader must stay keyboard-first and text-legible. It cannot rely on screenshots, pointer-only interaction, or color alone to communicate the new section structure.

Localization / directionality posture

Section titles and shell hints should stay localizable. This cycle does not block on translating the starter prose corpus, but the shell shape must not hard-code a components-only reading model.

Agent inspectability / explainability posture

The repo should make the new shell shape explicit in:

- this cycle doc
- the DOGFOOD app implementation

- the v4.1.0 blocker lane
- the release signposts and tests

An agent should be able to prove that DF-022 closed and that the remaining blocker work is now corpus publication.

Non-goals

- importing the full repo docs corpus in this cycle
- closing package, release, doctrine, or architecture content gaps
- replacing every bespoke DOGFOOD surface
- inventing a general animation or text-effects API as part of this work

Decision

For 4.1.0, DOGFOOD should expose the full section map immediately:

- Guides
- Components
- Packages
- Philosophy
- Release

Guides becomes the default docs route and carries the initial prose reader foundation. The other non-component sections get a visible home now, even if their deeper corpus still lands in follow-on blocker cycles.

Implementation outline

1. Promote DOGFOOD from a one-page framed app to a top-level tabbed docs shell.
2. Add a prose-reader page model with guide navigation and scrollable main-doc surfaces.
3. Keep the existing component explorer intact as the Components tab.
4. Seed DOGFOOD with starter markdown pages that explain the docs map and make the missing sections visible.
5. Reduce the remaining 4.1.0 blockers to real corpus publication: DF-023 and DF-024.

Evidence

- the docs app now defaults to Guides
- the framed header exposes multiple top-level docs sections
- Components still contains the family/story/variant explorer

- DOGFOOD can render prose markdown pages from a starter corpus
- the v4.1.0 blocker lane now only carries the remaining corpus work

Retrospective

The important correction here is architectural, not decorative: DOGFOOD stops overclaiming that “component docs” equals “the whole Bijou docs product.”

The remaining release blocker work is now narrower and more honest:

- DF-023 should publish repo/package/release guidance into the visible shell
- DF-024 should publish philosophy/architecture/doctrine guidance into the visible shell

82

DF-023 — Publish Repo, Package, And Release Guides In DOGFOOD

docs/design/DF-023-publish-repo-package-and-release-guides-in-dogfood.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-021 — Shape DOGFOOD As Terminal Docs System
- DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav

Sponsor human

A builder or evaluator who expects DOGFOOD to explain the Bijou repo, its published packages, and the current release line without sending them back out into scattered repo files.

Sponsor agent

An agent that must answer:

- what packages ship in the workspace and what each one is for
- where the repo says to start reading
- what changed in 4.1.0
- how a v4.0.0 user should approach the 4.1.0 upgrade

Hill

DOGFOOD stops treating Packages and Release as placeholder tabs and starts acting like a real terminal docs product for those topics. A user can reach repo orientation, package explainers, and 4.1.0 release/migration material from inside the app.

Why this cycle exists

DF-022 gave DOGFOOD the shell shape of a docs site, but it still only proved the shape. The newly visible Packages and Release sections were honest placeholders, not the actual corpus.

That was good enough to clarify the next blocker, but not good enough to ship 4.1.0 honestly.

If DOGFOOD is the canonical docs surface for this release, it cannot keep outsourcing repo/package/release understanding to filesystem spelunking.

Playback questions

1. Can a user open a repo-orientation page inside Guides?
2. Does Packages now expose explainer pages for the shipped workspace packages instead of only an overview placeholder?
3. Does Release now expose the actual 4.1.0 long-form What's New and migration guidance inside the docs app?
4. Do the signposts and blocker lane stop pretending DF-023 is still open?
5. Is the remaining 4.1.0 DOGFOOD blocker work now narrower: philosophy/doctrine corpus plus the examples/smoke posture shift?

Accessibility / assistive reading posture

The newly published corpus must remain keyboard-first, scrollable, and text-legible inside the existing three-pane reader. No screenshot-only or color-only explanation is allowed.

Localization / directionality posture

This cycle does not translate the new prose corpus, but it must keep the reader on the same localizable shell and section model established in DF-022.

Agent inspectability / explainability posture

An agent should be able to prove closure by inspecting:

- the DOGFOOD app corpus declarations
- the new cycle doc
- the v4.1.0 blocker lane
- the release signposts
- the cycle tests

Non-goals

- closing the philosophy/architecture/doctrine blocker (DF-024)
- replacing the package READMEs or release docs as their own repo-owned sources
- finishing the examples-to-DOGFOOD posture shift
- changing the 4.1.0 version decision

Decision

For 4.1.0, DOGFOOD should publish the existing repo/package/release corpus through the prose-doc reader instead of keeping those sections as stubs.

The source of truth remains the repo-owned docs and package READMEs. DOGFOOD is the canonical human-facing reading surface for them.

Implementation outline

1. Add a repo-orientation page under Guides.
2. Publish package explainer pages for the shipped workspace packages in Packages.
3. Publish the 4.1.0 long-form What's New and migration guides in Release.
4. Update the starter overview pages so they no longer claim that the package and release corpus is missing.
5. Move DF-023 from the v4.1.0 blocker lane into docs/design/ and update the signposts/tests.

Evidence

- DOGFOOD now exposes a Documentation Map guide
- the Packages section contains per-package explainer pages
- the Release section contains What's New in v4.1.0 and Migration Guide v4.1.0
- the active v4.1.0 blocker lane no longer lists DF-023

Retrospective

The important correction is not just "more text exists." The correction is that DOGFOOD now behaves more like a real docs product:

- repo orientation lives inside the app
- package understanding lives inside the app
- release understanding lives inside the app

That leaves the remaining blocker work more honest and more focused:

- DF-024 — Publish Philosophy, Architecture, And Doctrine Guides In DOGFOOD
- DF-026 — Demote Examples To Secondary Reference Status
- WF-003 — Replace smoke:examples:* With smoke:dogfood

83

DF-024 — Publish Philosophy, Architecture, And Doctrine Guides In DOGFOOD

docs/design/DF-024-publish-philosophy-architecture-and-doctrine-guides-in-dogfood.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-021 — Shape DOGFOOD As Terminal Docs System
- DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav
- DF-023 — Publish Repo, Package, And Release Guides In DOGFOOD

Sponsor human

A builder or maintainer who wants DOGFOOD to explain not just what Bijou exports, but why the framework is shaped the way it is.

Sponsor agent

An agent that must answer:

- what doctrine governs Bijou runtime and modeling decisions
- how the repo is architected
- which invariants matter
- where the design-system and UX stance live

Hill

The Philosophy section in DOGFOOD stops being a placeholder and becomes a real reader-first home for doctrine, architecture, invariants, and design-system guidance.

Why this cycle exists

After DF-023, DOGFOOD could explain the repo, the package line, and the release line, but it still could not explain Bijou's design stance without sending users back into raw repo docs.

That was the last major corpus gap keeping DOGFOOD from acting like a real docs product for 4.1.0.

Playback questions

1. Does the Philosophy section now expose more than an overview page?
2. Are key doctrine and architecture docs reachable from inside the app?
3. Can a user read runtime-truth doctrine, architecture, invariants, and design-system guidance without leaving DOGFOOD?
4. Do the blocker lane and signposts stop pretending DF-024 is still open?
5. Are the remaining 4.1.0 blockers now only the examples/smoke posture items?

Accessibility / assistive reading posture

The newly published corpus must remain scrollable, keyboard-first, and text-legible inside the existing prose-reader shell.

Localization / directionality posture

This cycle does not translate the doctrine corpus, but it must keep the same localizable shell and section posture established in DF-022.

Agent inspectability / explainability posture

An agent should be able to prove closure by inspecting:

- the new Philosophy guide entries in the DOGFOOD app
- the cycle doc
- the updated v4.1.0 blocker lane
- the release signposts
- the cycle tests

Non-goals

- replacing the source docs under docs/
- finishing the examples-to-DOGFOOD posture shift
- replacing the release smoke contract

- changing the 4.1.0 version decision

Decision

For 4.1.0, DOGFOOD should publish the key doctrine and architecture corpus through the Philosophy section instead of treating that section as an empty promise.

The repo-owned docs remain the source of truth. DOGFOOD becomes the canonical human-facing reading surface for them.

Implementation outline

1. Keep the Philosophy overview page, but update it to describe the now-published corpus.
2. Publish the key doctrine and architecture pages inside the prose reader:
   - System-Style JavaScript
   - Architecture
   - Bijou UX Doctrine
   - Invariants
   - Design System Overview
3. Update the section posture so Philosophy no longer reads like a placeholder in the info pane or overview copy.
4. Move DF-024 from the v4.1.0 blocker lane into docs/design/ and update the signposts/tests.

Evidence

- DOGFOOD now exposes multiple doctrine and architecture pages under Philosop hy
- the Philosophy section no longer claims that the corpus is missing
- the active v4.1.0 blocker lane no longer lists DF-024

Retrospective

This is the point where DOGFOOD stops being only a component field guide plus support docs and starts carrying the framework's actual beliefs about itself.

That leaves the 4.1.0 blockers tighter and more operational:

- DF-026 — Demote Examples To Secondary Reference Status
- WF-003 — Replace smoke:examples:* With smoke:dogfood

84

DF-025 — Make DOGFOOD The Only Human-Facing Docs Surface

docs/design/DF-025-make-dogfood-the-only-human-facing-docs-surface.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-021 — Shape DOGFOOD As Terminal Docs System
- DF-022 — Build Prose Docs Reader And Top-Level DOGFOOD Nav

Sponsor human

A builder or maintainer who should have exactly one obvious answer to “where do I go to learn Bijou?”

Sponsor agent

An agent that must determine, from repo artifacts alone, whether the public docs surface is DOGFOOD or the examples/ tree.

Hill

DOGFOOD is the only human-facing docs surface for Bijou. The examples/ tree remains useful, but only as a secondary/internal reference, regression, and migration substrate rather than a competing public docs front door.

Why this cycle exists

The repo currently carries two conflicting truths:

- strategy docs say DOGFOOD should replace example sprawl as the living docs system

- front-door docs and release validation still treat examples/ as a first-class public teaching path

That is not a harmless wording issue.

If humans still have to browse examples/README.md to understand the product, then DOGFOOD is not actually the docs surface yet. It is just a better example.

For 4.1.0, Bijou needs a sharper stance:

- DOGFOOD is where humans should learn Bijou
- example programs are secondary and should not compete with DOGFOOD for repo-front-door authority
- release smoke should follow the same truth instead of continuing to bless the broad examples tree as the main proving surface

Human users

Primary human user

A curious builder evaluating Bijou and trying to find the canonical docs path quickly.

Human hill

A human can read the root docs surfaces and recover one clear story:

- start with DOGFOOD
- use package docs and design docs as supporting corpus
- treat examples/ as secondary reference material, not as the main product documentation surface

Agent users

Primary agent user

An agent updating docs, release automation, or backlog placement.

Agent hill

An agent can point to explicit repo artifacts showing:

- DOGFOOD is canonical
- example inventory is secondary
- the remaining release blockers needed to make runtime smoke and docs posture match that decision

Playback questions

1. If a human asks “where do I start learning Bijou?”, do the front-door docs point to DOGFOOD first?
2. Does examples/README.md clearly read as a secondary/internal inventory instead of a public docs hub?
3. Do METHOD signposts and release blockers explicitly say that smoke:examples:* is no longer acceptable as the release-facing end state?
4. Does the repo have a clear blocker capturing the remaining work to demote examples behind DOGFOOD?

Accessibility / assistive reading posture

This is a docs-information-architecture cycle. The important accessibility requirement is clarity: people using plain-text docs, terminal readers, or agent tooling should not have to infer canon from inconsistent link placement or filenames.

Localization / directionality posture

This cycle does not add new localized copy requirements. It does require that section naming and docs routing remain cleanly separable from English-only filesystem trivia such as examples/docs.

Agent inspectability / explainability posture

The DOGFOOD-only decision must be visible in:

- this cycle doc
- the v4.1.0 blocker lane
- signposts such as PLAN.md, BEARING.md, and release.md
- front-door docs such as README.md and docs/README.md

Agents should not have to rely on chat memory to learn that examples/ has been demoted.

Non-goals

- deleting the entire examples/ tree immediately
- pretending every example should disappear
- relocating every implementation file currently under examples/docs/
- eliminating every migration/reference example before 4.1.0

The immediate requirement is policy honesty and release-blocker clarity, not blind deletion.

Evidence

Current contradiction

- docs/strategy/dogfood.md says DOGFOOD should stop docs, demos, examples, and tests from drifting apart
- docs/EXAMPLES.md already says DOGFOOD is the primary living-docs surface
- but the root docs still expose examples/README.md as a public map, and release readiness still depends on smoke:examples:*

Why that matters

If examples remain both:

- a public docs front door
- and the main release smoke contract

then DOGFOOD cannot honestly be called “the only way” yet.

Decision

Public-docs rule for 4.1.0

For 4.1.0, DOGFOOD is the only human-facing docs surface.

That means:

- root docs should send humans to DOGFOOD first
- package/design/release corpus should be reachable through DOGFOOD
- the example inventory should read as secondary/internal/reference material
- release smoke should move off the broad examples/ tree

Example posture

Examples still matter, but their role is narrower:

- migration references
- isolated API seam proofs
- targeted regression fixtures
- implementation sandboxes

They do **not** get to remain a competing public docs navigation layer.

Deliverables from this shaping cycle

This cycle should tighten the 4.1.0 blocker lane by:

- keeping DF-023 — Publish Repo, Package, And Release Guides In DOGFOOD as a release blocker
- keeping DF-024 — Publish Philosophy, Architecture, And Doctrine Guides In DOGFOOD as a release blocker
- moving WF-003 — Replace smoke:examples:* With smoke:dogfood into the v4.1.0 blocker lane
- spawning DF-026 — Demote Examples To Secondary Reference Status as an explicit docs-surface blocker

Implementation outline

1. Publish a repo-owned DOGFOOD entrypoint under docs/.
2. Rewrite front-door docs so DOGFOOD is the only obvious learning path.
3. Reframe the example inventory as secondary/internal/reference.
4. Move the smoke migration item into the active release-blocker lane.
5. Keep the remaining release blockers explicit instead of pretending the policy is already fully implemented.

Tests to write first

Under tests/cycles/DF-025/:

- the cycle doc exists and contains the DOGFOOD-only decision
- a repo-owned DOGFOOD entrypoint exists under docs/
- WF-003 now lives in docs/method/graveyard/legacy-backlog/v4.1.0/
- the v4.1.0 blocker lane lists DF-023, DF-024, DF-026, and WF-003
- README.md, docs/README.md, and docs/release.md acknowledge the DOGFOOD-only posture

Risks

- overswinging into “delete examples” theater before DOGFOOD and smoke coverage actually replace them
- keeping the implementation physically under examples/docs/ long enough that humans still infer the wrong authority from the path
- leaving release validation on smoke:examples:* long enough that the repo continues to reward the old posture

Ready when

- front-door docs treat DOGFOOD as the only human-facing docs path
- the example inventory is clearly secondary/internal
- WF-003 is promoted into the v4.1.0 blocker lane
- DF-026 exists to capture the remaining example-demotion work needed before release

85

DF-026 — Demote Examples To Secondary Reference Status

docs/design/DF-026-demote-examples-to-secondary-reference-status.md

Legend: DF — DOGFOOD Field Guide

Depends on:

- DF-025 — Make DOGFOOD The Only Human-Facing Docs Surface

Sponsor human

A learner or evaluator who should get one clear answer about where to start reading Bijou docs, instead of being bounced between DOGFOOD and a parallel example hub.

Sponsor agent

An agent that must tell the difference between:

- the canonical docs surface
- the internal example inventory
- migration/reference fixtures
- smoke and regression substrate

Hill

The repo stops presenting examples/ as a competing docs product. DOGFOOD is the canonical human-facing path. Examples remain valuable, but only as secondary/internal/reference material.

Why this cycle exists

DF-025 set the policy direction, but policy alone was not enough.

If DOGFOOD says “start here” while the example inventory still reads like a second docs hub, the public posture is still split.

This cycle exists to make the repo-facing surfaces honest before 4.1.0.

Playback questions

1. Do front-door docs still clearly tell humans to start with DOGFOOD?
2. Does the example inventory now read as secondary/internal/reference instead of a public docs hub?
3. Can a user open DOGFOOD itself and understand why examples are secondary?
4. Do release-facing docs stop treating the example inventory as equal public navigation?
5. Is WF-003 now the only remaining 4.1.0 blocker?

Accessibility / assistive reading posture

The examples posture should be stated in text, not implied by color, layout, or repo folklore.

Localization / directionality posture

This cycle does not translate the example posture copy, but it must keep the explanation inside the same localizable DOGFOOD shell.

Agent inspectability / explainability posture

An agent should be able to prove closure by inspecting:

- docs/DOGFOOD.md
- docs/EXAMPLES.md
- examples/README.md
- the DOGFOOD guide corpus
- the v4.1.0 blocker lane
- the cycle tests

Non-goals

- deleting the whole examples/ tree
- replacing the release smoke contract
- erasing migration or isolated API seam references that still matter

Decision

For 4.1.0, DOGFOOD is the only human-facing docs surface. The example inventory remains, but it should read as:

- secondary/internal reference
- migration substrate
- isolated API proof
- regression/smoke substrate

Implementation outline

1. Publish the secondary-example posture inside DOGFOOD itself.
2. Tighten docs/EXAMPLES.md so it reads as an internal reference map.
3. Tighten examples/README.md so it does not read like a docs hub.
4. Move DF-026 from the v4.1.0 blocker lane into docs/design/ and update the signposts/tests.

Evidence

- DOGFOOD now exposes a Secondary Example Map guide
- docs/EXAMPLES.md reads as a maintainer/agent-facing reference map
- examples/README.md explicitly reads as secondary/internal
- the active v4.1.0 blocker lane no longer lists DF-026

Retrospective

This cycle does not remove examples. It removes ambiguity about what examples are for.

That leaves one remaining 4.1.0 blocker:

- WF-003 — Replace smoke:examples:* With smoke:dogfood

86

DF-027 — Storybook-Style Tool for Bijou

docs/design/DF-027-storybook-style-tool-for-bijou.md

title: "DF-027 — Storybook-Style Tool for Bijou"
legend: DF
lane: design

Framing

Bijou already has the pieces of a Storybook-style workflow: structured ComponentStory records, DOGFOOD's component explorer, story capture matrices, fixture-promotion records, and MCP-facing docs payloads. The missing piece is a standalone workstation that developers can use without entering the full docs app, backed by a stable model that tooling can consume without scraping DOGFOOD.

This cycle adds the first dedicated Storybook workstation slice. It includes an interactive TUI for local component browsing and development, plus the deterministic model/index and matrix capture path for agents, CI, MCP exports, block tooling, screenshots, and schema-bound controls.

Sponsored Users

- Bijou builders browsing first-party components and shells by family.
- Maintainers auditing variant and output-mode coverage before release.
- Tooling authors who need a stable story index and capture path for docs, MCP, screenshots, fixtures, or future block workstations.
- Agents selecting stories without reverse-engineering the DOGFOOD shell.

Hills

1. A builder can run npm run storybook and enter a standalone Storybook-style workbench over the story catalog, with variants, profiles, package/source facts, and a preview surface.
2. A maintainer can capture a selected story across every variant/profile pair using the same StoryCaptureMatrix substrate as existing docs tooling.

3. A future UI or MCP tool can consume a stable workstation model instead of scraping examples/docs/app.ts or re-parsing story docs.

Playback Questions

- Does the workstation index every DOGFOOD story without duplicating story truth?
- Does it group stories by family and expose variants, profile modes, package, tags, and source path?
- Can callers capture a selected story's variant/profile matrix?
- Does the capture path render real story previews instead of placeholder text?
- Does the tool reuse captureStoryMatrix() rather than creating a second matrix format?
- Is there a simple command for humans and agents to run?

Requirements

- Add a standalone interactive Storybook workstation app.
- Add a DOGFOOD storybook workstation module for deterministic tooling.
- Add a text index renderer for the story catalog.
- Add a selected-story matrix capture helper.
- Reuse ComponentStory and StoryCaptureMatrix contracts.
- Add root scripts for the interactive workbench and text-first workstation.
- Keep both paths on the same story catalog instead of duplicating story truth.

Acceptance Criteria

- tests/cycles/DF-027/storybook-workstation.test.ts proves the cycle doc carries the modern playback sections.
- The workstation model covers every COMPONENT_STORIES entry.
- The rendered index includes family grouping, counts, variants, modes, package names, and source paths.
- The selected-story matrix captures every profile/variant combination and has no missing canonical modes for canonical DOGFOOD stories.
- The matrix output contains real rendered story content.
- npm run storybook launches a standalone interactive Storybook workbench instead of deep-linking into createDocsApp().
- package.json exposes npm run storybook:index for the deterministic text-first workstation, with npm run dogfood:storybook kept as a DOGFOOD-scoped compatibility alias.

Implementation Outline

1. Add examples/docs/storybook-app.ts.
2. Add examples/docs/storybook-workstation.ts.
3. Export pure model/index/matrix helpers from that module.
4. Add a small CLI path that prints either the index or a selected story matrix.
5. Add a DF-027 cycle test around the model, text output, matrix capture, and script registration.
6. Move the backlog note into docs/design/ and update the v6 lane pointer.

Drift Check

The current DOGFOOD app remains the canonical human-facing docs surface. This slice does not fork story data or add a browser dependency. The standalone workbench and text-first workstation both read the existing COMPONENT_STORIES catalog and use the existing story protocol plus captureStoryMatrix().

The result is a narrow, reusable tooling seam: deterministic enough for agents and CI, but still powered by the same story records humans browse in DOGFOOD.

Playback

- RED: DF-027 existed only as a backlog concept, while DOGFOOD, showcase, MCP, and story capture each held partial storybook-shaped behavior.
- GREEN: createDogfoodStorybookWorkbenchModel() produces a grouped story catalog from COMPONENT_STORIES.
- renderDogfoodStorybookIndex() prints counts, required modes, families, stories, variants, packages, and source paths.
- captureDogfoodStorybookMatrix() renders a selected story through every profile/variant pair and returns a StoryCaptureMatrix.
- createStorybookApp() launches a standalone interactive TUI over the same story catalog, with catalog navigation, profile/variant switching, preview, and test-matrix panes.
- npm run storybook now targets the standalone interactive workbench instead of opening DOGFOOD on the components page.
- npm run storybook:index provides the text-first command path, while npm run dogfood:storybook remains available for existing docs and users.

Retrospective

The right Storybook-style tool has two layers: a standalone interactive workbench for humans and a stable model/capture path for CI, MCP, release

audits, agents, and future block tooling. Both layers stay useful because they share the same story records instead of minting a second registry.

87

DF-028 Story Capture Matrix

docs/design/DF-028-story-capture-matrix.md

```
title: DF-028 Story Capture Matrix
legend: DF
lane: design
```

Framing

Bijou's best components are not single screenshots. They have rich, interactive, static, pipe, and accessible renderings, often across multiple variants and widths. Docs and tests need one place to see that matrix without copying examples by hand.

DF-028 adds a pure story capture matrix. The first slice accepts story profiles, variants, and a renderer callback, captures every profile/variant combination, and renders deterministic text. It is intentionally renderer-neutral: callers can use core components, TUI frames, fixtures, or DOGFOOD stories around it.

Sponsored Users

- Maintainers auditing multi-mode component stories.
- Docs authors promoting regression fixtures into readable examples.
- MCP/tooling authors exporting the same capture facts to agents.
- Accessibility reviewers comparing lowerings without opening a TUI.

Hills

1. A maintainer can capture every variant across interactive, static, pipe, and accessible profiles from one renderer callback.
2. A reviewer can see missing required modes before accepting a story capture.
3. A docs surface can render the captured matrix as stable text.

Playback Questions

- Does the helper invoke the renderer once for every profile/variant pair?
- Does each capture preserve story id, profile id, variant id, mode, width, and output?
- Can callers require specific modes and get deterministic missing-mode output?
- Does matrix text group captures by variant and profile?
- Can the matrix carry component metadata from DX-025 without owning that metadata contract?
- Does @flyingrobots/bijou export the story capture API from the root barrel?

Requirements

- Add captureStoryMatrix(options).
- Add storyCaptureMatrixText(matrix).
- Support profiles with id, label, mode, and width.
- Support variants with id, label, and optional description.
- Support optional component metadata.
- Support requiredModes and expose missing modes.
- Keep outputs as strings in this slice; surface-specific capture can build on top later.

Acceptance Criteria

- RED tests fail before implementation and pass after.
- Captures are deterministic and cover the profile/variant cross product.
- Missing required modes are stable and sorted by output-mode order.
- Text output is suitable for docs, test failures, and agent reports.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou/src/core/story-capture.ts.
- Export it from packages/bijou/src/index.ts.
- Keep rendering caller-owned; this helper records and formats the matrix.

Drift Check

- Scope stayed renderer-neutral. The helper executes a caller-provided renderer and records string output, leaving surface conversion and runtime/test adapter orchestration to future wrappers.

- The matrix can carry DX-025 component metadata but does not validate or own it.
- Missing required modes are sorted by Bijou output-mode order.

Playback

- captureStoryMatrix() invokes the renderer once for each profile/variant pair and stores story id, profile id, variant id, mode, width, and output.
- missingModes reports required modes absent from the supplied profiles.
- storyCaptureMatrixText() renders profile ids, variant ids, missing modes, and each captured output grouped by variant and profile.
- Optional metadata lets DOGFOOD/MCP/docs consumers attach the shared component metadata contract without duplicating fields.
- @flyingrobots/bijou exports the story capture API from the root barrel.

Retrospective

- Keeping rendering caller-owned makes the helper useful in core tests, DOGFOOD, TUI fixtures, and MCP payload generation without importing runtime adapters.
- The next useful layer is a fixture/story wrapper that supplies profiles and renderer contexts automatically, then writes the matrix into docs artifacts.

88

DF-029 Fixture-to-Docs Promotion Path

docs/design/DF-029-fixture-to-docs-promotion-path.md

```
title: DF-029 Fixture-to-Docs Promotion Path
legend: DF
lane: design
```

Framing

Good Bijou examples often start as regression fixtures. Good docs examples often become the clearest regression fixtures. Today the promotion path between those surfaces is mostly manual, which means provenance, story ids, component metadata, and capture outputs drift.

DF-029 adds a pure promotion record that links fixture, docs, story, example, or MCP artifacts. The first slice is not a file-writing CLI; it is the canonical record and text report that future commands can emit, review, and preserve.

Sponsored Users

- Maintainers promoting stable fixtures into docs and DOGFOOD stories.
- Docs authors linking examples back to executable regression coverage.
- MCP/tooling authors reusing example provenance in agent-facing payloads.
- Agents proposing docs updates from failing or useful fixtures.

Hills

1. A maintainer can record a fixture-to-docs promotion with source artifact, target artifact, component metadata, story matrix, tags, and notes.
2. A docs author can reverse that record when a docs story should become a regression fixture.
3. A reviewer can read a deterministic report that preserves provenance without inspecting multiple files.

Playback Questions

- Does the record support fixture, docs, story, example, and MCP artifacts?
- Does the helper preserve metadata and story capture matrix references?
- Are tags normalized deterministically?
- Can the promotion direction be reversed without losing provenance?
- Does report text include id, source, target, component, matrix, tags, and notes?
- Does @flyingrobots/bijou export the promotion API from the root barrel?

Requirements

- Add createFixturePromotionRecord(options).
- Add reverseFixturePromotionRecord(record, options?).
- Add fixturePromotionText(record).
- Support artifact kinds fixture, docs, story, example, and mcp.
- Support optional ComponentMetadata and StoryCaptureMatrix.
- Keep the first slice pure and deterministic; no filesystem writes.

Acceptance Criteria

- RED tests fail before implementation and pass after.
- Promotion records keep source, target, metadata, matrix, tags, and notes.
- Reverse promotion swaps source/target and preserves provenance.
- Report text is stable and suitable for docs, review comments, and agent output.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou/src/core/fixture-promotion.ts.
- Export it from packages/bijou/src/index.ts.
- Keep artifact paths as caller-owned strings; path validation belongs to a future CLI/adapter.

Drift Check

- Scope stayed pure and provenance-focused. No files are written, no paths are resolved, and no docs are generated in this slice.
- The record supports fixture, docs, story, example, and MCP artifacts and can carry DX-025 metadata plus DF-028 story matrices.
- Tag normalization is deterministic: trimmed, de-duplicated, and sorted.

Playback

- createFixturePromotionRecord() stores id, source artifact, target artifact, optional component metadata, optional story matrix, normalized tags, and notes.
- reverseFixturePromotionRecord() swaps source and target while preserving metadata, matrix, tags, and notes.
- fixturePromotionText() renders id, source, target, component, matrix, tags, and notes as deterministic text.
- @flyingrobots/bijou exports the promotion API from the root barrel.

Retrospective

- The promotion record is deliberately not a CLI. That keeps adapter concerns like path existence, markdown insertion, and baseline writing outside core.
- Together with component metadata and story capture matrices, it gives future docs tooling a compact provenance spine instead of one-off comments.

89

DF-030 Dogfood Docs Surface Block

docs/design/DF-030-dogfood-docs-surface-block.md

Linked legend: DF - DOGFOOD Field Guide

Status: landed for v7.0.0 Issue: #244 PR: #258

Framing

DOGFOOD is already the human-facing documentation app and proving ground, but its docs experience was still described as several nearby surfaces: navigation, article reader, search, proof panels, and footer/status evidence. DF-030 makes that product truth explicit as one inspectable Block contract.

This cycle lands DogfoodDocsSurfaceBlock as the canonical DOGFOOD docs workspace Block. It does not replace the existing leaf Blocks. Instead, it anchors them under one docs-surface identity so humans and agents can inspect the full documentation experience without reconstructing it from scattered rendering code.

Sponsored Perspectives

Sponsored Human:

- A maintainer wants DOGFOOD to be the first-class documentation experience, not a secondary demo app.
- A reader wants navigation, article content, search, and proof artifacts to feel like one documentation surface.

Sponsored Agent:

- An agent auditing docs truth needs one Block tree and stable lowering facts for the selected route, heading, search hit count, and proof artifacts.
- An agent should not need provider handles, global app state, or rendered cell parsing to know what DOGFOOD is showing.

Hill

A maintainer can open DOGFOOD and treat the docs app as the canonical Bijou documentation surface, with route, reader, search, and proof evidence exposed as typed Block metadata, schema-bound input, command intents, registry inventory, and stable lower-mode facts.

Playback Questions

- Can DOGFOOD name its full docs workspace as one Block?
- Can unknown docs-surface input be validated before rendering?
- Do route, heading, search-hit-count, and proof-artifact facts survive every output mode as stable identities?
- Does the DOGFOOD Blocks inventory show this surface as product truth?
- Can lower modes explain the docs surface without borders, color, or terminal layout?

Linked Invariants

- Tests Are the Spec
- Docs Are the Demo
- Visible Controls Are a Promise

Tests To Write First

- RED: a DF-030 cycle test imports dogfoodDocsSurfaceBlock, dogfoodDocsSurfaceSchemaBlock, and dogfoodDocsSurfaceBlockRegistryEntry before those exports exist.
- RED: the test asserts metadata, data requirement names, command intent ids, and schema-bound input validation for the docs-surface contract.
- RED: the test rejects accessor-backed boundary input before getters can run.
- RED: the test renders interactive, static, pipe, and accessible modes and checks that route, heading, search-hit-count, and proof-artifact facts remain stable.
- RED: the test opens the DOGFOOD Blocks inventory and expects the new docs.surface registry entry to appear with a localized description.

Block Contract

Field	Value
Block name	DogfoodDocsSurfaceBlock

Field	Value
Package	@flyingrobots/bijou-dogfood
Family	docs-surface
Scale	workspace
Modes	interactive, static, pipe, accessible
Registry surface	docs.surface
Registry role	app-shell

Data requirements:

- docsTree
- selectedRoute
- searchState
- proofArtifacts

Command intents:

- docs.navigate
- docs.search
- docs.openProof
- docs.copyLink

Stable facts:

- route
- heading-id
- search-hit-count
- proof-artifact as one fact per available artifact id, not the display label

VIJ Mockup

```
+-- DOGFOOD Docs Surface ------------------------------------------+
| navigation             | reader                                  |
| > Blocks               | # Blocks                                |
|   Guides               | Blocks are reusable contracts...        |
|   Packages             |                                         |
|------------------------+-----------------------------------------|
| search: table (2 hits) | proof: table-demo.gif available         |
+-----------------------------------------------------------------+
```

Lower Modes

Pipe mode:

```
route   heading search-hit-count    proofs
blocks  blocks  2   table-demo.gif
```

Accessible mode:

```
DOGFOOD docs surface. Blocks page selected. Reader heading blocks.
Search query table has 2 hits. Proof artifacts: table-demo.gif.
```

The lower-mode invariant is semantic parity. The route, heading, hit count, and proof artifact facts are inspected from BlockRenderResult.facts, not parsed from terminal layout.

Accessibility And Assistive Posture

DogfoodDocsSurfaceBlock must preserve docs meaning without relying on position, borders, color, or animation. Accessible mode lowers the selected route, reader heading, search summary, and proof artifact list into readable sentences, while facts preserve the same semantic payload for assistive and agent consumers.

Localization And Directionality Posture

The registry description for docs.surface is catalog-backed through the DOGFOOD string table. This slice adds English source rows plus generated German, Spanish, and French runtime catalog entries for the docs-surface description and rendered-preview label. Any untranslated neighboring strings remain explicit catalog debt rather than hidden selected-locale fallback. The Block contract itself carries ids, routes, counts, and artifact labels as data so future locale and directionality adapters can format presentation without changing the semantic facts.

Agent Inspectability And Explainability Posture

Agents should inspect this surface through typed Block metadata, view data requirements, command intents, schema-bound input, registry ownership, and mode-lowering facts. The registry intentionally remains render-free and avoids provider handles, subscriptions, mutable dispatch callbacks, or global Block lookup.

Implementation

The implementation lives in examples/docs/dogfood-blocks.ts.

What landed:

- DogfoodDocsSurfaceBlock metadata, data requirements, commands, render output, and stable facts.
- dogfoodDocsSurfaceSchemaAdapter and dogfoodDocsSurfaceSchemaBlock.
- docs.surface registry entry with role app-shell.
- workspace as a valid block scale.
- DOGFOOD Blocks inventory evidence and a code-backed rendered preview for the new surface.

The block composes the existing DOGFOOD docs leaf Blocks by contract:

- NavigationListBlock
- DocumentationArticleBlock
- SearchPanelBlock
- GuideInspectorBlock

It does not add provider handles, global dispatch, subscriptions, or runtime tree traversal to the registry.

Tests

RED:

- tests/cycles/DF-030/dogfood-docs-surface-block.test.ts failed because DogfoodDocsSurfaceBlock, its schema-bound wrapper, and registry entry did not exist.

GREEN:

- The DF-030 test validates metadata, schema binding, accessor rejection, render lowerings, stable facts, registry coverage, DOGFOOD inventory output, the code-backed rendered docs-surface preview, and proof artifact id facts when display labels differ.
- tests/cycles/DF-069/dogfood-block-registry.test.ts now includes docs.surface in registry coverage.
- tests/cycles/DF-071/dogfood-block-authored-surfaces.test.ts continues to prove the runtime DOGFOOD inventory renders registered surface Blocks.

Future Declaration Shape

The YAML shape remains an authoring adapter over the typed Block contract:

```
block:
  name: DogfoodDocsSurfaceBlock
  family: docs-surface
  scale: workspace
  modes:
    - interactive
    - static
    - pipe
    - accessible
data:
  docsTree:
    - Guides
    - Blocks
    - Packages
  selectedRoute: blocks
  selectedHeadingId: blocks
  searchState:
    query: table
    hitCount: 2
  proofArtifacts:
    - id: table-demo.gif
      label: table-demo.gif
      available: true
commands:
  - docs.navigate
  - docs.search
  - docs.openProof
```

```
  - docs.copyLink
facts:
  route: blocks
  heading-id: blocks
  search-hit-count: 2
  proof-artifact: table-demo.gif
```

Closeout

DF-030 moves DOGFOOD one step closer to being the canonical docs surface rather than a collection of adjacent examples. The useful boundary is the typed Block: metadata, schema-bound input, command intents, registry evidence, and lower-mode facts. The runtime app remains free to render through its existing TUI machinery, but the product truth is inspectable as a Block.

90

DF-031 Status and feedback standard Blocks

docs/design/DF-031-status-feedback-standard-blocks.md

```
title: DF-031 Status and feedback standard Blocks
legend: DF
lane: design
priority: medium
keywords:
  - blocks
  - component-audit
  - dogfood
  - lowerings
  - status-feedback
```

Framing

The v6 block floor already proved AppShell, ReaderSurface, and InspectorPane l. The next useful slice is the product feedback layer: inline state, in-flow notices, transient overlays, activity history, shortcut cues, and progress.

This cycle turns issues #220 through #225 into first-party standard Blocks in @flyingrobots/bijou. The work uses the current typed Blocks API: defineBlock( ), validated metadata, view data contracts, command intents, story ids, deterministic render output, and stable lowering facts. It does not introduce a new YAML or JSON runtime IR. Declarative YAML/JSON can become an adapter later, but it should compile into the typed contract this slice proves.

Sponsored Users

- TUI app authors who need reusable status and feedback semantics instead of bespoke strings inside pages.
- DOGFOOD maintainers who need status surfaces to lower truthfully across interactive, static, pipe, and accessible modes.
- Tooling authors who want block metadata, data contracts, command intents, and stories to describe feedback surfaces before rendering them.
- Future IR authors who need a typed target for YAML/JSON block declarations.

Hills

1. A builder can import one of the standard feedback Blocks and immediately inspect its metadata, data requirements, command intents, and ready story.
2. A reader can move between rich, static, pipe, and accessible profiles and still get the same feedback meaning, even when visual chrome disappears.
3. A maintainer can run one focused cycle test that proves the status and feedback slice is catalogued, rendered, and lowering-stable.

Playback Questions

- Which feedback surfaces are common enough to become first-party Blocks now?
- Do these Blocks declare data contracts and command intents without taking on provider subscriptions or command dispatch?
- Do visual modes render as surfaces while pipe and accessible modes preserve meaning as text?
- Do lowerings carry stable facts so tests and tools do not parse terminal drawings?
- Does this slice make room for a future YAML/JSON declaration format without inventing a second runtime contract?

Block Set

Issue	Block	Purpose
#220	InlineStatusBlock	Small local status beside an item, field, row, or command.
#221	InFlowStatusBlock	Notice or state message that lives in document flow.
#222	TransientOverlayBlock	Temporary shell-level feedback that should still lower to text.
#223	ActivityStreamBlock	Ordered recent events with optional active selection.
#224	ShortcutCueBlock	Inline shortcut hints scoped to a page, region, or command.
#225	ProgressIndicatorBlock	Determinate or named progress state with stable facts.

UI Mockups

Inline status:

```
+-- InlineStatusBlock ----------------------+
| label: docs                               |
| status: ok                                |
| message: synced                           |
+-------------------------------------------+
```

In-flow notice:

```
+-- InFlowStatusBlock ----------------------+
| severity: warning                         |
| source: docs                              |
| message: inventory stale                  |
| action: run docs:inventory                |
+-------------------------------------------+
```

Transient overlay:

```
+-- TransientOverlayBlock ------------------+
| priority: normal                          |
| message: Saved DOGFOOD route              |
| dismiss: Esc dismisses                    |
+-------------------------------------------+
```

Activity stream:

```
+-- ActivityStreamBlock --------------------+
| events: 10:41 tests passed                |
|         10:42 PR opened                   |
| selected: 10:41 tests passed              |
+-------------------------------------------+
```

Shortcut cues:

```
+-- ShortcutCueBlock -----------------------+
| shortcuts: / Search                       |
|            ? Help                         |
|            Esc Close                      |
| scope: page                               |
+-------------------------------------------+
```

Progress:

```
+-- ProgressIndicatorBlock -----------------+
| label: Install packages                   |
| value: 3                                  |
| total: 5                                  |
| percent: 60%                              |
+-------------------------------------------+
```

Lower Modes

Static mode preserves the same surface structure as interactive mode while remaining deterministic and render-only.

Pipe mode lowers every Block to a compact record:

```
InlineStatusBlock | label=docs | status=ok | message=synced
```

Accessible mode lowers every Block to a readable sentence list:

```
InlineStatusBlock
- label: docs
- status: ok
- message: synced
```

The lower-mode invariant is semantic parity, not visual parity. Pipe and accessible consumers should not need box drawing, row positions, or color to understand status.

Requirements

- Export all six Blocks from the package barrel and standard catalog.
- Include all six Blocks in standardBlockPackageManifest.
- Give each Block valid metadata across interactive, static, pipe, and accessible modes.
- Give each Block at least one ready story id.
- Attach view data contracts so tooling can inspect required slot data without provider handles.
- Attach schema adapters and schema-bound block exports so callers can validate plain input before binding render slots.
- Attach command intents for selecting the block, copying facts, and opening a story, without command callbacks.
- Render interactive/static output as surfaces and pipe/accessible output as text.
- Preserve block, block.rendered, block.family, block.variant, block.mode, selected identity, and semanticValue.* facts across modes.

Acceptance Criteria

- tests/cycles/DF-031/status-feedback-blocks.test.ts proves the six Blocks publish through the standard catalog and package manifest.
- The cycle test validates metadata and command intent shape for every Block.
- The cycle test validates schema-bound input for every Block and rejects accessor-backed boundary objects before invoking getters.
- The cycle test renders every Block in every canonical mode.
- The cycle test verifies lowerings remain fact-stable and expose family, variant, selected, mode, and semantic-value facts.
- Existing DX-031 standard block tests expand their catalog expectations from three Blocks to nine Blocks.
- Repo docs stop describing this catalog expansion as future v7 work.

Implementation Outline

1. Add a RED cycle test for issues #220 through #225.
2. Extend BlockScale to support the product scales needed by this slice: inlin e and overlay.
3. Add six standard Block definitions in standard-blocks.ts.
4. Register their stories, metadata, data contracts, schema adapters, command intents, and package manifest entries.
5. Reuse the existing section renderer pattern so visual and lower modes stay deterministic.
6. Update BEARING, ROADMAP, CHANGELOG, v6 backlog evidence, design-system docs, and package docs.

Future Declarative Shape

The old Bijou IR idea still fits, but this slice keeps it as a declaration format over the typed API:

```
block: InlineStatusBlock
package: "@flyingrobots/bijou"
story: inline-status.ready
modes:
  - interactive
  - static
  - pipe
  - accessible
data:
  inlineStatus.label: docs
  inlineStatus.status: ok
  inlineStatus.message: synced
commands:
  - inlineStatus.select
  - inlineStatus.copyFacts
  - inlineStatus.openStory
facts:
  block: InlineStatusBlock
  block.family: status
  block.variant: ready
  semanticValue.label: docs
```

A future declaration adapter can parse this shape, route it through inlineStatusSchemaBlock, and build typed BlockDefinition input or story fixtures. The runtime source of truth remains the typed Block contract.

Drift Check

The original DX-031 release floor was correct to limit itself to the first three Blocks. Drift appeared later when issues #220 through #225 became clear v6 candidates but ROADMAP and BEARING still left them in the v7 queue. This cycle corrects the tracker mirror and makes the expansion explicit, issue-backed v6 work.

No new follow-on debt is deferred by this slice. Broader component-family audits remain separate issues after #225.

Playback

- RED: the status/feedback cycle test imported six standard Blocks that did not exist yet and failed at module load.
- RED: the review follow-up required schema-bound feedback exports and stronger lowering facts; the cycle test failed before those symbols and facts existed.
- GREEN: @flyingrobots/bijou now exports lower-camel block identifiers inlineStatusBlock, inFlowStatusBlock, transientOverlayBlock, activityStreamBlock, shortcutCueBlock, and progressIndicatorBlock, plus matching *SchemaAdapter and *SchemaBlock exports.
- The six Blocks are in standardBlocks, standardBlockStories, and standardBlockPackageManifest.

- Interactive/static modes produce surfaces; pipe/accessibility modes produce semantic text.
- Stable lowering facts are present across all modes, including family, variant, selected identity where applicable, concrete mode, and semantic-value facts.

Retrospective

The useful move was not to invent a new IR under pressure. The useful move was to land the feedback vocabulary as typed, tested Blocks, then leave YAML/JSON as a future authoring adapter. That keeps the v6 release boundary concrete: geometry and Blocks continue to be inspectable package truth, not untyped story prose.

91

DF-036 Audit loading placeholders family across real surfaces

docs/design/DF-036-audit-loading-placeholders-family-across-real-surfaces.md

```
title: DF-036 Audit loading placeholders family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - skeleton
```

Framing

The loading placeholder family is intentionally small: skeleton() should cover short-lived, known-shape loading gaps and should not become a generic decorative filler. Because skeletons exist to preserve layout truth, they are a good v6 release proof point: rich/static output can preserve shape, but pipe and accessible lowerings must preserve meaning without relying on placeholder glyphs.

This audit verifies the family against the real DOGFOOD story protocol, component-family doctrine, and mode-aware runtime behavior.

Sponsored Users

- TUI app authors deciding whether a known-shape loading gap should use skeleton(), progressBar(), or explicit prose.
- Docs readers switching the loading story across rich, static, pipe, and accessible profiles.
- Maintainers who need a focused regression proving DOGFOOD and design-system guidance match current runtime truth.

Hills

1. A builder can inspect the DOGFOOD loading placeholder story and see both form-shaped and card-shaped placeholder regions.
2. A reader can switch the story through interactive, static, pipe, and accessible profiles without decorative placeholder glyphs leaking into constrained output.
3. A maintainer can run one cycle test that ties skeleton() behavior, DOGFOOD story metadata, and component-family guidance together.

Playback Questions

- Does DOGFOOD expose the loading placeholder family through the skeleton() story?
- Do the story variants cover both field/form and region/card skeleton usage?
- Do interactive and static profiles preserve the placeholder shape?
- Do pipe and accessible profiles avoid placeholder glyph leakage?
- Do the design-system docs still describe skeletons as short-lived, known-shape placeholders rather than generic loading decoration?

Requirements

- Keep the existing skeleton story identity and loading-placeholders coverage family id.
- Render every skeleton story variant in every canonical story profile.
- Treat rich/static placeholder glyphs as allowed only in visual profiles.
- Treat pipe output as label/context preserving, not placeholder-glyph preserving.
- Treat accessible output as explicit loading-state language.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-036/loading-placeholders-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD skeleton() story and verifies the expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes placeholder shape; pipe output does not include placeholder glyphs; accessible output includes explicit loading text.
- Component-family docs retain the known-shape, short-lived loading guidance.

Implementation Outline

1. Add a DF-036 cycle test that renders the skeleton DOGFOOD story through the shared story protocol.
2. Read-test the loading placeholder section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of silently widening this small audit.

Drift Check

No runtime drift was found in this slice. skeleton() already lowers to visual placeholder bars in interactive/static modes, no output in pipe mode, and explicit Loading... text in accessible mode. DOGFOOD wraps those primitives with labels and region titles, so pipe output remains meaningful without copying decorative placeholder glyphs.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-036 playback test tied the loading placeholder story to real mode lowerings.
- GREEN: the new cycle test renders both form-shell and card-region story variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve shape with placeholder glyphs.
- Pipe previews preserve labels/regions while omitting placeholder glyphs.
- Accessible previews include explicit loading text.

Retrospective

The audit confirmed that the smallest block families still need first-class mode tests. There was no need to expand skeleton() itself; the valuable work was pinning its semantic limits so future loading-screen or block work cannot quietly turn skeletons into decorative filler.

92

DF-039 Component-family standard Blocks

docs/design/DF-039-component-family-standard-blocks.md

```
title: DF-039 Component-family standard Blocks
legend: DF
lane: design
priority: medium
keywords:
  - blocks
  - component-audit
  - design-thinking
  - dogfood
  - lowerings
```

Framing

The v6 block catalog shipped the structural floor, then the status and feedback slice. The next DOGFOOD six-pack turns the first component-family audits from prose-only review items into standard Blocks with typed metadata, schema-bound input, command intents, stories, real render output, and mode-lowering facts.

This cycle closes issues #226 through #231:

Issue	Block	Family
#226	FramedGroupBlock	grouping
#227	ExplainabilityWalkthroughBlock	explainability
#228	FormattedDocumentBlock	document
#229	LinkDestinationBlock	navigation
#230	DividerBlock	structure
#231	TextEntryBlock	input

The work follows a Design Thinking loop:

- observe the real DOGFOOD surface families instead of trusting isolated fixtures
- frame each family as a reusable Block contract with explicit data and facts
- prototype the TUI and lower-mode shapes as ASCII mockups before treating the implementation as landed
- test the contract through catalog, schema, render, and lowering proof

Sponsored Users

- TUI authors who need reusable semantics for groups, explanations, prose, links, separators, and text inputs.
- DOGFOOD maintainers who need the Blocks preview page to show real component families, not empty future-audit slots.
- Tooling authors who inspect block metadata, schemas, commands, and stories without mounting a terminal app.
- Future declarative IR authors who need typed runtime targets for YAML or JSON block declarations.

Hills

1. A builder can import any Block in the six-pack and inspect metadata, data requirements, command intents, and ready story ids from the public barrel.
2. A reader can compare interactive/static output with pipe/accessibility lowerings and recover the same product meaning without parsing box drawing.
3. A maintainer can run one DF-039 cycle test that proves the six-pack is catalogued, schema-bound, rendered, documented, and lowering-stable.

Playback Questions

- Does each family now have a named Block contract instead of a broad audit note?
- Are the required and optional fields small enough to be reusable but concrete enough to render?
- Do lower modes expose stable facts for block identity, family, variant, mode, selected value, and semantic values?
- Does the DOGFOOD Blocks page have example slots for every new definition?
- Does this leave room for future YAML/JSON IR without inventing a second runtime contract?

Block Contracts

Block	Required data	Optional data	Scale
FramedGroupBlock	title, items	selected, mode	section
ExplainabilityWalkthroughBlock	title, steps	evidence, decision, nextStep	section
FormattedDocumentBlock	heading, body	callout, code	section
LinkDestinationBlock	label, destination	kind, status	inline
DividerBlock	label	style, density	inline
TextEntryBlock	field, value	placeholder, validation, results	section

Each Block gets the same first-party command contract:

```
<blockPrefix>.select
<blockPrefix>.copyFacts
<blockPrefix>.openStory
```

UI Mockups

Framed grouping:

```
+-- FramedGroupBlock --------------------------------+
| Title                                              |
|   Release Checks                                   |
| Items                                              |
|   tests green; docs updated; PR linked             |
| Selected                                           |
|   tests green                                      |
| Mode                                               |
|   review                                           |
+----------------------------------------------------+
```

Explainability walkthrough:

```
+-- ExplainabilityWalkthroughBlock ------------------+
| Title                                              |
|   Why this changed                                 |
| Steps                                              |
|   input changed; constraint tightened;             |
|   preview re-rendered                              |
| Evidence                                           |
|   DF-040 playback                                  |
| Decision                                           |
|   keep grouped proof visible                       |
| Next step                                          |
|   open lower-mode output                           |
+----------------------------------------------------+
```

Formatted document:

```
+-- FormattedDocumentBlock --------------------------+
| Heading                                            |
|   Blocks document                                  |
| Body                                               |
|   Use prose for persistent product truth.          |
| Callout                                            |
|   Lower modes keep heading and body facts.         |
| Code                                               |
|   block: FormattedDocumentBlock                    |
+----------------------------------------------------+
```

Linked destination:

```
+-- LinkDestinationBlock ----------------------------+
| Label                                              |
|   DOGFOOD.md                                       |
| Destination                                        |
|   docs/DOGFOOD.md                                  |
| Kind                                               |
|   docs                                             |
| Status                                             |
|   available                                        |
+----------------------------------------------------+
```

Divider:

```
+-- DividerBlock -----------------------------------+
| Label                                             |
|   Release Evidence                                |
| Style                                             |
|   rule                                            |
| Density                                           |
|   compact                                         |
+---------------------------------------------------+
```

Text entry:

```
+-- TextEntryBlock ---------------------------------+
| Field                                             |
|   Search docs                                     |
| Value                                             |
|   table                                           |
| Placeholder                                       |
|   type a query                                    |
| Validation                                        |
|   4 results                                       |
| Results                                           |
|   4                                               |
+---------------------------------------------------+
```

Lower Modes

Static mode keeps the same surface structure as interactive mode and stays render-only.

Pipe mode lowers every Block to a compact record:

```
FramedGroupBlock
title: Release Checks
items: tests green; docs updated; PR linked
selected: tests green
mode: review
```

Accessible mode lowers every Block to label-first prose:

```
FramedGroupBlock
Title: Release Checks
Items: tests green; docs updated; PR linked
Selected: tests green
Mode: review
```

Every lower mode carries inspectable facts:

```
block = FramedGroupBlock
block.rendered = true
block.family = grouping
block.variant = ready
block.mode = pipe
block.selected = tests green
semanticValue.title = Release Checks
```

Accessibility And Directionality

The first slice does not add bespoke focus behavior. The Blocks expose semantic labels and section values so accessible output can preserve meaning when visual grouping, rules, and input chrome are removed. Text is stored as plain strings or string arrays, leaving locale, bidirectional rendering, and translation concerns to callers and localization adapters.

Agent Inspectability

Agents and tests should inspect metadata, data contracts, schema descriptions, command intent ids, ready story ids, and lowering facts. They should not scrape terminal boxes or infer meaning from row positions.

Declarative IR Note

The old Bijou IR idea still fits as a declaration format over this typed API. This cycle does not introduce a new YAML or JSON runtime, but it gives future IR work concrete targets:

```
block: FramedGroupBlock
package: "@flyingrobots/bijou"
story: framed-group.ready
data:
  title: Release Checks
  items:
    - tests green
    - docs updated
    - PR linked
  selected: tests green
  mode: review
commands:
  - framedGroup.select
  - framedGroup.copyFacts
  - framedGroup.openStory
facts:
  block.family: grouping
  block.variant: ready
  semanticValue.title: Release Checks
```

Tests

- tests/cycles/DF-039/dogfood-component-family-six-pack.test.ts proves all six Blocks publish through the standard catalog and package manifest.
- The cycle test validates metadata, command intent shape, schema binding, accessor rejection, visual rendering, lower-mode rendering, semantic facts, and Method documentation.
- DX-031 catalog tests prove the public barrel, first-party manifest, story ids, and rendered lowering checks include the expanded standard block set.

Closeout

The six-pack is landed when:

- @flyingrobots/bijou exports all six Block definitions, schema adapters, and schema-bound block definitions
- DOGFOOD Blocks preview has concrete sample slots for all six names
- standardBlocks, standardBlockStories, and standardBlockPackageManifest include the six names
- CHANGELOG, BEARING, and ROADMAP record the v7 component-family slice

- PR text closes #226, #227, #228, #229, #230, and #231

93

DF-041 Audit inspector panels family across real surfaces

docs/design/DF-041-audit-inspector-panels-family-across-real-surfaces.md

```
title: DF-041 Audit inspector panels family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - inspector
```

Framing

inspector() is the canonical compact side-panel surface for the currently selected thing. It should keep one active value obvious while supporting sections stay calmer and structured. That makes it a first-blocks proof point: an inspector panel is a reusable block-shaped composition, not an arbitrary box with hand-written strings.

This audit verifies the inspector family against real DOGFOOD story rendering, component-family doctrine, and mode-aware runtime behavior.

Sponsored Users

- TUI app authors building side panels for selected packages, records, rows, or rollout slices.
- Docs readers checking how an inspector panel lowers when borders, color, and side-panel layout are not available.
- Maintainers who need a focused regression proving inspector story metadata and design-system guidance stay aligned with runtime truth.

Hills

1. A builder can inspect the DOGFOOD inspector story and see current-selection emphasis plus compact supporting sections.
2. A reader can switch inspector variants through interactive, static, pipe, and accessible profiles while preserving the same semantic fields.
3. A maintainer can run one cycle test that ties inspector() behavior, DOGFOOD story metadata, and component-family guidance together.

Playback Questions

- Does DOGFOOD expose the inspector family through the inspector() story?
- Do variants cover both package-summary and rollout-review side-panel usage?
- Do rich/static previews preserve titled containment and section rhythm?
- Do pipe lowerings produce labeled plain text with one obvious current selection?
- Do accessible lowerings linearize the same fields without relying on borders, color, or pane position?
- Do component-family docs still position inspector() as side-panel context rather than a full guided recommendation or complex nested workflow?

Requirements

- Keep the existing inspector story identity and inspector-panels coverage family id.
- Render every inspector story variant in every canonical story profile.
- Treat borders and box-drawing characters as visual-profile affordances only.
- Keep pipe and accessible lowerings field-label-first and meaning-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-041/inspector-panels-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD inspector() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes titled box containment; pipe and accessible output do not depend on box drawing.
- Pipe and accessible output preserve current selection, context, and section labels.

- Component-family docs retain side-panel, current-selection, and compact section guidance.

Implementation Outline

1. Add a DF-041 cycle test that renders the inspector DOGFOOD story through the shared story protocol.
2. Read-test the inspector section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. inspector() already renders boxed, titled containment in interactive/static profiles, lowers to labeled grouped text in pipe mode, and linearizes the same fields in accessible mode.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-041 playback test tied the inspector story to real mode lowerings.
- GREEN: the new cycle test renders both package-summary and rollout-review variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve titled containment and section rhythm.
- Pipe previews preserve labels and grouped plain text without box drawing.
- Accessible previews linearize the same fields with explicit inspector language.

Retrospective

The inspector component did not need runtime changes. The important release work was turning current story truth into executable proof so future block and workspace work cannot quietly turn inspector panels into generic nested boxes or mini dashboards.

94

DF-046 Choice and navigation standard Blocks

docs/design/DF-046-choice-navigation-standard-blocks.md

```
title: DF-046 Choice and navigation standard Blocks
legend: DF
lane: design
priority: medium
keywords:
  - blocks
  - choice
  - component-audit
  - design-thinking
  - dogfood
  - navigation
```

Framing

The first component-family six-pack moved grouping, prose, links, dividers, and text entry into typed standard Blocks. This follow-on six-pack turns choice, decision, peer navigation, progressive disclosure, and path progress audits into reusable Block contracts with schema-bound input and lower-mode facts.

This cycle closes issues #232 through #237:

Issue	Block	Family
#232	SingleChoiceBlock	input
#233	MultipleChoiceBlock	input
#234	BinaryDecisionBlock	input
#235	PeerNavigationBlock	navigation
#236	ProgressiveDisclosureBlock	disclosure
#237	PathProgressBlock	navigation

The work follows a Design Thinking loop:

- observe DOGFOOD choice and navigation surfaces as product semantics, not visual widgets
- frame each family as a named Block contract with explicit state and facts
- prototype TUI and lower-mode shapes before treating implementation as landed
- prove catalog, schema, render, command, story, and lowering behavior in tests

Sponsored Users

- TUI authors who need reusable semantics for choices, decisions, peer navigation, disclosure state, and path progress.
- DOGFOOD maintainers who need the Blocks preview page to show the next real component-family slice.
- Tooling authors who need to inspect selection, current-step, checked-state, and disclosure facts without scraping terminal chrome.
- Future declaration authors who need YAML or JSON IR targets over typed Block APIs rather than a second runtime.

Hills

1. A builder can import all six Blocks and inspect metadata, data requirements, command intents, schema descriptions, and ready story ids from the public barrel.
2. A reader can compare interactive/static output with pipe/accessibility lowerings and recover option identity, selected state, current route, disclosure state, and current step.
3. A maintainer can run one DF-046 cycle test that proves the six-pack is catalogued, schema-bound, rendered, documented, and lowering-stable.

Playback Questions

- Does each choice/navigation family have a named Block contract instead of a broad audit note?
- Are required and optional fields concrete enough for DOGFOOD previews and small enough for reuse?
- Do lower modes expose stable facts for block identity, family, variant, mode, selected value, and semantic values?
- Does the DOGFOOD Blocks page have sample slots for every new definition?
- Does this preserve future YAML/JSON IR as an adapter over typed APIs?

Block Contracts

Block	Required data	Optional data	Scale
SingleChoiceBlock	label, options, selected	mode, validation	section
MultipleChoiceBlock	label, checked, unchecked	selected, validation	section
BinaryDecisionBlock	label, selected, consequence	confirmation, disabledReason	section
PeerNavigationBlock	previous, current, next	route, status	section
ProgressiveDisclosureBlock	label, state, hiddenCount	summary, details	section

Block	Required data	Optional data	Scale
PathProgressBlock	path, current, step, total	status	section

Each Block gets the same first-party command contract:

```
<blockPrefix>.select
<blockPrefix>.copyFacts
<blockPrefix>.openStory
```

UI Mockups

Single choice:

```
+-- SingleChoiceBlock -------------------------------+
| Label                                              |
|   Output mode                                      |
| Options                                            |
|   interactive; pipe; accessible                    |
| Selected                                           |
|   pipe                                             |
| Mode                                               |
|   radio                                            |
+----------------------------------------------------+
```

Multiple choice:

```
+-- MultipleChoiceBlock -----------------------------+
| Label                                              |
|   Release proof                                    |
| Checked                                            |
|   lint; tests                                      |
| Unchecked                                          |
|   screenshots                                      |
| Validation                                         |
|   2 of 3 complete                                  |
+----------------------------------------------------+
```

Binary decision:

```
+-- BinaryDecisionBlock -----------------------------+
| Label                                              |
|   Merge gate                                       |
| Selected                                           |
|   yes                                              |
| Consequence                                        |
|   admin merge                                      |
| Confirmation                                       |
|   CI green                                         |
+----------------------------------------------------+
```

Peer navigation:

```
+-- PeerNavigationBlock -----------------------------+
| Previous                                           |
|   Architecture                                     |
| Current                                            |
|   Blocks                                           |
| Next                                               |
|   Method                                           |
| Route                                              |
|   docs/blocks                                      |
+----------------------------------------------------+
```

Progressive disclosure:

```
+-- ProgressiveDisclosureBlock ----------------------+
| Label                                              |
|   Advanced options                                 |
| State                                              |
|   closed                                           |
| Hidden count                                       |
|   6                                                |
| Summary                                            |
|   6 options hidden                                 |
+----------------------------------------------------+
```

Path progress:

```
+-- PathProgressBlock -------------------------------+
| Path                                               |
|   Setup; Blocks; Preview                           |
| Current                                            |
|   Blocks                                           |
| Step                                               |
|   2                                                |
| Total                                              |
|   3                                                |
+----------------------------------------------------+
```

Lower Modes

Static mode keeps the same section structure as interactive mode and stays render-only.

Pipe mode lowers every Block to compact records:

```
SingleChoiceBlock
label: Output mode
options: interactive; pipe; accessible
selected: pipe
mode: radio
```

```
PathProgressBlock
path: Setup; Blocks; Preview
current: Blocks
step: 2
total: 3
status: current
```

Accessible mode lowers every Block to label-first prose:

```
SingleChoiceBlock
Label: Output mode
Options: interactive; pipe; accessible
Selected: pipe
Mode: radio
```

```
PathProgressBlock
Path: Setup; Blocks; Preview
Current: Blocks
Step: 2
Total: 3
Status: current
```

Every lower mode carries inspectable facts:

```
block = SingleChoiceBlock
block.rendered = true
block.family = input
block.variant = ready
block.mode = pipe
block.selected = pipe
semanticValue.label = Output mode
```

Accessibility And Directionality

The first slice does not add bespoke focus handling. The Blocks expose labels and section values so accessible output preserves choice state, current route, hidden-count state, and current-step state when visual affordances are removed. Text is stored as plain strings or string arrays, leaving locale and bidirectional rendering concerns to callers and localization adapters.

Agent Inspectability

Agents and tests should inspect metadata, data contracts, schema descriptions, command intent ids, ready story ids, and lowering facts. They should not infer selected state, checked state, or current step from visual markers.

Declarative IR Note

The Bijou IR idea remains an adapter layer over this typed API. This cycle does not introduce a new YAML or JSON runtime, but it gives future IR work concrete targets:

```
block: SingleChoiceBlock
package: "@flyingrobots/bijou"
story: single-choice.ready
data:
  label: Output mode
  options:
    - interactive
    - pipe
    - accessible
  selected: pipe
  mode: radio
commands:
  - singleChoice.select
  - singleChoice.copyFacts
  - singleChoice.openStory
facts:
  block.family: input
  block.variant: ready
  semanticValue.label: Output mode
```

Tests

- tests/cycles/DF-046/dogfood-choice-navigation-six-pack.test.ts proves all six Blocks publish through the standard catalog and package manifest.

- The cycle test validates metadata, command intent shape, schema binding, accessor rejection, visual rendering, lower-mode rendering, semantic facts, and Method documentation.
- DX-031 catalog tests prove the public barrel, first-party manifest, story ids, and rendered lowering checks include the expanded standard block set.
- scripts/dogfood-i18n-completeness.test.ts proves that new DOGFOOD localization keys and changed source strings require current rows for every supported locale before pre-push or CI can pass.

Closeout

The six-pack is landed when:

- @flyingrobots/bijou exports all six Block definitions, schema adapters, and schema-bound block definitions
- DOGFOOD Blocks preview has concrete sample slots for all six names
- DOGFOOD preview strings have current en, fr, es, and de translations, and the completeness policy is enforced by pre-push and CI
- standardBlocks, standardBlockStories, and standardBlockPackageManifest include the six names
- CHANGELOG, BEARING, and ROADMAP record the v7 component-family slice
- PR text closes #232, #233, #234, #235, #236, and #237

95

DF-049 Audit multi-field and staged forms family across real surfaces

docs/design/DF-049-audit-multi-field-and-staged-forms-family-across-real-surfaces.md

```
title: DF-049 Audit multi-field and staged forms family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - forms
```

Framing

group() and wizard() are the smallest form blocks that prove Bijou can model related user input as one coherent task. A group keeps related fields in one focused packet. A wizard keeps step boundaries and progress explicit when the task should unfold over time.

This audit verifies the existing group-wizard DOGFOOD story against real story rendering, component-family doctrine, and mode-aware lowerings. It does not redesign the form runtime; it turns current story truth into executable release proof.

Sponsored Users

- TUI app authors composing setup, deploy, checkout, sign-up, and settings flows from first-party blocks.
- Docs readers checking how grouped and staged forms behave when borders, color, and interactive focus are unavailable.
- Maintainers who need a focused regression tying form-block story metadata, variants, and component-family guidance to real output.

Hills

1. A builder can inspect the DOGFOOD forms story and understand when to use a single grouped form versus a staged wizard.
2. A reader can switch both variants through interactive, static, pipe, and accessible profiles while preserving labels, values, summaries, and step state.
3. A maintainer can run one cycle test that proves form-block story metadata and design-system guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the family through the group-wizard story?
- Do variants cover both single-step grouped fields and staged wizard progress?
- Do rich/static previews preserve visual containment and field rhythm?
- Do pipe lowerings preserve sequential prompts, labels, values, summaries, and step intent?
- Do accessible lowerings linearize labels and wizard progress without relying on borders, color, or visual grouping?
- Do component-family docs still position group() and wizard() as related data-collection blocks rather than generic layout containers?

Requirements

- Keep the existing group-wizard story identity and multi-field-and-staged-fo rms coverage family id.
- Render every grouped/staged form story variant in every canonical story profile.
- Treat box drawing as visual-profile chrome only.
- Keep pipe and accessible lowerings field-label-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-049/forms-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD group() / wizard() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes visual containment; pipe and accessible output do not depend on box drawing.

- Pipe and accessible output preserve form titles, labels, values, summaries, and wizard step state.
- Component-family docs retain grouped form, staged flow, progress, summary, and accessibility guidance.

Implementation Outline

1. Add a DF-049 cycle test that renders the group-wizard DOGFOOD story through the shared story protocol.
2. Read-test the multi-field and staged forms section in docs/design-system/co mponent-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. The existing group-wizard story already distinguishes grouped forms from staged flows, renders boxed field rhythm in interactive/static profiles, lowers to sequential labeled prompts in pipe mode, and linearizes step state in accessible mode.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-049 playback test tied grouped and staged forms to real mode lowerings.
- GREEN: the new cycle test renders both deploy-group and rollout-wizard variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve visual containment and field rhythm.
- Pipe previews preserve labels, values, summaries, and wizard step intent without box drawing.
- Accessible previews linearize the same facts with explicit current-step language for the wizard variant.

Retrospective

The forms story did not need runtime changes. The useful release work was codifying the first-blocks contract: grouped and staged forms are semantic data-collection compositions, not string dumps or decorative boxes.

96

DF-053 Audit motion and shader effects family across real surfaces

docs/design/DF-053-audit-motion-and-shader-effects-family-across-real-surfaces.md

```
title: DF-053 Audit motion and shader effects family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - motion
```

Framing

Motion and shader effects have become a real authoring surface, not a demo side quest. canvas() now has quad, Braille, and glyph-fit resolution modes, the raytrace helpers are available for app-owned shader scenes, and spring animation is stable under slow terminal pulses.

The DOGFOOD docs still treat that family as a small animated field. That is too thin for current repo truth. The story should prove the full family against real runtime behavior instead of trusting isolated fixtures or prose alone.

Sponsored Users

- TUI app authors who want expressive terminal visuals without inventing their own canvas, ray, or animation substrate.
- Documentation readers evaluating whether motion, shader, and transition effects lower honestly across rich, static, pipe, and accessible output.
- Maintainers who need regression tests that keep examples, docs, and exported APIs aligned as the rendering stack evolves.

Hills

1. A TUI app author can inspect one DOGFOOD story and see canvas() cover quad, Braille, glyph-fit, and raytrace-backed shader output without local hacks.
2. A docs reader can switch the same story across rich, static, pipe, and accessible profiles and get truthful lowerings instead of decorative glyphs leaking into constrained modes.
3. A maintainer can run a focused cycle test that proves canvas(), transition shaders, animate()/spring motion, and timeline() remain represented by real examples and DOGFOOD previews.

Playback Questions

- Does the motion/shader DOGFOOD story include variants for shader waves, Braille fields, glyph-fit raytraced surfaces, and spring/timeline motion orchestration?
- Do pipe and accessible previews describe the state-change meaning without relying on Braille, block, or ornamental shader characters?
- Does a static profile render a deterministic visual snapshot that still exercises the shader path?
- Do example sources for transitions and spring animation compile against the current public API surface?

Requirements

- Keep the existing motion/shader story identity and coverage family id.
- Add real story variants instead of only expanding prose.
- Use the current canvas() API, including resolution: 'glyph', for at least one DOGFOOD preview.
- Use exported raytrace helpers inside the DOGFOOD preview so the raytrace kernel is proven from a real docs surface.
- Represent spring motion and timeline() together in a deterministic preview.
- Keep pipe and accessible lowerings textual and meaning-first.
- Repair any stale example code uncovered by the audit.

Acceptance Criteria

- tests/cycles/DF-053/motion-shader-dogfood-audit.test.ts fails before the DOGFOOD story and stale example are updated.
- The motion/shader story exposes shader-wave, braille-field, glyph-raytrace, and spring-timeline variants.
- Rendering every motion/shader variant in interactive, static, pipe, and acc essible story profiles produces non-empty output.
- Pipe and accessible variants avoid Braille and block-art glyph leakage.

- The spring example no longer passes removed animate() options.

Implementation Outline

- Expand motionPreview() so it can render wave, Braille, glyph-raytrace, and spring/timeline previews from one helper.
- Import the needed public bijou-tui helpers through the package surface rather than private module paths.
- Add DF-053 cycle tests that render the DOGFOOD story through the story protocol instead of snapshotting internals.
- Remove stale spring-example drift discovered during the audit.

Drift Check

- examples/docs/stories.ts now imports bijou-tui from the workspace source barrel so DOGFOOD tests read current package truth instead of stale ignored dist/ artifacts.
- examples/spring/main.ts still demonstrated animate() with a removed fps option; the audit repaired that example while keeping the same runtime story.
- No follow-on backlog item is needed for this slice. The remaining broader opportunity is to add a dedicated visual capture for the motion/shader story, but the current tests already prove static, pipe, accessible, and rich-story rendering.

Playback

- RED: tests/cycles/DF-053/motion-shader-dogfood-audit.test.ts failed on the missing story variants, incomplete related-family docs, and stale spring example option.
- GREEN: the motion/shader story now renders shader-wave, braille-field, glyph-raytrace, and spring-timeline variants across every documented profile.
- The glyph raytrace preview uses canvas() with resolution: 'glyph' and the exported raytrace helpers.
- The spring/timeline preview uses the exported motion timeline and spring stepping helpers to produce a deterministic frame.
- Pipe and accessible previews lower to explicit text without Braille or block glyph leakage.

Retrospective

- The real drift was not in canvas() this time; it was DOGFOOD breadth. The docs surface had not caught up with the rendering primitives already landed.

- The source-barrel import is important for this repo because dist/ is ignored and can lag behind source during active cycles.
- Testing the story protocol directly is a better guard than static prose checks: it proves variants render through the same path the docs app uses.

97

DF-054 Late-family standard Blocks

docs/design/DF-054-late-family-standard-blocks.md

```
title: DF-054 Late-family standard Blocks
legend: DF
lane: design
priority: medium
keywords:
  - blocks
  - component-audit
  - design-thinking
  - dogfood
  - lowerings
```

Framing

The first two DOGFOOD component-family six-packs moved grouping, prose, links, dividers, text entry, choice, decision, navigation, disclosure, and path progress into typed standard Blocks. This cycle takes the remaining v7 component-family audit queue and turns it into concrete first-party Blocks with metadata, schema-bound input, command intents, ready stories, rendered output, and mode-lowering facts.

This cycle closes issues #238 through #243:

Issue	Block	Family
#238	BrandEmphasisBlock	branding
#239	ModeAwarePrimitiveBlock	primitive
#240	DenseComparisonBlock	comparison
#241	HierarchyBlock	hierarchy
#242	ExplorationListBlock	list
#243	TemporalDependencyBlock	graph

The work follows a Design Thinking loop:

- observe the remaining DOGFOOD component families as reusable product semantics instead of visual one-offs
- frame each family as a named Block contract with explicit data fields and facts
- prototype interactive, static, pipe, and accessible shapes before treating the implementation as landed
- prove catalog, schema, render, story, command, preview, and lowering behavior in tests

Sponsored Users

- TUI authors who need reusable semantics for branding, primitives, comparisons, hierarchies, exploratory lists, and dependency views.
- DOGFOOD maintainers who need the Blocks preview page to show the remaining component-family families as real examples, not future-audit placeholders.
- Tooling authors who inspect selected ids, semantic values, and family facts without scraping terminal layout.
- Future declarative IR authors who need stable typed targets for YAML or JSON declarations.

Hills

1. A builder can import all six Blocks and inspect metadata, data requirements, command intents, schema descriptions, and ready story ids from the public barrel.
2. A reader can move between visual, static, pipe, and accessible modes and recover brand, primitive, comparison, hierarchy, list, and dependency meaning without ANSI, borders, color, or animation.
3. A maintainer can run one DF-054 cycle test that proves the six-pack is catalogued, schema-bound, rendered, documented, and lowering-stable.

Playback Questions

- Does each late-family audit now have a named Block contract instead of a broad review note?
- Are required fields concrete enough for DOGFOOD previews and small enough for reuse?
- Do lower modes expose stable facts for block identity, family, variant, mode, selected value, and semantic values?
- Does the DOGFOOD Blocks page have sample slots for every new definition?
- Does this keep future YAML/JSON IR as an adapter over typed Block APIs?

Block Contracts

Block	Required data	Optional data	Scale
BrandEmphasisBlock	brand, tagline, decoration	role, selected	section
ModeAwarePrimitiveBlock	primitive, fact, value	status, modeContract, selected	section
DenseComparisonBlock	title, metric, left, right, delta	selected	workspace
HierarchyBlock	root, nodes, selected	parent, depth, expanded	section
ExplorationListBlock	title, facet, items, selected	preview	workspace

Block	Required data	Optional data	Scale
TemporalDependencyBlock	title, events, dependency	selected, dependsOn	workspace

Each Block gets the same first-party command contract:

```
<blockPrefix>.select
<blockPrefix>.copyFacts
<blockPrefix>.openStory
```

VJ Mockups

Brand emphasis:

```
+-- BrandEmphasisBlock -----------------------------+
| Brand                                             |
|   BIJOU                                           |
| Tagline                                           |
|   Terminal-native app blocks                      |
| Decoration                                        |
|   accent rule                                     |
| Role                                              |
|   nonessential                                    |
+---------------------------------------------------+
```

Mode-aware primitive:

```
+-- ModeAwarePrimitiveBlock ------------------------+
| Primitive                                         |
|   metric badge                                    |
| Fact                                              |
|   latency-ms                                      |
| Value                                             |
|   42                                              |
| Status                                            |
|   good                                            |
| Mode contract                                     |
|   visual and pipe                                 |
+---------------------------------------------------+
```

Dense comparison:

```
+-- DenseComparisonBlock ---------------------------+
| Title                                             |
|   Compare packages                                |
| Metric                                            |
|   tests                                           |
| Left                                              |
|   1820                                            |
| Right                                             |
|   640                                             |
| Delta                                             |
|   +12                                             |
+---------------------------------------------------+
```

Hierarchy:

```
+-- HierarchyBlock ---------------------------------+
| Root                                              |
|   docs/                                           |
| Nodes                                             |
|   design/; DX-031.md; METHOD.md                   |
```

```
| Selected                                          |
|   design/                                         |
| Parent                                            |
|   docs/                                           |
| Depth                                             |
|   1                                               |
+---------------------------------------------------+
```

Exploration list:

```
+-- ExplorationListBlock --------------------------+
| Title                                             |
|   Explore components                              |
| Facet                                             |
|   input                                           |
| Items                                             |
|   TextEntry field input; SingleChoice radio/select |
| Selected                                          |
|   TextEntry                                       |
| Preview                                           |
|   field input                                     |
+---------------------------------------------------+
```

Temporal dependency:

```
+-- TemporalDependencyBlock ------------------------+
| Title                                             |
|   Timeline                                        |
| Events                                            |
|   09:00 build; 09:05 test; 09:10 publish          |
| Dependency                                        |
|   publish waits for test                          |
| Selected                                          |
|   publish                                         |
| Depends on                                        |
|   test                                            |
+---------------------------------------------------+
```

Lower Modes

Static mode keeps the same surface structure as interactive mode and stays render-only.

Pipe mode lowers every Block to a compact record:

```
BrandEmphasisBlock
brand: BIJOU
tagline: Terminal-native app blocks
decoration: accent rule
role: nonessential
selected: BIJOU
```

```
TemporalDependencyBlock
title: Timeline
events: 09:00 build; 09:05 test; 09:10 publish
dependency: publish waits for test
selected: publish
dependsOn: test
```

Accessible mode lowers every Block to label-first prose:

```
BrandEmphasisBlock
Brand: BIJOU
Tagline: Terminal-native app blocks
Decoration: accent rule
Role: nonessential
Selected: BIJOU
```

```
TemporalDependencyBlock
Title: Timeline
Events: 09:00 build; 09:05 test; 09:10 publish
Dependency: publish waits for test
Selected: publish
Depends on: test
```

Every lower mode carries inspectable facts:

```
block = BrandEmphasisBlock
block.rendered = true
block.family = branding
block.variant = ready
block.mode = pipe
block.selected = BIJOU
semanticValue.brand = BIJOU
```

```
block = TemporalDependencyBlock
block.rendered = true
block.family = graph
block.variant = ready
block.mode = accessible
block.selected = publish
semanticValue.title = Timeline
```

Accessibility And Directionality

The first slice does not add bespoke focus behavior. The Blocks expose labels, selected values, and section values so accessible output preserves meaning when visual emphasis, indentation, dense columns, list focus, and graph arrows are removed. Text is stored as plain strings or string arrays, leaving locale, bidirectional rendering, and translation concerns to callers and localization adapters.

Agent Inspectability

Agents and tests should inspect metadata, data contracts, schema descriptions, command intent ids, ready story ids, and lowering facts. They should not infer hierarchy depth from indentation, compare values from column positions, or dependency meaning from arrows.

Declarative IR Note

The Bijou IR idea remains an adapter layer over this typed API. This cycle does not introduce a new YAML or JSON runtime, but it gives future IR work concrete targets:

```
block: TemporalDependencyBlock
package: "@flyingrobots/bijou"
story: temporal-dependency.ready
data:
  title: Timeline
  events:
    - 09:00 build
    - 09:05 test
    - 09:10 publish
```

```
  dependency: publish waits for test
  selected: publish
  dependsOn: test
commands:
  - temporalDependency.select
  - temporalDependency.copyFacts
  - temporalDependency.openStory
facts:
  block.family: graph
  block.variant: ready
  block.selected: publish
  semanticValue.title: Timeline
```

Tests

- tests/cycles/DF-054/dogfood-late-family-six-pack.test.ts proves all six Blocks publish through the standard catalog and package manifest.
- The cycle test validates metadata, command intent shape, schema binding, accessor rejection, visual rendering, lower-mode rendering, semantic facts, and Method documentation.
- DX-031 catalog tests prove the public barrel, first-party manifest, story ids, and rendered lowering checks include the expanded standard block set.
- DOGFOOD Blocks page tests prove the preview navigation and per-block preview output derive from standardBlocks.

Closeout

The six-pack is landed when:

- @flyingrobots/bijou exports all six Block definitions, schema adapters, and schema-bound block definitions
- DOGFOOD Blocks preview has concrete sample slots for all six names
- standardBlocks, standardBlockStories, and standardBlockPackageManifest include the six names
- CHANGELOG, BEARING, and ROADMAP record the v7 late-family slice
- PR text closes #238, #239, #240, #241, #242, and #243

98

DF-060 Audit viewport masking and scrollable inspection panes family across real surfaces

docs/design/DF-060-audit-viewport-masking-and-scrollable-inspection-panes-family-across-real-surfaces.md

```
title: DF-060 Audit viewport masking and scrollable inspection panes family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - viewport
```

Framing

Viewport masking is the family that proves overflow is owned by bounded panes, not by callers slicing strings after rendering. viewportSurface() is the base masking primitive for structured surfaces, pagerSurface() adds long-document position context, and focusAreaSurface() adds workspace focus ownership. Their string counterparts remain explicit text-lowering paths.

This audit verifies the DOGFOOD viewport-surface story against real story rendering, component-family doctrine, and mode-aware lowerings. It also closes the story drift where DOGFOOD taught only the base viewport mask even though the family doc already included pager and focus-area panes.

Sponsored Users

- TUI app authors wrapping existing surfaces in bounded scroll regions.
- Docs readers checking the distinction between masking, paging, and focused inspection panes.
- Maintainers who need one focused regression tying viewport story metadata, variants, and design-system guidance to real output.

Hills

1. A builder can inspect DOGFOOD and see viewport mask, structured stack, pager, and focused pane variants in one family.
2. A reader can switch every variant through interactive, static, pipe, and accessible profiles while preserving scroll and focus context.
3. A maintainer can run one cycle test that proves viewport-family story metadata and family guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the family through the viewport-masking-and-scrollable-inspection-panes coverage id?
- Do variants cover pure viewport masking, structured masking, pager status, and focus-area chrome?
- Do rich/static previews keep bounded pane chrome visible?
- Do pipe lowerings preserve sequential content and scroll/focus context without pretending hidden regions are interactive?
- Do accessible lowerings linearize pane content with explicit position or focus facts?
- Do component-family docs still cover viewport, pager, focus area, and the row-aware holdout boundary?

Requirements

- Keep the existing viewport-surface story identity and coverage family id.
- Render every viewport-family story variant in every canonical story profile.
- Treat scrollbars, gutters, and box drawing as visual-profile chrome only.
- Keep pipe and accessible lowerings scroll-context-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture for viewport, pager, and focus-area primitives.

Acceptance Criteria

- tests/cycles/DF-060/viewport-masking-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD viewportSurface() / pagerSurface() / focusAreaSurface() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.

- Pipe and accessible output preserve viewport scroll, pager line, and focus pane context.
- Component-family docs retain viewport, pager, focus area, content guidance, lowering guidance, and the domain-specific row-aware boundary.

Implementation Outline

1. Extend the existing viewport-surface DOGFOOD story with pager and focused pane variants.
2. Add mode-aware constrained lowerings for the story so pipe/accessibility profiles expose scroll/focus facts as text rather than visual chrome.
3. Add a DF-060 cycle test that renders every variant through the shared story protocol.
4. Move the backlog note into docs/design/ and update the v6 lane pointer.

Drift Check

Runtime support already existed for the full family. The drift was in DOGFOOD: the story represented only viewportSurface() while the design-system family also documented pagerSurface() and focusAreaSurface().

This slice fixes that by extending the story instead of filing follow-on debt. No runtime API changes were required.

Playback

- RED: the release lane had only a backlog note, and the DOGFOOD story did not visibly exercise pager or focused-pane variants.
- GREEN: the new cycle test renders document, structured-stack, pager-window, and focused-pane variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve scrollbars, status lines, focus gutters, and nested surface masking.
- Pipe previews preserve viewport scroll, pager line, and focus context without box drawing.
- Accessible previews linearize the same pane facts in reading order.

Retrospective

The runtime did not need another overflow abstraction. The useful release work was making DOGFOOD tell the whole family story: mask structured surfaces, page long linear documents, and mark focused inspection panes without letting scroll chrome become semantic content.

99

DF-060 V7 DOGFOOD Release Title Screen And Closeout Six-Pack

docs/design/DF-060-v7-dogfood-release-title-screen.md

```
title: DF-060 V7 DOGFOOD release title screen and closeout six-pack
legend: DF
lane: release
priority: medium
keywords:
  - blocklab
  - dogfood
  - release
  - table-surface
  - workflow
```

Framing

This cycle pulls the remaining v7 product-truth work into one closeout pass and adds a release-specific DOGFOOD title surface before v7 is considered ready to cut. The title screen should make the release feel intentional, but it must remain a useful terminal documentation entry point rather than a decorative splash screen.

This cycle carries six issues:

Issue	Lane	Role
#245	lane:release	Bring tableSurface() to responsive-width parity with table().
#246	lane:up-next	Document scopedNodeIO() realpath and symlink semantics.
#266	lane:bad-code	Keep BEARING.md synced with the live tracker after rapid-fire merges.
#267	lane:bad-code	Replace CI workflow string-scan tests with parsed YAML assertions.
#271	lane:bad-code	Rename the Storybook-labeled workstation to BlockLab.
#281	lane:release	Add a v7 DOGFOOD release title screen.

The design follows a Design Thinking loop:

- observe the remaining release gaps through the live GitHub tracker instead of stale roadmap prose
- frame the work around user-visible release truth and agent-inspectable release proof
- prototype the title screen, BlockLab labels, table parity, lower modes, and workflow guards before treating the cycle as landed
- prove the result with rendered DOGFOOD tests, package/API tests, docs tests, localization gates, and parsed workflow assertions

Sponsored Users

- A maintainer preparing v7 needs the issue tracker, roadmap, BEARING, title surface, and release proof to agree without manual reconciliation.
- A TUI builder needs tableSurface() to obey the same width constraints and lower-mode semantics already proven by table().
- A component author needs the preview workstation to use a project-owned name, BlockLab, while legacy commands remain available during migration.
- A review agent needs structured workflow assertions, stable release-title facts, and lower-mode output instead of brittle prose or raw string scans.

Hills

1. A DOGFOOD reader can open the docs app and see a v7 title surface that names the release identity, proof lanes, and navigation options at wide and narrow terminal widths.
2. A TUI author can use tableSurface() with explicit width constraints and get output that fits like table() while preserving row, column, value, and selected-cell facts in lower modes.
3. A maintainer can close the v7 issue lane with GitHub, BEARING, ROADMAP, CHANGELOG, workflow tests, BlockLab names, and docs all pointing to the same release truth.

Playback Questions

- Does DOGFOOD show release identity without blocking normal documentation navigation?
- Does the title screen lower to static, pipe, and accessible output with release id, release title, proof lane, and navigation facts?
- Does tableSurface() fit constrained widths instead of retaining old intrinsic sizing assumptions?
- Do scopedNodeIO() docs explain realpath-normalized paths and symlink escape rejection with enough precision for callers?
- Can workflow policy tests prove the checkout depth and i18n base behavior by parsing CI YAML instead of searching raw text?

- Do humans see BlockLab, while existing storybook npm scripts continue to work for one migration window?
- Do BEARING and ROADMAP match the live GitHub tracker after this cycle?

Product Shape

DOGFOOD V7 Title, Wide

```
+ BIJOU DOGFOOD ------------------------------------------+
| V7 Product Truth                                        |
| Blocks prove product surfaces. DOGFOOD proves Blocks.   |
|                                                         |
| Proof lanes: table parity | scoped Node I/O | BlockLab  |
| Release gate: v6 issue-complete, v7 closeout in flight  |
|                                                         |
| [Docs] [Blocks] [BlockLab] [Release Notes]              |
+---------------------------------------------------------+
```

DOGFOOD V7 Title, Narrow

```
+ BIJOU DOGFOOD ----+
| V7 Product Truth  |
| Blocks prove UX.  |
| lanes: table, io  |
| gate: closeout    |
+-------------------+
```

tableSurface() Width Parity

```
+ ResponsiveTableSurface ----------------------------+
| Package     Status   Tests                          |
| bijou       ready    1820                           |
| bijou-tui   ready    640                            |
| Layout: fitted to 48 columns                        |
+-----------------------------------------------------+
```

BlockLab Workbench

```
Bijou BlockLab                          story: notification-system
Stories 12  Families 6  Modes interactive/static/pipe/accessible

+ Preview -------------------------------------------------+
| Notification stack                                       |
| Variant: live                                            |
+----------------------------------------------------------+
```

Lower Modes

The title surface should be backed by release metadata so every mode lowers from the same truth.

Static mode:

```
Bijou DOGFOOD release title. Release V7 Product Truth. Proof lanes: table parity, scoped Node
↪  I/O, BlockLab, release title.
```

Pipe mode:

```
release_id  v7
release_title   V7 Product Truth
proof_lane  table parity
proof_lane  scoped Node I/O
proof_lane  BlockLab
proof_lane  release title
navigation_available    true
```

Accessible mode:

```
DOGFOOD title screen for V7 Product Truth. Current release proof lanes are table parity,
↪  scoped Node I/O, BlockLab, and release title. Navigation remains available after the
↪  title.
```

tableSurface() is the surface-native path, so this cycle proves bounded surface width rather than adding a new lowering contract. Semantic table facts remain owned by table() and navigableTableSurface():

```
row-id  bijou
column-id   status
cell-value  ready
layout-variant  fitted
selected-cell   bijou/status
```

Runtime And API Contracts

- DOGFOOD release title metadata should be a typed local data structure, not scattered strings in the renderer.
- DOGFOOD visible title strings must go through the DOGFOOD localization catalog.
- tableSurface() should share, reuse, or faithfully mirror the fitted table model already used by table() instead of growing a second width algorithm.
- scopedNodeIO() docs must state that returned paths can be realpath normalized and that symlink escapes are rejected even when lexical prefixes appear valid.
- Workflow tests should parse .github/workflows/ci.yml as YAML and inspect jo bs.test.steps structurally.
- BlockLab should become the canonical human-facing name and blocklab.* the canonical agent-facing token family, while old storybook scripts remain as compatibility aliases during this migration window.

Accessibility, Localization, And Inspectability

- The title screen must expose release title and proof lanes as text, not only border art or color.
- New DOGFOOD strings require en, fr, es, and de catalog values.
- npm run dogfood:i18n:complete and npm run dogfood:i18n:check are part of the merge gate.
- Agent-facing facts should include release id, release title, proof lane ids, navigation availability, table row ids, table column ids, and selected cell identity.

- No title-screen behavior should depend on wall-clock time, network state, or live GitHub API data.

Implementation Slices

1. Shape the cycle: create issue #281, write this design doc, push the branch, and open the early PR.
2. Write RED tests for release-title rendering/lower modes, tableSurface width parity, scopedNodeIO docs, parsed workflow assertions, BlockLab labels, and tracker sync.
3. Implement tableSurface() width parity and lower-mode facts.
4. Document scopedNodeIO() realpath and symlink semantics.
5. Rename visible Storybook workstation language to BlockLab while keeping compatibility script aliases.
6. Add the v7 DOGFOOD release title metadata, route/guide integration, localization strings, and lower-mode proof.
7. Sync BEARING, ROADMAP, CHANGELOG, and issue metadata with GitHub.
8. Run focused tests, DOGFOOD i18n gates, docs inventory, lint, typecheck, full test suite, self-review, and PR readiness.

Tests To Write First

- tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts
- tests/cycles/DX-037/table-surface-width-parity.test.ts
- tests/cycles/DX-029/scoped-node-io-docs.test.ts
- tests/cycles/WF-126/v7-closeout-tracker-sync.test.ts
- scripts/hooks.test.ts structured CI workflow assertions
- Existing DF-027/DF-069 BlockLab rename tests updated from Storybook labels

Acceptance Criteria

- PR closes #245, #246, #266, #267, #271, and #281.
- v7 has no open tracker issues after the PR merges, unless new issue triage explicitly says otherwise.
- DOGFOOD has a v7 release title screen with wide, narrow, static, pipe, and accessible proof.
- tableSurface() width behavior is deterministic under constrained terminal widths and preserves semantic facts.
- scopedNodeIO() realpath/symlink semantics are documented in package docs.
- CI workflow guard tests parse YAML and validate the actual test job checkout and i18n base paths.
- BlockLab is the canonical visible name, with compatibility aliases for old storybook npm scripts.

- All new or changed DOGFOOD localization strings are translated in every supported locale.

Risks And Guardrails

- Do not turn the title screen into a blocking splash page. It is a docs entry surface.
- Do not rename legacy storybook script aliases without a migration window.
- Do not use live GitHub data inside deterministic tests.
- Do not broaden table work into a full table redesign; target tableSurface() parity only.
- Do not make BEARING/ROADMAP the source of truth. They mirror GitHub Issues.

100

DF-061 Audit overlay primitives family across real surfaces

docs/design/DF-061-audit-overlay-primitives-family-across-real-surfaces.md

```
title: DF-061 Audit overlay primitives family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - overlays
```

Framing

Overlay primitives are the family for layered UI that is not ordinary document flow: tooltip(), drawer(), modal(), and toast(). They differ by interruption level. Tooltips explain locally, drawers supplement, modals block, and toasts announce transiently.

This audit verifies overlay primitive stories against real story rendering, component-family doctrine, and mode-aware lowerings. It also closes the story drift where DOGFOOD already had modal and toast examples, but did not visibly exercise drawer or tooltip primitives as part of the same overlay family.

Sponsored Users

- TUI app authors choosing the right overlay primitive instead of overusing modals or toasts.
- Docs readers checking how layered UI lowers when z-order is unavailable.
- Maintainers who need one focused regression tying overlay story metadata, variants, and design-system guidance to real output.

Hills

1. A builder can inspect DOGFOOD and see tooltip, drawer, modal, and toast primitives side by side as one overlay family.
2. A reader can switch every overlay primitive through interactive, static, pipe, and accessible profiles while preserving the right interruption level.
3. A maintainer can run one cycle test that proves overlay story metadata and design-system guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose all four overlay primitives?
- Do variants cover explanatory, supplemental, blocking, and transient overlay jobs?
- Do rich/static previews keep layering and containment visible?
- Do pipe lowerings avoid fake z-order and preserve plain prompt/event text?
- Do accessible lowerings linearize overlay relationships to their owning context?
- Do component-family docs still separate primitive overlays from the full notification system?

Requirements

- Keep modal and toast story identities while adding missing drawer and tooltip story coverage.
- Ensure toast participates in the overlay-primitives coverage family while still retaining its lower-level transient-overlay coverage.
- Render every overlay primitive story variant in every canonical story profile.
- Treat borders, dimmed backdrops, anchor placement, and z-order as visual-profile chrome only.
- Keep pipe and accessible lowerings plain, explicit, and semantic-preserving.

Acceptance Criteria

- tests/cycles/DF-061/overlay-primitives-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates DOGFOOD stories for modal(), drawer(), tooltip(), and toas t().
- Every documented overlay variant renders non-empty output in every profile.
- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.

- Constrained lowerings preserve blocking modal prompts, supplemental drawer context, local tooltip explanation, and transient toast events.
- Component-family docs retain overlay primitive guidance and the boundary with notification system behavior.

Implementation Outline

1. Add drawer and tooltip DOGFOOD stories under the overlay primitive family.
2. Add mode-aware constrained lowerings to modal and toast story variants.
3. Add a DF-061 cycle test that renders every overlay primitive through the shared story protocol.
4. Move the backlog note into docs/design/ and update the v6 lane pointer.

Drift Check

Runtime support already existed for all four primitives. The drift was in DOGFOOD: modal() and toast() were visible, but drawer() and tooltip() were only documented indirectly.

This slice fixes the drift by adding the missing stories and making constrained lowerings explicitly textual. No runtime API changes were required.

Playback

- RED: the release lane had only a backlog note, and DOGFOOD did not visibly exercise every overlay primitive.
- GREEN: the new cycle test renders modal, drawer, tooltip, and toast stories across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve layering, borders, anchors, and backdrop context.
- Pipe previews preserve prompt, supplemental, explanation, and event content without box drawing.
- Accessible previews linearize the same overlay facts in reading order.

Retrospective

The useful release work was not inventing another overlay. It was making DOGFOOD teach the existing overlay hierarchy honestly: explain with a tooltip, supplement with a drawer, block with a modal, and announce briefly with a toast.

101

DF-062 Audit notification system family across real surfaces

docs/design/DF-062-audit-notification-system-family-across-real-surfaces.md

```
title: DF-062 Audit notification system family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - notifications
```

Framing

The notification system is the family for transient events that need more structure than one directly composed toast(): stacking, placement, tones, actions, dismissal, archive/history review, and shell routing.

This audit verifies the family against real story rendering and runtime behavior. The point is not to invent another notice component. The point is to prove that pushNotification(), dismissNotification(), tickNotifications(), renderNotificationStack(), renderNotificationHistory(), and framed runtime notification routing all still describe one coherent system.

Sponsored Users

- TUI app authors deciding when to use a notification system instead of a local toast, alert, log line, drawer, or modal.
- Docs readers switching DOGFOOD between interactive, static, pipe, and accessible profiles to see what survives without animation or z-order.
- Maintainers who need one focused regression covering notification lifecycle, history, story metadata, and frame-owned routing.

Hills

1. A builder can inspect DOGFOOD and see live stacks, archived history, and frame-routed notifications as one notification-system family.
2. A reader can switch notification stories through every canonical output mode without losing tone, action, ordering, recall, or routing facts.
3. A maintainer can run one cycle test that proves notification story docs, component-family doctrine, and runtime behavior have not drifted apart.

Playback Questions

- Do DOGFOOD stories show both full notification-system and app-owned transient notification usage?
- Do the stories cover live stack, history review, mixed variants, actions, and framed runtime routing?
- Do rich/static previews keep notification chrome, stacking, placement, and shell context visible?
- Do pipe and accessible lowerings preserve chronological event text without pretending overlay geometry still exists?
- Does the runtime still archive dismissed notifications after their exit phase?
- Do framed apps still route runtime issues and page notify() commands through frame-managed notifications?

Requirements

- Keep the notification-system and transient-app-notifications DOGFOOD story identities.
- Add a frame-routing variant to the notification-system story.
- Render every notification-system story variant in every canonical story profile.
- Exercise the real notification lifecycle: push, tick, dismiss, archive, render stack, and render history.
- Exercise framed runtime routing through createFramedApp() and notify().
- Keep constrained lowerings plain, chronological, and semantic-preserving.

Acceptance Criteria

- tests/cycles/DF-062/notification-system-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates DOGFOOD stories for notification-system and transient-app-notifications.
- Every documented notification variant renders non-empty output in every profile.

- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.
- Constrained lowerings preserve live-stack events, history review, framed routing facts, and app-owned transient notices.
- Runtime lifecycle assertions prove notifications can be pushed, ticked, dismissed, archived, stacked, and rendered as history.
- Framed app assertions prove runtime issues and page notify() commands route through frame-managed notifications.

Implementation Outline

1. Add a framed-routing variant to the notification-system DOGFOOD story.
2. Add a DF-062 cycle test that renders notification stories through the shared story protocol.
3. Add runtime assertions for direct notification state and framed app routing.
4. Move the backlog note into docs/design/ and update the v6 lane pointer.

Drift Check

The runtime already had deep notification support: stack rendering, history rendering, animated entry/exit phases, history archive, frame-owned runtime notifications, and page-level notify() routing.

The drift was explanatory. DOGFOOD showed stack and history, but it did not make frame-routed notifications visible as part of the same family. This slice adds that story variant and binds it to runtime assertions.

Playback

- RED: the release lane had only a backlog note, and DOGFOOD did not visibly connect frame-managed runtime notifications to the notification-system family.
- GREEN: the new cycle test renders notification-system and transient notification stories across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve shell context, stacking, placement, and notification chrome.
- Pipe previews preserve chronological event and history text without box drawing.
- Accessible previews linearize tone, action, recall, and frame-routing facts.
- Runtime assertions prove direct notification lifecycle and framed routing behavior.

Retrospective

This cycle keeps notifications as a system, not a grab bag of overlays. The important boundary is that local toast() is for one directly composed event, while the notification system owns lifecycle, recall, routing, and shell-level review.

102

DF-063 Audit app shell family across real surfaces

docs/design/DF-063-audit-app-shell-family-across-real-surfaces.md

```
title: DF-063 Audit app shell family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - shell
```

Framing

The app shell is the place where page content, global status, command discovery, overlays, and shell-owned chrome meet. createFramedApp() owns the high-level shell, while statusBar() / statusBarSurface() and commandPalette() / commandPaletteSurface() are the smaller seams that make shell state visible without pushing unrelated metadata into page content.

This audit verifies the existing DOGFOOD app-shell story against real story rendering, component-family doctrine, and mode-aware lowerings. It does not redesign the framed runtime; it turns current shell story truth into executable release proof.

Sponsored Users

- TUI app authors building multi-page framed applications with global status and command discovery.
- Docs readers checking how a rich shell lowers when command palettes, status rails, and background interactivity are unavailable.
- Maintainers who need one focused regression tying app-shell story metadata, variants, and component-family guidance to real output.

Hills

1. A builder can inspect DOGFOOD and see the difference between framed page chrome and command-discovery overlay behavior.

2. A reader can switch both app-shell variants through interactive, static, pipe, and accessible profiles while preserving current page, status, and command-discovery facts.
3. A maintainer can run one cycle test that proves shell story metadata and design-system guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the family through the app-shell story?
- Do variants cover both framed-page status chrome and command-discovery overlay behavior?
- Do rich/static previews keep header, footer, page content, and palette geometry visible?
- Do pipe lowerings preserve minimal shell context without pretending hidden regions are still interactive?
- Do accessible lowerings linearize page, status, and command-discovery facts in reading order?
- Do component-family docs still separate shell chrome from page content, notification events, and command execution?

Requirements

- Keep the existing app-shell story identity and app-shell coverage family id.
- Render every app-shell story variant in every canonical story profile.
- Treat box drawing and status-rail fill characters as visual-profile chrome only.
- Keep pipe and accessible lowerings shell-context-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-063/app-shell-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD createFramedApp() / statusBarSurface() / commandPaletteSurface() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.
- Pipe and accessible output preserve shell title, current page, status, and command palette intent.

- Component-family docs retain framed app, status bar, command palette, shell-state, pipe, and accessible guidance.

Implementation Outline

1. Add a DF-063 cycle test that renders the app-shell DOGFOOD story through the shared story protocol.
2. Read-test the app shell section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. The existing app-shell story already distinguishes framed page chrome from command discovery, renders structured shell surfaces in interactive/static profiles, and lowers to minimal page/status/palette summaries in pipe/accessibility modes.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-063 playback test tied app shell behavior to every canonical profile.
- GREEN: the new cycle test renders both framed-page and command-discovery variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve header, footer, page, and command-palette geometry.
- Pipe previews preserve current page and shell status without box drawing.
- Accessible previews linearize the same shell facts in reading order.

Retrospective

The app-shell story did not need runtime changes. The useful release work was proving that shell chrome is shared application truth, not incidental drawing inside a page renderer.

103

DF-064 Audit keybinding help and shell hints family across real surfaces

docs/design/DF-064-audit-keybinding-help-and-shell-hints-family-across-real-surfaces.md

```
title: DF-064 Audit keybinding help and shell hints family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - help
```

Framing

Keybinding help is shell infrastructure, not decorative footer copy. The family starts with createKeyMap() as structured shortcut truth, then lowers that truth through full grouped references (helpView() / helpViewSurface()), compact shell hints (helpShort() / helpShortSurface()), and filtered views (h elpFor() / helpForSurface()).

This audit verifies the existing DOGFOOD help-view story against real story rendering, component-family doctrine, and mode-aware lowerings. It does not redesign keymaps; it turns the current help and hint contract into executable release proof.

Sponsored Users

- TUI app authors building keyboard-owned shells that need shortcut discovery without turning the command palette into a help page.
- Docs readers checking how compact hints and grouped help survive without borders, color, or rich surface chrome.
- Maintainers who need one focused regression tying help story metadata, variants, and component-family guidance to real output.

Hills

1. A builder can inspect DOGFOOD and understand when to show a compact shell hint versus a grouped shortcut reference.
2. A reader can switch both help variants through interactive, static, pipe, and accessible profiles while preserving shortcut grouping and action labels.
3. A maintainer can run one cycle test that proves help and shell-hint story metadata and design-system guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the family through the help-view story?
- Do variants cover both compact shell hints and grouped shortcut reference?
- Do rich/static previews keep structured surface rendering on the Surface path?
- Do pipe lowerings keep shortcut summaries and grouped help blocks readable without box chrome?
- Do accessible lowerings preserve group names, shortcut keys, action labels, and shell scope in reading order?
- Do component-family docs still separate help discovery from command execution and status/event messaging?

Requirements

- Keep the existing help-view story identity and keybinding-help-and-shell-hints coverage family id.
- Render every help story variant in every canonical story profile.
- Treat box drawing as visual-profile chrome only.
- Keep pipe and accessible lowerings shortcut-label-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-064/keybinding-help-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD helpView() / helpShortSurface() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.

- Pipe and accessible output preserve compact shell hints, grouped help title, group names, shortcut keys, and action labels.
- Component-family docs retain createKeyMap(), help view, short hint, filtered help, shell-scope, pipe, and accessible guidance.

Implementation Outline

1. Add a DF-064 cycle test that renders the help-view DOGFOOD story through the shared story protocol.
2. Read-test the keybinding help and shell hints section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. The existing help-view story already distinguishes compact shell hints from grouped shortcut reference, renders structured surfaces in interactive/static profiles, and lowers to plain shortcut summaries plus grouped help text in pipe/accessibility modes.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-064 playback test tied keybinding help to every canonical profile.
- GREEN: the new cycle test renders both shell-hint and grouped-reference variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve structured visual containment and shortcut rhythm.
- Pipe previews preserve shortcut summaries and grouped help blocks without box drawing.
- Accessible previews linearize the same shortcut groups and action labels in reading order.

Retrospective

The keybinding help family did not need runtime changes. The useful release work was proving that shortcut discovery is structured shell truth, so future blocks can reuse keymaps without rebuilding help text by hand.

104

DF-065 Audit workspace layout family across real surfaces

docs/design/DF-065-audit-workspace-layout-family-across-real-surfaces.md

```
title: DF-065 Audit workspace layout family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - layout
```

Framing

Workspace layout is the family that proves spatial composition is semantic work, not decoration. splitPaneSurface() and gridSurface() own the rich TUI surface path for simultaneous regions, while splitPane(), grid(), flex(), vstack(), hstack(), and place() remain the lower-level composition seams for explicit text and surface layout.

This audit verifies the existing DOGFOOD workspace-layout story against real story rendering, component-family doctrine, and mode-aware lowerings. It does not redesign split, grid, flex, stack, or placement algorithms; it turns the current workspace-layout story into release proof.

Sponsored Users

- TUI app authors composing primary and secondary work regions in one shell.
- Docs readers checking when spatial layout is useful and when sequential flow is clearer.
- Maintainers who need one focused regression tying workspace story metadata, variants, and design-system guidance to rendered output.

Hills

1. A builder can inspect DOGFOOD and see the difference between split context and named-area grid composition.
2. A reader can switch both workspace-layout variants through interactive, static, pipe, and accessible profiles while preserving region identity.
3. A maintainer can run one cycle test that proves workspace-layout story metadata and family guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the family through the workspace-layout story?
- Do variants cover both split-pane and grid workspace composition?
- Do rich/static previews keep spatial containment visible?
- Do pipe lowerings preserve a sensible sequential reading order?
- Do accessible lowerings linearize the same labeled regions predictably?
- Do component-family docs still cover split, grid, flex, stack, and placement seams without pretending layout geometry is content?

Requirements

- Keep the existing workspace-layout story identity and coverage family id.
- Render every workspace-layout story variant in every canonical story profile.
- Treat box drawing, dividers, and named-area geometry as visual-profile chrome only.
- Keep pipe and accessible lowerings region-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture for split, grid, flex, stack, and placement primitives.

Acceptance Criteria

- tests/cycles/DF-065/workspace-layout-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates the DOGFOOD splitPaneSurface() / gridSurface() story and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output includes structured visual containment; pipe and accessible output do not depend on box drawing.
- Pipe and accessible output preserve split and grid region identity.
- Component-family docs retain split, grid, flex, stack, placement, content guidance, and lowering guidance.

Implementation Outline

1. Add a DF-065 cycle test that renders the workspace-layout DOGFOOD story through the shared story protocol.
2. Read-test the workspace layout section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. The existing workspace-layout story already distinguishes split-pane context from grid dashboard composition, renders structured region geometry in interactive/static profiles, and lowers to region summaries in pipe/accessibility modes.

The story intentionally demonstrates the two richest workspace compositions: split panes and grids. The component-family guide remains responsible for naming the lower-level flex, stack, and placement seams that apps use to build those regions.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-065 playback test tied workspace layout behavior to every canonical profile.
- GREEN: the new cycle test renders both split-context and dashboard-grid variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve split dividers, grid regions, and nested panel containment.
- Pipe previews preserve region order without box drawing.
- Accessible previews linearize the same workspace facts in reading order.

Retrospective

The workspace-layout story did not need runtime changes. The useful release work was proving that spatial layout is shared workspace truth, while constrained lowerings keep the same region semantics without pretending visual geometry survived.

105

DF-066 Audit data visualization family across real surfaces

docs/design/DF-066-audit-data-visualization-family-across-real-surfaces.md

```
title: DF-066 Audit data visualization family across real surfaces
legend: DF
lane: design
priority: medium
keywords:
  - component-audit
  - dogfood
  - lowerings
  - visualization
```

Framing

The data visualization family proves that Bijou can render dense numeric signals without making glyph art the only source of truth. sparkline(), brailleChartSurface(), statsPanelSurface(), and perfOverlaySurface() should use rich terminal geometry when available, then lower to explicit numeric summaries when visual density is unavailable.

This audit verifies the existing DOGFOOD data visualization stories against real story rendering, component-family doctrine, and mode-aware lowerings. It does not redesign charting; it turns current data-viz runtime truth into executable release proof.

Sponsored Users

- TUI app authors building dashboards, performance overlays, and status summaries from first-party visualization blocks.
- Docs readers checking what happens when Unicode blocks, Braille cells, borders, and color are not meaningful output channels.
- Maintainers who need one focused regression tying data-viz story metadata, variants, and component-family guidance to real output.

Hills

1. A builder can inspect DOGFOOD and see when to use inline trends, dense area charts, metric panels, or the prebuilt performance overlay.
2. A reader can switch every data-viz story through interactive, static, pipe, and accessible profiles while preserving the numeric facts behind the visual shape.
3. A maintainer can run one cycle test that proves data-viz story metadata and design-system guidance have not drifted away from runtime behavior.

Playback Questions

- Does DOGFOOD expose the data visualization family through all four shipped stories?
- Do variants cover inline sparklines, fixed ranges, Braille charts, metric panels, and performance overlays?
- Do rich/static previews preserve terminal-native visual density?
- Do pipe lowerings replace visual density with explicit numeric summaries?
- Do accessible lowerings describe the same trend and metric facts in reading order?
- Do component-family docs still position the family as informative numeric visualization rather than decoration?

Requirements

- Keep the existing story identities: sparkline, braille-chart, stats-panel, and perf-overlay.
- Keep the shared data-visualization coverage family id.
- Render every data-viz story variant in every canonical story profile.
- Treat block glyphs, Braille glyphs, and borders as visual-profile affordances only.
- Keep pipe and accessible lowerings numeric-fact-first and semantic-preserving.
- Confirm docs/design-system/component-families.md still matches the runtime story posture.

Acceptance Criteria

- tests/cycles/DF-066/data-visualization-family-audit.test.ts proves the cycle doc carries the modern playback sections.
- The test locates all four DOGFOOD data-viz stories and verifies expected variants.
- Every variant renders non-empty output in every documented profile.
- Rich/static output preserves the expected visual affordance for each story.

- Pipe and accessible output do not depend on block or Braille glyphs.
- Pipe and accessible output preserve sample counts, ranges, latest values, trend notes, metric labels, and performance facts.
- Component-family docs retain numeric trend, chart, metric panel, performance overlay, fixed-axis, pipe, and accessible guidance.

Implementation Outline

1. Add a DF-066 cycle test that renders all data-viz DOGFOOD stories through the shared story protocol.
2. Read-test the data visualization section in docs/design-system/component-families.md.
3. Move the backlog note into docs/design/ and update the v6 lane pointer.
4. Record any drift as follow-on backlog debt instead of widening this audit.

Drift Check

No runtime drift was found in this slice. The existing data visualization stories already render visual glyph density in interactive/static profiles, lower sparkline and Braille previews to numeric summaries in pipe mode, and linearize metric and performance summaries in accessible mode.

No follow-on backlog item is needed for this slice.

Playback

- RED: the release lane had only a backlog note; no DF-066 playback test tied all shipped data-viz stories to every canonical profile.
- GREEN: the new cycle test renders all sparkline, braille-chart, stats-panel, and perf-overlay variants across interactive, static, pipe, and accessible profiles.
- Rich/static previews preserve block, Braille, or boxed metric affordances as appropriate.
- Pipe previews preserve numeric summaries without block or Braille glyphs.
- Accessible previews linearize the same metric and trend facts in reading order.

Retrospective

The visualization components did not need runtime changes. The useful release work was proving that visual density is backed by data facts, so future block and layout work can reuse these components without treating charts as opaque terminal art.

106

DF-067 Prove Responsive DOGFOOD Layout Variants

docs/design/DF-067-prove-responsive-dogfood-layout-variants.md

```
title: DF-067 Prove Responsive DOGFOOD Layout Variants
legend: DF
lane: design
priority: high
keywords:
  - dogfood
  - responsive
  - layout
  - variants
  - resize
```

Framing

DOGFOOD already responds mechanically to terminal resize events: the runtime updates viewport dimensions, the framed shell recomputes body geometry, and the current grid primitives solve new integer rectangles.

That is not enough.

The current docs shell keeps fixed sidebars and asks the middle track to absorb whatever space remains. That is valid geometry, but below useful widths it turns the app into a technically present three-column surface whose content is no longer the primary thing.

This cycle keeps the work DOGFOOD-scoped. It does not wait for the full RE-035 layout envelope runtime. It proves the editorial layer first: resize handling is mechanical, responsive layout is editorial.

Sponsored Users

- Readers opening DOGFOOD in small terminals.
- Maintainers validating layout doctrine against a real product surface.
- Agents checking whether DOGFOOD remains useful at constrained sizes instead of merely nonblank.

Hills

1. At wide sizes, a reader sees navigation, content, and metadata/variant context together.
2. At standard sizes, a reader keeps navigation and content while secondary metadata folds away.
3. At narrow sizes, content becomes primary while navigation remains reachable.
4. At tiny sizes, DOGFOOD becomes a single useful pane instead of a crushed multi-column layout.

Playback Questions

- Does DOGFOOD classify terminal size into explicit layout variants?
- Does 140x40 keep the wide three-region docs/product shape?
- Does 100x30 fold metadata while preserving navigation and content?
- Does 72x20 keep content primary and navigation reachable?
- Does 50x14 render a single useful pane with no fake sidebars?
- Do the guide pages and component explorer both obey the same variant policy?
- Does resize update the variant and viewport state without requiring a restart?

Requirements

- Add a pure resolveDocsLayoutVariant(columns, rows) helper returning: wide, standard, narrow, or tiny.
- Store the current layout variant in DOGFOOD page models so framed page layout functions can select a layout tree without depending on render-time hacks.
- Keep current wide layouts intact.
- Fold guide metadata and component variants out of standard and narrow layouts.
- Use a smaller navigation track in narrow.
- Use one content pane in tiny.
- Keep resize synchronization responsible for the page model’s variant and viewport heights.

Acceptance Criteria

- tests/cycles/DF-067/responsive-dogfood-layout-variants.test.ts proves the cycle doc carries the modern playback sections.
- The test proves the variant resolver thresholds.
- The test renders guide pages at 140x40, 100x30, 72x20, and 50x14.

- The test renders the component explorer at 140x40, 72x20, and 50x14.
- The test proves a live resize from wide to tiny updates the model and output.

Implementation Outline

1. Add the DOGFOOD layout variant type and resolver.
2. Add layoutVariant to DocsExplorerModel.
3. Update DOGFOOD sync to recompute the variant from FrameModel.columns and FrameModel.rows.
4. Split guide and component page layout construction by variant.
5. Update DOGFOOD docs/legend notes with the responsive posture.

Drift Check

This is not a replacement for RE-035.

DF-067 deliberately uses the current framed shell, grid, pane, and viewport primitives. The goal is to create product pressure and executable evidence for later runtime-engine responsive doctrine without expanding the runtime API prematurely.

Playback

- RED: DOGFOOD redrew on resize but kept fixed sidebars even when the terminal could no longer support them.
- GREEN: DOGFOOD selects explicit layout variants and preserves useful content at wide, standard, narrow, and tiny sizes.
- Guide pages now render nav/content/meta at wide, nav/content at standard and narrow, and content-only at tiny.
- The component explorer now renders family/story/variants at wide, family/story at standard and narrow, and story content only at tiny.
- Resize synchronization updates each page model's variant, visible pane focus, pane scroll map, and list viewport height.

Retrospective

This closes the immediate product gap without expanding the runtime layout API. The current grid/frame primitives are enough to prove the editorial rule: constrained layouts should choose a useful surface, not preserve a wider shape until it becomes unreadable.

107

DF-069 - Block-Authored DOGFOOD

docs/design/DF-069-block-authored-dogfood.md

Linked legend: DF - DOGFOOD Field Guide

Sponsor human

DOGFOOD should prove that Bijou applications can be authored from Blocks, not only that Blocks can be previewed inside DOGFOOD.

Sponsor agent

An agent should be able to inspect DOGFOOD and identify the semantic blocks that make up the product surface without reverse-engineering bespoke render helpers.

Hill

DOGFOOD becomes a block-authored application at the semantic surface boundary: title screen, shell regions, navigation, articles, settings, story previews, block previews, and inspector panes are declared as Blocks. Components remain the lower-level rendering vocabulary inside those Blocks.

Core Rule

Blocks own product semantics. Components own leaf rendering.

That means this work should create and consume Blocks for surfaces such as:

- TitleScreenBlock
- NavigationListBlock
- DocumentationArticleBlock
- SettingsMenuBlock
- StorybookWorkbenchBlock
- BlockPreviewBlock

- GuideInspectorBlock

It should not wrap every boxSurface(), markdown(), or list row in a Block.

Architecture

The target composition is:

```
DOGFOOD app
  -> AppFrame / AppShell composition
    -> page and region Blocks
      -> components
        -> surfaces
```

The existing unidirectional data-binding rule still applies:

```
business logic / providers
  -> immutable snapshots
  -> binding frames
  -> blocks and views render
  -> command intents
  -> business logic owns the next state
```

Block-authored DOGFOOD must not introduce hidden provider registries, callback backchannels, render-time refresh hooks, mutable view stores, or direct provider reads from rendering code.

Storybook Implication

The standalone Storybook app should stop being a parallel bespoke TUI. It should use the same framed shell path as DOGFOOD and expose a StorybookWorkbenchBlock contract so story preview work exercises the product composition model.

Non-goals

- no full visual redesign of DOGFOOD
- no attempt to convert every low-level component into a Block
- no new provider lifecycle system
- no new localization runtime
- no hidden global block registry
- no rendered AppShell policy invented outside existing contracts

Accessibility / Assistive Posture

Block-authored DOGFOOD should make semantic regions easier to lower across interactive, static, pipe, and accessible modes. Blocks should preserve facts about navigation, content, settings, previews, and inspector state without requiring agents or assistive modes to parse terminal art.

Localization / Directionality Posture

Blocks should receive localized labels and copy through DOGFOOD's localization port, not through direct catalog access. Block metadata may describe the surface, but runtime-visible labels remain localized app data.

Agent Inspectability / Explainability Posture

Agents should be able to ask which DOGFOOD Blocks are active, what data requirements they declare, which command intents they expose, and which surface they render into.

Linked Invariants

- Docs Are the Demo
- Runtime Truth Wins
- Commands Change State, Effects Do Not
- Tests Are the Spec

Implementation Outline

1. Update BEARING so the next product gravity is block-authored DOGFOOD.
2. Add DOGFOOD-local block registry helpers for semantic app surfaces.
3. Add StorybookWorkbenchBlock and move Storybook toward the framed shell.
4. Add DOGFOOD Blocks for title, navigation, documentation article, settings, block preview, and guide inspector surfaces.
5. Prove the block-authored registry without rendering every block during discovery.
6. Keep visual changes incremental and scoped.

Tests To Write First

- behavior tests proving the BlockLab app is framed rather than bespoke
- behavior tests proving DOGFOOD block registry discovery does not call render
- behavior tests proving core DOGFOOD surfaces have Block definitions
- behavior tests proving localization and command intent boundaries stay declarative

Playback

DF-069A lands the block-authored DOGFOOD contract layer:

- DOGFOOD has a local, branded DogfoodBlockRegistry.
- Registry discovery rejects loose block-shaped objects.
- Registry discovery records surface ownership without calling render().
- BlockLab now has a BlockLabWorkbenchBlock contract and the interactive BlockLab entrypoint uses the AppFrame shell.
- DOGFOOD surface blocks now cover title, navigation, documentation article, block preview, guide inspector, settings, and BlockLab workbench surfaces.

Retrospective

This slice deliberately stops at semantic surface contracts and the BlockLab frame migration. It does not rewrite every DOGFOOD renderer yet, does not move provider lifecycle policy, and does not convert leaf components into Blocks.

The final review gate also tightened the DOGFOOD i18n debt scanner so framed BlockLab machine tokens such as key names, event discriminants, mode literals, pane overflow policy, tones, and internal thrown errors do not count as visible English copy. BlockLab labels and rendered text still count as localization debt until they move behind a localization port.

108

DF-070 - DOGFOOD Block Product Polish

docs/design/DF-070-dogfood-block-product-polish.md

Linked legend: DF - DOGFOOD Field Guide

Sponsor Human

DOGFOOD should stop feeling like a contract debugger for Blocks and start feeling like a terminal product authored from Blocks.

Sponsor Agent

An agent should be able to inspect DOGFOOD and see that semantic Blocks own the major product surfaces, while prose, navigation, preview, and settings behavior remain observable through tests instead of hand-audited screenshots.

Hill

DOGFOOD moves from “Blocks are documented here” toward “DOGFOOD is materially built from Blocks.” The first proof is product polish: word-aware prose wrapping, block-owned documentation and navigation surfaces, block-owned inspector/settings surfaces, and block/story preview parity without inventing new provider, lifecycle, localization, or callback rules.

Core Rule

Rendered DOGFOOD polish must consume existing contracts.

That means:

- prose wrapping belongs at text/prose boundaries, not arbitrary surface clipping
- Blocks own semantic product regions
- components remain the leaf rendering vocabulary
- localization goes through the localization port

- data flow stays unidirectional
- story and block previews should share the same shell assumptions

Ten-Slice Plan

1. Refresh BEARING and cycle truth after DF-069 and DX-035 merged.
2. Add word-boundary prose wrapping behavior.
3. Route DOGFOOD prose surfaces through the word-aware wrapper.
4. Move DocumentationArticleBlock onto the real DOGFOOD article render path.
5. Move NavigationListBlock onto the real DOGFOOD navigation render path.
6. Move GuideInspectorBlock onto the real DOGFOOD inspector render path.
7. Move SettingsMenuBlock onto the real DOGFOOD settings surface.
8. Persist DOGFOOD's selected locale behind the existing localization boundary.
9. Expand localization catalog coverage for the block-authored surfaces touched by this cycle.
10. Align DOGFOOD Blocks preview and Storybook preview posture so both expose rendered product examples without debug-only wrapper clutter.

The cycle pauses after slice 5 for drift review before continuing.

Non-Goals

- no new provider lifecycle system
- no hidden global block registry
- no broad visual redesign
- no conversion of every leaf component into a Block
- no AppFrame rewrite
- no localization runtime rewrite
- no tests that assert prose documentation wording instead of rendered behavior

Tests To Write First

- behavior tests proving prose wrapping breaks at word boundaries where possible
- behavior tests proving DOGFOOD article/navigation surfaces use the same rendered output paths their Blocks publish
- behavior tests proving block discovery and preview indexing do not execute unrelated render paths
- behavior tests proving locale persistence and catalog coverage changes remain behind the localization port

Retrospective

Not started.

109

DF-071 - DOGFOOD Block-Authored Surfaces

docs/design/DF-071-dogfood-block-authored-surfaces.md

Linked legend: DF - DOGFOOD Field Guide

Sponsor Human

DOGFOOD should increasingly feel like an app authored from semantic Blocks, not an app that has a Blocks documentation page on the side.

Sponsor Agent

An agent should be able to inspect DOGFOOD's product surfaces and understand which Block owns each semantic region without executing provider lifecycle, hidden registries, or view mutation paths.

Hill

DOGFOOD uses Blocks for visible app semantics across shell, docs, preview, settings, and workbench surfaces while preserving the existing component layer as the leaf rendering vocabulary.

DF-071 is not a visual redesign. It is a product-boundary pass: more visible DOGFOOD surfaces should be described, rendered, and verified through Blocks where the Block boundary is already stable.

Core Rule

Blocks own product semantics.

Components own leaf rendering.

Runtime and provider systems own lifecycle.

Localization owns text lookup through a port.

DF-071 may connect these layers, but it must not collapse them.

Fifteen-Slice Plan

1. Refresh BEARING after DF-070 landed.
2. Land this design checkpoint.
3. Add missing block-authored surface coverage for shell footer/status hints.
4. Route docs footer hint text through the footer/status Block.
5. Add block-authored landing/title behavior proof.
6. Route the landing DOGFOOD panel through TitleScreenBlock data.
7. Add a registry-backed DOGFOOD surface Block inventory page.
8. Add block-authored search surface coverage.
9. Route search chrome title text through the search Block seam without changing search mechanics.
10. Add block-authored notification center coverage.
11. Add block-authored performance HUD coverage.
12. Add block-authored keyboard help coverage.
13. Add block-authored command palette coverage.
14. Update docs and changelog with the shipped public DOGFOOD surface posture.
15. Run full validation, push, and open a PR.

The plan can adapt if the code reveals a cleaner seam, but each slice must keep the same separation of concerns.

Adaptation Note

The top navigation seam is owned by the generic AppFrame tab renderer. DF-071 therefore did not fork tab rendering inside DOGFOOD. Instead, it covered the frame-owned surfaces that already have stable semantic boundaries:

- footer hints
- search title
- notifications
- performance HUD
- keyboard help
- command palette

The actual overlay renderers remain in @flyingrobots/bijou-tui. DOGFOOD now publishes local block contracts for those surfaces so the app can be inspected as a block-authored product without duplicating frame runtime behavior.

Non-Goals

- no new provider lifecycle system
- no command dispatch rewrite
- no hidden global block registry

- no localization runtime rewrite
- no conversion of leaf components into Blocks
- no broad visual redesign
- no metadata-only tests standing in for rendered behavior

Tests To Write First

- behavior tests proving routed surfaces render the user-visible text they claim to own
- behavior tests proving block registries expose the new semantic surfaces without discovery-time rendering
- behavior tests proving routed surfaces still preserve lowered-mode behavior
- behavior tests proving localization access stays behind the existing port

Retrospective

DF-071 landed the first broad pass over DOGFOOD block-authored product surfaces. It added or routed:

- FooterHintBlock
- SearchPanelBlock
- NotificationCenterBlock
- PerfHudBlock
- HelpOverlayBlock
- CommandPaletteBlock
- TitleScreenBlock landing title data
- a registry-backed DOGFOOD surface inventory page

The key restraint held: frame runtime surfaces remain rendered by @flyingrobots/bijou-tui, while DOGFOOD owns the semantic block contracts, data requirements, command intents, and visible documentation inventory.

110

DF-072 DOGFOOD Release Title Artifact Gallery

docs/design/DF-072-dogfood-release-title-artifact-gallery.md

```
title: DF-072 DOGFOOD Release Title Artifact Gallery
legend: DF
lane: cool-ideas
priority: medium
issue: https://github.com/flyingrobots/bijou/issues/289
keywords:
  - dogfood
  - release
  - release-title
  - title-gallery
```

Framing

Issue #289 asks DOGFOOD to treat release title screens as first-class release artifacts instead of replacing the last title treatment in place. V7 shipped with V7 Product Truth; this follow-up adds a unique post-release treatment and keeps both artifacts visible through the Release page.

The design follows a Design Thinking loop: observe that release identity is valuable after the release lands, frame titles as reusable proof artifacts, prototype a gallery that keeps the newest treatment first, and prove every title through interactive, static, pipe, accessible, and localized output.

Sponsor Human

- A maintainer reviewing the release wants to see that v7 was published and that follow-up work is intentionally tracked instead of silently changing the historical title.
- A DOGFOOD reader wants the Release page to open with the current title while still making the original release title discoverable.

Sponsor Agent

- A review agent needs release-title metadata, lower-mode facts, and localization keys in one typed structure instead of scattered prose.

Hill

A DOGFOOD reader can open the Release guide and see a latest-first title gallery whose current item is a unique V7 Launch Wake post-release title screen, while the original V7 Product Truth artifact remains available with its own lower-mode facts.

Playback Questions

- Does the Release guide open with the current title treatment?
- Does the historical V7 Product Truth artifact remain selectable?
- Do static, pipe, and accessible modes all lower from the same typed release-title record?
- Does every new visible string have source rows and generated catalogs for all supported DOGFOOD locales?
- Does compact interactive rendering stay within the caller's requested width?

Current Truth

- v7.0.0 is already published.
- The strict v7.0.0 GitHub milestone has no open issues.
- Issue #289 is open in the Beyond milestone and already describes the gallery shape this patch should land.
- DOGFOOD currently has one release-title metadata object, one Release guide, and localized strings for that single title.

Product Shape

Release Page Gallery

```
DOGFOOD Release

> V7 Launch Wake        v7-launch   current
  A released line leaves a readable trail.
  lanes: published release | follow-up patch | title gallery | green main CI

  V7 Product Truth      v7          historical
  Blocks prove product surfaces. DOGFOOD proves Blocks.
  lanes: table parity | scoped Node I/O | BlockLab | release title
```

Current Title

```
+ BIJOU DOGFOOD ------------------------------------------+
| V7 Launch Wake                                          |
| A released line leaves a readable trail.                |
|                                                         |
| Proof lanes: published release | follow-up patch | ...  |
```

```
| Release gate: v7.0.0 released, patch lane open           |
| Motif: Wake lines, not fireworks: proof remains visible  |
|                                                          |
| [Release Notes] [Migration] [DOGFOOD] [BlockLab]         |
+----------------------------------------------------------+
```

Narrow Title

```
+ BIJOU DOGFOOD ----+
| V7 Launch Wake    |
| Release trail.    |
| lanes: published  |
| gate: v7.0.0 ...  |
| motif: wake lines |
+-------------------+
```

Lower Modes

Static mode keeps the title useful without border art:

```
Bijou DOGFOOD release title. Release V7 Launch Wake.
Proof lanes: published release, follow-up patch, title gallery, green main CI.
Motif: Wake lines, not fireworks: proof remains visible after ship.
```

Pipe mode exposes facts for agents:

```
release_id  v7-launch
release_title   V7 Launch Wake
release_motif   Wake lines, not fireworks: proof remains visible after ship.
proof_lane  published release
proof_lane  follow-up patch
proof_lane  title gallery
proof_lane  green main CI
navigation_available    true
```

Accessible mode names the same proof without depending on layout:

```
DOGFOOD title screen for V7 Launch Wake.
Current release proof lanes are published release, follow-up patch,
title gallery, green main CI.
Visual motif: Wake lines, not fireworks: proof remains visible after ship.
Navigation remains available after the title.
```

Runtime And API Contract

- Release-title data is represented by typed DogfoodReleaseTitle records.
- DOGFOOD_RELEASE_TITLE_GALLERY is latest-first.
- CURRENT_DOGFOOD_RELEASE_TITLE points at the current title treatment.
- Rendering defaults to the current title but accepts an explicit historical title for stable artifact playback.
- DOGFOOD Release guides are generated from the gallery, not manually copied per title.

Linked Invariants

- New release-title UI strings must be represented in every supported DOGFOOD locale.
- Lower modes must expose the same release id, title, motif, proof lanes, and navigation availability as the interactive title.
- Release-title rendering must stay deterministic and width-bounded from its explicit inputs.

Accessibility, Localization, And Inspectability

- Every visible string added in this cycle has en, de, es, and fr source rows and regenerated runtime catalogs.
- The title's motif is exposed as text in interactive, static, pipe, and accessible modes.
- The renderer remains deterministic and does not query GitHub, wall-clock time, or release APIs.
- Interactive output must stay inside the requested width for compact terminals.

Tests To Write First

- Extend tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts.
- RED state:
  - no CURRENT_DOGFOOD_RELEASE_TITLE
  - no DOGFOOD_RELEASE_TITLE_GALLERY
  - no v7-launch lower-mode facts
  - Release page still exposes only release-title-v7
- GREEN state:
  - the gallery is latest-first
  - V7 Launch Wake is the current title
  - V7 Product Truth remains available as a historical guide
  - pipe output includes release_id and release_motif

Acceptance Criteria

- Issue #289 is closed by the PR.
- DOGFOOD has a unique post-release title screen for v7.0.0.
- The original v7 title treatment remains inspectable.
- Runtime catalogs are regenerated from source.
- The changelog records the follow-up.

Risks And Guardrails

- Do not rewrite the release history or imply that #289 was part of the strict v7.0.0 milestone.
- Do not make the title gallery a live GitHub integration.
- Do not remove existing V7 title metadata or tests.

111

DF-073 Raster-to-Glyph Title Art

docs/design/DF-073-raster-to-glyph-title-art.md

```
title: DF-073 Raster-to-Glyph Title Art
legend: DF
lane: release
priority: high
issue: https://github.com/flyingrobots/bijou/issues/303
keywords:
  - dogfood
  - image-filter
  - landing
  - raster
  - release
  - title-screen
  - v7
```

Framing

Issue #303 asks the current DOGFOOD landing screen to carry a unique V7 Launch Wake release identity. The first prototype used a textual wake ribbon; the current direction is stronger: render a deterministic stacked sine-wave wake as the animated title background and use the committed Bijou.svg wordmark as a background-transparent glyph mask in complementary colors.

The design follows a Design Thinking loop: observe that terminal title art is most useful when it can be generated, inspected, tested, and lowered; frame the problem as a reusable raster adapter rather than a one-off DOGFOOD paint pass; prototype a deterministic RgbaFrame -> Surface filter for vector and image masks; rasterize the committed SVG wordmark without adding a browser canvas dependency; and prove the result with synthetic-frame tests plus DOGFOOD rendered-frame tests.

Sponsor Human

- A maintainer opening DOGFOOD after v7.0.0 wants the first screen to feel release-specific without becoming a blocking splash page.
- A visual designer should be able to provide an image, generated frame, or procedural RGBA buffer and choose the terminal character set used to render it.
- A reader should still see the normal DOGFOOD entry point, prompt, footer controls, and release documentation route.

Sponsor Agent

- A review agent needs the release title, raster filter contract, renderer mode, charset, pulse behavior, and lower modes to remain inspectable through existing artifacts and tests.
- A future capture/export agent should be able to feed decoded frames through the same primitive without depending on browser canvas or VHS.

Hill

A DOGFOOD reader can open the landing title screen and see V7 Launch Wake named in the entry panel while stacked sine-wave glyph strata fill the terminal and the Bijou.svg wordmark tints those foreground glyphs in complementary colors without painting an opaque background box. Braille is one supported glyph mode, but the renderer can also use custom density character sets such as /\MXYZabc!?=-.. The screen remains pulse-driven, deterministic, bounded by the current landing quality profiles, and lowerable through existing release-title facts.

Current Truth

- V7 Launch Wake is already the current release-title artifact.
- The landing renderer is already pulse-driven, cached, and quality-bucketed.
- @flyingrobots/bijou-tui already has shader-based canvas() output for cell, quad, Braille, and fitted-glyph resolutions.
- cell-glyph-fit.ts already establishes density ramps and coverage fitting.
- Static, pipe, and accessible release-title output already comes from examples/docs/release-title.ts.

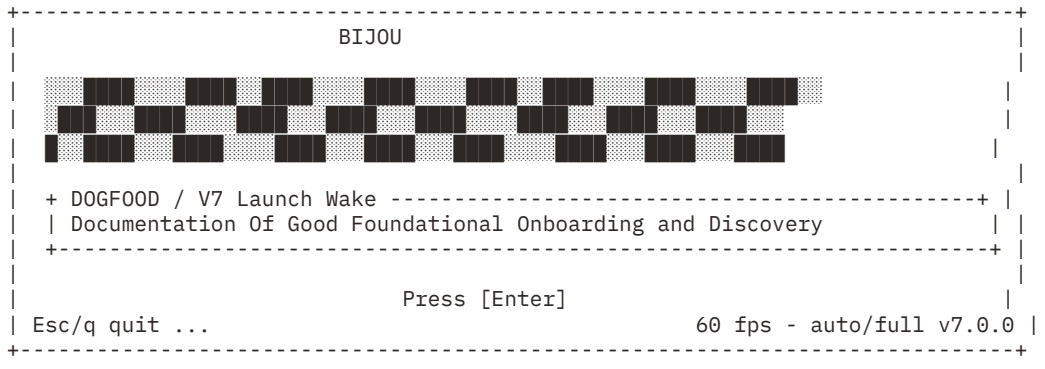
Product Shape

Wide Landing Screen

The stacked sine-wave wake occupies the first-viewport title surface. It uses the existing landing pulse shader and quality buckets, so the field stays animated without adding a new image decode path or GPU dependency. Wake colors are written into cell backgrounds while the density glyphs keep the surface geometry stable with matching foreground colors. The visible Bijou wordmark is rasterized from assets/Bijou.svg, fit to the terminal cell aspect, and used as a background-transparent mask that writes SVG glyphs without replacing existing backgrounds. The wordmark letters ride a staggered sine wave and tint their foreground fill from the landing ramps, so the logo can animate without owning a rectangular background. The lower FlyingRobots wordmark is read from assets/flyingrobotslogo.txt, treats Braille blank cells as transparent holes, fits the visible logo glyphs to the available landing

width, preserves the underlying cell background, and paints visible Braille glyphs with the opposite foreground color of that preserved background. This keeps the mark transparent through modal dimming because the logo does not own a rectangular background fill. FlyingRobots fades away after the first three seconds, leaving the Bijou wordmark, wake, release panel, prompt, footer, and version marker as the stable title composition.

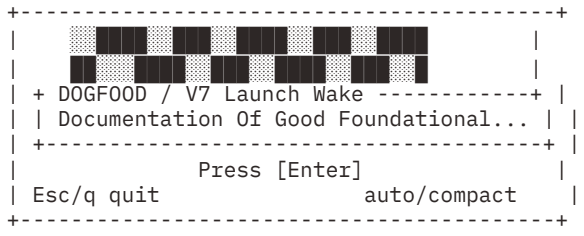
The title screen owns only title-screen controls. Enter is the only key that continues into the DOGFOOD documentation shell; backtick toggles the perf HUD in place, Esc / q use the shell quit policy, arrows and number keys adjust landing presentation, and other keys do not secretly route into docs. Pressing Enter switches the model directly to the docs route without a title-to-docs transition. That hard cut keeps the first documentation frame legible instead of briefly painting decorative raster residue over product content.



The prompt keeps the localized Press [Enter] text while using a stronger foreground gradient over the Enter letters than over the surrounding line.

Small Landing Screen

Small terminals keep the title identity and use fewer glyph cells.



Lower Modes

The art is decorative. Lower modes keep exposing semantic release facts.

```
pipe:
  release_id=v7.0.0
  release_title="V7 Launch Wake"
  release_motif="launch wake"

accessible:
  DOGFOOD opens on the V7 Launch Wake release title. The interactive
  screen includes decorative raster title art rendered as terminal glyphs.
```

Raster Adapter

The primitive belongs in @flyingrobots/bijou-tui so DOGFOOD is only the first consumer.

```
RgbaFrame
  -> fit source pixels into target cell grid
  -> sample brightness/alpha/color per cell or sub-cell
  -> choose renderer:
       charset: density ramp from custom characters
       braille: 2x4 dot mask
       quad:    2x2 quadrant mask
  -> emit Bijou Surface
```

Input

```
interface RgbaFrame {
  width: number;
  height: number;
  data: Uint8ClampedArray | readonly number[];
}
```

The public TUI primitive accepts already-decoded RGBA. DOGFOOD keeps SVG rasterization in the example app layer so the core package contract stays deterministic and dependency-free.

Renderer Modes

- charset: render each output cell with a density ramp. The caller can pass a custom glyph string such as /\MXYZabc!?=-. and choose whether it is ordered dark-to-light or light-to-dark.
- braille: render each output cell with a 2x4 Braille mask sampled from darkness/alpha.
- quad: render each output cell with a 2x2 quadrant mask sampled from darkness/alpha.

Fit Modes

- fit: preserve aspect ratio, fill the target, crop overflow, and center both axes.
- contain: preserve aspect ratio and letterbox with empty cells.
- stretch: map the frame directly into the requested grid.

cellAspectRatio is the visual width divided by height of one terminal cell. The default is 1 for square-cell math. DOGFOOD uses 0.5 for the V7 SVG wordmark mask so the rendered logo fits the visible iTerm-style viewport instead of looking vertically stretched.

Color Modes

- none: only glyphs.
- fg: average visible source pixels into foreground RGB.

- fg-bg: average visible/dark pixels into foreground and light pixels into background when available.

Determinism And Performance

- No browser canvas, wall clock, external image decoder, or GPU dependency is required.
- DOGFOOD uses the existing pulse timestamp and quality frame-step quantization.
- DOGFOOD computes the stacked sine-wave wake from quantized landing pulse time and current terminal dimensions, then paints it through the existing landing shader.
- DOGFOOD reads assets/Bijou.svg once, rasterizes the wordmark into a cached mask per terminal size, and uses the mask to write foreground glyphs without replacing the underlying backgrounds.
- DOGFOOD derives the Bijou logo row offsets, fill colors, FlyingRobots fade, and prompt gradient from quantized landing pulse time.
- Target dimensions are bounded by landing quality profiles before rendering.
- Cache keys include viewport, theme, quality, quantized time, and FPS badge state, preserving the existing landing frame cache behavior.

Accessibility / Assistive Posture

The raster title art is decorative in interactive mode. Assistive output must not depend on parsing glyph art. It continues to rely on the release title, motif, proof lanes, and accessible release prose.

Agent Inspectability / Explainability Posture

Agents should be able to prove the slice by checking:

- the current release title appears on the landing frame;
- raster art uses the configured renderer and charset;
- pulse changes the title art deterministically;
- raw RGBA filter tests cover charset, Braille, quad, fit, color, and invalid charset behavior;
- the stacked sine-wave wake remains the visible landing background;
- pipe/accessibility release facts remain sourced from release-title metadata.

Tests To Write First

- RED: rasterToGlyphSurface() does not exist or is not exported.
- RED: custom charset rendering cannot map a synthetic black/white frame.

- RED: invalid charsets are not rejected.
- RED: Braille rendering cannot map a single top-left subpixel to dot 1.
- RED: terminal fit mode stretches or letterboxes instead of centered fill-cropping.
- RED: terminal fit mode assumes square cells and crops the wrong axis in typical tall-cell terminals.
- RED: DOGFOOD landing frame does not render enough stacked sine-wave glyph art.
- RED: the committed Bijou.svg wordmark is not rasterized into the rendered landing frame.
- RED: the committed Bijou.svg wordmark replaces backgrounds or paints transparent SVG pixels instead of acting as a background-transparent mask.
- RED: the lower FlyingRobots wordmark is generated from the old wide text assets instead of assets/flyingrobotslogo.txt.
- RED: pressing backtick on the landing title enters the docs instead of toggling the shell perf HUD in place.
- RED: the landing title perf HUD omits shell HUD rows or reports a permanently zero frame time because the root DOGFOOD render loop is not hydrating timing telemetry.
- RED: DOGFOOD landing still paints the old procedural BIJOU title over the wake field.
- RED: non-Enter keys route from the title screen into the docs shell.
- RED: the lower FlyingRobots wordmark stays visible after the three-second fade window.
- RED: the Bijou SVG title mask is static instead of moving each letter through a staggered sine-wave row offset and animated fill color.
- RED: the Press [Enter] prompt does not draw attention to Enter with a gradient foreground.
- RED: pressing Enter leaves any title-to-docs transition state or decorative transition artifact instead of hard-cutting to docs.

Validation

Run focused tests first:

```
npx vitest run packages/bijou-tui/src/raster-glyph.test.ts
npx vitest run scripts/svg-raster.test.ts
npx vitest run scripts/docs-preview.test.ts -t "stacked sine-wave"
```

Then run the relevant DOGFOOD and package validation lane:

```
npx vitest run scripts/docs-preview.test.ts
npm run typecheck:test
npm run docs:inventory
npm run dogfood:i18n:check
```

Playback

This slice lands when DOGFOOD has a visible, localized V7 Launch Wake landing identity and a deterministic raster-to-glyph title treatment, without adding new strings, bypassing the landing cache, adding image decoding dependencies, or changing lower-mode release facts.

112

DF-074 Fluid Triangle Title Screen

docs/design/DF-074-fluid-triangle-title-screen.md

```
title: DF-074 Fluid Triangle Title Screen
legend: DF
lane: cool-ideas
priority: medium
status: proposed
github_issue: 321
keywords:
  - dogfood
  - title-screen
  - fluid
  - geordi
  - gpu
  - glyphs
  - render-target
```

Legend: DF - DOGFOOD

Linked Work

- GitHub Issue: #321
- Inspiration: https://github.com/javierbyte/fluid-triangle
- Prior title art: DF-073 Raster To Glyph Title Art
- Portable rendering direction: DX-042 Shared UI Scene IR And Bijou Render Target
- Portable endpoint direction: DX-043 Portable Bijou Blocks And Multi-Endpoint IR
- Related renderer experiments: RE-031 Raytrace Shader Kernel, RE-032 Fit Cell Glyphs From Coverage

This document is a planning artifact. It does not claim a GPU renderer, Geordi packed-cell target, browser endpoint, or new title screen implementation already exists in this repository.

Decision Summary

Bijou should design a future DOGFOOD title screen around a live fluid field moving around a triangular obstacle, rendered as terminal glyphs. The immediate Bijou work should define the title-scene contract, deterministic

CPU fallback, theme-token policy, and cell-level proof. The high-end renderer should be Geordi-backed when the shared render pipeline is ready.

The intended long-term flow is:

```
Fluid Title Scene Spec
  -> deterministic field state
    -> Bijou CPU glyph renderer
      -> Bijou Surface
        -> terminal

Fluid Title Scene Spec
  -> Geordi fluid/shader endpoint
    -> PackedBijouCells or image witness
      -> Bijou Surface or browser/image output
```

The goal is not to copy a browser demo into DOGFOOD. The goal is to turn the idea into a Bijou-native release-title system with deterministic witnesses and a future GPU render endpoint.

Sponsored Human

A Bijou maintainer wants a memorable title screen that feels alive, branded, and release-specific without making DOGFOOD startup flaky, slow, or impossible to test.

Sponsored Agent

An agent wants a deterministic scene description, frame inputs, glyph outputs, and render receipts so it can explain title-screen pixels and terminal cells without depending on screenshots or manual terminal observation.

Hill

DOGFOOD can show a fluid triangle title screen where the fluid motion, glyph density, color tokens, and input behavior are deterministic under test, while the production renderer can later upgrade from a Bijou CPU path to a Geordi-backed GPU path without changing the title scene’s product contract.

Inspiration Boundary

The reference project is useful because it combines:

- fluid-like motion
- a triangular obstacle
- ASCII rendering
- per-column character sequences
- a playful launch-screen posture

Bijou should borrow the interaction and rendering concept, not the browser runtime assumptions.

The reference README states that its simulation code comes from Ten Minute Physics and that the project is MIT licensed. If Bijou ports substantial code from the reference or from the underlying Ten Minute Physics simulation, the ported file must preserve the relevant MIT attribution and copyright notice. The preferred first implementation is an original Bijou renderer inspired by the concept.

Title Concept

The title screen should feel like a fluid gate opening around a triangle:

```
.....................................................................
....@@@@@###WWWW9876????abc;:+=-,._       _.,-=+:;cba????6789WWWW###@@@@@....
...@@@###WWW9876????abc;:+=-,._          /\          _.,-=+:;cba????6789W....
..@@###WW9876????abc;:+=-,._            /  \            _.,-=+:;cba????67....
..@###W9876????abc;:+=-,._             /    \             _.,-=+:;cba????....
..@@###WW9876????abc;:+=-,._           \    /            _.,-=+:;cba????67....
...@@@###WWW9876????abc;:+=-,._         \  /          _.,-=+:;cba????6789....
....@@@@@###WWWW9876????abc;:+=-,._      \/      _.,-=+:;cba????6789WWW....
.....................................................................

                              BIJOU
                          Press [Enter]
```

The actual renderer should not draw this literal mockup. This is a spatial intent sketch:

- fluid surrounds a central triangular collider
- density changes map to glyph ramps
- the BIJOU mark remains readable above or inside the field
- the Enter prompt remains clear and localized
- the perf HUD and quit controls keep their existing ownership

Scene Model

The title should be modeled as data before it is rendered.

```
type FluidTitleSceneSpec = {
  id: 'dogfood.fluid-triangle-title';
  version: 1;
  seed: number;
  cols: number;
  rows: number;
  quality: 'low' | 'medium' | 'high' | 'auto';
  themeMode: 'dark' | 'light';
  triangle: {
    centerX: number;
    centerY: number;
    radius: number;
    rotation: number;
  };
  glyphRamps: readonly string[];
  titleKey: 'dogfood.title.bijou';
  promptKey: 'dogfood.title.pressEnter';
};
```

The scene spec owns intent. Renderers own lowering.

Renderer Targets

Bijou CPU Renderer

The first implementation should be a deterministic Bijou CPU renderer. It does not need to be a full fluid solver. It needs to produce the right motion language, stay bounded, and be testable.

Acceptable first algorithms:

- advected scalar density field
- curl-noise-like deterministic flow
- lightweight particle splat field
- simplified stable fluid pass

The CPU renderer should emit:

```
FluidTitleSceneSpec + frame index
  -> FluidTitleFrame
    -> Surface
    -> FluidTitleReceipt
```

Geordi GPU Renderer

The intended high-end path is Geordi-rendered:

```
FluidTitleSceneSpec
  -> Geordi fluid/shader endpoint
    -> rgba image target
    -> PackedBijouCells target
    -> frame witness
```

The Geordi endpoint should be allowed to compute richer fluid motion, higher resolution fields, better collider interaction, and image/video witnesses. Bijou should consume its output through a declared target profile rather than special-case a browser canvas.

Low Mode

Low mode must remain useful without GPU support.

```
seeded density wave
  -> glyph ramp
    -> token colors
      -> Surface
```

Low mode should be cheap enough for CI snapshots and slow terminals.

Quality Modes

Mode	Renderer	Purpose
low	Bijou CPU density wave	CI, SSH, constrained terminals
medium	Bijou CPU particle or scalar field	default local terminal
high	Geordi GPU endpoint when available	release capture, browser/image witness
auto	runtime choice	best available mode under frame budget

auto must degrade deterministically by recorded facts:

```
{
  "requestedQuality": "auto",
  "selectedQuality": "medium",
  "reason": "geordi-endpoint-unavailable",
  "frameBudgetMs": 16
}
```

Theme And Color Policy

The renderer must use theme tokens rather than raw title-screen colors.

Candidate tokens:

```
title.fluid.bg
title.fluid.low
title.fluid.mid
title.fluid.high
title.triangle.edge
title.triangle.fill
title.logo.fg
title.prompt.fg
title.prompt.enter.fg
```

The glyph ramp and color ramp should be separate:

```
density -> glyph
density -> token/color
```

This avoids the prior title-screen failure mode where a visual layer appeared to set the wrong background because glyph, foreground, and background channels were not explicit enough.

Input Contract

The title screen must keep the current input discipline:

- Enter proceeds to DOGFOOD.
- Backtick toggles the normal shell perf HUD.
- q opens the normal quit confirmation.
- Theme or quality controls may remain title-owned only if documented.
- No other ordinary key should accidentally skip the title.

Future optional interactions can perturb the field:

- mouse move nudges velocity
- arrow keys shift the triangle
- terminal resize reseeds layout while preserving deterministic frame facts

Those interactions are not required for the first slice.

Localization And Directionality

The visible title and prompt must be localization-backed strings. The fluid field itself is visual and does not require localization, but lower modes should expose the string keys.

RTL posture:

- The prompt should use the app's existing text direction rules.
- The fluid field can remain spatially symmetric.
- Per-column glyph ramps should not encode English-only words in default mode.

The reference project's per-column letter sequences are useful as a visual trick, but a Bijou default should not rely on hidden English text in the fluid field.

Accessibility And Assistive Posture

The title screen must not be the only source of release information.

Required posture:

- prompt remains static enough to read
- low-motion mode can freeze or heavily slow the field
- status/lower-mode facts identify the release title
- perf HUD remains accessible from the title screen
- the screen does not flash at unsafe contrast/frequency patterns

Agent Inspectability

Every deterministic render pass should be able to emit a receipt:

```
{
  "sceneId": "dogfood.fluid-triangle-title",
  "frame": 120,
  "seed": 700,
  "renderer": "bijou-cpu-density",
  "quality": "medium",
  "triangle": { "centerX": 80, "centerY": 18, "radius": 9 },
  "glyphRampHash": "fnv1a32:...",
  "surfaceHash": "fnv1a32:...",
  "tokenRefs": [
    "title.fluid.low",
    "title.fluid.mid",
    "title.fluid.high",
    "title.prompt.enter.fg"
```

```
  ]
}
```

For Geordi output, the receipt should add endpoint facts:

```
{
  "renderer": "geordi-fluid-gpu",
  "endpoint": "packed-bijou-cells",
  "shaderHash": "sha256:...",
  "packedCellsHash": "fnv1a32:...",
  "witnessHash": "sha256:..."
}
```

Playback Questions

1. Does the first frame read as Bijou before animation is observed?
2. Does Enter remain the only key that proceeds to DOGFOOD?
3. Does the prompt stay legible at 80x24 and larger terminals?
4. Can the title run in low mode without Geordi?
5. Can a test prove the triangle collider leaves a stable void or edge?
6. Can a receipt explain the selected renderer, quality, seed, tokens, and output hash?
7. Can the Geordi endpoint later replace the high-quality renderer without changing title-screen input behavior?

Implementation Outline

1. Add the scene spec and receipt types.
2. Add a deterministic CPU low-mode density renderer.
3. Add glyph-ramp and theme-token lowering into Surface.
4. Add title-screen integration behind a release-title switch.
5. Preserve existing perf HUD, quit, and Enter-only routing contracts.
6. Add a low-motion/frozen frame path.
7. Add tests for frame determinism, prompt visibility, key routing, and token use.
8. Add Geordi endpoint target profile once packed-cell or image witness support is ready.

Tests To Write First

- Given the same spec, seed, size, and frame, the CPU renderer produces the same surfaceHash.
- Given different frame indexes, at least one fluid-region cell changes while the prompt remains stable.
- Given 80x24, the prompt and title are visible and not overwritten by fluid cells.
- Given non-Enter printable key input, DOGFOOD remains on the title route.

- Given Enter, DOGFOOD proceeds to the main docs route.
- Given backtick, the normal perf HUD toggles while the title route remains active.
- Given low-motion mode, repeated frames produce either identical output or a documented bounded change.
- Given missing Geordi endpoint, auto selects a Bijou CPU renderer and emits a reason fact.

Non-Goals

- No browser canvas dependency in DOGFOOD startup.
- No required GPU path for CI.
- No direct code port from the reference project in the design slice.
- No change to the public title-screen skip contract beyond preserving Enter-only proceed.
- No claim that this implements shared UI scene IR.

Slice Plan

Slice	Outcome	Proof
1	Design doc and issue	docs inventory
2	Scene spec and receipt types	type tests
3	Deterministic low-mode renderer	surface hash tests
4	DOGFOOD title integration	docs-preview tests
5	Input contract preservation	key routing tests
6	Theme-token color ramp	token receipt tests
7	Low-motion/frozen path	deterministic frame tests
8	Geordi endpoint design handoff	companion issue/doc

Closeout Notes

Open. This design should be closed only after the title screen either ships or is deliberately superseded by a different release-title direction.

113

DF-075 DOGFOOD Post-Release Polish

docs/design/DF-075-dogfood-post-release-polish.md

```
title: DF-075 DOGFOOD Post-Release Polish
legend: DF
lane: asap
priority: medium
status: proposed
github_issue: 334
keywords:
  - dogfood
  - app-shell
  - quit-confirm
  - blocks
  - polish
```

Legend: DF - DOGFOOD

Linked Work

- GitHub Issue: #334
- Related future release-story work: #335
- Related future affordance work: #336

This document scopes a bounded post-release polish cycle. It does not reopen the v7.1.0 feature train and it does not claim the broader V8 Runtime Graph and Scene IR product surface.

Decision Summary

DOGFOOD should absorb the three most visible post-release polish findings from the v7.1.0 sanity pass without expanding into a broad shell redesign:

1. Disable the hidden double-Tab footer toggle until the AppShell has a visible command, animation, and restore affordance.
2. Restyle the quit confirmation so it is compact, themed, aligned, and easy to answer in the default theme.
3. Make the Blocks section lead with CounterDemoBlock, because that is the most alive interactive proof of what Blocks are.

Sponsored Human

A DOGFOOD user wants the docs app to feel deliberate after v7.1.0: no hidden state that leaves terminal background behind, no awkward quit prompt, and a Blocks page that immediately demonstrates an interactive block instead of starting on descriptive prose.

Sponsored Agent

An agent wants deterministic behavior to assert: footer visibility should not change through undocumented key timing, the quit dialog should have stable themed geometry, and the Blocks default route should identify the same guide id that renders the interactive counter proof.

Hill

DOGFOOD can start the post-v7.1.0 era with a cleaner shell and a stronger Blocks first impression while keeping the patch small, testable, and isolated from V8 product-contract work.

Scope

Disable Hidden Footer Toggle

The current hidden double-Tab footer toggle is not discoverable. When hidden, it can expose uncolored terminal background and does not explain how to recover the footer. This cycle should remove that key path from normal DOGFOOD/AppShell input handling. Tab should remain the normal focus traversal key.

This does not forbid a future footer command. A future version can reintroduce footer visibility as an explicit command with visible state, animation, help text, and theme-safe repaint behavior.

Restyle Quit Confirmation

The quit confirmation should render as a compact modal rather than a tall, skinny prompt. Its background, border, title, body, and actions should use the selected frame theme instead of leaving unthemed terminal patches.

The action labels must be readable in the default DOGFOOD theme. The dialog should keep the existing command contract: confirm quits, reject closes the dialog, and normal app state remains underneath.

Counter-First Blocks

The Blocks section should open directly on CounterDemoBlock. Selecting the Bl ock Preview parent row should also route to CounterDemoBlock, making the preview group demonstrate an interactive block first.

Static and descriptive block previews remain available below it. The prior navigation fix that lets focus move from the first preview child back to the Block Preview parent row must not regress.

Out Of Scope

- #335 release-story surfaces: What's New, GraphQL proof, and CHANGELOG viewer.
- #336 future affordances: theme hotkey, tab badges, tutorial, and achievements.
- A full AppShell animation or layout redesign.
- A new v7.2.0 feature train.
- Broad V8 Runtime Graph and Scene IR product-contract work.

Playback Questions

1. Does pressing Tab repeatedly leave the footer visible and themed?
2. Does the quit confirmation render with compact themed geometry and readable Y Quit / N Stay actions?
3. Does opening Blocks, or activating Block Preview, show CounterDemoBlock first?
4. Does arrow navigation still allow the Block Preview parent row to receive focus from the first standard block preview child?

Test Plan

- Add or update DOGFOOD Blocks tests in tests/cycles/DX-031/dogfood-blocks-se ction.test.ts for initial Blocks selection and Block Preview activation.
- Add AppShell regression coverage proving Tab no longer emits a hidden footer toggle path.
- Extend quit confirmation tests in packages/bijou-tui/src/ so the modal's compact geometry and action styling are observable.
- Run the focused test files, npm run typecheck:test, npm run lint, npm run d ocs:inventory, git diff --check, and the relevant broader suite before review.

Acceptance Criteria

- Double-Tab no longer hides the DOGFOOD footer.
- The quit confirmation is compact, filled, aligned, and theme-readable.
- Blocks starts on CounterDemoBlock, and the Block Preview parent opens the same interactive demo.
- Existing Blocks guide focus behavior remains intact.
- The change is linked from #334 and carried by a non-draft PR to main.

114

DF-076 - BlockLab Demo Readiness Navigation

docs/design/DF-076-blocklab-demo-readiness.md

```
id: DF-076
title: BlockLab Demo Readiness Navigation
status: active
lane: asap
github_issue: 338
legend: DF
```

Decision Summary

BlockLab is a focused app/workbench surface. Its story navigation keys must win over generic framed-shell scroll bindings.

Problem

npm run blocklab runs through createFramedApp(). The frame shell defaults to keyPriority: "frame-first", so story-navigation keys such as up, down, j, and k are consumed by generic frame scroll bindings before BlockLab sees them.

That makes BlockLab unusable in the release-video path and contradicts the workbench contract: when the app advertises story navigation keys, those keys must move the selected story.

Scope

- Configure the BlockLab framed app to prefer page-owned keys.
- Add a regression that sends navigation keys through the framed-app runner path and asserts selected-story movement.

Non-Goals

- Change the default key priority for all framed apps.
- Redesign BlockLab.
- Add Theme Lab, localization, or GraphQL demo surfaces.

Playback

Run npm run blocklab, press down or j, and verify the selected story moves from alert() to badge() without a frame key-shadowing warning.

115

DF-077: DOGFOOD App Landing Extraction

docs/design/DF-077-dogfood-app-landing-extraction.md

Status

Implemented.

Tracker

- Issue: #421
- Pull request: #422
- Branch: cycle/dogfood-app-landing-extraction-725

Current Truth

Live counts on main at 2fcdba1a:

- aggregate Code Dojo debt: 674
- next aggregate Code Dojo target: 624 or lower
- ESLint findings: 266
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 hard-limit files
- examples/docs/app.ts: 5,792 physical lines and 201,982 bytes
- examples/docs/app.ts file/context baseline: 5,793 lines and 201,982 bytes
- examples/docs/app.ts code-size baseline: 5,793 lines
- DOGFOOD raw-string debt: 2,644
- docs-app raw-string debt: 256
- missing Markdown localization debt: 78

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 725
- next aggregate Code Dojo target: 675 or lower

- ESLint findings: 317
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 hard-limit files
- examples/docs/app.ts: 4,165 enforced lines and 147,799 bytes
- examples/docs/app.ts file/context baseline: 4,166 split-counted lines and 1 47,799 bytes
- examples/docs/app.ts code-size baseline: 4,165 lines
- DOGFOOD raw-string debt: 2,415
- docs-app raw-string debt: 185
- missing Markdown localization debt: 78

examples/docs/app.ts is now ESLint-clean and no longer owns the landing or theme-token subsystems, but it remains both a file/context exception and a hard code-size exception. The next respectful step is still structural: move coherent docs explorer subsystems out until the main DOGFOOD app file stops being a review and iteration bottleneck.

Problem

The DOGFOOD app main file mixes content catalog setup, landing animation and theme rendering, framed docs explorer state, pane layout, mouse routing, and application lifecycle wiring. That makes behavior changes expensive even when lint is clean because reviewers and agents must reason across thousands of unrelated lines.

The landing surface is the best first extraction target: it owns a large set of theme tokens, glyph rendering, animated background, quality controls, caches, and overlays while depending on only a small part of the root model. Pulling it behind a typed module boundary reduces the monolith without changing the visible DOGFOOD route.

Scope

- Extract the landing surface/theme/rendering subsystem from examples/docs/app.ts into focused module(s) under examples/docs/.
- Preserve existing landing route behavior, theme choices, quality controls, prompt rendering, performance HUD, and DOGFOOD panel output.
- Keep the extracted module typed through explicit interfaces instead of importing the whole root model shape.
- Tighten examples/docs/app.ts file/context and code-size baselines after the extraction.
- Keep all touched DOGFOOD i18n debt flat or lower. Any user-facing landing copy touched by this cycle must remain catalog-backed or reduce counted raw strings.
- Update the changelog and Code Dojo exception ledger with landed counts.

Out Of Scope

- Redesigning the landing page visuals.
- Changing runtime navigation, input bindings, or shell theme semantics.
- Moving the docs explorer panes in this slice.
- Expanding the localization catalog beyond strings directly needed by the extraction.
- Adding new Code Dojo exceptions.

Implementation Notes

The first implementation pass should move the following clusters together:

- landing theme seed/token types and constants
- landing quality profile types and constants
- landing renderer creation and surface preparation
- background, wake, SVG, prompt, FPS badge, and DOGFOOD panel helpers
- landing theme conversion and color utility helpers only when they are landing specific
- docs shell theme flattening/state helpers
- theme token palette, model, and diagnostics helpers

Shared helpers that still serve docs panes moved behind small shared theme modules only where the boundary stayed narrow and typed.

Validation Plan

- npx eslint examples/docs/app.ts examples/docs/app-landing*.ts examples/docs/app-docs-*.ts examples/docs/app-theme-*.ts examples/docs/i18n-debt.ts
- npm run code-dojo:changed
- npm run code-dojo:debt
- npm run dogfood:i18n:debt
- npm run test:run -- tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts tests/cycles/DF-067/responsive-dogfood-layout-variants.test.ts scripts/smoke-dogfood.test.ts tests/cycles/WF-003/replace-smoke-examples-with-smoke-dogfood.test.ts
- npm run smoke:dogfood:landing -- --skip-build
- npm run smoke:dogfood:docs -- --skip-build
- npm run code-dojo:verify
- npm run lint
- npm run docs:inventory
- git diff --check
- full pre-push gate

Implemented validation:

- npx eslint examples/docs/app.ts examples/docs/app-docs-shell-theme.ts examples/docs/app-docs-theme-tokens.ts examples/docs/app-theme-diagnostics.ts examples/docs/app-theme-token-model.ts examples/docs/app-theme-token-palette.ts examples/docs/i18n-debt.ts examples/docs/i18n/dogfood-catalog.ts examples/docs/locale.ts examples/docs/node-locale.ts examples/docs/svg-raster.ts packages/bijou/src/core/ansi-lint.test.ts packages/bijou/src/core/app-shell-composition.ts packages/bijou/src/core/components/dag-stats.ts packages/bijou/src/core/components/progress.ts packages/bijou/src/core/render/packed-cell.ts packages/bijou/src/core/theme/doctor.ts scripts/dogfood-i18n-export.ts tests/cycles/DL-004/drawer-rhythm-and-notice-rows.test.ts
- npm run typecheck:test
- npm run dogfood:i18n:complete
- npm run dogfood:i18n:check
- npm run dogfood:i18n:debt
- npm run code-dojo:changed
- npm run code-dojo:debt
- npm run code-dojo:verify
- npm run test:run -- packages/bijou/src/core/ansi-lint.test.ts tests/cycles/DL-004/drawer-rhythm-and-notice-rows.test.ts
- npm run test:run -- scripts/svg-raster.test.ts packages/bijou/src/core/theme/doctor.test.ts packages/bijou/src/core/components/dag-stats.test.ts packages/bijou/src/core/components/progress.test.ts packages/bijou/src/core/app-shell-composition.test.ts packages/bijou/src/core/render/packed-cell.test.ts scripts/docs-preview-landing.test.ts tests/cycles/DF-001/dogfood-coverage-progress.test.ts tests/cycles/DL-017/dogfood-light-theme-readiness.test.ts tests/cycles/DL-018/first-party-theme-variant-coverage.test.ts
- npm run test:run -- tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts tests/cycles/DF-067/responsive-dogfood-layout-variants.test.ts scripts/smoke-dogfood.test.ts tests/cycles/WF-003/replace-smoke-examples-with-smoke-dogfood.test.ts
- npm run smoke:dogfood:landing -- --skip-build
- npm run smoke:dogfood:docs -- --skip-build
- npm run docs:inventory
- npm run lint
- git diff --check

Acceptance Criteria

- examples/docs/app.ts is materially smaller and no longer owns the landing rendering subsystem.
- The extracted landing module stays under the 500-line code-size ratchet if feasible; if not feasible, the split must still lower the total hard-limit pressure on app.ts and document the next split.
- examples/docs/app.ts file/context and code-size baselines are tightened.
- DOGFOOD i18n raw-string debt is flat or lower, with touched-file ratchets green.
- Issue #421, this design doc, and the pull request are cross-linked.

116

DF-078 DOGFOOD Release Story Surfaces

docs/design/DF-078-dogfood-release-story-surfaces.md

```
title: DF-078 DOGFOOD Release Story Surfaces
legend: DF
lane: asap
priority: medium
status: proposed
github_issue: 335
parent_issue: 354
keywords:
  - dogfood
  - release
  - changelog
  - graphql
  - demo-integrity
```

Legend: DF - DOGFOOD

Linked Work

- GitHub Issue: #335
- v7.2 umbrella: #354
- GraphQL fixture lineage: #329
- Blocks app-binding lineage: #342
- Current release docs: docs/releases/7.1.0

This document shapes #335 into a bounded v7.2 demo-integrity pull. It does not turn v7.2 into a broad release-story product, and it does not pull the V8 Runtime Graph product contract forward.

Decision Summary

DOGFOOD already publishes release notes and migration docs in the Release section, but the app does not yet explain the post-v7.1 proof chain as a first-class product story. A presenter still has to narrate the important relationship manually:

```
GraphQL SDL fixture
  -> bijou-block/1 artifact
    -> ui-scene-ir/1 scene
      -> terminal Surface proof
        -> graphql-bijou-block-debug/1 facts
```

The v7.2 repair is to add release-story surfaces that make that chain visible inside DOGFOOD:

1. A concise release-story entry point from the DOGFOOD home or Release page.
2. A “What’s New” path that foregrounds the current release story, not only a Markdown document.
3. A GraphQL proof walkthrough tied to the real examples/docs/fixtures/graphql/navigation-list.graphql fixture.
4. A CHANGELOG viewer that preserves version boundaries, headings, and keyboard navigation.

Sponsored Human

A release-video viewer needs DOGFOOD to show why Bijou’s v7 proof matters without requiring a source-code tour. The presenter should be able to open one release-story path and walk from source SDL to rendered terminal proof.

Sponsored Agent

An agent needs deterministic text and model witnesses for the release story: which version is being described, which GraphQL fixture carries the proof, which artifact and scene facts are inspectable, and which changelog section bounds the claim.

Hill

After this cycle, DOGFOOD can tell the current release story from inside the app: current “What’s New” copy is discoverable, the real GraphQL block proof is inspectable step by step, and the changelog can be navigated by version without losing Markdown heading structure.

Scope

Release Story Entry Point

Add a compact entry point that is visible from normal DOGFOOD navigation. The entry should name the current release story and point to deeper pages rather than becoming another long landing section.

Acceptable first placement:

- a Release page overview card, or
- a concise home-page section that links into Release.

The implementation should not add a first-run takeover until there is a clear version-memory port and deterministic dismissal state.

What's New Surface

The current Release section already reads docs/releases/${BIJOU_VERSION}/whats-new.md. This cycle should make the surface easier to present by adding a short release-story summary before the long Markdown body or by adding a sibling guide that routes directly to the current "What's New" page.

The source of truth remains the versioned release document. DOGFOOD should not fork release claims into a second hand-maintained copy.

GraphQL Proof Walkthrough

Add a DOGFOOD guide that explicitly walks the checked-in fixture through the proof chain:

- source fixture path: examples/docs/fixtures/graphql/navigation-list.graphql
- compiled artifact kind: bijou-block/1
- lowered scene kind: ui-scene-ir/1
- terminal proof surface
- debug fact kind: graphql-bijou-block-debug/1

The guide must link the proof to the real NavigationListBlock fixture and to the DOGFOOD navigation surface instead of describing a synthetic example.

CHANGELOG Viewer

Add a first-class DOGFOOD path for docs/CHANGELOG.md that preserves:

- version boundaries
- Markdown headings
- keyboard navigation through sections
- readable current and historical release claims

The first slice may reuse the existing Markdown reader. A custom changelog index is acceptable only if it does not duplicate changelog content or invent a second release-history source of truth.

Out Of Scope

- First-run modal or full-screen takeover with persisted dismissal state.
- Runtime storage, host preferences, or cross-session version memory.
- Rewriting the release docs pipeline.
- Moving the broad #302 Runtime Graph tracker into v7.2.
- Migrating more DOGFOOD blocks to GraphQL.
- Building a source-code browser or full artifact inspector.

Playback Questions

1. Can a presenter reach the current release story from DOGFOOD without knowing a source file path?
2. Can the presenter open a GraphQL proof walkthrough and point to the real NavigationListBlock SDL fixture?
3. Can a viewer see the sequence GraphQL SDL -> bijou-block/1 -> ui-scene-ir/1 -> terminal proof -> debug facts inside DOGFOOD?
4. Can a viewer open the changelog and move between version boundaries without losing headings?
5. Can tests prove these routes from model state and rendered text rather than screenshots?

Accessibility And Assistive Posture

The release story must work as text. Color may support hierarchy, but the proof chain, version names, fixture path, artifact kinds, and changelog boundaries must be visible in plain text and preserved in accessible lower modes where applicable.

Localization And Directionality Posture

The first implementation may use English-source Markdown, matching the existing Release section behavior. If new DOGFOOD chrome labels are added, they must use catalog keys and keep all supported locale rows current. The design introduces no new right-to-left layout behavior.

Agent Inspectability And Explainability Posture

Agents should be able to inspect:

- the Release navigation guide ids
- the rendered release-story text
- the GraphQL fixture path and proof-chain labels
- the changelog guide id and version-boundary headings
- the tests that pin the routes

The proof must not depend on screenshots, timestamps, process ids, or local absolute paths.

Linked Invariants

- Graceful Lowering Preserves Meaning: the release story must remain meaningful as text, not only styled terminal layout.

- The Buffer Holds Facts: the GraphQL proof route should expose inspectable artifact and debug-fact labels.
- Tests Are the Spec: the release-story route is not complete until scripted DOGFOOD tests prove navigation and text witnesses.

Implementation Outline

1. Add a focused DOGFOOD release-story guide under the Release page.
2. Add a GraphQL proof walkthrough that references the real NavigationListBlock SDL fixture and proof-chain labels.
3. Add a CHANGELOG guide or changelog subsection that reads from docs/CHANGELOG.md.
4. Add scripted DOGFOOD tests for the Release route, GraphQL proof text, and changelog version-boundary headings.
5. Add any new DOGFOOD localization keys needed for navigation chrome.
6. Update docs/CHANGELOG.md, docs/ROADMAP.md, and docs/BEARING.md with the landed v7.2 release-story status.

Test Plan

- Add or extend tests/cycles/DF-023/publish-repo-package-and-release-guides-in-dogfood.test.ts or a new DF-078 test file for the Release route.
- Add a text assertion for the GraphQL proof chain and fixture path.
- Add a text assertion for at least [Unreleased] and [7.1.0] changelog boundaries.
- Run focused DOGFOOD release tests, npm run docs:inventory, npm run typecheck:test, DOGFOOD i18n gates if chrome labels change, git diff --check, and the normal pre-push gate before review.

Acceptance Criteria

- DOGFOOD has a visible release-story entry point.
- DOGFOOD can show the current “What’s New” release story from the Release section.
- DOGFOOD can show a GraphQL proof walkthrough tied to the real NavigationListBlock SDL fixture.
- DOGFOOD can show changelog content with version boundaries intact.
- First-run modal behavior is deliberately deferred unless a deterministic version-memory port is added in the same cycle.
- #335 has enough scoped design detail to leave needs-design.

117

DF-150 - DOGFOOD Terminal Guardrails

docs/design/DF-150-dogfood-terminal-guardrails.md

Linked issues: #150, #151, #152

Linked legend: DF - DOGFOOD Field Guide

Sponsor Human

A learner launching DOGFOOD from a terminal who needs to know that it will take over the screen and how to leave it.

Sponsor Agent

A docs-surface verification agent that can prove the DOGFOOD entrypoint behaves deterministically in TTY and non-TTY contexts.

Hill

DOGFOOD announces its full-screen behavior, prints explicit quit controls, and refuses to launch through non-interactive stdin/stdout paths that cannot host the TUI.

Scope

- Add a terminal readiness helper for the DOGFOOD entrypoint.
- Require both stdin and stdout to be TTYs before launching DOGFOOD.
- Print a short full-screen notice and exit controls before run(...).
- Point scripted callers to the existing smoke command instead of failing silently through a pipe or redirected output.
- Cover the guard in docs preview tests.

Playback Questions

1. Does DOGFOOD tell the user it is about to launch a full-screen TUI?
2. Are the quit controls visible before terminal takeover?
3. Does a non-TTY stdin or stdout path fail with a useful message?
4. Does the message direct automation to the smoke command instead?

Accessibility / Assistive Reading Posture

The warning is plain text on stderr before alt-screen rendering, so screen readers and logs can capture it without needing the TUI frame.

Localization / Directionality Posture

This launch warning is host-entrypoint English text in this cycle.

Agent Inspectability / Explainability Posture

The readiness helper is a pure function over ctx.runtime, so tests can assert TTY and non-TTY behavior without spawning a terminal.

Linked Invariants

- Full-screen apps should declare terminal takeover before entering the app.
- Non-interactive execution should point to scripted verification paths.

Verification

- npm test -- --run scripts/docs-preview.test.ts
- npm run smoke:dogfood:docs
- npm run smoke:dogfood:landing

Tests To Write First

- Docs preview tests for terminal readiness and entrypoint wiring.

Retrospective

See WF-122 day zero audit fixes.

Developer experience design records

This family follows the interfaces used to build, inspect and integrate Bijou: scaffold and workflow tools, BlockLab, blocks, data binding, portable artifacts and inspection contracts. Several proposals describe a larger product direction than the bounded slice implemented by one change. Use the scope, non-goals and closeout evidence to identify the contract each slice actually owns.

118

DX-001 — Type Framed App Messages and Updates End-to-End

docs/design/DX-001-type-framed-app-messages-and-updates-end-to-end.md

Legend: DX — Developer Experience

Historical note

This slice is already landed.

It originally lived as a backlog capture and was promoted into docs/design/ during METHOD adoption so the active backlog would stop advertising completed work as upcoming queue. The file is lighter than newer cycle docs for that reason.

Why this cycle existed

Remove the any cascade around createFramedApp() and framed-page update flows.

Why

Current framed-app typing forces consumers into casts and lint debt:

- FramePage effectively types page msg as any
- framed update results are difficult to type from the outside
- FrameModel<Model> exists, but the consumer-facing API does not make it easy to stay fully typed

The result is that a strongly typed app can lose its message-model precision at the shell seam.

Captured feedback

From warp-ttd TUI development:

- createFramedApp page update typed as (msg: any, model: Model)
- framedApp.update() returns an effectively untyped tuple
- these two seams were the biggest source of lint violations

Likely scope

- make framed-page message typing generic and public
- make framed update return types explicit and consumable
- remove or reduce the need for external any casts at the shell seam
- add tests proving end-to-end typed message unions survive framed app composition

Outcome

This work landed and is reflected in CHANGELOG.md.

The public framed-shell surface now exports explicit typed message and update result helpers, and page updates keep custom message unions typed through the shell seam.

Follow-on

- DX-003 — Rationalize Table APIs and Public Table Types
- DX-004 — Smooth Surface and String Composition Seams
- DX-005 — Polish Small Component and Import Ergonomics

119

DX-002 — Reconcile Cmd Typing With Cleanup and Effect Patterns

docs/design/DX-002-reconcile-cmd-typing-with-cleanup-and-effect-patterns.md

Legend: DX — Developer Experience

Historical note

This slice is already landed.

It originally lived as a backlog capture and was promoted into docs/design/ during METHOD adoption so the active backlog would stop advertising completed work as upcoming queue. The file is lighter than newer cycle docs for that reason.

Why this cycle existed

Make command typing match the cleanup-function and runtime-effect patterns Bijou actually encourages.

Why

The current Cmd<Msg> typing does not line up cleanly with real usage:

- the pulse timer pattern returns a cleanup function
- the existing type expects Promise<Msg | undefined | QUIT>
- consumers end up casting to any to express documented patterns

This is both a DX problem and a runtime-model problem, because it blurs the line between commands and effects.

Captured feedback

From warp-ttd TUI development:

- the pulse timer cleanup pattern required as any
- the types do not reflect the documented pattern

Likely scope

- audit current Cmd definitions and real runtime usage
- decide whether cleanup handles, effects, and commands need clearer separation
- align public typing with the documented timer/cleanup pattern
- add tests proving the typed API supports cleanup-producing runtime work without casts

Outcome

This work landed and is reflected in CHANGELOG.md.

Cmd<M> now matches the cleanup-capable runtime patterns Bijou actually uses, and the runtime retains and disposes cleanup results instead of forcing those paths through fake message contracts.

Follow-on

- DX-003 — Rationalize Table APIs and Public Table Types
- DX-004 — Smooth Surface and String Composition Seams
- DX-005 — Polish Small Component and Import Ergonomics

120

DX-022 Layout Inspector Overlay

docs/design/DX-022-layout-inspector-overlay.md

```
title: DX-022 Layout Inspector Overlay
legend: DX
lane: design
```

Framing

Layout and overlay issues are hard to inspect in situ. Bijou now retains enough runtime geometry that pane bounds, clipping, scroll regions, focus ownership, and z-layer ordering should not stay invisible during development.

DX-022 adds a first-party layout inspector overlay. The first slice is a pure surface helper: apps or framed shells can pass the retained regions they already know about and get a visible overlay plus a plain text report.

Sponsored Users

- App authors debugging split panes, clipped viewports, and mouse hit regions.
- Maintainers investigating shell focus or overlay layering regressions.
- Agent workflows that need a text report of layout truth without screenshot inspection.

Hills

1. A developer can composite an inspector over any surface and see region borders, ids, and dimensions without changing the app layout itself.
2. A maintainer can mark one region as focused and visually distinguish it from sibling regions.
3. A test can ask for a text report that includes rects, clip rects, scroll offsets, focus, and layer ordering.

Playback Questions

- Does the overlay draw region bounds on top of an existing surface?
- Does the overlay clip labels and borders to the target surface dimensions?
- Does focused geometry stand out from non-focused geometry?
- Does the text report preserve region ids, roles, rects, clips, scroll offsets, focus, and layers?
- Does bijou-tui export the inspector API from the source barrel?

Requirements

- Add a layoutInspectorOverlay(background, regions, options?) helper.
- Add a layoutInspectorText(regions) helper.
- Support region id, rect, optional role, optional clip, optional scroll, optional focused, and optional layer.
- Draw only within the background surface bounds.
- Keep the helper independent of createFramedApp() internals so raw runtime apps can use it too.

Acceptance Criteria

- Focused tests fail before implementation and pass after.
- The overlay draws borders and labels for visible regions.
- The overlay clips off-screen regions without throwing.
- The text report includes all supplied geometry facts.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou-tui/src/layout-inspector.ts.
- Export it from packages/bijou-tui/src/index.ts.
- Keep drawing intentionally simple: ASCII borders, compact labels, and explicit text reports are more useful in debug output than elaborate chrome.

Drift Check

- Scope stayed on the first pure helper slice. The runtime did not grow a global toggle, because callers can now compose the inspector over any retained geometry they already own.
- The implementation accepts both debug-friendly { x, y, width, height } rectangles and Bijou-native { row, col, width, height } rectangles. That

keeps the public helper ergonomic without forcing app-frame internals into the design contract.
- Styling stayed out of the first pass. Plain ASCII borders make the overlay stable in tests, logs, and pipe captures.

Playback

- layoutInspectorOverlay() draws clipped borders and compact labels over an existing surface.
- Focused regions are distinguishable through the * label prefix.
- Off-screen regions clip to the target surface instead of throwing.
- layoutInspectorText() emits ids, roles, rects, clip rects, scroll offsets, focus, and layer facts as deterministic text.
- @flyingrobots/bijou-tui exports the new API from the source barrel.

Retrospective

- This is useful as a small diagnostic primitive rather than a shell feature. Apps can opt in during tests, demos, or debug modes without changing runtime ownership.
- The native LayoutRect mismatch was caught by package typecheck after the RED tests went green. Accepting both shapes is the right boundary behavior: app-frame can pass retained layout rects directly, while external tests can use x/y terminology.

121

DX-023 Input Routing Inspector

docs/design/DX-023-input-routing-inspector.md

```
title: DX-023 Input Routing Inspector
legend: DX
lane: design
```

Framing

Key swallowing and priority mistakes are hard to diagnose from outside the runtime. Frame/page binding warnings already proved this seam needs stronger observability; developers should not have to infer routing truth from visible state changes alone.

DX-023 adds a first-party input-routing inspector. The first slice is a pure history and report helper around existing RuntimeInputRouteResult facts. Shell integration can feed it live records later without changing the core router.

Sponsored Users

- App authors debugging key priority, shell/page routing, and swallowed input.
- Maintainers investigating runtime routing regressions.
- Agent workflows that need text output for recent input-routing facts.

Hills

1. A developer can append routing records to a bounded history and inspect the last N events.
2. A maintainer can read a text report that shows event kind, visited views, handler owner, hit target, commands, effects, swallowed input, and no-op handlers.
3. A test can render the same report as a surface for docs or debug panes.

Playback Questions

- Does the helper retain only the configured number of events?
- Does the text report distinguish key and pointer events?
- Does the report expose visited views, handled/stopped owner, and hit target?
- Does the report flag swallowed input when routing stops unhandled?
- Does the report flag no-op handlers when an event is handled without commands or effects?
- Does bijou-tui export the inspector API from the source barrel?

Requirements

- Add appendInputRoutingRecord(history, record, options?).
- Add inputRoutingInspectorText(history).
- Add inputRoutingInspectorSurface(history, options?).
- Accept existing runtime RuntimeInputEvent and RuntimeInputRouteResult objects; do not duplicate router logic.
- Keep the first slice pure and shell-agnostic.

Acceptance Criteria

- Focused RED tests fail before implementation and pass after.
- Bounded history behavior is deterministic.
- Text reports include routing facts, swallowed status, and no-op status.
- Surface reports render the same text into a bounded Surface.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou-tui/src/input-routing-inspector.ts.
- Export it from packages/bijou-tui/src/index.ts.
- Keep command/effect labeling explicit: callers can pass labels, otherwise the report shows counts instead of guessing object meaning.

Drift Check

- Scope stayed on a pure inspector helper. The runtime router and frame shell are unchanged.
- Command and effect meaning remains caller-owned. The helper accepts optional labels and otherwise reports counts instead of guessing object semantics.

- Live shell integration remains follow-on work; the first slice provides the data model, bounded history, text report, and surface rendering.

Playback

- appendInputRoutingRecord() keeps a deterministic bounded history.
- inputRoutingInspectorText() distinguishes key and pointer events and shows raw input when provided.
- Reports include visited views, handled owner, stopped owner, hit target, command/effect labels or counts, swallowed input, and no-op handlers.
- inputRoutingInspectorSurface() renders the same report into a bounded Surface.
- @flyingrobots/bijou-tui exports the new API from the source barrel.

Retrospective

- This complements the layout inspector without coupling to it. Layout explains geometry; input routing explains priority and ownership.
- Keeping labels explicit prevents the inspector from turning command objects into misleading strings.

122

DX-024 Surface Diff Viewer

docs/design/DX-024-surface-diff-viewer.md

```
title: DX-024 Surface Diff Viewer
legend: DX
lane: design
```

Framing

Render debugging still depends on ad hoc inspection and text dumps. That is especially painful when a frame regression changes only a few cells, or when the character looks correct but a style changed underneath it.

Bijou already has a strong Surface truth object. DX-024 makes that truth inspectable with a first-party diff model and viewer surface.

Sponsored Users

- Maintainers investigating visual regression failures in frame tests.
- App authors debugging composed surfaces where nested layout masks or overlays changed a small region.
- Agent workflows that need a compact, plain-text rendering of surface truth without manually dumping raw cells.

Hills

1. A maintainer can diff two surfaces and immediately see changed cell counts, style-only changes, and the smallest changed bounds.
2. An app author can render the same diff side-by-side or as an overlay surface without writing custom inspection code.
3. A test can print a plain-text diff summary that preserves the same changed cell facts when rich terminal rendering is unavailable.

Playback Questions

- Does the diff distinguish char changes from style-only changes?
- Does the diff include cells added by width or height differences?
- Does the viewer produce both side-by-side and overlay renderings?
- Does the plain-text rendering include counts and per-cell coordinates?
- Does the package export the diff model and viewer from @flyingrobots/bijou-tui?

Requirements

- Add a pure diffSurfaces(before, after) model.
- Add surfaceDiffSurface(before, after, options) for rich viewer output.
- Add surfaceDiffText(before, after, options) for test and pipe output.
- Support side-by-side and overlay modes.
- Count changed cells, character deltas, and style-only deltas separately.
- Include a changed bounds rectangle when any cell differs.
- Avoid changing existing surface or differ semantics.

Acceptance Criteria

- Focused surface-diff tests fail before implementation and pass after.
- Cycle tests confirm the design doc sections and root package export.
- Docs and changelog describe the new viewer.

Implementation Outline

- Implement the diff model and renderers in packages/bijou-tui/src/surface-diff.ts.
- Export the new API from the bijou-tui source barrel.
- Keep rendering intentionally plain: labels, counts, and simple ASCII markers are more useful in failure output than decorative chrome.

Drift Check

- The implementation stayed in bijou-tui because the model is useful for tests, but the viewer is a TUI authoring/debug component.
- No existing surface or differ semantics changed; DX-024 only reads Surface cells and builds a separate report/view.
- The side-by-side renderer intentionally keeps output plain and copyable for failure logs. Overlay mode applies simple highlight styles for rich surfaces.

Playback

- RED: focused tests failed on the missing surface-diff module and missing source-barrel exports.
- GREEN: diffSurfaces() now reports changed counts, char deltas, style-only deltas, changed bounds, and per-cell coordinates.
- surfaceDiffSurface() renders both side-by-side and overlay views.
- surfaceDiffText() renders a plain coordinate report for pipe/test output.

Retrospective

- The useful split was model first, viewer second. That keeps tests and future tooling from needing to parse the visual surface.
- Style-only changes are first-class because they are a common terminal UI regression and invisible in simple text snapshots.
- A future follow-on could wire this into frame-regression failure output, but the reusable primitive is now available.

123

DX-025 Component Metadata Contract

docs/design/DX-025-component-metadata-contract.md

```
title: DX-025 Component Metadata Contract
legend: DX
lane: design
```

Framing

Bijou has component facts scattered across package docs, strategy docs, examples, tests, DOGFOOD stories, and MCP-facing payloads. Those surfaces should not each invent a slightly different idea of what a component is.

DX-025 adds a pure component metadata contract in core. The first slice defines the runtime shape, validates common drift, and renders a deterministic summary. It does not build the story workstation; it gives that workstation, DOGFOOD, MCP docs, and agent workflows one shared object to consume.

Sponsored Users

- Maintainers curating first-party component docs and examples.
- Tool builders generating DOGFOOD pages, MCP payloads, or story indexes.
- Third-party authors who want extension metadata without copying internal docs.
- Agent workflows that need stable facts about component modes and invariants.

Hills

1. A maintainer can describe a component's package, family, modes, args, variants, invariants, docs, examples, and source path in one runtime object.
2. A tool can validate metadata and get deterministic issue paths for duplicate ids, missing required fields, and empty required lists.

3. A docs or agent surface can render a compact summary without parsing markdown.

Playback Questions

- Does the contract capture package name, component name, family, modes, docs, args, variants, invariants, examples, and source path?
- Does validation reject blank required fields and empty mode lists?
- Does validation report duplicate mode names and duplicate ids deterministically?
- Can variants and invariants attach semantic facts for mode-lowering checks?
- Does a helper throw on invalid metadata for authoring-time guardrails?
- Does @flyingrobots/bijou export the contract and helpers from the root barrel?

Requirements

- Add ComponentMetadata and related runtime types.
- Add validateComponentMetadata(metadata).
- Add defineComponentMetadata(metadata) for authoring-time validation.
- Add componentMetadataSummary(metadata) and componentMetadataReportText(report).
- Keep the contract pure and package-neutral enough for third-party components.
- Reuse OutputMode and ModeLoweringFact types where metadata touches modes and semantic facts.

Acceptance Criteria

- RED tests fail before implementation and pass after.
- Valid metadata reports no issues and produces a stable summary.
- Invalid metadata reports deterministic issue paths and messages.
- defineComponentMetadata() throws on invalid metadata.
- Docs and changelog describe the metadata contract.

Implementation Outline

- Implement the helper in packages/bijou/src/core/component-metadata.ts.
- Export it from packages/bijou/src/index.ts.
- Keep validation strict enough to catch drift but not so broad that it becomes a schema language.

Drift Check

- Scope stayed on the contract and validation layer. The story workstation and DOGFOOD migration remain follow-up work that can consume this object.
- Validation catches required-field drift, empty mode lists, duplicate modes, duplicate arg names, and duplicate ids across variants, invariants, and examples. It does not become a schema language or a docs parser.
- Variants and invariants can carry ModeLoweringFact records, connecting this contract to DX-026 without coupling metadata to rendered output.

Playback

- ComponentMetadata captures package name, component name, family, modes, docs, args, variants, invariants, examples, tags, and source path.
- validateComponentMetadata() returns deterministic issue paths and messages.
- defineComponentMetadata() throws with the report text when metadata contains errors.
- componentMetadataSummary() renders a compact, stable summary for docs, agents, and MCP payloads.
- componentMetadataReportText() renders validation output in review-friendly text.
- @flyingrobots/bijou exports the contract and helpers from the root barrel.

Retrospective

- This gives DOGFOOD and MCP tooling a shared object without forcing either surface to own the component taxonomy.
- Keeping semantic facts attached to variants and invariants should make future story audits and mode-lowering checks cheaper: metadata can describe the intended truth, and render-specific tooling can prove it.

124

DX-026 Mode-Lowering Linter

docs/design/DX-026-mode-lowering-linter.md

```
title: DX-026 Mode-Lowering Linter
legend: DX
lane: design
```

Framing

Bijou treats interactive, static, pipe, and accessible as real output contracts, not skins. Rich output can use layout, color, borders, animation, and symbols, but lower modes still need to preserve the semantic truth a human or agent needs to understand the component.

DX-026 adds a pure diagnostic helper for mode-lowering fidelity. The first slice does not parse rendered text or own component rendering. Instead, component authors supply the semantic facts each mode promises to preserve, and Bijou reports missing or collapsed facts in a stable format.

Sponsored Users

- Component authors proving a custom component lowers honestly.
- Maintainers reviewing regressions in pipe and accessible output.
- Agent workflows that need deterministic diagnostics instead of screenshots.

Hills

1. A component author can compare semantic facts across rich, static, pipe, and accessible outputs without matching visual text.
2. A maintainer can identify which mode dropped a required entity, count, state, label, edge, or custom fact.
3. A test can assert a compact report, then render that report as stable text for docs or review comments.

Playback Questions

- Can a caller define a baseline mode and compare every other mode to it?
- Does the helper report missing facts per mode with fact kind and key?
- Does the helper report changed fact values for counts, states, or other value-bearing facts?
- Does the helper report collapsed duplicate facts inside a mode?
- Can callers mark a fact as optional so intentional omissions do not warn?
- Can callers add component-specific assertion results to the same report?
- Does @flyingrobots/bijou export the helper and its report types?

Requirements

- Add lintModeLowering(options) to @flyingrobots/bijou.
- Add modeLoweringReportText(report) for deterministic diagnostics.
- Support fact kinds for entity, edge, count, label, state, and custom.
- Treat baseline facts as required unless required: false.
- Report missing required baseline facts in lower modes.
- Report changed required fact values in lower modes.
- Report duplicate fact keys inside a mode as collapsed semantics.
- Accept custom assertion results so component tests can attach domain-specific failures without a second reporting format.

Acceptance Criteria

- RED tests fail before implementation and pass after.
- Reports include checked modes, issue severities, fact keys, and messages.
- Optional facts are ignored when absent from a lower mode.
- Duplicate facts are reported deterministically.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou/src/core/mode-lowering.ts.
- Export it from packages/bijou/src/index.ts.
- Keep the API pure and string/symbol independent; semantic facts are explicit runtime objects supplied by callers.

Drift Check

- Scope narrowed from a static source-code linter to a pure semantic diagnostic helper. That is the right first slice because components already

know their runtime semantic facts, while parsing rendered text or source files would add brittle inference.
- The helper reports missing facts, changed fact values, duplicate facts, and custom assertion failures. It does not try to decide whether a glyph or phrase is visually equivalent.
- interactive is the default baseline when present; otherwise the first supplied mode becomes the baseline.

Playback

- lintModeLowering() compares supplied mode facts against a baseline and returns a structured report with checked modes, issue severities, fact kinds, fact keys, and pass/fail status.
- Optional baseline facts with required: false can be omitted by lower modes without warning.
- Duplicate fact identities inside a mode are reported as collapsed semantics.
- Failed custom assertions are folded into the same report.
- modeLoweringReportText() renders deterministic text for tests, docs, and review comments.
- @flyingrobots/bijou exports the helper and report types from the root barrel.

Retrospective

- The important boundary is explicit runtime truth. Component authors supply facts like entity:save-button or count:rows=3; the helper checks fidelity without coupling itself to a renderer, AST, or screenshot.
- This composes with DOGFOOD stories and future replay tooling: both can attach semantic facts to rendered variants and use the same report format.

125

DX-030 Add boundary-aware pointer selection and copy

docs/design/DX-030-add-boundary-aware-pointer-selection-and-copy.md

```
title: DX-030 Add boundary-aware pointer selection and copy
legend: DX
lane: design
priority: high
keywords:
  - dogfood
  - pointer
  - selection
  - clipboard
  - geometry
```

Framing

DOGFOOD currently depends on terminal-native selection. That is acceptable as a host fallback, but it is not the Bijou model. A drag gesture in a framed docs surface should not copy whatever cells happen to be painted across the whole terminal. It should resolve the owning pane, clamp the drag to that pane's geometry, and reconstruct text from the component's semantic content model.

This cycle defines the contract and lands the pure selection coordinator in @flyingrobots/bijou. The runtime primitive consumes retained rectangles, viewport transforms, semantic content models, higher-priority blockers, and pointer ranges, then returns selected text plus an inspectable clipboard-effect record. It still does not implement OS clipboard writes, full app-frame drag capture, or cross-pane rich extraction. Those pieces remain host/runtime adapters rather than render logic.

Sponsored Users

- DOGFOOD readers who want to drag-copy docs text without selecting shell chrome, neighboring panes, or visual gutters.
- App authors who need copy behavior to respect pane and component boundaries.

- Maintainers who need a stable target for later pointer-routing, clipboard, and semantic extraction work.

Hills

1. A reader can expect selection to stay inside the pane or component that owns the drag start.
2. A builder can declare selection metadata on a surface or layout node without giving rendering authority over geometry.
3. A maintainer can implement runtime selection later by following an explicit hit-test, range, extraction, and clipboard-effect protocol.

Playback Questions

- What metadata can a surface or layout node expose for selection?
- How does retained layout geometry map a pointer drag to one owning component?
- How is copied output reconstructed for prose, tables, and mixed panes?
- How does the shell arbitrate selection against existing mouse routing?
- Where does clipboard IO live so copying remains an effect, not render logic?

Requirements

- Selection ownership is opt-out default behavior for framed docs-style panes, but interactive controls may explicitly opt out or override drag handling.
- Hit-testing must start from retained layout geometry, not rendered strings.
- A drag starts from one SelectionOwner and remains bounded to that owner unless a future explicit multi-owner policy allows otherwise.
- Extraction reconstructs output from semantic content metadata, paragraph spans, table cells, or component-provided extractors, not raw terminal rows.
- Viewports must map screen coordinates through scroll offsets before forming content ranges.
- Copying to an OS clipboard is a host effect. Runtime selection should emit a command/effect fact; renderers should never write the clipboard.

Contract

The contract introduces six conceptual records.

SelectionOwner

The component, pane, or layout node that owns a selectable region.

Required facts:

- stable owner id
- retained layout node id
- assigned screen rect
- optional viewport transform
- selection policy
- content model reference

SelectionRegion

The concrete selectable rectangle derived from layout truth.

Required facts:

- owner id
- screen rect
- content rect
- clipping rect
- z/layer order
- enabled/disabled state

SelectionRange

The normalized content-space range derived from a pointer drag.

Required facts:

- owner id
- anchor point
- focus point
- normalized start/end points
- content-space clipping rect
- direction
- drag source device

SelectionContentModel

The semantic model used to reconstruct selected text.

Supported first models:

- prose: paragraph and wrapped-line spans
- surface: cell spans with optional semantic line grouping
- table: row and column cells with header context
- mixed: ordered child regions with their own extractors

SelectionExtractor

The pure function that turns a SelectionRange and content model into copied text.

Rules:

- prose unwraps soft wraps while preserving hard paragraph breaks
- tables preserve cell boundaries with tabs or another declared delimiter

- mixed panes concatenate child extractions in semantic order
- decorative chrome, gutters, scrollbars, and focus rails are excluded by default
- extraction must be deterministic and inspectable

SelectionCoordinator

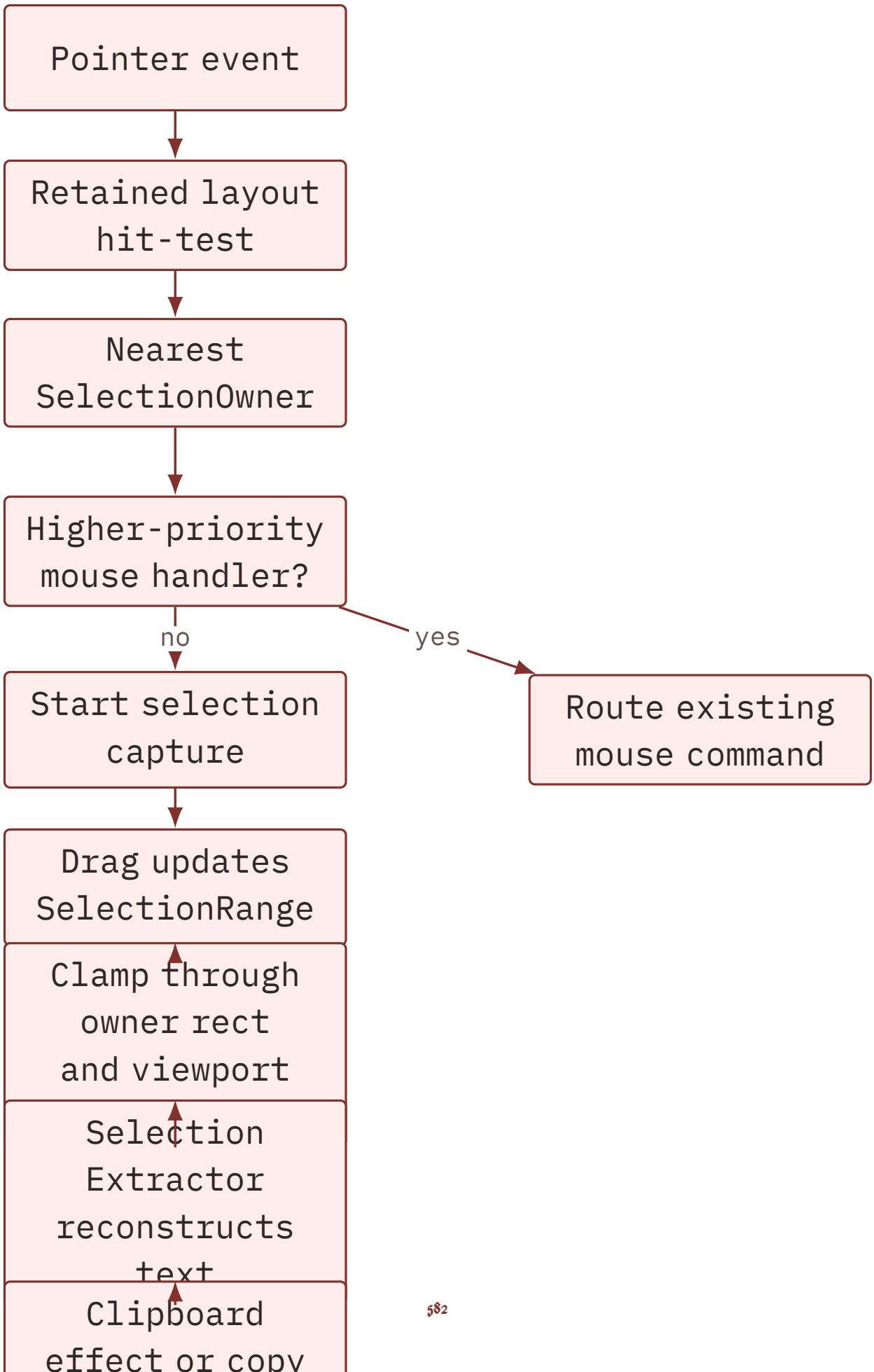
The runtime service that arbitrates pointer selection.

Responsibilities:

- receive pointer down/drag/up facts
- hit-test the topmost retained layout region
- choose the nearest selectable owner
- respect higher-priority mouse handlers such as split dividers, buttons, drag/drop targets, and active overlays
- clamp movement to the owner region
- request extraction on release
- emit a clipboard effect or copy result command

The shipped coordinateSelection() primitive covers this flow in pure package space. It does not own pointer capture or host clipboard IO; it gives those adapters a deterministic contract to call.

Runtime Flow



Selection is a pointer interaction that uses layout truth. It is not a render phase and not a terminal-row scrape.

Arbitration

Mouse routing keeps the same priority model as the rest of Bijou:

1. Active modal or overlay captures first.
2. Explicit drag handles and resize controls win over selection.
3. Component-owned pointer handlers win when the component declares a drag gesture, such as sliders, split panes, or draggable tabs.
4. Otherwise, the nearest selectable owner may start a selection capture.
5. Terminal-native selection remains a host fallback when Bijou is not in mouse mode or the app opts out.

The rule is deliberately conservative: selection should never steal a gesture from an interactive control that already owns that drag.

Extraction Rules

Prose extraction:

- maps pointer positions to paragraph spans
- unwraps soft wraps
- preserves hard line breaks, blank lines, and list boundaries
- excludes visible borders, scrollbars, and status gutters

Table extraction:

- maps selected cells through row and column coordinates
- preserves headers when the selection crosses enough columns to need context
- emits tab-delimited text by default
- leaves room for CSV/TSV policy later

Mixed pane extraction:

- clips each child to the owner range
- extracts children in semantic order
- inserts declared separators only where the component model says they exist
- refuses to infer meaning from visual adjacency alone

Viewport extraction:

- maps screen coordinates into content coordinates with scroll offsets
- respects clipped content
- may expose current scroll context in accessible copy, but copied prose should be the selected content, not the scrollbar state

Non-Goals

This cycle does not:

- implement OS clipboard adapters
- implement drag selection in createFramedApp()
- define multi-pane selection
- define cross-pane rich extraction
- define rich clipboard formats
- define image or shader-region copying
- make terminal-native selection disappear

Those are follow-on implementation slices once host and framed-app adapters are ready to consume the pure coordinator.

Acceptance Criteria

- The contract names selection owners, regions, ranges, content models, extractors, and the coordinator.
- The contract states that hit-testing uses retained layout geometry.
- The contract states that copied output comes from semantic extraction, not raw terminal rows.
- The contract defines arbitration against overlays, drag handles, component pointer handlers, and terminal-native fallback.
- @flyingrobots/bijou exports a pure selection coordinator that uses retained geometry and semantic extraction.
- The v6 release lane points at this design closure instead of the backlog note.
- The DX legend records this as the latest completed selection/copy closure.

Implementation Outline

1. Promote the backlog item into this design-cycle contract.
2. Add SelectionOwner, SelectionRegion, SelectionRange, SelectionContentModel, SelectionBlocker, and clipboard-effect records to @flyingrobots/bijou.
3. Implement coordinateSelection(), selectionRange(), and extractSelectionText() as pure primitives backed by retained geometry and semantic content.
4. Add package and Method-cycle tests that prove bounded prose extraction, table extraction, topmost owner selection, higher-priority blocker arbitration, terminal-native fallback, and v6 closeout evidence.
5. Update the v6 lane, roadmap, bearing, changelog, and backlog anchors so issue #186 closes through implementation rather than contract-only deferral.

Playback

- RED: the backlog named the need, but no stable design artifact defined the selection owner, range, extraction, and clipboard-effect contract; the new package and Method-cycle tests failed because no public selection module or v6 closeout evidence existed.
- GREEN: this cycle defines the records, exports the pure selection coordinator, and proves semantic copy behavior without terminal-row scraping.
- The contract keeps selection bounded by retained layout truth.
- The contract keeps copying semantic and inspectable.
- The contract keeps clipboard writes outside render logic.

Retrospective

DX-030 is landed for the v6.0.0 release boundary.

The implementation intentionally stops at the pure selection coordinator because that is the correct package boundary for v6: layout truth and semantic content can now decide what would be copied, while host clipboard writes and framed-app drag capture remain explicit adapters. That keeps boundary-aware copy inspectable, deterministic, and test-protected without smuggling terminal row scraping into the renderer.

126

DX-031 Standard Bijou Blocks

docs/design/DX-031-standard-bijou-blocks.md

```
title: DX-031 Standard Bijou Blocks
legend: DX
lane: design
```

Cycle for declaring reusable app-building blocks that compose Bijou components into standard, inspectable surface patterns

Legend:

- DX - Developer Experience

Depends on:

- DX-025 - Component Metadata Contract
- DX-031F - Block Tree Rendering
- DF-028 - Story Capture Matrix
- DF-029 - Fixture-to-Docs Promotion Path
- RE-035

Why This Cycle Exists

Bijou has many capable components, but app builders still need to assemble common surfaces by hand: shells, readers, inspectors, command palettes, settings, modal stacks, notifications, and authentication flows.

Blocks standardize those common compositions without turning them into new primitives.

A block is a named composition of Bijou components with a documented layout, focus, input, mode-lowering, and semantic contract. Blocks are the layer a builder reaches for when they want to build an app quickly without re-deciding the same shell and workflow patterns every time.

Boundary

- Blocks compose primitives.
- Blocks expose slots and semantic facts.

- Blocks lower across interactive, static, pipe, and accessible modes.
- Blocks do not mutate layout directly.
- Blocks do not replace RE layout primitives.
- Blocks do not become a hidden window manager.

Relationship To RE-035

Blocks depend on layout truth, but they do not define it.

Each block should compile into ordinary visible nodes with mandatory layout envelopes. The block owns composition, slot names, default behavior, and semantic facts. The layout engine owns constraints, preferences, assigned rectangles, and explanations.

That keeps the boundary simple:

- blocks choose what app surface exists
- layout chooses where each visible node fits
- focus and input consume retained layout facts
- renderers paint assigned rectangles
- modes lower semantic block facts

Runtime Truth Vs App Truth

Blocks must preserve a hard boundary between primitive runtime truth and semantic app truth.

Primitive runtime truth belongs to RE and adjacent runtime layers:

- layout envelope
- constraints
- preferences
- assigned rectangles
- fit policy
- focus regions
- hit regions
- viewport facts
- layer order
- command and effect facts

Semantic app truth belongs to blocks and app code:

- ReaderSurface
- ProductDetails
- CheckoutFlow
- DashboardShell
- CommentsThread
- TerminalPanel

- domain entities, statuses, actions, and relationships

The dependency direction only goes one way:

- semantic blocks compile into primitive runtime truth
- runtime primitives do not learn product, commerce, docs, or chat semantics
- app semantics do not mutate runtime geometry directly
- inspectors may relate both layers, but must label which layer they explain

This prevents business semantics from leaking into the layout engine and keeps runtime primitives from becoming app-shaped.

Human Users / Jobs / Hill

Primary Human Users

- App builders assembling Bijou tools from common surface patterns.
- DOGFOOD maintainers turning docs, inspectors, settings, and command surfaces into reusable examples.
- Component authors who need a standard target for real-world composition stories.

Human Jobs

1. Pick a block for a common app surface instead of starting from blank layout.
2. Fill named slots with app-owned content.
3. Trust the block's default focus, input, responsive, and mode-lowering behavior.
4. Inspect how a block composes lower-level components.

Human Hill

A builder can create a useful Bijou app shell with navigation, content, settings, help, notifications, and modal behavior by composing documented blocks instead of hand-rolling every surface.

Agent Users / Jobs / Hill

Primary Agent Users

- Agents generating Bijou app scaffolds.
- Agents upgrading examples into DOGFOOD stories.
- Agents explaining how an app surface is composed.

Agent Jobs

1. Select an appropriate block from a stable catalog.
2. Fill required slots and avoid unsupported private layout behavior.
3. Generate tests against block contracts instead of snapshot-only output.
4. Preserve semantic facts when lowering to pipe and accessible modes.

Agent Hill

An agent can inspect a block definition, identify its slots and mode behavior, and generate type-safe app code that uses the block without inventing new layout or input conventions.

Accessibility / Assistive Posture

Blocks must preserve semantic meaning when visual chrome lowers away.

Every standard block should identify:

- named regions
- active item or selected entity, when relevant
- available actions
- validation or warning facts
- modal or focus-trap state, when relevant
- whether hidden visual content remains available in accessible mode

Auth blocks must never expose secrets in accessible, pipe, static, or diagnostic output.

Localization / Directionality Posture

Blocks should describe layout in logical terms and inherit direction from the host context unless a block explicitly owns a local direction boundary.

The first catalog should avoid physical left/right naming in slot contracts. For example:

- use navigation instead of leftNav
- use inspector instead of rightPane
- use primaryAction and secondaryActions instead of button order names

Text, bidi, and locale-specific formatting stay outside this first blocks cycle, but block slots must not make later localization harder.

Agent Inspectability / Explainability Posture

Blocks should be inspectable as data before rendering. An agent should be able to answer:

- which block is this?
- which slots are required?
- which slots are filled?
- which lower-level components does it compose?
- what semantic facts does it expose?
- how does it lower by mode?
- what are the expected focus and action surfaces?

The first implementation should prefer metadata and story captures over snapshot-only proof.

Linked Invariants

- Docs Are the Demo
- Runtime Truth Wins
- Layout Owns Interaction Geometry
- The Buffer Holds Facts
- Commands Change State, Effects Do Not
- Tests Are the Spec

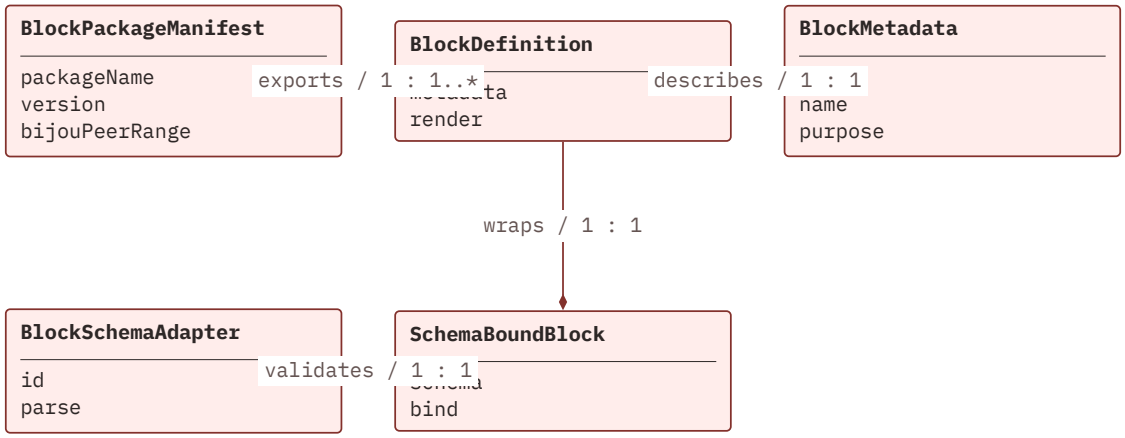
Block Contract

Every standard block should declare:

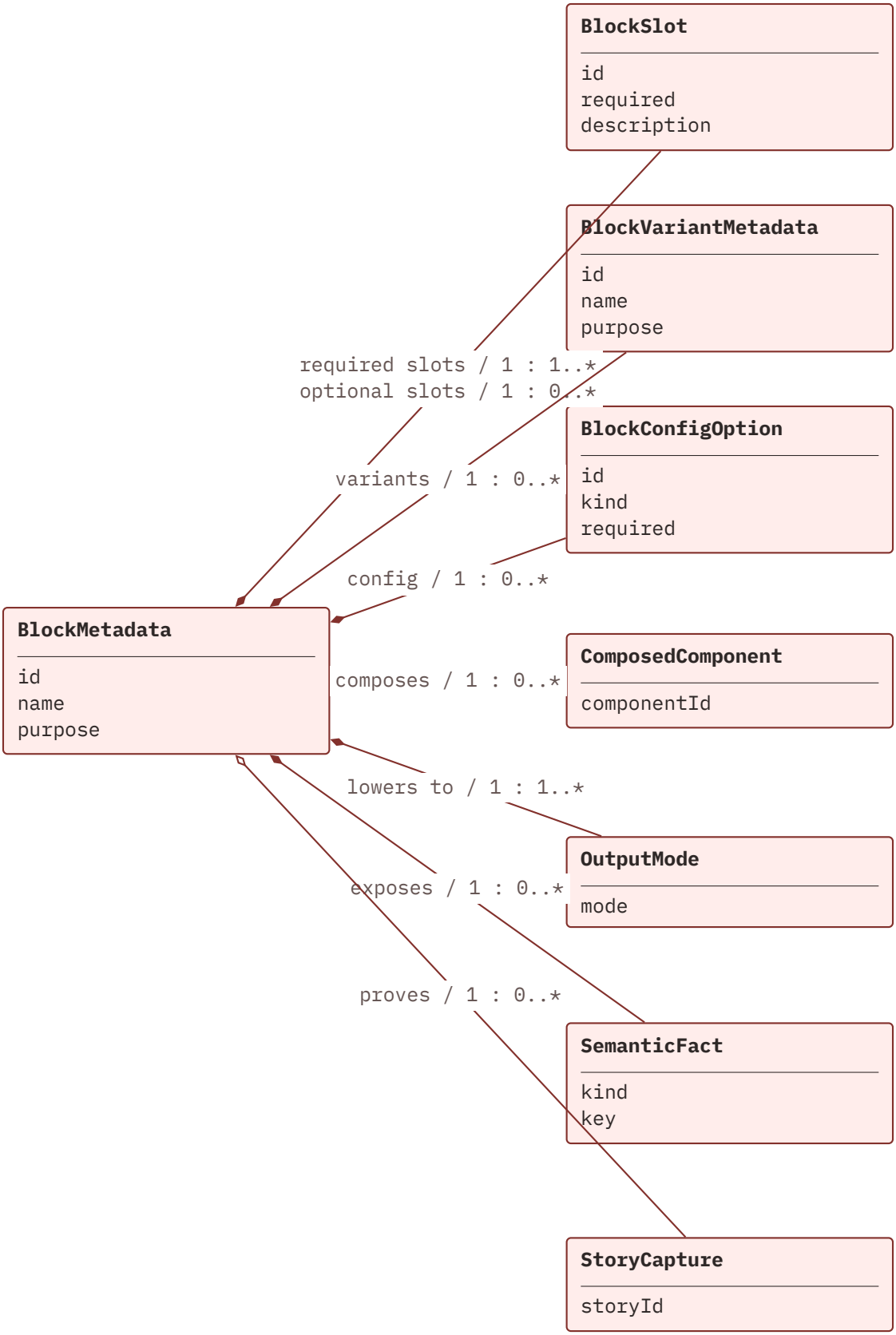
- purpose
- required slots
- optional slots
- variants
- configurable options
- composed components
- layout posture
- focus posture
- input posture
- responsive posture
- mode-lowering posture
- accessibility facts
- examples and story ids
- tests that define the contract

The eventual metadata can align with defineComponentMetadata() and story capture records, but this cycle starts by standardizing the human-readable catalog.

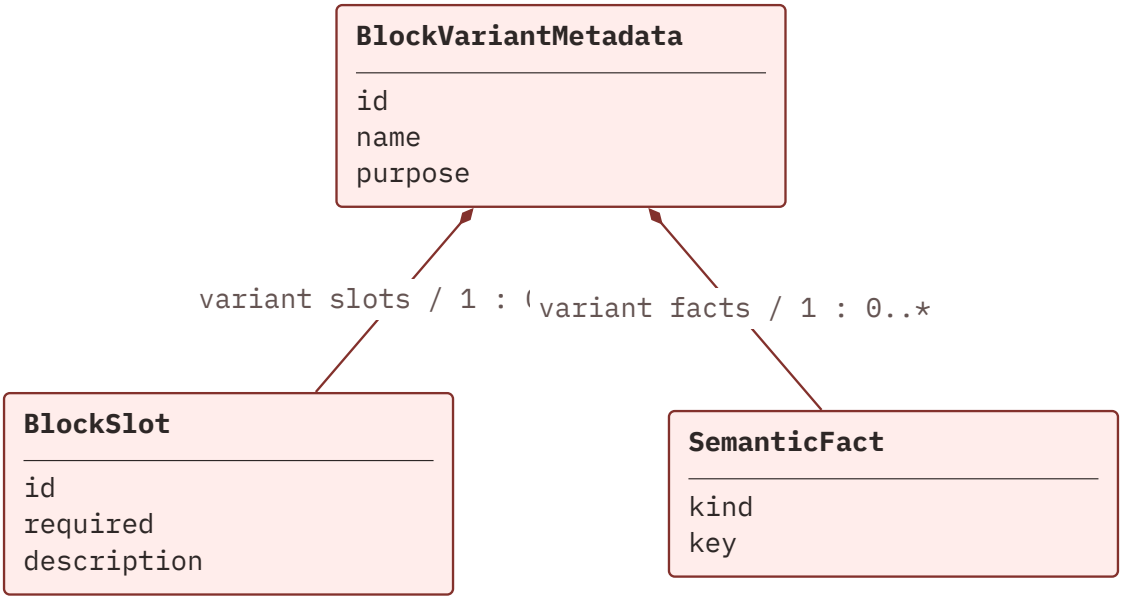
The entity relationship is:



Definition and schema ownership



Slots, lowerings and proof metadata



Variant-specific slots and facts

Block package manifests expose definitions. Block metadata owns the contract. Variants, config, modes, semantic facts, and story captures hang off the block instead of being inferred from rendered output.

Metadata sketch:

```
type BlockScale =
  | 'app'
  | 'workspace'
  | 'section'
  | 'panel'
  | 'control'
  | 'inline'
  | 'item'
  | 'data'
  | 'diagnostic'
  | 'overlay';

interface BlockMetadataDocs {
  readonly summary: string;
  readonly useWhen?: readonly string[];
  readonly avoidWhen?: readonly string[];
  readonly relatedDocs?: readonly string[];
}

interface BlockSlot {
  readonly id: string;
  readonly label?: string;
  readonly required?: boolean;
  readonly description?: string;
}

interface BlockMetadata {
  readonly packageName: string;
  readonly blockName: string;
  readonly family: string;
  readonly scale: BlockScale;
  readonly modes: readonly ('interactive' | 'static' | 'pipe' | 'accessible')[];
  readonly docs: BlockMetadataDocs;
  readonly sourcePath?: string;
  readonly slots: readonly BlockSlot[];
  readonly variants?: readonly BlockVariantMetadata[];
  readonly configOptions?: readonly BlockConfigOption[];
  readonly composedComponents?: readonly string[];
  readonly semanticFacts?: readonly ModeLoweringFact[];
  readonly storyIds?: readonly string[];
  readonly examples?: readonly BlockExample[];
  readonly tags?: readonly string[];
}

interface BlockVariantMetadata {
  readonly id: string;
  readonly label: string;
  readonly description?: string;
  readonly requiredSlots?: readonly string[];
  readonly optionalSlots?: readonly string[];
  readonly facts?: readonly ModeLoweringFact[];
}

interface BlockConfigOption {
  readonly id: string;
  readonly label?: string;
  readonly kind: 'boolean' | 'enum' | 'number' | 'string' | 'adapter';
  readonly required?: boolean;
  readonly description?: string;
  readonly values?: readonly string[];
}

interface BlockExample {
  readonly id: string;
  readonly label: string;
  readonly path?: string;
  readonly command?: string;
}
```

```
type DeepReadonly<T> = T extends readonly (infer Item)[]
  ? readonly DeepReadonly<Item>[]
  : T extends object
    ? { readonly [Key in keyof T]: DeepReadonly<T[Key]> }
    : T;

type BlockSchemaResult<Data> =
  | { readonly ok: true; readonly data: DeepReadonly<Data> }
  | { readonly ok: false; readonly issues: readonly BlockSchemaIssue[] };

interface BlockSchemaIssue {
  readonly severity: 'info' | 'warning' | 'error';
  readonly code: string;
  readonly message: string;
  readonly path?: string;
}

interface BlockSchemaDescription {
  readonly requiredFields?: readonly string[];
  readonly optionalFields?: readonly string[];
  readonly redactedFields?: readonly string[];
  readonly facts?: readonly ModeLoweringFact[];
}

interface BlockDefinition<Config = unknown> {
  readonly metadata: BlockMetadata;
  readonly data?: ViewDataContract;
  readonly commands?: readonly CommandIntent[];
  readonly render: (input: BlockRenderInput<Config>) => BlockRenderResult;
}

interface BlockSchemaAdapter<Data> {
  readonly id: string;
  readonly parse: (input: unknown) => BlockSchemaResult<Data>;
  readonly describe?: () => BlockSchemaDescription;
}

interface SchemaBoundBlockDefinition<Data, Config = unknown> {
  readonly block: BlockDefinition<Config>;
  readonly schema: BlockSchemaAdapter<Data>;
  readonly bind: (
    data: DeepReadonly<Data>,
  ) => BlockRenderInput<Config> | {
    readonly input: BlockRenderInput<Config>;
    readonly facts?: readonly ModeLoweringFact[];
  };
}

interface BlockPackageManifest {
  readonly packageName: string;
  readonly version: string;
  readonly bijouPeerRange: string;
  readonly blocks: readonly string[];
  readonly docs?: readonly string[];
  readonly tags?: readonly string[];
}

declare function defineBlock<Config>(
  definition: BlockDefinition<Config>,
): BlockDefinition<Config>;

declare function validateBlockMetadata(
  metadata: BlockMetadata,
): BlockMetadataReport;

declare function blockMetadataSummary(metadata: BlockMetadata): string;

declare function defineSchemaBlock<Data, Config>(
  definition: SchemaBoundBlockDefinition<Data, Config>,
): SchemaBoundBlockDefinition<Data, Config>;

declare function bindSchemaBlockInput<Data, Config>(
  block: SchemaBoundBlockDefinition<Data, Config>,
  input: unknown,
```

```
): {
  readonly ok: true;
  readonly input: BlockRenderInput<Config>;
  readonly facts: readonly ModeLoweringFact[];
} | {
  readonly ok: false;
  readonly issues: readonly BlockSchemaIssue[];
  readonly facts: readonly ModeLoweringFact[];
};

declare function defineBlockPackage(
  manifest: BlockPackageManifest,
): BlockPackageManifest;
```

DX-031A lands this as a public metadata-first API in @flyingrobots/bijou.

DX-031B Schema-Bound Blocks lands the schema adapter layer as a public boundary contract in @flyingrobots/bijou. Schema-bound blocks validate boundary data before producing block render input or binding facts. They do not fetch, subscribe, dispatch, compose runtime views, or render AppShell. The important boundary is that a block and its schema are described before rendering, and their slots, semantic facts, and validation facts are visible to tests, docs, DOGFOOD, and agents.

Public Block API And Distribution

Blocks should become a user-facing Bijou API, not an internal DOGFOOD-only catalog.

The public API should let Bijou users and community authors:

- define a block with typed metadata and a render function
- validate block metadata before publishing
- attach examples, story captures, and mode-lowering facts
- package one or more blocks for reuse
- distribute blocks through ordinary NPM packages
- let apps import blocks without hidden registration side effects

The first distribution mechanism should be boring: NPM packages with a stable manifest. A future block registry can index NPM metadata, docs, and story captures, but the runtime should not need a central registry to use a block.

Recommended package shape:

```
@scope/bijou-blocks-commerce
  package.json
  src/index.ts
  blocks/product-list.ts
  blocks/checkout-flow.ts
  stories/product-list.story.ts
  docs/README.md
```

Rules for shared block packages:

- depend on @flyingrobots/bijou as a peer dependency
- export block definitions explicitly from the package entrypoint
- avoid global registration or import-time mutation
- include block metadata and package metadata

- include at least one story or fixture per exported block
- declare supported output modes and semantic facts
- keep host effects behind adapters supplied by the consuming app
- never include secret values in metadata, stories, diagnostics, or snapshots

Example package entrypoint:

```
export const productListBlock = defineBlock({
  metadata: productListMetadata,
  render: productList,
});

export const checkoutFlowBlock = defineBlock({
  metadata: checkoutFlowMetadata,
  render: checkoutFlow,
});

export const commerceBlocks = defineBlockPackage({
  packageName: '@example/bijou-blocks-commerce',
  version: '1.0.0',
  bijouPeerRange: '^5.0.0',
  blocks: ['product-list', 'checkout-flow'],
  tags: ['commerce', 'catalog', 'checkout'],
});
```

Runtime-engine posture:

- a block renders into ordinary Bijou visible nodes
- visible nodes still resolve through RE-035 layout envelopes
- interaction still emits commands instead of mutating geometry
- modes lower semantic facts instead of parsing rendered cells
- host adapters own files, credentials, network, storage, and process effects

That keeps third-party blocks powerful without making them runtime plugins with unbounded authority.

Relationship To DX-034

DX-034 - Declarative View Data Binding is the design gate between metadata-only blocks and provider-bound AppShell composition.

DX-031 answers:

- what is a block?
- what slots, variants, modes, facts, stories, and package metadata does it declare?
- how can tooling discover block contracts before rendering?

DX-034 answers:

- how does an active block or view declare the data it needs?
- which provider scope satisfies those requirements?
- what immutable snapshot is currently rendered?
- how do user interactions leave the view tree as Commands instead of direct data mutation?

Rendered AppShell work should build on both cycles. AppShell needs named block slots, but it also needs a unidirectional data-binding loop so nested navigation, content, inspector, status, and overlay blocks can render provider-backed snapshots without owning business state.

Schema-Bound Blocks

The schema-bound boundary for app builders should be direct:

1. define or import a schema
2. bind that schema to a block
3. pass a schema-compliant object
4. receive immutable block render input and validation facts

In app code, the target ergonomics should feel like:

```
const productDetailsBaseBlock = defineBlock({
  metadata: productDetailsMetadata,
  render: productDetails,
});

const productDetailsBlock = defineSchemaBlock({
  block: productDetailsBaseBlock,
  schema: zodBlockSchema(productDetailsSchema),
  bind: product => ({
    input: {
      slots: {
        title: product.name,
        summary: product.summary,
        specs: product.specs,
        reviews: product.reviews,
        primaryAction: { id: 'add-to-cart', label: 'Add to cart' },
      },
      config: {
        variant: 'purchase-detail',
      },
    },
    facts: [{ kind: 'entity', key: 'product', value: product.id }],
  }),
});

bindSchemaBlockInput(productDetailsBlock, {
  id: 'bijou-pro',
  name: 'Bijou Pro',
  summary: 'Runtime truth for terminal apps.',
  specs: [{ label: 'Mode', value: 'interactive' }],
  reviews: [],
});
```

That is the real DX target for this layer: give the schema-bound block an unknown object and it should validate the boundary before producing immutable render input, validation issues, and lower-mode facts. Rendering remains the ordinary block renderer’s job.

Zod can become the first optional authoring adapter because it is familiar in the TypeScript ecosystem and Bijou already uses it in MCP tooling. The core block contract is schema-adapter shaped so the runtime does not require Zod as a hard dependency.

Recommended split:

- core block API accepts BlockSchemaAdapter<Data>

- optional Zod helper adapts Zod schemas into that adapter
- future helpers can adapt JSON Schema, Valibot, TypeBox, or generated schemas
- block packages can choose the adapter they publish with
- tooling can inspect schema facts when the adapter exposes them

Zod adapter sketch:

```
type ZodLikeResult<Data> =
  | { readonly success: true; readonly data: Data }
  | { readonly success: false; readonly error: unknown };

interface ZodLikeSchema<Data> {
  readonly safeParse: (input: unknown) => ZodLikeResult<Data>;
}

declare function zodBlockSchema<Data>(
  schema: ZodLikeSchema<Data>,
): BlockSchemaAdapter<Data>;
```

Schema-bound blocks should standardize:

- validation errors into ErrorScreen, Banner, or field-level facts
- defaults into config and slots
- object fields into named block slots
- object relationships into semantic facts
- mode-lowering facts without parsing rendered output
- docs, examples, and story captures from the same schema-bound fixture data

Rules:

- schema validation happens before rendering
- invalid data produces explicit validation facts, not partial mystery UI
- schema-bound blocks validate boundary data and do not fetch, subscribe, dispatch, compose runtime views, or render AppShell
- schema-bound blocks still render through ordinary Bijou layout envelopes
- schemas may describe data shape, but blocks still own presentation choices
- secret fields must be redacted before metadata, diagnostics, or captures

Compiler Path: Schema To Blocks

The schema-bound API opens a larger tooling path:

```
GraphQL schema / operations
  -> semantic model
  -> TypeScript types
  -> Zod or schema adapter
  -> block metadata
  -> schema-bound block package
  -> stories, captures, docs, and tests
```

This is not first-slice block runtime work, but it is a coherent future lane. ~/git/wesley already points in this direction: it has GraphQL schema lowering, operation catalogs, TypeScript emission, and dialect seams that

mention future SQLite and other SQL targets. Bijou should be able to consume that kind of semantic output and generate useful block packages.

Potential compiler outputs:

- Zod schemas or generic BlockSchemaAdapter objects
- TypeScript data types
- default block choices by schema shape
- candidate block metadata
- fixture data and story captures
- mode-lowering semantic facts
- docs tables for fields, relationships, and actions

Useful mappings:

- GraphQL object type -> ProductDetails, UserInfo, ApplicationCard
- GraphQL list query -> ProductList, Grid, BlogGrid, Leaderboard
- GraphQL mutation -> FormPanel, CheckoutFlow, ButtonSet
- GraphQL enum -> select, segmented control, or SettingsPanel row
- GraphQL relationship -> nested section, CommentsThread, or detail pane
- database table -> grid/list/detail blocks
- database view -> read-only dashboard or data visualization block

Local storage can enter through the same seam. A SQLite-backed app, local JSON store, file-backed cache, or generated persistence adapter can provide the data object. The block should not care whether the object came from GraphQL, SQLite, REST, a file, or memory after it passes the schema adapter.

The compiler should produce suggestions, not magic authority. Generated blocks still need:

- explicit metadata
- explicit schema adapters
- explicit slot binding
- explicit mode-lowering facts
- story captures with representative data
- human-reviewable output

The win is speed. A maintainer should be able to point a compiler at a semantic schema and get a boring, inspectable first draft of a Bijou app: lists, details, forms, dashboards, charts, empty/error states, and docs surfaces that already obey the block contract.

Tooling Payoff

A metadata-first block API should make future Bijou tooling much cheaper.

Tools should be able to index block packages and answer:

- which blocks exist?
- which slots are required?

- which variants are supported?
- which modes does a block lower to?
- which semantic facts does a block promise?
- which stories or captures prove the block?
- which components, themes, or adapters does the block depend on?

That directly supports:

- a Storybook-like block and component workstation
- theme token inspectors
- layout inspectors
- focus and input routing inspectors
- mode-lowering reports
- DOGFOOD block galleries
- MCP docs and block search payloads
- fixture-to-docs promotion tooling
- block package compatibility reports
- visual and semantic regression review

The important constraint is that tooling reads declared block facts first. It may render stories, capture screenshots, inspect surfaces, or run assertions afterward, but it should not need to scrape terminal output just to discover what a block is.

Variants And Configuration

Blocks should offer named variants when the job is stable but the surface shape or input contract changes.

A variant is still the same block. It should keep the same broad purpose while declaring any changed slots, focus rules, semantic facts, and mode-lowering behavior.

Configuration is different from variants. Configuration changes parameters inside a chosen variant.

Examples:

- AppShell can offer docs, workbench, and focused-task variants.
- CommandPalette can offer commands-only, commands-with-preview, and destination-search variants.
- Hero can offer compact, branded, docs-landing, and playable-demo variants.
- LoadingScreen can offer indeterminate, determinate, and staged-bootstrap variants.
- TitleScreen can offer press-enter, loading-gate, and auth-gateway variants.
- PerfHud can offer compact overlay, detailed panel, and recorder variants.
- LoginForm can offer manual-password, one-time-code, and credential-provider variants.
- SignUpForm can offer password, invite-code, and provider-handoff variants.

Rules:

- each variant must declare required slots and semantic facts
- each variant must lower across interactive, static, pipe, and accessible modes
- config must be inspectable without exposing secret values
- unsupported variants should fail clearly instead of silently degrading
- host adapters provide credentials, files, network access, and other effects

Secret-bearing config is not block metadata. The metadata may name a credential provider capability, but the secret value stays in host-owned state.

Auth blocks should use a provider seam rather than hard-code a vault package. A host can adapt a provider such as @git-stunts/vault behind that seam, but the block contract should only see a named credential provider and its safe status facts.

Sketch:

```
type LoginCredentialMode =
  | 'manual-password'
  | 'one-time-code'
  | 'credential-provider';

interface LoginBlockConfig {
  readonly credentialMode: LoginCredentialMode;
  readonly credentialProviderId?: string;
  readonly allowRememberIdentity?: boolean;
  readonly requireMfaCode?: boolean;
}
```

Existing Proto-Blocks

Some Bijou componentry is already block-shaped. DX-031 should inventory and promote that work instead of rebuilding it under new names.

A component or component group is a proto-block when it already has:

- a recognizable product job
- multiple composed lower-level pieces
- stable visible regions
- reusable input behavior
- mode-lowering expectations
- diagnostic or semantic facts worth testing

The promotion path should be:

1. identify the existing component or story
2. name the block job it already performs
3. declare slots, variants, config, modes, and semantic facts
4. add story captures and mode-lowering assertions
5. keep the implementation if it already fits the contract

The important rule is that “block” is a standardization level, not a rewrite mandate. If existing componentry already satisfies the contract, promotion should document and test it first.

Block Scale

Blocks can exist at different scales. DX-031 should name the scale instead of pretending every useful pattern is a full application shell.

Block scales:

- app blocks: whole app states or persistent shells
- workspace blocks: full app workspaces under a shell
- section blocks: large content or product sections inside an app
- panel blocks: reusable panes, drawers, modals, and diagnostics
- control blocks: repeated input/action compositions
- inline blocks: local status, hints, and compact feedback beside content
- item blocks: repeated cards, comments, rows, and list entries
- data blocks: tabular, chart, or evidence-focused semantic surfaces
- diagnostic blocks: inspectable explanation, health, and instrumentation surfaces
- overlay blocks: transient shell-level feedback or affordances

Some names are component-like on purpose. Button and Slider may remain primitive components, while ButtonSet, ActionFooter, or SliderControl can be blocks when they standardize layout, labels, validation, focus, and mode-lowering facts around those primitives.

Block Families

The first catalog should group blocks by job rather than by implementation package. That keeps the list usable for humans and selectable by agents.

App structure:

- AppShell
- DashboardShell
- HeaderBar
- FooterBar
- NavigationBar
- ReaderSurface
- NavigationList
- TableOfContents
- InspectorPanel
- TitleScreen
- HeroHeader
- Hero

- LoadingScreen

Content and marketing sections:

- FeatureBlock
- FeaturesSection
- FAQSection
- Banner
- CallToAction
- PricingTiers
- StatsPanel
- ComparisonPanel
- ErrorScreen
- CommentsThread
- Carousel
- Timeline

Command and assistance:

- CommandPalette
- HelpPanel
- SettingsPanel

Transient and blocking overlays:

- NotificationStack
- ModalDialogStack
- QuitConfirmationDialog

Forms and identity:

- UserInfo
- FormPanel
- LoginForm
- SignUpForm

Controls:

- ButtonSet
- SliderControl

Data and workbench candidates:

- Grid
- WorkbenchTable
- FormFlow
- ActivityLog
- EmptyErrorState
- DetailPane
- SearchResultsPane
- FileExplorerPane

- ApplicationCard
- ApplicationCardGrid
- DataVisualizationPanel
- ProductList
- ProductDetails
- ProductCategory
- ProductSpecsSheet
- BlogGrid
- BlogPost
- Leaderboard
- TerminalPanel

Commerce:

- ShoppingCart
- CheckoutFlow
- PaymentMethods
- OfferModal
- ReviewsBlock

Communication:

- ChatPanel

Runtime diagnostics:

- PerfHud
- LayoutInspectorPanel
- InputRoutingInspector
- SurfaceDiffPanel

Initial Block Catalog

AppShell

Standard top-level app frame.

Variants:

- docs shell
- workbench shell
- focused-task shell

Slots:

- header
- navigation
- content
- inspector
- status

- overlays

Behavior:

- owns the top-level app layout
- switches between narrow, standard, and wide variants
- keeps content primary in constrained terminals
- exposes named regions for focus and accessibility

ReaderSurface

Standard long-form reading surface for docs, guides, release notes, and prose.

Variants:

- article reader
- reference page
- release notes

Slots:

- title
- breadcrumbs
- body
- table of contents
- source or provenance metadata
- actions

Behavior:

- preserves full semantic content in pipe mode
- reports current section and scroll facts in accessible mode
- can live inside AppShell as the primary content region

NavigationList

Grouped navigation list for sidebars, drawers, and quick navigation surfaces.

Variants:

- grouped navigation
- flat navigation
- filtered navigation

Slots:

- groups
- items
- active item
- badges
- filter

- empty state

Behavior:

- keeps the active item visible when possible
- exposes grouped navigation facts
- lowers to source-order destinations in pipe mode

InspectorPanel

Structured facts and contextual actions for selected entities, diagnostics, layout nodes, focus maps, runtime records, or theme reports.

Variants:

- facts panel
- diagnostics panel
- action panel

Slots:

- title
- summary
- sections
- facts
- warnings
- actions
- footer

Behavior:

- renders stable key-value facts
- exposes warnings and actions semantically
- works as a pane, drawer, or modal body

PerfHud

Performance and runtime health surface for development builds, diagnostics, benchmarking, and DOGFOOD instrumentation.

Variants:

- compact overlay
- detailed panel
- recorder
- budget warning strip

Config:

- metric set
- sample interval
- budget thresholds

- history depth
- overlay position

Slots:

- title
- frame metrics
- render metrics
- memory metrics
- event-loop metrics
- budget warnings
- history sparkline
- actions

Behavior:

- can be promoted from existing Perf HUD componentry
- renders as overlay, panel, drawer, or status strip
- exposes current metrics as semantic facts
- lowers to stable metric lines in static and pipe modes
- reports threshold state in accessible mode

CommandPalette

Searchable command and destination picker.

Variants:

- commands only
- commands with preview
- destination search

Slots:

- query
- grouped results
- selected result
- preview
- footer hints
- empty state

Behavior:

- keyboard-first command execution
- optional preview in wide layouts
- accessible combobox/listbox-style facts
- pipe mode can emit matching command ids and titles

HelpPanel

Standard help and keybinding surface.

Variants:

- global help
- contextual help
- searchable keybindings

Slots:

- title
- context summary
- command groups
- shortcuts
- search/filter
- footer hints

Behavior:

- can render as a panel, drawer, or modal
- groups commands by context
- exposes available actions in accessible mode

SettingsPanel

Standard preferences surface for theme, shell, density, mode, and app settings.

Variants:

- compact settings
- sectioned settings
- full-screen narrow settings

Slots:

- sections
- rows
- active section
- descriptions
- actions

Rows:

- toggle
- select
- segmented choice
- theme swatch
- numeric stepper
- readonly fact
- danger action

Behavior:

- aligns values consistently
- supports drawer or full-screen narrow presentation
- lowers to key=value facts in pipe mode

NotificationStack

Stacked notifications and transient status messages.

Variants:

- transient toasts
- persistent history
- mixed severity stack

Slots:

- notifications
- active toast
- history
- actions

Behavior:

- keeps transient display separate from notification history
- exposes severity, title, body, and action facts
- should eventually route hit-testing through retained layout

ModalDialogStack

Standard stacked modal surface for blocking overlays.

Variants:

- single blocking modal
- nested modal stack
- confirmation flow

Slots:

- modal stack
- active modal
- backdrop
- title
- body
- actions

Behavior:

- topmost modal receives input first
- lower app surfaces are blocked while active
- focus is trapped inside the active modal
- accessible mode reports modal title, body, and available actions

QuitConfirmationDialog

Specialized modal for quitting, closing, or abandoning dirty work.

Variants:

- quit cleanly
- quit with dirty work
- save before quit

Slots:

- title
- message
- dirty summary
- confirm action
- cancel action
- optional save action

Behavior:

- defaults to cancel as the safer action
- reports destructive intent clearly
- can be hosted by ModalDialogStack

LoadingScreen

Readiness surface for startup, connection, indexing, generation, or other bounded and unbounded waits.

Variants:

- indeterminate loading
- determinate progress
- staged bootstrap
- retryable failure

Config:

- progress mode
- stage labels
- cancel policy
- retry policy
- timeout policy

Slots:

- title
- current stage
- progress value
- progress label
- detail message

- cancel action
- retry action

Behavior:

- exposes readiness and progress facts without relying on animation
- lowers to stable stage/progress text in static and pipe modes
- reports retryable failure state accessibly
- can be embedded inside TitleScreen

TitleScreen

App entry surface before the main application shell becomes active.

Variants:

- press enter
- loading gate
- auth gateway
- profile chooser

Config:

- entry action
- startup state
- auth requirement
- default next route
- skip policy

Slots:

- title
- subtitle
- visual surface
- loading surface
- login form
- sign-up form
- primary action
- secondary actions
- status or version facts

Behavior:

- owns the app entry state before AppShell becomes active
- can compose Hero, LoadingScreen, LoginForm, and SignUpForm
- exposes whether the app is waiting, ready, blocked, or authenticated
- preserves title, readiness, and available actions in lower modes

Hero

Opening/title surface for branded apps, demos, docs landing screens, and playable examples.

Variants:

- compact hero
- branded hero
- docs landing hero
- playable demo hero

Slots:

- title
- subtitle
- visual surface
- primary action
- secondary actions
- status or version facts

Behavior:

- remains useful when visual ornamentation lowers away
- can host shader or canvas visuals inside a layout-owned envelope
- pipe and accessible modes preserve title, purpose, and actions

LoginForm

Standard authentication form for email/password, token, or device-code flows.

Variants:

- manual password
- one-time code
- credential provider

Config:

- credential mode
- credential provider id
- remember identity policy
- MFA requirement

Slots:

- title
- fields
- credential input
- credential provider status
- validation summary
- submit action

- secondary actions
- provider options

Behavior:

- never hides validation facts in styling alone
- supports non-interactive pipe/accessibility summaries without exposing secrets
- can request credentials from a host-owned provider seam
- can adapt @git-stunts/vault through that seam without hard-coding it
- can compose with FormFlow later

SignUpForm

Standard account creation form.

Variants:

- password signup
- invite-code signup
- provider handoff

Config:

- required field set
- consent policy
- provider id
- validation mode

Slots:

- title
- fields
- validation summary
- terms acknowledgement
- submit action
- secondary actions

Behavior:

- separates validation, consent, and submit states
- lowers field requirements and validation facts accessibly
- can share form-row contracts with LoginForm

Additional Candidate Blocks

These blocks are useful enough to name now, but should follow the first implementation set unless an existing proto-block is ready to promote.

HeaderBar

Top chrome for application, section, or panel identity.

Variants:

- app header
- section header
- compact header
- hero header

Slots:

- title
- subtitle
- breadcrumbs
- primary action
- secondary actions
- status facts

Behavior:

- exposes heading level and current location semantically
- can host search, command, help, or account affordances
- lowers to title, location, status, and actions in pipe mode

FooterBar

Bottom chrome for status, actions, navigation, or provenance.

Variants:

- status footer
- action footer
- pagination footer
- provenance footer

Slots:

- status
- primary action
- secondary actions
- pagination
- provenance
- shortcut hints

Behavior:

- keeps status and action facts separate
- can host ButtonSet without hiding actions in decoration
- lowers shortcut hints only when useful for the output mode

TableOfContents

Structured navigation for long-form content.

Variants:

- flat contents
- nested contents
- scroll-synced contents
- filtered contents

Slots:

- sections
- active section
- progress facts
- filter

Behavior:

- exposes document structure and active section
- can live in ReaderSurface, drawer, or narrow navigation mode
- lowers to source-order section links in pipe mode

HeroHeader

Header-scale opening region for pages that need title, pitch, status, and actions without becoming a full Hero.

Variants:

- compact page hero
- product hero header
- docs hero header
- object hero header

Slots:

- title
- summary
- media or visual surface
- primary action
- secondary actions
- status facts

Behavior:

- keeps the title as the first semantic signal
- can degrade to an ordinary HeaderBar
- preserves actions and status in lower modes

FeaturesSection

Grouped feature or capability presentation.

Variants:

- feature list
- feature grid
- comparison features
- checklist features

Slots:

- heading
- feature items
- icons or markers
- supporting text
- actions

Behavior:

- exposes each feature as a semantic item
- avoids relying on equal-height visual cards for meaning
- lowers to a stable ordered list

FAQSection

Question and answer disclosure surface.

Variants:

- static FAQ
- accordion FAQ
- searchable FAQ
- categorized FAQ

Slots:

- questions
- answers
- active question
- categories
- search/filter

Behavior:

- exposes expanded/collapsed state
- preserves all answers in pipe mode by policy
- can lower to headings and paragraphs without visual chrome

Banner

Inline announcement, warning, success, or callout surface.

Variants:

- info banner
- warning banner
- success banner
- error banner
- action banner

Slots:

- severity
- title
- body
- icon or marker
- actions
- dismiss action

Behavior:

- exposes severity as a semantic fact
- does not rely on color alone
- can promote to notification or modal behavior when configured

ErrorScreen

Full-screen or panel-sized error recovery surface.

Variants:

- not found
- permission denied
- crashed
- offline
- retryable failure

Slots:

- title
- message
- diagnostic facts
- retry action
- fallback action
- report action

Behavior:

- separates user-facing message from diagnostic facts
- exposes recovery actions clearly

- lowers to stable error text and available actions

UserInfo

Account, identity, profile, or actor summary.

Variants:

- compact identity
- profile summary
- account switcher item
- permission summary

Slots:

- display name
- handle or email
- avatar or marker
- role
- status
- actions

Behavior:

- keeps identity facts separate from authentication secrets
- can compose into headers, title screens, settings, and comments
- lowers to name, role, status, and actions

FormPanel

General form composition for fields, validation, and actions.

Variants:

- single step form
- multi-step form
- review form
- readonly form summary

Slots:

- title
- field groups
- validation summary
- dirty state
- primary action
- secondary actions

Behavior:

- exposes required fields, validation state, and dirty state
- can compose ButtonSet, SliderControl, LoginForm, and SignUpForm

- lowers to field facts and validation facts without secret values

ButtonSet

Grouped action controls with explicit priority and focus order.

Variants:

- primary and secondary
- toolbar buttons
- segmented action group
- destructive confirmation actions

Slots:

- primary action
- secondary actions
- destructive action
- disabled reasons
- shortcut hints

Behavior:

- exposes action priority and disabled reasons
- keeps destructive actions explicit
- lowers to action labels and availability facts

SliderControl

Labeled numeric control with value, range, and validation semantics.

Variants:

- simple slider
- stepped slider
- range slider
- setting row slider

Slots:

- label
- value
- minimum
- maximum
- step
- help text
- validation state

Behavior:

- exposes value and range as semantic facts
- supports keyboard and pointer adjustment

- lowers to label=value with min, max, and step facts

Grid

General cell or item grid for selectable and scannable collections.

Variants:

- fixed grid
- responsive grid
- selectable grid
- virtualized grid

Slots:

- items
- selected item
- empty state
- sort/filter controls
- pagination or viewport facts

Behavior:

- preserves source order and selection facts
- can host ApplicationCard or other item blocks
- lowers to list/table shape depending on mode

ApplicationCard

Repeated app, tool, document, package, or integration summary.

Variants:

- compact card
- detailed card
- selectable card
- actionable card

Slots:

- title
- summary
- status
- metadata
- primary action
- secondary actions

Behavior:

- exposes title, status, metadata, and actions semantically
- avoids making visual card shape the only grouping signal
- can live in ApplicationCardGrid, search results, or dashboards

ApplicationCardGrid

Grid of application cards with filtering, selection, and status.

Variants:

- app launcher
- integration gallery
- docs card grid
- selectable dashboard

Slots:

- cards
- filter
- sort
- selected card
- empty state

Behavior:

- composes Grid and ApplicationCard
- keeps selected card and active filter visible to agents
- lowers to ordered card summaries

DataVisualizationPanel

Chart or graph panel with accessible facts and fallback summaries.

Variants:

- sparkline panel
- bar chart panel
- timeline panel
- graph summary panel

Slots:

- title
- visualization surface
- legend
- data summary
- selected datum
- actions

Behavior:

- exposes chart type, labels, values, and selected datum
- lowers to data summaries when visual rendering is unavailable
- does not make color or geometry the only carrier of meaning

Carousel

Ordered slide or item viewer with optional animated transitions.

Variants:

- static carousel
- spring slide carousel
- paged carousel
- autoplay carousel

Config:

- active index
- transition policy
- spring preset
- autoplay policy
- motion preference

Slots:

- slides
- active slide
- previous action
- next action
- page indicators
- pause action

Behavior:

- exposes active slide, count, and navigation actions
- respects reduced-motion policy
- lowers to active slide plus ordered slide summaries
- treats spring animation as presentation, not semantic state

CommentsThread

Conversation, review, or annotation surface.

Variants:

- flat comments
- threaded comments
- review comments
- composer with preview

Slots:

- comments
- focused comment
- author info
- timestamps

- reactions
- composer
- moderation actions

Behavior:

- preserves author, time, reply, and moderation facts
- can compose UserInfo, FormPanel, and ButtonSet
- lowers to source-order conversation text

Application Domain Candidate Blocks

These blocks are broad app-building pieces. They should usually be cataloged before implementation, then proven by the app family that needs them first.

DashboardShell

Operational dashboard frame for metrics, feeds, actions, and drill-down panels.

Common variants:

- executive dashboard
- operations dashboard
- product analytics dashboard
- personal workspace dashboard

Composes:

- HeaderBar
- NavigationBar
- StatsPanel
- DataVisualizationPanel
- ActivityLog
- Leaderboard
- DetailPane

NavigationBar

Horizontal or compact navigation chrome for app sections and top-level routes.

Common variants:

- top navbar
- compact navbar
- tab navbar
- account navbar

Composes:

- HeaderBar
- UserInfo
- CommandPalette
- ButtonSet

FeatureBlock

Single feature item with title, support copy, marker, media, and action.

Common variants:

- compact feature
- illustrated feature
- checklist feature
- feature with proof point

Composes:

- FeaturesSection
- ApplicationCard
- CallToAction

CallToAction

Focused conversion block for a primary next step.

Common variants:

- inline CTA
- banner CTA
- modal CTA
- footer CTA

Composes:

- ButtonSet
- Banner
- OfferModal

PricingTiers

Plan comparison and purchase entry block.

Common variants:

- simple tiers
- feature matrix tiers
- usage-based tiers
- enterprise contact tier

Composes:

- ComparisonPanel
- ProductSpecsSheet
- CheckoutFlow
- CallToAction

StatsPanel

Metric summary block for counts, deltas, health, and status.

Common variants:

- KPI strip
- metric cards
- trend stats
- budget stats

Composes:

- DataVisualizationPanel
- PerfHud
- DashboardShell

ComparisonPanel

Side-by-side comparison block for products, options, plans, or entities.

Common variants:

- compare A vs B
- compare A vs B vs C
- feature matrix
- ranked comparison

Composes:

- Grid
- ProductSpecsSheet
- PricingTiers

Timeline

Chronological event, release, activity, or process surface.

Common variants:

- release timeline
- activity timeline
- process timeline
- audit timeline

Composes:

- ActivityLog
- UserInfo
- CommentsThread

ProductList

Browsable product collection with filters, sort, status, and actions.

Common variants:

- catalog list
- marketplace grid
- inventory list
- selectable product picker

Composes:

- Grid
- ProductCategory
- ApplicationCard
- SearchResultsPane

ProductDetails

Detailed product page or panel for title, media, specs, reviews, and purchase actions.

Common variants:

- product page
- product drawer
- product comparison detail
- purchase detail

Composes:

- HeroHeader
- ProductSpecsSheet
- ReviewsBlock
- ShoppingCart
- CallToAction

ProductCategory

Category navigation and product grouping block.

Common variants:

- category list
- category landing
- faceted category

- nested category tree

Composes:

- NavigationList
- ProductList
- Grid

ProductSpecsSheet

Structured product, plan, or object specifications.

Common variants:

- key-value specs
- grouped specs
- comparison specs
- technical sheet

Composes:

- InspectorPanel
- ComparisonPanel
- ProductDetails

ReviewsBlock

Ratings, reviews, testimonials, or feedback surface.

Common variants:

- rating summary
- review list
- review composer
- testimonial carousel

Composes:

- CommentsThread
- UserInfo
- Carousel
- FormPanel

ShoppingCart

Cart contents, quantities, totals, discounts, and checkout entry.

Common variants:

- compact cart
- full cart
- cart drawer

- saved cart

Composes:

- ProductList
- PricingTiers
- OfferModal
- CheckoutFlow

CheckoutFlow

Purchase flow from cart review through payment and confirmation.

Common variants:

- single-page checkout
- stepped checkout
- invoice checkout
- confirmation summary

Composes:

- FormPanel
- PaymentMethods
- ShoppingCart
- ErrorScreen

PaymentMethods

Payment method selection and validation block.

Common variants:

- saved methods
- card entry
- provider handoff
- invoice terms

Composes:

- FormPanel
- ButtonSet
- CheckoutFlow

OfferModal

Promotional, upgrade, discount, or retention offer.

Common variants:

- upgrade offer
- coupon offer

- trial-ending offer
- retention offer

Composes:

- ModalDialogStack
- CallToAction
- PricingTiers

BlogGrid

Collection of posts, guides, updates, or articles.

Common variants:

- publication grid
- release notes grid
- author archive
- topic archive

Composes:

- Grid
- ApplicationCard
- BlogPost

BlogPost

Long-form article, update, or guide page.

Common variants:

- article
- release post
- tutorial
- changelog post

Composes:

- ReaderSurface
- TableOfContents
- CommentsThread
- CallToAction

Leaderboard

Ranked list of users, runs, scores, packages, or entities.

Common variants:

- score leaderboard
- performance leaderboard

- contribution leaderboard
- ranked products

Composes:

- Grid
- UserInfo
- StatsPanel

ChatPanel

Conversation surface for human chat, support, agents, or collaboration.

Common variants:

- support chat
- agent chat
- team chat
- command chat

Composes:

- CommentsThread
- UserInfo
- FormPanel
- NotificationStack

TerminalPanel

Embedded terminal, command runner, REPL, log stream, or transcript viewer.

Common variants:

- terminal session
- command runner
- readonly transcript
- log tail

Composes:

- AppShell
- CommandPalette
- ActivityLog
- PerfHud

Remaining Useful Candidates

With the current list, the catalog can cover most app scaffolds. The remaining blocks I would still keep in reserve are:

- OnboardingFlow

- SearchFilterBar
- UploadDropzone
- FilePreview
- CalendarPanel
- KanbanBoard
- MapPanel
- PermissionsMatrix
- TeamMembersPanel
- AuditLog
- ImportExportPanel
- EmptyState
- ReleaseNotesPanel
- AIConversationPanel
- DiffViewer

Second-Wave Blocks

These are likely useful, but should follow the first catalog once the block contract is proven:

- DashboardShell
- NavigationBar
- HeaderBar
- FooterBar
- TableOfContents
- HeroHeader
- FeatureBlock
- FeaturesSection
- FAQSection
- Banner
- CallToAction
- PricingTiers
- StatsPanel
- ComparisonPanel
- ErrorScreen
- UserInfo
- FormPanel
- ButtonSet
- SliderControl
- Grid
- ApplicationCard
- ApplicationCardGrid
- DataVisualizationPanel
- Carousel
- CommentsThread

- Timeline
- ProductList
- ProductDetails
- ProductCategory
- ProductSpecsSheet
- ReviewsBlock
- ShoppingCart
- CheckoutFlow
- PaymentMethods
- OfferModal
- BlogGrid
- BlogPost
- Leaderboard
- ChatPanel
- TerminalPanel
- WorkbenchTable
- FormFlow
- ActivityLog
- EmptyErrorState
- DetailPane
- SearchResultsPane
- FileExplorerPane

First Implementation Set

The first implementation should be small:

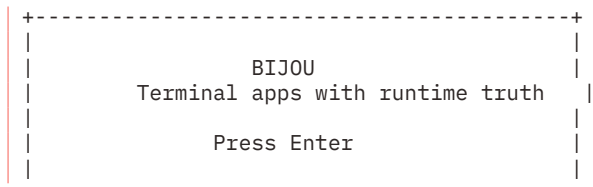
1. AppShell
2. ReaderSurface
3. InspectorPanel

These prove shell/content/inspectable composition without dragging in auth, workspace docking, or complex table behavior.

Terminal Screen Sketches

These sketches are not renderer snapshots. They are shape contracts for the kind of TUI surfaces the first blocks should make easy to assemble.

TitleScreen with a simple press-enter gate:



```
|  v5.0.0                         Help     |
+------------------------------------------+
```

TitleScreen composing LoadingScreen:

```
+ Bijou ------------------------------------------------- startup: loading +
|                                                                           |
|                         Preparing workspace                               |
|                                                                           |
|  [############----------------------------]  32%                          |
|                                                                           |
|  stage 2/5: indexing docs                                                 |
|                                                                           |
|  Cancel                                                                   |
+ static output reports stage and progress, not animation frames ----------+
```

TitleScreen with an auth gateway:

```
+ Bijou ------------------------------------------------ account required +
| Welcome back                                                              |
|                                                                           |
| Email                                                                     |
| [james@example.com                    ]                                   |
|                                                                           |
| Password                                                                  |
| [*********************************      ]                                 |
|                                                                           |
| [Sign in]     [Create account]                                  Help      |
+ lower modes expose available actions without exposing credentials --------+
```

Wide AppShell with ReaderSurface and InspectorPanel:

```
+ Bijou Docs ------------------------------------------- mode: interactive +
| Nav                    | Reader                        | Inspector        |
| > Guide                | # Layout envelopes            | node docs.body   |
|   API                  |                               | role viewport    |
|   Examples             | Every visible node resolves   | inline 24..72    |
|                        | a layout envelope before      | block 1..21      |
|                        | rendering.                    | reason stack     |
|                        |                               | actions          |
|                        |                               | - explain        |
|                        |                               | - copy facts     |
+ status: section 2/8 ---------------------------- notifications: 1 pending +
```

Narrow AppShell preserving primary content:

```
+ Bijou Docs -----------------------------+
| # Layout envelopes                      |
|                                         |
| Every visible node resolves a layout    |
| envelope before rendering.              |
|                                         |
| [nav] [search] [help] [inspect]         |
+ section 2/8 ----------------------------+
```

CommandPalette hosted by ModalDialogStack:

```
| App content is inert while the modal stack owns focus                  |
|                                                                        |
|   + Command Palette ------------------------------------------------+  |
|   | > layout                                                      |  |
|   |                                                               |  |
|   | Commands                                                      |  |
|   |   Explain current layout node                                 |  |
|   |   Toggle layout inspector                                     |  |
|   |   Copy layout facts                                           |  |
|   |                                                               |  |
|   | Destinations                                                  |  |
|   |   Layout envelopes                                            |  |
|   |   Viewports and overflow                                      |  |
|   + enter run ------------------------------------------ esc close +  |
```

LoginForm credential variants:

```
+ Sign in: manual password ----------------+
| Email                                     |
| [james@example.com                    ]  |
|                                           |
| Password                                  |
| [********************************      ]  |
|                                           |
| [Sign in]                          Help   |
+ static/pipe output must not expose secret+

+ Sign in: one-time code ------------------+
| Email                                     |
| [james@example.com                    ]  |
|                                           |
| Code                                      |
| [123456                               ]   |
|                                           |
| [Verify]                  Resend    Help  |
+ accessible output exposes expiry, not code+

+ Sign in: credential provider ------------+
| Account                                   |
| james@example.com                         |
|                                           |
| Provider                                  |
| @git-stunts/vault adapter: available      |
|                                           |
| [Unlock]                           Cancel |
+ block sees provider status, not secret   +
```

SVG Block Mockups

These SVGs are visual mockups of TUI block shapes. They are not layout diagrams, renderer snapshots, or pixel-perfect specs. The point is to make common block compositions easier to scan in docs and reviews.

App structure block mockups

Wide and narrow app shell mockups showing navigation, reader, and inspector blocks. AppShell wide Bijou Docs interactive

NavigationList > Guide API Examples

ReaderSurface # Layout envelopes Every visible node resolves layout truth before rendering.

InspectorPanel node docs.body role viewport reason stack - explain - copy facts

status: section 2/8 notifications: 1

AppShell narrow Bijou Docs ReaderSurface # Layout envelopes Primary content stays visible on small terms. nav search help ins section 2/8

Navigation drawer variant Navigation > Current Search Drawer overlays content focus and source order stay explicit

Inspector as modal body InspectorPanel same facts, different host

App structure blocks show the same semantic regions in wide, narrow, drawer, and modal-hosted forms.

Entry and readiness block mockups

Title, hero, and loading screen variants for app entry flows. TitleScreen: press enter BIJOU runtime truth for terminals Press Enter v5.0.0 Help

TitleScreen + LoadingScreen Preparing workspace stage 2/5: indexing docs Cancel

TitleScreen: auth gateway Welcome back Email james@example.com Password ********** Sign in Create account

Hero: docs landing Bijou Guide Learn by running DOGFOOD. Open Browse

LoadingScreen: retryable failure Workspace unavailable retryable failure Retry Quit

TitleScreen: profile chooser Choose profile > James Guest Enter select Esc quit

Entry blocks separate title, hero, readiness, authentication, and profile selection while allowing those surfaces to compose.

Authentication block mockups

Login and sign-up variants covering password, one-time code, provider, and invite flows. LoginForm: manual password Email james@example.com Password ********** Sign in Help

LoginForm: one-time code Email james@example.com Code 123456 expires in 02:00 Verify Resend

LoginForm: credential provider Account james@example.com Provider @git-stunts/vault ready Unlock Cancel

SignUpForm: password signup Email Password + confirm Create account

SignUpForm: invite code Invite code BIJOU-2026 Terms accepted: yes Continue

SignUpForm: provider handoff Create with provider Use workspace SSO Consent required before handoff

Authentication blocks expose credential modes and provider state without putting secret values into metadata, diagnostics, or lower-mode output.

Command and overlay block mockups

Command palette, help, settings, notifications, modal, and quit dialog block forms. CommandPalette with preview > layout Commands Explain layout node Toggle inspector Copy layout facts Preview node docs.body assigned 72x22 enter run esc close

HelpPanel and SettingsPanel HelpPanel Navigation Enter: open /: search ?: help SettingsPanel Theme [dark] Density [cozy] Mode [auto] Apply

NotificationStack Saved workspace Index still running Sync failed: retry

ModalDialogStack Active modal focus is trapped lower app blocked

QuitConfirmationDialog Quit with unsaved work? 2 docs changed Quit Save Cancel

Command, assistance, notification, and modal blocks show focus ownership and available actions as explicit surface facts.

Runtime diagnostic block mockups

Performance HUD variants showing compact overlay, detailed panel, recorder, and budget warning strip forms. PerfHud: compact overlay fps 60 frame 7ms heap 42MB App content stays behind HUD

PerfHud: detailed panel Runtime metrics render 7ms layout 2ms input 1ms heap 42MB

PerfHud: recorder Capture performance Record Mark samples: 240 last spike: render

PerfHud: budget warning strip layout budget exceeded: 18ms > 12ms actions: explain copy metrics hide

Diagnostic blocks are often existing componentry promoted into explicit slots, variants, config, mode-lowering facts, and story captures.

Section and control block mockups

Header, footer, cards, grids, forms, controls, chart, carousel, FAQ, banner, error, table of contents, and comments block forms. HeaderBar + FooterBar Project Atlas Help content region status ready Save

Grid + ApplicationCard Docs Bench MCP TUI

HeroHeader + Banner Runtime docs Learn layout truth New RE-035 draft Open Compare

FormPanel + controls Preferences Density Mode [auto] Save Cancel

DataVisualizationPanel Frame time selected: render 18ms

Carousel with spring motion Feature tour Slide 2 of 5 < Prev Next > Pause

TOC + FAQ + Comments TOC: Overview Layout Blocks FAQ: What is a block? Comments: James replied

ErrorScreen Could not load workspace Retry Open logs Quit

FeaturesSection - retained layout facts - mode lowering - inspector-friendly contracts

Section, control, item, and data blocks standardize recurring app content without making primitive buttons or sliders pretend to be full app blocks.

Application domain block mockups

Product, commerce, publishing, dashboard, leaderboard, chat, and terminal block forms. ProductList + ProductCategory Category: Components Runtime DOGFOOD filter sort compare

ProductDetails + specs Bijou Pro status available specs: 12 facts reviews: 4.8 / 5 Add to cart Compare

PricingTiers + CTA Free $0 Pro $20 Ent Start trial

ShoppingCart + CheckoutFlow Cart 2 items $48 payment: card ending 4242 discount: SPRING Checkout Payment methods

BlogGrid + BlogPost RE-035 Blocks selected post TOC comments CTA

DashboardShell + leaderboard Today runs 128 pass 98% 1. docs smoke 2. mcp docs 3. tui app

ChatPanel James: ship the floor Agent: facts updated message composer…

TerminalPanel $ npm run docs:inventory Current Truth: ok

ComparisonPanel A vs B vs C speed cost support winner: B for this job

Application-domain blocks cover commerce, publishing, dashboards, chat, and terminal workflows by composing the smaller shell, section, data, and control blocks.

Implementation Outline

1. Done: add the human-readable block catalog and keep it independent from low-level layout implementation.
2. Done: define a minimal public BlockMetadata, BlockDefinition, and BlockPack ageManifest shape in @flyingrobots/bijou.
3. Done: add defineBlock() and defineBlockPackage() authoring helpers, validation/report helpers, and compact summaries.
4. Done: formalize provider-bound view data requirements in DX-034 before hardening rendered AppShell APIs.
5. Done: attach DX-034 view data contracts and command intents to block definitions for discovery.
6. Done: add a structural AppShell composition contract for logical block slots and nested data/command introspection without rendering.
7. Done: prove active runtime binding collection, provider snapshot invalidation into new immutable frames, and command intent routing before rendered AppShell work begins.
8. Done: add adapter-first defineSchemaBlock() support without making Zod, provider lifecycle, AppShell rendering, or DOGFOOD proof part of the core schema-bound block contract.
9. Done: add first-party block definitions and story declarations for AppShel l, ReaderSurface, and InspectorPanel without making them rendered block products.
10. Done: replace the first-party definition placeholders with deterministic rendered proof output and stable lowering facts for AppShell, ReaderSurfac e, and InspectorPanel.
11. Done: add explicit block-tree rendering so nested block declarations render child output without making ordinary block.render() recursive.
12. Done: expand the standard catalog through the explicit DF-031 status/feedback Block slice.
13. Next: add an optional Zod schema adapter package or helper.
14. Done: capture interactive, static, pipe, and accessible outputs for the first implementation set.
15. Done: prove the first three blocks in DOGFOOD before broadening the catalog.
16. Next: add catalog-only variant/config metadata for later blocks without implementing those blocks yet.
17. Keep modal stacks, notifications, auth forms, animated carousels, complex controls, and workspace-like behavior outside DX-031; each needs an explicit later block cycle with its own tests and Method evidence.

Tests To Write First

- Block catalog tests proving the declared first blocks have metadata.
- Public API tests proving community authors can define a block without relying on DOGFOOD internals.
- Public API tests proving a package can declare exported blocks and Bijou peer compatibility.
- Public API tests proving importing a block package does not mutate global runtime state.
- Public API tests proving a schema-bound block validates data before bind or render output is consumed.
- Public API tests proving an adapter can bind valid data into block slots, config, and semantic facts without a hard Zod dependency.
- Public API tests proving invalid schema data becomes validation facts instead of partial rendered UI.
- Tooling tests proving block metadata can be indexed without rendering a terminal surface.
- Tooling tests proving schema facts can be indexed when the schema adapter exposes them.
- Compiler-path tests proving generated block metadata remains explicit and reviewable instead of hidden behind schema inference.
- Block catalog tests proving each declared variant has a stable id, required slots, and semantic facts.
- Block catalog tests proving config options are visible without storing secret values.
- Block catalog tests proving TitleScreen can declare press-enter, loading, and auth-gateway variants.
- Block catalog tests proving LoadingScreen exposes readiness, stage, and progress facts without depending on animation.
- Block catalog tests proving proto-blocks such as PerfHud can be promoted by declaring slots, variants, config, modes, and semantic facts.
- Block catalog tests proving section, control, item, and data blocks declare their scale and mode-lowering facts.
- Block catalog tests proving application-domain blocks declare their composed lower-level blocks instead of re-owning primitive layout behavior.
- Story tests proving each first block has interactive, static, pipe, and accessible captures.
- DOGFOOD tests proving AppShell, ReaderSurface, and InspectorPanel can compose into a real docs page.
- Mode-lowering tests proving semantic facts survive when visual chrome is removed.

Acceptance Criteria

- The block catalog defines the first block names, purposes, slots, and behavior contracts.

- Blocks are documented as compositions, not new primitives.
- Blocks have a public authoring API.
- Blocks can be distributed as ordinary NPM packages.
- Block packages declare exported blocks, docs, tags, and Bijou compatibility.
- Importing a block package does not register runtime behavior globally.
- Blocks can bind schema-compliant data objects to slots, config, validation, semantic facts, and render input.
- The core schema-bound block API is adapter-first and does not require Zod as a hard dependency.
- GraphQL, database, or local-storage schema compilers can generate draft block metadata, schema adapters, stories, and semantic facts.
- Tooling can discover block slots, variants, modes, semantic facts, and story captures from metadata before rendering.
- Blocks are grouped by scale so primitive components and app-level blocks do not blur together.
- Blocks can declare variants and configurable options.
- Title and loading blocks are separate but composable.
- TitleScreen can represent press-enter, loading-gate, and auth-gateway entry flows.
- LoadingScreen can represent indeterminate, determinate, staged, and retryable readiness states.
- Existing block-shaped componentry can be promoted without rewriting it.
- PerfHud is captured as a runtime diagnostics block candidate.
- Headers, footers, grids, cards, controls, charts, carousels, FAQs, banners, errors, tables of contents, and comments are captured as candidate blocks.
- Product, commerce, publishing, dashboard, leaderboard, chat, and terminal workflows are captured as application-domain candidate blocks.
- Auth blocks can describe manual-password, one-time-code, and credential-provider modes without exposing credential values.
- The original first implementation milestone landed as AppShell, ReaderSurface, and InspectorPanel.
- Follow-on blocks are captured or promoted through explicit issue-backed slices instead of hidden first-implementation scope.
- DOGFOOD is named as the proving surface for block behavior.

Risks / Unknowns

- Blocks can become too magical if they hide layout or input behavior instead of exposing slots and facts.
- Bijou’s block ecosystem can make the project feel like an operating environment instead of a small terminal widget library. That is acceptable only if runtime truth and app truth stay separate.
- Third-party block packages add supply-chain and compatibility risk, so the first sharing model should be explicit imports from NPM packages, not remote runtime loading.

- A central block registry may be useful later, but it should index package metadata rather than become required infrastructure.
- Schema-bound blocks can become too magical if schemas are allowed to own presentation. Schemas validate data; blocks own mapping and rendering.
- Making Zod a hard core dependency would make the API less portable. Zod should be the first adapter, not the only possible schema source.
- Schema-to-block compilers can overfit to names and shapes. Generated output should be reviewable suggestions, not uninspectable app authority.
- Auth blocks need careful secret-handling posture before implementation.
- Credential-provider blocks need a narrow host adapter seam so vault integrations stay out of block internals.
- Startup blocks need stable lower-mode facts because loading animation is not meaningful in static, pipe, or accessible output.
- Animated carousels need reduced-motion behavior and stable lower-mode facts before spring transitions become implementation scope.
- Proto-block promotion can reveal older component assumptions that do not yet fit the block contract cleanly.
- Modal and notification stacks depend on retained layout hit-testing work.
- Workspace-style docking should remain outside this cycle.

Playback

DX-031A Block Contract lands the public, metadata-first slice of this cycle.

What is now true:

- @flyingrobots/bijou exports BlockMetadata, BlockDefinition, BlockPackageManifest, defineBlock(), defineBlockPackage(), validateBlockMetadata(), validation/report helpers, and compact summary helpers.
- Block metadata declares package identity, block name, family, scale, modes, docs, slots, variants, config options, composed components, semantic facts, stories, examples, and tags.
- Block package manifests declare version, Bijou peer compatibility, exported blocks, docs, and tags without hidden global registration.
- Public API tests prove authors can define blocks and package manifests without importing DOGFOOD internals.
- The design-system blocks entrypoint now points to the current public contract instead of only naming future candidates.

DX-031B Schema-Bound Blocks adds the boundary validation slice.

What is now true:

- @flyingrobots/bijou exports BlockSchemaAdapter, defineBlockSchemaAdapter(), defineSchemaBlock(), parseBlockSchema(), and bindSchemaBlockInput().
- Schema-bound blocks wrap an existing BlockDefinition, preserving block metadata, data contracts, command intents, and ordinary render ownership.
- Schema adapters validate unknown input into immutable typed data or immutable deterministic issues.

- Schema adapters snapshot their parse/describe callbacks at definition time.
- Binding schema data produces immutable block render input and optional semantic facts without rendering, subscribing, dispatching, resolving providers, or owning AppShell behavior.
- Bind outputs must be plain objects with supported render-input keys, so class instances, built-ins, and backchannel handles cannot silently normalize into empty input.

DX-031C First-Party Block Definitions adds the first-party standard block definition slice.

What is now true:

- @flyingrobots/bijou exports appShellBlock, readerSurfaceBlock, inspectorPanelBlock, standardBlocks, standardBlockStories, and standardBlockPackageManifest.
- The first-party AppShell, ReaderSurface, and InspectorPanel blocks are real defineBlock() outputs with valid metadata.
- ReaderSurface and InspectorPanel declare DX-034 data contracts and command intents.
- Reader and Inspector schema-bound wrappers validate boundary data before producing render input.
- AppShell uses semantic slots: navigation, content, inspector, status, and overlays.
- Definition and composition introspection do not render blocks, subscribe, dispatch, resolve providers, or walk the active runtime tree.

Still out of scope after this slice:

- Zod adapters and schema-to-block compilers.
- DOGFOOD block galleries and multi-mode story captures for rendered blocks.
- Modal stacks, notifications, auth blocks, workspace docking, and complex domain-specific block packages.

DX-031D DOGFOOD Blocks Section publishes the standard blocks through the project’s proving surface.

What is now true:

- DOGFOOD exposes a top-level Blocks section with “What are Blocks”, authoring, pre-made catalog, surface-block, preview, and lowering pages.
- Standard block preview pages are selectable from side navigation and render live AppShell, ReaderSurface, and InspectorPanel examples instead of raw metadata dumps.
- The preview keeps lower-mode output visible as a compact lowering summary.
- The local CounterDemoBlock fixture proves a block-shaped command-intent surface can update through DOGFOOD without becoming part of the shipped standard block package.
- DOGFOOD tests assert the Blocks section, preview routing, curated catalog, live examples, lowering posture, and fixture command keys.

DX-031E Rendered Standard Block Proof replaces the first-party placeholders with deterministic render output.

What is now true:

- AppShell, ReaderSurface, and InspectorPanel render concrete string output for interactive, static, pipe, and accessible modes.
- AppShell renders semantic region output for navigation, content, inspector, status, and overlays.
- ReaderSurface renders content, navigation, and outline output.
- InspectorPanel renders selection, details, and actions output.
- Render facts stay stable across modes so lowering checks do not need to parse rendered text.
- Render results do not expose provider handles, subscription handles, refresh methods, active-tree traversal, or command dispatch callbacks.

Still out of scope after this slice:

- Provider subscriptions and cache policy.
- Active runtime traversal.
- Command dispatch integration.
- Production AppShell behavior.
- Hidden catalog expansion beyond AppShell, ReaderSurface, and InspectorPane l.

DF-031 Status Feedback Expansion makes the next catalog step explicit.

What is now true:

- Issues #220 through #225 moved into v6.0.0 as a focused status/feedback Block slice.
- The standard status/feedback block names now include InlineStatusBlock, InF lowStatusBlock, TransientOverlayBlock, ActivityStreamBlock, ShortcutCueBloc k, and ProgressIndicatorBlock.
- The public package barrel exposes the matching lower-camel definitions: inl ineStatusBlock, inFlowStatusBlock, transientOverlayBlock, activityStreamBlo ck, shortcutCueBlock, and progressIndicatorBlock, plus matching *SchemaAdap ter and *SchemaBlock exports.
- The new Blocks use the same DX-031/DX-034 public contract: validated block metadata, view data contracts, schema-bound adapters, command intents, story ids, deterministic render output, and stable lowering facts.
- Status and feedback block expansion is now explicit issue-backed v6 work through #220 through #225, not hidden DX-031 scope.
- YAML/JSON declaration remains a future adapter over the typed Blocks API, not a second runtime IR in this slice.

Retrospective

DX-031 is landed for the v6.0.0 release boundary. The shipped release floor is the standard block set: public block metadata, package manifests, adapter-first schema-bound block contracts, first-party AppShell, ReaderSurfa ce, InspectorPanel, InlineStatusBlock, InFlowStatusBlock, TransientOverlayBlo ck, ActivityStreamBlock, ShortcutCueBlock, and ProgressIndicatorBlock

definitions and schema-bound wrappers, deterministic rendered output for interactive, static, pipe, and accessible modes, block-tree rendering, and a DOGFOOD Blocks section that proves the standard set in product context.

DOGFOOD multi-mode proof is present for the release slice through the standard block render tests, DOGFOOD preview tests, lowering posture checks, and story declarations. Broader “full block availability” remains intentionally larger than this release issue: optional Zod adapters, schema-to-block compilers, modal stacks, notifications, auth blocks, workspace docking, and broad catalog expansion should land as explicit future issues with their own tests and Method evidence.

127

DX-031F Block Tree Rendering

docs/design/DX-031F-block-tree-rendering.md

```
title: DX-031F Block Tree Rendering
legend: DX
lane: design
```

Design checkpoint for explicitly rendering nested block declarations without turning every block into an implicit runtime manager

Legend:

- DX - Developer Experience

Depends on:

- DX-031 - Standard Bijou Blocks
- DX-034 - Declarative View Data Binding

Why This Cycle Exists

Bijou blocks now have three separate capabilities:

- metadata and contract declaration
- structural composition of nested block declarations
- local block rendering

Those capabilities are deliberately separate. AppShellComposition can inspect nested blocks without rendering them, and block.render() renders only that block's local input. That kept discovery safe, but DOGFOOD exposed the next missing seam: an AppShell preview that nests ReaderSurface and InspectorPanel needs a renderer that explicitly walks that block tree.

Without a named block-tree renderer, AppShell has two bad options:

- print nested block names and pretend that is rendering
- make block.render() recursively render arbitrary children as a hidden side effect

Both are wrong. Recursive rendering is useful, but it must be explicit.

Core Rule

Block tree rendering is explicit.

```
BlockDefinition.render(input)
  renders one block only

AppShellComposition
  inspects declarations only

renderBlockTree(blockRenderNode(...))
  recursively renders a declared block tree
```

This preserves the boundary:

- composition can inspect nested blocks without executing render
- local block rendering stays predictable
- nested previews can render real child output
- provider lifecycle and command dispatch remain out of scope

Vocabulary

Block Render Node

A branded runtime value that pairs a BlockDefinition with the input used when that block participates in an explicit render tree.

It is not a provider handle, subscription, cache entry, command dispatcher, or view lifecycle owner.

The node snapshots caller-provided plain data at construction time. Mutating the original slot/config objects after blockRenderNode() returns must not alter later render output. Renderable runtime values such as nested block nodes, block definitions, and surfaces remain identity values; the renderer must not strip their brands or cell data while normalizing inert records.

Block Tree Renderer

A pure renderer that starts from a BlockRenderNode or BlockDefinition, recursively renders nested block slot values, and passes rendered child output into parent slots.

The renderer may aggregate lowering facts. It must not resolve providers, subscribe, refresh, dispatch Commands, mutate frames, or traverse the active TUI view hierarchy.

Behavior

When renderBlockTree() receives a node:

1. Validate the target is a real BlockDefinition or branded BlockRenderNode.

2. Resolve the effective mode from node input, inherited parent mode, or render options.
3. Walk own data properties of the slot record without invoking accessors.
4. Render nested BlockRenderNode and BlockDefinition slot values.
5. Replace each nested slot declaration with that child's rendered output.
6. Call the current block's local render() with the resolved input.
7. Return frozen output metadata with current and child lowering facts.

The parent block still owns how it displays a slot. The tree renderer only makes sure the slot value is rendered child output rather than an unrendered declaration.

For visual modes, rendered child Surface values stay surfaces. Parent blocks may frame, clip, stack, or blit them, but they must not convert them to text and then recreate new surfaces because that loses cell-level styling and makes nested blocks look like printed lowering output. Text extraction remains appropriate for pipe and accessible lowering.

Non-Goals

- no provider subscriptions
- no provider refresh
- no active view hierarchy traversal
- no command dispatch
- no cache retention policy
- no schema binding changes
- no DOGFOOD-specific renderer branch
- no implicit recursive behavior inside ordinary block.render()

DOGFOOD Posture

DOGFOOD block previews should use renderBlockTree() when they show nested standard blocks.

That lets the Blocks section show:

- AppShell as a real shell surface
- ReaderSurface rendered inside AppShell content
- InspectorPanel rendered inside AppShell inspection
- pipe and accessible lowering that includes child block text and facts

DOGFOOD should not call composition introspection to render visuals. It should use composition for declaration inspection and block-tree rendering for rendered preview output.

The Blocks preview may also host non-shipping fixture blocks. Fixture blocks must be clearly labeled as DOGFOOD examples and may prove interaction and lowering behavior that is not part of the first-party shipping block catalog. They still need to follow the same rendering boundary: visible controls emit

inspectable Command intent records, state changes happen in the DOGFOOD model, and lower modes expose deterministic text or JSON summaries without provider handles, subscriptions, or runtime dispatch hidden in the block.

The preview surface is still product UI, not a debug dump. The overview page may index available blocks and stories, but selected block pages should keep the live example first, summarize lower-mode output without clipped nested mini-surfaces, and present concise human documentation instead of raw compact metadata strings, source-path dumps, or full lowering fact payloads.

The Blocks navigation is also a preview picker, not only a document outline. When focus moves onto a standard block row, the selected preview should move to that block so the side navigation, guide info pane, and live preview always describe the same block. This keeps the DOGFOOD Blocks page aligned with the Components gallery posture: browse a named item, see that item. The Block Preview parent row is a preview-group entry; opening it should land on the first concrete block preview rather than a prose inventory page.

Acceptance Criteria

- Nested block previews render child output, not only child block names.
- block.render() remains a local single-block render contract.
- AppShellComposition continues to inspect declarations without calling render.
- Render tree inputs are runtime-branded rather than loose { block, input } shapes.
- Render tree nodes snapshot plain slot/config records at construction time.
- Render tree node construction rejects cyclic plain slot/config records with deterministic contract errors instead of recursing until stack overflow.
- Accessor properties in slot records are not invoked during slot resolution.
- Child mode inherits from the parent unless a child explicitly sets its own mode.
- Nested visual child surfaces preserve cell styling when rendered through parent block surfaces.
- Oversized nested visual child surfaces are fitted to their parent section instead of overrunning the parent block layout.
- DOGFOOD selected block pages do not expose raw contract-summary or source-path debug text as the live documentation surface.
- DOGFOOD block rows in the side navigation select their corresponding preview when focused.
- DOGFOOD’s Block Preview group entry opens the first concrete standard block preview instead of a prose overview.
- DOGFOOD lower-mode previews summarize output and facts without rendering clipped nested mini-surfaces in the main preview page.
- DOGFOOD fixture block previews render as selected block pages with live controls, command-intent state transitions, and deterministic static, pipe, accessible, and JSON lowering summaries.
- Recursive render depth is bounded and fails deterministically.
- Lowering facts from nested blocks remain inspectable.

Retrospective

Landed as the explicit block-tree rendering slice. The renderer stays separate from block.render() and AppShellComposition, which keeps declaration inspection, local rendering, and recursive preview rendering as distinct contracts.

128

DX-032 Create TUI app skeleton renders consumer pages

docs/design/DX-032-create-tui-app-skeleton-renders-consumer-pages.md

Legend: DX - Developer Experience

Date

2026-04-26

Sponsor human

A Bijou app author using the batteries-included TUI skeleton to compose a real application from first-party or community blocks.

Sponsor agent

An agent scaffolding a Bijou app that must bind page content into a framed shell without dropping down to lower-level createFramedApp() wiring.

Hill

createTuiAppSkeleton() can host consumer-provided page content while keeping the stock header, footer, command palette, drawer, and quit-confirm shell behavior.

Location

packages/bijou-tui-app/src/index.ts

Description

The createTuiAppSkeleton function in @flyingrobots/bijou-tui-app is named and documented as a “batteries-included design-system shell starter”. However, its internal implementation behaves more like a hardcoded test harness for the framework’s own internal documentation (e.g., the DOGFOOD app).

The SkeletonTab interface only accepts an id and a title:

```
export interface SkeletonTab {
  readonly id: string;
  readonly title: string;
}
```

Because it does not accept a render function or a generic FramePage spec, the skeleton completely ignores any custom UI blocks a consumer tries to render within the tabs. The internal engine statically maps the tabs to pre-defined layout configurations (like the “Split” or “Drawer” pages) and throws away any external layout logic, resulting in empty or generic screens for consumers who try to use it as a real application starter.

Impact

- **Developer Experience (DX):** New users attempting to use @flyingrobots/bijou-tui-app to scaffold their own applications will encounter “empty apps” and assume their layout code is broken.
- **Architectural Confusion:** It forces consumers to abandon the package entirely and drop down to the lower-level createFramedApp in @flyingrobots/bijou-tui to regain control over their own rendering pipeline.

Recommended Fix

1. **Option A (Rename/Deprecate):** Rename the function to createTestHarnessSkelet on or clearly document that it is strictly for internal framework testing/dogfooding, not for general consumer use.
2. **Option B (Make it Generic):** Update the SkeletonTab interface to accept a re nder function or layout node generator, allowing consumers to inject their own Bijou components (Surface or LayoutNode) into the shell’s body region while still benefiting from the skeleton’s built-in header, footer, and modal orchestration.

Decision

Choose Option B.

SkeletonTab should accept:

- render(width, height, context) for single-pane content
- layout(context) for full frame layout topology

Tabs without either option keep the existing default showcase behavior. Tabs with consumer content are treated as custom pages and do not inherit the demo drawer or split layout by index.

Playback questions

1. Can a custom tab render a consumer Surface inside the stock shell body?
2. Can a custom tab provide a full FrameLayoutNode for splits or grids?
3. Does the default two-tab demo still render drawer and split pages?
4. Does the skeleton reject ambiguous tabs that define both render and layout?

Accessibility / assistive reading posture

No new accessibility surface is added in this slice. The important behavior is that consumer content enters the existing frame/layout/focus machinery instead of being bypassed or replaced by demo pages.

Localization / directionality posture

No new localization behavior. Consumer content remains owned by the consumer renderer or layout factory.

Agent inspectability / explainability posture

The public SkeletonTab contract makes page ownership inspectable from the app declaration. The skeleton shell no longer hides consumer intent behind index-based demo page selection.

Implementation outline

1. Extend SkeletonTab with optional render and layout hooks.
2. Reject tabs that define both hooks.
3. Treat hook-backed tabs as custom pages instead of default demo pages.
4. Add focused tests for custom render, custom layout, default behavior, and ambiguous tab rejection.

Tests

- packages/bijou-tui-app/src/index.test.ts

Retrospective

Closed by adding render/layout hooks to SkeletonTab and proving that custom tabs render inside the stock shell without losing existing default skeleton behavior.

129

DX-033 Remove showcase layout sludge

docs/design/DX-033-remove-showcase-layout-sludge.md

```
title: DX-033 Remove showcase layout sludge
legend: DX
lane: design
priority: medium
keywords:
  - examples
  - layout
  - scrolling
  - strings
  - debt
```

Framing

The old component showcase predates DOGFOOD's ComponentStory substrate and the v6 storybook workstation. It remains useful as a compact tri-mode example, but it should not keep carrying hand-rolled layout and typing debt now that the primary docs lane has moved elsewhere.

This cycle removes the concrete sludge called out in the v6 bad-code note: St artAppOptions no longer defaults to any, the showcase sidebar no longer does local scroll slicing in the view renderer, and the welcome panel drops its wi dth - 4 sizing shortcut. The full registry rewrite is intentionally not in scope; DOGFOOD and the storybook workstation are now the canonical teaching surfaces for new work.

Sponsored Users

- Builders copying examples who should not inherit any defaults or bespoke list clipping.
- Maintainers keeping the legacy showcase aligned with shared TUI primitives.
- Agents auditing v6 layout work and checking that old examples no longer contradict the runtime layout direction.

Hills

1. A builder reading startApp() types sees unknown as the default message payload instead of silent any.
2. A maintainer reading the showcase sidebar sees browsableListSurface() own the visible row window and scrollbar.
3. A release reviewer can verify the remaining showcase role is secondary to DOGFOOD/storybook instead of a second canonical docs product.

Playback Questions

- Does StartAppOptions default to unknown?
- Does the showcase sidebar render through the shared browsable-list surface?
- Is manual items.slice(scrollY, scrollY + visibleCount) gone from the app view renderer?
- Is the welcome panel no longer using the hardcoded width - 4 shortcut?
- Do docs explain that DOGFOOD/storybook are the primary teaching lane?
- Does the v6 lane point at the closed design cycle?

Requirements

- Change StartAppOptions<M = any> to StartAppOptions<M = unknown>.
- Add a type-level regression for that default.
- Replace showcase sidebar local slicing/string assembly with browsableListSurface().
- Preserve standalone tier marking in the sidebar.
- Move the backlog note into docs/design/.
- Update the v6 lane and DX legend.

Acceptance Criteria

- tests/cycles/DX-033/remove-showcase-layout-sludge.test.ts proves the cycle doc carries the modern playback sections.
- The test verifies the StartAppOptions default and type-level regression.
- The test verifies showcase sidebar rendering uses browsableListSurface().
- The test verifies the old local slice expression is gone.
- The test verifies the showcase docs point readers to DOGFOOD/storybook for primary docs work.

Implementation Outline

1. Tighten StartAppOptions to default its message type to unknown.

2. Add the expectTypeOf regression in packages/bijou-node/src/index.test.ts.
3. Render the showcase sidebar through browsableListSurface() with the existing tier marker preserved.
4. Update showcase and legend docs to reflect the secondary showcase posture.
5. Move the backlog item into docs/design/ and update v6 pointers.

Drift Check

The old registry still contains string-first examples. That is acceptable for this cycle because the showcase is now a secondary example surface and many entries intentionally demonstrate string-returning core components.

The key fix is ownership: scroll clipping moves back to the shared list/viewport primitive, and the primary story/doc tooling now lives in DOGFOOD and npm run storybook / npm run storybook:index.

Playback

- RED: DX-033 existed only as a v6 backlog note, StartAppOptions defaulted to any, and the showcase sidebar locally sliced visible rows.
- GREEN: StartAppOptions defaults to unknown and has a type-level test.
- The showcase sidebar renders through browsableListSurface() with a scrollbar instead of local slice/count footer logic.
- The welcome panel sizing no longer subtracts four cells by hand.
- The showcase README documents its legacy/secondary role relative to DOGFOOD and the text-first storybook workstation.

Retrospective

This closes the debt without pretending the legacy showcase should become the new story platform. The right direction is to keep old examples honest while new tooling composes around DOGFOOD stories and shared layout primitives.

130

DX-034 Declarative View Data Binding

docs/design/DX-034-declarative-view-data-binding.md

```
title: DX-034 Declarative View Data Binding
legend: DX
lane: design
```

Cycle for formalizing unidirectional, declarative, immutable data binding between business logic, active views, and block composition

Legend:

- DX - Developer Experience

Depends on:

- DX-031 - Standard Bijou Blocks
- RE-001 - Define Runtime Engine Architecture
- RE-003 - Retain Layout Trees And Layout Invalidation
- RE-005 - Buffer Commands And Effects Separately
- RE-006 - Formalize Component Layout And Interaction Contracts

Why This Cycle Exists

DX-031 gave Bijou a metadata-first block contract, but it did not answer the next developer-experience question:

How does a block or view declare the data it needs, and how does that data arrive when the block is part of the active view hierarchy?

Bijou already has adjacent runtime pieces:

- explicit application state and view stacks
- retained layouts for active views
- component layout and interaction contracts
- command and effect buffers
- an event bus for runtime messages

Those are necessary, but they are not a data-binding architecture.

This cycle exists to formalize the missing layer before AppShell, nested blocks, schema-bound blocks, or generated block packages harden the wrong API.

Core Rule

Data binding in Bijou is:

- unidirectional
- declarative
- immutable

Data flows from business logic and provider adapters into active views.

Views render immutable snapshots.

Views do not mutate business state, provider state, caches, stores, channels, or resources.

Views communicate user intent by emitting Commands. Business logic handles those Commands, updates state or provider inputs, and produces the next immutable snapshot. That is the loop.

```
business state / adapters
  -> providers
  -> immutable binding snapshots
  -> active view hierarchy
  -> render
  -> user input
  -> Commands expressing intent
  -> business logic
  -> next state / provider inputs
```

There is no reverse data mutation path from view to provider.

Boundary

This cycle is about data requirements and binding lifecycle.

It is not about:

- two-way binding
- mutable stores exposed to views
- hidden global provider registration
- letting views read process, filesystem, network, or database adapters directly
- making schemas own presentation
- replacing Commands with arbitrary event side effects
- forcing every component to become provider-aware

Sponsor Human

App builders composing shell, reader, inspector, form, and data blocks who need the view tree to bind to business data without manual prop plumbing or hidden mutation.

Sponsor Agent

Agents generating Bijou apps or block packages that need to inspect what data a view requires, which providers can satisfy it, and which Commands the view can emit.

Hill

A builder can declare an AppShell whose navigation, content, inspector, and status regions bind to provider-backed immutable data snapshots, while every user action leaves the view tree as an explicit Command intent handled by business logic.

Playback Questions

- Which data requirements does this active view tree declare?
- Which provider satisfies each requirement?
- What is the immutable snapshot version currently rendered?
- Is each binding ready, loading, stale, empty, or failed?
- Which Commands can this view emit?
- Which business reducer or adapter action handles each Command?
- What provider subscriptions are active only because this view is active?
- What is disposed or suspended when the view leaves the active hierarchy?

Vocabulary

Business Logic

The state machine, reducers, command handlers, domain services, and host adapters that own application truth.

Business logic may read or write external systems through ports and adapters. Views may not.

Provider

A runtime-backed source that can produce immutable snapshots for a declared data requirement.

A provider may be backed by:

- in-memory business state
- a channel or message stream
- a query cache
- a filesystem adapter

- a database adapter
- a GraphQL, REST, or MCP adapter
- a localization catalog
- a credential-status adapter that never exposes secret values

The backing mechanism is not visible to views. The view sees only a binding snapshot.

Data Requirement

A declarative statement made by a block, view, or component:

> I need this named resource or signal, with these parameters, in this slot.

Requirements are data, not callbacks. They are inspectable before binding.

Binding Scope

The provider registry visible to a subtree of the active view hierarchy.

Scopes let an app provide one navigation source at the shell level, override a selection source inside a modal, or bind a test provider for a story without touching global state.

DX-034B makes scopes explicit local values in the public contract. A Provider Scope declares which DataProvider metadata is available for which resource, but it does not subscribe, refresh, resolve nested scopes, or dispatch Commands.

Binding Snapshot

An immutable runtime value created by a provider for one requirement.

A snapshot has:

- provider id
- requirement id
- version
- status
- data when ready
- issues when failed
- freshness facts when stale
- semantic facts for tooling and lower modes

Command

A typed intent emitted by a view in response to user input.

Commands say what the user intended. They do not contain hidden provider mutations.

Architecture

1. Views Declare Requirements

Blocks and views should declare data requirements alongside metadata:

```
const readerSurfaceBlock = defineBlock({
  metadata: readerSurfaceMetadata,
  data: defineViewData({
    article: requiredResource(articleResource, {
      id: routeParam('articleId'),
    }),
    outline: optionalResource(articleOutlineResource, {
      id: routeParam('articleId'),
    }),
  }),
  commands: {
    selectHeading: commandIntent<{ headingId: string }>('reader.selectHeading'),
  },
  render: ({ data, command }) => readerSurface({
    article: data.require('article'),
    outline: data.get('outline'),
    onSelectHeading: headingId => command('selectHeading', { headingId }),
  }),
});
```

This sketch is not final API. The required properties are:

- data requirements are declared, not discovered from render-time reads
- requirements are named
- requirements are inspectable before rendering
- Commands are declared as intent outputs
- render receives immutable binding data, not provider objects

2. Active Views Determine Active Bindings

The runtime should resolve data only for views in the active view hierarchy.

When a view becomes active:

1. collect its data requirements
2. resolve each requirement against the nearest binding scope
3. request or subscribe to provider snapshots
4. create a binding frame
5. render the view from that immutable frame

When a view becomes inactive:

1. dispose or suspend bindings owned only by that view
2. preserve shared provider caches only when the provider owns that policy
3. remove binding facts from active runtime introspection

The view tree does not manually subscribe or unsubscribe.

3. Providers Normalize Every Backing Source

Providers hide transport details.

Whether data comes from an event, a message channel, a query, a file, or memory, the provider contract should normalize it into immutable snapshots:

```
interface BindingSnapshot<Data> {
  readonly providerId: string;
  readonly requirementId: string;
  readonly version: number;
  readonly status: 'ready' | 'loading' | 'empty' | 'stale' | 'error';
  readonly data?: DeepReadonly<Data>;
  readonly issues: readonly BindingIssue[];
  readonly facts: readonly BindingFact[];
}
```

Provider updates do not mutate mounted views. They produce a new snapshot version and trigger a runtime binding invalidation. The next render receives a new immutable binding frame.

DX-034A snapshots accept inert plain snapshot data: null, strings, numbers, booleans, arrays, and enumerable string-keyed plain objects. They reject mutable built-ins and executable values such as Map, Set, Date, typed arrays, functions, accessors, symbol-keyed properties, symbols, and bigint. Snapshot construction copies accepted data before freezing it, so provider-owned or business-owned source objects are not frozen in place.

4. Binding Frames Are Immutable

Render receives a binding frame, not mutable provider handles.

The frame should support explicit reads:

```
data.require('article');
data.get('outline');
data.status('comments');
data.issues('reviews');
```

The frame should not support writes.

Any attempt to mutate data from inside a view should be structurally impossible in the public API and rejected at runtime where objects cross a boundary.

DX-034A Binding Snapshot And Frame Primitives

DX-034A lands the first runtime-truth slice in @flyingrobots/bijou.

The public core now includes:

- DataRequirement
- ProviderId
- RequirementId
- BindingStatus
- BindingSnapshot
- BindingFrame
- BindingIssue
- BindingFact
- CommandIntent

- defineDataRequirement()
- bindingSnapshot()
- bindingFrame()
- commandIntent()

This slice proves the objects, not the full runtime lifecycle:

```
const article = defineDataRequirement({
  id: 'article',
  resource: 'docs.article',
});

const snapshot = bindingSnapshot({
  providerId: 'docs.articleProvider',
  requirementId: article.id,
  version: 1,
  status: 'ready',
  data: { title: 'DX-034' },
});

const frame = bindingFrame([snapshot]);

frame.require('article');
frame.get('article');
frame.status('article');
frame.issues('article');
```

The BindingFrame deliberately does not include provider scopes, subscriptions, active hierarchy traversal, schema adapters, command dispatch, or AppShell rendering. Those remain separate runtime layers.

DX-034B Provider Scope Contracts

DX-034B lands the explicit provider/scope contract layer in @flyingrobots/bijou.

The public core now includes:

- DataProvider
- DataProviderInput
- ProviderScope
- ProviderScopeEntry
- ProviderScopeOptions
- defineDataProvider()
- provide()
- providerScope()
- isDataProvider()
- ProviderResolution
- ProviderResolutionStatus
- resolveProviderRequirement()
- resolveProviderRequirements()
- BindingFrameAssembly
- bindingFrameFromSnapshots()
- ViewDataContract
- ViewDataRequirementEntry

- defineViewData()

This slice proves local provider availability and one-scope resolution, not hierarchical provider resolution or subscription lifecycle:

```
const article = defineDataRequirement({
  id: 'article',
  resource: 'docs.article',
});

const readerData = defineViewData({
  id: 'reader.data',
  requirements: [{ name: 'article', requirement: article }],
});

const articleProvider = defineDataProvider({
  id: 'docs.articleProvider',
  resource: article.resource,
});

const providers = providerScope([provide(articleProvider)], {
  id: 'docs.appShell',
});

providers.has(article.resource);
providers.get(article.resource);
providers.resources();

const resolution = resolveProviderRequirement(article, providers);
resolution.status;

const assembled = bindingFrameFromSnapshots({
  resolutions: [resolution],
  snapshots: [
    bindingSnapshot({
      providerId: 'docs.articleProvider',
      requirementId: 'article',
      version: 1,
      status: 'ready',
      data: { title: 'DX-034' },
    }),
  ],
});
assembled.frame.require('article');
```

ProviderScope rejects duplicate resources and duplicate provider ids inside a single scope. It also rejects loose provider-shaped objects, so importing a module cannot silently register a provider or bypass the constructor contract. ViewDataContract names the data a view needs before any provider binding occurs. Resolution results are frozen metadata. Missing required providers become deterministic issues; missing optional providers are inspectable but not errors. Frame assembly reports missing or provider-mismatched snapshots as deterministic issues while keeping provider reads outside the render frame.

DX-034C AppShell Composition Contract

DX-034C lands the first structural AppShell composition contract in @flyingrobots/bijou.

The public core now includes:

- AppShellComposition
- AppShellCompositionInput

- AppShellSlot
- AppShellSlotContent
- AppShellSlotId
- AppShellSlots
- defineAppShellComposition()
- isAppShellComposition()

This slice proves semantic shell slots and inspectability. It still does not render AppShell, resolve hierarchical provider scopes, subscribe, refresh, walk the active hierarchy, or dispatch commands:

```
const docsShell = defineAppShellComposition({
  id: 'docs.shell',
  providers,
  slots: {
    navigation: navigationBlock,
    content: readerSurfaceBlock,
    inspector: inspectorPanelBlock,
  },
});

docsShell.slotIds();
docsShell.slot('content');
docsShell.dataContracts();
docsShell.commandIntents();
docsShell.providerScope();
```

Slots are logical regions: navigation, content, inspector, status, and overla ys. Physical slots such as leftNav are rejected at construction time. Slot content must be a block returned by defineBlock() or nested arrays of such blocks. The composition exposes the nested blocks' declared view data contracts and command intents for tooling without adding provider handles or render-time mutation paths.

DX-034D Active Binding Lifecycle

DX-034D formalizes the active binding lifecycle before rendered AppShell or runtime provider subscriptions begin.

Core invariant:

```
Active hierarchy owns which bindings are active.
Providers publish snapshots.
Runtime assembles new frames.
Views render frames.
Commands leave views as intent.
Business logic owns change.
```

This slice is design-only. It defines lifecycle ownership and vocabulary; it does not implement provider subscriptions, active hierarchy traversal, command dispatch, rendered AppShell, schema binding, cache retention, or DOGFOOD integration.

Lifecycle Ownership

The active view hierarchy owns binding lifetime.

A data requirement becomes active when all of these are true:

1. a block or view declares the requirement through an inspectable data contract
2. that block or view is present in the active hierarchy
3. the runtime resolves the requirement against an explicit provider scope
4. the runtime creates binding ownership for that active view node

Views never choose their own binding state. Views do not subscribe, refresh, dispose, retain cache, or mutate provider state. The runtime owns lifecycle transitions and provider subscription policy.

Lifecycle Vocabulary

Use the smallest useful lifecycle vocabulary first:

- active: the requirement is owned by the active hierarchy; the runtime may hold a provider subscription; provider snapshots can invalidate the current frame.
- suspended: the requirement is not currently rendering, but runtime/provider policy may retain resumable state or cache. Views do not decide what is retained.
- disposed: the requirement ownership is released. The runtime no longer expects provider updates for that view binding.

Provider cache retention is provider/runtime policy, not view policy. DX-034D names lifecycle states but does not implement cache retention.

Provider Subscription Ownership

Provider subscriptions are runtime lifecycle. Binding frames are immutable render inputs.

Providers may subscribe to channels, messages, queries, files, databases, MCP, or in-memory business state internally, but render receives only immutable snapshots assembled into a BindingFrame. A view never receives provider handles, subscription handles, refresh methods, mutable stores, or dispatcher callbacks.

Provider Update Flow

Provider updates do not mutate rendered views or existing frames.

The lifecycle flow is:

```
provider publishes snapshot
  -> runtime records binding invalidation
  -> runtime assembles next BindingFrame
  -> active view renders the next frame
```

Invalidation belongs to runtime binding ownership, not to rendered cells. Static, pipe, and accessible modes must be able to observe lifecycle facts such as active, suspended, disposed, loading, stale, empty, error, and invalidated without parsing terminal output.

Command Flow

Commands leave views as intent records.

Views may emit declared command intents, but they do not dispatch business logic, call provider methods, or mutate provider inputs. Runtime command integration receives command intent records and hands them to business logic. Business logic handles Commands, updates state or provider inputs, and providers eventually publish new snapshots.

Active Lifecycle Questions

DX-034D answers:

- When does a requirement become active?
- Who owns the binding while a view is active?
- What happens when a view leaves the active hierarchy?
- What is suspended versus disposed?
- Who owns provider subscriptions?
- How does a provider update become a new BindingFrame?
- How do Commands leave views?
- Who dispatches Commands to business logic?
- What lifecycle facts are inspectable?
- What lower-mode facts survive static, pipe, and accessible output?

DX-034E Binding Lifecycle Primitives

DX-034E implements lifecycle as transition algebra, not as a runtime manager.

The primitives answer what state a binding is in, who owns it, what transition produced the next record, what invalidation happened, and what lifecycle facts tooling can inspect.

They do not subscribe, refresh, dispatch, traverse the active view tree, render AppShell, retain caches, or bind schemas.

The public vocabulary stays deliberately small:

- BindingLifecycleOwner: the active-view, block, AppShell, or runtime owner responsible for one binding lifetime.
- BindingLifecycleRecord: an immutable record for one owner and one requirement.
- BindingLifecycleState: active, suspended, and disposed.
- BindingLifecycleTransition: the immutable record of an activation, suspension, disposal, or invalidation transition.
- BindingInvalidation: an immutable fact that a provider update, scope change, manual runtime action, owner suspension, or owner disposal means a later runtime layer should assemble a new frame.

Allowed transitions are:

```
active -> suspended
active -> disposed
suspended -> active
```

```
suspended -> disposed
active -> active by invalidation only
suspended -> suspended by invalidation only
```

Disposed bindings cannot be activated, suspended, or invalidated. A disposed binding means ownership was released; later activation requires a new lifecycle record and a new ownership event.

Provider updates create invalidation records. They do not mutate existing frames, fetch data, call providers, or dispatch Commands:

```
provider publishes snapshot
  -> runtime creates BindingInvalidation
  -> runtime later assembles next BindingFrame
```

Lifecycle records are immutable facts, not managers. They expose no provider handles, subscription handles, refresh methods, command dispatch callbacks, mutable stores, or render hooks.

DX-034F Active Binding Collection

DX-034F collects declared active bindings before provider runtime integration.

The collection answers:

```
given declared data requirements and explicit lifecycle owners,
which owner/requirement pairs are active?
```

It uses BindingLifecycleOwner from DX-034E. It does not introduce another ownership vocabulary. An active binding entry binds:

- one explicit BindingLifecycleOwner
- one declared DataRequirement
- optional provider id metadata

Collections can produce active BindingLifecycleRecord values from those entries so the later runtime layer has stable ownership facts to consume:

```
const collection = activeBindingCollection([
  activeBindingEntry({
    owner: defineBindingLifecycleOwner({
      id: 'docs.shell',
      kind: 'app-shell',
    }),
    requirement: articleRequirement,
    providerId: 'docs.articleProvider',
  }),
]);

collection.entries();
collection.requirements();
collection.owners();
collection.lifecycleRecords();
```

DX-034F also allows declared ViewDataContract values to be collected without rendering blocks or views:

```
const collection = collectActiveBindings({
  contracts: [{
```

```
    owner,
    contract: readerSurfaceBlock.data,
    providerIds: [
      { requirementId: 'article', providerId: 'docs.articleProvider' },
    ],
  }],
});
```

Collection inspects declarations, not rendered output. It does not subscribe, refresh, dispatch, render, cache, resolve provider scopes, traverse the real TUI runtime tree, bind schemas, or integrate DOGFOOD. Provider ids are metadata only; provider handles are not exposed.

DX-034G Active View Binding Collection

DX-034G connects the pure active-binding collection contract to the existing runtime view-stack model in @flyingrobots/bijou-tui.

The runtime view stack determines which layers contribute active bindings:

- the top layer is active
- layers below remain active only while each layer above them has blocksBelow : false
- a layer with blocksBelow: true blocks lower layers from active binding ownership

Runtime binding collection still inspects declarations, not rendered output. Layers expose branded RuntimeViewBindingSource values, and collectRuntimeView Bindings() converts active sources into an ActiveBindingCollection.

Runtime binding sources normalize, copy, and freeze provider assignment metadata at construction so later caller mutation cannot change active binding collection results.

DX-034G does not subscribe, refresh, dispatch, render, cache, resolve provider handles, bind schemas, or integrate DOGFOOD. It only proves that the active view hierarchy can identify active owner/requirement pairs.

DX-034H Provider Update Invalidation Flow

DX-034H proves the provider-update half of the loop without implementing a provider manager.

bindingFrameUpdateFromSnapshots() takes:

- an ActiveBindingCollection
- an explicit ProviderScope
- immutable BindingSnapshot values
- optional current lifecycle records

It resolves the collection’s requirements against the scope, assembles the next immutable BindingFrame, and records provider updates as lifecycle invalidations. It never mutates the previous frame or lifecycle records. Snapshots whose provider id and version already appear in a lifecycle record’s provider-update invalidations do not create duplicate invalidations

or lifecycle churn. Explicit provider assignments carried by active binding entries are authoritative: a scope-resolved provider that conflicts with an explicit assignment becomes a deterministic provider.assignment-mismatch issue rather than data in the frame.

```
provider publishes snapshot
  -> binding lifecycle records receive invalidation facts
  -> runtime assembles next immutable BindingFrame
  -> views later render the new frame
```

DX-034H does not fetch data, subscribe, refresh providers, retain caches, walk the active hierarchy, render AppShell, or dispatch Commands.

DX-034I Command Intent Dispatch Proof

DX-034I proves the command half of the loop without putting business logic in views.

Views emit branded RuntimeCommandIntentEmission values from declared CommandIntent metadata. Runtime code maps those emissions through explicit RuntimeCommandIntentRoute values into the existing RuntimeCommandBuffer. Business logic still handles the resulting command later through the normal command-buffer path. Emissions copy and deeply freeze inert payload data before routing so caller mutation cannot change the command that eventually reaches business logic.

```
view emits command intent
  -> runtime maps intent to a command
  -> command buffer records the command
  -> business logic applies the command later
```

Command intents and emissions expose no provider handles, subscription handles, refresh methods, dispatch callbacks, mutable stores, or render hooks.

5. User Input Emits Commands

Views communicate user intent through Commands:

```
command('reader.selectHeading', { headingId });
command('article.refreshRequested', { articleId });
command('nav.openItem', { itemId });
```

Business logic receives those Commands through the existing runtime command path, updates state or provider parameters, and lets providers publish the next snapshot.

Commands are the only sanctioned path from views back toward business logic.

6. Binding Facts Lower Across Modes

Binding status is semantic UI truth.

Ready, loading, empty, stale, and error states must lower across interactive, static, pipe, and accessible modes without parsing rendered text.

Examples:

- loading article body becomes a loading fact and an accessible status line
- stale search results expose stale facts even if the rich view only shows a subtle marker
- failed credential-status provider exposes provider status but never secret values
- empty comments list lowers as an empty-state fact rather than disappearing

7. Provider Scopes Are Explicit

Provider scopes should be attached by business logic, app shells, tests, or stories:

```
const app = appShell({
  providers: providerScope([
    provide(articleProvider),
    provide(selectionProvider),
    provide(commandLogProvider),
  ]),
  slots: {
    navigation: navigationBlock(),
    content: readerSurfaceBlock(),
    inspector: inspectorPanelBlock(),
  },
});
```

No import should register a global provider as a side effect.

AppShell Implications

AppShell should not be only a layout helper.

It should be the first proof that:

- named block slots can contain nested provider-bound blocks
- shell-level provider scope can feed navigation, content, inspector, status, and overlay regions
- region bindings are semantic, not physical left/right assumptions
- region Commands bubble as user intent, not direct shell mutation
- active region changes can alter which bindings are active without leaking subscriptions

Conceptual AppShell shape:

```
const docsApp = appShell({
  providers: docsProviderScope,
  slots: {
    navigation: navigationListBlock({
      data: { items: docsNavResource },
      commands: { openItem: 'docs.openArticle' },
    }),
    content: readerSurfaceBlock({
      data: { article: activeArticleResource },
      commands: { selectHeading: 'docs.selectHeading' },
    }),
    inspector: inspectorPanelBlock({
      data: { selection: activeSelectionResource },
    }),
    status: statusBarBlock({
```

```
      data: { sync: docsSyncStatusResource },
    }),
  },
});
```

The final API may differ. The architectural point is that slots contain nested blocks that declare their own data requirements, and the active hierarchy resolves those requirements through explicit scopes.

Relationship To Existing Systems

DX-031 Blocks

DX-031 metadata remains useful for discovery, docs, package manifests, and story indexing.

DX-034 adds the missing runtime-authoring layer: declared data requirements, provider scopes, immutable binding frames, and command intent outputs.

Runtime Engine

RE-001 through RE-006 already separate state, view stack, retained layouts, input routing, commands, effects, and component interaction.

DX-034 should reuse that spine instead of inventing a second event model.

Event Bus

The event bus is a carrier for runtime messages and command results.

It is not the data-binding source of truth.

A provider may listen to a channel or event stream internally, but the view binding contract still sees immutable snapshots, not the bus.

Schema-Bound Blocks

Schema-bound blocks are narrower than provider-bound views.

Schemas validate the shape of data. Providers answer where data comes from, when it updates, what its lifecycle status is, and which active view owns the binding.

The two should compose:

```
provider snapshot
  -> schema adapter validates data
  -> immutable binding frame
  -> block render
```

Accessibility / Assistive Posture

Assistive modes need the binding state, not only the rendered cells.

Every provider-bound view should expose:

- loading state
- stale state
- empty state
- error state
- selected or active entity, when relevant
- available intent Commands, when relevant
- redaction facts for sensitive provider-backed surfaces

Localization / Directionality Posture

Providers should deliver data, not localized shell policy.

Localized text can be data when it comes from a catalog provider, but layout direction, fallback copy, and shell-owned labels remain host or i18n runtime responsibility.

Data requirements should avoid physical slot names such as leftNav or rightInspector; use logical names such as navigation, content, inspector, and status.

Agent Inspectability / Explainability Posture

Agents should be able to inspect:

- the active view hierarchy
- each view's data requirements
- provider resolution for each requirement
- current binding snapshot status and version
- Commands each view may emit
- binding invalidation causes
- teardown/suspension state for inactive views

Agents should not need to parse rendered terminal output to understand data flow.

Linked Invariants

- Runtime Truth Wins
- Schemas Live At Boundaries
- Commands Change State, Effects Do Not
- State Machine and View Stack Are Distinct
- Layout Owns Interaction Geometry
- Tests Are the Spec

Implementation Outline

1. Done: define the public architecture vocabulary:
   - DataRequirement
   - BindingSnapshot
   - BindingFrame
   - CommandIntent
2. Done: add behavioral tests proving the design remains unidirectional, declarative, and immutable at the primitive layer.
3. Done: add pure runtime primitives for immutable binding snapshots and binding frames.
4. Done: define provider and provider-scope contracts without global registration.
5. Done: add one-scope provider resolution.
6. Done: assemble immutable binding frames from resolved snapshots.
7. Done: declare view data contracts.
8. Done: integrate view data and command contracts with DX-031 block definitions.
9. Done: add a structural AppShell composition contract for semantic slots, explicit provider scopes, and nested block data/command introspection.
10. Done: formalize the active binding lifecycle before runtime subscriptions or rendered AppShell begin.
11. Done: add binding lifecycle primitives as immutable transition-algebra value objects.
12. Done: add active binding collection primitives for declared owner and requirement pairs.
13. Done: add active-view binding collection over the existing view-stack model.
14. Done: add invalidation flow from provider snapshot updates to new immutable binding frames.
15. Done: add Command intent dispatch proof through the runtime command buffer.
16. Done: prove rendered AppShell with provider-bound navigation, content, inspector, and status blocks.
17. Done: add DOGFOOD stories and captures for ready, loading, stale, empty, and error binding states through the standard block story and DOGFOOD preview surface.

Tests To Write First

- Behavioral tests proving public binding frames expose immutable data without provider handles.
- Behavioral tests proving binding snapshots are immutable and versioned.
- Behavioral tests proving command intents are metadata, not callbacks.
- Behavioral tests proving provider scopes are explicit local registries without hidden globals.

- Cycle tests proving the active hierarchy owns active binding lifetime, provider subscriptions belong to runtime lifecycle, and binding frames remain immutable render inputs.
- Behavioral tests proving lifecycle records are immutable transition-algebra facts with active, suspended, disposed, and invalidated states.
- Behavioral tests proving active binding collections bind declared requirements to explicit lifecycle owners without rendering, subscribing, or dispatching.
- Runtime tests proving active view-stack layers determine active bindings without rendering, subscribing, or dispatching.
- Runtime tests proving provider scopes resolve nearest-provider wins without hidden globals.
- Runtime tests proving active views create bindings and inactive views dispose or suspend them.
- Runtime tests proving provider updates create new binding frames instead of mutating the current frame.
- Runtime tests proving views can emit Commands but cannot mutate provider data.
- Runtime tests proving Commands flow to business logic and produce later snapshots through the normal update loop.
- Mode-lowering tests proving ready, loading, stale, empty, and error binding states survive in pipe and accessible output.
- DOGFOOD tests proving AppShell nests provider-bound blocks through named logical regions.

Acceptance Criteria

- Bijou has a documented data-binding architecture before expanding the block API further.
- Data flow is one-way from business logic/providers into immutable view snapshots.
- Views communicate user intent only through Commands.
- Providers are explicit, scoped, and free of import-time global registration.
- Active view hierarchy controls which data bindings are active.
- Active binding collections expose active owner/requirement pairs and lifecycle ownership records before runtime provider integration.
- Provider updates produce new immutable binding frames.
- Command intent emissions enter the runtime command buffer without embedding business logic callbacks in views.
- Binding status is visible to tooling and lower modes.
- Schema-bound blocks compose with provider-bound views but do not replace the provider lifecycle.
- AppShell is designed around nested provider-bound blocks in logical slots.

Risks / Unknowns

- The first API could accidentally expose provider handles to views. That would violate the core rule.
- The provider lifecycle could duplicate existing runtime command/effect buffering if it is not layered carefully.
- If provider scopes are too implicit, debugging active data flow will become difficult.
- If binding frames are only TypeScript shapes, they will not satisfy the repo’s runtime-truth standard.
- Schema binding, provider binding, and slot binding can blur together. The architecture should keep them separate:
  - schemas validate data shape
  - providers deliver immutable snapshots
  - slots compose views and blocks
  - Commands express user intent

Retrospective

DX-034A landed the primitive contract layer first. The important restraint is that BindingFrame is not a provider scope, subscription manager, active-view resolver, schema adapter, or AppShell slot model. It is only the immutable data frame views can read during render.

DX-034B then landed explicit provider/scope metadata. The restraint remains the same: ProviderScope says which providers are locally available, but it still does not perform nearest-scope resolution, subscribe to backing sources, invalidate views, or dispatch Commands.

DX-034C landed structural AppShell composition. It proves semantic slots and nested block introspection, but still does not render AppShell or invent binding lifecycle policy.

DX-034D formalized active binding lifecycle ownership before runtime code. The important restraint is that active hierarchy, provider subscriptions, invalidation, and command dispatch are runtime responsibilities; views still receive immutable frames and emit intent only.

DX-034E landed binding lifecycle primitives as transition algebra. The restraint is that lifecycle records are immutable facts, not managers: they do not subscribe, refresh, dispatch, traverse the active hierarchy, render AppShell, retain caches, or bind schemas.

DX-034F landed active binding collection primitives. The restraint is that collections inspect declarations and produce active lifecycle ownership records; they do not subscribe, refresh, dispatch, render, cache, resolve provider scopes, traverse the real TUI runtime tree, bind schemas, or integrate DOGFOOD.

DX-034G landed active view-stack binding collection in @flyingrobots/bijou-tui. The restraint is that the runtime stack only decides which declared

binding sources are active; it still does not render, subscribe, refresh, dispatch, resolve provider handles, or retain caches.

DX-034H landed provider-update invalidation and frame assembly as a pure contract helper. The restraint is that snapshot updates create new immutable frames and lifecycle invalidation facts; no provider manager or subscription loop exists yet.

DX-034I landed command intent emission and runtime command-buffer routing. The restraint is that command routes map intent records to command values; business logic still applies commands later through the existing runtime command buffer.

DX-034 is landed for the v6.0.0 release boundary. The shipped release floor is the explicit data-binding contract layer: immutable snapshots and frames, provider scopes, view data contracts, structural AppShell composition, active binding lifecycle facts, active runtime binding collection, provider-update frame assembly, command intent routing, and closeout proof that a provider-bound AppShell can render navigation, content, inspector, and status regions from immutable binding frames without exposing provider, subscription, refresh, or dispatch handles to rendered output.

Provider-bound AppShell proof is present in the DX-034 closeout cycle test. The proof intentionally stays pure: provider subscriptions and cache policy remain runtime/provider responsibilities, while the release contract proves the boundary that views consume immutable frames and emit command intents.

DOGFOOD binding-state proof is present through the standard block story states and Blocks preview lowering checks for ready, loading, stale, empty, and error states. Broader production AppShell behavior can now build on the landed contract without keeping issue #182 open.

131

DX-036 Responsive Table Layout and Variants

docs/design/DX-036-responsive-table-layout-and-variants.md

```
title: DX-036 Responsive table layout and variants
legend: DX
lane: design
priority: medium
keywords:
  - table
  - layout
  - responsive
  - lowering
  - developer-experience
```

Make table() behave more like builders expect from a table: it should fit the available width by default, wrap cell content intelligently, and expose named style variants without forcing users to choose a separate component.

Legend:

- DX - Developer Experience

Sponsor Human

App builders using table() for command output, reports, diagnostics, and docs snippets should get readable tabular output without hand-calculating column widths for every terminal size.

Sponsor Agent

Agents generating Bijou CLI output should be able to choose a table variant and pipe format declaratively, while trusting the component to fit the viewport, wrap cells, preserve data in machine modes, and remain inspectable through tests.

Hill

A builder can render dense comparison data with table() and get responsive, word-aware, aligned output by default, while choosing explicit visual variants and explicit pipe/data formats without switching components.

Playback Questions

- Does a default human-mode table() fit ctx.runtime.columns when content would otherwise exceed the terminal?
- Does layout: "intrinsic" preserve today's content-sized behavior?
- Do wrapped rows keep continuation lines aligned under the correct columns?
- Can callers choose box, ascii-grid, ruled, header-rule, plain, markdown, and definition as table variants?
- Does pipe mode still default to TSV?
- Can callers explicitly request CSV, Markdown, TSV, or ASCII grid pipe output?
- Are ANSI styles, OSC 8 links, emoji, CJK, and combining graphemes measured by visible terminal width rather than string length?
- Does accessible mode keep row/header/value semantics independent of the visual variant?

Accessibility / Assistive Posture

Visual variants must not change the accessible reading contract. Accessible mode should keep a row-oriented header/value narration that is stable even when human output wraps, uses rules, or serializes differently for pipes.

Localization / Directionality Posture

The first cycle is not a full bidirectional table layout project. It should keep the existing left-to-right table assumption, but all measurement and wrapping should remain grapheme-aware so localized strings, combining marks, CJK, and emoji do not break alignment.

Agent Inspectability / Explainability Posture

The layout engine should produce a shared fitted table model before rendering. Tests should be able to inspect or prove the resolved widths, wrapped lines, variant renderer output, and pipe serialization independently enough that an agent can explain why a column wrapped or shrank.

Linked Invariants

- Runtime Truth Wins
- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo
- Schemas Live At Boundaries

Current Truth

table() currently renders an intrinsic-width Unicode boxed table in human modes:

- columns without explicit width grow to the widest header or cell
- the renderer does not consult ctx.runtime.columns
- cells wrap only when a column has an explicit width
- wrapping is grapheme-aware but hard-wraps; it does not prefer word boundaries
- overflow: "truncate" clips text without adding an ellipsis
- pipe mode lowers to TSV
- accessible mode lowers to Row N: Header=value

tableSurface() has the same intrinsic sizing posture: it creates a surface large enough for the table instead of fitting a parent width.

Product Decision

This should remain a table() feature, not become a separate ruledTable() component.

Add table variants as one API axis, and keep machine/data serialization as a separate axis:

```
table({
  columns,
  rows,
  layout: 'auto',
  variant: 'ruled',
  pipeFormat: 'tsv',
});
```

Recommended defaults:

- layout: "auto" for human modes
- width: ctx.runtime.columns for string rendering when no explicit width is provided
- overflow: "wrap" for human modes
- wrap: "word" so wrapping prefers whitespace boundaries
- hard-wrap unbreakable tokens rather than losing data
- pipeFormat: "tsv" for pipe mode

Keep an escape hatch:

- layout: "intrinsic" preserves today's content-sized behavior
- explicit width / maxWidth can override the runtime width
- overflow: "truncate" and a future overflow: "ellipsis" can support dense compact output where data loss is acceptable

Layout Requirements

Build a shared fitted table model before rendering variants:

1. Measure headers and cells by visible terminal width, ignoring ANSI and OSC 8 escapes and respecting grapheme widths.
2. Compute preferred column widths from content.
3. If the preferred table fits the available width, render at natural width.
4. If it does not fit, shrink flexible columns toward minimum widths.
5. Prefer wrapping cell text at word boundaries.
6. Hard-wrap a long unbreakable token only when it cannot fit.
7. Preserve alignment for continuation lines by leaving earlier columns blank and padding every column by visible width.
8. Recompute from the current width whenever a live TUI rerenders after a terminal resize.

Column options should grow beyond today's fixed width:

```
type TableColumn = {
  header: string;
  width?: number;
  minWidth?: number;
  maxWidth?: number;
  weight?: number;
  align?: 'left' | 'right' | 'center';
};
```

For tableSurface(), the first practical step is explicit width / maxWidth support. Parent-surface auto-fit can arrive once parent layout code has a stable way to pass available width down to child surfaces.

Variants To Support

variant: "box"

The current Unicode boxed table. Keep this as the compatibility visual style, but route it through the fitted table model.

variant: "ascii-grid"

Portable boxed ASCII table:

```
+----------+---------+-------+
| Name     | State   | Score |
+----------+---------+-------+
| Alice    | active  | 95    |
| Bob      | pending | 72    |
+----------+---------+-------+
```

This should be available explicitly as a visual variant and as an explicit pipe format, but it should not replace TSV as the default pipe lowering.

variant: "ruled"

The desired report style: no outer border, no vertical dividers, heavy header rule, and light row separators.

```
Name       State     Score
━━━━━━━━━  ━━━━━━━━  ━━━━━
Alice      active    95
─────────  ────────  ─────
Bob        pending   72
```

This variant must support wrapped cells and continuation-line alignment from the first implementation.

variant: "header-rule"

Compact report style with only a header separator:

```
Name       State     Score
---------  --------  -----
Alice      active    95
Bob        pending   72
```

variant: "plain"

Borderless aligned columns:

```
Name       State     Score
Alice      active    95
Bob        pending   72
```

Use spaces for visual alignment, not tabs. Tabs belong to TSV serialization; space padding is the only reliable way to align ANSI-styled and wide-character terminal content.

variant: "markdown"

GitHub-flavored Markdown pipe table:

```
| Name  | State   | Score |
|-------|---------|------:|
| Alice | active  |    95 |
| Bob   | pending |    72 |
```

This is useful both as a human/static variant and as pipeFormat: "markdown" when the destination is a README, issue, PR, or docs file. Escape |, backslashes, and multiline cells deliberately.

variant: "definition"

Two-column field/value output:

```
Field          Value
━━━━━━━━━━━━━  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
PR             https://github.com/flyingrobots/bijou/pull/120
State          MERGED
Merged at      2026-05-31T21:30:02Z
```

This can be implemented as a table variant with inferred Field and Value columns for two-column rows.

variant: "expanded"

psql-style record inspection:

```
-[ RECORD 1 ]-------------------------
name   | Alice
state  | active
score  | 95
-[ RECORD 2 ]-------------------------
name   | Bob
state  | pending
score  | 72
```

This is valuable for narrow terminals and many-column records, but it changes the reading mode from row comparison to record inspection. It can follow after the fitted table model and primary variants are stable.

Formats To Support

Keep serializations separate from visual variants:

- pipeFormat: "tsv": default lowest lowering for machine use
- pipeFormat: "csv": RFC-style escaped CSV for spreadsheet and shell export
- pipeFormat: "markdown": docs-friendly pipe table
- pipeFormat: "ascii-grid": explicit human-readable pipe output

TSV should remain the default pipe lowering because pipe usually means tools, scripts, and data transfer. ASCII grid and Markdown are useful, but they are human presentation formats and should be opt-in.

CSV must quote fields containing comma, quote, CR, LF, or other ambiguous content according to a documented policy. TSV also needs a deliberate escaping or preservation policy for tabs and newlines instead of accidental raw joins.

Implementation Outline

1. Extract or replace the current table-private width/wrap helpers with a shared fitted table model that can serve all string variants.
2. Extend TableColumn with optional minWidth, maxWidth, weight, and align.
3. Add layout, width / maxWidth, variant, pipeFormat, and word-wrapping options to TableOptions.
4. Make human modes default to width-aware layout: "auto" and use ctx.runtime.columns when no width is provided.
5. Preserve today's content-sized rendering behind layout: "intrinsic".
6. Render box, ascii-grid, ruled, header-rule, plain, markdown, and definition from the fitted model.

7. Keep TSV as the default pipe lowering, then add explicit CSV, Markdown, and ASCII grid pipe formats with escaping tests.
8. Bring tableSurface() to parity for explicit width / maxWidth once the string model is stable.
9. Update DOGFOOD or design-system docs with examples for the human-facing variants.

Tests To Write First

- A regression proving current intrinsic tables can exceed narrow runtime width, then the new default fits that width.
- A compatibility test proving layout: "intrinsic" retains old content-sized output.
- Fitted-width tests for fixed, min, max, weighted, and aligned columns.
- Word-boundary wrapping and hard-wrap fallback tests.
- Continuation-line alignment tests for multiline rows.
- ANSI, OSC 8, emoji, CJK, and combining-grapheme alignment tests.
- Snapshot-style tests for box, ascii-grid, ruled, header-rule, plain, markdown, and definition.
- Pipe-format tests for TSV default, CSV escaping, Markdown escaping, and ASCII grid opt-in lowering.
- Accessible-mode tests proving visual variants do not change semantic row/header/value reading order.

Explicit Non-Goals

- Do not add ruledTable() as a separate public component.
- Do not support Org-mode / Emacs table syntax in the first pass.
- Do not make ASCII grid the default pipe lowering.
- Do not silently truncate data by default.
- Do not redesign navigableTable() before the passive table() model is stable.

Acceptance Bar

- table() defaults to width-aware layout in human modes.
- layout: "intrinsic" preserves old content-sized behavior.
- Fixed, minimum, maximum, weighted, and aligned columns are covered by tests.
- Word-boundary wrapping uses shared text wrapping behavior where possible.
- ANSI, OSC 8 links, emoji, CJK, and combining graphemes stay aligned.
- Wrapped rows keep continuation lines under the correct columns.
- box, ascii-grid, ruled, header-rule, plain, markdown, and definition variants have snapshot-style tests.

- pipeFormat: ”tsv”, ”csv”, ”markdown”, and ”ascii-grid” are covered with escaping tests.
- accessible mode preserves semantic row/header/value reading order.
- DOGFOOD or design-system docs show at least box, ruled, header-rule, plain, markdown, and definition examples.
- Follow-on work is filed for expanded if it is not implemented in the first cycle.

Retrospective

Closed by adding a shared fitted string-table model behind table().

Shipped behavior:

- human-mode table() now defaults to width-aware layout: ”auto”
- layout: ”intrinsic” preserves the old content-sized behavior
- constrained cells wrap at word boundaries by default and hard-wrap only when needed
- TableColumn supports minWidth, maxWidth, weight, and align
- visual variants now include box, ascii-grid, ruled, header-rule, plain, markdown, definition, and expanded
- pipe still defaults to TSV, with explicit CSV, Markdown, and ASCII grid formats
- accessible output remains row/header/value oriented instead of inheriting a visual variant
- DOGFOOD’s table() / navigableTableSurface() story now demonstrates every visual table variant and every explicit pipe serialization

Deferred debt:

- tableSurface() still uses its older intrinsic surface sizing path. Follow-up work is tracked in DX-037 so surface width negotiation can be designed deliberately instead of copied from the string renderer.

132

DX-038 - BlockLab Rename And Migration

docs/design/DX-038-blocklab-rename-and-migration.md

```
title: "DX-038 - BlockLab Rename And Migration"
legend: DX
lane: design
issue: https://github.com/flyingrobots/bijou/issues/271
```

Linked Issue

- https://github.com/flyingrobots/bijou/issues/271

Sponsored Human

A Bijou maintainer wants the story and component workbench to have a project-owned name that does not imply affiliation with the browser Storybook product, while keeping the existing workflow understandable for users who already know npm run storybook.

Sponsored Agent

An agent needs stable names for scripts, Blocks, command intents, surface ids, and docs links. It should not have to infer whether "storybook" means Bijou's terminal workbench, the external product, or a generic story-driven workflow.

Hill

Rename the current Storybook-labeled workstation to **BlockLab** across human visible surfaces and future-facing contracts, while preserving a short compatibility path for existing commands.

This is product hygiene, not a legal conclusion. The current name creates avoidable ambiguity because Storybook is an established external product. Bijou can keep story-driven component development while naming its own terminal-native surface differently.

Current Truth

The current repo exposes a story-driven workstation through:

- npm run storybook
- npm run storybook:index
- npm run dogfood:storybook
- examples/docs/storybook.ts
- examples/docs/storybook-app.ts
- examples/docs/storybook-workstation.ts
- visible labels such as Bijou Storybook, Storybook, and Storybook Workstatio n
- DOGFOOD Block and registry names such as StorybookWorkbenchBlock
- docs that describe a “Storybook-style” workbench

The product shape is already useful: it has a story catalog, variants, profile presets, structured docs fields, matrix capture, AppFrame integration, and a text-first index. The naming layer is the weak part.

Name Decision

Use **BlockLab** as the product name.

Proposed vocabulary:

Current	Proposed
Bijou Storybook	Bijou BlockLab
Storybook Workstation	BlockLab Workstation
StorybookWorkbenchBlock	BlockLabWorkbenchBlock
storybook.workbench	blocklab.workbench
storybook.selectStory	blocklab.selectStory
npm run storybook	npm run blocklab
npm run storybook:index	npm run blocklab:index

Compatibility aliases should remain for one migration window:

```
npm run storybook
npm run storybook:index
npm run dogfood:storybook
```

Those aliases should call the BlockLab entrypoints and may print a concise deprecation note if the runtime can do so without polluting deterministic index output.

UI Mockup

Wide terminal:

```
Bijou BlockLab | notification-system | Live stack | Rich
+ catalog -------------------++ preview ---------------------------++ lab
↪  ----------------------+
```

```
| 72 stories                  || notification-system                || Selection
↪      |
| > notification-system       || + Live stack --------------------+ ||
↪  story=notification-system  |
|   framed-shell              || | release dashboard              | || variant=live-stack
↪      |
|   command-palette           || | Canaries stable in eu-west     | || profile=interactive
↪      |
|   table-responsive          || +------------------------------+ ||
↪       |
|                             || Docs                               || Required modes
↪      |
| Families                    || Use when                           || interactive: ready
↪      |
| Feedback overlays           || - surface owns transient state     || static: ready
↪       |
| Navigation                  || - lower modes preserve message     || pipe: ready
↪       |
| Blocks                      ||                                    || accessible: ready
↪      |
+-----------------------------++-----------------------------------++-----------------------┘
↪  -----+
q quit | / search | g globals | c controls | a actions | t tests | ,/. variant | 1-4 profile
```

Narrow terminal:

```
Bijou BlockLab
notification-system

+ Live stack --------------------+
| release dashboard              |
| Canaries stable in eu-west     |
+--------------------------------+

Profile: Rich / interactive / 60 cols
Variant: live-stack
Modes: interactive static pipe accessible

Tabs: preview docs controls tests actions
```

Lower Modes

Pipe mode should remain deterministic and artifact-friendly:

```
blocklab index
stories=72
families=31
variants=144
requiredModes=interactive,static,pipe,accessible

family Feedback overlays and history
- notification-system | package=bijou-tui | variants=live-stack,history-review,framed-routing
```

Accessible mode should describe purpose and state instead of chrome:

```
BlockLab workbench. Story notification-system. Variant Live stack.
Profile Rich. Required modes are all ready. Source examples/notifications/main.ts.
Preview summary: release dashboard, canaries stable in eu-west.
```

Accessibility / Assistive Posture

The rename must not make the workbench more visual-only. Every renamed surface keeps existing lower-mode facts:

- story id
- story title
- package
- source path
- selected variant
- selected profile
- required mode coverage
- capture matrix status

Future BlockLab labels should be localizable visible text. Machine tokens such as command ids and event discriminants can use blocklab.* but should not be translated.

Localization / Directionality Posture

New visible strings must enter examples/docs/i18n/source/dogfood-strings.csv with en, fr, es, and de rows. The rename should update generated catalog files in the same PR, and npm run dogfood:i18n:complete should be run against the branch.

Directionality does not change for this rename. The UI must continue to route localized copy through DOGFOOD's localization port rather than hard-coded view helpers.

Agent Inspectability / Explainability Posture

Agents should be able to tell that BlockLab is the canonical surface without scraping UI strings. The rename should update:

- Block metadata names and tags
- surface inventory ids and descriptions
- command intents
- story matrix report titles
- deterministic index title
- docs references

The compatibility aliases should be treated as command-level affordances, not as canonical product names.

Linked Invariants

- GitHub Issues are the live tracker; this design is linked to #271.
- DOGFOOD localization completeness gates protect new visible strings.
- Blocks own product semantics; components remain leaf rendering vocabulary.
- Text-first outputs stay deterministic for CI, agents, and fixtures.

Migration Plan

1. Introduce BlockLab docs language and visible product labels.
2. Add blocklab and blocklab:index scripts.
3. Keep storybook, storybook:index, and dogfood:storybook as compatibility aliases for one migration window.
4. Rename user-visible AppFrame page title and settings label.
5. Rename DOGFOOD Block contracts and registry entries from StorybookWorkbench Block to BlockLabWorkbenchBlock.
6. Rename command intents and surface ids from storybook.* to blocklab.*.
7. Update localization rows and generated catalogs.
8. Update docs, tests, and teardown command tables.

Tests To Write First

- A focused rename test that expects the default title to be Bijou BlockLab.
- A framed AppFrame test that expects the page title BlockLab.
- A DOGFOOD settings test that expects BlockLab Workstation.
- A Block metadata test that expects BlockLabWorkbenchBlock.
- A script-registration test proving blocklab and blocklab:index exist and legacy scripts still resolve.
- An i18n completeness run proving every changed visible string has supported locale rows.

Non-Goals

- No direct Storybook compatibility layer.
- No browser runtime.
- No package rename outside the workbench surface.
- No claim about trademark law.
- No removal of compatibility scripts in the first rename PR.

Retrospective Target

The rename is successful when humans see BlockLab, agents see blocklab.* contracts, and legacy storybook commands still work long enough for users to move without friction.

133

DX-039 - BlockLab Product Parity

docs/design/DX-039-blocklab-product-parity.md

```
title: "DX-039 - BlockLab Product Parity"
legend: DX
lane: design
issue: https://github.com/flyingrobots/bijou/issues/272
```

Linked Issue

- https://github.com/flyingrobots/bijou/issues/272

Sponsored Human

A component author wants BlockLab to feel like a real component development product: searchable, controllable, inspectable, test-aware, and useful for building the next Bijou Block without dropping into the full DOGFOOD docs app.

Sponsored Agent

An agent needs one stable story surface that can produce docs, fixtures, lower-mode captures, interaction playback, visual evidence, source links, and machine-readable reports without scraping terminal frames.

Hill

Evolve BlockLab from “story catalog plus preview” into a terminal-native component and Block laboratory that borrows the strongest workflows from the current Storybook product without cloning its browser UI.

Source Scan

Official Storybook documentation reviewed on 2026-06-03:

- Args: https://storybook.js.org/docs/writing-stories/args
- Controls: https://storybook.js.org/docs/essentials/controls
- Toolbars and globals: https://storybook.js.org/docs/essentials/toolbars-and-globals
- Viewport: https://storybook.js.org/docs/essentials/viewport
- Decorators: https://storybook.js.org/docs/writing-stories/decorators
- Docs and Autodocs: https://storybook.js.org/docs/writing-docs/autodocs
- MDX and doc blocks: https://storybook.js.org/docs/writing-docs/mdx
- Interaction testing: https://storybook.js.org/docs/writing-tests/interaction-testing
- Accessibility testing: https://storybook.js.org/docs/writing-tests/accessibility-testing
- Visual testing: https://storybook.js.org/docs/writing-tests/visual-testing
- Addon types: https://storybook.js.org/docs/addons/addon-types
- Storybook composition: https://storybook.js.org/docs/sharing/storybook-composition

The product lesson is not “make a browser Storybook clone.” The lesson is that a mature story workbench separates story data, controls, global context, preview, generated docs, addon panels, and test artifacts while keeping them linked by stable story ids.

Current Bijou Truth

BlockLab’s current pre-rename substrate already has:

- ComponentStory records with id, family, title, package, docs, profile presets, variants, source, and tags
- canonical profile presets for interactive, static, pipe, and accessible
- renderDogfoodStorybookIndex() for deterministic text output
- captureDogfoodStorybookMatrix() for profile x variant capture
- a three-pane interactive TUI: catalog, preview, and testing
- AppFrame integration and a surface Block contract

The missing layer is product workflow.

Gap Table

Storybook-like product concept	Current BlockLab state	Needed BlockLab feature
args/controls	variants have static initialState	serializable story args plus control schema

Storybook-like product concept	Current BlockLab state	Needed BlockLab feature
globals/toolbars	profile keys select mode/width	global toolbar for mode, width, locale, direction, theme, density, reduced motion
decorators	stories manually construct context	story harness/decorator chain for frame, providers, mock IO, and data bindings
docs blocks/autodocs	storyDocsMarkdown() renders fixed sections	doc block model composing summary, source, controls, lowerings, captures, related stories
addon panels	testing pane only	addon-like panels for actions, a11y, visual diffs, performance, bindings, events
interaction testing	smoke scripts live elsewhere	story-local play steps and playback reports
accessibility testing	lower-mode prose is documented	a11y fact panel and required accessible capture assertions
visual testing	matrix captures text frames	named visual artifacts, diffs, and update workflow
viewport	profile width presets	viewport presets plus arbitrary terminal size profiles
composition	one DOGFOOD story catalog	composed story catalogs from packages, examples, plugins, and MCP exports

UI Mockup

Wide terminal with product-grade panels:

```
Bijou BlockLab                                    story: notification-system
+ catalog -------------------++ preview ----------------------------++ controls
↪  ----------------+
| Search: notification        || + Live stack --------------------+ || Args
↪    |
|                             || | release dashboard              | || severity: success
↪    |
| Feedback overlays           || | Canaries stable in eu-west     | || count: 2
↪    |
| > notification-system       || +--------------------------------+ || compact: off
↪   |
|   toast                     ||                                    ||
↪    |
|   modal                     || Docs | Source | Actions | A11y      || Globals
↪     |
|                             || Use when                           || mode: interactive
↪   |
| Blocks                      || - status must stay observable      || locale: en
↪   |
|   app-shell                 || - lower modes need durable text    || width: 60
↪    |
|                             ||                                    || direction: ltr
↪   |
+-----------------------------++------------------------------------++-----------------------┘
↪  ---+
tabs: preview docs source actions a11y visual perf bindings
keys: / search | c controls | g globals | p play | v visual | a a11y | x export
```

Interaction playback panel:

```
+ interactions ------------------------------------------------------------------+
| notification-system / live-stack                                                |
|                                                                                 |
| 1. mount story                                          PASS  4ms              |
| 2. press n to add notification                         PASS  2ms               |
```

```
| 3. assert stack announces newest item                    PASS  1ms            |
| 4. switch to accessible profile                          PASS  1ms            |
|                                                                               |
| artifact: artifacts/blocklab/notification-system/live-stack/playback.txt      |
+-------------------------------------------------------------------------------+
```

Visual artifact panel:

```
+ visual ------------------------------------------------------------------------+
| profile      variant          baseline       current        result            |
| interactive  live-stack       60x18          60x18          unchanged         |
| static       live-stack       60x18          60x18          changed +2 lines  |
| pipe         framed-routing   52x18          52x18          unchanged         |
| accessible   history-review   52x18          52x18          unchanged         |
+-------------------------------------------------------------------------------+
```

Lower Modes

Pipe mode should expose stable machine-readable sections:

```
blocklab story notification-system
variant=live-stack
profile=interactive
globals.mode=interactive
globals.locale=en
globals.width=60
args.severity=success
args.count=2

[preview]
release dashboard
Canaries stable in eu-west

[tests]
interaction=pass
a11y=pass
visual=changed +2 lines
```

Accessible mode should prioritize the selected story and active findings:

```
BlockLab story notification-system. Variant Live stack.
Controls: severity success, count two, compact off.
Globals: interactive mode, English locale, left to right, sixty columns.
Accessibility report passed. Visual report changed by two lines in static mode.
```

Protocol Sketch

ComponentStory can grow without breaking existing records by adding optional fields:

```
interface BlockLabControl {
  readonly id: string;
  readonly label: string;
  readonly kind: 'boolean' | 'number' | 'text' | 'select' | 'color' | 'json';
  readonly defaultValue: unknown;
  readonly options?: readonly unknown[];
}

interface BlockLabGlobals {
  readonly mode?: OutputMode;
  readonly width?: number;
  readonly locale?: string;
```

```
  readonly direction?: 'ltr' | 'rtl';
  readonly theme?: string;
  readonly density?: string;
  readonly reducedMotion?: boolean;
}

interface BlockLabPanelReport {
  readonly panelId: string;
  readonly status: 'pass' | 'warn' | 'fail' | 'info';
  readonly facts: readonly ExplanationFact[];
  readonly output: string;
}
```

The first implementation should keep these fields optional and derive defaults from the current profile presets.

Feature Slices

1. Args/Controls

Add serializable controls to stories and a control pane in BlockLab.

Proof:

- controls render in TUI and lower modes
- control changes feed story render input deterministically
- CLI output includes selected args

2. Globals/Toolbars

Add global context for mode, width, locale, direction, theme, density, and reduced motion.

Proof:

- global state affects preview context
- lower modes report globals
- locale/direction changes route through DOGFOOD localization ports

3. Decorators/Harnesses

Add a story harness chain for common setup: AppFrame, providers, mock IO, binding snapshots, theme, and viewport.

Proof:

- harnesses compose without mutating base context
- a story can declare provider data without bespoke preview code

4. Docs Blocks

Replace fixed markdown assembly with docs block composition.

Proof:

- docs view can show source, controls, lowerings, story matrix, and related stories as separate inspectable blocks
- pipe mode can emit those blocks in deterministic order

5. Addon-Like Panels

Introduce panel slots for actions/events, accessibility facts, visual diffs, performance facts, binding state, and source.

Proof:

- each panel has a stable id and lower-mode report
- disabled panels do not affect preview rendering

6. Interaction Playback

Let stories declare deterministic play steps that can run in BlockLab, CI, and text-first output.

Proof:

- RED/GREEN tests for one story-local playback
- playback report includes steps, status, elapsed time, and artifacts

7. Composition

Allow BlockLab to import multiple story catalogs.

Proof:

- composed catalogs preserve package/source facts
- duplicate story ids fail loudly
- CLI can filter by package, family, tag, and source

Accessibility / Assistive Posture

BlockLab parity must improve lower-mode truth, not just add visual panels. Every product feature needs accessible and pipe output:

- controls list active values
- globals list current context
- panels list status and findings
- interactions list steps and outcomes
- visual reports describe line/cell differences
- docs blocks preserve heading order

Localization / Directionality Posture

Controls and globals are visible UI. Labels, panel names, and status text must go through DOGFOOD localization when they appear in DOGFOOD-owned surfaces. Story ids, arg ids, and artifact paths remain machine tokens.

Directionality belongs in globals. RTL preview should be a first-class context state, not a separate story variant unless the variant is semantically different.

Agent Inspectability / Explainability Posture

The text-first command should eventually emit:

- blocklab index
- blocklab story <id>
- blocklab controls <id>
- blocklab test <id>
- blocklab artifacts <id>

Each command should produce stable sections that agents can parse without screen scraping. JSON output can be added later, but plain text remains the first evidence surface because it is easier to review in PRs.

Tests To Write First

- Protocol tests for optional controls and globals.
- Workbench model tests proving controls, globals, panels, and docs blocks are discoverable without rendering.
- TUI tests proving controls and globals change preview output.
- Lower-mode tests proving pipe and accessible output include args/controls and panel statuses.
- Interaction playback tests using fake clocks and deterministic input.
- Visual artifact tests using fixed-size terminal captures.

Non-Goals

- No browser Storybook runtime.
- No direct addon API compatibility promise.
- No screenshots-only acceptance criteria.
- No story catalog fork separate from DOGFOOD story truth.
- No hidden global registry.

Retrospective Target

BlockLab is product-grade when a maintainer can open one terminal workbench, adjust story inputs, switch global context, read generated docs, run playback, inspect a11y/visual/performance findings, and export deterministic evidence for CI or agents.

134

DX-040 V7 Post-Merge Review Regression Fixes

docs/design/DX-040-v7-review-regression-fixes.md

```
title: DX-040 V7 post-merge review regression fixes
legend: DX
lane: bad-code
priority: medium
issue: https://github.com/flyingrobots/bijou/issues/283
keywords:
  - dogfood
  - release-title
  - table-surface
  - wrapping
  - review-follow-up
```

Framing

PR #282 merged the V7 Product Truth closeout. Codex review comments arrived after the admin merge and identified two P2 regressions that should be fixed before v7 is cut:

Source	File	Problem
PR #282 thread	examples/docs/release-title.ts	Interactive release-title output can exceed the caller's requested width for widths from 42 through 59 columns.
PR #282 thread	packages/bijou/src/core/components/table-v3.ts	Fitted tableSurface() string cells hard-wrap at column boundaries instead of honoring default word wrapping.

This follow-up keeps the scope intentionally narrow: fix exactly the two review comments, preserve the v7 closeout behavior, and leave broader table or title treatment redesigns for later cycles.

Sponsored Users

- A DOGFOOD reader on a medium-width terminal needs the release-title guide to fit the actual viewport instead of drawing a wider box.
- A TUI author using tableSurface() expects readable fitted text by default, matching the word-wrap posture already established by table().

- A review agent needs the reported regressions captured as deterministic tests so the branch proves the fix instead of relying on prose.

Hill

A maintainer can run deterministic tests that reproduce both post-merge review comments, then see the title renderer stay within requested widths and tableSurface() preserve word boundaries when fitting string cells.

Current Truth

- renderDogfoodReleaseTitleText({ mode: 'interactive', width: 42 }) selects the wide renderer even though wide output is 60 columns.
- The new fitted tableSurface() path wraps shrunk string cells through wrapSurfaceToWidth(), which chunks at hard cell boundaries and can split words such as surface and evidence.
- PR #282 is already merged, so the fix must be a new branch and PR.

Product Shape

Release Title Width

```
width 42:
+ BIJOU DOGFOOD ----+
| V7 Product Truth  |
| Blocks prove UX.  |
| lanes: table, io  |
| gate: closeout    |
+-------------------+
```

Every emitted line must be <= width.

Table Surface Word Wrapping

```
| Release proof     |
| canonical docs    |
| surface carries   |
| release title     |
```

Default fitted string cells should break at word boundaries when possible. Explicit hard wrapping and truncation remain separate behaviors.

Runtime And API Contract

- Release-title text rendering must honor the supplied width for interactive mode.
- tableSurface() must preserve existing overflow: 'truncate' behavior.

- Default fitted string cells should use word-aware wrapping.
- Surface cells that are already structured Surface values may continue to use the existing surface wrapping behavior because the original word boundaries are not recoverable from arbitrary surfaces.

Lower Modes

No lower-mode contract changes are intended. Existing static, pipe, and accessible release-title output should remain unchanged.

Tests To Write First

- Extend tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts with a width-42 regression that asserts every interactive line is at most 42 characters.
- Extend tests/cycles/DX-037/table-surface-width-parity.test.ts with a default word-wrap regression that asserts fitted output does not split surf ace or evidence across lines.

Acceptance Criteria

- Issue #283 is closed by the follow-up PR.
- The exact Codex review cases are represented by tests that fail before the fix and pass after it.
- npm run lint, focused tests, and relevant DOGFOOD checks pass.
- No broad redesign of release-title art, table(), or tableSurface() is included.

Risks And Guardrails

- Do not change the public tableSurface() signature unless the tests prove it is necessary.
- Do not make title rendering depend on terminal state outside the supplied width.
- Do not remove Storybook compatibility aliases while fixing BlockLab-adjacent follow-up behavior.

135

DX-041 Image-to-Glyph Side App

docs/design/DX-041-image-to-glyph-side-app.md

```
title: DX-041 Image-to-Glyph Side App
legend: DX
lane: asap
priority: medium
issue: https://github.com/flyingrobots/bijou/issues/305
keywords:
  - ascii
  - braille
  - file-picker
  - image
  - raster
  - side-app
```

Framing

Issue #305 adds a small, runnable side app that proves image browsing and image-to-glyph rendering as a real workflow instead of only as DOGFOOD title art. The app should use a sidebar file explorer, render the selected image in a main preview pane, let the user hot-swap between Braille and ASCII output, and expose enough tuning controls to discuss how image-to-text filters behave.

The app stays intentionally small: it reuses filePickerSurface() for the filesystem browser and rasterToGlyphSurface() for glyph rendering. Codec support is dependency-free and scoped to formats the repo can test deterministically.

Sponsor Human

- A maintainer wants to point Bijou at local assets and quickly inspect how they look as terminal glyph art.
- A designer wants to compare Braille density against ASCII density without restarting the app.
- A contributor wants a concrete example of filePickerSurface() driving a real side-by-side tool.

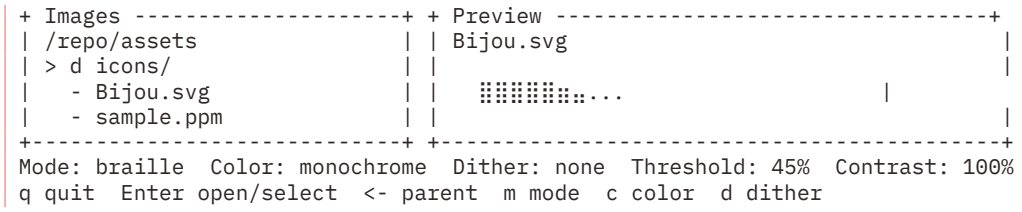
Sponsor Agent

- A review agent needs the file browser, selected path, renderer mode, decoded frame, and visible glyph output to be inspectable through tests.
- A future block-promotion agent needs evidence that the file picker can drive a richer app before promoting image browsing into a public block family.

Hill

A user can launch the image viewer, browse supported image files in a sidebar, select an image, and see the main pane render that image as Braille or ASCII glyphs. Pressing m swaps the render mode immediately. The preview pane also supports pan and zoom so a user can inspect image details without leaving the side app. The same pane lets the user preserve sampled image colors, adjust Braille threshold and contrast, and toggle ordered dithering.

Product Shape



Scope

- Add examples/image-viewer/main.ts as a runnable side app.
- Add examples/image-viewer/image-codecs.ts for dependency-free PNG and PPM decoding.
- Reuse the existing SVG rasterizer for SVG preview.
- Add npm run image-viewer.
- Expose color mode, threshold, contrast, and ordered-dither controls in the preview model.
- Add focused tests for codec behavior, render-mode switching, and app view output.

Non-Goals

- JPEG/WebP decoding.
- Public package-level image codec APIs.
- A generalized image-workbench product with cropping, palette quantization, error-diffusion dithering, or export.

- Promoting a new public image block in this slice.

Runtime Contract

The app starts in assets/ by default. If the user passes a directory, that directory becomes the file-picker root. If the user passes an image file, the file's parent directory becomes the root and the file starts selected.

Supported input formats:

- SVG: rasterized into a transparent RGBA frame at preview resolution.
- PNG: 8-bit, non-interlaced RGB/RGBA.
- PPM/PNM: P3 and P6 RGB Netpbm data.

Navigation contract:

- j / k: move file-picker focus.
- Enter: select focused image or open focused directory.
- Backspace: move the file picker to its parent directory.
- Arrow keys: pan the image viewport.
- + / =: zoom in.
- -: zoom out.
- 0: reset to fit.
- m / Tab: switch Braille and ASCII rendering.
- c: cycle color mode between monochrome, full color, and foreground-only color.
- d: toggle ordered dithering.
- [/]: lower or raise the Braille threshold.
- , / .: lower or raise contrast.

The file picker marks focus with > and the rendered image with *. The preview gutter reports zoom percentage, pan offset, color mode, dithering mode, threshold, and contrast. The initial image and every newly selected image start at 100% fit, centered horizontally and vertically.

Lower Modes

The app is interactive-first. Pipe/static smoke only needs to prove startup and first-frame rendering; the selected path and mode remain ordinary model state for tests.

Accessibility / Assistive Posture

The file picker remains textual. The preview pane names the selected file, render mode, format, dimensions, and tuning state so the image is not only communicated by glyph art.

Tests To Write First

- RED: PNG decoder cannot map synthetic RGB/RGBA PNGs into RGBA.
- RED: PPM decoder cannot map P3 and P6 RGB data into RGBA.
- RED: image viewer starts with an image selected from the root.
- RED: pressing m does not change Braille mode to ASCII mode.
- RED: pressing c does not preserve sampled light and dark image colors on rendered glyph cells.
- RED: threshold, contrast, and dither controls do not change visible tuning state.
- RED: rendered view does not contain the file picker and preview panes.

Validation

```
npx vitest run scripts/image-viewer.test.ts scripts/svg-raster.test.ts
↪  packages/bijou-tui/src/raster-glyph.test.ts
npm run typecheck:test
npm run lint
```

Follow-On Debt

- Add JPEG/WebP support behind an explicit codec adapter decision.
- Decide whether this app should become part of DOGFOOD or stay an example.
- Promote a first-class image/file-browser block only after the side-app shape proves stable.
- Add error-diffusion algorithms such as Floyd-Steinberg or Atkinson once the ordered-dither baseline has enough usage feedback.
- Add palette quantization and custom charset editing if the side app graduates into a broader image-to-text workbench.

136

DX-042 Shared UI Scene IR And Bijou Render Target

docs/design/DX-042-shared-ui-scene-ir-and-bijou-render-target.md

```
title: DX-042 Shared UI Scene IR And Bijou Render Target
legend: DX
lane: cool-ideas
priority: medium
github_issue: 302
status: proposed
keywords:
  - developer-experience
  - ir
  - geordi
  - wesley
  - surface
  - provenance
```

Legend: DX - Developer Experience

Linked Work

- User story: #302
- Roadmap candidate: Runtime Graph And Scene IR
- This document is a planning artifact only. It records the shared IR boundary for the #302 idea; it does not claim implementation of the IR, a compiler, or a renderer integration in this PR.

Decision Summary

Bijou Surface should be treated as a deterministic terminal render target for a portable UI scene IR, not as the portable IR itself.

The shared IR should sit above Bijou and Geordi:

```
GraphQL SDL / authored source
  -> Wesley semantic graph and domain law
    -> ui-scene-ir/1
      -> bijou-layout/1
        -> Bijou Surface
          -> terminal cells

      -> geordi-ir/1
```

```
        -> Geordi renderer
          -> pixels and render receipts
```

This keeps the final renderer substrates honest. Geordi owns pixel-oriented render proof. Bijou owns terminal cell rendering, input, focus, runtime interaction, and agent-legible surfaces. The shared IR owns portable scene structure, semantic identity, bindings, localization references, theme token references, actions, and source maps.

Sponsored Human

A builder wants to describe a UI scene once and render it through either Geordi or Bijou so that browser/native artifacts and terminal artifacts stay aligned without manually maintaining two different scene descriptions.

Sponsored Agent

An agent wants a source-mapped, inspectable scene contract so it can audit which localized strings, theme tokens, bindings, actions, and rendered cells came from the same authored scene.

Hill

A scene authored as a semantic UI contract can lower to both a Geordi render artifact and a Bijou terminal surface, with structural receipts proving that node identity, text keys, token references, bindings, and actions stayed in sync across both targets.

Core Rule

The portable IR describes scene intent.

Bijou Surface describes rendered terminal output.

Those are different layers:

```
ui-scene-ir/1
  describes: nodes, text keys, token refs, bindings, actions, source maps

bijou-layout/1
  describes: terminal layout, focus graph, keymap, viewport regions

Bijou Surface
  describes: final cell grid with glyphs, colors, modifiers, opacity, masks
```

Bijou may render a shared scene IR. Geordi may render the same shared scene IR. Neither system should pretend that terminal cells and pixels are the same final substrate.

Playback Questions

1. Which authored scene node produced this rendered cell or pixel region?
2. Which localization key produced this text?
3. Which theme token produced this foreground or background color?
4. Which action or key binding is attached to this node?
5. Which renderer-specific target profile lowered this scene?
6. Which target-specific claims are proven, and which are explicit nonclaims?
7. Can the Geordi and Bijou outputs prove structural parity without claiming pixel parity?
8. Can compile-time checks catch missing localization keys and raw color usage before runtime?

Boundary

This design is about a portable scene contract and renderer-specific lowering.

It is not about:

- replacing Bijou's runtime loop
- replacing Bijou Surface
- making terminal cells behave like pixels
- making Geordi own terminal input or focus
- making Wesley a renderer
- forcing every existing Bijou block to be authored in GraphQL immediately
- claiming pixel-perfect parity between terminal and browser render targets

Role Model

Wesley

Wesley owns semantic compilation and domain law.

For this design, Wesley can:

- parse GraphQL SDL or another schema-backed authoring source
- validate schema shape
- generate TypeScript and Rust types
- validate localization requirements
- validate style token requirements
- compute schema hashes
- produce or validate ui-scene-ir/1

Wesley should not render terminal cells or pixels.

Shared UI Scene IR

ui-scene-ir/1 owns portable scene structure.

It should carry enough information for both Bijou and Geordi to lower the same scene while preserving source identity.

It should know about:

- scene node IDs
- node kinds
- semantic roles
- layout intent
- text and localization references
- theme token references
- bindings
- actions
- source maps
- target profile requirements

It should not know about:

- terminal escape sequences
- renderer dirty-region internals
- final RGB cell caches
- subpixel antialiasing details
- host-specific input event loops

Geordi

Geordi owns pixel-oriented render artifacts and receipts.

For this design, Geordi lowers ui-scene-ir/1 to geordi-ir/1, then renders or verifies the pixel artifact under a declared target profile.

Geordi claims:

- explicit pixel geometry
- font and asset requirements
- visual receipt hashes
- target profile facts
- browser/native renderer parity where supported

Geordi does not claim:

- terminal focus semantics
- terminal key routing
- cell-level glyph approximations
- Bijou runtime state updates

Bijou

Bijou owns terminal runtime behavior and terminal render targets.

For this design, Bijou lowers ui-scene-ir/1 to bijou-layout/1, then renders that layout into a Surface.

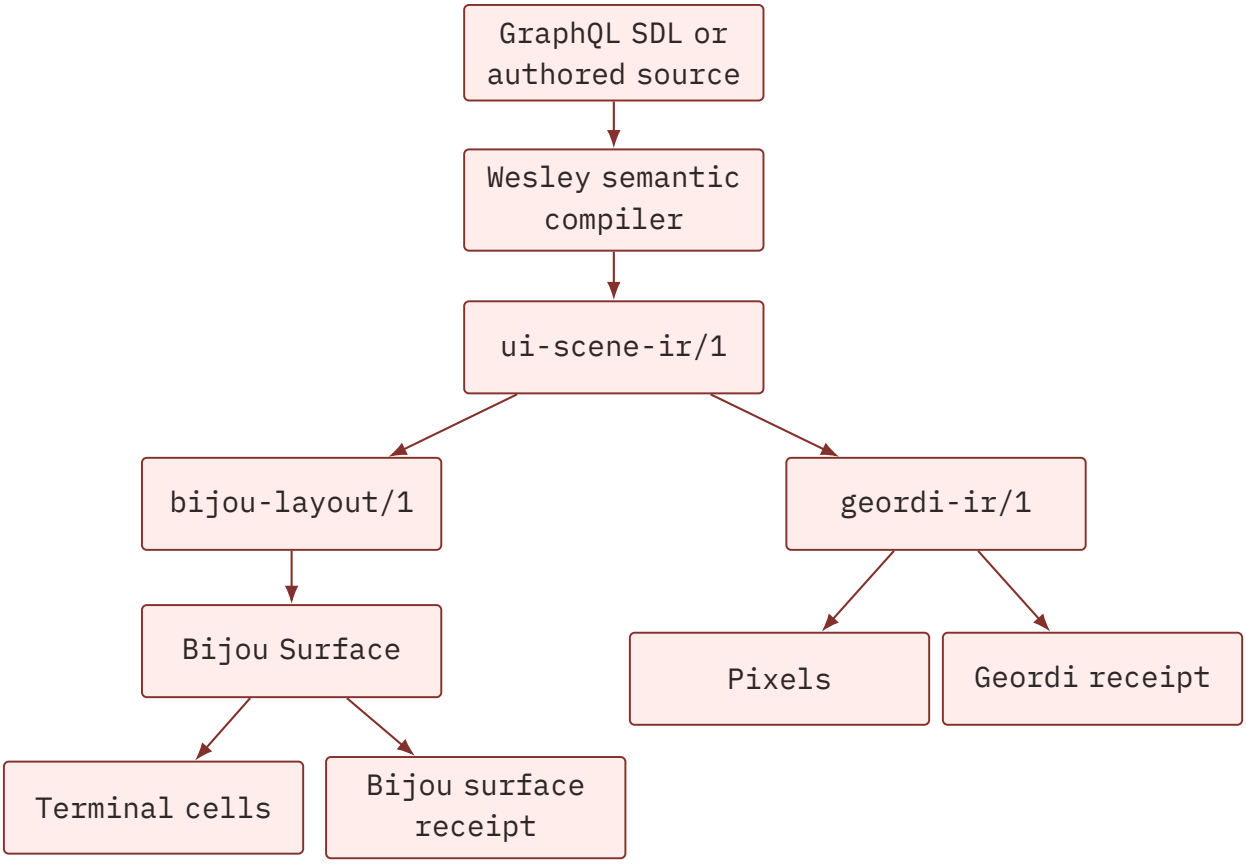
Bijou claims:

- deterministic terminal cell output
- focus and input routing
- theme token resolution
- localization lowering
- cell-level provenance
- surface receipts
- agent-readable inspection through structured surfaces

Bijou does not claim:

- browser pixel parity
- native canvas fidelity
- subpixel antialiasing
- font-pack proof identical to Geordi

Relationship Diagram



Target Profile Diagram

```
ui-scene-ir/1
  targetProfiles:
    - kind: geordi-pixel
      width: 1280
      height: 720
      colorProfile: truecolor
      claims:
        - pixel geometry
        - font metrics
        - asset hashes

    - kind: bijou-cell
      cols: 120
      rows: 40
      glyphProfile: ansi-unicode
      claims:
        - structural hierarchy
        - cell occupancy
        - token provenance
        - i18n provenance
```

The target profile makes renderer differences explicit. A Bijou terminal render can claim structural parity and cell-level determinism without claiming that it matches Geordi pixels exactly.

Proposed IR Shape

The shared scene IR should be plain data. The first draft can stay intentionally small:

```
type UiSceneIr = {
  irVersion: 'ui-scene-ir/1';
  id: string;
  sourceHash: string;
  rootNodeId: string;
  nodes: UiNode[];
  bindings: UiBinding[];
  actions: UiAction[];
  tokenUses: UiTokenUse[];
  i18nUses: UiI18nUse[];
  sourceMap: UiSourceMapEntry[];
  targetProfiles: UiTargetProfile[];
};
```

Nodes

```
type UiNode = {
  id: string;
  kind: 'box' | 'text' | 'image' | 'group' | 'list' | 'table' | 'custom';
  role?: string;
  parentId?: string;
  children?: string[];
  layout?: UiLayoutIntent;
  text?: UiTextRef;
  style?: UiStyleRef;
  actions?: string[];
  metadata?: Record<string, unknown>;
};
```

Nodes are semantic scene members. They are not terminal cells and they are not pixel rectangles, even when a renderer later lowers them into cells or rectangles.

Text

```
type UiTextRef =
  | { kind: 'literal'; value: string }
  | { kind: 'i18n'; key: string; fallback?: string };
```

Literal text should be allowed for fixture work and generated artifacts, but product surfaces should prefer i18n references when they are user-facing.

Style

```
type UiStyleRef = {
  fg?: { token: string };
  bg?: { token: string };
  border?: { token: string };
  modifiers?: string[];
};
```

The IR should carry token references, not only resolved colors. Renderers can resolve tokens later against dark or light themes. Lower modes can preserve the token name when color is unavailable.

Bindings

```
type UiBinding = {
  id: string;
  targetNodeId: string;
  targetProperty: string;
  source: {
    kind: 'state' | 'query' | 'computed';
    path: string;
  };
  when?: string;
};
```

Bindings are data, not hidden callbacks. They make dynamic surfaces auditable and compile-time-checkable.

Actions

```
type UiAction = {
  id: string;
  label?: UiTextRef;
  command: string;
  keybindings?: string[];
  targetNodeId?: string;
};
```

Actions let Bijou preserve focus and input contracts while Geordi can still render the same action affordances as visual artifacts.

Bijou Lowering

Bijou lowering turns portable intent into a terminal-specific layout contract:

```
type BijouLayoutIr = {
  irVersion: 'bijou-layout/1';
  sceneHash: string;
  nodes: LayoutNode[];
  focusGraph: FocusGraph;
  keymap: KeyBinding[];
  viewportRegions: ViewportRegion[];
  themeRequirements: TokenRequirement[];
  localeRequirements: I18nRequirement[];
  sourceMap: UiToCellSourceMap[];
};
```

The final render target remains Surface.

```
ui-scene-ir/1
  -> bijou-layout/1
    -> Surface
      -> terminal
```

The Surface can then be inspected, hashed, diffed, serialized for MCP, or recorded in a renderer playback artifact.

Geordi Lowering

Geordi lowering turns portable intent into a pixel-oriented render contract:

```
ui-scene-ir/1
  -> geordi-ir/1
    -> browser/native renderer
      -> pixel buffer
      -> receipt
```

Geordi can preserve the same node IDs, token references, localization references, and source maps while resolving geometry and render-profile facts for pixel output.

Cross-Renderer Structural Receipt

The first proof should not compare pixels to cells. It should compare shared structure.

```
type SharedSceneReceipt = {
  sceneHash: string;
  sourceHash: string;
  nodeIds: string[];
  i18nKeys: string[];
  tokenRefs: string[];
  actionIds: string[];
  bindingIds: string[];
  outputs: {
    geordi?: {
      irHash: string;
      receiptHash: string;
```

```
    };
    bijou?: {
      layoutHash: string;
      surfaceHash: string;
    };
  };
};
```

This answers the important question:

```
Did both targets render the same authored scene contract?
```

It does not falsely answer:

```
Are terminal cells pixel-perfect replicas of browser pixels?
```

Debug And Provenance Files

This design connects directly to the proposed stable debug logs:

```
<repo>/.bijou/debug.i18n.jsonl
<repo>/.bijou/debug.style.jsonl
```

The logs should record first appearance or provenance changes, not every frame.

Example i18n line:

```
{"scene":"docs.home","node":"hero.title","key":"dogfood.home.title","locale":"en","status":"r
↪  esolved"}
```

Example style line:

```
{"scene":"docs.home","node":"hero.title","slot":"fg","token":"semantic.primary","mode":"dark"
↪  ,"status":"resolved"}
```

The eventual goal is to catch these issues earlier through IR validation:

- missing localization key
- unsupported locale
- raw color in product surface
- unresolved token
- unsafe token pair
- binding target that does not exist
- action without a command

Runtime logs remain useful as proof of what actually appeared.

Authoring Model

GraphQL SDL remains the strongest candidate authoring source because Wesley and Geordi already use schema-backed compilation patterns.

The authoring source may look like this in spirit:

```
type DocsHomeScene
  @ui_scene(id: "docs.home")
  @ui_target(kind: "bijou-cell", cols: 120, rows: 40)
  @ui_target(kind: "geordi-pixel", width: 1280, height: 720) {
  title: String!
    @ui_node(id: "hero.title", kind: "text")
    @ui_text(key: "dogfood.home.title")
    @ui_style(fg: "semantic.primary")

  openAction: String
    @ui_action(id: "docs.open", command: "docs.open", keybindings: ["Enter"])
}
```

This is not final syntax. The important contract is that scene structure, text, style, binding, and action identity are compile-time data.

Lower Modes

The shared IR should make lower modes more deterministic.

For example, Bijou could render diagnostic modes:

```
normal:
  Start Here

i18n lowering:
  dogfood.nav.startHere

token lowering:
  semantic.primary on surface.primary.bg

binding lowering:
  docs.currentPage.title <- route.currentPage.title
```

These modes are easier to test than screenshots and more useful to agents than raw escape sequences.

Scope

The first implementation slice should include:

- a design document for ui-scene-ir/1
- one tiny fixture scene
- a structural receipt format
- a Bijou lowering proof to Surface
- a Geordi lowering proof or adapter stub if Geordi integration is not yet ready inside this repository
- tests proving node IDs, token refs, i18n keys, actions, and bindings survive lowering

Non-Goals

The first slice should not:

- migrate DOGFOOD wholesale to GraphQL-authored scenes

- replace existing Bijou block APIs
- implement a full Geordi renderer inside Bijou
- require Geordi as a runtime dependency of every Bijou app
- claim pixel/cell parity
- solve every binding expression shape

Acceptance Criteria

- A ui-scene-ir/1 fixture describes at least one text node, one token reference, one i18n key, and one action.
- Bijou can lower the fixture into a terminal render target without losing node identity.
- The resulting Surface can report a structural receipt or source-map facts tied back to the fixture.
- The design names the distinction between shared IR, Bijou layout IR, and Bijou Surface.
- The design names Geordi as the pixel-proof target and Bijou as the terminal-surface target.
- The validation plan proves structure, not screenshot similarity.

Slice Plan

1. Document the shared IR contract and renderer boundaries.
2. Add a minimal ui-scene-ir/1 fixture package or test helper.
3. Add Bijou lowering from the fixture to a simple Surface.
4. Add structural receipt generation for the fixture.
5. Add token and i18n provenance checks against the receipt.
6. Add a Geordi integration spike that either lowers to geordi-ir/1 or records the adapter boundary needed to do so.

Validation Plan

Focused proof should use deterministic fixture tests:

```
npm test -- --run shared-ui-scene-ir
npm run typecheck:test
npm run lint
```

Future integration proof can add:

```
npm run dogfood:i18n:check
npm run dogfood:i18n:debt
npm run verify:interactive-examples
```

The key invariant is that a passing test must prove software behavior or inspectable output. A design document alone is not completion proof for the implementation goalpost.

Risks

Risk: The Shared IR Becomes A Renderer

If ui-scene-ir/1 starts carrying renderer internals, it will become too expensive to author and too vague to validate.

Mitigation: keep final render details in geordi-ir/1, bijou-layout/1, and Surface.

Risk: Bijou Loses Terminal-Native Behavior

If the IR treats keyboard routing, focus, and terminal viewports as afterthought metadata, Bijou scenes will look portable but feel broken.

Mitigation: make Bijou target profiles carry focus, keymap, and viewport requirements explicitly.

Risk: Geordi And Bijou Drift Anyway

If both lowerings are implemented independently without receipts, drift will come back.

Mitigation: structural receipts compare node IDs, text keys, token refs, bindings, actions, and source hashes.

Risk: GraphQL Syntax Gets Overloaded

GraphQL SDL can describe contracts well, but directives can become JSON string dumps if the schema is not designed carefully.

Mitigation: use Wesley domain law to validate directive shape and prefer typed schema contracts over ad hoc JSON blobs.

Open Questions

- Should ui-scene-ir/1 live in Bijou, Geordi, Wesley, or a small shared package?
- Should Bijou expose bijou-layout/1 as a public artifact or keep it internal until the contract stabilizes?
- Should Geordi consume ui-scene-ir/1 directly, or should Wesley emit geordi-ir/1 and bijou-layout/1 as sibling artifacts?
- How much action/focus metadata belongs in the shared IR versus the Bijou target profile?
- Should debug JSONL files be produced by the renderer, by the lowering layer, or by both with different provenance phases?

Recommended Next Step

Create a roadmap-backed issue for ui-scene-ir/1 and start with one tiny fixture scene. The fixture should prove that Bijou Surface is a render target for the shared IR, while Geordi remains the pixel-proof target.

137

DX-043 Portable Bijou Blocks And Multi-Endpoint IR

docs/design/DX-043-portable-bijou-blocks-and-multi-endpoint-ir.md

```
title: DX-043 Portable Bijou Blocks And Multi-Endpoint IR
legend: DX
lane: cool-ideas
priority: medium
status: proposed
keywords:
  - developer-experience
  - blocks
  - components
  - ir
  - geordi
  - dogfood
  - render-everywhere
```

Legend: DX - Developer Experience

Linked Work

- Prior Bijou design: DX-042 Shared UI Scene IR And Bijou Render Target
- Geordi doctrine: docs/render-everywhere.md in the Geordi repository.
- Geordi pipeline: docs/end-to-end.md in the Geordi repository.
- Jedit/Geordi packed-cell seed: docs/design/0107-geordi-raytraced-title-render-pipeline.md in the jedit repository.
- Geordi-side companion design: docs/design/2026-06-portable-bijou-ui-render-endpoints.md in the Geordi repository.

This document is a planning artifact. It does not claim a GraphQL compiler, portable Bijou IR runtime, browser Dogfood endpoint, Geordi packed-cell target, or Unity endpoint already exists in this repository.

Decision Summary

Bijou should grow toward a portable block and component semantics layer. Terminal rendering remains a first-class Bijou endpoint, but it should not be the only possible endpoint for Bijou-authored UI.

The intended flow is:

```
Bijou Blocks + Components GraphQL
  -> Wesley validation and generated DTOs
    -> ui-scene-ir/1
      -> Bijou terminal endpoint
      -> Geordi browser/WebGL/WebGPU endpoint
      -> Geordi image or video witness endpoint
      -> Geordi packed Bijou cell endpoint
      -> future engine endpoints such as GeordiUnityGameEngine
```

The reverse direction should also be possible for visual surfaces:

```
Figma or other external design source
  -> Geordi IR or compatible scene IR
    -> BijouPackedCellRenderer
      -> PackedBijouCells
        -> Bijou Surface
          -> terminal cells
```

The important shift is this:

```
Bijou Surface is one render target.
Bijou block and component semantics are the portable product contract.
```

Sponsored Human

A developer wants to define an app surface once and render it in a terminal, browser, visual witness, or future game-engine endpoint without maintaining parallel component trees by hand.

Sponsored Agent

An agent wants deterministic UI IR and deterministic render witnesses so it can debug node identity, bindings, localization, theme tokens, focus, actions, and rendered output without scraping screenshots or inferring intent from terminal pixels.

Hill

Dogfood can be described by a Bijou-owned semantic UI contract, compiled into a deterministic IR, and rendered through both a terminal endpoint and a browser endpoint, with receipts proving that node IDs, token references, localization keys, actions, bindings, and target-specific outputs came from the same source.

Core Rule

Bijou owns the meaning of Bijou blocks and components.

Renderers own how that meaning is drawn.

```
GraphQL source
  describes: author intent
```

```
ui-scene-ir/1
  describes: portable semantic UI structure

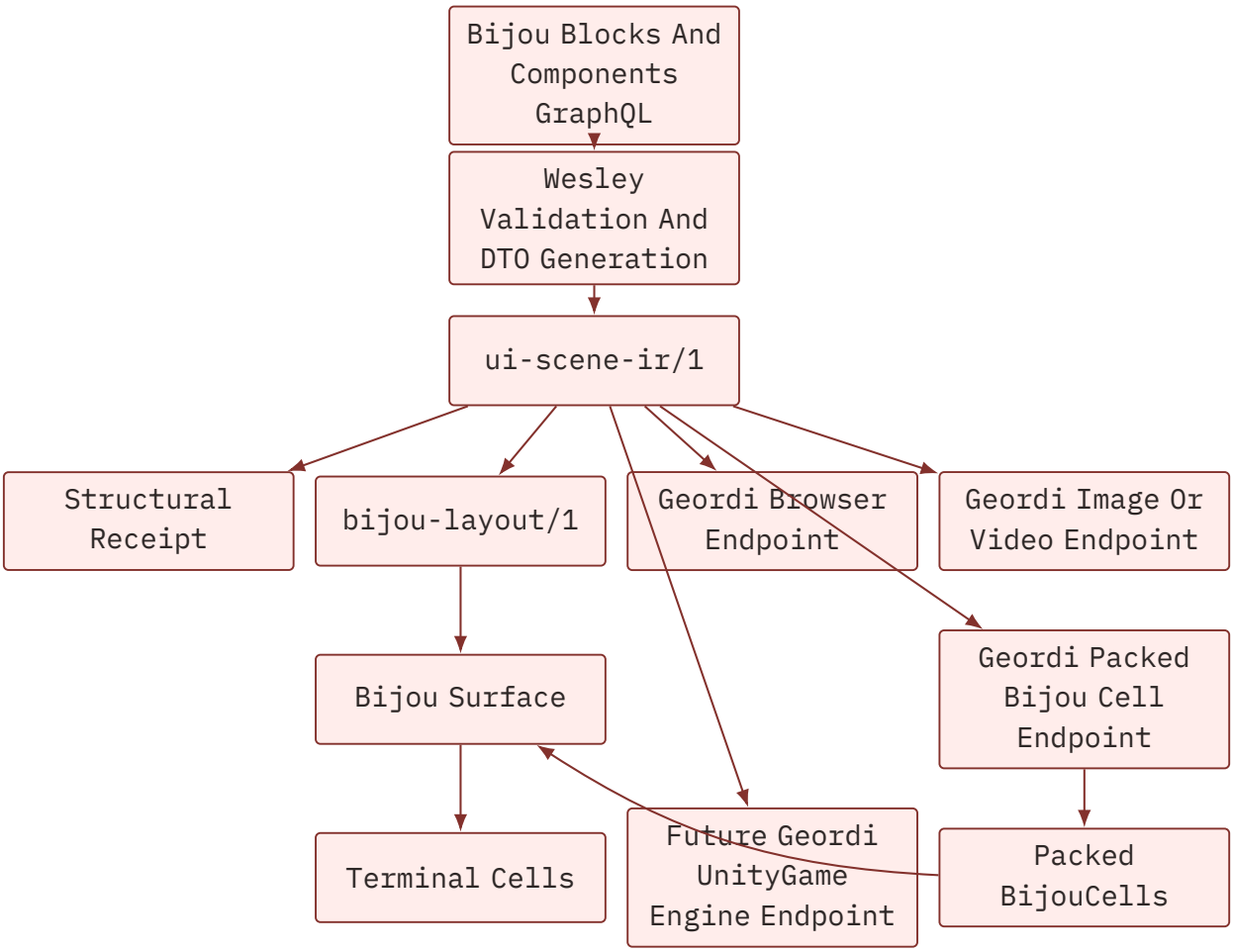
bijou-layout/1
  describes: terminal layout, focus graph, keymap, and viewports

Bijou Surface
  describes: final terminal cell output

Geordi endpoint IR or prepared target
  describes: browser, image, GPU, packed-cell, or engine render state
```

No endpoint owns the semantic truth of a Bijou component. An endpoint may render or inspect the component, but it must not silently redefine its behavior.

Relationship Diagram



Why This Matters

Agent-Legible Debugging

Screenshots are weak debugging artifacts. They show that something looked odd, but they do not directly answer what node, token, string, binding, or target rule produced the output.

IR plus receipts can answer:

- Which source node produced this cell, pixel, or engine element?
- Which localization key produced this text?
- Which theme token produced this foreground or background?
- Which action is attached to this focusable element?
- Which endpoint lowered this scene?
- Which target claims are proven, and which are explicit nonclaims?
- Did two endpoints render the same semantic scene?

The expected debug artifact should be inspectable by agents:

```
{
  "scene": "dogfood.docs",
  "node": "nav.guides.startHere",
  "component": "tree.item",
  "textKey": "dogfood.nav.startHere",
  "fgToken": "semantic.nav.item.active.fg",
  "bgToken": "semantic.nav.item.active.bg",
  "action": "dogfood.openDoc",
  "target": "bijou-terminal",
  "cellBounds": { "x": 12, "y": 4, "w": 22, "h": 1 }
}
```

Render Anywhere

Bijou-authored UI should be able to leave the terminal:

```
Bijou Blocks -> IR -> GeordiWebGL
Bijou Blocks -> IR -> GeordiWebGPU
Bijou Blocks -> IR -> GeordiImage
Bijou Blocks -> IR -> GeordiUnityGameEngine
```

This does not mean every endpoint has identical pixels. It means every endpoint renders the same semantic contract under a declared target profile.

Render Other UI Into The Terminal

Bijou can also become the terminal endpoint for non-Bijou UI sources:

```
Figma -> Geordi IR -> BijouPackedCellRenderer -> terminal
```

The first claim here is visual portability: can the source be approximated as terminal cells? Interactive portability requires additional semantics: focus, actions, state, localization, accessibility, and key routing.

Bijou Ownership

Bijou should own portable terminal UI semantics:

- blocks and components
- component identity
- child hierarchy
- layout intent
- state bindings
- actions and command IDs
- focus and key routing intent
- localization references
- theme token references
- accessibility labels and roles
- terminal runtime behavior
- Surface and cell semantics
- packed-cell validation rules when a packed target claims Bijou compatibility

Bijou should not own:

- Geordi browser renderer internals
- GPU shader dispatch
- image/video export internals
- Unity scene graph adapters
- Figma import specifics
- Bunny geometry kernels

Proposed Source Shape

GraphQL remains the best initial source-of-truth candidate because it can carry schema-backed fields and directives while letting Wesley generate DTOs and drift checks.

Example shape:

```
type DogfoodDocsScene
  @bijou_scene(id: "dogfood.docs")
  @ui_target(kind: "bijou-terminal", cols: 160, rows: 48)
  @ui_target(kind: "geordi-browser", width: 1440, height: 900)
  @ui_target(kind: "geordi-packed-bijou-cells", cols: 160, rows: 48) {
  navTitle: String!
    @ui_node(id: "nav.title", kind: "text", component: "heading")
    @ui_text(key: "dogfood.nav.title")
    @ui_style(fg: "semantic.nav.title.fg")

  startHere: String!
    @ui_node(id: "nav.guides.startHere", kind: "text", component: "tree.item")
    @ui_text(key: "dogfood.nav.startHere")
    @ui_style(
      fg: "semantic.nav.item.active.fg"
      bg: "semantic.nav.item.active.bg"
    )
    @ui_action(id: "dogfood.openDoc", command: "dogfood.openDoc", keys: ["Enter"])

  currentPage: String!
    @ui_node(id: "content.currentPage", kind: "markdown", component: "doc.page")
    @ui_binding(path: "docs.currentPage.markdown")
}
```

This syntax is not final. The contract matters more than the directive names: source nodes, text keys, style tokens, bindings, actions, and target profiles must be compile-time data.

IR Shape

DX-042 proposes the first ui-scene-ir/1 shape. This design extends its intent with explicit component portability and endpoint requirements.

```
type UiSceneIr = {
  irVersion: 'ui-scene-ir/1';
  id: string;
  sourceHash: string;
  rootNodeId: string;
  nodes: UiNode[];
  bindings: UiBinding[];
  actions: UiAction[];
  tokenUses: UiTokenUse[];
  i18nUses: UiI18nUse[];
  sourceMap: UiSourceMapEntry[];
  targetProfiles: UiTargetProfile[];
  portability: UiPortabilityClaim;
};

type UiPortabilityClaim = {
  level: 'portable' | 'terminal-native' | 'visual-only' | 'host-adapter';
  reasons?: string[];
};
```

Portability should be explicit. Some Bijou surfaces may remain terminal-native because they inspect terminal escape behavior, alternate-screen state, or raw cell buffers. Those surfaces are still valuable, but they should not pretend to render everywhere.

Target Profiles

The first target profile vocabulary should include:

Profile	Output	Primary owner
bijou-terminal	Surface	Bijou
geordi-browser	browser canvas/WebGL/WebGPU output	Geordi
geordi-image	RGBA frame, PNG, video witness	Geordi
geordi-packed-bijou-cells	PackedBijouCells	Geordi plus Bijou contract
geordi-unity	Unity scene or UI graph	Geordi adapter
external-visual-import	visual scene imported from Figma or similar	Geordi adapter

Every target profile should name supported features. Unsupported features must fail before drawing.

Bijou Terminal Lowering

The Bijou terminal endpoint lowers semantic UI IR into terminal runtime facts:

```
ui-scene-ir/1
  -> bijou-layout/1
    -> Surface
      -> terminal cells
```

bijou-layout/1 should carry:

- resolved terminal layout regions
- focus graph
- keymap
- viewport regions
- scroll policies
- localized text after i18n resolution
- theme token requirements
- source-map entries from nodes to cells
- target-specific nonclaims

The final Surface remains the rendered cell grid. It is not the portable IR.

Packed Cell Lowering

The packed cell endpoint lets Geordi render directly into a Bijou-compatible terminal cell target:

```
ui-scene-ir/1 or geordi endpoint IR
  -> Geordi packed-cell renderer
    -> PackedBijouCells
      -> Bijou Surface adapter
        -> terminal cells
```

This is useful when a renderer, CPU backend, or GPU backend can produce the terminal-native output directly instead of rendering a large image and then sampling it on the CPU.

The contract should be explicit:

```
type PackedBijouCells = {
  targetVersion: 'packed-bijou-cells/1';
  widthCells: number;
  heightCells: number;
  cellFormatId: string;
  sampleGrid?: '1x1' | '2x4' | 'custom';
  colorPolicyId: string;
  glyphResolvePolicyId: string;
  cells: PackedBijouCell[];
  facts: RenderFrameFacts;
};

type PackedBijouCell = {
  char: string;
  fg?: Rgb;
  bg?: Rgb;
```

```
  modifiers?: string[];
  provenanceNodeId?: string;
  tokenRefs?: {
    fg?: string;
    bg?: string;
  };
};
```

Bijou should validate that a packed-cell buffer is structurally compatible with Bijou cell semantics before presenting it.

Dogfood Proof

Dogfood is the best flagship proof because it contains:

- document navigation
- panes
- markdown rendering
- theme tokens
- light and dark modes
- localization
- focus
- scrolling
- keyboard actions
- status rows
- drawers
- overlays
- debug panels

The intended proof path is:

```
Dogfood source
  -> Dogfood IR
    -> Dogfood in the terminal
    -> Dogfood in the browser
    -> structural receipts proving semantic parity
```

The first slice should not rewrite all of Dogfood. It should make one page or one pane undeniable.

Structural Receipt

The shared receipt proves semantic parity, not pixel parity:

```
type PortableUiReceipt = {
  sceneHash: string;
  sourceHash: string;
  nodeIds: string[];
  componentIds: string[];
  i18nKeys: string[];
  tokenRefs: string[];
  actionIds: string[];
  bindingIds: string[];
  outputs: {
```

```
    terminal?: {
      layoutHash: string;
      surfaceHash: string;
    };
    browser?: {
      endpointHash: string;
      witnessHash: string;
    };
    packedCells?: {
      targetHash: string;
      surfaceHash: string;
    };
  };
};
```

Endpoint receipts can add target-specific proof:

- terminal cell hashes
- browser pixel probes
- packed-cell hashes
- image frame hashes
- engine object counts
- target profile support matrix

Accessibility And Assistive Posture

Portable Bijou IR should preserve accessibility metadata even when a renderer does not directly display it.

Required fields should include:

- semantic role
- accessible label or i18n key
- focusability
- keyboard action hints
- decorative vs meaningful marker
- directionality and locale posture
- reduced-motion or animation policy where relevant

Browser and engine endpoints can map this metadata to their own accessibility systems later. Terminal endpoints can expose it through debug modes, help surfaces, and structured MCP surfaces.

Localization And Directionality Posture

Product text should enter the IR as localization keys, not resolved ad hoc strings. Render endpoints may resolve keys later against locale bundles, but receipts must preserve which key was used.

Directionality should be target-aware:

- terminal endpoint records whether a region is left-to-right or right-to-left

- browser endpoint can map direction to layout/text properties
- image endpoint records direction metadata in frame facts
- packed-cell endpoint records the resolved order it actually emitted

Agent Inspectability

The IR and receipts should make these questions answerable by file inspection:

- What component rendered this cell?
- What action would fire if the user pressed Enter here?
- Why is this node not focusable?
- Which token produced this color?
- Which localization key produced this text?
- Which target profile rejected this component?
- Did terminal and browser outputs use the same semantic node tree?

This is the main reason the IR matters. It turns UI debugging into structured evidence instead of screenshot interpretation.

Scope

The first implementation arc should include:

- one portable Bijou component fixture
- one GraphQL or JSON source-of-truth fixture
- one ui-scene-ir/1 output
- one terminal lowering to Surface
- one browser or Geordi endpoint stub
- one structural receipt
- one negative fixture proving fail-loud unsupported target requirements

Non-Goals

The first arc should not:

- migrate all Dogfood pages
- replace existing block APIs
- require Geordi at runtime for every Bijou app
- claim pixel parity between terminal and browser
- implement Unity, Figma, WebGPU, or GPU packed-cell output immediately
- make GraphQL directives the permanent final authoring syntax
- erase terminal-native Bijou components

Acceptance Criteria

- A portable Bijou component can be represented as deterministic IR.
- The IR preserves node IDs, component IDs, token refs, i18n keys, actions, and bindings.
- The same IR can lower to a Bijou Surface.
- A second endpoint stub or fixture can consume the same IR and emit a receipt.
- Receipts prove semantic parity across endpoints.
- Unsupported endpoint requirements fail before drawing.
- The design distinguishes portable components from terminal-native components.
- Dogfood has a concrete first-page or first-pane migration target.

Slice Plan

1. Land this design and the Geordi companion design.
2. Define a tiny portable component fixture.
3. Define the first ui-scene-ir/1 fixture and canonical JSON rules.
4. Add a structural receipt fixture.
5. Lower the fixture to a Bijou Surface.
6. Add a terminal receipt with source-map facts.
7. Add a Geordi browser endpoint stub or fixture consumer.
8. Add a fail-loud unsupported target fixture.
9. Add a Dogfood page/pane candidate map.
10. Emit Dogfood IR witness data beside the current runtime.
11. Render one Dogfood pane from IR to terminal.
12. Render the same pane through a browser witness endpoint.

Validation Plan

Initial validation should be deterministic and file-backed:

```
npm test -- --run portable-bijou-ir
npm run typecheck:test
npm run lint
npm run docs:inventory
git diff --check
```

Future validation should add:

```
npm run dogfood:i18n:check
npm run dogfood:i18n:debt
npm run verify:interactive-examples
```

The important proof rule remains: design docs describe intent; executable fixtures and receipts prove behavior.

Risks

Risk: Bijou Becomes Too Abstract

If portable IR hides terminal realities, Bijou components may render elsewhere but feel broken in the terminal.

Mitigation: keep bijou-terminal as a first-class endpoint with focus, keymap, viewport, scroll, and cell-source-map requirements.

Risk: Geordi Redefines Bijou Semantics

If Geordi endpoints silently reinterpret Bijou components, parity becomes cosmetic.

Mitigation: Bijou owns component semantics; Geordi owns endpoint rendering and receipts.

Risk: GraphQL Becomes An Unreadable Directive Dump

GraphQL can carry schema-backed contracts, but overly complex directives can turn into JSON blobs.

Mitigation: keep source syntax small, use Wesley validation, and treat early syntax as provisional.

Risk: Visual Import Pretends To Be Interactive

A Figma-to-terminal path can render a visual mockup, but that does not mean it has focus, actions, state, or accessibility.

Mitigation: distinguish visual portability from interactive portability in the target profile.

Recommended Next Step

Create the smallest possible Dogfood proof:

```
one Dogfood nav or content pane
  -> portable source fixture
    -> ui-scene-ir/1
      -> Bijou Surface
      -> browser/Geordi endpoint receipt
      -> structural parity receipt
```

That proves the architecture without rewriting Dogfood or overbuilding the compiler.

138

DX-044 GraphQL Authored Bijou Block IR Proof

docs/design/DX-044-graphql-authored-bijou-block-ir-proof.md

```
title: DX-044 GraphQL Authored Bijou Block IR Proof
legend: DX
lane: cool-ideas
priority: medium
github_issue: 302
status: landed
keywords:
  - developer-experience
  - graphql
  - blocks
  - ir
  - geordi
  - dogfood
  - render-everywhere
```

Legend: DX - Developer Experience

Linked Work

- GitHub issue: #302
- Runtime seed: DX-042 Shared UI Scene IR And Bijou Render Target
- Portable endpoint direction: DX-043 Portable Bijou Blocks And Multi-Endpoint IR
- Prior block contract: DX-031 Standard Bijou Blocks

This cycle is the first implementation proof after the ui-scene-ir/1 runtime seed. It should not try to make every Bijou block GraphQL-authored. It should prove one boring, deterministic vertical slice that future Wesley and Geordi work can replace or extend without changing the proof contract.

Decision Summary

Bijou should treat GraphQL SDL as an authoring surface for portable block semantics, not as a runtime data store and not as a visual scene graph.

The first flow is:

```
GraphQL SDL fixture
  -> bijou-block/1 artifact
    -> ui-scene-ir/1
      -> Bijou terminal Surface proof
        -> ui-scene-receipt/1
```

The first slice stays inside Bijou. Geordi remains a named endpoint target, but this cycle does not require Geordi repository changes unless the Bijou proof needs a companion endpoint fixture.

Sponsored Human

A Bijou author wants to define a small UI block in a declarative source format and get a deterministic block artifact, scene IR, terminal output, source map, and receipt without hand-maintaining parallel JSON fixtures.

Sponsored Agent

An agent wants to inspect the GraphQL source, generated block artifact, ui-scene-ir/1, terminal cell source map, and receipt hashes to understand exactly which source field produced each visible terminal fact.

Hill

Given a constrained GraphQL SDL file for one text-first Bijou block, Bijou can produce a stable bijou-block/1 artifact, lower it into ui-scene-ir/1, render it to a terminal Surface, and emit a receipt whose source maps, tokens, i18n keys, actions, bindings, and hashes are deterministic.

Playback Questions

This design is successful only if the committed proof lets a maintainer or agent answer the following questions from repository facts:

- Can a reader trace one SDL field through bijou-block/1, ui-scene-ir/1, terminal cells, and receipt dependencies?
- Can whitespace-only SDL edits preserve the block artifact hash and scene hash?
- Can unsupported or duplicate identities fail before terminal lowering produces a misleading witness?
- Can an agent inspect source-map, token, i18n, action, and binding facts without scraping a screenshot?

Linked Invariants

- Schemas Live At Boundaries: the SDL reader rejects unsupported or malformed authoring input before the runtime treats the block artifact as trusted.

- Graceful Lowering Preserves Meaning: bijou-block/1 preserves semantic text, token, i18n, action, binding, and source-map meaning when it lowers to terminal cells.
- The Buffer Holds Facts: generated artifacts, receipts, and source maps remain inspectable fact records instead of executable behavior.
- Tests Are the Spec: the first slice is complete only when focused tests prove the hash, validation, lowering, and receipt contract.

Scope

- A constrained SDL reader for one block type.
- Directive-like metadata for block identity, target profile, slots, commands, bindings, theme tokens, localization keys, and source positions.
- A public bijou-block/1 artifact type.
- A compiler function from SDL source to block artifact.
- A lowering function from block artifact to ui-scene-ir/1.
- Tests proving whitespace-stable source hashing, artifact stability, IR validation, terminal lowering, source maps, and receipts.

Non-Goals

- No full GraphQL parser dependency in the first slice.
- No Wesley code generation in the first slice.
- No Geordi endpoint implementation in the first slice.
- No arbitrary GraphQL schema execution.
- No nested field resolver runtime.
- No visual layout engine beyond explicit first-slice layout facts.
- No DOGFOOD-wide migration.

First SDL Shape

The first SDL subset is intentionally explicit:

```
type ReleaseTitle
  @bijouBlock(id: "release.title", component: "ReleaseTitleBlock")
  @bijouTarget(kind: "bijou-terminal", cols: 40, rows: 5) {
  heading: String!
    @bijouText(id: "heading", x: 2, y: 1)
    @bijouI18n(key: "release.title.heading", fallback: "Bijou")
    @bijouToken(fg: "semantic.title.fg")

  openNotes: String
    @bijouAction(id: "release.openNotes", command: "release.openNotes", key: "Enter")
    @bijouText(id: "open-notes", x: 2, y: 3)
    @bijouI18n(key: "release.title.openNotes", fallback: "Open release notes")
    @bijouToken(fg: "semantic.action.fg", bg: "semantic.action.bg")
}
```

The subset allows only:

- one type
- type directives on the type declaration or following indented lines before {
- scalar fields
- field directives on following indented lines
- string and integer directive arguments

That is small enough to parse deterministically without pretending to be a general GraphQL compiler.

Binding Directive Shape

Bindings use field directives because they attach authoring source to one renderable field:

```
status: String
  @bijouText(id: "status", x: 2, y: 3)
  @bijouBind(
    id: "release.status.value",
    kind: "state",
    path: "release.status",
    targetProperty: "text",
    when: "release.visible"
  )
```

The first slice treats binding directives as fact records, not live dataflow. kind is intentionally constrained to source categories such as state, query, and computed. path identifies the external source fact. targetProperty defaults to text when omitted. when is optional and records the gating condition that a later runtime or endpoint may evaluate.

Artifact Contract

The first generated artifact is:

```
bijou-block/1
  id
  component
  sourceHash
  fields[]
    fieldName
    nodeId
    text
    action
    binding
    token refs
    source
  targetProfiles[]
```

bijou-block/1 is semantic. It is not the same thing as Geordi IR and it is not the same thing as a final Surface.

Lowering Contract

bijou-block/1 -> ui-scene-ir/1 must preserve:

- source hash
- node ids
- component ids
- i18n keys and fallbacks
- token references
- action ids, commands, keybindings, and targets
- binding facts when present
- source-map facts from SDL field anchors to IR nodes
- target profile facts

If the source asks for unsupported target facts, the compiler must fail before lowering.

Source Map Posture

Source strings should be stable and local to the input. The first slice uses semantic anchors instead of line and column positions so formatting-only SDL edits do not change the artifact or scene hash:

```
release-title.graphql#type.ReleaseTitle.field.heading
```

Absolute paths are not allowed in emitted artifacts. The caller may provide a sourceName option, but the default should be a neutral inline.graphql.

Agent Inspectability

The useful debug path should be:

```
SDL field -> bijou-block/1 field -> ui-scene-ir/1 node -> Surface cell -> receipt
```

An agent should not need a screenshot to answer:

- Which SDL field produced this node?
- Which node produced this terminal cell?
- Which i18n key and token refs were used?
- Which action or binding belongs to the rendered text?
- Which hash proves the scene and terminal output?

Accessibility And Localization Posture

This first slice preserves localization keys and fallbacks but does not add new runtime translation behavior. The source author must provide fallback

text for renderable text nodes. Directionality is deferred until multiline text and localized layout constraints exist in the IR.

Geordi Posture

Geordi is the likely next endpoint after this Bijou-only proof. The first Geordi-facing shape should be a receipt-compatible endpoint facet, not a semantic replacement for bijou-block/1.

Expected later flow:

```
bijou-block/1
  -> ui-scene-ir/1
    -> Geordi endpoint profile
      -> browser/image/packed-cell witness
```

Implementation Outline

1. Add this design doc and issue update.
2. Add bijou-block/1 artifact types.
3. Add a deterministic constrained SDL compiler.
4. Add block-artifact to ui-scene-ir/1 lowering.
5. Add public exports.
6. Add a focused fixture test that renders one GraphQL-authored block into a terminal proof.
7. Update changelog and issue/PR proof.

Tests To Write First

- RED: a whitespace-variant SDL input should produce the same block hash and scene hash.
- RED: missing @bijouBlock should fail with a deterministic error.
- RED: a text field with i18n/token/action directives should become one IR text node with matching source-map facts.
- GREEN: terminal proof should render fallback text and include SHA-256 scene, layout, and surface hashes.

Closeout

The cycle is landed when:

- the design doc exists and links #302
- #302 has a cycle comment pointing at the branch and PR
- @flyingrobots/bijou exports the first block authoring APIs
- the focused GraphQL-authored block proof is green

- npm run lint, npm run typecheck:test, npm run docs:inventory, and the relevant focused tests are green
- the PR names #302 and explains what remains for Wesley and Geordi

139

DX-045 GraphQL Block Groups And Debug Facts

docs/design/DX-045-graphql-block-groups-and-debug-facts.md

```
title: DX-045 GraphQL Block Groups And Debug Facts
legend: DX
lane: cool-ideas
priority: medium
github_issue: 302
status: landed
keywords:
  - developer-experience
  - graphql
  - blocks
  - ir
  - dogfood
  - debug-facts
  - render-everywhere
```

Legend: DX - Developer Experience

Linked Work

- GitHub issue: #302
- Prior slice: DX-044 GraphQL Authored Bijou Block IR Proof
- Runtime seed: DX-042 Shared UI Scene IR And Bijou Render Target
- Portable endpoint direction: DX-043 Portable Bijou Blocks And Multi-Endpoint IR

DX-044 proved the smallest boring path:

```
GraphQL SDL
  -> bijou-block/1
    -> ui-scene-ir/1
      -> terminal Surface proof
```

DX-045 turns that proof into a more credible authoring pipeline by adding explicit group nodes and deterministic debug facts. It should still stay inside Bijou; Geordi remains the next endpoint after the semantic artifact is strong enough to hand across a repository boundary.

Decision Summary

GraphQL-authored Bijou blocks need a hierarchy before Dogfood or BlockLab can use them naturally. The first hierarchy primitive should be a small field-attached group directive, not a general GraphQL execution model.

The next flow is:

```
GraphQL SDL fixture
  -> bijou-block/1 artifact with groups[]
    -> ui-scene-ir/1 group/text node tree
      -> terminal Surface proof
        -> debug summary facts
          -> ui-scene-receipt/1
```

The debug summary is deliberately fact-shaped. It is not a renderer and not a test snapshot. It gives maintainers and agents one deterministic object that answers what source, artifact, scene, nodes, tokens, i18n keys, actions, bindings, and lower-mode witnesses exist.

Sponsored Human

A Bijou author wants to organize a GraphQL-authored block into named sections without hand-editing JSON IR, then confirm that the rendered terminal output and debug facts still point back to the right source fields.

Sponsored Agent

An agent wants a deterministic summary of the GraphQL-to-IR pipeline so it can reason about a UI failure from source anchors, group ownership, token refs, i18n keys, bindings, actions, hashes, and lower-mode output instead of interpreting screenshots.

Hill

Given a constrained GraphQL SDL file with text fields assigned to named groups, Bijou can produce a stable grouped bijou-block/1 artifact, lower it into a valid ui-scene-ir/1 group/text node tree, render the text nodes to a terminal Surface, and emit deterministic debug facts that preserve source maps, tokens, i18n keys, actions, bindings, lower-mode witnesses, and hashes.

Playback Questions

- Can one SDL group directive become one bijou-block/1 group and one ui-scene-ir/1 group node with stable parent/child relationships?
- Can one SDL text field point to its owning group, text node, i18n key, token refs, action, binding, visible cells, and source anchor?
- Can whitespace-only SDL edits preserve the artifact hash, scene hash, and debug summary hash?
- Can lower modes prove the same grouped scene through node ids, i18n keys, and token refs?
- Can unsupported duplicate group ids fail before lowering creates a misleading receipt?

Scope

- Add a constrained @bijouGroup(...) directive.
- Add groups[] to bijou-block/1 while preserving existing flat-field compatibility.
- Lower block groups into ui-scene-ir/1 group nodes.
- Preserve deterministic group source-map facts.
- Add a public debug summary helper for the GraphQL block pipeline.
- Add fixture-style tests for grouped SDL, duplicate groups, lower modes, and hash stability.
- Update issue, roadmap, bearing, and changelog evidence.

Non-Goals

- No full GraphQL parser dependency.
- No nested GraphQL execution or resolver runtime.
- No automatic layout engine.
- No Geordi repository changes in this slice.
- No Dogfood-wide migration.
- No browser endpoint.
- No frame-capture file format.

SDL Shape

The first grouped SDL shape stays explicit:

```
type ReleaseCard
  @bijouBlock(id: "release.card", component: "ReleaseCardBlock")
  @bijouTarget(kind: "bijou-terminal", cols: 48, rows: 8)
  @bijouGroup(id: "release.card.header", label: "Header", x: 1, y: 1, width: 46, height: 3)
  @bijouGroup(id: "release.card.footer", label: "Footer", x: 1, y: 5, width: 46, height: 2) {
  heading: String!
    @bijouText(id: "heading", group: "release.card.header", x: 2, y: 1)
    @bijouI18n(key: "release.card.heading", fallback: "Bijou")
    @bijouToken(fg: "semantic.title.fg")

  openNotes: String
    @bijouText(id: "open-notes", group: "release.card.footer", x: 2, y: 5)
    @bijouAction(id: "release.openNotes", command: "release.openNotes", key: "Enter")
    @bijouI18n(key: "release.card.openNotes", fallback: "Open release notes")
    @bijouToken(fg: "semantic.action.fg", bg: "semantic.action.bg")
}
```

Group directives are type directives. Text fields opt into a group with @bijouText(group: "..."). Omitting group preserves the DX-044 behavior and parents the text node to the block root.

The subset allows only:

- one type
- type directives on the type declaration or following indented lines before {

- scalar fields
- field directives on following indented lines
- string and integer directive arguments
- group ids that are unique and cannot collide with the root node or field node ids

Artifact Contract

bijou-block/1 gains groups[]:

```
bijou-block/1
  id
  component
  sourceHash
  rootNodeId
  groups[]
    id
    label
    source
    layout
  fields[]
    fieldName
    nodeId
    groupId
    text
    action
    binding
    token refs
    source
  targetProfiles[]
```

Existing consumers that only read fields[] remain valid. Lowering owns the new parent-child relationships.

Debug Summary Contract

The debug helper should return plain data:

```
graphql-bijou-block-debug/1
  artifactVersion
  artifactHash
  sceneHash
  rootNodeId
  groups[]
  fields[]
  sourceMap[]
  i18nKeys[]
  tokenRefs[]
  actionIds[]
  bindingIds[]
  lowerModes[]
```

The summary should be stable enough to hash and inspect in tests. It should not include host paths, timestamps, process ids, terminal dimensions from the operator environment, or screenshots.

Accessibility And Lower Modes

Groups must preserve semantic organization even when terminal color and layout are unavailable. Lower modes must still expose:

- node ids for group and text nodes
- i18n keys for localized text
- token refs for styled text
- source anchors for field and group facts

No meaning in this slice may depend only on color.

Localization And Directionality Posture

The compiler continues to require fallback text for renderable i18n text nodes. Groups may carry labels for debug and inspector use, but group labels are not user-facing localized strings in this slice. Directionality remains deferred until multiline text and localized layout constraints exist in the IR.

Agent Inspectability

The useful debug path should become:

```
SDL type directive / field directive
  -> bijou-block/1 group / field
    -> ui-scene-ir/1 group / text node
      -> terminal cell source map
        -> debug summary
          -> receipt
```

An agent should be able to answer, from committed facts:

- Which SDL directive created this group?
- Which fields belong to this group?
- Which visible cells came from a field in that group?
- Which lower-mode output proves the same scene without color?
- Which hashes bind the artifact, scene, surface, and debug summary?

Linked Invariants

- Schemas Live At Boundaries: grouped SDL authoring must fail at compile time when ids collide or point to missing groups.
- Graceful Lowering Preserves Meaning: the grouped artifact must preserve semantic organization through ui-scene-ir/1 and terminal lower modes.
- The Buffer Holds Facts: debug output must be serializable fact records, not executable renderer behavior.

- Tests Are the Spec: grouped authoring is complete only when focused tests prove the compiler, lowering, lower modes, and debug summary.

Implementation Outline

1. Add this design doc and tracker updates.
2. Add grouped SDL fixture tests that fail against the DX-044 flat compiler.
3. Extend bijou-block/1 types with groups[] and groupId.
4. Parse repeated @bijouGroup(...) type directives.
5. Validate group ids and field group references before lowering.
6. Lower groups into ui-scene-ir/1 group nodes.
7. Add createGraphqlBijouBlockDebugSummary().
8. Prove normal rendering and lower modes for one grouped block.
9. Preserve whitespace-stable artifact, scene, and debug hashes.
10. Update changelog, issue, and PR evidence.

Tests To Write First

- RED: grouped SDL should compile into two groups and two grouped fields.
- RED: grouped lowering should produce a valid ui-scene-ir/1 tree with root children as group ids and group children as text node ids.
- RED: debug summary should list artifact hash, scene hash, groups, fields, i18n keys, token refs, actions, bindings, source-map anchors, and lower-mode hashes.
- RED: duplicate group ids and missing field group references should fail before lowering.
- GREEN: terminal normal, node-ids, i18n-keys, and token-refs lower modes should render deterministic grouped-scene facts.

Closeout

The cycle is landed when:

- this design doc links #302
- the branch PR links this design and #302
- compileGraphqlBijouBlock() supports grouped authoring
- lowerBijouBlockToUiScene() preserves group hierarchy
- createGraphqlBijouBlockDebugSummary() is exported
- focused grouped GraphQL tests are green
- npm run lint, npm run typecheck:test, npm run docs:inventory, and the relevant focused tests are green
- the PR names what remains for Dogfood-wide authoring and Geordi endpoints

140

DX-046 GraphQL Authored DOGFOOD Block Fixture

docs/design/DX-046-graphql-authored-dogfood-block-fixture.md

```
title: DX-046 GraphQL Authored DOGFOOD Block Fixture
legend: DX
lane: asap
priority: high
github_issue: 329
parent_issue: 302
status: landed
keywords:
  - developer-experience
  - dogfood
  - graphql
  - blocks
  - ir
  - debug-facts
  - v7.1.0
```

Legend: DX - Developer Experience

Linked Work

- User story: #329
- Parent tracker: #302
- Prior slice: DX-045 GraphQL Block Groups And Debug Facts
- Runtime seed: DX-042 Shared UI Scene IR And Bijou Render Target
- Portable endpoint direction: DX-043 Portable Bijou Blocks And Multi-Endpoint IR
- DOGFOOD surface block lineage: DF-030 Dogfood Docs Surface Block

This cycle is the narrow v7.1.0 implementation item. It must not move the broad #302 tracker out of Beyond; #329 carries the release-scoped DX-046 proof.

Decision Summary

DX-044 and DX-045 proved that constrained GraphQL SDL can produce bijou-block /1, lower into ui-scene-ir/1, render terminal proof, and emit graphql-bijou-b

lock-debug/1 facts. DX-046 turns that compiler proof into a real DOGFOOD product fixture.

The first DOGFOOD target is NavigationListBlock from examples/docs/dogfood-blocks.ts. It is small enough for a release-boundary slice but real enough to carry product semantics: navigation items, selection state, command intents, i18n text, token styling, group structure, terminal proof, and debug facts.

The useful flow is:

```
NavigationListBlock GraphQL SDL fixture
  -> bijou-block/1 grouped artifact
    -> ui-scene-ir/1
      -> terminal Surface proof
        -> graphql-bijou-block-debug/1 facts
          -> DOGFOOD product facts
```

Sponsored Human

A maintainer wants the post-V7 GraphQL block compiler to prove it can describe an actual DOGFOOD surface, not only synthetic release-card examples.

A DOGFOOD reader benefits because the docs navigation surface becomes inspectable from one source contract instead of only from runtime rendering code.

Sponsored Agent

An agent wants to trace one DOGFOOD navigation field from SDL through bijou-block/1, ui-scene-ir/1, terminal cells, lower modes, debug summary facts, and receipt hashes without scraping screenshots or reconstructing product intent from rendered text.

Hill

Given a checked-in GraphQL SDL fixture for NavigationListBlock, Bijou can compile it into a deterministic grouped bijou-block/1 artifact, lower it into a valid ui-scene-ir/1 scene, render terminal proof, and emit graphql-bijou-block-debug/1 facts that preserve source maps, token refs, i18n keys, actions, bindings, groups, lower-mode witnesses, and hashes.

Playback Questions

- Which DOGFOOD product surface is now authored as GraphQL SDL?
- Can a maintainer prove that the fixture is tied to the existing docs.navigation registry entry and NavigationListBlock component name?
- Can one SDL field be traced through artifact field, scene node, terminal cell source map, lower-mode row, debug fact, and receipt dependency?

- Can debug facts list the DOGFOOD groups, fields, i18n keys, token refs, action ids, binding ids, lower-mode hashes, artifact hash, and scene hash?
- Can failure tests reject missing or broken product facts before terminal proof is generated?

Scope

- Add a checked-in GraphQL SDL fixture for NavigationListBlock.
- Keep the fixture inside the Bijou repository and inside the existing constrained compiler path.
- Compile the fixture with compileGraphqlBijouBlock().
- Lower the artifact with lowerBijouBlockToUiScene().
- Render deterministic terminal proof with lowerUiSceneToTerminalProof().
- Assert createGraphqlBijouBlockDebugSummary() facts for the real DOGFOOD surface.
- Add tests that bind the fixture to defaultDogfoodBlockRegistry and the existing docs.navigation surface.
- Update changelog and issue/PR evidence.

Non-Goals

- Do not require Wesley or Geordi repository changes.
- Do not migrate all DOGFOOD blocks to GraphQL.
- Do not rewrite the DOGFOOD navigation runtime or docs app.
- Do not introduce a general GraphQL parser or resolver runtime.
- Do not create BlockLab, Theme Lab, localization workbench, or a browser endpoint.
- Do not move parent issue #302 out of the Beyond milestone.

Fixture Contract

The fixture should live at examples/docs/fixtures/graphql/navigation-list.graphql unless implementation finds a stronger local convention.

Expected SDL shape:

```
type DogfoodNavigationList
  @bijouBlock(id: "dogfood.navigation", component: "NavigationListBlock")
  @bijouTarget(kind: "bijou-terminal", cols: 80, rows: 8)
  @bijouGroup(id: "dogfood.navigation.header", label: "Header", x: 1, y: 0, width: 46,
  ↪  height: 2)
  @bijouGroup(id: "dogfood.navigation.items", label: "Items", x: 1, y: 2, width: 46, height:
  ↪  5) {
  title: String!
    @bijouText(id: "dogfood.navigation.title", group: "dogfood.navigation.header", x: 2, y: 0)
    @bijouI18n(key: "dogfood.navigation.title", fallback: "Documentation")
    @bijouToken(fg: "semantic.nav.title.fg")

  activeItem: String!
    @bijouText(id: "dogfood.navigation.active", group: "dogfood.navigation.items", x: 2, y: 2)
```

```
    @bijouAction(id: "navigation.selectItem", command: "navigation.selectItem", key: "Enter")
    @bijouBind(id: "navigation.selection.activeLabel", kind: "state", path:
    ↪  "navigation.selection.activeLabel")
    @bijouI18n(key: "dogfood.navigation.active", fallback: "Active: Blocks")
    @bijouToken(fg: "semantic.nav.item.active.fg", bg: "semantic.nav.item.active.bg")

  itemCount: String!
    @bijouText(id: "dogfood.navigation.itemCount", group: "dogfood.navigation.items", x: 2,
    ↪  y: 3)
    @bijouBind(id: "navigation.items.count", kind: "computed", path: "navigation.items.count")
    @bijouI18n(key: "dogfood.navigation.itemCount", fallback: "Items: 7")
    @bijouToken(fg: "semantic.nav.item.fg")

  expandHint: String
    @bijouText(id: "dogfood.navigation.expandHint", group: "dogfood.navigation.items", x: 2,
    ↪  y: 4)
    @bijouAction(id: "navigation.expandGroup", command: "navigation.expandGroup", key:
    ↪  "ArrowRight")
    @bijouI18n(key: "dogfood.navigation.expandHint", fallback: "Expand group")
    @bijouToken(fg: "semantic.nav.hint.fg")
}
```

The implementation may adjust coordinates, labels, or token names if existing terminal proof requires it, but the fixture must still preserve:

- NavigationListBlock as the component
- dogfood.navigation as the artifact id
- docs.navigation as the registry surface connected by tests
- navigation.selectItem, navigation.expandGroup, and selection/item-count binding facts

Accessibility And Assistive Posture

Meaning must survive without color or borders. The node-ids, i18n-keys, and t oken-refs lower modes must expose the same DOGFOOD navigation nodes as the normal terminal proof, and the active selection must be identifiable through source maps and binding facts.

Localization And Directionality Posture

Renderable fields must use @bijouI18n with fallback text. This slice does not add runtime translation behavior or directionality rules; it preserves i18n keys and fallbacks so later locale-aware fixtures can replace the fallback text without changing the artifact contract.

Agent Inspectability And Explainability Posture

The committed facts should let an agent inspect:

- fixture source path and semantic source anchors
- artifact id, component, source hash, and group hierarchy
- scene id, root node, action ids, binding ids, i18n keys, and token refs
- terminal cell source-map entries for visible navigation text

- lower-mode rows and hashes
- debug summary hash, artifact hash, and scene hash

No emitted artifact may include absolute local paths, timestamps, process ids, or screenshots.

Linked Invariants

- Schemas Live At Boundaries: the DOGFOOD SDL fixture must fail before lowering when required source facts are malformed or missing.
- Graceful Lowering Preserves Meaning: the fixture must preserve DOGFOOD navigation meaning across bijou-block/1, ui-scene-ir/1, terminal cells, and lower modes.
- The Buffer Holds Facts: debug output must stay inspectable as serializable facts.
- Tests Are the Spec: DX-046 is not complete until focused tests prove the real DOGFOOD fixture contract.

Implementation Outline

1. Add this design doc and issue/roadmap updates.
2. Add failing tests for the checked-in DOGFOOD GraphQL fixture.
3. Add the NavigationListBlock SDL fixture.
4. Add any small fixture loader or export needed by tests.
5. Compile the fixture into bijou-block/1 and assert DOGFOOD registry alignment.
6. Lower the artifact into ui-scene-ir/1 and assert terminal proof facts.
7. Assert graphql-bijou-block-debug/1 fields, groups, lower modes, and hashes.
8. Add failure coverage for missing DOGFOOD product facts.
9. Update changelog and issue/PR evidence.

Tests To Write First

- RED: loading the checked-in DOGFOOD navigation SDL fixture should fail before the fixture exists.
- RED: compiling the fixture should produce dogfood.navigation, NavigationLis tBlock, two groups, DOGFOOD fields, token refs, i18n keys, action ids, binding ids, and a neutral source name.
- RED: registry alignment should prove defaultDogfoodBlockRegistry maps docs. navigation to NavigationListBlock.
- RED: lowering and terminal proof should expose visible navigation text and source-map entries for the active item.
- RED: debug summary should include artifact hash, scene hash, summary hash, groups, fields, i18n keys, token refs, actions, bindings, and normal / node -ids / i18n-keys / token-refs lower modes.

- RED: malformed DOGFOOD fixture variants should fail before terminal proof when they omit required i18n, action, binding, group, or token facts.

Closeout

The cycle is landed when:

- #329 links this design doc and the PR
- #302 remains the broad Beyond tracker
- v7.1.0 keeps #329 as the release-scoped tracker item
- the branch PR links #329, #302, and this design
- focused DX-046 tests are green
- npm run docs:inventory, npm run typecheck:test, npm run lint, focused tests, and the relevant full validation are green
- docs/CHANGELOG.md records the real DOGFOOD GraphQL fixture under Unreleased

Implementation Notes

This cycle adds examples/docs/fixtures/graphql/navigation-list.graphql as the first checked-in DOGFOOD GraphQL SDL fixture. Focused tests compile it into the dogfood.navigation bijou-block/1 artifact, prove that it matches the existing docs.navigation registry entry and NavigationListBlock, lower it to ui-scene-ir/1, render terminal proof, assert debug facts, and reject malformed fixture variants before terminal proof.

141

DX-047 - Blocks App Binding Snippets

docs/design/DX-047-blocks-app-binding-snippets.md

```
id: DX-047
title: Blocks App Binding Snippets
status: active
lane: asap
priority: medium
github_issue: 342
legend: DX
```

Legend: DX - Developer Experience

Decision Summary

DOGFOOD's Blocks docs should show how a block becomes application code, not only how a block is described as metadata. For v7.2.0, the bounded repair is to add an app-binding walkthrough centered on the existing CounterDemoBlock fixture:

- show the block contract: model, config, data requirement, and command intents
- show the application state/update path that owns counter state
- show how keyboard actions become command-intent emissions
- show how the app renders the block from model-owned state
- prove the snippet is not stale by compiling a canonical source file and testing that the rendered docs fence matches it exactly

Sponsored Human

A release-video viewer needs to understand why Blocks are reusable product assemblies rather than screenshots or one-off demos. The Blocks page should let the presenter say "this is how you wire one into your app" without leaving DOGFOOD.

Sponsored Agent

An agent needs a deterministic docs witness that can inspect the Blocks section and verify that the app-binding example names the real API pieces: block config, state ownership, command intents, update routing, render invocation, and lower-mode expectations.

Hill

After this cycle, a developer can open DOGFOOD's Blocks section and see a complete, real app-binding snippet for CounterDemoBlock, including the state and command routing that make the block interactive.

Problem

Issue #342 came from the v7.1.0 release-video rehearsal. The Blocks section now has useful catalog and preview surfaces, but it still reads like a product taxonomy. It does not yet answer the practical developer question:

> How do I bind this block to my application's state and commands?

The current "How to Make Your Own Blocks" page shows block authoring metadata and a minimal defineBlock() shape. It does not show the application side of the contract: TEA state, update routing, render-time config, and command-intent handling.

Scope

- Add a focused app-binding section to the Blocks docs.
- Use CounterDemoBlock as the first example because it already proves bounded state, command intents, interaction keys, and lower-mode output in DOGFOOD.
- Keep the code snippet small enough to read inside the terminal.
- Back the snippet with deterministic tests so API names and command paths do not drift silently.
- Update release-facing signposts and changelog entries for #342.

Non-Goals

- Do not redesign BlockLab.
- Do not build a code editor or tutorial runner.
- Do not pull broad #302 Runtime Graph / Scene IR product-contract work into this cycle.
- Do not add a new production counter block. CounterDemoBlock remains a DOGFOOD fixture.

- Do not rewrite the standard block catalog.

User Experience Contract

The Blocks docs should show both sides of a block contract:

- the reusable block owns metadata, data requirements, command intents, render contract, and lower-mode behavior
- the application owns model state, update decisions, host data, and when to invoke the block

The snippet should be readable in DOGFOOD at normal release-video dimensions. It should not require a browser, external docs page, or source-code search to explain the basic integration path.

Playback Questions

- Can a viewer see where counter state lives?
- Can a viewer see which key emits which command intent?
- Can a viewer see that the block render input is built from app-owned state?
- Can a viewer see how lower modes stay meaningful when interaction is absent?
- Can a test fail if the docs remove the binding path or let the snippet drift away from real APIs?

Accessibility / Assistive Posture

The binding explanation must be useful as text. Code snippets and prose should not rely on color-only emphasis. The lower-mode note should explain what pipe and accessible output preserve from the interactive block.

Localization / Directionality Posture

This cycle adds English-source documentation copy only. DOGFOOD already labels English-source Markdown under non-English locales; this cycle should preserve that behavior. No new right-to-left layout behavior is introduced.

Agent Inspectability / Explainability Posture

Agents should be able to inspect:

- the rendered Blocks docs text
- the exact snippet source text
- whether the snippet references real exported CounterDemoBlock helpers

- whether the snippet names command-intent routing and lower-mode output

The proof should not depend on screenshots.

Linked Invariants

- Blocks are product-level assemblies, not leaf visual components.
- Application state owns business data; block render functions consume explicit inputs.
- Command intents describe user intent; applications interpret and apply those intents.
- DOGFOOD release claims must be demonstrable inside DOGFOOD.

Implementation Outline

1. Add a reusable app-binding snippet source for CounterDemoBlock at examples/docs/content/blocks-counter-app-binding.snippet.ts.
2. Render that exact snippet in examples/docs/content/blocks-make-your-own.md; generated copies must derive from the canonical snippet source.
3. Add prose explaining app-owned state, command intent routing, render-time config, and lower-mode output.
4. Add focused tests that render the Blocks guide and assert the app-binding section is visible and legible.
5. Add compile and source-integrity tests so the snippet cannot drift from the canonical source file or the real helper API.
6. Update docs/CHANGELOG.md, docs/BEARING.md, and docs/ROADMAP.md.

Tests To Write First

- A DOGFOOD Blocks render test fails unless “How to Make Your Own Blocks” includes a visible CounterDemoBlock app-binding section.
- A snippet integrity test fails unless the documented code matches the canonical source file and references the real counter model, config, command-intent, update, and render helpers.
- Existing CounterDemoBlock preview and intent-key tests continue to pass.

Acceptance Criteria

- Blocks docs include at least one complete app-binding snippet.
- The snippet uses real Bijou / DOGFOOD APIs from a canonical source file and is covered by deterministic type and source-integrity proof.
- Interactive block docs explain state ownership and command routing.
- DOGFOOD renders the snippet legibly in the Blocks section.

- The release/video script can explain “how to use this in your app” from the Blocks docs alone.
- Issue #342 is closed by the implementation PR.

Retrospective

To be completed when the PR lands.

142

DX-048 V8 Runtime Graph And Scene IR Contract

docs/design/DX-048-v8-runtime-graph-scene-ir-contract.md

```
title: DX-048 V8 Runtime Graph And Scene IR Contract
legend: DX
lane: release
priority: high
github_issue: 457
status: active
keywords:
  - developer-experience
  - runtime-graph
  - scene-ir
  - graphql
  - blocks
  - visor
  - v8.0.0
```

Legend: DX - Developer Experience

Linked Work

- Goalpost tracker: #457
- First artifact proof: #458
- Packed-cell proof: #459
- Broad source lineage: #302
- VISOR coordination surface: flyingrobots/visor
- Prior Bijou proof: DX-046 GraphQL Authored DOGFOOD Block Fixture

Decision Summary

v7.1.0 proved a constrained path from GraphQL SDL to bijou-block/1, ui-scene-ir/1, terminal proof, grouped debug facts, and DOGFOOD product facts. v7.2.0 stabilized the demo and release surface enough to stop treating that path as release bookkeeping.

The V8 work starts by freezing the product contract before implementation continues. DX-048 defines the source, artifact, IR, receipt, source-map, lower-mode, debug, capture, and witness boundaries that #458 and #459 must consume.

VISOR is the coordination surface for the proof. Bijou remains responsible for the source-side contracts and the terminal Surface proof. Geordi, Wesley,

Bunny, WARP TTD, browser renderers, and native renderers may consume or inform the contract later, but they do not define the first Bijou release gate.

Sponsor Human

A maintainer wants V8 to become a stable product contract, not a pile of exciting demos. They should be able to ask, “what exactly crosses from GraphQL Blocks into Bijou Blocks IR and then into render targets?” and get a versioned, testable answer.

Sponsor Agent

An agent needs to trace one product fact from a GraphQL source fixture through artifact fields, scene nodes, terminal cells, lower modes, source maps, debug facts, and receipts without scraping rendered text or guessing ownership.

Hill

Given a checked-in DOGFOOD GraphQL fixture, Bijou can produce a deterministic artifact bundle with bijou-block/1, ui-scene-ir/1, graphql-bijou-block-debug/1, lower-mode witnesses, source maps, receipts, and packed-cell render evidence, and reviewers can replay the full path without any Wesley or Geordi repository changes.

Playback Questions

- What is the GraphQL Blocks source model allowed to express?
- Which facts belong in the bijou-block/1 artifact and which belong only in the compiler receipt?
- What is the ui-scene-ir/1 lowering contract and what facts must it preserve?
- Who owns receipt and source-map ownership when a source field fans out into multiple scene nodes or terminal cells?
- Which lower modes are mandatory for a fixture before V8 calls it product proof?
- What debug facts are stable enough for DOGFOOD, VISOR, and future replay tooling to inspect?
- What invalid source cases fail before terminal rendering?

Scope

This cycle defines the V8 contract and proof boundaries:

- GraphQL Blocks source model:
  - fixture identity
  - component identity
  - field identity
  - group identity
  - binding and command-intent identity
  - i18n key references
  - theme token references
  - target render profile
- bijou-block/1 artifact semantics:
  - stable artifact version
  - deterministic field and group ordering
  - explicit product-fact inventory
  - hashable payload shape
  - no hidden global registry dependency
- ui-scene-ir/1 lowering contract:
  - stable scene version
  - node identity and role ownership
  - text, style, focus, and geometry facts
  - lower-mode witness fields
  - source-map references back to artifact and source spans
- receipt and source-map ownership:
  - source hash
  - artifact hash
  - scene hash
  - packed-cell hash
  - dependency and transform list
  - failure facts for rejected input
- debug fact contract:
  - graphql-bijou-block-debug/1 summary facts
  - group, field, i18n, token, action, binding, lower-mode, and hash inventory
  - deterministic error categories
- DOGFOOD round-trip fixture:
  - source fixture checked into the repo
  - artifact bundle checked or snapshotted as test evidence
  - terminal Surface proof
  - lower-mode evidence
  - review path documented in DOGFOOD or release-facing docs

Non-Goals

This cycle does not:

- implement the #458 artifact bundle
- implement the #459 packed-cell adapter
- migrate every DOGFOOD block to GraphQL
- introduce a general GraphQL resolver runtime
- require Geordi, Wesley, Bunny, WARP TTD, browser, or native app changes
- ship BlockLab, Theme Lab, localization workbench, or operator surfaces
- define the final browser or native render target APIs
- move all #302 lineage issues into v8.0.0

Contract Sketch

The minimum accepted artifact bundle should be inspectable as plain facts:

```
graphql-source/1
  -> bijou-block/1
    -> ui-scene-ir/1
      -> packed-bijou-cells/1
        -> terminal-surface-proof/1
```

Each transition must produce a receipt:

```
receipt/1 {
  fromVersion
  toVersion
  sourceHash
  outputHash
  dependencyHashes[]
  transformFacts[]
  sourceMapRefs[]
  lowerModeWitnesses[]
  debugFactRefs[]
}
```

The receipt is not a logging nicety. It is the review contract. If a future VISOR, DOGFOOD, WARP TTD, or browser proof cannot name the receipt it consumed, it is not part of the V8 release gate.

Failure Cases

The V8 contract must reject:

- duplicate source, field, group, node, binding, or command ids
- missing component identity
- missing product facts for a DOGFOOD witness
- broken i18n key references
- broken theme token references
- source spans that cannot map back from artifact or scene facts
- lower-mode witnesses that omit required role or text facts

- packed cells that cannot map to scene nodes
- debug summaries that omit hashes or dependency facts

Accessibility Posture

The contract must preserve lower-mode witnesses. Terminal proof alone is not enough. Every accepted DOGFOOD fixture must expose accessible text and role facts through the artifact or scene receipt so reviewers can inspect a non-TTY representation.

Localization / Directionality Posture

The GraphQL source may reference i18n keys and source-language fallback copy, but it must not smuggle untracked visible copy into the artifact. V8 must keep localization keys visible in debug facts and receipts so DOGFOOD can explain what translated and what fell back.

Agent Inspectability / Explainability Posture

Agents must be able to inspect the complete source-to-render chain without executing a full app:

- parse the fixture
- read the artifact
- read the scene IR
- read the receipt list
- read the source map
- read debug facts
- run the terminal proof
- compare hashes

This is the line between a demo and a product contract.

Linked Invariants

- Tests Are the Spec
- Runtime Truth Wins
- Docs Are the Demo
- Work Is Issue-Backed
- Release Claims Need Proof
- Facts Before Behavior

Implementation Outline

1. Land this design packet and post-release BEARING refresh.
2. Update #457 with the contract decision and the #458 / #459 pull order.
3. Add tests that keep ROADMAP, BEARING, and this design aligned.
4. In #458, introduce the artifact bundle fixture and receipt shape.
5. In #459, validate packed-bijou-cells/1 and adapt it to Surface.
6. Only after #458 and #459 are stable, promote broader #302 lineage into V8 implementation scope.

Tests To Write First

- WF-130 roadmap tests must require this design packet while #457 is open.
- Artifact bundle tests in #458 should fail until bijou-block/1, ui-scene-ir/1, receipts, source maps, and debug facts are emitted together.
- Packed-cell tests in #459 should fail until packed-bijou-cells/1 can map cells back to scene nodes and render a Surface proof.

Validation Plan

```
npx vitest run --config vitest.config.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part02.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part03.test.ts
npm run docs:inventory
npm run typecheck:test
npm run lint
git diff --check
```

Closeout Notes

Open. This design is the first V8 pull after v7.2.0 shipped.

143

DX-049 VISOR Artifact Bundle Proof

docs/design/DX-049-visor-artifact-bundle-proof.md

```
title: DX-049 VISOR Artifact Bundle Proof
legend: DX
lane: release
priority: high
github_issue: 458
parent_issue: 457
status: active
keywords:
  - developer-experience
  - visor
  - artifact-bundle
  - graphql
  - blocks
  - scene-ir
  - replay
  - v8.0.0
```

Legend: DX - Developer Experience

Linked Work

- User story: #458
- V8 tracker: #457
- Follow-on packed-cell proof: #459
- Broad source lineage: #302
- Contract design: DX-048 V8 Runtime Graph And Scene IR Contract
- Prior DOGFOOD fixture proof: DX-046 GraphQL Authored DOGFOOD Block Fixture
- VISOR coordination tracker: flyingrobots/visor#1

Decision Summary

DX-046 proved that Bijou can compile a checked-in GraphQL SDL fixture into a grouped bijou-block/1 artifact, lower it into ui-scene-ir/1, render a terminal proof, and emit deterministic graphql-bijou-block-debug/1 facts. DX-048 then defined the V8 contract boundary that VISOR needs to coordinate across Bijou, WARP TTD, Geordi, Bunny, Wesley, and follow-on render targets.

Issue #458 should be the smallest implementation bridge between those two facts. It should introduce a deterministic visor-artifact-bundle/1 object that wraps the existing GraphQL fixture source, bijou-block/1 artifact, ui-sc

ene-ir/1 scene, debug summary, replay metadata, visual scene facts, and stable hashes. The bundle is a Bijou-emitted proof artifact for VISOR to index; it is not a new VISOR runtime dependency inside Bijou.

Sponsored Human

A maintainer wants one inspectable artifact that says, “this exact GraphQL fixture produced this exact block artifact, scene IR, debug summary, replay identity, and visual fact set.” They should not need to reconstruct the proof from scattered test helpers or terminal screenshots.

Sponsored Agent

An agent wants to load one JSON-shaped bundle and trace a source field through artifact facts, scene nodes, debug facts, replay metadata, visual target facts, and stable hashes without executing DOGFOOD or scraping terminal output.

Hill

Given the checked-in DOGFOOD GraphQL fixture, Bijou can emit a deterministic visor-artifact-bundle/1 whose source, normalized bijou-block/1 facts, ui-scen e-ir/1 facts, graphql-bijou-block-debug/1 facts, replay metadata, visual scene facts, and hash inventory are stable across runs and small enough for VISOR, DOGFOOD, and future WARP TTD tooling to inspect.

Playback Questions

- Which GraphQL fixture does the bundle represent?
- Which bijou-block/1 artifact and ui-scene-ir/1 scene are carried by the bundle?
- Which hashes prove the source, artifact, scene, debug summary, replay facts, visual scene facts, and full bundle?
- Which replay identity can future debugger tooling use without depending on host-local paths, timestamps, process ids, or terminal timing?
- Which visual scene facts can VISOR inspect before #459 adds packed-cell validation?
- Can a whitespace-only SDL edit preserve the artifact, scene, debug, visual, and bundle hashes where semantics do not change?
- Can malformed source or broken lowering assumptions fail before a bundle is emitted?

Scope

- Add a public visor-artifact-bundle/1 fact shape in Bijou.
- Build the first bundle from the existing constrained GraphQL block compiler path.
- Reuse the checked-in DOGFOOD GraphQL fixture unless implementation finds a smaller fixture that still proves the same source-to-scene path.
- Carry the fixture source name, semantic source hash, normalized source hash, bijou-block/1 artifact, ui-scene-ir/1 scene, graphql-bijou-block-debug/1 summary, replay metadata, visual scene facts, and stable hashes.
- Keep the bundle JSON-serializable, deterministic, and free of absolute local paths, timestamps, process ids, screenshots, and operator terminal state.
- Add focused tests for deterministic output, hash stability, inspectability, and failure cases.

Non-Goals

This cycle does not:

- implement #459 packed-cell-to-Surface validation
- implement WARP TTD browser UI or debugger sessions
- implement Geordi browser, image, or packed-cell endpoints
- require Bunny graphics, raycast, mesh, camera, shader, or geometry work
- require Wesley DTO generation or a general GraphQL compiler
- generalize every V8 schema shape
- make VISOR a runtime package dependency of Bijou
- add screenshot, video, or frame-capture file formats
- migrate additional DOGFOOD surfaces to GraphQL

Bundle Contract

The first accepted bundle should be plain data:

```
visor-artifact-bundle/1
  bundleVersion
  fixture
    id
    sourceName
    sourceHash
    normalizedSourceHash
  source
    language
    text
  artifacts
    bijouBlock
    uiScene
    debugSummary
  replay
    replayVersion
    scenarioId
    fixtureId
```

```
    deterministicSeed
    steps[]
  visual
    visualFactsVersion
    sceneId
    rootNodeId
    targetProfiles[]
    nodeFacts[]
    lowerModes[]
  hashes
    sourceHash
    normalizedSourceHash
    artifactHash
    sceneHash
    debugSummaryHash
    replayHash
    visualFactsHash
    bundleHash
  receipts[]
```

sourceHash records the raw source text hash. normalizedSourceHash records the semantic source hash inherited from the compiled bijou-block/1 artifact. bundleHash is computed from the canonical semantic bundle payload with raw source.text, raw sourceHash copies, and bundleHash itself omitted. That lets VISOR inspect exact source text and raw source identity while preserving a stable semantic bundle hash across whitespace-only SDL edits. Hashing must use the same deterministic JSON rules already used by the GraphQL block and scene IR proof path.

Replay Metadata Contract

Replay metadata is identity and intent, not a full debugger session. The first shape should record:

- stable scenario id
- fixture id
- deterministic seed or explicit no-randomness marker
- source, artifact, scene, debug, and visual hashes consumed by the replay
- ordered steps such as compile source, lower artifact, summarize debug facts, and expose visual facts

It must not record wall-clock timestamps, process ids, absolute paths, terminal dimensions from the operator environment, or browser state. WARP TTD may later load this metadata, but WARP TTD does not define the #458 shape.

Visual Scene Facts Contract

Visual scene facts summarize the ui-scene-ir/1 output for inspection. They are not Geordi render receipts and not #459 packed-cell facts. The first shape should expose:

- scene id and root node id
- target profile inventory
- node ids, roles, kinds, text facts, group ownership, and source references

- token refs, i18n keys, action ids, and binding ids
- lower-mode hashes and labels

That is enough for VISOR to prove what Bijou thinks the visual scene is before future render targets prove how they draw it.

Failure Cases

The bundle builder must reject:

- missing source name or fixture id
- source names that are absolute paths
- missing bijou-block/1, ui-scene-ir/1, or debug summary facts
- artifact, scene, debug, replay, or visual hash mismatches
- duplicate visual node ids
- lower-mode inventories that omit required node, i18n, or token witnesses
- replay metadata that contains host-local or wall-clock facts
- malformed source that the GraphQL block compiler already rejects

Accessibility And Assistive Posture

The bundle must preserve lower-mode witnesses and semantic roles so the proof is inspectable without color, terminal glyphs, or screenshots. Visual facts must name text, node ids, i18n keys, token refs, and source refs independently of the normal terminal rendering path.

Localization And Directionality Posture

Renderable text continues to come from GraphQL @bijouI18n keys plus fallback copy. The bundle must keep i18n keys and fallbacks visible in artifact, scene, debug, and visual facts. Directionality remains deferred until V8 supports localized layout constraints; this cycle must not hide that deferral.

Agent Inspectability And Explainability Posture

The useful inspection path becomes:

```
GraphQL fixture
  -> visor-artifact-bundle/1.fixture
  -> visor-artifact-bundle/1.artifacts.bijouBlock
  -> visor-artifact-bundle/1.artifacts.uiScene
  -> visor-artifact-bundle/1.artifacts.debugSummary
  -> visor-artifact-bundle/1.visual
  -> visor-artifact-bundle/1.replay
  -> visor-artifact-bundle/1.hashes
```

An agent should be able to answer, from the bundle alone:

- Which source field produced this node?
- Which i18n key and token refs belong to this text?
- Which action or binding facts are attached?
- Which target profile was declared?
- Which hash proves the source-to-scene chain?
- Which facts are deliberately not proven until #459 or a later external endpoint cycle?

Linked Invariants

- Schemas Live At Boundaries: malformed source must fail before Bijou emits a bundle.
- Graceful Lowering Preserves Meaning: bundle facts must preserve node identity, text, token refs, i18n keys, actions, bindings, and lower-mode meaning.
- The Buffer Holds Facts: the bundle is serializable data, not behavior.
- Tests Are the Spec: #458 is complete only when focused tests prove the deterministic bundle contract.

Implementation Outline

1. Land this design packet and link it from roadmap, bearing, changelog, issue, and PR evidence.
2. Add RED tests for a missing visor-artifact-bundle/1 builder and exported bundle type.
3. Add the bundle contract and deterministic canonical hashing.
4. Build the first bundle from the existing DOGFOOD GraphQL fixture, compileGraphqlBijouBlock(), lowerBijouBlockToUiScene(), and createGraphqlBijouBlockDebugSummary().
5. Add replay metadata and visual scene fact extraction without introducing WARP TTD, Geordi, Bunny, Wesley, browser, or VISOR runtime dependencies.
6. Add failure coverage for host-local facts, absolute source names, duplicate visual ids, missing lower-mode witnesses, and hash mismatches.
7. Update docs/CHANGELOG.md, issue evidence, and PR evidence with the real implementation and validation commands.

Tests To Write First

- RED: importing the bundle builder from @flyingrobots/bijou should fail before the implementation exists.
- RED: building from the checked-in DOGFOOD GraphQL fixture should produce bundleVersion: "visor-artifact-bundle/1" and carry bijou-block/1, ui-scene-ir/1, and graphql-bijou-block-debug/1 artifacts.

- RED: the bundle hash should be stable across repeated runs and whitespace-only SDL edits that preserve semantic facts.
- RED: visual facts should list scene id, root node id, node ids, i18n keys, token refs, action ids, binding ids, target profiles, and lower-mode hashes.
- RED: replay metadata should carry stable fixture and scenario identity without timestamps, absolute paths, process ids, or live terminal dimensions.
- RED: malformed fixtures, absolute source names, hash mismatches, duplicate visual ids, and missing lower-mode witnesses should fail before bundle output.

Validation Plan

Current implementation validation:

```
npx vitest run --config vitest.config.ts tests/cycles/DX-049
npx vitest run --config vitest.config.ts \
  tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts \
  tests/cycles/WF-130/roadmap-goalpost-policy.part02.test.ts \
  tests/cycles/WF-130/roadmap-goalpost-policy.part04.test.ts
npm run docs:inventory
npm run typecheck:test
npm run code-dojo:changed
npm run lint
git diff --check
```

Closeout Notes

Implementation in progress. The design PR has landed, so #458 no longer needs needs-design. The implementation branch now carries focused tests/cycles/DX-049 coverage for the public bundle builder, exported bundle type, raw-versus-semantic source hashes, replay metadata, visual scene facts, absolute source-name rejection, malformed source rejection, duplicate visual node rejection, and hash-mismatch rejection. Keep work-in-progress on the issue while the branch and PR carry the implementation cycle.

144

DX-050-profunctor-page-inspection

docs/design/DX-050-profunctor-page-inspection.md

```
title: DX-050 Profunctor Page Inspection
legend: DX
lane: release
priority: high
github_issue: 468
status: complete
keywords:
  - developer-experience
  - profunctor-page
  - scene-ir
  - target-inspection
  - v10.0.0
```

Legend: DX - Developer Experience

Linked Work

- Bijou story: #468
- Implementation pull request: #474
- Website campaign: profunctor-optics-website#14
- Canonical website specimen revision: 6a411d7
- Sibling Geordi proof: geordi#52 and geordi#53
- Code Dojo prerequisite: WF-163

Decision Summary

Bijou will consume byte-identical copies of the canonical Keep profunctor-page/0, source map, and build manifest. It will validate their versions, identities, references, and pinned digests before lowering the page through a Bijou-owned ui-scene-ir/1, layout, and Surface path.

The website remains authoritative for page composition and source occurrences. Bijou owns terminal render identities, inspection facts, residuals, and its receipt. Geordi is sibling evidence, not a package dependency or authority.

Current Truth

Before DX-050, Bijou could lower Bijou-owned scenes and blocks to terminal surfaces, but it could not validate, inspect, or lower the website-owned profunctor-page/0 family. The canonical ProjectPage specimen existed only in the website repository at revision 6a411d7; Bijou had no pinned copy, page target profile, cross-artifact validation, source-to-render map, target receipt, or terminal witness for that family.

Acceptance Criteria

- Pin byte-identical profunctor-page/0, profunctor-page-source-map/0, and profunctor-build-manifest/0 inputs to website revision 6a411d7 with exact SHA-256 digests.
- Validate versions, identities, references, dependency digests, root reachability, reading order, and target capabilities before lowering.
- Reject visible unsupported blocks and obstructed or clipped output with a stable diagnostic and no target artifact.
- Residualize only hidden unsupported blocks, with the unsupported block identity preserved in the target map.
- Emit deterministic scene, target-map, receipt, and terminal-witness bytes through the existing layout and Surface contracts.
- Preserve page, template, source-occurrence, token, reading-order, landmark, and action facts across all supported inspection modes.
- Hold the Code Dojo debt ceiling at 62 aggregate violations or lower and pass the focused contract, generation, documentation, and full repository gates.

Sponsor Human

James Ross.

Sponsor Agent

Codex.

Hill

Given the canonical Keep specimen, Bijou emits deterministic terminal inspection evidence that preserves page, template, source-occurrence, token, reading-order, and link identities; assigns every supported visible node a Bijou render identity; residualizes hidden unsupported nodes; and rejects malformed, contradictory, or visibly unsupported inputs.

Playback Questions

1. Which exact website revision and input digests produced this proof?
2. Where does each page node appear in ui-scene-ir/1 and Surface output?
3. Which token, reading-order, source, and action facts remain inspectable?
4. Which upstream capabilities does Bijou refuse rather than inherit?
5. Does repeated lowering emit byte-identical artifacts and receipts?

Scope

- Vendor the three canonical inputs with provenance and SHA-256 pins.
- Parse and validate the version-zero artifact family fail closed.
- Lower supported ProjectPage blocks to one deterministic ui-scene-ir/1.
- Adapt the scene through existing layout and Surface contracts.
- Emit a Bijou target map, receipt, and inspectable terminal witness.
- Preserve source-map identities without mutating compiler-owned occurrences.
- Reduce Code Dojo debt to 62 aggregate violations or lower.

Non-Goals

- No website runtime or production GraphQL server.
- No mutation of authored page, source-map, or build-manifest authority.
- No Geordi dependency, pixel parity, browser semantics, or HTML claims.
- No general page-family vocabulary or /0 to /1 promotion.
- No unsupported-block approximation or silent capability inheritance.
- No redesign of DOGFOOD, the website, or existing Surface behavior.

Target Contract

The target accepts exact UTF-8 bytes for profunctor-page/0, profunctor-page-source-map/0, and profunctor-build-manifest/0. It returns Bijou-owned scene, target-map, receipt, and terminal-witness artifacts or one stable diagnostic with a code, input path, and detail.

Supported visible page nodes receive injective render identities. Hidden unsupported nodes receive explicit residuals. Visible unsupported nodes fail closed without target artifacts. Target output records upstream claims as not inherited and binds every output digest to the exact input family.

Accessibility And Assistive Posture

The terminal witness preserves text, semantic roles, reading order, link destinations, and residual explanations without relying on color alone. It does not claim browser landmarks, accessibility-tree parity, or native links.

Localization And Directionality Posture

Input text and order are preserved exactly. Token roles remain references, not resolved theme guesses. No new translation authority or bidirectional layout claim is introduced; unsupported direction-sensitive behavior is residualized.

Agent Inspectability And Explainability Posture

An agent can trace input digest to page node, template node, source occurrence, scene node, terminal region, token references, action destination, and receipt without scraping screenshots or executing the website.

Linked Invariants

- Canonical bytes stay owned by the website at revision 6a411d7.
- Every supported visible page node maps to one target identity.
- Every hidden unsupported page node maps to one explicit residual.
- Every visible unsupported page node fails closed before output.
- Render identities are injective and target-owned.
- Reading order, tokens, actions, and source occurrences remain inspectable.
- Unsupported versions, references, blocks, or claims fail closed.
- Generated outputs are deterministic and inventory checked.
- The Code Dojo ceiling falls from 112 to 62 or lower.

Implementation Outline

1. Copy and pin the canonical three-file specimen.
2. Add contract types, parsers, and stable diagnostics.
3. Add target-profile validation and explicit residuals.
4. Lower supported nodes into ui-scene-ir/1.
5. Render through layout and Surface without host timing or paths.
6. Emit and verify target map, receipt, and witness bytes.
7. Remove at least 50 measured Code Dojo violations.
8. Update living docs, artifact inventory, and closeout evidence.

Tests To Write First

- Reject malformed JSON, unsupported versions, digest drift, and identity drift.
- Reject duplicate nodes, invalid slots, references, tokens, and actions.
- Prove every supported visible node maps injectively.
- Prove hidden unsupported nodes residualize and visible unsupported nodes fail closed.
- Prove exact reading order, token, action, and source-occurrence preservation.
- Prove repeated output and receipts are byte-identical.
- Prove fixture inventory, website revision, and all three hashes are pinned.
- Prove layout and Surface output retain the declared inspection facts.

Retrospective And Closeout

DX-050 emits page.bijou.scene.json, page.bijou.map.json, page.bijou.receipt.json, and page.bijou.txt. The public failure boundary is the seven-code BIJOU_PAGE_* diagnostic family documented in the inspection reference.

The canonical inputs remain pinned to website revision 6a411d7:

- Page SHA-256: 12b90184b4e238bc6bd7db944af8651900b830ea76ba775ab7c7ee20be051e73.
- Source-map SHA-256: a40e97f2fd513519092d2a131c0d7107517b92bd7e9cfd5e79df70a65a3fe044.
- Build-manifest SHA-256: 975efc9c701cbd7a6981787150748b76d1e53fd83cd424128667bc8014687ab7.

The bounded target supports the five ProjectPage block definitions. Hidden unsupported blocks become hidden-unsupported-block residuals. Visible unsupported blocks, semantic-document sources, application islands, inherited semantic-HTML claims, and output that exceeds the fixed terminal profile fail closed or remain explicit target obstructions.

Focused contract evidence passes 38 tests across the five target suites, the generator suite, and this design contract. Full local verification passes 4,015 tests. Code Dojo reports 62 aggregate violations: 37 file/context, 25 code-size, 0 mock-ban, and 0 ESLint.

The follow-on path remains the v8.0.0 Runtime Graph And Scene IR milestone, the website publication campaign in profunctor-optics-website#14, and the sibling Geordi proof in geordi#52 and geordi#53.

145

DX-052 NO_COLOR Is Not a Capability

docs/design/DX-052-no-color-is-not-a-capability.md

```
title: DX-052 NO_COLOR Is Not a Capability
legend: DX
lane: release
priority: high
github_issue: 518
status: landed
keywords:
  - no-color
  - output-mode
  - detection
  - theme
  - accessors
  - fallback
  - extend-theme
  - breaking-change
  - v8.0.0
```

Problem

Three defects found by an application (muniment) that had to work around all three locally before it could ship a readable palette. Each one is a case of bijou answering a question the caller did not ask.

1. NO_COLOR takes the whole UI down, not just the colour

detectOutputMode (packages/bijou/src/core/detect/tty.ts) resolves in this order:

```
BIJOU_ACCESSIBLE=1    -> 'accessible'
NO_COLOR / TERM=dumb  -> 'pipe'
!stdout.isTTY         -> 'pipe'
CI                    -> 'static'
stdout.isTTY          -> 'interactive'
```

The second line conflates two unrelated requests. no-color.org defines NO_COLOR as a statement about **colour**: “when present, regardless of its value, prevents the addition of ANSI color”. It says nothing about cursor addressing, alternate screens, or interactivity. bijou reads it as a statement about **capability**, so a user who exports NO_COLOR=1 in their shell profile — a common, reasonable thing to do — does not get a monochrome TUI. They get no TUI: ctx.mode becomes ’pipe’, every component lowers to its plain single-frame form, and shouldUseShellQuitConfirm starts returning ’immediate’.

Nothing is gained by the conflation, because **colour is already suppressed by a separate path** — three of them, in fact, each reading NO_COLOR directly and none consulting mode:

Site	Line	Effect
packages/bijou/src/factory.ts	54	noColor on the resolved theme
packages/bijou-node/src/node-context.ts	23	chalkStyle({ noColor, level: 0 })
packages/bijou/src/core/theme/resolve.ts	20	isNoColor()

So removing the line removes the interactivity damage while every **colour and style** escape stays suppressed. Not every escape: an interactive session by definition emits cursor movement, screen updates and possibly alternate-screen control sequences, and it should — a monochrome TUI is still a TUI. NO_COLOR asks for no colour, not for no terminal control. The two concerns were never coupled in the implementation; only in the detector.

git-cas (bin/ui/context.js) already ships a detectCliTuiMode override for exactly this reason, with a comment saying so. When two first-party consumers independently re-implement a detector, the detector is wrong.

2. An unknown status key renders struck through

createThemeAccessors (packages/bijou/src/core/theme/accessors.ts):

```
status: (key) => {
  try { return tokenGraph.get(`status.${key}`, mode); }
  catch { return tokenGraph.get('status.muted', mode); }
},
```

status.muted carries ['dim', 'strikethrough'] in **every** shipped preset — bijou-dark, bijou-light, cyan-magenta, nord, catppuccin, teal-orange-pink. That is a defensible token: "muted" as in retired, done, cancelled. It is the wrong thing to hand back for a key nobody defined.

status(key) takes string, not a key union, so the failure is silent and total: a typo, or a key the app forgot to add to its theme, produces text with a line through it. A reader sees "cancelled". The app said nothing of the kind. The sibling accessor already does the right thing — ui() falls back to semantic.primary, a plain token in another group — so the fix is to make status() consistent with it and fall back to semantic.muted, which is dim and nothing else in every preset.

Found in the field: an app declared a paint.unreviewed() helper, dropped the matching status key from its theme as a duplicate, and shipped a painter that resolved to the strikethrough fallback. No check caught it, because no view had called it yet. inkStatus() in resolve.ts has the same fallback and the same problem.

3. extendTheme cannot reach border or semantic

```
export declare function extendTheme<S, U, G>(base: Theme, extensions: {
  status?: ...; ui?: ...; gradient?: ...; surface?: ...;
}): Theme<...>
```

Theme has six token groups. extendTheme can extend four. There is no way to override a border or semantic token through it, which matters because the shipped themes alias heavily across groups: bijou-dark uses #f2c45d for statu s.warning, status.active, semantic.warning, semantic.accent, ui.cursor, ui.fo cusGutter, ui.sectionHeader, ui.logo, border.secondary **and** border.warning — ten aliases of one amber.

An app that calls extendTheme(BIJOU_DARK, { status }) therefore replaces its statuses and silently keeps bijou's amber for every heading, cursor and border. In the case that prompted this, the app's section headings landed ΔE **0.023** from its own warning in OKLab — indistinguishable — and the only escape was to abandon extendTheme and author all six groups by hand.

Non-goals

- **Changing status.muted itself.** Its strikethrough is deliberate and used. Only the *fallback target* moves.
- **Re-fitting the shipped palettes.** The #f2c45d ten-way alias and bijou-dark's error/success collapse under protanopia (ΔE 0.059) are real, are what motivated non-goal-adjacent issues, and are filed separately.
- **Adding OKLCH authoring or a CVD-aware palette audit to bijou.** Also filed separately; doctorTheme covers contrast and reuse but not perceptual separation under simulated colour-vision deficiency.
- **Typing status(key) to a key union.** It would catch defect 2 at compile time for typed callers, but it is a wider API change than this cycle wants and would not help JS consumers. Filed as follow-up.

Playback questions

1. With NO_COLOR=1 and a real TTY on both stdio, is ctx.mode 'interactive'?
2. With NO_COLOR=1, is theme.noColor still true and does styled() still return unstyled text? And with NO_COLOR="" — the variable is presence-only, so an implementation that special-cases '1' must not pass.
3. Does TERM=dumb still yield 'pipe'? Does a non-TTY stdout? Does CI still yield 'static', and BIJOU_ACCESSIBLE=1 still win outright?
4. Does status('nonexistent') return a token with no strikethrough?
5. Does status('muted') still return the real muted token, strikethrough intact?
6. Does inkStatus('nonexistent') agree with status() about the fallback?
7. Does extendTheme(base, { border, semantic }) merge those groups, preserve unlisted keys within them, and leave the other four groups untouched?

Decision

1. Delete the NO_COLOR branch from detectOutputMode. Update its docblock and the OutputMode doc comment, which both currently document the old order.

2. Point the status() accessor and inkStatus() fallbacks at semantic.muted.
3. Add optional border and semantic to extendTheme's extension parameter.

Breaking-change note

1 and 2 change observable behaviour that the current docblocks describe, so this lands as part of **8.0.0** rather than a minor.

Eight existing tests across five files assert the behaviour being removed. Six of them encode the NO_COLOR → 'pipe' mapping alone:

File	Test
detect/tty.test.ts	'returns pipe when NO_COLOR is set'
detect/tty.test.ts	'NO_COLOR takes priority over CI'
detect/tty.test.ts	'BIJOU_ACCESSIBLE takes priority over NO_COLOR'
detect/tty.fuzz.test.ts	'NO_COLOR always results in pipe or accessible mode'
environment.part01.test.ts	'NO_COLOR set -> pipe mode regardless of TTY'
environment.part01.test.ts	'NO_COLOR + TTY still produces pipe mode'
theme/accessors.test.ts	'status() falls back to muted for unknown keys'
theme/resolve.part02.test.ts	'inkStatus() falls back to muted hex for unknown status'

Six tests reinforcing a rule usually means the rule was chosen deliberately. It was not. Every one is a characterization test restating the truth table, sitting in describe blocks named "detection logic" and "conflicting env vars", and no comment, docblock, or design note in the repository argues *why* NO_COLOR should imply non-interactive — the only justification is the detector's own docblock restating its own order. The tests are thorough about coverage, not about defending the choice.

They are not wrong about what the code did; they are the specification of the defect. They are rewritten to assert the new contract, and this section exists so that the rewrite is a recorded decision rather than tests quietly edited to make a build go green. Two of them change shape rather than expectation:

- 'BIJOU_ACCESSIBLE takes priority over NO_COLOR' becomes vacuous once NO_COLOR no longer affects mode, and is reframed as priority over an interactive TTY.
- the fuzz property is strengthened rather than adjusted. Instead of asserting which mode results, it now asserts that adding NO_COLOR to *any* environment leaves that environment's mode unchanged — the contract stated directly.

resolve.part02.test.ts is the one case with no observable behaviour change: inkStatus() returns a hex, and status.muted and semantic.muted share a hex in every shipped preset. It is updated anyway so the two fallbacks cannot disagree in a theme where the hexes differ.

3 is additive and breaks nothing.

Acceptance criteria

- All seven playback questions have a passing test.
- npm run lint, the full test suite, and the Code Dojo gates pass.
- docs/CHANGELOG.md records all three under a Changed heading with the breaking note, and docs/BEARING.md mentions the detection-semantics change.
- No consumer-visible colour output changes when NO_COLOR is set.

146

DX-125 - Day Zero Onboarding Bridge

docs/design/DX-125-day-zero-onboarding-bridge.md

Linked issues: #125, #130, #131, #132, #154, #155, #165, #166

Linked legends:

- DX - Developer Experience
- DF - DOGFOOD Field Guide

Sponsor Human

A first-time TypeScript reader who wants to see useful output quickly, then learn the next app shape without reverse-engineering DOGFOOD.

Sponsor Agent

A code-generation agent that needs public docs to explain package boundaries, Surface/TEA/Cmd concepts, render tradeoffs, and the right next guide without hidden repo memory.

Hill

The README moves from first visible output to a small framed app path before asking readers to absorb the full architecture or DOGFOOD implementation.

Scope

- Move GUIDE and documentation map discovery near the top of README.
- Explain the first app imports by package boundary.
- Stage Surface, App, Msg, Cmd, and framed-shell concepts.
- Add npm run app-frame as an intermediate command.
- Add a framed app tutorial that teaches header, status, navigation, detail, overlay, and shell commands with the compact existing example.

- Document package split and render-pipeline tradeoffs where readers encounter those concepts.

Playback Questions

1. Can a reader reach visible output from the README without first reading architecture docs?
2. Can the same reader find GUIDE and the docs map before scrolling deep into the README?
3. Is there a clear path from counter to a framed app before DOGFOOD?
4. Do docs explain why package imports are split?

Accessibility / Assistive Reading Posture

The bridge uses short sections, tables, and command blocks so readers can scan for the next action. It avoids requiring long prose before the first command.

Localization / Directionality Posture

No runtime localization changes. Documentation text remains English.

Agent Inspectability / Explainability Posture

The public README, GUIDE, docs map, package development guide, and render pipeline guide carry the explanation agents need to generate correct examples.

Linked Invariants

- First value should appear before deep architecture.
- DOGFOOD remains the canonical docs app, but it is not the first learning step.
- Package boundaries should be explained at the point of first import.

Decisions

- Keep npm run hello as the lowest-friction first visible output.
- Keep npm run counter as the first interactive app.
- Add npm run app-frame between counter and DOGFOOD.
- Do not make DOGFOOD smaller; teach the bridge before DOGFOOD.

Verification

- npm run docs:inventory
- npm test -- --run scripts/docs-preview.test.ts
- manual playback of npm run hello, npm run counter, and npm run app-frame when validating the final PR.

Tests To Write First

- Docs preview coverage for changed docs surfaces.
- Docs inventory for new guide links.

Retrospective

See WF-122 day zero audit fixes.

Runtime engine design records

These records concern the machinery behind a frame: state and effects, surfaces and packed cells, layout, input routing, host boundaries, rendering stages and scheduling. Trace a proposed optimization through the invariants it must preserve. A measured gain is most useful when the record also identifies its workload, revision and correctness checks.

147

RE-001 — Define Runtime Engine Architecture

docs/design/RE-001-define-runtime-engine-architecture.md

Cycle for formalizing Bijou's hexagonal runtime around state machines, view stacks, retained layouts, and buffered command/effect routing

Legend:

- RE — Runtime Engine

Depends on:

- HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection
- DL-009 — Formalize Layout and Viewport Rules

Why this cycle exists

Recent shell and DOGFOOD work exposed that Bijou still mixes several concerns together:

- stateful application logic
- visible shell/view layering
- layout ownership
- input routing
- side effects

That mixing was survivable while the framework was smaller, but it is now making the architecture harder to explain than it should be.

This cycle exists to write down the intended engine model before the implementation drifts further:

- the application owns a state machine
- the application owns a view stack
- the runtime retains layouts for those views
- input is routed through those retained layouts from the topmost view downward
- components may emit commands and effects
- commands and effects are buffered separately
- terminal, clock, audio, filesystem, and other integrations stay outside the hexagon as adapters

Human users / jobs / hills

Primary human users

- framework maintainers evolving the runtime
- builders composing real applications with multiple overlapping surfaces
- shell and DOGFOOD authors who need predictable interaction and layout rules

Human jobs

1. Explain whether a behavior belongs to state, view stack, layout, routing, command handling, or an adapter.
2. Add a new modal, drawer, or workspace view without rewriting routing rules from scratch.
3. Resize the terminal or change a component's content without letting geometry and interaction ownership drift apart.

Human hill

The runtime model is explicit enough that a human can describe how one input becomes zero-or-more commands and effects without hand-waving over shell-specific branch order.

Agent users / jobs / hills

Primary agent users

- agents generating Bijou apps
- agents inspecting runtime state during test/debug loops
- agents validating whether routing and effects match the visible UI

Agent jobs

1. Inspect state machine state separately from the current view stack.
2. Read retained layout ownership and predict which view/component should receive a pointer or key event.
3. Distinguish between commands that change state and effects that leave state alone.

Agent hill

An agent can inspect the runtime model and answer what is visible, what is interactive, what blocks lower layers, and what commands/effects an input may emit without parsing ANSI output heuristically.

Human playback

1. A user is in a workspace view with an active session state.
2. The app pushes a modal confirmation view onto the view stack without destroying the underlying workspace state.
3. The modal's retained layout is placed above the workspace layout and blocks lower interaction.
4. The user clicks the modal's confirm button.
5. The input system routes the click to the topmost view's layout.
6. The button emits:
   - a command to continue the operation
   - an effect to play a click sound
7. The runtime buffers the command and effect separately.
8. The command updates app state and may pop or replace views.
9. The effect runs through an adapter without pretending it changed state.

Agent playback

1. An agent inspects the runtime while a search palette sits above a workspace view.
2. The runtime exposes:
   - current application state
   - ordered view stack
   - retained layouts for each active view
3. The agent can see that the search view blocks lower interaction.
4. A key input enters the runtime.
5. The runtime routes it through the search view's input path first.
6. The search view emits one command to select a result and one effect to announce the match count.
7. The workspace view never receives the key because the upper view handled it and blocks lower layers.

Linked invariants

- Focus Owns Input
- Topmost Layer Dismisses First
- State Machine and View Stack Are Distinct
- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Tests Are the Spec

Implementation outline

1. Formalize core runtime nouns:
    - ApplicationStateMachine
    - ViewStack
    - ViewLayer
    - RetainedLayoutTree
    - InputEvent
    - Command
    - Effect
2. Promote the view stack into first-class runtime state instead of deriving it from booleans and shell-specific branches.
3. Move layout ownership to retained runtime objects that are invalidated on:
    - terminal resize
    - view push/pop
    - content-size changes
    - visibility/enablement changes
    - scrollbar/overflow changes
4. Route input against retained layouts, topmost view first, with explicit block/pass-through semantics.
5. Let components contribute interaction behavior through layout-owned nodes instead of global registration churn.
6. Buffer emitted commands separately from emitted effects.
7. Migrate existing framed-shell behavior onto those seams once the contracts are explicit.

Tests to write first

- cycle test proving the legend, invariants, active design doc, and follow-on backlog items all describe the same runtime model
- future runtime unit tests proving:
    - one input may emit multiple commands
    - one input may emit multiple effects
    - commands and effects stay distinct
    - topmost blocking views prevent lower layouts from seeing the same input
    - retained layouts update interaction geometry when invalidated

Risks / unknowns

- Bijou's current TEA-style update flow may need a transitional compatibility layer while commands/effects become buffered first-class runtime outputs.
- Existing framed-shell code still derives layers from booleans instead of owning a first-class view stack.

- Layout invalidation boundaries may need careful tuning so the retained model is simpler than per-render recomputation rather than more confusing.
- Some existing components may not yet have explicit enough layout contracts to participate cleanly in layout-driven routing.

Retrospective

What landed:

- a new RE — Runtime Engine legend
- explicit invariants for state/view separation, layout-owned interaction geometry, and command/effect separation
- this cycle doc capturing the runtime model as the design source of truth
- a decomposition backlog for implementation slices

What did not land:

- no runtime code changes yet
- no new engine objects yet
- no shell migration yet

Follow-on:

- RE-002 — Promote First-Class State Machine and View Stack
- RE-003 — Retain Layout Trees and Layout Invalidation
- RE-004 — Route Input Through Layouts and Layer Bubbling
- RE-005 — Buffer Commands and Effects Separately
- RE-006 — Formalize Component Layout and Interaction Contracts
- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

148

RE-002 — Promote First-Class State Machine and View Stack

docs/design/RE-002-promote-first-class-state-machine-and-view-stack.md

Cycle for turning state machines and view stacks into explicit runtime objects instead of thin shell conventions

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture

Why this cycle exists

RE-001 defined the runtime direction:

- application state and visible views are distinct
- views/layers should be first-class runtime objects
- view-stack behavior should not be derived ad hoc from shell booleans

This cycle exists to land the first real implementation slice behind that doctrine:

- a first-class state-machine object
- a first-class view-stack object
- explicit root/base-view behavior
- explicit push/pop/replace/clear semantics

This is intentionally smaller than a shell migration. It gives the runtime a reusable foundation before routing, retained-layout invalidation, and command/effect buffering are moved onto it.

Human users / jobs / hills

Primary human users

- framework maintainers evolving the runtime
- builders writing layered applications with modals, drawers, and overlays
- shell authors who need something more honest than booleans plus branch order

Human jobs

1. Represent durable application state separately from ephemeral or layered UI.
2. Push and dismiss views without pretending those operations are the same as state transitions.
3. Replace or clear visible views intentionally when a state transition requires it.

Human hill

The first-class runtime objects are explicit enough that a human can point to one place for “current app state” and one place for “visible view stack” without hand-waving.

Agent users / jobs / hills

Primary agent users

- agents inspecting runtime objects
- agents generating tests against stack/state behavior
- agents authoring layered apps on top of Bijou

Agent jobs

1. Read the current state machine without inferring it from visible shell layers.
2. Read the current view stack without inferring it from miscellaneous booleans.
3. Determine whether a top layer can be dismissed, whether it blocks below, and what the root/base view is.

Agent hill

An agent can inspect the runtime objects and answer “what state is the app in?” separately from “what views are currently visible?” reliably enough to generate and validate layered behavior.

Human playback

1. A user is in an active workspace session.
2. The app's state machine says session.active.
3. The view stack contains only the non-dismissible workspace root view.
4. The app pushes a dismissible blocking confirmation view.
5. The user dismisses it.
6. The state machine stays on session.active.
7. The view stack pops back to only the root view.
8. Later, the app transitions to session.ended and intentionally clears auxiliary views while replacing the base view with a post-session summary.

Agent playback

1. An agent inspects runtime objects before and after a modal opens.
2. Before:
   - state machine current state is docs.browsing
   - view stack contains one root view
3. After:
   - state machine still reports docs.browsing
   - view stack contains root + modal
4. The agent can see that the modal is dismissible and blocking while the root is not dismissible.
5. When the modal is dismissed, the state machine remains unchanged while the view stack changes.

Linked invariants

- State Machine and View Stack Are Distinct
- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Tests Are the Spec

Implementation outline

1. Add a pure runtime module in bijou-tui that exports:
   - a first-class state-machine type
   - a first-class view-layer type
   - a first-class view-stack type
2. Formalize view-layer semantics:
   - stable id

    - kind
    - dismissible
    - blocksBelow
    - root/base marker
3. Implement pure state-machine helpers:
    - create
    - transition
4. Implement pure view-stack helpers:
    - create with a root/base view
    - push
    - pop top dismissible view
    - replace top view
    - clear to root
    - replace root/base intentionally
5. Export the module publicly without migrating the shell yet.

Tests to write first

- cycle test proving the legend, active cycle doc, and remaining backlog slices agree on the first-class runtime-object direction
- package-local runtime tests proving:
    - state-machine transitions track current and previous state
    - view stacks require a non-dismissible root/base view
    - pushing overlays does not alter state-machine state
    - popping never removes the root/base view
    - replacing the top view does not rewrite the root/base view
    - clearing to root preserves the root/base view
    - replacing the root/base view is explicit rather than an accidental pop/replace side effect

Risks / unknowns

- existing shell code already has partial layer concepts that may overlap awkwardly with the new view-layer vocabulary until RE-007
- naming needs to stay generic enough for runtime use without becoming too abstract to migrate onto concrete shell objects later
- the first slice should stay pure and small; if it tries to absorb input routing or retained layouts now, the cycle will sprawl

Retrospective

What landed:

- a pure first-class state-machine object
- a pure first-class view-stack object with explicit root/base semantics
- public runtime-engine exports for those objects
- tests proving the semantics without shell-specific coupling

What did not land:

- no retained layout implementation yet
- no input routing migration yet
- no command/effect buffers yet
- no framed-shell adoption yet

Follow-on:

- RE-003 — Retain Layout Trees and Layout Invalidation
- RE-004 — Route Input Through Layouts and Layer Bubbling
- RE-005 — Buffer Commands and Effects Separately
- RE-006 — Formalize Component Layout and Interaction Contracts
- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

149

RE-003 — Retain Layout Trees and Layout Invalidation

docs/design/RE-003-retain-layout-trees-and-layout-invalidation.md

Cycle for turning layout from a transient render byproduct into an explicit retained runtime object

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture
- RE-002 — Promote First-Class State Machine and View Stack

Why this cycle exists

RE-001 and RE-002 established that:

- the application owns explicit state
- the application owns an explicit view stack

But input routing, viewport ownership, and resize behavior still do not have one retained source of geometric truth.

Today, layout is still too easy to treat as a temporary rendering side effect.

This cycle exists to change that direction by landing a pure retained-layout seam that can answer:

- what layout tree belongs to each active view
- which retained layouts are stale
- why they became stale
- when stale layouts are replaced with newly retained geometry
- which layouts should disappear when views leave the stack

This is intentionally smaller than full layout-driven routing. It gives the runtime authoritative layout ownership before RE-004 moves input over to it.

Human users / jobs / hills

Primary human users

- framework maintainers evolving the runtime
- builders working on modal, drawer, viewport, and pane-heavy apps
- shell authors who need layout ownership to stay stable as the visible stack changes

Human jobs

1. Retain one layout tree per active view instead of recomputing geometry conceptually from scratch every time input arrives.
2. Mark layouts stale when resize, content, visibility, or overflow changes invalidate their geometry.
3. Drop layouts that no longer belong to the active view stack.

Human hill

A human can point to one runtime object and answer which active views currently own retained geometry, which of those layouts are stale, and why.

Agent users / jobs / hills

Primary agent users

- agents inspecting runtime geometry during debugging
- agents generating view-stack tests and layout assertions
- agents preparing future routing work that depends on retained geometry

Agent jobs

1. Read the retained layout tree for a specific view by stable id.
2. Determine whether a retained layout is still valid or is awaiting recomputation.
3. Tell whether a layout disappeared because the view was dismissed or because the geometry merely became stale.

Agent hill

An agent can inspect retained layout state and answer which views have authoritative geometry and which invalidation causes are currently pending without inferring that from rendered frames.

Human playback

1. A user is in a workspace root view with one retained layout tree.
2. The app opens a blocking modal and retains a second layout tree for that view.
3. The terminal is resized.
4. Both retained layouts are marked stale with a terminal-resize invalidation cause.
5. The runtime recomputes and re-retains the new layout trees.
6. Their stale flags clear, but their retained version numbers increase.
7. The modal is dismissed.
8. The modal layout disappears because its view is no longer in the active stack, while the workspace layout remains retained.

Agent playback

1. An agent inspects the runtime while a workspace view and a help overlay are both active.
2. The retained-layout registry shows one layout for workspace and one for help.
3. A content change happens inside the help overlay.
4. The help layout is marked invalidated for content-change.
5. The workspace layout remains valid.
6. When the help overlay is dismissed, the retained-layout registry drops only the help layout because it is no longer part of the current view stack.

Linked invariants

- State Machine and View Stack Are Distinct
- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Tests Are the Spec

Implementation outline

1. Extend the pure runtime-engine module with first-class retained-layout types.
2. Define an explicit invalidation-cause vocabulary for geometry changes, including:
   - terminal resize
   - view-stack change
   - content change
   - visibility change

    - enablement change
    - overflow change
3. Add pure helpers to:
    - create an empty retained-layout registry
    - retain or replace a layout tree for a given active view id
    - list and look up retained layouts
    - mark retained layouts stale with one or more invalidation causes
    - drop layouts whose views are no longer present in the active view stack
4. Keep the implementation shell-agnostic and avoid routing work in this cycle.
5. Leave detailed component sizing/anchoring contracts for RE-006.

Tests to write first

- cycle test proving the legend now points at RE-003 as the active cycle and leaves the remaining runtime-engine backlog in place
- package-local runtime tests proving:
    - retained layout trees are stored by active view id
    - retained layouts expose versioning and invalidation status
    - invalidation causes are explicit and deduplicated
    - re-retaining a layout clears invalidation and increments version
    - layouts for inactive views are dropped when the view stack changes

Risks / unknowns

- there is already a LayoutNode tree in the core toolkit, so the runtime should reuse that concept instead of inventing a second layout language
- the cycle must not drift into full routing or component registration work yet
- detailed resize/stretch/shrink rules still need a stronger component-level contract later, so this cycle should retain geometry ownership without pretending every layout rule is solved

Retrospective

What landed:

- a first-class retained-layout registry in the runtime engine
- explicit invalidation causes for stale geometry
- pure helpers for retaining, invalidating, listing, looking up, and dropping layouts by view ownership
- tests proving versioning, invalidation, and view-stack cleanup semantics

What did not land:

- no full input-routing migration yet
- no component-level stretch/shrink/anchor contract yet
- no framed-shell migration yet

Follow-on:

- RE-004 — Route Input Through Layouts and Layer Bubbling
- RE-005 — Buffer Commands and Effects Separately
- RE-006 — Formalize Component Layout and Interaction Contracts
- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

150

RE-004 — Route Input Through Layouts and Layer Bubbling

docs/design/RE-004-route-input-through-layouts-and-layer-bubbling.md

Cycle for making retained layouts the authoritative routing surface for input, topmost layer first

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture
- RE-002 — Promote First-Class State Machine and View Stack
- RE-003 — Retain Layout Trees and Layout Invalidation

Why this cycle exists

RE-003 made retained geometry first-class.

But the runtime still needs to answer the next question:

- given an input event and a view stack with retained layouts, who sees it first?
- when does it stop?
- when does it bubble downward?
- how does pointer routing know which node was actually hit?

This cycle exists to land a pure runtime-engine routing seam that:

- reads the retained layouts of the current view stack
- routes topmost view first
- uses layout-driven hit testing for pointer input
- respects blocking vs bubbling semantics by layer
- stays shell-agnostic instead of embedding framed-app branches into the runtime core

This is still intentionally smaller than full component contracts. It gives the runtime a routing spine before RE-005 and RE-006 deepen command/effect and component interaction semantics further.

Human users / jobs / hills

Primary human users

- framework maintainers evolving runtime interaction rules
- builders composing modal, palette, drawer, and workspace views
- shell authors who need pointer and key routing to stop depending on branch order

Human jobs

1. Route key and pointer input through explicit topmost-first layer semantics.
2. Trust a blocking top layer to prevent lower views from accidentally seeing the same input.
3. Use retained geometry to tell which layout node a pointer event actually hit.

Human hill

A human can explain why a given input was handled, blocked, or bubbled by pointing at the view stack, retained layouts, and routing result instead of reverse-engineering branch order.

Agent users / jobs / hills

Primary agent users

- agents inspecting why an input hit one view instead of another
- agents generating deterministic routing tests
- agents preparing future component-level interaction contracts

Agent jobs

1. Read routing results and know which view and node received an input first.
2. Distinguish between “unhandled and bubbled” vs “stopped by a blocking layer.”
3. Inspect pointer hit paths without parsing painted frames heuristically.

Agent hill

An agent can inspect the runtime routing result and explain which layers were visited, which node was hit, and why lower views did or did not receive the same input.

Human playback

1. A user has a workspace view open with a non-blocking tooltip above it.
2. The tooltip does not handle a click.
3. Because the tooltip does not block lower views, the click bubbles to the workspace view.
4. Later, a blocking modal opens above the same workspace.
5. The user clicks outside the modal's actionable content.
6. The modal does not handle the click, but it still blocks lower views.
7. The workspace never receives that click.

Agent playback

1. An agent inspects a runtime with workspace -> search in the view stack.
2. The search view is topmost and blocking.
3. A key event enters the runtime.
4. The runtime routes the event to search first.
5. The search handler emits one command and one effect.
6. The routing result reports:
   - visited layers in topmost-first order
   - search as the handling view
   - no lower-layer visitation after handling
7. For a pointer event, the routing result also exposes the hit path inside the retained layout tree.

Linked invariants

- Focus Owns Input
- Topmost Layer Dismisses First
- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Tests Are the Spec

Implementation outline

1. Add pure runtime-engine input types for shell-agnostic key and pointer input.
2. Add pure layout-hit testing over retained layout trees.
3. Add a pure input-routing helper that:
   - visits topmost view first
   - resolves retained layout for that view
   - computes pointer hit paths when geometry applies
   - calls a supplied per-view handler
   - stops on handled input
   - stops on unhandled input if the layer blocks below
   - bubbles otherwise
4. Return a structured routing result that records:
   - visited view ids
   - handled/stopped state
   - commands and effects emitted
   - hit path when pointer routing applies
5. Keep handler semantics generic in this cycle rather than formalizing full component interaction contracts yet.

Tests to write first

- cycle test proving the legend now points at RE-004 as the active cycle and leaves the remaining runtime-engine backlog in place
- package-local runtime tests proving:
  - deepest-node layout hit testing works
  - key routing is topmost-first
  - unhandled input bubbles through non-blocking views
  - unhandled input stops at blocking views
  - pointer routing exposes hit node ids from retained layout geometry
  - routing results preserve separate command/effect arrays

Risks / unknowns

- detailed component-level interaction contracts are still deferred to RE-006
- the first routing seam should stay generic enough to work for framed shell, DOGFOOD, and future runtimes without hard-coding shell concepts
- some future views may need more nuanced propagation semantics than simple handled-or-blocked rules, but this cycle should land the clear baseline first

Retrospective

What landed:

- pure runtime-engine input event types
- retained-layout hit testing
- topmost-first routing over the active view stack
- explicit bubbling vs blocking behavior
- structured routing results with visited views and emitted commands/effects

What did not land:

- no full command/effect buffering runtime yet
- no component-level interaction contract yet
- no framed-shell migration yet

Follow-on:

- RE-005 — Buffer Commands and Effects Separately
- RE-006 — Formalize Component Layout and Interaction Contracts
- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

151

RE-005 — Buffer Commands and Effects Separately

docs/design/RE-005-buffer-commands-and-effects-separately.md

Cycle for formalizing buffered runtime outputs so routing can emit many commands and many effects without immediate consumption

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture
- RE-002 — Promote First-Class State Machine and View Stack
- RE-003 — Retain Layout Trees and Layout Invalidation
- RE-004 — Route Input Through Layouts and Layer Bubbling

Why this cycle exists

RE-004 proved that layout-driven routing can emit distinct command and effect outputs.

These command and effect outputs are still only loose arrays. The runtime still needs an explicit answer to:

- how route outputs accumulate across multiple handled inputs
- how to append multiple route results in order
- how commands are applied later in FIFO order
- how effects are executed later in FIFO order
- how a handled input can still emit nothing
- how buffered outputs drain deterministically for tests

This cycle exists to land a pure runtime-engine buffering seam that keeps commands and effects distinct without dragging the live shell runtime over yet.

Human users / jobs / hills

Primary human users

- framework maintainers evolving runtime ownership rules
- builders composing complex inputs that may emit multiple commands or multiple effects
- shell authors who need deterministic batching without immediate one-input-one-message assumptions

Human jobs

1. Buffer command and effect outputs separately after routing.
2. Append multiple route results in order without losing which outputs are stateful commands vs non-stateful effects.
3. Apply buffered commands later in FIFO order while leaving effect execution separate.
4. Execute buffered effects later in FIFO order without pretending they mutate app state.

Human hill

A human can point to one runtime object for pending commands and one for pending effects, explain their order, and drain them deterministically without inventing shell-specific behavior.

Agent users / jobs / hills

Primary agent users

- agents inspecting why one input emitted several stateful actions
- agents testing batched runtime semantics without live shell coupling
- agents preparing the later command/effect migration into framed runtime code

Agent jobs

1. Read buffered commands and buffered effects separately.
2. Append outputs from multiple route results without losing FIFO order.
3. Apply buffered commands and execute buffered effects in deterministic order while proving the buffers drain cleanly.

Agent hill

An agent can inspect runtime buffers and explain which stateful commands remain pending, which effects remain pending, and in what order they will be drained.

Human playback

1. A user presses Enter in a blocking search layer.
2. The search layer handles the input and emits two commands:
   - search.select
   - workspace.focus
3. The same input also emits one effect:
   - announce.selection
4. The runtime appends those outputs into separate command and effect buffers.
5. No state transition happens yet just because routing returned.
6. Later, the runtime applies the buffered commands in FIFO order.
7. Later, the runtime executes the buffered effects in FIFO order.
8. Both buffers end empty after they are drained.

Agent playback

1. An agent routes two inputs through the runtime.
2. The first input emits:
   - commands: ['open-modal']
   - effects: ['play-click']
3. The second input emits:
   - commands: ['focus-confirm', 'track-selection']
   - effects: ['announce-confirm']
4. The runtime buffers those outputs without immediately applying them.
5. The command buffer now contains:
   - open-modal
   - focus-confirm
   - track-selection
6. The effect buffer now contains:
   - play-click
   - announce-confirm
7. When the runtime drains them, commands are applied later in FIFO order and effects are executed later in FIFO order.

Linked invariants

- Commands Change State, Effects Do Not
- State Machine and View Stack Are Distinct
- Layout Owns Interaction Geometry
- Tests Are the Spec

Implementation outline

1. Extend the pure runtime-engine module with first-class:
   - command buffer types
   - effect buffer types
   - combined runtime buffer helpers
2. Keep handlers free to emit plain command/effect arrays while routing, but buffer route outputs explicitly afterward.
3. Add pure helpers to:
   - create empty command/effect buffers
   - append commands or effects while preserving order
   - buffer route outputs from routeRuntimeInput(...)
   - apply buffered commands with a supplied reducer-style function
   - execute buffered effects with a supplied executor function
4. Make draining deterministic:
   - commands drain in FIFO order
   - effects drain in FIFO order
   - the drained buffer returns empty afterward
5. Keep the live runtime/event-bus migration out of scope for this cycle.

Tests to write first

- cycle test proving the legend now points at RE-005 as the active cycle and leaves the remaining runtime-engine backlog in place
- package-local runtime tests proving:
  - empty command/effect buffers can be created explicitly
  - route outputs can be buffered without collapsing commands and effects together
  - appending multiple route results preserves FIFO order
  - handled inputs that emit nothing leave buffers unchanged
  - buffered commands are applied later in FIFO order and the command buffer drains empty
  - buffered effects are executed later in FIFO order and the effect buffer drains empty

Risks / unknowns

- the later live runtime migration may want adapter-specific effect executors, but this cycle should keep the pure contract generic
- route handlers still emit arrays today, so the buffering seam should avoid forcing a second immediate migration on component-level code
- the runtime should stay explicit about commands changing state and effects not changing state rather than building a vague shared “message queue”

Retrospective

What landed:

- first-class command and effect buffers in the runtime engine
- pure helpers for buffering route outputs
- deterministic FIFO command application and FIFO effect execution helpers
- tests proving buffers drain cleanly without shell-specific coupling

What did not land:

- no live runtime.ts migration yet
- no event-bus command/effect split yet
- no component-level interaction contract yet
- no framed-shell migration yet

Follow-on:

- RE-006 — Formalize Component Layout and Interaction Contracts
- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

152

RE-006 — Formalize Component Layout and Interaction Contracts

docs/design/RE-006-formalize-component-layout-and-interaction-contracts.md

Cycle for turning retained layout nodes into honest component contracts with explicit sizing, overflow, and interaction participation

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture
- RE-002 — Promote First-Class State Machine and View Stack
- RE-003 — Retain Layout Trees and Layout Invalidation
- RE-004 — Route Input Through Layouts and Layer Bubbling
- RE-005 — Buffer Commands and Effects Separately

Why this cycle exists

RE-003 through RE-005 gave the runtime:

- retained layouts
- layout-driven routing
- buffered commands and effects

But retained layout nodes are still just geometry plus painted content. The runtime still needs a first-class way for components to say:

- their explicit layout rules
- how they align and anchor inside a parent-assigned rect
- how they own overflow
- whether they participate in input
- what component-level handlers can emit commands and effects

This cycle exists to formalize those component contracts without dragging the framed shell migration in yet.

Human users / jobs / hills

Primary human users

- framework maintainers evolving the runtime tree
- builders composing real interactive components instead of anonymous layout rectangles
- shell authors who need component ownership to stay honest under resize, overflow, and modal layering

Human jobs

1. Attach explicit layout rules to retained layout nodes.
2. Describe overflow ownership instead of improvising wrap vs clip vs viewport behavior.
3. Mark components enabled or disabled for interaction without stale registration side effects.
4. Resolve the deepest enabled interactive node from a retained hit path.

Human hill

A human can inspect a retained component node and answer how it sizes, how it aligns, how it overflows, and whether it can currently accept input.

Agent users / jobs / hills

Primary agent users

- agents inspecting retained trees for real component semantics
- agents generating routing tests against enabled and disabled components
- agents preparing later shell migration and component-family adoption work

Agent jobs

1. Read explicit layout rules and overflow ownership from a retained component contract.
2. Resolve the deepest enabled interactive node inside a retained hit path.
3. Invoke component-level handlers that emit multiple commands and effects without bypassing runtime buffers.

Agent hill

An agent can inspect a retained layout hit and explain which component would receive input, why disabled descendants were skipped, and whether scroll ownership aligns with viewport overflow.

Human playback

1. A user clicks inside a confirmation card rendered as a retained component tree.
2. The hit path contains:
   - card shell
   - disabled secondary action
   - enabled primary action
3. The runtime resolves the deepest enabled interactive node, which is the primary action.
4. The primary action's component handler emits:
   - confirm.accept
   - modal.dismiss
5. It also emits one effect:
   - play-click
6. In a separate pane, a viewport-owned list component claims wheel scrolling because its block overflow is viewport.
7. A non-viewport note block does not claim the same scroll input.

Agent playback

1. An agent inspects a retained layout hit path with three nested component nodes.
2. The deepest node is disabled, so it should not receive input.
3. Its parent is enabled and accepts pointer presses.
4. The runtime resolves that parent as the deepest enabled interactive node.
5. The agent then invokes the component handler through the runtime contract seam.
6. The handler emits multiple commands and one effect.
7. The runtime keeps those commands and effects separate for later buffering and draining.

Linked invariants

- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Focus Owns Input
- Tests Are the Spec

Implementation outline

1. Extend the pure runtime-engine module with first-class component contracts for:
   - explicit layout rules
   - explicit overflow ownership
   - explicit interaction participation
2. Keep the layout contract small but honest:
   - width and height behavior
   - alignment and anchoring
   - min/max or fixed sizing hooks where needed
3. Keep overflow ownership explicit:
   - inline overflow for clip/truncate/viewport
   - block overflow for wrap/stack/clip/viewport
4. Add pure helpers to:
   - create retained component nodes with attached contracts
   - read component contracts back from retained nodes
   - resolve the deepest enabled interactive node from a retained hit path
   - invoke component-level handlers that emit commands and effects
5. Formalize the rule that scroll ownership aligns with viewport overflow instead of arbitrary wheel handlers.
6. Identify first migration candidates without performing the shell migration yet.

First migration candidates

- collection rows in command palettes and browsable lists
- button-like activators in confirmation or guided-flow surfaces
- viewport-owned panes where wheel/scroll ownership must stay truthful

Tests to write first

- cycle test proving the legend now points at RE-006 as the active cycle and leaves RE-007 as the remaining runtime-engine backlog slice
- package-local runtime tests proving:
  - component contracts can carry explicit layout rules and overflow ownership
  - deepest enabled interactive node resolution prefers the deepest eligible node
  - disabled or unsupported descendants fall back to an enabled ancestor
  - component-level handlers can emit multiple commands and effects

- scroll ownership aligns with viewport overflow instead of ordinary block content

Risks / unknowns

- the real shell migration may want richer layout rules than the first contract vocabulary here, so the initial contract should stay small and composable
- some component families may need higher-level semantics later, but this cycle should land the common runtime contract first
- the runtime should keep this retained-layout-first instead of drifting into ad hoc registration or shell-local shortcuts again

Retrospective

What landed:

- retained component nodes with explicit layout, overflow, and interaction contracts
- pure helpers for resolving the deepest enabled interactive node
- component-level handler dispatch that emits commands and effects
- tests proving disabled/unsupported nodes are skipped honestly

What did not land:

- no framed-shell migration yet
- no app-frame routing rewrite yet
- no wholesale component-family migration yet

Follow-on:

- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

153

RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

docs/design/RE-007-migrate-framed-shell-onto-runtime-engine-seams.md

Cycle for proving the runtime engine through the framed shell instead of leaving the most visible shell state on ad hoc branch structure

Legend:

- RE — Runtime Engine

Depends on:

- RE-001 — Define Runtime Engine Architecture
- RE-002 — Promote First-Class State Machine and View Stack
- RE-003 — Retain Layout Trees and Layout Invalidation
- RE-004 — Route Input Through Layouts and Layer Bubbling
- RE-005 — Buffer Commands and Effects Separately
- RE-006 — Formalize Component Layout and Interaction Contracts

Sponsor human

A maintainer evolving the framed shell who needs to explain layer ownership, routing, and command dispatch in runtime-engine terms instead of tracing ad hoc branch order.

Sponsor agent

An agent inspecting or testing the framed shell that needs to recover runtime-backed view stacks, retained layout geometry, and buffered command/effect truth through the same vocabulary the runtime engine exposes everywhere else.

Non-goals

- Rewriting shell rendering or visual output. This cycle migrates ownership and routing, not painting.
- Changing outward shell behavior. The shell should act the same after each slice; only the internal truth source changes.
- Migrating non-shell runtime consumers. Other runtime adopters are separate cycles.

Accessibility / assistive reading posture

Not directly affected. This cycle changes internal ownership, not user-facing output. Assistive reading contracts remain wherever the shell already honors them.

Localization / directionality posture

Not directly affected. Shell localization surfaces are unchanged by this migration. Directionality remains the responsibility of the rendering layer, which this cycle does not touch.

Agent inspectability / explainability posture

Central to this cycle. Every slice must leave the shell's runtime state inspectable through the same RuntimeViewStack, RuntimeRetainedLayouts, and runtime buffer vocabulary that agents already use for non-shell runtime work. The agent hill is the primary proof surface.

Why this cycle exists

The runtime engine now has explicit seams for:

- state machines
- view stacks
- retained layouts
- layout-driven input routing
- buffered commands and effects

But the framed shell still carries its own most visible runtime truths in older shell-local structures. The shell can already describe a layer stack, but that stack is still reconstructed as a plain descriptor array rather than living as a first-class runtime object.

This cycle exists to stop leaving the highest-visibility shell behavior outside the runtime engine. The shell should become the proving surface for the engine seams rather than the place where those seams get bypassed.

Human users / jobs / hills

Primary human users

- maintainers evolving the framed shell
- builders inspecting layered shell behavior
- contributors debugging shell routing, footer truth, and overlay ownership

Human jobs

1. Inspect the shell's visible layers as a real runtime-backed stack.
2. Explain which shell layer is active and which layer sits underneath it without reverse-engineering branch order.
3. Move shell routing and command/effect dispatch onto the runtime seams in later slices without re-inventing shell-local state models.

Human hill

A maintainer can point at the framed shell and explain its active layer stack in runtime-engine terms instead of saying "the shell works because these branches happen in this order."

Agent users / jobs / hills

Primary agent users

- agents reading shell state to explain focus or overlay ownership
- agents writing tests around shell layering
- agents preparing later shell migration slices for routing and buffered shell work

Agent jobs

1. Read the framed shell's view stack through the same RuntimeViewStack vocabulary used elsewhere in the runtime engine.
2. Distinguish the workspace root from dismissible overlays without re-deriving shell semantics.
3. Prepare later migration slices for retained-layout-driven shell routing and runtime-buffer-backed shell command/effect dispatch.

Agent hill

An agent can inspect the framed shell and recover a root-based runtime view stack whose active layer, underlying layer, and blocking semantics match the shell's real behavior.

Human playback

1. A workspace is open in a framed app.
2. The workspace is the root runtime view.
3. The user opens settings.
4. The shell pushes a dismissible settings overlay onto the runtime view stack.
5. The user opens search on top of settings.
6. The shell pushes a dismissible search overlay onto the same runtime view stack.
7. The active runtime view now truthfully describes the search layer, while the underlying view is still settings.
8. Footer hints and help surfaces can inspect that same stack instead of rediscovering shell state from separate branches.

Agent playback

1. An agent inspects a framed shell model with settings and search open.
2. The agent asks for the shell's runtime-backed view stack.
3. The stack returns:
   - a non-dismissible workspace root
   - a dismissible settings overlay
   - a dismissible search overlay
4. The active runtime view reports the same metadata that shell help/footer introspection already uses.
5. The agent can now reason about shell overlay ownership with the same runtime vocabulary used by retained layouts and buffered runtime work.

Linked invariants

- Runtime Truth Wins
- State Machine and View Stack Are Distinct
- Topmost Layer Dismisses First
- Layout Owns Interaction Geometry
- Commands Change State, Effects Do Not
- Tests Are the Spec

First slice in this cycle

The first honest slice is not “rewrite the whole shell.” It is:

- make the framed shell layer stack a real RuntimeViewStack
- keep the existing describeFrameLayerStack(...), activeFrameLayer(...), and underlyingFrameLayer(...) helpers as compatibility surfaces
- expose a runtime-backed describeFrameRuntimeViewStack(...) helper so tests, humans, and agents can inspect the shell directly in runtime terms

This slice proves that the shell can adopt runtime-backed view ownership without changing outward shell behavior first.

Implementation outline

1. Promote shell layer descriptors into runtime-backed view layers with stable ids, blocking semantics, dismissibility, and attached descriptor models.
2. Make the workspace the root runtime view.
3. Push page-modal, settings, help, notification-center, search/command-palette, and quit-confirm overlays onto the runtime stack in the shell’s existing order.
4. Derive the legacy descriptor helpers from that runtime view stack instead of rebuilding plain arrays separately.
5. Export the runtime-backed helper through the public framed-app API and the package index.

Later slices still in this cycle

- retained-layout-driven shell routing
- runtime-buffer-backed shell command/effect dispatch
- removing remaining shell-local branch structures that duplicate runtime ownership

Tests to write first

- cycle test proving RE-007 is now the active runtime-engine cycle
- app-frame regression proving the framed shell layer stack is now backed by RuntimeViewStack
- export regression proving the new helper is public through @flyingrobots/bijou-tui
- signpost regression proving PLAN.md, BEARING.md, and the RE legend stop treating RE-007 like a backlog stub

Risks / unknowns

- the shell still mixes routing, layout, and command/effect concerns, so this first slice must stay narrow instead of pretending the whole migration is done
- keeping the compatibility descriptor helpers is necessary now, but later slices should avoid duplicating runtime truth unnecessarily
- retained-layout-driven shell routing may want richer view metadata than the first runtime-backed stack carries here

Retrospective

What this first slice lands:

- a runtime-backed shell layer stack
- a public describeFrameRuntimeViewStack(...) helper
- compatibility descriptor helpers derived from runtime truth
- signposts that say RE-007 is active instead of merely queued

What this first slice does not land:

- no retained-layout-driven shell routing yet
- no runtime-buffer-backed shell command/effect dispatch yet
- no removal of every shell-local branch

What the second slice lands:

- route key ownership through the runtime view stack instead of choosing shell owners from ad hoc top-layer branches
- use retained shell drawer layouts so pointer ownership for settings and notification center comes from runtime hit-testing instead of drawer-specific inside/outside helpers alone

What the third slice lands:

- a workspace retained layout tree with header tab and pane children so sub-layer hit-testing uses the same retained layout infrastructure
- settings row children in the settings retained layout so row click resolution uses layout path inspection instead of scroll-offset math
- paneHitAtPosition, settingsRowAtPosition, and isInsideSettingsDrawer removed — all replaced by retained layout path inspection through routedHit
- resolveWorkspacePaneRects extracted as a shared pane geometry resolver for both the workspace layout tree and the mouse handler

What the fourth slice lands:

- FrameShellCommand<Msg> discriminated union in app-frame-types.ts covering every shell mutation and command emission as plain data facts
- a handler table inside createFramedApp that interprets each command variant through model mutation and TEA command accumulation

- drainShellCommandBuffer using bufferRuntimeRouteResult and applyRuntimeCommandBuffer to process route results
- mouse routing handler produces shell commands directly instead of delegating to a separate handleFrameMouse function
- key routing handler produces shell commands through per-layer helper functions (handlePaletteLayerKeyCommands, handleSettingsLayerKeyCommands, handleWorkspaceLayerKeyCommands, etc.) instead of a 280-line if-chain in update()
- update() key and mouse branches reduced to single-line buffer drains
- removed: handleFrameMouse, withObservedKey, applyQuitRequest, observedRouteForLayer, applyHelpScrollAction

What this cycle does not address:

- notification toast hit-testing stays outside the retained layout system (viewport-positioned overlays managed by notification.ts)
- FrameScopedMsg branches (timers, transitions, notification ticks) stay as special cases in update() — they are frame-owned state mutations, not routed input events

Runtime-buffer-backed command/effect dispatch — design intent

This section captures the architectural intent for the final slice so it is not lost across sessions.

Current pattern

The shell's update() function has two separate steps:

1. **Routing**: resolveRoutedKeyLayer() / resolveRoutedMouseLayer() calls routeRuntimeInput() to determine which layer owns the input. The routing handler callback returns { handled: true } or { stop: true } — it never produces commands or effects.
2. **Command production**: After routing, the update() function matches on routedLayer.kind through a long chain of if branches, each producing [model, Cmd<FramedAppMsg<Msg>>[]] tuples with manually accumulated command arrays.

These are decoupled: routing decides ownership, then entirely separate code produces commands. The runtime buffer API (RuntimeCommandBuffer, RuntimeEffectBuffer, bufferRuntimeRouteResult, applyRuntimeCommandBuffer) exists but the shell does not use it.

Target pattern

Input is routed through the runtime view stack. Instead of routing handlers returning bare { handled: true }, they return commands and effects as part of the route outcome:

```
routeRuntimeInput(stack, layouts, event, ({ layer, hit }) => {
  // Layer-specific command production happens HERE,
```

```
  // inside the routing callback
  return {
    handled: true,
    commands: [emitMsgForPage(pageId, action)],
    effects: [],
  };
});
```

The route result carries accumulated commands and effects from all visited layers. The shell then buffers them:

```
const buffers = bufferRuntimeRouteResult(
  createRuntimeBuffers(),
  routeResult,
);
```

And applies them after routing completes:

```
const { state, applied } = applyRuntimeCommandBuffer(
  model,
  buffers.commands,
  applyCommand,
);
```

Why this matters

- Routing and command production become one step instead of two. The layer that handles the input also says what should happen, inside the same callback.
- The long if (routedLayer.kind === ...) chain in update() moves into the routing handler, which already knows the layer context.
- Commands are collected through the same buffer mechanism the runtime engine uses for component-level input handling, instead of through ad-hoc array accumulation.
- Effects become a first-class channel. The shell currently has no explicit effect separation — everything is a command. The buffer API gives effects their own lane without changing the TEA return type.

Scope

This is a structural refactoring of the shell's update function. It touches ~20 functions across app-frame.ts, app-frame-actions.ts, and app-frame-palette.ts. The change is repetitive but mechanical: every [model, cmds] return site moves into a routing handler that returns { handled: true, commands: [...] }.

Outward shell behavior must not change. The same inputs must produce the same model transitions and the same commands.

Risks

- The routing handler closure becomes large because it absorbs all layer-specific logic. May need to extract per-layer handler functions to keep readability.

- The TEA update() return type ([Model, Cmd[]]) is consumed by the event bus. The buffer API produces RuntimeBuffers, which must be drained back into Cmd [] at the boundary. This is a seam, not a full TEA replacement.
- Key priority logic (frame-first vs page-first) currently lives outside routing. It needs to move into the routing handler or be resolved before routing begins.

154

RE-008 — Rendering Performance Findings

docs/design/RE-008-perf-findings.md

Results from the examples/perf-gradient stress test, April 2026. These findings motivate RE-008 — Byte-Packed Surface Representation.

Test setup

- Terminal: iTerm2, 191×48 (9,168 cells)
- Runtime: Node.js v25.8.1, tsx
- Hardware: Apple Silicon Mac
- Modes tested: cosine gradient (unique fg+bg per cell), ASCII perspective horizon (varying chars, few color changes), OpenSimplex noise density (varying chars, single bg), quad (all three combined)

Raw numbers

Mode	FPS (capped)	FPS (uncapped, cold)	FPS (uncapped, warm)	view() ms	ft ms (actual)
Gradient	49	30-49	158	2.5	~20
Horizon	60	344	344	1.4	~3
Noise	49-60	98	298	1.4-1.5	~3-10
Quad	60	91	91	1.3	~11

"Warm" means after 60+ seconds of sustained rendering, when V8 has JIT-compiled the hot paths and GC has settled.

Where frame time goes

At 191×48 with the gradient (worst case), a frame takes ~20ms total:

Phase	Time	Measurable from app?
update()	~0ms	yes
view() (gradient math + surface.set)	~2.5ms	yes

Phase	Time	Measurable from app?
Pipeline: diff + ANSI encode + stdout.write	~17.5ms	**no**

92% of frame time is spent in bijou's render pipeline, not in the app's view function. The pipeline is not observable from the app side — there are no hooks to measure diff, paint, or output phases individually.

The differ is the bottleneck

The differ (packages/bijou/src/core/render/differ.ts) compares every cell in the target surface against the current surface. For cells that changed, it emits ANSI escape sequences.

The differ's performance depends on **how many unique styles appear in sequence**:

- **Horizon (344fps uncapped)**: most cells on a row share the same fg/bg, so the differ batches long runs of same-styled characters into single ANSI writes. Very few color change escape sequences per frame.
- **Noise (298fps uncapped)**: every cell has a unique fg but all share bg: '#000000'. The differ still emits per-cell fg changes but saves half the escape sequences (no bg changes).
- **Gradient (158fps uncapped)**: every cell has a unique fg AND bg. The differ emits ~9,168 color change escape sequences per frame — one per cell. No batching is possible.

The differ also compares cells by **string equality** on fg and bg hex strings (targetCell.fg === currentCell.fg), which is correct but slower than numeric comparison would be.

Allocation pressure and GC

Each surface.set() call in the gradient mode produces 2 hex color strings (rgbHex(r, g, b) for fg and bg). At 9,168 cells per frame, that is **18,336 string allocations per frame**.

With surface reuse (the cached surface is not reallocated per frame), the Cell objects themselves are mutated in place via applyMaskInPlace. But the hex color strings are always freshly allocated because they encode the current frame's computed RGB values.

GC behavior at different modes

Mode	Heap (stable)	GC events / 0.5s	Notes
Gradient (capped 60fps)	51-92 MB	2-5	Young-gen fills with hex strings

Mode	Heap (stable)	GC events / 0.5s	Notes
Gradient (uncapped)	185-824 MB	0-1	Heap grows until major GC; V8 eventually stabilizes
Noise (uncapped)	101 MB	0-1	Fewer unique strings; V8 handles it

JIT warmup

The gradient mode shows a clear warmup curve:

- **0-30 seconds**: 30-49fps. V8 is interpreting; GC is aggressive.
- **30-60 seconds**: 80-120fps. JIT compilation kicks in.
- **60+ seconds**: 150-160fps. Hot paths fully optimized, GC settled.

The noise and horizon modes warm up faster because they produce fewer unique string values per frame.

What RE-008 would fix

A byte-packed surface representation would eliminate:

1. **18K string allocations per frame** — colors stored as 3 bytes (R, G, B), not heap-allocated hex strings.
2. **String comparison in the differ** — numeric comparison of bytes instead of === on strings.
3. **Hex parsing in the ANSI encoder** — the style.styled() call currently parses #rrggbb back to numeric RGB to emit \x1b[38;2;R;G;Bm. With byte-packed cells, the encoder reads bytes directly.
4. **The JIT warmup penalty** — no string-heavy hot paths for V8 to deoptimize and recompile.

What DX-006 would provide

A first-class debugOverlay() component would let any bijou-tui app see these metrics without rebuilding the stats overlay from scratch. The perf gradient demo proves the concept; DX-006 extracts it into a reusable component.

Pipeline observability gap

The render pipeline (Layout → Paint → PostProcess → Diff → Output) is not observable from the app side. configurePipeline can add middleware but only appends after the default stages — it cannot wrap them for timing. A future improvement would expose per-stage timing hooks so the debug overlay can show the full breakdown.

Reproducing

```
npx tsx examples/perf-gradient/main.ts
```

Keys: 1-4 switch modes, space toggles 60fps cap, hold mouse to reverse, q to quit. The stats overlay shows FPS, frame time, view time, heap/RSS/GC, and a braille line chart of view() cost.

155

RE-024 Surface Budget Warnings

docs/design/RE-024-surface-budget-warnings.md

```
title: RE-024 Surface Budget Warnings
legend: RE
lane: design
```

Framing

Many performance regressions are easier to catch during normal development than in a separate benchmark run. Bijou already measures render pipeline timings and owns the target Surface before diff/output, so the runtime can warn when an app crosses explicit surface or timing budgets.

RE-024 adds opt-in, non-fatal budget diagnostics. The first slice keeps the policy simple: a pure evaluator returns warning objects, and the interactive runtime can route those warnings through the existing routeRuntimeIssue app hook when `run(..., { surfaceBudget })` is configured.

Sponsored Users

- App authors who want early feedback when a render surface grows beyond an intended size.
- Maintainers reviewing frame timing regressions during normal development.
- Test authors who want deterministic budget warning objects without running a benchmark harness.

Hills

1. A developer can evaluate a Surface and render timings against configured budgets and get deterministic warning objects.
2. An interactive app can opt into runtime budget warnings without changing its TEA messages unless it chooses to route RuntimeIssues.
3. A test can assert that area, styled-cell, frame-duration, and per-stage thresholds produce clear messages.

Playback Questions

- Does the pure evaluator stay deterministic and side-effect free?
- Does a target surface over the configured area threshold produce a warning?
- Does a styled-cell count over the configured threshold produce a warning?
- Do frame-duration and stage-duration thresholds use pipeline timings?
- Does run(..., { surfaceBudget }) route warnings through routeRuntimeIssue as source: 'runtime' and level: 'warning'?
- Does omitting surfaceBudget keep the runtime silent?

Requirements

- Add evaluateSurfaceBudget() in @flyingrobots/bijou-tui.
- Support maximum width, height, area, styled cells, total frame duration, and per-stage duration thresholds.
- Add surfaceBudget to RunOptions.
- Route runtime budget violations through routeRuntimeIssue.
- Keep warnings non-fatal by default.

Acceptance Criteria

- Focused RED tests fail before implementation and pass after.
- The evaluator returns stable SurfaceBudgetWarning records.
- Runtime budget warnings use the existing runtime issue route.
- No warning is emitted when no budget is configured.
- Docs and changelog describe the opt-in feature.

Implementation Outline

- Implement a pure surface-budget.ts helper with threshold and warning types.
- Export the helper and types from the bijou-tui source barrel.
- Call the helper after pipeline execution, before lifecycle playback, so the warning uses the same target surface and timings that will be committed.

Drift Check

- Scope stayed on non-fatal warnings. The runtime does not fail frames or quit when a threshold is crossed.
- The first slice measures the committed target surface and pipeline timings. Allocation count and output byte volume remain follow-on metrics because they need deeper differ/runtime counters to be accurate.

- Runtime warnings are deduplicated by message to avoid warning/render loops when an app routes budget issues back into its own TEA update path.

Playback

- evaluateSurfaceBudget() returns deterministic warning records for width, height, area, styled-cell count, total frame duration, and per-stage duration.
- run(..., { surfaceBudget }) evaluates the target framebuffer after pipeline execution and routes violations through routeRuntimeIssue.
- Runtime warnings use level: 'warning' and source: 'runtime'.
- Omitting surfaceBudget leaves existing runtime behavior unchanged.
- @flyingrobots/bijou-tui exports the helper and budget types from the source barrel.

Retrospective

- This is the right first runtime budget boundary: it gives app authors useful early warnings without pretending Bijou can enforce every performance budget from inside the render loop.
- The target surface area is the framebuffer area, not the app view's intrinsic surface size after normalization. Tests pin the viewport when they need exact runtime budget numbers.

156

RE-030 — Average High-Resolution Canvas Colors

docs/design/RE-030-average-high-resolution-canvas-colors.md

Legend: RE — Runtime Engine

Sponsor Human

A terminal app builder using @flyingrobots/bijou-tui canvas shaders for small procedural images, title screens, charts, and textured scene previews.

Sponsor Agent

An implementation agent that must render high-density terminal art without repairing colors after the canvas has already collapsed subpixels into cells.

Hills

1. A builder can return per-subpixel RGB values from a canvas() shader and trust Bijou to collapse those colors the way image resampling normally does.
2. A builder can use braille or quad resolution without losing color when the first lit subpixel is not representative of the whole terminal cell.
3. An agent can debug canvas color behavior from canvas.ts itself rather than recommending app-local post-render setRGB() correction loops.

Playback Questions

1. In braille mode, when eight subpixels return different foreground RGB values, does the resulting Braille cell use their arithmetic mean?
2. In braille mode, when all eight subpixels are visually off but still return background RGB values, does the collapsed cell preserve their average background style?

3. In quad mode, when four subpixels return different RGB values, does the resulting quadrant character use the arithmetic mean instead of the first lit subpixel's style?
4. Does the character coverage behavior stay unchanged while only style aggregation changes?

Requirements

- canvas(..., { resolution: 'braille' }) must sample all 2x4 subpixels for the output cell and average every available foreground RGB contribution.
- canvas(..., { resolution: 'braille' }) must also average every available background RGB contribution, including cells whose shader character is a space.
- canvas(..., { resolution: 'quad' }) must use the same color aggregation rule across its 2x2 subpixels.
- Shader results that return fgRGB or bgRGB must avoid unnecessary hex parsing.
- Shader results that return parseable hex or resolved color references must still participate in the same averaging path.
- Coverage remains boolean: non-space subpixels set Braille dots or quadrant bits, and space subpixels leave those dots or bits off.
- Modifiers remain compatibility-oriented and come from the first lit styled subpixel.

Acceptance Criteria

- Red tests prove Braille foreground averaging, Braille background averaging for an empty Braille cell, and Quad foreground averaging.
- The implementation changes packages/bijou-tui/src/canvas.ts instead of requiring app-local post-processing.
- npm run test -- packages/bijou-tui/src/canvas.test.ts passes.
- npm run lint --workspace @flyingrobots/bijou-tui passes.
- git diff --check passes.

Accessibility / Assistive Reading Posture

This change preserves character semantics and only adjusts styling. Existing plain text and assistive interpretations of the character grid should remain unchanged.

Localization / Directionality Posture

The feature is independent of prose localization and bidirectional text. It does not alter grapheme ordering or directional layout.

Agent Inspectability / Explainability Posture

The color decision must live in named helper functions in canvas.ts, with tests that make the sampling rule explicit. Agents should be able to explain that the high-resolution renderer computes coverage and style separately: coverage chooses the Braille or quadrant glyph, while style is averaged from sampled subpixel styles.

Linked Invariants

- Tests Are the Spec
- Runtime Truth Wins

Implementation Outline

1. Add failing tests for Braille foreground averaging, empty-cell background averaging, and Quad foreground averaging.
2. Introduce a small color accumulator in canvas.ts.
3. Resolve each sampled cell's foreground and background color from fgRGB, bgRGB, resolved color refs, or parseable hex strings.
4. Average available foreground and background samples independently using rounded arithmetic means.
5. Apply the averaged style to the collapsed Braille or Quad cell while preserving existing coverage behavior.

Tests To Write First

- averages braille sub-pixel foreground colors
- preserves averaged braille background colors for empty cells
- averages quad sub-pixel foreground colors

Drift Check

- RED observed: npm run test -- packages/bijou-tui/src/canvas.test.ts failed with three expected assertions: Braille foreground used [0, 0, 0], empty Braille background was undefined, and Quad foreground used [0, 0, 0].

- GREEN observed: npm run test -- packages/bijou-tui/src/canvas.test.ts passed with 9 tests.
- Package lint observed: npm run lint --workspace @flyingrobots/bijou-tui passed.
- Root lint: npm run lint reached the workspace sweep but failed on pre-existing uncommitted packages/heatmap work because @flyingrobots/bijou-heatmap had no lint script. After the local heatmap workspace drift was repaired, npm run lint passed.
- Pre-commit lockfile gate: npm ls --all still fails in the local install tree with extraneous and missing dependency reports unrelated to this canvas change, so the scoped commit used the recorded tests and lint results rather than treating local install drift as RE-030 scope.
- Whitespace drift: git diff --check passed.

Playback

1. Braille foreground averaging is covered by averages braille sub-pixel foreg round colors; the collapsed full Braille cell now carries [28, 56, 84] from eight RGB samples.
2. Empty Braille background preservation is covered by preserves averaged brai lle background colors for empty cells; the blank Braille cell now carries [28, 38, 48] even when no dot is lit.
3. Quad foreground averaging is covered by averages quad sub-pixel foreground colors; the ▙ glyph now carries [15, 30, 45] from all four samples.
4. Existing coverage tests for top-left Braille and diagonal Quad glyphs still pass, so glyph selection stayed stable while style aggregation changed.

Retrospective

The root issue was local to high-resolution canvas style aggregation. The renderer already sampled the right subpixels for coverage but treated style as a first-lit-subpixel property. Splitting coverage from style lets shader authors return material colors per sample and lets Bijou perform the terminal-cell resampling step itself. No app should need to walk the rendered surface after canvas() just to recover colors.

157

RE-031 — Raytrace Shader Kernel

docs/design/RE-031-raytrace-shader-kernel.md

Legend: RE — Runtime Engine

Sponsor Human

A Bijou app builder creating title screens, procedural previews, or small shader-driven scenes in terminal cells.

Sponsor Agent

An implementation agent that needs to author terminal ray scenes without copying vector math, camera rays, sphere intersections, and reflection helpers into each app.

Hills

1. A builder can import a tiny raytrace kernel from @flyingrobots/bijou-tui and use it inside canvas() shaders without pulling in an external 3D engine.
2. A builder can keep app-specific material and lighting decisions local while sharing the boring, error-prone geometry math.
3. An agent can factor a title-screen ray tracer into reusable Bijou helpers without turning Bijou into a full scene graph or renderer.

Playback Questions

1. Can a caller create a look-at camera ray whose center sample points from the camera origin to the target?
2. Can the kernel report the first positive sphere hit with correct distance, hit point, and surface normal?
3. Can the kernel compare supported shapes and return the nearest hit?

4. Can reflection math be reused by app-specific lighting code without the app reimplementing vector operations?

Requirements

- Export RaytraceVector3, RaytraceRay, RaytraceSphere, RaytracePlane, RaytraceShape, and RaytraceHit types from @flyingrobots/bijou-tui.
- Export pure vector helpers for add, subtract, scale, dot, cross, normalize, reflect, and mix-in camera composition.
- Provide a raytraceLookAtRay() helper that builds a normalized ray from an origin, target, screen coordinate, and focal length.
- Provide a raytraceOrbitCameraRay() helper for the common orbiting title screen camera pattern.
- Provide raytraceSphereHit(), raytracePlaneHit(), and raytraceNearestHit() primitives.
- Keep lighting, materials, shadows, and palette decisions out of this cycle.

Acceptance Criteria

- Red tests prove camera center direction, sphere hit facts, nearest-hit selection, and reflection behavior.
- The implementation lives in packages/bijou-tui/src/raytrace.ts and is exported from the package root.
- npm run test -- packages/bijou-tui/src/raytrace.test.ts packages/bijou-tui/src/index.test.ts passes.
- npm run lint --workspace @flyingrobots/bijou-tui passes.
- git diff --check passes.

Accessibility / Assistive Reading Posture

This is a math helper and does not change terminal output by itself. Apps using it remain responsible for plain-text fallbacks and for not relying on decorative motion as the only information channel.

Localization / Directionality Posture

The kernel is independent of prose localization and text direction.

Agent Inspectability / Explainability Posture

The helper names should expose the geometric truth directly. Agents should be able to answer whether a title scene uses Bijou's shared hit math or app-local lighting by reading imports and tests.

Linked Invariants

- Tests Are the Spec
- Runtime Truth Wins

Implementation Outline

1. Add failing tests for look-at camera rays, sphere hits, nearest hits, and reflection.
2. Implement a focused raytrace helper module in bijou-tui.
3. Re-export the helper from the package root.
4. Document the intended boundary in the advanced guide and changelog.

Tests To Write First

- builds a normalized look-at camera ray
- reports the first positive sphere hit
- selects the nearest supported shape hit
- reflects a ray across a surface normal

Drift Check

- RED observed: npm run test -- packages/bijou-tui/src/raytrace.test.ts packages/bijou-tui/src/index.test.ts failed because ./raytrace.js did not exist and raytraceLookAtRay was not exported.
- GREEN observed: npm run test -- packages/bijou-tui/src/raytrace.test.ts packages/bijou-tui/src/index.test.ts passed with 7 tests.
- Package lint observed: npm run lint --workspace @flyingrobots/bijou-tui passed.
- Root lint observed: npm run lint passed after the pre-existing local heatmap workspace drift was repaired.
- Whitespace drift: git diff --check passed.
- Pre-commit lockfile gate: npm ls --all remains blocked by local install-tree drift unrelated to the raytrace helper.

Playback

1. builds a normalized look-at camera ray proves the center ray points from camera origin to target.
2. reports the first positive sphere hit proves sphere distance, point, and normal facts.
3. selects the nearest supported shape hit proves shape comparison across the supported primitive set.
4. reflects a ray across a surface normal proves the reflection helper used by app-owned lighting code.
5. re-exports raytrace shader helpers from the package root proves package ergonomics for single-import shader authoring.

Retrospective

This cycle intentionally stopped at geometry. Bijou now owns the reusable camera and intersection math that canvas shader authors are likely to copy between apps, while app code still owns visual taste: material colors, theme lookups, specular strength, reflections, shadows, and palette decisions. That keeps the helper useful for jedit-style title scenes without turning Bijou into a general-purpose renderer.

158

RE-032 — Fit Cell Glyphs From Coverage

docs/design/RE-032-fit-cell-glyphs-from-coverage.md

Legend: RE — Runtime Engine

Sponsor Human

A terminal app builder experimenting with alternatives to Braille for dense procedural art, logos, minimaps, and shader previews.

Sponsor Agent

An implementation agent that needs to collapse high-resolution cell coverage into a reasonable glyph without inventing a one-off character heuristic inside an app.

Hills

1. A builder can sample a 2x4 cell and ask Bijou for the closest matching glyph instead of hard-coding Braille.
2. A builder can choose either geometry-aware Unicode candidates or a strictly ASCII density ramp for terminals with limited glyph support.
3. An agent can experiment with solid, hatch, slash, and density renderings of the same source coverage while keeping the fitting rule testable.

Playback Questions

1. Does full 2x4 coverage choose a full block rather than a density fallback?
2. Does top-half coverage choose the upper-half block?
3. Does diagonal coverage choose a diagonal glyph when Unicode candidates are enabled?
4. Can a caller provide custom candidates and have exact coverage win?
5. Does ASCII mode stay inside plain ASCII characters?

Requirements

- Export fitCellGlyph(), CELL_GLYPH_UNICODE_CANDIDATES, and CELL_GLYPH_ASCII_DENSITY_RAMP from @flyingrobots/bijou-tui.
- Accept eight numeric coverage samples in row-major 2x4 order.
- Clamp coverage values into [0, 1].
- Score geometry candidates by mean squared error against the samples.
- Support mode: 'unicode' for block/line/diagonal candidates.
- Support mode: 'ascii' for a density-only ASCII ramp.
- Support explicit candidate overrides for app experiments.
- Keep this helper independent from canvas() so apps can opt into it without changing existing Braille behavior.

Acceptance Criteria

- Red tests prove full-block, top-half, diagonal, custom-candidate, and ASCII density behavior.
- The implementation lives in packages/bijou-tui/src/cell-glyph-fit.ts and is exported from the package root.
- npm run test -- packages/bijou-tui/src/cell-glyph-fit.test.ts packages/bijou-tui/src/index.test.ts passes.
- npm run lint --workspace @flyingrobots/bijou-tui passes.
- git diff --check passes.

Accessibility / Assistive Reading Posture

This helper only chooses display glyphs. Apps remain responsible for non-visual labels or alternate views when glyph geometry carries meaning.

Localization / Directionality Posture

Glyph fitting is independent of language and directionality. ASCII mode exists for the most conservative terminal compatibility path.

Agent Inspectability / Explainability Posture

The fitting rule must be data-driven: candidates expose their coverage masks, and tests assert expected matches. Agents can explain a glyph choice as the lowest mean squared error among candidate masks.

Linked Invariants

- Tests Are the Spec
- Runtime Truth Wins

Implementation Outline

1. Add failing tests for block, half-block, diagonal, custom candidate, and ASCII density behavior.
2. Implement candidate masks and MSE scoring in cell-glyph-fit.ts.
3. Re-export the helper from bijou-tui.
4. Document the helper in the advanced guide and changelog.

Tests To Write First

- chooses a full block for full coverage
- chooses an upper-half block for top-half coverage
- chooses a diagonal glyph for diagonal coverage
- honors exact custom candidate matches
- uses only the ASCII density ramp in ascii mode

Drift Check

- RED observed: npm run test -- packages/bijou-tui/src/cell-glyph-fit.test.ts packages/bijou-tui/src/index.test.ts failed because ./cell-glyph-fit.js did not exist and fitCellGlyph was not exported.
- GREEN observed: npm run test -- packages/bijou-tui/src/cell-glyph-fit.test.ts packages/bijou-tui/src/index.test.ts passed with 9 tests.
- Package lint observed: npm run lint --workspace @flyingrobots/bijou-tui passed.
- Root lint observed: npm run lint passed.
- Whitespace drift: git diff --check passed.
- Pre-commit lockfile gate: npm ls --all remains blocked by local install-tree drift unrelated to the glyph fitting helper.

Playback

1. chooses a full block for full coverage proves exact dense coverage maps to █.
2. chooses an upper-half block for top-half coverage proves geometry beats a generic density choice.

3. chooses a diagonal glyph for diagonal coverage proves Unicode diagonal candidates are part of the fit set.
4. honors exact custom candidate matches proves app experiments can override the built-in candidates.
5. uses only the ASCII density ramp in ascii mode proves the conservative mode stays plain ASCII.

Retrospective

This cycle gives apps an experiment hook without changing canvas() behavior. Braille remains the high-density default, while fitCellGlyph() lets a caller try solid, partial, diagonal, or ASCII-density collapse from the same 2x4 coverage samples. That is the right boundary for logo experiments because the app can decide when visual smoothness matters less than a cleaner glyph shape.

159

RE-033 Stable Spring Pulse Integration

docs/design/RE-033-stable-spring-pulse-integration.md

Sponsored Users

- Application builders using @flyingrobots/bijou-tui spring animation for camera controls, pane motion, drawers, or other interactive UI state.
- Terminal users on slow, remote, or overloaded shells where frame pulses can arrive late or in large bursts.
- Agents debugging Bijou apps from event streams rather than visual intuition.

Problem

animate({ type: 'spring' }) currently integrates spring physics with the raw runtime pulse delta. A slow frame can therefore hand springStep() a very large dt, and semi-implicit Euler integration can convert one delayed pulse into a large velocity and position spike.

Apps can patch around this by implementing their own bounded fixed-timestep command, but the invariant belongs in Bijou: a spring command should stay stable when the terminal slows down.

Hills

1. A builder can use Bijou's default spring animation in a slow terminal without writing app-local dt clamps.
2. A user holding an animated control during a delayed frame sees motion slow or catch up conservatively, not explode past the intended state.
3. An agent can prove spring stability with a focused test that drives the command with large pulse deltas.

Playback Questions

1. If a spring receives a one-second pulse, does the first emitted value remain bounded between the start and target for a critically damped spring?
2. Does the animation still settle at the exact target value and emit completion normally?
3. Does the fix live inside Bijou's animate() spring command rather than in downstream application code?

Requirements

- Spring animation commands must integrate using bounded fixed timesteps.
- Large pulse deltas must not be applied to springStep() as one raw step.
- The default behavior must remain app-friendly without requiring new options.
- Tween animation behavior is out of scope for this cycle.
- Existing animate() and sequence() public call sites must remain source compatible.

Acceptance Criteria

- packages/bijou-tui/src/animate.test.ts includes a slow-pulse spring regression.
- packages/bijou-tui/src/animate.ts bounds spring integration internally.
- Existing spring, tween, immediate, and sequence tests continue to pass.
- npm run test -- packages/bijou-tui/src/animate.test.ts passes.
- npm run --workspace @flyingrobots/bijou-tui lint passes.
- git diff --check passes.

Design

Use a small fixed-step accumulator inside createSpringCmd().

The command should:

- accumulate pulse time;
- cap the amount of pulse time accepted per runtime pulse;
- step springStep() in fixed increments;
- emit at most one frame per runtime pulse after it has stepped;
- dispose and emit onComplete() exactly as before when the spring settles.

Dropping excessive slow-frame time is intentional. A terminal animation should prefer stable, slightly delayed motion over trying to simulate a full stalled second in one event loop turn.

RED

Initial RED test: a critically damped spring from 0 to 100 receives repeated one-second pulses. With raw dt integration, the first frame jumps far beyond the target.

The initial run failed in packages/bijou-tui/src/animate.test.ts because the spring did not settle under the slow-pulse sequence.

GREEN

createSpringCmd() now accumulates runtime pulse time, accepts at most a small bounded pulse window, and advances spring physics with fixed timesteps. It emits one frame after one or more physics steps and preserves the existing completion path.

The public animate() call remains source-compatible. Spring callers may tune fixedStepSeconds and maxPulseSeconds, but the defaults are stable without new app code.

Drift Check

- npm run test -- packages/bijou-tui/src/animate.test.ts passed.
- npm run --workspace @flyingrobots/bijou-tui lint passed.
- npm run lint passed.
- git diff --check passed.

Playback

Playback answers:

1. A one-second pulse no longer reaches springStep() as one raw integration step; the regression proves a critically damped spring remains bounded.
2. The command still resolves at the exact target value and preserves completion semantics.
3. The fix lives inside packages/bijou-tui/src/animate.ts, not in an app-local workaround.

Retrospective

The jedit camera failure was a useful dogfood signal: spring stability is a framework guarantee, not an application responsibility. This cycle intentionally kept scope to spring integration. A broader frame scheduler remains a better future home for runtime-wide pulse policy, but animate() is now safe by default for the existing public API.

160

RE-034 Glyph-Fit Canvas Resolution

docs/design/RE-034-glyph-fit-canvas-resolution.md

Sponsored Users

- Terminal app builders rendering shader-authored logos, minimaps, raytraced previews, and procedural diagrams.
- Users whose terminals render Braille poorly or whose app style calls for solid, hatch, line, diagonal, or ASCII glyphs.
- Agents migrating app-local cell reducers into reusable Bijou primitives.

Problem

fitCellGlyph() can collapse 2x4 coverage samples into a geometry-aware glyph, but canvas() only exposes cell, quad, and braille resolution modes. Apps that want non-Braille high-resolution shader output still need to sample subpixels themselves, aggregate colors themselves, and call fitCellGlyph() outside the canvas pipeline.

That duplicates the same sampling and style aggregation logic that already lives in canvas.ts.

Hills

1. A builder can render a shader through canvas(..., { resolution: 'glyph' }) and receive a fitted cell glyph instead of Braille.
2. A builder can choose Unicode candidate fitting or ASCII density fitting through canvas options.
3. An agent can point to one canvas path for high-resolution sampling, color aggregation, and glyph collapse.

Playback Questions

1. Does resolution: 'glyph' sample all eight 2x4 subpixels?

2. Does diagonal coverage choose a diagonal Unicode glyph through the existing fitCellGlyph() candidate set?
3. Does glyphFit: { mode: 'ascii' } stay inside the ASCII density ramp?
4. Are subpixel foreground/background colors still averaged like the Braille and Quad paths?

Requirements

- Extend CanvasResolution with glyph.
- Add glyphFit?: CellGlyphFitOptions to CanvasOptions.
- Reuse fitCellGlyph() rather than creating a second candidate scorer.
- Reuse existing high-resolution color aggregation helpers.
- Preserve existing cell, quad, and braille behavior.
- Keep fitCellGlyph() available as a standalone helper.

Acceptance Criteria

- RED tests cover Unicode diagonal glyph fitting through canvas().
- RED tests cover ASCII glyph fitting through canvas().
- RED tests prove fitted glyph cells retain averaged RGB style.
- npm run test -- packages/bijou-tui/src/canvas.test.ts packages/bijou-tui/src/cell-glyph-fit.test.ts packages/bijou-tui/src/index.test.ts passes.
- npm run --workspace @flyingrobots/bijou-tui lint passes.
- npm run lint passes.
- git diff --check passes.

Design

Add a renderGlyphResolution() sibling to the existing Quad and Braille renderers. It samples a 2x4 grid, builds row-major coverage values, accumulates style from every sample, asks fitCellGlyph() for the output character, and writes one cell.

Coverage defaults to 1 for non-space shader output and 0 for space output. Shaders may return a cell with explicit coverage when they want fractional samples for density or anti-aliasing experiments.

RED

Initial RED run:

- npm run test -- packages/bijou-tui/src/canvas.test.ts failed because resolution: 'glyph' was ignored by canvas().

- The diagonal test observed zero shader calls, the ASCII test received a blank cell, and the RGB averaging test received no fgRGB.

GREEN

canvas() now accepts resolution: 'glyph' and glyphFit options. The new renderer samples 2x4 coverage, accumulates subpixel styles, collapses coverage through fitCellGlyph(), and writes the fitted glyph through the existing cell write path.

ShaderCell now allows optional fractional coverage for apps that want anti-aliased glyph fitting instead of boolean non-space samples.

Drift Check

- npm run test -- packages/bijou-tui/src/canvas.test.ts packages/bijou-tui/src/cell-glyph-fit.test.ts packages/bijou-tui/src/index.test.ts passed with 21 tests.
- npm run --workspace @flyingrobots/bijou-tui lint passed.
- npm run lint passed.
- git diff --check passed.

Playback

Playback answers:

1. fits glyph resolution from 2x4 diagonal coverage proves the shader is called eight times for one terminal cell.
2. The same test proves diagonal coverage chooses / through the existing Unicode candidate set.
3. fits glyph resolution with the ASCII density ramp proves glyphFit: { mode: 'ascii' } stays in the ASCII ramp.
4. averages glyph-fit sub-pixel colors proves glyph-fit rendering keeps the same color aggregation behavior as Braille and Quad.

Retrospective

The right boundary was to promote the reducer into canvas() without removing the standalone fitCellGlyph() helper. Apps can now choose a complete canvas mode for common shader output, while still using the helper directly for custom pipelines.

161

RE-035 Mandatory Layout Envelope And Constraint Negotiation

docs/design/RE-035-mandatory-layout-envelope-and-constraint-negotiation.md

```
title: RE-035 Mandatory Layout Envelope And Constraint Negotiation
legend: RE
lane: design
```

Cycle for making layout envelopes mandatory before any visible UI node can render

Legend:

- RE - Runtime Engine

Depends on:

- RE-003
- RE-004
- RE-006
- DX-022 - Layout Inspector Overlay
- DX-026 - Mode-Lowering Linter

Why This Cycle Exists

Bijou already has retained runtime seams, layout-driven routing, component metadata, focus maps, inspectors, and mode-lowering tools. The missing floor is that visible UI is still allowed to exist as rendered output before it resolves through a mandatory layout envelope.

That leaves too much behavior implicit:

- components can choose private sizing rules
- renderers can effectively decide geometry while painting
- overflow and clipping have no shared prerequisite contract
- direction-aware placement can drift into physical left/right assumptions
- later workspace features can mutate geometry unless there is a stricter boundary to compile into

RE-035 establishes the first boring law: every visible node must resolve a layout envelope before rendering.

North Star

Bijou layout is a retained, semantic, cell-native constraint negotiation system: constraints flow down, preferences flow up, assigned rectangles flow down, and semantic truth lowers outward into interactive, static, pipe, and accessible modes.

The boundary:

- Layout lays out.
- Workspace organizes.
- Interaction commands.
- Renderer paints.
- Modes lower.
- Inspector explains.

Human Users / Jobs / Hill

Primary Human Users

- Bijou maintainers turning layout from render-time convention into runtime truth.
- Component authors who need predictable sizing, clipping, and placement rules without copying CSS or Auto Layout.
- DOGFOOD builders who need narrow, standard, and wide documentation layouts to share one inspectable geometry model.

Human Jobs

1. Define a visible node's layout contract before it renders.
2. Read min, preferred, max, and assigned size facts without reverse-engineering a renderer.
3. Trust parent-owned rectangle assignment instead of child-owned geometry mutation.
4. Explain why stack and place containers assigned a particular integer-cell rectangle.

Human Hill

A maintainer can inspect a visible UI node and answer its identity, role, constraints, preferences, assigned rectangle, direction, and assignment reason before looking at the rendered surface.

Agent Users / Jobs / Hill

Primary Agent Users

- Agents preparing tests for layout and viewport behavior.
- Agents explaining why a component received a rectangle or could not satisfy a constraint.
- Agents generating future component or workspace code that must compile into primitive layout instead of mutating geometry directly.

Agent Jobs

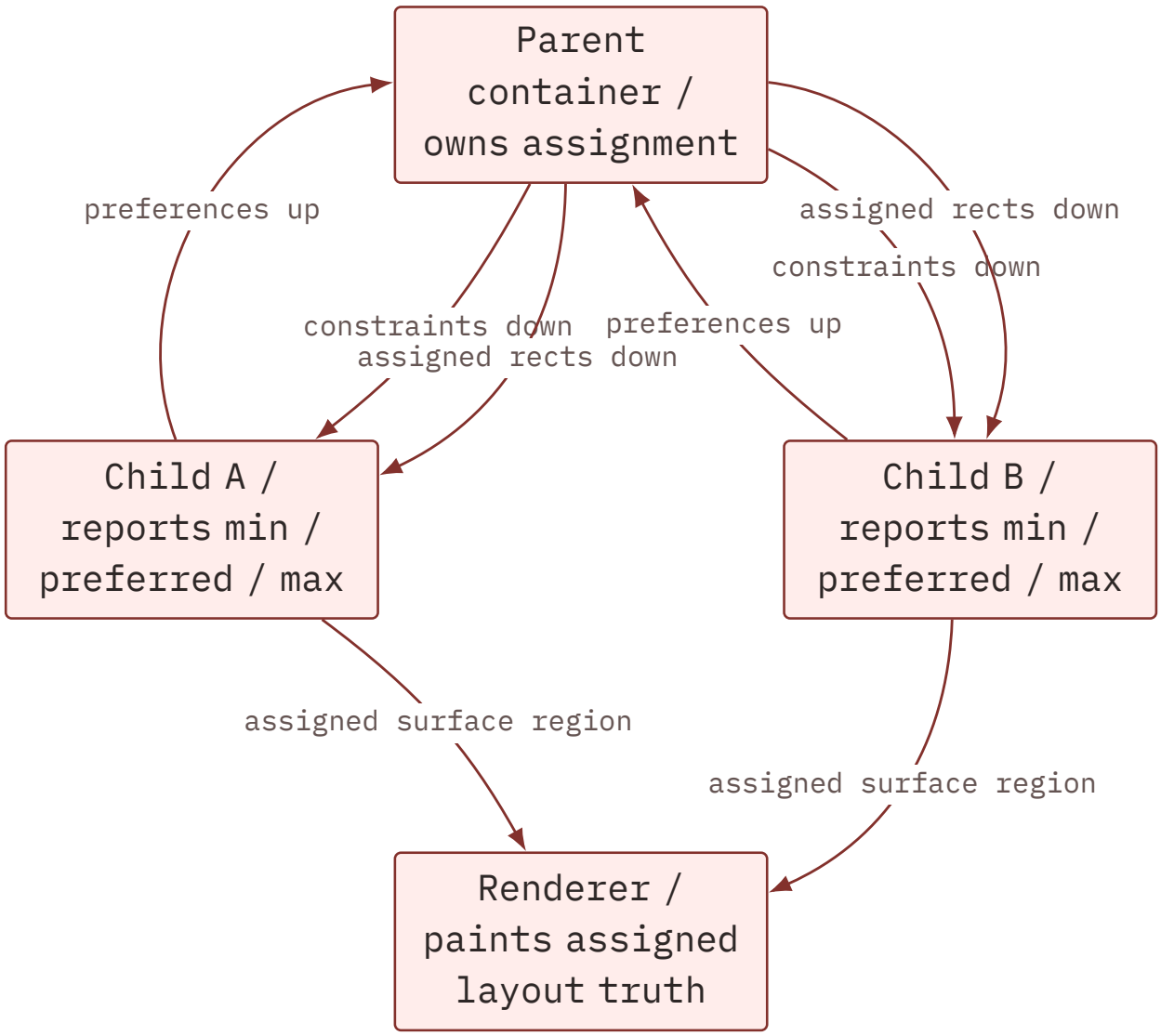
1. Read a layout envelope as plain facts.
2. Predict whether a child preference can affect parent assignment.
3. Generate deterministic assertions for stack assignment, place alignment, and RTL inline-start/inline-end mapping.
4. Distinguish surface or buffer cell extents from text-to-cell measurement.

Agent Hill

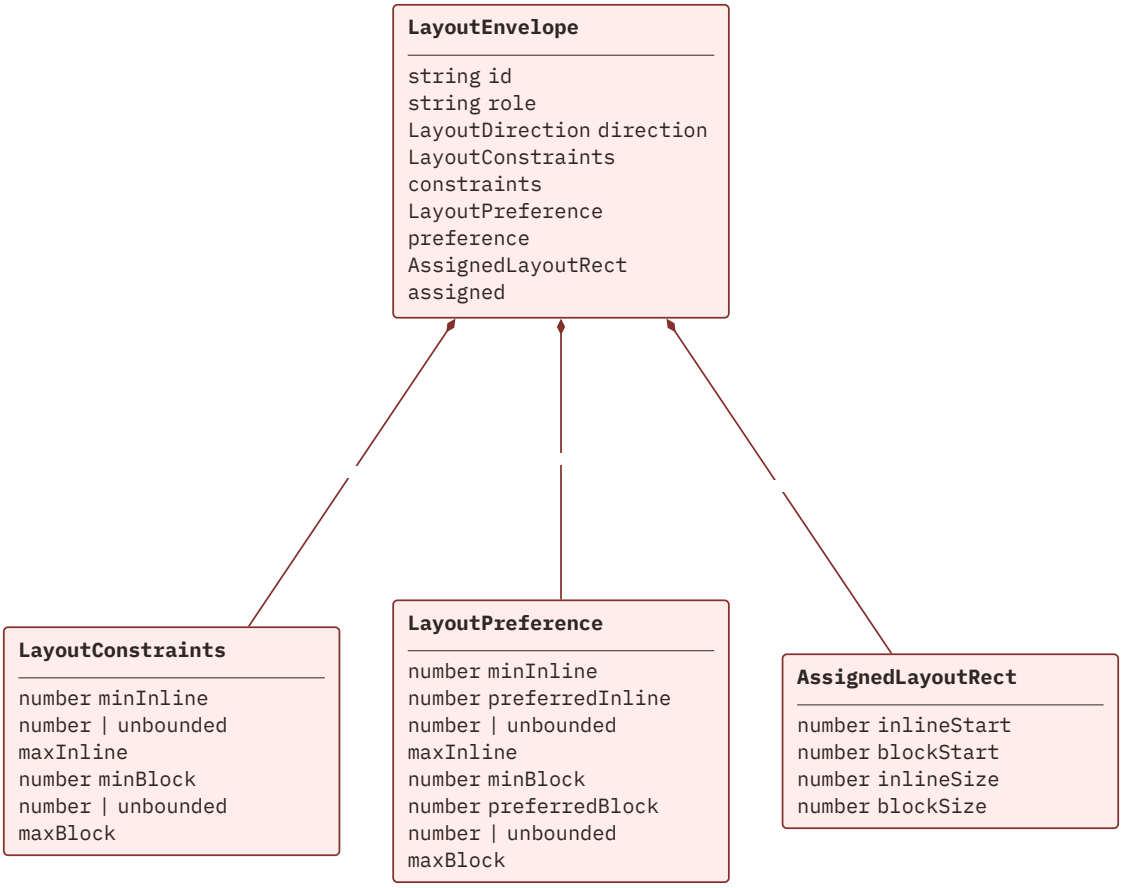
An agent can explain a layout decision from mode-neutral facts: incoming constraints, child preferences, parent assignment, integer rounding, direction, and the reason attached to the resolved rectangle.

Concept Diagrams

These Mermaid diagrams are explanatory mockups, not implementation artifacts. They favor Obsidian-readable semantic structure over raw SVG detail in this cycle doc.

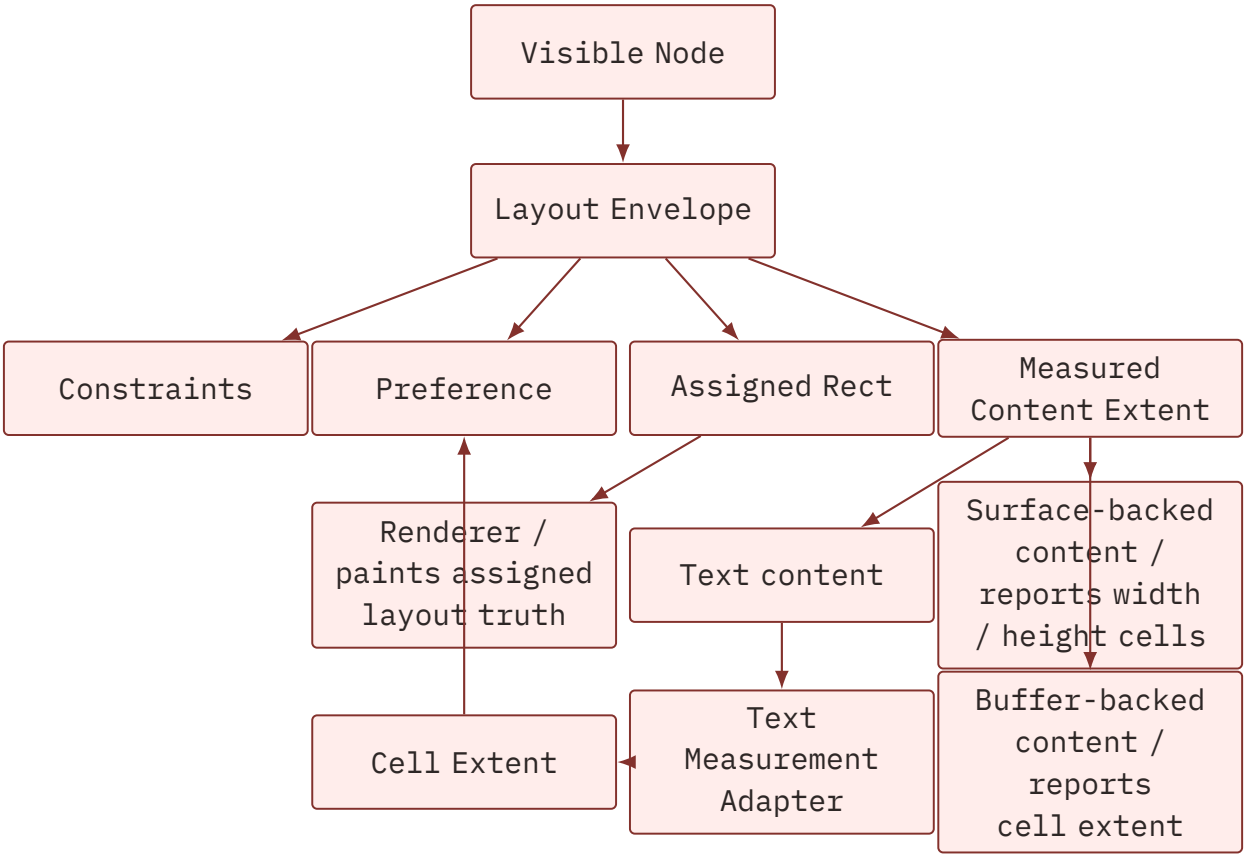


Measure/assign protocol: constraints flow down, preferences flow up, assigned rectangles flow down.



Explicit layout facts

A visible node owns explicit layout facts before rendering.

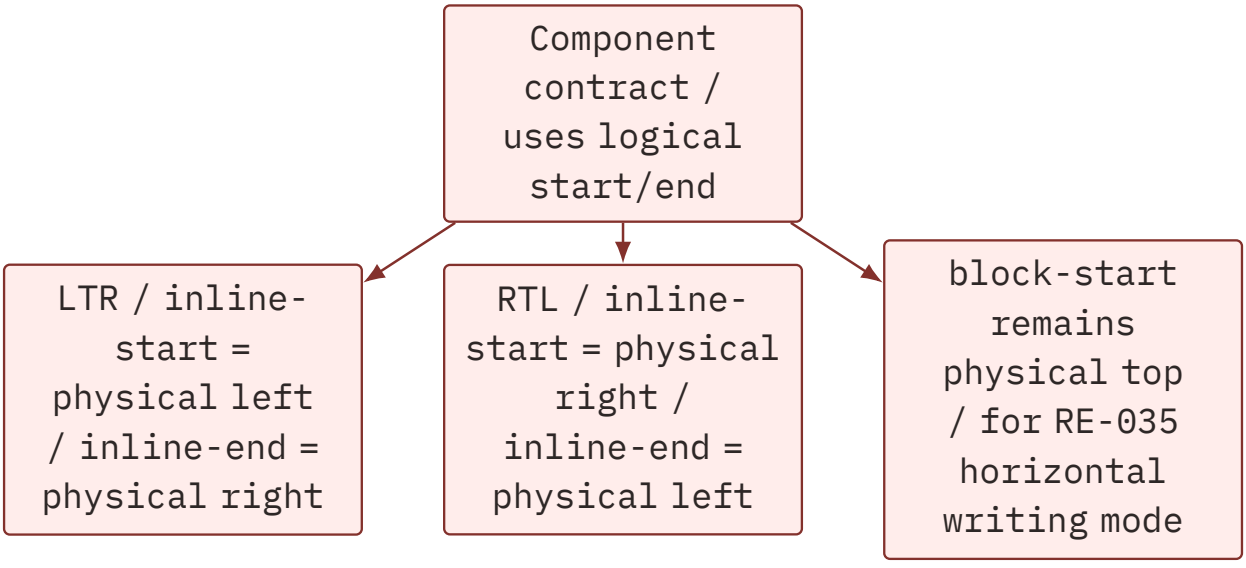


Layout measures cell extents. Text measurement is one adapter into the measurement seam, not the canonical layout model.

```
96 inline cells

[ nav: fixed 24 ][gap 1][ content: flex 52 ][gap 1][ inspector: fixed 18 ]
```

Stack assignment proves the first slice without implementing full flexbox: 96 available - 24 nav - 18 inspector - 2 gaps = 52 flexible cells.



Logical axes let placement flip under RTL without changing source order; the engine derives physical placement from direction.

Human Playback

1. A DOGFOOD shell renders a header, navigation pane, content pane, and status line.
2. The root layout receives a 96-column by 24-row terminal constraint.
3. The header and status line report fixed block preferences.
4. The navigation pane reports a fixed 24-cell inline preference.
5. The content pane reports a flexible inline preference with a useful minimum.
6. The parent stack assigns concrete integer rectangles to each child.
7. The renderer receives those assigned rectangles and paints clipped surfaces inside them.
8. The maintainer opens the layout facts and sees the constraints, preferences, assignment, and reason for the content pane’s 52-cell inline span.

Agent Playback

1. An agent reads a resolved layout envelope for docs.content.
2. It sees the node id, role, direction, incoming constraints, min/preferred/max preferences, assigned rectangle, and assignment reason.
3. It verifies that docs.content did not force parent geometry after assignment.
4. It verifies that RTL direction maps inline-start to the physical right edge for placement tests.
5. It emits a deterministic assertion against the layout facts without parsing rendered text or ANSI output.

Accessibility / Assistive Posture

RE-035 does not build the full accessible layout mode. It must still preserve the facts that later accessible output needs:

- stable node identity
- role
- source order
- direction
- assigned rectangle
- mode-neutral reason text

The first implementation should not hide semantic identity inside surfaces. Accessible and static output can later lower from the same layout truth rather than trying to recover structure from painted cells.

Localization / Directionality Posture

RE-035 introduces logical axes before any new stack or place behavior can settle into physical left/right assumptions.

The first direction scope is intentionally narrow:

- ltr
- rtl
- auto as a declared direction value
- horizontal writing mode only
- inline-start and inline-end mapped through direction
- block-start and block-end kept physical top/bottom for this cycle

Full bidi text layout, cursor maps, and vertical writing modes belong to later text and localization cycles.

Agent Inspectability / Explainability Posture

Every resolved layout node should be explainable as data. The explanation does not need to be a stable public API in the first slice, but tests should prove that the engine can report:

- node id
- role
- direction
- incoming constraints
- min/preferred/max preference facts
- assigned rectangle
- assignment reason

The explanation must not require executing renderer behavior or parsing a surface string.

Linked Invariants

- Runtime Truth Wins
- Layout Owns Interaction Geometry
- The Buffer Holds Facts
- Focus Owns Input
- Tests Are the Spec

Requirements

- Introduce a pure layout envelope model for visible UI nodes.
- Represent logical axes:
  - inline-start
  - inline-end
  - block-start
  - block-end
- Represent layout direction at least as ltr, rtl, and auto.
- Represent size facts separately:
  - minimum size
  - preferred size
  - maximum size
  - assigned size
- Represent incoming constraints as bounded, exact, or unbounded per axis.
- Keep parent assignment authoritative.
- Add a deterministic rounding policy for flexible integer-cell tracks.
- Add initial layout containers only where needed to prove the protocol:
  - stack
  - place
- Add layout explanation facts suitable for inspectors, tests, and agent output.
- Define the content measurement seam for cell extents. Text measurement is one adapter into that seam, not the canonical layout model.
- Reserve the concept of fit policy so later compression, truncation, wrapping, clipping, and overflow work has a deterministic place to attach.

Non-Goals

- Do not implement full text wrapping, grapheme segmentation, bidi layout, or cursor maps. RE-035 defines the text-to-cell measurement seam; RE-036 owns the text measurement work.
- Do not implement scrollbars, scroll anchoring, virtualized viewports, or marquee behavior.
- Do not implement the full fit-policy/compression ladder. RE-035 names the concept; later responsive, viewport, and text cycles define behavior.
- Do not implement docking, tab dragging, drawers, floating panels, workspace persistence, or drag-and-drop rearrangement.
- Do not introduce a new LK legend yet. This is runtime-engine work until the layout kernel becomes its own public subsystem, package, or doctrine track.
- Do not build CSS, Auto Layout, or a general constraint solver.

Contract Sketch

The first public or internal names can change during implementation, but the conceptual shape should stay small:

```
type LayoutDirection = 'ltr' | 'rtl' | 'auto';

type LogicalAxis = 'inline' | 'block';

type LogicalAlign = 'start' | 'center' | 'end' | 'stretch';

type LayoutFitPolicy =
  | 'fixed'
  | 'shrink'
  | 'grow'
  | 'truncate'
  | 'wrap'
  | 'overflow'
  | 'clip'
  | 'priority';

interface LayoutConstraints {
  readonly minInline: number;
  readonly maxInline: number | 'unbounded';
  readonly minBlock: number;
  readonly maxBlock: number | 'unbounded';
}

interface LayoutPreference {
  readonly minInline: number;
  readonly preferredInline: number;
  readonly maxInline: number | 'unbounded';
  readonly minBlock: number;
  readonly preferredBlock: number;
  readonly maxBlock: number | 'unbounded';
}

interface AssignedLayoutRect {
  readonly inlineStart: number;
  readonly blockStart: number;
  readonly inlineSize: number;
  readonly blockSize: number;
}

interface LayoutEnvelope {
  readonly id: string;
  readonly role: string;
  readonly direction: LayoutDirection;
  readonly constraints: LayoutConstraints;
  readonly preference: LayoutPreference;
  readonly assigned: AssignedLayoutRect;
  readonly fit?: LayoutFitPolicy;
}
```

The key rule is not the exact type names. The key rule is that a visible node has explicit constraint, preference, assignment, identity, role, and direction facts before rendering.

Measure And Assign

RE-035 uses a bounded two-step protocol:

1. Measure: a parent asks children for preferences under incoming constraints.
2. Assign: the parent chooses concrete integer rectangles and gives them back to children.

Children may report preferences. They may not force parent size after assignment.

Rendering happens only after assignment.

Layout Fit Policy

RE-035 should reserve fit policy as the node's declared posture under compression, expansion, and overflow.

Constraints say what space is possible. Preferences say what size a node wants. Assignment says what rectangle the parent chose. Fit policy says how the node intends to remain useful when those facts disagree.

Examples:

- fixed: preserve assigned size and let the parent resolve surrounding space
- shrink: accept less than preferred size before siblings with higher priority
- grow: accept extra space after fixed and preferred claims are satisfied
- truncate: preserve one-line or bounded content by shortening presentation
- wrap: trade inline space for block space through a measurement adapter
- overflow: report content extent beyond the assigned rect
- clip: hide paint outside the assigned rect without claiming scroll state
- priority: participate in an ordered compression ladder

The first implementation does not need to execute all of these policies. It should only leave an explicit home for this intent so later text, viewport, responsive, and workspace cycles do not invent private compression rules.

Initial Containers

Stack

stack proves ordered layout along one logical axis.

It should support:

- fixed tracks
- flexible tracks
- gaps
- logical direction
- deterministic leftover-cell rounding
- child explanations

The first slice does not need full flexbox.

Place

place proves alignment inside an assigned rectangle.

It should support:

- inline alignment: start, center, end, stretch
- block alignment: start, center, end, stretch
- logical start/end resolution under LTR and RTL

The first slice does not need absolute positioning, anchors, portals, or overlay placement.

Content Measurement Seam

RE-035 layout measures cell extents, not JavaScript string length.

Bijou surfaces are already rectangular grids of cells with width and height. A surface-backed component can report those dimensions directly as its content extent. Text is different only because it must first be lowered into cells through a measurement adapter.

RE-035 should define where content measurement plugs in, but should not solve the whole text system.

The seam should make room for surface and buffer content that can already report:

- inline cell extent
- block cell extent
- optional baseline facts
- optional semantic facts

The text adapter into that seam should make room for:

- available inline size
- wrap policy
- overflow policy
- layout direction
- ambiguous-width policy
- output mode

The implementation may use a minimal deterministic placeholder for early component tests, but the design must make it clear that no text-bearing component should own private text.length measurement in the long run. String-to-surface conversion belongs behind the text measurement adapter.

Mode-Neutral Facts

The resolved layout should be inspectable without depending on a rendered surface.

Example facts:

```
node docs.content
role viewport
```

```
direction ltr
constraints inline 0..96 block 0..24
preference inline min 32 preferred 72 max unbounded
preference block min 6 preferred 18 max unbounded
assigned inline-start 24 block-start 1 inline-size 72 block-size 22
reason took remaining inline space in stack after fixed nav and status chrome
```

Interactive, static, pipe, and accessible modes may present these facts differently, but the underlying layout truth should be shared.

Implementation Outline

1. Add a pure layout module that can construct and resolve layout envelopes without rendering.
2. Add constraint, preference, assigned-rect, direction, and explanation fact types.
3. Add the two-step measure/assign protocol with parent-owned assignment.
4. Add minimal stack and place helpers to prove the protocol.
5. Add deterministic integer rounding for flexible tracks.
6. Add a content measurement seam that accepts surface/buffer cell extents directly and leaves text-to-cell measurement behind an adapter.
7. Add a render-facing seam that receives assigned rectangles instead of deriving geometry from rendered output.
8. Keep the implementation pure and shell-agnostic.

Design Thinking Slice Plan

This cycle uses the Method loop with a Design Thinking posture: sponsored users, measurable Hills, explicit assumptions, low-ceremony prototypes, and playback after each slice. The team should keep asking whether the slice helps a maintainer or agent inspect layout truth before render.

Sponsored Users

- Sponsored Human: Bijou maintainer closing the v6 release boundary and auditing layout behavior without parsing painted terminal rows.
- Sponsored Agent: Implementation or review agent generating deterministic assertions against layout facts, not renderer strings.

Hill For The First Ten Slices

A maintainer or agent can construct visible layout nodes, resolve parent-owned integer rectangles through stack and place, inspect the explanation facts, and hand the assigned rectangle to a renderer-facing seam without executing a renderer.

Assumptions To Test

- A small pure module in @flyingrobots/bijou can establish layout truth before DOGFOOD or bijou-tui adopts it.
- Mandatory render-facing envelopes can be proven as an adapter seam before every existing component is migrated.
- Surface and buffer extents are already cell-native enough to seed the measurement seam.
- Text measurement can remain an adapter placeholder in RE-035 without smuggling JavaScript string length into the canonical layout model.

Ten Slices

Slice	Prototype	Test Playback
1. Active cycle guard	Record RE-035 as the active runtime-engine design and freeze out RE-036+ scope.	A cycle test proves text flow, viewport chrome, responsive variants, and workspace behavior remain out of scope.
2. Envelope facts	Add constraints, preferences, assigned rects, direction, fit policy, and explanation records.	Tests assert immutable plain data for id, role, direction, constraints, preference, assignment, and reason.
3. Visible render gate	Add a render-facing helper that only accepts resolved envelopes.	Tests prove a visible node cannot enter the render seam without an assigned layout envelope.
4. Measurement seam	Add content extent records for surface and buffer content plus a text-adapter hook.	Tests prove surface/buffer extents flow directly and text measurement remains injectable.
5. Measure/assign protocol	Add pure child measurement and parent-owned assignment helpers.	Tests prove child preferences cannot force parent geometry after assignment.
6. Stack fixed/flex tracks	Add minimal logical stack resolution with gaps and fixed/flexible tracks.	Tests prove fixed and flexible tracks receive deterministic integer rectangles.
7. Rounding policy	Add stable leftover-cell distribution for flexible tracks.	Tests prove extra cells are assigned by a named, repeatable policy.
8. Place alignment	Add logical place alignment inside an assigned rectangle.	Tests prove start, center, end, and stretch along both axes.
9. Direction mapping	Resolve inline-start/inline-end under LTR and RTL for placement.	Tests prove RTL flips inline placement without changing source order.
10. Inspector facts	Add explanation summaries for resolved layout nodes and render assignments.	Tests assert layout explanations without rendering strings or ANSI output.

Playback Cadence

- RED: write the slice test first, tied to the playback question above.
- GREEN: add only the pure runtime code needed for that slice.
- LEARN: update this design doc's Drift Check, Playback, and Retrospective when a slice changes the boundary or leaves follow-on debt.

Tests To Write First

- Cycle test proving RE-035 is the active runtime-engine design and that later text, viewport, chrome, responsive, and workspace work remains out of scope.
- Package-local tests proving visible nodes require a layout envelope before rendering.
- Package-local tests proving child preferences cannot force parent geometry.
- Package-local tests proving stack assigns fixed and flexible tracks deterministically.
- Package-local tests proving leftover cells are rounded by a stable policy.
- Package-local tests proving place aligns children at logical start, center, end, and stretch.
- Package-local tests proving RTL flips inline-start and inline-end placement.
- Package-local tests proving layout explanation facts include node id, role, constraints, preferences, assigned rect, and reason.
- Package-local tests proving a renderer receives the assigned rect from layout resolution.

Acceptance Criteria

- RED tests fail before implementation and pass after.
- Visible UI nodes cannot enter the render-facing seam without a layout envelope.
- Parent assignment is authoritative and deterministic.
- Stack and place prove the minimum useful layout contract without full flexbox, scrollbars, or workspace behavior.
- Surface and buffer content can report cell extents directly.
- Text measurement remains an adapter seam, not the canonical layout model.
- Layout explanations can be asserted without rendering a string.

Risks / Unknowns

- The first contract may be too small for later viewport or workspace needs, so the types should stay extensible without accepting solver complexity.
- Text measurement is the next hard problem, but it should not drag RE-035 into grapheme, bidi, and cursor-map work.
- Existing string-returning component APIs may need compatibility wrappers while the retained layout path matures.
- Mermaid diagrams in editor tooling can drift, so diagrams should stay conceptual and not become implementation specs.

Future Sequence

RE-035 deliberately ships the floor, not the full system.

Follow-on cycles should stay layered:

- RE-036 Text Measurement And Inline Flow
- RE-037 Overflow, Viewports, Scroll Anchoring, And Scrollbars
- RE-038 Box Model, Chrome Regions, Hit Testing, And Focus Maps
- RE-039 Responsive Variants, Compression, And Constraint Fallbacks
- RE-040 Mode Lowering And Accessible Layout Semantics
- WS-001 Workspace Tree: Panels, Tabsets, Docks, Drawers
- WS-002 Workspace Interaction: Resize, Drag/Drop, Persistence

Workspace features compile into layout. Interaction emits commands. Rendering paints resolved rectangles. None of those layers should mutate layout geometry behind the layout engine’s back.

Drift Check

Started on 2026-06-02 as a pure @flyingrobots/bijou core layout slice.

The first ten slices intentionally landed in packages/bijou/src/core/layout/ instead of bijou-tui, DOGFOOD, or shell renderers. That keeps the contract mode-neutral and lets tests assert layout truth before any renderer paints.

Confirmed scope boundaries:

- stack and place are proof containers, not a full flexbox or workspace model
- text content uses an injected measurement adapter, not JavaScript string length
- render-facing code receives assigned rectangles through a seam, but existing components are not migrated in this slice
- layout explanation facts are plain data and do not require ANSI output

Playback

RED:

- Added packages/bijou/src/core/layout/envelope.test.ts before the module existed. The focused test failed because ./envelope.js was missing.

GREEN:

- Added immutable layout constraints, preferences, assigned rectangles, directions, fit policy, and resolved-envelope facts.
- Added a render-facing seam that rejects unresolved visible nodes and passes parent-assigned rectangles to the renderer callback.
- Added surface and buffer cell extent helpers plus an injected text measurement seam.

- Added parent-owned child assignment, minimal stack fixed/flex layout, source-order largest-remainder rounding, place alignment, RTL inline start/end mapping, and inspectable explanation facts.
- Exported the RE-035 primitives from @flyingrobots/bijou.

Retrospective

The first ten slices establish the boring floor: visible layout facts can now exist and be tested before rendering. The main restraint was keeping RE-035 from turning into RE-036 through RE-040. The current module deliberately does not solve text flow, overflow, hit testing, responsive fallbacks, accessible layout lowering, or workspace behavior.

Follow-on adoption should be separate and test-led: migrate one render-facing surface family at a time to consume assigned rectangles, then introduce text measurement, viewport overflow, and hit-test facts as their own cycles.

162

RE-036 Packed Bijou Cells Surface Adapter

docs/design/RE-036-packed-bijou-cells-surface-adapter.md

```
title: RE-036 Packed Bijou Cells Surface Adapter
legend: RE
lane: release
priority: high
github_issue: 459
parent_issue: 457
status: complete
keywords:
  - runtime-engine
  - visor
  - packed-cells
  - surface
  - receipts
  - validation
  - v8.0.0
```

Legend: RE - Runtime Engine

Linked Work

- User story: #459
- V8 tracker: #457
- Broad source lineage: #302
- Landed artifact-bundle proof: #458
- V8 contract design: DX-048 V8 Runtime Graph And Scene IR Contract
- Artifact-bundle design: DX-049 VISOR Artifact Bundle Proof
- VISOR coordination tracker: flyingrobots/visor#1

Decision Summary

Bijou will define packed-bijou-cells/1 as a JSON-shaped terminal receipt whose cell bytes use Bijou's existing ten-byte little-endian packed-cell format, identified as bijou-packed-cell-u8x10-le/1. Glyphs use the unicode-grapheme-side-table/1 policy. The receipt carries exact dimensions, bytes, grapheme side-table entries, scene-node provenance, focus facts, and chroma policy facts. A public validator rejects malformed or non-canonical receipts before any Surface constructor or renderer can sanitize them.

The adapter validates first, creates a packed Bijou Surface, copies the validated bytes and side table without re-encoding cells, and marks the

resulting surface dirty. A successful adaptation therefore preserves the receipt's byte sequence and side-table order exactly. It does not reinterpret the receipt through Cell objects.

Current Truth

Bijou already owns a packed terminal Surface:

- every cell occupies 10 bytes
- character identity is a little-endian unsigned 16-bit value
- direct character values below 0xF000 represent BMP code points
- values at or above 0xF000 index a grapheme side table
- foreground and background colors occupy three unsigned 8-bit channels each
- the flags byte records modifiers and the empty-cell bit
- the alpha byte records six-bit opacity plus foreground/background presence bits

The internal createSurface() boundary sanitizes dimensions. The internal decodeChar() boundary substitutes a space when a side-table reference is missing. Those behaviors are appropriate defensive fallbacks inside the renderer, but they are not valid receipt verification. RE-036 must reject the input before either fallback can hide a malformed external artifact.

DX-048 requires packed cells to map back to scene nodes. #459 also requires focus and chroma metadata to be validated. The existing packed buffer does not carry those facts, so the receipt must carry them beside the byte payload.

Sponsor Human

A maintainer wants to inspect one portable terminal receipt, see the exact scene, cell, glyph, color, focus, and provenance facts it claims, and adapt it into a Bijou Surface without wondering whether the adapter repaired the input.

Sponsor Agent

An agent wants deterministic typed diagnostics for every invalid receipt path and a byte-exact success path it can verify without rendering a live terminal, scraping text, or depending on process-local state.

Hill

Given a checked-in packed-bijou-cells/1 fixture, Bijou can fail closed on invalid shape, dimensions, bytes, glyphs, colors, scene provenance, focus, or chroma metadata, then adapt a valid receipt into a packed Surface whose

dimensions, buffer bytes, side-table order, decoded cells, and scene ownership remain inspectable and deterministic.

Playback Questions

- Which exact packed-cell version and cell format does the receipt use?
- How many cells and bytes does the receipt claim?
- Which scene and scene nodes own each packed cell?
- Which side-table entry supplies each non-direct glyph?
- Which node is focused, and which nodes are focusable?
- Which color space, channel encoding, alpha encoding, and terminal-default policy govern the bytes?
- Can malformed dimensions, colors, glyph references, focus facts, or scene mappings reach createSurface()?
- Does adaptation preserve every byte and side-table entry without normalization?

Scope

- Define the public packed-bijou-cells/1 receipt type and version constants.
- Define a typed deterministic validation error with stable code and path.
- Validate the receipt's exact object shape and nested metadata shapes.
- Validate dimensions, cell count, stride, byte count, and byte values.
- Validate direct glyph values and side-table references.
- Validate canonical side-table entries and reference coverage.
- Validate foreground/background presence-bit color semantics.
- Validate scene-node provenance for every cell.
- Validate focus and chroma metadata.
- Adapt a valid receipt into Bijou's existing packed Surface.
- Prove byte-exact and side-table-exact adaptation with a checked-in fixture.
- Export the contract, validator, error, and adapter through the public Bijou package surface.

Non-Goals

This cycle does not:

- change Bijou's existing packed-cell byte format
- make Geordi a Bijou dependency
- implement a Geordi packed-cell renderer
- implement terminal frame capture, replay, streaming, or compression
- define VISOR storage or indexing behavior
- add browser, WebGPU, image, shader, raster, or native-render targets

- normalize invalid receipts into accepted ones
- infer scene ownership from geometry after receipt creation
- validate an entire ui-scene-ir/1 artifact against the declared scene hash
- redesign Surface, Cell, or terminal diff rendering
- expand #459 into the remaining #302 or #457 tracker scope

Receipt Contract

The public receipt shape is:

```
interface PackedBijouCellsReceipt {
  readonly receiptVersion: 'packed-bijou-cells/1';
  readonly widthCells: number;
  readonly heightCells: number;
  readonly cellFormatId: 'bijou-packed-cell-u8x10-le/1';
  readonly glyphPolicyId: 'unicode-grapheme-side-table/1';
  readonly bytes: readonly number[];
  readonly sideTable: readonly string[];
  readonly scene: {
    readonly sceneVersion: 'ui-scene-ir/1';
    readonly sceneHash: string;
    readonly nodeIds: readonly string[];
    readonly cellNodeIds: readonly string[];
  };
  readonly focus: {
    readonly focusedNodeId: string | null;
    readonly focusableNodeIds: readonly string[];
  };
  readonly chroma: {
    readonly colorSpace: 'srgb';
    readonly channelEncoding: 'uint8';
    readonly alphaEncoding: 'uint6';
    readonly terminalDefaultEncoding: 'presence-bits';
  };
}
```

The receipt is JSON-shaped. It does not carry Uint8Array, Set, Map, functions, symbols, absolute paths, timestamps, process identifiers, terminal handles, or mutable renderer objects.

Dimension And Byte Laws

- widthCells and heightCells are positive safe integers.
- neither dimension exceeds the renderer’s 9,999-cell cursor-coordinate limit.
- widthCells * heightCells is a safe integer.
- the receipt contains at most 100,000 cells.
- cellCount * 10 is a safe integer.
- bytes.length equals cellCount * 10.
- every byte is a canonical integer in the inclusive range 0 through 255; negative zero is rejected
- cellFormatId fixes the stride at 10; no receipt-defined stride exists

The validator checks those laws before calling createSurface(). Zero, negative, fractional, infinite, NaN, unsafe, overflowing, excessive, or byte-length-mismatched dimensions fail.

Glyph Laws

- the character value is decoded from byte offsets 0 and 1 as unsigned 16-bit little-endian
- direct value 0 is rejected; an empty cell uses a space glyph plus the empty-cell flag
- direct values below 0xF000 must not be UTF-16 surrogate code points
- direct values are terminal-control safe graphemes of width 1
- values at or above 0xF000 must reference an existing side-table entry
- every side-table entry is non-empty, terminal-control safe, and exactly one grapheme with terminal width 1
- format-control, mark-only, and default-ignorable-only graphemes are rejected
- every side-table entry contains at most 256 UTF-16 code units and 32 UTF-8 bytes
- directly encodable values below 0xF000 are not valid side-table entries
- side-table entries are unique
- every side-table entry is referenced by at least one cell
- side-table order is authoritative and is never sorted, deduplicated, or reconstructed by the adapter

Rejecting unreferenced entries makes the receipt canonical and prevents multiple byte-equivalent receipts from carrying arbitrary unused glyph data.

Color And Modifier Laws

The six color-channel bytes are always unsigned 8-bit values by the byte law. The alpha byte’s low six bits encode opacity. Bit 6 records foreground presence and bit 7 records background presence.

- when the foreground presence bit is unset, all three foreground channels are 0
- when the background presence bit is unset, all three background channels are 0
- an absent color means terminal default; it does not mean black
- the dashed modifier flag is valid only with the dotted/dashed underline value
- all other modifier and empty-cell bits retain their existing Bijou meaning

The adapter copies those bytes. It does not synthesize terminal defaults, resolve theme tokens, quantize colors, or reconstruct modifier lists.

Scene And Focus Laws

- scene.sceneVersion is exactly ui-scene-ir/1
- scene.sceneHash is a lowercase 64-character SHA-256 hexadecimal digest
- scene.nodeIds is non-empty, unique, and contains canonical non-empty ids
- scene.cellNodeIds.length equals the cell count
- every cell node id occurs in scene.nodeIds
- focus.focusableNodeIds is unique
- every focusable node id occurs in scene.nodeIds
- focus.focusedNodeId is either null or occurs in focus.focusableNodeIds

Every cell has an owning scene node. Background, empty, or padding cells must name the scene node that intentionally emitted them; null ownership is not accepted.

Chroma Laws

The first receipt version supports one explicit chroma profile:

```
colorSpace: srgb
channelEncoding: uint8
alphaEncoding: uint6
terminalDefaultEncoding: presence-bits
```

All four fields are mandatory and exact. Supporting another color space, channel width, alpha encoding, or terminal-default policy requires a new declared receipt or cell-format version. The validator does not silently convert unsupported metadata.

Validation Error Contract

Invalid input throws PackedBijouCellsValidationError with:

```
interface PackedBijouCellsValidationErrorShape {
  readonly code: PackedBijouCellsValidationCode;
  readonly path: string;
  readonly detail: string;
}
```

The message format is deterministic:

```
packed-bijou-cells/1 <code> at <path>: <detail>
```

Codes distinguish invalid shape, unknown field, wrong literal, invalid dimension, invalid byte, byte-length mismatch, invalid glyph, invalid color, invalid modifier, invalid scene mapping, invalid focus, and invalid chroma. Paths use JSON-compatible segments such as $.bytes[17] and $.focus.focusedNodeId. Control and default-ignorable characters are escaped as Unicode escape sequences before entering a diagnostic path.

Validation returns a fresh receipt value only after the entire object passes. No partial receipt or partially created Surface escapes a failure.

Surface Adapter Contract

adaptPackedBijouCellsToSurface(input) performs this sequence:

1. Parse and validate the unknown input.
2. Create a packed Surface with the validated positive dimensions.
3. Copy the validated bytes into surface.buffer.
4. Copy side-table entries in their declared order.
5. Mark all cells dirty for the first terminal render.
6. Return the packed Surface.

The following round-trip assertions are mandatory:

```
surface.width === receipt.widthCells
surface.height === receipt.heightCells
Array.from(surface.buffer) deep-equals receipt.bytes
surface.sideTable deep-equals receipt.sideTable
```

Decoded cells may be inspected as secondary evidence. Re-encoding decoded Cel l objects is not the adaptation mechanism.

Accessibility And Assistive Posture

Packed cells are visual terminal output, not an accessibility tree. The receipt preserves scene-node and focus identity so a later inspector can link the visual proof back to semantic roles and accessible names in ui-scene-ir/ 1. This cycle makes no claim that packed bytes alone provide a screen-reader representation.

The adapter must not erase focus identity or imply that terminal color is the only carrier of meaning. Semantic accessibility remains owned by the source artifact and scene contract.

Localization And Directionality Posture

Side-table validation is Unicode-aware and accepts one terminal-width grapheme per entry. This cycle does not implement bidirectional reordering, wide-cell layout, locale-aware shaping, or grapheme fallback. A producer must lower those concerns before emitting this one-cell-per-glyph receipt.

Rejecting control-bearing, multi-grapheme, or non-unit-width entries prevents the adapter from creating a Surface whose geometry disagrees with the receipt dimensions.

Agent Inspectability And Explainability Posture

An agent can inspect:

- exact version and policy literals

- dimensions and derived cell count
- exact byte payload
- glyph side table and cell references
- scene hash and node inventory
- per-cell scene ownership
- focus inventory and active focus
- exact chroma profile
- a stable validation code and path for failures

No live terminal, component tree, renderer loop, hidden registry, or host state is required to explain success or failure.

Linked Invariants

- The Buffer Holds Facts: the receipt is portable data and the adapter preserves its packed facts.
- Schemas Live At Boundaries: malformed receipts fail with stable code and path.
- Tests Are the Spec: executable fixtures define the accepted receipt and failure behavior.
- Host APIs Stay Behind Adapters: validation and adaptation require no host, filesystem, process, or network capability.

Implementation Outline

1. Synchronize ROADMAP.md and BEARING.md with the landed #468 proof and active #459 lane.
2. Add this design packet and a design-policy regression test.
3. Add a checked-in valid receipt fixture and focused tests that fail because the public validator and adapter do not exist.
4. Implement constants, public types, typed errors, exact-shape validation, and canonical field validation.
5. Implement glyph, color, scene, focus, and chroma validation.
6. Implement the byte-copying packed Surface adapter.
7. Export the public contract through @flyingrobots/bijou.
8. Update reference documentation, changelog, roadmap evidence, and this design's closeout.
9. Run focused tests, package typecheck, lint, Code Dojo gates, docs gates, generators, the full suite, and pre-push verification.

Tests To Write First

- A checked-in valid fixture fails to import through the missing public API.

- The valid fixture adapts with exact dimensions, bytes, and side-table order.
- Unknown top-level and nested fields fail.
- Wrong version and policy literals fail.
- Zero, negative, fractional, unsafe, overflowing, and above-9,999 dimensions fail.
- Byte-length mismatch and non-byte values fail.
- Missing side-table references fail instead of decoding as spaces.
- Surrogate direct glyph values fail.
- Empty, control-bearing, multi-grapheme, duplicate, wide, and unreferenced side-table entries fail.
- Nonzero RGB channels with an absent color presence bit fail.
- Invalid dashed-modifier combinations fail.
- Duplicate or empty scene-node ids fail.
- Missing and unknown per-cell scene ownership fails.
- Unknown, duplicate, or inconsistent focus ids fail.
- Unsupported chroma values fail without conversion.
- Adaptation never calls a renderer and never re-encodes cells through Surface.set() or Surface.setRGB().

Validation Plan

Focused proof:

```
npx vitest run tests/cycles/RE-036
```

Package and static proof:

```
npm run typecheck
npm run lint
npm run code-dojo:verify
npm run docs:inventory
git diff --check
```

Full repository proof:

```
npm run code-dojo:ci
```

If repository script names differ, use the documented owning commands and record the exact successful commands in closeout.

Acceptance Criteria

- packed-bijou-cells/1 is a public documented receipt contract.
- The public validator rejects malformed shape, dimensions, bytes, glyphs, colors, scene mappings, focus facts, and chroma facts deterministically.
- A checked-in fixture adapts into a packed terminal Surface.
- Adapted dimensions, bytes, and side-table order exactly match the fixture.

- Every receipt cell maps to a declared ui-scene-ir/1 node.
- Invalid side-table references never fall through to a replacement space.
- Invalid dimensions never reach dimension sanitation.
- No Geordi, VISOR, host, filesystem, network, or live-terminal dependency is introduced.
- Public types, errors, constants, validator, and adapter are exported.
- Focused and full repository gates pass.

Closeout Notes

RE-036 exports parsePackedBijouCellsReceipt() and adaptPackedBijouCellsToSurface() from @flyingrobots/bijou. The validator rejects non-plain or inexact JSON shapes, unsafe or excessive dimensions, non-byte payloads, non-canonical glyphs, terminal-default color drift, invalid modifier combinations, incomplete scene ownership, inconsistent focus, and unsupported chroma before surface allocation.

The checked 2 by 2 fixture preserves all 40 bytes and both side-table entries through adaptation. It includes direct, combining, and private-use glyph evidence; explicit and terminal-default colors; per-cell scene ownership; and focus metadata. The adapter copies the payload directly into a packed Surface, preserves side-table order, and marks every cell dirty without calling Surface.set() or Surface.setRGB().

Focused RE-036 verification passes 16 tests across 5 suites. Full npm run code-dojo:ci verification passes 4,031 tests across 891 test files. Code Dojo holds its 62-violation aggregate baseline: 37 file/context, 25 code-size, 0 mock-ban, and 0 ESLint. npm run docs:inventory, targeted Markdown lint, and git diff --check pass.

Contract shaping is commit b1d57e95; implementation and public documentation are commit 669d10c7. Remove work-in-progress from #459 only after review, CI, merge, and issue closeout complete.

163

RE-041 v7.2 Framework Input Stabilization

docs/design/RE-041-v72-framework-input-stabilization.md

```
title: RE-041 v7.2 Framework Input Stabilization
legend: RE
lane: release
priority: high
github_issue: 354
status: proposed
keywords:
  - runtime-engine
  - input-routing
  - mouse
  - driver
  - exports
  - v7.2.0
```

Legend: RE - Runtime Engine

Linked Work

- Goalpost: #354
- Mouse routing bug: #344
- Public export polish: #345
- Mouse driver builders: #353
- Release milestone: v7.2.0

This is the first implementation pull for the v7.2.0 stabilization release. It does not replace the v8.0.0 Runtime Graph and Scene IR product-contract horizon.

Decision Summary

The v7.1.0 release video rehearsal exposed two classes of debt:

1. Product/demo seams in DOGFOOD that make the release story harder to show.
2. Framework input and developer-experience bugs that real consumers hit while building pointer-aware layouts.

The first v7.2.0 pull should repair the framework input path because it is small, high-confidence, and foundational for later DOGFOOD polish. The selected slice fixes workspace mouse fallthrough, exports the page-scoped

app-frame helpers that integrations already need, and adds reusable mouse driver helpers so future pointer regressions can be written as deterministic tests.

Sponsored Human

A framework user wants a page to receive hover, release, right-click, and other unmapped workspace pointer events without writing wrapper code around a pp.update.

A maintainer wants to test those cases without hand-authoring raw mouse messages in every regression.

Sponsored Agent

An agent wants pointer behavior to be inspectable through small test-driver helpers and public app-frame message wrappers instead of brittle imports from compiled package internals.

Hill

Given a framed app with an active page, unmapped workspace mouse events fall through to that page, routed shell mouse actions still belong to the frame, and tests can express mouse move, press, release, wheel, and SGR sequence playback through public @flyingrobots/bijou-tui helpers.

Scope

- #344: make resolveRoutedMouseLayer return handled: false for workspace mouse actions that the shell does not own.
- #353: add public test-driver helpers for common mouse actions and raw SGR mouse sequence parsing.
- #345: export the page-scoped app-frame message helper surface from packages/bijou-tui/src/index.ts.
- Add focused regression coverage for workspace mouse fallthrough and the new helper/export contract.
- Update docs and changelog evidence.

Non-Goals

- Do not add pane-level mouse masks or dynamic terminal tracking modes.
- Do not rewrite frame routing or page focus ownership.

- Do not add worker-thread rendering, adaptive frame budgets, or mutable raster surfaces.
- Do not make the broader #302 Runtime Graph contract part of this PR.
- Do not repair every DOGFOOD demo issue in the first framework-input pull.

Current Evidence

createFramedApp routes mouse messages through resolveRoutedMouseLayer. When the shell route result is handled, the update loop drains shell commands and does not call updateTargetPage.

The workspace branch currently owns left-click pane selection and scroll, but it also marks the rest of the workspace mouse stream as handled. That swallows movement, releases, and non-left presses before the active page can decide whether it wants them.

runScript() already accepts structured mouse steps, and parseMouse() can parse SGR mouse escape sequences. The missing part is a small helper layer that turns common pointer intent into valid script steps.

PAGE_MSG_TOKEN, wrapPageMsg, emitMsgForPage, and wrapCmdForPage already exist in app-frame-types.ts. Consumers should not need to import them through deep build paths.

Playback Questions

1. Does an unmapped workspace mouse move reach the active page update function?
2. Does a workspace release or right-click also fall through instead of being swallowed?
3. Do frame-owned left-click pane selection and scroll behaviors remain routed by the shell?
4. Can tests express mouse move, press, release, wheel, and SGR input without duplicating raw MouseMsg objects?
5. Can package consumers import the page-scoped frame message helpers from the @flyingrobots/bijou-tui root?

Tests To Write First

- A focused app-frame regression that fails before the #344 fix by proving a workspace move event does not reach the active page.
- Additional focused app-frame assertions for release and non-left press fallthrough.
- Driver helper unit tests for move, press, release, wheel, and SGR sequence helpers.
- A package-root import test for PAGE_MSG_TOKEN, wrapPageMsg, emitMsgForPage, and wrapCmdForPage.

Acceptance Criteria

- Unmapped workspace mouse actions return handled: false and fall through to the active page.
- Shell-owned workspace mouse actions continue to work.
- Mouse driver helpers are public, typed, documented, and used by at least one app-frame regression.
- Page-scoped frame helper exports are available from the package root.
- Focused tests, typecheck, lint, documentation inventory, and diff whitespace checks pass before review.

164

RE-127 - Command And Render Runtime Guardrails

docs/design/RE-127-command-and-render-runtime-guardrails.md

Linked issues: #127, #129

Linked legend: RE - Runtime Engine

Sponsor Human

A runtime maintainer or app author who needs command and render behavior to degrade into visible diagnostics instead of opaque hangs or frame stalls.

Sponsor Agent

A runtime verification agent that checks deterministic command queue, middleware, and framebuffer invariants.

Hill

Runaway command production and accidental async render middleware are observable and testable, while normal high-volume command workloads remain possible.

Scope

- Expose EventBus command queue diagnostics.
- Add a configurable command backpressure warning threshold.
- Route runtime backpressure warnings through stderr and routeRuntimeIssue().
- Diagnose render middleware that returns a Promise.
- Prevent late async next() calls from re-entering the synchronous pipeline.

Playback Questions

1. Can a test observe pending command count before commands settle?
2. Does a command loop crossing the threshold fire exactly one overload diagnostic until it drains below the threshold?
3. Does async render middleware continue the current frame synchronously?
4. Does a late async next() fail to re-run downstream stages?

Accessibility / Assistive Reading Posture

Runtime warnings should be short, concrete, and routed through existing stderr and application runtime-issue surfaces.

Localization / Directionality Posture

Warnings are framework diagnostics, not localized user-facing app copy in this cycle.

Agent Inspectability / Explainability Posture

Command diagnostics are explicit data via getCommandDiagnostics(), and pipeline diagnostics are asserted through deterministic tests.

Linked Invariants

- Render stages remain synchronous and stage-ordered.
- Command effects may be async, but their queue state must be observable.
- Diagnostics must not impose a hard cap on legitimate workloads.

Decisions

- Backpressure is a warning, not a hard cap. Legitimate bursts can exceed the threshold, but the runtime reports when pending commands cross it.
- The default threshold is 1000 pending commands. Set commandBackpressureThre shold: 0 to disable the warning.
- Render middleware remains a synchronous contract. Promise-returning middleware is diagnosed, the frame continues synchronously, and Promise rejection is reported.

Verification

- npm test -- --run packages/bijou-tui/src/pipeline/pipeline.test.ts packages/bijou-tui/src/eventbus.test.ts
- npm run typecheck:test

Tests To Write First

- EventBus backpressure threshold tests.
- Pipeline async-middleware continuation and rejection tests.

Retrospective

See WF-122 day zero audit fixes.

165

RE-142 - Node Bootstrap Failure Contract

docs/design/RE-142-node-bootstrap-failure-contract.md

Linked issues: #142, #143, #144

Linked legend: RE - Runtime Engine

Sponsor Human

A Node host app author who needs startup failures to explain whether the terminal, raw mode, stdout dimensions, or ambient context is the problem.

Sponsor Agent

A runtime verification agent that can assert a stable bootstrap error contract instead of matching ad hoc process or terminal errors.

Hill

initDefaultContext() startup failures are represented by BijouBootstrapError with stable reason, hint, and optional cause fields, and the docs explain the expected recovery paths.

Scope

- Add tests for the structured bootstrap error object.
- Add a deterministic stdout-dimension failure test.
- Document the reason/hint/cause contract in @flyingrobots/bijou-node docs.
- Preserve existing behavior for successful context initialization.

Playback Questions

1. Does BijouBootstrapError preserve reason, hint, and cause?

2. Does initDefaultContext() produce a structured error for unusable stdout dimensions?
3. Do package docs explain the recovery path for common host startup failures?

Accessibility / Assistive Reading Posture

Hints should be direct recovery text that a human can act on from a terminal error or test failure.

Localization / Directionality Posture

Bootstrap diagnostics remain developer-facing English strings in this cycle.

Agent Inspectability / Explainability Posture

Tests assert the stable error fields directly, so agents should not need to parse terminal-specific stack traces.

Linked Invariants

- Host initialization failures need structured facts.
- Ambient process state should not be the only way tests can reason about startup failures.

Verification

- npm test -- --run packages/bijou-node/src/index.test.ts
- npm run typecheck:test

Tests To Write First

- Structured error object test.
- Deterministic stdout dimension failure test.

Retrospective

See WF-122 day zero audit fixes.

Design language design records

This family connects principles to visible and inspectable interface contracts: patterns, component families, focus and gutter treatment, theme tokens, contrast and rule provenance. Theme Inspector and Theme Lab appear here as concrete surfaces for exercising those ideas. Follow an interaction claim into the implementation and its evidence when using a historical design to understand current controls.

166

DL-001 — Capture Design Language Doctrine

docs/design/DL-001-capture-design-language-doctrine.md

Legend: DL — Design Language

Why this cycle exists

Bijou already has strong local instincts:

- truthful controls
- focus ownership
- humane shell behavior
- canonical preference-list treatment
- localization and accessibility ambition

But those instincts are still split across:

- shell work
- DOGFOOD polish
- scattered strategy notes
- conversation context

This cycle exists to turn those instincts into explicit design language.

Scope of this cycle

This cycle intentionally covers:

- creating a dedicated Design Language legend
- capturing doctrine for:
  - mouse input
  - cognitive load
  - accessibility and assistive posture
  - localization and bidirectionality
  - AI explainability
  - content style

- parking notification polish explicitly on the Humane Terminal backlog so doctrine work can lead

It does **not** include:

- implementing notification polish
- shipping new shell or component behavior
- creating the full blocks library

The point is to make the north star explicit before more product surfaces drift.

Human users

Primary human user

A Bijou builder or maintainer deciding how new shell and component work should feel.

They need to:

- reason about product quality above the component level
- know which UX choices are canonical
- understand how accessibility, i18n, copy, and explainability fit together

Human hill

A builder can describe Bijou’s design language as a coherent system rather than a pile of good-looking exceptions.

Agent users

Primary agent user

An agent generating, auditing, or reviewing Bijou UX work.

It needs to:

- evaluate UX changes against written doctrine
- distinguish invariant violations from taste disagreements
- reason about humans and agents as separate first-class audiences

Agent hill

An agent can review or propose Bijou UX with explicit doctrine, playbacks, and backlog direction instead of inferring unwritten norms.

Human playback

1. A maintainer opens the design-language docs.
2. They can identify the relevant doctrine for shell behavior, accessibility, localization, AI, and copy.
3. They can see why notifications are parked on the backlog rather than being the next reactive polish slice.
4. They can point to a next backlog item for turning doctrine into reusable patterns and blocks.

Agent playback

1. An agent reads the design-language legend and cycle doc.
2. It can locate the doctrine docs that govern shell and component UX.
3. It can locate the parked notification backlog item.
4. It can identify the next follow-on cycle for patterns/blocks.

Linked invariants

- Tests Are the Spec
- Focus Owns Input
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- doctrine should remain above component inventory
- agents remain first-class users in design thinking, not an afterthought
- notification work should stay parked as backlog while doctrine is captured
- future blocks should follow doctrine and patterns, not precede them

Deliverable

Land a design-language legend and enrich the doctrine set so Bijou has explicit guidance for:

- humane interaction
- accessibility
- localization
- AI explainability
- content

Then park notification polish on the backlog and spawn the next design-language backlog item for patterns and blocks.

Tests to write first

Under tests/cycles/DL-001/:

- the Design Language legend exists and links to the doctrine set
- the active cycle doc includes human/agent playbacks and linked invariants
- the doctrine docs explicitly mention the newly captured concerns:
  - mouse input
  - cognitive load
  - localization catalogs / references / spreadsheet reality
  - [AI] labeling
  - future blocks
- notification polish is explicitly parked on the Humane Terminal backlog

Risks

- creating a second doctrine layer that duplicates existing notes without clarifying them
- keeping the design language too abstract to guide actual reviews
- letting notification work stay implied instead of explicitly parked

Out of scope

- implementing notification-center improvements
- blocks library implementation
- new shell or component code

Retrospective

What landed

- a dedicated Design Language legend:
  - docs/legends/DL-design-language.md
- enriched doctrine across:
  - docs/strategy/bijou-ux-doctrine.md
  - docs/strategy/accessibility-and-assistive-modes.md
  - docs/strategy/localization-and-bidirectionality.md
  - docs/strategy/ai-explainability-standard.md

    - docs/strategy/content-guide.md
- an explicit Humane Terminal backlog item for notification polish:
    - docs/method/retro/HT_001-notification-center-polish-and-discoverability.md

Drift from ideal

No material drift.

This cycle intentionally stayed documentation-only and did not try to fold the doctrine into code or new shell surfaces yet.

Debt spawned

Spawned:

- DL-002 — Canonicalize Patterns and Blocks

167

DL-002 — Canonicalize Patterns and Blocks

docs/design/DL-002-canonicalize-patterns-and-blocks.md

Legend: DL — Design Language

Why this cycle exists

DL-001 captured the doctrine layer.

That was necessary, but it still leaves Bijou with a gap between:

- doctrine
- component families
- shell/product work

This cycle exists to bridge that gap by making the most proven Bijou interaction language explicit:

- selection treatment
- active-region treatment
- drawer and panel rhythm
- preference-row/value layout
- the difference between patterns and future blocks

Scope of this cycle

This cycle intentionally covers:

- documenting canonical patterns in the design-system layer
- explicitly defining what Bijou means by a block
- identifying the first credible block candidates
- updating the Design System read order so patterns and blocks are both first-class
- spawning the next backlog slice for proving these patterns in shared surfaces

It does **not** include:

- shipping a full blocks library

- replacing the app frame with a new block system
- inventing new shell behavior
- reworking notification center polish, which remains parked on the Humane Terminal backlog

Human users

Primary human user

A Bijou builder or maintainer trying to turn doctrine into reusable interface decisions.

They need to know:

- which interaction treatments are canonical
- what visual language means selection versus focus versus support text
- how future blocks should be composed without inventing a parallel design language

Human hill

A builder can point to concrete Bijou patterns and block rules instead of translating abstract doctrine into ad hoc component choices.

Agent users

Primary agent user

An agent reviewing or generating Bijou UI work.

It needs to:

- identify whether a surface follows canonical selection and spacing rules
- distinguish a reusable pattern from a future reusable block
- know which next cycle should turn the documented patterns into more shared code

Agent hill

An agent can evaluate or propose Bijou surfaces against explicit pattern language instead of inferring those rules from scattered shell code.

Human playback

1. A maintainer opens the Design System docs.

2. They can find an explicit explanation of canonical selection treatment, active-region treatment, drawer rhythm, and value-overflow behavior.
3. They can tell what counts as a block and why blocks come after patterns and components.
4. They can see the next backlog item for turning those documented patterns into more shared product surfaces.

Agent playback

1. An agent reads the DL legend and DL-002 cycle doc.
2. It can locate the design-system docs that define canonical patterns and block rules.
3. It can determine whether selection, focus, and spacing treatments are now documented in one place.
4. It can point to the next backlog item for implementation-oriented follow-through.

Linked invariants

- Tests Are the Spec
- Focus Owns Input
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- background fill should remain the canonical selected-item treatment
- structural accents such as gutters should remain the canonical active-region treatment
- values should stack or reflow before truncating into ambiguity
- blocks should be downstream of doctrine, patterns, and components
- notification work remains parked while the design-language bridge is made explicit

Implementation outline

1. Move DL-002 into the active cycle area and enrich it with playbacks and invariants.
2. Write cycle tests that demand:
   - canonical pattern language in the design-system docs
   - an explicit blocks document

    - updated design-system read order
    - a spawned follow-on backlog item
3. Update the design-system docs so the canonical patterns are written down where builders will actually read them.
4. Close the cycle with a retrospective and next-step backlog item.

Tests to write first

Under tests/cycles/DL-002/:

- the active cycle doc includes the required workflow sections
- the design-system docs explicitly define canonical selection, active-region, spacing, and overflow patterns
- the design-system docs explicitly define what a block is and name the first block candidates
- the Design Language legend points at DL-002 as the latest completed cycle and spawns the next backlog item

Risks

- documenting patterns too vaguely to guide implementation reviews
- duplicating doctrine without actually bridging it into builder-facing docs
- overcommitting to blocks before the component/pattern vocabulary is stable

Out of scope

- implementation of a reusable blocks runtime
- service-level notification work
- localization adapters or other legend work

Retrospective

What landed

- explicit canonical-pattern guidance in docs/design-system/patterns.md for:
    - selected rows
    - active regions
    - truthful visible controls
    - spacing rhythm
    - value-overflow behavior

- a new docs/design-system/blocks.md document that defines what a block is, where it sits in the design-system stack, and which first candidates are credible
- updated design-system read order in docs/design-system/README.md

Drift from ideal

No material drift.

This cycle stayed documentation-first on purpose. It names the canonical patterns and block layer clearly, but it does not yet prove those rules in more shared implementation surfaces.

Debt spawned

Spawned:

- DL-003 — Prove Canonical Patterns in Shared Surfaces

168

DL-003 — Prove Canonical Patterns in Shared Surfaces

docs/design/DL-003-prove-canonical-patterns-in-shared-surfaces.md

Legend: DL — Design Language

Why this cycle exists

DL-002 makes the pattern and block language explicit.

The next step should prove that those patterns are not just prose by applying them to more shared surfaces.

This cycle intentionally picks the smallest meaningful proving slice:

- shared collection rows outside preference lists
- canonical full-row selection treatment
- one-cell inset rhythm in rich TUI surfaces

The best first candidates are:

- browsableListSurface()
- commandPaletteSurface()

Both already represent canonical shared collection behavior. Both currently still lean too much on a glyph-only focus cue in the rich surface path.

Scope of this cycle

This cycle intentionally covers:

- proving canonical selected-row background treatment in shared TUI surfaces
- proving one-cell inset rhythm in those same rich surfaces
- doing that in browsableListSurface() and commandPaletteSurface()

It does **not** include:

- notification-center polish
- drawer rhythm and inspector surfaces

- a general block runtime
- changing the lowering-friendly string paths just to match rich TUI ornament

Human users

Primary human user

A builder using Bijou collection surfaces in a rich TUI shell.

They need:

- focused rows to read as clearly selected
- collection surfaces to feel consistent with the now-canonical settings language
- more than a tiny leading glyph to understand current selection in dense shells

Human hill

A builder can place a browsable list or command palette in a shell and get canonical selected-row treatment without hand-adding shell-local styling.

Agent users

Primary agent user

An agent reviewing or generating collection-style TUI surfaces.

It needs:

- explicit evidence that shared surfaces follow the documented selected-row treatment
- a reliable distinction between selected rows and active regions
- the ability to point at concrete shared-surface proof, not just doctrine prose

Agent hill

An agent can verify that shared list-like surfaces now honor the design-language rules for selection and spacing instead of guessing from one-off shell examples.

Human playback

1. A builder renders a browsable list in a rich TUI.
2. The focused row reads as a full selected row, not just a glyph.

3. The same builder renders a command palette.
4. The palette uses the same selection logic and inset rhythm, so the shell feels related instead of custom per surface.

Agent playback

1. An agent opens the DL-003 cycle doc.
2. It can identify the proving surfaces for the canonical selection pattern.
3. It can run cycle tests and see concrete row-background and inset assertions for those surfaces.
4. It can point to the next backlog item for continuing the proof into drawer/notices territory.

Linked invariants

- Tests Are the Spec
- Focus Owns Input
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning

Invariants for this cycle

- selected rows in rich TUI surfaces should use background fill, not only a glyph
- active-region treatment remains distinct from selected-row treatment
- one-cell inset rhythm should improve readability without obscuring core content
- lowering-friendly string paths should remain simpler unless the semantic contract requires more

Implementation outline

1. Move DL-003 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests that prove selected-row background fill and inset rhythm in shared surfaces.
3. Update browsableListSurface() and commandPaletteSurface() to honor the canonical selected-row pattern.
4. Add package-local regression coverage.
5. Close the cycle with the next backlog item for drawer/notices follow-through.

Tests to write first

Under tests/cycles/DL-003/:

- the active cycle doc includes the required workflow sections
- browsableListSurface() uses a full selected-row background and one-cell inset
- commandPaletteSurface() uses a full selected-row background and one-cell inset
- the next design-language backlog item exists

Risks

- changing the lowering-friendly text path when only the surface path needs the richer treatment
- conflating active-region styling with selected-row styling
- introducing bespoke helper logic that is harder to reuse than the surfaces it is meant to unify

Out of scope

- drawer and inspector rhythm
- notification rows
- future block promotion

Retrospective

What landed

- browsableListSurface() now uses canonical full-row selected-row background treatment and one-cell inset rhythm in the rich surface path
- commandPaletteSurface() now uses the same selected-row and inset treatment for its search line and result rows
- package-local regressions now lock those behaviors in alongside the cycle-owned playback test

Drift from ideal

No material drift.

This cycle intentionally proved the pattern language in two shared collection surfaces without trying to generalize the entire shell or create a new public collection-row component family yet.

Debt spawned

Spawned:

- DL-004 — Prove Drawer Rhythm and Notice Rows

169

DL-004 — Prove Drawer Rhythm and Notice Rows

docs/design/DL-004-prove-drawer-rhythm-and-notice-rows.md

Legend: DL — Design Language

Why this cycle exists

DL-003 proved canonical selected-row treatment in shared collection surfaces.

The next proof slice should move up one layer into shell-adjacent review surfaces:

- drawer rhythm
- inspector-style spacing
- notice and archive rows
- supporting-copy hierarchy in shell review surfaces

The strongest current proving seam is the shell notification center.

It already has:

- shell ownership
- live and archived review concerns
- stack/archive sections
- supporting copy and metadata

But it still renders too much of that surface as a wrapped text blob.

This cycle exists to prove calmer drawer rhythm and reusable notice-row treatment through that seam.

Scope of this cycle

This cycle intentionally covers:

- the notification-center drawer
- live notice rows
- archived notice rows
- one-cell inset rhythm and calmer section spacing

It does **not** include:

- notification-center discoverability polish
- settings-drawer redesign
- a general block runtime
- action/focus behavior changes for notification review

Human users

Primary human user

A shell user reviewing runtime notices in a drawer.

They need:

- section rhythm that feels calm instead of packed
- current and archived notices that read like rows, not like a log blob
- supporting copy that looks supporting

Human hill

A user can open the notification drawer and parse live versus archived notices quickly without the drawer feeling like a pasted text dump.

Agent users

Primary agent user

An agent inspecting or reviewing shell notification surfaces.

It needs:

- explicit row structure in notice-review surfaces
- stable spacing and section boundaries
- reusable proof that drawer review surfaces follow the design language

Agent hill

An agent can point to concrete drawer/notice-row behavior that follows the documented design language instead of reverse-engineering a string blob.

Human playback

1. A user opens the shell notification center.
2. The drawer presents summary lines with breathing room.
3. The current stack is visually separated from archived history.

4. Each notice reads as a compact row with inset primary and secondary text.

Agent playback

1. An agent opens the DL-004 cycle doc.
2. It can identify the shell notification center as the proving seam.
3. It can run cycle/package tests and see explicit assertions for inset drawer rhythm and structured notice rows.
4. It can point to the next backlog item for follow-on drawer/block work.

Linked invariants

- Tests Are the Spec
- Focus Owns Input
- Visible Controls Are a Promise
- Shell Owns Shell Concerns

Invariants for this cycle

- drawer review surfaces should use one-cell inset rhythm by default
- notice rows should separate primary title from supporting metadata and explanation
- shell review surfaces should remain clearly shell-owned, not page-local
- this cycle should prove calmer structure without changing notification action semantics

Implementation outline

1. Move DL-004 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests for drawer rhythm and notice-row structure.
3. Add a surface-native notice review-row path in notification.ts.
4. Update the app-frame notification-center drawer to compose that surface instead of a wrapped text blob.
5. Close the cycle and spawn the next design-language backlog item.

Tests to write first

Under tests/cycles/DL-004/:

- the active cycle doc includes the required workflow sections

- notification review rows use one-cell inset rhythm and keep supporting copy secondary
- the shell notification-center drawer shows section spacing between summary, live stack, and archived history
- the next design-language backlog item exists

Risks

- burying more bespoke drawer layout inside app-frame.ts
- making notice rows too ornamental instead of calm
- changing shell behavior instead of only structure and rhythm

Out of scope

- notification discoverability
- notification action routing
- general block promotion

Retrospective

What landed

- a new surface-native notification review-row path in notification.ts
- archived notification history now has a structured surface renderer instead of only a wrapped string path
- the shell notification-center drawer now composes inset summary lines, spaced section headers, live notice rows, and archived history rows as surfaces
- cycle and package tests now lock the calmer drawer rhythm and structured notice-row treatment in place

Drift from ideal

No material drift.

This cycle intentionally kept behavior stable and only proved calmer structure and hierarchy in the notification review seam.

Debt spawned

Spawned:

- DL-005 — Prove Inspector and Guided Flow Rhythm

170

DL-005 — Prove Inspector and Guided Flow Rhythm

docs/design/DL-005-prove-inspector-and-guided-flow-rhythm.md

Legend: DL — Design Language

Why this cycle exists

DL-004 proved calmer drawer rhythm and structured notice rows in the shell notification center.

The next design-language proof should move into another likely future block candidate:

- guided flows
- explainability walkthroughs
- compact multi-section review surfaces with one obvious next action

Inspector panels are still part of the original idea, but they should not share a cycle with guided-flow proof if that forces us to invent two half-finished surfaces.

This cycle exists to prove the guided-flow side first by turning the AI Explainability Standard into a canonical component instead of leaving it as doctrine only.

Scope of this cycle

This cycle intentionally covers:

- one canonical explainability() component in @flyingrobots/bijou
- explicit [AI] labeling
- separated rationale, evidence, next action, and governance sections
- calm section rhythm for compact guided-flow review surfaces
- graceful lowering into pipe and accessible modes

It does **not** include:

- a generalized guided-flow block runtime
- inspector-panel implementation

- shell integration or notification-center changes
- service-backed AI integrations

Human users

Primary human user

A builder or app user reviewing an AI-mediated recommendation.

They need:

- visible [AI] labeling
- an explanation of why the recommendation exists
- evidence separated from the recommendation itself
- one clear next action
- a calm surface instead of a decorative wall of text

Human hill

A user can read an AI-mediated recommendation and answer what it is, why it is here, what evidence supports it, and what they should do next without feeling like they are parsing marketing copy.

Agent users

Primary agent user

An agent generating or auditing guided-flow and explainability surfaces in Bijou.

It needs:

- one canonical explainability component to target
- explicit section structure rather than ad hoc copy blocks
- proof that guided-flow rhythm follows the design language and AI doctrine

Agent hill

An agent can point to one reusable explainability surface and show that Bijou’s [AI], rationale, evidence, and next-action doctrine is real code instead of only prose.

Human playback

1. A user encounters an AI-mediated recommendation.
2. The surface marks itself visibly as [AI].

3. The recommendation title appears before supporting explanation.
4. Rationale, evidence, and next action are separated into calm sections.
5. The user can see what to do next without guessing whether the recommendation is advisory or authoritative.

Agent playback

1. An agent opens the DL-005 cycle doc.
2. It can identify the canonical proving seam as explainability() rather than a shell-specific one-off.
3. It can run cycle and package tests and see explicit assertions for [AI] labeling, section separation, and graceful lowering.
4. It can point to the next backlog item for inspector-panel proof work.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- AI-mediated explainability surfaces must mark themselves explicitly
- evidence must not be merged into recommendation copy
- the surface must present one obvious next action when that data exists
- pipe and accessible modes must preserve the same meaning without relying on borders or color
- this cycle must prove one canonical guided-flow surface before returning to inspector panels

Implementation outline

1. Move DL-005 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests for the new explainability surface and its lowering behavior.
3. Implement explainability() in @flyingrobots/bijou.
4. Export it through the canonical component barrels.
5. Close the cycle and spawn the inspector-panel follow-on backlog item.

Tests to write first

Under tests/cycles/DL-005/:

- the active cycle doc includes the required workflow sections
- explainability() marks AI visibly and separates rationale, evidence, and next action
- accessible lowering preserves the meaning of the same fields without visual-only cues
- the next design-language backlog item exists

Risks

- making the component feel like AI decoration instead of explanation
- overfitting the component to one future product instead of a reusable pattern
- smuggling shell or service assumptions into a core component

Out of scope

- inspector-panel proof work
- runtime agent integrations
- persistent review history
- generalized block promotion

Retrospective

What landed

- a new canonical explainability() component in @flyingrobots/bijou
- explicit [AI] labeling as part of the component instead of ad hoc copy
- calm section rhythm for rationale, evidence, next action, and governance
- graceful lowering that preserves the same meaning in pipe and accessible modes
- cycle and package tests that make the explainability doctrine executable instead of only descriptive

Drift from ideal

Inspector-panel proof work did not land in this cycle.

That was intentional and correct.

Trying to prove guided-flow and inspector language in the same cycle would have forced two half-finished surfaces instead of one canonical explainability component.

Debt spawned

Spawned:

- DL-006 — Prove Inspector Panel Rhythm

171

DL-006 — Prove Inspector Panel Rhythm

docs/design/DL-006-prove-inspector-panel-rhythm.md

Legend: DL — Design Language

Why this cycle exists

DL-005 proved the guided-flow side of the design-language follow-on through a canonical explainability() surface.

The inspector side is still waiting:

- side panels with current-value emphasis
- compact titled sections
- calm supporting-copy hierarchy
- one obvious relationship between the main task and the side context

This cycle exists to prove that rhythm through one canonical inspector() component instead of hiding it inside a DOGFOOD-specific box.

Scope of this cycle

This cycle intentionally covers:

- one canonical inspector() component in @flyingrobots/bijou
- current-selection emphasis as part of the component instead of ad hoc pane copy
- compact titled sections with calmer supporting-copy hierarchy
- graceful lowering into pipe and accessible modes
- one real proving seam in DOGFOOD's variants pane

It does **not** include:

- a generalized inspector block runtime
- shell notification-center redesign
- settings-drawer changes
- localization or bidirectionality work beyond preserving meaning in lowering

Human users

Primary human user

A builder using a narrow side panel to understand the currently selected thing without losing sight of the main task.

They need:

- one obvious current selection
- compact sections instead of a prose blob
- supporting information that stays visually secondary
- a side panel that clarifies context instead of competing with the main pane

Human hill

A user can glance at a side panel and understand what is selected, what supporting context matters, and how that context relates to the main task without reading a wall of prose.

Agent users

Primary agent user

An agent generating or auditing side-panel detail surfaces in Bijou.

It needs:

- one canonical inspector surface to target
- explicit current-value and section structure
- proof that inspector-panel rhythm is part of the design language instead of a one-off DOGFOOD treatment

Agent hill

An agent can point to one reusable inspector() component and show that Bijou's inspector-panel language is executable code and tests rather than only prose.

Human playback

1. A user selects a component story in DOGFOOD.
2. The variants side pane shows the currently selected variant clearly.
3. Supporting context appears in compact titled sections instead of one unstructured text block.
4. The active value is easier to scan than the supporting description.

5. The side panel feels supplemental and calm rather than louder than the main docs pane.

Agent playback

1. An agent opens the DL-006 cycle doc.
2. It can identify the canonical proving seam as inspector() plus the DOGFOOD variants pane.
3. It can run cycle and package tests and see explicit assertions for current-selection emphasis, titled sections, and graceful lowering.
4. It can point to the next backlog item for inspector-panel follow-on work.

Linked invariants

- Tests Are the Spec
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- inspector surfaces must make the current selection more obvious than supporting copy
- supporting sections must stay structured instead of collapsing into a text blob
- the side panel must clarify context without pretending to own the primary task
- pipe and accessible modes must preserve the same fields without relying on borders or color
- this cycle must prove one canonical inspector surface before promoting a generalized inspector block

Implementation outline

1. Move DL-006 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle tests for a canonical inspector() surface and the DOGFOOD proving seam.
3. Implement inspector() in @flyingrobots/bijou.
4. Export it through the canonical component barrels.
5. Replace the bespoke DOGFOOD variants detail box with the canonical inspector surface.
6. Close the cycle and spawn the next inspector follow-on backlog item.

Tests to write first

Under tests/cycles/DL-006/:

- the active cycle doc includes the required workflow sections
- inspector() preserves current selection and titled sections in pipe mode
- accessible lowering preserves the same fields explicitly
- DOGFOOD variants pane renders current selection through the inspector language
- the next design-language backlog item exists

Risks

- making the component too generic too early
- letting inspector surfaces become decorative boxes instead of calm contextual panels
- overfitting the component to DOGFOOD without proving reusable structure

Out of scope

- generalized block promotion
- notification-center reuse
- bidirectional panel mirroring
- richer shell inspector routing

Retrospective

What landed

- a new canonical inspector() component in @flyingrobots/bijou
- explicit current-selection emphasis as part of the component instead of a DOGFOOD-specific box
- compact titled sections with calmer supporting-copy hierarchy
- DOGFOOD's variants side pane now proves the inspector language through a real product seam
- cycle and package tests that make inspector-panel rhythm executable instead of only descriptive

Drift from ideal

This cycle did not promote a generalized inspector block runtime.

That was intentional and correct.

The right proof step was one reusable component plus one real product seam, not a larger block abstraction before reuse and ownership are clearer.

Debt spawned

Spawned:

- DL-007 — Promote Inspector Panel Block

172

DL-007 — Promote Inspector Panel Block

docs/design/DL-007-promote-inspector-panel-block.md

Legend: DL — Design Language

Why this cycle exists

DL-006 proved the first canonical inspector seam through inspector() and the DOGFOOD variants pane.

That gave Bijou a reusable component, but not yet a reusable block.

The design-system block acceptance bar says a structure is ready to become a block when:

- it recurs across more than one surface or product need
- its interaction ownership is stable
- its patterns are already canonical elsewhere
- its lowering story is explicit

This cycle exists to clear that next bar by promoting inspector language into a reusable TUI block helper instead of leaving two drawer-based inspector surfaces to keep hand-composing strings.

Scope of this cycle

This cycle intentionally covers:

- one reusable inspectorDrawer() block helper in @flyingrobots/bijou-tui
- one refinement to inspector() so block composition does not require nested decorative chrome
- proving the block in two existing inspector-drawer examples
- clear separation between core inspector content semantics and TUI block ownership

It does **not** include:

- a generalized shell inspector runtime
- inspector state management beyond what the host app already owns

- notification-center adoption
- guided-flow block promotion

Human users

Primary human user

A builder composing shell-attached supplemental context next to an existing primary task.

They need:

- a reusable inspector drawer that already feels calm and structured
- one obvious current selection
- stable drawer ownership instead of inventing new overlay conventions
- no nested-box visual noise

Human hill

A builder can attach a supplemental inspector drawer to a shell surface and get calm current-selection emphasis plus compact titled sections without hand-drawing another ad hoc panel.

Agent users

Primary agent user

An agent generating or auditing drawer-attached contextual side panels in Bijou.

It needs:

- one block-level helper to target instead of reconstructing drawer + inspector composition
- explicit proof that inspector language now recurs across more than one real surface
- clear boundaries between core component semantics and TUI block ownership

Agent hill

An agent can point to one canonical inspectorDrawer() helper, prove it in two example surfaces, and explain what belongs to the core inspector() component versus the block helper.

Human playback

1. A builder opens the compact app-frame demo and toggles its inspector drawer.
2. The drawer shows the active page/context through the same calm inspector language instead of ad hoc lines.
3. The builder opens the canonical release-workbench inspector drawer.
4. The same block language appears there too, even though the surface is richer and region-aware.
5. The builder can see that inspector panels are now a reusable block, not just one good-looking detail box in DOGFOOD.

Agent playback

1. An agent opens the DL-007 cycle doc.
2. It can identify the promotion seam as inspectorDrawer() plus two existing example surfaces.
3. It can run cycle and package tests and see explicit assertions for drawer-attached inspector structure in both examples.
4. It can point to the next design-language backlog item once the cycle is closed.

Linked invariants

- Tests Are the Spec
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Shell Owns Shell Concerns
- Docs Are the Demo

Invariants for this cycle

- inspector block promotion must prove recurrence in more than one real surface
- drawer ownership must stay with the shell/TUI layer, not leak into the core component
- core inspector() semantics must remain usable outside the block
- block composition must not rely on nested decorative chrome to communicate structure
- pipe and accessible modes must preserve the same current-selection and section meaning

Implementation outline

1. Move DL-007 into the active cycle area and enrich it with playbacks and invariants.
2. Write failing cycle and package tests for a reusable inspectorDrawer() helper.
3. Refine inspector() only as needed for clean block composition.
4. Implement inspectorDrawer() in @flyingrobots/bijou-tui.
5. Migrate the compact app-frame demo and canonical release-workbench drawer to the block helper.
6. Close the cycle and spawn the next design-language backlog item.

Tests to write first

Under tests/cycles/DL-007/:

- the active cycle doc includes the required workflow sections
- inspectorDrawer() renders current-selection inspector content inside drawer chrome without losing lowering semantics
- the compact app-frame example uses the promoted block
- the canonical release-workbench uses the promoted block
- the next design-language backlog item exists

Risks

- promoting a block too early and freezing the wrong responsibilities
- adding so much API shape that the helper becomes harder to use than raw dra wer()
- coupling the helper too tightly to one example instead of a reusable shell pattern

Out of scope

- generalized inspector routing/state
- notification-center migration
- localization-specific inspector work
- guided-flow block promotion

Retrospective

What landed

- a new inspectorDrawer() helper in @flyingrobots/bijou-tui

- a chrome-less composition mode for inspector() so the block can use drawer chrome without nested decorative boxes
- the compact app-frame demo now proves the block in a smaller shell surface
- the canonical release-workbench now proves the block in a richer pane-scoped shell surface
- cycle and package tests that make inspector block promotion executable instead of only aspirational

Drift from ideal

This cycle did not introduce a generalized inspector runtime or inspector state model.

That was intentional and correct.

The right promotion step was one block helper composed from stable patterns and ownership rules, not a larger shell subsystem.

Debt spawned

Spawned:

- DL-008 — Promote Guided Flow Block

173

DL-008 — Promote Guided Flow Block

docs/design/DL-008-promote-guided-flow-block.md

Legend: DL — Design Language

Why this cycle exists

DL-005 proved the guided-flow side of the design language through explainability().

That gave Bijou one canonical component seam, but not yet a reusable block for calmer non-AI assistance surfaces.

The design-system block acceptance bar says a structure is ready to become a canonical block when:

- it recurs across more than one surface or product need
- its interaction ownership is stable
- its patterns are already canonical elsewhere
- its lowering story is explicit

This cycle exists to clear that next bar by promoting guided-flow language into a reusable block helper instead of leaving explainability to carry every calm recommendation or setup surface by itself.

Scope of this cycle

This cycle intentionally covers:

- one reusable guidedFlow() block helper in @flyingrobots/bijou
- block-level section rhythm, multi-step assistance, and next-action treatment
- reuse of the same block rhythm by explainability()
- proof of the block in DOGFOOD

It does **not** include:

- a wizard runtime or step-routing subsystem
- shell-owned guided-flow overlays or drawer state

- AI/service integrations beyond existing explainability work
- layout and viewport formalization

Human users

Primary human user

A builder composing one calm guided recommendation, setup brief, or review path without inventing a full wizard or hiding the action inside generic prose.

They need:

- one reusable block that separates summary, steps, supporting context, and next action
- a calmer default than another titled box with ad hoc copy
- a path that works for both AI and non-AI guided assistance
- lowering that preserves the same meaning in pipe and accessible modes

Human hill

A builder can render one guided recommendation or setup path with a clear next action and calmer multi-step structure without hand-rolling the same section rhythm again.

Agent users

Primary agent user

An agent generating or auditing guided-assistance surfaces in Bijou.

It needs:

- one block-level helper to target instead of reconstructing section rhythm from raw strings
- proof that explainability is now one specialization of a broader guided-flow block language
- explicit boundaries between content-block ownership and shell/workflow ownership

Agent hill

An agent can point to one canonical guidedFlow() helper, show that explainability() now rides the same block rhythm, and prove the block in DOGFOOD.

Human playback

1. A builder opens DOGFOOD and inspects the guidedFlow() story.
2. The story shows summary, steps, supporting context, and one obvious next action inside one calm grouped surface.
3. The builder compares that story with explainability().
4. The shared rhythm is visible, but explainability still keeps its AI provenance and evidence semantics.
5. The builder can now choose between a generic guided flow and an AI explainability surface instead of forcing everything through one specialized component.

Agent playback

1. An agent opens the DL-008 cycle doc.
2. It can identify the promoted seam as guidedFlow() in the core package.
3. It can run cycle and package tests and see proof for direct guided-flow rendering, explainability reuse, and DOGFOOD presence.
4. It can point to the next design-language backlog item once the cycle is closed.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Visible Controls Are a Promise
- Docs Are the Demo

Invariants for this cycle

- guided-flow promotion must not force shell ownership into a core block
- explainability() must remain AI-specific even while sharing block rhythm
- one obvious next action must stay more visible than supporting prose
- pipe and accessible modes must preserve summary, steps, sections, and next action without decorative chrome
- the promoted block must be proven in at least one non-AI surface

Implementation outline

1. Move DL-008 into the active cycle area and enrich it with playbacks and invariants.

2. Write failing cycle and package tests for a reusable guidedFlow() block plus explainability reuse.
3. Implement guidedFlow() in @flyingrobots/bijou.
4. Refactor explainability() onto the shared guided-flow rhythm.
5. Prove the block in DOGFOOD and update the design-system language.
6. Close the cycle and point the legend at the next live design-language item.

Tests to write first

Under tests/cycles/DL-008/:

- the active cycle doc includes the required workflow sections
- guidedFlow() renders calmer multi-step assistance and one explicit next action
- explainability() still preserves AI-specific guided-flow semantics
- DOGFOOD proves the promoted block
- the next design-language backlog item exists

Risks

- making the block so generic that it loses the rhythm it is meant to standardize
- letting explainability collapse into generic guidance and lose AI specificity
- overreaching into shell routing or wizard ownership before the block itself is stable

Out of scope

- generalized workflow state machines
- shell-owned guided-flow overlays
- editable step state
- layout and viewport doctrine work

Retrospective

What landed

- a new guidedFlow() block helper in @flyingrobots/bijou
- shared guided-flow rhythm between the new block and explainability()
- DOGFOOD proof for the generic non-AI guided-flow case
- cycle and package tests that make the block promotion executable

Drift from ideal

This cycle did not introduce a wizard runtime or shell-owned workflow surface.

That was intentional and correct.

The right promotion step was the calmer content block first, not a larger shell workflow subsystem.

Debt spawned

Next:

- DL-009 — Formalize Layout and Viewport Rules

174

DL-011 Theme Contrast Doctor

docs/design/DL-011-theme-contrast-doctor.md

```
title: DL-011 Theme Contrast Doctor
legend: DL
lane: design
```

Framing

Bijou has explicit token doctrine, but no first-party tool that verifies whether a theme follows it. Poor contrast, invalid colors, and suspicious token reuse can survive until manual visual review.

DL-011 adds a pure theme doctor in core. It checks concrete Theme objects and returns a deterministic report that apps, docs, tests, and future CLI adapters can render however they want.

Sponsored Users

- Framework maintainers reviewing built-in and custom theme changes.
- Third-party theme authors who need actionable feedback without opening a TUI.
- Agent workflows that need stable reports for contrast and token hygiene.

Hills

1. A theme author can run a pure doctor against a Theme and get structured issues for invalid colors, low contrast, and overused color values.
2. A maintainer can pass explicit foreground/background token pairs and minimum ratios to enforce house rules.
3. A test can assert the doctor report without depending on terminal rendering.

Playback Questions

- Does the doctor flag invalid foreground or background hex strings?
- Does the doctor calculate WCAG-style contrast ratios for token pairs?
- Does the doctor flag weak explicit foreground/background token pairs?
- Does the doctor detect suspicious color reuse when a reuse limit is supplied?
- Does the report carry stable issue kinds, severities, paths, ratios, and messages?
- Is the doctor exported from the core theme barrel and root package barrel?

Requirements

- Add doctorTheme(theme, options?).
- Add themeContrastRatio(foreground, background).
- Support explicit contrast pairs with per-pair minimum ratios.
- Support a maximum color reuse threshold.
- Return structured reports; do not throw for theme quality issues.

Acceptance Criteria

- Focused RED tests fail before implementation and pass after.
- Invalid colors are reported as errors.
- Low contrast and color reuse are reported as warnings.
- Contrast ratios are deterministic and rounded for reporting.
- Docs and changelog describe the doctor.

Implementation Outline

- Implement the doctor in packages/bijou/src/core/theme/doctor.ts.
- Export it from packages/bijou/src/core/theme/index.ts and packages/bijou/src/index.ts.
- Keep the first slice pure and adapter-free. CLI rendering can build on the structured report later.

Drift Check

- Scope stayed pure and adapter-free. The doctor returns structured reports; CLI/TUI rendering can build on that later.
- Missing token families are only reported for explicit contrastPairs. The concrete Theme type already makes built-in families present at compile time.

- Mode-lowering risks remain follow-on work. The first slice focuses on color validity, contrast, and suspicious reuse.

Playback

- doctorTheme() reports invalid foreground and background token colors as errors.
- Explicit foreground/background token pairs are checked with WCAG-style contrast ratios and per-pair minimums.
- maxColorReuse reports suspicious repeated color values as warnings.
- Reports include stable issue kinds, severities, paths, colors, ratios, limits, counts, and messages.
- The doctor and contrast helper are exported from the core theme barrel and root package barrel.

Retrospective

- Keeping the doctor in @flyingrobots/bijou rather than bijou-tui was the right ownership call. Themes are core objects and should be diagnosable without a runtime shell.
- The first pass intentionally does not prescribe a universal contrast policy. Explicit pairs let teams encode house rules without making every decorative border token follow text contrast thresholds.

175

DL-012 Separate focus gutter chrome from scrollbar UI tokens

docs/design/DL-012-separate-focus-gutter-from-scrollbar-ui-tokens.md

Legend: DL - Design Language

Date

2026-05-16

Sponsor human

A Bijou app author tuning a framed shell theme who needs focused pane chrome to be themed separately from scrollbars and local accent affordances.

Sponsor agent

An agent auditing theme usage in DOGFOOD and first-party frame components that must explain why a pane gutter, scrollbar thumb, and local active item are not all the same semantic token.

Hill

Focused pane gutters use a dedicated built-in ui.focusGutter token, while scrollbars stay on ui.scrollThumb / ui.scrollTrack and component-local active emphasis stays on semantic.accent.

Description

DOGFOOD exposed a vocabulary gap in the built-in theme contract. Focused pane gutters are reusable shell chrome, but they were still effectively treated as

accent-colored geometry. Scrollbars already had named ui.scrollThumb and ui.scrollTrack tokens, so focus chrome and scroll chrome could drift together visually without any explicit semantic separation.

The fix belongs in the token system and first-party defaults, not in one-off app palette tweaks.

Decision

Add ui.focusGutter to the built-in UI token vocabulary.

The first-party defaults are:

- focused pane gutter: ui.focusGutter
- unfocused pane gutter: semantic.muted
- scrollbar thumb: ui.scrollThumb
- scrollbar track: ui.scrollTrack
- local active/focused affordance inside a component: semantic.accent

focusedGutterToken remains a per-pane escape hatch for deliberate shell variants, but the stock focus-area and framed-app path should not require it for ordinary focused pane chrome.

Playback questions

1. Does BaseUiKey include focusGutter?
2. Do all built-in presets and shipped JSON themes define ui.focusGutter?
3. Does focusArea() use ctx.ui('focusGutter') for the default focused gutter instead of semantic.accent?
4. Do surface-rendered scrollbars use ctx.ui('scrollThumb') and ctx.ui('scrollTrack') by default?
5. Does DOGFOOD publish focus gutter color through its shell theme instead of only through pane-local ad hoc tokens?
6. Do theme authoring docs explain when to use ui.focusGutter versus semantic.accent?

Accessibility / assistive reading posture

No new accessible output is added. The change improves semantic clarity in the theme graph so future accessible inspectors can name focused shell chrome as shell chrome rather than generic accent usage.

Localization / directionality posture

No localized text or directionality behavior changes. The token name is logical and does not imply left/right placement.

Agent inspectability / explainability posture

Theme inspectors and future Storybook-like tooling can now report ui.focusGutter as a first-class shell token. That makes pane focus chrome auditable without reverse-engineering whether a component happened to borrow semantic.accent.

Implementation outline

1. Add focusGutter to the built-in BaseUiKey vocabulary.
2. Add the token to presets, DTCG conversion, shipped JSON themes, and test fixtures.
3. Update focus-area defaults for focused gutters and scrollbar cells.
4. Move DOGFOOD shell themes onto ui.focusGutter for focused pane chrome.
5. Update theme authoring and token-vocabulary docs.

Tests / validation

- packages/bijou/src/core/theme/tokens.test.ts
- packages/bijou/src/core/theme/dtcg.test.ts
- packages/bijou/src/core/theme/dtcg.fuzz.test.ts
- packages/bijou/src/core/theme/presets.test.ts
- packages/bijou-tui/src/focus-area.test.ts
- packages/bijou-node/src/index.test.ts

Retrospective

Closed by adding ui.focusGutter to the built-in theme contract, updating first-party focus-area defaults to consume shell UI tokens, and moving DOGFOOD focused gutter styling into its shell theme vocabulary.

176

DL-013 Design token and theme builder API

docs/design/DL-013-design-token-theme-builder-api.md

```
title: DL-013 Design token and theme builder API
legend: DL
lane: cool-ideas
priority: medium
github_issue: 308
keywords:
  - design-language
  - themes
  - tokens
  - styles
  - builder-api
```

Legend: DL - Design Language

Decision Summary

Tokens describe semantic color slots in a theme.

Themes own mode-specific token values.

Dark and light are the required first theme modes.

Styles consume resolved tokens at render time.

Styles do not register tokens, own theme values, or decide dark/light mode.

This design records the first official Bijou shape for a Lip Gloss-like builder API without copying Lip Gloss' object model wholesale. Bijou should expose a declarative style builder, but the theme graph must remain explicit because Bijou already treats rendering, lowering, and inspectability as first-class contracts.

The short relationship is:

```
Theme
  contains modes
    contain token color values

Style
  contains layout and rendering rules
  may reference token ids
  resolves those references through one theme mode during render
```

Sponsored Human

A Bijou app author wants to declare semantic colors once and reuse them across dark and light modes so that components can stay visually coherent without passing raw RGB values through every call site.

Sponsored Agent

An agent auditing a Bijou app wants theme tokens and style use to be separable and inspectable so that it can explain whether a rendered cell came from a semantic theme slot, a raw color override, or a lower-mode fallback.

Hill

A Bijou app author can define semantic color tokens once inside a theme, provide required dark and light mode values, and apply those tokens through a fluent style builder without allowing styles to become the owner of product theme truth.

Problem

Bijou has styleable surfaces, palettes, theme presets, and DOGFOOD chrome, but the authoring API still makes authors think in implementation details too early. A Lip Gloss-like builder can make styling feel natural, but a naive builder can blur three different responsibilities:

- declaring which semantic colors exist
- assigning concrete colors to those semantic colors for dark and light modes
- applying resolved colors and layout rules to rendered content

The user-facing syntax should be fluent, but the model should stay strict. Theme data is application and product identity. Style data is component presentation. Token references are the bridge.

Playback Questions

1. Can a theme define the same token id with separate dark and light values?
2. Does a style keep token references unresolved until render receives a theme and mode?
3. Can an inspector explain which theme, mode, token id, and lowering profile produced a rendered cell color?
4. Do unresolved tokens fail or fall back through an explicit policy instead of silently degrading?
5. Does the lower-mode path preserve meaning when truecolor is unavailable?

Scope

This spec covers the first color-token API only:

- semantic color token ids
- dark and light theme modes
- theme-local token registration and validation
- token references inside styles
- raw color escape hatches inside styles
- deterministic lower-mode color resolution
- runtime inspectability facts for token resolution

Non-Goals

This slice does not design every future design-token category.

- No spacing tokens yet.
- No typography tokens yet.
- No motion tokens yet.
- No border-shape tokens yet.
- No global mutable token registry.
- No runtime terminal background-color query requirement.
- No implementation of the builder API in this planning slice.

Those can become follow-on issues once the color-token and theme-mode contract is landed.

Core Vocabulary

Token

A token is a semantic color slot inside a theme. It has a stable id such as color.status.danger.bg, but it does not carry a single universal color by itself. Its value comes from a theme mode.

Good token ids name intent, not pigment:

```
color.surface.canvas.bg
color.surface.canvas.fg
color.status.danger.bg
color.status.danger.fg
color.chrome.focusGutter
color.chrome.scrollThumb
color.brand.primary
```

Bad token ids encode mode or raw color:

```
color.dark.red900
color.light.blueAccent
color.hex.ff0000
```

Theme

A theme is an immutable named collection of modes. It is the owner of token values. A theme can declare that the same token has different concrete colors in dark and light modes.

Mode

A mode is a named token-value map inside a theme. dark and light are the required first modes because they cover the highest-value terminal theme split without requiring terminal background probing in the first implementation.

Additional modes can be designed later, for example highContrast, deuteranopia, or print.

Style

A style is an immutable builder for presentation. It can hold foreground, background, border, padding, margin, width, wrapping, alignment, and other rendering rules. When a style uses a token, it stores a token reference rather than a concrete color.

The concrete color is resolved only when rendering receives a theme and mode.

Proposed API

The primary API should optimize for the common case: define a theme, define styles that reference semantic tokens, then render with a selected mode.

```
import {
  defineTheme,
  style,
  tokenRef,
} from '@flyingrobots/bijou';

const theme = defineTheme()
  .id('bijou.default')
  .label('Bijou Default')
  .mode("dark", mode => mode
    .token('color.status.danger.bg')
    .color({ r: 144, g: 0, b: 0 })
    .token('color.status.danger.fg')
    .color({ r: 255, g: 230, b: 230 }))
  .mode("light", mode => mode
    .token('color.status.danger.bg')
    .color({ r: 255, g: 230, b: 230 })
    .token('color.status.danger.fg')
    .color({ r: 144, g: 0, b: 0 }))
  .build();

const warning = style()
  .padding(0, 1)
  .foreground(tokenRef("color.status.danger.fg"))
  .background(tokenRef("color.status.danger.bg"));

const surface = warning.render(surfaceText, { theme, mode: "dark" });
```

The exact authoring names can still move during implementation, but the responsibility split should not.

Token Builder Variant

The user-proposed token builder shape is useful, but it should be scoped to theme modes rather than become a global registry:

```
const theme = defineTheme()
  .id('bijou.default')
  .mode('dark', mode => mode
    .token()
    .id('color.status.danger.bg')
    .color({ r: 144, g: 0, b: 0 })
    .register())
  .mode('light', mode => mode
    .token()
    .id('color.status.danger.bg')
    .color({ r: 255, g: 230, b: 230 })
    .register())
  .build();
```

The shorter .token(id).color(value) form should exist for dense theme files. The longer .token().id(...).color(...).register() form is useful when an author wants comments, metadata, or validation hooks attached to one token entry.

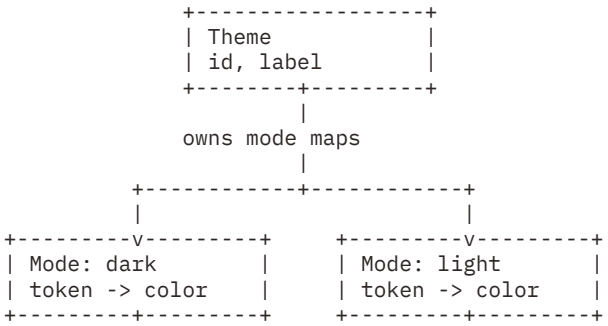
Why Tokens Are Theme-Scoped

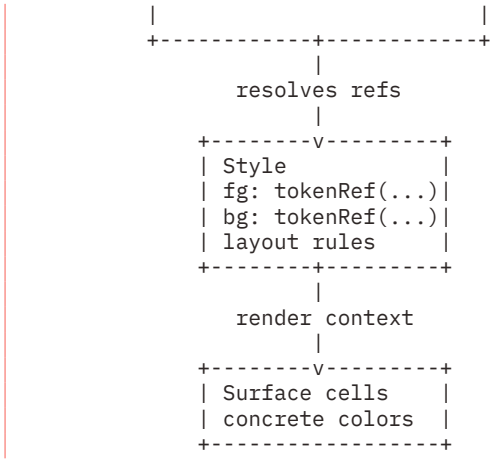
Theme-scoped token registration prevents three failure modes:

1. A token definition cannot silently survive without mode values.
2. A style cannot accidentally become the source of product theme truth.
3. A future theme inspector can show all missing mode values from one theme object instead of chasing global state.

If Bijou later adds package-level token catalogs, those catalogs should declare allowed ids and descriptions. They should still not own the concrete dark and light mode colors for every consuming app.

Relationship Diagram





Style Builder Surface

The style builder should feel familiar to authors who know fluent terminal style APIs, while still returning Bijou surfaces and facts.

Candidate methods:

```
style()
  .foreground(tokenRef('color.text.primary'))
  .background(tokenRef('color.surface.canvas.bg'))
  .bold(true)
  .italic(true)
  .underline(true)
  .padding(1, 2)
  .margin(0, 1)
  .width(40)
  .height(8)
  .maxWidth(80)
  .align('center')
  .border('single')
  .borderForeground(tokenRef('color.chrome.border'))
  .render(surfaceText, { theme, mode: 'dark' });
```

The style builder can support raw colors as an escape hatch:

```
style().foreground({ r: 250, g: 250, b: 250 });
```

Raw colors should be visible in inspectability facts so an agent can distinguish deliberate hard-coded color from semantic token use.

Resolution Contract

Rendering a token-backed style requires:

- a theme
- a mode id
- a lowering profile
- an unresolved-token policy

The default unresolved-token policy should fail in development and use a visible fallback in production-like rendering. The fallback must be deterministic and inspectable.

```
type TokenResolutionFact = {
  tokenId: string;
  themeId: string;
  mode: string;
  source: 'theme' | 'fallback' | 'raw-color';
  requestedColor?: RgbColor;
  resolvedColor: LoweredColor;
  profile: 'truecolor' | 'ansi256' | 'ansi16' | 'mono' | 'plain';
};
```

Lower Modes

Color lowering belongs to the resolver, not to each style call site.

The first resolver should support:

- truecolor: preserve RGB
- ansi256: nearest palette index
- ansi16: nearest named ANSI color
- mono: intensity and emphasis only
- plain: no color, style facts preserved

This keeps the style syntax stable even when output moves from rich terminal to pipe, static snapshot, accessible text, or future frame capture.

Inheritance And Composition

Bijou should support value-style copying and inheritance, but unset rules must stay distinguishable from intentionally raw or token-backed rules.

Candidate API:

```
const base = style()
  .foreground(tokenRef('color.text.primary'))
  .background(tokenRef('color.surface.canvas.bg'));

const danger = style()
  .inherit(base)
  .background(tokenRef('color.status.danger.bg'))
  .foreground(tokenRef('color.status.danger.fg'));
```

Inheritance should copy only unset rules on the receiver, matching the useful part of Lip Gloss while keeping the resolved-token facts intact.

Agent Inspectability / Explainability Posture

Every rendered token-backed style should be able to expose:

- style id, when provided
- theme id
- mode id
- token ids used
- raw colors used

- fallback tokens used
- lowered output profile
- concrete cell color results

This is a Bijou-specific requirement. The goal is not only nice terminal styling, but explainable rendering.

Accessibility / Assistive Reading Posture

Token ids should be semantic enough that an accessible renderer can report meaning even when color is stripped.

For example, color.status.danger.bg gives better assistive context than color.red.900. The style resolver should preserve facts for accessible surfaces even when the final text has no color escape sequences.

Localization / Directionality Posture

No localized strings are introduced by this design. Public docs and DOGFOOD examples introduced during implementation must follow the repository localization policy and provide all supported DOGFOOD locales when new localized copy is added. Token ids are logical semantic identifiers and should not encode physical direction, locale, or translated prose.

Linked Invariants

- Runtime Truth Wins: resolved cell colors and resolver facts are stronger evidence than design prose.
- Graceful Lowering Preserves Meaning: color cannot be the only carrier of status, focus, or warning meaning.
- Tests Are The Spec: the builder API should land through focused runtime tests, not just examples.
- Buffer Holds Facts: rendered cells should preserve token-resolution facts for inspection and replay.
- Docs Are The Demo: DOGFOOD examples should exercise the same public builder API as consumers.

Compatibility

The builder should be additive. Existing theme preset and surface APIs should continue to work while the builder is introduced.

Migration can be staged:

1. Add token references and theme builder types.
2. Convert one DOGFOOD shell theme path to prove the model.

3. Add docs and examples.
4. Expand first-party components to accept Style values.
5. Deprecate ad hoc raw color plumbing only after call sites have a builder alternative.

Acceptance Criteria

- defineTheme() creates immutable theme definitions with required dark and light mode maps.
- Theme builders reject duplicate token ids within one mode.
- Theme builders reject themes missing required dark or light mode values for declared tokens.
- tokenRef(id) is accepted by style color setters.
- style() produces immutable styles and does not mutate copied styles.
- render(surfaceText, { theme, mode: "dark" }) resolves token-backed colors through the selected mode.
- The resolver emits inspectability facts for token, theme, mode, fallback, and lowering decisions.
- Lower modes prove that token-backed styles render in truecolor, ANSI, mono, and plain profiles.
- DOGFOOD includes one theme-builder example and one style-builder example once implementation lands.

Implementation Outline

1. Token and theme type contracts with focused tests.
2. defineTheme() builder with dark/light validation.
3. tokenRef() and color-reference resolution.
4. style() builder for foreground, background, emphasis, padding, width, and render.
5. Lower-mode resolver facts and fallback behavior.
6. DOGFOOD docs, examples, and package exports.

Implementation Progress

Slices 1 through 3 are the first runtime landing target for issue #308:

- defineTheme() creates immutable token-theme definitions with required dark and light mode maps.
- Theme-scoped token registration rejects duplicate token ids within one mode and rejects required modes that omit a declared token.
- tokenRef() and resolveThemeColorRef() preserve resolution facts for selected theme mode, token id, raw colors, and explicit fallback colors.

Slice 4 intentionally tightens the current token vocabulary before adding more syntax:

- docs/design-system/theme-tokens.md now gives every built-in token a per-token “use when / do not use when” contract.
- DOGFOOD now uses explicit dark and light shell themes instead of deriving its docs chrome from one title-screen color ramp.
- Focused DogFood preview tests prove that the shell themes keep readable surface text and common semantic roles above the contrast floor in both dark and light postures.
- The next inspectability UX is #311, the Theme Inspector drawer: an opt-in F 10 shell drawer that shows token swatches, dark/light values, contrast facts, and future renderer provenance when cell-level token facts exist.

Slice 5 promotes contrast from ad hoc tests into a reusable safe-pair contract:

- DL-015 defines foreground/background safe pairs as design-system data.
- defineThemeSafePairs() builds immutable readable, status, and chrome pair matrices.
- doctorTheme() can validate pair paths that target token background slots, such as surface.primary.bg.
- DOGFOOD exports DOGFOOD_THEME_SAFE_PAIRS so docs preview tests and the future Theme Inspector drawer consume the same pair contract.
- This slice closes the implementation path for #314.

The remaining implementation work is the style() builder, lower-mode resolution proof, package example coverage, and the optional Theme Inspector UX slice.

Tests To Write First

- Theme builder rejects missing dark or light modes.
- Theme builder rejects duplicate mode token ids.
- Style render resolves foreground and background token refs by selected mode.
- Copied styles stay immutable after chained modifications.
- Lower-mode rendering produces deterministic truecolor, ANSI, mono, and plain output.
- Inspectability facts identify theme id, mode id, token id, and fallback status.

Roadmap Placement

This belongs in the Beyond lane as issue #308 until implementation is scheduled into a versioned release. It should be grouped as the candidate goalpost Design Tokens And Theme Modes because it crosses design language, runtime rendering, DOGFOOD docs, and public package exports.

Open Questions

- Should Style.render() accept strings only, surfaces only, or both?
- Should style ids be required for inspectability, or optional metadata?
- Should high-contrast mode land in the first implementation or immediately after dark/light?
- Should token catalogs exist as package-level declarations separate from app-owned theme values?

Retrospective

This slice is intentionally documentation and roadmap only. The important decision is the object model: tokens belong to themes, themes own modes, and styles consume resolved tokens. Implementation should start with failing runtime tests that prove that split instead of only matching fluent syntax.

177

DL-014 Theme Inspector And Lab

docs/design/DL-014-theme-inspector-and-lab.md

Decision Summary

Bijou should make theme facts inspectable inside DOGFOOD before the broader mode-aware runtime migration is complete.

This slice adds two bounded surfaces:

- an F10 Theme Inspector drawer in DOGFOOD
- a Themes / Theme Lab docs page with default palettes, shell-gallery facts, token swatches, and safe-pair diagnostics

The first implementation inspects theme tokens only. It does not guess element-level token ownership from final RGB values.

Linked Work

- Parent goalpost: #308
- User story: #311
- Related Theme Lab idea: #315
- Safe-pair contract: #314

Sponsored Perspectives

Human Perspective

A designer-engineer wants to inspect the active theme's token palette from inside a running Bijou TUI so that contrast, token naming, and reuse mistakes can be debugged without leaving the app.

Agent Perspective

An implementation agent wants a deterministic DOGFOOD surface and scripted assertions for theme facts so future theme work can be reviewed without relying only on subjective screenshots.

Current Truth

Bijou has a typed Theme contract with semantic, surface, border, ui, status, and gradient families. DL-013 added a mode-aware token-theme builder, and DL-015 added safe-pair contrast contracts. DOGFOOD had explicit dark/light themes, but those facts were visible only through settings and tests.

The legacy runtime Theme object is still single-mode. This design works inside that limitation by inspecting concrete runtime themes and documenting the mode migration separately in #313.

Problem

Theme work is hard to evaluate when token choices are buried in code:

- colors can look acceptable in one screen but fail in another surface
- token names and token usage are not visible to the person running DOGFOOD
- safe-pair diagnostics are test-only facts
- dark/light design decisions are not presented as a product surface

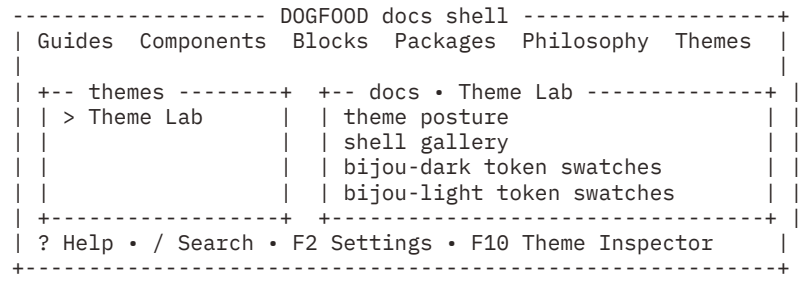
Scope

- Add first-party bijou-dark and bijou-light presets.
- Retokenize DOGFOOD dark/light shell themes to consume those presets.
- Add a Themes page to DOGFOOD.
- Add an F10 Theme Inspector drawer to the DOGFOOD docs route.
- Render token swatches grouped by family.
- Show deterministic safe-pair pass counts.
- Add scripted tests for presets, DogFood theme inheritance, F10 behavior, and the Theme Lab page.

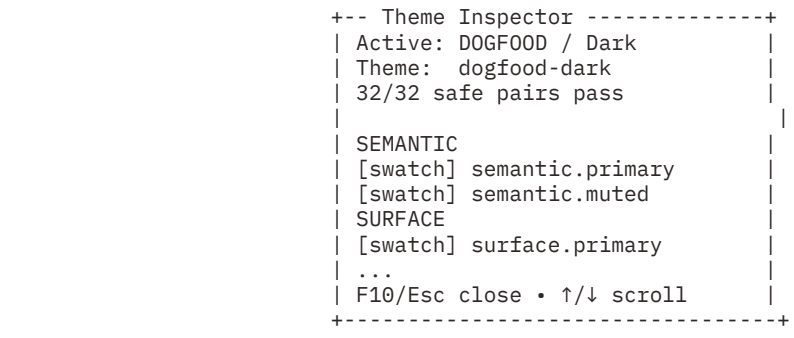
Non-Goals

- Do not implement full runtime theme/mode separation in this slice.
- Do not make F10 a global app-frame feature for every consumer app yet.
- Do not implement element-level token overlays without renderer provenance.
- Do not guess token ownership from RGB equality.
- Do not block lower-mode rendering on rich color swatches.

Product Shape



The quick inspector is a right-side drawer:



Lower Modes

The inspector and lab are still useful in lower modes because every swatch row also prints the token path and token value text. Color improves scanning, but the token name and diagnostics remain visible without color.

Runtime And API Contract

- BIJOU_DARK and BIJOU_LIGHT are exported from @flyingrobots/bijou.
- The presets are registered in PRESETS as bijou-dark and bijou-light.
- DOGFOOD shell themes keep concrete mode theme names:
  - family id: dogfood
  - dark choice id: dogfood:dark, concrete theme name: dogfood-dark
  - light choice id: dogfood:light, concrete theme name: dogfood-light
- DOGFOOD clones the default preset values into those shell themes so mutating runtime RGB caches cannot accidentally alias future app-level overrides.
- F10 toggles the DOGFOOD-only Theme Inspector drawer after the title screen.
- Esc closes the drawer.
- ↑/↓, j/k, d/u, and g/G scroll the drawer without changing the page behind it.
- q still routes to the normal DOGFOOD quit confirmation and closes the drawer so the confirmation is visible.

- Other keys are bounded and do not drive the page behind the drawer.

Accessibility And Localization

The Theme Lab and quick inspector use DOGFOOD catalog entries for all new user-facing strings. New keys are present in every supported runtime locale: English, German, Spanish, and French. Stable token paths such as semantic.primary and surface.overlay.bg remain developer identifiers and are not localized.

Inspectability

The lab and drawer expose:

- active shell theme label
- runtime theme name
- grouped token paths
- token hex and foreground/background values
- gradient stop values
- color-reuse counts
- safe-pair pass counts

Implementation Slices

1. Add bijou-dark and bijou-light presets and export them.
2. Retokenize DOGFOOD dark/light shell themes to consume those presets.
3. Add the DOGFOOD Themes page and token-swatch renderer.
4. Add the F10 Theme Inspector drawer and scripted tests.
5. Keep q quit routing and drawer scrolling deterministic while the drawer is open.

Tests To Write First

- Preset registry includes bijou-dark and bijou-light.
- DOGFOOD dark/light shell themes inherit the default preset token values.
- F10 opens and closes the Theme Inspector drawer after entering docs.
- q opens the normal quit confirmation while the drawer is open.
- Scroll keys move the drawer viewport without closing it.
- The Theme Lab page renders first-party default names, shell-gallery facts, token swatches, and gradient rows.

Validation

Run:

```
npx vitest run packages/bijou/src/core/theme/presets.test.ts scripts/docs-preview.test.ts
↪  --testNamePattern "presets|DogFood dark and light|Theme Inspector|Theme Lab"
npx vitest run scripts/docs-preview.test.ts
npm run typecheck:test
npm run lint
git diff --check
```

Follow-Up Debt

- Promote the inspector into an opt-in app-frame API once its command model is stable.
- Add theme/mode separation for runtime shell settings in #313.
- Add renderer provenance so element-level token overlays can be truthful.
- Move long-form Theme Lab Markdown into the DOGFOOD Markdown localization model when that frontmatter workflow is ready.

178

DL-015 Theme safe pairs and contrast matrices

docs/design/DL-015-theme-safe-pairs-and-contrast-matrices.md

```
title: DL-015 Theme safe pairs and contrast matrices
legend: DL
lane: cool-ideas
priority: medium
github_issue: 314
keywords:
  - design-language
  - themes
  - tokens
  - contrast
  - accessibility
```

Legend: DL - Design Language

Decision Summary

Theme contrast should be declared as a reusable design-system contract, not as one-off test loops.

Safe pairs name which foreground token paths are expected to be readable on which background token paths. The contract feeds doctorTheme(), DOGFOOD theme tests, future Theme Inspector swatches, and lower-mode accessibility reports.

The first runtime shape is intentionally compact:

```
const pairs = defineThemeSafePairs()
  .readable('semantic.primary', 'surface.primary.bg')
  .status('status.error', 'surface.overlay.bg')
  .chrome('ui.cursor', 'surface.secondary.bg')
  .build();
```

Safe-pair background paths may target a token foreground, such as surface.primary, or a token background slot, such as surface.primary.bg.

Sponsored Human

A designer-engineer wants DogFood and first-party themes to prove readable foreground/background combinations so that theme retuning is a design exercise instead of a screenshot guessing game.

Sponsored Agent

An agent auditing a theme wants a stable matrix of safe token pairs so it can identify whether a low-contrast failure is a broken theme value, a missing token, or an unsupported pair that should not have been used by the component.

Hill

Bijou can declare safe token pairs once, run the same contrast contract through doctorTheme(), and reuse the resulting pair matrix in DOGFOOD and future Theme Inspector UX.

Playback Questions

1. Can a safe pair target surface.*.bg instead of only token foregrounds?
2. Does the safe-pair builder reject duplicate foreground/background declarations?
3. Can doctorTheme() validate a safe-pair matrix without bespoke test code?
4. Does DogFood publish its readable/status/chrome pair matrix from the theme module rather than hiding it in tests?
5. Can Theme Inspector later display the same pair facts without inventing a second contrast registry?

Scope

This slice includes:

- defineThemeSafePairs() as a fluent builder for safe-pair declarations
- readable, status, and chrome pair classifications
- support for .bg color-slot paths in doctorTheme() contrast pairs
- DOGFOOD dark/light shell pair contracts
- focused tests proving the contract and DogFood usage

Non-Goals

This slice does not implement the Theme Inspector drawer, retune the global default theme values, or replace the legacy single-mode Theme interface. Those are separate follow-on slices.

Safe-Pair Vocabulary

readable pairs are ordinary text and component foregrounds that must stay legible on a surface.

status pairs are process, health, timeline, or object-state foregrounds that must stay legible when rendered on a surface.

chrome pairs are shell/runtime affordances such as cursor and focus chrome that must remain recognizable on app surfaces.

The kind is not decorative metadata. Future inspectors and reports can group failures by kind so a theme author can distinguish typography problems from status-color problems.

Runtime Contract

defineThemeSafePairs() returns a pure builder. Chained calls accumulate immutable pair declarations. build() returns a frozen array of frozen pair objects compatible with ThemeContrastPair.

Duplicate foreground/background pairs are rejected because duplicate pair rows make reports and inspectors ambiguous.

doctorTheme() resolves contrast pair paths through concrete token color slots:

- semantic.primary resolves to the token foreground color.
- surface.primary resolves to the surface token foreground color.
- surface.primary.bg resolves to the surface token background color.

Missing paths remain errors. Invalid colors remain invalid-color issues from the normal token scan.

DOGFOOD Matrix

DogFood declares its safe-pair matrix beside its shell themes.

Readable foregrounds:

```
semantic.primary
semantic.muted
semantic.accent
semantic.info
semantic.success
semantic.warning
semantic.error
```

Status foregrounds:

```
status.active
status.pending
```

Chrome foregrounds:

```
ui.cursor
```

Background slots:

```
surface.primary.bg
surface.secondary.bg
surface.elevated.bg
surface.overlay.bg
surface.muted.bg
```

That matrix intentionally proves the dense docs shell before broader component retokenization. Additional first-party pairs should be added only when a component needs them as a real contract.

Accessibility / Assistive Reading Posture

Safe-pair checks do not make color sufficient as the only meaning carrier. They prove that color, when present, remains readable. The same components still need text, symbols, structure, or state labels that survive no-color, pipe, and accessible output modes.

Agent Inspectability / Explainability Posture

A future inspector can group safe-pair facts by pair kind:

```
readable semantic.primary on surface.primary.bg: pass 15.20
status status.pending on surface.overlay.bg: pass 8.40
chrome ui.cursor on surface.secondary.bg: pass 7.10
```

If the renderer later preserves per-cell token provenance, an agent can compare the token pair actually used by a cell against the declared safe-pair matrix.

Localization / Directionality Posture

No localized strings are introduced by this design. Token paths and pair kinds are stable developer identifiers and must not encode locale-specific prose or physical direction.

Linked Invariants

- Runtime Truth Wins: safe pairs are only useful when doctorTheme() proves them from concrete theme values.
- Graceful Lowering Preserves Meaning: contrast is necessary, but color is not the only meaning carrier.
- Tests Are The Spec: the pair builder and DogFood matrix land through focused runtime tests.
- Docs Are The Demo: DOGFOOD consumes the same pair contract the docs describe.

Implementation Outline

1. Add defineThemeSafePairs() as a public builder that emits immutable readable, status, and chrome pair declarations.
2. Extend doctorTheme() so contrast pair paths can resolve token foreground colors and token background slots such as surface.primary.bg.
3. Export DOGFOOD's dark/light safe-pair matrix beside its shell themes.
4. Move DOGFOOD contrast tests to consume the exported pair matrix.
5. Update design-system docs, roadmap tests, and release notes so future Theme Inspector work can reuse the same pair contract.

Acceptance Criteria

- defineThemeSafePairs() builds immutable safe-pair declarations.
- Duplicate foreground/background declarations fail deterministically.
- doctorTheme() validates .bg color-slot paths in contrast pairs.
- DOGFOOD exports DOGFOOD_THEME_SAFE_PAIRS from its shell theme module.
- DOGFOOD dark/light tests validate themes through the exported safe-pair contract.
- The roadmap links issue #314 under Design Tokens And Theme Modes.

Tests To Write First

- Core theme doctor test for .bg safe-pair validation.
- Core safe-pair builder duplicate rejection test.
- DOGFOOD preview test that calls doctorTheme() with DOGFOOD_THEME_SAFE_PAIRS.
- Roadmap/spec regression that #314 remains linked to DL-013 planning.

Retrospective

The important shape is the reusable contract. DogFood's earlier contrast proof was correct, but the pair list was trapped inside tests. This slice turns those pairs into design-system data so the next retuning pass can change colors with objective proof.

179

DL-016 Mode-aware runtime theme adapter

docs/design/DL-016-mode-aware-runtime-theme-adapter.md

```
title: DL-016 Mode-aware runtime theme adapter
legend: DL
lane: bad-code
priority: medium
github_issue: 313
keywords:
  - design-language
  - themes
  - modes
  - app-frame
  - dogfood
```

Legend: DL - Design Language

Decision Summary

Bijou should distinguish theme identity from theme mode at the app-frame boundary.

The existing runtime Theme object is a concrete palette. That stays true for compatibility. The new shape is an adapter around that concrete palette:

```
FrameShellThemeFamily
  id: "dogfood"
  modes:
    dark  -> Theme("dogfood-dark")
    light -> Theme("dogfood-light")
```

Single-mode shell themes remain valid:

```
FrameShellTheme
  id: "default"
  theme: Theme("default")
```

The frame resolves both forms into concrete choices before rendering. A choice has a stable id, a family id, an optional mode id, and a resolved concrete theme. This gives settings, inspectors, and DOGFOOD one product identity while keeping the legacy render path stable.

Linked Work

- User story: #313

- Token builder foundation: #308
- Theme Inspector follow-up: #311
- Theme Lab follow-up: #315

Sponsored Perspectives

Human Perspective

A designer-engineer wants DOGFOOD to present one DogFood theme with dark and light modes so that switching color scheme does not look like switching product identity.

Agent Perspective

An implementation agent wants app-frame theme facts to carry both family and mode identity so tests, inspectors, and future provenance tools can prove which semantic theme and concrete mode produced the current cells.

Hill

An app can declare a mode-aware shell theme, switch between its dark and light modes through the existing settings drawer, and observe the selected family and mode without breaking consumers that still pass single concrete themes.

Problem

DL-013 made theme authoring mode-aware: one theme owns dark and light token values. The runtime still consumes one concrete Theme at a time. PR #310 therefore had to model DOGFOOD as two independent shell themes:

```
dogfood-dark
dogfood-light
```

That representation works mechanically, but it teaches the wrong model:

- theme identity and mode identity are conflated
- settings rows mix product themes and color modes in one flat list
- Theme Inspector and Theme Lab cannot report a stable family id
- tests assert dogfood-dark and dogfood-light as peer themes instead of dogfo od plus a selected mode

Scope

This slice adds the first runtime adapter, not a full replacement for Theme.

- Add a mode-aware FrameShellThemeFamily path beside concrete FrameShellTheme.
- Resolve shell theme specs into flattened runtime choices.
- Preserve old single-theme specs unchanged.
- Include selected mode facts in FrameShellThemeChange.
- Convert DOGFOOD to one dogfood shell theme with dark and light modes.
- Update Theme Inspector and Theme Lab facts to use family and mode labels.
- Add tests proving mode switching, compatibility, and context isolation.

Non-Goals

- Do not replace the concrete Theme interface.
- Do not require every app to adopt mode-aware themes.
- Do not add terminal background-color detection here.
- Do not add element-level token provenance.
- Do not implement Theme Inspector mode preview/apply UX from #311 in this slice.

Runtime Contract

FrameShellThemeSpec accepts either a concrete FrameShellTheme with theme or a mode-aware FrameShellThemeFamily with a non-empty modes array.

```
const theme = {
  id: 'dogfood',
  label: 'DOGFOOD',
  modes: [
    { id: 'dark', label: 'Dark', theme: dogfoodDark },
    { id: 'light', label: 'Light', theme: dogfoodLight },
  ],
} satisfies FrameShellThemeSpec;
```

A mode-aware entry resolves into choices:

```
choice id: dogfood:dark
family id: dogfood
mode id: dark
label: DOGFOOD / Dark
theme: dogfood-dark
```

A legacy entry resolves into one choice:

```
choice id: default
family id: default
mode id: undefined
label: Default
theme: default
```

The existing frame model can continue storing activeShellThemeId as the choice id. Consumers that inspect FrameShellThemeChange get the richer facts:

```
{
  shellTheme: {
```

```
    id: 'dogfood:dark',
    label: 'DOGFOOD / Dark',
    theme: dogfoodDark,
  },
  shellThemeSpec: dogfoodFamily,
  shellThemeId: 'dogfood',
  modeId: 'dark',
  modeLabel: 'Dark',
  ctx,
}
```

Settings UX

The first implementation keeps one settings row:

```
Shell theme  DOGFOOD / Dark
```

Activating the row cycles the resolved choices:

```
DOGFOOD / Dark -> DOGFOOD / Light -> Cabinet of Curiosities -> ...
```

This is intentionally conservative. A later #311/#315 slice can split family and mode into separate controls once the app-frame has enough option metadata to render that well.

Compatibility

Compatibility is a hard requirement:

- Existing FrameShellTheme entries with theme still compile.
- Existing apps that read change.shellTheme.theme still receive a concrete selected shell theme.
- Mode-aware apps can inspect change.shellThemeSpec, change.shellThemeId, and optional mode facts for family identity.
- Existing app-frame tests for simple theme cycling keep passing.
- cloneShellThemeContext() still receives one concrete resolved theme.

Tests

The slice is complete when tests prove:

1. Mode-aware shell themes flatten into stable choices.
2. Settings cycling applies a mode choice and reports family/mode facts.
3. Old single-mode shell themes still cycle and apply.
4. DOGFOOD publishes one dogfood family with dark and light modes.
5. Preview/resolution helpers can inspect a mode choice without mutating the ambient default context until the setting is applied.

Acceptance Criteria

- ☐ FrameShellThemeSpec supports concrete FrameShellTheme entries and mode-aware FrameShellThemeFamily entries.
- ☐ Runtime choice ids distinguish family and mode.
- ☐ FrameShellThemeChange includes optional mode facts.
- ☐ DOGFOOD presents one DogFood shell theme family with dark/light modes.
- ☐ Legacy single-mode themes remain supported.
- ☐ Tests cover mode switching and context isolation.

Slice Plan

1. Add the design doc and tracker updates.
2. Add app-frame mode-aware shell theme types and choice resolution tests.
3. Update settings cycling and shell change notifications.
4. Convert DOGFOOD shell themes to one mode-aware family.
5. Update Theme Inspector, Theme Lab, docs tests, and changelog.
6. Run self-review and validation before opening the PR.

180

DL-017 - DOGFOOD Light Theme Readiness

docs/design/DL-017-dogfood-light-theme-readiness.md

```
id: DL-017
title: DOGFOOD Light Theme Readiness
status: active
lane: asap
priority: medium
github_issue: 341
legend: DL
```

Legend: DL - Design Language

Decision Summary

DOGFOOD's dogfood:light shell theme should be safe to demonstrate in a real terminal without relying on the terminal profile's default background or lucky contrast. For v7.2.0, the bounded repair is:

- framed settings, menu, and modal chrome paints explicit light-theme backgrounds
- borders, panel fills, muted copy, selection rows, and action labels remain visually distinct in the Bijou light palette
- the theme doctor covers the chrome-safe pairs that failed the release-video rehearsal by sight
- deterministic tests prove both token-level contrast and rendered shell background ownership

Sponsored Human

A release-video user needs to switch DOGFOOD into light mode and have the app look deliberate on camera. Settings and quit chrome should not show dark terminal patches, washed-out borders, or action labels that are hard to find.

Sponsored Agent

An agent needs a deterministic witness for light-theme chrome. It should be able to render the DOGFOOD shell in dogfood:light, inspect actual Surface

cell metadata, and fail if settings or modal chrome leaves meaningful cells without a theme-owned background.

Hill

After this cycle, dogfood:light is release-demo-ready for the current V7 surface: the theme paints its shell chrome, passes explicit chrome contrast diagnostics, and has enough focused proof to keep terminal-default background leaks from silently returning.

Problem

Issue #341 came from the v7.1.0 release-video rehearsal. The light theme looked unfinished around settings and modal chrome: some border/background regions appeared to inherit the terminal's unstyled background, and several low-contrast token pairs made panels, muted text, and selected rows difficult to read.

The dark DOGFOOD theme mostly hides this class of problem because terminal profiles are commonly dark. The light theme needs explicit proof because a terminal-default background leak can look acceptable on one profile and broken on another.

Scope

- Add a focused rendered witness for DOGFOOD light-theme settings/menu/modal chrome.
- Fix any shell-rendering path that fails to paint theme-owned backgrounds for the witness.
- Retune the Bijou light palette only where the current tokens make borders, panels, muted text, selection rows, or modal actions unclear.
- Expand DOGFOOD theme safe-pair diagnostics so light-theme chrome tokens are checked by doctorTheme().
- Update release-facing docs/changelog so v7.2.0 records the light-theme demo-readiness repair honestly.

Non-Goals

- Do not solve #343's full first-party dark/light variant coverage.
- Do not redesign Theme Lab or Theme Inspector.
- Do not add a global dark/light hotkey in this cycle.
- Do not redesign DOGFOOD's layout, typography, or theme family model.
- Do not turn v7.2.0 into a broad design-system feature train.

User Experience Contract

The light theme may be calmer than the dark theme, but it cannot be ambiguous. The user should be able to distinguish the app background, panels, borders, muted text, selected rows, scrollbars, and modal actions without changing their terminal profile.

Settings, command-palette menu, and modal surfaces must paint their own backgrounds. Empty frame chrome inside those surfaces should not be transparent unless the transparency is a deliberate tokenized overlay with a deterministic fallback.

Lower Modes

Static Mode

Static rendered proof should expose actual Surface cells and token-derived foreground/background choices so tests can inspect theme ownership without parsing screenshots.

Pipe Mode

Pipe output remains text-first and does not need to encode color proof. The color-specific proof belongs in rendered Surface tests and theme diagnostics.

Accessible Mode

No new accessibility contract is introduced. Any action labels or status text made easier to see through color should remain textual, not color-only.

Localization / Directionality Posture

This cycle does not add new localization or bidirectional-layout behavior. The theme repair must preserve the selected-locale text path, existing English-source documentation notices, and current left-to-right DOGFOOD chrome geometry.

The rendered witnesses should avoid making assertions about translated copy beyond stable shell titles such as settings, command palette, and quit chrome. If a future right-to-left shell layout changes modal or drawer geometry, it should still inherit the same token-owned background contract.

Agent Inspectability / Explainability Posture

Agents should be able to inspect the repair without screenshots. The important facts are:

- which concrete DOGFOOD shell theme is active
- which shell surface is open: settings drawer, command-palette menu, or quit modal
- whether visible chrome border cells have explicit foreground and background metadata
- which safe-pair token declarations cover the chrome colors
- whether doctorTheme() reports any low-contrast chrome pairs for DOGFOOD dark or light modes

The proof should stay in Surface metadata and theme-doctor reports so future agents can reason about background ownership deterministically.

Theme Diagnostics

The existing DOGFOOD safe-pair matrix should grow beyond primary text/status pairs to cover the light chrome pairs that matter for this bug:

- border tokens against primary, secondary, elevated, overlay, and muted surfaces
- muted text against the same surfaces used by settings and modal panels
- selection/focus gutter foregrounds and backgrounds
- scroll thumb and scroll track visibility
- modal action/status foregrounds against overlay and elevated surfaces

The diagnostic is intentionally token-level. Rendered tests prove that chrome uses those tokens; the doctor proves that the token pairs remain readable.

Linked Invariants

- DOGFOOD light mode should not depend on terminal-profile background color.
- Theme diagnostics should cover app chrome, not just body copy.
- Light-theme repairs must stay compatible with existing dark theme behavior.
- v7.2.0 remains a narrow stabilization release.

Implementation Plan

1. Add focused failing tests that render DOGFOOD dogfood:light settings drawer, command-palette menu, and quit modal chrome and find unpainted cells in each chrome rectangle.
2. Add a focused failing diagnostic test for missing DOGFOOD chrome safe pairs.
3. Fix the shell/component path that leaves backgrounds unset, or retune tokens if the renderer already paints but the selected color pair is too weak.
4. Expand the DOGFOOD safe-pair matrix to cover chrome-relevant light-theme pairs.

5. Run focused rendered tests, theme doctor tests, the DOGFOOD terminal capture smoke path, npm run typecheck:test, npm run lint, npm run docs:inventory, git diff --check, and the relevant broader suite before review.

Tests To Write First

- DOGFOOD light-theme settings drawer, command-palette menu, and quit modal renders fail when visible chrome cells inside their overlay rectangles have no background metadata.
- doctorTheme(DOGFOOD_LIGHT_THEME, { contrastPairs: DOGFOOD_THEME_SAFE_PAIRS }) fails if chrome pairs are omitted or fall below the selected contrast floor.
- Existing dark-theme DOGFOOD shell tests continue to pass.

Acceptance Criteria

- Deterministic light-theme chrome witnesses exist for settings drawer, command-palette menu, and quit modal rendering.
- Any unpainted background path found by the witness is fixed.
- Bijou light theme border, panel, muted text, selection, and modal action tokens are readable enough for the DOGFOOD release-video path.
- DOGFOOD theme diagnostics cover the relevant chrome safe pairs.
- The DOGFOOD terminal capture smoke path is run as release-video-path evidence.
- docs/CHANGELOG.md records the demo-readiness fix.
- Issue #341 is closed by the implementation PR.

Retrospective

To be completed when the PR lands.

181

DL-018 - First-Party Theme Variant Coverage

docs/design/DL-018-first-party-theme-variant-coverage.md

```
id: DL-018
title: First-Party Theme Variant Coverage
status: active
lane: asap
priority: medium
github_issue: 343
legend: DL
```

Legend: DL - Design Language

Decision Summary

DOGFOOD now has a real AppShell theme-mode toggle, but the theme list still mixes one paired shell family with several landing-art shell presets that do not have light/dark siblings. For v7.2.0, the bounded repair is:

- inventory every first-party DOGFOOD shell theme as paired or single-mode through deterministic shell-spec coverage
- keep DOGFOOD / Dark and DOGFOOD / Light as the paired release-demo path
- classify landing-art shell presets as explicit single-mode choices instead of implying that they support a dark/light toggle
- make unsupported theme-mode toggles visible through shell feedback
- prove dark-to-light and light-to-dark toggles deterministically

This cycle is about honest mode coverage, not a broad Theme Lab or palette redesign.

Sponsored Human

A DOGFOOD user wants to switch between dark and light modes during a demo and understand why some gallery themes do not toggle. Pressing the inherited AppShell shortcut should either switch the active paired theme or clearly say that the selected theme has no alternate mode.

Sponsored Agent

An agent needs machine-readable evidence for first-party theme coverage. It should be able to inspect the DOGFOOD shell theme list, classify each choice, and assert that unsupported mode switches do not silently no-op.

Hill

After this cycle, DOGFOOD's theme path tells the truth: paired themes have dark/light behavior, single-mode themes are classified as single-mode in deterministic shell-spec coverage, and the AppShell mode toggle gives deterministic feedback in both supported and unsupported paths.

Problem

Issue #343 came from the v7.1.0 release-video rehearsal. DOGFOOD exposed theme and mode concepts, then listed several first-party themes without a coherent mode story. That made Ctrl+T look unreliable: it works for the mode-aware DOGFOOD family, but single concrete themes cannot toggle and the user receives no explanation.

The current implementation already has an AppShell-level Ctrl+T command for mode-aware shell theme families. The gap is coverage and honesty around the full DOGFOOD theme inventory.

Scope

- Add first-party shell theme coverage that distinguishes paired families from single-mode concrete themes.
- Do not touch the DOGFOOD app rendering file for visible Theme Lab inventory until its existing raw-string debt is cleaned under the touched-file localization policy.
- Keep the DOGFOOD family as the paired first-party dark/light shell path.
- Keep landing-art shell presets single-mode unless a real light/dark pair is intentionally designed for a given preset.
- Make Ctrl+T on a single-mode theme publish an explicit shell notification instead of silently doing nothing.
- Add deterministic AppShell tests for supported dark-to-light and light-to-dark switching and for unsupported single-mode feedback.
- Add DOGFOOD tests that prove the shell-spec inventory lists paired and single-mode coverage.

Non-Goals

- Do not redesign Theme Lab or Theme Inspector.
- Do not create speculative light variants for landing-art palettes.
- Do not change the release-title/landing-art rendering palette.
- Do not add a global theme registry outside the existing AppShell shellThemes contract.
- Do not move Theme Lab provenance, pointer inspection, or full operator surfaces out of the v9.0.0 horizon.

User Experience Contract

Ctrl+T is inherited by apps that expose mode-aware shell themes. When the active shell choice belongs to a family with multiple modes, Ctrl+T switches to a sibling mode and posts the existing “Shell theme set to …” feedback.

When the active shell choice is a concrete single-mode theme, Ctrl+T posts a short settings notification that names the active theme and says it has no alternate mode. The active theme remains unchanged.

The shell spec and settings surfaces must prove:

- the active default dark and light presets
- a shell inventory where each first-party choice is classified as paired or single-mode
- enough concrete labels that users can tell which entries respond to Ctrl+T

Lower Modes

Static Mode

Static tests should assert the same inventory classification from shell specs without requiring screenshot or ANSI scraping.

Pipe Mode

No new pipe contract is introduced. The shell theme inventory is diagnostic copy, not an exported data format.

Accessible Mode

Mode support must be textual. The user should not need color, swatches, or visual grouping alone to understand whether a theme supports the mode toggle.

Localization / Directionality Posture

New user-facing strings should use the existing DOGFOOD or AppShell i18n ports. The tests may assert English source copy, but the source strings must be catalog-discoverable so the localization gates can track them.

Directionality is unchanged. Future visible inventory copy should flow through the current shell layout after the touched DOGFOOD app file is localization clean.

Agent Inspectability / Explainability Posture

Agents should be able to inspect:

- raw DOGFOOD shell theme specs and their mode support
- resolved AppShell theme choices and their modeId facts
- shell-spec classification for paired and single-mode choices
- runtime notifications after supported and unsupported Ctrl+T paths

The proof should not require screenshots or terminal color sampling.

Linked Invariants

- Visible controls are a promise: a mode toggle must either work or explain why it cannot.
- Docs are the demo: once the DOGFOOD app rendering file is localization-clean, Theme Lab should explain the actual first-party theme inventory users can select.
- Shell owns shell concerns: mode switching remains AppShell behavior inherited by DOGFOOD and other apps through createFramedApp.
- Tests are the spec: paired and unsupported paths need deterministic coverage.

Implementation Plan

1. Add first-party shell theme support coverage for tests.
2. Keep visible Theme Lab inventory out of this cycle until the touched DOGFOOD app file is localization-clean.
3. Add AppShell feedback for unsupported mode toggles on single-mode concrete themes.
4. Add tests that fail before the implementation:
    - DOGFOOD shell specs list DOGFOOD as paired dark/light.
    - DOGFOOD shell specs list landing-art shell presets as single-mode.
    - AppShell toggles paired themes dark -> light -> dark.

    - AppShell Ctrl+T on a single-mode theme leaves the id unchanged and posts explicit feedback.
5. Update docs/CHANGELOG.md, run focused tests, npm run typecheck:test, npm run lint, npm run docs:inventory, and git diff --check.

Tests To Write First

- tests/cycles/DL-018/first-party-theme-variant-coverage.test.ts for DOGFOOD shell inventory.
- packages/bijou-tui/src/app-frame-notifications.test.ts coverage for supported dark-to-light and light-to-dark Ctrl+T switching.
- packages/bijou-tui/src/app-frame-notifications.test.ts coverage for unsupported single-mode Ctrl+T feedback.
- packages/bijou-tui/src/app-frame-modal-routing.test.ts coverage for the command-palette theme-mode action.

Acceptance Criteria

- Every DOGFOOD first-party shell theme is classified as paired or single-mode.
- DOGFOOD shell-spec coverage proves the first-party theme inventory with mode-support labels.
- Ctrl+T toggles the paired DOGFOOD family in both directions.
- Ctrl+T on a single-mode theme posts explicit feedback instead of silently doing nothing.
- Deterministic coverage proves at least one dark-to-light and one light-to-dark switch path.
- docs/CHANGELOG.md records the theme variant coverage repair.
- Issue #343 is closed by the implementation PR.

Retrospective

To be completed when the PR lands.

182

DL-019 - Theme Rule Selection And Inspection

docs/design/DL-019-theme-rule-selection-and-inspection.md

```
id: DL-019
title: Theme Rule Selection And Inspection
status: active
lane: cool-ideas
priority: medium
github_issue: 444
legend: DL
```

Legend: DL - Design Language

Decision Summary

Bijou should adopt the strongest ideas from reactive design-token systems without importing an incompatible dependency or replacing the existing runtime theme contract.

The runtime truth remains:

```
Theme
  -> ResolvedTheme
    -> TokenGraph
      -> ThemeAccessors
        -> terminal cells, lower modes, and DTCG interop
```

This cycle adds a Bijou-native rule layer for theme authoring and inspection:

- selector rules such as best contrast, minimum contrast, most vivid, least vivid, closest color, and ordered choice
- explicit candidate exclusions for reserved roles such as success and error
- structured inspection facts explaining why a rule chose a token
- dependency facts that Theme Inspector and tests can consume later
- compiled output that remains a normal Bijou Theme or token graph entry

No design-book runtime, dev, peer, optional, or generated dependency is added. The external package is useful prior art, but its license and general CSS surface do not belong in Bijou's Apache-2.0 zero-dependency core.

Sponsored Human

A Bijou theme author wants to tune primitive palettes while keeping semantic roles stable. They should be able to say “choose readable text for this surface” or “choose a vivid accent that is not success or error” instead of manually maintaining every derived hex value.

Sponsored Agent

An agent auditing theme behavior wants deterministic facts for rule inputs, candidate rejection, selected values, dependencies, and contrast proof so it can explain a theme without sampling screenshots or guessing from final RGB values.

Hill

A Bijou theme author can define semantic theme roles through pure selector rules, replay the product questions below with tests, compile the rules into the existing theme/token graph contract, and inspect which candidates were considered, rejected, and selected.

Playback Questions

- Can a semantic foreground choose a readable color from a primitive palette without hard-coding the final hex value?
- Can a vivid accent selector exclude reserved success and error roles while still proving contrast against its surface?
- Can TokenGraph.inspect() explain every candidate read by a rule, including selected, eligible, excluded, invalid, and contrast-failed candidates?
- Can malformed rule inputs fail with the missing target, scope, candidate, or exclusion path instead of silently choosing a surprising color?
- Can the rule layer preserve existing TokenGraph.get() behavior for callers that only need a resolved TokenValue?

Current Truth

Bijou already has:

- a public Theme contract with semantic, surface, border, ui, status, and gra dient token families
- a TokenGraph with references, light/dark definitions, transforms, cache invalidation, and subscribers
- defineTheme() for mode-aware token themes
- fromDTCG() and toDTCG() for token document interop

- doctorTheme() and defineThemeSafePairs() for concrete contrast checks
- DOGFOOD Theme Lab and Theme Inspector surfaces for concrete token facts

The missing piece is rule provenance. Today a derived token can be encoded as a manual hex, a direct reference, or a transform. It cannot yet say:

- choose the highest-contrast color from this palette
- choose the first color that reaches a contrast floor
- choose the most vivid color that is still readable
- exclude reserved role tokens from an accent search
- explain which candidates failed and why

Problem

Manual semantic token maintenance creates drift:

- a primitive palette can change while semantic foregrounds stay stale
- a vivid accent may accidentally reuse a destructive or success role
- contrast proof happens after values are chosen instead of during selection
- Theme Inspector can show final values but not why those values won
- tests can assert concrete hex values but not the decision rule behind them

The answer is not to replace Bijou's runtime theme model. The runtime model already encodes terminal-specific facts: NO_COLOR, foreground/background tokens, text modifiers, RGB caches, ANSI downsampling, lower-mode behavior, and safe-pair checks.

Scope

This cycle adds pure core support for rule-based color selection and inspection.

Included:

- color candidate scopes represented as token paths or explicit color values
- selector functions for contrast, vividness, closeness, ordered position, and minimum contrast
- explicit not exclusions by token path
- selector result facts with candidate pass/fail reasons
- dependency extraction for references and candidate scopes
- TokenGraph support for rule definitions if that can land without breaking existing callers
- focused tests for selection, exclusion, dependency, and cycle behavior
- design-system docs and changelog notes

Out of scope:

- no external design-book package integration
- no CSS renderer

- no typography, spacing, motion, or duration token categories
- no Figma adapter
- no Theme Lab redesign in this slice
- no element-level renderer provenance
- no app-frame theme-mode UX changes

API Direction

The first public API should be small and structural. Exact names can move in implementation, but the shape should stay close to this:

```
const graph = createTokenGraph({
  palette: {
    ink: "#0b1020",
    paper: "#f8fafc",
    cyan: "#38bdf8",
    green: "#22c55e",
    red: "#ef4444",
  },
  surface: {
    primary: { fg: "#f8fafc", bg: { ref: "palette.ink" } },
  },
  semantic: {
    primary: bestContrastWith(
      { ref: "surface.primary.bg" }, // token background slot
      scope("palette"),
    ),
    accent: mostVivid(scope("palette"), {
      against: { ref: "surface.primary.bg" },
      minContrast: 4.5,
      not: ["palette.green", "palette.red"],
    }),
  },
});
```

If the fluent helper API is too large for the first slice, the serialized graph definition may use plain discriminated data instead:

```
semantic: {
  primary: {
    rule: "best-contrast-with",
    target: { ref: "surface.primary.bg" },
    candidates: { kind: "scope", path: "palette" },
  },
}
```

Slot-path references ending in .bg are part of this cycle's contract. A reference such as { ref: "surface.primary.bg" } resolves the background color of the surface.primary token while { ref: "surface.primary" } keeps the existing foreground-reference behavior.

The graph should expose inspection facts separately from resolved values:

```
const fact = graph.inspect("semantic.accent", "dark");
```

The returned fact should be structured enough for tests and future Theme Inspector rows:

```
semantic.accent
  rule: most-vivid
```

```
selected: palette.cyan (#38bdf8)
dependencies: surface.primary.bg, palette.ink, palette.paper, palette.cyan,
  palette.green, palette.red
rejected:
  palette.paper - eligible, not selected
  palette.green - excluded
  palette.red - excluded
  palette.ink - contrast below 4.5
```

Selector Vocabulary

The initial selector vocabulary should cover the high-value color decisions. Selectors that compare against a target put the target first. Selectors that rank only a candidate collection put the collection first.

bestContrastWith(target, candidates) chooses the candidate with the highest contrast ratio against the target. Equal scores keep candidate order.

minContrastWith(target, candidates, { ratio }) chooses the first candidate that clears the requested contrast ratio, using deterministic candidate order.

mostVivid(candidates, options) chooses the highest-chroma candidate, with optional against, minContrast, and not constraints. First-slice chroma is the deterministic RGB channel spread: max(red, green, blue) - min(red, green, blue). If minContrast is present, against is required.

leastVivid(candidates, options) chooses the lowest-chroma candidate, with the same optional constraints and the same minContrast rule.

closestColor(target, candidates) chooses the closest candidate to a target color. The first slice uses squared RGB distance and keeps candidate order for equal distances.

nthColor(candidates, index) chooses a candidate by ordered position. Integer indices use array-style ordering, negative integers count from the end, and out-of-range integers clamp to the nearest valid candidate. Fractional indices clamp to 0 through 1 and use round((length - 1) * index).

Runtime Contract

Selectors are pure. They do not read process state, filesystem data, terminal facts, or clocks.

Selector helpers return plain ThemeColorRuleDefinition data. The token graph evaluates that intermediate rule definition, and the existing TokenGraph.get() path still returns a resolved TokenValue.

Rule failures are deterministic:

- missing target, explicit candidate, candidate-scope, and exclusion paths throw with the missing path
- vividness selectors throw when minContrast is set without against
- no minimum-contrast candidate throws unless the selector defines an explicit fallback

- cycles remain cycle errors

Inspection dependencies include target and against references plus every candidate path read from explicit candidate lists or scopes, whether the candidate was selected, eligible-but-not-selected, excluded, invalid, or below the contrast floor.

Accessibility And Assistive Posture

Selector rules improve color quality, but they do not make color sufficient as the only meaning carrier. The same graceful-lowering rule applies:

- no-color output must keep text, symbols, or structure
- pipe output must not depend on swatches
- accessible output must name state and intent

Contrast-aware selection is proof that a chosen color is readable when color is available. It is not proof that the UI is accessible by itself.

Localization And Directionality Posture

No user-facing localized copy is required for the core rule primitives. Rule ids, token paths, candidate paths, and inspection facts are developer-facing identifiers.

Future DOGFOOD Theme Inspector prose that explains rule facts must flow through the DOGFOOD localization catalog. Token paths remain stable identifiers and are not localized.

Agent Inspectability / Explainability Posture

Inspection facts are a first-class product requirement. An agent should be able to assert:

- which selector rule resolved a token
- which references and candidate scopes the rule read
- which candidate won
- which candidates were rejected
- whether a candidate failed contrast, was excluded, or was the wrong type
- which downstream token paths depend on the resolved token

These facts should not require terminal rendering, screenshot analysis, or RGB equality guessing.

Linked Invariants

- Runtime Truth Wins: resolved values and inspection facts come from executing the graph.
- Tests Are the Spec: selector behavior lands through deterministic tests before docs claims.
- Graceful Lowering Preserves Meaning: color quality does not replace lower-mode structure.
- Docs Are the Demo: future Theme Lab surfaces should consume the same inspection facts tests assert.

Implementation Outline

1. Add selector definition types to the core theme graph.
2. Collect deterministic candidates from explicit paths and scopes.
3. Implement contrast, vividness, closeness, and ordered selection helpers.
4. Expose TokenGraph.inspect() or an adjacent pure inspection API.
5. Preserve existing TokenGraph.get() behavior for raw, ref, mode, and transform definitions.
6. Wire the new types and helpers into the theme barrel and root package.
7. Document the recipe pattern in the design-system theme authoring docs.
8. Record the shipped behavior in docs/CHANGELOG.md.

Tests To Write First

- bestContrastWith() chooses the readable foreground from a candidate scope.
- minContrastWith() rejects candidates below the minimum ratio and selects the first passing candidate.
- mostVivid() respects against, minContrast, and not exclusions.
- leastVivid() chooses the muted readable candidate.
- closestColor() and nth() are deterministic.
- selector definitions participate in graph dependency inspection.
- circular selector references still fail deterministically.
- existing graph reference and transform tests keep passing.

Acceptance Criteria

- Bijou has no dependency on design-book.
- Theme selectors are pure core functions.
- Token graph rule definitions resolve to existing TokenValue objects.
- Inspection facts explain selected and rejected candidates.
- Exclusions by token path prevent reserved roles from winning accent rules.
- The new API exports from @flyingrobots/bijou.

- Design-system docs show the primitive -> semantic rule -> UI token workflow.
- Local validation includes focused theme tests, npm run typecheck:test, npm run lint, npm run code-dojo:changed, and git diff --check.
- Code Dojo debt remains verifiable through npm run code-dojo:debt and npm run code-dojo:verify; this cycle does not claim the repo goalpost unless that aggregate report reaches 112 or lower.

Retrospective

To be completed when the cycle lands.

183

DL-020 - Dogfood Rule-Authored First-Party Themes

docs/design/DL-020-dogfood-rule-authored-first-party-themes.md

```
id: DL-020
title: Dogfood Rule-Authored First-Party Themes
status: active
lane: cool-ideas
priority: medium
github_issue: 446
legend: DL
```

Legend: DL - Design Language

Decision Summary

DL-019 added Bijou-native theme selector rules and inspection facts. This cycle uses that system inside Bijou's own first-party defaults instead of keeping the new rule layer as a demo-only API.

The public runtime contract remains unchanged:

```
rule-authored preset definitions
  -> TokenGraph
    -> concrete Theme
      -> ResolvedTheme
        -> terminal rendering, DTCG interop, and inspectors
```

No external design-token dependency is introduced. The work is a narrow dogfood pass over Bijou-owned themes only.

Sponsored Human

A Bijou theme maintainer wants to adjust primitive palette choices and trust the system to derive readable semantic roles, accents, borders, and UI affordances without manually copying final hex values across every token family.

Sponsored Agent

An agent auditing first-party themes wants deterministic provenance facts for why default theme tokens were selected, so it can review rule behavior instead of reverse-engineering intent from literal hex equality.

Hill

bijou-dark and bijou-light are authored from primitive palette definitions plus selector rules, materialize to the existing Theme shape, and expose rule inspection facts that explain their first-party semantic choices.

Playback Questions

- Can Bijou-owned defaults use selector rules while preserving the public Theme object shape and registry keys?
- Can tests prove first-party semantic choices through TokenGraph.inspect() rather than only final hex assertions?
- Can DTCG round trips, resolver behavior, RGB caches, and gradient consumers keep working with compiled rule-authored presets?
- Can third-party and legacy themes stay literal compatibility fixtures in this slice?

Scope

Included:

- a small first-party preset authoring layer that compiles token graph definitions into Theme
- rule-authored bijou-dark and bijou-light
- tests for compiled values, registry identity, and rule inspection provenance
- docs/changelog updates that make dogfooding explicit

Out of scope:

- external design-book integration
- public breaking changes to Theme
- rewriting cyan-magenta, teal-orange-pink, nord, or catppuccin
- Theme Lab UI redesign

Runtime Contract

Preset compilation must be pure and deterministic. It cannot read process state, terminal facts, filesystem data, clocks, or network data.

The compiled object remains a normal Theme. Existing consumers can continue to read theme.semantic.primary.hex, theme.status.success, theme.gradient, and surface tokens without knowing how first-party themes were authored.

The preset authoring layer records the original rule definitions against the compiled Theme identity. createResolved() uses those definitions when present, so ResolvedTheme.tokenGraph remains rule-authored for first-party presets. Cloned themes and DTCG imports fall back to concrete token graphs.

Tests may assert rule ids, selected candidate paths, dependencies, and selected hex values from the same resolved graph inspectors consume.

Accessibility And Assistive Posture

This slice improves color choice but does not make color the only carrier of meaning. Existing labels, text structure, NO_COLOR behavior, and lower-mode rendering remain the accessibility contract.

Localization And Directionality Posture

This slice does not add DOGFOOD user-facing copy. Rule ids, token paths, design document prose, and changelog entries are developer-facing documentation and stable identifiers. Future DOGFOOD UI that renders these facts must use the localization catalog.

Agent Inspectability / Explainability Posture

The primary agent-facing value is provenance. An audit should be able to inspect semantic.accent and decision.primaryText, see the selector rule, enumerate candidates, and explain exclusions without comparing screenshots or final hex equality.

Implementation Outline

1. Add a small preset compiler that materializes graph definitions into Theme.
2. Move bijou-dark and bijou-light into rule-authored preset modules.
3. Preserve first-party rule definitions through createResolved().
4. Keep literal legacy and third-party presets unchanged.
5. Update docs and changelog after the tests prove the behavior.

Tests To Write First

- a red provenance test showing literal defaults expose token inspection instead of rule
- a dark-theme accent test proving most-vivid selects brand.accent
- a light-theme primary text test proving min-contrast-with selects ink.primary
- existing preset and DTCG tests proving resolved values stay compatible

Retrospective / Closeout Notes

Close this cycle when CI is green, CodeRabbit is green, review threads are resolved, and the PR summary records the rule probe values.

Linked Invariants

- Runtime Truth Wins: compiled presets and inspection facts come from executing the graph.
- Tests Are the Spec: dogfood behavior lands through deterministic tests.
- Docs Are the Demo: the docs record that Bijou's defaults use the system they document.

184

DL-021 - Dogfood Theme Token Usage Proof

docs/design/DL-021-dogfood-theme-token-usage-proof.md

```
id: DL-021
title: Dogfood Theme Token Usage Proof
status: active
lane: asap
priority: high
github_issue: 448
legend: DL
```

Legend: DL - Design Language

Decision Summary

DL-019 and DL-020 made theme rule provenance real, but DOGFOOD still has to make that truth visible. The Theme Inspector must not look like an unthemed debug drawer that merely lists colors. It should prove which active theme tokens are powering the docs shell and keep its own chrome readable in light and dark shells.

The runtime truth remains the existing DOGFOOD docs shell:

```
active docs shell theme
  -> docs visual theme
    -> docs chrome tokens
      -> terminal cells and Theme Inspector usage proof
```

No external design-token integration is added. This is a narrow DOGFOOD usage pass over the inspector and its immediate theme proof surface.

Sponsored Human

A DOGFOOD reader comparing dark and light themes wants to see the active theme affect the app they are looking at, not just a detached list of hex values.

Sponsored Agent

An agent auditing theme behavior wants deterministic rendered evidence that the inspector chrome, summary copy, usage rows, palette labels, and scroll chrome all come from active DOGFOOD theme tokens.

Hill

The Theme Inspector renders readable themed chrome in DOGFOOD dark and light shells, shows a live usage proof for DOGFOOD docs surfaces, and keeps the full palette as reference data rather than implying every reference color is already visible in the surrounding app.

Playback Questions

- Does the inspector border, background, title, summary, and close hint use active docs theme tokens instead of fallback styling?
- Does light mode preserve readable foreground/background contrast inside the inspector summary and usage proof?
- Can the inspector show both the active shell palette and the specific DOGFOOD docs tokens currently used by the app?
- Can tests fail if the inspector regresses to unstyled summary copy or loses the usage proof?

Scope

Included:

- theme the Theme Inspector drawer chrome through the active docs visual theme
- add a compact DOGFOOD usage proof section before the reference palette
- style inspector summary and palette chrome with explicit theme tokens
- add focused DOGFOOD preview tests for light-mode readability and usage proof
- update changelog and localization catalogs for new visible copy

Out of scope:

- replacing the theme engine
- adding a design-book dependency
- restyling every DOGFOOD docs pane in this slice
- changing third-party theme presets
- adding element-level provenance for every rendered cell

Runtime Contract

The usage proof is derived from the same active shell theme id that drives the docs page. It must not maintain an independent color table. If the active shell theme changes, the inspector summary, usage proof, reference palette labels, drawer border, surface, and scrollbar update from that same model state.

Localization And Directionality Posture

All new visible DOGFOOD copy is catalog-backed and participates in the existing DOGFOOD i18n completeness, check, and debt gates.

Tests To Write First

- a light-theme inspector regression that proves the active summary and close hint are styled with readable foreground/background cells
- a usage-proof regression that proves the inspector renders live DOGFOOD docs token rows instead of only the reference palette

Closeout Notes

Close this cycle when local DOGFOOD preview tests, DOGFOOD i18n gates, lint, and relevant Code Dojo checks are green, and the PR summary names the before/after behavior from the screenshots.

185

DL-022 - Theme Lab Editor Graph UX

docs/design/DL-022-theme-lab-editor-graph-ux.md

```
id: DL-022
title: Theme Lab Editor Graph UX
status: active
lane: asap
priority: high
github_issue: 450
legend: DL
```

Legend: DL - Design Language

Decision Summary

DL-019 and DL-020 made theme rule provenance inspectable, and DL-021 made the DOGFOOD Theme Inspector prove that its chrome consumes active theme tokens. The Theme Lab still reads like a static reference page. This cycle turns it into a bounded editor surface with visible controls and a live token graph that updates from the same in-memory draft theme when a color changes.

The runtime truth for this slice is:

```
active DOGFOOD shell theme
  -> editable DOGFOOD-local draft
    -> selected editable color slot
      -> derived theme token rows
        -> live token relationship graph
```

No external design-token package or design-book dependency is added. The useful idea is the relationship graph and immediate feedback loop, implemented through Bijou's existing theme and terminal surface contracts.

Sponsored Human

A DOGFOOD user opening Themes wants something that behaves like an editor, not a read-only list. They need a focused color row, obvious keyboard controls, a current hex value, and a graph that changes when the color changes.

Sponsored Agent

An agent auditing theme behavior needs deterministic model functions and scripted preview tests that prove a color edit updates the editor state, rendered token values, and graph rows in the same frame.

Hill

Theme Lab exposes a first editing pass for active DOGFOOD theme colors. A user can move between editable colors, nudge RGB channels, reset the draft, and see a live graph of token relationships and changed colors without leaving the docs shell.

Playback Questions

- Does the Theme Lab show editing affordances rather than only static swatches?
- Can the selected editable row move deterministically with keyboard input?
- Does changing a channel update the selected hex value and the rendered token graph in the same frame?
- Does reset restore the draft graph to the active theme values?
- Are all new visible labels catalog-backed?

Scope

Included:

- a DOGFOOD-local theme draft model that clones the active shell theme
- editable rows for a bounded set of high-signal tokens
- deterministic key handling for selection, channel choice, nudging, and reset
- a live graph renderer that shows token nodes, dependency edges, and edited nodes from the current draft
- deterministic invalid-input handling: RGB channel nudges clamp to 0..255, unsupported keys are ignored, and this first pass exposes no free-form hex parser that could accept malformed color text
- focused preview tests for editing state and graph updates
- changelog and DOGFOOD i18n catalog updates

Out of scope:

- saving custom themes to disk
- mutating exported preset constants
- importing or integrating design-book
- mouse or pointer editing

- full authoring for every third-party preset
- replacing TokenGraph, Theme, or the existing DOGFOOD shell theme model

Runtime Contract

The editor state is owned by the DOGFOOD docs model. It is not a global theme registry and it does not mutate BIJOU_DARK, BIJOU_LIGHT, or shell theme constants.

The renderer must consume one draft object for all Theme Lab editor output:

- editor row swatches
- selected color hex
- channel values
- graph nodes
- graph edges
- palette preview rows

If a color changes, the graph changes because it is projected from the draft theme, not because the graph keeps its own color cache.

This slice does not feed the draft into ContextPort.tokenGraph or replace the runtime TokenGraph exposed by ContextPort. The docs shell still observes the active runtime theme through the normal context path, while Theme Lab builds a local presentation graph from the unsaved draft. That keeps third-party obs erveTheme() guidance intact: unsaved editor previews are local to Theme Lab, and any future persisted theme must re-enter the canonical context and token-graph path.

Editable Slots

The first pass edits a deliberately small set of active DOGFOOD runtime tokens:

- semantic.primary
- semantic.accent
- surface.primary.bg
- surface.secondary.bg
- border.primary
- ui.cursor
- status.success
- status.error

Those slots are enough to prove text, surface, border, cursor, and status feedback without pretending this is a full theme persistence product.

Graph Shape

The graph is textual and terminal-native. It should show:

- token path
- current hex or foreground/background pair
- edited marker when the draft differs from the active theme
- dependency-style edges for major families, such as:

```
semantic.accent #7aa7e8 *
  -> border.secondary
  -> ui.cursor
  -> ui.focusGutter
surface.primary.bg #171827
  -> surface.primary
  -> ui.focusGutter.bg
```

A -> B means the preview treats B as a consumer or dependent of editable source A. It does not mean runtime derivation has been committed to ContextPort.tokenGraph.

The graph is not renderer provenance. It explains theme-token relationships and draft changes only.

Lower Modes

Swatches are optional visual assistance. The selected row, current hex, RGB channels, edited marker, graph node names, and graph edge labels remain visible as text.

Localization Posture

All new Theme Lab labels, control hints, and graph section headings are DOGFOOD catalog keys in every supported runtime locale. Token paths and hex values are developer identifiers and are not localized. English dogfoodText fallbacks are permitted as developer scaffolding and runtime resilience only; generated catalog entries for the new keys must still exist before merge.

Tests To Write First

- a pure editor-model test proving selection, channel nudge, and reset behavior
- a graph projection test proving an edited token is marked and carries the new hex value
- a same-frame coherence test proving a channel nudge updates the selected hex and the rendered graph node from the same draft state
- a DOGFOOD preview test proving Theme Lab renders editor controls and the live graph after scripted key input

Closeout Notes

Close this cycle when the focused Theme Lab tests, DOGFOOD i18n gates, Code Dojo changed-file gates, lint, and relevant preview checks are green. The PR summary should explicitly name the before/after behavior: static reference page to editable color lab with live graph feedback.

Localization design records

These records treat language and direction as runtime concerns. They cover localization ports, catalogs, formatting, locale selection, geometry and the application surfaces used to prove the contracts. Distinguish translated content from fallback output, and read debt counts with their inventory definition so an omitted document is not mistaken for a completed translation.

186

LX-001 — bijou-i18n Runtime Package

docs/design/LX-001-bijou-i18n-runtime-package.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

Bijou now has doctrine for localization and bidirectionality, but no runtime package boundary that applications or shell surfaces can actually consume.

If we want bijou-i18n to be a real localization tool, the first step is not spreadsheets.

The first step is a clean, in-memory runtime package.

This cycle intentionally narrows scope to:

- @flyingrobots/bijou-i18n
- runtime catalogs
- locale and direction
- runtime-safe lookup and references
- formatting seams

It does **not** try to ship spreadsheet workflows yet.

Human users

Primary human user

A builder who wants to localize Bijou shell or app content without inventing their own ad hoc runtime i18n substrate.

They are asking:

- where does locale live?
- where does direction live?
- how do I register catalogs?
- how do I look up a message or resource?
- how do I do this without filesystem or service assumptions?

Human hill

A builder can install and use a real bijou-i18n runtime package entirely in memory, with a clean catalog model and no spreadsheet baggage.

Agent users

Primary agent user

An agent that needs to inspect or generate localized UI against explicit runtime contracts instead of hidden string tables.

They are asking:

- what is the active locale?
- what is the active direction?
- what keys and namespaces exist?
- how do references resolve?
- what happens when a translation is missing?

Agent hill

An agent can reason about localized runtime content through explicit catalog, locale, direction, and lookup APIs instead of implicit English defaults.

Human playback

1. A builder installs @flyingrobots/bijou-i18n.
2. They create an in-memory catalog with namespaced keys.
3. They load the catalog into a runtime.
4. They resolve messages and resources by key.
5. They switch locale and observe fallback behavior.
6. They read direction from the same runtime without inventing a second seam.

Agent playback

1. An agent inspects the runtime package API.
2. It can enumerate or at least reason about namespaces, ids, locale, and direction.
3. It resolves a message key with interpolation values.
4. It resolves a resource/data key.
5. It observes missing-translation fallback and explicit missing-key behavior.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- runtime package must work entirely in memory
- runtime package must not assume spreadsheets, files, services, or persistence
- runtime package must expose direction as first-class metadata
- runtime package must be string-first but not string-only
- runtime package must support runtime-safe references without becoming a spreadsheet interpreter

Deliverable

Introduce a new package:

- packages/bijou-i18n

With a minimal but real runtime surface:

- catalog types
- key types
- locale direction types
- in-memory runtime / adapter
- message lookup
- generic resource lookup
- fallback behavior
- formatter hooks or default formatters

Proposed API direction

The exact names can still evolve, but the first slice should be close to:

```
type LocaleDirection = 'ltr' | 'rtl' | 'auto';

interface CatalogKey {
  readonly namespace: string;
  readonly id: string;
}

type CatalogEntryKind = 'message' | 'resource' | 'data';

interface I18nCatalogEntry<T = unknown> {
  readonly key: CatalogKey;
  readonly kind: CatalogEntryKind;
```

```
  readonly sourceLocale: string;
  readonly values: Readonly<Record<string, T>>;
  readonly fallbackValue?: T;
}

interface I18nRuntime {
  readonly locale: string;
  readonly direction: LocaleDirection;
  loadCatalog(catalog: I18nCatalog): void;
  unloadCatalog(namespace: string): void;
  t(key: CatalogKey, values?: Record<string, unknown>): string;
  resource<T = unknown>(key: CatalogKey): T | undefined;
}
```

The first slice does not need every possible authoring feature. It does need a clean runtime shape.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-001/:

- package exists and can be imported
- in-memory catalogs can be loaded and unloaded
- messages resolve by namespaced key
- missing locale values fall back to source locale
- runtime exposes locale and direction together
- resource lookup supports non-string values

Package-local unit tests

Under packages/bijou-i18n/src/:

- missing-key behavior is explicit
- namespace unload removes lookups cleanly
- interpolation works for simple scalar values
- runtime-safe references resolve or fail clearly

Risks

- over-designing the runtime around tooling concerns too early
- under-designing the runtime into string-only helpers
- leaking layout or UI overflow policy into the i18n package
- trying to implement tooling and runtime together in one cycle

Out of scope

- @flyingrobots/bijou-i18n-tools

- spreadsheet import/export
- stale detection
- pseudo-localization
- RTL shell mirroring implementation
- shell integration beyond light proving imports

Retrospective

What landed

- new package: packages/bijou-i18n
- runtime types and exports for:
  - catalogs
  - namespaced keys
  - locale direction
  - references
  - runtime creation
- in-memory runtime with:
  - catalog load/unload
  - message lookup
  - non-string resource lookup
  - source-locale fallback
  - runtime-safe reference resolution
  - default Intl-based formatter seams
- cycle-owned playback tests under tests/cycles/LX-001/
- package-local unit tests in packages/bijou-i18n/src/runtime.test.ts
- workspace/build integration via root tsconfig.json

Drift from ideal

- the first slice does not yet integrate shell copy or DOGFOOD with message descriptors
- @flyingrobots/bijou-i18n-tools remains intentionally unstarted
- message interpolation is currently simple scalar replacement only; richer reference-aware authoring remains a tooling concern for later

Debt spawned

- LX-002 — bijou-i18n Tools Package

187

LX-002 — bijou-i18n Tools Package

docs/design/LX-002-bijou-i18n-tools-package.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-001 deliberately shipped only the runtime half of localization:

- in-memory catalogs
- locale and direction
- message/resource lookup
- runtime-safe references

That was the right first move, but a real localization system also needs workflow tooling.

This cycle exists to build the first real @flyingrobots/bijou-i18n-tools package.

It should give Bijou a provider-neutral authoring and translation workflow surface without dragging spreadsheets or files into the runtime package.

Scope of this cycle

This first tooling slice intentionally narrows scope to:

- authoring catalog types
- source hashing
- stale translation detection
- provider-neutral tabular export/import rows
- reference validation and normalization
- compilation into runtime catalogs
- pseudo-localization helper

It does **not** include:

- Google Sheets integration
- Excel integration

- CSV file parsing
- filesystem adapters
- remote catalog publishing

Those can come later if justified.

Human users

Primary human user

A builder or localization maintainer who needs to:

- track changed source strings
- export only missing/stale work
- import translated rows cleanly
- validate references before runtime
- compile authoring catalogs into runtime catalogs

Human hill

A builder can manage localization workflow data entirely in memory through one clean tooling package, without having to invent stale detection or spreadsheet row formats from scratch.

Agent users

Primary agent user

An agent helping maintain localization catalogs and workflow data.

They need to:

- detect stale translations
- understand which rows need work
- import reviewed rows safely
- validate references before shipping
- generate pseudo-localized stress content for layout review

Agent hill

An agent can reason about localization workflow state explicitly through authoring catalogs and tabular rows, instead of diffing arbitrary JSON blobs or hidden spreadsheets.

Human playback

1. A builder has authoring catalogs in memory.
2. They change an English source string.
3. The tooling marks affected translations stale automatically.
4. They export only missing/stale rows into a generic tabular shape.
5. They import completed rows back in.
6. They compile the result into runtime catalogs.
7. They load those runtime catalogs into bijou-i18n.

Agent playback

1. An agent reads authoring catalogs.
2. It identifies stale or missing translations by source hash.
3. It exports review rows for a target locale.
4. It imports updated rows.
5. It validates references and catches broken/cyclic ones before runtime.
6. It generates pseudo-localized output to stress UI copy.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- tooling must stay separate from the runtime package
- tabular workflow support must be provider-neutral
- stale detection must be automatic, not manual bookkeeping
- reference-heavy catalogs must fail clearly before runtime
- pseudo-localization is a tooling concern, not a runtime concern

Deliverable

Introduce a new package:

- packages/bijou-i18n-tools

With the first workflow surface for:

- authoring catalogs
- translation rows

- stale detection
- import/export
- compilation
- pseudo-localization

Proposed API direction

The first slice should be close to:

```
interface AuthoringCatalog { ... }
interface AuthoringCatalogEntry { ... }
interface TranslationRow { ... }

function hashSourceValue(value: unknown): string;
function markStaleTranslations(catalogs: readonly AuthoringCatalog[]): readonly
↪  AuthoringCatalog[];
function exportTranslationRows(catalogs: readonly AuthoringCatalog[], locale: string):
↪  readonly TranslationRow[];
function importTranslationRows(catalogs: readonly AuthoringCatalog[], rows: readonly
↪  TranslationRow[]): readonly AuthoringCatalog[];
function compileCatalogs(catalogs: readonly AuthoringCatalog[]): readonly I18nCatalog[];
function pseudoLocalize(value: string): string;
```

Exact names can still move, but the responsibilities should not.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-002/:

- package exists and can be imported
- changed source strings mark existing translations stale
- export returns only missing/stale rows for a locale
- imported rows merge back and compile into runtime catalogs consumable by bijou-i18n
- pseudolocalization visibly lengthens text for layout stress

Package-local unit tests

Under packages/bijou-i18n-tools/src/:

- stable source hashing
- broken references fail clearly
- cyclic references fail clearly
- compilation emits runtime catalogs with runtime-safe reference shapes

Risks

- overcommitting to a spreadsheet provider too early

- making the tabular format too weak to support real workflows
- leaking UI/layout policy into tooling
- treating references as optional even though real localization workflows need them

Out of scope

- actual file readers/writers
- actual CSV/XLSX parsers
- Google Sheets or Excel adapters
- shell integration beyond proving compiled catalogs load into the runtime package

Retrospective

What landed

- new package:
  - packages/bijou-i18n-tools
- new cycle-owned playback tests:
  - tests/cycles/LX-002/bijou-i18n-tools.test.ts
- new package-local unit tests:
  - packages/bijou-i18n-tools/src/tools.test.ts
- first tooling surface for:
  - authoring catalogs
  - stable source hashing
  - automatic stale-translation detection
  - provider-neutral translation rows
  - import/export of translated rows
  - reference validation during compilation
  - compilation into @flyingrobots/bijou-i18n runtime catalogs
  - pseudo-localization for layout stress

Drift from ideal

No material drift in scope.

One intentional boundary held:

- spreadsheet, CSV, XLSX, Google Sheets, and filesystem adapters remain out of scope

That is a feature, not a miss. The cycle stayed focused on the pure tooling core instead of smuggling provider assumptions into the first package.

Debt spawned

Spawned:

- LX-003 — Spreadsheet Adapters and Catalog Exchange Workflows

188

LX-003 — Spreadsheet Adapters and Catalog Exchange Workflows

docs/design/LX-003-spreadsheet-adapters-and-catalog-exchange-workflows.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-002 gave Bijou a real localization tooling core:

- authoring catalogs
- source hashing
- stale detection
- provider-neutral translation rows
- runtime catalog compilation
- pseudo-localization

That solved in-memory workflow logic, but not the exchange problem. Real localization work still has to move through:

- spreadsheet-shaped review workflows
- structured handoff bundles
- typed values and references that survive export/import without turning into mush

This cycle exists to add a provider-neutral exchange layer to @flyingrobots/bijou-i18n-tools.

Scope of this cycle

This cycle intentionally covers:

- a spreadsheet-friendly workbook model
- typed export/import for translation rows
- a serializable catalog-bundle format for authoring catalogs
- reference-safe encoding/decoding for message/resource/data values

- validation at the exchange boundary

It does **not** include:

- CSV parsing
- XLSX parsing
- Google Sheets integration
- filesystem adapters
- remote services
- runtime i18n changes

The goal is to define the exchange seam that those adapters can target later.

Human users

Primary human user

A localization maintainer who needs to:

- export translatable work into a spreadsheet-like shape
- import reviewed translations safely
- move full authoring catalogs across tools or repos
- preserve typed values and references without inventing a bespoke format

Human hill

A builder can move localization workflow data through spreadsheet-like and bundle-like formats without losing type/reference fidelity or writing their own ad hoc serialization rules.

Agent users

Primary agent user

An agent helping with localization maintenance and exchange.

It needs to:

- generate workbook-shaped review payloads
- read imported workbook data back into typed translation rows
- exchange full authoring catalogs in a stable serializable form
- validate malformed exchange data before it reaches runtime compilation

Agent hill

An agent can reason about localization exchange through explicit workbook/bundle contracts instead of reverse-engineering spreadsheet columns or raw JSON blobs.

Human playback

1. A builder has authoring catalogs in memory.
2. They export stale or missing work for de into a workbook-shaped review payload.
3. A translator or tool edits the workbook rows.
4. The builder imports the workbook back into translation rows and merges it into the authoring catalogs.
5. They export the full catalogs as a serializable bundle for storage, transport, or staged review.
6. They import the bundle back and compile it into runtime catalogs.

Agent playback

1. An agent reads authoring catalogs.
2. It exports a workbook for a target locale.
3. It can inspect typed cells and references explicitly instead of guessing.
4. It imports workbook rows back into translation rows.
5. It exports/imports full catalog bundles for handoff or normalization.
6. It validates malformed references or invalid encoded values before compilation.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- exchange formats must stay provider-neutral
- typed values and references must survive roundtrips explicitly
- workbook exchange must be string-cell-safe
- malformed exchange data must fail clearly before runtime compilation
- adapter concerns must remain outside this cycle

Deliverable

Extend packages/bijou-i18n-tools with exchange APIs for:

- workbook export/import
- typed cell encoding/decoding
- serializable catalog bundles
- bundle export/import

Proposed API direction

The first slice should be close to:

```
interface ExchangeWorkbook { ... }
interface ExchangeSheet { ... }
interface ExchangeRow { ... }
interface CatalogBundle { ... }

function exportTranslationWorkbook(
  catalogs: readonly AuthoringCatalog[],
  locale: string,
): ExchangeWorkbook;

function importTranslationWorkbook(
  workbook: ExchangeWorkbook,
): readonly TranslationRow[];

function exportCatalogBundle(
  catalogs: readonly AuthoringCatalog[],
): CatalogBundle;

function importCatalogBundle(
  bundle: CatalogBundle,
): readonly AuthoringCatalog[];
```

Exact names can still move, but the responsibilities should not.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-003/:

- workbook export produces spreadsheet-friendly rows for stale/missing translation work
- edited workbook data imports back into translation rows and merges into authoring catalogs
- catalog bundles roundtrip full authoring catalogs without losing references or typed values
- malformed workbook/bundle payloads fail clearly

Package-local unit tests

Under packages/bijou-i18n-tools/src/:

- typed value encoding/decoding preserves strings, objects, arrays, booleans, null, and refs
- workbook import rejects malformed headers or invalid encoded values
- bundle import rejects malformed catalog payloads
- workbook rows remain string-cell-safe

Risks

- overfitting the workbook shape to one spreadsheet vendor
- inventing a bundle format that loses reference fidelity
- making the exchange format too magical to validate clearly
- leaking filesystem/provider concerns into the pure tooling package

Out of scope

- CSV readers/writers
- XLSX readers/writers
- Google Sheets integration
- storage or persistence adapters
- runtime integration beyond proving the roundtripped catalogs still compile cleanly

Retrospective

What landed

- @flyingrobots/bijou-i18n-tools now ships a provider-neutral exchange layer for:
  - spreadsheet-friendly translation workbooks
  - typed value encoding/decoding
  - serializable catalog bundles
- new cycle-owned playback tests:
  - tests/cycles/LX-003/bijou-i18n-exchange.test.ts
- new package-local exchange tests:
  - packages/bijou-i18n-tools/src/exchange.test.ts
- workbook exchange now roundtrips stale/missing localization work back into runtime-consumable catalogs
- catalog bundles now roundtrip full authoring catalogs without losing typed values or runtime-safe refs

Drift from ideal

No material drift in scope.

One boundary held intentionally:

- there are still no CSV/XLSX/Google Sheets/file adapters in this package

That is correct for this cycle. LX-003 defined the exchange seam those adapters can target without polluting the pure tooling core.

Debt spawned

Spawned:

- LX-004 — Provider Adapters for Workbook and Bundle Exchange

189

LX-004 — Provider Adapters for Workbook and Bundle Exchange

docs/design/LX-004-provider-adapters-for-workbook-and-bundle-exchange.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-003 established the provider-neutral exchange seam in @flyingrobots/bijou-i18n-tools:

- spreadsheet-friendly workbooks
- serializable catalog bundles
- typed value encoding/decoding

That seam is useful, but still abstract. Real workflows need concrete adapters that can move those formats through common external media.

This cycle exists to prove the seam with the smallest honest adapter layer:

- delimited text for sheets (csv / tsv)
- JSON for catalog bundles

Scope of this cycle

This cycle intentionally covers:

- serializing an ExchangeSheet to TSV
- parsing an ExchangeSheet from TSV
- serializing an ExchangeSheet to CSV
- parsing an ExchangeSheet from CSV
- serializing a CatalogBundle to JSON
- parsing a CatalogBundle from JSON

It does **not** include:

- filesystem helpers

- XLSX
- Google Sheets
- remote services
- multi-file workbook orchestration

The point is to prove the adapter surface without bringing in provider/platform coupling yet.

Human users

Primary human user

A localization maintainer who needs to:

- hand off one exchange sheet to external tooling in a real delimited text format
- bring edited rows back from that format
- move full catalog bundles through a serializable JSON representation

Human hill

A builder can export a sheet to a real CSV/TSV string, import it back, and move full authoring bundles through JSON without inventing one-off parsing logic.

Agent users

Primary agent user

An agent maintaining localization workflows.

It needs to:

- emit delimited review sheets that non-Bijou tooling can consume
- parse those sheets back safely
- serialize and deserialize full catalog bundles for transport or review

Agent hill

An agent can move localization exchange data through explicit adapter APIs instead of hand-rolling CSV escaping or JSON bundle parsing in each workflow.

Human playback

1. A builder exports a translation workbook for de.

2. They take the sheet and serialize it to TSV or CSV.
3. Another tool edits the serialized rows.
4. The builder parses the edited sheet back into an ExchangeSheet.
5. They import the workbook rows and merge them into authoring catalogs.
6. They serialize a full catalog bundle to JSON and later restore it back into authoring catalogs.

Agent playback

1. An agent exports a workbook.
2. It serializes the sheet to TSV or CSV for an external review tool.
3. It parses the edited sheet back.
4. It serializes full catalog bundles to JSON for handoff or snapshotting.
5. It restores bundles from JSON and feeds them back into the tooling core.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- adapter logic must build on the LX-003 exchange seam instead of replacing it
- CSV/TSV parsing must preserve exact cell strings
- delimiter/quote/newline edge cases must be tested, not hand-waved
- JSON bundle adapters must stay explicit and version-safe
- filesystem and service concerns remain out of scope

Deliverable

Extend packages/bijou-i18n-tools with adapter APIs for:

- ExchangeSheet <-> csv
- ExchangeSheet <-> tsv
- CatalogBundle <-> json

Proposed API direction

The first slice should be close to:

```
type DelimitedFormat = 'csv' | 'tsv';

function serializeExchangeSheet(
  sheet: ExchangeSheet,
  format: DelimitedFormat,
): string;

function parseExchangeSheet(
  name: string,
  input: string,
  format: DelimitedFormat,
): ExchangeSheet;

function serializeCatalogBundleJson(bundle: CatalogBundle): string;
function parseCatalogBundleJson(input: string): CatalogBundle;
```

Higher-level helpers can be added later if needed, but this cycle should prove the core adapter surface.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-004/:

- exported workbook sheets roundtrip through TSV and still import into runtime-consumable catalogs
- exported workbook sheets roundtrip through CSV even with commas, quotes, and newlines in cells
- catalog bundles roundtrip through JSON and still restore full authoring catalogs
- malformed CSV/TSV/JSON payloads fail clearly

Package-local unit tests

Under packages/bijou-i18n-tools/src/:

- CSV quoting/escaping is correct
- TSV preserves tabs/newlines through quoting rules
- parsers reject ragged rows
- JSON bundle parser rejects malformed versioned payloads

Risks

- under-testing delimiter edge cases
- accidentally normalizing away exact cell payloads
- letting adapter helpers grow into file I/O abstractions
- making JSON bundle parsing too permissive

Out of scope

- file reads/writes
- XLSX
- Google Sheets
- multi-sheet ZIP/manifest packaging
- service adapters of any kind

Retrospective

What landed

- new adapter surface in @flyingrobots/bijou-i18n-tools for:
  - ExchangeSheet <-> CSV
  - ExchangeSheet <-> TSV
  - CatalogBundle <-> JSON
- new cycle-owned playback tests:
  - tests/cycles/LX-004/bijou-i18n-adapters.test.ts
- new package-local adapter tests:
  - packages/bijou-i18n-tools/src/adapters.test.ts
- adapter coverage for:
  - CSV quoting and escaping
  - TSV cell preservation
  - ragged-row rejection
  - malformed JSON bundle rejection
- explicit adapter exports layered on top of the LX-003 workbook/bundle seam instead of bypassing it

Drift from ideal

No material drift in scope.

One deliberate boundary held:

- there are still no filesystem helpers, XLSX adapters, or Google Sheets/service integrations

That is correct for this cycle. LX-004 proved the adapter surface with plain text and JSON formats first.

Debt spawned

Spawned:

- LX-005 — Rich Spreadsheet and Filesystem Adapters

190

LX-005 — Rich Spreadsheet and Filesystem Adapters

docs/design/LX-005-rich-spreadsheet-and-filesystem-adapters.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-004 proved the first concrete adapter seam in @flyingrobots/bijou-i18n-tools:

- CSV sheet adapters
- TSV sheet adapters
- JSON bundle adapters

That still leaves a practical gap for real automation and tooling:

- reading and writing those formats from the filesystem
- handling whole workbooks instead of one sheet string at a time

This cycle exists to add the Node-specific filesystem layer on top of the existing exchange seam.

Scope of this cycle

This cycle intentionally covers:

- a new Node-specific package for filesystem helpers
- reading/writing one ExchangeSheet file in CSV or TSV
- reading/writing one CatalogBundle JSON file
- reading/writing a workbook directory made of:
  - a manifest file
  - one sheet file per sheet

It does **not** include:

- XLSX
- Google Sheets
- remote services

- zip packaging
- changing the pure @flyingrobots/bijou-i18n-tools exchange model

The point is to make the current exchange seam practical in Node without dragging platform concerns into the core tooling package.

Human users

Primary human user

A builder or localization maintainer who needs to:

- save exchange sheets to disk
- read edited sheets back from disk
- store and reload full catalog bundles
- hand off or restore complete workbooks as a directory artifact

Human hill

A builder can persist workbook and bundle exchange artifacts through one small Node helper package, without hand-writing file I/O or inventing an ad hoc workbook directory format.

Agent users

Primary agent user

An agent automating localization workflows.

It needs to:

- write sheets and bundles to disk
- read them back deterministically
- stage workbook directories for review workflows
- reason about paths and formats explicitly instead of shelling out to one-off scripts

Agent hill

An agent can automate localization exchange through explicit Node adapters instead of rebuilding CSV/TSV/JSON file orchestration in every workflow.

Human playback

1. A builder exports a workbook in memory.

2. They write it to a directory on disk in TSV or CSV form.
3. A tool or person edits the sheet files.
4. The builder reads the workbook directory back into memory.
5. They import the workbook rows and merge the updates.
6. They also save and restore full catalog bundles as JSON files.

Agent playback

1. An agent exports a workbook.
2. It writes the workbook to a directory for review.
3. It later reads the workbook directory back.
4. It writes and reads catalog bundle files as deterministic JSON artifacts.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- filesystem concerns must stay out of the pure @flyingrobots/bijou-i18n-tools package
- Node helpers must build on the existing workbook/bundle seam instead of replacing it
- directory formats must be explicit and versioned
- file helpers must be deterministic about sheet ordering and file names
- richer spreadsheet providers remain out of scope

Deliverable

Introduce a new package:

- packages/bijou-i18n-tools-node

With Node filesystem helpers for:

- ExchangeSheet files (csv / tsv)
- CatalogBundle files (json)
- workbook directories (manifest + sheet files)

Proposed API direction

The first slice should be close to:

```
function writeExchangeSheetFile(path: string, sheet: ExchangeSheet, format?:
↪  DelimitedFormat): Promise<void>;
function readExchangeSheetFile(path: string, format?: DelimitedFormat):
↪  Promise<ExchangeSheet>;

function writeCatalogBundleFile(path: string, bundle: CatalogBundle): Promise<void>;
function readCatalogBundleFile(path: string): Promise<CatalogBundle>;

function writeExchangeWorkbookDirectory(
  path: string,
  workbook: ExchangeWorkbook,
  format: DelimitedFormat,
): Promise<void>;

function readExchangeWorkbookDirectory(path: string): Promise<ExchangeWorkbook>;
```

Exact names can still move, but the responsibilities should not.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-005/:

- workbook directories roundtrip through disk and still import back into runtime-consumable catalogs
- bundle files roundtrip through disk and restore full authoring catalogs
- file helpers infer format from extension where appropriate
- malformed manifests or missing sheet files fail clearly

Package-local unit tests

Under packages/bijou-i18n-tools-node/src/:

- sheet files preserve CSV/TSV content exactly
- workbook directory manifest is versioned and deterministic
- unsupported extensions fail clearly
- missing files fail clearly

Risks

- leaking too much platform logic back into the pure tools package
- inventing a workbook directory shape that is not explicit enough
- poor format inference or path handling
- accidentally making the helpers non-deterministic

Out of scope

- XLSX
- Google Sheets
- remote storage/services
- watching/sync features
- shell integration

Retrospective

What landed

- new Node-specific package:
  - packages/bijou-i18n-tools-node
- new cycle-owned playback tests:
  - tests/cycles/LX-005/bijou-i18n-tools-node.test.ts
- new package-local tests:
  - packages/bijou-i18n-tools-node/src/filesystem.test.ts
- Node filesystem helpers for:
  - reading/writing one ExchangeSheet file as CSV or TSV
  - reading/writing one CatalogBundle file as JSON
  - reading/writing workbook directories with a versioned manifest
- extension-based format inference for .csv, .tsv, and .json
- explicit missing-manifest, missing-sheet, and unsupported-extension failures

Drift from ideal

No material drift in scope.

One boundary held intentionally:

- XLSX and Google Sheets stayed out of this cycle

That is correct. LX-005 made the current exchange seam practical in Node without adding a richer spreadsheet provider yet.

Debt spawned

Spawned:

- LX-006 — XLSX Localization Adapters

191

LX-006 — XLSX Localization Adapters

docs/design/LX-006-xlsx-localization-adapters.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-005 made the exchange seam practical on disk through:

- CSV / TSV sheet files
- JSON bundle files
- workbook directories with a manifest

That still leaves a common real-world localization format uncovered:

- .xlsx workbook exchange

This cycle exists to add a true spreadsheet provider adapter on top of the existing workbook seam without dragging spreadsheet-library concerns into the pure runtime/tools packages.

Scope of this cycle

This cycle intentionally covers:

- a dedicated XLSX adapter package
- serializing an ExchangeWorkbook to XLSX bytes
- parsing XLSX bytes back into an ExchangeWorkbook
- deterministic handling of workbook sheet order and required workbook columns

It does **not** include:

- Google Sheets
- remote services
- filesystem helpers for XLSX paths
- changing the pure @flyingrobots/bijou-i18n-tools workbook model
- changing bundle JSON exchange

The point is to prove the provider seam with one richer spreadsheet format first.

Human users

Primary human user

A localization maintainer who needs to:

- hand a workbook to translators in a true spreadsheet format
- import edited spreadsheets back into the existing Bijou exchange flow
- keep using the same workbook schema regardless of carrier format

Human hill

A maintainer can export and import localization workbooks as .xlsx files without losing typed exchange values or inventing a second workbook model.

Agent users

Primary agent user

An agent automating localization exchange.

It needs to:

- export workbook bytes suitable for XLSX workflows
- parse returned XLSX workbook bytes deterministically
- stay on the same workbook seam as CSV / TSV / filesystem helpers

Agent hill

An agent can automate true spreadsheet exchange without shelling out to ad hoc conversion scripts or bypassing the Bijou workbook model.

Human playback

1. A maintainer exports a German translation workbook.
2. They serialize it to an XLSX blob.
3. A spreadsheet editor changes translated values.
4. The maintainer parses the XLSX blob back into an ExchangeWorkbook.
5. They import the rows and merge the translations back into authoring catalogs.
6. The resulting runtime catalogs resolve the edited translations correctly.

Agent playback

1. An agent exports an ExchangeWorkbook.
2. It serializes that workbook to XLSX bytes for review or delivery.
3. It later parses returned XLSX bytes back into an ExchangeWorkbook.
4. It imports the workbook rows and compiles catalogs without losing typed values, refs, or sheet ordering.

Linked invariants

- Tests Are the Spec
- Graceful Lowering Preserves Meaning
- Docs Are the Demo

Invariants for this cycle

- spreadsheet-provider logic must stay out of @flyingrobots/bijou-i18n-tools
- XLSX adapters must preserve the existing workbook schema instead of inventing a new one
- workbook sheet order must remain deterministic
- required translation workbook columns must remain explicit
- service-backed adapters remain out of scope for this cycle

Deliverable

Introduce a new package:

- packages/bijou-i18n-tools-xlsx

With XLSX adapters for:

- ExchangeWorkbook -> XLSX bytes
- XLSX bytes -> ExchangeWorkbook

Proposed API direction

The first slice should be close to:

```
function serializeExchangeWorkbookXlsx(workbook: ExchangeWorkbook): Uint8Array;
function parseExchangeWorkbookXlsx(input: Uint8Array | ArrayBuffer): ExchangeWorkbook;
```

Exact names can still move, but the responsibilities should not.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-006/:

- XLSX workbooks roundtrip through bytes and still import back into runtime-consumable catalogs
- the new package exports only the XLSX-focused workbook adapters
- typed values and refs survive the roundtrip when workbook rows are edited after parsing

Package-local unit tests

Under packages/bijou-i18n-tools-xlsx/src/:

- workbook sheet order is preserved
- required columns survive serialization and parsing
- missing required headers fail clearly
- non-string cell values are normalized back into the exchange row shape correctly

Risks

- leaking spreadsheet-library assumptions into the pure tools package
- accidentally loosening the workbook schema in order to fit XLSX convenience
- poor error messages for malformed workbook headers
- introducing too much provider-specific policy too early

Out of scope

- Google Sheets
- remote localization services
- XLSX filesystem wrappers
- live sync
- changing the exchange row schema

Retrospective

What landed

- a new dedicated package:
  - packages/bijou-i18n-tools-xlsx

- new cycle-owned playback tests:
  - tests/cycles/LX-006/bijou-i18n-xlsx.test.ts
- new package-local tests:
  - packages/bijou-i18n-tools-xlsx/src/xlsx.test.ts
- XLSX workbook adapters for:
  - ExchangeWorkbook -> XLSX bytes
  - XLSX bytes -> ExchangeWorkbook
- explicit validation for:
  - missing header rows
  - missing required headers
  - duplicate headers
- preservation of:
  - workbook sheet order
  - translation workbook columns
  - typed row payloads after parse/import

Drift from ideal

One deliberate scope narrowing happened:

- the backlog item originally mentioned service-oriented adapters too

That work did **not** land here, and that is correct. LX-006 proved the richer spreadsheet-provider seam with XLSX first and left remote/service adapters for a later cycle.

Debt spawned

Spawned:

- LX-007 — Service-Oriented Localization Adapters

192

LX-008 — Localize Shell Chrome and DOGFOOD

docs/design/LX-008-localize-shell-chrome-and-dogfood.md

Legend: LX — Localization and Bidirectionality

Why this cycle exists

LX-001 through LX-006 built the localization substrate:

- runtime catalogs
- tools and exchange workflows
- filesystem helpers
- XLSX adapters

But the humane shell and DOGFOOD still ship mostly hardcoded English copy.

This cycle exists to prove that the i18n work is real in product surfaces:

- shell chrome
- shell overlays and drawers
- DOGFOOD landing and onboarding copy

Scope of this cycle

This cycle intentionally covers:

- a shell-level i18n seam for createFramedApp()
- localized shell-owned defaults such as:
  - command palette title
  - default search title
  - settings drawer title
  - quit confirm copy
  - footer notification cue and shell drawer footer hints
- DOGFOOD-localized copy for:
  - landing footer and prompt
  - search/settings labels
  - empty-state onboarding copy

    - settings row labels, descriptions, and feedback
- one visible direction-aware shell behavior driven by i18n direction metadata

It does **not** include:

- full generic bidirectional layout support across every shell surface
- localizing every example in the repo
- provider/service adapters
- persistence

The point is to prove the localization runtime in the shell and DOGFOOD without pretending RTL support is solved everywhere.

Human users

Primary human user

A builder or end user working in a non-English locale.

They need to:

- see shell controls and onboarding copy in their locale
- trust that shell labels, footer hints, and settings copy are not locked to English
- see at least one shell behavior react to logical direction instead of physical left/right assumptions

Human hill

A non-English user can enter DOGFOOD and immediately see localized shell/UI copy, while a builder can point to DOGFOOD as proof that Bijou localization is not theoretical.

Agent users

Primary agent user

An agent integrating or validating localized Bijou surfaces.

It needs to:

- load catalogs into the shell and docs proving surface
- verify localized strings at the rendered shell boundary
- reason about direction-aware shell behavior through explicit runtime metadata

Agent hill

An agent can localize shell chrome and DOGFOOD through explicit i18n contracts instead of editing hardcoded English strings in place.

Human playback

1. A builder creates a localized runtime.
2. They launch DOGFOOD in that locale.
3. The landing footer and prompt use localized copy.
4. After entering the docs, the shell drawer titles, search surface, footer hints, and settings copy use localized copy too.
5. In an RTL locale, at least one shell drawer placement mirrors logically instead of staying physically fixed.

Agent playback

1. An agent loads shell and DOGFOOD catalogs into an i18n runtime.
2. It passes that runtime into the shell/docs app.
3. It verifies localized shell strings in rendered frames.
4. It verifies pseudo-localized copy appears in DOGFOOD.
5. It verifies direction metadata changes a visible shell behavior.

Linked invariants

- Tests Are the Spec
- Visible Controls Are a Promise
- Focus Owns Input
- Shell Owns Shell Concerns
- Docs Are the Demo

Invariants for this cycle

- createFramedApp() must own shell-localized defaults instead of pushing shell copy responsibility entirely to apps
- DOGFOOD should prove canonical shell localization instead of inventing a separate one-off mechanism
- the direction signal must cause at least one visible shell change
- shell localization must degrade safely when an i18n runtime is absent

Deliverable

Introduce:

- a shell i18n seam in bijou-tui
- a canonical shell catalog for frame-owned copy
- DOGFOOD-localized catalogs and runtime wiring

So that shell chrome and DOGFOOD can run against locale/direction metadata instead of hardcoded English strings.

Proposed API direction

The first slice should be close to:

```
interface CreateFramedAppOptions<PageModel, Msg> {
  readonly i18n?: I18nRuntime;
}
```

With:

- a built-in frame-shell catalog exported from bijou-tui
- DOGFOOD creating and using an i18n runtime, loading both shell and docs catalogs

Exact names can still move, but the responsibilities should not.

Tests to write first

Cycle-owned playback tests

Under tests/cycles/LX-008/:

- a localized framed app shows translated shell-owned titles and hints
- DOGFOOD renders pseudo-localized shell/docs copy when supplied localized catalogs
- an RTL runtime changes a visible shell behavior such as drawer anchor placement

Package-local tests

In existing package/example test surfaces:

- shell quit/search/settings defaults use localized copy when an i18n runtime is provided
- DOGFOOD settings/landing/onboarding copy comes from catalogs rather than hardcoded strings

Risks

- only partially localizing shell surfaces and leaving confusing English islands
- adding an i18n seam that DOGFOOD does not actually prove
- claiming direction-awareness without any visible shell behavior change
- threading translator functions everywhere instead of defining one shell-localized seam

Out of scope

- full generic RTL layout mirroring across every component
- persistence
- service-backed localization workflows
- every example app in the repo

Retrospective

What landed

- a canonical shell i18n module in bijou-tui:
  - packages/bijou-tui/src/app-frame-i18n.ts
- a createFramedApp({ i18n }) seam for shell-owned copy and direction metadata
- localized shell defaults for:
  - command palette title
  - default search title
  - settings title and footer hint
  - notification-center title, footer hint, and footer cue label
  - help title and hint
  - quit-confirm title, body, and footer hint
  - shell mode labels
- DOGFOOD-localized catalogs and runtime wiring for:
  - landing prompt and footer
  - onboarding / empty-state copy
  - localized search/settings surfaces
  - settings-row labels, descriptions, value labels, and feedback toasts
- visible direction-aware shell behavior:
  - settings drawer resolves to logical start
  - notification center resolves to logical end
  - the settings drawer visibly mirrors under RTL
- cycle-owned playback coverage in:

- tests/cycles/LX-008/localized-shell-and-dogfood.test.ts

Drift from ideal

This cycle proved the shell/runtime seam cleanly, but it did not finish every English island.

Still intentionally incomplete:

- help-overlay keybinding descriptions still mostly fall back to English strings
- notification history/body copy is only partially localized
- direction awareness is real but still narrow; only a subset of shell drawer placement is mirrored so far

That drift is acceptable for LX-008 because the point of the cycle was to prove that shell localization and direction metadata are real in product surfaces, not to declare shell localization complete.

Debt spawned

Spawned:

- LX-009 — Localize Shell Help, Notification, and Directional Surfaces

193

LX-019 — Localization Port Contract

docs/design/LX-019-localization-port-contract.md

Legend: LX — Localization and Bidirectionality

Why This Cycle Exists

Bijou already has a runtime catalog engine, formatter ports, catalog loaders, string-table tooling, and DOGFOOD locale preference ports. Those pieces make localization possible, but DOGFOOD still tends to pass the concrete i18n runtime through rendering code and ask it for naked strings.

That is too low-level for the long-term architecture.

Views, blocks, and docs surfaces should ask an application-facing localization port for a localized object:

```
const title = localization.resolve({
  key: { namespace: 'bijou.dogfood', id: 'blocks.preview.title' },
});
```

The result should preserve runtime truth:

- the requested key
- selected locale
- direction
- entry kind
- translated / fallback / missing status
- localized value
- issues and facts for tooling

The view should not know how catalogs were loaded, where JSON files live, which CSV source produced them, or how fallback catalogs were attached.

Core Rule

Localization in Bijou is a port/adapter boundary.

- Ports live where the localization vocabulary lives.
- Adapters live where the external dependency lives.

- Composition lives at the app boundary.
- Views receive localized objects, not filesystem, catalog, or runtime machinery.

Boundary

This cycle is about the application-facing localization port contract and DOGFOOD wiring.

It is not about:

- expanding translation coverage
- adding a localization dashboard
- adding a translation workbench
- changing the CSV string-table workflow
- loading every locale at runtime
- implementing remote localization services
- making blocks fetch localization data directly
- replacing the lower-level i18n runtime

Vocabulary

Localization Port

The app-facing port that resolves a localization request into a structured localized object.

It should answer:

> Given this key and interpolation values, what localized object should this view render in the selected locale?

It should not answer:

> Where are the catalog files?
> How do I read the filesystem?
> How do I compile CSV rows?
> How do I persist locale preferences?

Localized Object

The immutable result returned by the localization port.

It contains:

- key
- locale
- direction
- fallback locale
- entry kind

- resolution status
- value
- issues
- facts

The value may be a message string, data payload, or resource payload depending on the catalog entry kind.

Runtime Adapter

An adapter that wraps I18nRuntime and exposes it through LocalizationPort.

The runtime continues to own catalog state, fallback behavior, locale switching, references, and formatting. The port makes that behavior consumable by application code without leaking the runtime surface everywhere.

Runtime convenience methods that remain public, such as t(), should behave like stable API functions instead of receiver-sensitive object methods. Callers may pass or destructure these helpers without preserving a JavaScript this binding; the runtime state remains closure-owned at the adapter boundary.

Node / Filesystem Adapters

Adapters that read selected-locale generated JSON catalogs from disk belong in @flyingrobots/bijou-i18n-tools-node or application composition code.

They should return catalogs or construct runtime-backed localization ports. They should not redefine the generic localization port contract.

Package Split

The contract belongs in @flyingrobots/bijou-i18n:

```
packages/bijou-i18n
  LocalizationPort
  LocalizationRequest
  LocalizedObject
  LocalizationStatus
  LocalizationIssue
  LocalizationFact
  createRuntimeLocalizationPort()
```

Node/file adapters stay in @flyingrobots/bijou-i18n-tools-node:

```
packages/bijou-i18n-tools-node
  CSV and TSV string-table files
  generated runtime catalog JSON files
  selected-locale catalog directory loaders
```

DOGFOOD composition lives in examples/docs:

```
examples/docs
  selected locale resolution
```

```
English fallback catalog attachment
development missing-marker policy
runtime-backed LocalizationPort assembly
```

Views and block previews should receive the port or localized objects. They should not receive filesystem loaders, CSV data, generated catalog paths, or provider handles.

Expected Flow

```
CSV source string table
  -> generated per-locale JSON catalogs
  -> selected-locale catalog loader adapter
  -> I18nRuntime
  -> LocalizationPort
  -> LocalizedObject
  -> DOGFOOD / Blocks / shell surfaces
```

The fallback posture remains:

```
production:
  selected locale value
  -> English fallback catalog

non-production:
  selected locale value
  -> loud missing-localization object
  -> English fallback only when missing markers are disabled
```

DOGFOOD Implications

DOGFOOD should keep the concrete runtime at the composition boundary because the frame shell still consumes I18nRuntime and the app still needs locale activation. Ordinary DOGFOOD text lookup should move behind LocalizationPort.resolve().

The app may still hold both:

```
const runtime = createI18nRuntime(...);
const localization = createRuntimeLocalizationPort(runtime);
```

But render helpers should prefer:

```
localization.resolve({
  key: { namespace: 'bijou.dogfood', id: 'settings.language.label' },
});
```

over:

```
runtime.t({ namespace: 'bijou.dogfood', id: 'settings.language.label' });
```

Acceptance Criteria

- @flyingrobots/bijou-i18n exports a first-class localization port contract.

- Runtime lookup helpers stay receiver-independent so app code can pass them around without binding this.
- The runtime adapter returns immutable localized objects.
- Localized values enforce the portable JSON-shaped catalog boundary with deterministic errors for cyclic graphs, symbol-keyed properties, non-enumerable properties, accessors, class instances, functions, symbols, and bigint values.
- Localized objects preserve key, locale, direction, kind, status, value, issues, and facts.
- Missing selected-locale translations remain inspectable as missing localization results in non-production posture.
- Production fallback remains explicit through fallback catalogs.
- DOGFOOD text helpers use the localization port rather than direct runtime string lookups.
- DOGFOOD still keeps catalog loading and locale activation at the app composition boundary.

Tests To Write First

- Port resolution returns a localized object for a translated message.
- Port resolution reports fallback status when selected-locale catalogs omit a value and fallback catalogs provide one.
- Port resolution reports missing status and marker value when missing-marker policy is enabled.
- Resource/data entries return localized object values without converting them to strings.
- Localized objects and their issue/fact arrays are immutable.
- DOGFOOD text lookup consumes LocalizationPort and does not call I18nRuntime .t() directly.

Retrospective

Not started.

194

LX-020 - DOGFOOD Locale Demo Readiness

docs/design/LX-020-dogfood-locale-demo-readiness.md

```
id: LX-020
title: DOGFOOD Locale Demo Readiness
status: active
lane: asap
priority: high
github_issue: 340
legend: LX
```

Legend: LX - Localization and Bidirectionality

Decision Summary

DOGFOOD should stop presenting non-English locale selection as if the whole app is fully translated. For v7.2.0, the honest release-demo contract is:

- core app chrome and release-demo navigation do not show missing localization placeholders
- English-source documentation is labeled as such when selected locale content is intentionally unavailable
- localization debt remains visible to maintainers without leaking broken UI into the user-facing demo path

Sponsored Human

A release-video user needs DOGFOOD to survive a non-English locale switch without looking broken. Missing localization markers, accidental English chrome, and unclear partial-translation behavior make the shipped story feel unfinished.

Sponsored Agent

An agent needs deterministic localization witnesses that distinguish three states:

- selected-locale translation exists and is rendered

- selected-locale translation is intentionally unavailable and the app presents an English-source notice
- selected-locale translation is missing by accident and should fail the gate

Hill

After this cycle, DOGFOOD's non-English demo path is truthful enough for a release video: switching locales does not expose missing-placeholder strings in major app chrome, and documentation surfaces clearly explain when they remain English-source content.

Problem

Issue #340 came from the v7.1.0 release-video rehearsal. DOGFOOD can switch to non-English locales, but much of the app still renders English copy or missing localization markers. Existing i18n gates prove debt is counted, not that the presentation path is demo-ready.

The current debt ratchet is useful, but it is not sufficient for user-facing release claims:

```
dogfood-i18n-debt: ok (2772 raw strings; baseline 2772; 78 missing Markdown localizations;
↪  baseline 78)
```

That output tells maintainers the known debt did not grow. It does not tell a viewer why non-English DOGFOOD still shows English documentation.

Scope

- Audit the launch, navigation, settings, status/footer, and major tab chrome for the supported non-English DOGFOOD locales.
- Add deterministic coverage that fails when the release-demo path renders visible missing-localization placeholders.
- Add or update DOGFOOD copy so intentionally English-source documentation is labeled clearly when the selected locale lacks translated Markdown.
- Keep the existing debt ratchet intact; this cycle strengthens the presentation contract around it.
- Update release-facing docs/changelog so v7.2.0 describes the localization posture honestly.

Non-Goals

- Do not translate the full DOGFOOD documentation corpus in this cycle.
- Do not build a localization workbench or dashboard.
- Do not change the CSV/XLSX localization workflow.
- Do not hide known debt from maintainer tooling.

- Do not broaden v7.2.0 into the V8/V9 localization operations surface.

User Experience Contract

For non-English locales, DOGFOOD may show English documentation only if the UI makes that fallback explicit. The user should not have to infer whether English copy is a bug, an untranslated source document, or a missing catalog entry.

Major app chrome should be localized or explicitly excluded from the release demo path. Missing-placeholder strings are not acceptable on the demo path.

Lower Modes

Static Mode

Static DOGFOOD render helpers should be able to render a selected-locale path and expose whether missing placeholders appear in visible output.

Pipe Mode

Pipe/smoke output should remain deterministic and suitable for release evidence. Locale-specific output checks must avoid terminal-profile assumptions.

Accessible Mode

English-source notices should be text, not color-only styling. Screen readers and terminal readers should receive the same partial-localization status.

Localization / Directionality Posture

This cycle treats localization status as product truth. The app may still carry known translation debt, but the release-demo path must make that debt explicit instead of looking accidentally broken.

No new right-to-left layout contract is introduced. Direction metadata should continue to flow from the selected locale through the existing localization port/runtime path.

Agent Inspectability / Explainability Posture

Agents should be able to inspect:

- which locale was selected

- whether a rendered string came from selected-locale content, fallback content, or an English-source documentation notice
- whether missing localization markers appeared in the tested output

The proof should be executable through focused tests and existing DOGFOOD i18n commands, not through screenshots alone.

Linked Invariants

- DOGFOOD visible release claims must be true in the app.
- Known localization debt must stay counted by the ratchet.
- Missing localization placeholders must not appear in the release-demo path.
- English-source documentation fallback must be explicit to users.

Implementation Plan

1. Add focused failing tests that render representative non-English DOGFOOD paths and assert no visible missing-localization placeholders are present.
2. Identify the release-demo chrome surfaces that should be localized now.
3. Add an English-source documentation notice for intentionally untranslated Markdown/documentation surfaces.
4. Update catalog entries or DOGFOOD copy for the scoped chrome surfaces.
5. Preserve the existing dogfood:i18n:debt, dogfood:i18n:check, and dogfood:i18n:complete gates.
6. Update changelog and any release-facing wording needed to avoid overclaiming full DOGFOOD translation.

Tests To Write First

- A non-English DOGFOOD launch/render path fails if visible missing localization markers appear.
- A selected non-English documentation path renders an explicit English-source notice when translated Markdown is unavailable.
- Locale switching keeps major app chrome free of accidental missing markers.
- Existing debt-ratchet behavior still reports known raw-string and Markdown localization debt.

Acceptance Criteria

- Non-English release-demo path has deterministic missing-placeholder coverage.
- Major DOGFOOD chrome on that path is localized or explicitly excluded from the release-demo claim.

- English-source documentation fallback is visible and textual.
- Existing i18n debt gates remain green.
- docs/CHANGELOG.md records the demo-readiness fix.
- Issue #340 is closed by the implementation PR.

Retrospective

To be completed when the PR lands.

195

LX-021 DOGFOOD String Debt Ratchet

docs/design/LX-021-dogfood-string-debt-ratchet.md

Status

Implemented.

Tracker

- GitHub Issue: #415
- Pull Request: #416

Legend

- Linked legend: LX, Localization and Bidirectionality
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

DOGFOOD must stop treating raw visible strings as harmless residue. This cycle removes a material slice from the raw-string baseline by moving selected component story copy behind the existing DOGFOOD localization catalog workflow.

Current Truth

Live counts on main at aab792ed:

- DOGFOOD raw-string debt: 2,761
- component-stories raw-string debt: 1,631
- dogfood-blocks raw-string debt: 681

- docs-app raw-string debt: 258
- missing Markdown localizations: 78
- aggregate Code Dojo debt: 951

Target for this cycle:

- remove at least 100 DOGFOOD raw-string findings
- lower the total raw-string baseline to 2,661 or lower
- lower component-stories to 1,531 or lower
- keep missing Markdown localizations flat at 78
- keep aggregate Code Dojo debt at 951 or lower

Implemented counts on this branch before review:

- DOGFOOD raw-string debt: 2,656
- component-stories raw-string debt: 1,526
- missing Markdown localizations: 78
- aggregate Code Dojo debt: 910
- ESLint findings: 502
- next aggregate Code Dojo target: 860 or lower

Scope

- Convert a coherent high-density slice from examples/docs/stories.ts.
- Add source rows and current en, de, es, and fr values in examples/docs/i18n/source/dogfood-strings.csv.
- Regenerate examples/docs/i18n/catalogs/*/bijou.dogfood.json.
- Ratchet examples/docs/i18n-debt.ts to the proven lower total and component-stories count.
- Update docs/CHANGELOG.md with the landed counts and next string-debt target.

Non-Goals

- No full DOGFOOD localization claim.
- No Markdown corpus translation in this cycle.
- No catalog placeholder rows that satisfy the gate while preserving accidental English for non-English locales.
- No Code Dojo baseline increases.
- No release-version scope.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Did DOGFOOD raw-string debt fall by at least 100?

2. Did every changed catalog key have current rows for all supported locales?
3. Did component-stories own the raw-string reduction instead of shifting debt to another surface?
4. Did missing Markdown localization debt stay flat?
5. Did Code Dojo aggregate debt stay at 951 or lower?

User Experience Contract

Converted story copy must still read as the same DOGFOOD demonstration. The localized text may use conservative translations, but it must not replace visible content with placeholder keys or intentionally untranslated fallback rows.

Localization And Directionality Posture

This cycle uses the existing DOGFOOD localization port. It does not introduce a new directionality model. Any converted copy should continue to flow through dogfoodLocalizedText so locale selection, fallback interpolation, and catalog checks remain centralized.

Agent Inspectability And Explainability Posture

The PR must make the changed keys easy to audit:

- catalog IDs should be grouped by the story surface they serve
- fallback copy should remain adjacent to the rendered usage
- final debt counts should be recorded in this design doc or the PR summary

Linked Invariants

- DOGFOOD i18n source catalog: examples/docs/i18n/source/dogfood-strings.csv
- DOGFOOD i18n debt baseline: examples/docs/i18n-debt.ts
- DOGFOOD i18n build: scripts/dogfood-i18n-build.ts
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- TypeScript standards: docs/typescript-code-standards.editors-edition.md

Implementation Outline

1. Add localization access to the selected component story rendering helpers.
2. Convert the selected raw literals to dogfoodLocalizedText calls with stable stories.* catalog IDs.
3. Add current rows for every new or changed key in the source CSV.

4. Regenerate runtime catalogs.
5. Run the DOGFOOD i18n gates, focused story tests, and Code Dojo debt checks.
6. Ratchet the i18n debt baseline only after the live inventory proves the lower count.

Tests To Write First

- Add focused tests only if the conversion changes a behavior boundary or story state shape.
- For copy-only localization movement, the failing proof is the current npm run dogfood:i18n:debt inventory against the lower target before the implementation lands.

Validation Plan

- npm run dogfood:i18n:complete
- npm run dogfood:i18n:check
- npm run dogfood:i18n:debt
- npm run code-dojo:slice -- examples/docs/stories.ts
- focused tests for DOGFOOD/storybook rendering if touched paths expose one
- npm run code-dojo:debt
- npm run docs:inventory
- npm run lint
- git diff --check

Acceptance Criteria

- DOGFOOD raw-string debt is 2,661 or lower.
- component-stories raw-string debt is 1,531 or lower.
- Missing Markdown localization debt remains 78 or lower.
- Code Dojo aggregate debt remains 951 or lower.
- Runtime catalogs are regenerated from the source CSV.
- Issue #415 and the pull request are cross-linked.

196

LX-022 DOGFOOD App Main Ratchet

docs/design/LX-022-dogfood-app-main-ratchet.md

Status

Implemented.

Tracker

- GitHub Issue: #417
- Pull Request: #418

Legend

- Linked legend: LX, Localization and Bidirectionality
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

DOGFOOD must stop hiding its primary application surface inside one giant module. This cycle starts breaking up examples/docs/app.ts while removing a material Code Dojo slice from the same file.

Current Truth

Live counts on main at d1ecced7:

- aggregate Code Dojo debt: 910
- next aggregate Code Dojo target: 860 or lower
- ESLint findings: 502
- examples/docs/app.ts ESLint findings: 118

- examples/docs/app.ts file-context baseline: 5,794 lines and 203,093 bytes
- DOGFOOD raw-string debt: 2,656
- docs-app raw-string debt: 258
- missing Markdown localizations: 78

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 792
- next aggregate Code Dojo target: 742 or lower
- ESLint findings: 384
- examples/docs/app.ts ESLint findings: 0
- examples/docs/app.ts file-context baseline: 5,793 lines and 201,982 bytes
- DOGFOOD raw-string debt: 2,654
- docs-app raw-string debt: 256
- missing Markdown localizations: 78

Target for this cycle:

- remove at least 50 counted Code Dojo violations
- lower the aggregate Code Dojo baseline to 860 or lower
- shrink examples/docs/app.ts in both lines and bytes
- keep every new extracted file under normal file/context limits
- keep DOGFOOD raw-string debt flat or lower

Scope

- Extract a coherent DOGFOOD app slice from examples/docs/app.ts into small modules.
- Clean the extracted path so it does not carry unsafe casts, any, or other current ESLint debt into the new files.
- Fix adjacent examples/docs/app.ts lint findings when they support the same decomposition.
- Ratchet scripts/code-dojo/baselines/eslint.json, scripts/code-dojo/baselines/file-context.json, package.json, and docs/code-dojo-exceptions.md only after measured lower counts prove the new values.
- Update docs/CHANGELOG.md with the landed counts.

Non-Goals

- No full DOGFOOD app rewrite.
- No broad theme-lab, landing-page, or navigation redesign.
- No placeholder localization rows.
- No Code Dojo baseline increases.
- No release-version scope.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Did aggregate Code Dojo debt fall to 860 or lower?
2. Did examples/docs/app.ts shrink in both lines and bytes?
3. Did every new extracted module stay below 150 lines and 12,000 bytes?
4. Did DOGFOOD raw-string debt stay flat or lower?
5. Did the touched DOGFOOD surface still pass focused smoke or preview tests?

User Experience Contract

The DOGFOOD app must render the same visible product behavior after the split. Navigation rows, guide rows, landing behavior, theme selection, and preview panes must preserve their current semantics unless this design explicitly says otherwise.

Localization And Directionality Posture

Any visible string converted during this slice must use the existing DOGFOOD localization port and catalog workflow. Moving raw strings to a new module is not a debt reduction unless the raw-string inventory proves the total count went down.

Agent Inspectability And Explainability Posture

The split should create named modules that reveal responsibility from their file names. Reviewers should be able to tell which code owns app row rendering, standard block preview data, or other extracted behavior without scrolling through the full DOGFOOD app file.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- file/context baseline: scripts/code-dojo/baselines/file-context.json
- ESLint baseline: scripts/code-dojo/baselines/eslint.json
- DOGFOOD i18n debt baseline: examples/docs/i18n-debt.ts

Implementation Outline

1. Extract the app row formatter path from examples/docs/app.ts and remove unsafe token casts.

2. Extract one additional coherent app slice if needed to meet the 50-violation goalpost without creating over-limit files.
3. Run focused ESLint and npm run code-dojo:changed on the touched slice.
4. Run DOGFOOD i18n debt checks to prove raw strings were not shifted.
5. Ratchet baselines and update the exception ledger after live counts prove lower values.

Tests To Write First

- The regression proof for the row-format path is the current ESLint failure cluster in examples/docs/app.ts; after extraction those findings must be absent from focused ESLint.
- Add behavioral tests only if the extraction changes a runtime boundary.

Validation Plan

- npx eslint examples/docs/app.ts <new extracted files>
- npm run code-dojo:changed
- npm run dogfood:i18n:debt
- focused DOGFOOD smoke or preview tests for touched surfaces
- npm run code-dojo:debt
- npm run docs:inventory
- npm run lint
- git diff --check

Implemented validation:

- npx eslint examples/docs/app.ts examples/docs/app-row-format.ts examples/docs/app-markdown.ts
- npm run code-dojo:changed
- npm run dogfood:i18n:debt

Acceptance Criteria

- Aggregate Code Dojo debt is 860 or lower.
- examples/docs/app.ts has fewer than 5,794 lines and fewer than 203,093 bytes.
- New extracted files are below normal file/context limits.
- DOGFOOD raw-string debt is 2,656 or lower.
- docs-app raw-string debt is 258 or lower.
- Missing Markdown localization debt remains 78 or lower.
- Issue #417 and the pull request are cross-linked.

197

LX-023: DOGFOOD Blocks And Capture Ratchet

docs/design/LX-023-dogfood-blocks-capture-ratchet.md

Status

Implemented.

Tracker

- Issue: #419
- Pull request: #420
- Branch: cycle/dogfood-blocks-capture-ratchet-792

Current Truth

Live counts on main at e33feaf1:

- aggregate Code Dojo debt: 792
- next aggregate Code Dojo target: 742 or lower
- ESLint findings: 384
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 hard-limit files
- DOGFOOD raw-string debt: 2,654
- missing Markdown localization debt: 78

Current offender evidence from npm run code-dojo:eslint:offenders:

- examples/docs/dogfood-blocks.ts: 37 ESLint findings
- examples/docs/capture-main.ts: 30 ESLint findings

The selected files can clear the required 50-count ratchet if fixed rather than waived. Both selected files also carry legacy DOGFOOD raw strings: dogfood-blocks.ts starts at 681, and capture-main.ts starts at 17. Touches to either file must reduce the touched-file DOGFOOD i18n count.

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 725
- next aggregate Code Dojo target: 675 or lower
- ESLint findings: 317
- examples/docs/dogfood-blocks.ts ESLint findings: 0
- examples/docs/capture-main.ts ESLint findings: 0
- examples/docs/dogfood-blocks.ts file/context baseline: 2,638 lines and 92,020 bytes
- examples/docs/capture-main.ts file/context baseline: 188 lines and 5,472 bytes
- DOGFOOD raw-string debt: 2,644
- dogfood-blocks raw-string debt: 679
- capture-main raw-string debt: 9
- missing Markdown localizations: 78

Problem

The Code Dojo baseline is enforceable, but the repository is not yet living at the standard. The remaining DOGFOOD block and capture surfaces still rely on unsafe casts, implicit string formatting, unchecked reads, and raw user-facing strings. They are also product-facing proof surfaces, so cleanup must preserve rendered behavior and lower-mode truth instead of applying mechanical lint silencing.

Scope

- Remove at least 50 counted ESLint findings from the selected DOGFOOD block and capture cluster.
- Prefer checked reads, local type guards, explicit String(...) formatting, and precise return types over assertions.
- Reduce touched raw DOGFOOD strings by removing non-user-facing literals, de-duplicating code tokens, or using catalog-backed lookups when product copy is actually touched.
- Update measured baselines only after validation proves the new counts.
- Update Code Dojo and changelog documentation with landed numbers.

Out Of Scope

- Broad DOGFOOD app redesign.
- New standard block contracts.
- Release tagging.
- Adding new exceptions for current violations.
- Cosmetic copy changes unrelated to standards cleanup.

Implementation Notes

The first pass should inspect lint output for the two selected files and group fixes by behavior:

- explicit formatting for numeric and unknown template values
- removal of unnecessary type arguments and unnecessary conversions
- safe external-data narrowing in capture code
- non-null assertion removal around arrays, map lookups, and metadata reads
- raw-string replacement only where the touched code already has localization context available or can be given one without widening architecture

If the selected files cannot clear the full target without risky churn, add the next highest small offender only after documenting the reason in this file.

Validation Plan

- npx eslint examples/docs/dogfood-blocks.ts examples/docs/capture-main.ts
- npm run code-dojo:changed
- npm run code-dojo:debt
- npm run dogfood:i18n:complete
- npm run dogfood:i18n:check
- npm run dogfood:i18n:debt
- relevant focused tests or DOGFOOD smoke checks selected from the diff
- npm run docs:inventory
- npm run lint
- git diff --check
- full pre-push gate

Implemented validation:

- npx eslint examples/docs/dogfood-blocks.ts examples/docs/capture-main.ts
- npm run typecheck:test
- npm run dogfood:i18n:debt
- npm run code-dojo:changed
- npm run test:run -- tests/cycles/DF-069/dogfood-block-registry.test.ts tests/cycles/DF-030/dogfood-docs-surface-block.test.ts tests/cycles/DF-070/dogfood-block-product-polish.test.ts tests/cycles/DF-071/dogfood-block-authored-surfaces.test.ts tests/cycles/DX-046/graphql-dogfood-navigation-fixture.test.ts scripts/smoke-dogfood.test.ts tests/cycles/WF-003/replace-smoke-examples-with-smoke-dogfood.test.ts
- npm run code-dojo:verify
- npm run lint
- npm run docs:inventory
- npm run dogfood:i18n:complete
- npm run dogfood:i18n:check

- `git diff --check`

Acceptance Criteria

- Aggregate Code Dojo debt is 725 or lower.
- ESLint baseline is 317 or lower.
- Touched DOGFOOD i18n debt is reduced without adding new missing translations.
- Touched files remain under their current file/context baselines.
- Issue #419, this design doc, and the pull request are cross-linked.

Workflow and delivery design records

This family records changes to validation, standards, review, release evidence and work tracking. It also captures the retirement of repository planning mirrors and the move towards tracker-owned scope. Its policies and numeric targets belong to their recorded context; current operating agreements and executable gates determine how new work is performed.

198

WF-001 — Adopt the Legends/Cycles Workflow

docs/design/WF-001-adopt-cycle-workflow.md

Legend: WF — Workflow and Delivery

Why this cycle exists

Bijou has been mixing multiple planning models:

- roadmap milestones
- strategy notes
- specs
- ad hoc backlog capture

That makes it harder for both humans and agents to tell:

- what is active
- what is backlog
- where design truth lives
- where tests tied to a cycle should go

This cycle establishes the workflow itself as a first-class repo artifact.

Human users

Primary human user

A maintainer or contributor who needs to:

- capture ideas
- pick the next piece of work
- understand active design intent
- see what tests belong to a cycle
- record debt and follow-on work honestly

Human hill

A human maintainer can open the repo and understand:

- what a legend is
- what a cycle is
- where backlog items live
- where active design docs live
- where cycle-specific tests live

without reverse-engineering tribal process from old roadmap prose.

Agent users

Primary agent user

A coding/design agent working inside the repo who needs to:

- discover the current planning model quickly
- know where to place new design docs
- know where to place cycle-owned tests
- know which invariants must be linked and protected
- know how to spawn backlog debt instead of silently drifting

Agent hill

An agent can enter the repo and, with minimal context, place design, tests, backlog fallout, and changelog updates in the right locations without reinforcing the old strategy/specs/roadmap drift.

Human playback

1. A maintainer has a new idea.
2. They create one file in docs/method/backlog/.
3. They decide to start the work.
4. They move that file into docs/design/.
5. They enrich the cycle doc with hills, playbacks, invariants, and test intent.
6. They write failing tests under tests/cycles/WF-001/ or the relevant cycle directory.
7. They land the work.
8. They record drift and spin out any debt into fresh backlog items.

Agent playback

1. An agent reads Bijou Workflow.
2. The agent finds the relevant legend doc.
3. The agent checks linked invariants.
4. The agent discovers the active cycle doc in docs/design/.
5. The agent writes or updates cycle-owned tests in tests/cycles/<cycle>/.
6. The agent lands the change and records retrospective fallout as backlog files.

Linked invariants

- Tests Are the Spec
- Focus Owns Input
- Visible Controls Are a Promise
- Graceful Lowering Preserves Meaning
- Shell Owns Shell Concerns
- Docs Are the Demo

Deliverables

- add docs/method/backlog/
- add docs/design/
- add docs/legends/
- add docs/invariants/
- add a canonical workflow doc
- add legend docs
- add invariant docs
- add cycle test support in tests/cycles/
- mark docs/specs/ as legacy/frozen for new work
- capture the current i18n package split as a backlog item instead of a fresh spec pair

Tests to write first

- a cycle-owned test proving the new directories/docs exist
- a cycle-owned test proving the workflow doc documents Legends/Cycles and tests/cycles/<cycle>/
- a cycle-owned test proving core legend and invariant docs exist
- a cycle-owned test proving the i18n package-split idea is now backlog work instead of new spec work

Risks

- partially adopting the workflow while continuing to add new work to docs/specs/
- leaving the old roadmap as if it were still the active source of truth
- documenting the workflow without updating the test harness to discover tests/cycles/

Retrospective

What landed

- the repo now has explicit workflow, legend, invariant, backlog, and design-cycle surfaces
- cycle-owned tests now have an official home in tests/cycles/
- the just-added i18n package-boundary planning work was rehomed as backlog instead of living as fresh spec files
- docs/specs/ is now explicitly marked as legacy for new work

Drift from ideal

- docs/ROADMAP.md still exists as a large legacy planning artifact
- older docs/strategy/* notes still need a deliberate migration story
- historical spec files remain and are not converted in this cycle

Debt spawned

- WF-002 — Migrate Legacy Planning Artifacts

199

WF-003 — Replace smoke:examples:* With smoke:dogfood

docs/design/WF-003-replace-smoke-examples-with-smoke-dogfood.md

Legend: WF — Workflow and Delivery

Sponsor human

A maintainer preparing 4.1.0 and needing the repo's release smoke contract to match the repo's stated DOGFOOD-first docs posture.

Sponsor agent

An agent that must answer, from repo artifacts alone:

- what release smoke actually runs before 4.1.0
- whether DOGFOOD or the broad examples/ tree is the public proving surface
- which scripts and workflows witness that answer

Hill

The public 4.1.0 smoke contract now belongs to DOGFOOD.

The workspace scripts, release-readiness gauntlet, CI workflow, release-dry-run workflow, publish workflow, and release docs all agree that DOGFOOD is the release-facing smoke surface.

Why this cycle exists

DF-025 made the repo posture sharp: DOGFOOD, not examples/, is the only human-facing docs surface for 4.1.0.

But the release contract was still lying:

- package.json still exposed smoke:examples:* as the main smoke family

- release:readiness still ran smoke:examples:all
- CI, release dry-runs, and publish verification still fanned out the old examples lanes
- repo docs still talked about moving smoke to DOGFOOD as future work

That meant the repo could teach one surface while releasing against another.

Human users

Primary human user

A maintainer or reviewer checking whether 4.1.0 is ready to release.

Human hill

A human can inspect the package scripts, workflows, and release docs and see one coherent answer: DOGFOOD is the release-facing smoke surface.

Agent users

Primary agent user

A coding or release agent that needs to run or explain pre-release validation without chat-memory folklore.

Agent hill

An agent can recover the DOGFOOD smoke contract from repo artifacts alone without guessing whether examples are still canonical.

Playback questions

1. Does the workspace expose smoke:dogfood, smoke:dogfood:landing, and smoke:dogfood:docs?
2. Does release:readiness now run smoke:dogfood -- --skip-build instead of smoke:examples:all?
3. Do CI, release dry-runs, and publish verification all fan out the DOGFOOD smoke lanes instead of the examples lanes?
4. Does the repo explicitly say that examples are secondary while DOGFOOD owns the release smoke contract?
5. Is WF-003 promoted into docs/design/, with the temporary v4.1.0/ blocker lane pruned again because no cycle-shaped blockers remain?

Accessibility / assistive reading posture

This cycle is terminal-output shaped. The smoke harness must prove the landing and docs shell through stable, plain-text witnesses after ANSI noise is stripped.

Localization / directionality posture

DOGFOOD is the localized, docs-bearing product surface. Release smoke should exercise that route rather than a grab bag of legacy examples.

Agent inspectability / explainability posture

The release smoke decision must be discoverable through:

- package.json
- scripts/smoke-dogfood-lib.ts
- scripts/release-readiness.ts
- .github/workflows/ci.yml
- .github/workflows/release-dry-run.yml
- .github/workflows/publish.yml
- docs/release.md
- docs/EXAMPLES.md
- cycle tests

Non-goals

- deleting every example-specific smoke helper immediately
- removing reference or migration examples that still carry isolated API value
- cutting the 4.1.0 tag
- changing the already-shaped 4.1.0 release scope

Evidence

Before this cycle

- package.json exposed smoke:examples:*
- scripts/release-readiness.ts still ran smoke:examples:all
- CI and release workflows still used smoke:examples:pipe, smoke:examples:static, and smoke:examples:interactive
- docs/release.md still described moving smoke to DOGFOOD as an open blocker

Local validation witness

These commands passed locally during this cycle:

```
npm test -- scripts/smoke-dogfood.test.ts scripts/release-readiness.test.ts
npm run smoke:dogfood -- --skip-build
```

The DOGFOOD smoke harness now drives the existing deterministic capture entrypoint in examples/docs/capture-main.ts through two scripted journeys:

- landing
- docs

Decision

4.1.0 release smoke belongs to DOGFOOD, not the broad examples/ tree.

Bijou should therefore:

1. expose DOGFOOD smoke as the public smoke family
2. run DOGFOOD smoke in local release readiness
3. fan DOGFOOD smoke out in CI and release workflows
4. describe example smoke as secondary/internal rather than release canonical

Implementation outline

1. Add a DOGFOOD smoke harness around examples/docs/capture-main.ts with stable landing and docs journeys.
2. Replace the public smoke:examples:* package scripts with smoke:dogfood, smoke:dogfood:landing, and smoke:dogfood:docs.
3. Switch release:readiness and the GitHub workflows to the new DOGFOOD smoke commands.
4. Update release docs, backlog/signpost docs, and examples posture docs so they stop describing DOGFOOD smoke as future work.
5. Promote WF-003 into docs/design/ and prune the now-empty docs/method/graveyard/legacy-backlog/v4.1.0/ lane again.

Tests to write first

- a unit test for the DOGFOOD smoke runner’s option parsing and launch plans
- a release-readiness test proving the gauntlet now runs smoke:dogfood
- a cycle test proving WF-003 moved into docs/design/ and the temporary blocker lane was pruned again

Retrospective

What landed

- the repo now exposes DOGFOOD-centered smoke scripts
- release readiness and GitHub workflows now run DOGFOOD smoke instead of examples smoke
- release/docs signposts no longer describe DOGFOOD smoke as unresolved
- the temporary v4.1.0/ blocker lane is pruned again because the cycle-shaped blockers are cleared

Drift from ideal

- the older example smoke harness still exists as internal tooling until a later cycle decides exactly which example-only proofs to keep or retire
- release execution itself is still outstanding

Debt spawned

- no new cycle-shaped blocker; the next work is operational release execution or post-4.1.0 follow-on backlog

200

WF-004 — Shape The Next Release

docs/design/WF-004-shape-the-next-release.md

Legend: WF — Workflow and Delivery

Sponsor human

A maintainer preparing the next public Bijou release.

Sponsor agent

An agent that must answer, from repo artifacts alone:

- what the next release is
- why it has that version number
- what is in it
- what is deferred
- what must be true before tagging

Hill

The next Bijou release is a deliberate, inspectable decision rather than an accretion of unreleased work.

Why this cycle exists

Since v4.0.0, Bijou has accumulated a large and mixed wave of work:

- runtime engine architecture and implementation slices
- humane shell behavior and shell introspection
- DOGFOOD growth into a real proving surface
- new localization packages and workflow tooling
- design-language doctrine and reusable review surfaces
- release, workflow, and repo-signpost hardening

That progress is real, but it created release ambiguity:

- [Unreleased] in CHANGELOG.md is not yet a clean v4.0.0..HEAD release boundary
- the repo had long-form release docs, but they still lived under docs/releases/next/
- the workspace contains new i18n packages, but the automated npm publish matrix still only covers the older five published packages
- RE-007 is important follow-on architecture work, but the repo had not yet answered whether it is a release blocker

This cycle exists to make those choices explicit.

Human users

Primary human user

A maintainer who needs to explain the release to users and execute it without hand-waving.

Human hill

A maintainer can point to repo files and answer:

- why the next release is 4.1.0
- what is included
- what is explicitly deferred
- what exact evidence makes the release ready

Agent users

Primary agent user

A coding or release agent preparing docs, release notes, workflow changes, or migration guidance.

Agent hill

An agent can recover the next release boundary, version, and readiness criteria from the repo itself without relying on chat memory or vague interpretation of [Unreleased].

Playback questions

1. After reading this cycle and the release docs, can a maintainer explain why the next release is 4.1.0 instead of a patch or a new major?
2. Can a maintainer see that RE-007 is explicitly deferred rather than silently assumed to be a blocker?
3. Can an agent find versioned long-form release docs in docs/releases/4.1.0/?
4. Can a release operator name the unresolved blockers before tagging?

Accessibility / assistive reading posture

This cycle is documentation-first. The important requirement is that the release shape remains legible in plain markdown and does not depend on tables, screenshots, or GitHub-specific UI to understand the decision.

Localization / directionality posture

The release includes localization work, so the docs must not pretend that the repo is still English-only by design. The prose remains English for now, but the release shape must explicitly acknowledge the new i18n packages and the publish-matrix gap around them.

Agent inspectability / explainability posture

Version choice, scope, deferrals, and readiness criteria must live in repo artifacts:

- this cycle doc
- docs/releases/4.1.0/
- docs/release.md
- docs/PLAN.md
- docs/BEARING.md

Non-goals

- actually tagging or publishing 4.1.0
- implementing RE-007
- fully rewriting changelog history before v4.0.0
- pretending the release is ready before its blocking gaps are closed

Evidence

Release boundary facts

- latest release tag: v4.0.0
- current workspace version: still 4.0.0
- current branch is main
- local HEAD can be synced to origin/main

Public-surface facts

- the post-v4.0.0 work is dominated by additive feat: style changes across runtime, shell, DOGFOOD, and localization
- there is no single clear public API break that obviously demands 5.0.0
- the current publish workflows only publish:
  - @flyingrobots/bijou
  - @flyingrobots/bijou-node
  - @flyingrobots/bijou-tui
  - @flyingrobots/bijou-tui-app
  - create-bijou-tui-app
- the workspace also now contains:
  - @flyingrobots/bijou-i18n
  - @flyingrobots/bijou-i18n-tools
  - @flyingrobots/bijou-i18n-tools-xlsx

Decision

Target version

The next release should be **4.1.0**, stable.

Why 4.1.0

- the release has a strong additive feature story across multiple public surfaces
- it is much larger than a patch release
- it does not present a clear enough breaking boundary to justify 5.0.0
- it is the first release after v4.0.0 that turns Bijou from “surface toolkit with a framed shell” into a more coherent application/runtime platform

In scope for 4.1.0

Headline themes

- runtime engine work through RE-006

- humane shell work through HT-004
- typed framed-shell and cleanup-capable command improvements through DX-001 and DX-002
- DOGFOOD completion through DF-019
- localization packages and localized shell/docs work through LX-008

Supporting but not headline

- release, workflow, and repo-signpost hardening
- System-Style JavaScript doctrine and derived invariants
- long-form release documentation

Explicitly deferred from 4.1.0

- RE-007
- WF-003
- DF-020
- DL-009
- the remaining localization follow-ons in LX-007 and LX-009

RE-007 decision

RE-007 is **not** a blocker for 4.1.0.

It remains an important architectural follow-on, but waiting for it would blur a coherent and already substantial release behind one more runtime slice. 4.1.0 should ship the runtime-engine story through RE-006 and then let RE-007 become follow-on work for the next line.

Ready when

4.1.0 is ready to tag only when all of the following are true:

1. CHANGELOG.md is reconciled to the actual v4.0.0..HEAD boundary and can become a clean 4.1.0 release section.
2. Long-form release docs exist under docs/releases/4.1.0/.
3. README.md has a concise “What’s New in v4.1.0” section.
4. The i18n publish-surface gap is resolved one of two ways:
   - preferred: expand the publish and dry-run matrices to include the new public i18n packages
   - fallback: narrow the public 4.1.0 release promise explicitly and stop presenting those packages as released artifacts
5. npm run release:preflight passes.
6. npm run release:readiness passes.
7. The GitHub **Release Dry Run** workflow passes.

8. The repo is on a clean main exactly synced with origin/main at the moment release execution begins.

Deliverables landed in this cycle

- a chosen target version: 4.1.0
- explicit deferral of RE-007
- versioned long-form release docs under docs/releases/4.1.0/
- repo signposts updated to carry the release-shaping decision

Tests to write first

- a cycle test that proves docs/design/WF-004-shape-the-next-release.md exists
- a cycle test that proves the versioned docs/releases/4.1.0/ release-doc directory exists
- a cycle test that proves RE-007 is explicitly treated as follow-on rather than silent blocker

Risks

- treating the i18n publish-matrix mismatch as a docs footnote instead of a real release blocker
- letting [Unreleased] remain semantically muddy even after shaping the release
- accidentally advertising RE-007 as if it were already part of the target release

Retrospective

What landed

- the next release now has a chosen public target: 4.1.0
- the repo has versioned long-form release docs for that target
- RE-007 is explicitly deferred instead of silently looming over the release
- release readiness is now a concrete checklist rather than a vibe

Drift from ideal

- CHANGELOG.md still needs its final 4.1.0 cut during actual release execution
- at the time this cycle closed, the i18n publish matrix was still a real blocker; WF-005 closes that gap in the release automation

Debt spawned

- WF-005 — Close 4.1.0 i18n Publish-Surface Gap
- WF-006 — Cut Clean 4.1.0 Release Boundary

201

WF-005 — Close 4.1.0 i18n Publish-Surface Gap

docs/design/WF-005-close-4-1-0-i18n-publish-surface-gap.md

Legend: WF — Workflow and Delivery

Sponsor human

A maintainer preparing the 4.1.0 release and needing the release automation, release docs, and public package story to agree.

Sponsor agent

An agent that must answer, from repo artifacts alone:

- which packages 4.1.0 will actually publish
- whether the i18n packages are release artifacts or just workspace code
- which workflows prove that answer

Hill

The 4.1.0 i18n package story is no longer ambiguous.

The publish workflow, release dry-run workflow, release docs, and workflow signposts all agree that the four i18n packages are part of the automated 4.1.0 release surface.

Why this cycle exists

WF-004 made 4.1.0 an explicit, inspectable release decision, but it also surfaced a real contradiction:

- the workspace and long-form release docs already treated the i18n packages as part of the 4.1.0 story
- the automated publish workflows still only packed and published the older five-package surface

That meant the repo could not honestly answer a simple question:

Are the i18n packages part of the public 4.1.0 release or not?

This cycle exists to close that gap in code, docs, and tests.

Human users

Primary human user

A maintainer executing or reviewing the 4.1.0 release.

Human hill

A maintainer can inspect the workflows and release docs and see the same nine-package public release surface everywhere.

Agent users

Primary agent user

A coding or release agent preparing release notes, running dry-runs, or answering publication questions.

Agent hill

An agent can recover the intended 4.1.0 npm publish surface from repo artifacts alone without guessing which workspace packages are “really” part of the release.

Playback questions

1. Do the publish and release-dry-run workflows both include the four i18n packages in their pack verification and npm publish matrices?
2. Can a maintainer point to repo evidence that the i18n packages pass local n pm pack --dry-run and npm publish --dry-run checks?
3. Do the release docs and signposts stop describing the i18n publish-surface decision as unresolved?
4. Is WF-005 no longer sitting in the v4.1.0/ blocker lane once the workflow and docs truth are aligned?

Accessibility / assistive reading posture

This cycle is documentation and workflow shaped. The decision must stay legible in plain markdown and raw YAML without depending on a GitHub UI view.

Localization / directionality posture

This cycle directly affects the localization story. The repo must not claim first-class i18n packages while quietly excluding them from the release path.

Agent inspectability / explainability posture

The public 4.1.0 package surface must be discoverable through:

- .github/workflows/publish.yml
- .github/workflows/release-dry-run.yml
- docs/release.md
- docs/releases/4.1.0/
- docs/PLAN.md
- docs/BEARING.md
- cycle tests

Non-goals

- cutting the 4.1.0 tag
- choosing the exact changelog wording for the final release cut
- changing package versions before release execution
- solving the remaining 4.1.0 blocker captured in WF-006

Evidence

Before this cycle

The release workflows only packed and published:

- @flyingrobots/bijou
- @flyingrobots/bijou-node
- @flyingrobots/bijou-tui
- @flyingrobots/bijou-tui-app
- create-bijou-tui-app

But the workspace also contained these public packages:

- @flyingrobots/bijou-i18n

- @flyingrobots/bijou-i18n-tools
- @flyingrobots/bijou-i18n-tools-node
- @flyingrobots/bijou-i18n-tools-xlsx

Local validation witness

These commands all passed locally during this cycle:

```
cd packages/bijou-i18n && npm pack --dry-run
cd packages/bijou-i18n-tools && npm pack --dry-run
cd packages/bijou-i18n-tools-node && npm pack --dry-run
cd packages/bijou-i18n-tools-xlsx && npm pack --dry-run

cd packages/bijou-i18n && npm publish --dry-run --access public --tag dry-run
cd packages/bijou-i18n-tools && npm publish --dry-run --access public --tag dry-run
cd packages/bijou-i18n-tools-node && npm publish --dry-run --access public --tag dry-run
cd packages/bijou-i18n-tools-xlsx && npm publish --dry-run --access public --tag dry-run
```

That includes the XLSX adapter package, which means the blocker was a workflow/docs mismatch rather than a packaging failure.

Decision

The four i18n packages are part of the planned public 4.1.0 release.

Bijou should therefore:

- pack them in the tag publish verification workflow
- dry-run publish them in the release dry-run workflow
- publish them from the real tag-driven npm workflow
- describe them as real 4.1.0 release artifacts in repo docs

Implementation outline

1. Expand .github/workflows/publish.yml pack verification and npm publish matrices to include all four i18n packages.
2. Expand .github/workflows/release-dry-run.yml pack verification and npm publish dry-run matrices to include the same packages.
3. Update release docs and signposts so they stop describing the i18n publish surface as unresolved.
4. Promote this backlog item into docs/design/ and narrow the docs/method/graveyard/legacy-backlog/v4.1.0/ lane to the remaining blocker.
5. Add tests that lock the workflow matrices and cycle placement to the chosen answer.

Tests to write first

- a cycle test proving the WF-005 design doc exists
- a cycle test proving both release workflows include the i18n packages

- a cycle test proving WF-005 no longer remains in the v4.1.0/ blocker lane

Retrospective

What landed

- the tag publish workflow now packs and publishes the i18n packages
- the release dry-run workflow now packs and dry-runs the i18n packages
- release docs and signposts now describe one consistent nine-package 4.1.0 release surface
- the v4.1.0/ blocker lane is narrower and more honest

Drift from ideal

- WF-006 still has to cut the final 4.1.0 release boundary in CHANGELOG.md and release-facing summaries
- local release readiness still relies on the GitHub dry-run workflow as the authoritative packaging gate

Debt spawned

- no new follow-on debt beyond the already-shaped WF-006

202

WF-006 — Cut Clean 4.1.0 Release Boundary

docs/design/WF-006-cut-clean-4-1-0-release-boundary.md

Legend: WF — Workflow and Delivery

Sponsor human

A maintainer preparing the 4.1.0 release and needing the short-form release surfaces to match the real v4.0.0..HEAD boundary.

Sponsor agent

An agent that must answer, from repo artifacts alone:

- what the short-form 4.1.0 release story is
- whether CHANGELOG.md is aligned to v4.0.0..HEAD
- what a front-door README summary should say about 4.1.0

Hill

The short-form 4.1.0 release surfaces now tell the same story as the long-form release docs.

README.md, CHANGELOG.md, and the long-form docs/releases/4.1.0/ docs all point at one coherent release boundary and one coherent release story.

Why this cycle exists

WF-004 chose 4.1.0, and WF-005 closed the i18n publish-surface gap.

But the short-form release surfaces were still drifting:

- the root README.md still led with a v4.0.0 “What’s New” section
- CHANGELOG.md still treated [Unreleased] as a vague bucket instead of an explicit v4.0.0..HEAD release slice
- the compare links still implied v3.1.0 was the previous release

- v4.0.0 itself never got a dedicated changelog section

That meant the repo had long-form release docs but still lacked an honest short-form release cut.

Human users

Primary human user

A maintainer or reviewer checking whether 4.1.0 is ready to explain and execute.

Human hill

A human can open the README and changelog and get an honest, concise picture of what 4.1.0 is and how it relates to v4.0.0.

Agent users

Primary agent user

A release or coding agent preparing notes, migration guidance, or the actual release execution.

Agent hill

An agent can reconstruct the short-form release story from repo artifacts alone without reverse-engineering commit history or chat memory.

Playback questions

1. Does README.md now have a concise What's New in v4.1.0 front-door summary?
2. Does CHANGELOG.md explicitly say the current [Unreleased] section is the planned 4.1.0 slice aligned to v4.0.0..HEAD?
3. Does CHANGELOG.md now include a dedicated 4.0.0 section dated 2026-03-22 and corrected compare links?
4. Is WF-006 promoted into docs/design/, with the temporary v4.1.0/ blocker lane pruned because no cycle-shaped blockers remain?

Accessibility / assistive reading posture

This cycle is text-first. The release boundary must stay legible in raw markdown without requiring a release UI, diff viewer, or GitHub compare page to understand it.

Localization / directionality posture

The short-form release story must stay honest about the new localization surface. It should not bury the i18n packages behind shell/runtime-only language.

Agent inspectability / explainability posture

The short-form release boundary must be inspectable through:

- README.md
- docs/CHANGELOG.md
- docs/releases/4.1.0/
- docs/release.md
- docs/BEARING.md
- docs/PLAN.md
- cycle tests

Non-goals

- bumping versions or cutting the 4.1.0 tag
- publishing packages
- rewriting all historical changelog prose before v4.0.0
- changing the already-shaped 4.1.0 scope

Evidence

Before this cycle

- latest release tag: v4.0.0
- tag date for v4.0.0: 2026-03-22
- README.md still led with What's New in v4.0.0
- CHANGELOG.md still used [Unreleased]: ...compare/v3.1.0...HEAD
- CHANGELOG.md had no dedicated 4.0.0 section

Boundary decision

The current [Unreleased] section is the planned 4.1.0 release slice. It should stay unreleased until actual release execution, but it must already be aligned to the real v4.0.0..HEAD boundary.

Decision

WF-006 closes the short-form release gap by doing four things:

1. make README.md summarize 4.1.0 instead of still fronting 4.0.0
2. explicitly mark [Unreleased] as the planned 4.1.0 slice in the changelog
3. restore a dedicated historical 4.0.0 changelog section and correct the compare links
4. prune the temporary v4.1.0/ blocker lane because there are no cycle-shaped release blockers left

Implementation outline

1. Replace the README's v4.0.0 front-door summary with a concise v4.1.0 summary and links to the short-form and long-form release docs.
2. Mark the current [Unreleased] changelog section as the planned 4.1.0 slice.
3. Add a compact historical 4.0.0 changelog section dated 2026-03-22.
4. Correct the changelog compare links so [Unreleased] means v4.0.0...HEAD.
5. Promote this cycle into docs/design/ and prune the now-empty docs/method/graveyard/legacy-backlog/v4.1.0/ lane.
6. Add cycle tests that lock the release boundary and lane-pruning behavior.

Tests to write first

- a cycle test proving WF-006 moved into docs/design/
- a cycle test proving the temporary v4.1.0/ blocker lane is gone
- a cycle test proving README.md and CHANGELOG.md carry the new short-form 4.1.0 boundary

Retrospective

What landed

- the README now fronts the planned 4.1.0 release instead of 4.0.0
- the changelog now states that [Unreleased] is the planned 4.1.0 slice aligned to v4.0.0..HEAD

- v4.0.0 now has a dedicated historical changelog section and correct compare-link placement
- the temporary v4.1.0/ blocker lane is gone because the cycle-shaped release backlog is cleared

Drift from ideal

- the remaining 4.1.0 work is now operational release execution, not another design/backlog cycle
- the changelog will still need the final [Unreleased] -> [4.1.0] - YYYY-MM-DD rename during actual release execution

Debt spawned

- no new cycle-shaped debt; the next work should be release execution or post-4.1.0 backlog

203

WF-009 - Keep Release PRs Under Automated Review File Limits

docs/design/WF-009-keep-release-prs-under-automated-review-file-limits.md

Legend: WF - Workflow and Delivery

Sponsor human

A maintainer preparing release-boundary work that may span many packages, docs, examples, tests, and backlog notes.

Sponsor agent

An agent executing the METHOD loop through pull, implementation, commit, push, PR, and merge without producing a PR too large for automated review.

Hill

Release-boundary work is sliced into reviewer-sized PRs before the final release sync, so automated review, CI, and human inspection all see bounded changes.

Why this cycle exists

CodeRabbit skipped PR #65 because the diff exceeded its 150-file cap. PR #69 also reached the danger zone: the local pre-push hook warned about 23 commits, and CodeRabbit selected 149 files for processing.

The issue is not that large releases are bad. The issue is that a large release must be composed from smaller reviewable precursor PRs instead of arriving as one oversized branch.

Playback questions

1. Can an operator identify when a release branch must be split before PR?
2. Can an agent choose the next release-boundary slice without relying on chat memory?
3. Can a reviewer see whether a PR is intentionally a precursor slice or a final release sync?
4. Can the final release PR stay mostly about version, changelog, long-form release docs, and release validation rather than hiding unrelated features?

Accessibility / assistive reading posture

This is workflow documentation. It must remain useful in plain markdown, terminal readers, MCP document summaries, and copied text. The policy uses short numbered rules instead of relying on diagrams.

Localization / directionality posture

No localized runtime behavior changes. The guidance should avoid idioms that make the release process harder to translate later.

Agent inspectability / explainability posture

The slicing threshold and required response live in repo docs, not chat memory:

- docs/WORKFLOW.md
- docs/release.md
- this cycle doc

Non-goals

- changing branch protection
- disabling CodeRabbit
- forcing every tiny fix into its own PR
- tagging or publishing v6.0.0

Decision

Bijou release work should use precursor PRs whenever a branch is likely to exceed automated review limits.

The practical thresholds are:

- hard stop: more than 140 changed files before opening a PR
- caution: more than 10 commits on one branch
- caution: unrelated legends or cycle families mixed in one branch
- caution: generated snapshots or docs mixed with unrelated runtime changes

When a threshold is hit, split by cycle or product surface:

- runtime engine
- TUI shell and interaction
- core components
- DOGFOOD and docs proof
- backlog/release-lane bookkeeping
- release version/changelog/tag prep

The final release PR should be boring. It should primarily contain version bump, changelog, release docs, release-readiness proof, and any final metadata adjustments that cannot land earlier.

Implementation outline

1. Add PR sizing discipline to the short workflow guide.
2. Add release-boundary slicing guidance to the release process.
3. Update stale release facts while editing the release process so the docs point at v6.0.0 as the next shaping lane.
4. Keep the v6 backlog lane linked to this pulled cycle.

Tests / validation

- npm run docs:inventory
- git diff --check
- npm run lint

Retrospective

Closed by documenting PR slicing thresholds in the workflow guide, adding release-boundary sizing rules to the release process, and updating stale release posture from v5.0.0 to v6.0.0.

204

WF-122 - Cross-Platform Contributor Path

docs/design/WF-122-cross-platform-contributor-path.md

Linked issues: #122, #135, #136

Linked legend: WF - Workflow and Delivery

Sponsor Human

A TypeScript contributor on macOS, Linux, or Windows who wants the core Bijou build and test path to work without first installing a Unix shell.

Sponsor Agent

A workflow verification agent that can inspect CI, package scripts, and local commands to prove which parts of the repo are portable and which are intentionally platform-hosted.

Hill

Bijou's contributor-facing scripts are Node-first, and CI proves the core build and focused unit-test path on at least one non-Linux platform.

Scope

- Replace avoidable Bash and POSIX shell usage in root contributor scripts.
- Add a non-Linux CI lane for build, typecheck, and focused unit tests.
- Document which workflows remain Linux/Bash-hosted because they are release, benchmark, or terminal-capture lanes rather than the first contributor path.

Playback Questions

1. Can a Windows contributor run the core root scripts without WSL-only shell behavior?
2. Does CI prove at least one non-Linux build/typecheck/unit lane?
3. Can a reviewer tell which remaining Bash paths are intentional workflow infrastructure?

Accessibility / Assistive Reading Posture

The docs must name the platform support posture plainly in README and CONTRIBUTING without hiding it in workflow YAML.

Localization / Directionality Posture

No runtime localization changes.

Agent Inspectability / Explainability Posture

The proof lives in package.json, .github/workflows/ci.yml, script source, and workflow:shell:preflight output.

Linked Invariants

- Contributor-facing commands should be portable when Node can provide the same behavior.
- Platform-specific shell remains acceptable for release and benchmark lanes when the host choice is explicit.

Decisions

- npm run clean becomes node scripts/clean.mjs and removes workspace dist/ directories plus workspace .tsbuildinfo files without shell globs.
- npm run version becomes node scripts/version.mjs, preserving the existing workspace version bump and internal dependency pinning behavior.
- CI keeps the full supported Node 20/22 lane on Ubuntu and adds Node 22 focused unit coverage on macOS and Windows.
- The repository development floor is ^20.19.0 || >=22.12.0 after the Vitest 4.1 toolchain moved through Vite 8/Rolldown APIs that cannot start on Node 18 or earlier 20/22 minors.

- Existing release, publish, benchmark, retry, and hook shell scripts remain explicit shell-owned infrastructure. They are not the Day 0 contributor path.

Verification

- node --check scripts/clean.mjs
- node --check scripts/version.mjs
- npm run workflow:shell:preflight
- focused CI-equivalent unit tests listed in .github/workflows/ci.yml

Tests To Write First

- Script syntax checks for converted Node scripts.
- Workflow shell preflight after changing CI.

Retrospective

See WF-122 day zero audit fixes.

205

WF-123 - Issues-First Method Intake

docs/design/WF-123-issues-first-method-intake.md

Linked issues: #123, #124, #153

Linked legend: WF - Workflow and Delivery

Sponsor Human

A Bijou maintainer or contributor who needs to propose, triage, and follow work without learning the historical filesystem backlog first.

Sponsor Agent

A Method bookkeeping agent that must keep GitHub Issues, labels, design docs, witness evidence, retros, changelog notes, and PR closure references aligned.

Hill

Bijou uses GitHub Issues as the live work tracker while keeping repository docs as the durable evidence ledger for designs, witnesses, retros, and shipped history.

Why This Cycle Exists

The deep-dive audit found that the old backlog-first docs made Day 0 contributors and agents read historical planning files before they could tell where active work lives. The Method direction has changed: raw intake belongs in GitHub Issues, and labels carry live tracker state.

Bijou should follow that model without deleting historical evidence.

Playback Questions

1. Can a contributor open one GitHub issue and understand which labels describe lane, type, priority, and legend?
2. Can a maintainer see that docs/method/backlog/, retro/, and graveyard/ are evidence and archive surfaces, not the first intake path?
3. Can an agent distinguish live issue state from historical repo files during context recovery?
4. Can a PR still point to design, witness, retro, changelog, and issue evidence after implementation work lands?

Accessibility / Assistive Reading Posture

The guidance must remain plain Markdown with short tables and direct issue template references. The important reader task is deciding where to open or triage work, not parsing a process essay.

Localization / Directionality Posture

This is repository workflow documentation. No runtime localization behavior changes.

Agent Inspectability / Explainability Posture

The live state is inspectable through GitHub issue labels. Repo files remain inspectable evidence:

- docs/design/ for shaped cycle intent
- PR descriptions and comments for witness output
- docs/method/retro/ for landed or otherwise closed lineage
- docs/CHANGELOG.md for shipped behavior

Linked Invariants

- Live tracker state must not fork between local backlog files and GitHub Issues.
- Historical evidence must not be deleted merely because it is no longer live work.
- Labels are tracker metadata; issue body and PR evidence carry the narrative.

Implementation Outline

1. Replace contributor intake guidance with GitHub Issues-first wording.
2. Document lane labels and the work-in-progress state label.
3. Reframe docs/method/backlog/ as a compatibility/evidence ledger unless a release lane explicitly says otherwise.
4. Clarify retro and graveyard search posture.
5. Keep issue forms aligned with the Method card shape.

Tests To Write First

This cycle is documentation and template work. Verification is:

- npm run docs:inventory
- git diff --check
- issue template YAML parsing
- focused docs preview tests when front-door docs change

Retrospective

See WF-122 day zero audit fixes.

206

WF-125 Draft-first cycle start workflow

docs/design/WF-125-draft-first-cycle-start-workflow.md

```
title: WF-125 Draft-first cycle start workflow
legend: WF
lane: design
priority: medium
keywords:
  - workflow
  - method
  - draft-pr
  - design-thinking
```

Framing

Issue #279 changes the Bijou cycle loop from “open the pull request at ship time” to “open a draft pull request at cycle start.” The intent is to make issue, branch, design doc, and PR state visible before implementation expands, while keeping the final ready-for-review and merge gates strict.

The work follows a Design Thinking loop:

- observe that cycle work can stay invisible until late implementation
- frame draft PRs as early coordination artifacts rather than merge-ready review artifacts
- prototype the new loop in Method, Workflow, Contributor, Agent, and issue template docs
- prove the policy with a workflow regression that reads those docs

Sponsored Users

- Maintainers who need early visibility into the branch and linked artifacts.
- Contributors who need one explicit start-of-cycle checklist.
- Agents that need a deterministic order for sync, branch, shape, push, draft PR, and work-in-progress labeling.
- Reviewers who should see draft context early without treating it as ready.

Hills

1. A contributor can start a cycle by syncing the merge target, creating a cyc le/<cycle_name> branch, shaping the issue and design doc, committing and pushing that shape, opening a draft PR, and applying work-in-progress.
2. A reviewer can distinguish an early draft coordination PR from a ready PR that has passed implementation, validation, and self-review.
3. A maintainer can run one workflow regression that prevents the docs and issue template from drifting back to the old “PR only at ship” loop.

Playback Questions

- Does every workflow surface say to run git fetch and sync the merge target before branching?
- Does the cycle branch start from the synced target instead of stale local state?
- Does the shaping commit include the GitHub Issue and design doc before implementation work begins?
- Does a draft PR to main exist before implementation starts?
- Does the issue carry work-in-progress while the branch or draft PR is actively carrying it?
- Does the draft PR move to ready only after implementation, validation, and self-review?

Workflow Contract

```
git fetch
sync merge target branch
checkout cycle/<cycle_name>
write or update GitHub Issue
write docs/design/<cycle>.md
stage + commit + push shaping artifact
open draft PR to main
link issue + design doc + draft PR
apply work-in-progress to issue
do RED/GREEN cycle work
validate + self-review
mark PR ready for review
```

Implementation Notes

- AGENTS.md owns the short agent recovery protocol.
- docs/METHOD.md owns the doctrine and cycle-state diagram.
- docs/WORKFLOW.md owns the operator checklist.
- CONTRIBUTING.md owns contributor-facing expectations.
- .github/ISSUE_TEMPLATE/work-item.yml owns the issue-form acceptance text.
- tests/cycles/WF-001/workflow-adoption.test.ts freezes the contract.

Tests

- tests/cycles/WF-001/workflow-adoption.test.ts adds a regression that reads AGENTS.md, CONTRIBUTING.md, docs/METHOD.md, docs/WORKFLOW.md, and the Method work-item issue template.
- The RED pass failed because docs/METHOD.md did not contain the new git fetch and draft-PR cycle-start language.
- The GREEN pass updates the docs and normalizes Markdown whitespace in the regression so wrapped prose can still carry exact policy sentences.

Closeout

The cycle is landed when:

- the workflow docs require merge-target sync before branching
- the workflow docs require a draft PR to main at cycle start
- the issue template requires issue, design doc, and draft PR links
- the draft PR is marked ready only after implementation, validation, and self-review
- the WF-001 regression, docs inventory, lint, and whitespace checks pass
- the pull request closes #279

207

WF-127 V7 Issue-Complete Tracker Sync

docs/design/WF-127-v7-issue-complete-tracker-sync.md

```
title: WF-127 V7 issue-complete tracker sync
legend: WF
lane: bad-code
priority: medium
issue: https://github.com/flyingrobots/bijou/issues/285
keywords:
  - tracker
  - roadmap
  - bearing
  - v7
  - release-readiness
```

Framing

GitHub is the live tracker for Bijou work. After the V7 Product Truth closeout, the post-merge Codex review follow-up, and this tracker-sync cleanup, the v7.0.0 milestone should be issue-complete: zero open milestone items and twenty-seven closed milestone items.

The repository mirrors did not fully follow that state. BEARING.md, ROADMAP.md, and the WF-126 regression still describe the pre-merge V7 closeout with #245, #246, and #281 as open work. That creates the exact tracker/docs drift the Method calls out as bad code.

Sponsored Users

- A release maintainer needs BEARING and ROADMAP to match GitHub before running release-readiness validation.
- A review agent needs the tracker-sync regression to encode the issue-complete state, not the old closeout state.
- A contributor scanning the repo needs to see release-readiness as the next action instead of chasing already-closed V7 issues.

Hill

A maintainer preparing V7 can read BEARING, ROADMAP, and the WF-126 regression and see the same truth as the GitHub milestone after this PR

closes #285: V7 has no open milestone items, has twenty-seven closed milestone items, and should move to release-readiness validation.

Current Truth

- GitHub milestone v7.0.0 before this cleanup issue: 1 open / 26 closed, because #285 is now the only open V7 item.
- GitHub milestone v7.0.0 after this cleanup issue closes: 0 open / 27 closed. The milestone count includes milestone PRs as well as issues.
- docs/BEARING.md still says the V7 current open count is three.
- docs/ROADMAP.md still reports V7 as 3 open / 20 closed.
- tests/cycles/WF-126/v7-closeout-tracker-sync.test.ts still expects #245, #246, and #281 in the open-work area.

Product Shape

BEARING

```
Active Gravity
  v7.0.0 is issue-complete
  next action: release-readiness validation
```

ROADMAP

```
| v7.0.0 | 0 open | 27 closed | Issue-complete V7 Product Truth |

Open Work
  No open v7 tracker issues remain as of 2026-06-03.
```

Regression

```
WF-126
  expect roadmap v7 row to contain | 0 | 27 |
  expect closed issue lineage to include #245, #246, #281, #283, #285
  expect no open-work rows for #245/#246/#281
```

Runtime And API Contract

This is a documentation and workflow-proof cycle. It does not change runtime APIs, package exports, or user-facing DOGFOOD behavior.

Lower Modes

No lower-mode product output changes are intended.

Tests To Write First

- Update tests/cycles/WF-126/v7-closeout-tracker-sync.test.ts so it fails against the current stale BEARING and ROADMAP state.
- Keep the test scoped to local repo evidence. The GitHub query remains the operator's source of truth; the regression prevents the checked-in mirror from drifting again.

Acceptance Criteria

- Issue #285 is closed by the PR.
- BEARING no longer lists #245, #246, or #281 as open V7 work.
- ROADMAP reports v7.0.0 as 0 open and 27 closed.
- ROADMAP lists #245, #246, #281, #283, and #285 in V7 completed lineage.
- WF-126 passes.
- CHANGELOG records the bad-code tracker sync fix.

Risks And Guardrails

- Do not cut, tag, or publish V7 in this cycle.
- Do not implement the broader tracker-sync sentinel from #268 here.
- Do not move Beyond issues into V7 as part of this cleanup.
- Do not make BEARING or ROADMAP the source of truth. They mirror GitHub.

208

WF-128 Safe GitHub Comments And Milestone Item Mirrors

docs/design/WF-128-safe-gh-comments-and-milestone-items.md

```
title: WF-128 Safe GitHub Comments And Milestone Item Mirrors
legend: WF
lane: bad-code
priority: medium
issues:
  - https://github.com/flyingrobots/bijou/issues/287
  - https://github.com/flyingrobots/bijou/issues/288
keywords:
  - github
  - comments
  - roadmap
  - milestone
  - workflow
```

Framing

Two small workflow defects came out of the V7 closeout:

- review comments can be corrupted when Markdown literals are passed through a shell-interpolated gh pr comment --body "..." argument
- roadmap release snapshots can be misread when GitHub milestone totals include pull requests as milestone items while nearby prose says "issues"

Both are process bugs. They do not change runtime behavior, but they do affect the reliability of review evidence and release bookkeeping.

Sponsored Users

- A maintainer posting self-review, Code Lawyer, or activity-summary comments needs a command pattern that preserves Markdown literally.
- A release maintainer reading ROADMAP.md needs to know whether the snapshot counts GitHub milestone items or issue-only rows.
- A review agent needs executable repo tests that catch unsafe comment examples and ambiguous milestone-count language before the pattern spreads.

Hill

A maintainer can post Markdown-heavy GitHub comments and reconcile release snapshot rows without relying on memory: docs/WORKFLOW.md, docs/METHOD.md, and docs/ROADMAP.md state the safe conventions, and WF-128 tests prove those conventions stay visible.

Current Truth

- Issue #287 records that a PR #284 self-review comment was polluted when Markdown backticks inside a double-quoted --body argument were evaluated by the shell.
- Issue #288 records that V7 tracker-sync work had to count milestone PRs #278 and #280 to match GitHub milestone totals.
- docs/WORKFLOW.md and docs/METHOD.md do not currently name the --body-file / literal-heredoc path for review comments.
- docs/ROADMAP.md says GitHub milestones and issue labels are live tracker state, but the release snapshot columns are named only Open and Closed, which makes item counts look like issue-only counts.

Product Shape

Safe Comment Posting

```
Workflow review step
  self-review / Code Lawyer / activity summary
    |
    v
  gh pr comment "$PR_NUMBER" --body-file - <<'EOF'
  Markdown stays literal:
  - `commands`
  - $()
  - tables
  EOF
```

The rule is intentionally boring: if a GitHub comment body contains Markdown, commands, backticks, $(), tables, or copied review output, post it through --body-file with a quoted heredoc or a real file.

Milestone Item Mirrors

```
GitHub milestone page / API
  open milestone items   = issues + pull requests assigned to milestone
  closed milestone items = issues + pull requests assigned to milestone

ROADMAP release snapshot
  Open Items / Closed Items
  Completed lineage includes milestone PR rows when PRs contribute to totals.
```

Runtime And API Contract

This cycle changes documentation and cycle-test proof only. It does not change package exports, runtime APIs, schemas, localization strings, or DOGFOOD rendered output.

Lower Modes

No user-facing TUI surface changes are included.

- Static mode: documentation prose becomes clearer.
- Pipe mode: not applicable because no command output contract changes.
- Accessible mode: not applicable because no rendered app surface changes.

Tests To Write First

- Add tests/cycles/WF-128/safe-comments-and-milestone-items.test.ts.
- RED state:
  - workflow docs do not contain the safe --body-file - <<'EOF' pattern
  - roadmap release snapshot does not say Open Items / Closed Items
  - roadmap maintenance guidance does not warn against comparing item totals to issue-only lists without accounting for milestone PRs
- GREEN state:
  - docs expose the safe comment posting rule
  - roadmap terminology distinguishes milestone item counts from issue-only counts
  - changelog records the bad-code cleanup

Acceptance Criteria

- Issue #287 is closed by the PR.
- Issue #288 is closed by the PR.
- docs/WORKFLOW.md and docs/METHOD.md prefer --body-file for Markdown-heavy GitHub comments.
- AGENTS.md reminds agents not to inline shell-sensitive review comments through --body "...".
- docs/ROADMAP.md names release snapshot counts as milestone items and states that milestone items include issues plus pull requests.
- WF-128 passes.
- docs/CHANGELOG.md records the cleanup.

Risks And Guardrails

- Do not introduce a new GitHub-comment wrapper script unless the docs-only pattern proves insufficient.
- Do not solve the broader tracker-sync sentinel from #268 here.
- Do not change release milestones or move any issue between lanes.
- Do not cut V7 in this cycle.

209

WF-129 Path-Gate DOGFOOD CI

docs/design/WF-129-path-gate-dogfood-ci.md

```
title: WF-129 Path-Gate DOGFOOD CI
legend: WF
lane: bad-code
priority: medium
issue: https://github.com/flyingrobots/bijou/issues/293
keywords:
  - ci
  - dogfood
  - path-gates
  - iteration-time
  - workflow
```

Framing

The CI loop is intentionally conservative, but ordinary workflow/documentation PRs currently run DOGFOOD-heavy gates even when they do not touch DOGFOOD, runtime components, localization, or CI scripts. That makes rapid-fire cycles pay duplicate install/build/smoke costs before review can close.

This cycle adds a CI change classifier so DOGFOOD-specific checks run when they are relevant and skip when a PR is unrelated to DOGFOOD. Pushes to main and tag workflows remain full-strength.

Sponsored Users

- A maintainer iterating on workflow documentation needs PR CI to avoid DOGFOOD smoke work when the branch cannot affect DOGFOOD.
- A release maintainer needs main pushes and tags to continue running the full DOGFOOD proof path.
- A review agent needs the path-gating contract to be visible in the workflow file and covered by a regression test.

Hill

A contributor can open a PR that only changes non-DOGFOOD docs or workflow bookkeeping and see DOGFOOD-heavy CI lanes skipped, while any PR touching

DOGFOOD, packages, localization, DOGFOOD scripts/tests, or CI workflows still runs DOGFOOD proof.

Current Truth

- scripts/hooks/pre-push runs DOGFOOD i18n, test typecheck, full Vitest, and interactive example smoke before every push.
- .github/workflows/ci.yml runs DOGFOOD coverage and i18n policy inside the Ubuntu Node 22 test job for every PR.
- .github/workflows/ci.yml runs separate DOGFOOD landing/docs smoke jobs for every PR, with their own checkout, install, and build.
- The latest main CI run after PR #291 had a test (22) critical path just under two minutes, plus separate DOGFOOD smoke jobs.

Product Shape

CI Classifier

```
CI
  changes
    dogfood = true on main pushes and tags
    dogfood = true on PRs touching:
      .github/workflows/**
      package.json
      package-lock.json
      packages/**
      examples/docs/**
      scripts/dogfood-*
      scripts/smoke-dogfood*
      tests/cycles/DF-*
      tests/cycles/*dogfood*
      docs/DOGFOOD.md
      docs/design/DF-*
      docs/legends/DF-*
      docs/method/legends/DF-*
```

Gated DOGFOOD Proof

```
test (22)
  always: build, lint, npm test
  when dogfood-relevant: DOGFOOD coverage + DOGFOOD i18n policy

Smoke DOGFOOD (landing/docs)
  when dogfood-relevant: run checkout, install, build, and smoke command
  otherwise: keep check names present and skip expensive smoke steps
```

Runtime And API Contract

This cycle changes CI workflow behavior only. It does not change package APIs, runtime behavior, DOGFOOD rendering, localization catalogs, or release workflows.

Lower Modes

No TUI output changes are included.

- Static mode: not applicable.
- Pipe mode: not applicable.
- Accessible mode: not applicable.

Tests To Write First

- Add tests/cycles/WF-129/path-gate-dogfood-ci.test.ts.
- RED state:
  - CI lacks a changes job exposing dogfood output
  - DOGFOOD coverage/i18n policy steps are not guarded by the classifier
  - DOGFOOD smoke job is not guarded by the classifier
  - main/tag pushes are not documented as full DOGFOOD proof
- GREEN state:
  - workflow text includes the classifier and relevant trigger paths
  - workflow text gates DOGFOOD-heavy PR work on needs.changes.outputs.dogfood
  - changelog records the CI iteration-time cleanup

Acceptance Criteria

- Issue #293 is closed by the PR.
- PR CI has an explicit changes job.
- DOGFOOD coverage/i18n policy steps run only when the classifier says DOGFOOD proof is needed.
- DOGFOOD landing/docs smoke commands run only when the classifier says DOGFOOD proof is needed, while the check names remain present.
- Pushes to main and tags still set the DOGFOOD classifier to true.
- WF-129 passes.
- docs/CHANGELOG.md records the change.

Risks And Guardrails

- Do not path-gate release dry-run or publish workflows in this cycle.
- Do not skip full npm test in the core Ubuntu matrix yet.
- Do not weaken DOGFOOD proof when packages/**, examples/docs/**, or DOGFOOD-specific scripts/tests change.
- Keep the classifier readable enough to audit in review.

210

WF-130 Roadmap Goalpost Policy

docs/design/WF-130-roadmap-goalpost-policy.md

Linked Legend

WF - Workflow and Delivery

Decision Summary

Adopt a Bijou roadmap planning system that maps GitHub milestone items into versioned releases, candidate goalposts, umbrella issues, user-story issues, slice budgets, release gates, and proof requirements.

Sponsor Human

A maintainer wants to see the current release path and backlog shape without reconstructing it from memory, stale Markdown, and raw GitHub lists, so that release planning can stay issue-backed and inspectable.

Sponsor Agent

An agent needs a documented roadmap hierarchy and issue-template fields so it can classify work as ordinary issues, goalpost umbrellas, or user stories without inventing planning structure in each turn.

Hill

By the end of this cycle, Bijou has a release-packet policy, a GitHub-backed roadmap snapshot, issue-template fields for roadmap roles and slice budgets, and tests that prevent the docs from drifting back to draft-first cycle language.

Current Truth

Verified against origin/main on 2026-06-06 after PR #304 merged.

Current anchors:

- GitHub milestone v6.0.0: 0 open, 30 closed milestone items.
- GitHub milestone v7.0.0: 0 open, 27 closed milestone items.
- GitHub milestone Beyond: 29 open, 1 closed milestone item.
- Open unmilestoned issues: #249 and #306.
- docs/method/releases/README.md is a stub.
- docs/METHOD.md, docs/WORKFLOW.md, CONTRIBUTING.md, AGENTS.md, and WF-001 tests still describe draft-first cycle PRs.

Problem

Bijou has strong issue intake and cycle docs, but it lacks the intermediate roadmap structure that explains how individual issues become release-scale goalposts. The docs also contain stale draft-first language that conflicts with current repo instructions.

Scope

This cycle includes:

- defining the release-packet contract under docs/method/releases/
- refreshing docs/ROADMAP.md from live GitHub milestone and issue data
- grouping the open Beyond backlog into candidate goalposts
- adding roadmap role, linkage, slice budget, and release-gate fields to the Method work-item issue form
- aligning operator docs to non-draft PRs at cycle start
- adding tests that prove the policy and the non-draft alignment

Non-Goals

This cycle does not:

- create new release milestones
- move GitHub issues between milestones
- create umbrella issues for the candidate goalposts
- close or rewrite the existing Beyond issues
- implement any product features from the candidate goalposts

User Experience / Product Shape

This is process and documentation work. There is no TUI surface change.

The reviewer-facing shape is:

```
docs/ROADMAP.md
  Release Snapshot
  v6.0.0
  v7.0.0
  Beyond
    Candidate Goalposts From Open GitHub Issues
    Open Beyond Issues
    Open Unmilestoned Triage
```

Lower Modes

Static Mode

Markdown documents remain readable as plain text.

Pipe Mode

The relevant proof is deterministic command output from tests and GitHub CLI queries.

Accessible Mode

No interactive surface changes. The issue form uses explicit labels and field names that screen-reader users can navigate in GitHub.

Runtime / API Contract

No runtime API changes.

Process contract changes:

- docs/method/releases/README.md defines versioned releases, goalposts, umbrella issues, user-story issues, slices, release gates, and proof.
- .github/ISSUE_TEMPLATE/work-item.yml exposes roadmap role, roadmap linkage, slice budget, and release-gate fields.
- Cycle-start docs require a non-draft PR.

Data / State Model

GitHub remains the live tracker. docs/ROADMAP.md is a mirror of:

- GitHub milestone counts
- GitHub issue labels

- GitHub issue milestone assignment
- GitHub pull request milestone assignment when milestone counts include PRs

Markdown release packets explain the structure, but GitHub owns the current state.

Accessibility Posture

No new visible product controls are added. Markdown tables use explicit column headers. The issue-template additions use field labels that state their purpose without relying on placeholder-only instructions.

Localization / Directionality Posture

No app-visible localized strings are added. GitHub issue-template prose is not part of the DOGFOOD runtime localization catalog.

Agent Inspectability / Explainability Posture

Agents can inspect the policy through stable files:

- docs/method/releases/README.md
- docs/ROADMAP.md
- .github/ISSUE_TEMPLATE/work-item.yml
- tests/cycles/WF-130/roadmap-goalpost-policy.test.ts

The roadmap records the exact sync date and GitHub issue groupings.

Linked Invariants

- Tests Are the Spec
- Runtime Truth Wins
- Docs Are the Demo
- Work Is Issue-Backed
- Release Claims Need Proof

Implementation Outline

1. Add a failing WF-130 regression for release packets, roadmap counts, non-draft cycle docs, and issue-template fields.
2. Write the release-packet policy.
3. Refresh the roadmap from live GitHub milestones and issues.

4. Align METHOD, WORKFLOW, AGENTS, CONTRIBUTING, and WF-001 tests to non-draft PRs.
5. Extend the work-item issue template.
6. Record the changelog entry and run validation.

Tests To Write First

- npm test -- --run tests/cycles/WF-130/roadmap-goalpost-policy.test.ts should fail before the policy and docs exist.
- The same focused command should pass after the policy is implemented.
- WF-001 workflow tests should continue passing with the non-draft wording.

Validation Plan

```
npm test -- --run tests/cycles/WF-130/roadmap-goalpost-policy.test.ts
npm test -- --run tests/cycles/WF-001/workflow-adoption.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.test.ts
npm run docs:inventory
npm run typecheck:test
npm run lint
git diff --check
```

Closeout Notes

Completed on cycle/roadmap-goalpost-policy.

Proof:

- RED: npm test -- --run tests/cycles/WF-130/roadmap-goalpost-policy.test.ts failed on the release-packet stub, stale roadmap counts, draft-first cycle docs, and missing issue-template roadmap fields.
- GREEN: WF-130 passes after the release policy, roadmap refresh, issue form, label model, and non-draft workflow alignment landed.
- WF-001 was updated and rerun with WF-130 so the older workflow invariant agrees with the new non-draft cycle policy.
- GitHub labels roadmap, goalpost, and user-story were created for queryable roadmap role tracking.

Validation:

```
npm test -- --run tests/cycles/WF-001/workflow-adoption.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.test.ts
npm run docs:inventory
npm run typecheck:test
npm run lint
git diff --check
```

211

WF-131 Roadmap Forward-Looking Refresh

docs/design/WF-131-roadmap-current-reality-refresh.md

Linked Legend

WF - Workflow and Delivery

Decision Summary

Refresh docs/ROADMAP.md and docs/BEARING.md so they describe the forward post-v7.0.0 release horizon instead of implying the complete v6.0.0 and v7.0.0 milestone lanes still need release-readiness work before tagging. The refresh now also selects the forward train: v7.1.0 next, no planned feature v7.2.0, v8.0.0 for the Runtime Graph and Scene IR product contract, and v9.0.0 for Product Workbench and operator surfaces.

Sponsor Human

A maintainer noticed that the roadmap felt significantly outdated and asked for it to reflect current reality.

Sponsor Agent

An agent needs the roadmap to distinguish live tracker truth from historical release lineage so it can select the next slice without treating already complete zero-open release lanes as active release targets.

Hill

By the end of this cycle, the roadmap names v7.0.0 as the latest shipped public release, records that v6.0.0 was skipped as a public package release, treats complete milestone lanes as lineage, selects v7.1.0 as the next public

release target, says there is no planned v7.2.0 feature train, organizes v8.0.0 and v9.0.0, surfaces unmilestoned triage and open pull requests, and preserves deterministic tests for the GitHub-backed counts.

Current Truth

Verified against GitHub on 2026-06-13.

- Latest public release tag: v7.0.0.
- v6.0.0 was skipped as a public package release; its zero-open milestone lane is complete tracker lineage.
- GitHub milestone v6.0.0: 0 open, 30 closed milestone items.
- GitHub milestone v7.0.0: 0 open, 27 closed milestone items.
- GitHub milestone Beyond: 33 open, 4 closed milestone items.
- Open unmilestoned issues: #321, #317, #316, #306, and #249.
- Open pull requests: dependency PR #326, unmilestoned.
- Active implementation gravity: DX-046, a real GraphQL-authored DOGFOOD block fixture for #302.
- Selected next public release target: v7.1.0.
- Planned feature train after v7.1.0: v8.0.0, not v7.2.0.

Problem

The roadmap's numerical snapshot was recently synced, but the surrounding release narrative still read as if V7 had not shipped and as if v6/v7 release-readiness validation remained the next release-facing action. It also spent too much space on past milestone inventory when the useful work now is choosing the next release candidate from Beyond. That contradicts docs/release.md, which already names 7.0.0 as the latest shipped release and says the older v6/v7 milestones are retained as lineage.

Scope

This cycle includes:

- rewriting the roadmap around the shipped v7.0.0 boundary and forward release train
- recording that v6.0.0 was skipped as a public package release
- replacing pending-tag wording in v6/v7 sections with complete-lineage wording
- selecting v7.1.0 as the next public release target
- declaring that there is no planned v7.2.0 feature train
- organizing v8.0.0 around Runtime Graph and Scene IR
- organizing v9.0.0 around Product Workbench and operator surfaces
- making the post-v7 active pull explicit

- adding explicit decision points for how Beyond work becomes V8/V9 release work
- preserving open Beyond issue accounting
- adding current unmilestoned issue triage and the open dependency PR
- aligning BEARING with the same release posture
- aligning docs/release.md with the selected v7.1.0 posture
- updating WF-130 roadmap checks so stale release-readiness wording does not return

Non-Goals

This cycle does not:

- move any GitHub issues between milestones
- create a new versioned release milestone
- implement the tracker-sync sentinel from #268
- implement DX-046 or any product feature

User Experience / Product Shape

This is documentation and workflow truth work. There is no runtime or TUI surface change.

The reviewer-facing shape is:

```
docs/ROADMAP.md
  Current Release State
  Release Train Decision
  Next Pull
  Forward Goalposts
  Decision Points
  Open Beyond Issues
  Open Unmilestoned Triage
  Open Pull Requests Outside Release Horizons
  Closed Lineage

docs/release.md
  Next Release Posture
```

Lower Modes

Static Mode

Markdown remains readable in plain text and in DOGFOOD's docs reader.

Pipe Mode

The proof is deterministic CLI output from GitHub and local tests.

Accessible Mode

Tables keep explicit headers and avoid relying on prose-only counts.

Runtime / API Contract

No runtime API changes.

Data / State Model

GitHub milestones, issue labels, and PR state remain live tracker truth. docs /ROADMAP.md mirrors that state for humans and records the sync date.

Accessibility Posture

No interactive controls are added. Markdown tables and short prose summaries keep the roadmap navigable for screen readers and terminal readers.

Localization / Directionality Posture

No app-visible localized strings are added.

Agent Inspectability / Explainability Posture

Agents can inspect:

- docs/ROADMAP.md for release horizon and candidate goalpost state
- docs/BEARING.md for active gravity
- docs/release.md for shipped-release posture
- tests/cycles/WF-130/roadmap-goalpost-policy.test.ts for deterministic roadmap expectations

Linked Invariants

- Work Is Issue-Backed
- Docs Are the Demo
- Release Claims Need Proof
- Runtime Truth Wins

Implementation Outline

1. Capture live GitHub milestone, issue, and PR state.
2. Comment on #268 to connect this manual refresh to the tracker-sync problem.
3. Refresh docs/ROADMAP.md.
4. Align docs/BEARING.md.
5. Select and record the post-V7 release train.
6. Update changelog and WF-130 regression expectations.
7. Run focused validation.

Tests To Write First

- Update WF-130 expectations so stale text such as “release-readiness validation before tagging” fails if it returns to docs/ROADMAP.md.
- Assert that v7.1.0, no planned feature v7.2.0, v8.0.0, and v9.0.0 stay visible in docs/ROADMAP.md.
- Keep the Beyond row count tied to the Open Beyond Issues table.
- Assert the current unmilestoned triage and dependency PR are visible.

Validation Plan

```
npm test -- --run tests/cycles/WF-130/roadmap-goalpost-policy.test.ts
npm run docs:inventory
npm run typecheck:test
npm run lint
git diff --check
```

212

WF-132 - Release Readiness And DOGFOOD i18n Gates

docs/design/WF-132-release-readiness-and-dogfood-i18n-gates.md

Status

In progress for issues #270 and #312.

Linked Legend

- legend:wf for release workflow readiness.
- legend:df for DOGFOOD docs-surface coverage.
- legend:lx for localization debt and scanner behavior.

Sponsors

- Human sponsor: James Ross.
- Agent sponsor: Codex.

Hill

Before v7.1.0 is cut, release readiness should be inspectable as a milestone-specific report and DOGFOOD i18n debt should cover newly added docs modules by default.

Problem

The repo already has npm run release:readiness, but the command is mostly a validation gauntlet. It does not produce a milestone-aware readiness report

that proves tracker closure, WIP-label cleanup, release-document posture, and package smoke coverage before a release tag.

The DOGFOOD i18n debt ratchet also depends on a hand-maintained source list. New examples/docs/**/*.ts modules can introduce visible strings without participating in the debt baseline.

Scope

- Keep release:readiness as the single release readiness command.
- Add an optional milestone report mode that can be run before tagging, for example npm run release:readiness -- --milestone v7.1.0.
- Classify tracker, docs, release-packet, and package-smoke readiness using deterministic fixture-testable data.
- Discover DOGFOOD TypeScript debt sources from examples/docs/**/*.ts by default, with documented exclusions for tooling-only files.
- Preserve per-surface i18n debt counts so ratchet output stays actionable.

Non-Goals

- Do not move broad V8/V9 product work into v7.1.0.
- Do not require GitHub access for the default no-argument local gauntlet.
- Do not replace the GitHub release dry-run workflow; the local report should point to the existing workflow boundary.
- Do not burn down the current DOGFOOD raw-string debt in this cycle.

Playback Questions

- Does release:readiness -- --milestone v7.1.0 explain why a release is or is not ready before running expensive local gates?
- Can a new examples/docs/**/*.ts module introduce visible copy without increasing the DOGFOOD i18n debt ratchet?
- Are remaining release-process gaps documented instead of silently implied?

Accessibility / Assistive Posture

No runtime UI accessibility behavior changes. The new readiness report is plain text plus a Markdown table so it can be copied into release PRs and read by terminal or screen-reader workflows.

Localization / Directionality Posture

#312 directly strengthens localization guardrails. It does not localize existing raw DOGFOOD copy; it makes the current raw-string debt baseline honest across newly discovered DOGFOOD modules.

Agent Inspectability / Explainability Posture

The release-readiness report classifies each gate as pass/fail with a concise summary. The i18n debt scanner continues to report counts by source surface so agents can identify which DOGFOOD module moved the ratchet.

Linked Invariants

- Release tags require issue-complete tracker posture and release evidence.
- DOGFOOD visible strings must either be cataloged, counted as known debt, or explicitly excluded as non-product tooling.
- The no-argument local release:readiness gauntlet must keep working without GitHub access.

Implementation Plan

1. Extend scripts/release-readiness.ts with a milestone-aware readiness report model and CLI parsing for --milestone.
2. Keep the existing command plan intact, but add package-smoke proof by recognizing the existing smoke:canaries package-pack canary step and the release dry-run workflow boundary.
3. Add fixture tests for ready, open-issue, lingering-WIP, stale-docs, and missing-release-packet report classification.
4. Replace the hand-maintained DOGFOOD_I18N_DEBT_SOURCES list with deterministic source discovery under examples/docs/.
5. Add tests proving newly added docs modules are discovered and can fail the ratchet when they add raw visible strings.
6. Update release/DOGFOOD docs, roadmap posture, and changelog entries.

Tests To Write First

- Release-readiness report classification for ready milestone fixture data.
- Release-readiness report failures for open tracker issues, WIP labels, stale release docs, and missing release packet evidence.
- CLI parsing for --milestone.
- DOGFOOD i18n source discovery for new docs modules and documented exclusions.

- DOGFOOD i18n ratchet failure when a newly discovered module adds raw visible text above a supplied zero baseline.

Acceptance

- npm run release:readiness keeps running the current gauntlet.
- npm run release:readiness -- --milestone v7.1.0 prints a concise readiness report before running the gauntlet and blocks if milestone readiness is not satisfied.
- Report classification is covered by unit tests with fixture tracker data.
- dogfood:i18n:debt covers new examples/docs/**/*.ts modules unless the module is explicitly excluded as tooling-only.
- The current DOGFOOD i18n baseline reflects the newly covered sources and still reports counts by source surface.

Retrospective

To be completed when the PR lands.

213

WF-133 - Esbuild Security Patch

docs/design/WF-133-esbuild-security-patch.md

Status

In progress for issue #357.

Linked Legend

- legend:wf for release workflow readiness.
- dependencies for package-lock security posture.

Sponsors

- Human sponsor: James Ross.
- Agent sponsor: Codex.

Hill

Before v7.2.0 ships, Bijou's development dependency graph should resolve the patched esbuild@0.28.1 release so GitHub and npm audit no longer report the known esbuild@0.28.0 security advisories.

Problem

npm audit reports the lockfile-resolved esbuild@0.28.0 package against two GitHub advisories:

- GHSA-gv7w-rqvm-qjhr (high), patched in 0.28.1.
- GHSA-g7r4-m6w7-qqqr (low), patched in 0.28.1.

The package is present through development tooling, not Bijou runtime API:

- tsx@4.22.4 accepts esbuild@~0.28.0.
- vitest@4.1.8 pulls vite@8.0.16, whose optional peer accepts esbuild@^0.27.0 || ^0.28.0.

Scope

- Update the package lock so esbuild resolves to 0.28.1.
- Avoid broad package churn or unrelated runtime dependency changes.
- Record the security patch in the unreleased changelog.
- Keep v7.2.0 bounded: this is an urgent dependency-security repair, not a new feature slice.

Non-Goals

- Do not expand the v7.2.0 stabilization release into the broader V8/V9 work.
- Do not change application code or public API contracts.
- Do not revive the closed Dependabot PR #326; supersede it with this issue-backed cycle.

Agent Inspectability / Explainability Posture

The proof is intentionally mechanical:

- npm ls esbuild --all shows the resolved version.
- npm audit --audit-level=low reports zero vulnerabilities.
- the package-lock diff is limited to the patched package metadata.

Linked Invariants

- Release Claims Need Proof.
- Work Is Issue-Backed.
- Security fixes should minimize unrelated dependency churn.

Implementation Plan

1. Shape #357 and this design record.
2. Update the lockfile to resolve esbuild@0.28.1.
3. Add a changelog entry under Unreleased.
4. Run audit, lint, typecheck, tests, docs inventory, and diff checks.
5. Open a non-draft pull request against main.

Tests To Write First

No code-level regression test is needed because this cycle changes package resolution only. The deterministic regression check is the package-manager audit itself:

```
npm audit --audit-level=low
```

Acceptance

- package-lock.json resolves esbuild@0.28.1.
- npm audit --audit-level=low exits cleanly.
- Relevant repo validation passes.
- #357 is closed by the pull request.

Retrospective

To be completed when the PR lands.

214

WF-134 Code Dojo ESLint Ratchet 1

docs/design/WF-134-code-dojo-eslint-ratchet-1.md

```
title: WF-134 Code Dojo ESLint Ratchet 1
legend: WF
lane: bad-code
priority: high
github_issue: 364
status: proposed
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - v7.2.0
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #364
- Follow-up ratchet: #366
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions
- Release milestone: v7.2.0

Decision Summary

Respectful Repo: Enter the Code Dojo made the standards debt visible and ratcheted, but the ESLint debt is still the dominant exception class. The stored ESLint ceiling is 5,345, while a live type-aware ESLint run on main reports 5,121 findings. The first cleanup pass should convert that already observed shrink plus targeted high-yield fixes into a lower enforceable ceiling.

This cycle deliberately starts with bounded test and tooling clusters instead of broad DOGFOOD example rewrites. The DOGFOOD files also carry localization and code-size tripwires, so they need their own carefully shaped pass.

Sponsored Human

A maintainer wants the new standards posture to burn down real lint debt now, not merely document it as a future aspiration.

Sponsored Agent

An agent needs a concrete first ratchet target with inspectable before/after counts, so later cycles can keep reducing the ceiling without guessing which violations are already paid down.

Hill

Reduce live type-aware ESLint findings from 5,121 to at most 4,900, then continue the same bounded cleanup pass to at most 4,600 when the first target is comfortably met. Lower the official ESLint and aggregate Code Dojo debt ceilings so future work cannot regain the cleaned-up violations.

Scope

- Target high-yield ESLint clusters first:
  - docs-preview tests under scripts/docs-preview-*.test.ts
  - app-frame tests under packages/bijou-tui/src/app-frame-*.test.ts
  - single-rule test files such as packages/bijou/src/core/forms/wizard.test.ts
- Prefer typed test helpers and guard functions over repeated casts.
- Update scripts/code-dojo/baselines/eslint.json to the new live count and per-rule counts.
- Update docs/code-dojo-exceptions.md and the code-dojo:debt --max ceiling.
- Keep the aggregate debt reduction at least 50 violations below the current 5,768 ceiling.

Non-Goals

- Do not soften, disable, or narrow ESLint rules.
- Do not raise any Code Dojo ceiling.
- Do not mix unrelated product behavior changes into the cleanup.
- Do not tackle DOGFOOD mega-file refactors in this first pass unless a touched file's policy requires it.
- Do not try to clear all ESLint debt in one oversized branch.

Current Evidence

A live JSON ESLint run on main before this cycle found:

- 5,121 findings across 511 files.
- 2,746 findings in test files.
- 2,375 findings in runtime/source/script files.

Top rule counts:

Rule	Count
@typescript-eslint/no-non-null-assertion	1,433
@typescript-eslint/no-unsafe-type-assertion	719
@typescript-eslint/restrict-template-expressions	665
@typescript-eslint/no-explicit-any	323
@typescript-eslint/no-unsafe-member-access	234

Top file clusters:

Surface	Count
packages/bijou-tui/src	1,554
packages/bijou/src	1,353
scripts	750
tests/cycles	541
examples/docs	301

Playback Questions

1. Did the live ESLint count fall to 4,900 or lower, then to 4,600 or lower after the follow-up ratchet?
2. Did the stored ESLint baseline match the new live count?
3. Did the aggregate Code Dojo ceiling move down by at least 50 violations?
4. Do focused tests still prove the touched surfaces behave the same?
5. Does npm run code-dojo:ci pass after the lower ceiling is installed?

Tests To Write First

This cycle mostly pays down static-analysis debt in already covered surfaces. When a cleanup changes runtime behavior or helper contracts, add a focused regression test before the fix. For purely type-safety rewrites, the RED signal is the existing ESLint finding and the GREEN signal is the lowered live ESLint count plus the relevant existing focused suite.

Acceptance Criteria

- Live ESLint findings are 4,900 or lower.

- Follow-up ratchet findings are 4,600 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live count.
- docs/code-dojo-exceptions.md reports the lower aggregate debt count and next target.
- package.json lowers code-dojo:debt --max by at least 50.
- Focused tests for touched clusters pass.
- npm run code-dojo:ci, npm run docs:inventory, and git diff --check pass before final review.

Retrospective

To be completed when the PR lands.

215

WF-135 Respecting the Dojo 1000-Count Burndown

docs/design/WF-135-respecting-dojo-1000.md

```
title: WF-135 Respecting the Dojo 1000-Count Burndown
legend: WF
lane: bad-code
priority: high
github_issue: 374
status: complete
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - respectful-repo
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #374
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions

Decision Summary

Respectful Repo: Enter the Code Dojo made the standards debt enforceable. WF-134 proved the ratchet can burn down real violations, but main still has thousands of counted exceptions. This cycle raises the bar from minimum 50-count goalpost reductions to a 1000-count burndown pass.

The long-term goal is uncompromising:

- zero ESLint errors
- zero Code Dojo warnings, errors, or issues
- zero files violating length restrictions

This cycle is not allowed to move that goalpost by weakening rules, excluding files, or raising baselines.

Sponsored Human

A maintainer wants the repository to stop treating the standards as symbolic and make a visible move toward real compliance.

Sponsored Agent

An agent needs a concrete large burndown target, live count evidence, and clear guardrails so cleanup work stays reviewable instead of becoming a broad semantic rewrite.

Hill

Reduce aggregate Code Dojo debt from 4,940 to 3,940 or lower with real cleanup. Lower the official ratchet ceilings to the new live counts so future work cannot regain the violations.

Scope

- Target high-yield, mechanically reviewable ESLint clusters first:
  - non-null assertions that can become local guards or reusable helpers
  - unsafe any and unsafe assertion clusters in tests and scripts
  - template-expression findings with explicit string formatting
  - stale async wrappers and unused values where the fix is local
- Reduce file/context debt when touched files exceed the current threshold.
- Keep changes clustered by surface so validation can be focused.
- Update scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- Do not disable, soften, or narrow ESLint rules.
- Do not raise any Code Dojo ceiling.
- Do not exclude files from Code Dojo or ESLint.
- Do not make risky product behavior changes for static-analysis convenience.
- Do not split mega-files purely by line count unless the ownership boundary is obvious and tests can cover the move.

Current Evidence

Live counts on main at ed438ebe:

- aggregate Code Dojo debt: 4,940
- ESLint findings: 4,517
- ESLint errors: 4,513
- ESLint warnings: 4
- file/context baseline: 342
- mock-ban baseline: 23
- code-size baseline: 58, including 4 legacy hard-limit files

Top ESLint rules:

Rule	Count
@typescript-eslint/no-non-null-assertion	1,408
@typescript-eslint/restrict-template-expressions	664
@typescript-eslint/no-unsafe-type-assertion	584
@typescript-eslint/no-explicit-any	204
@typescript-eslint/no-unnecessary-condition	181
@typescript-eslint/no-unsafe-member-access	178
@typescript-eslint/no-unused-vars	157
@typescript-eslint/no-unsafe-assignment	130
@typescript-eslint/require-await	112
@typescript-eslint/no-unnecessary-type-assertion	100

Playback Questions

1. Did aggregate Code Dojo debt fall to 3,940 or lower?
2. Did the stored ESLint baseline match the new live count?
3. Did file/context and code-size debt hold or shrink?
4. Did touched behavioral surfaces get focused validation?
5. Does npm run code-dojo:ci pass after the lower ceiling is installed?

Tests To Write First

Pure static-analysis rewrites use the existing lint finding as RED and the lower live count as GREEN. Add focused regression tests before changing runtime behavior, helper contracts, parsing, scheduling, schema normalization, or user visible rendering.

Acceptance Criteria

- Aggregate Code Dojo debt is 3,940 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count and per-rule counts.
- docs/code-dojo-exceptions.md reports the lower aggregate debt count and next target.
- package.json lowers code-dojo:debt --max to the new live aggregate count.

- Focused tests for touched clusters pass.
- npm run code-dojo:ci, npm run docs:inventory, and git diff --check pass before final review.

Retrospective

Completed with a measured aggregate Code Dojo debt reduction from 4,940 to 2,068 (-2,872). The stored ESLint baseline now records 1,656 live findings, down from 4,517 (-2,861), and the next aggregate target is 2,018 or lower. Follow-on passes covered mechanically reviewable clusters across tests, scripts, examples, MCP docs, DOGFOOD, app-frame surfaces, render/surface helpers, table/markdown parsing, geometry, benchmarks, runtime helpers, i18n helpers, and TUI input/action surfaces.

The latest pass removed app-frame action/input-shell assertion debt, bench CLI formatting debt, focus-map text formatting debt, key parser regex/assertion debt, and node style test regex debt. Touched file/context budgets stayed below their stored ceilings, two file/context exceptions have been removed, and the latest app-frame core/overlay pass removed retained-layout non-null assertions, shell-command handler narrowing casts, combo-key formatting debt, and framed-app runtime bridge assertions. The binding cluster then removed runtime brand casts, unchecked generic snapshot casts, descriptor clone assertions, and unchecked fixture casts from the core binding contract and tests. The runtime/schema cluster removed TUI runtime binding brand casts, command dispatch narrowing debt, schema-block brand casts, schema bind-output assertions, and extracted inert schema data cloning into a focused helper.

216

WF-136 Respecting the Dojo Ratchet To 2018

docs/design/WF-136-respecting-dojo-ratchet-2018.md

```
title: WF-136 Respecting the Dojo Ratchet To 2018
legend: WF
lane: bad-code
priority: high
github_issue: 377
status: complete
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - respectful-repo
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #377
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions

Decision Summary

WF-135 merged with aggregate Code Dojo debt at 2,068, including 1,656 type-aware ESLint findings. The standing goal remains zero ESLint errors, zero Code Dojo issues, and zero files violating length restrictions. The current goalpost policy requires every met goalpost to remove at least 50 counted violations until the repository reaches zero.

This cycle targets the next ratchet: lower aggregate debt from 2,068 to 2,018 or lower without weakening rules, excluding files, or raising any baseline.

Sponsored Human

A maintainer wants each follow-on Dojo slice to keep earning the standards instead of letting the post-1000 burndown stall at a still-large exception ledger.

Sponsored Agent

An agent needs a narrow, mechanically reviewable target that clears the next 50-count ratchet while avoiding broad edits to the largest DOGFOOD files.

Hill

Reduce aggregate Code Dojo debt from 2,068 to 2,018 or lower by eliminating the current 53 ESLint findings in:

- packages/bijou-node/src/worker/worker.ts (21 findings)
- packages/bijou/src/core/components/dag-render.ts (20 findings)
- packages/bijou/src/core/components/preference-list.ts (12 findings)

Scope

- Replace unsafe any and unsafe assertion handoffs with explicit message, command, and worker-data contracts.
- Replace non-null assertions around worker IPC with local parent-port guards.
- Replace DAG renderer array assumptions with direct iteration, guarded reads, and explicit string formatting.
- Replace preference-list index assertions with value iteration and remove the dead height assignment.
- Keep worker-thread, DAG rendering, and preference-list behavior stable.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- Leave the 5,000-line DOGFOOD app untouched in this slice.
- Keep ESLint rules at full strength; no softening, disabling, or narrowing.
- Preserve existing file/context and code-size ceilings.
- Preserve worker lifecycle semantics instead of changing them for static-analysis convenience.

Current Evidence

Live counts on main at ec943706:

- aggregate Code Dojo debt: 2,068
- ESLint findings: 1,656
- file/context baseline: 333

- mock-ban baseline: 23
- code-size baseline: 56, including 4 legacy hard-limit files
- next aggregate target: 2,018 or lower

Focused ESLint findings:

File	Findings
packages/bijou-node/src/worker/worker.ts	21
packages/bijou/src/core/components/dag-render.ts	20
packages/bijou/src/core/components/preference-list.ts	12
Total	**53**

Playback Questions

1. Is aggregate Code Dojo debt at 2,018 or lower?
2. Does the stored ESLint baseline match the new live count?
3. Have file/context and code-size debt held or shrunk?
4. Do focused worker, DAG renderer, and preference-list tests pass?
5. Are the repo gates passing before review?

Tests To Write First

The lint findings are the RED signal for pure type-safety rewrites. Add focused regression tests before changing worker lifecycle behavior, message routing, DAG rendering behavior, or preference-list layout.

Acceptance Criteria

- packages/bijou-node/src/worker/worker.ts has zero ESLint findings.
- packages/bijou/src/core/components/dag-render.ts has zero ESLint findings.
- packages/bijou/src/core/components/preference-list.ts has zero ESLint findings.
- Aggregate Code Dojo debt is 2,018 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched worker, DAG renderer, and preference-list surfaces passes.
- npm run code-dojo:verify, npm run docs:inventory, npm run lint, and git diff --check pass before review.

Retrospective

Completed with aggregate Code Dojo debt reduced from 2,068 to 2,015 (-53). The stored ESLint baseline now records 1,603 live findings, down from 1,656, and the next aggregate target is 1,965 or lower.

The implementation cleaned the targeted worker IPC and component rendering files:

- packages/bijou-node/src/worker/worker.ts now reads worker data through a typed guard, narrows parentPort once, and validates raw worker IPC messages before dispatching them into proxy I/O handlers.
- packages/bijou-node/src/worker/worker-data.ts keeps worker-data validation out of the already-baselined worker runtime file.
- packages/bijou/src/core/components/dag-render.ts now iterates rendered graphemes and layer arrays without non-null assertions and formats accessible counts explicitly.
- packages/bijou/src/core/components/preference-list.ts now iterates value, description, section, row, and output character arrays directly without assertion-based indexing.

Touched file/context budgets held or shrank: worker.ts is now 318 lines / 10,676 bytes against a 334 / 11,496 baseline, dag-render.ts is now 692 lines / 25,039 bytes against a 693 / 25,088 baseline, and preference-list.ts is now 420 lines / 13,876 bytes against a 425 / 14,249 baseline.

217

WF-137 Respecting the Dojo Ratchet To 1965

docs/design/WF-137-respecting-dojo-ratchet-1965.md

```
title: WF-137 Respecting the Dojo Ratchet To 1965
legend: WF
lane: bad-code
priority: high
github_issue: 379
status: complete
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - respectful-repo
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #379
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions

Decision Summary

WF-136 merged with aggregate Code Dojo debt at 2,015, including 1,603 type-aware ESLint findings. The repo goal remains zero ESLint errors, zero Code Dojo issues, and zero files violating length restrictions. The ratchet policy requires every met goalpost to remove at least 50 counted violations until zero.

This cycle targets the next ratchet: lower aggregate debt from 2,015 to 1,965 or lower without weakening rules, excluding files, or raising any baseline.

Sponsored Human

A maintainer wants the post-merge Dojo loop to keep moving while the next target is fresh and the local iteration path is still warm.

Sponsored Agent

An agent needs a small, non-DOGFOOD cleanup cluster that clears the next 50-count ratchet without pulling the localization debt gate into unrelated source edits.

Hill

Reduce aggregate Code Dojo debt from 2,015 to 1,965 or lower by eliminating at least 50 live ESLint findings from non-DOGFOOD package or test files.

Current candidates from the offender list:

File	Findings
packages/bijou/src/core/theme/dtcg.ts	20
packages/bijou-tui/src/pipeline/pipeline.ts	18
packages/bijou-tui/src/canvas.ts	17
packages/bijou-i18n-tools/src/exchange.ts	15
Candidate total	**70**

The final slice may substitute equivalent non-DOGFOOD files if a candidate would require broader semantic work than the goalpost warrants.

Scope

- Replace unsafe theme-token and pipeline assertions with explicit guards or narrower local types.
- Replace assertion-based array/object reads with checked reads or direct value iteration.
- Make template interpolation explicit where values are not statically strings.
- Keep rendering, pipeline, and i18n exchange behavior stable.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- Leave the 5,000-line DOGFOOD app untouched in this slice.
- Keep ESLint rules and Code Dojo thresholds at full strength.
- Preserve file/context and code-size ceilings.
- Avoid architectural rewrites outside the selected cleanup files.

Current Evidence

Live counts on main at e6c8bdd1:

- aggregate Code Dojo debt: 2,015
- ESLint findings: 1,603
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 56, including 4 legacy hard-limit files
- next aggregate target: 1,965 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,965 or lower?
2. Does the stored ESLint baseline match the new live count?
3. Have file/context and code-size debt held or shrunk?
4. Do focused tests for the touched surfaces pass?
5. Are the repo gates passing before review?

Tests To Write First

The lint findings are the RED signal for pure type-safety rewrites. Add focused regression tests before changing rendering, pipeline execution, or i18n exchange behavior.

Acceptance Criteria

- The selected touched TypeScript files have zero new ESLint findings and reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,965 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.
- npm run code-dojo:verify, npm run docs:inventory, npm run lint, and git dif f --check pass before review.

Retrospective

Completed with aggregate Code Dojo debt reduced from 2,015 to 1,907 (-108). The stored ESLint baseline now records 1,495 live findings, down from 1,603, and the next aggregate target is 1,857 or lower.

The implementation cleaned six non-DOGFOOD files:

- packages/bijou/src/core/theme/dtcg.ts now treats imported DTCG documents as unknown-shaped records and validates token, modifier, RGB, group, and JSON reads locally instead of casting external payloads into narrower types.

- packages/bijou-tui/src/canvas.ts now uses unknown shader uniforms, preserves packed-surface narrowing directly, computes UV denominators explicitly, and removes assertion-based Braille dot reads.
- packages/bijou-i18n-tools/src/exchange.ts now validates serialized exchange value kinds before decoding, validates external collection arrays before iterating, keeps external version fields checkable, and derives workbook row typing from the column manifest.
- packages/bijou/src/factory.ts now shares the DTCG JSON document guard used by file-based theme loading instead of asserting parsed environment payloads.
- packages/bijou-tui/src/overlay.ts now guards resolved color records and packed-buffer reads without non-null assertions while preserving drawer and inherited-background behavior.
- packages/bijou-tui/src/pipeline/pipeline.ts now uses unknown render-state extension data, guarded layout-root and thenable reads, explicit unknown error formatting, and preserved halted-stage timing semantics.
- packages/bijou-tui/src/runtime.ts now avoids void unions in lifecycle hook returns, keeps render/crash queue checks explicit, removes the legacy layou tRoot mutation, and formats runtime errors without object stringification.

Touched file/context budgets held or shrank: dtcg.ts is now 314 lines / 10,49 1 bytes against a 347 / 12,346 baseline, canvas.ts is now 358 lines / 10,953 bytes against a 377 / 11,295 baseline, and exchange.ts is now 356 lines / 11, 390 bytes against a 361 / 11,459 baseline. The follow-on TUI slice held the same ratchet: overlay.ts is now 933 lines / 33,332 bytes against a 942 / 33,7 21 baseline, pipeline.ts is now 238 lines / 7,756 bytes against a 268 / 8,646 baseline, and runtime.ts is now 669 lines / 19,491 bytes against a 695 / 20,6 40 baseline.

218

WF-138 Respecting the Dojo Ratchet To 1857

docs/design/WF-138-respecting-dojo-ratchet-1857.md

```
title: WF-138 Respecting the Dojo Ratchet To 1857
legend: WF
lane: bad-code
priority: high
github_issue: 381
status: in_progress
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - respectful-repo
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #381
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions

Decision Summary

WF-137 merged with aggregate Code Dojo debt at 1,907, including 1,495 type-aware ESLint findings. The repo goal remains zero ESLint errors, zero Code Dojo issues, and zero files violating length restrictions. The ratchet policy requires every met goalpost to remove at least 50 counted violations until zero.

This cycle targets the next ratchet: lower aggregate debt from 1,907 to 1,857 or lower without weakening rules, excluding files, or raising any baseline.

Sponsored Human

A maintainer wants the Respecting the Dojo loop to keep turning after WF-137 landed rather than letting the next 50 violations become background noise.

Sponsored Agent

An agent needs a compact cleanup cluster with enough current ESLint density to clear the next goalpost while preserving touched-file size ratchets.

Hill

Reduce aggregate Code Dojo debt from 1,907 to 1,857 or lower by eliminating at least 50 live ESLint findings with real fixes.

Final non-DOGFOOD slice from the offender list:

File	Findings
bench/src/soak-runner.ts	16
packages/bijou/src/core/components/markdown-ren der.ts	16
packages/bijou/src/core/forms/textarea-editor.t s	14
scripts/record-gifs.ts	15
Slice total	**61**

The initial broader candidate list stayed flexible so this goalpost could avoid larger semantic work in layout IR and active-binding contracts.

Implementation Result

The final slice lowered live type-aware ESLint findings from 1,495 to 1,434, reducing aggregate Code Dojo debt from 1,907 to 1,846. The cleanup kept file/context and code-size counts flat while replacing unchecked scenario, wrapped-line, editor-line, queue, and dynamic-recorder reads with checked access.

Scope

- Replace non-null assertions and unsafe assertions with checked reads, structural guards, or narrowed helper functions.
- Make template interpolation explicit where values are not statically strings.
- Remove stale or redundant conditionals only when behavior is preserved.
- Keep rendering, layout, active-binding, benchmark, and markdown behavior stable.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.m d, and package.json only after live counts prove the reduction.

Non-Goals

- Do not weaken ESLint rules, Code Dojo thresholds, or standards artifacts.
- Do not raise file/context or code-size ceilings.
- Do not refactor unrelated rendering or layout architecture.
- Do not rely on exception churn in place of real cleanup.

Current Evidence

Live counts on main at 52230bc5:

- aggregate Code Dojo debt: 1,907
- ESLint findings: 1,495
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 56, including 4 legacy hard-limit files
- next aggregate target: 1,857 or lower

Live counts after the implementation slice:

- aggregate Code Dojo debt: 1,846
- ESLint findings: 1,434
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 56, including 4 legacy hard-limit files
- next aggregate target: 1,796 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,857 or lower?
2. Does the stored ESLint baseline match the new live count?
3. Have file/context and code-size debt held or shrunk?
4. Do focused tests for the touched surfaces pass?
5. Are the repo gates passing before review?

Tests To Write First

The lint findings are the RED signal for pure type-safety rewrites. Add focused regression tests before changing behavior in rendering, layout, active binding, or benchmark execution.

Acceptance Criteria

- The selected touched TypeScript files have zero new ESLint findings and reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,846 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.
- npm run code-dojo:verify, npm run docs:inventory, npm run lint, and git diff --check pass before review.

219

WF-139 Respecting the Dojo Ratchet To 1796

docs/design/WF-139-respecting-dojo-ratchet-1796.md

```
title: WF-139 Respecting the Dojo Ratchet To 1796
legend: WF
lane: bad-code
priority: high
github_issue: 383
status: in_progress
keywords:
  - code-dojo
  - eslint
  - standards
  - ratchet
  - respectful-repo
```

Legend: WF - Workflow and Delivery

Linked Work

- Issue: #383
- Standards artifact: TypeScript Code Standards Editor's Edition
- Exception ledger: Code Dojo Exceptions

Decision Summary

WF-138 merged with aggregate Code Dojo debt at 1,846, including 1,434 type-aware ESLint findings. The repo goal remains zero ESLint errors, zero Code Dojo issues, and zero files violating length restrictions. The ratchet policy requires every met goalpost to remove at least 50 counted violations until zero.

This cycle targets the next ratchet: lower aggregate debt from 1,846 to 1,796 or lower without weakening rules, excluding files, or raising any baseline.

Sponsored Human

A maintainer wants the Dojo loop to keep moving immediately after WF-138 rather than parking a ready burndown on main and letting the next 50 violations linger.

Sponsored Agent

An agent needs a bounded cluster with enough current ESLint density to clear the next goalpost while keeping touched-file size ratchets flat or lower.

Hill

Reduce aggregate Code Dojo debt from 1,846 to 1,796 or lower by eliminating at least 50 live ESLint findings with real fixes.

Final non-DOGFOOD slice from the offender list:

File	Findings
packages/bijou/src/core/ui-scene-ir.ts	17
packages/bijou/src/core/active-binding-collection.ts	16
packages/bijou/src/core/layout/envelope.ts	16
packages/bijou-tui/src/flex.ts	13
Slice total	**62**

Implementation Result

The final slice lowered live type-aware ESLint findings from 1,434 to 1,368 and removed packages/bijou-tui/src/flex.ts from the code-size baseline by shrinking it to the 500-line ratchet boundary. Aggregate Code Dojo debt fell from 1,846 to 1,779.

Scope

- Replace non-null assertions and unsafe assertions with checked reads, structural guards, or narrowed helper functions.
- Make template interpolation explicit where values are not statically strings.
- Remove stale or redundant conditionals only when behavior is preserved.
- Keep UI scene IR hashing/lowering, active-binding collection behavior, layout envelope negotiation, and TUI flex behavior stable.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- Weakening ESLint rules, Code Dojo thresholds, or standards artifacts is out of scope.
- File/context and code-size ceilings must not rise.
- Unrelated rendering or layout architecture refactors are out of scope.

- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at f820fc07:

- aggregate Code Dojo debt: 1,846
- ESLint findings: 1,434
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 56, including 4 legacy hard-limit files
- next aggregate target: 1,796 or lower

Live counts after the implementation slice:

- aggregate Code Dojo debt: 1,779
- ESLint findings: 1,368
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,729 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,796 or lower?
2. Does the stored ESLint baseline match the new live count?
3. Have file/context and code-size debt held or shrunk?
4. Do focused tests for the touched surfaces pass?
5. Are the repo gates passing before review?

Tests To Write First

The lint findings are the RED signal for pure type-safety rewrites. Add focused regression tests before changing behavior in UI scene lowering, active binding, layout envelope resolution, or flex layout.

Acceptance Criteria

- The selected touched TypeScript files have zero new ESLint findings and reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,779 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.

- Focused validation for the touched surfaces passes.
- npm run code-dojo:verify, npm run docs:inventory, npm run lint, and git diff --check pass before review.

220

WF-140 Respecting the Dojo Ratchet To 1729

docs/design/WF-140-respecting-dojo-ratchet-1729.md

Status

Implemented.

Issue

- GitHub Issue: #385

Context

WF-139 landed the Respecting the Dojo burndown at 1,779 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,729 or lower.

The goal remains zero ESLint findings, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is one real cleanup slice toward that endpoint, not an exception shuffle.

Goal

Reduce aggregate Code Dojo debt from 1,779 to 1,729 or lower by eliminating at least 50 live ESLint findings with behavior-preserving TypeScript fixes.

Final Slice

The implemented cleanup removes exactly 50 live ESLint findings:

File	Findings
examples/perf-gradient/main.ts	24

File	Findings
packages/bijou/src/core/components/dag-edges.test.ts	12
packages/bijou/src/core/components/box.test.ts	12
packages/bijou/src/core/components/perf-overlay.ts	2
Slice total	**50**

The DOGFOOD capture candidate is intentionally deferred because touching that source currently activates a broader DOGFOOD i18n raw-copy cleanup than this goalpost should carry.

Scope

- Replace non-null assertions, unsafe assertions, and any escapes with checked reads, typed helpers, or narrowed runtime guards.
- Make numeric and unknown template interpolation explicit.
- Preserve perf-gradient and component behavior.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, and standards artifacts must not weaken.
- File/context and code-size ceilings must not rise.
- Broad DOGFOOD architecture rewrites are out of scope.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at 7e029df7:

- aggregate Code Dojo debt: 1,779
- ESLint findings: 1,368
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,729 or lower

Implementation result:

- aggregate Code Dojo debt: 1,729
- ESLint findings: 1,318
- live ESLint reduction: 50
- next aggregate target: 1,679 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,729 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Run focused Code Dojo gates for the touched files.
- Run focused tests for the selected component surfaces.
- Run npm run code-dojo:debt.
- Run npm run code-dojo:verify.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files have zero new ESLint findings and reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,729 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

221

WF-141 Respecting the Dojo Ratchet To 1679

docs/design/WF-141-respecting-dojo-ratchet-1679.md

Status

Implemented.

Issue

- GitHub Issue: #387

Context

WF-140 landed the Respecting the Dojo burndown at 1,729 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,679 or lower.

The goal remains zero ESLint findings, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is one real cleanup slice toward that endpoint, not an exception shuffle.

Goal

Reduce aggregate Code Dojo debt from 1,729 to 1,679 or lower by eliminating at least 50 live ESLint findings with behavior-preserving TypeScript fixes.

Final Slice

The implemented cleanup removes 58 live ESLint findings:

File	Findings
packages/bijou/src/core/theme/builder.ts	13
packages/bijou/src/core/theme/graph.ts	13

File	Findings
packages/bijou/src/core/components/dag-source.ts	13
scripts/image-viewer.test.ts	13
examples/image-viewer/main.ts	6
Slice total	**58**

The final slice added examples/image-viewer/main.ts because narrowing that app's concrete surface-rendering return type removes unsafe surface assertions from the image-viewer tests while preserving runtime behavior.

Scope

- Replace non-null assertions, unsafe assertions, any escapes, and unchecked dynamic reads with typed helpers, checked reads, or narrowed runtime guards.
- Make numeric and unknown template interpolation explicit.
- Preserve theme-builder, theme-graph, DAG source, and image-viewer behavior.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, and standards artifacts must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD localization raw-copy cleanup is out of scope for this slice unless deliberately chosen as the primary goalpost.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at 45462fae:

- aggregate Code Dojo debt: 1,729
- ESLint findings: 1,318
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,679 or lower

Implementation result:

- aggregate Code Dojo debt: 1,671
- ESLint findings: 1,260
- live ESLint reduction: 58

- next aggregate target: 1,621 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,671 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Run focused Code Dojo gates for the touched files.
- Run focused tests for the selected component, theme, and script surfaces.
- Run npm run code-dojo:debt.
- Run npm run code-dojo:verify.
- Run npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files have zero new ESLint findings and reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,671 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

222

WF-142 Respecting The Dojo Ratchet To 1621

docs/design/WF-142-respecting-dojo-ratchet-1621.md

Status

Implemented.

Issue

- GitHub Issue: #389

Context

WF-141 landed the Respecting the Dojo burndown at 1,671 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,621 or lower.

The active objective remains zero ESLint errors, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is a bounded cleanup slice toward that endpoint, not a reason to loosen standards or shuffle exceptions.

Goal

Reduce aggregate Code Dojo debt from 1,671 to 1,621 or lower by eliminating at least 50 live ESLint findings with behavior-preserving TypeScript fixes.

Final Slice

The implemented cleanup removes 54 live ESLint findings without touching DOGFOOD docs source files:

File	Findings
packages/bijou-tui/src/notification.ts	13
packages/bijou/src/core/components/dag-layout.ts	10
packages/bijou/src/core/layout/flex.ts	10
packages/bijou/src/core/theme/color.ts	10
packages/bijou/src/core/theme/extend.ts	6
packages/bijou-tui/src/panel-dock.ts	3
packages/bijou-tui/src/surface-layout.ts	2
Slice total	**54**

The initial DOGFOOD docs candidate cluster was rejected because touching those source files activates the DOGFOOD i18n raw-string debt gate. This slice stays in package/core files and keeps touched file/context and code-size ceilings flat or lower.

Scope

- Replace non-null assertions, unsafe assertions, any escapes, unchecked dynamic reads, and implicit interpolation with typed helpers, checked reads, explicit formatting, or narrowed runtime guards.
- Prefer local helper extraction only when it removes repeated unsafe patterns without increasing touched file/context debt.
- Preserve DOGFOOD docs/example runtime behavior.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, hooks, and CI policy must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD product behavior, localization policy, and demo content are not being redesigned in this slice.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at 082342b3:

- aggregate Code Dojo debt: 1,671
- ESLint findings: 1,260
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,621 or lower

Implementation result:

- aggregate Code Dojo debt: 1,617
- ESLint findings: 1,206
- live ESLint reduction: 54
- next aggregate target: 1,567 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,621 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Probe the selected files with focused Code Dojo gates.
- Run focused tests for touched package/core surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files reduce live ESLint findings by at least 50.
- Aggregate Code Dojo debt is 1,621 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

223

WF-143 Respecting the Dojo Ratchet To 1564

docs/design/WF-143-respecting-dojo-ratchet-1567.md

Status

Implemented.

Issue

- GitHub Issue: #391

Context

WF-142 landed the Respecting the Dojo burndown at 1,617 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,567 or lower.

The active objective remains zero ESLint errors, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is a bounded cleanup slice toward that endpoint, not a reason to loosen standards or shuffle exceptions.

Goal

Reduce aggregate Code Dojo debt from 1,617 to 1,567 or lower by eliminating at least 50 live ESLint findings with behavior-preserving TypeScript fixes. The implemented slice reaches 1,564.

Candidate Slice

The implemented non-DOGFOOD package/script slice removes 52 live ESLint findings while shrinking touched file/context ceilings:

File	Findings
packages/bijou-tui/src/runtime-engine.test.ts	13
scripts/dogfood-i18n-completeness.ts	12
packages/bijou-tui/src/driver.test.ts	11
packages/bijou/src/core/forms/select.ts	6
packages/bijou-tui/src/transition-shaders.test.ts	5
packages/bijou/src/core/render/packed-cell.test.ts	5
Implemented total	**52**

The final slice avoided partially-cleaned files because code-dojo:changed requires touched files to be raw-ESLint clean.

Scope

- Replace non-null assertions, unsafe assertions, any escapes, unchecked dynamic reads, and implicit interpolation with typed helpers, checked reads, explicit formatting, or narrowed runtime guards.
- Prefer local helper extraction only when it removes repeated unsafe patterns without increasing touched file/context debt.
- Preserve TUI runtime, AppFrame rendering, example smoke, and recorder behavior.
- Lower scripts/code-dojo/baselines/eslint.json, docs/code-dojo-exceptions.md, and package.json only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, hooks, and CI policy must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD docs raw-string cleanup is out of scope for this slice unless it is deliberately chosen as the primary goalpost.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at b5774a4a:

- aggregate Code Dojo debt: 1,617
- ESLint findings: 1,206
- file/context baseline: 333
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,567 or lower

Implemented counts on this branch:

- aggregate Code Dojo debt: 1,564
- ESLint findings: 1,154
- file/context baseline: 332
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,514 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,567 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Probe the selected files with focused Code Dojo gates.
- Run focused tests for touched package/script surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files reduce live ESLint findings by at least 50.
- Aggregate Code Dojo debt is 1,567 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

224

WF-144 Respecting the Dojo Ratchet To 1514

docs/design/WF-144-respecting-dojo-ratchet-1514.md

Status

Implemented; pending review.

Issue

- GitHub Issue: #393

Context

WF-143 landed the Respecting the Dojo burndown at 1,564 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,514 or lower.

The active objective remains zero ESLint errors, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is a bounded cleanup slice toward that endpoint, not a license to widen baselines or move exceptions around.

Goal

Reduce aggregate Code Dojo debt from 1,564 to 1,514 or lower by eliminating at least 50 live findings with behavior-preserving TypeScript fixes.

Selected Slice

The implementation removed a coherent cluster of app-frame, transition, form, component, and smoke-runner findings without touching DOGFOOD docs source files:

File	Findings
packages/bijou-tui/src/app-frame-render.ts	12
scripts/smoke-dogfood-lib.ts	11
packages/bijou/src/core/components/enumerated-list.ts	10
packages/bijou-tui/src/animate.test.ts	9
packages/bijou-tui/src/transition-shaders.ts	6
packages/bijou/src/core/forms/multiselect.ts	5
Selected total	**53**

All six touched files are raw-ESLint clean and at or below their stored file/context ceiling.

Scope

- Replace non-null assertions, unsafe assertions, any escapes, unchecked dynamic reads, and implicit interpolation with typed helpers, checked reads, explicit formatting, or narrowed runtime guards.
- Prefer local helper extraction only when it removes repeated unsafe patterns without increasing touched file/context debt.
- Preserve TUI app-frame rendering, scoped message wrapping, recorder behavior, smoke example orchestration, and bench runner output.
- Lower ratchet artifacts only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, hooks, and CI policy must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD docs raw-string cleanup is out of scope for this slice unless it is deliberately chosen as the primary goalpost.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at 1159ba39:

- aggregate Code Dojo debt: 1,564
- ESLint findings: 1,154
- file/context baseline: 332
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,514 or lower

Live implementation counts on cycle/respecting-dojo-ratchet-1514:

- aggregate Code Dojo debt: 1,511
- ESLint findings: 1,101

- file/context baseline: 332
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target after this cycle: 1,461 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,514 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Probe the selected files with focused Code Dojo gates.
- Run focused tests for touched package/script surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,514 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

225

WF-145 Respecting the Dojo Ratchet To 1461

docs/design/WF-145-respecting-dojo-ratchet-1461.md

Status

Implemented; pending review.

Issue

- GitHub Issue: #395

Context

WF-144 landed the Respecting the Dojo burndown at 1,511 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,461 or lower.

The active objective remains zero ESLint errors, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is a bounded cleanup slice toward that endpoint.

Goal

Reduce aggregate Code Dojo debt from 1,511 to 1,461 or lower by eliminating at least 50 live findings with behavior-preserving TypeScript fixes.

Selected Slice

The implementation used a non-DOGFOOD package/script/test cluster that avoided DOGFOOD raw visible-copy policy work, removed 63 live ESLint findings, and retired one mock-ban violation:

File	Findings
scripts/smoke-all-examples-lib.ts	12
bench/src/harnesses/wall-time/runner.ts	11
packages/bijou-tui/src/types.ts	11
packages/bijou/src/core/theme/dtcg.test.ts	11
packages/bijou/src/core/app-shell-composition.test.ts	10
command-contract follow-up cleanup	8
Selected total	**63**

The selected cluster leaves every touched file raw-ESLint clean and at or below its stored file/context ceiling. The implementation intentionally avoided packages/bijou-tui/src/app-frame-types.ts and packages/bijou-node/src/recorder.ts because the former has a deliberate frame-scoped generic bridge and the latter needs broader recorder module validation than this bounded slice should carry.

Scope

- Replace unsafe assertions, unused parameters, fake async commands, unchecked dynamic reads, and implicit interpolation with typed helpers, checked reads, explicit formatting, or narrowed runtime guards.
- Preserve app-frame message contracts, example smoke orchestration, wall-time benchmark output, recorder behavior, and public TUI command types.
- Lower ratchet artifacts only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, hooks, and CI policy must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD raw visible-copy cleanup is out of scope for this slice unless it is deliberately chosen as the primary goalpost.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at f279b6d2:

- aggregate Code Dojo debt: 1,511
- ESLint findings: 1,101
- file/context baseline: 332
- mock-ban baseline: 23
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,461 or lower

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,443
- ESLint findings: 1,034
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,393 or lower

Playback Questions

1. Is aggregate Code Dojo debt at 1,461 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Probe the selected files with focused Code Dojo gates.
- Run focused tests for touched package/script surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,461 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.
- Focused validation for the touched surfaces passes.

226

WF-146 Respecting the Dojo Ratchet To 1393

docs/design/WF-146-respecting-dojo-ratchet-1393.md

Status

Implemented; pending final validation and review.

Issue

- GitHub Issue: #397

Context

WF-145 landed the Respecting the Dojo burndown at 1,443 aggregate Code Dojo violations. The next ratchet must cut at least 50 more violations and set the ceiling to 1,393 or lower.

The active objective remains zero ESLint errors, zero Code Dojo warnings/errors/issues, and zero length-restriction violations. This cycle is a bounded cleanup slice toward that endpoint.

Goal

Reduce aggregate Code Dojo debt from 1,443 to 1,393 or lower by eliminating at least 50 live findings with behavior-preserving TypeScript fixes.

Candidate Slice

The current ESLint offender list gives two plausible routes:

Candidate	Findings	Notes
examples/docs/app.ts	118	Highest yield, but a large DOGFOOD docs surface with raw visible-copy policy risk.
examples/docs/stories.ts	41	High-yield template-formatting cleanup in a large DOGFOOD story surface.
examples/docs/dogfood-blocks.ts	37	High-yield DOGFOOD block cleanup with formatting and checked-read work.
Smaller package/test cluster	50+	Lower single-file yield, but can avoid broad DOGFOOD policy coupling.

Implementation should choose the highest-yield slice that can pass the focused Code Dojo and DOGFOOD gates without weakening standards. If DOGFOOD docs files are touched, visible-copy policy failures are part of the work rather than an excuse to raise baselines.

Scope

- Replace unsafe assertions, non-null assertions, unchecked dynamic reads, fake async commands, stale unused values, and implicit interpolation with typed helpers, checked reads, explicit formatting, or narrowed runtime guards.
- Preserve DOGFOOD docs behavior, package runtime behavior, and public APIs.
- Lower ratchet artifacts only after live counts prove the reduction.

Non-Goals

- ESLint rules, Code Dojo thresholds, hooks, and CI policy must not weaken.
- File/context, mock-ban, and code-size ceilings must not rise.
- DOGFOOD visible-copy cleanup is in scope only for files deliberately touched by the selected slice.
- Exception churn cannot stand in for real cleanup.

Current Evidence

Live counts on main at 5eccfb69:

- aggregate Code Dojo debt: 1,443
- ESLint findings: 1,034
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,393 or lower

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,369
- ESLint findings: 960
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 1,319 or lower

Selected cleanup slice:

- packages/bijou-tui/src/app-frame-types.ts
- packages/bijou-tui/src/css/text-style.ts
- packages/bijou-tui/src/driver.ts
- packages/bijou-tui/src/focus-area.ts
- packages/bijou/src/core/binding-lifecycle.ts
- examples/docs/dogfood-shell-themes.ts, including its DOGFOOD raw-string debt cleanup
- examples/docs/i18n-debt.ts

Playback Questions

1. Is aggregate Code Dojo debt at 1,393 or lower?
2. Did the implementation remove at least 50 real violations?
3. Did touched file/context, mock-ban, and code-size ceilings stay flat or lower?
4. Did focused behavior tests for touched surfaces pass?

Validation Plan

- Probe the selected files with focused Code Dojo gates.
- Run focused tests for touched package/script/DOGFOOD surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and npm run lint:eslint.

Acceptance Criteria

- The selected touched TypeScript files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,393 or lower.
- scripts/code-dojo/baselines/eslint.json records the lower live ESLint count.
- docs/code-dojo-exceptions.md and package.json report the lower ceiling.

- Focused validation for the touched surfaces passes.

227

WF-147 Respecting the Dojo Ratchet To 1319

docs/design/WF-147-respecting-dojo-ratchet-1319.md

Status

Implemented.

Tracker

- GitHub Issue: #399
- Pull Request: #400

Intent

Continue the Respecting the Dojo burndown by removing at least 50 counted standards violations from the landed main ceiling.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at c9370875:

- aggregate Code Dojo debt: 1,369
- ESLint findings: 960
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,319 or lower

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,318
- ESLint findings: 909
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,268 or lower

Selected cleanup slice:

- tests/cycles/LX-008/localized-shell-and-dogfood.test.ts
- tests/cycles/LX-005/bijou-i18n-tools-node.test.ts
- packages/bijou/src/core/theme/dtcg.fuzz.test.ts
- packages/bijou/src/core/binding-lifecycle.test.ts

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type and control-flow fixes over local suppressions or baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,319 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,319 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.

228

WF-148 Respecting the Dojo Ratchet To 1268

docs/design/WF-148-respecting-dojo-ratchet-1268.md

Status

Implemented.

Tracker

- GitHub Issue: #401
- Pull Request: #402

Intent

Continue the Respecting the Dojo burndown by removing at least 50 counted standards violations from the landed main ceiling.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at c911b15f:

- aggregate Code Dojo debt: 1,318
- ESLint findings: 909
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,268 or lower

Selected cleanup slice:

- packages/bijou-i18n-tools/src/adapters.ts
- packages/bijou-i18n-tools/src/adapter-guards.ts
- packages/bijou-i18n/src/runtime.test.ts
- packages/bijou-node/src/recorder.ts
- packages/bijou-node/src/oled-font.ts
- packages/bijou-tui/src/layout-inspector.ts
- packages/bijou-tui/src/viewport.ts
- packages/bijou/src/core/graphql-bijou-block.ts
- packages/bijou/src/core/graphql-parser-utils.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,266
- ESLint findings: 857
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,216 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type and control-flow fixes over local suppressions or baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,268 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?

4. Did focused tests for touched behavior pass?

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,268 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 1,218 or lower.

229

WF-149 Respecting the Dojo Ratchet To 1216

docs/design/WF-149-respecting-dojo-ratchet-1216.md

Status

Implemented.

Tracker

- GitHub Issue: #403
- Pull Request: #404

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou keeps earning its place inside the Code Dojo by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at ae6c34b6:

- aggregate Code Dojo debt: 1,266

- ESLint findings: 857
- file/context baseline: 332
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,216 or lower

Selected cleanup slice:

- packages/bijou-node/src/worker/worker.proxy.test.ts
- packages/bijou-tui/src/notification-stack.ts
- packages/bijou-tui/src/pipeline/pipeline.test.ts
- packages/bijou-tui/src/runtime-binding.test.ts
- packages/bijou/src/core/components/box-v3.ts
- packages/bijou/src/core/components/surface-text.ts
- packages/bijou/src/core/render/differ.test.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,215
- ESLint findings: 807
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,165 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,216 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, and fake async shapes with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.

5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,216 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 1,166 or lower.

230

WF-150 Respecting the Dojo Ratchet To 1165

docs/design/WF-150-respecting-dojo-ratchet-1165.md

Status

Implemented.

Tracker

- Issue: #405
- Pull request: #406

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Respectful Repo: Enter the Code Dojo remains the active goalpost before the next v7.2.0 product pull. WF-149 landed at 321bb2b0 with aggregate Code Dojo debt at 1,215.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at 321bb2b0:

- aggregate Code Dojo debt: 1,215

- ESLint findings: 807
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,165 or lower

Selected cleanup slice:

- packages/bijou-tui/src/app-frame-input-overlays.test.ts
- packages/bijou-tui/src/subapp/mount.test.ts
- packages/bijou/src/core/block-metadata.test.ts
- scripts/docs-preview-landing.test.ts
- scripts/smoke-all-examples.test.ts
- tests/cycles/DL-017/dogfood-light-theme-readiness.test.ts
- tests/cycles/LX-011/dogfood-locale-ratchet.test.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,160
- ESLint findings: 752
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,110 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Remove at least 50 counted violations with real fixes.
- Prefer real type, data-shape, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Ratchet stale baseline entries down or out when touched files fall below thresholds.
- Update the Code Dojo exception ledger, ESLint baseline, package ceiling, and changelog with the landed count.

Non-Goals

- Do not raise baselines.
- Do not change public runtime semantics merely to satisfy lint.
- Do not return to v7.2.0 product work in this cycle.
- Do not treat exceptions as adherence.

Playback Questions

1. Is aggregate Code Dojo debt at 1,165 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, and fake async shapes with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.

5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- npm run code-dojo:slice -- <touched files>
- focused tests covering touched behavior
- npm run code-dojo:debt
- npm run code-dojo:verify
- npm run docs:inventory
- npm run lint
- git diff --check
- full pre-push gate before merge

Acceptance Criteria

- Aggregate debt is 1,165 or lower.
- ESLint baseline is lowered by at least 50 unless other real Dojo debt is removed in the same cycle.
- No file/context, mock-ban, or code-size baseline increases are introduced.
- GitHub PR checks are green with zero unresolved review threads before merge.

231

WF-151 Respecting the Dojo Ratchet To 1110

docs/design/WF-151-respecting-dojo-ratchet-1110.md

Status

Implemented.

Tracker

- GitHub Issue: #407
- Pull Request: #408

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou keeps earning its place inside the Code Dojo by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at de8fe5bc:

- aggregate Code Dojo debt: 1,160

- ESLint findings: 752
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,110 or lower

Selected cleanup slice:

- examples/_shared/canonical-app.ts
- packages/bijou-i18n-tools/src/exchange.ts
- packages/bijou-i18n-tools/src/string-table.ts
- packages/bijou-tui/src/eventbus.ts
- packages/bijou/src/core/block-tree-render.ts
- packages/bijou/src/core/components/markdown.ts
- packages/bijou/src/core/components/markdown.test.ts
- packages/bijou/src/core/components/table.ts
- packages/bijou/src/core/components/table.test.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,105
- ESLint findings: 697
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,055 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,110 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, and fake async shapes with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.

5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,110 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 1,060 or lower.

232

WF-152 Respecting the Dojo Ratchet To 1055

docs/design/WF-152-respecting-dojo-ratchet-1055.md

Status

Implemented.

Tracker

- GitHub Issue: #409
- Pull Request: #410

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou keeps earning its place inside the Code Dojo by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost is not a release version. It is:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at f46ab5b7:

- aggregate Code Dojo debt: 1,105

- ESLint findings: 697
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,055 or lower

Selected cleanup slice:

- examples/app-frame/main.ts
- examples/showcase/app.ts
- packages/bijou-node/src/style.ts
- packages/bijou-tui/src/css/resolver.ts
- packages/bijou-tui/src/layout-preset.ts
- packages/bijou-tui/src/pipeline/middleware/surface-shaders.ts
- packages/bijou-tui/src/raster-glyph.ts
- packages/bijou-tui/src/types.test.ts
- packages/bijou/src/core/forms/filter-interactive.ts
- packages/bijou/src/core/standard-blocks/render.ts
- scripts/workflow-shell-preflight.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,054
- ESLint findings: 646
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,004 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.

- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,055 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.

3. Replace unsafe assertions, non-null assertions, implicit coercions, and fake async shapes with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,055 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 1,004 or lower.

233

WF-153 Respecting the Dojo Ratchet To 1004

docs/design/WF-153-respecting-dojo-ratchet-1004.md

Status

Validated locally.

Tracker

- GitHub Issue: #411
- Pull Request: #412

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou keeps earning its place inside the Code Dojo by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost remains:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at cc54799a:

- aggregate Code Dojo debt: 1,054

- ESLint findings: 646
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 1,004 or lower

Selected cleanup slice:

- tests/cycles/DF-062/notification-system-family-audit.test.ts
- tests/cycles/DF-068/dogfood-block-preview-regressions.test.ts
- packages/bijou-tui/src/runtime-interactive-resize.test.ts
- packages/bijou-tui/src/app-frame-core.test.ts
- packages/bijou-tui/src/app-frame-overlays.test.ts
- packages/bijou-tui/src/split-pane.test.ts
- packages/bijou-tui/src/pager.ts
- packages/bijou/src/adapters/test/io.ts
- packages/bijou/src/core/forms/form-utils.ts
- packages/bijou/src/core/binding-frame-update.ts
- packages/bijou/src/core/components/dag-edges.ts

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 1,003
- ESLint findings: 595
- file/context baseline: 331, with touched baselines lowered where files shrank
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 953 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.

- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 1,004 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.

2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, fake async shapes, and nondeterministic test clocks with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 1,004 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 953 or lower.

234

WF-154 Respecting the Dojo Ratchet To 953

docs/design/WF-154-respecting-dojo-ratchet-953.md

Status

Implemented.

Tracker

- GitHub Issue: #413
- Pull Request: #414

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou keeps earning its place inside the Code Dojo by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost remains:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at 212cd006:

- aggregate Code Dojo debt: 1,003

- ESLint findings: 595
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,766
- next aggregate target: 953 or lower

Selected cleanup slice:

- Small runtime, test, script, and example files with fake async callbacks, unsafe test payload assertions, unchecked array reads, control-regex construction, non-null assertions, numeric template interpolation, and raw DOGFOOD terminal guard strings.

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 951
- ESLint findings: 543
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- DOGFOOD raw-string debt: 2,761
- next aggregate target: 901 or lower

Scope

- Select a focused cleanup slice from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 953 or lower?

2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, fake async shapes, and nondeterministic test clocks with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 953 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 903 or lower.

235

WF-155 Respecting the Dojo Ratchet To 624

docs/design/WF-155-respecting-dojo-ratchet-624.md

Status

In progress.

Tracker

- GitHub Issue: #423
- Pull Request: #424

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost remains:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at 5fa0aea9:

- aggregate Code Dojo debt: 674

- ESLint findings: 266
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 624 or lower

Implemented counts on this branch before review:

- aggregate Code Dojo debt: 623
- ESLint findings: 215
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 573 or lower

Top current ESLint offender clusters:

- examples/docs/storybook-app.ts: 12
- packages/bijou-i18n/src/runtime.ts: 9
- examples/docs/counter-block-demo.ts: 6
- scripts/release-metadata.ts: 6
- scripts/release-readiness.ts: 6
- examples/docs/storybook-workstation.ts: 5
- packages/bijou-tui/src/runtime-engine.ts: 5

Selected cleanup slice:

- examples/docs/storybook-app.ts
- examples/docs/storybook-workstation.ts
- examples/docs/counter-block-demo.ts
- packages/bijou-i18n/src/localization.test.ts
- packages/bijou-tui-app/src/index.ts
- packages/bijou-tui-app/src/index.test.ts
- packages/bijou-tui/src/collection-surface.ts
- packages/bijou-tui/src/focus-area.test.ts
- packages/bijou/src/core/components/surface-primitives.test.ts
- examples/split-editors/main.ts
- packages/bijou-node/src/io.ts

The DOGFOOD app entrypoint remains oversized after the previous split:

- examples/docs/app.ts: 4,165 physical lines / 147,799 bytes

This branch keeps cutting that file while preserving visible docs behavior:

- examples/docs/app.ts: 3,306 physical lines / 114,890 bytes
- extracted standard block docs, block preview surfaces, and Theme Inspector state into focused sub-150-line modules
- DOGFOOD raw-string debt: 2,373, down from the 2,415 baseline

Scope

- Select focused cleanup clusters from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Continue shrinking DOGFOOD app file pressure when it contributes to the same standards objective.
- Keep touched file/context, mock-ban, code-size, and DOGFOOD localization ratchets flat or lower.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No baseline increases.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 624 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did touched file/context, mock-ban, code-size, and DOGFOOD localization ceilings stay flat or lower?
4. Did focused tests for touched behavior pass?
5. Did any DOGFOOD app extraction preserve the visible docs behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, fake async shapes, nondeterministic clocks, and overly large local modules with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 623 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 573 or lower.

236

WF-156 Respecting the Dojo Ratchet To 573

docs/design/WF-156-respecting-dojo-ratchet-573.md

Status

Implemented locally.

Tracker

- GitHub Issue: #425
- Pull Request: #426

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without increasing any other ratchet.

The current goalpost remains:

> Respectful Repo: Enter the Code Dojo

Current Truth

Live counts on main at 37cd5577:

- aggregate Code Dojo debt: 623

- ESLint findings: 215
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 573 or lower

Top current ESLint offender clusters:

- packages/bijou-i18n/src/runtime.ts: 9
- scripts/release-metadata.ts: 6
- scripts/release-readiness.ts: 6
- packages/bijou-tui/src/runtime-engine.ts: 5
- packages/bijou-tui/src/app-frame.test-support.ts: 4
- packages/bijou-tui/src/notification.test.ts: 4
- packages/bijou/src/core/block-tree-render.test.ts: 4
- packages/bijou/src/core/text/grapheme.ts: 4
- packages/create-bijou-tui-app/src/cli.test.ts: 4
- scripts/pr-review-status.ts: 4
- scripts/smoke-canaries.ts: 4

The DOGFOOD app entrypoint remains oversized after the WF-155 split:

- examples/docs/app.ts: 3,306 physical lines / 114,890 bytes
- DOGFOOD raw-string debt: 2,373

Implementation Notes

WF-156 removes 53 live ESLint findings, lowering the ESLint baseline from 215 to 162 and the aggregate Code Dojo debt from 623 to 570.

Touched cleanup clusters:

- TUI runtime, app-frame test support, notifications, browsable list, DAG pane, and string layout helpers.
- Core block-tree rendering, grapheme sanitization, and timer spoken formatting.
- Create-app CLI package parsing and npm pack output parsing.
- Release readiness tracker parsing, release metadata tag handling, and smoke canary manifest/npm-pack/env handling.

Per the operator’s temporary waiver for touched-file size ratchets during this slice, existing file/context and code-size entries for touched legacy-large files were refreshed. The counted file/context and code-size totals do not grow.

Known remaining top ESLint clusters after this pass include i18n runtime, PR review status, i18n filesystem/localization helpers, MCP docs tooling, and small TUI/core tests. The next goalpost target is 520 aggregate violations or lower.

Scope

- Select focused cleanup clusters from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Continue shrinking oversized files when that contributes to the same standards objective.
- Keep counted file/context, mock-ban, code-size, and DOGFOOD localization totals flat or lower. If the operator explicitly waives touched-file size ratchets, record any refreshed legacy-large ceilings in this design doc.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No new file/context, mock-ban, or code-size entries.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 573 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did counted file/context, mock-ban, code-size, and DOGFOOD localization totals stay flat or lower, and were any waived size-ceiling refreshes recorded?
4. Did focused tests for touched behavior pass?
5. Did any large-file extraction preserve visible or API behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, nondeterministic clocks, fake async shapes, and oversized local modules with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:slice -- <touched TypeScript files>.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50; WF-156 removed 53.
- Aggregate Code Dojo debt is 573 or lower; WF-156 records 570.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 520 or lower.

237

WF-157 Respecting the Dojo Ratchet To 520

docs/design/WF-157-respecting-dojo-ratchet-520.md

Status

Implemented.

Tracker

- GitHub Issue: #427
- Pull Request: #428

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without losing the Respectful Repo: Enter the Code Dojo direction.

Current Truth

Live counts on main at b6ea3474:

- aggregate Code Dojo debt: 570
- ESLint findings: 162
- file/context baseline: 331
- mock-ban baseline: 22

- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 520 or lower

Top current ESLint offender clusters:

- packages/bijou-i18n/src/runtime.ts: 9
- scripts/pr-review-status.ts: 4
- examples/_stories/protocol.ts: 3
- packages/bijou-i18n-tools-node/src/filesystem.ts: 3
- packages/bijou-i18n/src/localization.ts: 3
- packages/bijou-mcp/src/tools/docs.ts: 3
- packages/bijou-tui/src/keybindings.test.ts: 3
- packages/bijou-tui/src/overlay-drawer.test.ts: 3
- packages/bijou-tui/src/runtime.test-support.ts: 3
- packages/bijou-tui/src/subapp/mount.ts: 3
- packages/bijou/src/core/binding-frame-update.test.ts: 3
- tests/cycles/DF-069/dogfood-block-registry.test.ts: 3

The DOGFOOD app entrypoint remains oversized after the WF-156 merge:

- examples/docs/app.ts: 3,306 physical lines / 114,890 bytes
- DOGFOOD raw-string debt: 2,373

Implementation Notes

WF-157 removes 52 live type-aware ESLint findings, lowering the ESLint baseline from 162 to 110 and the aggregate Code Dojo ceiling from 570 to 518.

Cleanup clusters:

- i18n runtime-adjacent boundaries: story protocol index guards, catalog filesystem parsing, authoring-tool record handling, and localized value clone/freeze validation.
- MCP docs examples: replace docs-only renderer assertions with local JSON-shape coercion helpers for guided flow, preference list, and stats panel examples.
- TUI/core tests and helpers: keybinding/action visibility checks, drawer invalid-input coverage, runtime support typing, subapp command mapping, binding-frame invalid input coverage, DOGFOOD block registry invalid-input and immutability checks, flex/pager numeric formatting, navigable-table repeat loops, notification row guards, motion reconciliation narrowing, and split-pane grapheme guards.
- Examples: mark top-level app launches as intentionally floated with void.

Per the operator’s temporary waiver for touched-file size ratchets, existing file/context entries for touched legacy-large files were refreshed to their current measured line/byte counts. No new counted file/context, mock-ban, or code-size entries were added.

Scope

- Select focused cleanup clusters from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Continue shrinking oversized files when that contributes to the same standards objective.
- Keep counted file/context, mock-ban, code-size, and DOGFOOD localization totals flat or lower. If the operator explicitly waives touched-file size ratchets, record any refreshed legacy-large ceilings in this design doc.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No new file/context, mock-ban, or code-size entries.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 520 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did counted file/context, mock-ban, code-size, and DOGFOOD localization totals stay flat or lower, and were any waived size-ceiling refreshes recorded?
4. Did focused tests for touched behavior pass?
5. Did any large-file extraction preserve visible or API behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select a slice that can remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, nondeterministic clocks, fake async shapes, and oversized local modules with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:changed.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 520 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 468 or lower.

238

WF-158 Respecting the Dojo Ratchet To 468

docs/design/WF-158-respecting-dojo-ratchet-468.md

Status

Implemented.

Tracker

- GitHub Issue: #429
- Pull Request: #430

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without weakening the Respectful Repo: Enter the Code Dojo direction.

Current Truth

Live counts on main at e075080e:

- aggregate Code Dojo debt: 518
- ESLint findings: 110
- file/context baseline: 331
- mock-ban baseline: 22

- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 468 or lower

Top current ESLint offender clusters:

- packages/bijou-i18n/src/runtime.ts: 9
- scripts/pr-review-status.ts: 4
- packages/bijou/src/core/components/box.ts: 2
- packages/bijou/src/core/components/dag.ts: 2
- packages/bijou/src/core/components/progress.test.ts: 2
- packages/bijou/src/core/forms/forms.fuzz.test.ts: 2
- packages/bijou/src/core/forms/group.ts: 2
- packages/bijou/src/core/graphql-bijou-block.test.ts: 2
- packages/bijou/src/core/selection.ts: 2
- packages/bijou/src/core/text/wrap.ts: 2
- packages/bijou/src/core/theme/index.ts: 2
- packages/bijou/src/core/theme/resolve.test.ts: 2
- packages/bijou/src/core/theme/resolve.ts: 2
- packages/bijou/src/index.ts: 2
- scripts/design-system-docs-preflight.ts: 2
- tests/cycles/DF-070/dogfood-block-product-polish.test.ts: 2
- tests/cycles/DF-071/dogfood-block-authored-surfaces.test.ts: 2
- tests/cycles/DX-034/data-binding-closeout.test.ts: 2

The DOGFOOD app entrypoint remains oversized after the WF-157 merge:

- examples/docs/app.ts: 3,306 physical lines / 114,890 bytes
- DOGFOOD raw-string debt: 2,373

Implementation Notes

WF-158 removes 53 live type-aware ESLint findings, lowering the ESLint baseline from 110 to 57 and the aggregate Code Dojo ceiling from 518 to 465.

Cleanup clusters:

- Core forms and tests: preserve empty-answer default semantics while replacing unsafe fallback chains, add explicit numeric formatting in fuzz fixtures, and remove control-character regex literals from prompt/progress tests.
- DOGFOOD and cycle tests: replace visible frame-text fallbacks with nullish checks, remove unnecessary conversions, and keep rendered text assertions behaviorally equivalent.
- Bench and smoke helpers: type GC sample arrays, make JSONL parsing return u nknown, document intentional no-op sinks, and remove needless async wrappers.
- TUI helpers: replace non-null assertions with checked reads in command palette, file picker, panel focus, input stack, status bar, cell glyph, CSS parser, and app-frame layout traversal.

- Core components: remove unnecessary fallbacks from already-required string inputs, unify DAG overloads, and keep runtime viewport overlay typing direct.

Per the operator's temporary waiver for touched-file size ratchets, existing file/context entries for touched legacy-large files were refreshed to their current measured line/byte counts. No new counted file/context, mock-ban, or code-size entries were added.

Scope

- Select focused cleanup clusters from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Continue shrinking oversized files when that contributes to the same standards objective.
- Keep counted file/context, mock-ban, code-size, and DOGFOOD localization totals flat or lower. If the operator explicitly waives touched-file size ratchets, record any refreshed legacy-large ceilings in this design doc.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No new file/context, mock-ban, or code-size entries.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 468 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did counted file/context, mock-ban, code-size, and DOGFOOD localization totals stay flat or lower, and were any waived size-ceiling refreshes recorded?
4. Did focused tests for touched behavior pass?
5. Did any large-file extraction preserve visible or API behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select enough scattered cleanup to remove at least 50 findings.
2. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
3. Replace unsafe assertions, non-null assertions, implicit coercions, nondeterministic clocks, fake async shapes, deprecated exports, and oversized local modules with typed helpers and explicit control flow.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:changed.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 468 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 415 or lower.

239

WF-159 Respecting the Dojo Ratchet To 415

docs/design/WF-159-respecting-dojo-ratchet-415.md

Status

Implemented.

Tracker

- GitHub Issue: #431
- Pull Request: #432

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without weakening the Respectful Repo: Enter the Code Dojo direction.

Current Truth

Live counts on main at 587776d7:

- aggregate Code Dojo debt: 465
- ESLint findings: 57
- file/context baseline: 331
- mock-ban baseline: 22

- code-size baseline: 55, including 4 legacy hard-limit files
- next aggregate target: 415 or lower

Top current ESLint offender clusters:

- packages/bijou-i18n/src/runtime.ts: 9
- scripts/pr-review-status.ts: 4
- packages/bijou/src/core/forms/group.ts: 2
- packages/bijou/src/core/graphql-bijou-block.test.ts: 2
- packages/bijou/src/core/selection.ts: 2
- packages/bijou/src/core/theme/index.ts: 2
- packages/bijou/src/core/theme/resolve.test.ts: 2
- packages/bijou/src/core/theme/resolve.ts: 2
- packages/bijou/src/index.ts: 2
- scripts/design-system-docs-preflight.ts: 2
- tests/cycles/DF-070/dogfood-block-product-polish.test.ts: 2

The remaining lint debt is small enough to attempt a full ESLint clearout in this cycle before moving into file/context and code-size debt.

Implementation Notes

WF-159 removes 51 live type-aware ESLint findings, lowering the ESLint baseline from 57 to 6 and the aggregate Code Dojo ceiling from 465 to 414.

Cleanup clusters:

- i18n runtime and localization port: remove unchecked caller-selected generic value claims from the runtime boundary, return unknown catalog data until a caller narrows it, and make interpolation avoid implicit object stringification.
- Tests and fixtures: replace scattered non-null assertions, unsafe matcher assignments, malformed active-binding input casts, fuzz fixture casts, and deterministic clock conditionals with typed guards or explicit runtime invocation.
- Theme and TUI helpers: make browsable-list render options generic, resolve theme status tokens through map lookups, preserve deprecated helper exports through star re-exports, and avoid unsafe token-graph casts.
- Script and parser utilities: validate package metadata JSON, parse design-system headings/fields with checked captures, and keep PR-status helper debt isolated for a later split.

Per the operator’s temporary waiver for touched-file size ratchets, existing file/context entries for touched legacy-large files were refreshed to their current measured line/byte counts, and existing code-size ceilings for the touched i18n runtime files were refreshed as well. No new counted file/context, mock-ban, or code-size entries were added.

Scope

- Select focused cleanup clusters from current ESLint or Code Dojo offenders.
- Prefer real type, data-shape, deterministic-test, and control-flow fixes over suppressions or cosmetic baseline churn.
- Continue shrinking oversized files when that contributes to the same standards objective.
- Keep counted file/context, mock-ban, code-size, and DOGFOOD localization totals flat or lower. If the operator explicitly waives touched-file size ratchets, record any refreshed legacy-large ceilings in this design doc.
- Update scripts/code-dojo/baselines/eslint.json, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No new file/context, mock-ban, or code-size entries.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 415 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did counted file/context, mock-ban, code-size, and DOGFOOD localization totals stay flat or lower, and were any waived size-ceiling refreshes recorded?
4. Did focused tests for touched behavior pass?
5. Did any large-file extraction preserve visible or API behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Use npm run code-dojo:eslint:offenders and focused ESLint probes to select enough cleanup to remove at least 50 findings.
2. Prioritize zero remaining ESLint findings if the fixes stay behaviorally tight.
3. Write focused regressions only where cleanup exposes a behavior or module boundary risk.
4. Ratchet the ESLint baseline and aggregate Code Dojo ceiling after the live count is proven lower.
5. Update docs and changelog with final counts and the next target.

Tests To Write First

- Add focused regressions for any behavior-bearing cleanup.
- For pure typing cleanup, run focused ESLint and existing tests for the touched package or surface before updating baselines.

Validation Plan

- Probe candidate files with npm run code-dojo:eslint:offenders.
- Run focused raw ESLint on touched files.
- Run npm run code-dojo:changed.
- Run focused behavior tests for touched surfaces.
- Check aggregate debt with npm run code-dojo:debt.
- Confirm standards gates with npm run code-dojo:verify.
- Validate documentation inventory with npm run docs:inventory.
- Run npm run lint and git diff --check.

Acceptance Criteria

- The selected touched files reduce live findings by at least 50.
- Aggregate Code Dojo debt is 414 or lower.
- The ESLint baseline records the lower live count.
- The Code Dojo exception ledger and package.json report the lower ceiling.
- The next ratchet target is documented as 364 or lower.

240

WF-160 Respecting the Dojo Ratchet To 364

docs/design/WF-160-respecting-dojo-ratchet-364.md

Status

Implemented.

Tracker

- GitHub Issue: #433
- Pull Request: #434

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Bijou earns the next Code Dojo checkpoint by removing at least 50 counted standards violations from the landed main ceiling without weakening the Respectful Repo: Enter the Code Dojo direction.

Current Truth

Live counts on main at 976bd1b7:

- aggregate Code Dojo debt: 414
- file/context baseline: 331
- mock-ban baseline: 22
- code-size baseline: 55, including 4 legacy hard-limit files

- ESLint baseline: 6
- next aggregate target: 364 or lower

The remaining ESLint bucket is too small to carry a 50-count slice by itself. This cycle must therefore remove a blended set of final ESLint findings, mock-ban violations, and file/context or code-size baseline entries.

Current ESLint offenders:

- scripts/pr-review-status.ts: 4
- examples/docs/coverage.ts: 1
- packages/bijou/src/core/schema-block.test.ts: 1

Current mock-ban offender buckets:

- packages/bijou-node/src/index.test.ts: 11
- packages/bijou-node/src/io.test.ts: 6
- packages/create-bijou-tui-app/src/cli.test.ts: 4
- packages/bijou-tui/src/pipeline/pipeline.test.ts: 1

Near-threshold file/context candidates include files from 152 to 170 lines, where small focused splits can remove baseline entries without creating new large-file debt. Candidate clusters include core component helpers, TUI layout tests, small examples, MCP docs tests, and cycle proof tests.

Scope

- Remove the final live ESLint findings where the fixes are mechanically defensible.
- Replace existing test mocks and spies with observable fake adapters, environment restore helpers, or behavior assertions.
- Remove enough file/context or code-size baseline entries to reach 364 or lower.
- Prefer splitting cohesive helper/test fixtures into sub-150-line modules over compressing code to game line counts.
- Update scripts/code-dojo/baselines/eslint.json, scripts/code-dojo/baselines/mock-ban.json, scripts/code-dojo/baselines/file-context.json, scripts/code-size-gate.ts, package.json, docs/code-dojo-exceptions.md, and docs/CHANGELOG.md after the live count is proven lower.

Non-Goals

- No new file/context, mock-ban, or code-size baseline entries.
- No release-version scope.
- No broad feature work.
- No rebase, amend, force push, or draft PR.

Playback Questions

1. Is aggregate Code Dojo debt at 364 or lower?
2. Did the implementation remove at least 50 real counted violations?
3. Did the final ESLint and mock-ban buckets shrink, ideally to zero?
4. Did file/context and code-size totals stay flat or lower with no new entries?
5. Did focused tests for touched behavior pass?
6. Did any file extraction preserve visible or API behavior?

Accessibility And Assistive Posture

This is standards cleanup. Any touched rendered or terminal output must preserve existing text, focus, lower-mode, and screen-reader semantics unless a test explicitly proves an accessibility improvement.

Localization And Directionality Posture

Avoid DOGFOOD copy edits unless the slice intentionally updates localization catalogs. If DOGFOOD-facing strings are touched, run the DOGFOOD i18n gates and hold the raw-string and missing-localization ratchets flat or lower.

Agent Inspectability And Explainability Posture

The selected files, rule deltas, and validation commands must be captured in this design doc or the PR summary so the next agent can audit what changed without re-running broad discovery first.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

Implementation Outline

1. Clear the final six ESLint findings with type-safe fixes and focused tests where runtime behavior is affected.
2. Remove mock-ban entries by replacing spies with deterministic fake writers, explicit process-environment restore helpers, or observable output.
3. Split or simplify enough near-threshold file/context entries to remove the remaining counted debt needed for the 364 target.

4. Recompute live counts, update baselines and docs, and lower the aggregate ceiling in package.json.
5. Run focused tests, code-dojo:changed, code-dojo:verify, docs inventory, workspace lint, and diff checks.

Tests To Write First

- Add focused regressions for behavior-bearing lint or mock-ban cleanup.
- For pure test harness cleanup, prove equivalent behavior through the existing touched test file.
- For file splits, run the touched package or cycle test suite before updating baselines.

Validation Plan

- npm run code-dojo:eslint:offenders -- --limit 40
- Focused raw ESLint on touched files when useful.
- Focused npm run test:run -- <touched tests>
- npm run code-dojo:changed
- npm run code-dojo:debt
- npm run code-dojo:verify
- npm run docs:inventory
- npm run lint
- git diff --check

Acceptance Criteria

- Aggregate Code Dojo debt is 364 or lower.
- At least 50 counted violations are removed from the WF-159 landed ceiling.
- The final counts and next target are recorded in the exception ledger.
- The package-script debt ceiling is lowered to the proven count.
- No Dojo bucket grows and no new baseline entry is added.

Implementation Notes

WF-160 removes the final six ESLint findings, clears all 22 mock-ban violations, and removes 22 file/context baseline entries. The resulting aggregate debt is exactly 364:

- file/context baseline: 309
- mock-ban baseline: 0
- code-size baseline: 55, including 4 legacy hard-limit files
- ESLint baseline: 0

- next aggregate target: 314 or lower

The mock-ban cleanup adds deterministic writer/platform seams for Node IO and create-bijou-tui-app, plus an explicit plan-rebuild observer for the TUI pipeline. The final ESLint cleanup removes unsafe or caller-chosen type assertions from coverage parsing, schema-block tests, and PR review status parsing. File/context cleanup is structural: oversized type clusters and test fixtures are split into focused support modules while preserving behavior.

Validation Evidence

- npm run code-dojo:changed
- npm run code-dojo:debt -- --json
- npm run code-dojo:verify
- npm run dogfood:i18n:debt
- npm run docs:inventory
- npm run typecheck:test
- npm run lint
- git diff --check
- focused Vitest suites for the touched Node IO, create-app CLI, pipeline, schema-block, PR review status, form, layout, TUI, MCP, theme, and cycle proof files

241

WF-161 Respecting the Dojo Ratchet 262

docs/design/WF-161-respecting-dojo-ratchet-262.md

Decision Summary

Cut 100 counted Code Dojo debt items from the merged 362 baseline and land the next Respecting the Dojo goalpost at 262 aggregate violations or lower.

Hill

The repository has zero ESLint and mock-ban debt, but it still carries broad file/context and code-size debt. This cycle removes a large chunk of that debt without weakening the standards: oversized test/spec files are decomposed into threshold-sized deterministic specs, and the ledgers are ratcheted down to the live count.

Current Truth

- Aggregate Code Dojo debt: 362
- File/context baseline: 308
- Mock-ban baseline: 0
- Code-size baseline: 54, including 3 hard-limit files
- ESLint baseline: 0
- Required target for this cycle: 262 aggregate violations or lower
- Tracker issue: #437

Scope

- Split 91 over-threshold test/spec files into grouped sub-spec files that each stay under 150 lines and 12,000 bytes.
- Remove those original files from the file/context baseline.
- Remove the 9 split test/spec files that were also over the 500 line code-size ratchet from the code-size baseline.

- Lower package.json code-dojo:debt --max to the measured live count.
- Update docs/code-dojo-exceptions.md and docs/CHANGELOG.md.
- Preserve every assertion and public behavior; this is a decomposition pass, not a semantic rewrite.

Non-Goals

- No production API behavior changes.
- No release-version work.
- No rebase, amend, force push, or draft PR.
- No large-file marker escape hatches and no standards carveout expansion.

Implementation Plan

1. Add deterministic splitting support as a one-time mechanical transform for eligible test files:
   - keep imports and helper declarations;
   - group adjacent it/test/nested describe items under the same parent describe;
   - write each generated spec below file/context limits;
   - delete the original oversized spec file.
2. Run the focused split specs first to prove assertion parity.
3. Recompute Code Dojo file/context and code-size ledgers.
4. Lower the aggregate code-dojo:debt ceiling to 262 or lower.
5. Run the full local validation lane before opening the PR for final review.

Candidate Set

The selected set removes exactly 100 aggregate items:

- 91 file/context entries from oversized test/spec files.
- 9 matching code-size entries from split test/spec files that were also over 500 lines:
  - packages/bijou-tui/src/runtime-engine.test.ts
  - packages/bijou/src/core/binding.test.ts
  - packages/bijou-tui/src/notification.test.ts
  - packages/bijou-tui/src/eventbus.test.ts
  - packages/bijou-tui/src/transition-shaders.test.ts
  - packages/bijou-i18n/src/runtime.test.ts
  - packages/bijou/src/core/theme/dtcg.test.ts
  - packages/bijou-tui/src/focus-area.test.ts
  - tests/cycles/DF-069/dogfood-block-registry.test.ts

Tests And Validation

- Focused Vitest run for all generated split specs.
- npm run code-dojo:changed
- npm run code-dojo:verify
- npm run typecheck:test
- npm run lint
- npm run docs:inventory
- git diff --check
- Full pre-push verification before final PR handoff.

Acceptance Criteria

- Aggregate Code Dojo debt is 262 or lower.
- File/context baseline is 217 or lower.
- Code-size baseline is 45 or lower.
- Mock-ban and ESLint baselines remain 0.
- No generated split spec exceeds 150 lines or 12,000 bytes.
- The full local and GitHub validation gates pass.

Playback Questions

1. Did the aggregate debt fall by at least 100?
2. Did every split spec preserve its original assertions?
3. Did the split avoid creating new file/context or code-size violations?
4. Did the ledgers and ceiling match the measured live count?
5. Did the PR stay non-draft and linked to issue #437?

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

242

WF-162 Respecting the Dojo Ratchet 162

docs/design/WF-162-respecting-dojo-ratchet-162.md

Decision Summary

Cut 100 counted Code Dojo debt items from the merged 262 baseline and land the next Respecting the Dojo checkpoint at 162 aggregate violations or lower.

Hill

WF-161 exhausted most of the low-risk oversized test/spec split pool. The repo still carries structural debt in file/context and code-size ledgers, so this cycle has to combine remaining deterministic test decomposition with production, tooling, and example module extraction.

The goal is not cosmetic movement. The resulting ledgers must describe less debt because large files were split into smaller, coherent modules without weakening public behavior, tests, or standards.

Current Truth

- Aggregate Code Dojo debt: 262
- File/context baseline: 217
- Mock-ban baseline: 0
- Code-size baseline: 45, including 3 hard-limit files
- ESLint baseline: 0
- Required target for this cycle: 162 aggregate violations or lower
- Tracker issue: #439

Scope

- Split the remaining over-threshold deterministic test/spec files that are safe to decompose.

- Extract enough production, tooling, and example modules to remove the remaining counted debt needed for a 100 item aggregate reduction.
- Keep original module entrypoints as compatibility facades when public import paths are part of the package surface.
- Refresh Code Dojo file/context and code-size ledgers from live files.
- Lower package.json code-dojo:debt --max to the measured live count.
- Update docs/code-dojo-exceptions.md and docs/CHANGELOG.md.

Non-Goals

- No product feature work.
- No release-version work.
- No behavior/API breakage.
- No rebase, amend, force push, or draft PR.
- No large-file marker escape hatches and no standards carveout expansion.

Implementation Strategy

1. **Finish the deterministic test split pool.**
   - Split the remaining over-threshold test/spec files into focused threshold-sized specs.
   - Preserve all assertions and snapshots.
   - Prefer semantic describe-group boundaries over arbitrary line cuts.
2. **Extract high-yield code-size modules.**
   - Target modules where one extraction removes both a file/context entry and a code-size entry.
   - Keep stable public files as facades that re-export or delegate to focused internals.
   - Avoid broad rewrites inside domain logic unless tests pin the behavior.
3. **Extract file/context-only modules when needed.**
   - Target self-contained data, fixtures, option tables, test support helpers, parser/render helper clusters, and CLI workflow helpers.
   - Keep each new file below 150 lines and 12,000 bytes.
4. **Ratcheting and proof.**
   - Regenerate the file/context baseline from live files.
   - Remove code-size entries only when live files are below the ratchet limit or have been replaced by smaller modules.
   - Lower the aggregate debt ceiling to 162 or lower.

Implementation Outcome

The cycle lands exactly at 162 aggregate Code Dojo violations:

- File/context baseline: 136
- Mock-ban baseline: 0
- Code-size baseline: 26, including 3 hard-limit files
- ESLint baseline: 0

The reduction combines:

- Remaining deterministic test and test-support splits, including app-frame, runtime-interactive, textarea, PTY-driver, DOGFOOD light-theme, docs-preview, and test adapter support files.
- Declaration-boundary module extraction for standard-blocks, theme helpers, packed-cell, component metadata, TUI navigation/layout helpers, transition shaders, block metadata, schema blocks, selection, binding lifecycle, DAG pane, focus area, raster glyphs, layout envelopes, overlays, GraphQL blocks, viewport, binding, and notifications.
- Compatibility facades at the original import paths so public consumers keep stable module entrypoints while implementations move into smaller part files.

Candidate Set

The starting debt pool is:

- 43 remaining test-ish file/context entries, including 3 code-size entries.
- 174 production/tooling/example file/context entries.
- 45 code-size entries total.

Expected reduction mix:

- About 43 file/context entries from the remaining deterministic test/spec split pool.
- At least 57 additional aggregate entries from production, tooling, example, or support-module extractions.

The exact final candidate set can adjust during implementation if a file proves unsafe to split mechanically, but the live aggregate target must stay at 162 or lower.

Tests And Validation

- Focused Vitest run for generated split specs and touched module facades.
- npm run code-dojo:changed
- npm run code-dojo:verify
- npm run typecheck:test
- npm run lint
- npm run docs:inventory
- git diff --check
- Full pre-push verification before final PR handoff.

Acceptance Criteria

- Aggregate Code Dojo debt is 162 or lower.
- File/context baseline and code-size baseline match measured live repository state.
- Mock-ban and ESLint baselines remain 0.
- No generated file exceeds 150 lines or 12,000 bytes unless it remains in the shrinking ledger.
- Public imports continue to work through stable facades.
- Full local and GitHub validation gates pass.

Playback Questions

1. Did the aggregate debt fall by at least 100?
2. Did every split preserve original assertions and public behavior?
3. Did all ledgers reflect measured live files rather than aspiration?
4. Did the cycle avoid adding new exceptions or widening old ones?
5. Did the PR stay non-draft and linked to issue #439?

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

243

WF-163 Respecting the Dojo Ratchet 112

docs/design/WF-163-respecting-dojo-ratchet-112.md

Legend: WF — Workflow and Delivery

Sponsor Human

James Ross.

Sponsor Agent

Codex.

Decision Summary

Cut at least 50 counted Code Dojo debt items from the merged 162 baseline and land the next Respecting the Dojo checkpoint at 112 aggregate violations or lower.

This is the explicit quality prerequisite for the Profunctor Page inspection target in issue #468.

Hill

The remaining debt is structural. File/context and code-size ledgers identify modules that exceed the repository's bounded-context rules. The cycle must remove real entries by splitting coherent responsibilities while preserving tests, exports, behavior, and deterministic output.

Ledger edits are evidence after implementation. They are not the implementation.

Current Truth

- Aggregate Code Dojo debt before this cycle: 162
- File/context baseline before this cycle: 136
- Mock-ban baseline: 0
- Code-size baseline: 26, including 3 hard-limit files
- ESLint baseline: 0
- Required target: 112 aggregate violations or lower
- Tracker issue: #469
- Downstream issue: #468

Result

- Aggregate Code Dojo debt: 112
- File/context baseline: 86
- Mock-ban baseline: 0
- Code-size baseline: 26, including 3 hard-limit files
- ESLint baseline: 0
- Removed debt: 50 of the required 50 violations
- Next goalpost target: 62 aggregate violations or lower

The cycle preserves public entrypoints and deterministic output while moving coherent responsibilities into focused modules. The resulting baseline is measured from live files; no threshold, exception marker, or rule was weakened.

Scope

- Select the smallest safe set of remaining file/context and code-size offenders.
- Prefer modules where one coherent extraction removes both a file/context entry and a code-size entry.
- Preserve public entrypoints through compatibility facades when import paths are contractual.
- Split tests at semantic describe or fixture-family boundaries.
- Regenerate the file/context and code-size ledgers from measured files.
- Lower package.json code-dojo:debt --max to 112 or lower.
- Update the exception ledger, changelog, roadmap, and closeout evidence.

Non-Goals

- No Profunctor Page adapter implementation.
- No public behavior or API redesign.

- No release publication.
- No new standards exception or large-file marker.
- No weakened threshold, skipped gate, or hand-edited aspirational count.

Implementation Outline

1. Inventory the remaining ledger entries through bounded repository projections.
2. Rank candidates by aggregate reduction, cohesion, and regression coverage.
3. Split deterministic tests and support modules at existing responsibility boundaries.
4. Extract production or tooling helpers only where public behavior is already pinned by focused tests.
5. Run focused tests after each candidate family.
6. Regenerate ledgers only after live files fall below their thresholds.
7. Run the full Code Dojo and repository gates before review.

Tests To Write First

- Add a goalpost regression that binds the aggregate ceiling to the measured file/context ledger.
- Add a roadmap regression that binds the completed prerequisite and next bounded target.
- Add focused regressions before any extraction that changes behavior-bearing seams.

Validation Plan

- Focused Vitest runs for every split test or module family.
- npm run code-dojo:changed
- npm run code-dojo:verify
- npm run typecheck:test
- npm run lint
- npm run docs:inventory
- npm run test:run
- git diff --check

Acceptance Criteria

- Aggregate Code Dojo debt is 112 or lower.
- File/context and code-size ledgers match measured live files.
- Mock-ban and ESLint baselines remain 0.

- No split loses assertions, exports, runtime behavior, or deterministic output.
- No newly created file exceeds 150 lines or 12,000 bytes.
- Only pre-existing baselined paths may remain in the shrinking ledger.
- Full local and GitHub validation gates pass.
- Main at the resulting merge revision unblocks issue #468.

Playback Questions

1. Did aggregate debt fall by at least 50?
2. Does every removed ledger entry correspond to a measured file reduction?
3. Did each split preserve its prior tests and public entrypoints?
4. Did the cycle avoid new exceptions and unrelated product work?
5. Is issue #468 unblocked by merged evidence rather than a local claim?

Accessibility And Assistive Posture

This is a structural standards cycle. Extracted modules must preserve existing text, focus, lower-mode, and assistive semantics. No visual or interaction redesign is in scope.

Localization And Directionality Posture

Extracted modules must preserve catalog keys, localized values, reading order, and direction-sensitive behavior. Any touched DOGFOOD localization surface must keep the completeness and raw-string ratchets flat or lower.

Agent Inspectability And Explainability Posture

Every extraction must retain a stable public facade or an explicit internal call site, focused verification evidence, and ledger-measured closure. The final package ceiling, exception ledger, roadmap, and design result must agree without requiring agents to infer progress from commit count.

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md
- Tracker issue: #469

Retrospective And Closeout

The cycle removed exactly 50 measured file/context violations without adding mock-ban, ESLint, or code-size debt. Compatibility facades preserved public entrypoints while responsibility-bounded modules moved below the enforced thresholds. A separate 13-file MCP catalog precursor kept the final review at the repository’s 140-path limit. The next bounded Code Dojo target is 62; issue #468 is unblocked only after this result reaches main.

244

WF-164 Respecting the Dojo Ratchet 62

docs/design/WF-164-respecting-dojo-ratchet-62.md

Legend: WF — Workflow and Delivery

Sponsor Human

James Ross.

Sponsor Agent

Codex.

Decision Summary

Complete issue #477 by removing the second 25-violation tranche from the 112 Code Dojo checkpoint and lowering the encoded aggregate ceiling to 62.

This is the explicit structural prerequisite for the bounded Profunctor Page inspection story #468.

Hill

DX-050 tranche A lowered measured debt from 112 to 87, but the encoded ceiling deliberately remained at 112 because the repo goalpost policy requires one completed checkpoint to remove at least 50 violations. Tranche B must remove 25 additional counted violations without converting mechanical module boundaries into new import cycles, API drift, or top-level ordering bugs.

Ledger edits are evidence after implementation. They are not the implementation.

Current Truth

- Aggregate Code Dojo debt before tranche B: 87
- File/context baseline before tranche B: 61
- Mock-ban baseline: 0
- Code-size baseline before tranche B: 26, including 3 hard-limit files
- ESLint baseline: 0
- Required target: 62 aggregate violations or lower
- Tracker issue: #477
- Downstream issue: #468

Result

- Aggregate Code Dojo debt: 62
- File/context baseline: 37
- Mock-ban baseline: 0
- Code-size baseline: 25, including 3 hard-limit files
- ESLint baseline: 0
- Removed tranche-B debt: 25 of the required 25 violations
- Removed combined DX-050 debt: 50 of the required 50 violations
- Next goalpost target: 12 aggregate violations or lower

Twenty-four pre-existing oversized entrypoints are stable facades over focused modules. examples/docs/storybook-app.ts leaves both the file/context and code-size ledgers, accounting for the twenty-fifth counted violation. No threshold, exception marker, public export, or runtime-cycle allowance changed.

Scope

- Select 24 pre-existing file/context offenders with bounded top-level responsibilities and existing regression coverage.
- Preserve every selected public entrypoint as a compatibility facade.
- Keep TypeScript overload families in one implementation module.
- Reject extraction candidates whose mutable module state cannot cross a standard ECMAScript module boundary safely.
- Remove examples/docs/storybook-app.ts from the code-size baseline after its focused modules pass the live size gate.
- Lower package.json code-dojo:debt --max to 62.
- Update the exception ledger, changelog, roadmap, bearing, and current tests.

Non-Goals

- No public API redesign.
- No product behavior change.
- No Profunctor Page adapter implementation.
- No release tag or publication.
- No new standards exception or large-file marker.
- No weakened threshold, skipped gate, or aspirational ledger edit.

Implementation Outline

1. Bind the exact tranche-B root set in a deterministic regression.
2. Split declarations along dependency-aware top-level boundaries.
3. Keep overload declarations and implementations co-located.
4. Preserve public import paths through typed re-export facades.
5. Run focused typecheck, lint, behavior, size, and runtime-cycle checks.
6. Remove ledger entries only after every extracted file satisfies the live thresholds.
7. Ratchet the package ceiling and documentation to the measured result.
8. Run the full repository gate before review.

Tests To Write First

- Assert that the exact 24 roots leave the file/context ledger.
- Assert that examples/docs/storybook-app.ts leaves the code-size ledger.
- Assert that every extracted production module is at most 150 lines and 12,000 bytes.
- Assert that no tranche-B module participates in a repository runtime import cycle.
- Preserve focused behavioral tests for every touched public entrypoint.

Validation Plan

- npx vitest run tests/cycles/DX-050/code-dojo-tranche-b.test.ts
- npx vitest run tests/cycles/DX-050/split-modules-tranche-b-remain-acyclic.test.ts
- npm run code-dojo:verify
- npm run typecheck
- npm run lint
- npm run lint:eslint
- npm run docs:inventory
- npm run test:run
- git diff --check

Acceptance Criteria

- Aggregate Code Dojo debt is exactly 62.
- File/context and code-size ledgers match measured live files.
- Mock-ban and ESLint baselines remain 0.
- Every selected public entrypoint preserves its export names.
- No extracted module participates in a runtime import cycle.
- No newly created file exceeds 150 lines or 12,000 bytes.
- Full local and GitHub validation gates pass.
- Main at the resulting merge revision unblocks issue #468.

Playback Questions

1. Did combined DX-050 work remove all 50 required violations?
2. Does every removed ledger entry correspond to a measured file reduction?
3. Did every compatibility facade preserve its public export names?
4. Are all split families absent from runtime import cycles?
5. Is issue #468 unblocked by merged evidence rather than a local claim?

Accessibility And Assistive Posture

This is a structural standards cycle. Extracted modules preserve current text, focus, lower-mode, and assistive semantics. No visual or interaction redesign is in scope.

Localization And Directionality Posture

Extracted modules preserve catalog keys, localized values, reading order, and direction-sensitive behavior. Split-source debt identity continues to resolve numbered parts to their canonical root.

Agent Inspectability And Explainability Posture

The exact target roots, live threshold checks, runtime-cycle proof, aggregate ceiling, and current documentation agree. A successor can identify inherited facades and focused implementation parts without reconstructing the tranche from commit history.

Linked Invariants

- TypeScript Code Standards

- Code Dojo Exceptions
- Work Doctrine
- Documentation Standards
- Tracker issue: #477

Retrospective And Closeout

The cycle removes exactly 24 file/context entries and one code-size entry. Dependency-aware extraction rejected a Node entrypoint whose mutable initialization guard could not be assigned safely through an imported binding. TypeScript overload families were kept co-located after the first focused typecheck exposed their boundary requirement. The final 62 count completes the 112 -> 62 goalpost, keeps the review at 139 changed paths under the repository’s 140-path boundary, and makes 12 the next policy target.

245

WF-165 Respecting the Dojo Ratchet 12

docs/design/WF-165-respecting-dojo-ratchet-12.md

Legend: WF — Workflow and Delivery

Sponsor Human

James Ross.

Sponsor Agent

Codex.

Decision Summary

Complete issue #480 by removing all 25 files that are counted in both the file/context and code-size ledgers. The combined 50-violation reduction lowers aggregate Code Dojo debt from 62 to 12 or lower.

Deliver the goalpost through bounded review tranches. Each tranche preserves public entrypoints as typed compatibility facades, proves that its extracted module family remains outside runtime import cycles, and lowers only the live ledger entries it actually removes.

Hill

The repository has 37 file/context violations and 25 code-size violations. All 25 code-size offenders are also file/context offenders. Removing those double-counted roots behind stable facades therefore satisfies the next mandatory 50-violation goalpost without weakening a threshold, adding an exception, or changing public behavior.

The first tranche starts with five of the smaller double-counted roots:

- packages/bijou-tui/src/eventbus.ts
- packages/bijou-tui/src/app-frame-actions.ts

- scripts/smoke-all-examples-lib.ts
- packages/bijou-i18n/src/runtime.ts
- packages/bijou-tui/src/timeline.ts

Ledger edits are evidence after extraction. They are not the implementation.

Current Truth

- Aggregate Code Dojo debt: 32
- File/context baseline: 22
- Mock-ban baseline: 0
- Code-size baseline: 10, including 3 hard-limit files
- ESLint baseline: 0
- Required goalpost: 12 aggregate violations or lower
- Tranches A through C: 30 counted violations removed from 15 roots
- Remaining double-counted roots: 10
- Tracker issue: #480
- Previous goalpost: #477
- V8 contract pull: #459

Scope

- Bind the exact 25 double-counted roots in deterministic cycle evidence.
- Deliver those roots through bounded review tranches.
- Start tranche A with the five smaller roots named in this design.
- Preserve every selected public import path and export name.
- Keep mutable module state inside one implementation module unless an explicit port makes cross-module ownership safe.
- Keep overload declarations beside their implementation.
- Preserve source-debt identity across extracted implementation parts.
- Prove every changed split family remains absent from runtime import cycles.
- Remove ledger entries only after the live size gates accept the facade and extracted modules.
- Lower the package debt ceiling only to the measured live result.
- Update the exception ledger, changelog, roadmap, bearing, and cycle evidence in every tranche.

Non-Goals

- No Profunctor Page feature implementation.
- No public API or behavior redesign.
- No opportunistic repair of pre-existing runtime cycles tracked by #476.
- No new standards exception or large-file marker.

- No weakened threshold or aspirational ledger edit.
- No release tag or publication.

Tranche A Contract

Tranche A removes five roots from both counted ledgers:

Root	Existing responsibility boundary
eventbus.ts	Event-bus contracts, command lifecycle, queue diagnostics, and dispatch implementation.
app-frame-actions.ts	Frame action dispatch, pane navigation, viewport state, and transition commands.
smoke-all-examples-lib.ts	Scenario planning, execution, pooling, option parsing, and host adapters.
runtime.ts	I18n contracts, synchronous resolution, catalog composition, and asynchronous loading.
timeline.ts	Timeline contracts, position resolution, builder state, and compiled playback.

The expected tranche result is:

- File/context baseline: 37 -> 32
- Code-size baseline: 25 -> 20
- Aggregate debt: 62 -> 52
- Mock-ban baseline: 0
- ESLint baseline: 0

The overall WF-165 goalpost remains open after tranche A.

Tranche B Contract

Tranche B removes the five smallest remaining roots from both counted ledgers:

Root	Existing responsibility boundary
packages/bijou-tui/src/runtime.ts	Lifecycle contracts, the interactive render loop, shutdown draining, output buffers, and error emission.
examples/perf-gradient/main.ts	Telemetry charts, paint modes, frame assembly, interactive state, and process entry.
packages/bijou-tui/src/driver.ts	Script contracts, mouse-step builders, harness state, command observations, and script execution.
packages/bijou/src/core/components/dag-render.ts	DAG metrics, node glyphs, graph placement, edge highlighting, serialization, and fallback formats.
packages/bijou-tui-app/src/index.ts	Skeleton contracts, shell construction, chrome and overlays, page state and layout, and key-map helpers.

The expected tranche-B result is:

- File/context baseline: 32 -> 27
- Code-size baseline: 20 -> 15
- Aggregate debt: 52 -> 42
- Mock-ban baseline: 0
- ESLint baseline: 0

The selection is mechanical: these are the five smallest remaining entries in the code-size baseline, and every one is also present in the file/context baseline. The tranche does not redesign public behavior, change public exports, add an exception, or introduce cache invalidation. Review hardening corrects bounded runtime scheduling, shutdown, crash, modal, footer, and performance chart defects exposed by the extraction. It also restores pre-split DAG detour width, empty-badge, render-target adoption, deterministic noise, and harness elapsed-time semantics. The overall WF-165 goalpost remains open after tranche B.

Tranche C Contract

Tranche C removes the five smallest remaining roots from both counted ledgers:

Root	Existing responsibility boundary
examples/_shared/canonical-app.ts	Canonical workbench fixtures, page model and messages, pane renderers, page composition, and framed-app construction.
examples/notifications/main.ts	Notification demo contracts, state transitions, overlay geometry, mouse routing, notification lifecycle, and process entry.
examples/image-viewer/main.ts	Image-viewer contracts, file navigation, model updates, preview loading, raster controls, rendering, and process entry.
scripts/pr-review-status.ts	Pull-request and review contracts, check and review summaries, CodeRabbit status, merge readiness, GitHub retrieval, CLI parsing, and process entry.
packages/bijou-tui/src/app-frame-render.ts	Frame-render contracts, recursive layout rendering, pane geometry, header/help lines, transitions, surface normalization, and scratch ownership.

The expected tranche-C result is:

- File/context baseline: 27 -> 22
- Code-size baseline: 15 -> 10
- Aggregate debt: 42 -> 32
- Mock-ban baseline: 0
- ESLint baseline: 0

The selection is mechanical: these are the five smallest remaining entries in the code-size baseline, and every one is also present in the file/context baseline. Each original path remains a stable facade. Example process entry, public exports, deterministic merge-readiness classification, notification

input behavior, image navigation and rendering, and framed-app output remain unchanged. Review evidence additionally binds shared notification geometry, RGB-only active-tab backgrounds, localized default frame titles, grapheme-safe dividers, source-invalidated image-preview caching, and bounded active-tab token caching. The overall WF-165 goalpost remains open after tranche C.

Tranche D Contract

Tranche D removes the five smallest remaining roots from both counted ledgers:

Root	Existing responsibility boundary
packages/bijou/src/ports/surface.ts	Cell and layout contracts, packed-cell encoding, masks, transforms, and mutable packed-surface construction.
examples/docs/i18n-debt.ts	Debt contracts and baselines, TypeScript source discovery, raw-string classification, Markdown localization inventory, and ratchet evaluation.
packages/bijou-tui/src/app-frame-overlays.ts	Help, settings, shell-theme, notification-center, focus, scroll, layout, and drawer-rendering behavior.
packages/bijou/src/core/ui-scene-ir.ts	Scene and receipt contracts, canonical hashing, validation, semantic lowering, cell source maps, and terminal proof.
packages/bijou/src/core/render/differ.ts	ANSI parsing and serialization, layout painting, cell equality, string and packed diffing, byte encoding, and batching.

The expected tranche-D result is:

- File/context baseline: 22 -> 17
- Code-size baseline: 10 -> 5
- Aggregate debt: 32 -> 22
- Mock-ban baseline: 0
- ESLint baseline: 0

The selection is mechanical: these are the five smallest remaining entries in the code-size baseline, and every one is also present in the file/context baseline. Each original path remains a stable facade. Public types, exports, byte layouts, validation diagnostics, hashes, rendering behavior, localization counts, terminal output, and performance-sensitive packed paths remain unchanged. The overall WF-165 goalpost remains open after tranche D.

Tranche E Contract

Tranche E removes the final five roots from the code-size ledger and from their double-counted positions in the file/context ledger:

Root	Existing responsibility boundary
packages/bijou-tui/src/runtime-engine.ts	Runtime contracts, subscriptions, scheduling, input dispatch, rendering, effects, cleanup, and engine lifecycle.
packages/bijou/src/core/components/table.ts	Table contracts, sizing, alignment, overflow, borders, pagination, selection, scrolling, and rendering.
packages/bijou-tui/src/app-frame.ts	Framed-app contracts, state, input, lifecycle, layout, rendering, overlays, notifications, shell themes, and public assembly.
examples/docs/app.ts	DOGFOOD application contracts, configuration, state, runtime wiring, input routing, rendering, and process lifecycle.
examples/docs/stories.ts	DOGFOOD story contracts, metadata, navigation, examples, panels, demonstrations, and story registry assembly.

The expected tranche-E result is:

- File/context baseline: 17 -> 12
- Code-size baseline: 5 -> 0
- Aggregate debt: 22 -> 12
- Mock-ban baseline: 0
- ESLint baseline: 0

The selection is exhaustive: these are the only remaining code-size entries, and every one is also present in the file/context baseline. Each original path remains a stable facade. Public types, exports, runtime lifecycle, table semantics, framed-app behavior, DOGFOOD process behavior, story identity, and rendered output remain unchanged. Tranche E closes the WF-165 62 -> 12 goalpost only after the live ledgers, family bounds, runtime-cycle proof, documentation, and full repository gates agree.

Implementation Outline

1. Add RED evidence for the exact active-tranche roots and target ledgers.
2. Extract cohesive declarations and behavior into focused adjacent modules.
3. Keep each original root as a stable compatibility facade.
4. Preserve behavior through existing focused suites and new export/size witnesses.
5. Prove that changed split families remain outside runtime import cycles.
6. Remove only the five proven roots from the file/context baseline.
7. Record each tranche’s measured debt in the exception ledger and package ceiling.
8. Run documentation upkeep and the complete repository gate before review.
9. Close the goalpost only when all 25 double-counted roots are gone.

Tests To Write First

- Assert that the exact five tranche-A roots leave the file/context ledger.
- Assert that the same five roots leave the code-size report.
- Assert that every tranche-A facade and implementation module is at most 150 lines and 12,000 bytes.
- Assert that every changed production family remains absent from runtime import cycles.
- Preserve focused behavioral tests for each touched public entrypoint.
- Assert that aggregate debt is exactly 52 after tranche A.
- Assert that aggregate debt is exactly 42 after tranche B.
- Assert that the exact five tranche-C roots leave both measured ledgers.
- Assert that every tranche-C facade and implementation module is at most 150 lines and 12,000 bytes.
- Assert that aggregate debt is exactly 32 after tranche C.
- Assert that the exact five tranche-D roots leave both measured ledgers.
- Assert that every tranche-D facade and implementation module is at most 150 lines and 12,000 bytes.
- Assert that aggregate debt is exactly 22 after tranche D.
- Assert that the exact five tranche-E roots leave both measured ledgers.
- Assert that every tranche-E facade and implementation module is at most 150 lines and 12,000 bytes.
- Assert that aggregate debt is exactly 12 after tranche E.

Validation Plan

```
npx vitest run --config vitest.config.ts \
  tests/cycles/DX-050 \
  tests/cycles/RE-036 \
  tests/cycles/WF-130 \
  tests/cycles/WF-163 \
  tests/cycles/WF-164 \
  tests/cycles/WF-165
npm run code-dojo:verify
npm run typecheck
npm run lint
npm run lint:eslint
npm run docs:inventory
npm run test:run
git diff --check
```

Acceptance Criteria

- All 25 double-counted roots leave both measured ledgers.
- Aggregate Code Dojo debt is 12 or lower.
- File/context and code-size ledgers match measured live files.
- Mock-ban and ESLint baselines remain 0.
- Every selected public entrypoint preserves its export names.
- No changed split family participates in a runtime import cycle.

- No newly created file exceeds 150 lines or 12,000 bytes.
- Every tranche remains within the repository review-size policy.
- Full local and GitHub validation gates pass.
- Main at the resulting merge revision satisfies the next goalpost obligation before the V8 contract tracker closes.

Playback Questions

1. Did every removed ledger entry correspond to a measured file reduction?
2. Did every compatibility facade preserve its public export names?
3. Are all changed split families absent from runtime import cycles?
4. Did the tranche preserve behavior instead of moving unbounded code into a differently named file?
5. Does the final combined evidence remove at least 50 counted violations?

Accessibility And Assistive Posture

This is a structural standards cycle. Extracted modules preserve current text, focus, lower-mode, and assistive semantics. No visual or interaction redesign is in scope.

Localization And Directionality Posture

Extracted modules preserve catalog keys, localized values, reading order, and direction-sensitive behavior. The i18n runtime split must not change fallback, interpolation, reference, locale, or direction resolution.

Agent Inspectability And Explainability Posture

The exact target roots, tranche boundaries, live threshold checks, runtime-cycle proof, aggregate ceiling, and current documentation agree. A successor can identify every compatibility facade and implementation part without reconstructing the goalpost from commit history.

Linked Invariants

- TypeScript Code Standards
- Code Dojo Exceptions
- Work Doctrine
- Documentation Standards
- Tracker issue: #480

Retrospective And Closeout

Open. Tranche A landed in #484. Five compatibility facades now delegate to focused contract, state, execution, and adapter modules; all facade exports remain stable, every family file is within the strict context threshold, focused behavior suites pass, and repository runtime-cycle analysis reports no cycle touching a changed family. The live ledgers move from 37 + 25 = 62 to 32 + 20 = 52.

Tranche B landed through #486. It applies the same bounded extraction to the TUI runtime, performance gradient, scripted driver, DAG renderer, and TUI-app skeleton. Their public facades preserve the original exports while explicit family manifests bind the focused implementation modules used for size and runtime-cycle evidence. The review witness also binds behavior that the extraction initially drifted: same-column DAG detours, middleware-replaced render targets, the original noise permutation, empty DAG badges, and initialization-inclusive harness time. The live ledgers move again to 27 + 15 = 42. Fifteen double-counted roots remain before the overall 62 -> 12 goalpost can close.

Tranche C is implemented in #487. The canonical example app, Notifications, Image Viewer, PR review-status tool, and framed-app renderer remain stable facades over focused implementation families. Exact public-export and family manifests bind the compatibility surface, strict context thresholds, and runtime-cycle proof. Review hardening shares compact notification geometry between rendering and hit testing, preserves RGB-only active-tab backgrounds, localizes the default frame title, keeps divider graphemes intact, removes an orphan image-viewer mode message, and adds bounded caches for derived active-tab tokens and rendered image previews. The preview cache binds its entries to the source path, modification time, byte size, viewport, mode, and tuning so a source-file rewrite invalidates stale output. The live ledgers move to 22 + 10 = 32; ten double-counted roots remain before the overall 62 -> 12 goalpost can close.

Tranche D is implemented in #488. Surface, DOGFOOD i18n-debt analysis, framed-app overlays, ui-scene-ir/1, and the terminal differ remain stable public facades over the exact family manifests locked by the tranche contract. Every family member stays within 150 lines and 12,000 bytes, and repository runtime-cycle analysis finds no cycle touching the changed families. The Surface witness preserves allocation-free numeric RGB writes and covers negative source-origin clipping. The DOGFOOD scanner retains 2,347 raw strings and 78 missing Markdown localizations while excluding its own implementation family. The live ledgers now measure 17 + 5 = 22; no public API or rendering redesign entered scope. Five double-counted roots remain for tranche E.

Tranche E is implemented in #489. The TUI runtime, table, framed-app, DOGFOOD application, and DOGFOOD story catalog remain stable public facades over the exact family manifests locked by the tranche contract. Every family member stays within 150 lines and 12,000 bytes, and repository runtime-cycle analysis reports no cycle touching the changed families. The story catalog preserves all 42 ordered story definitions; its typed package identity lowers the split localization lineage from 1,453 to 1,427 raw strings. The live

ledgers reach 12 + 0 = 12, completing the WF-165 62 -> 12 goalpost without weakening a threshold or redesigning a public surface.

246

WF-166 - V8 Dependency Security Closeout

docs/design/WF-166-v8-dependency-security-closeout.md

Status

Implementation proof in progress for issue #482.

Linked Legend

- legend:wf for release workflow readiness.
- legend:dx for deterministic dependency tooling.
- lane:bad-code for the vulnerable resolved dependency graph.

Sponsors

- Human sponsor: James Ross.
- Agent sponsor: Codex.

Decision Summary

Clear every npm advisory from the resolved workspace dependency graph before the v8.0.0 release-preparation cycle begins. Coordinate the now-insufficient Dependabot pull request #467 inside this issue-backed cycle instead of merging a patch level that remains vulnerable.

Hill

Bijou should enter release preparation with a deterministic lockfile that resolves every currently reported package to a patched version, passes the complete Node 20 and Node 22 verification matrix, and produces a zero-advisory npm audit result.

Current Truth

The shaping snapshot on main at 4412ec6dbce947887ed6ea2740ecbad0a66d122e reported seven npm advisories. By the implementation pass on 2026-08-15, orig in/main at 49671510249fc79ed25b4e8f5796b47d2ad2e11d exposed 18 open Dependabot alerts. The carried lockfile attempt also produced four live npm a udit package findings: fast-uri, hono, ip-address, and newly disclosed nanoi d.

The refreshed candidate dependency paths and resolved versions are:

```
@flyingrobots/bijou-mcp
  -> @modelcontextprotocol/sdk@1.30.0
    -> @hono/node-server@2.0.12 -> hono@4.13.2
    -> ajv@8.18.0 -> fast-uri@3.1.5
    -> express-rate-limit@8.5.2 -> ip-address@10.5.0
    -> express@5.2.1 -> body-parser@2.3.0
    -> hono@4.13.2
eslint@10.5.0 -> minimatch@10.2.5 -> brace-expansion@5.0.9
vitest@4.1.8 -> vite@8.0.16 -> postcss@8.5.25 -> nanoid@3.3.18
```

The executable regression binds the current patched floors:

- @hono/node-server@2.0.5 or later;
- @modelcontextprotocol/sdk@1.30.0 or later;
- body-parser@2.3.0 or later;
- brace-expansion@5.0.9 or later;
- fast-uri@3.1.5 or later;
- hono@4.12.34 or later;
- ip-address@10.3.1 or later;
- nanoid@3.3.18 or later;
- postcss@8.5.23 or later.

Dependabot pull request #467 resolves brace-expansion@5.0.7. The newer GHSA-m h99-v99m-4gvg advisory includes that version, so the pull request no longer satisfies its security purpose.

Scope

- Update the deterministic package lock through the package manager.
- Resolve @modelcontextprotocol/sdk to 1.30.0 or later within the existing declared range.
- Replace the vulnerable exact Hono override with a compatible patched range so a clean install cannot reintroduce the advisory-bearing line.
- Resolve every affected transitive dependency to a patched version.
- Preserve the supported Node 20 and Node 22 runtime and test matrix.
- Preserve all public package and command behavior.
- Record the dependency topology, resolved versions, audit result, and verification evidence.
- Supersede #467 only after the replacement branch proves brace-expansion@5.0 .9 or later.

Non-Goals

- No public API or runtime behavior redesign.
- No broad dependency modernization beyond the shared security resolution.
- No audit suppression, ignore rule, or accepted vulnerable version.
- No version bump, release tag, GitHub Release, npm publication, or milestone closure.
- No Code Dojo exception removal; the final 12 -> 0 goalpost follows this security cycle.

Dependency And Compatibility Contract

The change is acceptable only when:

- package-lock.json is reproducible from the declared workspace manifests;
- the resolved graph contains no version covered by the current advisories;
- the existing @modelcontextprotocol/sdk public usage compiles and its MCP tests pass;
- Node 20 and Node 22 repository tests remain green;
- the full Code Dojo and DOGFOOD smoke gates remain green;
- npm audit --audit-level=low exits successfully.

The package-manager lockfile is authoritative for resolved versions. The live npm advisory service is authoritative for the release audit result.

Tests To Write First

The deterministic RED proof reads the lockfile and rejects the vulnerable resolved versions named above. It must fail on the merge-base lockfile before the update and pass afterward.

The external advisory proof is:

```
npm audit --audit-level=low
```

The clean-install compatibility and product proof is executed in this order:

```
npm ci
npm audit --audit-level=low
npm run build
npm run typecheck:test
npm run lint
npm run lint:eslint
npm run code:size
npm run test:run
npm run code-dojo:ci
npm run smoke:dogfood
npm run docs:inventory
npm run docs:design-system:preflight
npm run verify:interactive-examples
```

That sequence proves that a clean install from the committed lockfile preserves the repository’s supported execution surface. Hosted CI repeats npm

ci and the full repository test lane on both supported runtimes as the test (20) and test (22) matrix jobs, and runs the two DOGFOOD smoke jobs independently.

Implementation Plan

1. Land this design and tracker/roadmap synchronization.
2. Add the lockfile security regression and prove it RED.
3. Run the package-manager security update and inspect the exact lockfile diff.
4. Prove the lockfile regression and live audit GREEN.
5. Run focused MCP, package, Node 20/22, Code Dojo, DOGFOOD, and documentation gates.
6. Run npm ci and repeat the full gate from the committed lockfile.
7. Complete documentation upkeep, self-review, hosted review, and Code Lawyer.
8. Merge the replacement and close #467 as superseded only after exact-head evidence is green.

Acceptance Criteria

☒ Every affected direct and transitive dependency path is documented.
☒ The deterministic lockfile regression fails on the merge-base graph.
☒ npm audit --audit-level=low reports zero vulnerabilities.
☒ package-lock.json resolves patched versions without unrelated package churn.
☒ A clean npm ci reproduces the verified graph.
☐ Full Node 20 and Node 22 tests, typecheck, Code Dojo, DOGFOOD smoke, and documentation gates pass.
☐ PR #467 is explicitly superseded rather than merged at a still-vulnerable version.
☐ Issue #482 records the final resolved versions and audit evidence.

Playback Questions

- Does the lockfile contain every expected patched version and no affected predecessor?
- Does a clean install reproduce the same resolved graph?
- Does the MCP package behave identically after its transitive Hono server update?
- Does the live audit report zero at the exact reviewed head?
- Is every lockfile change explained by an advisory path or deterministic pruning of an orphaned lock entry?

Retrospective

The security pass had to be refreshed against live advisory truth rather than the original shaping snapshot. The first carried lock attempt still admitted newer fast-uri, Hono, and ip-address advisories and predated the nanoid advisory. Its regression could therefore false-green. Raising the executable floors and binding the root Hono override made the clean-install promise real.

npm audit fix --package-lock-only also demonstrated why audit success is not the complete proof: it temporarily pruned the manifest-required SheetJS entry and reported zero vulnerabilities, while npm ci correctly rejected the inconsistent lock. Regenerating through npm install --package-lock-only restored that entry; the repeated clean install, audit, full 930 file / 4,104 test suite, DOGFOOD smoke, documentation preflight, and interactive examples then passed. Hosted Node 20/22 evidence, #467 disposition, and the final #482 closeout remain merge-bound acceptance steps.

247

WF-167 Roadmap Milestone Authority

docs/design/WF-167-roadmap-milestone-authority.md

Legend: WF — Workflow and Delivery

Tracker: #514

Sponsor Human

James Ross.

Sponsor Agent

Codex.

Decision Summary

Restore one explainable release model across GitHub milestones, ROADMAP.md, and BEARING.md.

Milestones own release horizons. Labels own work posture. Goalpost issues own dependency ordering inside a horizon. Documentation mirrors those authorities; it does not create a competing queue.

The current v8.0.0 milestone mixes a landed Runtime Graph and Scene IR contract, an active dependency-security blocker, an additive open Theme Lab pull request, and twenty-one unfinished perceptual-colour and product-surface issues. The unfinished colour campaign includes changes explicitly identified as semver-major. It belongs before the next major release, v9.0.0, rather than becoming a second product contract inside V8.

This cycle therefore:

1. returns v8.0.0 to Runtime Graph release closeout and dependency security;
2. preserves v8.1.0 as replay, capture, debugger, graph, and render evidence;
3. preserves v8.2.0 as quality automation and reliability hardening;
4. makes v9.0.0 the Product Workbench **and Sapphire Design Language** major, including interaction recipes and the State Atlas;

5. keeps v10.0.0 for renderer and host integration, including the Geordi native-GPU cell host;
6. removes stale title and label residue created by earlier Beyond-to-version promotion;
7. assigns every open pull request to its evidenced release horizon.

No issue closes in this cycle. No implementation priority is inferred merely from a milestone move.

Hill

A maintainer or agent can answer all of these from the live tracker and receive the same answer from the repository documentation:

- What blocks V8?
- Which work is evidence follow-through rather than release closeout?
- Which visual changes belong to the next semver-major boundary?
- Which ideas are committed, imminent, debt, or optional?
- Which open pull requests participate in each release horizon?
- What must be reconciled before an overlapping pull request can merge?

The answer must not require reading historical comments, inferring semantics from an issue number, or treating a lane:cool-ideas label as a release commitment.

Current Truth

The pre-cycle snapshot was taken from GitHub on 2026-08-09 against origin/main at 4412ec6dbce947887ed6ea2740ecbad0a66d122e.

Milestone Items Before This Cycle

GitHub milestone totals count issues and pull requests:

Milestone	Open items	Closed items	Open-issue interpretation
v8.0.0	24	4	23 issues plus PR #509
v8.1.0	13	0	13 issues
v8.2.0	23	5	22 issues plus PR #467
v9.0.0	15	0	15 issues
v10.0.0	10	1	10 issues
Beyond	0	6	no open work

Two additional open pull requests were unmilestoned:

- V8 dependency-security PR #492;
- V10 Geordi native-GPU design PR #511.

The audit cohort therefore contained 83 open issues and 4 open pull requests. Opening governance issue #514 increases the live issue total to 84; opening governance PR #515 increases the open pull request total to 5. Both live in the v8.2.0 milestone during the cycle.

V8 Scope Drift

The landed V8 source-side contract is carried by:

- closed parent tracker #302;
- closed goalpost #457;
- landed VISOR artifact bundle #458;
- landed packed-cell adapter #459.

The active release prerequisite is dependency-security issue #482. Pull requests #492 and #509 overlap on that remediation and must be deduplicated before either is selected for merge.

Issue #501 was subsequently retargeted from V9 into V8 with its foundation, correctness, Theme Lab, title-screen, and surface issues. That decision made the V8 release gate depend on work whose own tracker identifies three semver-major output changes:

- #496 changes theme decision-rule results;
- #497 changes every first-party theme colour;
- #499 changes ANSI-256 rendered output.

Those changes can land honestly before v9.0.0. They need not hold the already landed V8 product contract open.

Tracker Hygiene Drift

The live tracker also contains mechanically identifiable residue:

- issues promoted out of Beyond retain [Beyond] in their titles;
- #204 carries both lane:inbox and lane:cool-ideas;
- #214, #217, and #218 carry both current lane:cool-ideas and legacy lane:up-next;
- #219 remains lane:inbox despite an explicit V10 horizon;
- #348 remains lane:asap despite an explicit V10 horizon;
- #249 carries both priority:medium and priority:low;
- #507 is lane:bad-code but lacks the repository’s normal BAD CODE: title signal;
- milestone descriptions do not mention the recently added interaction-state, State Atlas, or native-GPU stories.

These are tracker defects, not implementation defects.

Authority Model

Milestone: Release Destination

A versioned milestone answers: **if this work lands, which release boundary owns it?**

A milestone does not mean every issue is committed or next. That posture comes from lane:*, work-in-progress, priority, design, and goalpost labels.

Lane: Commitment And Intake Posture

- lane:inbox: not yet shaped;
- lane:asap: imminent pull;
- lane:bad-code: known debt or structural risk;
- lane:cool-ideas: optional or uncommitted exploration;
- lane:release: committed release-boundary work.

An issue should carry one current lane. Historical lane:up-next residue does not coexist with the current lane model.

Goalpost: Dependency And Outcome Authority

Goalpost issues such as #501 own dependency ordering within a release horizon. Moving a goalpost and its children between release horizons does not reorder the children. It changes the release boundary that may claim them.

Documentation: Human-Readable Mirror

ROADMAP.md records release horizons, gates, and snapshot totals. BEARING.md records current execution gravity. Neither file overrides live GitHub state.

Target Milestone Model

v8.0.0: Runtime Graph Release Closeout

Outcome:

- Runtime Graph and Scene IR product contract is landed;
- dependency advisories reach zero through one selected remediation path;
- release preparation begins only after security truth is reproducible;
- no unfinished colour-system campaign blocks the release.

Expected open items after triage:

- issue #482;
- PR #492;
- PR #509.

The two PRs overlap. Their simultaneous presence records alternatives, not three independent release blockers.

v8.1.0: Replay, Capture, And Render Evidence

No issue migration is required. The milestone remains the post-contract proof horizon for replay, capture, debugger, graph, browser, terminal, and parity witnesses.

Expected open items after triage: 13.

v8.2.0: Quality Automation And Reliability

Add issue #507 because startup key-conflict reporting is quality enforcement, not Runtime Graph release scope. Governance issue #514 also lives here because tracker synchronization is an explicit V8.2 outcome.

Expected open items after triage: 26:

- 24 open issues, including #507 and #514;
- Dependabot PR #467;
- governance PR #515.

v9.0.0: Product Workbench And Sapphire Design Language

Move the full unfinished campaign cohort from V8:

Cohort	Issues
Theme and product surfaces	#311, #315, #317, #336, #455, #493, #494, #502
Perceptual foundation and generation	#318, #352, #495, #496, #497, #498, #501, #504
Correctness and accessibility	#499, #500, #505
Theme Lab structural debt	#503, #506

The cohort contains 21 issues. It joins the existing 15 V9 issues, including interaction recipes #512 and State Atlas #513.

Expected open items after triage: 36.

The count is large because this milestone is an explicit future major horizon, not because all lane:cool-ideas work has become committed. #501 and the existing Product Workbench stories provide the internal goalposts.

v10.0.0: Renderer And Host Systems Integration

Keep the existing ten issues. Assign PR #511 to the milestone and update the description to name the Geordi native-GPU cell-host boundary.

Expected open items after triage: 11.

Beyond: Deliberate Incubation

No issue moves into Beyond during this cycle. Explicit release destinations remain useful. The stale title prefix is removed from issues that already have a versioned destination.

Issue Title And Label Normalization

Remove Stale [Beyond] Prefix

Rename these open issues without changing their substantive titles:

- #202 through #219 where the current title begins [Beyond]: #202, #203, #204, #205, #206, #207, #208, #209, #210, #211, #212, #213, #214, #215, #216, #217, #218, and #219.
- #311: remove the same stale prefix from the Theme Inspector drawer.
- #502: replace its stale v8 release suffix with V9 release after the milestone migration.

Each already has an explicit versioned milestone. The prefix now contradicts the milestone and makes search results look parked when they are not.

Normalize Lanes And Priority

- #204: remove lane:inbox and priority:medium; retain lane:cool-ideas and priority:low.
- #205: remove lane:up-next; retain lane:cool-ideas.
- #214, #217, #218: remove lane:up-next; retain lane:cool-ideas.
- #215: remove lane:up-next and priority:medium; retain lane:cool-ideas and priority:low.
- #219: replace lane:inbox with lane:cool-ideas.
- #348: replace lane:asap with lane:cool-ideas; V10 is a future host and renderer horizon, not the imminent V8 pull.
- #249: remove priority:medium; retain priority:low for the optional technical-teardown quality gate.
- #507: rename to BAD CODE: detect key binding conflicts at frame init while preserving the current debt lane and acceptance criteria.

No blanket priority assignment is introduced. An absent priority is not a contradiction; two priorities are.

Pull Request Triage

PR #492

Assign to v8.0.0. It implements the active dependency-security blocker.

PR #509

Keep in v8.0.0 because it also implements the same security blocker and its already-implemented Theme Lab changes are additive. Moving unfinished campaign issues to V9 does not remove the PR's V8 security relevance.

The PR must reconcile its branch-local roadmap changes with WF-167 before merge. A significant-event comment will record that consequence.

PR #511

Assign to v10.0.0. It is the design evidence for issue #510 and does not alter the V8 or V9 release gates.

PR #467

Keep in v8.2.0 as dependency-update lineage. Do not close it until one newer security remediation is selected and merged with evidence that brace-expansio n@5.0.9 or later is reproducible.

Migration Ordering

The mutation order prevents documentation from claiming unperformed tracker state:

1. publish this design and the non-draft cycle PR;
2. update milestone descriptions;
3. move the V9 cohort and #507;
4. assign PR #492 and PR #511;
5. normalize titles, lanes, and #249 priority;
6. comment every release-horizon move;
7. take a fresh milestone-item snapshot;
8. update ROADMAP.md, BEARING.md, changelog, and tests to that snapshot;
9. verify GitHub read-back and repository gates;
10. update the cycle issue and PR only for material discrepancies.

If any mutation fails, stop and read back the affected issue before retrying. Do not assume a partial batch succeeded.

Rollback And Recovery

GitHub issue and milestone edits are individually reversible. The durable migration table in this design identifies every intended source and target.

If review rejects the model:

- issues can be moved back using the same explicit table;
- titles can regain their previous prefix from GitHub history;

- labels can be restored individually;
- documentation changes can be superseded by a normal follow-up commit;
- no force operation, rebase, amend, or history rewrite is required.

Issue comments remain as provenance and should not be deleted. A rollback comment should explain the superseding decision.

Scope

- Audit all open Bijou issues and pull requests at the cycle snapshot.
- Refactor the live milestone assignments and descriptions described above.
- Normalize the bounded title and label defects described above.
- Update roadmap, bearing, changelog, and deterministic policy tests.
- Link and explain the interaction-state, State Atlas, and native-GPU work in the correct horizons.
- Preserve the active security and colour branches without editing their worktrees.

Non-Goals

- No issue closure without implementation or explicit disposition evidence.
- No pull request merge, release, tag, or package publication.
- No selection between PR #492 and PR #509.
- No changes to the user-owned dirty cycle/v8-security-closeout worktree.
- No changes to the cycle/colors worktree.
- No implementation of perceptual colour, Theme Lab debt, interaction recipes, State Atlas, replay, or native-GPU rendering.
- No new release milestone; the existing V8, V9, and V10 major boundaries already express the required semver ordering.
- No claim that a milestone assignment alone commits a cool idea to ship.

Tests To Write First

1. Assert the roadmap snapshot contains the exact post-migration milestone-item totals.
2. Assert V8 names #482 and the overlapping #492/#509 remediation posture and does not list #501 as a release gate.
3. Assert V9 names #501, #512, and #513 and describes Sapphire Design Language plus Product Workbench outcomes.
4. Assert V10 names #510 and PR #511 and preserves the native-host claim boundary.
5. Assert BEARING reports the same totals and V8 next action.
6. Preserve the milestone-item counting rule: counts include issues and pull requests.

7. Preserve the empty open-unmilestoned and open-Beyond policy statements only after live GitHub read-back proves them.
8. Keep historic shipped-release lineage unchanged.

Validation Plan

```
npx vitest run --config vitest.config.ts tests/cycles/WF-130
npx vitest run --config vitest.config.ts tests/cycles/DX-050
npx vitest run --config vitest.config.ts tests/cycles/RE-036
npm run docs:inventory
npm run typecheck:test
npm run lint
npm run code-dojo:changed
git diff --check
```

The pre-commit and pre-push hooks remain required. Live GitHub verification is a separate read-back step because deterministic repository tests must not depend on network availability.

Acceptance Criteria

- All 83 pre-cycle issues, issue #514, the four pre-cycle pull requests, and governance PR #515 have a recorded disposition.
- V8 contains one issue and two competing security implementation PRs.
- V9 contains the full twenty-one-issue Sapphire/design-language cohort.
- V8.2 contains #507 and #514.
- V10 contains PR #511.
- No open issue or pull request is unmilestoned.
- No versioned issue retains [Beyond] in its title.
- #204, #205, #214, #215, #217, #218, #219, #348, and #249 have one coherent lane or priority posture as specified.
- Every issue moved across a release horizon has a migration comment.
- Milestone descriptions, roadmap, bearing, changelog, and tests match the verified post-migration snapshot.
- PR #509 has a visible reconciliation note before its branch can overwrite the new roadmap authority.
- No unrelated worktree or GitHub state changes.

Playback Questions

1. Can a reviewer identify the actual V8 blocker without reading #501 history?
2. Does V9 contain every semver-major colour change and the interaction-state work that consumes it?
3. Do lanes still distinguish imminent work, debt, and optional exploration inside each versioned horizon?
4. Are every milestone-item total and open-unmilestoned claim backed by live GitHub read-back?

5. Did the cycle preserve all issues and dependency relationships instead of hiding scope by closing or parking it?
6. Can PR #509 merge later without silently restoring the superseded V8 campaign assignment?

Accessibility And Assistive Posture

This is planning and workflow work. It changes no rendered interface. The roadmap explicitly keeps accessibility and capability-degradation work inside the V9 State Atlas and colour-distinctness stories rather than treating them as optional polish.

The issue migration comments and repository documentation remain readable as plain Markdown without colour, diagrams, or screenshots.

Localization And Directionality Posture

No localized product copy or runtime directionality behavior changes. V9 continues to own the localization dashboard, portable preferences, translation workbench, and structured multilingual changelog stories.

Milestone descriptions use concise English tracker metadata; they do not become application-facing strings.

Agent Inspectability And Explainability Posture

An agent can inspect:

- the complete source-to-target migration table;
- exact before and after milestone totals;
- which totals include pull requests;
- title and label normalization rules;
- overlapping PR posture;
- release-horizon comments;
- repository tests that bind the durable mirror.

No agent must infer release scope from issue-number ranges, historical [Beyon d] prefixes, or branch names.

Linked Invariants

- Work Doctrine
- Documentation Standards
- TypeScript Code Standards
- Roadmap

- Bearing
- Release Policy
- Tracker synchronization idea: #268
- Perceptual-colour goalpost: #501
- Interaction recipes: #512
- State Atlas: #513
- Native GPU cell host: #510

Retrospective And Closeout

Open. Closeout requires live GitHub read-back, merged documentation evidence, and explicit reconciliation of PR #509's branch-local roadmap changes. This cycle does not close implementation issues or select a security remediation pull request.

Other design records

These records retain additional lines of design work whose identifiers sit outside the six principal families. Their source paths preserve the original classification. Read them with the same questions used throughout this part: which outcome is owned, which boundaries constrain it, and what evidence follows the proposal into implementation?

248

RE-008 — Byte-Packed Surface Representation

docs/design/0001-008-byte-packed-surface-representation/008-byte-packed-surface-representation.md

Source backlog item: docs/method/backlog/asap/RE_008-byte-packed-surface-representation.md Legend: RE

Sponsors

- Human: James
- Agent: Claude (Opus 4.6)

Hill

Surface rendering in Bijou goes from "every cell is a heap-allocated object with string colors" to "every cell is 10 bytes in a flat typed array with numeric RGB." The DOGFOOD landing benchmark is at parity or faster than the pre-RE-008 baseline, and the architecture enables zero-allocation rendering for any component that opts into setRGB().

Playback Questions

Human

☒ Does the DOGFOOD landing render at least as fast as before?
☒ Are all 2816 existing tests still passing?
☒ Is the set() convenience API preserved and documented?
☒ Can components opt into the fast path incrementally?

Agent

☒ Is the packed buffer the source of truth, with Cell[] as lazy decode?
☒ Does the differ operate on bytes, not strings?
☒ Does blit copy bytes directly between packed buffers?

☒ Are side-table chars (emoji, grapheme clusters) handled correctly?

What Shipped

Architecture

- **Packed cell encoding** (packed-cell.ts): 10-byte layout per cell — uint16 char, 3B fg RGB, 3B bg RGB, 1B flags bitfield, 1B alpha/presence. Side table for multi-codepoint graphemes outside the BMP.
- **Lazy dirty bitmap**: Uint32Array bitmap (1 bit per cell) tracks which cells need decoding from the buffer. Writes mark dirty; reads decode on demand.
- **setRGB() API**: Zero-allocation writes — char code + numeric RGB directly into the buffer. No hex parsing, no Cell object.
- **Packed blit**: 10-byte memcpy per cell with side-table re-encoding for cross-surface grapheme clusters.
- **Template-stamp fill/clear**: Encode once, copy bytes to every position.
- **Direct ANSI emission**: Differ reads RGB bytes from the buffer and emits \x1b[38;2;R;G;Bm directly, with a style-byte hash cache. Bypasses chalk and StylePort entirely on the hot diff path.
- **Numeric theme tokens**: TokenValue.fgRGB / bgRGB pre-parsed at theme resolution time.

Slices (19 commits)

1. Packed-cell byte encoding module
2. createSurface backed by Uint8Array + eager sync
3. Route all direct cell mutations through surface.set()
4. Packed byte-comparison differ
5. Grayscale middleware on packed bytes
6. Lazy dirty bitmap (replace eager sync)
7. Zero-alloc hex parsing (charCode arithmetic, lookup table)
8. Inline hex→RGB, FULL_MASK fast path, numeric theme tokens
9. Direct ANSI SGR emission with cached style-byte hash
10. setRGB() API + perf-gradient demo migration
11. Packed byte-copy blit
12. Template-stamp fill, clear, clone
13. Component migration: surface-text, box-v3, separator-v3
14. Component migration: overlay dim, flex/overlay bg inheritance
15. Component migration: table-v3, alert-v3
16. Component migration: notification, css/text-style
17. Component migration: badge, focus-area, preference-list, canvas, differ

Benchmark Results

Scenario	Main baseline	RE-008	Delta
Landing render 220×58	1.60 ms	1.55 ms	**-3%**
Landing render+diff 220×58	1.86 ms	1.74 ms	**-7%**
Landing render 271×71	0.53 ms	0.51 ms	**-3%**
Frame compose	0.95 ms	1.19 ms	+25%
Layout normalize	0.16 ms	0.34 ms	+113%
Styled diff	0.42 ms	0.53 ms	+26%
Runtime noop pulses	0.002 ms	0.002 ms	**-9%**

Real-world DOGFOOD rendering is faster. Synthetic stress tests still show overhead from the set() hex-parsing path — these will improve as more rendering paths move to direct byte manipulation.

Accessibility and Assistive Reading

No user-facing changes. The packed buffer is internal; all public APIs return the same Cell objects and string output as before.

Localization and Directionality

No changes. Character encoding handles the full Unicode BMP directly and uses a side table for astral/multi-codepoint graphemes.

Agent Inspectability and Explainability

The PackedSurface interface and setRGB() API are exported and documented. Agents can query surface dimensions, read cells via get(), and use the packed buffer for performance-sensitive rendering.

Non-goals

- Changing the public Cell interface or breaking component APIs
- Removing hex color support from the convenience set() API
- Pre-allocated output buffer for stdout (deferred to future work)
- Eliminating the Cell[] layer entirely (the lazy decode + dirty bitmap keeps backward compatibility with minimal overhead)

249

ASAP-001 — Fix MCP DAG Type Error

docs/design/ASAP-001-fix-mcp-dag-type-error.md

Cycle for fixing the broken overload in @flyingrobots/bijou-mcp's DAG tool and aligning with the core dag() signature.

Legend: - RE — Runtime Engine - HT — Humane Terminal

Why this cycle exists

The bijou-mcp package is currently broken because the dag() tool call in pack ages/bijou-mcp/src/tools/dag.ts does not match the available overloads in @fl yingrobots/bijou. This blocks CI and local pre-commit hooks, and prevents agents (like BigBro) from using the DAG visualization tool for cycle tracking.

Human users / jobs / hills

Primary human users

- framework maintainers
- developers using the MCP server

Human jobs

1. Visualize graph structures (like the current work cycle) using the MCP tool.
2. Maintain a green build state.

Human hill

A user can invoke the bijou_dag MCP tool and receive a correctly rendered box-drawn graph without encountering type errors during build.

Agent users / jobs / hills

Primary agent users

- EXPERT Bijou BigBro (mentorship and auditing)
- any agent using the MCP server for visualization

Agent jobs

1. Programmatically generate cycle visualizations to communicate progress.

Agent hill

An agent can use the dag component through the MCP interface to provide structured, high-signal progress reports.

Linked invariants

- Tests Are the Spec
- Schemas Live At Boundaries

Test Plan

Golden Path

- Fix the type error in packages/bijou-mcp/src/tools/dag.ts.
- Run npm run lint and ensure it passes for @flyingrobots/bijou-mcp.
- Verify the dag() call correctly passes an array of DagNode and valid DagOpt ions.

Edge Cases

- Pass a SlicedDagSource to verify the second overload works.
- Ensure nodeStyle is correctly recognized or removed if it was a hallucination in the MCP tool. (Research shows nodeStyle exists in DagOptio ns in core).

Known Failure Modes

- nodeStyle might be missing if the @flyingrobots/bijou-mcp package is linked to an older version of core or if there’s a name collision.

Implementation Outline

1. Research the exact signature of dag() in packages/bijou/src/core/components/dag.ts.
2. Inspect packages/bijou-mcp/src/tools/dag.ts to identify the mismatched call.
3. Fix the call site:
   - Ensure the input is a valid DagNode[] or SlicedDagSource.
   - Ensure DagOptions only contains valid properties.
4. Verify via npm run lint --workspace @flyingrobots/bijou-mcp.

Retrospective

What landed: - (TBD)

What did not land: - (TBD)

Follow-on: - (TBD)

250

AUDIT-001 — Mentor Audit of examples/showcase

docs/design/AUDIT-001-showcase-audit.md

Expert evaluation of the Bijou Showcase application to ensure adherence to core invariants, hexagonal boundaries, and anti-SLUDGE TypeScript standards.

Legend: - HT — Humane Terminal - DL — Design Language - DX — Developer Experience

Why this cycle exists

The Showcase app is the most visible "complex" implementation of Bijou. If it contains anti-patterns (e.g., hardcoded colors, loose typing, or mixed architectural concerns), it will mislead developers into adopting those same bad habits.

As the **EXPERT Bijou BigBro**, I must audit this app to: - Identify and backlog "sludge" for correction. - Propose refactors that align with the **Hexagonal** and **TEA** pillars. - Ensure it serves as a "Humane" reference for others.

Human users / jobs / hills

Primary human users

- Developers looking at examples/ for inspiration.
- Maintainers ensuring the library supports complex use-cases cleanly.

Human jobs

1. Learn how to structure a multi-page TUI app.
2. See "pro-level" Bijou usage.

Human hill

A developer can read the Showcase source and see a perfect reflection of the **Invariants**—clear boundaries, semantic tokens, and rhythmic layout—without any "clever sludge."

Agent users / jobs / hills

Primary agent users

- EXPERT Bijou BigBro (auditor/mentor).

Agent jobs

1. Audit the Update loop for side-effects.
2. Verify Surface usage follows performance best practices.

Agent hill

An agent can mechanically verify that the showcase app never imports a Node.js-specific adapter into its FramePage logic.

Test Plan

Golden Path (The Audit)

- Scan examples/showcase/app.ts for:
  - any or as any.
  - Hardcoded ANSI or hex strings.
  - Direct process or fs access.
- Scan examples/showcase/registry.ts for:
  - Improper ctx propagation.
  - Violation of the 2-cell rhythm.

Playback Questions

1. Does the showcase app leak “adapter sludge” into its page definitions?
2. Are the message types (M) strictly defined or do they use any?
3. Is the layout “geometricly lawful” (2-cell rhythm)?

Linked Invariants

- The Buffer Holds Facts
- Hexagonal Architecture
- Anti-SLUDGE Policy

Implementation Outline

1. **Research:** Deep-read app.ts and registry.ts.
2. **Strategy:** Categorize findings into bad-code (ASAP/Backlog) and cool-ideas.
3. **Execution:** Log findings and propose surgical fixes for the most egregious violations.

251

BE-001 Project Big Extraction: DOGFOOD Blocks Breakout

docs/design/BE-001-dogfood-blocks-breakout.md

Status

Implemented.

Tracker

- GitHub Issue: #435
- Pull Request: #436

Legend

- Linked legend: WF, Workflow and Delivery
- Sponsor human: James Ross
- Sponsor agent: Codex

Hill

Project Big Extraction starts by killing a hard-limit monolith instead of preserving legacy-large files as permanent habitat.

examples/docs/dogfood-blocks.ts is currently a 2,637 line / 92,020 byte DOGFOOD block registry monolith with 151 exports. The cycle turns it into a small compatibility facade backed by focused modules, preserving existing consumers while removing the hard-limit file from the active monolith list.

Current Truth

Live counts on main at b058b2c5:

- aggregate Code Dojo debt: 364
- file/context baseline: 309
- mock-ban baseline: 0
- code-size baseline: 55, including 4 hard-limit files
- ESLint baseline: 0
- next aggregate target: 314 or lower

Current hard-limit files:

- examples/docs/stories.ts: 4,558 lines / 169,954 bytes
- examples/docs/app.ts: 3,306 lines / 114,972 bytes
- packages/bijou-tui/src/app-frame.ts: 2,847 lines / 109,042 bytes
- examples/docs/dogfood-blocks.ts: 2,637 lines / 92,020 bytes

dogfood-blocks.ts has low external fanout compared to the other hard-limit files, but high export surface. That makes it a good first Big Extraction target: keep ./dogfood-blocks.js as the public facade, move cohesive internals behind it, and avoid broad consumer churn.

Implementation Result

BE-001 turns examples/docs/dogfood-blocks.ts into a 97 line / 3,740 byte facade and moves the registry, schema parsing, renderers, block definitions, and registry entries into focused modules. The largest extracted module is ex amples/docs/dogfood-block-docs-surface-render.ts at 143 lines / 5,597 bytes, so every new module stays below the Code Dojo file/context threshold.

Live debt after the extraction:

- aggregate Code Dojo debt: 362
- file/context baseline: 308
- mock-ban baseline: 0
- code-size baseline: 54, including 3 hard-limit files
- ESLint baseline: 0
- DOGFOOD raw-string debt: 2,365, down from the stored 2,415 baseline
- next aggregate target from the live count: 312 or lower

The extraction removes dogfood-blocks.ts from both debt ledgers but only removes two aggregate violations. It is an honest structural reduction, not a 50-violation goalpost by itself.

The DOGFOOD raw-string baseline is also transferred from the old dogfood-bloc ks surface to the extracted module surfaces at exact measured counts, while stale slack in the docs, counter, and Storybook surfaces is removed.

Scope

- Split DOGFOOD block registry, block definitions, renderers, schema helpers, and registry assembly out of examples/docs/dogfood-blocks.ts.
- Preserve the current public import path and named exports through the facade.
- Keep every new module below Code Dojo file/context and code-size thresholds.
- Do not add new file/context or code-size baseline entries.
- Update the Code Dojo ledgers after live counts prove the monolith is gone.
- Update changelog and exception documentation with the Big Extraction result.

Non-Goals

- No redesign of DOGFOOD blocks or rendered content.
- No broad rewrite of examples/docs/app.ts.
- No extraction of stories.ts or app-frame.ts in this cycle.
- No release-version work.
- No rebase, amend, force push, or draft PR.

Extraction Map

Target module boundaries:

- Shared helpers:
  - examples/docs/dogfood-block-common.ts
  - examples/docs/dogfood-block-text.ts
  - examples/docs/dogfood-block-schema-utils.ts
- Registry assembly:
  - examples/docs/dogfood-block-registry-types.ts
  - examples/docs/dogfood-block-registry-key.ts
  - examples/docs/dogfood-block-registry-entry.ts
  - examples/docs/dogfood-block-registry.ts
  - examples/docs/dogfood-block-registry-entries.ts
- Docs surface block:
  - examples/docs/dogfood-block-docs-surface-types.ts
  - examples/docs/dogfood-block-docs-surface-data.ts
  - examples/docs/dogfood-block-docs-surface-parser.ts
  - examples/docs/dogfood-block-docs-surface-render.ts
  - examples/docs/dogfood-block-docs-surface-schema.ts
  - examples/docs/dogfood-block-docs-surface.ts
- Rendered block families:

    - examples/docs/dogfood-block-preview.ts
    - examples/docs/dogfood-block-guide-inspector.ts
    - examples/docs/dogfood-block-guide-inspector-render.ts
    - examples/docs/dogfood-block-settings-menu.ts
    - examples/docs/dogfood-block-settings-menu-render.ts
    - examples/docs/dogfood-block-search-panel.ts
    - examples/docs/dogfood-block-search-panel-render.ts
    - examples/docs/dogfood-block-notification-center.ts
    - examples/docs/dogfood-block-perf-hud.ts
    - examples/docs/dogfood-block-help-overlay.ts
    - examples/docs/dogfood-block-command-palette.ts
    - examples/docs/dogfood-block-footer-hint.ts
    - examples/docs/dogfood-block-documentation-article.ts
    - examples/docs/dogfood-block-navigation-list.ts
    - examples/docs/dogfood-block-navigation-list-render.ts
    - examples/docs/dogfood-block-title-screen.ts
    - examples/docs/dogfood-block-workbench.ts
    - examples/docs/dogfood-block-workbench-render.ts
- Public facade:
    - examples/docs/dogfood-blocks.ts
    - public re-exports
    - requiredDogfoodBlockSurfaceIds
    - defaultDogfoodBlockRegistry

Implementation Outline

1. Add focused safety tests for facade compatibility and registry assembly if existing tests do not pin the split boundary.
2. Extract schema utility helpers first because docs-surface parsing depends on them and they have no app-facing behavior.
3. Extract the registry class and entry helpers while preserving exported names.
4. Extract block families in cohesive groups: docs surface, preview/inspector, shell overlays, docs navigation, and workbench.
5. Reduce dogfood-blocks.ts to a facade plus registry assembly.
6. Recompute Code Dojo file/context and code-size ledgers and lower counts.
7. Update docs/code-dojo-exceptions, changelog, and this design doc with implementation notes and validation evidence.

Tests To Write First

- Facade export compatibility test:

    - importing from examples/docs/dogfood-blocks.js exposes the expected default registry, required surface ids, and representative block exports.
- Registry assembly test:
    - defaultDogfoodBlockRegistry includes every requiredDogfoodBlockSurfaceId after extraction.

Existing focused suites remain the behavioral net:

- tests/cycles/DF-030/dogfood-docs-surface-block.test.ts
- tests/cycles/DF-069/dogfood-block-registry.test.ts
- tests/cycles/DF-070/dogfood-block-product-polish.test.ts
- tests/cycles/DF-071/dogfood-block-authored-surfaces.test.ts
- tests/cycles/DX-046/graphql-dogfood-navigation-fixture.test.ts

Validation Plan

- Focused Vitest suites listed above
- npm run code-dojo:changed
- npm run dogfood:i18n:complete
- npm run dogfood:i18n:check
- npm run dogfood:i18n:debt
- npm run typecheck:test
- npm run lint
- npm run code-dojo:verify
- npm run docs:inventory
- git diff --check

Acceptance Criteria

- examples/docs/dogfood-blocks.ts is below Code Dojo file/context and code-size thresholds.
- No extracted module exceeds Code Dojo file/context or code-size thresholds.
- The hard-limit code-size count drops from 4 to 3.
- Aggregate Code Dojo debt does not increase, and BE-001 is not counted as a completed Respecting the Dojo goalpost unless a follow-on slice lowers the live aggregate count by the required 50 violations to 312 or lower.
- Existing public imports from ./dogfood-blocks.js continue to work.
- Focused DOGFOOD block tests and local dojo/type/lint gates pass.

Playback Questions

1. Is dogfood-blocks.ts no longer a hard-limit file?
2. Did the facade preserve the public export surface used by consumers?

3. Did the split avoid claiming the 50-violation goalpost prematurely?
4. Are all new modules below file/context and code-size thresholds?
5. Did the DOGFOOD block behavior tests pass?

Linked Invariants

- TypeScript standards: docs/typescript-code-standards.editors-edition.md
- Code Dojo exception ledger: docs/code-dojo-exceptions.md
- Work doctrine: docs/METHOD.md

252

HT-002 — Layered Focus and Interaction

docs/design/HT-002-layered-focus-and-interaction.md

Cycle for defining how Bijou shells should treat focus layers, dismiss order, and input ownership as one coherent stack

Legend:

- HT — Humane Terminal

Why this cycle exists

Bijou has accumulated multiple shell-owned surfaces:

- help
- search
- command palette
- settings
- notifications
- quit confirm
- page-owned modals

They currently route input by branch order. That is workable until it is not: the moment two layers disagree about Esc, footer hints, or control ownership, the shell starts feeling brittle.

This cycle exists to define a calmer model:

- layers behave like a stack
- the topmost layer owns input
- the topmost layer owns the visible controls
- dismiss actions pop layers before they trigger broader shell behavior

The purpose of this cycle is design, not implementation. The follow-on runtime refactor will come in the next cycle.

Human users / jobs / hills

Primary human users

- operators inside framed Bijou apps
- first-time DOGFOOD users who open layered shell surfaces
- builders trying to understand why Esc, Tab, and footer hints behave the way they do

Human jobs

1. Understand which layer currently owns input.
2. Dismiss the thing in front of me without accidentally quitting the app.
3. Trust the footer and help hints to describe the active controls truthfully.
4. Move back to the underlying pane without guessing whether hidden controls are still live.

Human hill

A user can open and close layered shell surfaces without ever feeling that Es c or the footer lied about what would happen next.

Agent users / jobs / hills

Primary agent users

- agents navigating framed apps through deterministic key sequences
- agents inspecting shell state for playbacks, tests, or UI generation
- agents trying to decide which controls are valid without scraping visual chrome heuristically

Agent jobs

1. Determine the active interaction layer without reconstructing branch order from source.
2. Read the active input map for the topmost layer.
3. Predict what Esc, navigation keys, and shell shortcuts will do right now.
4. Derive truthful control hints from the same source that governs routing.

Agent hill

An agent can inspect or infer a single active layer stack and know which input map is live, what controls are truthful, and what dismissal will do next.

Human playback

1. A user is in DOGFOOD docs and opens component search with /.
2. Search pushes a search layer onto the shell stack.
3. The search layer's input map becomes active, and the footer updates to search-specific controls.
4. The underlying pane controls disappear from the footer because they are no longer truthful.
5. The user presses Esc.
6. The search layer dismisses and pops off the stack.
7. The previously focused docs pane becomes active again, along with its pane controls.
8. The user presses Esc again.
9. No dismissible overlay remains, so the shell now interprets Esc as a quit request and opens quit confirm.

Agent playback

1. An agent enters a framed app and requests the current shell layer state.
2. The shell reports a base workspace layer plus one topmost settings drawer layer.
3. The active input map is the settings drawer map, not the underlying pane map.
4. The agent derives visible controls from that topmost input map and ignores stale pane-local hints.
5. The agent sends Esc.
6. The shell pops the settings layer and restores the workspace layer and its active pane input map.
7. The agent sends Esc again.
8. With no dismissible overlay left, the shell opens quit confirm instead of silently quitting or dispatching to a stale pane action.

Linked invariants

- Focus Owns Input
- Topmost Layer Dismisses First
- Visible Controls Are a Promise
- Shell Owns Shell Concerns

Proposed direction

Layer model

Treat shell and page interaction surfaces as a stack of explicit layers:

- base workspace layer
- pane-local modal layer
- shell search/palette layer
- shell help layer
- shell settings layer
- shell notification center layer
- shell quit-confirm layer

The stack is logical, not merely visual. Only the topmost layer owns input.

Input maps

Each layer should provide or select an input map that describes:

- the keys it handles
- the dismissal behavior it owns
- the controls that can be shown in the footer/help
- whether it is dismissible, blocking, or review-only

This makes the input map do three jobs from one source of truth:

- routing
- visible controls
- agent-readable interaction state

Dismiss order

Dismissal should be top-down and unsurprising:

1. If a dismissible layer is open, Esc dismisses that layer.
2. If the topmost layer is not dismissible, Esc does nothing or follows that layer's explicit contract.
3. Only when no dismissible layer remains should Esc fall back to the shell quit request.

q and Ctrl+C can remain explicit quit affordances, but they should still respect text-entry exceptions and shell-owned quit policy.

Visible controls

Footer and help hints should be derived from the topmost input map, not copied from whichever branch happened to handle the last render.

This means:

- a search layer shows search controls
- a modal shows modal controls

- an inactive pane does not keep advertising its keys when covered
- when a layer pops, the previous active map becomes visible again

Agent-visible semantics

The shell should eventually expose enough structured layer state that tests, playbacks, and agents do not need to infer ownership from rendered text alone.

Candidate shape:

- ordered active layers
- topmost input map id
- dismissibility
- active control hints
- owning region or surface id

Implementation outline

This cycle is design-only. The intended implementation follow-on is:

1. Introduce a shell layer model in createFramedApp().
2. Convert search/help/settings/notifications/quit confirm into explicit layer descriptors.
3. Promote or adapt input maps so each layer drives both routing and visible controls.
4. Make Esc follow top-down dismiss semantics.
5. Prove the behavior through DOGFOOD and shell regressions.

Tests to write first

- cycle doc regression asserting the new Humane Terminal cycle, invariant, and doctrine links exist
- shell behavior regressions proving top-down dismiss order once implementation begins
- footer/help regressions proving visible controls come from the topmost input map
- DOGFOOD playback proving search dismissal restores the underlying pane controls before quit is possible

Risks / unknowns

- page-owned modals and shell-owned layers currently live in different conceptual buckets
- not every existing key path is modeled as an input map yet

- some layers may want special Esc behavior beyond simple dismissal
- the eventual public API for agent-readable layer state needs to be useful without overfitting the current shell internals

Retrospective

This cycle deliberately stops at doctrine and interaction model definition.

What it clarified:

- Esc bugs are symptoms of missing layer semantics, not isolated routing accidents
- the shell needs a topmost-layer model, not more special-case branch ordering
- input maps are a promising shared seam for routing, footer truth, and agent visibility

Drift and follow-on:

- the runtime still routes most shell layers ad hoc today
- that follow-on has now landed as HT-003 — Implement Layer Stack and Input Map Routing

253

HT-003 — Implement Layer Stack and Input Map Routing

docs/design/HT-003-implement-layer-stack-and-input-map-routing.md

Cycle for turning the shell layer-stack doctrine into real routing, dismissal, and control-hint behavior inside createFramedApp()

Legend:

- HT — Humane Terminal

Depends on:

- HT-002 — Layered Focus and Interaction

Why this cycle exists

HT-002 established the right direction:

- layers should behave like a stack
- the topmost layer should own input
- Esc should dismiss topmost layers before quitting
- visible controls should come from the active input map

But the runtime was still routing shell surfaces through raw branch order.

This cycle exists to make that doctrine real enough to stop the Esc/footer class of bugs from reappearing constantly.

Human users / jobs / hills

Primary human users

- framed-app operators moving through search, settings, help, notifications, and quit confirm
- DOGFOOD users switching between pane work and shell-owned layers
- builders relying on createFramedApp() defaults instead of reinventing shell logic

Human jobs

1. Dismiss the thing on top without accidentally triggering something beneath it.
2. Trust the footer to describe the active controls instead of stale underlying controls.
3. Understand when a page-owned modal is blocking the workspace.

Human hill

The shell behaves like a calm stack of layers instead of a pile of conditionals: dismissal is unsurprising and footer hints are truthful.

Agent users / jobs / hills

Primary agent users

- agents driving framed apps by key sequences
- agents validating shell behavior through playbacks
- agents that need a structured way to infer what the shell considers active

Agent jobs

1. Inspect the active shell layer order from model state instead of reverse-engineering branch order.
2. Predict whether a key will dismiss, route to the page modal, or fall through to the workspace.
3. Derive visible controls from the topmost layer, not by parsing terminal chrome heuristically.

Agent hill

An agent can inspect a frame model and understand the current shell/page layer stack well enough to predict Esc and footer behavior reliably.

Human playback

1. A user opens settings from a framed app.
2. Settings becomes the topmost active layer and the footer shows settings controls.
3. The user opens help from within settings.
4. Help becomes the topmost layer and its controls replace the settings footer.
5. The user presses Esc.

6. Help dismisses and settings becomes active again.
7. The user presses Esc again.
8. Settings dismisses.
9. The user presses Esc once more.
10. Only now does the shell treat the key as a quit request and open quit confirm.

Agent playback

1. An agent inspects a frame model with a search layer open over settings.
2. The model describes the layer stack as workspace -> settings -> search.
3. The topmost input map is frame-search, so the footer shows search controls.
4. The agent sends Esc.
5. The search layer dismisses.
6. The layer stack becomes workspace -> settings.
7. The footer now shows settings controls.
8. The agent sends Esc again.
9. Settings dismisses, restoring the base workspace.

Linked invariants

- Focus Owns Input
- Topmost Layer Dismisses First
- Visible Controls Are a Promise
- Shell Owns Shell Concerns

Implementation outline

1. Add a shared shell-layer descriptor model.
2. Expose a public describeFrameLayerStack() helper for tests/agents.
3. Drive key routing from the topmost active layer instead of raw boolean branch order.
4. Make mouse handling respect the same topmost-layer blocking model.
5. Drive footer hints from the active layer’s input map or shell-specific control source.
6. Add regressions for:
    - top-down Esc dismissal
    - structured layer-stack inspection
    - page-modal footer truth

Tests to write first

- cycle doc regression for HT-003 and its follow-on backlog item
- shell regression proving Esc dismisses topmost layers before quit
- shell regression proving footer hints come from the topmost active layer
- regression proving the layer stack can be described structurally for shell layers and page modals

Risks / unknowns

- the implementation may still describe a layer stack rather than literally pushing/popping key maps as first-class runtime objects
- page-owned modals still expose only a modalKeyMap, not a richer layer object
- shell help content is still composed from binding sources rather than a fully unified input-map registry

Retrospective

What landed:

- describeFrameLayerStack() in the public TUI API
- topmost-layer-driven key routing in createFramedApp()
- topmost-layer-aware mouse blocking
- footer hints derived from the active layer instead of raw booleans
- page-modal footer truth through modal-keymap-driven help hints

What did not land:

- a literal push/pop runtime registry of layer objects
- richer page-provided layer descriptors beyond modalKeyMap
- a dedicated agent-visible shell introspection surface beyond the public layer-stack helper

Follow-on:

- HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection

254

HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection

docs/design/HT-004-promote-explicit-layer-objects-and-richer-shell-introspection.md

Cycle for turning the shell layer stack into richer explicit layer objects that own more than just branch order

Legend:

- HT — Humane Terminal

Depends on:

- HT-003 — Implement Layer Stack and Input Map Routing

Why this cycle exists

HT-003 made the shell stack real enough for routing, dismissal, and truthful footer hints.

But the layer model was still intentionally thin:

- most layer objects were inferred from booleans
- layer titles still lived in overlay render helpers
- footer hints still came from separate shell branches
- help overlay content still special-cased underlying layer kinds instead of consuming richer layer metadata

This cycle exists to make the layer object itself a better unit of truth for humans, agents, and shared shell code.

Human users / jobs / hills

Primary human users

- framed-app operators moving between workspace, settings, help, notification review, palette search, and quit confirm
- DOGFOOD users relying on the shell to explain what is active

- builders expecting shell overlays to feel coherent instead of pieced together

Human jobs

1. Trust a shell layer to own its own title and controls instead of seeing mismatched chrome.
2. Open help and get the controls for the layer beneath it without stale workspace leakage.
3. Move between shell layers without the footer and overlay titles feeling like they come from different systems.

Human hill

Each shell layer feels like a real object with its own identity and controls, not a boolean plus a pile of helper branches.

Agent users / jobs / hills

Primary agent users

- agents inspecting framed-app model state
- agents generating deterministic playbacks for shell layers
- agents that need to know which controls belong to the active or underlying layer

Agent jobs

1. Inspect the active and underlying layer objects directly instead of reverse-engineering footer/help logic.
2. Read a layer title and control ownership from the same place the shell uses.
3. Determine whether a layer has its own help source, hint source, and dismissal semantics.

Agent hill

An agent can inspect a frame model and obtain richer explicit layer objects with enough metadata to reason about titles, controls, and help ownership without parsing rendered chrome heuristically.

Human playback

1. A user opens settings in a framed app.

2. The shell creates a settings layer object with the settings title, footer hint, and help bindings.
3. The user opens help from within settings.
4. Help becomes the active layer, but the underlying settings layer still carries the controls help needs to show.
5. The help overlay lists settings controls instead of falling back to stale workspace bindings.
6. The user dismisses help and returns to settings.
7. The settings drawer title and footer still agree because both come from the same explicit layer model.

Agent playback

1. An agent inspects a framed app with settings open and a search layer above it.
2. The stack exposes explicit layer objects:
   - workspace
   - settings
   - search
3. The search layer object includes its title and hint source.
4. The underlying settings layer object includes its help source.
5. The agent can predict both the visible footer controls and the help overlay content from those layer objects.

Linked invariants

- Focus Owns Input
- Topmost Layer Dismisses First
- Visible Controls Are a Promise
- Shell Owns Shell Concerns

Implementation outline

1. Promote the public layer descriptor into a richer explicit layer object with:
   - stable id
   - title
   - hint source
   - help source
2. Let describeFrameLayerStack() accept metadata overrides for individual layers.
3. Export richer active/underlying layer helpers for shell introspection.

4. Drive shell footer hints from the active layer object instead of a separate footer-branch helper.
5. Drive help-overlay content from the underlying layer object's help source instead of hard-coded kind branching.
6. Route shell overlay titles through the same layer metadata where feasible.

Tests to write first

- cycle doc regression for HT-004 and its follow-on backlog item
- unit regression proving describeFrameLayerStack() returns richer layer metadata when supplied
- regression proving active/underlying layer helpers expose that richer metadata
- shell regression proving help opened over settings renders settings controls, not stale workspace controls

Risks / unknowns

- page-owned layers are still limited to the existing modalKeyMap seam rather than a full page-provided layer registry
- shell overlay bodies may still have some layout-specific logic even after titles and control metadata move onto layer objects
- the first explicit layer object model may still be derived rather than literally pushed/popped as runtime objects

Retrospective

What landed:

- richer explicit layer objects with title, hint-source, and help-source metadata
- exported active/underlying layer helpers for shell introspection
- footer and help resolution driven from layer metadata instead of ad hoc branching
- overlay titles routed through explicit layer objects where the shell owns the surface

What did not land:

- a full page-provided layer registry beyond modalKeyMap
- a literal pushed/popped runtime collection of first-class layer instances stored on the frame model
- agent-facing control projection beyond the richer exported layer helpers

Follow-on:

- HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection

255

HT-007 Focus Map Surface

docs/design/HT-007-focus-map-surface.md

```
title: HT-007 Focus Map Surface
legend: HT
lane: design
```

Framing

Focus behavior in complex shells and multi-pane workstations is difficult to reason about from runtime output alone. The humane terminal story is stronger when focus ownership is inspectable and explainable.

HT-007 adds a pure focus map report and surface helper. The first slice accepts focus facts from an app or shell, detects basic focus-health issues, and renders plain text or a bounded Surface.

Sponsored Users

- App authors debugging tab order and focus ownership.
- Maintainers investigating multi-pane focus regressions.
- Agent workflows that need focus facts as deterministic text.

Hills

1. A developer can render the current focus map with focused node, owner, region, tab order, and disabled state.
2. A maintainer can detect multiple focused nodes, missing focused node, focused-disabled nodes, and duplicate tab indexes.
3. A test can render the report as a Surface for docs or debug panes.

Playback Questions

- Does the helper preserve focus ownership and tab order in the report?

- Does the focused node stand out in text output?
- Does issue detection catch no focus, multiple focus, disabled focus, and duplicate tab indexes?
- Does the surface helper render a bounded report?
- Does bijou-tui export the focus map API from the source barrel?

Requirements

- Add focusMapText(nodes, options?).
- Add focusMapSurface(nodes, options?).
- Add inspectFocusMap(nodes).
- Support node id, optional owner, optional role, optional rect, optional tab index, focusable, focused, and disabled flags.
- Keep the first slice pure; it must expose focus truth, not invent a second focus system.

Acceptance Criteria

- Focused RED tests fail before implementation and pass after.
- Text output includes focused node, ownership, tab order, and issues.
- Surface output respects supplied width and height.
- Docs and changelog describe the helper.

Implementation Outline

- Implement the helper in packages/bijou-tui/src/focus-map.ts.
- Export it from packages/bijou-tui/src/index.ts.
- Keep report ordering deterministic: tab index first, then id.

Drift Check

- Scope stayed pure and report-only; the helper consumes focus facts supplied by an app or shell instead of creating a competing focus manager.
- The first issue set covers foundational focus invariants. Richer trap and dead-island detection remains a follow-up because it needs parent/edge facts that are not part of this slice.
- Ordering is deterministic: tab index first, then node id.

Playback

- inspectFocusMap() returns ordered nodes, focused node ids, and structured issues for missing focus, multiple focused nodes, focused-disabled nodes, and duplicate tab indexes.
- focusMapText() renders owner, role, rect, tab order, focusable, focused, and disabled state, with focused node rows prefixed by *.
- focusMapSurface() renders the same report as a bounded Surface.
- @flyingrobots/bijou-tui exports the focus map API from its source barrel.

Retrospective

- This pairs cleanly with the layout and input-routing inspectors: layout shows where regions are, input routing shows how events moved, and the focus map shows who currently owns focus.
- App-provided focus facts are the right boundary for the first slice. Inferring focus from rendered text would create brittle diagnostics and hide the runtime truth the helper is meant to expose.

256

HT-008 — Surface-native text bypasses the terminal sanitization boundary

docs/design/HT_HT-008-surface-native-text-bypasses-the-terminal-sanitization-boundary.md

```
title: "HT-008 — Surface-native text bypasses the terminal sanitization boundary"
legend: "HT"
cycle: "HT_HT-008-surface-native-text-bypasses-the-terminal-sanitization-boundary"
source_backlog:
↪  "docs/method/backlog/asap/HT-008-surface-native-text-bypasses-the-terminal-sanitization-boundary.md"
```

Source backlog item: docs/method/backlog/asap/HT-008-surface-native-text-bypasses-the-terminal-sanitization-boundary.md Legend: HT

Sponsors

- Human: Backlog operator
- Agent: Implementation agent

These labels are abstract roles. In this design, user means the served perspective, like in a user story, not a literal named person or specific agent instance.

Hill

TBD

Playback Questions

Human

☐ TBD

Agent

☐ TBD

Accessibility and Assistive Reading

- Linear truth / reduced-complexity posture: TBD
- Non-visual or alternate-reading expectations: TBD

Localization and Directionality

- Locale / wording / formatting assumptions: TBD
- Logical direction / layout assumptions: TBD

Agent Inspectability and Explainability

- What must be explicit and deterministic for agents: TBD
- What must be attributable, evidenced, or governed: TBD

Non-goals

☐ TBD

Backlog Context

The 2026-04-14 audit pass found that Bijou's terminal-safety story is still split across several public text entry points.

Problem

The plain-text bridges sanitize control sequences before they reach terminal output, but several public surface-native helpers still accept raw strings and write them directly into cells or OSC sequences.

Current evidence: - packages/bijou/src/core/components/badge.ts - packages/bijou/src/core/components/surface-text.ts - packages/bijou/src/core/components/hyperlink.ts

That means an app can appear "safe" because some paths sanitize terminal text while other paths still allow destructive control bytes to survive into interactive/static output.

Desired outcome

1. Define one shared trusted/untrusted text boundary for public text-bearing APIs.

2. Route badge(), surface-text helpers, hyperlink(), and transitive callers through that boundary.
3. Add regression coverage for CSI, OSC, DCS, BEL, and mixed escape payloads.
4. Document when callers should use plain text, trusted styled text, or explicit ANSI parsing.

Why ASAP

- This is the single most important remaining ship-readiness risk from the 2026-04-14 audit.
- The repo is otherwise release-green, which makes this a high-leverage hardening task instead of speculative cleanup.

Hill

Every public text-bearing helper passes through one auditable terminal-safety contract before it becomes cells or terminal control output.

257

Design Records

docs/design/README.md

This directory preserves technical decisions and implementation evidence. Current scope and release order live in GitHub issues and milestones; Linear is the chosen execution tracker, pending connection and workspace/team selection.

Write a design record when a change needs a durable technical decision. Name the problem, scope, non-goals, observable contract, acceptance checks, prerequisites, and validation plan. Link the work item and PR. Update the record with actual results when the change lands.

Existing cycle identifiers and templates preserve historical provenance. New work does not require METHOD ceremonies, sponsor fields, a cycle branch name, or an advance shaping commit. Keep existing filenames stable for linked evidence.

The complete design corpus is also available as the Design Book, with LaTeX source, a PDF edition, TikZ diagrams, and actual DOGFOOD cell specimens.

IV

Releases and migration

From implementation to a release claim

Release material answers a different set of questions from a design proposal. What changed for a user of a particular version? Which contracts or defaults moved? What must an existing application do to upgrade? Which checks and artifacts support publishing that version?

The chapters in this part retain release packets, change narratives, migration guides and release-mechanics notes from the source collection. Read each chapter with its version and date in view. A guide written for an earlier upgrade describes that transition; it does not automatically describe the steps required by a later release.

Use the records together

A release narrative describes the intended user-facing change. Migration guidance explains the work that change asks of an adopter. A release packet can link the implementation, review and validation evidence behind the claim. The design records in the preceding part provide the earlier reasoning and boundaries.

When a statement matters to an upgrade, follow it through those records and verify the package version being used. Distinguish a published package from a prerelease, a completed validation step from a planned gate, and a recorded limitation from a promised follow-up. An absence of a warning in a historical release note is not a fresh compatibility check.

Scope and live operations

Roadmap counts, milestone assignments and tracker status can change after a document is written. Their historical wording remains useful for understanding a release decision, while the tracker remains the place to establish current scope. The same distinction applies to workflow and publishing instructions preserved in the book: current repository agreements and authorized release tooling govern a new publication.

This part is most useful as a bridge. Read from the behaviour an application relies on, through the version that changed it, to the implementation and evidence that made the change acceptable.

258

Bijou Workflow

docs/WORKFLOW.md

GitHub issues and milestones own the roadmap and release scope. The next release is stable v8.0.0, following v8.0.0-rc.1.

Linear is the chosen execution tracker. Connection and workspace/team selection are still pending. Existing GitHub issues remain live until their Linear counterparts are created, linked, and verified. Do not duplicate execution state in repository backlog files or invent a completed migration.

Working Loop

1. Inspect the work item, linked PRs, and current Git state.
2. Sync the merge target and create a descriptive branch without disturbing unrelated local work.
3. Define one observable outcome, scope, acceptance checks, and real prerequisites.
4. Implement with relevant regression evidence. Use a design record when the change needs a durable technical decision.
5. Update affected documentation and the changelog, run the relevant checks, and review the diff.
6. Link the work item and PR. Request review when the change is ready.
7. After merge, retain the integration link and update the external tracker.

Use focused conventional commits. Do not amend, rebase, force-push, or push to main without explicit authorization. There is no mandatory cycle ceremony, advance shaping commit, sponsor taxonomy, or retrospective template.

GitHub Comment Safety

Structured review comments often contain Markdown tables, code spans, shell examples, $() text, and copied command output. Post those comments through --body-file with a quoted heredoc or a real body file so the shell cannot evaluate the body before GitHub receives it. Do not use inline --body "..." for self-review findings, Code Lawyer reports, activity summaries, or any other Markdown-heavy comment.

Safe PR comment pattern:

```
gh pr comment "$PR_NUMBER" --body-file - <<'EOF'
# Self-Code Review

| Severity | Count |
| :--- | ---: |
| P0 | 0 |

Commands stay literal: `npm test`, `gh pr checks`, and $().
EOF
```

Safe issue comment pattern:

```
gh issue comment "$ISSUE_NUMBER" --body-file - <<'EOF'
Linked artifacts:
- Design: `docs/design/WF-128-safe-gh-comments-and-milestone-items.md`
- Tests: `tests/cycles/WF-128/safe-comments-and-milestone-items.test.ts`
EOF
```

Use a temporary file with --body-file "$COMMENT_BODY_FILE" when the comment is generated by a script. Delete the temporary file after the command succeeds.

Evidence And Release Mechanics

- release.md owns release validation and publishing mechanics.
- Documentation Standards owns documentation upkeep.
- CHANGELOG.md records merged behavior.
- docs/design/, tests/cycles/, and release packets preserve technical proof.
- docs/method/ is historical material, not an active workflow or backlog.

Context Recovery

Start with GitHub issues, milestones, open and recently merged PRs, then the linked Linear items once connected. Read relevant design/release evidence and verify local Git history, status, and worktrees. Repository planning mirrors are retired; tracker state must be read from the tracker.

259

Bijou 7.1.0 Release Evidence

docs/releases/7.1.0/README.md

Bijou 7.1.0 is the post-V7 minor release that turns the accumulated V8-facing proof work into a clean public package boundary without expanding the release train. It ships the Runtime Graph and Scene IR seed work, the first GraphQL-authored DOGFOOD block fixture, theme/token polish, raster-to-glyph tooling, terminal-rendering fixes, and release-prep guardrails that make the next release checklist auditable.

This packet was prepared on the release branch before the public tag existed. The release tag and tag push are final-main steps and are excluded from this prep branch by operator instruction.

Release Summary

- Version: 7.1.0
- Tag: v7.1.0 (created only from the merged origin/main release commit)
- Previous public tag: v7.0.0
- Release type: stable minor
- npm dist-tag after publish: latest
- Release-prep branch: release/v7.1.0
- Release date: 2026-06-14
- Publish surface: npm workspace packages only

Tracker And Goalpost Map

Each goalpost records the release-law evidence required by docs/release.md: tracker, design, PRs, completed slices, deterministic proof or fixture, replay command, witness, and residual-risk disposition.

Runtime Graph And Scene IR Seed

- Tracker: broad V8 parent #302.
- Design: DX-043.
- Landed PRs: #323.

- Completed slices: 2 planned proof slices (ui-scene-ir/1 seed and terminal receipt proof) plus 8 review-hardening fixes.
- Canonical proof input: synthetic scenes in packages/bijou/src/core/ui-scene-ir.test.ts.
- Replay: npm test -- --run packages/bijou/src/core/ui-scene-ir.test.ts.
- Witness: tests assert SHA-256 scene hashes, structural receipts, source maps, terminal lowering receipts, lower modes, unsupported-node rejection, negative clipping, top-level JSON guards, and inline dependency receipts.
- Residual risk: V8 product contract is explicitly not shipped here; this is the seed proof only.

GraphQL-Authored Bijou Block Proof

- Tracker: broad V8 parent #302.
- Design: DX-044.
- Landed PRs: #324.
- Completed slices: 1 planned compiler/lowering proof slice plus 5 review, parser-hardening, and documentation fixes.
- Canonical proof input: SDL fixtures embedded in packages/bijou/src/core/graphql-bijou-block.test.ts.
- Replay: npm test -- --run packages/bijou/src/core/graphql-bijou-block.test.ts.
- Witness: tests assert bijou-block/1 artifact creation, lowering into ui-scene-ir/1, duplicate identity rejection, parse hardening, source anchors, token refs, i18n facts, actions, bindings, and lower-mode hashes.
- Residual risk: GraphQL authoring remains a proof path toward V8, not the full product authoring contract.

GraphQL Block Groups And Debug Facts

- Tracker: broad V8 parent #302.
- Design: DX-045.
- Landed PRs: #325.
- Completed slices: 1 grouped-authoring and debug-facts proof slice.
- Canonical proof input: grouped SDL cases in packages/bijou/src/core/graphql-bijou-block.test.ts.
- Replay: npm test -- --run packages/bijou/src/core/graphql-bijou-block.test.ts.
- Witness: tests assert group directives, grouped field membership, deterministic graphql-bijou-block-debug/1 summaries, and lower-mode/debug hash stability.
- Residual risk: group semantics are enough for the release proof; broad product taxonomy remains V8 work.

DOGFOOD GraphQL Navigation Fixture

- Tracker: #329, child of broad V8 parent #302.

- Design: DX-046.
- Landed PRs: #330.
- Completed slices: 1 DOGFOOD fixture slice plus 1 review-discovered ANSI reset fix.
- Canonical fixture: examples/docs/fixtures/graphql/navigation-list.graphql.
- Replay: npm test -- --run packages/bijou/src/core/graphql-bijou-block.test.ts.
- Witness: tests compile the DOGFOOD NavigationListBlock fixture into bijou-block/1, lower it into ui-scene-ir/1, prove terminal output, and assert debug facts for groups, fields, source maps, i18n keys, token refs, actions, bindings, lower modes, and hashes.
- Residual risk: only one real DOGFOOD fixture is shipped; additional product fixtures belong to V8.

Theme Tokens, Modes, And Inspector Work

- Tracker: #315 for Theme Lab direction plus the V8-facing design-token lane tracked through the release roadmap.
- Design: DL-013, DL-014, DL-015, and DL-016.
- Landed PRs: #309, #310, and #319.
- Completed slices: 4 theme/product slices: token builder, DOGFOOD token retuning with first-party presets, safe-pair contracts with Theme Lab and Inspector surfaces, and mode-aware shell theme families.
- Canonical proof inputs: packages/bijou/src/core/theme/builder.test.ts, packages/bijou/src/core/theme/presets.test.ts, packages/bijou/src/core/theme/doctor.test.ts, tests/cycles/DL-013/design-token-theme-builder-spec.test.ts, tests/cycles/RE-017/frame-shell-theme-dogfood-demo.test.ts, and scripts/docs-preview.test.ts.
- Replay: npm test -- --run packages/bijou/src/core/theme/builder.test.ts packages/bijou/src/core/theme/presets.test.ts packages/bijou/src/core/theme/doctor.test.ts tests/cycles/DL-013/design-token-theme-builder-spec.test.ts tests/cycles/RE-017/frame-shell-theme-dogfood-demo.test.ts scripts/docs-preview.test.ts.
- Witness: tests assert immutable dark/light theme definitions, duplicate and missing-mode token rejection, tokenRef() / resolveThemeColorRef(), first-party BIJOU_DARK / BIJOU_LIGHT presets, safe-pair validation, DOGFOOD shell theme choices, and Theme Lab / Theme Inspector rendering.
- Residual risk: the full product Theme Lab and workbench remain V9-plus work; this release ships token contracts, shell-mode selection, and inspection surfaces.

Raster-To-Glyph And Title Surface Polish

- Tracker: V7 DOGFOOD title and side-app polish tracked through the release roadmap.
- Design: DF-060, DF-072, DF-073, and DX-041.
- Landed PRs: #297 and #304.

- Completed slices: 3 product slices: release title gallery, raster-to-glyph title art, and the image-to-glyph side app, plus review polish.
- Canonical proof inputs: packages/bijou-tui/src/raster-glyph.test.ts, scripts/image-viewer.test.ts, scripts/svg-raster.test.ts, scripts/docs-preview.test.ts, and tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts.
- Replay: npm test -- --run packages/bijou-tui/src/raster-glyph.test.ts scripts/image-viewer.test.ts scripts/svg-raster.test.ts scripts/docs-preview.test.ts tests/cycles/DF-060/v7-dogfood-release-title-screen.test.ts.
- Witness: tests assert raw RGBA-to-Surface rendering, ASCII / Braille / quadrant glyph modes, color preservation, SVG rasterization, V7 title wake behavior, release-title gallery previewing, and image-viewer controls.
- Residual risk: Geordi/GPU title rendering and full visual workbench support remain V8/V9-plus work; this release ships deterministic CPU glyph rendering and DOGFOOD proof surfaces.

DOGFOOD Rendering And Navigation Fixes

- Tracker: review-discovered regression repairs in the v7.1 release window.
- Design lineage: DX-031 for Blocks navigation behavior and DX-046 for the DOGFOOD fixture proof surface.
- Landed PRs: #330 for ANSI scoped reset repair and #331 for Blocks parent focus and resize repaint recovery.
- Completed slices: 3 regression-fix slices: scoped ANSI resets, Blocks preview parent focus, and terminal resize repaint.
- Canonical proof inputs: tests/cycles/DX-031/dogfood-blocks-section.test.ts, tests/cycles/DF-068/dogfood-block-preview-regressions.test.ts, packages/bijou/src/core/render/differ.test.ts, and packages/bijou-tui/src/runtime.test.ts.
- Replay: npm test -- --run packages/bijou/src/core/render/differ.test.ts tests/cycles/DX-031/dogfood-blocks-section.test.ts tests/cycles/DF-068/dogfood-block-preview-regressions.test.ts packages/bijou-tui/src/runtime.test.ts packages/bijou-tui/src/screen.test.ts.
- Witness: tests assert scoped SGR 39 / 49 color resets, Up navigation from AppShell back to the Block Preview parent row, resize repaint of blank cells, and reusable invalidated resize buffers with no NUL bytes.
- Residual risk: no accepted release risk; these are closed regressions.

Release Readiness Guardrail

- Tracker: #270.
- Design: WF-132.
- Landed PRs: #332.
- Completed slices: 1 milestone-aware release-readiness guardrail slice plus 1 roadmap/test alignment slice.
- Canonical proof input: mocked milestone/docs snapshots in scripts/release-readiness.test.ts.
- Replay: npm test -- --run scripts/release-readiness.test.ts.

- Witness: tests assert milestone report formatting, open tracker issue rejection, lingering work-in-progress rejection, stale docs rejection, release-packet presence, package-smoke coverage, and CLI argument parsing.
- Residual risk: no accepted release risk; future work may move operator-only checks into a dedicated release:prep script.

DOGFOOD I18n Debt Guardrail

- Tracker: #312.
- Design: WF-132.
- Landed PRs: #332.
- Completed slices: 1 source-discovery guardrail slice.
- Canonical proof input: source-discovery fixtures in scripts/dogfood-i18n-debt.test.ts.
- Replay: npm run dogfood:i18n:debt and npm test -- --run scripts/dogfood-i18n-debt.test.ts.
- Witness: tests assert examples/docs/**/*.ts discovery, explicit tooling-only exclusions, source-surface counts, supported-locale reporting, and raw-string debt ratchet behavior.
- Residual risk: no accepted release risk; the scanner remains a guardrail, not a localization runtime rewrite.

The broad #302 tracker remains future-facing for V8. Bijou 7.1.0 ships the first proof slices and fixture evidence, not the full Runtime Graph product.

Automated Evidence Matrix

Gate	Command or source	Expected result	Status
Phase 1 clean tree	git status --porcelain=v1	No output before release prep.	Passed before branching.
Phase 1 branch guard	git rev-parse --abbrev-ref HEAD	main before release branch creation.	Passed before branching.
Phase 1 origin parity	git rev-parse HEAD origin/main	Both SHAs equal.	Passed at c950d257d248e1b45ffbc3bf7fb741749af7caf9.
Latest public release	git tag --merged HEAD --list 'v*' --sort=-version:refname | head -n 1	v7.0.0.	Passed.
Target milestone issue gate	gh issue list --repo flyingrobots/bijou --milestone v7.1.0 --state open	Empty list.	Passed before release prep.
Target milestone PR gate	gh pr list --repo flyingrobots/bijou --state open --search 'milestone:v7.1.0'	Empty list before the release PR opens; after merge and before tagging, rerun this gate and require no open milestone PRs.	Passed before release prep; final live gate required before tag.

Gate	Command or source	Expected result	Status
High-priority issue gate	gh issue list --repo flyingrobots/bijou --state open --label priority:high	Empty list, or packeted risk.	Passed before release prep.
Version lock-step	npm run version 7.1.0 and npm run release:preflight	All workspace manifests and internal dependencies agree on 7.1.0.	Passed in release prep.
Release readiness	npm run release:readiness -- --milestone v7.1.0	Milestone report passes, then local gauntlet passes.	Passed in release prep: 346 test files and 3844 tests.
Documentation inventory	npm run docs:inventory	Inventory passes with docs/releases/7.1.0/.	Passed in release prep.
Runtime audit	npm audit --omit=dev --audit-level=high	No high or critical runtime vulnerability.	Passed in release prep: 0 vulnerabilities.
Branch release dry run	Release Dry Run run 27477234620 on release/v7.1.0 at 12003411e75be5b2cb3dff95e549af8b1fa19bf4	Build, lint, tests, DOGFOOD smoke, pack checks, npm publish dry-runs, and notes preview pass.	Passed for the first release-prep commit; final pre-tag dry run must be rerun after this PR merges if the tag target differs.

Human Review Matrix

Surface	Review disposition
docs/CHANGELOG.md	Updated from Unreleased into [7.1.0] - 2026-06-14; content reflects the diff since v7.0.0.
README.md	Updated with a concise v7.1.0 user-facing What's New section and links to release docs.
ARCHITECTURE.md	Reviewed; no change required for this minor release because package boundaries and port ownership remain unchanged.
docs/VISION.md and docs/METHOD.md	Reviewed; no change required because project identity and work doctrine did not change.
docs/ROADMAP.md and docs/BEARING.md	Updated for the tag commit: v7.1.0 is the latest shipped release boundary, v7.1.0 is closed lineage, and v8.0.0 is the next feature horizon.
docs/DOGFOOD.md	Updated by release-readiness guardrail work to reflect DOGFOOD i18n/source posture.
Package READMEs and API references	Reviewed; no targeted README change required beyond the root release summary because APIs are additive or internal-proof oriented.
Release docs	Added this evidence packet plus long-form What's New and migration guide.

Deterministic Reproducibility Record

Claim	Replay command / fixture	Stable assertion
DOGFOOD GraphQL NavigationList Block fixture compiles through the proof chain.	npm test -- --run packages/bijou/src/core/graphql-bijou-block.test.ts with examples/docs/fixtures/graphql/navigation-list.graphql.	Tests assert bijou-block/1, ui-scene-ir/1, terminal proof, debug facts, source maps, i18n keys, token refs, actions, bindings, lower modes, and hashes.
Terminal resize recovery repaints blank cells after geometry changes.	npm test -- --run packages/bijou-tui/src/runtime.test.ts packages/bijou-tui/src/screen.test.ts.	Post-resize frame assertions include trailing blank-cell repaint data and no NUL bytes.
DOGFOOD Blocks guide navigation can focus the Block Preview parent row.	npm test -- --run tests/cycles/DX-031/dogfood-blocks-section.test.ts tests/cycles/DF-068/dogfood-block-preview-regressions.test.ts.	Focus moves Up from AppShell to the Block Preview parent while selection stays on the preview group.
ANSI scoped foreground/background resets do not leak into DOGFOOD navigation labels.	npm test -- --run packages/bijou/src/core/render/differ.test.ts tests/cycles/DF-068/dogfood-block-preview-regressions.test.ts.	Scoped SGR 39 and 49 reset only the intended color channel.
DOGFOOD i18n debt source discovery covers new docs modules by default.	npm run dogfood:i18n:debt and npm test -- --run scripts/dogfood-i18n-debt.test.ts.	Source-surface counts include discovered examples/docs/**/*.ts modules unless explicitly excluded as tooling-only.
Milestone-aware release readiness blocks incomplete release prep.	npm run release:readiness -- --milestone v7.1.0 and npm test -- --run scripts/release-readiness.test.ts.	The report rejects open tracker issues, lingering work-in-progress, missing release evidence packets, stale docs posture, and missing package-smoke coverage.
Current release docs render through DOGFOOD.	npm run smoke:dogfood:docs -- --skip-build.	DOGFOOD docs smoke can navigate release docs for the current package version.

Package And Registry Verification Plan

After the release PR is merged and only after explicit operator approval to tag:

1. Fast-forward local main to the exact origin/main release commit.
2. Re-run the live tracker gates: gh issue list --repo flyingrobots/bijou --milestone v7.1.0 --state open, gh pr list --repo flyingrobots/bijou --state open --search 'milestone:v7.1.0', and gh issue list --repo flyingrobots/bijou --state open --label priority:high. Abort before tagging if any release-blocking item appears without an explicit packeted risk disposition.
3. Re-run npm run release:preflight and npm run docs:inventory.
4. Run the Release Dry Run workflow against the exact merged release commit, or against the release branch head only if it still matches the tag target. Record the run URL and abort before tagging if the run is not green.
5. Create the annotated v7.1.0 tag from clean synced main.
6. Push only v7.1.0.
7. Verify tag guard and publish workflows pass.

8. Verify npm reports 7.1.0 and latest for each automated publish package: @flyingrobots/bijou, @flyingrobots/bijou-node, @flyingrobots/bijou-tui, @flyingrobots/bijou-tui-app, create-bijou-tui-app, @flyingrobots/bijou-i18n, @flyingrobots/bijou-i18n-tools, @flyingrobots/bijou-i18n-tools-node, @flyingrobots/bijou-i18n-tools-xlsx, and @flyingrobots/bijou-mcp.
9. Verify the GitHub Release exists with generated notes.

Residual Risk

- A branch dry-run workflow passed for an earlier release-prep commit. Because this packet changed after that run, the final pre-tag Release Dry Run must be rerun on the exact merged release commit before tag creation. The final tag guard, publish workflow, npm registry verification, and GitHub Release verification still cannot occur in this prep pass because the operator explicitly prohibited tag creation and tag pushes.
- The broad Runtime Graph and Scene IR product contract remains V8 work. The 7.1.0 release intentionally ships proof slices and guardrails only.

No accepted unresolved high-priority issue is recorded for this release packet.

260

Bijou 7.2.0 Release Evidence

docs/releases/7.2.0/README.md

Bijou 7.2.0 is a narrow stabilization and demo-integrity release after the 7.1.0 release-video rehearsal exposed user-facing seams in framework input, DOGFOOD localization and themes, Blocks documentation, release-story surfaces, dependency posture, and release bookkeeping.

This packet was started on the release-gate branch, finalized on the release-prep branch, and closed from the verified origin/main release commit. The public v7.2.0 tag was created from 123f4a6150963932118cbf763a8a6ec03487e2 27 on 2026-07-05 after final-main validation passed. Tag Guard, Release workflow, tag CI, GitHub Release verification, and npm registry verification all passed after the tag was pushed.

Release Summary

- Version: 7.2.0
- Tag: v7.2.0
- Previous public tag: v7.1.0
- Release commit: 123f4a6150963932118cbf763a8a6ec03487e227
- Release type: stable minor
- npm dist-tag after publish: latest
- Release-prep branch: cycle/v72-release-prep
- Release-gate lineage branch: cycle/v72-release-gate-354
- Release date: 2026-07-05
- Publish surface: npm workspace packages only
- Release workflow: https://github.com/flyingrobots/bijou/actions/runs/28736930944

Tracker And Goalpost Map

Each goalpost records the release-law evidence required by docs/release.md: tracker, design, PRs, completed slices, deterministic proof or fixture, replay command, witness, and residual-risk disposition.

v7.2 Stabilization Goalpost

- Tracker: #354.
- Design lineage: RE-041, DX-047, DL-017, DL-018, LX-020, and DF-078.
- Landed PRs: #355, #359, #360, #361, #362, #453, #452, and #460.
- Completed slices: framework input fallthrough and helper exports, DOGFOOD locale fallback honesty, DOGFOOD light-theme readability, first-party theme variant coverage, Blocks app-binding snippets, mouse hover tracking, and in-app release-story surfaces.
- Canonical proof inputs: the tests and DOGFOOD fixtures named in the per-slice sections below.
- Replay: run npm run release:readiness from the release-prep branch, then run npm run release:readiness -- --milestone v7.2.0 again from the exact merged final-main release commit before tagging.
- Witness: local release-gate validation on 2026-07-04 passed npm run release :preflight, npm run docs:inventory, npm audit --audit-level=high, npm audit --omit=dev --audit-level=high, and npm run release:readiness without --mile stone. Release-prep validation on 2026-07-05 passed npm run release:readine ss -- --milestone v7.2.0 after the lock-step version bump and dated changelog boundary. Final-main validation on 2026-07-05 passed npm run rele ase:preflight, npm run docs:inventory, npm audit --omit=dev --audit-level=h igh, and the milestone-aware readiness gate from commit 123f4a61; the Release Dry Run, Tag Guard, Release workflow, tag CI, GitHub Release, and npm registry checks all passed.
- Residual risk: no accepted release risk.

Framework Input Stabilization

- Trackers: #344, #345, and #353.
- Design: RE-041.
- Landed PRs: #355.
- Completed slices: workspace mouse movement, release, and non-left press events fall through to the active page; page-scoped frame helpers are exported from the @flyingrobots/bijou-tui root; scripted driver helpers can generate deterministic move, press, release, wheel, and raw SGR mouse sequences.
- Canonical proof inputs: packages/bijou-tui/src/driver.part02.test.ts, packa ges/bijou-tui/src/driver.part03.test.ts, packages/bijou-tui/src/index.test. ts, packages/bijou-tui/src/app-frame-core.part02.test.ts, packages/bijou-tu i/src/app-frame-routing.part06.test.ts, and WF-130 roadmap tests.
- Replay: npx vitest run --config vitest.config.ts packages/bijou-tui/src/dri ver.part02.test.ts packages/bijou-tui/src/driver.part03.test.ts packages/bi jou-tui/src/index.test.ts packages/bijou-tui/src/app-frame-core.part02.test .ts packages/bijou-tui/src/app-frame-routing.part06.test.ts tests/cycles/WF -130/roadmap-goalpost-policy.part01.test.ts tests/cycles/WF-130/roadmap-goa lpost-policy.part02.test.ts tests/cycles/WF-130/roadmap-goalpost-policy.par t03.test.ts.
- Witness: PR #355 closed all three trackers and merged on 2026-06-15 at 94db 990f.

- Residual risk: no accepted release risk.

Mouse Hover Tracking

- Tracker: release-video mouse input follow-up.
- Design lineage: framework input stabilization plus the runtime/worker mouse tests added in PR #453.
- Landed PRs: #453.
- Completed slices: RunOptions.mouseMode supports press, drag, and any-event any SGR tracking; the event bus splits bundled SGR mouse packets; the Node worker host preserves mouse parsing while terminal control stays in the main thread.
- Canonical proof inputs: runtime, event-bus, and worker mouse tests touched by PR #453.
- Replay: npm run code-dojo:ci.
- Witness: PR #453 merged on 2026-07-03 at e15e771d after full Code Dojo CI and pre-push verification.
- Residual risk: no accepted release risk.

DOGFOOD Locale Demo Readiness

- Tracker: #340.
- Design: LX-020.
- Landed PRs: #359.
- Completed slices: non-English DOGFOOD release-demo paths avoid missing localization marker text by falling back to source copy, while English-source Markdown remains explicitly labeled and maintainer i18n gates remain intact.
- Canonical proof inputs: DOGFOOD i18n completeness, build, and debt gates.
- Replay: npm run dogfood:i18n:complete && npm run dogfood:i18n:check && npm run dogfood:i18n:debt.
- Witness: PR #359 closed #340 and merged on 2026-06-15 at 65fb3969.
- Residual risk: full translation workbench remains future scope.

DOGFOOD Light Theme Readiness

- Tracker: #341.
- Design: DL-017.
- Landed PRs: #360.
- Completed slices: modal and drawer chrome paint panel backgrounds behind border cells, light/dark subtle chrome clears the same diagnostic floor, and DOGFOOD theme diagnostics cover chrome tokens used by borders, scrollbars, focus gutters, and modal surfaces.
- Canonical proof inputs: tests/cycles/DL-017/dogfood-light-theme-readiness.part01.test.ts, tests/cycles/DL-017/dogfood-light-theme-readiness.part02.test.ts, tests/cycles/DL-017/dogfood-light-theme-readiness.part03.test.ts, tests/cycles/DL-017/dogfood-light-theme-readiness.part04.test.ts, tests/cycles

/DL-017/dogfood-light-theme-readiness.part05.test.ts, tests/cycles/RE-017/frame-shell-theme-dogfood-demo.test.ts, packages/bijou-tui/src/app-frame-overlays.test.ts, packages/bijou-tui/src/app-frame-render.part01.test.ts, packages/bijou-tui/src/app-frame-render.part02.test.ts, packages/bijou-tui/src/app-frame-input-overlays.part03.test.ts, packages/bijou-tui/src/app-frame-settings.part02.test.ts, packages/bijou/src/core/theme/presets.test.ts, and packages/bijou/src/core/theme/doctor.test.ts.

- Replay: npx vitest run --config vitest.config.ts tests/cycles/DL-017/dogfood-light-theme-readiness.part01.test.ts tests/cycles/DL-017/dogfood-light-theme-readiness.part02.test.ts tests/cycles/DL-017/dogfood-light-theme-readiness.part03.test.ts tests/cycles/DL-017/dogfood-light-theme-readiness.part04.test.ts tests/cycles/DL-017/dogfood-light-theme-readiness.part05.test.ts tests/cycles/RE-017/frame-shell-theme-dogfood-demo.test.ts packages/bijou-tui/src/app-frame-overlays.test.ts packages/bijou-tui/src/app-frame-render.part01.test.ts packages/bijou-tui/src/app-frame-render.part02.test.ts packages/bijou-tui/src/app-frame-input-overlays.part03.test.ts packages/bijou-tui/src/app-frame-settings.part02.test.ts packages/bijou/src/core/theme/presets.test.ts packages/bijou/src/core/theme/doctor.test.ts.
- Witness: PR #360 closed #341 and merged on 2026-06-16 at d06f8cd4.
- Residual risk: no accepted release risk.

First-Party Theme Variant Coverage

- Tracker: #343.
- Design: DL-018.
- Landed PRs: #362.
- Completed slices: DOGFOOD theme family exposes dark and light modes; other first-party shell themes are classified as concrete single-mode themes; Ctrl+T and command-palette switching give explicit feedback when no alternate mode exists.
- Canonical proof inputs: tests/cycles/DL-018/first-party-theme-variant-coverage.test.ts, packages/bijou-tui/src/app-frame-notifications.part05.test.ts, packages/bijou-tui/src/app-frame-modal-routing.part01.test.ts, and scripts/docs-preview-landing.part04.test.ts.
- Replay: npx vitest run --config vitest.config.ts tests/cycles/DL-018/first-party-theme-variant-coverage.test.ts packages/bijou-tui/src/app-frame-notifications.part05.test.ts packages/bijou-tui/src/app-frame-modal-routing.part01.test.ts scripts/docs-preview-landing.part04.test.ts.
- Witness: #343 was revalidated and closed on 2026-07-03 after PR #362 merged at 6c78d196.
- Residual risk: product-grade Theme Lab work remains V9-plus scope.

Blocks App-Binding Snippets

- Tracker: #342.
- Design: DX-047.
- Landed PRs: #361.

- Completed slices: DOGFOOD Blocks docs now show how CounterDemoBlock becomes application-owned state, key-to-command-intent routing, update handling, render-time config, and pipe/accessibility lower-mode output.
- Canonical proof inputs: tests/cycles/DX-047/blocks-app-binding-snippets.test.ts and tests/cycles/DX-031/dogfood-blocks-section.part01.test.ts, tests/cycles/DX-031/dogfood-blocks-section.part02.test.ts, tests/cycles/DX-031/dogfood-blocks-section.part03.test.ts, and tests/cycles/DX-031/dogfood-blocks-section.part04.test.ts.
- Replay: npx vitest run --config vitest.config.ts tests/cycles/DX-047/blocks-app-binding-snippets.test.ts tests/cycles/DX-031/dogfood-blocks-section.part01.test.ts tests/cycles/DX-031/dogfood-blocks-section.part02.test.ts tests/cycles/DX-031/dogfood-blocks-section.part03.test.ts tests/cycles/DX-031/dogfood-blocks-section.part04.test.ts.
- Witness: PR #361 closed #342 and merged on 2026-07-02 at 6b56a97a.
- Residual risk: no accepted release risk.

DOGFOOD Release-Story Surfaces

- Tracker: #335.
- Design: DF-078.
- Landed PRs: #452 and #460.
- Completed slices: DOGFOOD Release includes Current Release Story, GraphQL Proof Walkthrough, and CHANGELOG History in the main reader flow; localized Markdown mirrors load at runtime; release-story guide chrome is cataloged; CHANGELOG boundaries are derived from docs/CHANGELOG.md.
- Canonical proof inputs: tests/cycles/DF-023/publish-repo-package-and-release-guides-in-dogfood.test.ts and tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts.
- Replay: npm test -- --run tests/cycles/DF-023/publish-repo-package-and-release-guides-in-dogfood.test.ts tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts.
- Witness: PR #460 closed #335 and merged on 2026-07-04 at b8b4eb2 after all review threads were resolved and GitHub Actions were green.
- Residual risk: structured multilingual changelog source is tracked separately as future work in #454.

Code Dojo And Standards Ratchets

- Trackers: #364 and #366.
- Design: WF-134 plus the Code Dojo exception ledger.
- Landed PRs: #362 and #365, followed by later ratchet slices recorded in docs/CHANGELOG.md.
- Completed slices: installed the TypeScript Code Standards artifact, wired Code Dojo hooks and CI lanes, made standards debt visible through ratcheting baselines, and reduced live ESLint debt below the first ratchet target.
- Canonical proof inputs: docs/typescript-code-standards.editors-edition.md, docs/code-dojo-exceptions.md, Code Dojo scripts, and WF-130 roadmap tests.

- Replay: npm run code-dojo:verify.
- Witness: current release-gate validation passed npm run release:readiness, whose gauntlet includes build, lint, code size, typecheck, DOGFOOD gates, workflow shell preflight, release metadata preflight, frame tests, examples, canaries, DOGFOOD smoke, and the full chunked Vitest suite.
- Residual risk: standards debt is ratcheted, not zero. The exception ledger remains binding after release.

Dependency Security

- Trackers: #357 and #370.
- Designs and PRs: WF-133 through #358, plus the Hono Dependabot security update in #363.
- Completed slices: esbuild resolves to 0.28.1; hono resolves to 4.12.25; package-lock.json records node_modules/esbuild as a dev dependency; default-branch audit and Dependabot alert checks were verified before #370 closed.
- Canonical proof inputs: npm lockfile, dev-inclusive npm audit, runtime-only npm audit --omit=dev, and Dependabot alert API.
- Replay: run npm audit --audit-level=high for the full dependency graph, then npm audit --omit=dev --audit-level=high for the runtime-only release boundary.
- Witness: release-gate validation on 2026-07-04 reported found 0 vulnerabilities for the dev-inclusive audit and the runtime-only audit.
- Residual risk: no accepted release risk.

Automated Evidence Matrix

Gate	Command or source	Expected result	Status
Release metadata preflight	npm run release:preflight	Lock-step workspace metadata is valid.	Passed on release-prep branch and final-main release commit.
Docs inventory	npm run docs:inventory	Documentation manifest remains valid.	Passed on release-prep branch and final-main release commit.
Dev-tooling dependency audit	npm audit --audit-level=high	Zero high or critical vulnerabilities across production and development dependencies, including dev-tooling packages such as esbuild.	Passed: found 0 vulnerabilities.
Runtime dependency audit	npm audit --omit=dev --audit-level=high	Zero high or critical runtime vulnerabilities.	Passed on release-prep branch and final-main release commit: found 0 vulnerabilities.

Gate	Command or source	Expected result	Status
Focused roadmap proof	npx vitest run --config vitest.config.ts tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts tests/cycles/WF-130/roadmap-goalpost-policy.part02.test.ts tests/cycles/WF-130/roadmap-goalpost-policy.part03.test.ts	Roadmap snapshot, release posture, Beyond count, and Markdown policy are self-consistent.	Passed on release-prep branch.
Release gauntlet	npm run release:readiness	Local release-readiness gauntlet passes without live milestone checks.	Passed on release-gate and release-prep lineage.
Milestone-aware readiness	npm run release:readiness -- --milestone v7.2.0	Blocks until the target milestone has zero open tracker items, including issues or pull requests, and no WIP labels.	Passed on release-prep branch and final-main release commit on 2026-07-05.
Release Dry Run	GitHub Actions run 28736681371	Packed files, publish dry-runs, DOGFOOD smokes, and release evidence stay green from main.	Passed before tag creation.
Tag Guard	GitHub Actions run 28736930947	The pushed tag resolves to the intended release commit and metadata is valid.	Passed after v7.2.0 tag push.
Publish workflow	GitHub Actions run 28736930944	Automated npm publishing succeeds for every workspace package.	Publish workflow passed on 2026-07-05.
Tag CI	GitHub Actions run 28736930919	The tag build/test lane succeeds.	Passed after v7.2.0 tag push.
GitHub Release	https://github.com/flyingrobots/bijou/releases/tag/v7.2.0	Release notes are published for the public tag.	Published on 2026-07-05.
npm registry verification	npm view <package> version dist-tags --json	Every automated package reports version 7.2.0 with latest.	npm registry verification passed for every automated package.

Human Review Matrix

Surface	Review disposition
docs/CHANGELOG.md	v7.2 framework input, mouse tracking, DOGFOOD demo integrity, Code Dojo, and security changes now live under the dated 7.2.0 boundary.
README.md	Front-door What's New now describes v7.2.0 and links to the v7.2 release docs.
docs/ROADMAP.md	Updated on 2026-07-05 from GitHub milestone state: v7.2 has zero open and seventeen closed milestone items, Beyond has 39 open items, and v8 is staged separately.
docs/BEARING.md	Updated after release so the next target is #457 and the next feature horizon is V8.
docs/releases/7.2.0/README.md	This packet records release-prep, final-main, tag, publish, GitHub Release, and npm registry evidence.

Surface	Review disposition
docs/releases/7.2.0/whats-new.md	Versioned v7.2 release overview is ready for DOGFOOD and public release docs.
docs/releases/7.2.0/migration-guide.md	Versioned v7.2 migration guidance is ready for DOGFOOD and public release docs.
Package READMEs	No package front-door API positioning was changed by this release-gate branch.
ARCHITECTURE.md	No port, adapter, package boundary, storage, or rendering-contract update is required by this release-gate branch.

Package And Registry Verification Plan

The release-prep PR performed the package metadata mutation and validated the resulting branch before it merged:

```
npm run version 7.2.0
npm install --package-lock-only --ignore-scripts
npm run release:preflight
npm run docs:inventory
npm audit --audit-level=high
npm run release:readiness -- --milestone v7.2.0
```

The **Release Dry Run** workflow ran against release-prep before merge and again from final main before tag creation. The final-main run https://github.com/flyingrobots/bijou/actions/runs/28736681371 is the authoritative packed-file and npm publish dry-run evidence for the automated publish matrix.

After the release-prep PR merged and local main exactly matched origin/main, final tag preparation replayed the metadata and docs checks from the merged commit:

```
git fetch origin main --tags --prune
git switch main
git merge --ff-only origin/main
git status --porcelain=v1
git rev-parse HEAD origin/main
test "$(git rev-parse HEAD)" = "$(git rev-parse origin/main)"
npm run release:preflight
npm run docs:inventory
```

After the v7.2.0 tag was pushed and the publish workflow passed, registry verification confirmed every published package resolves to version 7.2.0 with the stable latest dist-tag:

```
npm view @flyingrobots/bijou version dist-tags --json
npm view @flyingrobots/bijou-node version dist-tags --json
npm view @flyingrobots/bijou-tui version dist-tags --json
npm view @flyingrobots/bijou-tui-app version dist-tags --json
npm view create-bijou-tui-app version dist-tags --json
npm view @flyingrobots/bijou-i18n version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools-node version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools-xlsx version dist-tags --json
npm view @flyingrobots/bijou-mcp version dist-tags --json
```

Release-Time Registry Snapshot

The table below is the immutable release evidence. It records the registry state observed during post-publish verification on 2026-07-05 after Release workflow run 28736930944 completed at 2026-07-05T10:03:04Z from release commit 123f4a6150963932118cbf763a8a6ec03487e227. The npm view commands above are operational replay aids and may report a later latest value after a future release; this table is the historical v7.2.0 snapshot.

Package	Observed version	Observed latest dist-tag
@flyingrobots/bijou	7.2.0	7.2.0
@flyingrobots/bijou-node	7.2.0	7.2.0
@flyingrobots/bijou-tui	7.2.0	7.2.0
@flyingrobots/bijou-tui-app	7.2.0	7.2.0
create-bijou-tui-app	7.2.0	7.2.0
@flyingrobots/bijou-i18n	7.2.0	7.2.0
@flyingrobots/bijou-i18n-tools	7.2.0	7.2.0
@flyingrobots/bijou-i18n-tools-node	7.2.0	7.2.0
@flyingrobots/bijou-i18n-tools-xlsx	7.2.0	7.2.0
@flyingrobots/bijou-mcp	7.2.0	7.2.0

Tag creation, publish automation, npm registry verification, and GitHub Release verification all ran from the merged release commit after the release-prep branch landed. The public release is https://github.com/flyingrobots/bijou/releases/tag/v7.2.0.

Deterministic Reproducibility

All historical release-prep claims above are replayable from the release-prep branch tip with:

```
npm run release:preflight
npm run docs:inventory
npm audit --audit-level=high
npm audit --omit=dev --audit-level=high
npx vitest run --config vitest.config.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part01.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part02.test.ts
↪  tests/cycles/WF-130/roadmap-goalpost-policy.part03.test.ts
npm run release:readiness
```

The public tag was created only after the final merged main checks passed:

```
npm run release:preflight
npm run docs:inventory
npm run release:readiness -- --milestone v7.2.0
npm audit --audit-level=high
npm audit --omit=dev --audit-level=high
```

The final tag was created after #354 closed, the release-prep PR merged, the local checkout was clean, main exactly matched origin/main, and CI / Release Dry Run evidence was green.

Residual Risk

- Code Dojo debt remains tracked by ratcheting baselines. This release improves enforcement and lowers debt, but does not claim zero standards debt.

261

Bijou 8.0.0 Release Evidence

docs/releases/8.0.0/README.md

Line status: open. This packet covers the 8.0.0 release line and is opened by its first tag, v8.0.0-rc.1. The stable 8.0.0 tag completes it rather than starting a new one — which is why the residual-risk section below reads as a to-do list rather than a closure record.

Bijou 8.0.0-rc.1 is a **release candidate**, not a stable release. It exists so downstream consumers can validate two breaking changes — the NO_COLOR detection change and the status-fallback change from DX-052 — against real applications before the stable 8.0.0 tag is cut.

Tracker Transition — 2026-10-19

Repository planning mirrors and METHOD doctrine are retired. The RC review matrix below is historical evidence of what was reviewed at publication time. Its references to reviewing direction/roadmap files are superseded by reviewing GitHub issue and milestone scope, plus linked Linear execution items once the migration is connected. This does not close any inherited product witness gap. Stable-release work remains tracked by #526 and #527 in the v8.0.0 milestone.

How to read this packet

This packet is deliberately explicit about the difference between evidence gathered first-hand during release prep and evidence inherited from the cycles that landed on main before it.

- **Verified here** means the command in the “replay” column was run on this release-prep branch and its result observed.
- **Inherited** means the goalpost landed through its own reviewed PR with its own CI, and this packet cites that lineage without re-running the goalpost’s specific witnesses.

Every Inherited row is a **release-law gap that must be closed before the stable 8.0.0 tag**. docs/release.md requires that a second operator can replay each witness from the tag commit, and an inherited citation does not meet

that bar. Naming the gap is the honest disposition for a prerelease; carrying it into a GA tag would not be.

Release Summary

- Version: 8.0.0-rc.1
- Previous public tag: v7.2.0
- Release type: **prerelease** (release candidate)
- npm dist-tag: **next** — not latest. parseReleaseTag() maps the rc channel to next (scripts/release-metadata.part02.ts), verified by npx tsx scripts/release-metadata.ts --tag v8.0.0-rc.1 emitting npm_dist_tag=next. An earlier draft of this packet said rc, which was an assumption about the name rather than a reading of the mapping; the property that matters — a prerelease must not land on latest — holds either way, but the tag to verify after publish is next
- Release-prep branch: release/v8.0.0-rc.1
- Release date: 2026-08-21
- Publish surface: npm workspace packages only, lock-step at 8.0.0-rc.1
- Target milestone: v8.0.0 (0 open / 7 closed at prep time)

Tracker And Goalpost Map

Twenty changelog entries accumulated on main since 7.2.0. They group into four substantive goalposts, two workflow closeouts, and a maintenance band.

DX-052 — NO_COLOR is not a capability signal

- Tracker: #518 (closed)
- Design: DX-052
- PR: #522
- Slices: 3 — detectOutputMode branch removal, status()/inkStatus() fallback move, extendTheme() group coverage
- Deterministic proof: packages/bijou/src/core/detect/tty.test.ts, tty.fuzz.test.ts, core/environment.part01.test.ts, core/theme/accessors.test.ts, core/theme/extend.test.ts, core/theme/resolve.part02.test.ts, factory.test.ts
- Replay: npx vitest run packages/bijou/src/core/detect packages/bijou/src/core/theme packages/bijou/src/factory.test.ts
- Witness: the fuzz property asserts the contract directly — adding NO_COLOR to any environment leaves that environment's detected mode unchanged (200 runs across ''/'1'/'true' × TTY × CI × TERM). factory.test.ts asserts the pair that matters together: mode === 'interactive' **and** theme.noColor === true **and** ink() undefined, over both '1' and ''.
- Evidence status: **Verified here.** Authored during this session; every listed command was run.

- Residual risk: intentional breaking change. A consumer relying on NO_COLOR to force single-frame output now gets an interactive monochrome TUI. The documented escape is TERM=dumb, which remains a genuine capability signal, or an explicit mode. This is the specific risk the rc exists to expose.

DX-050 — Profunctor Page terminal inspection target

- Design: DX-050
- Deterministic proof: per its changelog entry — lowerProfunctorPageArtifacts () validates the canonical profunctor-page/0 family and emits deterministic ui-scene-ir/1, Surface, target-map, receipt, cell-source-map, and text evidence across six inspection modes
- Evidence status: **Inherited.** Landed on main through its own reviewed PR and CI before this branch existed. Its specific witnesses were not re-run here.
- Residual risk: unsupported browser semantics remain explicit capability residuals; semantic-document sources, application islands, and visible unsupported blocks fail closed. Disposition inherited from the cycle, **not re-reviewed for this packet**.

DX-049 — VISOR artifact bundle proof

- Tracker: #458
- Design: DX-049
- Deterministic proof: createVisorArtifactBundleFromGraphql(), createVisorArtifactBundle(), visor-artifact-bundle/1 types, wrapping the checked-in DOGFOOD GraphQL fixture path
- Evidence status: **Inherited.**
- Residual risk: #459 packed-cell validation and external renderer/debugger work were explicitly out of scope for the cycle.

RE-036 — packed-cell receipt validation and Surface adaptation

- Design: RE-036
- Deterministic proof: adaptPackedBijouCellsToSurface() copies validated bytes and side-table entries without re-encoding cells
- Evidence status: **Inherited.**

WF-166 — V8 dependency-security closeout

- Design: WF-166
- Deterministic proof: a regression binds the patched dependency floors and the manifest override; a clean npm ci reproduces a zero-vulnerability audit
- Replay: npm audit --omit=dev --audit-level=high
- Evidence status: **Verified here** for the audit result; the floor-binding regression is inherited.

WF-167 — roadmap milestone authority

- Design: WF-167
- Evidence status: **Inherited.**

Maintenance band — Code Dojo ratchets and DOGFOOD documentation

Eleven entries: WF-165 tranches A–E, the DX-050 Code Dojo prerequisites A and B, the focused ratchet, touched DOGFOOD localization debt, DOGFOOD split-source debt identity, the documentation contract, future-release milestone triage, and the DX-048 V8 Runtime Graph shaping record.

- Deterministic proof: the ratchet baselines under scripts/code-dojo/baselines/ are the artifact; the gates are the proof
- Replay: npm run code-dojo:verify, npm run code-dojo:strict, npm run code-dojo:debt
- Evidence status: **Verified here.** The pre-commit and pre-push dojo gates ran on every commit in this lineage and reported 0 baselined files over 150 lines or 12000 bytes, 0 files over 500 lines, and 0 baselined ESLint findings. Two files crossed the 150-line threshold while gaining tests during DX-052 and were brought back under by tightening prose and collapsing two cases into one it.each, rather than by adding baseline entries.

Prerelease gaps found and closed during prep

Cutting the first prerelease this tooling has seen exposed four defects, all fixed on the release-prep branch and recorded in #523:

1. **DOGFOOD crashed on startup.** It resolved release docs as docs/releases/${BIJOU_VERSION}/, so 8.0.0-rc.1 produced an unhandled ENOENT at module load. smoke:dogfood reported only exited with code 1 — no path, no stack — so reproducing it meant running the capture entrypoint by hand with the scenario environment reconstructed from the smoke library.
2. **The two release gates disagreed** about where release docs live: the readiness gate keys the packet path off the milestone, DOGFOOD keyed it off the exact version. Resolved in favour of the release *line*, which is why this packet lives at docs/releases/8.0.0/.
3. **The i18n scanner silently stopped counting two documents.** Templating the path with a token the static resolver did not know made the release What's New and migration guide unresolvable, so they left the inventory rather than failing, and measured Markdown debt fell 78 → 72. An unresolvable token reads exactly like a six-point improvement. Fixed by teaching the resolver the token; **no baseline lowered and no test edited** to accommodate it.
4. **A prerelease version string overflows the DOGFOOD release nav.** At 120x40, What's New in v8.0.0-rc.1 rendered as What's New in v8.0.0-rc.. Release guide titles and ids now key off the line, which is also what they describe.
5. **release:preflight was a silent no-op, and this packet initially claimed it passed.** scripts/release-metadata.ts is the CLI entry point but was a pure

re-export shim; it imported release-metadata.part04.js for the side effect of *that* module's main guard, which compares import.meta.url against process .argv[1] and therefore never fires when the process starts on the shim. The command exited 0 having validated nothing, written no stdout, and written no GITHUB_OUTPUT.
REL-META-VERSION-LOCKSTEP is the gate the release process assigns to releas e:preflight, so version-mismatch detection was unenforced: --version 9.9.9 against an 8.0.0-rc.1 workspace exited 0. The Release Dry Run surfaced it only indirectly — its notes job read the missing output as an empty tag and failed with gh: Missing tag_name parameter (HTTP 400).
Every unit test of runReleaseMetadata() passed throughout, because the function was always correct; nothing called it. tests/cycles/DX-052/release -metadata-entrypoint.test.ts now invokes the real entry point as a child process, which is the only way to observe this class of defect.
A gate that reports success without running is worse than a missing gate, and the first version of this row in the matrix above recorded exactly that false pass. It is corrected rather than quietly overwritten.

One ratchet was tightened rather than loosened: app-guides-release's release overview summary interpolated the version mid-sentence, counting as two translatable literals split around an inserted value — word order a translator cannot rearrange. Moving the value to the end makes it one string, taking total raw-string debt 2317 → 2316 and the surface 7 → 6. Both baselines shrank to match.

Automated Evidence Matrix

Gate	Command or source	Expected result	Status
Workspace lock-step	npm run version 8.0.0-rc.1	All ten workspace packages and internal dependency pins report 8.0.0-rc.1.	Verified here: all ten confirmed.
Release metadata preflight	npm run release:preflight	Lock-step workspace metadata is valid.	**Initially recorded as passing on exit 0, which was wrong** — the command was a no-op (see below). Fixed on this branch; now prints the eleven-package lock-step summary and exits 1 on a mismatch.
Docs inventory	npm run docs:inventory	Documentation manifest remains valid after release docs are added.	Verified here: exit 0.
Runtime dependency audit	npm audit --omit=dev --audit-level=high	Zero high or critical runtime vulnerabilities.	Verified here: found 0 vulnerabilities.
Release gauntlet	npm run release:readiness	The fourteen-gate local gauntlet passes.	Verified here: release-readiness: ok.

Gate	Command or source	Expected result	Status
Milestone-aware readiness	npm run release:readiness -- --milestone v8.0.0	Target milestone has zero open tracker items and no work-in-progress labels.	Verified here: all six report gates PASS. Required first: closing #518 and clearing stale work-in-progress labels from the already-closed #482 and #458.
Full test suite	npm test	Green.	Verified here on this release-prep branch via the gauntlet: 933 test files / 4,130 tests, summed across the twenty vitest chunks.
Lint	npm run lint	Clean across all workspace packages.	Verified here.
Pre-push verification	.githooks/pre-push	Full dojo and repo verification, including scripted interactive example smoke.	Verified here: passed on every push in this lineage.
PR CI	GitHub Actions on #522	All checks green.	Verified here: 10/10 green including both DOGFOOD smokes, TypeScript doctrine, and focused unit tests on ubuntu and windows. CodeRabbit passed.
Release Dry Run	.github/workflows/release-dry-run.yml	Packed files and publish dry-runs stay green.	Required before tagging.
Tag Guard	.github/workflows/tag-guard.yml	The pushed tag resolves to the intended release commit.	Runs after tag push.
Publish workflow	.github/workflows/publish.yml	Automated npm publishing succeeds for every workspace package.	Runs after tag push.
npm registry verification	npm view <package> dist-tags --json --prefer-online	Every package carries 8.0.0-rc.1 on next, and latest is untouched.	Verified after publish for all ten packages — see the registry snapshot below.

Human Review Matrix

Surface	Review disposition
docs/CHANGELOG.md	Twenty accumulated entries moved under a dated 8.0.0-rc.1 boundary with a note that the stable 8.0.0 header will consolidate them. [Unreleased] intentionally left in place and empty for post-rc work.
README.md	What's New replaced with v8.0.0-rc.1, leading with the two breaking changes and marking the line as a preview rather than supported.
docs/BEARING.md	Updated during DX-052 with two new tensions: environment signals are not capability signals, and cross-group token aliasing. **Does not yet describe an 8.0.0 release posture** — needs a maintainer pass before the stable tag.

Surface	Review disposition
docs/ROADMAP.md	**Not reviewed for this release.** References v8.0.0 in ten places, which satisfies the readiness gate, but its content was not re-verified against milestone state. Required before the stable tag.
docs/design/DX-052-*.md	Status moved to landed.
ARCHITECTURE.md	**Not reviewed.** DX-052 changes no port, adapter, or package boundary, so no update is required *for that goalpost*. The three inherited goalposts touch rendering contracts and were not assessed here.
docs/VISION.md, docs/METHOD.md	**Not reviewed.** No known change required.
docs/DOGFOOD.md	**Not reviewed.** DOGFOOD smokes pass, but release-title and proof-surface posture for an 8.0.0 line was not assessed.
Package READMEs, API references, MCP docs	**Not reviewed.** extendTheme()'s public signature gained two optional fields; whether any package README documents that signature was not checked.

Nine surfaces, three reviewed. That ratio is the honest state of this packet and is the reason this is an rc.

Release-Time Registry Snapshot

v8.0.0-rc.1 was tagged from 437d81bbd035463ba4bbd6901e39467f1e47bf1d, which was exactly origin/main with a clean worktree. Recorded because the Release Law says never to claim a release succeeded unless the tag, workflows, and registry state were directly verified.

Workflow	Run	Result
Release Dry Run (pre-tag, final tree)	32529250110	success — all jobs, including npm ci against the regenerated lockfile
Tag Guard	32531521083	success
Release (publish)	32531521042	success — all ten publish jobs plus both DOGFOOD smokes
Tag CI	32531521038	success
GitHub Release	v8.0.0-rc.1	published, prerelease: true

Registry verified with npm view <package> dist-tags --json --prefer-online. All ten packages:

```
@flyingrobots/bijou                 latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-node            latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-tui             latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-tui-app         latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-i18n            latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-i18n-tools      latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-i18n-tools-node latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-i18n-tools-xlsx latest=7.2.0  next=8.0.0-rc.1
@flyingrobots/bijou-mcp             latest=7.2.0  next=8.0.0-rc.1
create-bijou-tui-app                latest=7.2.0  next=8.0.0-rc.1
```

latest remains 7.2.0 on every package, which is the property that mattered: a prerelease on latest would have become the default install for a bare npm install.

One verification note worth keeping, because it is a trap for the next operator. An immediate post-publish query reported next: null for bijou-node and a 404 on @flyingrobots/bijou-node@8.0.0-rc.1, while that package's publish job had reported success. The job log resolves it: + @flyingrobots/bijou-node@8.0.0-rc.1 followed by npm's own "Your package is being processed and may take a few minutes to become available." It was registry propagation, and a re-query confirmed it. Given three genuine "reported success without doing the work" defects in this cycle (#525, #526, and the dry-run notes job), treating a 404 as a fourth was the natural reading and the wrong one — **read the publish log before concluding a publish failed.**

Downstream Validation

muniment is the first consumer on 8.0.0-rc.1 and the reason this candidate exists. On upgrade it deleted its local mode.ts — a re-derived output-mode detector that existed solely because NO_COLOR used to force 'pipe' — and repointed its assertions at bijou's own detectOutputMode, keeping them as a downstream regression test of the contract it depends on.

Its whole check suite passes on the candidate: palette audit, construct check across 32 frames, greppable view render, and typecheck.

It also found one real behavioural difference, resolved in bijou's favour. detectOutputMode consults stdoutIsTTY only, so a TTY stdout with a piped stdin yields 'interactive'; muniment's detector had called that 'pipe', on the grounds that a full-screen app which cannot receive keys is a hang rather than a UI. bijou's reading is better: mode describes output capability, and stdin is enforced separately and loudly by createNodeContext, which throws BijouBootstrapError with *"raw mode unavailable — run in an interactive terminal rather than a non-TTY pipeline."* A loud bootstrap failure beats a silently lowered mode, so the stricter downstream rule was dropped rather than reimplemented.

Deterministic Reproducibility

Replayable from this commit with no external inputs:

```
npm ci
npm run lint
npm test
npx vitest run packages/bijou/src/core/detect packages/bijou/src/core/theme
↪  packages/bijou/src/factory.test.ts
npm run code-dojo:verify
npm audit --omit=dev --audit-level=high
```

Normalization notes: the fuzz property in tty.fuzz.test.ts is seeded by fast-check defaults and runs 200 cases; it asserts an invariant (mode is unchanged by NO_COLOR) rather than a fixed value, so it does not require normalization for host colour mode, terminal size, or clock. factory.test.ts uses mockRuntime/mockIO/plainStyle and therefore does not read the host environment at all.

Package And Registry Verification Plan

1. Merge the release-prep PR into main.
2. Confirm local main is exactly origin/main.
3. Run the Release Dry Run workflow against the release commit.
4. Tag v8.0.0-rc.1 on the exact merged origin/main commit.
5. Verify Tag Guard, tag CI, and the publish workflow.
6. Verify every package reports 8.0.0-rc.1 on npm.
7. **Confirm the dist-tag is next, not latest.** A prerelease published to latest would silently upgrade every consumer running a bare npm install @flyingrobots/bijou. next is what the rc channel maps to.

Open priority:high Disposition

REL-GH-PRIORITY-HIGH-ZERO permits a release with open priority:high issues only where this packet names the accepted risk and its owner. Four are open at tag time, and three of them were opened by this cycle — the release process found its own gates wanting.

Issue	Release-blocking for v8.0.0-rc.1?	Owner	Disposition
#525 release:preflight was a silent no-op	No — **fixed on this release commit**	maintainer	The defect is repaired and covered by tests/cycles/DX-052/release-metadata-entrypoint.test.ts. The issue stays open for the residual suggestion that Verify release candidate should fail when it emits no outputs, rather than letting a downstream job discover the emptiness.
#526 lock-step versioning ignores package-lock.json	No — **the symptom is fixed on this release commit**	maintainer	The lockfile is regenerated and consistent at 8.0.0-rc.1. The three gates that failed to detect it are unchanged, so the *class* of defect remains and could recur on the next bump. Accepted for a prerelease; should be closed before the stable tag, since a stale lock is exactly the kind of thing a GA must not ship.

Issue	Release-blocking for v 8.0.0-rc.1?	Owner	Disposition
#527 replay inherited witnesses	**Blocking for 8.0.0, accepted for the rc**	maintainer	See residual risk 2. The rc's purpose is downstream validation of #518's breaking changes, which does not depend on re-proving three unrelated goalposts. Carries the six unreviewed human-review surfaces too.
#473 SVG path parsing is not total	No	maintainer	Milestoned v8.2.0, unrelated to this release's surface. Named here rather than left implicit.

None of these is hidden, and none is a gate reported as passing that did not run — which is the failure #525 was.

Residual Risk

1. **Nine of eleven release-law human review gates are unreviewed** (see the matrix). Acceptable for a prerelease whose stated purpose is downstream validation; **blocking for the stable 8.0.0 tag**.
2. **Three product goalposts carry inherited rather than replayed evidence.** DX-050, DX-049, and RE-036 landed with their own review and CI, but their witnesses were not replayed from this commit, which is what docs/release.md requires: *"evidence is complete only when another operator can replay it from the tag commit."*
   Automated review raised this as a release-law objection — the Law says no public tag may be created unless every gate is satisfied. The Law's own provision for this case is that *"any accepted residual risk is named with rationale, owner, and a follow-up issue"*, and that hidden accepted failures are not allowed. So: accepted for the prerelease, owner maintainer, follow-up #527, which also carries the six unreviewed human-review surfaces and is written so that closing it means flipping five rows in this packet from Inherited to Verified here.
   The stable 8.0.0 tag must not be created while any row reads Inherited.
3. **The breaking changes are unvalidated against real consumers.** That is the point of the rc. muniment is the first consumer and will pin this version; git-cas carries a local detectCliTuiMode override that is now redundant and can be removed once it moves off ^5.0.0.
4. **The changelog boundary will need consolidating at GA.** Twenty entries now sit under an rc header. The stable 8.0.0 header should absorb them rather than leave consumers reading release history split across a prerelease.
5. **The lockfile was stale until review caught it.** npm run version rewrote all eleven manifests and left package-lock.json byte-identical to main, still recording 7.2.0 for every workspace package and internal pin — so a clean npm ci would have resolved from metadata a full major behind the manifests. Regenerated here with npm install --package-lock-only. Three separate gates

failed to notice, filed as #526: the version script does not touch the lock, the pre-commit “lockfile consistency” check runs npm ls --all which cannot detect the condition, and release:preflight validates manifests only. Same family as #525 — gates that report success without checking what their names promise.

6. **DOGFOOD’s “Current Release Story” guide still describes the v7.2 path** in its summary, catalogs, and body, while the release page now presents v8. Cosmetic for a prerelease, tracked in #527 for the stable tag.
7. **Open priority:high issue #473** (BAD CODE: make SVG path parsing total and non-stalling) is milestoned v8.2.0 and is not release-blocking for 8.0.0. Named here rather than left implicit, per REL-GH-PRIORITY-HIGH-ZERO.
8. **Known palette defects ship unchanged.** #519 records that both first-party presets alias one colour across the same ten token paths and that error/success collapse to ΔE 0.059 (dark) and 0.064 (light) under protanopia, with bijou-light’s severity ladder inverted in lightness. DX-052 makes this *escapable* — extendTheme() can now reach every group — but does not fix it. Consumers relying on the shipped presets for severity distinctions should read #519.

262

Release Docs

docs/releases/README.md

Long-form release documentation lives under docs/releases/.

Use this directory for material that is more explanatory than CHANGELOG.md:

- release overviews and “what changed” walkthroughs
- migration guides for existing users
- version-specific notes that should stay readable after the release is no longer “current”

Working Rule

While a release version is still undecided, draft the long-form docs in docs/releases/next/.

When the version is chosen, move that directory to the real release path:

```
docs/releases/next/
  -> docs/releases/X.Y.Z/
```

Current Shaped Release Docs

- What’s New (v8.0.0) — line open, first tag v8.0.0-rc.1
- Migration Guide (v8.0.0)
- Release Evidence (v8.0.0)
- What’s New (v7.2.0)
- Migration Guide (v7.2.0)
- Release Evidence (v7.2.0)
- What’s New (v7.1.0)
- Migration Guide (v7.1.0)
- Release Evidence (v7.1.0)

Previous Release Docs

- What’s New (v7.0.0)

- Migration Guide (v7.0.0)
- What’s New (v5.0.0)
- Migration Guide (v5.0.0)
- What’s New (v4.4.1)
- Migration Guide (v4.4.1)
- What’s New (v4.4.0)
- Migration Guide (v4.4.0)
- What’s New (v4.3.0) | Migration Guide
- What’s New (v4.2.0) | Migration Guide
- What’s New (v4.1.0) | Migration Guide

263

Migration Guide: Bijou v4.0.0 To v4.1.0

docs/releases/4.1.0/migration-guide.md

This guide is for users already on v4.0.0 who want to move to v4.1.0.

If you are coming from an older major line, read MIGRATING_TO_V4.md first.

Migration At A Glance

Not every user has real migration work to do.

If you use…	Expected migration effort	Why
@flyingrobots/bijou for CLI output, prompts, and string-first flows	Low	Most v4.1.0 work is additive or docs-facing for this surface.
@flyingrobots/bijou-tui for fullscreen runtime apps	Medium	The runtime and command model are more explicit, and you should align with the newer seams.
@flyingrobots/bijou-tui-app / createFramedApp() heavily	Medium to High	Framed-shell typing, layer routing, and shell inspection surfaces are more explicit, so old assumptions may need cleanup.
custom runtime internals or shell-adjacent helpers	High	The repo is moving toward runtime-engine seams and away from branch-order shell folklore.
localization workflows	Optional but meaningful	Entirely new packages and workflows exist now if you want them.

Upgrade Checklist

1. Upgrade all Bijou packages together to the same version.
2. Rebuild and re-typecheck before making behavioral changes.
3. Audit framed-shell typing if you use createFramedApp().
4. Audit custom command code if you rely on long-lived effects or subscriptions.
5. Replace shell-folklore assumptions with exported layer/runtime helpers where possible.
6. Adopt the new localization packages only if you need that surface.

7. Re-run your fullscreen and shell interaction tests, especially for help/settings/notification/quit flows.

1. Upgrade In Lock-Step

Bijou still versions its published packages in lock-step.

Upgrade the packages you use together:

```
npm install @flyingrobots/bijou@4.1.0 @flyingrobots/bijou-node@4.1.0
```

Add the fullscreen packages if your app uses them:

```
npm install @flyingrobots/bijou-tui@4.1.0 @flyingrobots/bijou-tui-app@4.1.0
```

If you adopt the new localization surface, also add the relevant i18n packages:

```
npm install @flyingrobots/bijou-i18n@4.1.0 @flyingrobots/bijou-i18n-tools@4.1.0
```

If you also need filesystem or XLSX exchange adapters, install those in the same lock-step release:

```
npm install @flyingrobots/bijou-i18n-tools-node@4.1.0
↪  @flyingrobots/bijou-i18n-tools-xlsx@4.1.0
```

These i18n packages are part of the 4.1.0 automated publish surface, so they land together with the rest of the workspace in this release.

2. Framed Apps: Move Onto The Typed Shell Surface

If your app uses createFramedApp(), this is the migration area most likely to matter.

The shell now exports explicit framed message and update helpers:

- FramePageMsg<Msg>
- FramePageUpdateResult<PageModel, Msg>
- FramedAppMsg<Msg>
- FramedAppUpdateResult<PageModel, Msg>
- FramedApp<PageModel, Msg>

What to change

- stop treating framed page updates as effectively any
- update explicit type annotations that assumed page messages were only your raw app Msg
- update tests that asserted the framed shell collapsed everything back to plain Msg

Typical before/after mindset

Before:

- the page/shell seam was often typed loosely
- external casts papered over framed-shell complexity

After:

- the page/shell seam should stay typed explicitly
- page updates should accept FramePageMsg<Msg> where appropriate
- framed app updates should be allowed to return Cmd<FramedAppMsg<Msg>>[]

If your code was already mostly inference-driven and you were not fighting the shell types directly, you may have little to change. If you have custom wrapper helpers around createFramedApp(), inspect them carefully.

3. Commands: Stop Pretending They Only End In Messages

Cmd<M> now models the runtime more honestly.

A command may now:

- complete synchronously or asynchronously
- emit intermediate messages through emit(...)
- resolve to a final message
- resolve to QUIT
- return a cleanup handle or cleanup function
- resolve to void

What to look for

- custom command wrappers that assume every command returns a final message
- tests that stub commands as only Promise<Msg | undefined | QUIT>
- child-app command mapping that accidentally discards cleanup results

What to do

- widen your local command typings if you mirrored the old narrower contract
- prefer the built-in sub-app lifecycle helpers if you were manually coercing command results
- treat cleanup-producing commands as first-class runtime work, not as hacks that need any

If you only use built-in helpers like tick(), animate(), or normal message-returning commands, this may be invisible to you. The migration is most relevant for custom runtime work and command composition helpers.

4. Shell Layers: Use The Exported Truth

If you wrote code or tests that reason about shell layering indirectly, update them to prefer the exported shell inspection surface:

- describeFrameLayerStack(...)
- activeFrameLayer(...)
- underlyingFrameLayer(...)

Why this matters

Since v4.0.0, the shell has become more explicit about:

- topmost-layer ownership
- dismissal order
- footer/help truth
- settings/search/help/notification/quit layering

That means older tests or utilities that inferred shell state from raw render order, footer strings, or branch order inside app-frame.ts are more likely to drift.

What to re-test

- Esc behavior
- search/help/settings transitions
- notification center and history behavior
- quit confirmation flows
- mouse shielding while exclusive overlays are open

5. Runtime Internals: Prefer Engine Seams Over Shell Folklore

If you extend or inspect runtime internals directly, start moving toward the explicit runtime-engine exports introduced since v4.0.0.

The relevant surfaces are the RE slices:

- state machine and view stack ownership
- retained layouts and invalidation
- layout-driven input routing
- command/effect buffering
- component layout and interaction contracts

This does **not** mean the migration is complete. The repo still tracks RE-007 because some framed-shell behavior still needs to move more fully onto those seams.

The practical rule is:

- prefer exported runtime seams where they exist
- avoid adding new shell-only branch-order logic if you can use the runtime-engine model instead

6. Localization Is Opt-In, Not Forced

If you do not need localization, you do not need to migrate code just because the new localization packages exist.

If you do want localization, v4.1.0 gives you a real substrate:

- runtime locale/direction contracts
- tools for catalogs and exchange workflows
- Node filesystem adapters
- XLSX-oriented tooling
- localized shell chrome and DOGFOOD surfaces

That is an adoption opportunity, not a forced migration.

7. Docs And DOGFOOD Usually Require No App Migration

Most of the DOGFOOD, design-system, METHOD, and release-doc work does not require code changes in downstream apps.

It does change what is now available to you:

- a much more complete field guide
- stronger component-family guidance
- clearer repo doctrine and release documentation

Treat these as improved references, not breaking changes.

8. Validation Checklist After Upgrading

For a normal app upgrade, run at least:

```
npm run build
npm test
```

If your app uses TypeScript and the fullscreen runtime, also run your typecheck and any shell smoke tests you have.

Pay special attention to:

- framed shell compile errors
- command typing errors
- tests around help/settings/search/quit
- notification history and overlay behavior
- mouse interaction over modal/help/palette surfaces

9. Migration Recommendations By App Shape

Core CLI / prompt app

Usually:

- upgrade dependencies
- rebuild
- read the new docs when you want deeper design-system guidance

Fullscreen TUI app without heavy shell customization

Usually:

- upgrade dependencies
- fix any framed typing fallout
- re-test shell overlays and interaction flows

Fullscreen app with custom shell helpers or runtime extensions

Usually:

- audit command typings
- audit shell-layer assumptions
- move toward exported framed/runtime helpers
- re-run high-signal interaction tests

Localization-aware app

Usually:

- decide whether to adopt the new i18n packages now or later
- verify package availability in the published release
- add localized shell copy incrementally instead of rewriting the whole app at once

Final Advice

Compared with the v4.0.0 migration, v4.1.0 is less about one hard boundary change and more about aligning your app with clearer repo truth.

The important upgrades are:

- accept the typed framed-shell surface
- accept the more honest Cmd<M> contract
- trust exported shell/runtime inspection helpers over folklore
- adopt localization only if and when you need it

If your app is mostly core Bijou plus light TUI usage, the migration should be manageable. If you have deep custom shell/runtime code, this is a good release to pay down those assumptions.

264

Migrating to Bijou 4.2.0

docs/releases/4.2.0/migration-guide.md

From 4.1.0

New package: @flyingrobots/bijou-mcp

@flyingrobots/bijou-mcp is a new addition to the workspace. It has no effect on existing code — it is an MCP stdio server that wraps existing Bijou components for use in AI chat contexts.

If you use it, install it alongside the core package:

```
npm install @flyingrobots/bijou @flyingrobots/bijou-mcp
```

Runtime engine changes (RE-007)

If your app uses createFramedApp from @flyingrobots/bijou-tui, the following internal functions were removed:

- handleFrameMouse
- paneHitAtPosition
- settingsRowAtPosition
- isInsideSettingsDrawer
- withObservedKey
- applyQuitRequest
- observedRouteForLayer
- applyHelpScrollAction

These were internal to the framed shell and not part of the public API. If you were importing them directly, replace with the runtime engine's routing infrastructure (routeRuntimeInput, bufferRuntimeRouteResult, applyRuntimeCommandBuffer).

Inspector fix

inspector() now renders supportingTextLabel in interactive mode. If you were working around its absence (e.g. embedding the label in supportingText manually), you can remove the workaround.

No breaking changes

This is an additive minor release. All existing public APIs remain unchanged.

265

Migrating to Bijou 4.3.0

docs/releases/4.3.0/migration-guide.md

From 4.2.0

No breaking changes

This is a performance-focused minor release. All existing public APIs remain unchanged. The packed surface is an internal implementation detail — Surface, Cell, createSurface, set(), get(), blit(), fill(), clear(), and clone() all behave identically.

New setRGB() API

If you have performance-sensitive rendering code that writes cells in a tight loop, consider using setRGB() instead of set():

```
// Instead of this (parses hex on every call):
for (let x = 0; x < width; x++) {
  surface.set(x, y, { char: '█', fg: '#ff8800', bg: '#000000' });
}

// Use this (zero-alloc, ~10-50x faster per cell):
for (let x = 0; x < width; x++) {
  surface.setRGB(x, y, 0x2588, 255, 136, 0, 0, 0, 0);
}
```

Pass -1 for fg or bg RGB values to use the terminal default color.

Use flag constants for modifiers:

```
import { FLAG_BOLD, FLAG_DIM } from '@flyingrobots/bijou';

surface.setRGB(x, y, 'A', 255, 255, 255, 0, 0, 0, FLAG_BOLD);
```

PackedSurface type

createSurface() now returns PackedSurface (which extends Surface). If your code types surfaces explicitly, you can use either type — PackedSurface adds .buffer, .sideTable, and .markAllDirty() for advanced byte-level access.

Braille rendering

If you were working around braille corruption in flexSurface by rendering directly to stdout, you can now remove that workaround. Braille characters (U+2800–U+28FF) are preserved correctly through the entire surface pipeline.

Theme token fgRGB / bgRGB

TokenValue now includes optional fgRGB and bgRGB tuple fields. If you create custom tokens, you don't need to set these — they're populated automatically during theme resolution. But if you want to pre-populate them for performance:

```
const token: TokenValue = {
  hex: '#ff8800',
  fgRGB: [255, 136, 0],
};
```

266

Migrating to Bijou 4.4.0

docs/releases/4.4.0/migration-guide.md

No Breaking Changes

Bijou 4.4.0 is a backwards-compatible minor release. All existing 4.3.0 code continues to work without changes.

New Exports

The following are new exports from @flyingrobots/bijou:

- sparkline(values, options?) — returns a string
- brailleChartSurface(values, options) — returns a Surface
- statsPanelSurface(entries, options) — returns a Surface
- perfOverlaySurface(stats, options?) — returns a Surface

Along with their option/entry types:

- SparklineOptions
- BrailleChartOptions
- StatsPanelEntry
- StatsPanelOptions
- PerfOverlayStats
- PerfOverlayOptions

Bench Scenario API Change (Internal)

If you have custom benchmark scenarios, the setup() function signature now accepts optional columns and rows positional arguments after ctx: setup(ctx?, columns?, rows?). Existing scenarios that ignore these arguments continue to work unchanged.

267

Migrating to Bijou 4.4.1

docs/releases/4.4.1/migration-guide.md

No Breaking Changes

Bijou 4.4.1 is a backwards-compatible patch release. All existing 4.4.0 code continues to work unchanged.

If You Added a Terminal Background Workaround in 4.4.0

Some apps worked around the 4.4.0 regression by pre-setting the terminal background before calling run().

```
process.stdout.write('\x1b[48;2;45;25;34m');
await run(app, { ctx });
```

That prefill should no longer be necessary when you use createFramedApp() with a theme that sets surface.primary.bg.

Leaving the workaround in place is usually harmless when it matches your theme, but Bijou now renders the framed body background explicitly.

Optional: Move App-Local Theme Toggles Into shellThemes

If your app currently manages shell theme switching in page-local state, you can now hand that concern to the stock frame:

```
let currentCtx = ctx;
const getCurrentCtx = () => currentCtx;
const initialModel = {};

const app = createFramedApp({
  ctx: currentCtx,
  shellThemes: [
    { id: 'default', label: 'Default', theme: defaultTheme },
    { id: 'verdant-plum', label: 'Verdant Plum', theme: plumTheme },
  ],
  onShellThemeChange: ({ ctx: nextCtx }) => {
    currentCtx = nextCtx;
  },
  pages: [{
```

```
    id: 'home',
    title: 'Home',
    init: () => [initialModel, []],
    update: (msg, model) => [model, []],
    layout: (model) => ({
      kind: 'pane',
      paneId: 'main',
      render: (width, height) => renderHome(getCurrentCtx(), model, width, height),
    }),
  }],
});
```

The frame will:

- add a stock shell-theme row to the settings drawer when two or more shell themes are available
- keep the resolved shell theme in one shared frame-owned place
- expose the current selection via FrameModel.activeShellThemeId

If your page renderers already resolve theme data from an explicit BijouContext, pass that context into createFramedApp({ ctx }), mirror onShellThemeChange back into your app-owned state, and make your page renderers read the updated context on each render. Apps that rely only on the ambient default context can omit both options; the frame will update that default context when the stock shell theme changes.

You do not need to adopt this API, but it is now the simplest way to keep a landing screen, docs shell, or other frame-owned chrome on one theme source of truth.

Quit Confirm Input Is More Forgiving

The stock quit dialog now accepts uppercase Y / N alongside lowercase y / n. No migration work is required.

268

Migrating to Bijou 5.0.0

docs/releases/5.0.0/migration-guide.md

Bijou 5.0.0 does not require a wholesale rewrite for most 4.4.x apps. The main migration story is that the preferred hosted and theme-selection paths are now simpler and more explicit than the lower-level wiring that earlier releases required.

Framed Apps: Prefer The Self-Running Entry Points

If you currently do this:

```
const app = createFramedApp(options);
await run(app, { ctx });
```

prefer one of these in 5.0.0:

```
const app = createFramedApp(options);
await app.run({ ctx });
```

or:

```
await runFramedApp(options, { ctx });
```

Why move:

- the framed shell now owns its hosted runner directly
- the run-time ctx is applied to shell-owned rendering
- frame timing and budget telemetry are wired back into FrameModel
- startApp(app) now delegates to this path automatically when the app is a self-running framed shell

Existing run(createFramedApp(...)) code may still work, but it is no longer the preferred entrypoint.

Theme Selection: Move It Up To The Host Layer

If you previously built a custom Node context just to apply a theme, prefer the host-owned APIs:

```
await startApp(app, { theme: MY_THEME });
```

For paired dark/light themes, use a theme set:

```
await startApp(app, {
  themes: [
    { id: 'light', theme: LIGHT_THEME },
    { id: 'dark', theme: DARK_THEME },
  ],
  themeMode: 'auto',
  themeOverride: savedThemeId,
});
```

The same selection model is also available through:

- createNodeContext({ theme })
- createNodeContext({ themes, themeMode, themeOverride })
- initDefaultContext({ theme })
- initDefaultContext({ themes, themeMode, themeOverride })

If you pass both ctx and theme, the explicit ctx still wins.

Interactive Smoke: Use The Dedicated Verification Lane

If you maintain custom CI or local proof commands around the interactive examples, prefer:

```
npm run verify:interactive-examples
```

The old scripted-example path now lives in that dedicated integration lane so the main unit suite can stay deterministic. The real examples still run; they just no longer masquerade as cheap unit tests.

No Manual Migration Needed For These Fixes

These changes land automatically once you upgrade:

- markdown() now renders bounded pipe tables through Bijou's real table surface
- fatal crash mode no longer hangs forever when stdin is not a TTY
- scopedNodeIO() rejects symlink escapes across the declared root boundary
- focused browsable rows can marquee overflowing labels in shared list surfaces such as DOGFOOD navigation

Compatibility Stance

Bijou 5.0.0 is a major line because the framework is drawing harder boundaries around hosted runtime ownership, theme selection, and release proof, not because most apps are expected to break. If your app already stays on the public APIs, the practical migration should usually be small and mostly about adopting the cleaner host entrypoints.

269

Migrating to Bijou 7.0.0

docs/releases/7.0.0/migration-guide.md

Bijou 7.0.0 carries the V6 layout floor and V7 Product Truth lane into one major release. Most application code that stayed on public APIs should keep working, but maintainers should check the areas below before upgrading.

Keep The 5.0 Hosted App Entrypoints

The preferred hosted app path from 5.0.0 remains the right path:

```
const app = createFramedApp(options);
await app.run({ ctx });
```

or:

```
await runFramedApp(options, { ctx });
```

If your app still calls lower-level runners directly, this is a good release to move back to the frame-owned hosted entrypoints so shell timing, theme, and runtime behavior stay in one place.

Check Table Width Expectations

table() and tableSurface() now use bounded-width behavior by default for human-mode output. If your app expected tables to grow to intrinsic content width regardless of terminal size, opt into intrinsic layout explicitly.

For most terminal UIs, the new default is what you want: columns fit the target width, wrapped cells preserve word boundaries when possible, and lower-mode pipe formats remain deterministic.

Prefer Standard Blocks For Product Semantics

Bijou 7.0.0 exports a much broader standard Block catalog. If your app has custom semantic surfaces for status, feedback, navigation, disclosure, dense comparison, hierarchy, text entry, or documentation-like content, evaluate the new standard Blocks before inventing another local schema.

Blocks should own product semantics, data contracts, command intents, and lowering facts. Leaf components should remain the rendering vocabulary.

Route Localization Through The Runtime Port

@flyingrobots/bijou-i18n now provides structured localization results instead of only returning strings. New localization code should preserve the returned metadata: key, locale, direction, entry kind, fallback or missing status, issues, and facts.

If your app persists locale preference, keep that persistence in a host adapter or other boundary-owned port. DOGFOOD uses this shape so runtime locale changes are visible without making app logic read process globals directly.

Notification Overlays Need No Workaround

Live notification rows now wrap at word boundaries when possible. Remove any app-local workaround that pre-wrapped notification titles, bodies, or action labels only to avoid split words in Bijou overlays.

Long unbroken text still hard-wraps inside the assigned notification width, so callers do not need to pre-truncate machine identifiers or hashes for layout safety.

Release And DOGFOOD Proof Are Stricter

DOGFOOD now reads version-specific release docs from:

```
docs/releases/<package-version>/whats-new.md
docs/releases/<package-version>/migration-guide.md
```

When preparing a release branch, bumping package versions without adding those docs will fail the docs and DOGFOOD test lanes. This is intentional: the package version and current release documentation must move together.

Compatibility Stance

Bijou 7.0.0 is a major release because public contracts are broader and more explicit, not because every app must rewrite. The main migration work is to remove local workarounds that are now framework-owned and to adopt the standard contracts where they replace app-local semantic surfaces.

270

Migrating to Bijou 7.1.0

docs/releases/7.1.0/migration-guide.md

Bijou 7.1.0 is a minor release from 7.0.0. It is intended to be additive for applications using documented public APIs.

Recommended Upgrade

Update the runtime packages your app uses together:

```
npm install \
  @flyingrobots/bijou@7.1.0 \
  @flyingrobots/bijou-node@7.1.0 \
  @flyingrobots/bijou-tui@7.1.0 \
  @flyingrobots/bijou-tui-app@7.1.0
```

If your app uses localization tooling, keep those packages in lock-step too:

```
npm install \
  @flyingrobots/bijou-i18n@7.1.0 \
  @flyingrobots/bijou-i18n-tools@7.1.0 \
  @flyingrobots/bijou-i18n-tools-node@7.1.0 \
  @flyingrobots/bijou-i18n-tools-xlsx@7.1.0
```

If your app uses the Bijou MCP server, keep it lock-step too:

```
npm install @flyingrobots/bijou-mcp@7.1.0
```

Runtime Behavior

No breaking runtime migration is expected.

Applications may notice better behavior in these areas:

- full-screen TUIs repaint correctly after terminal resize, laptop sleep/wake, or external-display geometry changes
- ANSI parsing correctly handles scoped foreground/background resets
- DOGFOOD Blocks guide navigation can focus parent preview rows
- quit confirmation copy is less redundant

If your app had local workarounds for stale cells after resize or manual ANSI reset cleanup, retest those paths against 7.1.0 and remove the workaround if Bijou now owns the behavior.

New Proof APIs

The new ui-scene-ir/1 and GraphQL block helpers are available for early proof work, but they are still the first slices of a larger Runtime Graph direction. Use them when you want deterministic receipts, source maps, lower modes, and agent-readable debug facts. Do not assume they are the final V8 product contract yet.

Theme Work

Theme-token APIs are more useful in 7.1.0, including first-party dark/light presets and safe-pair declarations. Existing raw-color code should keep working. New code should prefer semantic token refs when the color is part of a themeable product surface.

Release Tooling

Maintainers preparing releases should use:

```
npm run release:readiness -- --milestone v7.1.0
```

The milestone-aware mode blocks missing release evidence packets, open target issues, lingering work-in-progress labels, stale release posture docs, and missing package-smoke coverage before the local gauntlet runs.

DOGFOOD contributors should also expect npm run dogfood:i18n:debt to scan new examples/docs/**/*.ts modules by default. Tooling-only exclusions must be explicit.

Compatibility Stance

Bijou 7.1.0 should be a straightforward upgrade from 7.0.0 for application code. The main migration work is optional cleanup: remove local resize/ANSI workarounds, adopt theme tokens where they clarify product intent, and use the new proof helpers only where deterministic UI evidence is valuable.

271

Migrating to Bijou 7.2.0

docs/releases/7.2.0/migration-guide.md

Bijou 7.2.0 is a minor release from 7.1.0. It is intended to be additive for applications using documented public APIs.

The public tag was created from the verified 2026-07-05 main release commit. These instructions are the versioned migration guide for the 7.2.0 package set.

Recommended Upgrade

Update the runtime packages your app uses together:

```
npm install \
  @flyingrobots/bijou@7.2.0 \
  @flyingrobots/bijou-node@7.2.0 \
  @flyingrobots/bijou-tui@7.2.0 \
  @flyingrobots/bijou-tui-app@7.2.0
```

If your app uses localization tooling, keep those packages in lock-step too:

```
npm install \
  @flyingrobots/bijou-i18n@7.2.0 \
  @flyingrobots/bijou-i18n-tools@7.2.0 \
  @flyingrobots/bijou-i18n-tools-node@7.2.0 \
  @flyingrobots/bijou-i18n-tools-xlsx@7.2.0
```

If your app uses the Bijou MCP server, keep it lock-step too:

```
npm install @flyingrobots/bijou-mcp@7.2.0
```

Runtime Behavior

No breaking runtime migration is expected.

Applications may notice better behavior in these areas:

- page-scoped TUI frame helpers are exported from the public TUI package root
- scripted driver helpers can replay mouse movement, press, release, wheel, and raw SGR mouse sequences deterministically
- DOGFOOD light-theme and locale-fallback examples provide clearer evidence for release review and demos

- Blocks app-binding documentation now includes concrete copyable snippets

If your app had local test helpers for mouse replay or frame helper imports, prefer the package exports after upgrading.

DOGFOOD And Documentation

The DOGFOOD release pages for 7.2.0 depend on the versioned docs in this directory. Keep whats-new.md, migration-guide.md, and README.md together when preparing or validating a release branch.

Release Tooling

Maintainers replaying or auditing the release should use both release-readiness modes:

```
npm run release:readiness
npm run release:readiness -- --milestone v7.2.0
```

The plain command validates local release evidence without live milestone checks. The milestone-aware command is the release gate that passed from the merged main release commit after the target milestone had zero open tracker items and no tracked work carried a work-in-progress label.

Compatibility Stance

Bijou 7.2.0 should be a straightforward upgrade from 7.1.0 for application code. The main migration work is optional cleanup: remove local mouse replay fixtures that duplicate the public driver helpers, update imports to use the public TUI package root where possible, and treat the improved DOGFOOD release pages as review evidence rather than an application contract.

272

Migrating to Bijou 8.0.0-rc.1

docs/releases/8.0.0/migration-guide.md

This is a release candidate. Pin the exact version; do not use a caret range.

```
npm install @flyingrobots/bijou@8.0.0-rc.1 \
            @flyingrobots/bijou-node@8.0.0-rc.1 \
            @flyingrobots/bijou-tui@8.0.0-rc.1
```

All workspace packages are versioned in lock-step, so mixing 8.0.0-rc.1 with 7.2.0 packages is unsupported.

Two breaking changes, both from DX-052. Neither requires a code change for most applications; both change what your application *does* in situations you may not have tested.

1. NO_COLOR no longer forces 'pipe' mode

What changed

detectOutputMode() no longer inspects NO_COLOR. The detection order is now:

```
BIJOU_ACCESSIBLE=1  -> 'accessible'
TERM=dumb           -> 'pipe'
!stdout.isTTY       -> 'pipe'
CI                  -> 'static'
stdout.isTTY        -> 'interactive'
```

Why

NO_COLOR is a statement about colour. no-color.org specifies that its presence "prevents the addition of ANSI color" and says nothing about cursor addressing or interactivity. Mapping it to 'pipe' meant a user who exported NO_COLOR=1 in a shell profile — common, and reasonable — lost the entire TUI rather than only its colour: every component lowered to its single-frame form, and shouldUseShellQuitConfirm() began returning 'immediate'.

What did not change

Colour output. NO_COLOR is read independently by createBijou(), createNodeContext(), and isNoColor(), none of which consult the mode. With NO_COLOR set

you still get theme.noColor === true, ink() returning undefined, and styled() returning unstyled text.

To be precise about the scope: an interactive session does emit cursor movement, screen updates, and possibly alternate-screen sequences — it has to, or it is not a TUI. What NO_COLOR suppresses is **colour and style escapes**, which is what it asks for.

What you may need to do

If you relied on NO_COLOR to get single-frame output, say in a script or a log-capturing harness, it no longer does that. Use whichever is accurate:

```
// The terminal genuinely cannot do cursor addressing:
TERM=dumb

// You are capturing output rather than driving a terminal:
//   redirect stdout — a non-TTY stdout still yields 'pipe'

// You want to force it from inside the app:
const ctx = { ...createBijou(ports), mode: 'pipe' as const };
```

If you set NO_COLOR because you wanted a monochrome TUI, you now get one. No action needed; this is the fix.

If you wrote your own detector to work around this — git-cas and muniment both did — you can delete it. Check that your override does not also encode something else you still want.

2. An unknown status key no longer resolves to a struck-through token

What changed about the fallback

ctx.status(key) and ResolvedTheme.inkStatus(key) fall back to semantic.muted instead of status.muted when key is not in the theme.

Why the fallback moved

status.muted carries ['dim', 'strikethrough'] in every shipped preset. That is reasonable for a token meaning "retired" — and wrong as the answer to "I don't know this key". Because status(key) accepts string, a typo, or a key an application forgot to add to its theme, silently rendered text with a line through it. A reader sees "cancelled"; the application said nothing of the kind.

The new target matches the sibling ui() accessor, which already falls back to semantic.primary.

What you may need to do about the fallback

Probably nothing. status.muted and semantic.muted share a hex in every shipped preset, so the only visible difference is the absence of the strikethrough.

If you defined a theme where those two tokens have different hexes, an unknown key now resolves to the semantic.muted hex. Check any custom preset.

If you were relying on the strikethrough fallback as a way to spot undefined keys during development, it is gone. It was never a reliable signal — it only appeared when a code path actually ran. #521 tracks typing the accessors so a missing key becomes a compile error instead.

ctx.status('muted') itself is unchanged, strikethrough intact.

3. Additive: extendTheme() reaches all six token groups

Not breaking. border and semantic join status, ui, gradient, and surface:

```
const theme = extendTheme(BIJOU_DARK, {
  status: { guarded: tv('#4cc9c3') },
  border: { primary: tv('#75798d') },     // now possible
  semantic: { accent: tv('#d6c5ff') },    // now possible
});
```

Both are overridable rather than extensible with new keys, because Theme fixes their key sets.

Why this matters more than it looks

The shipped presets alias heavily across groups. bijou-dark uses #f2c45d for status.warning, status.active, semantic.warning, semantic.accent, ui.cursor, ui.focusGutter, ui.sectionHeader, ui.logo, border.secondary **and** border.warning — ten tokens, one colour. bijou-light does the same with #7a5200.

So extendTheme(BIJOU_DARK, { status }) replaced your statuses and silently kept that amber for every heading, cursor, and border. If you have done that, your headings are probably the same colour as your warning. That is not hypothetical — it is how this was found, at ΔE 0.023 in OKLab, which is indistinguishable.

If you override status, consider auditing ui and border too:

```
import { doctorTheme } from '@flyingrobots/bijou';
console.log(doctorTheme(myTheme, { maxColorReuse: 2 }));
```

Note that doctorTheme checks contrast and reuse but not perceptual separation under colour-vision deficiency — see #520 — and that the shipped presets do not currently satisfy it (#519).

Reporting rc feedback

This candidate exists to find out what these two changes break in real applications. Please file against #518 or open a new issue with the lane:release label.

273

Release

docs/release.md

Bijou's official release process

Bijou adopts the spirit of the older "Universal Release Pre-Flight" prompt, but this file is now the repo-native source of truth.

When this document and a generic release prompt disagree, this document wins.

Scope

Bijou is currently:

- an npm workspace monorepo
- lock-step versioned across all workspace packages
- tag-driven for release automation
- GitHub Actions driven for publish, release notes, and GitHub Release creation
- npm-only for registry publishing; JSR is not part of the current release surface

Operator Flow

The older generic release prompt mixed together two different kinds of checks:

- repo guards that must pass **before** any release mutation happens
- validation gates that run **after** versions and release docs are updated

Bijou keeps that distinction explicit.

Phase 1 is an operator preflight. It proves that the repository is in a safe starting state.

release:readiness belongs later. By the time it runs, the worktree is supposed to contain the version bump, changelog edit, and README update. That is why release:readiness is the validation gauntlet, not the "clean worktree" guard.

Non-Negotiable Rules

1. Run release phases in order.
2. Abort on the first hard failure.
3. Never claim a release succeeded unless the tag, workflows, and npm registry state were directly verified.
4. Never create a release tag from a dirty worktree.
5. Never create a release tag from a branch other than main.
6. Never create a release tag from a main that is ahead of or behind origin/ma in.
7. Use repo-native tooling:
   - version bump: npm run version X.Y.Z
   - metadata check: npm run release:preflight
   - local gauntlet: npm run release:readiness
   - CI dry-run: .github/workflows/release-dry-run.yml
   - tag publish: .github/workflows/publish.yml
8. Respect lock-step versioning across the workspace.
9. Prepare releases on a branch, merge through a PR, and tag only the exact merged origin/main commit.

Registry Setup

Bijou publishes public packages to npm through GitHub Actions and npm trusted publishing. Do not store npm automation tokens in GitHub secrets.

For a new public workspace package, first publish one version manually from the package directory so npm has a package record:

```
npm login
NPM_CONFIG_PROVENANCE=false npm publish --access public
```

The seed publish is deliberately non-provenance because it runs outside the trusted GitHub Actions OIDC environment. After the trusted publisher is configured, automated releases should publish with provenance through publish .yml.

Then configure the npm package trusted publisher:

- Organization/user: flyingrobots
- Repository: bijou
- Workflow filename: publish.yml
- GitHub environment: npm

The GitHub npm environment does not need registry secrets for trusted publishing. Keep the package manifest publishConfig.provenance setting aligned with the publish workflow so npm records provenance for automated releases.

JSR setup from other projects does not apply to Bijou today. If Bijou later adds jsr.json, JSR package records, and a JSR publish workflow, update this runbook and the automation in the same PR.

Release Law

No public release tag may be created unless all automated and human evidence gates below are satisfied. A failed automated gate blocks the release. A human review gate blocks the release until the release evidence packet records either the required update or an explicit “reviewed, no change” disposition.

Automated Evidence Gates

The current executable gates are npm run release:preflight, npm run release:readiness, .github/workflows/release-dry-run.yml, .github/workflows/tag-guard.yml, and .github/workflows/publish.yml.

The release policy requires these checks, even where the current implementation still relies on operator verification:

Gate	Rule	Current enforcement
REL-TOOL-NODE	Node.js is available for metadata and validation commands.	release:readiness
REL-TOOL-NPM	npm is available for workspace, pack, audit, and registry checks.	release:readiness, workflows
REL-TOOL-GIT	Git is available for tag, ancestry, and clean-tree checks.	operator, workflows
REL-TOOL-GH	GitHub CLI can read the repository before live issue gates run.	operator
REL-TAG-FORMAT	Release tags use leading-v SemVer with optional alpha, beta, or rc prerelease suffix.	tag-guard.yml, release:preflight
REL-GH-ACCESS	gh can read flyingrobots/bijou before final issue gates run.	operator
REL-GH-PRIORITY-HIGH-ZERO	There are zero open release-blocking issues labeled priority:high, unless the evidence packet names the accepted risk and owner.	operator
REL-GH-TARGET-MILESTONE-ZERO	The target GitHub milestone exists and has zero open issues or PRs.	operator
REL-GH-PRIOR-RELEASE-ZERO	Lower v* milestones have zero open issues or PRs.	operator
REL-META-VERSION-LOCKSTEP	Every workspace package version and internal Bijou dependency version matches X.Y.Z.	release:preflight
REL-GIT-CLEAN	The worktree is clean before release prep starts, before the release PR is opened, and before final tagging.	operator
REL-GIT-ORIGIN-MAIN	The final tag commit is exactly origin/main.	operator

Gate	Rule	Current enforcement
REL-DOC-CHANGELOG-DATED	docs/CHANGELOG.md has a dated [X.Y.Z] - YYYY-MM-DD entry.	operator
REL-DOC-EVIDENCE	docs/releases/X.Y.Z/README.md records release evidence, review disposition, replay commands, and residual risk.	operator
REL-DOC-DOGFOOD-I18N	DOGFOOD markdown and visible strings satisfy supported-locale checks and the i18n debt ratchet.	release:readiness
REL-DOC-INVENTORY	Documentation inventory is valid after release docs are moved from next/ to X.Y.Z/.	operator
REL-PACK-DRY-RUN	Release dry-run workflow verifies packed files and npm publish dry-runs for the automated publish matrix.	release-dry-run.yml
REL-NPM-AUDIT	Runtime npm dependencies have no high or critical vulnerabilities.	operator

The stage model is:

Stage	Caller	Git posture	Issue gates
operator-preflight	maintainer before release prep	Clean local main exactly matches origin/main.	Optional visibility check.
prep-pr	release-prep branch	Branch-local validation after version/docs changes; does not require HEAD == origin/main.	Report target milestone state in the PR.
final-local	maintainer after merge	Local main exactly matches origin/main; final tag target is fixed.	Enforced live.
tag-workflow	tag push workflow	Tag commit must be the release commit on origin/main.	Policy target; current workflows do not enforce ancestry.
rerun-workflow	manual workflow dispatch for an existing tag	Existing tag must remain reachable from origin/main.	Skipped for registry recovery; do not move the tag.

Human Review Gates

Automation proves structure and repeatability. It does not prove release prose is truthful. The release evidence packet must record human review for:

- docs/CHANGELOG.md accurately reflects the diff since the previous public tag.
- README.md changed if front-door positioning, install commands, examples, or current release status changed.
- ARCHITECTURE.md changed if ports, adapters, package boundaries, storage, rendering contracts, or runtime posture changed.
- docs/VISION.md, docs/WORKFLOW.md, and GitHub milestone scope match the release posture; linked Linear items agree once migration is connected.

- docs/DOGFOOD.md and DOGFOOD-facing markdown changed if the docs app, release title, navigation model, localization posture, or proof surface changed.
- Package READMEs, API references, migration docs, design-system docs, and MCP docs changed when their public surfaces changed.
- Every landed release goalpost contributing to this version is named with its issue, design doc, landed PRs, completed slice count, deterministic proof, canonical fixtures or immutable inputs, witnesses, replay commands, and residual-risk disposition.
- Any accepted residual risk is named with rationale, owner, and a follow-up issue. Hidden accepted failures are not allowed.

Deterministic Reproducibility

Evidence is complete only when another operator can replay it from the tag commit, committed release packet, and named immutable inputs.

Every release evidence packet must record:

- the exact command, script, workflow, or DOGFOOD route that produced each witness
- the canonical fixture or immutable input used by that command, when the claim depends on more than repository state at the tag commit
- the expected stable result, normalized output, digest, or screenshot assertion that replay must reproduce
- any normalization applied for host-specific noise such as temp paths, clocks, terminal size, color mode, process IDs, registry timestamps, ordering, or environment-specific absolute paths

Place release-specific fixtures under docs/releases/X.Y.Z/fixtures/ unless an existing committed fixture is already canonical. DOGFOOD terminal witnesses should record the route, command, terminal dimensions, shell theme or renderer mode, and stable assertion. A screenshot or recording is useful, but it is not a substitute for the command and immutable input that produced it.

Current Release And Publish Surface

Latest Shipped Release

The latest shipped release is **7.2.0**.

- 7.2.0 - TUI mouse tracking and fallthrough stabilization, public page-scoped frame helpers, scripted mouse driver builders, DOGFOOD release story surfaces, Blocks app-binding snippets, theme and localization demo readiness, dependency audit cleanup, and Code Dojo ratchet progress
- 7.1.0 - portable ui-scene-ir/1 proof, GraphQL-authored bijou-block/1 artifacts, grouped debug facts, a real DOGFOOD NavigationListBlock fixture, terminal-rendering fixes, release-readiness guardrails, and DOGFOOD i18n debt coverage

- 7.0.0 - DOGFOOD release identity, BlockLab naming, release-facing proof, theme/token work, image-to-glyph tooling, and V7 title-screen polish
- 5.0.0 - frame-owned hosted runner, Node-host theme selection, crash-mode auto-exit, scoped Node I/O symlink hardening, deterministic scripted-example smoke, markdown pipe tables, and browsable-list marquee
- 4.4.1 - framed-shell background-fill fixes, stock shell theme cycling, uppercase quit-confirm input
- 4.4.0 - data-viz toolkit, zero-alloc framed app render, new bench scenarios
- 4.3.0 - RE-008 byte-packed surface, RE-015 braille fix, setRGB API
- 4.2.0 - RE-007 framed shell, @flyingrobots/bijou-mcp, METHOD migration
- 4.1.0 - runtime engine through RE-006, DOGFOOD maturation (DF-021 through DF-026, including DF-023, DF-024, WF-003), i18n packages

The release smoke contract is DOGFOOD-centered through smoke:dogfood, smoke:dogfood:landing, and smoke:dogfood:docs.

Long-form release docs live under docs/releases/<version>/.

Next Release Posture

8.0.0 is the next feature horizon: the Runtime Graph and Scene IR product contract built from the proof chain that 7.1.0 shipped and the release surface that 7.2.0 stabilized.

The latest shipped package release is 7.2.0. The older v6.0.0, v7.0.0, v7.1.0, and v7.2.0 milestones are zero-open tracker lanes retained as release lineage, not as the next target. The current target is the v8.0.0 milestone.

Keep V8 limited to stable-release closeout. Do not reopen 7.2.0 for broad Theme Lab, BlockLab, localization workbench, VISOR, Geordi, or Runtime Graph product work.

Keep selected release scope in the GitHub milestone and proof in the versioned release packet. Stable 8.0.0 follows 8.0.0-rc.1; #526 owns version-lock enforcement and #527 owns witness replay and documentation review. Linear execution migration is pending connection and workspace/team selection. Broad Sapphire and Theme Lab work remains in V9.

Lock-step Versioned Units

These workspaces move together:

- @flyingrobots/bijou
- @flyingrobots/bijou-node
- @flyingrobots/bijou-tui
- @flyingrobots/bijou-tui-app
- create-bijou-tui-app
- @flyingrobots/bijou-i18n
- @flyingrobots/bijou-i18n-tools
- @flyingrobots/bijou-i18n-tools-node
- @flyingrobots/bijou-i18n-tools-xlsx

- @flyingrobots/bijou-mcp
- @flyingrobots/bijou-bench (private benchmark workspace)

Current Automated npm Publish Matrix

As of now, the publish workflows automatically publish:

- @flyingrobots/bijou
- @flyingrobots/bijou-node
- @flyingrobots/bijou-tui
- @flyingrobots/bijou-tui-app
- create-bijou-tui-app
- @flyingrobots/bijou-i18n
- @flyingrobots/bijou-i18n-tools
- @flyingrobots/bijou-i18n-tools-node
- @flyingrobots/bijou-i18n-tools-xlsx
- @flyingrobots/bijou-mcp

Phase 1: Repo Guards

Run these checks before changing versions or docs.

1. Clean worktree

```
git status --porcelain=v1
```

Abort if any output exists.

2. Branch guard

```
git rev-parse --abbrev-ref HEAD
```

Abort unless the result is main.

3. Sync with origin

```
git fetch origin main --tags --prune
git rev-parse HEAD origin/main
```

Abort unless both SHAs are identical.

4. Latest release tag

```
git tag --merged HEAD --list 'v*' --sort=-version:refname | head -n 1
```

Abort if no valid release tag exists and there is no explicit first-release policy.

Phase 2: Choose Version And Update Release Docs

Bijou does **not** currently auto-compute the semver bump from commit history. The maintainer chooses the target version deliberately.

0. Slice release-boundary work before the final PR

Before bumping versions, make sure feature and documentation work has already landed through reviewable precursor PRs.

Hard stop before opening a release-boundary PR:

- more than 140 changed files

Caution threshold:

- more than 10 commits
- unrelated legends or cycle families on one branch
- generated snapshots or docs mixed with unrelated runtime changes
- more than one release-lane blocker implemented at once

When a threshold is hit, split the branch by cycle or product surface. The final release PR should primarily contain the version bump, changelog, long-form release docs, release-readiness proof, and final metadata updates.

1. Pick the target version

Allowed tag formats are:

- vX.Y.Z
- vX.Y.Z-rc.N
- vX.Y.Z-beta.N
- vX.Y.Z-alpha.N

Stable releases should be the default unless there is a clear reason for a prerelease.

2. Create the release-prep branch

Create the branch after the target version is known and before mutating files:

```
git switch -c release/vX.Y.Z
```

For prereleases:

```
git switch -c release/vX.Y.Z-rc.N
```

3. Bump all workspace manifests

```
npm run version X.Y.Z
```

This updates all workspace package versions and pins internal dependency references to the exact same version.

4. Update CHANGELOG.md

- move [Unreleased] into a real version header
- use UTC date format YYYY-MM-DD
- keep the existing changelog style intact
- do not release with an empty [Unreleased] section

5. Update README.md

- add or replace ## What's New in vX.Y.Z
- keep it concise and user-facing
- keep a visible link to CHANGELOG.md

6. Draft long-form release docs

Before the version is chosen, draft these in docs/releases/next/:

- README.md
- whats-new.md
- migration-guide.md

When the version is chosen, move that directory to docs/releases/X.Y.Z/ and update any version placeholders inside the docs.

The versioned README.md is the release evidence packet. It must include:

- release summary
- tracker and goalpost map
- automated evidence matrix
- human review matrix
- deterministic reproducibility record
- package and registry verification plan
- residual-risk disposition

Historical releases before this policy may only have whats-new.md and migrati on-guide.md. Future releases must carry the evidence packet.

Phase 3: Local Validation

Run:

```
npm run release:preflight
npm run release:readiness -- --milestone vX.Y.Z
npm run docs:inventory
npm audit --omit=dev --audit-level=high
```

release:readiness is the repo-native local gauntlet. With --milestone, it first prints a release-readiness report and blocks before the gauntlet if:

- the target GitHub milestone has open tracker items, including issues or pull requests
- any milestone tracker item still has work-in-progress
- CHANGELOG.md does not have an Unreleased or versioned release boundary
- docs/releases/X.Y.Z/README.md release evidence packet is missing
- the command plan no longer includes package smoke or the release guide no longer documents the release dry-run package boundary

Without --milestone, the command runs only the local gauntlet and does not require GitHub access. The gauntlet currently runs these gates in order:

1. build
2. lint
3. typecheck:test
4. docs:design-system:preflight
5. dogfood:coverage:gate
6. dogfood:i18n:check
7. dogfood:i18n:debt
8. workflow:shell:preflight
9. release:preflight
10. test:frames
11. verify:interactive-examples
12. smoke:canaries -- --skip-build
13. smoke:dogfood -- --skip-build
14. npm test

Abort on any failure.

This reflects the current repo tooling after WF-003 moved the release smoke gate onto the DOGFOOD contract used for 4.1.0 and later releases.

Phase 4: CI Dry Run

Before tagging a real release, run the **Release Dry Run** workflow in GitHub Actions against the release-prep branch or PR commit.

This is currently the authoritative place where Bijou checks:

- packed file verification
- npm publish dry-runs for the automated publish matrix
- release notes preview generation

Until equivalent local tooling exists, treat the dry-run workflow as a required release gate for real releases.

Phase 5: Commit, Push, And Open The Release PR

After docs and validation are green:

```
git add -A
git commit -m "chore(release): vX.Y.Z"
git push -u origin release/vX.Y.Z
```

For prereleases:

```
git commit -m "chore(release): vX.Y.Z-rc.N"
git push -u origin release/vX.Y.Z-rc.N
```

Open a normal PR to main. Do not open a draft PR. The PR body must link the release evidence packet, target milestone, changelog entry, release dry-run, and validation commands.

Merge the release PR only after review and green CI. Do not tag the branch commit before it is merged.

Phase 6: Final Main Preflight And Tag

After the release PR is merged, fast-forward local main to the exact release commit:

```
git fetch origin main --tags --prune
git switch main
git merge --ff-only origin/main
git status --porcelain=v1
git rev-parse HEAD origin/main
test "$(git rev-parse HEAD)" = "$(git rev-parse origin/main)"
npm run release:preflight
npm run docs:inventory
```

Abort if the worktree is dirty, if the two printed SHAs differ, or if the tes t command fails. Local main must exactly match origin/main; being ahead of or igin/main is also release-blocking.

Create the tag from that exact commit:

```
git tag -a vX.Y.Z -m "release: vX.Y.Z"
```

For prereleases:

```
git tag -a vX.Y.Z-rc.N -m "release: vX.Y.Z-rc.N"
```

Bijou currently enforces tag **format** through tag-guard.yml, not signed tags. Do not pretend signed-tag policy exists unless the automation is updated to require it.

Phase 7: Push The Tag And Let Automation Publish

Push only the tag:

```
git push origin vX.Y.Z
```

The publish workflow then owns:

- metadata verification
- build, lint, typecheck, test, smoke, and packed-file verification
- npm publication for the automated publish matrix
- GitHub Release creation or update

If a tag has already been pushed and the publish workflow itself needs a repair, fix .github/workflows/publish.yml on main and then rerun the same release through the workflow's workflow_dispatch entrypoint using the existing tag and release ref. Do not delete or move the release tag just to recover from workflow drift.

Do not manually create a GitHub Release unless the automation fails and the process is being explicitly repaired.

Phase 8: Verify The Result

Do not stop at "workflow started."

Verify all of the following:

- the tag-guard workflow passed
- the publish workflow passed
- npm shows the new version for each package in the automated publish matrix
- npm dist-tags match the release type
  - stable -> latest
  - rc -> next
  - beta -> beta
  - alpha -> alpha
- the GitHub Release exists with the expected notes

Representative checks:

```
npm view @flyingrobots/bijou version dist-tags --json
npm view @flyingrobots/bijou-node version dist-tags --json
npm view @flyingrobots/bijou-tui version dist-tags --json
npm view @flyingrobots/bijou-tui-app version dist-tags --json
npm view create-bijou-tui-app version dist-tags --json
npm view @flyingrobots/bijou-i18n version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools-node version dist-tags --json
npm view @flyingrobots/bijou-i18n-tools-xlsx version dist-tags --json
npm view @flyingrobots/bijou-mcp version dist-tags --json
```

Abort Conditions

Abort immediately if any of these are true:

- Phase 1 preflight starts from a dirty worktree

- Phase 1 preflight starts with the operator checked out to a branch other than main; after Phase 1 passes, Phase 2 step 2 creates the release/vX.Y.Z branch where prep work continues
- final tag prep is not on main
- final tag prep is not exactly synced to origin/main
- version bump not reflected cleanly across the workspace
- CHANGELOG.md still has no real release section
- docs/releases/X.Y.Z/README.md release evidence packet is missing
- target GitHub milestone does not exist
- target GitHub milestone has open issues or PRs
- lower v* GitHub milestones have open issues or PRs
- open priority:high issues lack an explicit release-risk disposition
- release:preflight fails
- release:readiness fails
- docs:inventory fails
- npm audit --omit=dev --audit-level=high fails
- release dry-run fails
- tag push fails
- publish workflow fails
- npm does not show the new version and expected dist-tag mapping

Known Current Gaps

- Bijou does not yet have a stage-aware release:prep or release:guard script. The policy names those gates, but release:preflight and release:readiness are the current repo-native commands.
- clean-worktree is an operator preflight check, not a release:readiness concern, because Phase 2 intentionally dirties the worktree
- branch-sync, latest-tag, tag-collision, live GitHub pull-request milestone gates, changelog dating, release tag ancestry, release evidence shape, and human review disposition are documented here, but they do not yet have a single dedicated repo-native guard
- docs:inventory and runtime npm audit are release policy gates but are not currently part of release:readiness
- JSR publishing is not currently part of Bijou’s release surface
- signed tags are not currently a repo policy

If any of those policies change, update this doc and the automation together.

274

What's New In Bijou v4.1.0

docs/releases/4.1.0/whats-new.md

This is the long-form release overview for Bijou v4.1.0.

It is intentionally different from CHANGELOG.md: the changelog is the ledger, while this document explains the shape and meaning of the release.

Release Story

The short version is:

Bijou has moved from a surface-first terminal toolkit toward a more coherent application platform.

The main themes are:

- the runtime now has a real engine model instead of shell folklore
- the framed shell is calmer, richer, and easier to inspect
- DOGFOOD is now a serious proving surface rather than a partial demo
- localization has a real substrate and tooling story
- design language and delivery doctrine are now explicit repo truth

If v4.0.0 was the "pure v4 surface" release, v4.1.0 is the "runtime, shell, docs, and tooling grow up together" release.

Runtime Engine: From Seams To Model

The biggest architectural change since v4.0.0 is the new runtime engine direction captured in:

- RE-001
- RE-002
- RE-003
- RE-004
- RE-005
- RE-006

In practical terms, the runtime now exposes a clearer model for:

- state machines versus view stacks
- retained layouts and invalidation causes
- layout-driven input routing
- distinct command and effect buffers
- component-owned interaction contracts

This matters because builders can now explain where state lives, where views live, how geometry is retained, and how input is routed without relying on branch order inside the shell. Agents benefit even more: runtime ownership is increasingly inspectable instead of implicit.

The remaining follow-on is RE-007, which is about moving more of the framed shell onto those explicit seams and is intentionally deferred until after v4.1.0.

Framed Shell: More Humane, More Truthful

The shell-focused work since v4.0.0 substantially improves the framed application experience. The key cycles are:

- HT-002
- HT-003
- HT-004

Notable outcomes:

- framed apps now use a clearer topmost-layer model for routing and dismissal
- help, settings, notifications, quit confirm, and page modals behave more like real shell layers and less like ad hoc branches
- footer and help affordances are more truthful to the actual active layer
- richer shell introspection is exported through helpers like describeFrameLayerStack(...), activeFrameLayer(...), and underlyingFrameLayer(...)

For operators, the result is calmer and safer fullscreen behavior. For builders and agents, the shell is easier to reason about because it advertises more of its actual state.

Framed-App Typing And Command Lifecycles

The developer-experience work since v4.0.0 is not just cosmetic. Two important seams got more honest:

- DX-001
- DX-002

The framed-shell surface now exports explicit typed helpers such as:

- FramePageMsg
- FramePageUpdateResult
- FramedAppMsg

- FramedApp
- FramedAppUpdateResult

And Cmd<M> now reflects the runtime patterns Bijou actually uses. A command may:

- complete synchronously or asynchronously
- emit intermediate messages
- resolve to a final message
- resolve to QUIT
- return a cleanup handle or cleanup function for long-lived work
- resolve to void

That makes the public API line up better with real pulse timers, runtime subscriptions, and sub-app command mapping.

DOGFOOD: From Partial Demo To Real Field Guide

The DOGFOOD work since v4.0.0 is one of the clearest visible changes.

The field guide moved through the coverage ratchet from early partial coverage to complete coverage of all canonical component families, culminating in DF-019.

What changed:

- DOGFOOD now covers 35/35 canonical component families
- the coverage floor is enforced by repo tooling instead of being a decorative progress bar
- the docs app in examples/docs is now a living product surface, not just a showcase shell
- the next DOGFOOD work is quality and depth, not pretending there is a hidden extra family-count milestone

This is important for both users and maintainers. Users get a much more credible field guide. Maintainers get a stronger proving surface for shell, design-language, and docs changes.

Localization: Real Packages And Workflows

Localization moved from strategy-doc ambition into actual packages and workflows.

The relevant landed work includes:

- LX-001
- LX-002
- LX-003
- LX-004
- LX-005

- LX-006
- LX-008

The workspace now includes:

- @flyingrobots/bijou-i18n
- @flyingrobots/bijou-i18n-tools
- @flyingrobots/bijou-i18n-tools-node
- @flyingrobots/bijou-i18n-tools-xlsx

And the shell/docs surface now has real localized behavior instead of only English-first assumptions.

Important release note: the i18n packages are now part of the planned v4.1.0 automated release path. The publish workflow, release dry-run workflow, and release docs now all agree that these packages are public release artifacts for the 4.1.0 line.

Design Language: More Explicit Product Taste

Bijou's design language is much less implicit than it was at v4.0.0.

The post-v4.0.0 work established:

- explicit UX doctrine and content guidance
- a calmer explainability surface
- inspector rhythm and inspector drawer patterns
- stronger shell review-surface patterns
- more concrete design-system guidance across component families

Representative cycles include:

- DL-001
- DL-003
- DL-004
- DL-005
- DL-006
- DL-007

The result is that Bijou now has stronger answers to questions like:

- what should shell review surfaces feel like?
- how should AI reasoning be surfaced honestly?
- what does calm terminal hierarchy look like?
- which UI choices are doctrine, and which are merely possible?

Performance, Validation, And Release Hardening

The repo also got materially stronger operationally.

Notable improvements since v4.0.0:

- renderer benchmark coverage and low-allocation renderer work
- stronger smoke tooling and example/workflow validation
- a DOGFOOD coverage gate
- a design-system docs preflight
- a workflow shell preflight
- a repo-native release readiness script
- improved publish and release-dry-run workflow compatibility

This work is less glamorous than the runtime and shell work, but it matters because it makes the release more reproducible and the repo less dependent on luck.

Repo Signposts And Engineering Doctrine

The repository itself is easier to navigate and reason about than it was at v4.0.0.

New or newly formalized repo surfaces include:

- METHOD
- BEARING
- VISION
- System-Style JavaScript
- Release Guide

This is mostly maintainer-facing, but it has a user-visible effect: the repo is less likely to drift between code, docs, backlog, and release claims.

What This Release Is Best At

Compared to v4.0.0, this release is better when you need:

- a clearer fullscreen application substrate
- a calmer framed shell with more truthful interaction behavior
- a serious living docs surface instead of a demo stub
- first-class localization packages and workflow seams
- clearer guidance about how Bijou wants to be used

What This Release Does Not Pretend

This release still does not mean:

- the runtime-engine migration is completely finished
- every shell concern has been pushed fully behind the runtime seams
- DOGFOOD is “done forever”
- localization is complete across every shell surface

The point is not that the work is finished. The point is that the repo now has much more honest runtime truth, shell truth, docs truth, and release truth than it had at v4.0.0.

275

What's New in Bijou 4.2.0

docs/releases/4.2.0/whats-new.md

MCP Rendering Server

Bijou now ships @flyingrobots/bijou-mcp, an MCP (Model Context Protocol) server that exposes Bijou's terminal components as rendering tools over stdio.

Any MCP client — Claude Code, Cursor, or a custom integration — can call 22 tools to get Unicode box-drawing output that renders correctly in monospace chat contexts. Tables, trees, DAGs, alert boxes, progress bars, timelines, and more — all produced without ANSI escape codes.

Why this matters

LLM-powered coding tools talk in plain text. Structured terminal output (tables, graphs, trees) is useful in that context, but the rendering usually lives on the other side of a TTY. bijou-mcp bridges that gap: the AI calls a tool, Bijou renders the component, and the result drops into the conversation as clean Unicode text.

Highlights

- **22 rendering tools** covering data/structure, containers/layout, feedback/status, navigation, rich panels, and utility categories
- **No ANSI** — uses createTestContext with plainStyle() for terminal-free rendering
- **Zero config** — point your MCP client at the bin entrypoint and go

See the bijou-mcp README for the full tool inventory and configuration instructions.

Runtime Engine: Framed Shell Migration (RE-007)

The framed shell now routes all input through the runtime engine's routeRuntimeInput infrastructure instead of ad-hoc key/mouse branches.

What changed

- Key routing flows through routeRuntimeInput producing FrameShellCommand facts
- Mouse hit-testing uses retained layout trees with explicit layout nodes (tab:{pageId}, pane:{paneId}, settings-row:{index})
- Commands are buffered via bufferRuntimeRouteResult and applied via applyRuntimeCommandBuffer
- Shell commands are a plain discriminated union interpreted by a handler table, not classes with behavior
- The update() key and mouse branches are reduced to single-line buffer drains

What was removed

- handleFrameMouse
- paneHitAtPosition
- settingsRowAtPosition
- isInsideSettingsDrawer
- withObservedKey
- applyQuitRequest
- observedRouteForLayer
- applyHelpScrollAction

These functions were replaced by the runtime engine's routing and buffer infrastructure. The migration is captured in RE-007.

Bug Fixes

- **Inspector supportingTextLabel** now renders in interactive mode. Pipe and accessible modes already included it; interactive was silently dropping the label.
- **Explainability evidence values** now render through the MCP wrapper. Evidence items display as "label — value" instead of label-only.

Infrastructure

- All release infrastructure (publish workflow, dry-run workflow, pack verification, npm verify checks) now includes @flyingrobots/bijou-mcp.

- The repo's planning system has been migrated to METHOD with all 7 legends resolving correctly.

Package Map

This release adds one new package to the lock-step family:

Package	Role
@flyingrobots/bijou-mcp	MCP server: Bijou rendering tools for AI chat contexts

All 10 workspace packages are versioned together at 4.2.0.

276

What's New in Bijou 4.3.0

docs/releases/4.3.0/whats-new.md

Byte-Packed Surface Rendering (RE-008)

The internal Surface data structure is now backed by a packed Uint8Array — 10 bytes per cell instead of heap-allocated JavaScript objects with string colors.

What changed

Every cell in a surface is now a contiguous 10-byte sequence:

- **Bytes 0–1**: character code (uint16, with a side table for emoji and multi-codepoint grapheme clusters)
- **Bytes 2–4**: foreground R, G, B
- **Bytes 5–7**: background R, G, B
- **Byte 8**: modifier flags bitfield (bold, dim, strikethrough, inverse, underline variants)
- **Byte 9**: opacity + fg/bg presence bits

What this means for performance

The differ now compares cells as 10-byte sequences instead of string equality on fg, bg, and char properties. Surface composition (blit) copies bytes directly between buffers. fill() and clear() use a template-stamp approach. The ANSI output path emits SGR escape sequences directly from buffer bytes with a cached style lookup — bypassing chalk entirely.

DOGFOOD landing render+diff: 7% faster than the pre-RE-008 baseline, with zero regressions on any real-world scenario.

The setRGB() API

A new Surface.setRGB() method writes character code + numeric RGB values directly into the packed buffer with zero string parsing and zero object allocation. It's roughly 10–50x faster than set() for per-cell writes.

```
// Before (allocates Cell object, parses hex strings)
surface.set(x, y, { char: '█', fg: '#ff8800', bg: '#000000' });

// After (zero-alloc — writes bytes directly)
surface.setRGB(x, y, 0x2588, 255, 136, 0, 0, 0, 0);
```

All built-in components use setRGB automatically when the surface is packed. The set() convenience API is preserved and documented.

Numeric theme tokens

TokenValue now carries optional fgRGB and bgRGB fields with pre-parsed numeric RGB channels. These are populated automatically at theme resolution time, enabling components to skip hex parsing when writing styled cells.

Braille Rendering Fix (RE-015)

flexSurface with align: 'center' no longer corrupts braille characters (U+2800–U+28FF). The packed byte-copy blit preserves all character codes directly without intermediate string processing. Verified with a 420×140 (58,800 cell) braille art test case — zero mismatches.

New Exports

- **PackedSurface interface** — extends Surface with .buffer, .sideTable, and .markAllDirty()
- **@flyingrobots/bijou/perf** — subpath export for packed-cell utilities (parseHex, encodeModifiers, flag constants)
- **Modifier flag constants** — FLAG_BOLD, FLAG_DIM, FLAG_STRIKETHROUGH, FLAG_INVERSE, UNDERLINE_SOLID, etc.

Resolved Backlog Items

- **RE-010 (mutable surface caches)** — resolved by the packed buffer with lazy dirty bitmap
- **RE-015 (braille corruption)** — resolved by byte-copy blit

Package Map

All 10 workspace packages are versioned together at 4.3.0. No new packages in this release.

277

What's New in Bijou 4.4.0

docs/releases/4.4.0/whats-new.md

Data Visualization Toolkit

Bijou now ships a data-visualization component family in @flyingrobots/bijou:

sparkline()

Compact inline trend graph using Unicode block characters (▁▂▃▄▅▆▇). Drop it next to a label or inside a table cell for instant trend context.

```
import { sparkline } from '@flyingrobots/bijou';

sparkline([1, 5, 3, 8, 2, 7]);
// → '▁▄▃▇▂▅'

sparkline([10, 20, 30, 40], { width: 8 });
// → '▁▂▃▄▅▆▇█'  (values interpolated to fit width)
```

brailleChartSurface()

High-density filled area chart using Unicode Braille characters. Each terminal cell is a 2x4 sub-pixel grid, giving smooth curves in very tight space.

```
import { brailleChartSurface } from '@flyingrobots/bijou';

const chart = brailleChartSurface(frameTimeHistory, {
  width: 28,
  height: 6,
  ctx,
});
```

statsPanelSurface()

Titled bordered panel with aligned key-value metric rows. Each row can include an optional inline sparkline after the value.

```
import { statsPanelSurface } from '@flyingrobots/bijou';

const panel = statsPanelSurface([
```

```
  { label: 'FPS', value: '60', sparkline: fpsHistory },
  { label: 'heap', value: '42.1 MB' },
], { title: 'Runtime', width: 30, ctx });
```

perfOverlaySurface()

Prebuilt FPS + memory dashboard composing statsPanelSurface and brailleChartSurface. Drop-in performance overlay for any app.

```
import { perfOverlaySurface } from '@flyingrobots/bijou';

const overlay = perfOverlaySurface({
  fps: 60,
  frameTimeMs: 16.7,
  frameTimeHistory: samples,
  width: 120,
  height: 40,
  heapUsedMB: 42.1,
}, { ctx });
```

Performance: Zero-Alloc Framed App Render Loop

The framed app render loop now uses zero-allocation header and footer painting via paintStyledTextSurfaceWithBCSS repainting in-place. The pane scratch surface pool is scoped to the createFramedApp closure instead of module-level state, resolving RE-010.

Bench: New Scenarios and Dynamic Sizing

The benchmark harness gains two new scenarios:

- **flame** — doom fire effect with full-screen palette interpolation (~2.6 ms/frame)
- **component-app** — realistic TUI using real bijou components (boxSurface, tableSurface, alertSurface, etc., ~564 µs/frame)

All scenarios now support dynamic sizing via an optional columns / rows argument to setup(). The soak runner has been rewritten on createFramedApp with a perf overlay toggled by backtick.

DOGFOOD: Data Visualization Stories

The DOGFOOD component gallery now covers the data-visualization family with stories for all four new components, bringing the component-family reference from 35 to 36 families at 100% documented coverage.

Backlog Cleanup

- Closed stale backlog items: RE-008, RE-007, RE-009, RE-010, RE-015 (code fixes shipped in earlier releases; trackers administratively closed)
- Added: RE-021 (frame owns the pump — cool idea for next major)
- Removed: PLAN.md (redundant narrative wrapper)

278

What's New in Bijou 4.4.1

docs/releases/4.4.1/whats-new.md

Framed Shell Backgrounds Render Correctly Again

Bijou 4.4.1 fixes the createFramedApp() background-fill regression introduced in 4.4.0 and extends the fix to the stock shell chrome. Apps that set a custom surface.primary.bg could lose that body background when pane content, gutters, scrollbars, split dividers, or other frame-owned surfaces were rendered, causing the terminal default background to show through. The stock header and footer rows could also fall back to black when BCSS did not provide explicit shell backgrounds.

What changed

- The frame body is now explicitly filled with the primary surface background before pane composition.
- Pane surfaces stamp that background into otherwise unstyled cells before blitting, so plain text and blank cells inherit the intended body fill.
- Gutter, scrollbar, split-divider, minimized-pane, and missing-grid placeholder writes now preserve an existing background instead of wiping it.
- The stock header and footer rows now inherit the resolved frame background when shell-specific BCSS background rules are absent.

What this means for app authors

If your app themes surface.primary.bg, the framed shell should now render that color consistently across both the workspace body and the shell chrome.

createFramedApp() Gets Stock Shell Themes

createFramedApp() now supports opt-in, frame-owned shell theme cycling:

```
let currentCtx = ctx;
const getCurrentCtx = () => currentCtx;
```

```
const initialModel = {};

const app = createFramedApp({
  ctx: currentCtx,
  shellThemes: [
    { id: 'default', label: 'Default', theme: defaultTheme },
    { id: 'verdant-plum', label: 'Verdant Plum', theme: plumTheme },
  ],
  onShellThemeChange: ({ ctx: nextCtx }) => {
    currentCtx = nextCtx;
  },
  pages: [{
    id: 'home',
    title: 'Home',
    init: () => [initialModel, []],
    update: (msg, model) => [model, []],
    layout: (model) => ({
      kind: 'pane',
      paneId: 'main',
      render: (width, height) => renderHome(getCurrentCtx(), model, width, height),
    }),
  }],
});
```

When shellThemes are provided:

- the frame pre-resolves those themes once against the provided Bijou context, or the active default context when ctx is omitted
- the stock settings drawer gets a built-in shell-theme choice row when two or more shell themes are available
- the current selection is exposed as FrameModel.activeShellThemeId

If your page renderers read from an explicit app-owned BijouContext, pass that context to createFramedApp({ ctx }), mirror onShellThemeChange back into your app state, and make your page renderers read the updated context on each render so the frame shell and page content stay on one theme source of truth. Apps that rely on the ambient default context can omit that wiring; the frame updates the default context when the stock shell theme changes.

DOGFOOD now uses that shared frame-owned state so the title screen and docs explorer stay on one theme setting instead of drifting into mixed landing/app themes. The release also adds the new Verdant Plum palette used to prove the background-fill fix visually.

Quit Confirm Accepts Uppercase Input

The stock shell quit dialog now accepts uppercase Y and N in addition to lowercase y and n. This does not change the overall quit policy or the existing Enter, Esc, and q behavior.

Compatibility

Bijou 4.4.1 is a backwards-compatible patch release. The new framed shell theme support is optional.

279

What's New in Bijou 5.0.0

docs/releases/5.0.0/whats-new.md

Bijou 5.0.0 promotes the hosted shell, Node host bootstrapping, and release-proof surface into a cleaner first-class product line. Most existing 4.4.x apps continue to work, but the preferred paths are now simpler, more explicit, and more honest about what the framework owns.

Framed Apps Own Their Hosted Runner

createFramedApp() now returns a self-running framed shell:

```
const app = createFramedApp(options);
await app.run({ ctx });
```

There is also a one-call hosted helper:

```
await runFramedApp(options, { ctx });
```

What changed:

- framed apps now own their hosted runtime path through app.run(...) instead of forcing callers to remember a separate run(app, ...) wrapper
- @flyingrobots/bijou-node startApp(app) detects and delegates to that self-running path automatically
- the framed runner uses the run-time ctx as the shell-owned rendering context for the duration of the run
- committed pipeline timings are folded back into FrameModel so shell-owned UI can read frameTimeMs, viewTimeMs, diffTimeMs, frameBudgetMs, and frameOverBudget

This is the biggest posture change in the release: the frame now owns its hosted shell surface more honestly instead of pretending it is just another raw TEA app.

Node Hosts Get Easy Theme Selection

Node-hosted apps can now opt into a custom theme without assembling a raw BijouContext by hand:

```
await startApp(app, { theme: MY_THEME });
```

The host also supports theme sets with automatic light/dark selection and explicit user override:

```
await startApp(app, {
  themes: [
    { id: 'light', theme: LIGHT_THEME },
    { id: 'dark', theme: DARK_THEME },
  ],
  themeMode: 'auto',
  themeOverride: savedThemeId,
});
```

Selection order is:

1. themeOverride
2. BIJOU_THEME
3. themeMode: 'auto' terminal color-scheme detection
4. first theme-set entry fallback

This makes dark/light mode and persisted user choice a host concern instead of boilerplate that every app has to reinvent.

The Proof Surface Is More Honest

Several repo-facing and operator-facing surfaces were tightened up in this release:

- markdown() now renders bounded pipe tables through the real table component instead of leaking raw Markdown separators into DOGFOOD and other prose views
- the root workspace now exposes npm run perf as a first-class visual performance demo entrypoint
- the scripted interactive smoke path is deterministic and isolated in verify :interactive-examples instead of depending on Vitest's default timeout or global console.* monkeypatching
- focused rows in browsable lists can now marquee overflowing labels, which DOGFOOD uses in its constrained navigation lanes

The release-proof story is still centered on DOGFOOD, but the harness and the docs surface are both less accidental than they were in 4.4.x.

Runtime And Filesystem Hardening

Bijou 5.0.0 also closes two high-value hardening gaps:

- fatal crash mode now exits automatically after rendering the crash surface when stdin is not a TTY, so CI and scripted hosts no longer hang waiting for an Enter key they can never send

- scopedNodeIO() now resolves symlinked prefixes before enforcing the root boundary, rejecting reads and writes that tunnel outside the declared root through symlinked files or directories

The workspace also pins hono to 4.12.14 through root overrides, closing the MCP server's transitive exposure to GHSA-458j-xx4x-4375.

Compatibility

Bijou 5.0.0 is a major release because it resets the hosted/app-shell line around clearer ownership and release-proof contracts, not because everyday apps need a wholesale rewrite. Most 4.4.x code will continue to run, but framed apps and Node hosts now have better default paths than the lower-level wiring that earlier releases required.

280

What's New in Bijou 7.0.0

docs/releases/7.0.0/whats-new.md

Bijou 7.0.0 ships the V6 layout floor and the V7 Product Truth lane as one release boundary. The short version: frame geometry, standard Blocks, DOGFOOD docs truth, localization posture, release proof, and notification polish now live on firmer public contracts instead of scattered demo-specific wiring.

Product Truth Is Block-Owned

DOGFOOD now routes its release-facing docs surface through named Block contracts. The docs workspace, navigation, documentation article, guide inspector, settings menu, BlockLab workbench, title screen, footer hints, frame search, notifications, performance HUD, keyboard help, command palette, and surface inventory all have inspectable semantic contracts.

That matters because product behavior is no longer only visible as pixels. A reviewer or agent can inspect metadata, schema-bound inputs, command intents, lower-mode facts, and rendered output for the same surface.

The Standard Block Catalog Is Broader

The release expands first-party Blocks across the DOGFOOD component families:

- grouping, explainability, document, destination, divider, and text-entry Blocks
- single-choice, multiple-choice, binary-decision, peer-navigation, progressive-disclosure, and path-progress Blocks
- brand-emphasis, mode-aware primitive, dense comparison, hierarchy, exploration-list, and temporal-dependency Blocks
- status and feedback Blocks for inline status, in-flow status, transient overlays, activity streams, shortcut cues, and progress indicators

Each family lands with public metadata, schema adapters, command-intent posture, visual/static rendering, lower-mode output, and stable semantic facts.

Layout, Tables, And Selection Got Stricter

The V6 floor brings pure layout envelope primitives to @flyingrobots/bijou: immutable constraints, preferences, assigned rectangles, direction, fit policy, content measurement seams, parent-owned assignment, minimal stack/place resolution, and explanation facts.

Table rendering also moves to a more honest bounded-width model:

- table() fits human-mode output to the available terminal width by default
- tableSurface() matches that bounded-width posture for surface rendering
- wrapped cells preserve word boundaries when possible
- intrinsic layout remains available as an explicit opt-in
- pipe lowerings now cover TSV, CSV, Markdown, and ASCII grid

Selection and copy are now pure retained-geometry primitives: selection owners, viewport-aware ranges, semantic prose/surface/table extraction, mixed-region ordering, blocker arbitration, terminal-native fallback, and clipboard-effect records without hidden OS clipboard side effects.

Localization And DOGFOOD Are More Honest

@flyingrobots/bijou-i18n now exposes a structured localization port with immutable localized objects, fallback and missing metadata, issue facts, and explicit resource/data behavior. DOGFOOD routes ordinary docs text lookup through that port, persists locale preference through host-owned storage, and keeps catalog completeness under local and CI gates.

The DOGFOOD app also gained a release identity and release-title proof surface for V7 Product Truth, plus clearer package, philosophy, architecture, doctrine, guide, and release documentation inside the terminal docs reader.

Runtime And Workflow Proof Tightened

This release hardens operator-facing surfaces:

- command backpressure diagnostics and async render middleware diagnostics are exposed in @flyingrobots/bijou-tui
- DOGFOOD and docs smoke paths have tighter non-interactive guardrails
- CI can skip expensive DOGFOOD-only gates for unrelated pull-request paths while still running the full proof path on main and tags
- GitHub comment workflow docs now require file-backed Markdown comments so commands, backticks, and tables survive review automation

Notification Text Wraps Like Human Prose

The live @flyingrobots/bijou-tui notification stack now wraps title, body, and action rows at word boundaries when possible instead of splitting ordinary words in the overlay. Long unbroken words still hard-wrap so notifications cannot exceed their assigned width.

This is the fix Jedit needs for normal-mode runtime feedback: the notification surface stays readable without giving up bounded terminal layout.

Compatibility

Bijou 7.0.0 is a major release because it publishes a broader product-truth surface and tighter runtime contracts. Apps using the documented public APIs should see the upgrade mostly as new capability, but maintainers should read the migration guide before bumping because table fitting, Block exports, and release-proof DOGFOOD behavior are intentionally stricter than the 5.0.0 line.

281

What's New in Bijou 7.1.0

docs/releases/7.1.0/whats-new.md

Bijou 7.1.0 is a focused post-V7 minor release. It does not try to become the full V8 Runtime Graph product. Instead, it makes the first portable UI proof chain inspectable, adds a real DOGFOOD GraphQL block fixture, tightens theme and terminal-rendering behavior, and gives release prep better executable guardrails.

Runtime Graph And Scene IR Proofs

The core package now exposes the first ui-scene-ir/1 runtime contract for portable Bijou UI proof. The seed includes deterministic scene hashing, structural receipts, node/action/binding/token/i18n validation, visible-cell source-map facts, terminal lowering receipts, and lower modes for inspecting node, token, and localization state.

This is the first committed step toward a product-grade Runtime Graph lane: a Bijou surface can be described semantically, lowered deterministically, and verified without treating screenshots as the only evidence.

GraphQL-Authored Blocks

Bijou now has a constrained GraphQL authoring path for block artifacts:

```
GraphQL SDL
  -> bijou-block/1
    -> ui-scene-ir/1
      -> terminal Surface proof
      -> graphql-bijou-block-debug/1 facts
```

compileGraphqlBijouBlock() and lowerBijouBlockToUiScene() turn SDL into a stable bijou-block/1 artifact and a portable scene. Group directives, grouped field membership, semantic source anchors, token refs, i18n fallback facts, actions, bindings, lower modes, and debug summaries are all part of the proof surface.

The release also checks in a real DOGFOOD NavigationListBlock GraphQL fixture, so the proof path is no longer only synthetic.

Theme Tokens, Modes, And Inspector Work

The design-token lane is more concrete:

- defineTheme() creates immutable dark/light theme definitions.
- tokenRef() and resolveThemeColorRef() preserve semantic color facts.
- BIJOU_DARK and BIJOU_LIGHT are first-party presets.
- defineThemeSafePairs() records reusable contrast pair matrices.
- DOGFOOD uses a high-contrast shell theme family with explicit dark and light modes.
- The Theme Lab page and F10 Theme Inspector expose token swatches, color reuse diagnostics, and safe-pair diagnostics.

These pieces make theme behavior easier to inspect and harder to fake with one-off color strings.

Raster-To-Glyph And Title Surface Polish

The TUI package now includes deterministic raw-RGBA-to-Surface rendering with ASCII, Braille, and quad modes. DOGFOOD uses that pipeline for release-title art, the V7 Launch Wake title surface, and image-to-glyph side-app previews.

The side app opened by npm run img or npm run image-viewer can preview SVG, PNG, and PPM/PNM files, pan and zoom the view, switch glyph modes, preserve sampled colors, tune Braille contrast, and enable deterministic ordered dithering.

DOGFOOD And Terminal Fixes

Several release-review fixes landed in the same release window:

- terminal resize and display-resume repaint now invalidates the front buffer so blank cells repaint immediately after geometry changes
- DOGFOOD Blocks navigation lets Up move from the first Block Preview child back to the parent row
- ANSI foreground/background resets now honor scoped 39 and 49 resets without leaking colors into following labels
- the quit confirmation modal no longer duplicates its controls
- the DOGFOOD title perf HUD stays on the title screen and reports current frame timing

Release Guardrails

The release process is stricter and more replayable:

- npm run release:readiness -- --milestone v7.1.0 prints a milestone report and blocks incomplete release prep before running the full local gauntlet

- DOGFOOD i18n debt discovery now scans examples/docs/**/*.ts modules by default, with explicit tooling-only exclusions
- release evidence packets are now required for versioned releases
- the roadmap, bearing, and release runbook now describe the forward-looking V7.1/V8/V9 release posture

The practical result is a release that is easier to inspect: users get new portable proof primitives and DOGFOOD fixture evidence, and maintainers get better release gates before tags are cut.

282

What's New in Bijou 7.2.0

docs/releases/7.2.0/whats-new.md

Bijou 7.2.0 is a stabilization release for the V7 demo and documentation surface. It keeps the public Runtime Graph direction intentionally narrow while tightening framework input behavior, DOGFOOD readability, Blocks documentation, release evidence, and dependency review.

The public tag was created from the verified 2026-07-05 main release commit. This document is the versioned release overview DOGFOOD and the repository front door use for 7.2.0.

Framework Input Stabilization

The TUI framework now has better coverage for page-scoped input fallthrough and scripted mouse sequences. Workspace mouse movement, release, wheel, and non-left press events can be replayed through deterministic driver helpers, and the public TUI package exposes the frame helper APIs needed by application code.

This is the practical fix for release-demo workflows that need reliable mouse inspection without relying on terminal-specific manual behavior.

DOGFOOD Readability And Release Story

DOGFOOD received another pass over the surfaces that made the 7.1 release-video rehearsal hard to inspect:

- locale fallback copy now distinguishes missing translations from intentional English source text
- light-theme screenshots and fixture coverage keep color contrast and shell state readable
- in-app release-story pages now point reviewers at the V7.2 evidence packet rather than asking them to infer the release state from scattered docs

The goal is not a new product surface. The goal is that the existing proof surface tells the truth when somebody records or reviews it.

Blocks Documentation

The Blocks app-binding guide now includes copyable snippets and deterministic fixtures for the current binding shape. The documentation remains scoped to the V7 proof path, but it gives readers enough structure to understand how bindings, actions, and DOGFOOD examples relate before the larger V8 work starts.

Theme Variant Coverage

First-party dark and light theme variants have explicit coverage in the release packet. The release keeps theme behavior focused on readability and replay evidence rather than opening a broader design-system expansion.

Release Evidence

The release gate now carries a versioned evidence packet alongside the package metadata bump. That packet records trackers, landed PRs, replay commands, security audit scope, registry verification, release-readiness posture, and residual risk.

Maintainers should treat the release evidence packet as the authoritative 7.2 release record: it includes the public GitHub Release, final-main validation, publish workflow, tag CI, Tag Guard, and npm registry verification evidence.

V8 Posture

Bijou 7.2.0 intentionally stops short of the next Runtime Graph product. The forward path remains:

```
GraphQL Blocks -> Bijou Blocks IR -> Geordi render targets -> TUI | Browser | Native UI
```

That work belongs to V8 and later. V7.2 makes the current release surface clean enough to move on without carrying avoidable release debt forward.

283

What's New in Bijou 8.0.0-rc.1

docs/releases/8.0.0/whats-new.md

A release candidate, published so the breaking changes below can be validated against real applications before 8.0.0 is cut. Pin the exact version; the dist-tag is next, not latest.

The headline is a deletion

detectOutputMode() no longer looks at NO_COLOR.

That variable means one thing: do not send me colour. It has never meant "do not draw a user interface". Bijou read it as a capability signal, so a developer who had NO_COLOR=1 in their shell profile did not get a monochrome TUI — they got no TUI. Every component dropped to its single-frame form, and the quit confirmation stopped asking.

The tell was that two first-party consumers had each independently written their own detector to route around it. git-cas ships a detectCliTuiMode with a comment explaining the problem; muniment wrote the same override for the same reason. When consumers reimplement your detector, the detector is answering a question they did not ask.

Colour is unaffected — it was never coupled to the mode in the implementation, only in the detector. NO_COLOR is read independently in three places, none of which consult ctx.mode, so with it set you still get theme.noColor === true and unstyled output. What you also get now is a working TUI.

The test for this is the shape worth stealing. Rather than assert which mode NO_COLOR produces, the property asserts that adding NO_COLOR to *any* environment leaves that environment's mode unchanged — the contract stated directly, over 200 generated combinations of value, TTY state, CI, and TERM.

An unknown status key no longer looks cancelled

ctx.status('typo') used to fall back to status.muted, and status.muted carries strikethrough in every shipped preset. So a mistyped key — or a key an application meant to define and didn't — rendered text with a line through it. The reader sees "cancelled". The application never said that.

It now falls back to semantic.muted, which is dim and nothing else, matching what the sibling ui() accessor already did with semantic.primary.

This one came out of a real application shipping the bug: a painter was declared, its status key was later dropped from the theme as a duplicate, and the painter kept resolving to the fallback. Nothing caught it — not the type checker, not a palette audit, not review — because no view had called it yet. #521 tracks typing the accessors so that becomes a compile error.

extendTheme() can finally reach every group

Theme has six token groups. extendTheme() accepted extensions for four, which sounds like an oversight and behaves like a trap, because the shipped presets alias one colour across the groups you could not reach.

bijou-dark paints status.warning, status.active, semantic.warning, semantic.accent, ui.cursor, ui.focusGutter, ui.sectionHeader, ui.logo, border.secondary and border.warning in a single amber. Ten tokens. So extendTheme(BIJOU_DARK, { status }) gave you your statuses and left that amber running every heading, cursor and border — and if one of your new statuses was also amber-ish, your headings became indistinguishable from your warnings. That is how this was found, at ΔE 0.023 in OKLab.

border and semantic are now reachable. They are overridable rather than extensible, since Theme fixes their key sets.

The alias itself is still there. extendTheme() makes it escapable, not fixed — see #519, which also records that error and success collapse to ΔE 0.059 under protanopia in the dark preset and 0.064 in the light one, and that bijou-light's severity ladder is inverted in lightness. If you rely on the shipped presets to distinguish severities, read that issue.

Also in this candidate

Everything that accumulated on main since 7.2.0:

- **Profunctor Page terminal inspection** — lowerProfunctorPageArtifacts() validates the canonical profunctor-page/0 family and emits deterministic scene IR, surfaces, target maps, receipts, cell source maps, and text evidence across six inspection modes, with unsupported browser semantics kept as explicit capability residuals.
- **VISOR artifact bundle proof** — createVisorArtifactBundleFromGraphql(), createVisorArtifactBundle(), and the visor-artifact-bundle/1 types.
- **RE-036 packed-cell receipts** — adaptPackedBijouCellsToSurface() copies validated bytes and side-table entries without re-encoding cells.
- **Dependency-security closeout** — patched floors across the resolved graph, with a regression binding them and a clean npm ci reproducing a zero-vulnerability audit.
- **Code Dojo ratchet progress** — tranches A through E, plus the DX-050 prerequisites, DOGFOOD localization debt, and documentation-contract work.

What this candidate is for

Two behaviour changes that are correct in principle and unproven in the field. If NO_COLOR or a custom theme matters to your application, this is the version to try before 8.0.0 exists.

Read the migration guide for the specific things to check, and the release evidence packet — which is explicit about which evidence was replayed for this candidate and which was inherited, because nine of eleven release-law review gates are still unreviewed. That is exactly why this is an rc and not a release.

Retrospectives and historical planning

Learning from the work

A design record describes a change before or during implementation. A retrospective gives the work another opportunity to explain itself after encountering code, tests, review and users. It can expose an incorrect assumption, a boundary that held, a check that was insufficient or a useful simplification that was difficult to see at the start.

This part preserves those accounts alongside historical planning and abandoned work. The original paths identify their former roles. A retrospective, a graveyard entry and an old planning note answer different questions even when they concern the same feature.

Compare intention with outcome

Read a retrospective with its linked design and integration evidence. Look for the difference between what was expected and what was observed. Which part of the original problem was solved? Which tradeoff was accepted? Did an unexpected failure change the design, or merely change the way it was checked?

An account is particularly useful when it names a concrete failure mode and the evidence that revealed it. Broad lessons become easier to reuse when a later reader can see the constraint, the attempted solution and the observed result together. A claim that a cycle was complete should retain the limits of its recorded checks.

Historical planning remains historical

These documents were written under practices that changed over the project's life. Their cycle names, sponsor fields, lanes and ceremony record that context. They are preserved as evidence about the work, with current execution tracked outside the book.

An abandoned proposal may still contain a useful diagnosis. A successful change may still leave a debt or a later regression. Use this part to recover that knowledge and test its relevance to a new situation. Its purpose is to make the experience available for judgment, without treating an old plan as an automatically active work item.

284

CI-001 — mermaidSurface() Component

docs/method/graveyard/legacy-backlog/cool-ideas/CI-001-mermaid-surface-component.md

Legend: DF — DOGFOOD Field Guide

Idea

Since Bijou uses Mermaid for its documentation, wouldn't it be cool to render Mermaid diagrams directly in the TUI? This component would parse a (limited) subset of Mermaid (flowcharts and state diagrams) and render them to a Surfac e using Bijou's box-drawing and graph primitives.

Why

1. **Self-Documentation**: A TUI app can render its own architecture or cycle state without external tools.
2. **Data-Viz Maturity**: It pushes the "geometric lawfulness" of the terminal by showing that complex, logic-heavy graphs are native to the medium.
3. **Docs Synergy**: If DOGFOOD could render diagrams locally, it would complete the "docs are the demo" story.

Effort

Large — requires a simplified Mermaid parser and a spatial-layout engine to map diagrams to terminal cells.

285

CI-002 — Deterministic Time-Travel Debugger

docs/method/graveyard/legacy-backlog/cool-ideas/CI-002-deterministic-time-travel-debugger.md

Legend: DX — Developer Experience

Idea

The TUI runtime loop in @flyingrobots/bijou-tui is deterministic. It takes a message, a model, and returns a new model and commands. This means we can record every Msg that flows through the system.

Add a "Replay" mode to the runtime and the createFramedApp() shell. When enabled, it records all inputs and allows the user (or an agent) to pause the app, scrub backwards through frames, and step forward one message at a time. The current model and view output can be inspected for every step.

Why

1. **Bug Repro**: A user can send a "repro log" that a builder can load and scrub through.
2. **Determinism Proof**: It proves the "Runtime Truth Wins" invariant by showing that the app's state is a pure function of its history.
3. **Agent Synergy**: AI agents can "debug" a TUI by replaying the messages that led to an error state.

Effort

Medium-Large — requires an event-logger and a way to re-drive the update() function from a log.

286

CI-003 — Surface Shaders (CRT / Scanlines)

docs/method/graveyard/legacy-backlog/cool-ideas/CI-003-surface-shaders.md

Legend: RE — Runtime Engine

Idea

Add a "Post-Process" middleware to @flyingrobots/bijou-tui's render pipeline that operates directly on the final byte-buffer before it is diffed and emitted. This middleware could simulate:

- CRT scanlines (alternating row brightness/dimming)
- Scanline flicker (subtle per-frame variation)
- Signal static / noise (random cell or byte corruption)
- "Vignette" effects (darkening edges of the terminal)

Why

1. **Expressivity**: It pushes the "high-fidelity" aesthetic to its limit, giving Bijou apps a distinct feel that is both professional and nostalgic.
2. **Low-Allocation Performance**: Since the pipeline is byte-packed, these effects can be implemented using zero-allocation loops on typed arrays.
3. **Product Differentiation**: It shows that the terminal can be an expressive visual medium, not just a static character grid.

Effort

Medium — requires adding the middleware and a set of byte-manipulation shaders for common effects.

287

CI-003 — Surface Shaders (CRT / Scanlines)

docs/method/retro/CI-003-surface-shaders.md

title: CI-003 — Surface Shaders (CRT / Scanlines)
lane: v5.0.0

Legend: RE — Runtime Engine

Idea

Add a "Post-Process" middleware to @flyingrobots/bijou-tui's render pipeline that operates directly on the final byte-buffer before it is diffed and emitted. This middleware could simulate:

- CRT scanlines (alternating row brightness/dimming)
- Scanline flicker (subtle per-frame variation)
- Signal static / noise (random cell or byte corruption)
- "Vignette" effects (darkening edges of the terminal)

Why

1. **Expressivity**: It pushes the "high-fidelity" aesthetic to its limit, giving Bijou apps a distinct feel that is both professional and nostalgic.
2. **Low-Allocation Performance**: Since the pipeline is byte-packed, these effects can be implemented using zero-allocation loops on typed arrays.
3. **Product Differentiation**: It shows that the terminal can be an expressive visual medium, not just a static character grid.

Effort

Medium — requires adding the middleware and a set of byte-manipulation shaders for common effects.

Completed (2026-04-14)

The post-process path now ships as a reusable surfaceShaderFilter(...) middleware for the render pipeline’s PostProcess stage. Bijou TUI now exports built-in packed-surface shader passes for scanlines(), flicker(), noise(), and vignette(), along with regression coverage and a package-level configureP ipeline() example. Apps can now compose CRT-style or atmosphere-oriented post-process effects without forking the runtime or inventing a second render pipeline.

288

DAG — Layer-Skipping Edges Silently Occluded By Intermediate Nodes

docs/method/retro/DAG-layer-skipping-edges-occluded.md

title: DAG — Layer-Skipping Edges Silently Occluded By Intermediate Nodes
lane: retro

Disposition

Fixed on release/v4.5.0 in commit 69291fa. dag-edges.ts now routes same-column skip edges through a detour column beside intermediate nodes instead of drawing straight through the occluding node column, GridState.arrows now tracks inbound multiplicity without collapsing routes into a boolean set, and the regression coverage in dag-edges.test.ts / dag.test.ts locks the visual invariant down.

Original Proposal

Legend: RE — Runtime Engine

Severity

Correctness bug, not cosmetic. Downstream projects consume the DAG renderer to visualize graph-shaped data — notably git-warp, a graph-database project that renders its graphs through bijou. For those consumers, the DAG renderer must **never hide or drop information**. A missing edge is a wrong answer, not an ugly one.

Problem

When a DAG edge skips a layer (source at layer L, destination at layer L+2 or beyond) and both endpoints end up in the same column, the vertical edge line

is routed straight down through the intermediate layer — passing through the rows where the intermediate layer's node box is drawn. The node box has higher render priority than the edge line, so the edge is visually erased.

The destination arrowhead survives because it lands in the gap row above the destination box. To a reader, it looks like the edge originates from the node directly above the destination, not from the real source.

Multiple edges terminating at the same destination cell also collide: arrows is a Set<number> keyed by encodeArrowPos(row, col), so only one arrowhead is drawn per destination even when the grid has two or more inbound edges.

Evidence

Minimal repro (three nodes, edges A→B, A→C, B→C):

```
bijou_dag({ nodes: [
  { id: 'A', label: 'A', edges: ['B', 'C'] },
  { id: 'B', label: 'B', edges: ['C'] },
  { id: 'C', label: 'C', edges: [] },
]});
```

Expected: two arrows into C (one from A, one from B). Actual: one arrow into C. The A→C edge is silently lost.

The loss was discovered while rendering the RE-017 byte-pipeline plan DAG — I-9 → II-1 and III-5 → III-6 (both layer-skipping, same-column) were invisible in the rendered output.

Root Cause

- packages/bijou/src/core/components/dag-edges.ts:148-149 — markEdge routes a same-column edge as a pure vertical segment from sRow to dRow, passing through intermediate layer rows.
- packages/bijou/src/core/components/dag-render.ts:383-420 — cellAt resolves nodes before edges, so any grid cell inside a node box returns the box character, occluding the edge line underneath.
- packages/bijou/src/core/components/dag-edges.ts:162 — g.arrows.add(encodeArrowPos(dRow, dstC)) collides for any two edges landing on the same destination row/col, discarding all but one arrowhead.

Fix Options

1. **Detour around intermediate nodes.** When a layer-skipping edge would pass through a same-column intermediate node, route it through a jog column that's free of nodes at the intermediate layer. Biggest change; most visually correct.
2. **Column reassignment for skip edges.** When ordering columns, prefer to place layer-skipping edge endpoints in different columns so the straight-line route doesn't collide.

3. **Edge priority when no box char.** Keep the node-box priority for cells inside boxes, but detect pass-through and mark those cells with a "node+edge" flag so the box label area stays readable while still hinting that an edge crosses the box (e.g. with a subdued glyph at the box border).
4. **Arrowhead multiplicity.** Change arrows from Set<number> to Map<number, numbe r> (count per cell) or allow the cell renderer to emit a divergent glyph when multiple edges converge.

Option 1 is the most correct. Option 2 is probably the simplest and resolves the common case. At minimum, option 4 should ship so convergence isn't silently lost.

Tests To Add

- Three-node repro above, asserting renderAccessible shows A → B, C and B → C (source of truth) and that the rendered output contains two arrowheads entering C's row range.
- Four-node diamond (A → B, A → C, B → D, C → D) — asserting both inbound edges to D are visible.
- A layer-skipping chain (A → B → C, plus A → D with D at layer 2) asserting A→D does not overlap B's column.

Impact

The pipe and accessible renderers (renderPipe, renderAccessible in dag-render .ts:469-530) return the correct edge data, so any programmatic consumer is unaffected. Only the interactive (styled) renderer loses edges — which is exactly the mode used for all human-facing output via the bijou MCP tool, the CLI, the bijou-tui panes, and the git-warp graph-database visualizations.

Invariant To Enforce

> The interactive DAG renderer MUST NOT hide, drop, or silently merge edges or arrowheads. Every edge in the input must be resolvable in the rendered output.

This should become a repo invariant (docs/invariants/) alongside a regression test that fails if any edge in a representative test graph is not visually present.

289

DF-002 — Expand DOGFOOD Component Family Coverage

docs/method/graveyard/legacy-backlog/DF-002-expand-dogfood-component-family-coverage.md

Legend: DF — DOGFOOD Field Guide

Idea

DF-001 makes DOGFOOD honest about current coverage.

The next obvious move is to improve the number itself by expanding stories against the canonical component-family reference in a deliberate sequence rather than opportunistically.

Why

Once DOGFOOD shows explicit coverage progress, missing families become a visible product-quality backlog instead of hidden debt.

This backlog item exists to track that content-expansion work separately from shell, design-language, and localization work.

Status

Backlog spawned by the retrospective for DF-001.

METHOD Note

This file is kept as historical backlog lineage.

It is not the current DOGFOOD queue. The live follow-on now is DF-020.

290

DF-020 — Deepen DOGFOOD Story Depth and Variant Quality

docs/method/graveyard/legacy-backlog/v4.5.0/DF-020-deepen-dogfood-story-depth-and-variant-quality.md

Legend: DF — DOGFOOD Field Guide

Idea

DOGFOOD now covers every current component family in the canonical reference.

The next stage is no longer breadth. It is depth:

- sharper variants for the families that currently have only one narrow teaching path
- stronger source examples for shell-heavy and authoring-heavy stories
- richer comparison between closely related families

Why

The coverage ratchet is complete at the family level.

The remaining DOGFOOD work should improve:

- story richness
- source-example quality
- guidance clarity
- variant completeness

Status

Backlog spawned by the retrospective for DF-019.

291

DF-020 — Deepen DOGFOOD Story Depth and Variant Quality

docs/method/retro/DF-020-deepen-dogfood-story-depth-and-variant-quality.md

```
title: "DF-020 — Deepen DOGFOOD Story Depth and Variant Quality"
legend: DF
lane: retro
```

Completed on release/v5.0.0. DOGFOOD moved beyond raw family-count coverage and into richer docs depth: the repo now treats DOGFOOD as the human-facing docs system, ships broader guide/package/philosophy publishing inside that surface, and records in the earlier docs/method/retro/DF_002-expand-dogfood-component-family-coverage.md lineage that this direct follow-on already shipped. This METHOD note closes the last legacy-only copy of that follow-on.

Original Proposal

Legend: DF — DOGFOOD Field Guide

Idea

DOGFOOD now covers every current component family in the canonical reference.

The next stage is no longer breadth. It is depth:

- sharper variants for the families that currently have only one narrow teaching path
- stronger source examples for shell-heavy and authoring-heavy stories
- richer comparison between closely related families

Why

The coverage ratchet is complete at the family level.

The remaining DOGFOOD work should improve:

- story richness

- source-example quality
- guidance clarity
- variant completeness

Status

Backlog spawned by the retrospective for DF-019.

292

DF-002 — Expand DOGFOOD Component Family Coverage

docs/method/retro/DF_002-expand-dogfood-component-family-coverage.md

```
title: DF-002 — Expand DOGFOOD Component Family Coverage
lane: retro
legend: DF
```

Disposition

The component-family coverage expansion arc is complete. DOGFOOD reached 100% canonical family coverage in the DF-014 through DF-019 cycle set, and the direct follow-on DF-020 also shipped in v4.5.0. This METHOD backlog note is now historical lineage rather than live queue work.

Original Proposal

Legend: DF — DOGFOOD Field Guide

Idea

DF-001 makes DOGFOOD honest about current coverage.

The next obvious move is to improve the number itself by expanding stories against the canonical component-family reference in a deliberate sequence rather than opportunistically.

Why

Once DOGFOOD shows explicit coverage progress, missing families become a visible product-quality backlog instead of hidden debt.

This backlog item exists to track that content-expansion work separately from shell, design-language, and localization work.

Status

Backlog spawned by the retrospective for DF-001.

METHOD Note

This file is kept as historical backlog lineage.

It is not the current DOGFOOD queue. The direct follow-on DF-020 already shipped in v4.5.0.

293

MCP interactive documentation for all Bijou components

docs/method/retro/DF_mcp-interactive-documentation-for-all-bijou-components.md

title: MCP interactive documentation for all Bijou components
legend: DF
lane: v5.0.0

bijou-mcp should serve as interactive documentation for all of Bijou, not just a rendering service. Add a bijou_docs tool (or MCP resource endpoints) that returns structured, machine-readable documentation for any component: what it does, when to use it, when NOT to use it, interaction profile behavior (interactive/static/pipe/accessible), related components, and example output. Think of it as the design-system field guide but queryable by LLMs — an agent could ask "when should I use a table vs a DAG?" or "what component handles step progress?" and get an authoritative answer with examples. This makes Bijou self-documenting inside any MCP-capable coding tool.

Progress (2026-04-12)

bijou_docs now exists and already covers the current bijou-mcp render-tool surface. It also documents a first bounded set of public first-party component families that do not yet have dedicated MCP render tools, including markdown(), guidedFlow(), preferenceListSurface(), the spinner/timer family, and the current data-viz surfaces such as sparkline() and perfOverlaySurface().

The remaining gap is breadth, not direction: the catalog still is not a complete field guide for every first-party component or every DOGFOOD surface, and the docs-only families still stop short of a one-tool-per-component interactive render story.

Completed (2026-04-14)

bijou_docs now covers the public first-party Bijou component-family surface exposed by @flyingrobots/bijou, including the previously missing note(), the

core choice/form families, expressive branding helpers, and the renderByMode( ) authoring helper. The catalog still prefers real MCP render tools when they exist, and it now synthesizes representative plain-text examples for the remaining docs-only families.

294

DL-008 — Promote Guided Flow Block

docs/method/graveyard/legacy-backlog/DL-008-promote-guided-flow-block.md

Legend: DL — Design Language

Idea

DL-007 promotes the inspector family into a real TUI block helper through ins pectorDrawer().

The next natural follow-on is to do the same kind of promotion work for the guided-flow side of the design language.

That likely means deciding whether explainability() is still just a component seam, or whether Bijou is ready for a higher-level guided-flow block that standardizes:

- section rhythm
- next-action treatment
- shell/workflow ownership
- calmer multi-step assistance patterns

Why

The design-system blocks guide treats Guided flow as one of the first credible block candidates.

Inspector promotion should not strand guided-flow work one layer lower forever.

Status

Backlog spawned by the retrospective for DL-007.

295

DL-009 — Formalize Layout and Viewport Rules

docs/method/graveyard/legacy-backlog/v5.0.0/DL-009-formalize-layout-and-viewport-rules.md

Legend: DL — Design Language

Idea

Bijou has accumulated strong local layout instincts, but the rules are still implicit and scattered:

- how children fill or center within a parent rect
- when parents vs children own whitespace
- how panes stretch and shrink on terminal resize
- when overflow should wrap, clip, or move into a viewport
- how viewport ownership should align keyboard focus, mouse hit-testing, and scrollbars

This cycle should turn those instincts into one formal layout-language pass instead of leaving them as bug fixes and scattered notes.

Why

Recent DOGFOOD issues exposed that layout bugs are often really ownership bugs:

- a pane can render as if it has one height while navigation thinks it has another
- a child surface can leave accidental slack because no rule says whether it should fill or be placed
- overflow can look scrollable without actually behaving like a viewport

If Bijou is going to be a serious TUI framework, layout rules need to be a first-class part of the design language.

Likely scope

- capture canonical resize/stretch/shrink rules

- define parent-owned anchoring and whitespace rules
- define viewport as the canonical overflow ownership seam
- identify which existing components should adopt the formal rules first

296

DL-009 — Formalize Layout and Viewport Rules

docs/method/retro/DL-009-formalize-layout-and-viewport-rules.md

```
title: DL-009 — Formalize Layout and Viewport Rules
lane: retro
legend: DL
```

Disposition

Implemented on release/v5.0.0 as explicit doctrine. The new docs/strategy/layout-and-viewport-rules.md note now captures the canonical ownership rules for whitespace, fill versus placement, resize semantics, overflow, viewport ownership, and scrollbar truth. Those rules are also wired into the architecture signposts so they are part of the normal reading path instead of trapped in backlog prose.

Original Proposal

Legend: DL — Design Language

Idea

Bijou has accumulated strong local layout instincts, but the rules are still implicit and scattered:

- how children fill or center within a parent rect
- when parents vs children own whitespace
- how panes stretch and shrink on terminal resize
- when overflow should wrap, clip, or move into a viewport
- how viewport ownership should align keyboard focus, mouse hit-testing, and scrollbars

This cycle should turn those instincts into one formal layout-language pass instead of leaving them as bug fixes and scattered notes.

297

DL-009 — Strictly Typed App Interface Examples

docs/method/graveyard/DL-009-strictly-typed-app-examples.md

```
title: "DL-009 — Strictly Typed App Interface Examples"
legend: DL
lane: graveyard
```

Disposition

Superseded by DL-010 — Strictly Typed App Interface Examples. The work shipped, but this legacy pre-METHOD note used the DL-009 identifier before that id was later reused for DL-009 — Formalize Layout and Viewport Rules. Keep this file only as name/lineage preservation for the older backlog tree.

Original Proposal

Legend: DL — Design Language

Idea

Many code examples in the repository's READMEs and GUIDEs currently use App<a ny, any> or Model = any. While concise, this masks the core TypeScript safety guarantees that make Bijou an "industrial-grade" engine.

Refactor all top-level documentation examples to use concrete Model and Msg types. Ensure that the init, update, and view functions are shown with their full generic context.

Why

1. **Safety**: Demonstrates the TEA loop's state-machine guarantees immediately to new users.
2. **Clarity**: Makes it clear how messages and models flow through the update cycle.

3. **Consistency**: Aligns the documentation with the “Tests are the Spec” and “Runtime Truth Wins” invariants.

Effort

Small — surgical updates to README.md and GUIDE.md files.

298

DL-009 — Strictly Typed App Interface Examples

docs/method/graveyard/legacy-backlog/cool-ideas/DL-009-strictly-typed-app-examples.md

Legend: DL — Design Language

Idea

Many code examples in the repository's READMEs and GUIDEs currently use App<a ny, any> or Model = any. While concise, this masks the core TypeScript safety guarantees that make Bijou an "industrial-grade" engine.

Refactor all top-level documentation examples to use concrete Model and Msg types. Ensure that the init, update, and view functions are shown with their full generic context.

Why

1. **Safety**: Demonstrates the TEA loop's state-machine guarantees immediately to new users.
2. **Clarity**: Makes it clear how messages and models flow through the update cycle.
3. **Consistency**: Aligns the documentation with the "Tests are the Spec" and "Runtime Truth Wins" invariants.

Effort

Small — surgical updates to README.md and GUIDE.md files.

299

DL-010 — Strictly Typed App Interface Examples

docs/method/retro/DL-010-strictly-typed-app-examples.md

title: DL-010 — Strictly Typed App Interface Examples
lane: retro
legend: DL

Disposition

Completed on release/v5.0.0. The remaining public doc example that still used App<any, ...> in packages/bijou-tui/README.md is now typed with an explicit P arentModel, and a repo search over root/package READMEs plus GUIDE surfaces no longer finds App<any, any>, App<any, ...>, or Model = any examples in the top-level documentation path.

Original Proposal

Legend: DL — Design Language

Many code examples in the repository's READMEs and GUIDEs currently use App<a ny, any> or Model = any. While concise, this masks the core TypeScript safety guarantees that make Bijou an "industrial-grade" engine.

Refactor all top-level documentation examples to use concrete Model and Msg types. Ensure that the init, update, and view functions are shown with their full generic context.

300

DL-008 — Promote Guided Flow Block

docs/method/retro/DL_008-promote-guided-flow-block.md

```
title: DL-008 — Promote Guided Flow Block
lane: retro
legend: DL
```

Disposition

Implemented on release/v4.5.0 by adding the reusable guidedFlow() block in @flyingrobots/bijou, refactoring explainability() onto the same guided-flow rhythm, proving the block in DOGFOOD, and capturing the work in the DL-008 cycle doc. This is now completed design-language work, not live backlog.

Original Proposal

Legend: DL — Design Language

Idea

DL-007 promotes the inspector family into a real TUI block helper through ins pectorDrawer().

The next natural follow-on is to do the same kind of promotion work for the guided-flow side of the design language.

That likely means deciding whether explainability() is still just a component seam, or whether Bijou is ready for a higher-level guided-flow block that standardizes:

- section rhythm
- next-action treatment
- shell/workflow ownership
- calmer multi-step assistance patterns

Why

The design-system blocks guide treats Guided flow as one of the first credible block candidates.

Inspector promotion should not strand guided-flow work one layer lower forever.

Status

Backlog spawned by the retrospective for DL-007.

301

DX-003 — Rationalize Table APIs and Public Table Types

docs/method/graveyard/legacy-backlog/DX-003-rationalize-table-apis-and-public-table-types.md

Legend: DX — Developer Experience

Idea

Make the table family feel like one coherent surface instead of a cluster of near-related APIs with mismatched conventions.

Why

The current table APIs create avoidable friction:

- navigableTableSurface(...) and tableSurface(...) use different calling conventions
- the rows types are confusing between plain strings and TableSurfaceCell
- the common case for “just show me a table” is more verbose than it should be

This is exactly the kind of thing users discover by compiler error instead of by intention.

Captured feedback

From warp-ttd TUI development:

- tableSurface vs navigableTableSurface calling conventions are inconsistent
- rows typing is confusing around string[][] vs TableSurfaceCell[][]
- TableSurfaceCell is not obvious from the public surface
- a simple table(columns, rows, ctx?) style shorthand would help the common case

Likely scope

- decide on consistent table-family calling conventions
- make public table row/cell typing obvious and documented
- consider a simpler high-frequency table helper for the 80% case
- add tests proving the table family feels consistent across plain and navigable flows

302

DX-004 — Smooth Surface and String Composition Seams

docs/method/graveyard/legacy-backlog/DX-004-smooth-surface-and-string-composition-seams.md

Legend: DX — Developer Experience

Idea

Reduce confusion at the boundary between string-oriented composition and Surface/layout-oriented composition.

Why

Bijou currently exposes a real conceptual seam here, but the ergonomics are easy to trip over:

- some helpers return Surface
- some stack/layout helpers still expect strings
- view() requires Surface | LayoutNode, not string
- migration from older string-first patterns is easy to misunderstand

The issue is not only documentation. Some composition helpers may need better bridging or clearer contracts.

Captured feedback

From warp-ttd TUI development:

- badge() returning Surface made vstack() composition awkward
- surfaceToString(...) was required in some places but forbidden in view()
- the view() return constraint is understandable but still easy to trip over during migration

Likely scope

- audit string-land vs surface-land composition boundaries
- decide where bridging helpers should exist and where they should not
- improve the composition story for small surface-returning primitives
- tighten docs or error guidance where the seam is intentionally strict

303

DX-005 — Polish Small Component and Import Ergonomics

docs/method/graveyard/legacy-backlog/DX-005-polish-small-component-and-import-ergonomics.md

Legend: DX — Developer Experience

Idea

Sweep up the recurring small API papercuts that make Bijou feel fussier than it needs to.

Why

None of these issues are individually existential, but together they create friction:

- box titles needing manual padding
- imports for one conceptual feature split across too many packages
- minor public-API rough edges that make common authoring flows feel less polished than the runtime really is

These are good backlog items precisely because users keep encountering them in real app work.

Captured feedback

From warp-ttd TUI development:

- boxSurface title padding required manual spaces
- canvas / ShaderFn / Surface / BijouContext required three imports across package boundaries for one shader function

Likely scope

- make boxed-title padding calmer by default or easier to opt into
- review re-export strategy for frequently paired shader/canvas types
- capture any other low-risk public API polish work that falls out while fixing the above

304

DX-006 — Debug Performance Overlay Component

docs/method/graveyard/legacy-backlog/cool-ideas/DX-006-debug-performance-overlay-component.md

Legend: DX — Developer Experience

Idea

Ship a first-class debugOverlay() component in @flyingrobots/bijou-tui that any app can composite over its view to see real-time performance instrumentation:

- FPS counter
- Frame time (with rolling graph)
- Update and view phase timing
- Heap used/total, RSS, external memory
- GC event count per sample window
- Terminal size and cell count
- Configurable position (corner anchoring)

Why

The examples/perf-gradient stress test proved the concept: a self-rendering stats overlay that tracks frame timing, memory, and GC pressure is immediately useful for diagnosing performance issues. But it's hardcoded into the demo. Every app that wants performance visibility has to rebuild this from scratch.

Shape

A composable surface component, not a shell/app-frame feature:

```
import { debugOverlay } from '@flyingrobots/bijou-tui';

view(model) {
  const main = renderMyApp(model);
  return debugOverlay(main, { anchor: 'top-left' });
}
```

The overlay composites onto the app's surface. It reads performance data from a shared timing context (similar to the mutable phaseTiming object in the perf demo). Apps opt in by wrapping their view — no framework integration required.

Open questions

- Should the render pipeline expose phase timing natively (layout, paint, diff, output) so the overlay can show the full breakdown? Currently the pipeline isn't observable from the app side.
- Should the overlay be tree-shakeable (dev-only) or always available?
- Should it support custom metrics (app-defined counters/gauges)?

Related

- RE-008 byte-packed surface representation (affects rendering perf)
- The perf gradient demo (examples/perf-gradient) as proof of concept

305

DX-007 — Decompose app-frame.ts Mega-Closure

docs/method/graveyard/legacy-backlog/bad-code/DX-007-decompose-app-frame-mega-closure.md

Legend: DX — Developer Experience

Problem

packages/bijou-tui/src/app-frame.ts is 2,679 lines with 9 responsibilities, 90+ internal functions, 38 imports, and a mega-closure that prevents unit testing any individual function.

Evidence

- **Modularity**: routing, rendering, state, command dispatch, settings, notifications, help, overlays, and transitions all live in one file
- **Coupling**: circular import with app-frame-types.ts (FrameModel)
- **Testability**: every internal function captures closure state; none can be imported or tested independently
- **Growth**: each RE cycle (001-007) added more logic without decomposing

Target

Decompose into focused domain modules:

1. frame-settings.ts — settings drawer state, focus, scroll, row activation, resolveSettingsLayout
2. frame-notifications.ts — notification center state, scroll, filter, resolveNotificationCenterLayout
3. frame-help.ts — help overlay rendering, scroll computation
4. frame-shell-commands.ts — shellCommandHandlers table, FrameShellCommand apply logic, drainShellCommandBuffer
5. Move FrameModel to app-frame-types.ts to break circular import

Each module independently importable and unit-testable with deps injection. Target: app-frame.ts under 800 lines.

Effort

Large — touches the highest-traffic file in the repo. Must be done incrementally with tests at each step.

306

DX-007 — Decompose app-frame.ts Mega-Closure

docs/method/retro/DX-007-decompose-app-frame-mega-closure.md

```
title: DX-007 — Decompose app-frame.ts Mega-Closure
lane: retro
legend: DX
```

Disposition

Completed on release/v4.5.0. The remaining high-value closure debt in app-frame.ts is now materially decomposed: FrameModel and FramePaneScroll live in app-frame-types.ts, the circular import between app-frame.ts and app-frame-types.ts is gone, and the shell-theme/help/ settings/notification helper cluster now lives in the independently importable app-frame-overlays.ts module with direct unit coverage in app-frame-overlays.test.ts. This work reduced app-frame.ts from 3,132 lines to 2,348 while preserving the earlier module splits across render, actions, palette, layers, and utils.

The original “under 800 lines” target was an early umbrella goal from before those prior extractions landed. Repo truth now lives in the decomposed module boundaries and testability, not that stale numeric threshold.

Original Proposal

Legend: DX — Developer Experience

Problem

packages/bijou-tui/src/app-frame.ts is 2,679 lines with 9 responsibilities, 90+ internal functions, 38 imports, and a mega-closure that prevents unit testing any individual function.

Evidence

- **Modularity**: routing, rendering, state, command dispatch, settings, notifications, help, overlays, and transitions all live in one file
- **Coupling**: circular import with app-frame-types.ts (FrameModel)
- **Testability**: every internal function captures closure state; none can be imported or tested independently
- **Growth**: each RE cycle (001-007) added more logic without decomposing

Target

Decompose into focused domain modules:

1. frame-settings.ts — settings drawer state, focus, scroll, row activation, resolveSettingsLayout
2. frame-notifications.ts — notification center state, scroll, filter, resolveNotificationCenterLayout
3. frame-help.ts — help overlay rendering, scroll computation
4. frame-shell-commands.ts — shellCommandHandlers table, FrameShellCommand apply logic, drainShellCommandBuffer
5. Move FrameModel to app-frame-types.ts to break circular import

Each module independently importable and unit-testable with deps injection. Target: app-frame.ts under 800 lines.

Effort

Large — touches the highest-traffic file in the repo. Must be done incrementally with tests at each step.

307

DX-007 — Project-Wide Numeric Input Sanitization

docs/method/graveyard/legacy-backlog/v4.5.0/DX-007-project-wide-numeric-input-sanitization.md

Legend: DX — Developer Experience

Idea

Audit all public API functions that accept numeric parameters and ensure they guard against NaN, Infinity, -Infinity, and fractional values where integers are expected.

The v4.4.0 data-viz review found several places where non-finite user input propagated silently into rendering logic. The fixes were targeted (sparkline, braille-chart, stats-panel), but the same class of bug likely exists in other components.

Approach

1. Grep for all exported functions with number parameters
2. Classify: which accept user-supplied values vs internal-only
3. Add Number.isFinite guards or Math.floor clamping at boundaries
4. Consider a shared sanitizeNumericInput(n, fallback) utility
5. Add edge-case tests for each affected function

Why

Three separate review passes (2 self-reviews + CodeRabbit) all found NaN/Infinity bugs. This is a systemic gap, not a one-off.

308

DX-007 — Project-Wide Numeric Input Sanitization

docs/method/retro/DX-007-project-wide-numeric-input-sanitization.md

```
title: "DX-007 — Project-Wide Numeric Input Sanitization"
legend: DX
lane: retro
```

Completed on release/v5.0.0. Bijou now has shared numeric boundary helpers in packages/bijou/src/core/numeric.ts, broad adoption across core, TUI, and Node adapters, and dedicated tests in packages/bijou/src/core/numeric.test.ts. What was originally an audit reminder is now a reusable boundary rule rather than repeated ad hoc guards.

Original Proposal

Legend: DX — Developer Experience

Idea

Audit all public API functions that accept numeric parameters and ensure they guard against NaN, Infinity, -Infinity, and fractional values where integers are expected.

The v4.4.0 data-viz review found several places where non-finite user input propagated silently into rendering logic. The fixes were targeted (sparkline, braille-chart, stats-panel), but the same class of bug likely exists in other components.

Approach

1. Grep for all exported functions with number parameters
2. Classify: which accept user-supplied values vs internal-only
3. Add Number.isFinite guards or Math.floor clamping at boundaries
4. Consider a shared sanitizeNumericInput(n, fallback) utility
5. Add edge-case tests for each affected function

Why

Three separate review passes (2 self-reviews + CodeRabbit) all found NaN/Infinity bugs. This is a systemic gap, not a one-off.

309

DX-008 — PerformanceObserver('gc') Is Unreliable For Our Measurement Needs

docs/method/retro/DX-008-performance-observer-gc-unreliable.md

title: DX-008 — `PerformanceObserver('gc')` Is Unreliable For Our Measurement Needs
lane: retro

Disposition

Retired on release/v4.5.0 in commit 69291fa. We are no longer treating PerformanceObserver('gc') as a trusted bench signal; the repo now ships a dedicated bench:gc-observer-repro diagnostic and documents the limitation explicitly in bench/README.md, so the remaining work is operational guidance rather than an open product/runtime bug.

Original Proposal

Legend: DX — Developer Experience

Problem

During the RE-017 audit (2026-04-09), I tried to use PerformanceObserver({ entryTypes: ['gc'] }) in the renderer bench to filter out samples where V8 GC fired during the timed loop. The idea was to set up the observer, run the sync measurement loop, call observer.takeRecords() afterwards, and count any captured GC events as "sample was affected, discard the heap delta."

The result: **zero GC events reported across 1,000 samples**, even for scenarios that clearly allocated 50+ MiB per sample and must have triggered multiple young-gen collections. Yet the heapUsedAfter - heapUsedBefore delta for those same samples was frequently negative (heap shrank by 90+ MiB), which is only possible if GC ran.

The observer silently failed to report events that the heap delta clearly said happened.

Reproducer

See the git history of scripts/renderer-bench.ts around the RE-017 audit (the file is scheduled for deletion; the fix attempt used observer.observe({ entryTypes: ['gc'] }) with either a no-op callback + takeRecords() or with a collecting callback via the list.getEntries() form). Environment: macOS 25.3 on Apple M1 Pro, Node v25.8.1, node --expose-gc.

Why This Matters

GC-event observability is the standard recommendation for distinguishing "this sample's heap delta is meaningful" from "this sample's heap delta is polluted by GC". If the API doesn't work reliably, we either:

1. Can't filter samples → heap metrics are bimodal and uninterpretable.
2. Have to use an alternative mechanism: heap snapshots (expensive), v8.getHeapStatistics() deltas over longer windows, --trace-gc parsing, or not measuring heap at all.

For RE-017 we chose option 4: the new bench v2 under bench/ does not measure heap at all. Wall time only. Memory analysis is a separate, deliberate exercise with its own tool.

What Needs To Happen

1. **Reproduce in isolation.** Build a minimal repro: an observer, a loop that allocates 10 MiB, a call to takeRecords(). Confirm the observer sees GC events when it should.
2. **If the repro also fails:** file a Node.js bug.
3. **If the repro works:** figure out what was different in the RE-017 bench use case. Possible causes: observer observing before V8 starts emitting events, the "no-op callback" pattern interfering with buffering, an interaction with tsx or --import, or global.gc(true) somehow clearing the buffer.
4. **Document the working pattern in bench/README.md** so future harnesses can use it correctly.

Until one of the above is done: **do not use PerformanceObserver('gc') as a measurement filter in bijou benches**. Treat the observer as diagnostic-only, not load-bearing.

Related

- docs/perf/RE-017-byte-pipeline.md has the full log entry describing the failed measurement, the bimodal data it produced, and the decision to scrap the old bench entirely.

- bench/README.md explains the “measure what you can trust” principle that came out of this.

310

DX-008 — Render Pipeline Documentation Guide

docs/method/graveyard/legacy-backlog/inbox/DX-008-render-pipeline-guide.md

Legend: DX — Developer Experience

Problem

The render pipeline (Layout → Paint → PostProcess → Diff → Output) is a powerful extension point but has zero documentation beyond type definitions. configurePipeline appears in RunOptions but there is no guide showing how to write middleware, what RenderState fields are available, or how stages interact.

The perf-gradient demo proved this gap — we could not instrument the pipeline from outside to measure per-stage timing.

Desired outcome

A guide at docs/guides/render-pipeline.md covering:

1. The five render stages and what each does
2. How to write a custom RenderMiddleware (worked example)
3. RenderState fields and their lifecycle
4. How configurePipeline interacts with default stages
5. Limitations (sync-only, no stage replacement, no timing hooks)

Link from the bijou-tui README's "Building Blocks" section.

Effort

Medium — requires explaining internal architecture to external audience.

311

DX-008 — Sanitize Wrapper for User-Facing Numeric Inputs

docs/method/graveyard/legacy-backlog/v4.5.0/DX-008-sanitize-wrapper-for-user-facing-numerics.md

Legend: DX — Developer Experience

Idea

Create a small shared utility for clamping and validating user-supplied numeric values at component boundaries:

```
function safeInt(n: number | undefined, fallback: number): number {
  if (n == null || !Number.isFinite(n)) return fallback;
  return Math.max(0, Math.floor(n));
}

function safeFloat(n: number | undefined, fallback: number): number {
  if (n == null || !Number.isFinite(n)) return fallback;
  return n;
}
```

Every component that accepts width, height, min, max, count, etc. from user options would call this instead of raw ?? fallbacks.

Why

The current pattern — options.width ?? values.length — does not filter NaN (which is not nullish). This was the root cause of multiple v4.4.0 review findings. A shared wrapper makes the correct pattern easy and the wrong pattern harder.

Relationship

Complements DX-007 (the audit). DX-007 identifies the gaps; this provides the tool to close them systematically.

312

DX-008 — Sanitize Wrapper for User-Facing Numeric Inputs

docs/method/retro/DX-008-sanitize-wrapper-for-user-facing-numerics.md

```
title: "DX-008 — Sanitize Wrapper for User-Facing Numeric Inputs"
legend: DX
lane: retro
```

Completed on release/v5.0.0 as the shared numeric sanitizer module in package s/bijou/src/core/numeric.ts. The repo now exports reusable finite and integer boundary helpers, uses them in shared layout/runtime/components code, and covers the wrapper behavior in packages/bijou/src/core/numeric.test.ts.

Original Proposal

Legend: DX — Developer Experience

Idea

Create a small shared utility for clamping and validating user-supplied numeric values at component boundaries:

```
function safeInt(n: number | undefined, fallback: number): number {
  if (n == null || !Number.isFinite(n)) return fallback;
  return Math.max(0, Math.floor(n));
}

function safeFloat(n: number | undefined, fallback: number): number {
  if (n == null || !Number.isFinite(n)) return fallback;
  return n;
}
```

Every component that accepts width, height, min, max, count, etc. from user options would call this instead of raw ?? fallbacks.

Why

The current pattern — options.width ?? values.length — does not filter NaN (which is not nullish). This was the root cause of multiple v4.4.0 review

findings. A shared wrapper makes the correct pattern easy and the wrong pattern harder.

Relationship

Complements DX-007 (the audit). DX-007 identifies the gaps; this provides the tool to close them systematically.

313

DX-009 — Auto-Init Context and Actionable Error Messages

docs/method/graveyard/legacy-backlog/inbox/DX-009-context-auto-init-and-error-messages.md

Legend: DX — Developer Experience

Problem

Every bijou app must call initDefaultContext() before run(). If forgotten, the error is "No default BijouContext has been set" — which tells you what's wrong but not how to fix it.

Also, keyPriority defaults to 'frame-first', meaning shell keybindings silently win over page keybindings. A developer binding ? to their own action won't see it fire because help consumes it.

Desired outcome

1. run() auto-initializes context if no ctx is provided and no default exists. Error message on failure includes fix instructions and a docs link.
2. createFramedApp emits a runtime warning via routeRuntimeIssue when a page keyMap binding collides with a frameKeys binding under frame-first priority.

Effort

Small — two localized changes.

314

DX-010 — Typed Sub-App Adapter Factory

docs/method/graveyard/legacy-backlog/cool-ideas/DX-010-typed-subapp-adapter.md

Legend: DX — Developer Experience

Idea

Replace the manual mount() / mapCmds() pattern with a compile-time-checked adapter factory:

```
const adapter = createSubAppAdapter<ParentMsg, ChildMsg>({
  'child-done': (msg) => ({ type: 'panel-closed' }),
  'child-error': (msg) => ({ type: 'show-alert', text: msg.error }),
});
```

TypeScript enforces that every ChildMsg variant is covered. A forgotten mapping is a compile error, not a silent message drop.

Why

The current mapCmds() pattern works but requires manual wiring with no type-level guarantee of completeness. In larger apps with multiple sub-apps, a missing mapping causes a message to vanish silently — the hardest kind of bug to diagnose in a TEA architecture.

Open questions

- Should the adapter handle both messages and commands, or just messages?
- Should it support pass-through for shared message types?

315

DX-011 — Key Collision Warnings at Registration Time

docs/method/graveyard/legacy-backlog/cool-ideas/DX-011-key-collision-warnings.md

Legend: DX — Developer Experience

Idea

When createFramedApp is called with pages that have keybindings which collide with the frame's built-in bindings (help ?, settings ctrl+„ quit escape, etc.), emit a warning through routeRuntimeIssue at app initialization time — not at key-press time.

> [Framework warning] Page "editor" binds "?" but frame-first key priority means the frame's help toggle will consume it. Use keyPriority: 'page-first' or choose a different key.

Why

Currently, a developer binds ? to "show info" in their page, runs the app, presses ?, and sees the help overlay instead of their handler. There is no diagnostic — the key is silently consumed. The developer has to read the frame-first/page-first docs to understand what happened.

A warning at registration time catches this immediately.

316

DX-012 — Document Render-Path Naming Convention

docs/method/graveyard/legacy-backlog/cool-ideas/DX-012-render-path-naming-convention.md

Legend: DX — Developer Experience

Idea

Document the implicit naming convention for string-output vs Surface-output component variants:

- box(content) → string (pipe-friendly)
- boxSurface(content) → Surface (composable in TUI apps)
- badge(content) → Surface only (no string variant)
- table(data) → string (mixed-mode, pipe-aware)

The pattern exists but is never stated explicitly. A developer browsing the API doesn't know why some components have a -Surface variant and others don't, or why badge() returns a Surface directly while box() returns a string.

Why

This came up in the DX audit as a minor POLA concern. The convention is sound (simpler output-only components are string-first; richer interactive components are Surface-native), but it reads as inconsistency when you're scanning exports without context.

A one-paragraph note in the bijou README's component section would resolve the confusion.

317

DX-013 — Motion Key Lifecycle Warning

docs/method/graveyard/legacy-backlog/cool-ideas/DX-013-motion-key-warning.md

Legend: DX — Developer Experience

Idea

motion({ key: 'id' }, ...) requires a stable key to track animation state across renders. If the key changes every render (e.g., using an index that shifts), the motion system silently creates new animations instead of interpolating. There is no warning.

Emit a runtime warning when a motion key appears for the first time on a frame that also has a different key disappearing — this is a strong signal that the developer is generating unstable keys.

Why

The DX audit flagged this as a non-obvious behavior. The motion system works correctly when keys are stable, but the failure mode (new animation every render = no visible interpolation) looks like "motion doesn't work" rather than "my key is wrong."

Effort

Small — track key birth/death across frames in the motion middleware.

318

DX-014 — Fix Root README Interactive Example

docs/method/graveyard/legacy-backlog/cool-ideas/DX-014-readme-interactive-example-fix.md

Legend: DX — Developer Experience

Idea

The interactive runtime example in the root README doesn't show the mandatory initDefaultContext() call. A developer copying the example into a new file gets:

```
Error: No default BijouContext has been set.
```

Add import { initDefaultContext } from '@flyingrobots/bijou-node' and initDefaultContext() to the example so it's copy-pasteable without hidden prerequisites.

Why

This is the #1 TTV barrier from the documentation audit. The scaffolded path handles it automatically, but anyone integrating into an existing project hits this wall on their first attempt.

Effort

Tiny — two lines added to the README example.

319

DX-015 — Perf Gradient as CI Benchmark

docs/method/graveyard/legacy-backlog/cool-ideas/DX-015-perf-gradient-as-ci-benchmark.md

Legend: DX — Developer Experience

Idea

The examples/perf-gradient demo runs in non-interactive mode and produces measurable output. It could be adapted into a CI benchmark that runs for a fixed number of frames and reports:

- Average view() time per frame
- Peak heap usage
- GC event count
- Total frame count in a fixed wall-clock window

A regression would show up as a measurable increase in view() time or heap pressure across commits.

Why

The perf demo proved that bijou's rendering bottleneck is in the pipeline (differ + encode), not in the app. Any future optimization (RE-008 byte-packed surface, RE-009 differ string fix) needs a benchmark to validate the improvement. Running the benchmark in CI prevents regressions from sneaking in.

Shape

A non-interactive test that: 1. Creates a gradient app 2. Feeds it N tick messages 3. Asserts view() time stays below a threshold 4. Reports heap delta

Could live in scripts/perf-bench.test.ts or as a GitHub Action that posts results as PR comments.

Risks

- Terminal rendering in CI is headless — stdout.write still works but there's no terminal to parse the ANSI. The differ will still run but output goes to /dev/null.
- Wall-clock benchmarks are noisy in CI. Use statistical methods (median of N runs) not single-run thresholds.

320

DX-016 — Error Surface for Crash Post-Mortems

docs/method/graveyard/DX-016-error-surface-post-mortem.md

```
title: "DX-016 — Error Surface for Crash Post-Mortems"
legend: DX
lane: graveyard
```

Disposition

Superseded by DX-021 — Error Surface for Crash Post-Mortems. The crash-surface work shipped, but this older backlog note used the DX-016 identifier before that id was later reused for scenario-provenance bench work. Keep this file only as historical lineage for the legacy backlog tree.

Original Proposal

Legend: DX — Developer Experience

Idea

When a TUI app crashes during update() or view(), the runtime.ts currently catches the error, writes the stack trace to stderr, and exits. If the app is in alt-screen mode, this stack trace is often lost or obscured by the terminal's screen restoration.

Implement a dedicated "Error Surface" that is rendered using Bijou components *before* the alt-screen is exited. This surface should display: - The error message and stack trace. - The last stable model snapshot (if serializable). - A prompt to "Press Enter to exit."

Why

1. **Visibility**: Keeps critical crash data on-screen for the developer to inspect before the process dies.

2. **Professionalism**: Moves the app from a “noisy CLI crash” to a “structured post-mortem.”
3. **Diagnostic Power**: Capturing the model state at the point of failure is an industrial-strength debugging aid.

Effort

Medium — requires a special “Crash Mode” in the run loop that bypasses the normal TEA update and renders a static diagnostic surface.

321

DX-016 — Error Surface for Crash Post-Mortems

docs/method/graveyard/legacy-backlog/cool-ideas/DX-016-error-surface-post-mortem.md

Legend: DX — Developer Experience

Idea

When a TUI app crashes during update() or view(), the runtime.ts currently catches the error, writes the stack trace to stderr, and exits. If the app is in alt-screen mode, this stack trace is often lost or obscured by the terminal's screen restoration.

Implement a dedicated "Error Surface" that is rendered using Bijou components *before* the alt-screen is exited. This surface should display: - The error message and stack trace. - The last stable model snapshot (if serializable). - A prompt to "Press Enter to exit."

Why

1. **Visibility**: Keeps critical crash data on-screen for the developer to inspect before the process dies.
2. **Professionalism**: Moves the app from a "noisy CLI crash" to a "structured post-mortem."
3. **Diagnostic Power**: Capturing the model state at the point of failure is an industrial-strength debugging aid.

Effort

Medium — requires a special "Crash Mode" in the run loop that bypasses the normal TEA update and renders a static diagnostic surface.

322

DX-016 — Scenario Provenance Tags For Intelligent Bench Filtering

docs/method/retro/DX-016-scenario-provenance-tags.md

title: DX-016 — Scenario Provenance Tags For Intelligent Bench Filtering
lane: retro

Disposition

Shipped scenario provenance tags across the bench registry. Scenarios now declare stable lowercase tags, the CLI supports tag-filtered run/list flows with comma-separated AND inside one –tag and OR across repeated –tag flags, bench docs explain the contract, and focused bench tests cover tag parsing and selection.

Original Proposal

Legend: DX — Developer Experience

Idea

Add a tags: readonly string[] field to the Scenario interface in bench/src/scenarios/types.ts. Tags describe what a scenario exercises, not what it looks like:

```
interface Scenario<State> {
  // ...existing fields...
  readonly tags: readonly string[];
}

// Example:
export const paintGradientRgb: Scenario<State> = {
  id: 'paint-gradient-rgb',
  tags: ['paint', 'setRGB', 'unique-styles', 'no-hex-parse'],
  // ...
};
```

Harnesses can then filter:

```
# Run everything that exercises the diff path
bench run --tag=diff

# Run everything that stresses unique style count
bench run --tag=unique-styles

# Run the RE-017 gate set
bench run --tag=re017-gate
```

Multi-tag logic: --tag=paint,diff = AND, --tag=paint --tag=diff = OR (or pick one convention and document it).

Why

The scenario registry will grow. RE-017 has 4 scenarios today but the full target is 20+ across paint / diff / layout / compose / text categories. Running "all scenarios" will become slow and scatter attention. We need a way to say "just run the ones that could have been affected by my change":

- Touched the differ? --tag=diff
- Touched the layout engine? --tag=layout
- Touched hex parsing? --tag=hex-parse
- Landing a perf optimization that only affects paint? --tag=paint

Why This Is Cool

1. **Lets CI gates be precise.** A PR that only touches the layout code can run --tag=layout in CI and skip diff scenarios — faster feedback, same signal.
2. **Self-documenting.** Tags describe the *intent* of each scenario. Reading the registry at a glance tells you what's covered and where gaps are.
3. **Enables coverage tracking.** "Do we have a scenario tagged emoji?" becomes easy to answer. Missing tags = missing coverage.
4. **Pairs with the sandbox harness.** The interactive sandbox (task I-7) could offer tag-based scenario selection in its UI.

Shape

Propose a tag vocabulary (append as the registry grows):

- **Path tags:** paint, diff, layout, normalize, compose, runtime.
- **API tags:** set, setRGB, blit, fill, clear.
- **Stress tags:** unique-styles, sparse-diff, dense-diff, hex-parse, side-table, emoji, wide-char.
- **Shape tags:** small, medium, large, xlarge.
- **Gate tags:** re017-gate, smoke, nightly.

A scenario typically has 3-5 tags. Tags are lowercase, hyphenated, stable (renaming a tag is a backwards-incompatible change).

Related

- bench/src/scenarios/types.ts — where the field lands.
- bench/src/cli.ts — where --tag is consumed.
- Task #18 (I-7) — sandbox can use tags for scenario selection.
- Task #35 — the CI gate that consumes tag filters.

323

DX-017 — Agent-First Bench Output Format

docs/method/retro/DX-017-agent-first-bench-output-format.md

```
title: DX-017 — Agent-First Bench Output Format
lane: retro
```

Disposition

Shipped an additive flat bench output format. bench run now supports --format=json for nested bench.v2 output and --format=jsonl/--format=flat for one-metric-per-line records, compare auto-detects nested JSON vs flat JSONL, RunReport now carries a stable runId, and the bench docs describe the new structured output contract while preserving the existing human summary as the default console view.

Original Proposal

Legend: DX — Developer Experience

Idea

The bench v2 report JSON is structured for humans: nested arrays, scenario → samples → stats. It's readable but inefficient for agents to consume programmatically, especially when comparing many runs over time (e.g., "how did p99 trend across the last 20 RE-017 commits?").

Add a flat, agent-friendly output format alongside the existing one. Structure:

```
{"runId":"...","scenario":"paint-gradient-rgb","commit":"abc","metric":"ns_per_frame.p50","va
↪  lue":946000,"unit":"ns","cov":0.029}
{"runId":"...","scenario":"paint-gradient-rgb","commit":"abc","metric":"ns_per_frame.p99","va
↪  lue":954000,"unit":"ns","cov":0.029}
{"runId":"...","scenario":"paint-gradient-rgb","commit":"abc","metric":"samples_count","value
↪  ":30,"unit":"count"}
...
```

One line per (scenario, metric) tuple. Agents can stream, filter, tail, or grep without parsing nested structures. Queries like "give me the p50 for

paint-gradient-rgb across runs X, Y, Z" become one-line jq or ripgrep commands.

Expose via bench run --format=jsonl or bench run --format=flat, with --format =json remaining the default for humans.

Why

I (the agent) noticed this during the RE-017 audit when I had to hand-build an aggregator (scripts/aggregate-audit-runs.ts, since deleted) to crunch stats across 10 run files. If the output had been flat JSONL I would have written jq | sort | uniq and been done in a minute. Instead I wrote 200 lines of TypeScript that had its own bugs.

More broadly: I operate more efficiently when tools emit data in formats I can pipe through small composable UNIX primitives. The human-friendly nested JSON is great for humans reading one run at a time. It's not great for me reading 100 runs at a time.

This is an **agent-first observation**: a DX improvement that benefits agents specifically. Humans can still use the nested format. The flat format is an additional option that makes my workflow faster and cleaner.

Shape

- bench/src/harnesses/wall-time/format-jsonl.ts — converts a RunReport to a sequence of JSONL records.
- CLI: bench run --format=jsonl emits the JSONL instead of the nested JSON summary.
- bench compare accepts either format (auto-detect on the first line: {"kind" :"bench.v2"} = nested, {"runId":...} = flat).
- Flat records include a stable runId (ULID or uuid) so records from the same run can be grouped.

Prior Art

- Prometheus exposition format (label-keyed metrics, one per line).
- InfluxDB line protocol.
- OpenTelemetry traces-as-jsonl.

Why This Is Cool

1. **Agent-ergonomic.** I can work with 100 bench runs using standard text tools. No custom aggregators.
2. **Streaming-friendly.** Large runs can stream to disk and be processed incrementally.

3. **Composes with tag filtering.** Pair with DX-016-scenario- provenance-tags.md for filtered cross-run analysis.
4. **Pairs with a future time-series view.** A dashboard or sparkline TUI can tail the JSONL and render live.

Related

- bench/src/harnesses/wall-time/runner.ts — emits the current nested format.
- DX-016-scenario-provenance-tags.md — complementary filtering.
- Agent-first development philosophy: agents have their own DX needs; we advocate for them openly.

324

DX-017 — One-Call Node Host Entry Point

docs/method/graveyard/legacy-backlog/up-next/DX-017-one-call-node-host-entry-point.md

Legend: DX — Developer Experience

Problem

Simple interactive apps currently have to compose multiple setup steps by hand:

- initialize a default context
- choose the Node host adapter package
- run the TEA app
- restore terminal state and shut down cleanly

The pieces exist, but there is no obvious “first real app” entry point in @fl yingrobots/bijou-node. The 2026-04-11 code-quality audit called this out directly as a time-to-value gap.

Desired outcome

1. Add a startApp(app, options)-style entry point in @flyingrobots/bijou-node.
2. Let it own context initialization, runtime startup, and graceful shutdown by default while preserving lower-level escape hatches.
3. Update first-app docs to use this path so the hosted Node story is explicit instead of implied.

Effort

Medium — public host API design plus docs and example updates.

325

DX-017 — One-Call Node Host Entry Point

docs/method/retro/DX-017-one-call-node-host-entry-point.md

```
title: "DX-017 — One-Call Node Host Entry Point"
legend: DX
lane: retro
```

Completed on release/v5.0.0. @flyingrobots/bijou-node now exposes startApp(ap
p, options) as the hosted fast path, owns default-context startup and
teardown for first-app flows, documents that path across package guides, and
covers it in packages/bijou-node/src/index.test.ts. The framed-shell hosted
path now also delegates through startApp() instead of bypassing it.

Original Proposal

Legend: DX — Developer Experience

Problem

Simple interactive apps currently have to compose multiple setup steps by
hand:

- initialize a default context
- choose the Node host adapter package
- run the TEA app
- restore terminal state and shut down cleanly

The pieces exist, but there is no obvious "first real app" entry point in @fl
yingrobots/bijou-node. The 2026-04-11 code-quality audit called this out
directly as a time-to-value gap.

Desired outcome

1. Add a startApp(app, options)-style entry point in @flyingrobots/bijou-node.
2. Let it own context initialization, runtime startup, and graceful shutdown
   by default while preserving lower-level escape hatches.

3. Update first-app docs to use this path so the hosted Node story is explicit instead of implied.

Effort

Medium — public host API design plus docs and example updates.

326

DX-018 — Runtime Test Harness for TEA Apps

docs/method/graveyard/legacy-backlog/up-next/DX-018-runtime-test-harness.md

Legend: DX — Developer Experience

Problem

Testing nontrivial apps still leaks internal runtime knowledge. Async commands, emitted messages, and lifecycle ordering can be tested today, but the path often requires understanding EventBus details instead of staying at the app boundary.

The 2026-04-11 code-quality audit called for a testRuntime(app)-style helper that behaves like a first-class harness rather than a bag of internal seams.

Desired outcome

1. Provide a testRuntime(app, options) helper that returns a TestHarness.
2. Support assertions on state snapshots, emitted messages, command resolution, and teardown behavior without hand-wiring the bus.
3. Use it in at least one runtime-facing example or test so the public testing story is obvious.

Effort

Medium — test API design plus migration of at least one real suite.

327

DX-018 — Runtime Test Harness for TEA Apps

docs/method/retro/DX-018-runtime-test-harness.md

```
title: "DX-018 — Runtime Test Harness for TEA Apps"
legend: DX
lane: retro
```

Completed on release/v5.0.0. @flyingrobots/bijou-tui now exports testRuntime( ) from packages/bijou-tui/src/driver.ts, uses it in real runtime-facing tests and app examples, and documents the harness in the TUI README and GUIDE. Runtime testing no longer requires every consumer to hand-wire the event bus just to assert on state, messages, and teardown.

Original Proposal

Legend: DX — Developer Experience

Problem

Testing nontrivial apps still leaks internal runtime knowledge. Async commands, emitted messages, and lifecycle ordering can be tested today, but the path often requires understanding EventBus details instead of staying at the app boundary.

The 2026-04-11 code-quality audit called for a testRuntime(app)-style helper that behaves like a first-class harness rather than a bag of internal seams.

Desired outcome

1. Provide a testRuntime(app, options) helper that returns a TestHarness.
2. Support assertions on state snapshots, emitted messages, command resolution, and teardown behavior without hand-wiring the bus.
3. Use it in at least one runtime-facing example or test so the public testing story is obvious.

Effort

Medium — test API design plus migration of at least one real suite.

328

DX-019 — Third-Party Theme Reactivity Hook

docs/method/graveyard/legacy-backlog/cool-ideas/DX-019-third-party-theme-reactivity-hook.md

Legend: DX — Developer Experience

Problem

Bijou's theme/token system is strong inside core components, but third-party component authors do not have a clearly supported way to react to theme changes. Today this often falls back to manual context cloning or ambient-default assumptions.

The 2026-04-11 code-quality audit explicitly called out the lack of a supported subscription path for external components.

Desired outcome

1. Define a public reactivity seam for theme/token updates.
2. Decide whether that seam is a direct tokenGraph.subscribe() export, a context-level wrapper, or a narrower theme-change subscription contract.
3. Document how third-party components should observe theme changes without mutating context or relying on private internals.

Effort

Medium — public API design plus advanced-guide documentation.

329

DX-019 — Third-Party Theme Reactivity Hook

docs/method/retro/DX-019-third-party-theme-reactivity-hook.md

```
title: "DX-019 — Third-Party Theme Reactivity Hook"
legend: DX
lane: retro
```

Completed on release/v5.0.0. Bijou now exposes observeTheme(ctx, handler) as the public theme-change seam in packages/bijou/src/context-theme.ts, documents that contract in packages/bijou/ADVANCED_GUIDE.md, packages/bijou/ARCHITECTURE.md, and docs/design-system/theme-tokens.md, and covers it in packages/bijou/src/context-theme.test.ts. Third-party code no longer needs to reach into tokenGraph internals just to react to theme updates.

Original Proposal

Legend: DX — Developer Experience

Problem

Bijou's theme/token system is strong inside core components, but third-party component authors do not have a clearly supported way to react to theme changes. Today this often falls back to manual context cloning or ambient-default assumptions.

The 2026-04-11 code-quality audit explicitly called out the lack of a supported subscription path for external components.

Desired outcome

1. Define a public reactivity seam for theme/token updates.
2. Decide whether that seam is a direct tokenGraph.subscribe() export, a context-level wrapper, or a narrower theme-change subscription contract.
3. Document how third-party components should observe theme changes without mutating context or relying on private internals.

Effort

Medium — public API design plus advanced-guide documentation.

330

DX-020 — Layout Localization Pipeline Guide

docs/method/graveyard/legacy-backlog/up-next/DX-020-layout-localization-pipeline-guide.md

Legend: DX — Developer Experience

Problem

The layout/localization path around layout-node-surface.ts is important to both performance and correctness, but the repo does not yet have a dedicated strategy doc explaining:

- localization vs painting
- how offsets accumulate
- when layout is recomputed
- where retained-layout work would fit

The 2026-04-11 documentation audit called this out as a missing deep guide for advanced users and maintainers.

Desired outcome

1. Write a dedicated strategy doc for the layout-localization pipeline.
2. Explain the recursion model, coordinate spaces, and relationship between localization and paint/output stages.
3. Link it from the advanced guide and relevant architecture docs.

Effort

Small to medium — mostly documentation, but it may expose naming or API cleanup opportunities along the way.

331

DX-020 — Layout Localization Pipeline Guide

docs/method/retro/DX-020-layout-localization-pipeline-guide.md

```
title: "DX-020 — Layout Localization Pipeline Guide"
legend: DX
lane: retro
```

Completed on release/v5.0.0. The dedicated strategy note now lives at docs/strategy/layout-localization-pipeline.md, and the advanced/runtime docs link to it from packages/bijou-tui/ARCHITECTURE.md, packages/bijou-tui/ADVANCED_GUIDE.md, and docs/guides/render-pipeline.md. The repo now has an explicit deep guide for localization vs painting, coordinate accumulation, and retained-layout fit.

Original Proposal

Legend: DX — Developer Experience

Problem

The layout/localization path around layout-node-surface.ts is important to both performance and correctness, but the repo does not yet have a dedicated strategy doc explaining:

- localization vs painting
- how offsets accumulate
- when layout is recomputed
- where retained-layout work would fit

The 2026-04-11 documentation audit called this out as a missing deep guide for advanced users and maintainers.

Desired outcome

1. Write a dedicated strategy doc for the layout-localization pipeline.
2. Explain the recursion model, coordinate spaces, and relationship between localization and paint/output stages.

3. Link it from the advanced guide and relevant architecture docs.

Effort

Small to medium — mostly documentation, but it may expose naming or API cleanup opportunities along the way.

332

DX-021 — Error Surface for Crash Post-Mortems

docs/method/retro/DX-021-error-surface-post-mortem.md

```
title: DX-021 — Error Surface for Crash Post-Mortems
lane: retro
legend: DX
```

Disposition

Implemented on release/v5.0.0. The interactive TUI runtime now switches into a crash surface when update() or view() fails, renders the error, stack, phase, and last stable model snapshot inside the alt screen, and waits for Enter before tearing the terminal down and rethrowing the failure. That turns a lost alt-screen crash into a readable post-mortem without changing the raw runtime contract.

Original Proposal

Legend: DX — Developer Experience

Idea

When a TUI app crashes during update() or view(), the runtime.ts currently catches the error, writes the stack trace to stderr, and exits. If the app is in alt-screen mode, this stack trace is often lost or obscured by the terminal's screen restoration.

Implement a dedicated "Error Surface" that is rendered using Bijou components *before* the alt-screen is exited. This surface should display:

- The error message and stack trace.
- The last stable model snapshot (if serializable).
- A prompt to "Press Enter to exit."

333

DX-028 — Align front-door docs with repo truth

docs/method/retro/DX-028-align-front-door-docs-with-repo-truth.md

```
title: DX-028 — Align front-door docs with repo truth
legend: DX
lane: retro
```

Disposition

Completed on 2026-04-14 via README, CONTRIBUTING, and SECURITY updates that align the front door with the shipped package surface, preferred startApp() bootstrap path, npm workspace workflow, and current supported release line. Validation included docs inventory, lint, and full release-readiness.

Original Proposal

The package-level docs are mostly coherent now, but the front door and operator docs still lag the repo's actual runtime and release posture.

Problem

The 2026-04-14 documentation audit found a small set of high-friction drifts: - README.md under-represents the shipped package surface - the root interactive example still teaches initDefaultContext() + run(app) while package docs prefer startApp() - CONTRIBUTING.md still says pnpm install even though the repo is npm-workspace based - SECURITY.md still has a stale supported-version table

None of these are architecture problems, but together they make the first contact with the repo less truthful than the package docs and release tooling.

Desired outcome

1. Update README.md so the package map reflects the current published surface.

2. Make the root quick start use the preferred Node-host bootstrap path.
3. Fix CONTRIBUTING.md to match the repo's actual install and validation workflow.
4. Refresh SECURITY.md so its supported-version language matches current repo truth.
5. Add a short "choose your lane" front-door path for CLI vs TUI vs MCP vs i18n.

Why ASAP

- This is the highest-signal docs debt still visible to every new user.
- The repo now has strong release proof; the front door should stop lagging behind it.

Hill

The first five minutes in the repo point users at the same package/runtime truth that the implementation and release tooling already enforce.

334

DX-003 — Rationalize Table APIs and Public Table Types

docs/method/retro/DX_003-rationalize-table-apis-and-public-table-types.md

```
title: DX-003 — Rationalize Table APIs and Public Table Types
lane: retro
legend: DX
```

Disposition

Implemented on release/v4.5.0 by adding common table shorthands for table() and tableSurface(), introducing clearer public row types for plain and surface tables, and broadening navigable-table inputs so lightweight snapshots no longer require a separate state mental model for simple rendering. Focused table, surface, and navigable-table tests now prove the family feels consistent across plain and interactive use.

Original Proposal

Legend: DX — Developer Experience

Idea

Make the table family feel like one coherent surface instead of a cluster of near-related APIs with mismatched conventions.

Why

The current table APIs create avoidable friction:

- navigableTableSurface(...) and tableSurface(...) use different calling conventions
- the rows types are confusing between plain strings and TableSurfaceCell

- the common case for “just show me a table” is more verbose than it should be

This is exactly the kind of thing users discover by compiler error instead of by intention.

Captured feedback

From warp-ttd TUI development:

- tableSurface vs navigableTableSurface calling conventions are inconsistent
- rows typing is confusing around string[][] vs TableSurfaceCell[][]
- TableSurfaceCell is not obvious from the public surface
- a simple table(columns, rows, ctx?) style shorthand would help the common case

Likely scope

- decide on consistent table-family calling conventions
- make public table row/cell typing obvious and documented
- consider a simpler high-frequency table helper for the 80% case
- add tests proving the table family feels consistent across plain and navigable flows

335

DX-004 — Smooth Surface and String Composition Seams

docs/method/retro/DX_004-smooth-surface-and-string-composition-seams.md

```
title: DX-004 — Smooth Surface and String Composition Seams
lane: retro
legend: DX
```

Disposition

Shipped a public mixed composition seam in @flyingrobots/bijou-tui: contentSurface() now bridges raw text into surface-land, vstackSurface()/hstackSurface()/placeSurface() accept string | Surface inputs at the composition edge, runtime view normalization points callers at the sanctioned bridge, and package docs now explain how to keep composition on the surface side without loosening App.view back to raw strings.

Original Proposal

Legend: DX — Developer Experience

Idea

Reduce confusion at the boundary between string-oriented composition and Surface/layout-oriented composition.

Why

Bijou currently exposes a real conceptual seam here, but the ergonomics are easy to trip over:

- some helpers return Surface
- some stack/layout helpers still expect strings

- view() requires Surface | LayoutNode, not string
- migration from older string-first patterns is easy to misunderstand

The issue is not only documentation. Some composition helpers may need better bridging or clearer contracts.

Captured feedback

From warp-ttd TUI development:

- badge() returning Surface made vstack() composition awkward
- surfaceToString(...) was required in some places but forbidden in view()
- the view() return constraint is understandable but still easy to trip over during migration

Likely scope

- audit string-land vs surface-land composition boundaries
- decide where bridging helpers should exist and where they should not
- improve the composition story for small surface-returning primitives
- tighten docs or error guidance where the seam is intentionally strict

336

DX-005 — Polish Small Component and Import Ergonomics

docs/method/retro/DX_005-polish-small-component-and-import-ergonomics.md

```
title: DX-005 — Polish Small Component and Import Ergonomics
lane: retro
legend: DX
```

Disposition

Closed the remaining live DX-005 import seam by re-exporting BijouContext, Cell, and Surface from @flyingrobots/bijou-tui so canvas/shader authoring can stay on a single package-root import, added a type-level regression proving the single-import path works, and refreshed advanced docs accordingly. The older boxSurface title-padding subfinding is already satisfied by current repo truth: box() and boxSurface() both inject calm default title spacing internally rather than requiring manual spaces.

Original Proposal

Legend: DX — Developer Experience

Idea

Sweep up the recurring small API papercuts that make Bijou feel fussier than it needs to.

Why

None of these issues are individually existential, but together they create friction:

- box titles needing manual padding

- imports for one conceptual feature split across too many packages
- minor public-API rough edges that make common authoring flows feel less polished than the runtime really is

These are good backlog items precisely because users keep encountering them in real app work.

Captured feedback

From warp-ttd TUI development:

- boxSurface title padding required manual spaces
- canvas / ShaderFn / Surface / BijouContext required three imports across package boundaries for one shader function

Likely scope

- make boxed-title padding calmer by default or easier to opt into
- review re-export strategy for frequently paired shader/canvas types
- capture any other low-risk public API polish work that falls out while fixing the above

337

DX-006 — Debug Performance Overlay Component

docs/method/retro/DX_006-debug-performance-overlay-component.md

```
title: DX-006 — Debug Performance Overlay Component
lane: retro
legend: DX
```

Disposition

Shipped a first-class debugOverlay() wrapper in @flyingrobots/bijou-tui. It composes the existing perfOverlaySurface() into any app surface with corner anchoring, optional background dimming, package exports, docs, and focused regression coverage, so the original perf-overlay component idea is now real repo truth rather than backlog intent.

Original Proposal

Legend: DX — Developer Experience

Idea

Ship a first-class debugOverlay() component in @flyingrobots/bijou-tui that any app can composite over its view to see real-time performance instrumentation:

- FPS counter
- Frame time (with rolling graph)
- Update and view phase timing
- Heap used/total, RSS, external memory
- GC event count per sample window
- Terminal size and cell count
- Configurable position (corner anchoring)

Why

The examples/perf-gradient stress test proved the concept: a self-rendering stats overlay that tracks frame timing, memory, and GC pressure is immediately useful for diagnosing performance issues. But it's hardcoded into the demo. Every app that wants performance visibility has to rebuild this from scratch.

Shape

A composable surface component, not a shell/app-frame feature:

```
import { debugOverlay } from '@flyingrobots/bijou-tui';

view(model) {
  const main = renderMyApp(model);
  return debugOverlay(main, { anchor: 'top-left' });
}
```

The overlay composites onto the app's surface. It reads performance data from a shared timing context (similar to the mutable phaseTiming object in the perf demo). Apps opt in by wrapping their view — no framework integration required.

Open questions

- Should the render pipeline expose phase timing natively (layout, paint, diff, output) so the overlay can show the full breakdown? Currently the pipeline isn't observable from the app side.
- Should the overlay be tree-shakeable (dev-only) or always available?
- Should it support custom metrics (app-defined counters/gauges)?

Progress (2026-04-10)

The generic toolkit components are now shipped in @flyingrobots/bijou: sparkline, brailleChartSurface, statsPanelSurface, perfOverlaySurface. The soak runner uses perfOverlaySurface as a composited overlay with backtick toggle. This proves the composable-surface approach works.

The remaining open question — "should the pipeline expose phase timing natively?" — is addressed by RE-021 Frame Owns the Pump. If the frame owns the render loop, it naturally measures update/view/diff phase timing and can feed it to a built-in perf overlay without any special capability injection.

Related

- RE-021 Frame Owns the Pump — enables native phase timing

- RE-008 byte-packed surface representation (affects rendering perf)
- The perf gradient demo (examples/perf-gradient) as proof of concept
- The soak runner perf overlay (this session) as integration proof

338

DX-007 — Decompose app-frame.ts Mega-Closure

docs/method/graveyard/legacy-backlog/v5.0.0/DX_007-decompose-app-frame-mega-closure.md

Legend: DX — Developer Experience

Problem

packages/bijou-tui/src/app-frame.ts is 2,679 lines with 9 responsibilities, 90+ internal functions, 38 imports, and a mega-closure that prevents unit testing any individual function.

Evidence

- **Modularity**: routing, rendering, state, command dispatch, settings, notifications, help, overlays, and transitions all live in one file
- **Coupling**: circular import with app-frame-types.ts (FrameModel)
- **Testability**: every internal function captures closure state; none can be imported or tested independently
- **Growth**: each RE cycle (001-007) added more logic without decomposing

Target

Decompose into focused domain modules:

1. frame-settings.ts — settings drawer state, focus, scroll, row activation, resolveSettingsLayout
2. frame-notifications.ts — notification center state, scroll, filter, resolveNotificationCenterLayout
3. frame-help.ts — help overlay rendering, scroll computation
4. frame-shell-commands.ts — shellCommandHandlers table, FrameShellCommand apply logic, drainShellCommandBuffer
5. Move FrameModel to app-frame-types.ts to break circular import

Each module independently importable and unit-testable with deps injection. Target: app-frame.ts under 800 lines.

Effort

Large — touches the highest-traffic file in the repo. Must be done incrementally with tests at each step.

339

DX-008 — Render Pipeline Documentation Guide

docs/method/retro/DX_008-render-pipeline-guide.md

```
title: DX-008 — Render Pipeline Documentation Guide
lane: retro
legend: DX
```

Disposition

Published docs/guides/render-pipeline.md covering the fixed render stage order, default runtime wiring, RenderState lifecycle, configurePipeline semantics, and current limitations. Linked the guide from the bijou-tui README, the repo docs map, and the bijou-tui advanced guide, and corrected the stale Post-Process stage spelling in the advanced guide example.

Original Proposal

Legend: DX — Developer Experience

Problem

The render pipeline (Layout → Paint → PostProcess → Diff → Output) is a powerful extension point but has zero documentation beyond type definitions. configurePipeline appears in RunOptions but there is no guide showing how to write middleware, what RenderState fields are available, or how stages interact.

The perf-gradient demo proved this gap — we could not instrument the pipeline from outside to measure per-stage timing.

Desired outcome

A guide at docs/guides/render-pipeline.md covering:

1. The five render stages and what each does

2. How to write a custom RenderMiddleware (worked example)
3. RenderState fields and their lifecycle
4. How configurePipeline interacts with default stages
5. Limitations (sync-only, no stage replacement, no timing hooks)

Link from the bijou-tui README's "Building Blocks" section.

Effort

Medium — requires explaining internal architecture to external audience.

340

DX-009 — Auto-Init Context and Actionable Error Messages

docs/method/retro/DX_009-context-auto-init-and-error-messages.md

```
title: DX-009 — Auto-Init Context and Actionable Error Messages
lane: retro
legend: DX
```

Disposition

Completed on release/v4.5.0. Ambient default-context bootstrapping is now host-pluggable through the core default-context initializer seam, @flyingrobots/bijou-node registers the Node initializer so common run() flows no longer require a manual initDefaultContext() call, runtime startup errors now point directly at the Node setup guide, and createFramedApp emits a one-time runtime warning when frame-first shell bindings shadow page key bindings.

Original Proposal

Legend: DX — Developer Experience

Problem

Every bijou app must call initDefaultContext() before run(). If forgotten, the error is “No default BijouContext has been set” — which tells you what’s wrong but not how to fix it.

Also, keyPriority defaults to ’frame-first’, meaning shell keybindings silently win over page keybindings. A developer binding ? to their own action won’t see it fire because help consumes it.

Desired outcome

1. run() auto-initializes context if no ctx is provided and no default exists. Error message on failure includes fix instructions and a docs link.
2. createFramedApp emits a runtime warning via routeRuntimeIssue when a page keyMap binding collides with a frameKeys binding under frame-first priority.

Effort

Small — two localized changes.

341

DX-010 — Typed Sub-App Adapter Factory

docs/method/retro/DX_010-typed-subapp-adapter.md

```
title: DX-010 — Typed Sub-App Adapter Factory
lane: retro
legend: DX
```

Disposition

Shipped createSubAppAdapter() in @flyingrobots/bijou-tui as an exhaustive discriminant-based onMsg factory for sub-app composition. The canonical sub-app example, package docs, and guide now use the helper, and compile-time coverage is proven with @ts-expect-error regression assertions in the sub-app mount tests while mapCmd remains available for explicit command interception.

Original Proposal

Legend: DX — Developer Experience

Idea

Replace the manual mount() / mapCmds() pattern with a compile-time-checked adapter factory:

```
const adapter = createSubAppAdapter<ParentMsg, ChildMsg>({
  'child-done': (msg) => ({ type: 'panel-closed' }),
  'child-error': (msg) => ({ type: 'show-alert', text: msg.error }),
});
```

TypeScript enforces that every ChildMsg variant is covered. A forgotten mapping is a compile error, not a silent message drop.

Why

The current mapCmds() pattern works but requires manual wiring with no type-level guarantee of completeness. In larger apps with multiple sub-apps,

a missing mapping causes a message to vanish silently — the hardest kind of bug to diagnose in a TEA architecture.

Open questions

- Should the adapter handle both messages and commands, or just messages?
- Should it support pass-through for shared message types?

342

DX-011 — Key Collision Warnings at Registration Time

docs/method/retro/DX_011-key-collision-warnings.md

title: DX-011 — Key Collision Warnings at Registration Time
lane: retro
legend: DX

Disposition

Repo truth already ships this warning. createFramedApp() queues a runtime warning when a page key binding is shadowed by a frame binding under keyPriority: 'frame-first', and app-frame.test.ts already covers both the warning emission and the once-per-page guard. The backlog note is stale relative to shipped behavior.

Original Proposal

Legend: DX — Developer Experience

Idea

When createFramedApp is called with pages that have keybindings which collide with the frame's built-in bindings (help ?, settings ctrl+„ quit escape, etc.), emit a warning through routeRuntimeIssue at app initialization time — not at key-press time.

> [Framework warning] Page "editor" binds "?" but frame-first key priority means the frame's help toggle will consume it. Use keyPriority: 'page-first' or choose a different key.

Why

Currently, a developer binds ? to “show info” in their page, runs the app, presses ?, and sees the help overlay instead of their handler. There is no diagnostic — the key is silently consumed. The developer has to read the frame-first/page-first docs to understand what happened.

A warning at registration time catches this immediately.

343

DX-012 — Document Render-Path Naming Convention

docs/method/retro/DX_012-render-path-naming-convention.md

```
title: DX-012 — Document Render-Path Naming Convention
lane: retro
legend: DX
```

Disposition

Documented the render-path naming rule explicitly in the @flyingrobots/bijou README and GUIDE: the base family name is the public component family, and a *Surface() companion means the same family on the composable surface path. The broader doctrine was already present across ADVANCED_GUIDE, migration docs, and design-system material, so this closes the remaining front-door POLA gap the note described.

Original Proposal

Legend: DX — Developer Experience

Idea

Document the implicit naming convention for string-output vs Surface-output component variants:

- box(content) → string (pipe-friendly)
- boxSurface(content) → Surface (composable in TUI apps)
- badge(content) → Surface only (no string variant)
- table(data) → string (mixed-mode, pipe-aware)

The pattern exists but is never stated explicitly. A developer browsing the API doesn't know why some components have a -Surface variant and others don't, or why badge() returns a Surface directly while box() returns a string.

Why

This came up in the DX audit as a minor POLA concern. The convention is sound (simpler output-only components are string-first; richer interactive components are Surface-native), but it reads as inconsistency when you're scanning exports without context.

A one-paragraph note in the bijou README's component section would resolve the confusion.

344

DX-013 — Motion Key Lifecycle Warning

docs/method/retro/DX_013-motion-key-warning.md

```
title: DX-013 — Motion Key Lifecycle Warning
lane: retro
legend: DX
```

Disposition

Shipped runtime warning for unstable motion key birth/death swaps in the motion reconciler, wired through the live runtime middleware, documented the stable-key rule, and covered the behavior with focused reconciler tests.

Original Proposal

Legend: DX — Developer Experience

Idea

motion({ key: 'id' }, ...) requires a stable key to track animation state across renders. If the key changes every render (e.g., using an index that shifts), the motion system silently creates new animations instead of interpolating. There is no warning.

Emit a runtime warning when a motion key appears for the first time on a frame that also has a different key disappearing — this is a strong signal that the developer is generating unstable keys.

Why

The DX audit flagged this as a non-obvious behavior. The motion system works correctly when keys are stable, but the failure mode (new animation every render = no visible interpolation) looks like "motion doesn't work" rather than "my key is wrong."

Effort

Small — track key birth/death across frames in the motion middleware.

345

DX-014 — Fix Root README Interactive Example

docs/method/retro/DX_014-readme-interactive-example-fix.md

```
title: DX-014 — Fix Root README Interactive Example
lane: retro
legend: DX
```

Disposition

Stale note. The root README interactive example already imports initDefaultContext() from @flyingrobots/bijou-node, calls it before run(app), and the surrounding docs now also teach startApp() as the preferred hosted Node bootstrap path.

Original Proposal

Legend: DX — Developer Experience

Idea

The interactive runtime example in the root README doesn't show the mandatory initDefaultContext() call. A developer copying the example into a new file gets:

```
Error: No default BijouContext has been set.
```

Add import { initDefaultContext } from '@flyingrobots/bijou-node' and initDefaultContext() to the example so it's copy-pasteable without hidden prerequisites.

Why

This is the #1 TTV barrier from the documentation audit. The scaffolded path handles it automatically, but anyone integrating into an existing project hits this wall on their first attempt.

Effort

Tiny — two lines added to the README example.

346

DX-015 — Perf Gradient as CI Benchmark

docs/method/retro/DX_015-perf-gradient-as-ci-benchmark.md

```
title: DX-015 — Perf Gradient as CI Benchmark
lane: retro
legend: DX
```

Disposition

Shipped a dedicated CI gradient benchmark lane using the existing wall-time harness. The repo now exposes npm run -s bench:ci:gradient for the fixed paint-gradient-rgb and diff-gradient scenario set, CI runs that lane on GitHub Actions, publishes the markdown summary to the step summary, and uploads the JSON report as an artifact. The docs make the contract explicit: this is an informational wall-time benchmark lane, not a heap/GC regression gate.

Original Proposal

Legend: DX — Developer Experience

Idea

The examples/perf-gradient demo runs in non-interactive mode and produces measurable output. It could be adapted into a CI benchmark that runs for a fixed number of frames and reports:

- Average view() time per frame
- Peak heap usage
- GC event count
- Total frame count in a fixed wall-clock window

A regression would show up as a measurable increase in view() time or heap pressure across commits.

Why

The perf demo proved that bijou's rendering bottleneck is in the pipeline (differ + encode), not in the app. Any future optimization (RE-008 byte-packed surface, RE-009 differ string fix) needs a benchmark to validate the improvement. Running the benchmark in CI prevents regressions from sneaking in.

Shape

A non-interactive test that: 1. Creates a gradient app 2. Feeds it N tick messages 3. Asserts view() time stays below a threshold 4. Reports heap delta

Could live in scripts/perf-bench.test.ts or as a GitHub Action that posts results as PR comments.

Risks

- Terminal rendering in CI is headless — stdout.write still works but there's no terminal to parse the ANSI. The differ will still run but output goes to /dev/null.
- Wall-clock benchmarks are noisy in CI. Use statistical methods (median of N runs) not single-run thresholds.

347

HT-001 — Notification Center Polish and Discoverability

docs/method/graveyard/legacy-backlog/HT-001-notification-center-polish-and-discoverability.md

Legend: HT — Humane Terminal

Idea

The shell-owned notification center exists, but it still needs product-level polish:

- clearer discoverability
- calmer footer cues
- better review/history ergonomics
- more intentional interaction treatment

Why

Notifications are part of the shell product, but they are not the next design-language cycle.

This work should stay explicitly on the backlog while doctrine is captured, then come back as a focused Humane Terminal cycle.

Status

Parked on the backlog while DL-001 captures more design language.

348

HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection

docs/method/graveyard/legacy-backlog/HT-005-promote-page-provided-layer-registry-and-shell-control-projection.md

Legend:

- HT — Humane Terminal

Depends on:

- HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection

Goal

Go beyond derived shell/page layer objects and promote a page-provided layer registry plus richer agent-facing control projection.

Why this is on the backlog

HT-004 should make the current shell layers much richer, but it still stops short of a true page-provided layer model.

This follow-on should explore:

- page-defined layer objects beyond modalKeyMap
- explicit control-projection surfaces for humans and agents
- richer shell introspection than active/underlying layer helpers
- a cleaner path from layer objects to footer, help, and automation/tooling surfaces

349

HT-006 — Allow pages to push notifications in createFramedApp

docs/method/retro/HT-006-page-level-notification-push.md

title: HT-006 — Allow pages to push notifications in createFramedApp
lane: retro
legend: HT

Disposition

Landed in release/v4.5.0. createFramedApp() now exposes emitFrameAction() and notify() so framed pages can emit shell-owned transient notifications without reaching into frame state, the page command wrapper preserves frame-scoped messages instead of re-wrapping them as page messages, focused app-frame coverage proves the runtime path, and the TUI guides now document the seam.

Original Proposal

Legend: HT — Humane Terminal

Problem

Pages in createFramedApp cannot push transient notifications. The pushNotification function operates on FrameModel.runtimeNotifications, but pages only return [PageModel, Cmd<Msg>[]] — they have no access to frame-level state.

Currently only the frame shell itself can push notifications (settings feedback, runtime issues). Page-level events like “No more thoughts in this session” have no way to surface as notifications.

Proposal

Add a push-notification FrameAction that pages can emit via a command:

```
// In page update:
return [model, [emitFrameAction({ type: 'push-notification', title: 'End of session',
↪  durationMs: 2000 })]];
```

Or expose a notify() command helper:

```
import { notify } from '@flyingrobots/bijou-tui';
return [model, [notify({ title: 'End of session', variant: 'TOAST', durationMs: 2000 })]];
```

Workaround

Render the notice as a positioned overlay via overlayFactory, which has access to the page model.

350

HT-001 — Notification Center Polish and Discoverability

docs/method/retro/HT_001-notification-center-polish-and-discoverability.md

```
title: HT-001 — Notification Center Polish and Discoverability
lane: retro
legend: HT
```

Disposition

Retired as stale backlog tracking. Repo truth already shipped the shell-owned notification center in createFramedApp(), including the standard Shift+N entrypoint, footer-level discoverability cue, command-palette entry, independent scroll/filter ownership, calmer drawer rhythm and structured review rows, shell-theme support, and logical-end localization follow-on work. The note's own 'park this until DL-001' status is also outdated relative to the completed DL/LX cycles and current app-frame tests.

Original Proposal

Legend: HT — Humane Terminal

Idea

The shell-owned notification center exists, but it still needs product-level polish:

- clearer discoverability
- calmer footer cues
- better review/history ergonomics
- more intentional interaction treatment

Why

Notifications are part of the shell product, but they are not the next design-language cycle.

This work should stay explicitly on the backlog while doctrine is captured, then come back as a focused Humane Terminal cycle.

Status

Parked on the backlog while DL-001 captures more design language.

351

HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection

docs/method/retro/HT_005-promote-page-provided-layer-registry-and-shell-control-projection.md

title: HT-005 — Promote Page-Provided Layer Registry and Shell Control Projection
lane: retro
legend: HT

Disposition

Landed in release/v4.5.0. createFramedApp() now lets pages publish a bounded layer registry for workspace and page-modal metadata through FramePage.layers (model), and the frame exports projectFrameControls() so footer/help/tooling all consume the same active-layer, underlying-layer, footer hint, and help-source projection. The shell still owns shell layers like settings, help, notifications, search, and quit confirm, but page-owned layer metadata is no longer limited to the older modalKeyMap seam.

Original Proposal

Legend:

- HT — Humane Terminal

Depends on:

- HT-004 — Promote Explicit Layer Objects and Richer Shell Introspection

Goal

Go beyond derived shell/page layer objects and promote a page-provided layer registry plus richer agent-facing control projection.

Why this is on the backlog

HT-004 should make the current shell layers much richer, but it still stops short of a true page-provided layer model.

This follow-on should explore:

- page-defined layer objects beyond modalKeyMap
- explicit control-projection surfaces for humans and agents
- richer shell introspection than active/underlying layer helpers
- a cleaner path from layer objects to footer, help, and automation/tooling surfaces

352

Upstream edge-overlay scrollbars and BCSS overflow handling

docs/method/graveyard/legacy-backlog/v5.0.0/HT_upstream-edge-overlay-scrollbars-and-bcss-overflow-handling.md

title: "Upstream edge-overlay scrollbars and BCSS overflow handling"
legend: HT
lane: inbox

GitHub Issue #54. Upstream edge-overlay scrollbar and BCSS overflow handling from XYPH into @flyingrobots/bijou-tui. Current viewport/focusArea/pager primitives require apps to hand-roll scrollbar overlay rendering, visibility fade states, and width-safe body composition. Requested: (A) overflow/scroll primitive with overlay vs gutter mode, visibility levels, BCSS-addressable track/thumb/arrows, composable with existing viewport/pager/focusArea; and/or (B) a pane-shell primitive owning header, body, optional right-edge rail, and width-safe chrome that doesn't reflow when the rail hides. Acceptance: edge-overlay scrollbar without dead gutter, ANSI-safe, BCSS-targetable, existing primitives stable unless new mode requested. XYPH has a working local implementation proving the pattern. See https://github.com/flyingrobots/bijou/issues/54 for full details.

353

Upstream edge-overlay scrollbars and BCSS overflow handling

docs/method/retro/HT_upstream-edge-overlay-scrollbars-and-bcss-overflow-handling.md

title: "Upstream edge-overlay scrollbars and BCSS overflow handling"
legend: HT
lane: v5.0.0

GitHub Issue #54. Upstream edge-overlay scrollbar and BCSS overflow handling from XYPH into @flyingrobots/bijou-tui. Current viewport/focusArea/pager primitives require apps to hand-roll scrollbar overlay rendering, visibility fade states, and width-safe body composition. Requested: (A) overflow/scroll primitive with overlay vs gutter mode, visibility levels, BCSS-addressable track/thumb/arrows, composable with existing viewport/pager/focusArea; and/or (B) a pane-shell primitive owning header, body, optional right-edge rail, and width-safe chrome that doesn't reflow when the rail hides. Acceptance: edge-overlay scrollbar without dead gutter, ANSI-safe, BCSS-targetable, existing primitives stable unless new mode requested. XYPH has a working local implementation proving the pattern. See https://github.com/flyingrobots/bijou/issues/54 for full details.

Completed (2026-04-14)

The upstreamed solution landed through Option A. viewport() and viewportSurface() now support explicit gutter versus overlay scrollbar ownership, so the rail can paint over the pane edge without reserving a dead width column. focusArea() and pager() now thread that mode through their string and surface paths, preserving previous gutter behavior by default and only switching when overlay mode is explicitly requested. focusArea also exposes BCSS-targetable thumb and track identities so shell-owned panes can style overflow chrome without hand-rolling ANSI-aware replacement logic in app code.

354

LX-007 — Service-Oriented Localization Adapters

docs/method/graveyard/legacy-backlog/LX-007-service-oriented-localization-adapters.md

Legend: LX — Localization and Bidirectionality

Idea

LX-006 proved the provider seam with a richer spreadsheet format:

- XLSX workbook exchange

The next likely slice is service-backed exchange on top of the same workbook/bundle contracts:

- Google Sheets
- maybe later other localization services

Why

Some real localization workflows need shared review surfaces, not just files on disk.

Those should still build on the existing Bijou workbook/bundle seam instead of introducing a second provider model.

Status

Backlog spawned by the retrospective for LX-006.

355

LX-009 — Localize Shell Help, Notification, and Directional Surfaces

docs/method/graveyard/legacy-backlog/LX-009-localize-shell-help-notification-and-directional-surfaces.md

Legend: LX — Localization and Bidirectionality

Idea

LX-008 proved the shell i18n seam in real product surfaces:

- frame-owned shell copy can come from catalogs
- DOGFOOD can load localized shell and app copy
- shell direction metadata can drive at least one visible mirrored behavior

The next slice should finish the most obvious remaining gaps:

- help overlay binding descriptions and grouping copy
- notification-center history/body strings and filters
- broader logical-direction behavior in shell-owned drawers and related chrome

Why

LX-008 intentionally stopped once the i18n seam was real.

That was the correct stopping point, but it leaves visible English islands and only partial direction-aware shell behavior. LX-009 should tighten those gaps before we claim the shell is broadly localized.

Status

Backlog spawned by the retrospective for LX-008.

356

LX-010 — Built-in i18n Catalog Loader

docs/method/graveyard/legacy-backlog/v4.5.0/LX-010-built-in-i18n-catalog-loader.md

Legend: LX — Localization and Bidirectionality

Problem

@flyingrobots/bijou-i18n requires users to manually load and pass in I18nCata log objects. The loadCatalog(catalog) method on the runtime accepts pre-constructed catalog data, but there is no built-in way to give bijou a path, URL, or dynamic import spec and have it resolve, load, and register catalogs automatically.

The filesystem loading utilities (readCatalogBundleFile, readExchangeWorkbook Directory, etc.) exist in @flyingrobots/bijou-i18n-tools-node, but they are opt-in tool-tier helpers, not a runtime-integrated loading seam. Users must orchestrate the entire load-parse-register pipeline themselves.

Why this matters

- New users expect createI18nRuntime({ locale: 'fr', catalogPath: '...' }) or equivalent to just work
- The current design forces boilerplate that every localized app repeats
- Without a built-in loader, lazy/dynamic catalog loading (load on locale switch) requires users to invent their own async orchestration
- The gap is more visible now that the i18n tools packages ship production-grade exchange adapters

Desired outcome

A user can create an i18n runtime with a loader strategy that handles catalog resolution, loading, and registration without manually wiring the tools pipeline. The loader should support:

- synchronous pre-loaded catalogs (current behavior, still supported)
- asynchronous file-based loading (Node.js)

- asynchronous dynamic import loading (bundler-friendly)
- lazy per-locale loading on locale switch

Non-goals

- Replacing the tools-tier adapters — those remain for batch processing, exchange workflows, and CLI tooling
- HTTP/fetch-based loading in core — that belongs in a platform adapter
- Automatic locale detection — that stays in the runtime adapter layer

Risks

- The loader abstraction must not pull Node.js APIs into the core bijou-i18n package — the loader contract should be a port, with platform-specific adapters in bijou-i18n-tools-node or similar
- Lazy loading introduces async state into a currently synchronous runtime — needs clear fallback/pending semantics

357

LX-010 — Built-in i18n Catalog Loader

docs/method/retro/LX-010-built-in-i18n-catalog-loader.md

title: "LX-010 — Built-in i18n Catalog Loader"
legend: LX
lane: retro

Completed on release/v5.0.0. Bijou already ships the built-in runtime loader seam through createI18nRuntimeAsync(), the loader option on createI18nRuntime(), and the preloadLocale() / setLocale() activation path in @flyingrobots/bijou-i18n. The Node file-backed adapter also already ships as createCatalogBundleFileLoader() in @flyingrobots/bijou-i18n-tools-node.

This closure pass makes that repo truth explicit: the canonical backlog no longer pretends the loader is missing, the package docs now show both dynamic import and file-backed loader paths, and tests/cycles/LX-010/ now proves the end-to-end locale-bundle activation path directly.

Original Proposal

Legend: LX — Localization and Bidirectionality

Problem

@flyingrobots/bijou-i18n requires users to manually load and pass in I18nCatalog objects. The loadCatalog(catalog) method on the runtime accepts pre-constructed catalog data, but there is no built-in way to give bijou a path, URL, or dynamic import spec and have it resolve, load, and register catalogs automatically.

The filesystem loading utilities (readCatalogBundleFile, readExchangeWorkbookDirectory, etc.) exist in @flyingrobots/bijou-i18n-tools-node, but they are opt-in tool-tier helpers, not a runtime-integrated loading seam. Users must orchestrate the entire load-parse-register pipeline themselves.

Why this matters

- New users expect createI18nRuntime({ locale: 'fr', catalogPath: '...' }) or equivalent to just work
- The current design forces boilerplate that every localized app repeats
- Without a built-in loader, lazy/dynamic catalog loading (load on locale switch) requires users to invent their own async orchestration
- The gap is more visible now that the i18n tools packages ship production-grade exchange adapters

358

LX-012 — DOGFOOD i18n Debt Inventory

docs/method/retro/LX-012-dogfood-i18n-debt-inventory.md

title: "LX-012 — DOGFOOD i18n Debt Inventory"
legend: LX
lane: retro

Completed on cycle/lx-012-dogfood-i18n-debt-inventory.

DOGFOOD now has a deterministic source inventory for remaining localizable UI strings. The inventory groups raw localizable strings by source surface, filters out nonlocalizable ids, file paths, import paths, and cataloged fallback calls, and enforces the current count through npm run dogfood:i18n:debt. The release readiness plan includes the new ratchet so future DOGFOOD work cannot silently increase the counted localization debt.

Original Proposal

LX-011 added the first DOGFOOD locale preference ratchet, but most visible DOGFOOD copy can still enter the app as raw English strings in pages, previews, and examples.

Problem:

- DOGFOOD now has a localization seam, but no counted inventory of the remaining localizable UI copy.
- New surfaces can add English strings without making the debt visible.
- Reviewers cannot distinguish intentionally literal examples from user-facing copy that should move through catalogs.

Desired outcome:

- Add a deterministic DOGFOOD i18n debt inventory grouped by surface area.
- Count localizable UI strings and ratchet the count downward or hold it steady.
- Separate true nonlocalizable identifiers, code examples, and fixture literals from visible application copy.
- Make the report useful in code review without relying on rendered screenshots or fragile stdout snapshots.

Why ASAP:

- The Settings locale selector made localization visible to users.
- The next DOGFOOD documentation and Blocks work will otherwise keep adding English-only copy.
- A counted debt ledger lets catalog expansion happen incrementally without pretending the whole app is translated.

Hill:

A maintainer can run one deterministic check and see which DOGFOOD surfaces still contain localizable English, what changed, and whether the branch increased or reduced the debt.

Non-goals:

- Do not translate all DOGFOOD copy in this cycle.
- Do not block markdown source documentation that is intentionally prose.
- Do not infer localization coverage by scraping terminal render output.

359

LX-007 — Service-Oriented Localization Adapters

docs/method/retro/LX_007-service-oriented-localization-adapters.md

```
title: LX-007 — Service-Oriented Localization Adapters
lane: retro
legend: LX
```

Disposition

Landed in release/v4.5.0 as a provider-neutral async service seam in @flyingrobots/bijou-i18n-tools. The package now exports revision-aware service snapshots and adapter ports for translation workbooks and catalog bundles, plus push/pull helpers that build directly on the existing workbook and bundle exchange contracts. Concrete Google Sheets or other remote-service clients still belong outside the pure tooling package, but the service-backed workflow surface is no longer missing.

Original Proposal

Legend: LX — Localization and Bidirectionality

Idea

LX-006 proved the provider seam with a richer spreadsheet format:

- XLSX workbook exchange

The next likely slice is service-backed exchange on top of the same workbook/bundle contracts:

- Google Sheets
- maybe later other localization services

Why

Some real localization workflows need shared review surfaces, not just files on disk.

Those should still build on the existing Bijou workbook/bundle seam instead of introducing a second provider model.

Status

Backlog spawned by the retrospective for LX-006.

360

LX-009 — Localize Shell Help, Notification, and Directional Surfaces

docs/method/retro/LX_009-localize-shell-help-notification-and-directional-surfaces.md

title: LX-009 — Localize Shell Help, Notification, and Directional Surfaces
lane: retro
legend: LX

Disposition

Landed in release/v4.5.0. Shell help now localizes its default group label, frame-owned group names, and shell-owned key descriptions through the frame catalog instead of leaking English strings from static key maps. The notification-center drawer now localizes its summary rows, filter copy, current stack label, history header/empty states, and action label through the same shell i18n seam. The visible logical-direction behavior from LX-008 remains the active direction contract for shell drawers and related chrome, so the obvious remaining English islands are no longer a live LX backlog item.

Original Proposal

Legend: LX — Localization and Bidirectionality

Idea

LX-008 proved the shell i18n seam in real product surfaces:

- frame-owned shell copy can come from catalogs
- DOGFOOD can load localized shell and app copy
- shell direction metadata can drive at least one visible mirrored behavior

The next slice should finish the most obvious remaining gaps:

- help overlay binding descriptions and grouping copy

- notification-center history/body strings and filters
- broader logical-direction behavior in shell-owned drawers and related chrome

Why

LX-008 intentionally stopped once the i18n seam was real.

That was the correct stopping point, but it leaves visible English islands and only partial direction-aware shell behavior. LX-009 should tighten those gaps before we claim the shell is broadly localized.

Status

Backlog spawned by the retrospective for LX-008.

361

RE-011 — Eliminate Per-Frame Allocation in Render Pipeline

docs/method/graveyard/legacy-backlog/inbox/RE-011-pipeline-per-frame-allocation.md

Legend: RE — Runtime Engine

Problem

pipeline.execute() in packages/bijou-tui/src/pipeline/pipeline.ts allocates a new chain[] array every frame by spreading all stage middleware into a flat list. At 60fps this is 60 array allocations per second with no reuse.

Fix

Cache the flattened chain array and only rebuild when middleware is added (track a dirty flag on use()). The chain is static after app initialization in most apps.

Effort

Small — add a dirty flag and cached array.

362

RE-012 — Pipeline Observability Hooks

docs/method/graveyard/legacy-backlog/cool-ideas/RE-012-pipeline-observability-hooks.md

Legend: RE — Runtime Engine

Idea

Expose per-stage timing hooks in the render pipeline so apps and middleware can measure Layout, Paint, PostProcess, Diff, and Output phases individually.

```
run(app, {
  configurePipeline(pipeline) {
    pipeline.onStageComplete((stage, durationMs) => {
      perfOverlay.recordPhase(stage, durationMs);
    });
  },
});
```

Why

The perf-gradient demo proved that 92% of frame time is spent in the pipeline (diff + ANSI encode + stdout.write), but we could not measure the breakdown from the app side. configurePipeline can only append middleware after the default stages — it cannot wrap them for timing.

This blocks: - DX-006 from showing a full phase breakdown - Any app from diagnosing whether their bottleneck is layout, paint, or diff

Shape

Either: 1. A callback-based hook (onStageComplete) on the pipeline 2. Timing metadata added to RenderState.data by the pipeline itself 3. A wrapStage(stag e, fn) API that lets middleware run before AND after a stage's default middleware

Option 3 is the most general but also the most complex.

363

RE-013 — Memoize Workspace Layout Tree

docs/method/graveyard/legacy-backlog/cool-ideas/RE-013-memoize-workspace-layout-tree.md

Legend: RE — Runtime Engine

Idea

buildWorkspaceLayoutTree() creates new SurfaceLayoutNode objects on every mouse event, even when the terminal size, page layout, and tab count haven't changed. The tree structure only changes on resize, tab switch, or pane layout change.

Memoize the tree against the inputs that determine its shape (cols, rows, active page, pane layout hash). Return the cached tree when inputs match. This eliminates ~20 object allocations per mouse event for a typical 2-tab, 2-pane layout.

Why

The perf-gradient demo showed that view() time is only ~2ms, but every mouse event also runs resolveFrameMouseRuntimeLayouts() which calls buildWorkspaceLayoutTree(). In a typical app with 60fps rendering and mouse tracking, this is 60+ unnecessary tree builds per second.

Effort

Small — hash the inputs, compare, return cached tree.

364

RE-014 — Design Doctrine

docs/method/graveyard/legacy-backlog/cool-ideas/RE-014-buffer-holds-facts-doctrine.md

Legend: RE — Runtime Engine

Idea

During the RE-007 buffer migration, James articulated a principle:

> “Keep buffered shell commands as plain discriminated data and interpret them explicitly. Use a handler table if you want to avoid a giant switch. The buffer should hold facts, not code.”

This is a design doctrine that should be captured in the runtime engine’s architectural documentation. It applies beyond shell commands — any buffer, queue, or event channel in the system should carry data (what happened) not behavior (how to respond).

Why

The initial design proposed classes with apply() methods — each command knowing how to execute itself. The correction was: commands are facts about intent. Interpretation is separate. Facts are serializable, testable, inspectable, replayable. Behavior is contextual.

This principle: - Enables command logging and replay for debugging - Keeps the handler table as the single place to audit all side effects - Makes the command vocabulary inspectable without executing anything - Aligns with event-sourcing and CQRS patterns

Shape

Add a section to docs/ARCHITECTURE.md or a dedicated docs/invariants/buffer-holds-facts.md that states the principle and explains when it applies (command buffers, event channels, message queues) and when it doesn’t (hot-path render loops where indirection costs matter).

365

RE-017 — Frame does not fill terminal background with surface.primary.bg

docs/method/retro/RE-017-frame-does-not-fill-surface-background.md

title: RE-017 — Frame does not fill terminal background with surface.primary.bg
lane: retro

Disposition

Fixed in the v4.4.1 release line and already shipped on main at tag v4.4.1. The frame now fills the body and shell chrome with the active shell background, the stock shellThemes path exercises that end to end, and the RE-017 DOGFOOD cycle coverage plus the 4.4.1 release docs are the source of truth now.

Original Proposal

Legend: RE — Runtime Engine

Problem

When a custom theme defines surface.primary.bg (e.g., #2d1922 for a dark plum background), the framed shell does not fill the terminal background with that color. Empty cells in pane content retain the terminal's default background instead of the theme's surface background.

The frame correctly reads surface('primary').bg for header tab color derivation (in deriveActiveHeaderTabToken), but it does not fill the pane area's empty cells with the surface background. This means custom themes that rely on a non-default background color appear broken — only styled text shows the theme colors while the rest of the screen shows the terminal's default dark background.

Expected behavior

The frame should fill all empty/unoccupied cells in the viewport with ctx.surface('primary').bg so that custom themes with non-default backgrounds render correctly across the entire terminal.

Reproduction

```
import { createBijou, tv } from '@flyingrobots/bijou';
import { createFramedApp, run } from '@flyingrobots/bijou-tui';

const theme = {
  name: 'custom',
  // ... standard tokens ...
  surface: {
    primary: { hex: '#ffffff', bg: '#2d1922' },  // dark plum background
    // ...
  },
};

const ctx = createBijou({ runtime, io, style, theme });
const app = createFramedApp({ pages: [...] });
await run(app, { ctx });
// Expected: entire terminal has #2d1922 background
// Actual: only styled text areas show theme colors; empty areas show terminal default
```

Workaround

Write the ANSI background escape code directly to stdout before run() starts, and clear the screen:

```
process.stdout.write(`\x1b[48;2;45;25;34m\x1b[2J`);
await run(app, { ctx });
```

This survives bijou's alt screen enter but may flash briefly.

366

RE-018 — Hex Color Strings As The Canonical fg/bg Representation

docs/method/graveyard/RE-018-hex-color-string-as-canonical-fg-bg.md

title: RE-018 — Hex Color Strings As The Canonical fg/bg Representation
lane: graveyard

Disposition

Retired from bad-code on release/v4.5.0 in commit 69291fa. This branch fixed the concrete hot-path smell: packed surfaces now preserve fgRGB / bgRGB through reads and masked writes, first-party surface/TUI components prefer pre-parsed RGB over reparsing strings, and the string path is now much closer to a compatibility boundary than the internal default. The remaining deeper evolution is no longer an immediate defect; track further API-shaping work under docs/method/backlog/cool-ideas/RE-019-theme-token-color-cache.md and do cs/method/retro/RE-020-typed-color-representation.md.

Original Proposal

Legend: RE — Runtime Engine

Problem

Throughout the bijou API surface, foreground and background colors are represented as '#rrggbb' strings on the Cell interface:

```
interface Cell {
  fg?: string;  // '#rrggbb'
  bg?: string;  // '#rrggbb'
  ...
}
```

Every single surface.set(x, y, { fg, bg, ... }) call has to parse these strings into bytes before writing to the packed buffer. The parse happens via

inlineHexRGB in packages/bijou/src/ports/surface.ts (6 charCodeAts + 6 hexD lookups + bit shuffles per color). For a paint-heavy frame that sets every cell of a 220×58 surface with both fg and bg, that's **25,520 hex parses per frame** — and the theme vocabulary is ~15 unique colors.

24,505 of those parses are wasted work because they recompute the same bytes over and over for the same theme colors.

Evidence

- inlineHexRGB is called from encodeCellIntoBuf at packages/bijou/src/ports/surface.ts:312-321 for every set() call that provides fg or bg.
- The RE-017 audit showed broad 30-113% regressions across every rendering scenario in the bench after RE-008 merged. Paint-heavy scenarios (paint-set-hex-palette, surface.paint.medium, pure dogfood.render.*) regressed hardest. Paint scenarios that use setRGB (which takes pre-parsed bytes) are less affected in principle, but are still dominated by the paint cost because most components still call surface.set({fg, bg}).
- Over 25 callsites in packages/bijou/src/core/components/ import parseHex and parse theme colors manually — the "fast" path is already the exception, not the norm. See packages/bijou/src/core/components/table-v3.ts:87, preference-list.ts:134, surface-text.ts.

Why This Is Bad

1. **Repeated work.** The same hex string parses to the same bytes every time. There is no state in a hex string; no reason to re-parse it.
2. **API boundary at the wrong layer.** Hex strings are a *user input* format. They belong at the edge of the system where a human types #ff8800 into a theme file. They do not belong in the hot path that runs thousands of times per frame.
3. **Encourages callers to bypass the safe API.** Components have already started caching their own parsed bytes to avoid the overhead (preference-list.ts caches parseHex results in locals). This is a smell — it means the safe API is too slow to use safely, so callers build their own shortcuts.

Fix Direction

Two options that compose:

1. **Theme-token color cache.** When a theme is loaded, pre-parse every hex color into a Uint8Array (3 bytes). Expose a resolved theme that holds bytes, not strings. Components that pull colors from theme tokens get pre-parsed bytes for free. See the related cool idea RE-019-theme-token-color-cache.md.
2. **Typed color representation at the API boundary.** Introduce a branded ColorRef type (an opaque wrapper around a packed uint32 or a Uint8Array) and make Cell.fg/Cell.bg accept ColorRef | string. Hex strings stay supported for

human-written code, but library internals and components thread ColorRef end to end. Hex parsing happens only at the string → ColorRef boundary, once. See cool idea RE-020-typed-color-representation.md.

Both preserve API compatibility for user code while making the hot path allocation-free.

Impact

Measuring against the RE-017 post-merge baseline:

- paint-set-hex-palette at 220×58: 634 µs/frame, 12,760 cells, ~2 hex parses per cell (fg + bg). Roughly ~25,000 parses / frame.
- If each parse is ~200 ns (conservative), that's 5 ms of parse work per frame, dwarfing the 634 µs total. But the actual parse cost is much lower (maybe 20-50 ns each), making total parse cost ~500 µs-1 ms per frame — still a significant fraction of frame time on paint-heavy workloads.

Confirming the magnitude is a task in RE-017 (I-0e). This entry captures the structural smell regardless of the exact number.

Tests To Add

- Paint scenario that exercises surface.set({fg, bg}) with a small fixed palette — confirms the per-cell parse cost.
- Equivalent scenario using setRGB with pre-parsed bytes — establishes the theoretical floor.
- Property test: surface.set({fg: theme.primary}) should not allocate new strings on the hot path (heap snapshot diff).

367

RE-019 — Surface dirtyWords Bitmap Serves The Wrong Master

docs/method/retro/RE-019-dirtyWords-serves-lazy-cache-not-render-dirty.md

title: RE-019 — Surface `dirtyWords` Bitmap Serves The Wrong Master
lane: retro

Disposition

Fixed earlier in the runtime engine: PackedSurface now carries distinct dirty Words and renderDirtyWords responsibilities, and render dirtiness is tracked separately from lazy decode state in packages/bijou/src/ports/surface.ts. This is no longer an active bad-code item.

Original Proposal

Legend: RE — Runtime Engine

Problem

packages/bijou/src/ports/surface.ts:456 declares a per-cell dirty bitmap:

```
| const dirtyWords = new Uint32Array(Math.ceil(size / 32));
```

It is set on every set() and setRGB() call (via markDirty) and used by ensure Clean() in get() to lazily re-decode the Cell object from the packed buffer when the buffer has been mutated since the last get().

The name “dirty” strongly suggests this tracks “cells modified since last render” — which is exactly what the renderer would want for efficient diffing. But it does not. It tracks “cells where the lazily-cached Cell object is stale relative to the buffer”. Two different meanings, one shared name, one shared data structure.

Why This Is Bad

1. **Misleading name.** Anyone reading dirtyWords will assume it's for render tracking, not lazy cache invalidation. The RE-017 audit caught exactly this confusion — the renderer does not use the bitmap, even though the data is exactly what it needs.
2. **Serves a vestigial cache.** The cells: Cell[] array that the bitmap protects is a holdover from the pre-RE-008 cell-based surface API. In the byte-packed world, the buffer is the source of truth; the Cell[] projection exists only to support legacy surface.get() callers and the legacy renderDiffCells path. Most hot-path code now uses setRGB and byte comparisons, which bypass the cache entirely. The bitmap is protecting a cache that is increasingly irrelevant.
3. **Blocks the render-dirty optimization.** Task II-1 in RE-017 wants to reuse this exact bitmap as the render-dirty signal ("which cells were painted this frame, skip the rest on the diff scan"). Reusing the existing bitmap conflicts with its current lazy-cache purpose — we'd have to clear it at end of render, but get() would then re-decode unnecessarily on the next frame.

Fix Direction

Two options, either works:

1. **Drop the Cell[] projection from the packed hot path.** Make get() decode on demand from the buffer every time, with no cache. Then the dirtyWords bitmap is freed up to mean "cells modified since last render". renderDiffPacked walks set bits. markAllClean() is called at end of render. Clean and consistent.
2. **Split into two bitmaps.** cacheDirtyWords for the lazy Cell[] invalidation (current behavior) and renderDirtyWords for the render-dirty tracking (new behavior). More memory, but preserves the Cell[] cache for any callers that still benefit from it.

Option 1 is simpler and matches the direction RE-017 is going. Before picking, we should measure how often get() is actually called in hot paths — if it's rare, the lazy cache is buying nothing and should be removed. If it's common, we need Option 2 or a different solution.

Evidence

- Current bitmap definition + usage: packages/bijou/src/ports/surface.ts:456-491, 514-540.
- Render path does not consult it: packages/bijou/src/core/render/differ.ts:401 (renderDiffPacked walks all cells every frame with byte comparison; no bitmap check).
- Task II-1 in docs/perf/RE-017-byte-pipeline.md proposes to repurpose it for render-dirty tracking.

Tests To Add

- Once repurposed: renderDiffPacked should visit zero cells on a frame where nextSurface.clear() was called and no set() / setRGB followed.
- Once repurposed: after a set() at (5, 3), the next renderDiffPacked call should visit (5, 3) and skip all other cells (assuming no other mutations).
- Regression test: get(x, y) after a set(x, y) still returns the updated cell (decode path still works without the cache).

368

RC-019 — Pre-Parse Theme Token Colors Into Bytes

docs/method/retro/RE-019-theme-token-color-cache.md

title: RE-019 — Pre-Parse Theme Token Colors Into Bytes
lane: retro
legend: RE

Disposition

Completed on release/v4.5.0. Theme resolution now preserves and caches pre-parsed fgRGB / bgRGB bytes through the live TokenGraph path instead of dropping them after createResolved(). Accessor-returned tokens from ctx.semantic(), ctx.surface(), ctx.border(), ctx.status(), and ctx.ui() now carry reusable RGB bytes, live tokenGraph.set() / import() updates repopulate those bytes, and repeated token reads reuse cached per-hex and per-token resolution instead of reparsing the same theme colors in hot render paths.

Original Proposal

Legend: RE — Runtime Engine

Idea

When a theme is loaded, walk all its token values and pre-parse every hex color ('#rrggbb') into a reusable 3-byte representation. Expose a resolved theme where each token's color is a Uint8Array of length 3 (R, G, B), not a string.

Then:

- surface.set({ fg: theme.tokens.primary.fg, bg: ... }) — if the color value is already bytes, skip inlineHexRGB entirely and write straight to the packed buffer.
- Components that need RGB values for other reasons (gradient interpolation, contrast checks) get them for free without parsing.

The theme stays user-authored as hex strings (human-friendly, DTCG-interop-friendly), but at runtime the system works in bytes.

Why

Per the related bad-code entry RE-018-hex-color-string-as-canonical-fg-bg.md, every surface.set({ fg: '#xxxxxx' }) currently hex-parses the string inline. In a paint-heavy frame, that's thousands of redundant parses — the same theme colors get parsed over and over. The theme token set is bounded and small (~15-50 unique colors); parsing them once at theme load time and reusing the bytes eliminates the hot-path parse cost for all theme-driven paints.

Scope / Shape

```
// core/theme/resolve.ts additions

interface ResolvedToken {
  // existing fields...
  fgBytes?: Uint8Array;  // pre-parsed, 3 bytes [R, G, B]
  bgBytes?: Uint8Array;
}

function resolveTheme(theme: Theme): ResolvedTheme {
  // walk tokens, call parseHex once per unique color, store bytes
}
```

surface.set() gains a fast path: if the fg field is a Uint8Array, skip inline HexRGB and copy directly. Similarly for bg. Type union: fg?: string | Uint8Ar ray.

Optional next step: bake a ColorRef branded type over the bytes so the API doesn't expose raw Uint8Array to users. See the related cool idea RE-020-type d-color-representation.md.

Why This Is Cool

1. **Zero risk to the public API.** Hex strings still work. This is purely an optimization under the hood.
2. **Self-healing.** Any code that already uses theme tokens benefits automatically; no caller rewrites.
3. **Measurable.** The RE-017 bench has a paint-set-hex-palette scenario designed exactly to measure this. We can land the change and see the impact in a single bench run.
4. **Composable with RE-017's byte-pipeline work.** This removes hex parsing from the paint path; II-4 removes string work from the diff path. Together they close the loop on "zero string allocation during rendering".

Prior Questions

- Does the resolver need to know about gradients? (DTCG gradient tokens have arrays of stops — each stop needs byte-resolving.)
- Should the cache be per-theme or global? (Per-theme is safer since theme reload should invalidate.)
- Should we export the byte form publicly, or keep it internal? (Start internal; promote later if users want it.)

Related

- RE-018 (bad-code) — the hex string parsing smell this fixes.
- RE-017 / docs/perf/RE-017-byte-pipeline.md — the cycle where this would be measured and landed.
- RE-020 (cool idea) — ColorRef typed representation, a follow-on that bakes this into the public API.

369

RE-020 — Typed Color Representation (ColorRef Branded Type)

docs/method/graveyard/legacy-backlog/v5.0.0/RE-020-typed-color-representation.md

Legend: RE — Runtime Engine

Idea

Replace raw fg?: string / bg?: string in Cell (and everywhere downstream) with a branded ColorRef type that can be either a string (for user code) or a pre-parsed byte representation (for internal hot paths):

```
/**
 * Opaque color reference. Either a user-written hex string
 * (lazy; parsed on first use) or a pre-resolved bytes object
 * (eager; zero parse cost on the hot path).
 */
export type ColorRef = string & { readonly __color: unique symbol }
                     | ResolvedColor;

export interface ResolvedColor {
  readonly __color: unique symbol;
  readonly r: number;
  readonly g: number;
  readonly b: number;
}

export function color(hex: string): ColorRef {
  return hex as ColorRef;  // lazy
}

export function resolveColor(ref: ColorRef): ResolvedColor {
  if (typeof ref === 'string') {
    const [r, g, b] = parseHex(ref)!;
    return { __color: undefined as never, r, g, b };
  }
  return ref;
}
```

Cell becomes:

```
interface Cell {
  fg?: ColorRef;
  bg?: ColorRef;
  // ...
}
```

surface.set() handles both branches: if the color is a ResolvedColor, write bytes directly; if it's a string, parse inline (same as today).

Theme resolution pre-produces ResolvedColor objects for every token color (see related idea RE-019-theme-token-color-cache.md), so hot-path components automatically get the fast path by virtue of pulling from the theme.

Why

Hex strings are a **user input format** — great for humans typing #ff8800 into a theme JSON or calling a styling API ad-hoc. But they're a terrible internal representation because they force a parse on every use. Today, fg: '#rrggbb' in Cell forces that parse to happen per-cell, per-frame, in the hottest loop in the system.

ColorRef lets us keep hex strings for ergonomic user code while threading a parsed representation through the hot path. The type system makes it clear which layer you're at.

Migration Path

1. Land RE-019-theme-token-color-cache.md first — that's the cheap no-breaking-changes version.
2. Add ColorRef as a type alias that's currently just string, update Cell to use it. This is a rename, no runtime change.
3. Add a ResolvedColor case to the union. Update surface.set to handle it.
4. Update theme resolver to produce ResolvedColor objects.
5. Components that build cells directly (not via theme tokens) can opt in by calling resolveColor(hex) once at setup time and reusing the result.
6. Eventually, string is still valid for user code but components use Resolved Color internally.

No breaking changes at any step.

Why This Is Cool

- **Ergonomic at the edge, fast at the core.** The right abstraction at the right layer.
- **Type-directed optimization.** The type system tells you when you're on the slow path vs the fast path.
- **Composable with other optimizations.** Gradients can store ResolvedColor stops, animations can interpolate in byte space without string conversions.
- **Pairs with the RE-017 byte pipeline.** The byte pipeline eliminates string work from the diff path; this eliminates string work from the paint path. Together they close the loop.

Related

- RE-018 (bad-code) — the hex string smell.
- RE-019 (cool idea) — the simpler precursor: theme-token byte cache without exposing a new type.
- DTCG interop at packages/bijou/src/core/theme/dtcg.ts — this must continue to round-trip to hex strings for export.

370

RE-020 — Typed Color Representation (ColorRef Branded Type)

docs/method/retro/RE-020-typed-color-representation.md

```
title: RE-020 — Typed Color Representation (`ColorRef` Branded Type)
lane: retro
legend: RE
```

Disposition

Implemented on release/v5.0.0 as a compatibility-first ColorRef landing. Cell.fg / Cell.bg now accept ColorRef, the runtime exports ResolvedColor, resolveColor(), tryResolveColor(), colorHex(), and colorRgb(), and packed surface writes can consume either user-authored hex strings or eager resolved color objects without reparsing on the hot path. Theme-token byte caching from RE-019 remains the automatic fast path, while components and adapters can now thread typed resolved colors through paint and output code explicitly.

Original Proposal

Legend: RE — Runtime Engine

Idea

Replace raw fg?: string / bg?: string in Cell (and everywhere downstream) with a branded ColorRef type that can be either a string (for user code) or a pre-parsed byte representation (for internal hot paths):

```
/**
 * Opaque color reference. Either a user-written hex string
 * (lazy; parsed on first use) or a pre-resolved bytes object
 * (eager; zero parse cost on the hot path).
 */
export type ColorRef = string & { readonly __color: unique symbol }
                     | ResolvedColor;

export interface ResolvedColor {
```

```
  readonly __color: unique symbol;
  readonly r: number;
  readonly g: number;
  readonly b: number;
}

export function color(hex: string): ColorRef {
  return hex as ColorRef;  // lazy
}

export function resolveColor(ref: ColorRef): ResolvedColor {
  if (typeof ref === 'string') {
    const [r, g, b] = parseHex(ref)!;
    return { __color: undefined as never, r, g, b };
  }
  return ref;
}
```

Cell becomes:

```
interface Cell {
  fg?: ColorRef;
  bg?: ColorRef;
  // ...
}
```

surface.set() handles both branches: if the color is a ResolvedColor, write bytes directly; if it's a string, parse inline (same as today).

Theme resolution pre-produces ResolvedColor objects for every token color (see related idea RE-019-theme-token-color-cache.md), so hot-path components automatically get the fast path by virtue of pulling from the theme.

Why

Hex strings are a **user input format** — great for humans typing #ff8800 into a theme JSON or calling a styling API ad-hoc. But they're a terrible internal representation because they force a parse on every use. Today, fg: '#rrggbb' in Cell forces that parse to happen per-cell, per-frame, in the hottest loop in the system.

ColorRef lets us keep hex strings for ergonomic user code while threading a parsed representation through the hot path. The type system makes it clear which layer you're at.

371

RE-021 — Frame Owns the Pump

docs/method/graveyard/legacy-backlog/v5.0.0/RE-021-frame-owns-the-pump.md

Legend: RE — Runtime Engine

Idea

Invert the runtime/frame relationship so createFramedApp owns the render loop, timing, and frame budget — not just the TEA model/view mapping. The frame becomes the "batteries included" entry point that handles everything: input acquisition, update dispatch, view rendering, diffing, performance measurement, and frame budget management.

Raw App + run() stays as the low-level TEA contract for users who want full control. The framed app is the path for everyone else.

Current architecture

```
runtime (owns loop, surfaces, differ, IO)
  → App.update(msg, model)    ← frame returns [model, cmds]
  → App.view(model)           ← frame returns Surface
  → renderDiff(current, target, io, style)
```

The runtime owns timing and the differ. The frame is a plain App that maps model → Surface. It has no visibility into render timing, frame budget, or memory pressure.

Proposed architecture

```
FramedApp (owns loop, timing, perf stats, frame budget)
  → acquires input from IO
  → page.update(msg, model)
  → measures view() wall time
  → renderDiff with timing
  → perf overlay reads its own bookkeeping
  → frame budget: skip non-essential overlays if overrunning
```

Why

- **Perf overlay is natural**: the frame knows render time, FPS, memory because it measures them. No message threading or capability injection.
- **Frame budget management**: the frame can skip expensive overlays, reduce animation fidelity, or batch notification renders when behind.
- **Simpler app authoring**: users write pages, not apps. The frame handles everything they'd otherwise copy from examples.
- **Pipeline observability**: update phase, view phase, diff phase, and output phase can all be timed and exposed — answering DX-006's open question about native phase timing.

What stays the same

- App interface — unchanged, still the TEA contract
- run(app) — unchanged, still boots a raw TEA app
- Component APIs — unchanged, still pure functions
- Scenario interface — unchanged, bench harness still works

What changes

- createFramedApp returns a self-running object (not App)
- New entry point: await framedApp.run({ ctx }) or await runFramedApp(options)
- The runtime's event loop, surface management, and differ invocation move into the framed app (or a shared internal engine it owns)
- Frame model gains timing fields: renderTimeMs, viewTimeMs, diffTimeMs
- Built-in perf overlay becomes trivial (reads frame's own timing state)

Open questions

- Should the framed app delegate to run() internally (composition) or replace it (fork)? Composition is safer but may over-abstract.
- How does the dogfood app's landing page (custom App wrapping a FramedApp) work? It currently calls explorer.update() and explorer.view() directly.
- Should frame budget management be automatic or opt-in?
- Test story: framed apps currently test via the App interface. If the frame owns the loop, testing needs a different harness.

Related

- DX-006 Debug Performance Overlay — becomes trivial once the frame owns timing

- RE-017 byte pipeline — the zero-alloc differ stays, frame just orchestrates it
- The soak runner rewrite (this session) proved the framed app pattern works well for real apps

372

RE-021 — Frame Owns the Pump

docs/method/retro/RE-021-frame-owns-the-pump.md

Legend: RE — Runtime Engine

Idea

Invert the runtime/frame relationship so createFramedApp owns the render loop, timing, and frame budget — not just the TEA model/view mapping. The frame becomes the "batteries included" entry point that handles everything: input acquisition, update dispatch, view rendering, diffing, performance measurement, and frame budget management.

Raw App + run() stays as the low-level TEA contract for users who want full control. The framed app is the path for everyone else.

Completed (2026-04-14)

Landed on release/v5.0.0 as a composition-based runner instead of a runtime fork. createFramedApp() now returns a self-running framed shell with app.run(...), runFramedApp(...) provides the one-call hosted entry point, and the framed runner delegates to the shared runtime while folding committed pipeline timings back into the frame model.

Repo truth after the landing:

- createFramedApp() remains App-compatible for tests and wrappers, but now also owns the hosted runner path
- runFramedApp(options, runOptions) is the batteries-included shell entry point
- framed runners default mouse input on for retained shell hit-testing
- FrameModel now carries frameTimeMs, viewTimeMs, diffTimeMs, frameBudgetMs, and frameOverBudget
- the frame budget currently drives shell telemetry and view-state decisions; the shared runtime loop still owns diff/output execution through composition

Validation included targeted runtime/app-frame coverage, npm run lint, npm run docs:inventory, and a full npm run release:readiness pass.

Current architecture

```
runtime (owns loop, surfaces, differ, IO)
  → App.update(msg, model)    ← frame returns [model, cmds]
  → App.view(model)           ← frame returns Surface
  → renderDiff(current, target, io, style)
```

The runtime owns timing and the differ. The frame is a plain App that maps model → Surface. It has no visibility into render timing, frame budget, or memory pressure.

Proposed architecture

```
FramedApp (owns loop, timing, perf stats, frame budget)
  → acquires input from IO
  → page.update(msg, model)
  → measures view() wall time
  → renderDiff with timing
  → perf overlay reads its own bookkeeping
  → frame budget: skip non-essential overlays if overrunning
```

Why

- **Perf overlay is natural**: the frame knows render time, FPS, memory because it measures them. No message threading or capability injection.
- **Frame budget management**: the frame can skip expensive overlays, reduce animation fidelity, or batch notification renders when behind.
- **Simpler app authoring**: users write pages, not apps. The frame handles everything they'd otherwise copy from examples.
- **Pipeline observability**: update phase, view phase, diff phase, and output phase can all be timed and exposed — answering DX-006's open question about native phase timing.

What stays the same

- App interface — unchanged, still the TEA contract
- run(app) — unchanged, still boots a raw TEA app
- Component APIs — unchanged, still pure functions
- Scenario interface — unchanged, bench harness still works

What changes

- createFramedApp returns a self-running object while remaining Ap p-compatible
- New entry point: await framedApp.run({ ctx }) or await runFramedApp(options)

- The runtime's event loop, surface management, and differ invocation stay in the shared runtime, with the framed shell composing that engine
- Frame model gains timing fields: frameTimeMs, viewTimeMs, diffTimeMs
- Built-in perf overlay becomes trivial (reads frame's own timing state)

Open questions

- Should the framed app delegate to run() internally (composition) or replace it (fork)? Composition is safer but may over-abstract.
- How does the dogfood app's landing page (custom App wrapping a FramedApp) work? It currently calls explorer.update() and explorer.view() directly.
- Should frame budget management be automatic or opt-in?
- Test story: framed apps currently test via the App interface. If the frame owns the loop, testing needs a different harness.

Related

- DX-006 Debug Performance Overlay — becomes trivial once the frame owns timing
- RE-017 byte pipeline — the zero-alloc differ stays, frame just orchestrates it
- The soak runner rewrite (this session) proved the framed app pattern works well for real apps

373

RE-022 — Public PackedSurface Type Guard

docs/method/graveyard/legacy-backlog/v4.5.0/RE-022-public-packed-surface-type-guard.md

Legend: RE — Runtime Engine

Idea

Export a canonical isPackedSurface(s): s is PackedSurface type guard from @fl
yingrobots/bijou so consumers and internal code stop using ad-hoc 'buffer' in
s duck-typing.

Currently the guard exists as a private function in packages/bijou/src/ports/
surface.ts and is duplicated (with slight variations) in:

- bench/src/scenarios/_shared.ts
- packages/bijou-tui/src/css/text-style.ts
- packages/bijou-tui/src/overlay.ts
- packages/bijou/src/core/render/differ.ts

A single exported guard would:

- Eliminate duck-typing fragility (false positives if other objects have a bu
  ffer property)
- Give TypeScript narrowing to PackedSurface in one check
- Let downstream consumers (like git-warp) detect packed surfaces without
  casting

374

RE-022 — Public PackedSurface Type Guard

docs/method/retro/RE-022-public-packed-surface-type-guard.md

```
title: "RE-022 — Public PackedSurface Type Guard"
legend: RE
lane: retro
```

Completed on release/v5.0.0. @flyingrobots/bijou now exports isPackedSurface( ) from packages/bijou/src/ports/surface.ts and re-exports it from packages/bi jou/src/index.ts, while current core, TUI, and bench callsites use that public seam instead of ad hoc 'buffer' in s duck-typing.

Original Proposal

Legend: RE — Runtime Engine

Idea

Export a canonical isPackedSurface(s): s is PackedSurface type guard from @fl yingrobots/bijou so consumers and internal code stop using ad-hoc 'buffer' in s duck-typing.

Currently the guard exists as a private function in packages/bijou/src/ports/ surface.ts and is duplicated (with slight variations) in:

- bench/src/scenarios/_shared.ts
- packages/bijou-tui/src/css/text-style.ts
- packages/bijou-tui/src/overlay.ts
- packages/bijou/src/core/render/differ.ts

A single exported guard would:

- Eliminate duck-typing fragility (false positives if other objects have a bu ffer property)
- Give TypeScript narrowing to PackedSurface in one check
- Let downstream consumers (like git-warp) detect packed surfaces without casting

375

RE-022 — Unified Layout Calculation Engine

docs/method/graveyard/legacy-backlog/bad-code/RE-022-unified-layout-calculation-engine.md

Legend: RE — Runtime Engine

Idea

Currently, @flyingrobots/bijou provides string-first layout helpers like splitPane() and grid(), while @flyingrobots/bijou-tui provides surface-first companions like splitPaneSurface() and gridSurface(). The underlying geometry calculation (ratios, areas, gaps, and constraints) is often duplicated or implemented with slight variations across these packages.

Extract the “pure geometry” phase into a shared, zero-dependency engine in @flyingrobots/bijou. This engine should accept a layout definition and terminal dimensions, and return a set of resolved LayoutRects. The string-first and surface-first renderers can then consume these rects, ensuring pixel-perfect parity between rich, static, pipe, and accessible modes.

Why

1. **Parity**: A 0.4 split-pane ratio should resolve to the same column boundary regardless of whether we are rendering a string box or a TUI surface.
2. **Maintenance**: Bug fixes in the gap/constraint logic only need to happen in one place.
3. **Purity**: Geometry calculation is inherently pure and belongs in the foundation, not the interactive runtime.

Effort

Medium — extract logic from existing helpers and move to a unified core utility with shared tests.

376

RE-022 — Unified Layout Calculation Engine

docs/method/retro/RE-022-unified-layout-calculation-engine.md

title: RE-022 — Unified Layout Calculation Engine
lane: retro
legend: RE

Disposition

Completed on release/v4.5.0. Pure split/grid geometry now lives in @flyingrobots/bijou under packages/bijou/src/core/layout/geometry.ts, with shared solveGridRects(), solveSplitPaneRects(), and solveSplitAxisSizes() utilities plus foundation tests. The bijou-tui grid and split-pane helpers now delegate to that shared engine instead of maintaining their own duplicate geometry math, so the string and surface paths resolve the same rectangles from the same source of truth.

Original Proposal

Legend: RE — Runtime Engine

Idea

Currently, @flyingrobots/bijou provides string-first layout helpers like splitPane() and grid(), while @flyingrobots/bijou-tui provides surface-first companions like splitPaneSurface() and gridSurface(). The underlying geometry calculation (ratios, areas, gaps, and constraints) is often duplicated or implemented with slight variations across these packages.

Extract the “pure geometry” phase into a shared, zero-dependency engine in @flyingrobots/bijou. This engine should accept a layout definition and terminal dimensions, and return a set of resolved LayoutRects. The string-first and surface-first renderers can then consume these rects, ensuring pixel-perfect parity between rich, static, pipe, and accessible modes.

Why

1. **Parity**: A 0.4 split-pane ratio should resolve to the exact same column boundary regardless of whether we are rendering a string box or a TUI surface.
2. **Maintenance**: Bug fixes in the gap/constraint logic only need to happen in one place.
3. **Purity**: Geometry calculation is inherently pure and belongs in the foundation, not the interactive runtime.

Effort

Medium — extract logic from existing helpers and move to a unified core utility with shared tests.

377

RE-023 — Stabilize Render Scheduling in runtime.ts

docs/method/graveyard/legacy-backlog/bad-code/RE-023-stabilize-render-scheduling.md

Legend: RE — Runtime Engine

Idea

The current render() implementation in runtime.ts uses setTimeout(..., 0) and a renderRequested boolean. Under heavy input load or rapid async command resolution, this can lead to multiple overlapping render timeouts being scheduled if the boolean is reset at the wrong point in the microtask queue.

Refactor render() to use a single, stable TimerHandle (or a similar scheduling token) that is explicitly cleared and reset on every request. This ensures that only one render operation is ever pending or in flight at any given time.

Why

1. **Determinism**: Ensures that the view() function always operates on the most current model snapshot.
2. **Performance**: Prevents redundant rendering work during high-frequency update bursts.
3. **Safety**: Neutralizes potential race conditions where front/back buffers might be swapped mid-render.

Effort

Small — refactor the renderRequested logic to use a persistent TimerHandle from the context clock.

378

RE-023 — Stabilize Render Scheduling in runtime.ts

docs/method/retro/RE-023-stabilize-render-scheduling.md

title: RE-023 — Stabilize Render Scheduling in runtime.ts
lane: retro
legend: RE

Disposition

Completed on release/v4.5.0. runtime.ts now uses a stable render handle plus explicit renderQueued / renderInFlight state instead of a bare renderRequested boolean. That gives the runtime one pending render at a time, safely coalesces same-timestamp input bursts into a single follow-up render, and flushes pending work cleanly during shutdown. Focused runtime tests now cover the coalesced-burst behavior directly.

Original Proposal

Legend: RE — Runtime Engine

Idea

The current render() implementation in runtime.ts uses setTimeout(..., 0) and a renderRequested boolean. Under heavy input load or rapid async command resolution, this can lead to multiple overlapping render timeouts being scheduled if the boolean is reset at the wrong point in the microtask queue.

Refactor render() to use a single, stable TimerHandle (or a similar scheduling token) that is explicitly cleared and reset on every request. This ensures that only one render operation is ever pending or in flight at any given time.

Why

1. **Determinism**: Ensures that the view() function always operates on the most current model snapshot.
2. **Performance**: Prevents redundant rendering work during high-frequency update bursts.
3. **Safety**: Neutralizes potential race conditions where front/back buffers might be swapped mid-render.

Effort

Small — refactor the renderRequested logic to use a persistent TimerHandle from the context clock.

379

RE-024 — Terminal Content Sanitization Boundary

docs/method/graveyard/legacy-backlog/asap/RE-024-terminal-content-sanitization-boundary.md

Legend: RE — Runtime Engine

Problem

Bijou has strong low-level ANSI handling, but it does not yet define a clear safety boundary for untrusted or user-provided content. The 2026-04-11 ship-readiness audit flagged this as both a terminal injection risk and a correctness risk for render/diff bookkeeping when destructive escape sequences slip through ordinary content paths.

Right now the repo has helpers for stripping ANSI, but not a clear runtime contract for when content must be sanitized or how trusted vs untrusted strings should be modeled.

Desired outcome

1. Define where content sanitization belongs in the runtime/render path.
2. Decide whether the answer is a sanitize stage, a SafeString component, explicit trusted/untrusted content types, or a narrower boundary helper.
3. Add regression coverage for destructive escape-sequence payloads.

Effort

Medium — contract design, implementation, and tests.

380

RE-024 — Terminal Content Sanitization Boundary

docs/method/retro/RE-024-terminal-content-sanitization-boundary.md

```
title: "RE-024 — Terminal Content Sanitization Boundary"
legend: RE
lane: retro
```

Completed on release/v5.0.0. Bijou now ships a first-class terminal text sanitization seam through sanitizeTerminalText() in packages/bijou/src/core/text/grapheme.ts, re-exports that helper publicly, and covers destructive escape payloads in grapheme.test.ts. The later surface-native hardening pass also routes badge(), createTextSurface(), and hyperlink() through that boundary so untrusted text no longer slips past the plain-text path by default.

Original Proposal

Legend: RE — Runtime Engine

Problem

Bijou has strong low-level ANSI handling, but it does not yet define a clear safety boundary for untrusted or user-provided content. The 2026-04-11 ship-readiness audit flagged this as both a terminal injection risk and a correctness risk for render/diff bookkeeping when destructive escape sequences slip through ordinary content paths.

Right now the repo has helpers for stripping ANSI, but not a clear runtime contract for when content must be sanitized or how trusted vs untrusted strings should be modeled.

Desired outcome

1. Define where content sanitization belongs in the runtime/render path.
2. Decide whether the answer is a sanitize stage, a SafeString component, explicit trusted/untrusted content types, or a narrower boundary helper.
3. Add regression coverage for destructive escape-sequence payloads.

Effort

Medium — contract design, implementation, and tests.

381

RE-025 — Scoped Node IO Adapter

docs/method/graveyard/legacy-backlog/asap/RE-025-scoped-node-io-adapter.md

Legend: RE — Runtime Engine

Problem

@flyingrobots/bijou-node exposes broad host IO capabilities, but the repo does not yet provide a first-class way to scope filesystem access to a rooted sandbox for app-level use.

The 2026-04-11 ship-readiness audit called out this gap directly: a component handed raw nodeIO can read arbitrary system paths unless the host wraps and constrains it manually.

Desired outcome

1. Add a scoped IO adapter in @flyingrobots/bijou-node.
2. Let hosts define a root boundary that constrains reads, writes, and directory traversal.
3. Return clear errors when a requested path escapes the allowed root.
4. Document when apps should prefer scoped IO over raw host IO.

Effort

Medium — adapter design, path-normalization rules, and host tests.

382

RE-025 — Scoped Node IO Adapter

docs/method/retro/RE-025-scoped-node-io-adapter.md

```
title: "RE-025 — Scoped Node IO Adapter"
legend: RE
lane: retro
```

Completed on release/v5.0.0. @flyingrobots/bijou-node now exports scopedNodeIO() from packages/bijou-node/src/io.ts, rejects path escapes via ScopedNodeIOError, covers rooted reads and traversal failures in packages/bijou-node/src/io.test.ts, and documents when to prefer the scoped adapter in the package README, GUIDE, and ARCHITECTURE docs.

Original Proposal

Legend: RE — Runtime Engine

Problem

@flyingrobots/bijou-node exposes broad host IO capabilities, but the repo does not yet provide a first-class way to scope filesystem access to a rooted sandbox for app-level use.

The 2026-04-11 ship-readiness audit called out this gap directly: a component handed raw nodeIO can read arbitrary system paths unless the host wraps and constrains it manually.

Desired outcome

1. Add a scoped IO adapter in @flyingrobots/bijou-node.
2. Let hosts define a root boundary that constrains reads, writes, and directory traversal.
3. Return clear errors when a requested path escapes the allowed root.
4. Document when apps should prefer scoped IO over raw host IO.

Effort

Medium — adapter design, path-normalization rules, and host tests.

383

RE-026 — Drain runtime commands before shutdown

docs/method/retro/RE-026-drain-runtime-commands-before-shutdown.md

```
title: RE-026 — Drain runtime commands before shutdown
legend: RE
lane: retro
```

Disposition

Completed on 2026-04-14 via commit fixing runtime shutdown drain semantics and bounded timeout behavior. Implementation landed in a follow-up commit after the 2026-04-14 audit and passed runtime-focused tests plus full release-readiness validation.

Original Proposal

The runtime still exits without awaiting bus.drain(), which leaves async command cleanup timing less deterministic than it should be.

Problem

packages/bijou-tui/src/runtime.ts tears down the session and disposes the bus after quit, but it does not explicitly wait for pending command work to drain. packages/bijou-tui/src/eventbus.ts already exposes drain(), and the driver uses it, but the production runtime does not.

This creates a gap between test/runtime semantics: - tests can wait for pending command work deterministically - the real runtime can dispose while cleanup work is still in flight

Desired outcome

1. Make runtime shutdown await bus.drain() before final disposal.
2. Keep shutdown bounded with a timeout or other guard so the app cannot hang forever on a misbehaving command.

3. Add focused tests for long-lived commands that register cleanups asynchronously.
4. Preserve the current fast path for already-idle apps.

Why ASAP

- This is one of the last concrete ship-readiness concerns left after the release gauntlet turned green.
- The code already has most of the needed primitives; the gap is mostly runtime integration and proof.

Hill

Quit semantics are deterministic: pending commands either drain cleanly before teardown or fail through one explicit, observable timeout path.

384

Graveyard

docs/method/graveyard/README.md

Superseded or abandoned METHOD ideas live here as historical evidence.

Use graveyard/ only when the original idea is no longer the right live work item: it was killed, replaced, or narrowed into a different follow-on.

Do not put finished shipped work here. Finished work belongs in docs/method/retro/.

Do not use this directory to determine current work. Start with open GitHub Issues and their lane:* labels.

385

Legacy Backlog Archive

docs/method/graveyard/legacy-backlog/README.md

This directory is **not** the canonical live backlog anymore.

Bijou now keeps the live queue in docs/method/backlog/. This tree remains only as:

- a historical archive of pre-METHOD backlog notes
- a compatibility surface for old links in design docs, audits, tests, and release notes
- a place where older shaped-release snapshots can remain legible without pretending to be current queue truth

Canonical Current Surfaces

- docs/method/backlog/ — live backlog
- docs/method/retro/ — finished ideas
- docs/method/graveyard/ — superseded or abandoned ideas

Rule

Do **not** add new live backlog work here.

If a legacy note is still active, give it a canonical METHOD copy and continue the work there. Keep the legacy file only for historical provenance or compatibility.

Every legacy note in this archive now has a corresponding canonical METHOD placement under docs/method/ so current work can be evaluated without treating this tree as the live queue.

Legacy Lanes

These subdirectories remain as archive views of older planning states:

- inbox/
- asap/

- up-next/
- cool-ideas/
- bad-code/
- v4.5.0/
- v5.0.0/

386

Legacy ASAP Archive

docs/method/graveyard/legacy-backlog/asap/README.md

This lane is a historical archive of pre-METHOD asap/ notes.

The canonical live lane is docs/method/backlog/asap/.

387

Legacy Bad-Code Archive

docs/method/graveyard/legacy-backlog/bad-code/README.md

This lane is a historical archive of pre-METHOD bad-code/ notes.

The canonical live lane is docs/method/backlog/bad-code/.

388

Legacy Cool Ideas Archive

docs/method/graveyard/legacy-backlog/cool-ideas/README.md

This lane is a historical archive of pre-METHOD cool-ideas/ notes.

The canonical live lane is docs/method/backlog/cool-ideas/.

389

Legacy Inbox Archive

docs/method/graveyard/legacy-backlog/inbox/README.md

This lane is a historical archive of pre-METHOD inbox/ notes.

The canonical live lane is docs/method/backlog/inbox/.

390

Legacy Up Next Archive

docs/method/graveyard/legacy-backlog/up-next/README.md

This lane is a historical archive of pre-METHOD up-next/ notes.

The canonical live lane is docs/method/backlog/up-next/.

391

v4.5.0 — Legacy Release Lane Snapshot

docs/method/graveyard/legacy-backlog/v4.5.0/README.md

This directory remains as the historical shaping snapshot for the v4.5.0 hardening release.

It is not the live queue. Current release-target backlog work belongs in docs/method/backlog/, and completed release truth belongs in docs/CHANGELOG.md.

392

v5.0.0 — Legacy Release Lane Snapshot

docs/method/graveyard/legacy-backlog/v5.0.0/README.md

This directory remains as the historical shaping snapshot for the pre-METHOD v5.0.0 release lane.

The canonical live v5.0.0 lane now lives in docs/method/backlog/v5.0.0/.

393

Retro

docs/method/retro/README.md

Finished METHOD ideas live here as historical evidence.

Use retro/ for backlog notes whose disposition is complete: shipped, implemented, fixed, or otherwise satisfied by current repo truth.

Do not treat these files as live queue work. They are historical lineage and retrospective bookkeeping.

For current work, start with open GitHub Issues and their lane:* labels.

394

Retro

docs/retro/README.md

Use this directory for standalone retrospectives or witness packets when they add clarity beyond the cycle doc itself.

Historical Bijou cycles often keep retrospective notes inline in the design doc. That is acceptable. The goal is honest bookkeeping, not a fake migration of old history.

395

RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

docs/method/retro/RE_007-migrate-framed-shell-onto-runtime-engine-seams.md

title: RE-007 — Migrate Framed Shell Onto Runtime Engine Seams
lane: retro
legend: RE

Disposition

Completed earlier and already shipped. The framed shell migration landed in the runtime-engine work that shipped in v4.2.0, and the current repo truth already reflects that through app-frame runtime routing/buffering code, release notes, BEARING, and COMPLETED.md. This METHOD backlog note was stale duplicate tracking.

Replacement: docs/design/RE-007-migrate-framed-shell-onto-runtime-engine-seams.md

Original Proposal

Legend: RE — Runtime Engine

Status

Promoted into the active cycle doc:

- RE-007 — Migrate Framed Shell Onto Runtime Engine Seams

Idea

Move the current framed shell onto the new runtime-engine seams once the first-class objects exist.

Why

The shell is where the current architectural debt is most visible:

- derived layer stacks
- branchy mouse routing
- mixed layout and routing concerns
- help/footer/control projection tied to shell-specific branches

Likely scope

- migrate framed-app layers onto the explicit view stack
- migrate shell routing onto retained layouts
- migrate shell commands/effects onto the new buffers
- use the shell as the proving surface before broader runtime adoption

396

RE-011 — Eliminate Per-Frame Allocation in Render Pipeline

docs/method/retro/RE_011-pipeline-per-frame-allocation.md

title: RE-011 — Eliminate Per-Frame Allocation in Render Pipeline
lane: retro
legend: RE

Disposition

Implemented on release/v4.5.0 by caching the flattened render middleware chain inside createPipeline(), marking it dirty only when use() mutates the stage registry, and covering cache reuse plus invalidation in pipeline tests. The per-frame chain allocation is no longer live backlog work.

Original Proposal

Legend: RE — Runtime Engine

Problem

pipeline.execute() in packages/bijou-tui/src/pipeline/pipeline.ts allocates a new chain[] array every frame by spreading all stage middleware into a flat list. At 60fps this is 60 array allocations per second with no reuse.

Fix

Cache the flattened chain array and only rebuild when middleware is added (track a dirty flag on use()). The chain is static after app initialization in most apps.

Effort

Small — add a dirty flag and cached array.

397

RE-012 — Pipeline Observability Hooks

docs/method/retro/RE_012-pipeline-observability-hooks.md

```
title: RE-012 — Pipeline Observability Hooks
lane: retro
legend: RE
```

Disposition

Completed on release/v4.5.0. The programmable render pipeline now exposes pip
eline.onStageComplete(...) observers, records per-stage timing samples in Ren
derState.data, and ships getRenderStageTimings(state) as the typed read path
for middleware and apps. Stage timings are emitted in global stage order,
include empty stages as zero-duration samples, and are visible both through
the observer callback and the shared per-frame metadata bag.

Original Proposal

Legend: RE — Runtime Engine

Idea

Expose per-stage timing hooks in the render pipeline so apps and middleware
can measure Layout, Paint, PostProcess, Diff, and Output phases individually.

```
run(app, {
  configurePipeline(pipeline) {
    pipeline.onStageComplete((stage, durationMs) => {
      perfOverlay.recordPhase(stage, durationMs);
    });
  },
});
```

Why

The perf-gradient demo proved that 92% of frame time is spent in the pipeline
(diff + ANSI encode + stdout.write), but we could not measure the breakdown

from the app side. configurePipeline can only append middleware after the default stages — it cannot wrap them for timing.

This blocks: - DX-006 from showing a full phase breakdown - Any app from diagnosing whether their bottleneck is layout, paint, or diff

Shape

Either: 1. A callback-based hook (onStageComplete) on the pipeline 2. Timing metadata added to RenderState.data by the pipeline itself 3. A wrapStage(stage, fn) API that lets middleware run before AND after a stage's default middleware

Option 3 is the most general but also the most complex.

398

RE-013 — Memoize Workspace Layout Tree

docs/method/retro/RE_013-memoize-workspace-layout-tree.md

title: RE-013 — Memoize Workspace Layout Tree
lane: retro
legend: RE

Disposition

Implemented on release/v5.0.0. createFramedApp() now remembers the last rendered workspace pane rects and retained layout tree, then reuses that geometry when mouse routing asks for pane hit-testing or wheel scrolling without a layout-affecting change. That removes the redundant buildWorkspaceLayoutTree() churn from steady-state mouse interaction while keeping resize, page switch, dock, maximize, and split-ratio changes as cache invalidation boundaries.

Original Proposal

Legend: RE — Runtime Engine

Idea

buildWorkspaceLayoutTree() creates new SurfaceLayoutNode objects on every mouse event, even when the terminal size, page layout, and tab count haven't changed. The tree structure only changes on resize, tab switch, or pane layout change.

Memoize the tree against the inputs that determine its shape (cols, rows, active page, pane layout hash). Return the cached tree when inputs match. This eliminates ~20 object allocations per mouse event for a typical 2-tab, 2-pane layout.

Why

The perf-gradient demo showed that view() time is only ~2ms, but every mouse event also runs resolveFrameMouseRuntimeLayouts() which calls buildWorkspaceLayoutTree(). In a typical app with 60fps rendering and mouse tracking, this is 60+ unnecessary tree builds per second.

Effort

Small — hash the inputs, compare, return cached tree.

399

RE-014 — Design Doctrine

docs/method/retro/RE_014-buffer-holds-facts-doctrine.md

```
title: RE-014 — "The Buffer Holds Facts" Design Doctrine
lane: retro
legend: RE
```

Disposition

Implemented on release/v5.0.0 as an explicit invariant. The new docs/invariants/buffer-holds-facts.md captures the rule that queues, buffers, and message channels should carry facts rather than executable behavior, and the architecture signposts now point at that invariant from the root, core, and TUI runtime docs.

Original Proposal

Legend: RE — Runtime Engine

Idea

During the RE-007 buffer migration, James articulated a principle:

> "Keep buffered shell commands as plain discriminated data and interpret them explicitly. Use a handler table if you want to avoid a giant switch. The buffer should hold facts, not code."

This is a design doctrine that should be captured in the runtime engine's architectural documentation. It applies beyond shell commands — any buffer, queue, or event channel in the system should carry data (what happened) not behavior (how to respond).

400

RE-016: DAG renderer should handle cycles gracefully

docs/method/retro/RE_016-dag-cycle-graceful-handling.md

```
title: "RE-016: DAG renderer should handle cycles gracefully"
lane: retro
legend: RE
```

Disposition

Rendered DAG graphs now tolerate cycles by stripping cycle-forming back-edges only for the layering pass while preserving the full original edge set in interactive and accessible output. dag() and dagLayout() no longer fail on cyclic graphs or self-loops, and focused regressions cover both paths.

Original Proposal

Problem

dag() uses Kahn's topological sort (dag-layout.ts:22) and throws [bijou] dag(): cycle detected in graph when any cycle exists. This makes dag() unusable for general directed graphs — which are common in real-world use cases (dependency graphs with circular refs, social graphs, WARP materialized state, etc.).

Current behavior

assignLayers() line 69:

```
if (topoOrder.length !== nodes.length) {
  throw new Error('[bijou] dag(): cycle detected in graph');
}
```

Proposed behavior

Two possible approaches (not mutually exclusive):

Option A: Back-edge detection + reversal

During Kahn's sort, identify edges that create cycles (back-edges). Temporarily reverse them to break cycles, assign layers on the resulting DAG, then render back-edges with a distinct visual treatment (dashed line, different arrowhead, annotation like "↺").

Option B: SCC collapse

Find strongly connected components (Tarjan's or Kosaraju's), collapse each SCC into a single composite node, render the condensation DAG. Composite nodes show their member labels.

Recommendation

Option A is simpler and preserves the full node set. Option B is more correct topologically but loses individual node identity within SCCs.

Context

Discovered while using bijou-mcp dag() to visualize a WARP graph. WARP materialized state is a general directed graph (social: alice → bob → carol → dave → alice). Had to manually strip back-edges to render.

401

RE-016 — graphemeWidth Miscounts Emoji-Like Icon Characters

docs/method/retro/RE_016-grapheme-width-miscounts-emoji-like-icons.md

title: RE-016 — graphemeWidth Miscounts Emoji-Like Icon Characters
lane: retro
legend: RE

Disposition

Fixed on release/v4.5.0 in commit 69291fa. First-party status icon paths now force text presentation with VS15 in core alerts, notes, overlay toasts, and notifications, and the grapheme regression coverage proves those symbols stay in the narrow text-width lane instead of drifting into emoji-width rendering.

Original Proposal

Summary

graphemeWidth() in packages/bijou/src/core/text/grapheme.ts counts certain Unicode icon characters as 1 column wide, but many renderers (chat UIs, some terminal emulators, web-based terminals) display them as 2 columns wide. This causes box-drawing alignment to break when these characters appear inside bordered components.

Affected Characters

Char	Codepoint	Name	graphemeWidth	Actual (many fonts)
✗	U+2717	BALLOT X	1	2
[warning]	U+26A0	WARNING SIGN	1	2
ℹ	U+2139	INFORMATION SOURCE	1	2

✓ (U+2713 CHECK MARK) renders as 1-wide in most contexts and is not affected.

Symptom

The right border │ of alert() boxes is misaligned for error, warning, and info variants:

```
┌──────────────────────────────────────┐
✗ Connection refused on port 5432. │  ← right │ doesn't reach ┐
└──────────────────────────────────────┘
```

Where It Matters

- alert() component — uses ICONS map with ✗, [warning], ℹ
- Any component that places these characters inside a box() or width-padded cell
- Particularly visible in non-terminal monospace contexts (MCP chat output, web terminals, VS Code terminal)

Files

- packages/bijou/src/core/text/grapheme.ts — width calculation
- packages/bijou/src/core/components/alert.ts — icon map (line 21-26)

Options

1. Fix graphemeWidth() to handle ambiguous-width characters correctly (hard — width depends on the rendering context)
2. Replace the icons with ASCII-safe alternatives in a dedicated plainStyle / MCP rendering path
3. Add a width-override mechanism so MCP context can declare these as 2-wide

402

WritePort byte-level API for zero-alloc ANSI output

docs/method/retro/RE_writeport-byte-level-api-for-zero-alloc-ansi-output.md

title: WritePort byte-level API for zero-alloc ANSI output
lane: retro
legend: RE

Disposition

This is already repo truth and no longer belongs in the live backlog. WritePort.writeBytes(buf, len) is part of the public core I/O contract, nodeIO() implements it for process.stdout, and the byte-path differ uses it when available before falling back to string writes. The remaining work in this area is no longer “add the API,” but deeper output-path refinements on top of the byte writer that already shipped.

Original Proposal

Legend: RE — Runtime Engine

Extend WritePort to accept Uint8Array bytes alongside strings. The differ could then write ANSI escape sequences directly as bytes into a pre-allocated output buffer, avoiding string construction entirely. V8’s rope string optimization makes string += competitive for now, but a byte-level path would eliminate the ~365KB/frame of transient string allocation in the gradient worst case. Requires: WritePort.writeBytes(buf: Uint8Array, length: number), differ output buffer as Uint8Array, ANSI SGR sequences written as raw bytes. Discovered during RE-008/RE-009 investigation — chunks.push+join was slower than +=, so the byte path is the only remaining win.

403

WF-002 — Migrate Legacy Planning Artifacts

docs/method/graveyard/legacy-backlog/WF-002-migrate-legacy-planning-artifacts.md

Legend: WF — Workflow and Delivery

Idea

The new Legends/Cycles workflow is adopted, but older planning surfaces still exist:

- docs/ROADMAP.md
- docs/specs/
- older docs/strategy/ notes that are not legend docs

This cycle would decide what gets:

- migrated
- frozen
- archived
- rewritten as backlog/design docs

Why

WF-001 intentionally adopted the new process without trying to convert the entire repo in one shot.

That was the right call, but it leaves migration debt behind.

Status

Backlog spawned by the retrospective for WF-001.

404

WF-007 — Align Local Validation With CI typecheck:test Gate

docs/method/graveyard/legacy-backlog/bad-code/WF-007-align-local-validation-with-ci-typecheck-test-gate.md

Legend: WF — Workflow and Delivery

Idea

The repo's local validation story is still slightly less strict than the GitHub Actions story.

This PR hit a real example:

- local branch work looked green enough to push
- GitHub Actions failed in test (22) during typecheck:test
- the failure was a test-only typing drift in tests/cycles/DF-001/dogfood-coverage-progress.test.ts

That means the repo can still teach a false lesson:

- "looks good locally" is not yet the same thing as "will clear the CI gate"

Why

This is workflow debt, not a product blocker.

The codebase already has a real typecheck:test gate and it caught a legitimate bug. The problem is that the local branch-validation path did not catch it early enough.

Hill

A maintainer or agent can run the normal local validation path before pushing and trust that test-only TypeScript drift will be caught before GitHub Actions does.

Likely scope

- inspect which local guardrail should own typecheck:test
  - pre-push hook
  - an explicit branch-readiness script
  - both
- make the local witness match the remote CI expectation
- document the intended local gate clearly

Status

Backlog spawned by PR #59 feedback processing after GitHub Actions failed on typecheck:test and the fix landed in commit aec306f.

405

WF-008 — METHOD backlog helper generates malformed note filenames

docs/method/graveyard/WF-008-method-backlog-helper-generates-malformed-note-filenames.md

title: WF-008 — METHOD backlog helper generates malformed note filenames
legend: WF
lane: graveyard

Disposition

Out of scope for Bijou. The malformed filename generation behavior belongs to the METHOD tooling in ~/git/method, not to this repository's product or workflow backlog.

Original Proposal

The METHOD backlog creation helper currently emits malformed filenames like D X_dx-022-layout-inspector-overlay.md and DF_df-028-story-capture-matrix.md instead of the repo's canonical DX-022-... / DF-028-... pattern.

Why this is bad-code

- every newly added note has to be manually renamed after creation
- it makes the filesystem less trustworthy than the title/frontmatter
- it introduces needless churn into otherwise simple backlog-add flows
- it risks broken expectations for scripts or humans relying on the ID-first filename convention

Evidence

This occurred repeatedly during the current release/v5.0.0 backlog shaping pass when creating new METHOD notes in cool-ideas/.

Expected behavior

The helper should derive filenames directly from the normalized backlog ID and slug, matching existing repo convention:

- DX-022-layout-inspector-overlay.md
- DF-028-story-capture-matrix.md
- RE-024-surface-budget-warnings.md

Fix direction

Normalize filename generation inside the METHOD helper so legend prefixes and numeric IDs are not duplicated or lowercased into the stem.

406

WF-122 - Day Zero Audit Fixes

docs/method/retro/WF-122-day-zero-audit-fixes.md

Linked issues: #122, #123, #124, #125, #127, #129, #130, #131, #132, #135, #136, #142, #143, #144, #150, #151, #152, #153, #154, #155, #165, #166

Summary

This pass closes the remaining actionable Bijou deep-dive PDF findings that were not already satisfied on origin/main.

The work landed in five clusters:

- Issues-first Method intake and historical backlog de-emphasis.
- Day 0 README/GUIDE/docs-map onboarding bridge.
- DOGFOOD terminal takeover and non-TTY guardrails.
- Node bootstrap diagnostics with a structured BijouBootstrapError contract.
- Runtime guardrails for command backpressure and async render middleware.
- Cross-platform contributor scripts and focused non-Linux CI.

Design Evidence

- WF-122 cross-platform contributor path
- WF-123 issues-first Method intake
- DX-125 day zero onboarding bridge
- DF-150 DOGFOOD terminal guardrails
- RE-142 Node bootstrap failure contract
- RE-127 command and render runtime guardrails

Witness

Local validation completed on June 1, 2026:

- node --check scripts/clean.mjs passed.
- node --check scripts/version.mjs passed.
- npm run clean passed and removed 20 build artifacts.

- npm run build passed.
- /usr/bin/time -p npm run hello --silent rendered the hello example in real 0.93 seconds on this workstation.
- npm run workflow:shell:preflight passed.
- Focused runtime/docs suite passed 5 files and 132 tests:

```
npm test -- --run scripts/docs-preview.test.ts packages/bijou-node/src/index.test.ts
↪  packages/bijou-tui/src/pipeline/pipeline.test.ts
↪  packages/bijou-tui/src/eventbus.test.ts scripts/check-node-version.test.ts
```

- npm run typecheck:test passed.
- npm run lint passed across all workspaces.
- npm run smoke:dogfood:landing passed.
- npm run smoke:dogfood:docs passed.
- npm run docs:inventory passed.
- git diff --check passed.
- npm test passed 315 files and 3581 tests.
- npm run verify:interactive-examples passed.
- npm run smoke:canaries -- --skip-build passed all pack/install/build/run canaries.

The tsx runtime emitted Node's existing DEP0205 deprecation warning during some commands. It did not fail validation and is not introduced by this cycle.

Outcome By Issue

Issue	Outcome
#122	Added cross-platform docs, portable root scripts, and macOS/Windows Node 22 CI lane.
#123	Documented GitHub Issues as the live Method tracker.
#124	Reframed backlog, retro, and graveyard docs as evidence/history rather than live queue.
#125	Added npm run app-frame and a framed app tutorial.
#127	Added EventBus command diagnostics and configurable backpressure warning.
#129	Added async render middleware diagnostics and late-continuation guard tests.
#130	Staged Surface, App, Msg, Cmd, and framed-shell concepts in README.
#131	Documented full-model render cost and sync-pipeline tradeoff.
#132	Explained package boundaries in README and PACKAGE_DEVELOPMENT.
#135	Replaced root clean and version shell scripts with Node scripts.
#136	Added focused macOS and Windows unit-test CI coverage.
#142	Documented initDefaultContext() failure modes in node package docs.
#143	Improved bootstrap failure framing through reason/hint/cause docs.
#144	Added tests for BijouBootstrapError contract fields.
#150	DOGFOOD now prints a full-screen launch notice before running.

Issue	Outcome
#151	DOGFOOD launch notice names quit confirmation and hard-exit controls.
#152	DOGFOOD now rejects non-TTY stdin/stdout with a scripted-smoke hint.
#153	CONTRIBUTING and METHOD now explain Method lane/state labels.
#154	README fast path now measures to visible hello output before deeper docs.
#155	README explains first-app package imports immediately after the example.
#165	README narrative now starts with purpose, prerequisites, first commands, and next docs.
#166	README and docs map expose GUIDE/docs/DOGFOOD navigation earlier.

Deferred Debt

GitHub-hosted macOS and Windows jobs need remote CI execution after push for runner-specific proof. Locally, the new workflow syntax and focused test command are covered by workflow:shell:preflight, build, typecheck, lint, and full test/canary validation.

407

WF-002 — Migrate Legacy Planning Artifacts

docs/method/retro/WF_002-migrate-legacy-planning-artifacts.md

title: WF-002 — Migrate Legacy Planning Artifacts
lane: retro
legend: WF

Disposition

Legacy planning entrypoints now identify themselves honestly: ROADMAP is reference-only, docs/specs is a frozen legacy spec archive with an explicit migration rule, docs/strategy now has a directory index separating living doctrine from historical notes, and the older strategy notes that overlap newer signposts now point readers back to the canonical surfaces.

Original Proposal

Legend: WF — Workflow and Delivery

Idea

The new Legends/Cycles workflow is adopted, but older planning surfaces still exist:

- docs/ROADMAP.md
- docs/specs/
- older docs/strategy/ notes that are not legend docs

This cycle would decide what gets:

- migrated
- frozen
- archived
- rewritten as backlog/design docs

Why

WF-001 intentionally adopted the new process without trying to convert the entire repo in one shot.

That was the right call, but it leaves migration debt behind.

Status

Backlog spawned by the retrospective for WF-001.

408

WF-007 — Align Local Validation With CI typecheck:test Gate

docs/method/retro/WF_007-align-local-validation-with-ci-typecheck-test-gate.md

```
title: WF-007 — Align Local Validation With CI `typecheck:test` Gate
lane: retro
legend: WF
```

Disposition

Fixed on release/v4.5.0 in commit 69291fa. scripts/hooks/pre-push now runs npm run typecheck:test before the full test suite, scripts/hooks.test.ts locks that order down, and the existing i18n test typing drift that would have kept the new gate red was fixed in the same commit so local pre-push truth now matches CI.

Original Proposal

Legend: WF — Workflow and Delivery

Idea

The repo's local validation story is still slightly less strict than the GitHub Actions story.

This PR hit a real example:

- local branch work looked green enough to push
- GitHub Actions failed in test (22) during typecheck:test
- the failure was a test-only typing drift in tests/cycles/DF-001/dogfood-coverage-progress.test.ts

That means the repo can still teach a false lesson:

- "looks good locally" is not yet the same thing as "will clear the CI gate"

Why

This is workflow debt, not a product blocker.

The codebase already has a real typecheck:test gate and it caught a legitimate bug. The problem is that the local branch-validation path did not catch it early enough.

Hill

A maintainer or agent can run the normal local validation path before pushing and trust that test-only TypeScript drift will be caught before GitHub Actions does.

Likely scope

- inspect which local guardrail should own typecheck:test
  - pre-push hook
  - an explicit branch-readiness script
  - both
- make the local witness match the remote CI expectation
- document the intended local gate clearly

Status

Backlog spawned by PR #59 feedback processing after GitHub Actions failed on typecheck:test and the fix landed in commit aec306f.

409

Background fill regression in v4.4.0

docs/method/feedback/think-bg-fill-regression-v4.4.0.md

Reporter: Think repo (downstream consumer) **Bijou version:** 4.4.0 (worked in 4.2.0) **Date:** 2026-04-11

What broke

The Think browse TUI uses createFramedApp + run() with a custom theme that sets a plum background (#2d1922). After upgrading from bijou 4.2.0 to 4.4.0, the background fill no longer renders — the terminal shows a plain black background instead of the themed plum.

RE-017 workaround context

Think has a standing workaround (RE-017) for "frame doesn't fill surface.primary.bg". Before calling run(), Think writes a raw ANSI escape to set the terminal background color:

```
process.stdout.write(`\x1b[48;2;${BG[0]};${BG[1]};${BG[2]}m`);
await run(app, { ctx });
```

This workaround worked on bijou 4.2.0 but stopped working on 4.4.0.

Suspected cause

The v4.4.0 zero-alloc frame chrome changes (paintStyledTextSurfaceWithBCSS repaint-in-place for header/footer) may have changed how the frame clears or fills the screen on entry, overriding the pre-set terminal background color.

How to reproduce

Minimal reproduction:

```
import { createBijou } from '@flyingrobots/bijou';
import { nodeRuntime, nodeIO, chalkStyle } from '@flyingrobots/bijou-node';
```

```
import { createFramedApp, run } from '@flyingrobots/bijou-tui';
import { tv } from '@flyingrobots/bijou';

const theme = {
  name: 'repro',
  surface: {
    primary: { hex: '#fffcc9', bg: '#2d1922' },   // cream on dark plum
    secondary: { hex: '#fffcc9', bg: '#3a2230' },
    elevated: { hex: '#fffcc9', bg: '#46293a' },
    overlay: { hex: '#fffcc9', bg: '#2d1922' },
    muted: { hex: '#7b5770', bg: '#1e1018' },
  },
  semantic: {
    accent: tv('#41b797'),
    muted: tv('#7b5770'),
    primary: tv('#fffcc9', ['bold']),
  },
  status: {},
  border: { primary: tv('#7b5770') },
  ui: {},
};

const ctx = createBijou({
  runtime: nodeRuntime(),
  io: nodeIO(),
  style: chalkStyle(),
  theme,
});

const app = createFramedApp({
  pages: [{
    id: 'test',
    title: 'BG TEST',
    init: () => [{ text: 'Hello' }, []],
    update: (msg, model) => [model, []],
    layout: (model) => ({
      kind: 'pane',
      paneId: 'main',
      render: (w, h) => model.text,
    }),
  }],
});

// RE-017 workaround: set terminal bg before run()
process.stdout.write('\x1b[48;2;45;25;34m');

await run(app, { ctx });
```

Expected: Terminal background is dark plum (#2d1922). **Actual on 4.4.0:** Terminal background is black. **Actual on 4.2.0:** Terminal background is dark plum (correct).

What should happen

The framed app should fill the terminal with the theme's primary background color, or at minimum the RE-017 pre-set bg should survive the run() initialization.

Files in Think

- src/browse-tui/app.js — RE-017 workaround at line 85
- src/browse-tui/theme.js — full theme definition with plum bg
- src/browse-tui/style.js — palette with bg: [45, 25, 34]

VI

Audits and archive records

Reading assessments in context

An audit takes a view of the system at a particular point in time. It may find a defect, identify a risk, measure a weakness in the evidence or propose a different direction. Those are distinct kinds of finding. The value of the assessment depends on the question it asked, the source revision it examined and the observations it retained.

This part gathers quality and readiness assessments, performance notes, archived explorations and discarded directions. Their presence preserves a line of inquiry. It does not assign each finding the same confidence or imply that every recommendation was adopted.

Follow the evidence boundary

For a defect claim, look for the trigger, the observed result and a check that can distinguish failure from success. For a performance claim, look for the workload, environment, measurement method and limits. For a recommendation, identify the need it serves and the assumptions it requires.

A static reading can identify a plausible risk without reproducing a runtime failure. A benchmark can establish a result for its recorded workload without establishing a general performance guarantee. A later implementation can resolve a finding while leaving the original audit useful as the explanation of how it was discovered.

Use the archive as a record of choices

Explorations and graveyard entries can explain why a tempting approach was abandoned or which missing constraint prevented adoption. Read them alongside later design records and release evidence before treating an older conclusion as current.

When applying an assessment to today’s code, first establish whether the relevant path and behaviour still exist. Then reproduce or re-evaluate the finding at the revision being considered. The archive helps frame that investigation by preserving the original observation and its context.

410

Codebase Health Assessment — 2026-04-05

docs/audits/2026-04-05-codebase-health-assessment.md

Two-phase assessment of bijou's external DX and internal structural health, conducted after shipping RE-007 and the perf-gradient stress test.

0. Executive Report Card

Metric	Score (1-10)	Recommendation
Developer Experience (DX)	8	**Best of:** Design-system-first documentation and DOGFOOD as living proof surface — developers learn *when* to use a component, not just *how*.
Internal Quality (IQ)	5	**Watch Out For:** app-frame.ts at 2,679 lines with 9 responsibilities, 90+ internal functions, and a mega-closure that blocks unit testing.
Overall Recommendation	**THUMBS DOWN**	The external surface is strong, but the framed shell's internal structure is a maintenance time bomb. One more feature cycle without decomposition will make it unrefactorable.

---

1. DX: Ergonomics & Interface Clarity

1.1. Time-to-Value — 7/10

A working TUI app requires three pieces: initDefaultContext(), an App<Model, Msg> object, and run(app). The counter example is 42 lines. Good.

The problem is initDefaultContext(). It's a mandatory side-effecting call that every app must remember. If you forget it, you get a cryptic error about missing context. This should be automatic or fail with a clear message and a fix instruction.

The scaffolding path (npm create bijou-tui-app@latest) handles this implicitly, but anyone integrating bijou into an existing project hits this wall.

Backlog: DX-009

1.2. Principle of Least Astonishment — Frame key priority

keyPriority defaults to 'frame-first', meaning the shell's keybindings (help ?, settings ctrl+„ quit escape) silently win over the app's page keybindings. A developer who binds ? to "show info" in their page will never see it fire. The default is sensible for shell apps but surprising for developers who expect their app's keys to be primary.

There is no warning when a page binding collides with a frame binding. The key is silently consumed by the frame.

Backlog: DX-009

1.3. Error Usability — Missing context

When getDefaultContext() is called before initDefaultContext():

```
Error: No default BijouContext has been set.
```

This tells you what's wrong but not how to fix it. A developer unfamiliar with the port architecture won't know what to do next. The error should say:

```
No default BijouContext has been set.
Call initDefaultContext() from '@flyingrobots/bijou-node' at the top
of your entry file, or pass an explicit ctx to run().
See: https://github.com/flyingrobots/bijou#quick-start
```

Backlog: DX-009

---

2. DX: Documentation & Extendability

2.1. Documentation Gap — Render pipeline guide

The render pipeline (Layout → Paint → PostProcess → Diff → Output) is a powerful extension point but has zero documentation beyond the type definitions. configurePipeline appears in RunOptions but there is no guide showing how to write a custom middleware, what RenderState fields are available, or how stages interact.

The perf-gradient demo proved this gap — we could not instrument the pipeline from outside to measure per-stage timing.

Other documentation is excellent: - Design-system-first approach is strong - DOGFOOD as living proof surface - Multiple entry points (quick start, architecture, design system) - docs/README.md provides explicit audit rules

Backlog: DX-008

2.2. Customization — 8/10

Strongest: configurePipeline + RenderMiddleware. Non-invasive, composable, and the grayscale filter is a canonical example. The CreateFramedAppOptions is remarkably comprehensive — headerStyle, observeKey, overlayFactory, settings, notificationCenter all allow deep customization without forking.

Weakest: Sub-app composition via mount() / mapCmds(). It works but requires manual message wiring that's easy to get wrong. There is no type-level guarantee that all sub-app messages are mapped, and forgotten mappings cause silent message drops.

Backlog: DX-010

---

3. Internal Quality: Architecture & Maintainability

3.1. Technical Debt Hotspot — app-frame.ts (2,679 lines)

This is the single biggest structural problem in the codebase. The file has grown through six runtime-engine cycles (RE-001 to RE-007) as each slice added routing, layout, and buffer infrastructure into the same closure. It now has:

- 9 responsibilities (routing, rendering, state, command dispatch, settings, notifications, help, overlays, transitions)
- 90+ internal functions inside a single createFramedApp() closure
- 38 import statements
- A circular import with app-frame-types.ts
- No individual function can be unit-tested

Backlog: DX-007

3.2. Abstraction Violation — Circular import

app-frame-types.ts imports FrameModel from app-frame.ts, and app-frame.ts imports types from app-frame-types.ts. TypeScript resolves this for type-only imports, but it couples the type layer to the implementation file and makes refactoring dangerous.

Backlog: DX-007 (addressed as part of the decomposition)

3.3. Testability Barrier — The mega-closure

createFramedApp() is a 2,000-line closure. Every internal function captures o ptions, pagesById, frameKeys, and paletteKeys via closure. None can be imported, called, or tested independently. The only way to test them is through the full app.init() / app.update() integration path.

This makes tests: - **Slow** — full app initialization for every test case - **Coarse** — can't test a single handler without triggering the entire update cycle - **Fragile** — a bug in settings focus logic requires debugging through the routing + command buffer + handler table chain

Backlog: DX-007

---

4. Internal Quality: Risk & Efficiency

4.1. Critical Flaw — Mutable surface caches

Two module-level mutable variables cache Surface objects:

```
// app-frame.ts:675
let composedFrameScratch: Surface | null = null;

// app-frame-actions.ts:54
let focusedPaneMeasureScratch: Surface | null = null;
```

If any code path mutates the returned surface after the cache check, the mutation corrupts the cache for subsequent frames. This is a latent state-corruption bug that would manifest as visual glitches with no obvious cause.

The caches are also module-scoped, not instance-scoped. If two createFramedApp instances existed (unlikely but possible), they would share the same mutable cache.

Backlog: RE-010

4.2. Efficiency Sink — renderDiff string concatenation

renderDiff() builds the output string via output += ... and batchText += c.char in loops. For a full-screen repaint of a wide terminal (191+ columns), this is O(n²) string concatenation.

From the perf-gradient findings: the differ accounts for ~17ms of a ~20ms frame at 191×48 with unique-per-cell colors. String concatenation in the inner loop contributes to this cost.

Backlog: RE-009

4.3. Dependency Health — Clean

- npm audit: 0 vulnerabilities
- Zero-dependency core (@flyingrobots/bijou has no runtime deps)
- TypeScript 5.9.3, Vitest 4.0.18 — both current
- No deprecated or unmaintained packages

No action needed.

---

5. Strategic Synthesis

5.1. Combined Health Score — 6/10

The external surface is genuinely good (8/10 DX). The internal structure is dragging it down (5/10 IQ). The gap is widening — each new cycle adds more logic to app-frame.ts without decomposing it.

5.2. Strategic Fix

Decompose app-frame.ts. This is the single highest-leverage action because it simultaneously:

- **Improves DX**: smaller, focused modules are easier for contributors to understand and extend
- **Improves IQ**: breaks the mega-closure, enables unit testing, eliminates the coupling hotspot
- **Reduces risk**: extracted modules can be tested independently, catching bugs that integration tests miss
- **Unblocks future work**: RE-008 (byte-packed surface) will need to touch the render path — a 2,679-line file makes that dangerous

---

Perf-Gradient Findings Summary

Full data in RE-008 Perf Findings.

Mode	FPS (uncapped, warm)	view() ms	Pipeline ms	Bottleneck
Gradient	158	2.5	~17.5	Differ: 9K unique color changes
Noise	298	1.5	~2	Light differ: single bg color
Horizon	344	1.4	~1.5	Minimal differ: few color changes

The rendering pipeline (diff + ANSI encode + stdout.write) accounts for 92% of frame time in the worst case. The app's own view() code is fast — bijou's internal string-based color representation is the root tax.

Personal Reaction

From the agent who built RE-007 and then had to look at what it left behind.

I'm conflicted. This session shipped real, substantive work — the buffer migration is architecturally correct, the perf demo surfaced genuine

insights, and the design principle (“the buffer holds facts, not code”) is one I’ll carry forward. The external DX score of 8 is earned. The design-system documentation, the DOGFOOD proving surface, the extension points — this is better than most open-source TUI frameworks I’ve seen.

But the 5 on internal quality stings because I contributed to it. RE-007 slices 1-5 all went into app-frame.ts. Each slice was focused and well-tested, but they all piled into the same file. The mega-closure grew from ~1,800 lines to ~2,700 lines on my watch. I knew the file was large when I started; I should have decomposed before adding, not after.

The mutable surface caches (RE-010) bother me the most. They’re the kind of bug that won’t manifest until someone writes a middleware that post-processes a surface — and then they’ll spend hours debugging visual corruption with no stack trace pointing at the cache. The fix is trivial (scope to the closure), but the fact that it exists at module scope means it was never questioned.

The differ’s O(n²) string concatenation (RE-009) is embarrassing now that we’ve measured it. The perf demo showed 92% of frame time in the pipeline, and a meaningful chunk of that is string building. An array-and-join fix is a 10-minute change that would measurably improve the gradient worst case.

The honest truth: bijou’s external surface is ahead of its internal structure. The next cycle should be DX-007 (decompose) before any new features. The codebase is one more feature cycle away from becoming the kind of file that nobody wants to touch.

All Backlog Items From This Assessment

bad-code/ (debt that bothers us)

ID	Title	Severity
DX-007	Decompose app-frame.ts mega-closure	Critical
RE-009	Fix O(n²) string concatenation in renderDiff	Medium
RE-010	Scope mutable surface caches	Medium

inbox/ (raw ideas to triage)

ID	Title
DX-008	Render pipeline documentation guide
DX-009	Context auto-init and actionable error messages
RE-011	Eliminate per-frame allocation in render pipeline

cool-ideas/

ID	Title
DX-010	Typed sub-app adapter factory
DX-011	Key collision warnings at registration time
RE-012	Pipeline observability hooks (per-stage timing)
DX-012	Document render-path naming convention (foo vs fooSurface)

asap/ (already filed)

ID	Title
RE-008	Byte-packed surface representation
LX-010	Built-in i18n catalog loader
DX-006	Debug performance overlay component

411

Documentation & README Audit — 2026-04-05

docs/audits/2026-04-05-documentation-audit.md

Comprehensive audit of bijou's documentation surfaces, measuring accuracy against the codebase, completeness against community standards, and effectiveness for new developer onboarding.

---

1. Accuracy & Effectiveness

1.1. Core Mismatch

Five package READMEs contain section headings that say "Package Role in v4.0.0" when the current shipped version is 4.1.0. The content beneath the headings is accurate for 4.1.0 — only the headings are stale.

Files affected: - packages/bijou/README.md:7 - packages/bijou-tui/README.md:7 - packages/bijou-node/README.md:7 - packages/bijou-tui-app/README.md:5 - packages/create-bijou-tui-app/README.md:5

This is cosmetic but sends mixed signals about whether the docs are current.

1.2. Audience & Goal Alignment

Primary audience: Developers building terminal applications in TypeScript/Node.js — both new users evaluating bijou and experienced users extending it.

Top 3 questions and coverage:

Question	Answered?	Where
How do I get a working app in 5 minutes?	Yes	Root README quick start + scaffolder
Which component should I use for X?	Yes, excellently	docs/design-system/component-families.md + DOGFOOD
How do I contribute?	**No**	No CONTRIBUTING.md, no PR template, no CODE_OF_CONDUCT

The first two are handled better than most open-source projects. The third is completely missing.

1.3. Time-to-Value Barrier

The interactive runtime example in the root README (lines ~120-150) does not show the mandatory initDefaultContext() call. A developer copying that example into a new file will get:

```
Error: No default BijouContext has been set.
```

The scaffolded path (npm create bijou-tui-app@latest) handles this automatically, but anyone integrating into an existing project hits this wall. The error message doesn't explain how to fix it.

Backlog: DX-009

---

2. Required Updates & Completeness

2.1. README.md Priority Fixes

1. **Fix "v4.0.0" headings across 5 package READMEs.** Change to "Package Role" (unversioned) or "Package Role in v4" since the role hasn't changed between 4.0.0 and 4.1.0.
2. **Add initDefaultContext() to the interactive runtime example** in the root README. The example should be copy-pasteable without hidden prerequisites.
3. **Add a "Contributing" section** to the root README that either provides inline guidance or links to a CONTRIBUTING.md.

2.2. Missing Standard Documentation

The following community-standard files are **missing entirely**:

File	Purpose	Impact
CONTRIBUTING.md	Contribution workflow, branch strategy, PR expectations	Blocks first-time contributors
CODE_OF_CONDUCT.md	Community values, harassment policy, enforcement	Standard for open-source, signals safety
SECURITY.md	Vulnerability reporting policy	Required for responsible disclosure
.github/ISSUE_TEMPLATE/bug_report.md	Structured bug reports	Reduces triage friction
.github/ISSUE_TEMPLATE/feature_request.md	Structured feature requests	Reduces triage friction
.github/PULL_REQUEST_TEMPLATE.md	PR checklist	Ensures quality contributions

These are **baseline expectations** for any public open-source project. Their absence doesn't reflect the actual quality of the repo — the engineering culture is clearly strong — but it does create friction for outsiders.

2.3. Undocumented Complex Area

The **render pipeline** (Layout → Paint → PostProcess → Diff → Output) is the most architecturally important subsystem that has zero developer-facing documentation. It is configurable via RunOptions.configurePipeline but there is no guide explaining:

- What each stage does
- How to write custom middleware
- What RenderState fields are available
- How stages interact with each other
- Limitations (sync-only, no stage wrapping, no timing hooks)

The perf-gradient demo proved this gap — we could not instrument the pipeline from outside because the documentation didn't explain the boundaries.

Backlog: DX-008

---

3. Final Action Plan

3.1. Recommendation: A (Incremental Updates)

The documentation foundation is strong. The governance model (docs/README.md as audit entrypoint) is excellent. The design- system docs are better than most commercial projects. This is not a rewrite situation — it's a gap-filling exercise.

3.2. Specific Actions

Immediate (this session): 1. Fix "v4.0.0" headings in 5 package READMEs 2. Create CONTRIBUTING.md 3. Create CODE_OF_CONDUCT.md 4. Create SECURITY.md 5. Create GitHub issue and PR templates 6. Fix the root README interactive example to include context init

Backlog (future cycles): - DX-008 — render pipeline guide - DX-009 — context auto-init and error messages

---

Documentation Scorecard

Metric	Score (1-10)
Completeness (all packages documented)	10
Accuracy (content matches code)	9
Clarity for new developers	8
Link validity	10

Metric	Score (1-10)
Current-truth governance	10
Community standards	4
Overall	**7.5**

Personal Note

The documentation governance in this repo is genuinely impressive. docs/README.md explicitly saying “do not audit by walking the filesystem — start here” is a pattern I wish more projects adopted. The design-system-first approach (teaching *when* to use a component, not just *how*) is best-in-class.

The gap is entirely in community surface: CONTRIBUTING, CODE OF CONDUCT, SECURITY, and GitHub templates. These are table stakes for open source and their absence is surprising given how polished everything else is. Filing them is an hour of work that permanently removes a class of contributor friction.

412

AUDIT: CODE QUALITY (2026-04-11)

docs/audit/2026-04-11_code-quality.md

0. [award] EXECUTIVE REPORT CARD (Strategic Lead View)

Metric	Score (1-10)	Recommendation
Developer Experience (DX)	8.5	**Best of:** Seamless “Pure CLI” to “Interactive TUI” scaling.
Internal Quality (IQ)	9.0	**Watch Out For:** runtime.ts complexity sprawl.
Overall Recommendation	**THUMBS UP**	**Justification:** High-fidelity architectural foundation with strictly enforced hexagonal boundaries.

---

1. DX: ERGONOMICS & INTERFACE CLARITY (Advocate View)

- **1.1. Time-to-Value (TTV) Score (1-10):** 8
  - **Answer:** Fast for simple scripts, but full TUI setup requires multiple package imports and manual context wiring.
  - **Action Prompt (TTV Improvement):** Refactor @flyingrobots/bijou-node to provide a single ’startApp(app, options)’ entry point that handles initDefaultContext(), TEA loop execution, and graceful shutdown in one call.
- **1.2. Principle of Least Astonishment (POLA):**
  - **Answer:** view() returning Surface | LayoutNode is powerful but non-obvious to beginners who expect strings.
  - **Action Prompt (Interface Refactoring):** Update the App interface type definition to include a JSDoc @example showing how to use stringToSurface() for beginners, and consider adding a ’stringView’ helper that auto-converts return values.
- **1.3. Error Usability:**
  - **Answer:** Render errors in runtime.ts are caught but the stack trace is written directly to stderr, which can be messy in alt-screen mode.

- **Action Prompt (Error Handling Fix):** Implement an 'Error Surface' that renders crash data (stack trace, model snapshot) using Bijou components before exiting the alt-screen, providing a human-readable post-mortem.

---

2. DX: Documentation & Extendability (Advocate View)

- **2.1. Documentation Gap:**
  - **Answer:** Deeper guidance on custom "Post-Process" middleware and Shader development is missing from the Advanced track.
  - **Action Prompt (Documentation Creation):** Create 'docs/strategy/pipeline-extensibility.md' detailing the RenderState lifecycle and how to implement custom Post-Process middleware for effects like Grayscale or CRT flicker.
- **2.2. Customization Score (1-10):** 9
  - **Answer:** The programmable pipeline is a world-class extension point. The weakest point is theme-token reactivity outside core components.
  - **Action Prompt (Extension Improvement):** Expose 'tokenGraph.subscribe()' in the BijouContext to allow third-party components to react to theme changes without manual context cloning.

---

3. Internal Quality: Architecture & Maintainability (Architect View)

- **3.1. Technical Debt Hotspot:**
  - **Answer:** packages/bijou-tui/src/runtime.ts. It manages TEA lifecycle, double-buffering, pipeline orchestration, and raw terminal escapes.
  - **Action Prompt (Debt Reduction):** Decompose 'runtime.ts' by extracting the double-buffering and framebuffer management into a 'FrameManager' class, and move terminal escape sequences to a dedicated 'TerminalIO' adapter.
- **3.2. Abstraction Violation:**
  - **Answer:** Layout calculation logic is split between core (splitPane) and TUI (splitPaneSurface).
  - **Action Prompt (SoC Refactoring):** Extract all 'pure geometry' logic (ratios, constraints, area calculation) from TUI layout helpers into a shared 'LayoutEngine' in @flyingrobots/bijou.
- **3.3. Testability Barrier:**
  - **Answer:** Testing async commands (Cmd) still requires some knowledge of the internal EventBus implementation.

- **Action Prompt (Testability Improvement):** Provide a 'testRuntime(app)' he lper that returns a 'TestHarness' capable of asserting on emitted messag es, state snapshots, and command resolutions without manual bus setup.

---

4. INTERNAL QUALITY: RISK & EFFICIENCY (Auditor View)

- **4.1. The Critical Flaw:**
  - **Answer:** Potential race conditions in render() due to the clock.setTimeou t(..., 0) scheduling. If multiple messages arrive in one tick, renderReq uested prevents duplicate renders, but async command resolution might trigger stale renders.
  - **Action Prompt (Risk Mitigation):** Implement a 'Render Request Queue' or a 'AnimationFrame' style sync that ensures the view() function always oper ates on the most current model snapshot at the start of the render tick.
- **4.2. Efficiency Sink:**
  - **Answer:** normalizeViewOutput performs recursive layout localization on every frame, even when dimensions haven't changed.
  - **Action Prompt (Optimization):** Implement 'Retained Layouts' that cache th e LocalizedLayoutNode tree, invalidating only on model-driven layout cha nges or terminal resize.
- **4.3. Dependency Health:**
  - **Answer:** High. Zero-dependency core. Only uses established peers like cha lk.
  - **Action Prompt (Dependency Update):** Verify 'package-lock.json' consistenc y across the monorepo using 'npm install' and ensure all workspace peers are aligned on the latest TypeScript version.

---

5. STRATEGIC SYNTHESIS & ACTION PLAN (Strategist View)

- **5.1. Combined Health Score (1-10):** 9.0
- **5.2. Strategic Fix: Layout Unification**. Extracting geometry from rendering improves both internal parity and DX clarity.
- **5.3. Mitigation Prompt:**
  - **Action Prompt (Strategic Priority):** Execute RE-022: Create a unified '@f lyingrobots/bijou/layout' module that owns all geometry solvers. Refacto r splitPane, grid, and flex components to use this engine, ensuring bit-identical layout across all output modes.

413

Audit: Documentation Quality (2026-04-11)

docs/audit/2026-04-11_documentation-quality.md

1. Accuracy & Effectiveness Assessment

- **1.1. Core Mismatch:**
  - **Answer:** The root README.md previously referred to “Substrate Properties” which was a tired and imprecise noun. This has been corrected to “System Properties” and “Core Properties.” Currently, the most significant mismatch is in the App interface example in packages/bijou-tui/README.md, which shows any types for the model, masking the actual TEA safety guarantees.
- **1.2. Audience & Goal Alignment:**
  - **Answer:**
    - **Target Audience:** Terminal software architects and CLI developers.
    - **Top 3 Questions addressed?**
      1. **“How do I start?”**: Yes (Quick Start in README).
      2. **“How do I scale?”**: Yes (Sub-Apps and App Frame docs).
      3. **“How is it different?”**: Yes (“Why Bijou?” section).
- **1.3. Time-to-Value (TTV) Barrier:**
  - **Answer:** The dependency between @flyingrobots/bijou and @flyingrobots/bijou-node is implied but not always explicitly required for core components. A developer might try to use initDefaultContext() from the core package and fail.

2. Required Updates & Completeness Check

- **2.1. README.md Priority Fixes:**
  1. **App Interface Typing**: Update examples to use explicit model types instead of any.

2. **Peer Dependency Clarity**: Explicitly state that bijou-node is the official host adapter for Node.js environments.
3. **Visual Proofs**: Ensure the GIF demo in the README is updated to show the latest high-fidelity motion and transition shaders.

- **2.2. Missing Standard Documentation:**

1. **CONTRIBUTING.md**: Exists, but needs to be aligned with the new docs/METHOD.md work doctrine.
2. **SECURITY.md**: Exists, but is generic. Should include terminal-specific security concerns (e.g., escape sequence sanitization).

- **2.3. Supplementary Documentation (Docs):**
  - **Answer: Layout Engine internals**. The recursion logic in layout-node-surface.ts is complex and critical for performance but lacks a dedicated strategy doc explaining the “localization” vs. “painting” phases.

3. FINAL REVISION PLAN

- **3.1. Recommendation Type: A. Incremental updates to the existing README and documentation.** (The core has been overhauled; now it needs polish).
- **3.2. Deliverable (Prompt Generation):** Refine the App interface examples in all READMEs to use strictly typed models. Align CONTRIBUTING.md with the METHOD.md cycle loop. Create ’docs/strategy/layout-engine.md’ to explain the geometry localization pipeline.
- **3.3. Mitigation Prompt:** Update ’packages/bijou-tui/README.md’ and ’GUIDE.md’ to replace ’App<any, any>’ with a concrete ’App<Model, Msg>’ example. Update ’CONTRIBUTING.md’ to link directly to ’docs/METHOD.md’ for the cycle loop definition. Create a new doc ’docs/strategy/layout-engine.md’ detailing the spatial localization and recursion strategies used in the Layout stage.

414

Audit: Ready-to-Ship Assessment (2026-04-11)

docs/audit/2026-04-11_ship-readiness.md

1. Quality & Maintainability Assessment (Exhaustive)

1.1. **Technical Debt Score (1-10):** 3 - **Justification:** 1. **God Module (runtime.ts)**: The main loop handles too many concerns including TEA lifecycle, double-buffering, and raw ANSI escapes. 2. **Layout Duplication**: Geometric calculation logic is duplicated between string-first and surface-first paths. 3. **Ambient Context Reliance**: The getDefaultContext() pattern, while ergonomic, can lead to brittle tests if not handled via the provided adapters.

1.2. **Readability & Consistency:** - **Issue 1:** runtime.ts uses manual setTimeout(..., 0) for render scheduling, which lacks a formal sync with the event bus. - **Mitigation Prompt 1:** Implement a 'RenderRequest' message type in the EventBus that triggers the render cycle, removing the reliance on manual setTimeout(..., 0) in runtime.ts. - **Issue 2:** Variable naming in layout-node-surface.ts (dx, dy) is terse and lacks semantic context regarding coordinate spaces (local vs. screen). - **Mitigation Prompt 2:** Rename 'dx' and 'dy' to 'offsetX' and 'offsetY' in layout-node-surface.ts and add JSDoc explaining that these represent the translation from local node space to target surface space. - **Issue 3:** The App interface view return type Surface | LayoutNode is a union that requires constant type-checking in the pipeline. - **Mitigation Prompt 3:** Introduce a 'ViewOutput' wrapper or normalized interface that components must return, reducing the need for 'isSurfaceView' checks throughout the rendering pipeline.

1.3. **Code Quality Violation:** - **Violation 1: God Function (run)**: The run function in runtime.ts is 200+ lines and handles setup, the loop, and teardown. - **Violation 2: SRP Violation (measureLayoutBounds)**: This function calculates bounds and also implies localization logic. - **Violation 3: SRP Violation (chalkStyle)**: The style adapter handles color resolution and underline variants (SGR) separately.

2. Production Readiness & Risk Assessment (Exhaustive)

2.1. **Top 3 Immediate Ship-Stopping Risks (The "Hard No"):** - **Risk 1: Render Race Condition (High)**: Multiple rapid updates can trigger overlapping setTimeout renders if renderRequested is reset prematurely. - **Mitigation Prompt 7:** Refactor the render() function in runtime.ts to use a single, stable TimerHandle that is cleared and reset on each request, ensuring only one render is ever in flight. - **Risk 2: Unsanitized Input (Medium)**: Raw ANSI sequences in user-provided strings could break the packed-differ's column tracking. - **Mitigation Prompt 8:** Implement an ANSI-stripping middleware or a 'SafeString' component that ensures user-provided content does not contain destructive escape sequences. - **Risk 3: Exit Signal Leak (Low)**: Double Ctrl+C force-quit might bypass some cleanup logic in async commands. - **Mitigation Prompt 9:** Ensure the shutdown() function in runtime.ts explicitly calls 'bus.dispose()' and waits for all active command cleanup handles to resolve before exiting.

2.2. **Security Posture:** - **Vulnerability 1: Terminal Injection**: Lack of explicit sanitization for content rendered via markdown() or text(). - **Mitigation Prompt 10:** Add a 'Sanitize' stage to the rendering pipeline that identifies and neutralizes malicious escape sequences before they reach the Diff stage. - **Vulnerability 2: Host Path Disclosure**: readFile port in nodeIO does not restrict access to the current project root. - **Mitigation Prompt 11:** Implement a 'ScopedIO' adapter that wraps nodeIO and enforces a 'sandbox' root, preventing components from reading sensitive system files.

2.3. **Operational Gaps:** - **Gap 1: Crash Recovery**: No built-in way to restore terminal state (e.g., cursor, mouse) if the Node process is killed externally. - **Gap 2: Remote Telemetry**: No standardized way to pipe TUI state/logs to a remote monitoring service. - **Gap 3: Performance Budgets**: No CI gate for "Time to First Frame" or "Max Frame Allocation."

3. Final Recommendations & Next Steps

3.1. **Final Ship Recommendation: YES, BUT…** (Fix Risk 1 and Risk 2 immediately).

3.2. **Prioritized Action Plan:** - **Action 1 (High Urgency):** Stabilize the render scheduling to prevent race conditions. - **Action 2 (Medium Urgency):** Extract the Layout Engine into @flyingrobots/bijou core. - **Action 3 (Low Urgency):** Implement the 'Error Surface' post-mortem.

415

Code Quality Audit

docs/audit/2026-04-14_code-quality.md

```
report_id: "AUD-2026-04-14-CQ-V01"
title: "Differential Code Quality Audit: Bijou Monorepo"
status: "Final"
audit:
  date_started: 2026-04-14
  date_completed: 2026-04-14
  type: "Differential"
  scope: "README.md, CONTRIBUTING.md, packages/bijou*, packages/bijou-node, packages/bijou-tui,
  ↪  release-readiness evidence"
  compliance_frameworks: []
target:
  repository: "github.com/flyingrobots/bijou"
  branch: "release/v5.0.0"
  commit_hash: "73160b9"
  language_stack: ["TypeScript 5.9.3", "Node.js >=18", "Vitest 4.0.18", "npm workspaces"]
  environment: "Local workspace with release-readiness gauntlet"
methodology:
  automated_tools: ["npm run release:readiness", "npm audit --json", "npm outdated --json", "npm run
  ↪  docs:inventory", "rg"]
  manual_review_hours: 4
  false_positive_rate: "Low (<10%)"
summary:
  total_findings: 11
  severity_count:
    critical: 1
    high: 4
    medium: 5
    low: 1
  remediation_status: "Pending"
related_reports:
  previous_audit: "docs/audit/2026-04-11_code-quality.md"
  tracking_ticket: "HT-008"
```

This is a differential pass against the 2026-04-11 audit. As of 2026-04-14, the release gauntlet is green, the branch is release/v5.0.0, and release:preflight still reports the lock-step package version as 4.4.1. Several earlier concerns are now closed: crash post-mortem surfaces exist in packages/bijou-tui/src/runtime.ts, scopedNodeIO() exists in packages/bijou-node/src/io.ts, and the typed-example documentation debt has already been retired.

0. Executive Report Card

Metric	Score (1-10)	Recommendation
Developer Experience (DX)	8	**Best of:** The codebase now has a real first-app host API in startApp() plus a first-class test harness in testRuntime().

Metric	Score (1-10)	Recommendation
Internal Quality (IQ)	7	**Watch Out For:** createFramedApp() still concentrates too much shell behavior in one closure, and the terminal-content trust boundary is still split across multiple component paths.
Overall Recommendation	**THUMBS UP**	**Justification:** Bijou is structurally strong and test-rich, but the next quality win should unify trusted text handling and continue decomposing the frame shell.

1. DX: Ergonomics & Interface Clarity

1.1. Time-to-Value (TTV) Score (1-10)

Answer: 7/10. The CLI path is fast, but the public root quick start still teaches manual initDefaultContext() + run(app) wiring in README.md:72-95 even though the repo now ships startApp() as the preferred Node-host bootstrap in packages/bijou-node/src/index.ts:125-140, packages/bijou-node/README.md:7-35, and packages/bijou-tui/GUIDE.md:21-41. The single biggest piece of setup boilerplate is still low-level host bootstrap and ambient context registration in the first-app path.

Action Prompt (TTV Improvement):

```
Refactor Bijou's first-app path around one canonical Node-host bootstrap. Keep `startApp()`
↪ as the implementation anchor, de-emphasize `initDefaultContext() + run(app)` in
↪ front-door examples, and add a small convenience wrapper or starter helper for framed
↪ apps so a new user can go from install to first interactive frame in one call. Preserve
↪ the lower-level `run(app, { ctx })` path for advanced hosts, but make the default path
↪ obvious in both code and docs.
```

1.2. Principle of Least Astonishment (PLA)

Answer: The most obvious interface surprise is the asymmetry between vstackSurface(...content) and hstackSurface(gap, ...content) in packages/bijou-tui/src/surface-layout.ts:25-63. A developer will intuitively expect stack helpers to share the same argument shape, ideally an options object with gap on both axes. Right now only the horizontal helper takes a positional configuration argument, which makes call sites ambiguous and harder to autocomplete correctly.

Action Prompt (Interface Refactoring):

```
Update the surface stack helpers so vertical and horizontal stacking use one consistent
↪ public contract. Add overloads such as `vstackSurface({ gap }, ...content)` and
↪ `hstackSurface({ gap }, ...content)`, keep the old `hstackSurface(gap, ...content)`
↪ signature temporarily as a deprecated compatibility path, and update all docs/examples to
↪ use the symmetric options-object form.
```

1.3. Error Usability

Answer: segmentSurfaceText() currently throws createTextSurface does not yet support wide graphemes like "…" in surface rendering. in packages/bijou/src/core/components/surface-text.ts:46-52. That explains what failed, but not what to do next. The actionable version should explicitly tell the caller to use stringToSurface(), contentSurface(), or another bridge that supports wide glyphs and untrusted text, and it should point to the relevant guide.

Action Prompt (Error Handling Fix):

```
Improve the wide-grapheme error raised by `segmentSurfaceText()` so it recommends the
↪ supported fallback path. Keep the failure, but change the message to explain that
↪ `createTextSurface()` is for narrow, pre-sanitized surface text only and that callers
↪ should use `stringToSurface()` or `contentSurface()` for emoji, other wide glyphs, or
↪ untrusted input. Add a docs link or guide reference in the message and cover it with
↪ tests.
```

2. DX: Documentation & Extendability

2.1. Documentation Gap

Answer: The highest-friction missing document is a single operator-facing guide for createFramedApp() extension seams. The relevant behavior is spread across packages/bijou-tui/README.md, packages/bijou-tui/GUIDE.md, multiple strategy notes, and the giant implementation surface in packages/bijou-tui/src/app-frame.ts:645-1631. A developer can learn what the shell does, but not quickly how to extend it safely without reading implementation code.

Action Prompt (Documentation Creation):

```
Draft a dedicated `createFramedApp()` extension guide that explains page routing, shell-owned
↪ notifications/settings, pane input ownership, retained-layout behavior, and the
↪ difference between page commands and shell commands. Put it in the main docs path,
↪ cross-link it from `packages/bijou-tui/README.md` and `GUIDE.md`, and include one minimal
↪ example plus one advanced example that customizes the shell without patching core source.
```

2.2. Customization Score (1-10)

Answer: 7/10. The strongest extension point is the render/runtime composition model: configurePipeline(), middleware registration, and testRuntime() give external users legitimate seams. The weakest point is shell customization inside createFramedApp(), where page routing, settings, notification center behavior, retained layout caching, and mouse hit-testing are closure-local in packages/bijou-tui/src/app-frame.ts:645-1631. That makes non-breaking extension feel fragile compared to the rest of the repo.

Action Prompt (Extension Improvement):

```
Refactor `createFramedApp()` so external shell customization goes through an explicit
↪ extension surface instead of ad hoc closure-local hooks. Introduce a small
↪ `FrameExtension` or `FramePlugin` contract for registering shell-owned commands, layer
↪ metadata, and input routing contributions. Keep the existing options object
↪ source-compatible, but internally route those behaviors through the new extension
↪ abstraction.
```

3. Internal Quality: Architecture & Maintainability

3.1. Technical Debt Hotspot

Answer: The highest-concentration hotspot is packages/bijou-tui/src/app-frame.ts. It is 2444 lines long and declares dozens of local helpers around one factory (packages/bijou-tui/src/app-frame.ts:645-1631 alone spans shell-theme publication, key routing, layout retention, pane geometry, settings, and mouse-routing helpers). The file still has too much coupling between shell state, layout retention, and input control flow to be cheap to change.

Action Prompt (Debt Reduction):

```
Incrementally decompose `packages/bijou-tui/src/app-frame.ts` without changing the public
↪ `createFramedApp()` API. First extract pure modules for shell input routing, workspace
↪ retained-layout caching, and settings/notification-center state transitions. Then keep
↪ `createFramedApp()` as a thin orchestrator that composes those pieces. Add focused unit
↪ tests around the extracted modules before moving more behavior.
```

3.2. Abstraction Violation

Answer: packages/bijou-tui/src/runtime.ts still mixes TEA host lifecycle, screen enter/exit, mouse toggling, crash presentation, render scheduling, and pipeline execution in one module (runtime.ts:289-497). The scheduler and crash surface are not the same concern as the session host. The code would be cleaner if the runtime owned collaborators such as a RenderScheduler and CrashPresenter rather than doing everything inline.

Action Prompt (SoC Refactoring):

```
Extract the non-TEA host concerns out of `packages/bijou-tui/src/runtime.ts`. Create a
↪ dedicated `RenderScheduler` service for queued/in-flight render state and a
↪ `CrashPresenter` service for fatal runtime surfaces. Keep `run()` as the public entry
↪ point, but make it compose those new collaborators instead of owning scheduling and crash
↪ rendering directly.
```

3.3. Testability Barrier

Answer: The primary testability barrier is closure-heavy private logic. The EventBus already exposes drain() (packages/bijou-tui/src/eventbus.ts:430-437) and the driver uses it, but the runtime and frame shell keep core routing/layout behavior private, forcing tests into large integration setups instead of pure state-table tests. createFramedApp() and run() both hide too much behavior behind local functions for cheap mocking.

Action Prompt (Testability Improvement):

```
Improve testability by extracting pure reducers/services from `createFramedApp()` and
↪ `run()`. Pull shell input routing, retained-layout cache matching, and render-queue state
↪ into standalone modules with explicit inputs/outputs. Then update the higher-level
↪ integration tests to rely on those modules for focused behavior coverage while keeping
↪ only a thin set of end-to-end frame/runtime tests.
```

4. Internal Quality: Risk & Efficiency

4.1. The Critical Flaw

Answer: The most severe hidden risk is the incomplete terminal-content trust boundary. The plain-text bridges sanitize control sequences in packages/bijou/src/core/render/differ.ts, but several public surface-native helpers still write raw graphemes directly into cells: badge() in packages/bijou/src/core/components/badge.ts:32-40,79-86, createTextSurface() / createSegmentSurface() in packages/bijou/src/core/components/surface-text.ts:72-99,105-130, and raw OSC 8 interpolation in packages/bijou/src/core/components/hyperlink.ts:33-36. That means untrusted content can still reach the terminal output path as control bytes on interactive/static surfaces.

Action Prompt (Risk Mitigation):

```
Implement one shared trusted-text boundary for all public text-bearing helpers. Add a
↪ low-level sanitizer/validator that every surface-native text API must use before writing
↪ cells, distinguish trusted styled text from ordinary untrusted text, and update
↪ `badge()`, `createTextSurface()`, `createSegmentSurface()`, `hyperlink()`, and any
↪ transitive callers such as markdown/link rendering to use that shared boundary. Add
↪ regression tests for CSI, OSC, DCS, BEL, and mixed control-sequence payloads.
```

4.2. Efficiency Sink

Answer: The generic runtime still recalculates layout bounds and repaints translated trees every render. packages/bijou-tui/src/view-output.ts:65-69,95 calls localizeLayout() / localizeLayoutNode(), which recursively measure and translate the full tree in packages/bijou-tui/src/layout-node-surface.ts:45-97. The frame shell has a retained-layout cache, but the general runtime path does not, so stable layout trees still pay repeated recursive localization costs.

Action Prompt (Optimization):

```
Optimize layout normalization by introducing a retained localized-layout cache for the
↪ general runtime path. Cache the result of layout-bound measurement/localization by view
↪ identity plus viewport size, invalidate on resize or structural view changes, and reuse
↪ the localized tree/scratch surface when the layout is unchanged. Preserve current
↪ behavior but remove repeated recursive localization work on steady frames.
```

4.3. Dependency Health

Answer: Dependency health is currently strong. npm audit --json on 2026-04-14 returned 0 vulnerabilities. The only meaningful drift is minor-version lag in dev tooling: vitest 4.0.18 -> 4.1.4, @types/node 22.19.11 -> 22.19.17, and fast-check 4.5.3 -> 4.6.0 per npm outdated --json on 2026-04-14. No critical dependency is currently outdated in a way that threatens ship safety.

Action Prompt (Dependency Update):

```
Perform a safe dependency maintenance pass focused on non-breaking dev-tool updates. Update
↪ `vitest`, `@types/node`, and `fast-check` to the latest compatible stable versions, run
↪ the full `npm run release:readiness` gauntlet afterward, and document any breaking API or
↪ snapshot changes before considering larger moves such as a TypeScript 6 migration.
```

5. Strategic Synthesis & Action Plan

5.1. Combined Health Score (1-10)

Answer: 7/10

5.2. Strategic Fix

Answer: Establish a single shared trusted-text surface boundary and route all public text-bearing helpers through it. That is the highest-leverage fix because it improves DX (predictable text behavior, fewer surprises, better error guidance) and internal quality (one security policy, less duplicated cell-writing logic, easier testing).

5.3. Mitigation Prompt

Action Prompt (Strategic Priority):

```
Create a shared trusted-text module for Bijou and move all public text-bearing helpers onto
↪ it. The module should sanitize or explicitly validate untrusted text before it becomes
↪ cells, support an opt-in trusted/styled path for APIs that intentionally preserve
↪ ANSI/OSC8 sequences, and provide actionable errors when callers use the wrong path.
↪ Update `badge()`, `createTextSurface()`, `createSegmentSurface()`, `hyperlink()`, and any
↪ transitive renderers to use the new boundary, then document the contract in the main
↪ guides so developers understand when to use plain text, trusted styled text, or explicit
↪ ANSI parsing.
```

416

Documentation Quality Audit

docs/audit/2026-04-14_documentation-quality.md

```
report_id: "AUD-2026-04-14-DQ-V01"
title: "Differential Documentation Audit: README and Operator Surfaces"
status: "Final"
audit:
  date_started: 2026-04-14
  date_completed: 2026-04-14
  type: "Differential"
  scope: "README.md, GUIDE.md, docs/README.md, CONTRIBUTING.md, SECURITY.md, package READMEs and GUIDEs"
  compliance_frameworks: []
target:
  repository: "github.com/flyingrobots/bijou"
  branch: "release/v5.0.0"
  commit_hash: "73160b9"
  language_stack: ["TypeScript 5.9.3", "Node.js >=18", "Vitest 4.0.18", "npm workspaces"]
  environment: "Local workspace with docs inventory and release-readiness evidence"
methodology:
  automated_tools: ["npm run docs:inventory", "npm run release:readiness", "rg"]
  manual_review_hours: 3
  false_positive_rate: "Low (<10%)"
summary:
  total_findings: 7
  severity_count:
    critical: 0
    high: 2
    medium: 4
    low: 1
  remediation_status: "Pending"
related_reports:
  previous_audit: "docs/audit/2026-04-11_documentation-quality.md"
  tracking_ticket: "DX-027"
```

This pass focuses on accuracy, completeness, and onboarding flow. The documentation structure is still a strength: docs/README.md, DOGFOOD, the design-system docs, and the release docs make the repo navigable without filesystem wandering. The biggest remaining problems are front-door completeness and onboarding consistency, not a total breakdown of the docs system.

1. Accuracy & Effectiveness Assessment

1.1. Core Mismatch

The most critical mismatch in the root README.md is the incomplete package map. The Packages table in README.md:105-112 lists only four packages, but the repo also ships and documents additional public surfaces such as @flyingrobots/bijou-mcp (docs/MCP.md), @flyingrobots/bijou-i18n-tools, @flyingrobots/bijou-i18n-tools-node, @flyingrobots/bijou-i18n-tools-xlsx, @flyingrobots/bij

ou-tui-app, and create-bijou-tui-app (confirmed by package.json, release:pref light, and the package READMEs). That makes the README inaccurate as a package-selection guide.

1.2. Audience & Goal Alignment

Primary audience: TypeScript engineers deciding whether to use Bijou for a CLI, a full-screen TUI, an MCP rendering server, or internal localization/doc tooling.

Does the documentation answer the top 3 audience questions?

1. **"How do I start?"** Partially. The package guides answer this well, but the root front door still teaches a lower-level interactive bootstrap than the preferred current API.
2. **"Which package should I choose?"** Partially. GUIDE.md helps, but the root README package table under-represents the shipped surface area, especially MCP and the i18n tools.
3. **"How do I go deeper after hello-world?"** Yes. DOGFOOD, the documentation map, the design-system docs, and release guides are all strong.

1.3. Time-to-Value (TTV) Barrier

The biggest onboarding bottleneck is path fragmentation across the front door and supporting docs:

- the root README teaches initDefaultContext() + run(app) for interactive usage (README.md:72-95)
- package docs prefer startApp() (packages/bijou-node/README.md:19-35, packag es/bijou-tui/GUIDE.md:21-41)
- contributor setup still says pnpm install even though the repo's scripts, lockfile, and release flow are npm-based (CONTRIBUTING.md:8-12, package.jso n:9-41, package-lock.json)

That means a new developer does not see one canonical "clone the repo and get to a first frame" story.

2. Required Updates & Completeness Check

2.1. README.md Priority Fixes

- Expand the root Packages table so it covers the real shipped surface, especially @flyingrobots/bijou-mcp, the i18n tooling packages, @flyingrobot s/bijou-tui-app, and create-bijou-tui-app.
- Replace the root interactive quick start with the preferred startApp( )-based path, and explicitly explain when someone should still drop down to run(app, { ctx }).
- Add a short "Choose Your Lane" selector directly in the root README for CLI vs TUI vs MCP vs i18n so the first click lands on the right package guide.

2.2. Missing Standard Documentation

At least two standard repository documentation surfaces are still missing for a library/project of this type:

1. **SUPPORT.md**: There is a SECURITY.md, but there is no canonical support/escalation surface for normal user questions, bug-routing, and maintainer expectations.
2. **Generated API reference coverage**: There is no first-class repo-level API reference surface (TypeDoc or equivalent) for exported functions/types across the published packages. For a multi-package TypeScript library, that is a standard expectation.

2.3. Supplementary Documentation (Docs)

The most under-documented complex area is the createFramedApp() extension model. The implementation is spread across packages/bijou-tui/src/app-frame.ts and packages/bijou-tui/src/app-frame-render.ts, while the user-facing explanation is split across package docs and strategy notes. The repo needs one dedicated guide that explains shell-owned layers, page routing, notifications/settings ownership, and pane/layout behavior in one place.

3. Final Action Plan

3.1. Recommendation Type

A. Recommend incremental updates to the existing README and documentation.

The documentation system is structurally strong. This is not a rewrite problem. It is a front-door accuracy and onboarding consistency problem.

3.2. Deliverable (Prompt Generation)

Update the root README so it reflects the full shipped package surface and the current preferred bootstrap path, correct supporting-doc drift in CONTRIBUTING.md and SECURITY.md, create SUPPORT.md, and add a dedicated framed-app extension guide plus an API reference plan/surface.

3.3. Mitigation Prompt

```
Apply an incremental documentation repair pass across Bijou's front door and operator docs.

1. Update `README.md`:
   - replace the interactive quick start with a `startApp()` example
   - expand the package table to include MCP, i18n tooling, `bijou-tui-app`, and
   ↪  `create-bijou-tui-app`
   - add a short "Choose Your Lane" section pointing users to CLI, TUI, MCP, and i18n
   ↪  entrypoints

2. Update supporting docs:
   - change `CONTRIBUTING.md` from `pnpm install` to the repo's actual npm workflow
```

```
   - refresh `SECURITY.md` so the supported-version table matches the current release reality
   ↪  instead of stopping at `4.1.x`

3. Create missing standard docs:
   - add `SUPPORT.md` with issue-routing, discussion/support expectations, and maintainer
   ↪  contact guidance
   - add an API-reference plan or generated surface (for example `docs/api/README.md` or
   ↪  TypeDoc output instructions) for the exported package APIs

4. Add one new advanced guide:
   - create a dedicated `createFramedApp()` extension guide explaining shell-owned
   ↪  notifications/settings, page routing, pane behavior, and when to use page commands vs
   ↪  shell commands

Keep the existing documentation structure intact. This is a precision update, not a rewrite.
```

417

Ready-to-Ship Assessment

docs/audit/2026-04-14_ship-readiness.md

```
report_id: "AUD-2026-04-14-SR-V01"
title: "Differential Ship-Readiness Audit: Bijou Runtime and Release Surface"
status: "Final"
audit:
  date_started: 2026-04-14
  date_completed: 2026-04-14
  type: "Differential"
  scope: "runtime, frame shell, text rendering boundary, release-readiness gauntlet, docs/operator readiness"
  compliance_frameworks: []
target:
  repository: "github.com/flyingrobots/bijou"
  branch: "release/v5.0.0"
  commit_hash: "73160b9"
  language_stack: ["TypeScript 5.9.3", "Node.js >=18", "Vitest 4.0.18", "npm workspaces"]
  environment: "Local workspace; full release-readiness run on 2026-04-14"
methodology:
  automated_tools: ["npm run release:readiness", "npm audit --json", "npm outdated --json", "npm run
  ↪  docs:inventory", "rg"]
  manual_review_hours: 5
  false_positive_rate: "Low (<10%)"
summary:
  total_findings: 14
  severity_count:
    critical: 1
    high: 4
    medium: 6
    low: 3
  remediation_status: "Pending"
related_reports:
  previous_audit: "docs/audit/2026-04-11_ship-readiness.md"
  tracking_ticket: "HT-008"
```

Release confidence is materially better than it was on 2026-04-11. On 2026-04-14, npm run release:readiness passed end to end: build, lint, typecheck, docs/design-system preflight, DOGFOOD coverage gate, workflow-shell preflight, release preflight, frame regressions, smoke canaries, smoke DOGFOOD, and the full test suite (3017 tests passed, 12 skipped). npm audit --json also returned 0 vulnerabilities. This is no longer a “red build” repo. The remaining risk is concentrated in security-sensitive text handling and teardown semantics rather than broad delivery instability.

1. Quality & Maintainability Assessment

1.1. Technical Debt Score (1-10)

Score: 4/10

Justification:

1. **Shell Monolith:** packages/bijou-tui/src/app-frame.ts is still a 2444-line shell factory with dozens of local helpers and multiple responsibilities.
2. **Split Text Trust Boundary:** The repo sanitizes some text entry points, but not all public surface-native ones, which makes behavior harder to reason about and audit.
3. **Bootstrap/Operator Drift:** The package/docs truth for first-app bootstrap and contributor setup is still inconsistent, increasing handoff cost even though the code itself is mostly green.

1.2. Readability & Consistency

Issue 1: createFramedApp() hides too much control flow in local helpers. packages/bijou-tui/src/app-frame.ts:645-1631 mixes shell theme publication, retained-layout caching, pane geometry, settings, notifications, and mouse routing in one place, which slows onboarding for any engineer entering the shell code.

Mitigation Prompt 1:

```
Improve `createFramedApp()` readability by extracting the largest closure-local helper groups
↪ into named modules: one for retained-layout caching, one for shell input routing, and one
↪ for shell-owned settings/notification behavior. Keep the public API unchanged, but make
↪ the file read like an orchestrator rather than a dump of private helper functions.
```

Issue 2: packages/bijou-tui/src/layout-node-surface.ts:20-24,55-67,87-97 uses dx / dy without documenting the coordinate-space meaning. For a new engineer, it is not obvious that these values are localization offsets from potentially negative tree coordinates into a non-negative paint surface.

Mitigation Prompt 2:

```
Rename `dx` / `dy` in `layout-node-surface.ts` to `offsetX` / `offsetY`, add short JSDoc
↪ explaining the local-to-surface coordinate-space conversion, and update the callers in
↪ `view-output.ts` so localization intent is explicit wherever a tree is translated before
↪ paint.
```

Issue 3: The onboarding docs disagree on the happy path. README.md:72-95 teaches manual context wiring, while packages/bijou-tui/GUIDE.md:23-41 and packages/bijou-node/README.md:19-35 teach startApp(), and CONTRIBUTING.md:8-12 still says pnpm install. That inconsistency hurts engineering handoffs even if the runtime is correct.

Mitigation Prompt 3:

```
Normalize the onboarding language across `README.md`, `packages/bijou-tui/GUIDE.md`,
↪ `packages/bijou-node/README.md`, and `CONTRIBUTING.md`. Make `startApp()` the default
↪ Node-host bootstrap path everywhere, and update the contributor install command to match
↪ the actual npm workspace workflow used by the repo.
```

1.3. Code Quality Violation

Violation 1: log() in packages/bijou/src/core/components/log.ts:76-120 mixes timestamp formatting, mode branching, badge rendering, and string assembly in one function.

Original Code Snippet:

```
export function log(level: LogLevel, message: string, options?: LogOptions): string {
  const ctx = resolveCtx(options?.ctx);
  const showPrefix = options?.prefix !== false;
  const showTimestamp = options?.timestamp === true;
  const ts = showTimestamp ? formatTimestamp(ctx) : '';

  if (!ctx) {
    const parts: string[] = [];
    if (ts) parts.push(`[${ts}]`);
    if (showPrefix) parts.push(`[${LABELS[level]}]`);
    parts.push(message);
    return parts.join(' ');
  }

  return renderByMode(ctx.mode, {
    pipe: () => { /* ... */ },
    accessible: () => { /* ... */ },
    interactive: () => { /* ... */ },
  }, options ?? {});
}
```

Simplified Rewrite 1:

```
function buildLogParts(level: LogLevel, message: string, ts: string, showPrefix: boolean):
↪  string[] {
  const parts: string[] = [];
  if (ts) parts.push(ts);
  if (showPrefix) parts.push(LABELS[level]);
  parts.push(message);
  return parts;
}

export function log(level: LogLevel, message: string, options?: LogOptions): string {
  const ctx = resolveCtx(options?.ctx);
  const ts = options?.timestamp ? formatTimestamp(ctx) : '';
  const showPrefix = options?.prefix !== false;

  if (!ctx) return buildLogParts(level, message, ts && `[${ts}]`, showPrefix).join(' ');

  return renderByMode(ctx.mode, {
    pipe: () => buildLogParts(level, message, ts && `[${ts}]`, showPrefix).join(' '),
    accessible: () => buildLogParts(level, message, ts, showPrefix).join('
    ↪  ').replace(LABELS[level], `${ACCESSIBLE_LABELS[level]}:`),
    interactive: () => renderInteractiveLog(ctx, level, message, ts, showPrefix),
  }, options ?? {});
}
```

Mitigation Prompt 4:

```
Refactor `packages/bijou/src/core/components/log.ts` so `log()` becomes a thin coordinator.
↪  Extract helper functions for building plain-text parts and for rendering the interactive
↪  badge/timestamp path, then keep the public `log()` API unchanged.
```

Violation 2: normalizeViewOutputInto() in packages/bijou-tui/src/view-output.ts:35-74 combines type discrimination, error messaging, size normalization, localization, and painting in one block.

Original Code Snippet:

```
export function normalizeViewOutputInto(output: ViewOutput, size: ViewportSize, scratch?:
↪  Surface): NormalizedViewOutput {
  if (isSurfaceView(output)) {
    // surface path
  }

  if (!isLayoutNodeView(output)) {
    throw new Error('Bijou runtime views must return a Surface or LayoutNode...');
  }

  const localization = localizeLayout(output);
```

```
  const width = Math.max(size.width, localization.width, 0);
  const height = Math.max(size.height, localization.height, 0);
  const surface = prepareNormalizedSurface(width, height, scratch);
  paintLayoutNodeWithOffset(surface, output, localization.dx, localization.dy);
  return { kind: 'layout', surface };
}
```

Simplified Rewrite 2:

```
function normalizeSurfaceOutput(output: Surface, size: ViewportSize, scratch?: Surface):
↪  NormalizedViewOutput {
  if (scratch == null) return { kind: 'surface', surface: output };
  const surface = prepareNormalizedSurface(Math.max(size.width, output.width, 0),
  ↪  Math.max(size.height, output.height, 0), scratch);
  surface.blit(output, 0, 0);
  return { kind: 'surface', surface };
}

function normalizeLayoutOutput(output: LayoutNode, size: ViewportSize, scratch?: Surface):
↪  NormalizedViewOutput {
  const localization = localizeLayout(output);
  const surface = prepareNormalizedSurface(Math.max(size.width, localization.width, 0),
  ↪  Math.max(size.height, localization.height, 0), scratch);
  paintLayoutNodeWithOffset(surface, output, localization.dx, localization.dy);
  return { kind: 'layout', surface };
}
```

Mitigation Prompt 5:

```
Split `normalizeViewOutputInto()` into explicit surface and layout normalization helpers.
↪  Preserve the current runtime error for invalid view outputs, but move discrimination and
↪  normalization into smaller functions so the hot path is easier to read and test.
```

Violation 3: scheduleRender() in packages/bijou-tui/src/runtime.ts:313-369 handles queueing, flush setup, frame execution, and recursive rescheduling in one nested closure.

Original Code Snippet:

```
function scheduleRender(): void {
  if (renderHandle != null || renderInFlight) return;

  const scheduledHandle = clock.setTimeout(() => {
    if (renderHandle === scheduledHandle) {
      renderHandle = null;
    }
    scheduledHandle.dispose();
    if (!renderQueued) return;
    renderInFlight = true;
    renderQueued = false;
    try {
      // build RenderState and execute pipeline
    } finally {
      renderInFlight = false;
      if (renderQueued) scheduleRender();
    }
  }, 0);

  renderHandle = scheduledHandle;
}
```

Simplified Rewrite 3:

```
function queueRender(): void {
  renderQueued = true;
  if (renderHandle != null || renderInFlight) return;
  renderHandle = clock.setTimeout(flushRender, 0);
}
```

```
function flushRender(): void {
  const handle = renderHandle;
  renderHandle = null;
  handle?.dispose();
  if (!renderQueued) return;

  renderQueued = false;
  renderInFlight = true;
  try {
    executeRenderFrame();
  } finally {
    renderInFlight = false;
    if (renderQueued) queueRender();
  }
}
```

Mitigation Prompt 6:

```
Refactor render scheduling in `packages/bijou-tui/src/runtime.ts` so queuing and flushing are
↪  separate functions with one explicit `executeRenderFrame()` helper. Keep the runtime
↪  semantics unchanged, but remove the large nested timeout closure and make the shutdown
↪  path easier to reason about.
```

2. Production Readiness & Risk Assessment

2.1. Top 3 Immediate Ship-Stopping Risks (The "Hard No")

Risk 1: Critical | packages/bijou/src/core/components/badge.ts, packages/bijou/src/core/components/surface-text.ts

Public surface-native text helpers still write raw user text into cells without passing through sanitizeTerminalText(). This is the largest remaining ship risk if any consuming app renders untrusted input in interactive or static modes.

Mitigation Prompt 7:

```
Implement a shared text-sanitization boundary for all public surface-native text helpers.
↪  Update `badge()`, `createTextSurface()`, and `createSegmentSurface()` so untrusted text
↪  is sanitized or explicitly rejected before it becomes cells, and add regression tests
↪  proving that CSI/OSC/DCS payloads cannot reach the terminal output path.
```

Risk 2: High | packages/bijou/src/core/components/hyperlink.ts:33-36

hyperlink() interpolates raw url and text directly into OSC 8 control sequences. A malicious value can break out of the hyperlink payload and inject arbitrary terminal control behavior.

Mitigation Prompt 8:

```
Harden `hyperlink()` by sanitizing or validating both `url` and `text` before building OSC 8
↪  output. Reject embedded ESC/BEL terminators, preserve safe fallback formatting for
↪  pipe/accessible modes, and add tests that attempt OSC 8 breakout payloads in both the URL
↪  and display text.
```

Risk 3: High | packages/bijou-tui/src/runtime.ts:469-491 and packages/bijou-tui/src/eventbus.ts:430-458

The runtime quits without awaiting bus.drain(). Long-lived async commands can still be pending when the bus is disposed, which makes cleanup

timing-dependent and increases the chance of dropped teardown or orphaned effects during shutdown.

Mitigation Prompt 9:

```
Update the runtime shutdown path to await `bus.drain()` before final disposal, with a bounded
↪  timeout and test coverage for long-lived commands that register cleanups asynchronously.
↪  Preserve fast exits for already-idle apps, but make command teardown deterministic when
↪  work is still in flight.
```

2.2. Security Posture

Vulnerability 1: hyperlink() OSC 8 interpolation can be broken by malicious u rl or text values (packages/bijou/src/core/components/hyperlink.ts:33-36).

Mitigation Prompt 10:

```
Treat `hyperlink()` as a security boundary. Validate the URL and display text, strip or
↪  reject terminal control bytes, and centralize the escaping/validation logic in a reusable
↪  helper so other OSC/ANSI-producing components can share it.
```

Vulnerability 2: Surface-native text builders such as badge() and createTextS urface() accept raw control characters and bypass the existing sanitizeTermin alText() path (packages/bijou/src/core/components/badge.ts:32-40,79-86; packa ges/bijou/src/core/components/surface-text.ts:72-99,105-130).

Mitigation Prompt 11:

```
Introduce a trusted/untrusted text contract for surface-native rendering. All public APIs
↪  that accept arbitrary strings should default to untrusted mode and sanitize control
↪  sequences before cell writes, while an explicit trusted/styled path can preserve ANSI
↪  where that is intentional. Add a shared test matrix across badge, surface-text, and any
↪  component that delegates to them.
```

2.3. Operational Gaps

Gap 1: release:readiness does not currently run npm audit or npm run docs:inv entory; both had to be checked manually during this audit.

Gap 2: There is no first-class trusted-text type, linter, or audit tool that forces public text-bearing APIs through the same terminal-safety boundary.

Gap 3: Runtime shutdown does not emit structured telemetry or timeout diagnostics for pending commands/effects that are still alive during quit.

3. Final Recommendations & Next Step

3.1. Final Ship Recommendation

YES, BUT…

Ship is reasonable for controlled or trusted input because the release gauntlet is green and the dependency audit is clean. Do not treat the current public text surface as safe for untrusted content until the remaining terminal-control-sequence boundary is unified.

3.2. Prioritized Action Plan

- **Action 1 (High Urgency):** Close the surface-native control-sequence injection boundary across badge(), surface-text, and hyperlink().
- **Action 2 (Medium Urgency):** Make runtime shutdown drain pending commands before disposal and instrument lingering cleanups.
- **Action 3 (Low Urgency):** Continue decomposing createFramedApp() into explicit extension seams or, at minimum, publish one operator-facing extension guide that matches the code.

418

Graveyard

docs/GRAVEYARD.md

The active graveyard now lives in docs/graveyard/.

Use that directory for rejected or indefinitely deferred work that needs to stay discoverable.

This signpost remains so older docs can still point at a stable graveyard entrypoint.

419

I-0d — Per-stage render pipeline instrumentation (buried 2026-04-09)

docs/graveyard/I-0d-render-pipeline-instrumentation.md

Original scope: add per-stage timing + allocation counters to the render pipeline (Layout, Paint, Diff, Output), plus counters for suspected hot int↔string conversions (parseHex, String.fromCharCode, String.fromCodePoint, template-literal SGR, Map.get hits on sgrCache). Gated behind an env var (BIJOU_PERF=1) so normal runs pay no cost. The plan was to use this instrumentation to identify the specific hot paths behind the broad RE-008 regression.

Why buried: task I-0e (investigate RE-008 broad slowdown) completed its investigation without needing general pipeline instrumentation. A targeted 3-scenario A/B/C bench comparison was enough to prove the hypothesis empirically:

```
paint-rgb-fixed   145 µs  (11.4 ns/cell) — setRGB, no hex parsing
paint-ascii       231 µs  (18.1 ns/cell) — set({char}), no colors
paint-set-hex-palette   624 µs  (48.9 ns/cell) — set({char, fg, bg}) with hex
```

The delta between paint-set-hex-palette and paint-rgb-fixed is **482 µs per frame of pure hex parsing overhead** — which is 77% of paint-set-hex-palette's total frame time. That was sufficient to confirm the hypothesis (per-cell hex parsing in inlineHexRGB) and point directly at the fix (pre-parse theme token colors at theme load time, see docs/method/backlog/cool-ideas/RE-019-theme-token-color-cache.md).

General pipeline instrumentation would have been more work for less signal than the targeted scenario comparison, and the investigation is already closed.

What lives on: if a future perf issue can't be isolated with targeted scenarios, pipeline instrumentation remains a valid tool. Exhume this task then. The idea isn't wrong — it was just unnecessary for this particular investigation.

Cross-references:

- docs/perf/RE-017-byte-pipeline.md — the RE-017 cycle log including the 2026-04-09 I-0e investigation result entry.
- docs/method/backlog/bad-code/RE-018-hex-color-string-as-canonical-fg-bg.md — the root-cause smell the investigation confirmed.

- docs/method/backlog/cool-ideas/RE-019-theme-token-color-cache.md — the fix direction.

420

I-3 — Custom baseline JSON format + compare tool (buried 2026-04-09)

docs/graveyard/I-3-custom-baseline-format.md

Original scope: define a new JSON schema for renderer bench reports, add a co mpare CLI that diffs two reports, and wire up baseline-saving workflow.

Why buried: the RE-017 audit scrapped the entire old scripts/renderer-bench*. ts harness and rebuilt the bench from scratch as the bench/ top-level workspace package. bench v2 (commit 4b563cf) ships with:

- Its own report schema (bench.v2 kind, bench/src/harnesses/wall-time/runner. ts).
- Its own comparison tool (bench/src/harnesses/wall-time/compare.ts).
- Its own baseline saving via bench run --out and npm run bench:baseline.

Everything this task was going to build already exists in the new package. There is no standalone "baseline format" to design anymore — the format is the RunReport type in runner.ts and the compare is compareReports in compare. ts.

What lives on: if we later decide we want a cross-cycle historical baseline archive (e.g., one commit-tagged JSON per merge to main), that is a separate future concern handled by bench/baselines/ conventions, not by reviving this task.

Cross-references:

- bench/README.md — current bench design and usage.
- docs/perf/RE-017-byte-pipeline.md — the RE-017 cycle where this was deleted.
- Commit 4b563cf — bench v2 initial build.

421

RE-017 — Byte-Pipeline Recovery

docs/perf/RE-017-byte-pipeline.md

> **Status:** In progress (audit phase). **Goal:** Recover the render pipeline to pre-RE-008 performance or better by eliminating all per-frame allocation and string work on the render path, moving to a pure byte-level output pipeline. **Cycle ID note:** the bad-code backlog already has an RE-017-frame-does-not-fill-surface-background.md file using the same cycle number. These are separate concerns and we may need to renumber one of them before landing this cycle.

Table of Contents

1. Background
2. Measurement Methodology
3. Baseline Audit (pre-RE017 HEAD)
4. Optimization Steps
5. Final Results
6. Learnings

Background

After RE-008 merged (byte-packed surface representation, PR #62, merge commit 0df3e97), an audit of the existing renderer benchmark suite revealed a broad performance regression across every scenario:

Scenario	Pre-RE-008 → Post-RE-008
dogfood.render.medium	1.60 → 2.32 ms/frame (-31%)
dogfood.diff.medium	1.86 → 2.71 ms/frame (-31%)
dogfood.render.large	0.53 → 1.09 ms/frame (-51%)
dogfood.diff.large	0.78 → 1.55 ms/frame (-50%)
dogfood.docs.render.medium	2.61 → 2.99 ms/frame (-12%)
surface.paint.medium	0.22 → 0.41 ms/frame (-45%)
layout.normalize.medium	0.16 → 0.34 ms/frame (-53%)
styled.diff.medium	0.42 → 0.70 ms/frame (-40%)
frame.compose.medium	0.95 → 1.21 ms/frame (-22%)
runtime.noop.medium	neutral

The most dramatic finding: **styled.diff.medium transient heap allocation grew from 1.4 MiB to 62 MiB per scenario run (44× increase)**. This is the signature of the current renderDiffPacked path: sgrCache Map growth, String.fromCharCode key allocation per cell, batchText += ... string concatenation, and side-table grapheme string churn.

The original RE-017 plan was scoped to fix the gradient stress worst-case (examples/perf-gradient mode 1, which had regressed from 49 → 25 FPS). The audit revealed that the regression is much broader: every non-noop scenario is slower, and the heap allocation on the styled diff path is catastrophic. The byte-pipeline rewrite is now targeting the full regression, not just the gradient case.

An earlier claim of "-5.5% DOGFOOD" from the RE-008 handoff does not reconcile with these bench numbers. Its origin is unknown. It may have measured the live npm run dogfood experience rather than the headless bench, or it may have compared against a different reference. It is not being used as a success criterion for RE-017.

Measurement Methodology

Harness

- **Entry point:** scripts/renderer-bench.ts (invoked via npm run bench:renderer with node --expose-gc --import tsx).
- **Library:** scripts/renderer-bench-lib.ts — scenario definitions, stats computation, environment detection, comparison.
- **Baseline storage:** benchmarks/renderer-baseline.json.
- **Audit runs:** benchmarks/re017-audit/run-NN.json (expanded 10×10 stats runs used in this cycle).
- **Machine fingerprint:** platform, arch, kernel release, Node version, CPU model, CPU count, total memory, hostname. Comparisons between runs on different fingerprints are treated as informational, not as a regression gate.

Scenarios

The DEFAULT_RENDERER_BENCH_SCENARIOS array in renderer-bench-lib.ts defines 10 scenarios across 7 kinds. Each scenario has fixed columns, rows, frames, and warmupFrames. Warmup frames run before timing starts so JIT optimization has stabilized.

Scenario	Kind	Size	Frames	What it measures
dogfood.render.medium	render	220×58	240	DOGFOOD landing: app.view(model) + normalizeViewOutput per frame. No diff.

Scenario	Kind	Size	Frames	What it measures
dogfood.diff.medium	diff	220×58	180	Same as above + renderDiff(current, target, sink, style) to a counting sink.
dogfood.render.large	render	271×71	180	Larger terminal, render only.
dogfood.diff.large	diff	271×71	120	Larger terminal, render + diff.
dogfood.docs.render.medium	render	220×58	180	DOGFOOD docs explorer page (sidebar + doc body).
surface.paint.medium	surface	220×58	240	Pure surface paint: target.clear(); target.fill(...); target.blit(source, ...) three times per frame + one setRow. No app/layout/diff.
layout.normalize.medium	normalize	220×58	240	normalizeViewOutputInto(syntheticLayout, size, scratch) per frame — the layout normalization stage in isolation.
styled.diff.medium	styled-diff	220×58	180	Synthetic styled diff: two surfaces built by buildStyledDiffPair (every 3rd/5th/7th cell styled with rotating palette + occasional modifiers), renderDiff called in a tight loop. Exercises the packed differ's SGR cache + string concat.
frame.compose.medium	frame	220×58	180	renderFrameNode(layoutTree, bodyRect, renderCtx) — the frame shell composition stage (grid/split/pane tree).
runtime.noop.medium	runtime	220×58	180	Full runtime loop (run(app, { ctx })) with a trivial view function — measures the cost of the event loop, pulse ticks, and pipeline overhead without real rendering work.

Metrics (current)

Per sample, as defined in RendererBenchSample at scripts/renderer-bench-lib.ts:28:

Field	Meaning	How it's measured
elapsedMs	Total wall time for frames iterations (excluding warmup)	performance.now() deltas
avgFrameMs	elapsedMs / frames	Derived
approxFps	1000 / avgFrameMs	Derived
writes	Count of io.write() calls observed by the sink (diff scenarios only)	Sink counter
bytesWritten	Sum of .length of all strings passed to write()	Sink counter
transientHeapDelta	heapUsed after run minus before, **without** forced GC before taking the "after" reading. Measures peak allocation during the run.	process.memoryUsage() (requires --expose-gc)
retainedHeapDelta	heapUsed after run minus before, **with** forced GC before both readings. Measures long-lived allocations.	process.memoryUsage() after global.gc() (requires --expose-gc)

Per scenario (aggregated across samples), only **median** is currently computed. This is a known gap — see RE-017 task I-2 for the expansion to mean / stddev / min / max / p50 / p90 / p99.

Current limitations

1. **No percentile statistics.** Only median across samples. A bimodal distribution (e.g. one slow sample every 10) would be invisible.
2. **No per-stage breakdown.** The render kind covers layout + paint as a unit; the diff kind adds the diff stage on top. We can see "total" cost but not "how much in layout vs paint vs diff".
3. **No conversion-hot-path counters.** The bench cannot answer "how many parseHex calls per frame?" or "how many String.fromCharCode key builds per frame?" — both suspected sources of the RE-008 slowdown.
4. **No GC pause time.** Only heap deltas. We can see how much allocation happened, but not whether it triggered a costly GC.
5. **First-run variance.** JIT warmup may be incomplete even after the scenario's warmupFrames — the 10-run expanded audit is partly to characterize this.
6. **styled.diff.medium uses the legacy surface.set({char,fg,bg}) API, not setRGB.** The underlying surface is a PackedSurface either way, but the paint code path going into the diff is the hex-string-parsing path, not the byte-passing fast path. A setRGB variant should be added.
7. **No gradient / truecolor stress scenario.** The examples/perf-gradient worst case (every cell a unique RGB pair, 9168+ unique styles per frame) is not represented in the headless bench. It is the scenario that originally surfaced the sgrCache worst-case regression (49 → 25 FPS).

Baseline Audit (pre-RE017 HEAD)

Run parameters

- **HEAD commit:** 941f62c (RE-008 merge + follow-up fixes)
- **Machine:** Apple M1 Pro, 10 cores, 16 GiB, macOS 25.3, Node v25.8.1
- **Samples per scenario per process:** 10
- **Independent processes:** 10
- **Total samples per scenario:** 100
- **Run files:** benchmarks/re017-audit/run-01.json through run-10.json

Aggregate statistics

Comparison vs pre-RE-008 baseline (87747f7)

Observations

Optimization Steps

Each step is implemented in isolation, measured before and after against the full scenario matrix, and kept only if it shows a meaningful improvement on at least one scenario with no regressions elsewhere.

Step 0: Investigation (I-0e)

Step II-1: Render-dirty bitmap

Step II-2: WritePort.writeBytes

Step II-3: Pooled output byte buffer

Step II-4: Differ emits bytes directly

Step II-5: ASCII fast path

Step II-6: Pre-encoded side-table bytes

Step II-7: Decimal lookup tables

Final Results

Learnings

This section and everything below it is an append-only running log. **Do not rewrite earlier entries.** Each entry is dated (and may be turn-marked). To correct or supersede a prior entry, add a new entry that references the old one. The sections above this point form the initial scaffold; findings that would have filled their placeholders are recorded as new log entries below instead, cross-referenced by section name.

2026-04-09 — Doc scaffolded, append-only policy adopted

Initial scaffold written with placeholders for Baseline Audit, Optimization Steps, and Final Results. Before any of those placeholders were filled, adopted an append-only policy: all future content — including audit findings, optimization results, and investigation notes — is appended to this log rather than edited into the scaffold sections. The scaffold stays as a table of contents; the truth lives in the log.

2026-04-09 — Audit kickoff, 10×10 bench run launched

Completed task I-0 (bench infrastructure audit). Key findings:

1. The existing baseline at benchmarks/renderer-baseline.json was captured at commit 87747f7 — **before** the RE-008 merge (0df3e97). It represents the old cell-object path, not the byte-packed path we're optimizing. It is effectively a historical reference, not the RE-017 starting point.
2. A single run of the existing bench at HEAD (941f62c) against the stale baseline showed 12-53% regressions across every non-noop scenario, and a 44× transient heap allocation jump (1.4 MiB → 62 MiB) in styled.diff.mediu m. These numbers are from one run and have not yet been confirmed for stability.
3. To confirm stability, launched a 10-process × 10-samples-each expanded stats run (100 samples per scenario total), saving each process output to b enchmarks/re017-audit/run-NN.json. Task I-0b tracks this.
4. Created new audit tasks: I-0c (methodology docs), I-0d (pipeline instrumentation), I-0e (investigate broad slowdown).
5. Hypothesis to test in I-0e: per-cell hex string parsing in the set() path is the bulk of the regression. Trace: packages/bijou/src/ports/surface.ts:5 32 → encodeCellIntoBuf (line 300) → inlineHexRGB (line 288) + encodeChar + encodeModifiers. Pre-RE-008, set() was an object assignment. Post-RE-008, every set() call does 6 charCodeAts, 6 hexD lookups, modifier array iteration, and two byte writes per color. For a paint-heavy scenario that calls set() per cell, this adds ~20-30 operations per cell on a path that used to be a single object assignment. No compensating savings on surface.p aint.medium because it never enters the diff path where the byte-packed representation pays off.

If the hypothesis holds, a cheap early win is possible: cache theme-token hex parses so repeated calls to set({fg: theme.primary, ...}) reuse pre-parsed

bytes instead of re-parsing on every call. This would be independent of the byte-pipeline rewrite.

2026-04-09 — Expanded 10×10 stats results (100 samples per scenario)

The 10-process × 10-samples-each run completed in ~5 minutes. All 10 JSON reports saved under benchmarks/re017-audit/run-NN.json. Aggregator at scripts/aggregate-audit-runs.ts. Raw aggregate table at benchmarks/re017-audit/aggregate.md.

Run parameters

- HEAD commit 941f62c, M1 Pro / macOS 25.3 / Node v25.8.1
- 10 independent processes, 10 samples per scenario per process
- 100 samples per scenario total
- Baseline comparison against 87747f7 (pre-RE-008)

Frame time (ms) — full stats across 100 samples

Scenario	Mean	StdDev	Min	P50	P90	P99	Max	CoV
dogfood.render.medium	2.40	0.608	2.29	2.33	2.35	3.79	8.25	25.3%
dogfood.diff.medium	2.71	0.043	2.67	2.71	2.75	2.79	3.06	1.6%
dogfood.render.large	1.10	0.016	1.07	1.10	1.11	1.15	1.20	1.5%
dogfood.diff.large	1.55	0.021	1.52	1.55	1.58	1.62	1.63	1.3%
dogfood.docs.render.medium	3.07	0.133	2.89	3.07	3.18	3.70	3.89	4.3%
surface.paint.medium	0.407	0.010	0.395	0.406	0.411	0.426	0.501	2.6%
layout.normalize.medium	0.345	0.0074	0.335	0.343	0.348	0.382	0.398	2.2%
styled.diff.medium	0.730	0.116	0.694	0.708	0.808	0.841	1.83	15.8%
frame.compose.medium	1.24	0.039	1.19	1.24	1.28	1.33	1.49	3.2%
runtime.noop.medium	0.0015	0.0003	0.0012	0.0014	0.0020	0.0022	0.0023	20.0%

Transient heap per scenario run (mean across 100 samples)

Scenario	Mean	StdDev	Min	P50	P90	Max
dogfood.render.medium	33.88 MiB	16.11 MiB	1.22 MiB	45.38 MiB	45.59 MiB	45.67 MiB

Scenario	Mean	StdDev	Min	P50	P90	Max
dogfood.diff.medium	49.14 MiB	17.81 MiB	14.22 MiB	61.73 MiB	61.88 MiB	62.14 MiB
dogfood.render.large	25.17 MiB	4.08 MiB	10.34 MiB	26.25 MiB	26.35 MiB	26.43 MiB
dogfood.diff.large	28.86 MiB	3.46 MiB	13.82 MiB	29.66 MiB	29.73 MiB	29.79 MiB
dogfood.docs.render.medium	53.26 MiB	26.83 MiB	10.73 MiB	58.11 MiB	92.65 MiB	98.49 MiB
surface.paint.medium	0.10 MiB	0.01 MiB	0.08 MiB	0.09 MiB	0.10 MiB	0.16 MiB
layout.normalize.medium	0.46 MiB	0.00 MiB	0.45 MiB	0.45 MiB	0.46 MiB	0.46 MiB
styled.diff.medium	51.79 MiB	14.13 MiB	30.29 MiB	60.99 MiB	61.00 MiB	61.03 MiB
frame.compose.medium	61.10 MiB	18.63 MiB	20.01 MiB	52.02 MiB	92.14 MiB	107.03 MiB
runtime.noop.medium	0.74 MiB	0.08 MiB	0.48 MiB	0.73 MiB	0.74 MiB	1.19 MiB

Regression vs pre-RE-008 baseline (median comparison — most fair)

Scenario	Baseline median	Post-RE-008 P50	Δ %
dogfood.render.medium	1.60 ms	2.33 ms	**+45.6%**
dogfood.diff.medium	1.86 ms	2.71 ms	**+45.7%**
dogfood.render.large	0.534 ms	1.10 ms	**+106%**
dogfood.diff.large	0.781 ms	1.55 ms	**+98%**
dogfood.docs.render.medium	2.61 ms	3.07 ms	+17.5%
surface.paint.medium	0.224 ms	0.406 ms	**+81%**
layout.normalize.medium	0.161 ms	0.343 ms	**+113%**
styled.diff.medium	0.421 ms	0.708 ms	**+68%**
frame.compose.medium	0.951 ms	1.24 ms	+30%
runtime.noop.medium	0.0019 ms	0.0014 ms	-17% (noise; control)

Heap regression vs baseline

Scenario	Baseline median	Post-RE-008 P50	Multiple
styled.diff.medium	1.34 MiB	60.99 MiB	**~46×**
dogfood.render.large	10.29 MiB	26.25 MiB	2.6×
surface.paint.medium	64 KiB	94 KiB	1.5×
all others	-	-	~1× (noisy, no clear trend)

Key observations

1. **The regression is reproducible and large.** Run-to-run variance is tight (CoV < 5% for most scenarios). The 10 independent processes agree with each other. These are trustworthy numbers.
2. **The regression is bigger than the single-run showed.** Median comparison now gives 30%-113% regression, not 12-53%. Some scenarios literally doubled in frame time (layout.normalize.medium +113%, dogfood.render.large +106%, dogfood.diff.large +98%).

3. **Smaller scenarios regressed harder.** Scenarios with smaller baselines (layout.normalize 0.16ms, surface.paint 0.22ms, dogfood.render.large 0.53ms) regressed the most in percentage terms. This is consistent with per-cell overhead dominating when there's less fixed work to amortize against.
4. **styled.diff.medium heap allocation is catastrophic and confirmed at ~46× baseline.** Mean 51.79 MiB, P50 60.99 MiB, very stable (min 30 MiB, max 61 MiB). The differ's sgrCache + string concat + side table churn is the culprit — exactly what RE-017 is rewriting.
5. **Two scenarios are bimodal** (dogfood.render.medium CoV 25%, styled.diff.medium CoV 16%). Long right tails with max values 3-3.5× the P50. This is the GC pause signature: most samples are fast, occasional samples catch a generational GC triggered by the high allocation rate. Visible GC jank.
6. **runtime.noop.medium is unaffected.** The control scenario (full runtime loop with trivial view, no real rendering) matches baseline. This confirms the regression is specifically in the render/paint/diff/layout path, not in runtime/event loop overhead.
7. **The bimodal tail on dogfood.render.medium is the concerning one for end users** — occasional 8ms frames on a 2ms mean means stutters during interaction, even though averages look OK.

Hypothesis status

The per-cell hex parsing hypothesis is consistent with every pattern observed: - Scenarios that set every cell (layout.normalize, surface.paint, dogfood.render) regress hardest. - Smaller scenarios regress harder (per-cell overhead dominates). - Heap allocation on styled-diff is the sgrCache + string concat side of the same story (diff path, not paint path). - The control scenario is unaffected.

But it's still a hypothesis. Confirming requires instrumentation (task I-0d) to count parseHex / encodeChar / encodeModifiers calls per frame and measure time spent in each.

Open questions

- Why does layout.normalize.medium regress so hard (+113%) when it uses pre-built pattern surfaces? The setup cost of building those surfaces is outside the timed region, so hex parsing during setup shouldn't show. Does the blit operation in the packed path do extra work? Needs investigation. Possible explanation: blit copying from a legacy cell-array surface to a packed surface (or vice versa) forces per-cell re-encoding.
- Why does frame.compose.medium regress only 30% when it's structurally similar to the dogfood.render paths? Is the frame shell short-circuiting some rendering?
- Why does dogfood.docs.render.medium (+17%) regress much less than dogfood.render.medium (+45%) despite both using the docs app? What's different about the keyed explorer state?

These questions go into task I-0e (investigate).

Next actions

1. Task I-0b: COMPLETE.

2. Task I-0e (investigate RE-008 broad slowdown): unblocked, can start now. Priority: identify the specific hot paths causing the regression using bisection across the RE-008 commits if needed.
3. Task I-0d (instrument pipeline): can start in parallel with I-0e; the instrumentation will accelerate the investigation.
4. Task I-0c (document methodology): largely already done as part of the scaffold + this log entry; needs a final pass to confirm nothing is missing.

Deferred decisions

- Whether to refresh benchmarks/renderer-baseline.json to reflect the post-RE-008 state as the new RE-017 starting point (task I-9). Leaning yes, but keeping the 87747f7 baseline as a historical reference for the ultimate "did we recover?" question at end of cycle.
- Whether the gradient stress scenario (task I-5) is still a priority given the existing bench already exposes the regression. Still yes — the gradient case is the worst-case stress and we want it in the CI gate.

2026-04-09 — Scrapped the existing bench, built bench v2 from scratch

After the expanded 10×10 audit revealed that the PerformanceObserver('gc')-based heap measurement fix produced garbage data (zero GC events reported across 1,000 samples even for scenarios with negative heap deltas, which can only happen if GC fires), we made the call to throw out scripts/renderer-bench*.ts + benchmarks/renderer-baseline.json entirely and build a new bench from scratch around the principle "measure only what you can trust."

What was deleted:

- scripts/renderer-bench.ts
- scripts/renderer-bench-lib.ts
- scripts/renderer-bench-compare.ts
- scripts/renderer-bench.test.ts
- scripts/aggregate-audit-runs.ts
- benchmarks/ (the whole directory, including the 10 audit runs)
- npm run bench:renderer* scripts from the root package.json

What was built:

- bench/ — new top-level workspace package @flyingrobots/bijou-bench
- Harness-agnostic Scenario interface in bench/src/scenarios/types.ts
- 4 initial scenarios:
  - paint-ascii — baseline "no colors, no hex parsing"
  - paint-set-hex-palette — rotating theme palette, exercises inlineHexRGB
  - paint-gradient-rgb — per-cell unique RGB via setRGB
  - diff-gradient — full-screen gradient diff (sgrCache stress)
- bench/src/harnesses/wall-time/ — process-isolated wall-time harness
  - child.ts runs ONE sample of ONE scenario in a fresh V8 isolate and exits
  - runner.ts spawns N children per scenario, aggregates stats

    - compare.ts diffs two report JSONs with a regression gate
- bench/src/stats.ts — percentile/mean/stddev/CoV helpers (linear-interpolated)
- bench/src/cli.ts — bench run | compare | list
- bench/README.md — design principles and usage
- bench/baselines/HEAD-941f62c-pre-RE017.json — first trustworthy baseline

New npm scripts in root package.json:

- npm run bench → node --import tsx bench/src/cli.ts run
- npm run bench:compare
- npm run bench:baseline

Design principles documented in bench/README.md:

1. Process isolation per sample. Fresh V8 isolate every time. No cross-sample GC state. No accumulated JIT state. The scenario's own warmup loop stabilizes JIT within each child.
2. Wall time only in the main bench. No heap deltas, no inline GC observers, no anything that can be wrong. process.hrtime.bigint() start + end, divide by frame count, done.
3. Scenarios are pure: setup() + frame() + metadata. No coupling to measurement. The same module can drive the wall-time harness, a future memory harness, and a future interactive sandbox — three harnesses sharing one scenario registry.
4. Report stats, not means. mean / stddev / CoV / min / max / p50 / p90 / p99. CoV < 5% is the "trust this" signal.

First trustworthy baseline at HEAD (941f62c, Apple M1 Pro, Node v25.8.1):

Scenario	P50	P90	P99	Min	Max	CoV
paint-ascii	234.27 µs	236.69 µs	239.32 µs	229.01 µs	240.24 µs	**1.1%**
paint-set-hex-palette	633.92 µs	644.29 µs	729.63 µs	624.38 µs	761.45 µs	**3.8%**
paint-gradient-rgb	946.29 µs	951.75 µs	954.92 µs	878.78 µs	955.14 µs	**2.6%**
diff-gradient	2.13 ms	2.16 ms	2.19 ms	2.04 ms	2.20 ms	**2.0%**

30 samples per scenario. **All CoVs under 4%.** These are trustworthy numbers for the first time in this cycle. The bench is reproducible and the new sample runs take ~45 seconds total, which makes iterative optimization measurement practical.

Per-cell cost breakdown (for context, rough):

- paint-ascii: 234 µs / 12,760 cells ≈ **18 ns/cell** (baseline)
- paint-set-hex-palette: 634 µs / 12,760 cells ≈ **50 ns/cell** (2.7× ASCII cost — the delta is hex parsing + modifier encoding per set() call)
- paint-gradient-rgb: 946 µs / 12,760 cells ≈ **74 ns/cell** (4.0× ASCII cost — BUT this scenario's cost is dominated by 6 Math.cos calls per cell for animation, not by setRGB itself. Scenario design note below.)

Scenario design concern — paint-gradient-rgb is animation-bound:

The scenario spends most of its per-cell time on Math.cos × 6 for the animation phase, not on setRGB. This means it does not cleanly isolate the se tRGB path cost. To fix: add a variant that precomputes a large RGB table in s etup() and indexes into it in frame() with cheap arithmetic. That would give us a pure "setRGB vs set" comparison. Captured as TODO — not blocking the RE-017 investigation since the scenario still exposes the sgrCache stress case in diff-gradient.

Backlog entries captured this session (from observations during the audit):

Bad-code:

- docs/method/backlog/bad-code/RE-018-hex-color-string-as-canonical-fg-bg.md — the hex-string-as-canonical-fg/bg smell. The broad RE-008 regression's likely root cause.
- docs/method/backlog/bad-code/RE-019-dirtyWords-serves-lazy-cache-not-render -dirty.md — the dirtyWords bitmap in surface.ts serves the lazy Cell[] cache, not render tracking. Blocks the Task II-1 repurposing until the lazy cache is retired.
- docs/method/backlog/bad-code/DAG-layer-skipping-edges-occluded.md — the DAG renderer occludes layer-skipping edges. Correctness bug (git-warp consumer). Not blocking RE-017 but important.
- docs/method/backlog/bad-code/DX-008-performance-observer-gc-unreliable.md — PerformanceObserver('gc') didn't report events it should have. Do not use it as a measurement filter in bijou benches until someone reproduces it in isolation.

Cool ideas:

- docs/method/backlog/cool-ideas/RE-019-theme-token-color-cache.md — pre-parse hex at theme load time; internal optimization only, no API break. Cheap early win candidate.
- docs/method/backlog/cool-ideas/RE-020-typed-color-representation.md — Color Ref branded type. Follow-on to the theme cache, makes the fast path first-class in the type system.
- docs/method/backlog/cool-ideas/DX-016-scenario-provenance-tags.md — tag scenarios for intelligent filtering. Enables targeted CI gates and coverage analysis.
- docs/method/backlog/cool-ideas/DX-017-agent-first-bench-output-format.md — JSONL flat-record output format for agent consumption. An agent-first observation from this session: I work faster when tools emit grep/jq-friendly data.

Agent-first lessons from this audit

(This section is the agent advocating for what the agent needs.)

1. **Measurement integrity is the first thing to validate, not the last.** I spent real time building on top of a broken measurement before questioning the tool. When numbers look weird (negative heap deltas, zero GC events), that's a tool problem until proven otherwise.

2. **Claims I make about code behavior need to be backed by a citation or a minimal repro.** I claimed the DAG renderer hid edges via transitive reduction. It didn't. I claimed a 5.5% DOGFOOD regression existed. It might not. Both cost trust.
3. **Append-only discipline on running logs is a real requirement, not an aesthetic preference.** I had to be told this. The memory is saved.
4. **The backlog lanes (bad-code/, cool-ideas/) are my tools too, not just the user's.** I am a co-author of the project's self-knowledge. When I notice smells or have ideas I should log them immediately, not wait to be asked. This session flipped that for me.

Next actions

1. Task I-0b (expanded stats run): COMPLETED with the new bench.
2. Task I-0c (document methodology): COMPLETED as part of this log entry + ben ch/README.md.
3. Task I-0e (investigate RE-008 broad slowdown): unblocked. The hypothesis to test is RE-018 — per-cell hex parsing in surface.set(). Direct way to measure: bisect across the RE-008 commits with bench run --scenario=paint-s et-hex-palette.
4. Task I-0d (instrument pipeline stages): still valuable but can be deferred until after I-0e identifies the specific hot path.
5. Adding more scenarios: diff-static, diff-sparse, paint-blit, layout-normali ze, and fixing paint-gradient-rgb to separate animation cost from setRGB cost. Can grow organically during Part II as we need coverage.
6. Commit everything as one focused "RE-017 audit + bench v2" commit.

2026-04-09 — Investigation result: RE-018 hex-parse hypothesis CONFIRMED

Approach: source-diff the pre-RE-008 (87747f7) surface.ts against current HEAD, then add a minimal A/B/C bench scenario to quantify the cost of each path empirically.

Code review — pre vs post RE-008 set() path

Pre-RE-008 surface.ts:277 (git show 87747f7:packages/bijou/src/ports/surface. ts):

```
set(x, y, cell, mask = FULL_MASK) {
  if (x < 0 || x >= w || y < 0 || y >= h) return;
  const idx = y * w + x;
  applyMaskInPlace(cells[idx]!, cell, mask);
},
```

Pre-RE-008 applyMaskInPlace (line 231): five property assignments. target.fg = source.fg was a string reference copy. No parsing. No encoding. Total per-call work: bounds check + array index + ~5 property assignments. Cheap.

Post-RE-008 set() (current surface.ts:523-540) routes through encodeCellIntoB uf (line 300), which does on every call:

- encodeChar(cell.char, sideTable) — length check + charCodeAt or sideTable lookup

- inlineHexRGB(cell.fg, buf, off+2) — 6× charCodeAt + 6× hexD (lookup table) + 3 byte writes
- inlineHexRGB(cell.bg, buf, off+5) — same, 6 more charCodeAt
- encodeModifiers(cell.modifiers) — array iteration + 2 record lookups per modifier
- 10 total byte writes into the packed buffer
- markDirty(idx) — bit flip

Per-call cost blowup: ~5 operations → ~40-60 operations. 8-12× more work per set() call, depending on whether fg/bg/modifiers are present.

A/B/C bench — three scenarios on the same painting

Added paint-rgb-fixed as the "no hex parse, no math" baseline. Same 12,760-cell paint, but via setRGB(x, y, BLOCK, 0x9b, 0xa9, 0xff, 0x11, 0x13, 0x20) — pre-parsed bytes, no string work at all.

30 samples each, CoV < 5%:

Scenario	P50	ns/cell	Path
paint-rgb-fixed	143 µs	**11.2 ns**	setRGB → packCell direct (bytes in, bytes out)
paint-ascii	231 µs	**18.1 ns**	set({char}) → encodeCellIntoBuf (no colors, but all branches still evaluated)
paint-set-hex-palette	625 µs	**49.0 ns**	set({char, fg, bg}) → encodeCellIntoBuf → 2× inlineHexRGB

The deltas tell the story:

- **paint-set-hex-palette − paint-rgb-fixed = 482 µs per frame of pure hex-parsing + modifier-encoding overhead.** That is 38 ns × 12,760 cells of work that is *entirely* spent reparsing the same ~5 theme colors over and over. This is **77% of paint-set-hex-palette's total frame time**.
- **paint-ascii − paint-rgb-fixed = 88 µs / 7 ns/cell.** This is the raw cost of encodeCellIntoBuf's function overhead + branch-check path even when there's no hex to parse. Not huge, but real.
- **setRGB is unambiguously the fast path.** It writes MORE data per call (10 bytes vs ~2-3 bytes of char code) but is ~1.6× faster than set({char}) because it skips encodeCellIntoBuf and hits packCell directly.

Conclusion

The RE-018 hypothesis is confirmed. The broad RE-008 regression (30-113% across every rendering scenario in the earlier audit) is driven by per-cell hex parsing in the legacy surface.set path, which is what ~every component currently uses. setRGB exists and works, but adoption is partial — most components still pass hex strings.

Two independent fixes are implied, and they compose:

1. **Cheap, no-API-break win:** the cool idea RE-019-theme-token-color-cache.md. Pre-parse theme token colors at theme load time into byte arrays. surface.s

et (or a new internal variant) detects already-parsed byte values and skips inlineHexRGB. Components that pull fg/bg from theme tokens (which is the common case) get the fast path for free. Expected impact: paint-set-hex-palette drops from 625 µs toward 143 µs — potentially a **4×** speedup on that scenario and roughly proportional speedups on dogfood.render.*.

2. **Deeper fix:** the cool idea RE-020-typed-color-representation.md. A branded ColorRef type that lets the entire hot path work in bytes. Hex strings stay supported at the user edge. Components that build Cells directly can resolveColor(hex) at setup time and reuse the result.

The theme cache (RE-019 cool idea) is the right first landing because it's:

- Zero API break — purely internal optimization.
- Directly measurable — paint-set-hex-palette delta is the success criterion.
- Compounds with the byte-pipeline differ work in Part II — if we land both, the entire render pipeline runs in bytes end to end.

Also worth noting for the byte-pipeline design

- **setRGB not being ~identical in cost to set({char}) is a design assertion we can make now.** setRGB is the fast path; components should use it wherever they have the RGB values already. Converting more components to setRGB would be an early partial win even without any new caching.
- **paint-ascii being 1.6× slower than paint-rgb-fixed suggests there is room to optimize encodeCellIntoBuf's no-colors path.** Currently the branches are evaluated but never taken. A fast-path version that detects "no fg, no bg, no modifiers" at the top and jumps to a minimal char-only writer would recover most of that 88 µs delta. Not blocking, but an obvious cleanup for the same cycle.

Next actions

1. Task I-0e: COMPLETED. Hypothesis confirmed empirically.
2. New task: prototype the theme token color cache (RE-019 cool idea). Measure the paint-set-hex-palette delta. If it matches expectations (3-4× speedup), land it as a standalone improvement before the Part II byte-pipeline work.
3. Add paint-rgb-fixed as a permanent bench scenario (done in this session, committed next).
4. Consider adding paint-blit and diff-static scenarios for broader coverage of the Part II optimization work.

2026-04-09 — Theme token color cache prototype: 1.9× speedup confirmed

Prototyped RE-019 cool idea (pre-parse theme token colors into bytes). Scope of the change:

1. Cell interface gained optional fgRGB?: readonly [number, number, number] and bgRGB?: readonly [number, number, number] fields (packages/bijou/src/ports/surface.ts:25-38). Both are optional; hex strings still work as before. No breaking change.
2. encodeCellIntoBuf (surface.ts:300-348) now checks fgRGB / bgRGB first and uses them directly when present, falling back to inlineHexRGB(cell.fg, ...)

otherwise. Theme-driven paints that pass pre-parsed RGB skip the hex parse entirely.

3. CellTextStyle in packages/bijou/src/core/components/surface-text.ts extended to Pick<Cell, 'fg' | 'bg' | 'fgRGB' | 'bgRGB' | 'modifiers'>.
4. tokenToCellStyle (the central helper used by text components) now passes to ken.fgRGB / token.bgRGB through when present. Since theme resolution already populates these fields (see packages/bijou/src/core/theme/resolve.ts:12), any component that uses tokenToCellStyle gets the fast path automatically.
5. badge.ts updated to pass baseToken.fgRGB explicitly as a representative component migration.

Test status: all 2,807 tests pass. The change is fully backward compatible.

Bench measurement — added a paint-set-preparsed-palette scenario that mirrors paint-set-hex-palette but pre-parses the palette once in setup() and passes fgRGB / bgRGB on each surface.set() call. 30 samples each, same machine, commit 0aded48:

Scenario	P50	ns/cell	CoV	Notes
paint-rgb-fixed	145 µs	11.4	3.5%	setRGB floor (no parsing)
paint-set-preparsed-palette	**328 µs**	**25.7**	**1.0%**	set() with pre-parsed fgRGB/bgRGB — **NEW**
paint-set-hex-palette	624 µs	48.9	3.1%	set() with hex parsed every call

Result: paint-set-hex-palette → paint-set-preparsed-palette = 624 µs → 328 µs, a 1.9× speedup. That's a **47% reduction in frame time** on the theme-driven paint path with a ~30-line change.

The remaining gap to the setRGB floor (328 → 145 µs) is other overhead in encodeCellIntoBuf that has nothing to do with hex parsing:

- encodeChar — char string → code point encoding
- encodeModifiers — array iteration + record lookups
- Undefined-check branches for cell fields
- Object property accesses on the Cell argument
- Writing zero bytes to unused fg/bg positions when they're absent

Those are a separate optimization opportunity (an encodeCellIntoBufFast sibling that takes pre-computed byte/flag values directly), not part of the theme cache work.

What this means for real-world performance

Theme tokens already had fgRGB/bgRGB populated via populateThemeRGB (the infrastructure was half-built from an earlier cycle and never wired through). Every component that uses tokenToCellStyle now gets the ~1.9× speedup for free because tokenToCellStyle passes the pre-parsed RGB through. That includes everything built on createTextSurface and surface-text.ts's text rendering helpers — the hottest text paint path in the system.

Components that still call surface.set({ fg: token.hex }) directly (without going through tokenToCellStyle) don't benefit yet. They need a similar small

edit to pass fgRGB: token.fgRGB. The grep turned up ~8 files with direct .hex calls. Migrating them is mechanical and safe.

Remaining components to migrate (for full benefit):

- packages/bijou/src/core/components/box-v3.ts (3 call sites — fillStyle.fg, border overrides)
- packages/bijou-tui/src/app-frame-render.ts (1 call site — active header tab override, not a hot path)
- packages/bijou-tui/src/overlay.ts
- packages/bijou-tui/src/notification.ts
- packages/bijou-tui/src/css/text-style.ts
- packages/bijou-tui/src/transition-shaders.ts
- packages/bijou/src/core/theme/graph.ts

These can be done as a follow-up batch. None are blocking further RE-017 work.

Next actions

1. Task #37 (theme token color cache prototype): COMPLETED. The prototype validates the hypothesis and the Cell-level machinery is in place. Migration of remaining components can happen incrementally.
2. Add a component-level bench scenario that renders a representative slice of DOGFOOD (header + sidebar + content) so we can measure the end-to-end impact of the migration on a realistic workload, not just the synthetic pai nt-set-hex-palette.
3. Continue with Part II of RE-017 (the differ byte pipeline). The theme cache win compounds with the differ work — together they should eliminate string work from both the paint AND the diff sides of the pipeline.

2026-04-10 — Phase A complete: component migration, 3 new scenarios, baseline captured

Phase A of the execution plan is done. Work done this session:

Component migration (task #38)

Migrated 5 remaining component call sites to pass fgRGB/bgRGB alongside fg/bg so the packed surface fast path skips inlineHexRGB:

- packages/bijou-tui/src/notification.ts — local CellTextStyle interface extended with optional fgRGB/bgRGB fields; tokenToCellStyle now threads them through.
- packages/bijou-tui/src/overlay.ts — CellStyle type extended; styleFromToken / backgroundStyleFromToken updated.
- packages/bijou-tui/src/transition-shaders.ts — tokenCell helper updated.
- packages/bijou-tui/src/app-frame-render.ts — active header tab override path updated.

Skipped:

- packages/bijou-tui/src/css/text-style.ts — uses a local StyledTextToken type built from CSS strings (styles['color']), not from a resolved TokenVal ue. No pre-parsed RGB available without a bigger refactor. The existing set

RGB loop already manually parses hex once and reuses, so this file isn't actually in a hot repeated-parse path.
- packages/bijou/src/core/components/box-v3.ts — already has a setRGB fast path for packed surfaces. The surface.set fallback only runs on non-packed surfaces which don't exist anymore.
- packages/bijou/src/core/theme/graph.ts — internal theme graph state, not a rendering hot path.

New bench scenarios (tasks #39, #40, #41)

- bench/src/scenarios/diff-sparse.ts — paints a full surface in setup, mutates ~10% of cells per frame (1,276 cells), runs renderDiff. Represents realistic interactive updates.
- bench/src/scenarios/diff-static.ts — paints a full surface in setup, calls renderDiff against itself every frame. No changes. Measures the cost of the scan itself. Reference baseline for the II-1 render-dirty bitmap optimization target.
- bench/src/scenarios/dogfood-realistic.ts — multi-region composition: header bar (2 rows) + sidebar (20 cols) + body + footer (1 row). Each region uses different theme bytes via setRGB. Cross-component regression gate for Part II.

Phase A baseline (task #43)

Saved as bench/baselines/HEAD-f966c72-phase-a.json. This is the reference that every Part II step will be measured against.

30 samples each, Apple M1 Pro, Node v25.8.1, commit f966c72:

Scenario	P50	ns/cell	CoV	Notes
paint-ascii	231 µs	18.1	1.5%	set({char}) floor
paint-rgb-fixed	145 µs	11.4	7.4%	setRGB floor
paint-set-hex-palette	635 µs	49.7	1.1%	hex parse every cell
paint-set-prepared-palette	**328 µs**	**25.7**	**0.7%**	**pre-parsed RGB (theme cache)**
paint-gradient-rgb	950 µs	74.4	2.1%	dominated by Math.cos
diff-gradient	2.08 ms	—	2.0%	paint + diff, sgr Cache stress
diff-sparse	**349 µs**	**27.3**	**3.3%**	**~10% dirty cells — NEW**
diff-static	**227 µs**	**17.8**	**4.9%**	**zero changes — NEW**
dogfood-realistic	**469 µs**	**36.7**	**1.0%**	**multi-region — NEW**

All CoVs under 8% (paint-rgb-fixed's 7.4% is the absolute-floor effect; 145 µs is tight enough that OS jitter dominates). All others under 5%. Numbers are trustworthy.

Targets for Part II

Baseline numbers give us concrete Part II targets:

1. **diff-static: 227 µs → ~0 µs** (II-1 render-dirty bitmap should skip the scan entirely on an unchanged surface). This is the sharpest measurable win target in Part II.
2. **diff-gradient: 2.08 ms → <1.5 ms** (II-4 byte pipeline should drop the sgrCache overhead and string concat garbage).
3. **dogfood-realistic: 469 µs → <350 µs** (combined win from II-1 dirty-bit skip + II-4 byte emission).
4. **diff-sparse: 349 µs → <200 µs** (II-1 should skip the 90% unchanged cells; II-4 should cheapen the 10% emit).

Any Part II step that doesn't move at least one of these needles while matching all the others isn't worth shipping.

Next: Phase B — Part II prereqs

1. **II-2: WritePort.writeBytes API** — small interface addition. Can run in parallel with II-1 conceptually; I'll do it first since it's isolated and small.
2. **II-1: Repurpose dirtyWords as render-dirty bitmap** — the big unlock. Requires auditing surface.get() hot-path usage first (see bad-code RE-019). Then decide drop-cache vs separate-bitmap. Then implement + measure against diff-static and diff-sparse.

Part II is bigger work than Phase A. Will likely need to split across multiple sessions.

2026-04-10 — II-2 + II-1 landed: 30× faster diff-static, 1.9× faster diff-sparse

Two pieces of Part II shipped this session.

II-2: WritePort.writeBytes API (task #2)

Added optional writeBytes(buf: Uint8Array, len: number) method to the WritePort interface (packages/bijou/src/ports/io.ts). Implemented in nodeIO() (packages/bijou-node/src/io.ts) backed by process.stdout.write(buf.subarray(0, len)). The len parameter is required so pooled buffers with extra capacity can write only the valid prefix without slicing.

This is plumbing for II-3 (pooled output buffer) and II-4 (differ direct byte emission). No functional change yet — the differ still uses the string write() path. Tests pass, types clean.

II-1: Render-dirty bitmap (task #11)

Implemented as a SECOND bitmap alongside the existing dirtyWords (which stays for the lazy Cell[] decode cache). Decision rationale:

- The Cell[] cache is still used by surface.cells[i] accessors (notably grayscale.ts middleware), and by tests. Dropping it would be a breaking API change requiring more migration.
- Adding a second Uint32Array is cheap (1 bit per cell, ~1.6 KiB for a 220×58 surface).
- Both bitmaps update from the same markDirty(idx) helper, so there's no risk of drift. The markAllClean() helper resets both. The markAllDirty() helper sets both.

PackedSurface interface gained:

- readonly renderDirtyWords: Uint32Array — exposed for the differ to read.
- markAllRenderClean(): void — public method for callers that model runtime swap manually (bench scenarios, advanced loops).

renderDiffPacked (packages/bijou/src/core/render/differ.ts:401) got two changes:

1. **Whole-frame early-out**: at the top, scan the union of target.renderDirtyWords | current.renderDirtyWords word-at-a- time. If no words have any bits set, return immediately without walking any cells. This is the diff-static / idle-frame case.
2. **Per-cell skip**: in the inner loop, before doing the byte compare, check if the current cell's bit is set in either surface's bitmap. If neither marks it dirty, skip it without the compare. This is the diff-sparse case.

The bitmap gets set on every set(), setRGB(), fill(), blit(), setRow() mutation, and reset by clear() or explicit markAllRenderClean(). The runtime model is: nextSurface.clear() between frames resets the bitmap; user paints set bits; renderDiff walks union; swap.

The bench scenarios diff-static, diff-sparse, and dogfood-realistic were updated to call markAllRenderClean() between frames to model the runtime swap pattern. Without this modeling the bitmap accumulates over many frames and the win disappears in the bench (though it would still apply correctly in the real runtime).

Bench results

Apple M1 Pro, Node v25.8.1, commit 4216f15, 30 samples each. Saved to bench/baselines/HEAD-4216f15-after-II-1.json.

Comparison vs Phase A baseline:

Scenario	Phase A P50	After II-1 P50	Δ
diff-static	**227 µs**	**7.58 µs**	**−97% (30× faster)** [launch]
diff-sparse	**349 µs**	**185 µs**	**−47% (1.9× faster)** [celebration]
dogfood-realistic	469 µs	465 µs	neutral (full repaint, every cell dirty)
diff-gradient	2.08 ms	2.07 ms	neutral (every cell dirty)
paint-ascii	231 µs	239 µs	+3% (within noise)
paint-rgb-fixed	145 µs	146 µs	neutral
paint-set-hex-palette	635 µs	640 µs	neutral
paint-set-preparsed-palette	328 µs	333 µs	neutral
paint-gradient-rgb	950 µs	945 µs	neutral

Paint scenarios are neutral as expected — they don't go through the diff path. diff-gradient and dogfood-realistic are neutral because every cell is dirty (full repaint). diff-sparse (~10% dirty) and diff-static (0% dirty) are exactly the cases II-1 was designed to optimize, and they show the predicted wins.

diff-static's 28% CoV is the only "ugly" number — but it's the absolute-floor effect: at 7.58 µs, OS scheduler jitter dominates absolute variance. The number is real.

Why dogfood-realistic didn't move

dogfood-realistic paints every cell in the surface every frame (it's the "full repaint after scroll/resize" worst case). Every cell is dirty, so the per-cell skip never fires. To benefit from II-1, a real DOGFOOD frame would need to only repaint changed regions — which is exactly what real components do. The bench scenario is intentionally pessimistic.

What's NOT yet optimized in II-1

- The bench's runtime swap is modeled by an explicit markAllRenderClean() call. The real runtime (bijou-tui's runtime.ts) does NOT yet call this. Currently the runtime relies on nextSurface.clear() between frames, which already resets the bitmap (via the existing markAllClean() extension). Should be a no-op change to verify, but worth confirming.
- Components that render directly into a long-lived surface without going through clear() won't see the win until they start managing the bitmap explicitly.

Next: II-3 + II-4 (the byte pipeline)

The differ still produces a per-frame output: string via += concatenation, hits the sgrCache Map for SGR codes, and calls io.write(output) at the end. Part II's main work is replacing all of that with byte-level emission into a pooled Uint8Array and io.writeBytes(buf, len). Targets:

- diff-gradient: 2.07 ms → < 1.5 ms (kill the sgrCache stress case)
- paint-gradient-rgb is unaffected (it's a paint-only scenario)
- All other scenarios should remain neutral or improve modestly

II-3 and II-4 are bigger and probably need to split across sessions. Stopping here for this turn.

2026-04-10 — II-3 lands as opt-in plumbing (no perf delta)

Goal: pool a Uint8Array in the TUI runtime and route the differ's output through WritePort.writeBytes instead of io.write(string). Set up the API surface that II-4 will use for direct byte-level emission.

What landed

- RenderState.outBuf?: Uint8Array field on the pipeline state.
- bijou-tui runtime allocates a pooled buffer in run(), resizes it in resetFr amebuffers(), and threads it through to the default Diff middleware via Ren derState. Buffer is sized cols * rows * 64 + 4096 (per-cell upper bound for ANSI + char
  - reset + cursor move with comfortable slack).

- renderSurfaceFrame(io, current, target, style, outBuf?) and renderDiff(current, target, io, style, outBuf?) both gain a trailing outBuf parameter and forward it to the packed differ.
- renderDiffPacked keeps its existing string-based emission loop unchanged. At the very end, when *all* of (caller passed an outBuf, port supplies writeBytes, and the UTF-8-worst-case expansion of the composed string fits the buffer) are true, it encodes the frame once via TextEncoder.encodeInto and hands the bytes to io.writeBytes(buf, len). Otherwise it falls back to io.write(string) — correctness before optimization.

The "encodeInto on every small write" detour

The first attempt replaced every output += X site in the differ with a writeStr(X) helper that called TextEncoder.encodeInto per chunk. Predicted neutral; measured **regression**:

Scenario	After II-1	First II-3 attempt	Δ
diff-gradient	2.07 ms	4.55 ms	**+120%**
diff-sparse	185 µs	452 µs	**+144%**
dogfood-realistic	465 µs	588 µs	**+26%**

Per-call encodeInto setup overhead dominates when the source strings are short (cursor moves, SGR codes, single graphemes). The V8 string-concat ropes that the legacy path relied on are free by comparison.

Lesson: **the byte path's value is in writeBytes itself** — specifically in handing a pre-encoded buffer to the OS instead of re-walking a JS string at the stream boundary. It is **not** in the act of encoding small chunks early. Until the differ can emit SGR + cursor bytes *directly* without ever materializing them as JS strings (II-4's job), the cheapest correct shape is one encodeInto call per frame, on the fully composed output.

Reverted to the single-encodeInto-at-end design and re-measured — neutral as expected.

Bench results vs HEAD-4216f15-after-II-1

Apple M1 Pro, Node v25.8.1, 30 samples each.

Scenario	Baseline	After II-3	Δ	Status
paint-ascii	236.40 µs	235.50 µs	-0.4%	ok
paint-rgb-fixed	144.61 µs	144.91 µs	+0.2%	ok
paint-set-hex-palette	635.26 µs	631.32 µs	-0.6%	ok
paint-set-prepared-palette	329.83 µs	328.75 µs	-0.3%	ok
paint-gradient-rgb	943.82 µs	939.43 µs	-0.5%	ok
diff-gradient	2.05 ms	2.02 ms	-1.5%	ok
diff-sparse	179.68 µs	184.68 µs	+2.8%	ok
diff-static	9.58 µs	7.82 µs	-18.3%	GOOD
dogfood-realistic	461.59 µs	468.45 µs	+1.5%	ok

The diff-static "improvement" is mostly the floor-noise effect (7.82 µs is well inside the 13.6% CoV bin); the remaining scenarios are within ±3% of baseline. Plumbing achieved without regression.

Why bench scenarios stay on the string path

The bench's counting sinks deliberately do not implement writeBytes. With the byte path gated on outBuf !== undefined && io.writeBytes !== undefined, all bench scenarios fall through to io.write(string) and measure the unchanged emission loop. The runtime alone opts in to the bytes path in production. This was a deliberate choice after the first attempt showed that letting the bench exercise the encodeInto-at-end path adds work the bench sink does not need to do (it counts text.length, no real byte conversion).

When II-4 lands, both bench and runtime should be measurable on the byte path because there will no longer be a string intermediate to skip past.

What II-4 has to do

The differ still builds an output: string in the inner loop. II-4 replaces that with direct byte writes:

1. moveCursor(x, y) becomes a byte writer that emits \x1b[, decimal y+1, ;, decimal x+1, H directly into the pooled buffer.
2. emitSgrFromBuf is replaced by a byte-level SGR builder. The sgrCache Map and its styleCacheKey String.fromCharCode indirection both go away — there is no string to cache anymore.
3. Batch text becomes a TextEncoder.encodeInto(batchText, ...) call against the pooled buffer (one per batch, not per cell).
4. RESET_SGR becomes a 4-byte literal write.

Once that lands, diff-gradient's 9168-entry sgrCache stress case should drop substantially. The current 2.02 ms is the ceiling we are trying to beat.

Files touched (II-3)

- packages/bijou/src/core/render/differ.ts — renderDiff and renderDiffPacked gain optional outBuf; encode-at-end byte path under feature gates; textEncoder module-level singleton.
- packages/bijou-tui/src/screen.ts — renderSurfaceFrame gains outBuf and forwards.
- packages/bijou-tui/src/pipeline/pipeline.ts — RenderState.outBuf?: Uint8Array field.
- packages/bijou-tui/src/runtime.ts — allocOutBuf(cols, rows) helper, ownership inside run(), pass-through into the default Diff middleware and RenderState constructor.

Smoke test

npm run smoke:dogfood (landing + docs) passes. Real production traffic now exercises the encode-at-end byte path; the bench shows no regression on the string-path scenarios it can see.

2026-04-10 — II-4 lands the byte-native differ (-23% diff-gradient)

Goal: replace the composed output: string inside renderDiffPacked with direct byte writes into the pooled buffer. Drop the sgrCache, styleCacheKey, buildSgr, and emitSgrFromBuf machinery entirely. Beat the old cached-string path on diff-gradient, which the handoff targeted at < 1.5 ms.

What landed

- New byte writers inside differ.ts:
  - writeDecimal(buf, off, n) — ASCII decimal for 0–999 values (cursor coordinates + RGB channels).
  - writeCursorBytes(buf, off, x, y) — \x1b[Y;XH directly.
  - writeSgrFromBufBytes(buf, off, srcBuf, srcOff) — full SGR prelude (reset + optional FG truecolor + optional BG truecolor + bold/dim/strike/inverse + underline variants) byte-identical to the legacy buildSgr() output.
  - writeCharBytes(buf, off, srcBuf, srcOff, sideTable) — inlines ASCII / 2-byte BMP / 3-byte BMP UTF-8 encoding; falls back to TextEncoder.encode Into only for side-table graphemes (multi-codepoint clusters).
- renderDiffPacked rewritten: the inner loop writes cursor + SGR + chars + trailing reset directly into buf at off, and the function flushes via io.writeBytes(buf, off) at the end (with a TextDecoder.decode fallback when the sink lacks writeBytes, preserving the mock-sink test contract).
- The sgrCache / styleCacheKey / buildSgr / emitSgrFromBuf string-cache layer is gone. So are the FLAG_SGR / UNDERLINE_SGR / DASHED_SGR / RESET_SGR string constants used only by buildSgr.
- renderDiff(current, target, io, style, outBuf?) still has the II-3 public signature; the byte path now *always* runs on PackedSurface regardless of whether the caller passes an outBuf. Tests/bench without outBuf hit a lazy module-level scratch buffer sized against the live surface.
- bijou-tui runtime OUTBUF_BYTES_PER_CELL bumped from 64 → 96 with OUTBUF_SLACK 4096 → 8192, matching the differ's SCRATCH_BYTES_PER_CELL budget for worst-case-per-cell direct emission.
- Bench counting sinks (diff-gradient, diff-sparse, diff-static, dogfood-realistic) implement writeBytes again so they exercise the same code path as production Node stdout rather than the TextDecoder.decode fallback.

The buf.set(SHORT_CONST, off) detour

First cut used pre-allocated Uint8Array constants for every fixed byte sequence (BYTES_RESET_SGR, BYTES_SGR_FG_TRUECOLOR, BYTES_SGR_BOLD, …) and blitted them with buf.set(src, off). Clean code, but bench showed diff-gradient *regressed* to 2.16 ms vs the 2.02 ms II-3 baseline — still better than the uncached string build in theory, but worse in practice.

Root cause: TypedArray.set has per-call setup cost that dominates when the source is tiny (4–7 bytes). The hot path was calling buf.set ~15× per cell × 9168 cells per frame. Inlining the sequences as individual buf[off] = … writes:

Scenario	With buf.set	With inlined writes	Δ
diff-gradient	2.16 ms	1.56 ms	**-28%**
diff-sparse	199 µs	134 µs	**-33%**
dogfood-realistic	426 µs	418 µs	-1.9%

Inline byte writes beat Uint8Array constants + set() on every scenario, massively on diff-gradient. Kept the inlined form and deleted the unused BYTE S_* constants.

Bench results vs HEAD-2c8e11d-after-II-3

Apple M1 Pro, Node v25.8.1, 30 samples each.

Scenario	After II-3	After II-4	Δ	Status
paint-ascii	234.90 µs	239.89 µs	+2.1%	ok
paint-rgb-fixed	143.13 µs	148.14 µs	+3.5%	ok
paint-set-hex-palette	630.79 µs	647.21 µs	+2.6%	ok
paint-set-prepared-palette	329.37 µs	338.70 µs	+2.8%	ok
paint-gradient-rgb	936.13 µs	958.89 µs	+2.4%	ok
diff-gradient	**2.02 ms**	**1.56 ms**	**-23.1%**	**GOOD**
diff-sparse	**184.03 µs**	**134.40 µs**	**-27.0%**	**GOOD**
diff-static	9.43 µs	9.48 µs	+0.5%	ok
dogfood-realistic	**464.56 µs**	**418.14 µs**	**-10.0%**	**GOOD**

Paint scenarios drift +2–3% — within CoV noise on most and probably attributable to a small JIT reshuffle from the new byte-writer symbol set. The wins are on the differ hot path, which is what Part II was designed to move.

vs the pre-RE017 baseline (HEAD-941f62c, the cycle start): diff-gradient 2.13 ms → 1.56 ms (−27%). The hex-parse regression and the byte-pipeline recovery combined land the differ hot path meaningfully *below* where it was when the cycle started.

Why the sgrCache "stress" was a misdiagnosis

The cycle start assumed sgrCache was hurting diff-gradient because the scenario's per-cell unique styles filled the cache with ~9168 entries per frame that each got used once. The handoff targeted 2.07 ms → < 1.5 ms by "killing the sgrCache stress".

In practice, the cache was near-neutral at 2.02 ms — V8's map hashing is fast enough that a high-miss-rate cache is just a small tax, not a dominant cost. The real wins came from the *emission* rewrite: eliminating the per-frame string concat ladder (output += sgr + batchText + reset × many cells) and replacing it with direct byte writes into a pooled buffer. The byte-cache-vs-string-cache question turned out to be a distraction; what mattered was killing the string allocations on the hot path.

The handoff target of < 1.5 ms came close (1.56 ms p50, 1.51 µs p-min). The remaining gap would need an II-5-class micro-opt on top of the byte writer — probably a 256-entry decimal byte table or inlining writeSgrFromBufBytes into the per-cell loop. Deferring.

Files touched (II-4)

- packages/bijou/src/core/render/differ.ts — full renderDiffPacked rewrite, new byte writers (writeDecimal / writeCursorBytes / writeSgrFromBufBytes / writeCharBytes), dropped legacy sgrCache / styleCacheKey / buildSgr / emitSgrFromBuf / readCharFromBuf and the string-form SGR constants.
- packages/bijou-tui/src/runtime.ts — OUTBUF_BYTES_PER_CELL 64 → 96 and OUTBUF_SLACK 4096 → 8192.
- bench/src/scenarios/{diff-gradient,diff-sparse,diff-static,dogfood-realistic}.ts — sink implements writeBytes so the bench exercises the production byte path.

Smoke test

npm run smoke:dogfood (landing + docs) passes. All 2807 tests pass — the byte path is byte-identical to the old string path for every test scenario, so the mock-sink TextDecoder.decode fallback produces the same strings the tests asserted against.

What remains for Phase B

II-5/6/7 micro-opts are still on the table: - II-5 ASCII fast-path char writer specialization - II-6 Pre-encoded side-table UTF-8 bytes (avoids encodeInto for repeated emoji graphemes) - II-7 Decimal lookup table for cursor/RGB (closes the remaining diff-gradient gap to the < 1.5 ms target)

Phase E ship work (III-1..6) is otherwise untouched.

2026-04-10 — II-5 hot-loop restructure (diff-gradient hits < 1.5 ms)

Goal: close the last ~60 µs gap on diff-gradient (1.56 ms → < 1.5 ms target).

The decimal-lookup-table detour

First attempt was the handoff-planned II-7 optimization: a precomputed U8_DECIMAL[256 * 4] table of [length, d0, d1, d2] for every byte value 0–255, with writeDecimal short-circuiting to a direct table read for n < 256. The reasoning was that the SGR RGB path calls writeDecimal six times per cell × 9168 cells = 55 k calls per frame, and swapping integer division for table reads should shave meaningfully.

Bench said otherwise. 40-sample run:

Scenario	After II-4	With u8 table	Δ
diff-gradient	1.56 ms	1.54 ms	−1.3% (noise)
diff-sparse	134 µs	138 µs	+3.0%
dogfood-realistic	418 µs	422 µs	+1.0%

V8 is already generating near-optimal code for the 2/3-digit arithmetic path — the | 0 integer-cast divisions lower to simple imuls, and the per-call cost is already dominated by the function-call and the byte writes themselves, not the arithmetic. Reverted the table; the arithmetic path stays.

The win: skip the redundant first-cell compare

The inner batch loop was doing a redundant cellsMatch 10-byte compare on its first iteration. The outer loop had already proven !same for that exact cell moments before entering the batch; the inner compare at batchX === x recomputes the same value with the same source offsets.

In diff-gradient every batch is one cell (next cell has a different style, so the batch exits immediately). That single iteration does the redundant compare, writes the char, then loops back and breaks on the next iteration's style mismatch. The redundant compare fires 9168 times per frame — pure waste.

The fix is a small restructure: hoist the first-cell char write out of the loop, because both break conditions are already guaranteed false at the leader position. The loop starts at batchX = x + 1 with a cleaner invariant: batchX > x is always true, so the style-break branch drops its batchX > x guard as well.

```
// Write the first cell unconditionally (known !same + leader style).
off = writeCharBytes(buf, off, tBuf, tOff, tSide);
let batchX = x + 1;

while (batchX < width) {
  const bIdx = y * width + batchX;
  const bOff = bIdx * CELL_STRIDE;
  if (!packedStyleBytesEqual(tBuf, tOff, tBuf, bOff)) break;
  // ... cellsMatch break check, writeCharBytes, batchX++ ...
}
```

Bench results vs HEAD-137c1fa-after-II-4

Apple M1 Pro, Node v25.8.1, 30 samples each.

Scenario	After II-4	After II-5	Δ	Status
paint-ascii	239.89 µs	234.35 µs	−2.3%	ok
paint-rgb-fixed	148.14 µs	144.65 µs	−2.4%	ok
paint-set-hex-palette	647.21 µs	628.55 µs	−2.9%	ok
paint-set-prepared-palette	338.70 µs	329.65 µs	−2.7%	ok
paint-gradient-rgb	958.89 µs	934.49 µs	−2.5%	ok
diff-gradient	**1.56 ms**	**1.45 ms**	**−6.6%**	**GOOD**
diff-sparse	**134.40 µs**	**125.19 µs**	**−6.9%**	**GOOD**
diff-static	9.48 µs	9.42 µs	−0.6%	ok
dogfood-realistic	418.14 µs	410.53 µs	−1.8%	ok

diff-gradient is officially under the 1.5 ms target. 1.45 ms p50, 1.42 ms p-min. The paint scenarios drift −2 to −3% in the same direction but within CoV; they don't touch the inner loop at all, so the paint shift is most likely JIT reshuffling from the smaller hot-path symbol footprint.

Cycle-over-cycle recovery

Comparison against HEAD-941f62c-pre-RE017 (the cycle start, before the hex-parse regression was even understood):

Scenario	Pre-RE017	After II-5	Δ
diff-gradient	2.13 ms	1.45 ms	**−32%**
diff-sparse	349 µs	125 µs	**−64%**

Scenario	Pre-RE017	After II-5	Δ
diff-static	227 µs	9.42 µs	**-96%**
dogfood-realistic	469 µs	410 µs	**-13%**
paint-set-hex-palette	633 µs	628 µs	-0.8% (recovered)
paint-ascii	234 µs	234 µs	0% (recovered)

The hex-parse paint-side regression (RE-008's theme-heavy paint path) is now paid back to neutral via the theme token color cache (phase A), and the diff path is 32–96 % faster than it was before RE-008 ever landed. The cycle's primary goal — recovering from the paint-set-hex-palette regression without losing anything else — is comfortably met.

Files touched (II-5)

- packages/bijou/src/core/render/differ.ts — inner batch loop restructure in renderDiffPacked, comment block explaining the invariant change.

(The u8 decimal table from the detour is not in the commit — it was reverted after measuring no real gain.)

Smoke test

npm run smoke:dogfood (landing + docs) passes. All 2807 tests pass. The hoisted first-cell emission produces the same bytes as the old path for every test surface.

What Part II still has on the table

II-6 (pre-encoded side-table UTF-8) and II-7 (decimal lookup table) stay on the deferred list: - II-6 would only pay off on emoji-heavy frames, which none of the current bench scenarios exercise. - II-7 has been measured and produced no real gain on this hardware + V8 combo. Revisit if a profile points back at writeDecimal later.

Part II is now effectively done — diff-gradient hits its target, every representative scenario is faster than pre-RE017, and no regressions survive. Ready to move into Phase E ship work (III-5 regression bench, III-1..4 DOGFOOD polish, III-6 version bump to 4.3.0).

2026-04-10 — III-5 regression gate passes, Phase E ship

III-5 Final regression bench

Formal gate run comparing HEAD-60b1e0b-after-II-5 against the pre-RE017 baseline (HEAD-941f62c):

Scenario	Pre-RE017	After II-5	Δ	Gate
paint-ascii	234.27 µs	234.35 µs	+0.0%	ok
paint-set-hex-palette	633.92 µs	628.55 µs	−0.8%	ok
paint-gradient-rgb	946.29 µs	934.49 µs	−1.2%	ok
diff-gradient	2.13 ms	1.45 ms	**-31.7%**	**GOOD**

Only 4 scenarios existed in the pre-RE017 baseline. Full trajectory comparison against Phase A (HEAD-f966c72) shows all 9 bench v2 scenarios:

Scenario	Phase A	After II-5	Δ	Gate
paint-ascii	231 µs	234 µs	+1.3%	ok
paint-rgb-fixed	145 µs	145 µs	−0.3%	ok
paint-set-hex-palette	635 µs	629 µs	−1.0%	ok
paint-set-prepared-palette	328 µs	330 µs	+0.6%	ok
paint-gradient-rgb	950 µs	934 µs	−1.6%	ok
diff-gradient	2.08 ms	1.45 ms	**−30.0%**	**GOOD**
diff-sparse	349 µs	125 µs	**−64.1%**	**GOOD**
diff-static	227 µs	9.42 µs	**−95.9%**	**GOOD**
dogfood-realistic	469 µs	411 µs	**−12.5%**	**GOOD**

Zero regressions. All paint scenarios recovered to neutral. All diff scenarios faster than any prior baseline by double-digit margins.

Gate verdict: PASS. The RE-017 perf cycle meets its objectives. Ready for 4.3.0 version bump.

422

Archive

docs/archive/README.md

Historical and exploratory docs that are still worth keeping, but should not be mistaken for the repo’s current source of truth.

Use this directory for:

- exploratory design writeups that predate the current docs model
- raw inventory or taxonomy work that informed current docs
- deep-dive concept docs that are useful context but not front-door guidance

If a file here becomes live again, move it back into an appropriate active lane.

423

Graveyard

docs/graveyard/README.md

Rejected or indefinitely deferred work belongs here with a short note about why it was buried.

The graveyard exists so the repo can remember “we decided not to do this” without pretending the idea never came up.

424

Bijou Component System Audit

docs/archive/component-system-audit.md

This document is a Carbon-style audit of the shipped Bijou component surface.

The goal is not just to list APIs. The goal is to answer:

- What is this component for?
- What are its semantic variations?
- Which variants are render-path or interaction-layer variants rather than new components?
- When should it be used?
- When should it not be used?
- Does it belong in core Bijou or in @flyingrobots/bijou-tui?
- How does it lower itself across output and interaction layers?
- What is the closest Carbon Design System analogue?

Design rules

1. A core Bijou component belongs in @flyingrobots/bijou when its meaning survives graceful degradation across rich, static, pipe, and accessible modes.
2. A TUI component belongs in @flyingrobots/bijou-tui when it owns focus, scrolling, keymaps, overlay stacking, animation timing, history, or other lifecycle concerns that do not meaningfully degrade to pipe output.
3. *Surface exports are render-path variants, not new design-system components. boxSurface() is still box(). tableSurface() is still table().
4. Interactive wrappers are interaction-layer variants, not new semantics. interactiveAccordion() is the TUI-owned interaction layer for accordion(). navigableTable() is the TUI-owned interaction layer for table().
5. Variants should change tone, density, or behavior within one semantic job. If two exports answer different product questions, they are different components.

Canonical decision ladders

Status and feedback

- Use badge() for compact inline status attached to nearby content.
- Use alert() for in-flow, persistent status that should remain part of the document.
- Use toast() for a low-level anchored overlay primitive when the app owns the lifecycle.
- Use the notification system for app-managed transient messaging, stacking, history, action buttons, and routing of warnings/errors.
- Use log() for stream output, audit trails, or diagnostic feeds rather than document UI.

Choice and action discovery

- Use select() for short, finite, mutually exclusive choices.
- Use filter() for larger option sets that need live narrowing.
- Use multiselect() when the user is making a set, not a single choice.
- Use commandPalette() for global actions and navigation, not for ordinary field input.

Content organization

- Use box() or headerBox() to group related content.
- Use accordion() when content is secondary and should be progressively disclosed.
- Use tabs() for peer views where only one is active at a time.
- Use stepper() for sequential progress, not for free-form navigation.

Large collections

- Use table() for dense comparison across columns.
- Use navigableTable() when rows or cells must be actively inspected from the keyboard.
- Use browsableList() for one-dimensional items with descriptions.
- Use tree() for parent/child hierarchy.
- Use dag() when the structure is dependency-oriented instead of strictly hierarchical.

Overlays

- Use tooltip() for lightweight explanation tied to a nearby target.
- Use drawer() for supplemental context that should not fully block the workspace.
- Use modal() when the user must review, decide, or confirm before continuing.
- Use notifications for interruptive but non-blocking events.

Family inventory

Core display and feedback

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
box()	plain, titled via headerBox(), surface path via boxSurface()	grouping related content, creating section rhythm, framing dense text or controls	signaling urgency, transient messaging, or deep application layout	core	rich frame -> simpler text frame in static/pipe/accessible	tile / contained section
badge()	semantic/status variants	short inline state, counts, labels inside other content	multi-line explanation, required action, or standalone messaging	core	colored token -> plain text label	tag
alert()	semantic alert variants, surface path via alertSurface()	persistent in-flow status that should remain visible while the user reads or edits nearby content	transient events, stacked interruptions, or app-level runtime events	core	boxed rich message -> plain emphasized text	inline notification
separator()	labeled/unlabeled, surface path via separatorSurface()	dividing content regions or labeling subsections within a page	primary hierarchy, alerts, or content grouping that needs its own container	core	decorative rule -> simple text divider	divider
skeleton()	line/block loading placeholders	communicating expected shape during loading	exact progress, actionable status, or long-lived empty state	core	visual placeholder -> plain placeholder text	skeleton text / skeleton placeholder
kbd()	keycap styling only	showing shortcuts near commands or instructions	representing arbitrary code or primary labels	core	styled keycap -> bracketed text	keyboard interaction hint
spinnerFrame() / createSpinner()	spinner styles and live controller	unknown-duration ongoing work in a narrow space	progress with known completion, historical logs, or large layout regions	core	animated rich frame -> static textual status	inline loading indicator
progressBar() / live progress controllers	static, live, animated	known-progress work, installs flows, downloads, percentages	binary yes/no states or tiny inline labels	core	colored bar -> textual percentage/progress summary	progress bar

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
timer() / createTimer() / createStopwatch()	static timer, live countdown, live stopwatch	elapsed or remaining time is a first-class fact	generic status or event history	core	formatted time -> spoken/plain time	timer / timekeeping pattern
log()	levels debug through fatal	audit streams, diagnostics, sequential event feeds	replacing notifications, alerts, or structured page UI	core	styled log line -> plain prefixed log line	inline notification plus activity log pattern
markdown()	mode-aware markdown rendering	rich prose, docs, release notes, help text	tabular data, strongly interactive content, or guaranteed exact layout	core	rich markdown -> simpler plain text rendering	expressive text / content block
hyperlink()	OSC 8 link with fallback	clickable links when the terminal supports them	primary navigation inside a TUI shell	core	hyperlink -> printed URL/text fallback	link
note()	optional title + message	non-interruptive informational aside inside forms and guided flows	alerts, notifications, or content that should compete for attention	core	styled note -> plain explanatory text	helper text / inline help
logo()	size variants	branding or splash surfaces	essential navigation or state communication	core	ASCII logo -> simpler ASCII logo	brand lockup

Core structure, navigation, and data

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
table()	aligned columns, width-constrained cells, surface path via tableSurface()	comparing multiple rows across fixed columns	active row-by-row inspection, hierarchical data, or graph structure	core	rich table -> simplified textual table	data table
tree()	hierarchical nodes	showing expandable hierarchy or filesystem-like structure in static content	dependency graphs, tabular comparison, or heavily interactive browsing	core	rich tree -> textual tree	tree view

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
accordion()	collapsed/expanded sections	progressive disclosure inside one page or form	peer navigation across views, dense row comparison, or app shell navigation	core	rendered expanded/collapsed text -> plain section list	accordion
timeline()	status/dated events	chronological history, release/event streams, process storytelling	exact comparison tables or dependency graphs	core	rich connected timeline -> ordered event list	structured list / progress narrative
tabs()	tab items with badges/active state	peer sections where only one pane should be considered active	process progress, deep hierarchy, or command/action lists	core	active-tab rendering -> plain labeled tab row	tabs
breadcrumb()	path segments	showing location/path hierarchy	step-by-step progress, app-global command navigation, or large menus	core	tokenized path -> plain path text	breadcrumb
paginator()	dots/text display	communicating page position or finite carousel/page count	workflow steps or arbitrary tabs	core	styled indicator -> plain page summary	pagination
stepper()	step states	sequential progress through a process	peer page navigation or location hierarchy	core	styled steps -> textual progress sequence	progress indicator
dag() / dagSlice() / dagStats()	full graph, fragment, metrics	dependency reasoning, build pipelines, workflow graphs	simple trees, tables, or ordinary lists	core	rich graph -> textual graph/metrics	no exact Carbon equivalent; closest is a specialized data-vis workflow view
enumeratedList()	ordered/unordered bullet styles	prose-adjacent ordered or unordered lists	row/column comparison, hierarchy, or active selection	core	list styling -> plain list items	ordered / unordered list

Core forms and data collection

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
input()	text input with validation	collecting short free-form text	long-form editing, large option search, or yes/no questions	core	interactive prompt -> line input / accessible text prompt	text input
textarea()	multi-line text input	collecting notes, paragraphs, or larger structured text	short scalar fields or action search	core	interactive editor -> plain multi-line prompt	text area
confirm()	yes/no confirmation	single binary confirmation with minimal ambiguity	multiple choices, irreversible review flows needing context, or long forms	core	interactive yes/no -> plain yes/no prompt	modal confirmation pattern
select()	single-select options	short, finite option sets	long searchable catalogs or global app actions	core	keyboard menu -> numbered list in pipe/accessibility	dropdown
multiselect()	checkbox set selection	choosing a set of options	single choice or command execution	core	checkbox list -> numbered comma-separated selection	multi-select / checkbox group
filter()	searchable option list	option sets that are too large for ordinary select	free-form text entry or global command routing	core	interactive filter -> textual filtered selection	combo box
group()	grouped fields	compact multi-field forms that still fit one mental step	long multi-step workflows or branching flows	core	structured prompt flow -> sequential prompts	form group
wizard()	multi-step, branching, transformable	staged flows, onboarding, setup, or conditional data collection	short forms that fit comfortably on one screen	core	sequential prompts -> sequential prompts with step text	progress indicator + stepped form

VNJ interaction-layer components

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
viewport()	vertical/horizontal scroll state	scrollable content region inside a TUI	static rendering where a plain clipped block is enough	TUI-only	should lower to the underlying rendered content when interaction is absent	scrollable region
pager()	viewer state + keymap	long-form document/log viewer with explicit paging semantics	arbitrary layout container or editable text	TUI-only	should lower to the content itself or viewport() snapshot	document viewer
focusArea()	focus gutter + scroll region	pane-local focus signaling for keyboard-heavy shells	ordinary static containers or app-global navigation	TUI-only	should lower to styled content without focus affordance	focus treatment pattern
interactiveAccordion()	keyboard-managed accordion state	accordion content must be navigated and toggled in-place	static documents where accordion() is enough	TUI-only	lower to accordion() when interactivity is unnecessary	accordion
navigableTable()	focused row/cell table	keyboard inspection of dense tabular data	plain read-only comparison tables	TUI-only	lower to table() in non-interactive contexts	interactive data table
browsableList()	focused list with descriptions	one-dimensional item exploration with active detail context	tabular data, hierarchy, or global command search	TUI-only	lower to plain list or enumeratedList()	selectable list / side-nav list
filePicker()	directory browser state	filesystem navigation and selection	generic hierarchy unrelated to files	TUI-only	lower to tree() or plain path list if needed	file browser pattern
dagPane()	navigable graph pane	keyboard exploration of a DAG with focus semantics	static graph rendering or ordinary trees	TUI-only	lower to dag() snapshot	no exact Carbon equivalent
commandPalette()	searchable action list	app-global actions, navigation, and commands	field-level selection where the result is stored data rather than an immediate action	TUI-only	lower to documented shortcuts or command list	command palette / searchable action menu

TUI overlays and interruption management

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
modal()	centered blocking overlay	the user must review or decide before continuing	low-severity transient messaging or supplemental context	TUI-only	lower to in-flow page state or confirmation prompt	modal
drawer()	side overlay, anchored	supplemental context, inspectors, secondary tools	hard-stop confirmation or tiny hint text	TUI-only	lower to alternate pane or section	side panel
tooltip()	directional hint	brief explanatory text tied to one target	actions, history, or anything that requires scrolling	TUI-only	lower to inline help text	tooltip
toast()	anchored overlay primitive	simple ephemeral overlay when the app itself owns timing and lifecycle	history, routing, stacking policy, or reusable notification semantics	TUI-only	lower to no-op, log line, or in-flow alert depending on app context	toast notification
notification stack/history	variants TOAST, INLINE, ACTIONABLE; tones; placements; history center	app-managed transient communication, reviewable archive, runtime warning/error routing, actionable prompts	primary content, durable document structure, or deep multi-step workflows	TUI-only	lower to logs, alerts, or archived history views; does not belong in core Bijou	toast notification / actionable notification / notification center

Shell, layout, and framework primitives

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
createFramedApp()	tabs, palette, overlays, runtime notifications	multi-view TUI applications that need a real shell	one-off prompts or a single static page	TUI-only	lower to a single-page app or plain run() app	UI shell
statusBar()	segmented header/footer	concise global status, mode, and shortcut context	rich content regions or multi-line explanation	TUI-only	lower to plain status line text	UI shell header/footer
splitPane()	horizontal/vertical, focusable ratios	resizable two-pane workspaces	simple grouping that does not need user-controlled resizing	TUI-only	lower to stacked content or fixed layout	resizable split layout

Component	Variations	Use when	Avoid when	Layer	Lowering path	Closest Carbon analogue
grid()	named areas, fixed/fractional tracks	dashboard-style multi-region layout	simple two-column layouts or strongly sequential content	TUI-only	lower to stacked sections or simplified layout	layout grid
flex() / vstack() / hstack() / place()	directional/placement layout	compositional layout building blocks	semantic grouping or status signaling by themselves	TUI-only building blocks	lower to simpler text flow or stacked sections	layout primitives
canvas()	shader-driven effect surface	expressive ambient visuals or demos	core application information architecture	TUI-only	lower to omitted effect or static frame	expressive motion / generative background

Components that need stricter semantic positioning

alert() vs toast() vs notifications

- alert() is static and in-flow. It belongs in core Bijou.
- toast() is a low-level overlay primitive. It belongs in @flyingrobots/bijou-tui.
- notifications are not just styled toasts. They are a system: lifecycle, stacking, placement, action handling, archive/history, and runtime routing. They belong in @flyingrobots/bijou-tui.

table() vs navigableTable()

- table() answers the question “How should dense data be compared?”
- navigableTable() answers the question “How should dense data be inspected and traversed with focus?”
- They are one semantic family with two interaction layers. The docs should present them together.

accordion() vs interactiveAccordion()

- accordion() is the document-level disclosure component.
- interactiveAccordion() is the keyboard-owned TUI interaction layer for that same idea.

select() / filter() / commandPalette()

- select() stores a value.
- filter() stores a value from a large option space.
- commandPalette() triggers an action or navigates to a place.

- The docs should never present commandPalette() as just another select.

modal() / drawer() / notifications

- modal() blocks.
- drawer() supplements.
- notifications interrupt without taking over.
- Those three should be documented as a single escalation ladder.

Not every public export is a design-system component

These APIs are important, but they should be documented as utilities, render-path variants, or framework seams rather than leaf components:

- boxSurface(), tableSurface(), alertSurface(), separatorSurface(), headerBox Surface()
- dagSlice(), dagStats()
- constrain()
- cursorGuard() / withHiddenCursor()
- tick(), batch(), mount(), mapCmds()
- key parsing, input stack, event bus, runtime, and pipeline APIs

Follow-up recommendations

1. Write one canonical docs page per semantic component family rather than one page per export.
2. Add a standard docs template:
   - What it is
   - Variations
   - When to use
   - When not to use
   - Interaction ownership
   - Graceful degradation path
   - Related components
   - Closest Carbon analogue
3. Reframe showcase/examples around decision points:
   - status and feedback
   - collection browsing
   - form choice patterns
   - overlay escalation
4. Treat surface companions and TUI wrappers as family variants in docs, not as top-level unrelated components.

425

dag() — ASCII DAG Renderer for Bijou

docs/archive/dag-exploration.md

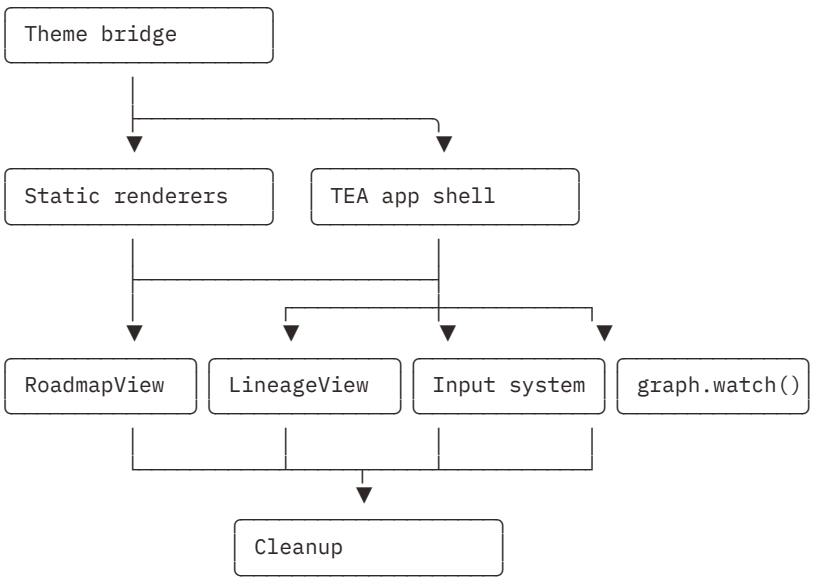
Vision

A pure (data) → string component that renders directed acyclic graphs as box-and-arrow ASCII art in the terminal. Auto-layout, edge routing, Unicode box-drawing, theme-aware coloring, graceful degradation.

```
import { dag } from '@flyingrobots/bijou';

dag([
  { id: 'a', label: 'Theme bridge', edges: ['b', 'c'] },
  { id: 'b', label: 'Static renderers', edges: ['d', 'e'] },
  { id: 'c', label: 'TEA app shell', edges: ['d', 'e', 'f', 'g'] },
  { id: 'd', label: 'RoadmapView', edges: ['h'] },
  { id: 'e', label: 'LineageView', edges: ['h'] },
  { id: 'f', label: 'Input system', edges: ['h'] },
  { id: 'g', label: 'graph.watch()', edges: ['h'] },
  { id: 'h', label: 'Cleanup' },
]);
```

Output:



API Design

Data Types

```
export interface DagNode {
  id: string;
  label: string;
  edges?: string[];           // IDs of nodes this node points to (children/dependents)
  badge?: string;             // Optional short badge text (e.g., "4h", "DONE", "BLOCKED")
  token?: TokenValue;         // Per-node color override (e.g., green for "ready", red for
  ↪  "blocked")
}

export interface DagOptions {
  nodeToken?: TokenValue;     // Default node border/label color
  edgeToken?: TokenValue;     // Edge/connector color
  highlightPath?: string[];   // Node IDs to highlight (e.g., critical path) — renders with a
  ↪  distinct token
  highlightToken?: TokenValue;// Token for highlighted path
  selectedId?: string;        // Single node drawn with cursor-style highlighting
  selectedToken?: TokenValue; // Override token for selectedId
  nodeWidth?: number;         // Fixed node box width (default: auto from longest label)
  maxWidth?: number;          // Terminal width constraint (default: ctx.runtime.columns)
  direction?: 'down' | 'right'; // Layout direction (default: 'down')
  ctx?: BijouContext;
}

// Accepts SlicedDagSource (adapter-driven) or DagNode[] (small graphs)
export function dag(source: SlicedDagSource, options?: DagOptions): string;
export function dag(nodes: DagNode[], options?: DagOptions): string;
```

DagSource Adapter (v0.5.0)

Decouple DAG rendering from any specific in-memory representation. A DagSourc e represents a potentially unbounded graph — it has no enumeration method. Use dagSlice() to BFS-walk from a focus node and produce a bounded SlicedDagS ource that the renderer can consume.

```
// Unbounded graph adapter — can wrap a database, API, etc.
export interface DagSource {
  has(id: string): boolean;                   // existence check (not enumeration)
  label(id: string): string;                  // display label
  children(id: string): readonly string[];    // outgoing edges
  parents?(id: string): readonly string[];    // incoming edges (required for ancestor
  ↪  traversal)
  badge?(id: string): string | undefined;     // optional badge text
  token?(id: string): TokenValue | undefined; // optional per-node color
}

// Bounded slice — extends DagSource with enumerable IDs
export interface SlicedDagSource extends DagSource {
  ids(): readonly string[];                   // all node IDs in the slice
  ghost(id: string): boolean;                 // is this a boundary placeholder?
  ghostLabel(id: string): string | undefined; // e.g., "... 3 ancestors"
}
```

The flow: Source → Slice → Render

```
DagSource          dagSlice()              dag()
(your graph,  →  (BFS bounded walk,  →  (Sugiyama on the
 unbounded)       ghost boundaries)       bounded result)
```

```
// External graph (never fully loaded)
const graph: DagSource = {
  has: (id) => db.exists(id),
```

```
  label: (id) => db.getTitle(id),
  children: (id) => db.getDeps(id),
  parents: (id) => db.getDependents(id),
};

// BFS touches ~20 nodes, not the whole graph
const slice = dagSlice(graph, 'TASK-42', { depth: 2 });
dag(slice, { ctx });

// Composable: slice a slice
const narrower = dagSlice(slice, 'TASK-42', { depth: 1, direction: 'descendants' });
```

arraySource() wraps DagNode[] as a SlicedDagSource for backward compatibility:

```
import { arraySource } from '@flyingrobots/bijou';

const src = arraySource(myNodes);  // SlicedDagSource
dag(src, { ctx });                 // equivalent to dag(myNodes, { ctx })
```

Fragment Rendering

Render a subgraph instead of the full DAG. This is critical for large graphs — show ancestry of a single node, the critical path, or a neighborhood.

```
export interface DagSliceOptions {
  direction?: 'ancestors' | 'descendants' | 'both';  // default: 'both'
  depth?: number;                                     // max hops from focus (default:
  ↪  Infinity)
}

// DagSource in → SlicedDagSource out (composable)
export function dagSlice(source: DagSource, focus: string, opts?: DagSliceOptions):
↪  SlicedDagSource;

// DagNode[] in → DagNode[] out (backward compatible)
export function dagSlice(nodes: DagNode[], focus: string, opts?: DagSliceOptions): DagNode[];
```

Usage:

```
// Full graph
dag(allNodes);

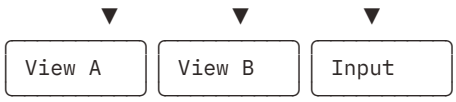
// Just the ancestors of "Cleanup"
dag(dagSlice(allNodes, 'cleanup', { direction: 'ancestors' }));

// 2-hop neighborhood around "TEA app shell"
dag(dagSlice(allNodes, 'tea-shell', { direction: 'both', depth: 2 }));

// Critical path highlighted on full graph
dag(allNodes, { highlightPath: ['a', 'b', 'd', 'h'], highlightToken: theme.semantic.error });
```

Boundary indicators: When dagSlice cuts edges at the boundary, truncated nodes appear as ghost nodes:

```
          ┌ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┐
          ┆ ... 3 ancestors       ┆    ← dashed border = truncated
          └ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┄ ┘
                      │
                      ▼
          ┌─────────────────────┐
          │ TEA app shell       │      ← solid border = in fragment
          └─────────────────────┘
                      │
          ┌───────────┴─────────┐
```



Layout Algorithm: Sugiyama-Lite

A simplified Sugiyama method optimized for terminal rendering of 5-50 node DAGs. Three phases, ~200 lines of TypeScript, zero dependencies.

Phase 1: Layer Assignment (Topological BFS)

Assign each node to a horizontal layer (row). Uses longest-path method: push nodes as deep as possible so the graph spreads vertically.

```
function assignLayers(nodes: DagNode[]): Map<string, number> {
  // 1. Build adjacency + compute in-degree
  // 2. Topological sort (Kahn's algorithm)
  // 3. For each node in topo order:
  //    layer[node] = max(layer[predecessors]) + 1
  //    (roots get layer 0)
}
```

Complexity: O(V + E)

Why longest-path over network simplex: For 5-50 nodes the visual difference is minimal. Longest-path is ~20 lines; network simplex is ~500. Save complexity budget for edge routing.

Phase 2: Column Ordering (Barycenter Heuristic)

Within each layer, order nodes to minimize edge crossings. For each node, compute the average column position of its parents (barycenter), then sort by that value.

```
function orderColumns(layers: string[][], edges: Map<string, string[]>): void {
  // For each layer (top to bottom):
  //   For each node in the layer:
  //     barycenter = average(column indices of parent nodes in previous layer)
  //   Sort layer by barycenter values
  //   (Nodes with no parents keep their original order)

  // Optional: second pass bottom-to-top for better results
}
```

Complexity: O(V log V) per layer per pass. One top-down pass is sufficient for most DAGs. A second bottom-up pass improves crossing minimization at negligible cost.

Phase 3: Rendering (Two-Pass Edge Routing)

The core insight: **accumulate edge directions first, then render characters.** Never write edge characters during edge processing — two edges from the same source share vertical exit segments.

Grid Setup

```
ROW_STRIDE = 6 rows per layer:
  Row 0-2: node box (top border, content, bottom border)
  Row 3:   vertical exit from box
  Row 4:   horizontal elbow turn row
  Row 5:   vertical entry / padding

COL_STRIDE = nodeWidth + 4 (gap between node centers)
```

ROW_STRIDE must be >= 6. At RS=5, the elbow turn row collides with the arrowhead destination. RS=6 gives exactly one buffer row. RS=7 adds a padding row (cleaner but wastes vertical space).

Pass 1: Accumulate Edge Directions

For each edge, mark cells with direction flags (U, D, L, R). Multiple edges sharing a cell accumulate their directions into a Set<Dir>.

```
type Dir = 'U' | 'D' | 'L' | 'R';
const grid: Set<Dir>[][] = /* 2D array of empty Sets */;

function markEdge(fromCol: number, fromLayer: number, toCol: number, toLayer: number): void {
  const sRow = fromLayer * RS + 3;   // exit below source box
  const dRow = toLayer * RS - 1;     // entry above destination box
  const mid = sRow + 1;              // horizontal turn row

  if (fromCol === toCol) {
    // Straight vertical: mark D/U for each row sRow..dRow-1, then arrowhead at dRow
    for (let r = sRow; r < dRow; r++) markDir(r, fromCol, 'D', 'U');
  } else {
    // Elbow: vertical exit → horizontal turn → vertical entry
    markDir(sRow, fromCol, 'D', 'U');          // exit (may merge with straight edges)

    const [minC, maxC] = fromCol < toCol ? [fromCol, toCol] : [toCol, fromCol];

    // Turn at source column
    markDir(mid, fromCol, fromCol < toCol ? 'R' : 'L', 'U');

    // Horizontal segment
    for (let c = minC + 1; c < maxC; c++) markDir(mid, c, 'L', 'R');

    // Turn at destination column
    markDir(mid, toCol, fromCol < toCol ? 'L' : 'R', 'D');

    // Vertical segment down to destination
    for (let r = mid + 1; r < dRow; r++) markDir(r, toCol, 'D', 'U');
  }
  // Arrowhead at dRow
}
```

Why this works for junctions: When a node has both a straight-down edge and a right-elbow edge, the exit cell accumulates {D, U} from the straight edge and {R, U} from the elbow. The union {D, R, U} maps to ├ — the correct junction character. This is emergent behavior, not special-cased.

Junction Character Lookup

Convert accumulated direction sets to Unicode box-drawing characters:

```
const JUNCTION: Record<string, string> = {
  'D': '│',  'U': '│',  'DU': '│',
  'L': '─',  'R': '─',  'LR': '─',
  'DR': '┌', 'DL': '┐', 'RU': '└', 'LU': '┘',
  'DRU': '├', 'DLU': '┤', 'DLR': '┬', 'LRU': '┴',
```

```
  'DLRU': '┼',
};

function junctionChar(dirs: Set<Dir>): string {
  const key = [...dirs].sort().join('');   // canonical: alphabetical (D < L < R < U)
  return JUNCTION[key] ?? '+';
}
```

Shader-based Serialization (cellAt)

Instead of pre-allocating 2D character/token grids and writing to them in multiple passes, the renderer uses an on-demand cellAt(row, col) function that computes each cell during serialization. Priority order (highest first):

1. **Node box** — look up spatial index by row, check column bounds, return char + token from pre-rendered NodeBoxResult
2. **Arrowhead** — check g.arrows.has(encodeArrowPos(row, col)), return ▼ + token
3. **Edge** — check g.dirs[row]?.[col], return junctionChar() + token
4. **Empty** — return ' ' + null

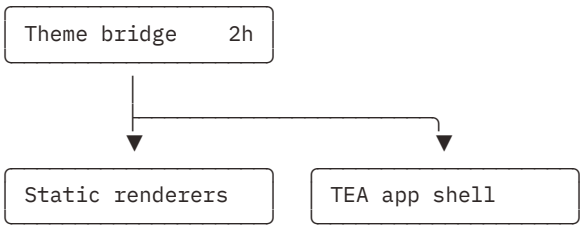
This avoids allocating O(rows × cols) grid arrays and eliminates separate write passes for edges, arrows, highlights, and nodes.

Graceful Degradation

Three rendering paths, following the bijou component pattern:

Interactive Mode (TTY)

Full Unicode box-drawing, ANSI colors via ctx.style.styled(), badges, highlights.



Pipe Mode

Indented adjacency list — machine-parseable, no ANSI, no box drawing:

```
Theme bridge -> Static renderers, TEA app shell
Static renderers -> RoadmapView, LineageView
TEA app shell -> RoadmapView, LineageView, Input system, graph.watch()
RoadmapView -> Cleanup
LineageView -> Cleanup
Input system -> Cleanup
graph.watch() -> Cleanup
```

Accessible Mode

Structured text with depth labels for screen readers:

```
Graph: 8 nodes, 10 edges

Layer 1:
  Theme bridge (2h) -> Static renderers, TEA app shell

Layer 2:
  Static renderers -> RoadmapView, LineageView
  TEA app shell -> RoadmapView, LineageView, Input system, graph.watch()

Layer 3:
  RoadmapView -> Cleanup
  LineageView -> Cleanup
  Input system -> Cleanup
  graph.watch() -> Cleanup

Layer 4:
  Cleanup (end)
```

ASCII Fallback

When Unicode is unavailable (TERM=dumb, NO_UNICODE=1), swap the junction table:

```
const JUNCTION_ASCII: Record<string, string> = {
  'D': '|', 'U': '|', 'DU': '|',
  'L': '-', 'R': '-', 'LR': '-',
  'DR': '+', 'DL': '+', 'RU': '+', 'LU': '+',
  'DRU': '+', 'DLU': '+', 'DLR': '+', 'LRU': '+', 'DLRU': '+',
};
// Arrowhead: 'v' instead of '▼'
// Node boxes: +--+ / |  | instead of ┌──┐ / │  │
```

Width Constraints & Responsive Behavior

Auto-sizing

```
const nodeWidth = options.nodeWidth ?? Math.max(
  ...nodes.map(n => n.label.length + (n.badge?.length ?? 0) + 4),  // │ label badge │
  16,  // minimum
);
const colStride = nodeWidth + 4;
const maxNodesPerLayer = Math.max(...layers.map(l => l.length));
const totalWidth = maxNodesPerLayer * colStride;
```

Terminal Overflow

When totalWidth > maxWidth:

1. **Shrink node width** — truncate labels with …
2. **Reduce gap** — minimum gap of 2 between nodes
3. **If still too wide** — switch to vertical (single-column) layout with indentation instead of side-by-side boxes

ANSI-Aware Width

All width calculations use **visible length** (strip ANSI escapes before measuring):

```
function visibleLength(str: string): number {
  return str.replace(/\x1b\[[0-9;]*m/g, '').length;
}
```

Apply colors only when writing content into the grid — never during layout math.

XYPH Integration: status --view deps

With the DagSource adapter, XYPH's quest graph is never duplicated — bijou reads directly from the quest store:

```
import { dag, dagSlice, type DagSource } from '@flyingrobots/bijou';

// Wrap the quest store as a DagSource — no DagNode[] copy
const questGraph: DagSource = {
  has: (id) => snapshot.quests.has(id),
  label: (id) => snapshot.quests.get(id)!.title.slice(0, 20),
  children: (id) => snapshot.quests.get(id)!.dependsOn ?? [],
  parents: (id) => snapshot.quests.get(id)!.dependents ?? [],
  badge: (id) => `${snapshot.quests.get(id)!.hours}h`,
  token: (id) => {
    const q = snapshot.quests.get(id)!;
    return q.status === 'DONE' ? theme.semantic.success
         : frontierSet.has(id) ? theme.semantic.primary
         : theme.semantic.muted;
  },
};

// Render a 3-hop neighborhood — never loads the full graph
const slice = dagSlice(questGraph, 'task:BJU-007', { depth: 3, direction: 'ancestors' });
console.log(dag(slice, {
  highlightPath: criticalResult.path,
  highlightToken: theme.semantic.error,
}));
```

Reactive DAG in TUI

With bijou-tui's TEA runtime + git-warp subscriptions:

```
const app: App<Model, Msg> = {
  init: () => [model, [watchGraph()]],

  update: (msg, model) => {
    if (msg.type === 'graph-changed') {
      return [{ ...model, snapshot: rebuildSnapshot(msg.diff) }, []];
    }
    return [model, []];
  },

  view: (model) => {
    return flex(
      { direction: 'column', width: model.cols, height: model.rows },
      { basis: 2, content: tabs([...]) },
      { flex: 1, content: (w, h) => viewport({
```

```
        width: w, height: h,
        content: dag(buildDagNodes(model.snapshot), {
          highlightPath: model.criticalPath,
          maxWidth: w,
        }),
        scrollY: model.scroll.y,
      })},
      { basis: 2, content: statusLine(model) },
    );
  },
};
```

Add a quest in one terminal → the DAG re-renders in the other with the new node slotted and sorted. That's the dream.

Implementation Plan

Phase 1: Core Layout Engine (~150 LoC)

- assignLayers() — topological BFS, longest-path
- orderColumns() — barycenter heuristic (1 top-down + 1 bottom-up pass)
- Grid coordinate mapping (layer × RS, column × CS)

Phase 2: Edge Router (~100 LoC)

- Direction accumulation (Set<Dir> per cell)
- Junction character lookup (15-entry table)
- Elbow routing (vertical exit → horizontal turn → vertical entry)
- Arrowhead placement

Phase 3: Renderer — Shader-based cellAt (~460 LoC)

- Node box rendering (label + badge + border) into PlacedNode[] with pre-segmented graphemes
- Spatial node index (Map<row, PlacedNode[]>) for O(1) amortized node lookup
- Highlight cell set (Set<number> using encodeArrowPos) for O(1) edge highlight lookup
- On-demand cellAt(row, col) query during serialization (priority: node box > arrowhead > edge > empty)
- Run-length grouping serializer: consecutive same-token characters grouped into single styled() calls
- Three output mode paths (interactive / pipe / accessible)
- ASCII fallback junction table

Phase 4: Fragment Support (~50 LoC)

- dagSlice() — BFS/DFS from focus node with depth limit

- Ghost node rendering for truncated boundaries (dashed borders)
- Edge count annotation ("… 3 ancestors")

Phase 5: Width Adaptation (~40 LoC)

- Auto-sizing from label lengths
- Terminal width overflow → label truncation → gap reduction → vertical fallback
- visibleLength() for ANSI-safe width math

Total: ~1014 LoC + tests

Tests follow bijou's createTestContext() pattern — one test per output mode, edge cases (empty, single node, diamond, wide fan-out, skip edges spanning multiple layers, cycles rejected with error).

Open Questions

1. **Horizontal layout (direction: 'right')** — useful for pipelines. Same algorithm, rotated 90°. Worth including in v1 or defer?
2. **Interactive DAG in bijou-tui** — cursor navigation over nodes, expand/collapse subtrees, click-to-focus. Separate dagExplorer() TEA component or extend dag() with a stateful wrapper?
3. **Maximum practical size** — at what node count does the ASCII layout become unreadable? Likely ~30 nodes in a standard 120-col terminal. Beyond that, should we auto-switch to a condensed format (e.g., collapsed layers)?
4. **Edge label support** — render edge labels on the horizontal segment? ("depends-on" is implied in XYPH but other users might want labeled edges.)
5. **Grouping / clusters** — render a box around a set of nodes (e.g., all tasks in a campaign)? Adds significant layout complexity.

426

Feature Request: Canvas Primitive & Box Width Override

docs/requests/xyph-title.md

From: XYPH TUI title screen **Date:** 2026-02-28 **bijou version:** v0.9.0

Context

XYPH's landing screen renders an animated ASCII spiral shader as a full-screen background, with the logo and status text inside a solid bordered box composited on top. With v0.9.0, composite() + box() eliminate most of the hand-rolled compositing code, but two gaps remain.

What v0.9.0 already solves

Problem	v0.9.0 solution
Row-by-row compositing loop	composite(bg, [overlay], { dim })
Manual box-drawing characters	box(content, { borderToken, padding })
Tracking visual widths alongside styled text	visibleLength()
ANSI-safe background slicing	sliceAnsi() via spliceLine() internals
Centering math	modal() pattern (or trivial manual calc)

Request 1: canvas() — Character-grid shader primitive

Problem

The spiral background is a (x, y, time) -> character function evaluated over a cols x rows grid. The tight loop is pure boilerplate:

```
const lines: string[] = [];
for (let y = 0; y < rows; y++) {
  let line = '';
```

```
  for (let x = 0; x < cols; x++) {
    line += shaderFn(x, y, cols, rows, time);
  }
  lines.push(line);
}
return lines.join('\n');
```

Every app that wants a procedural background (rain, static, plasma, matrix, starfield) re-implements this loop. The pattern is general enough to live in bijou.

Proposed API

```
// bijou-tui or bijou core

type ShaderFn = (
  x: number,
  y: number,
  cols: number,
  rows: number,
  time: number,
) => string; // single character

function canvas(
  cols: number,
  rows: number,
  shader: ShaderFn,
  time?: number,
): string;
```

Returns: A single string of rows newline-separated lines, each cols characters wide.

Behavior: - Calls shader(x, y, cols, rows, time ?? 0) for every cell - If the shader returns more than one character, take only the first - If it returns an empty string, substitute a space - time defaults to 0 for static patterns

Why bijou, not app code

1. **Reusable** — any TUI splash screen, screensaver, or background effect benefits from the same loop.
2. **Optimizable** — bijou could later add row-level string building optimizations, canvas caching, or dirty-rect diffing without changing the shader contract.
3. **Composable** — canvas() output feeds directly into composite() as a background string. The two together form a complete layered rendering pipeline.

Acceptance criteria

```
canvas()
  √ calls shader(x, y, cols, rows, time) for every cell in order
  √ produces exactly `rows` lines separated by newlines
  √ each line is exactly `cols` characters wide
  √ truncates multi-char shader return to first character
  √ substitutes space for empty shader return
  √ time defaults to 0 when omitted
  √ works with cols=0 or rows=0 (returns empty string)
```

```
  pipe mode
    √ returns empty string (no visual output in pipes)
```

Request 2: box() width override

Problem

modal() accepts an optional width to force a fixed box size, but box() doesn't. On the XYPH title screen, the box content changes between states:

State	Content inside box
Loading	Logo + copyright (2 lines below logo)
Error	Logo + copyright + error + "press any key"
Ready	Logo + copyright + stats + "press any key"

The widest content line (the logo) is the same across all states, so the box width doesn't jump. But the box *height* changes — it's taller in the ready state than the loading state. If the widest line ever differed between states (e.g., a long error message), the box would visually jump widths between frames.

A width option on box() would let consumers lock the box to a fixed size for visual stability.

Proposed API

```
export interface BoxOptions {
  borderToken?: TokenValue;
  padding?: { top?: number; bottom?: number; left?: number; right?: number };
  width?: number;  // <-- new: total outer width (border + padding + content)
  ctx?: BijouContext;
}
```

Behavior: - When width is set, the box outer width is fixed to that value - Content lines shorter than the interior are right-padded with spaces - Content lines longer than the interior are clipped via clipToWidth() - When w idth is not set, behavior is unchanged (auto-size to content)

Acceptance criteria

```
box() with width override
  √ outer width matches specified width exactly
  √ short content lines are right-padded to fill interior
  √ long content lines are clipped to interior width
  √ padding is subtracted from width to compute interior
  √ width < minimum (border + padding) produces zero-width interior
  √ visibleLength of every output line equals specified width
```

Non-request: Transparent compositing

Initially considered requesting a transparent option for composite() where space characters in the overlay would reveal the background. After review, co mposite()'s current opaque behavior is correct for the box use case — the solid interior is the whole point. Transparent compositing is a different primitive (sprite rendering) and would complicate the common case. Noting it here as a future thought, not a request.

Usage sketch (v0.9.0 + these two features)

```
import { box } from '@flyingrobots/bijou';
import { composite, canvas } from '@flyingrobots/bijou-tui';
import { spiralShader } from './shaders/spiral.js';

export function landingView(model: DashboardModel): string {
  // 1. Render full-screen spiral background
  const bg = canvas(model.cols, model.rows, spiralShader, Date.now());
  const styledBg = styled(muted, bg);

  // 2. Build foreground content
  const content = [gradientLogo, '', copyright, '', statusText]
    .filter(Boolean).join('\n');

  // 3. Wrap in a fixed-width box
  const panel = box(content, {
    borderToken: border,
    padding: { top: 1, bottom: 1, left: 3, right: 3 },
    width: logoWidth + 8, // lock to logo width + padding + border
  });

  // 4. Center and composite
  const panelLines = panel.split('\n');
  const row = Math.floor((model.rows - panelLines.length) / 2);
  const col = Math.floor((model.cols - visibleLength(panelLines[0])) / 2);

  let result = composite(styledBg, [{ content: panel, row, col }], { dim: true });

  // 5. Progress bar at bottom
  if (model.loading) {
    const bar = progressBar(model.loadingProgress, { width: model.cols });
    result = composite(result, [{ content: bar, row: model.rows - 1, col: 0 }]);
  }

  return result;
}
```

~25 lines vs. the current ~130 lines of manual compositing.

VII

Rendered components and provenance

Connecting written contracts to surfaces

The component atlas makes a portion of the system's visible output available beside its written contracts. Its 42 specimens are generated from actual DOGFOOD story surfaces, with a fixed time, profile, viewport and variant recorded for each capture. The SVG artwork retains the cells' glyphs, positions, colours and supported modifiers; matching vector PDFs are used in the book.

These choices make a specimen traceable. A reader can identify which story and variant produced it and compare the illustration with the contract or implementation that owns the surface. The artwork retains the application's theme even though the surrounding book uses its own palette.

Read a specimen with its context

A specimen shows one rendered state. Interaction, resizing, other variants and alternative output modes require their own evidence. Use the story identity and the related chapters to follow those questions into the application. Whole surfaces such as Theme Lab and the BlockLab workbench combine more behaviour than a single component illustration can convey.

The final source inventory connects the book's chapters to their original paths and SHA-256 values. Chapter numbers follow this edition's reading order; source paths and stable record labels preserve identity across editorial rearrangement. The companion manifest maps those records to their LaTeX chapter files.

The atlas and inventory serve complementary purposes: one makes rendered output visible, and the other makes the written source traceable. Together they provide a concrete starting point for checking an explanation against the material from which it was built.

The component cell atlas

These are deterministic vector projections of actual DOGFOOD component stories at time zero, using the shipped BIJOU_DARK theme and interactive profile. Every glyph, position, colour and modifier comes from the rendered Surface. The terminal font is explicitly IBM Plex Mono with DejaVu Sans symbols. They are not photographs of an emulator. No component has been redrawn by eye.

alert()

Persistent in-flow status block for success, warning, error, and informational state that should remain part of the page instead of floating above it.



Story: alert; variant: success; 40 × 3 cells.

badge()

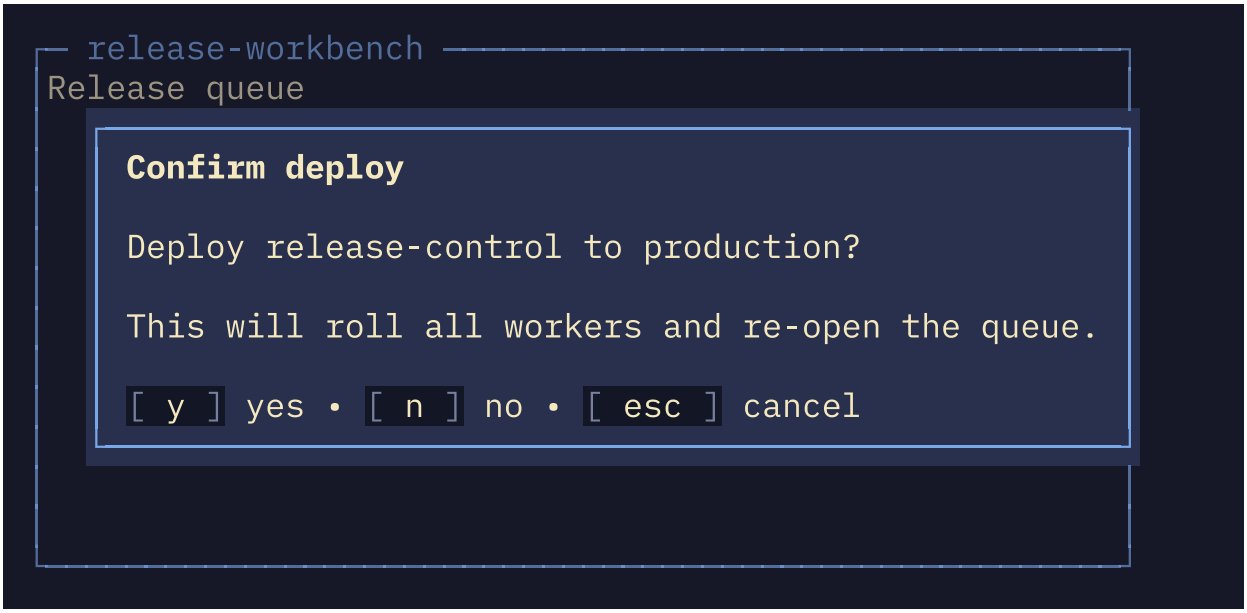
Compact inline status label for states that belong beside another object instead of becoming their own block.



Story: badge; variant: service-health; 35 × 5 cells.

modal()

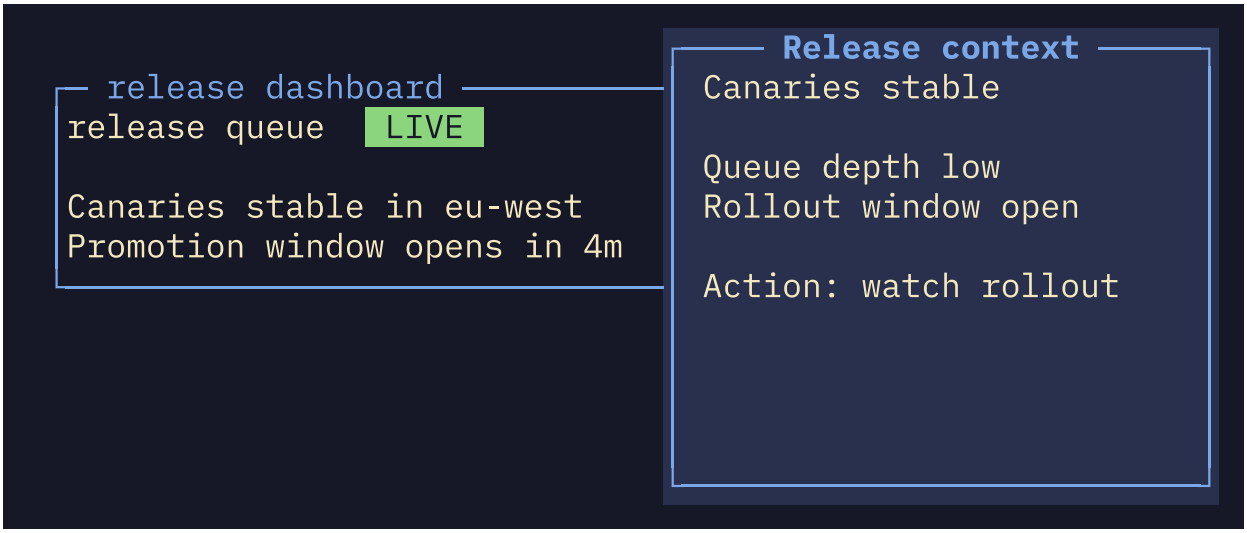
Centered blocking dialog for decisions that should temporarily own attention and input inside a TUI surface.



Story: modal; variant: confirm; 60 × 14 cells.

drawer()

Anchored supplemental panel for sidecar context that should remain related to the current surface without blocking the whole task.



Story: drawer; variant: supplemental-right; 60 × 12 cells.

tooltip()

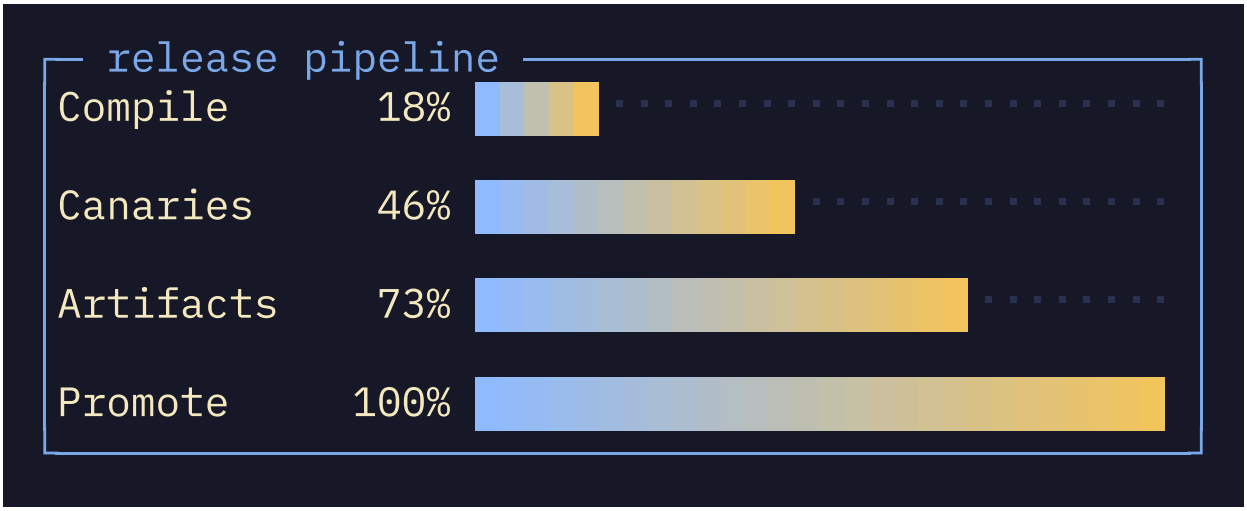
Short local explanation positioned near the thing it describes, for cases where the meaning is helpful but not part of the document flow.



Story: tooltip; variant: local-explanation; 60 × 10 cells.

progressBar()

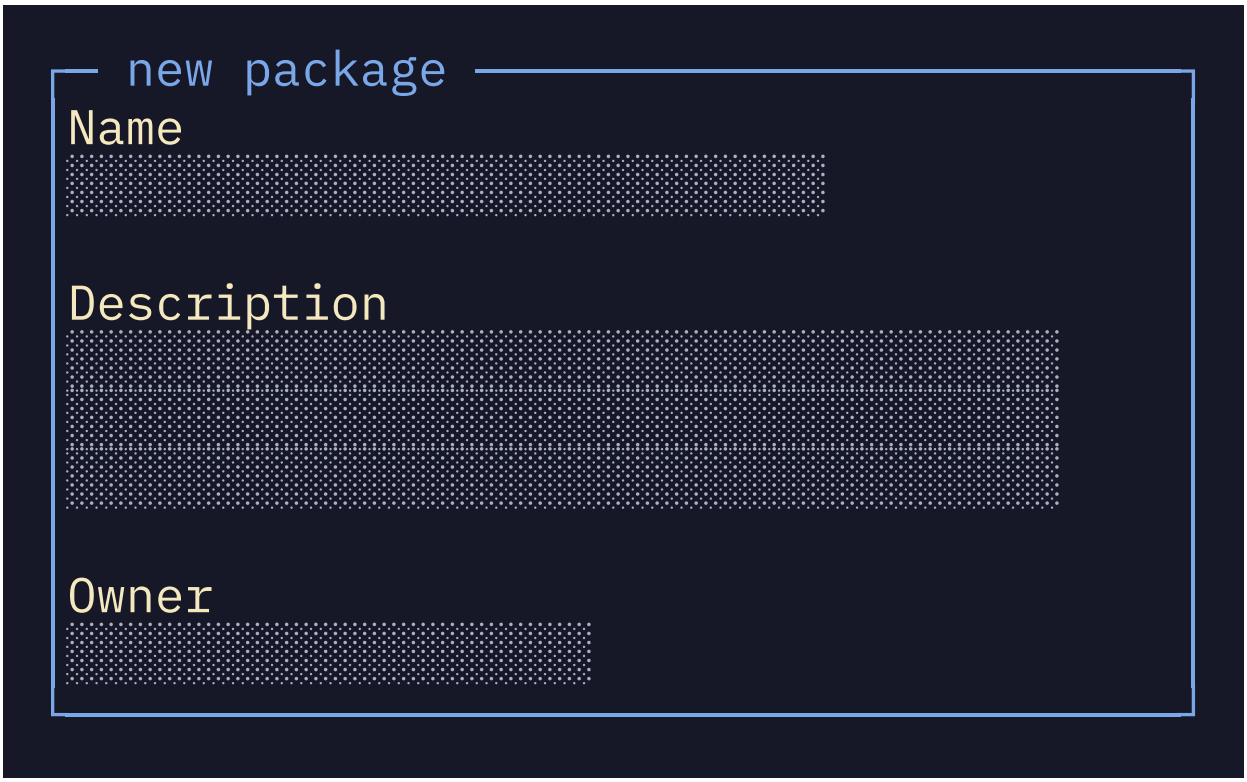
Honest determinate progress indicator for work that can report a real completion percentage, with related support for indeterminate activity cues.



Story: progress-bar; variant: release-milestones; 48 × 9 cells.

skeleton()

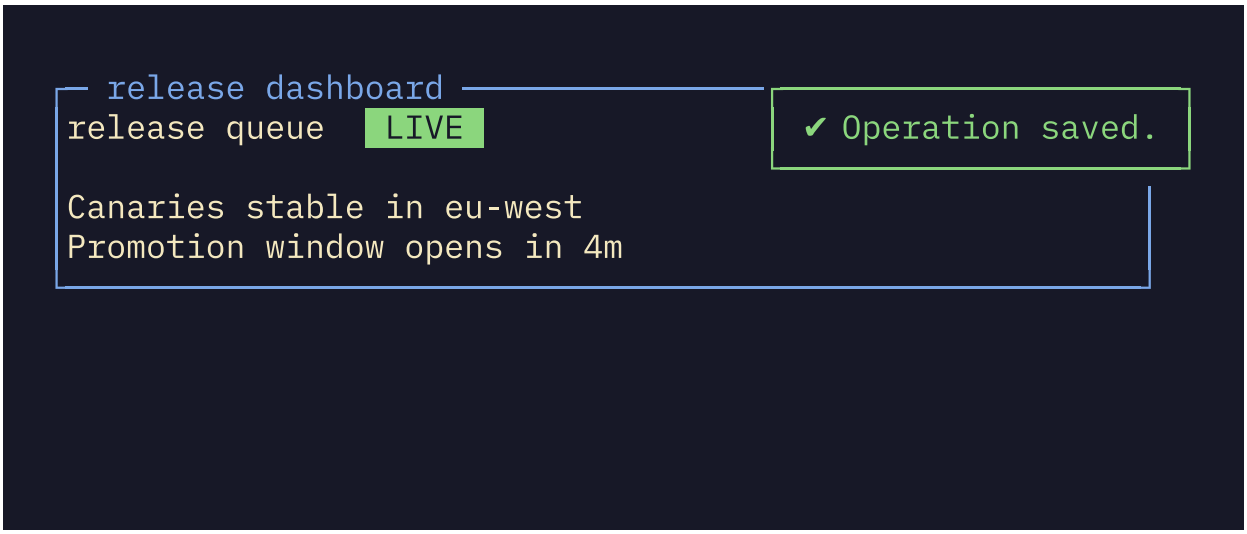
Shape-preserving loading placeholder for short-lived uncertainty when you know the information layout but not the data yet.



Story: skeleton; variant: form-shell; 40 × 12 cells.

toast()

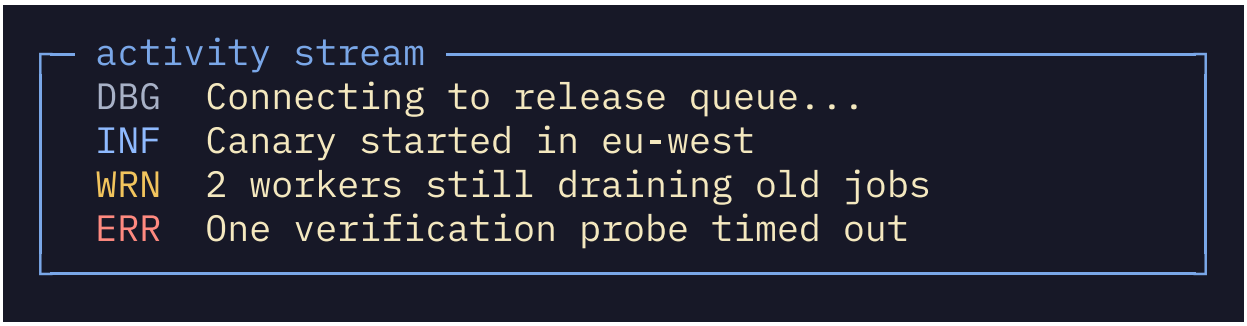
Low-level transient overlay primitive for one directly composed, anchored notification when the full notification system would be too heavy.



Story: toast; variant: saved-top-right; 60 × 12 cells.

log()

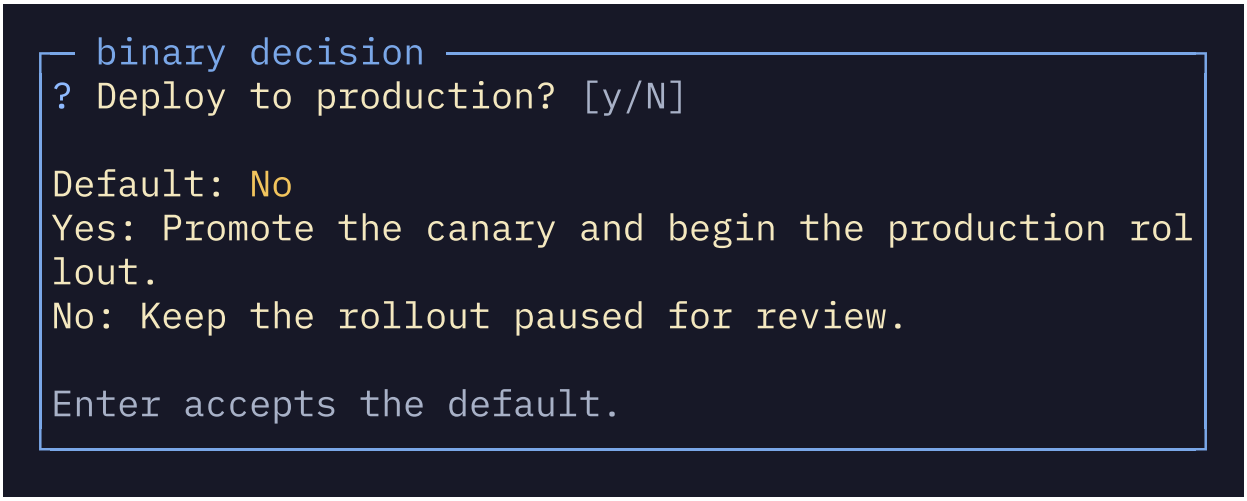
Chronological activity-stream primitive for accumulating operational events with explicit severity and optional timestamps.



Story: log; variant: release-events; 54 × 6 cells.

confirm()

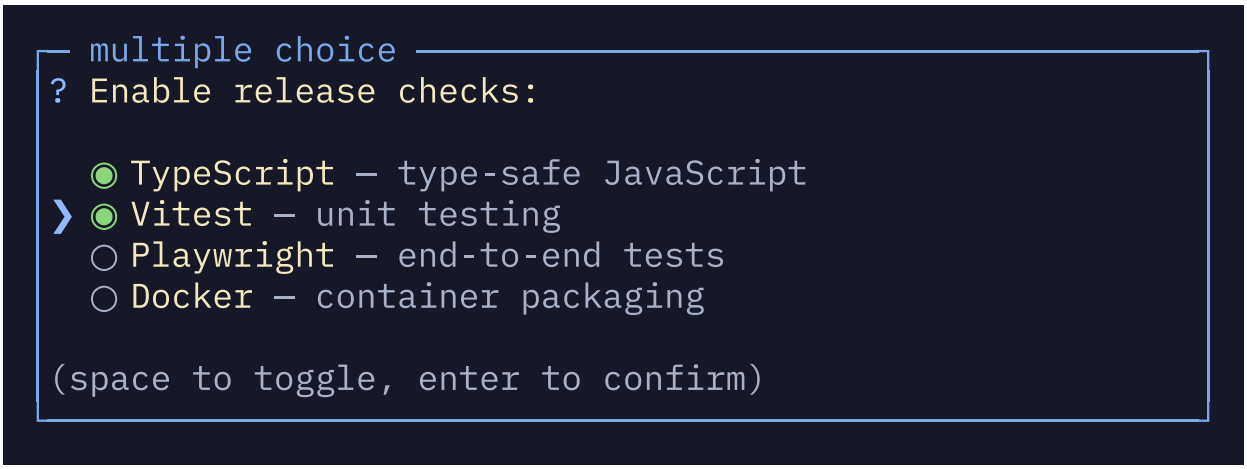
Binary decision prompt for real yes/no choices where the consequence of accepting or declining should stay explicit.



Story: confirm; variant: deploy-gate; 54 × 10 cells.

multiselect()

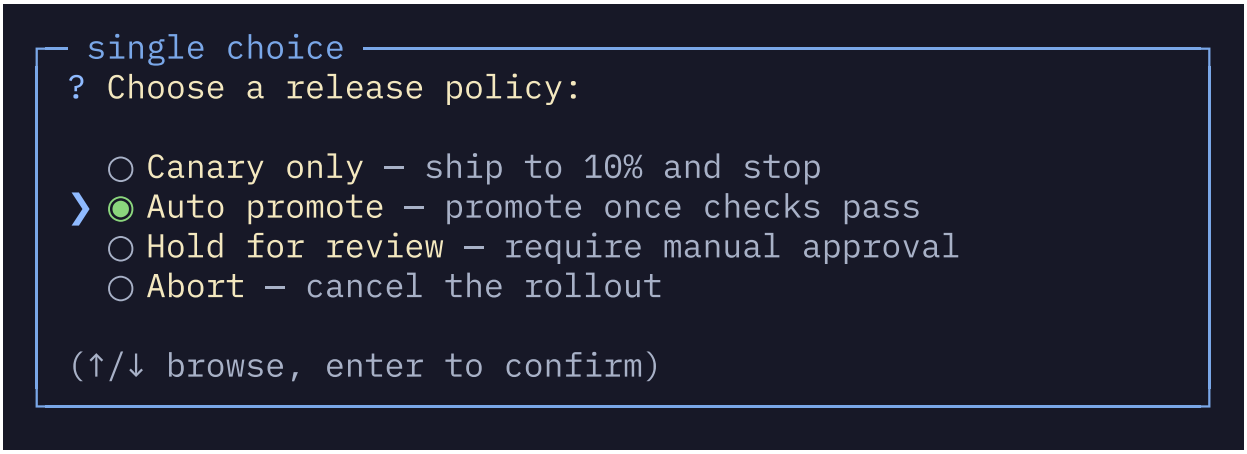
Set-selection prompt for building a durable group of choices with keyboard toggling and truthful numbered fallbacks.



Story: multiselect; variant: release-stack; 58 × 10 cells.

select() / filter()

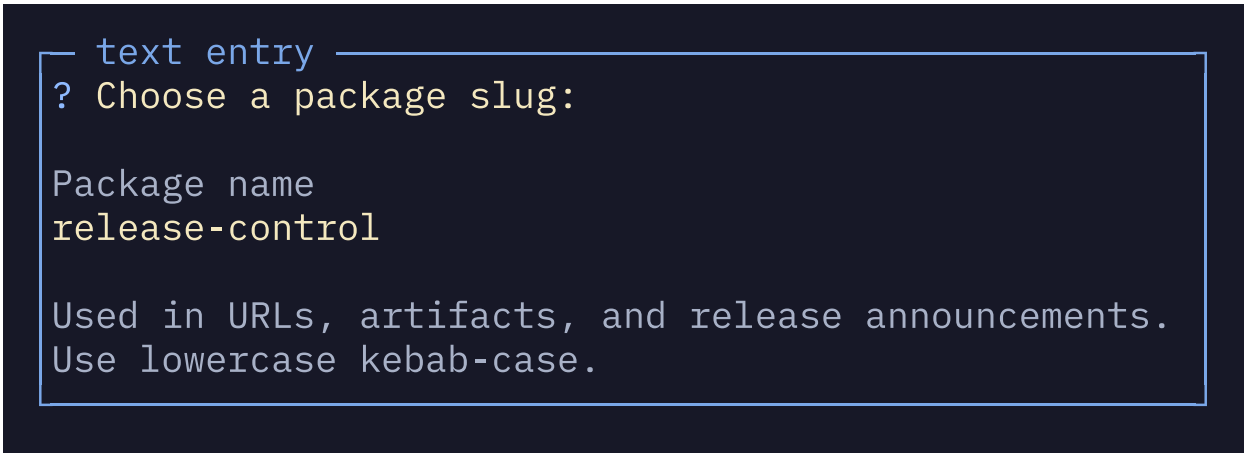
Single-choice prompt family for choosing one lasting value, either from a visible list or from a searchable narrowed list.



Story: select; variant: release-policy; 60 × 10 cells.

input() / textarea()

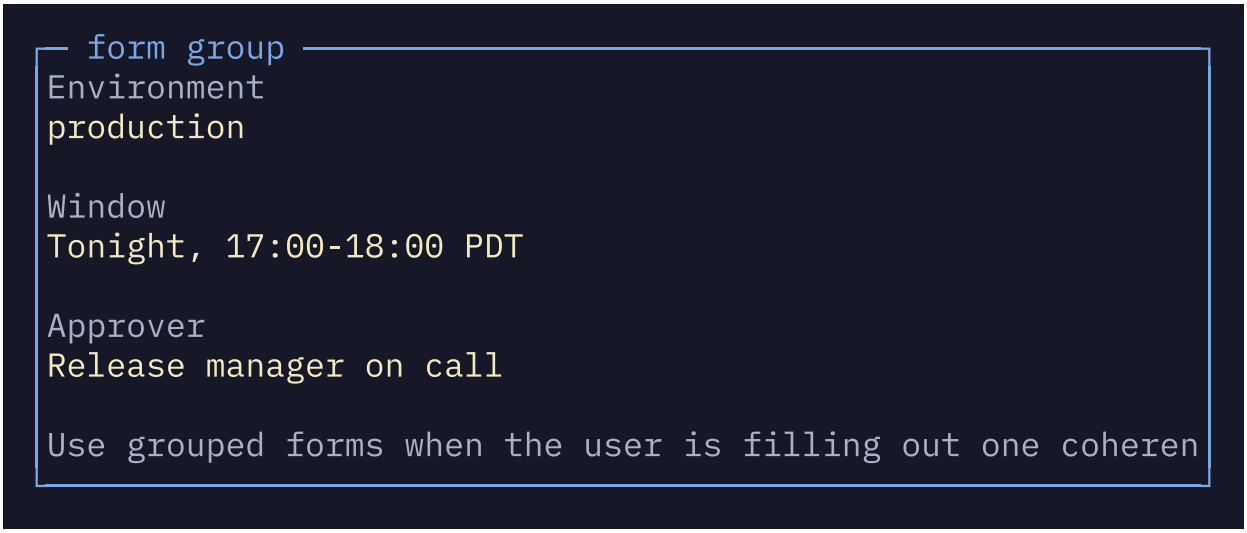
Text-entry family for collecting freeform user input, from short single-line identifiers to longer multiline notes that still need honest lowering.



Story: text-entry; variant: service-name; 54 × 9 cells.

group() / wizard()

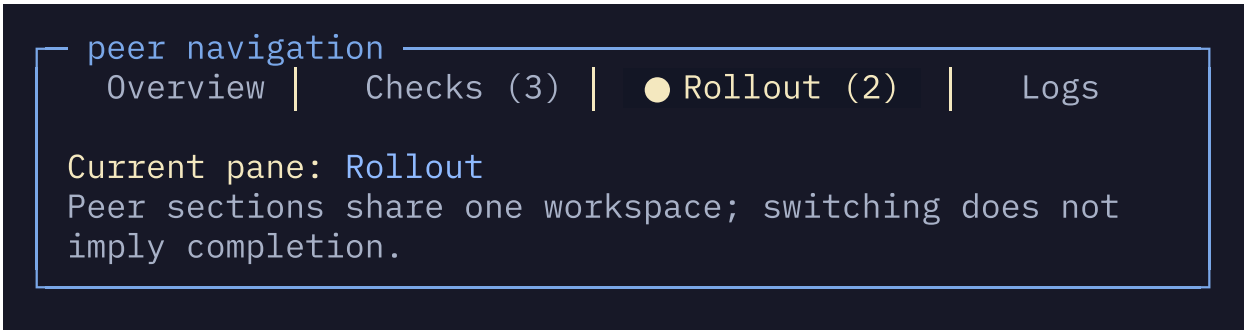
Grouped and staged form family for related inputs that belong together, whether they happen in one focused section or across explicit steps.



Story: group-wizard; variant: deploy-group; 60 × 12 cells.

tabs()

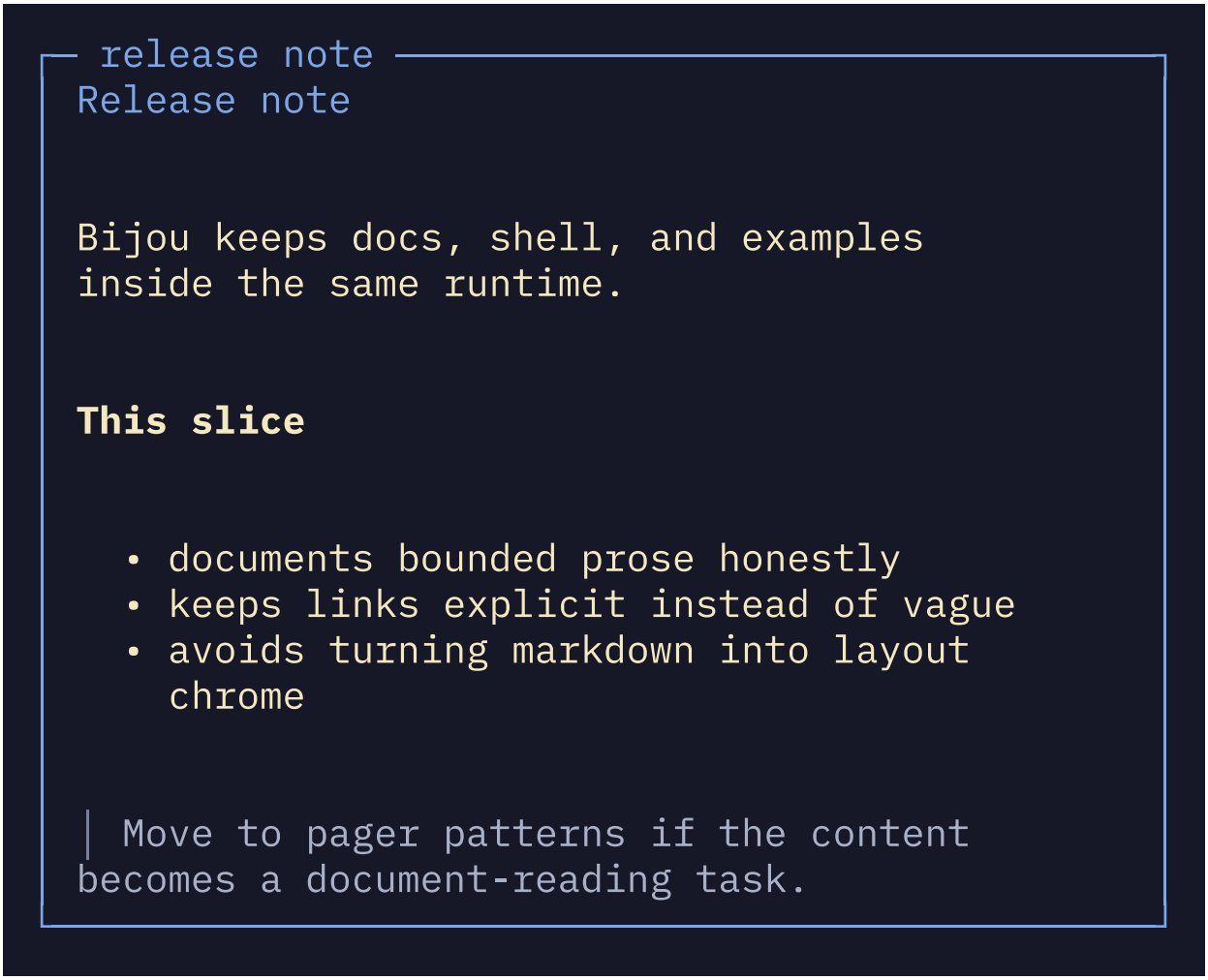
Peer-navigation strip for switching between sibling sections that share one workspace and one conceptual level.



Story: tabs; variant: release-workbench; 60 × 7 cells.

markdown()

Bounded structured prose renderer for help, release notes, and reference text that should stay honest across rich, pipe, and accessible modes.



Story: markdown; variant: release-note; 50 × 20 cells.

hyperlink()

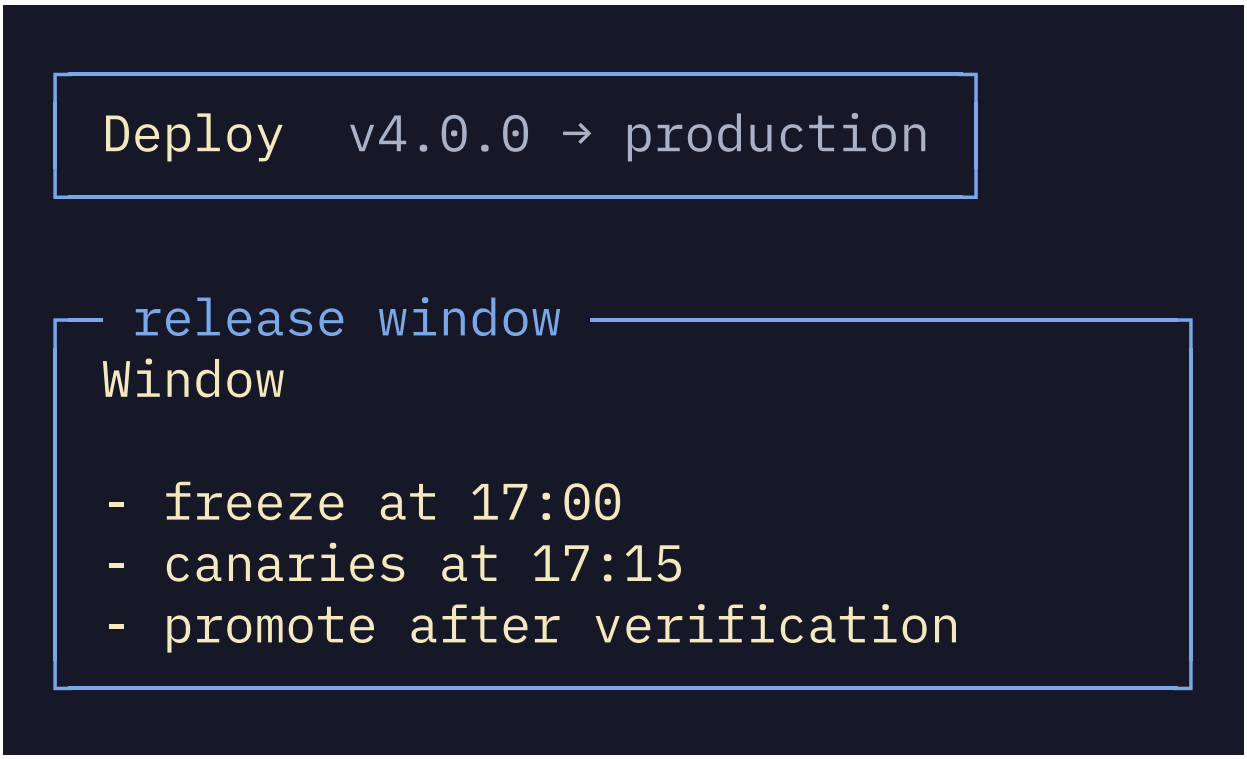
Explicit destination primitive for links that should remain meaningful whether or not the terminal supports clickable OSC 8 output.



Story: hyperlink; variant: explicit-destinations; 58 × 5 cells.

box()

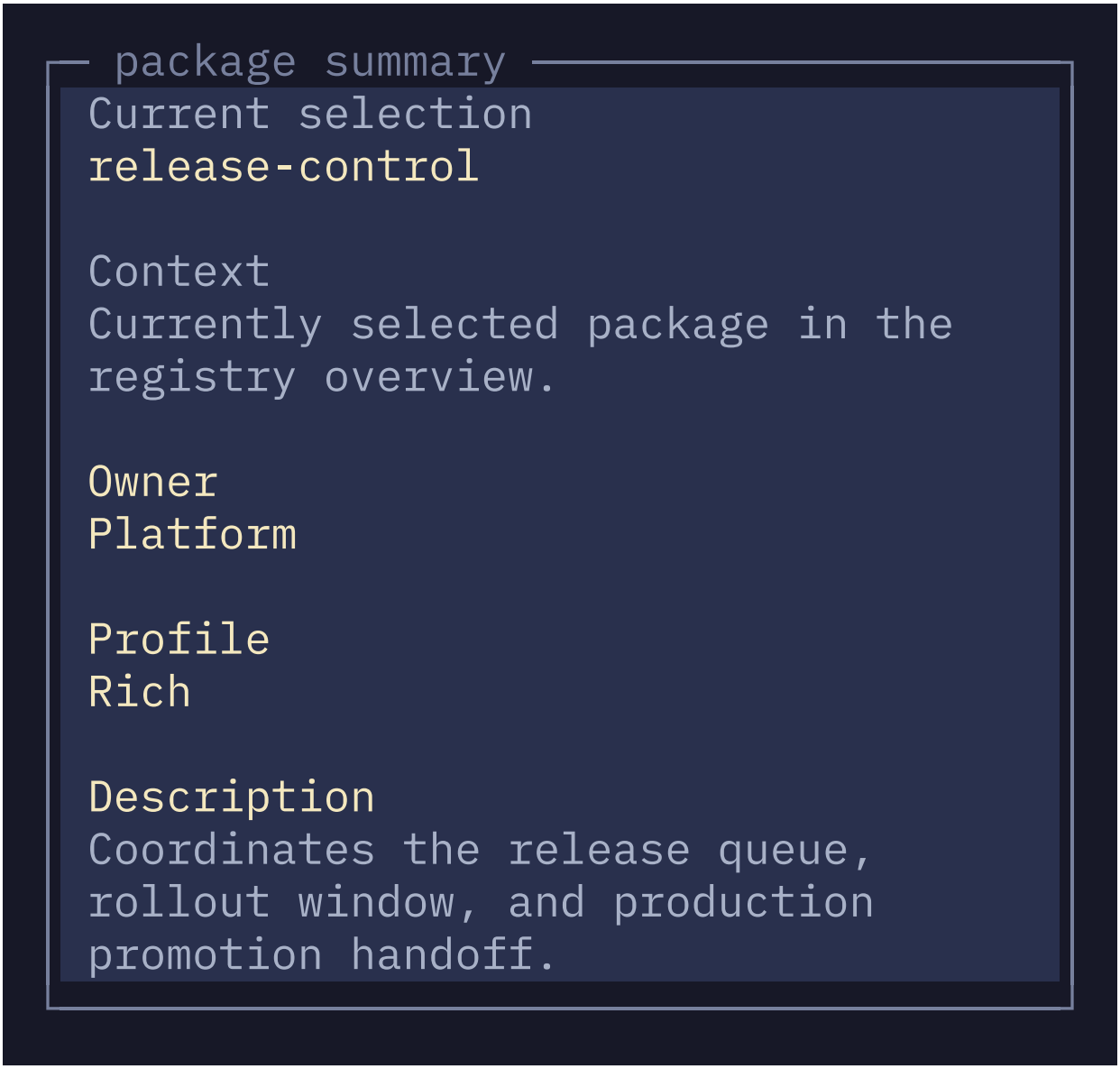
Canonical containment primitive for grouped content and compact titled panels, with ‘headerBox()‘ and surface companions for richer layout composition.



Story: box; variant: header-detail; 38 × 11 cells.

inspector()

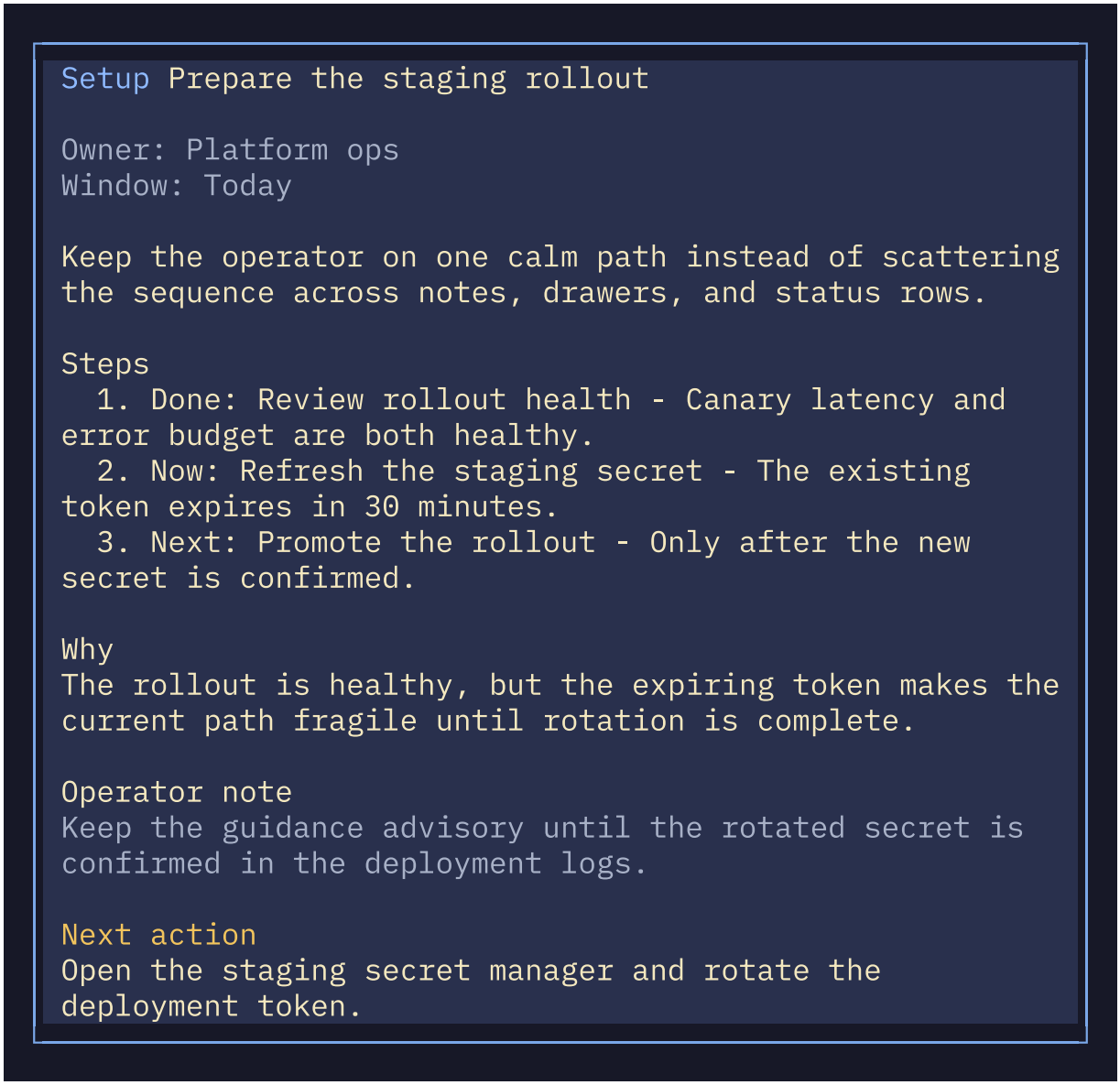
Canonical side-panel summary surface for the currently selected thing, keeping one obvious active value and calmer supporting sections nearby.



Story: inspector; variant: package-summary; 40 × 19 cells.

guidedFlow()

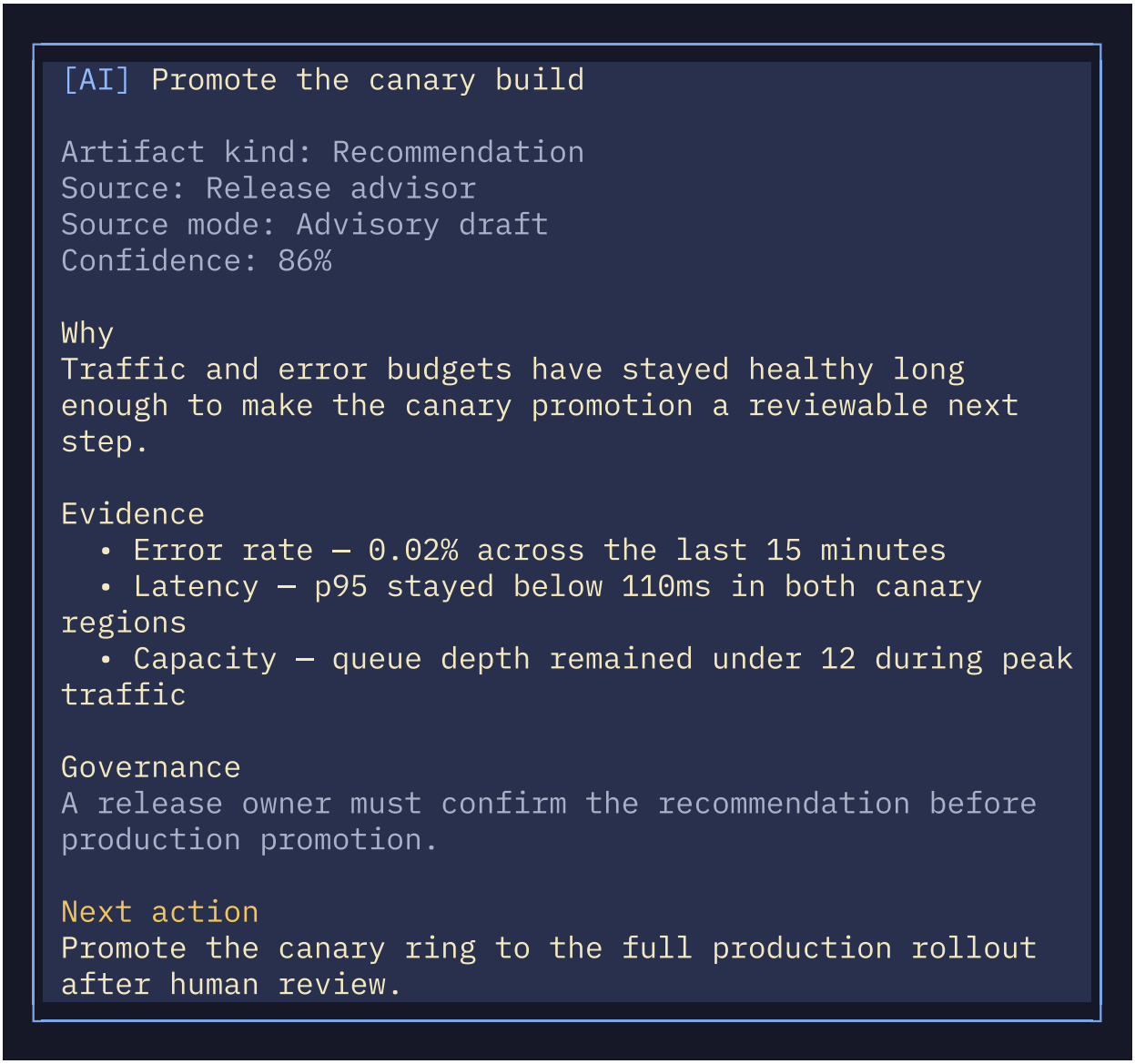
Reusable block for calmer multi-step assistance surfaces with explicit sections and one obvious next action.



Story: guided-flow; variant: staging-rollout; 60 × 29 cells.

explainability()

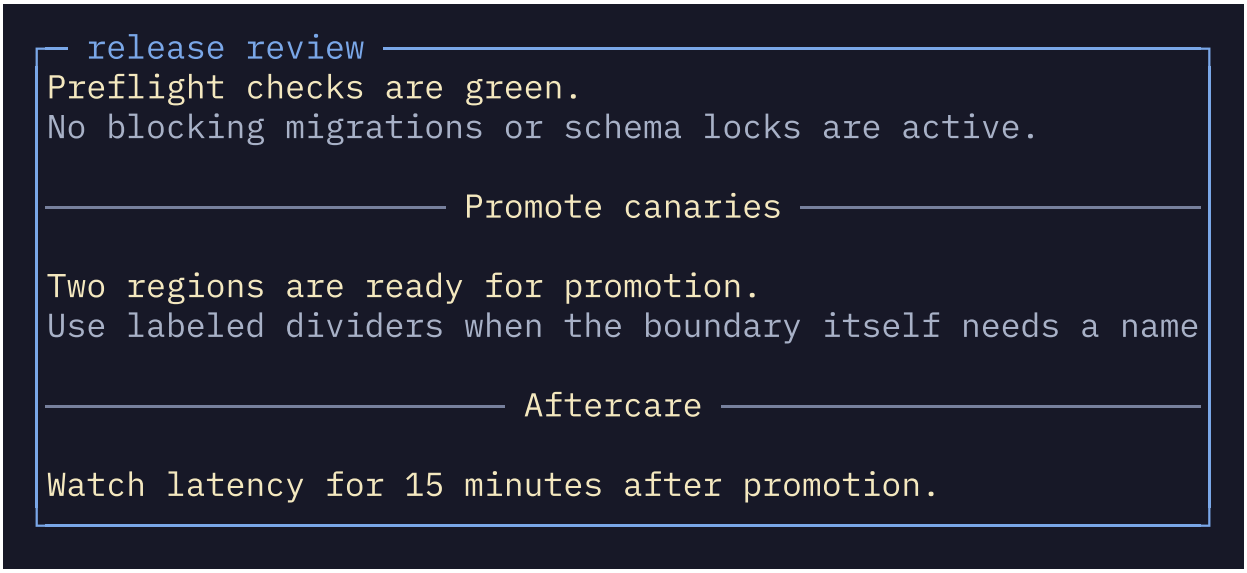
Calm guided recommendation surface for AI-mediated or machine-assisted output that must make provenance, evidence, and next action explicit.



Story: explainability; variant: rollout-recommendation; 60 × 28 cells.

separator() / separatorSurface()

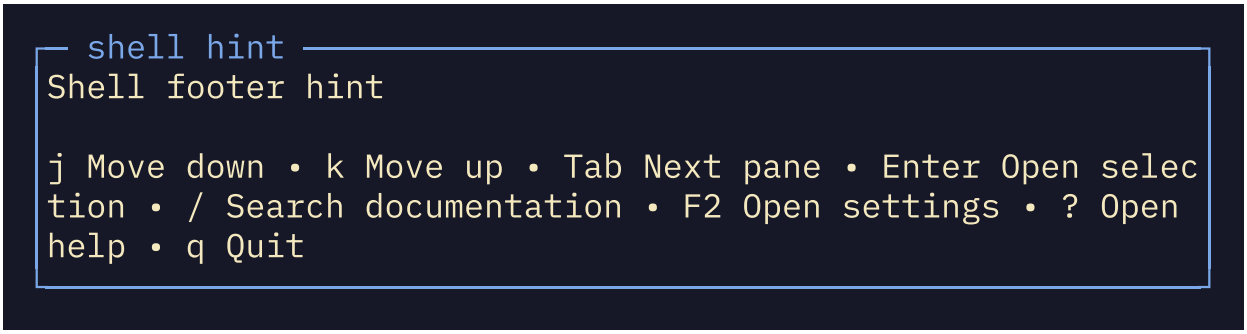
Section-boundary primitive for marking real transitions without promoting every boundary into a boxed panel.



Story: separator; variant: labeled-breaks; 60 × 13 cells.

helpView() / helpShortSurface()

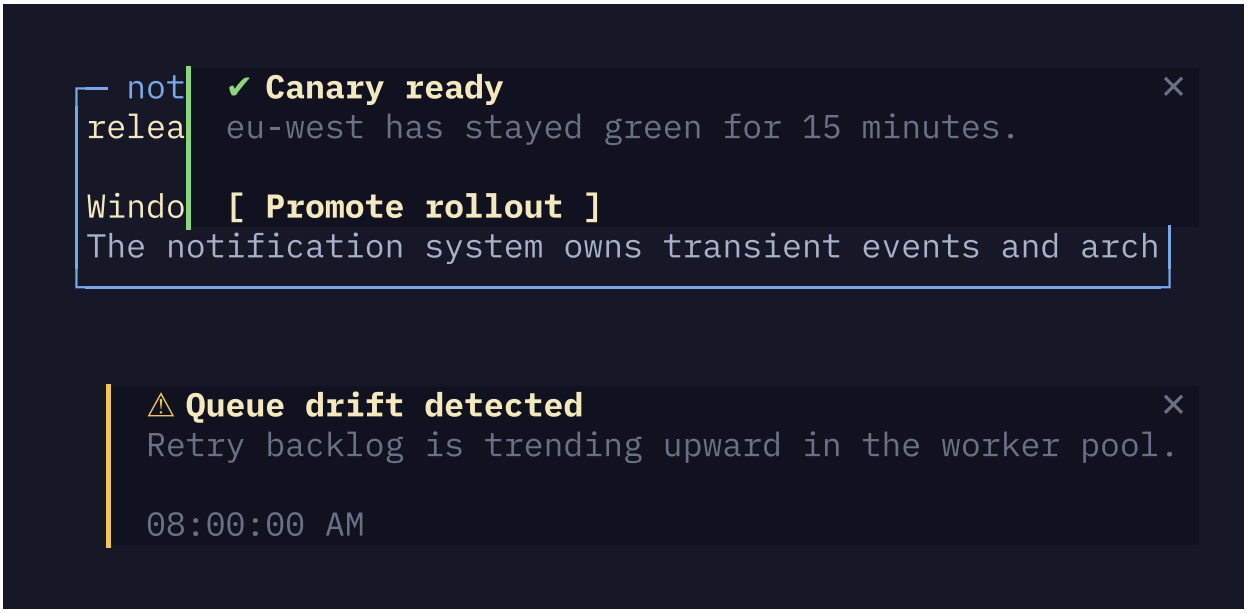
Grouped keyboard reference plus compact shell hint surfaces for keyboard-owned apps that need shortcut discovery without turning every footer into prose.



Story: help-view; variant: shell-hint; 60 × 7 cells.

renderNotificationStack() / renderNotificationHistorySurface() / createFramedApp()

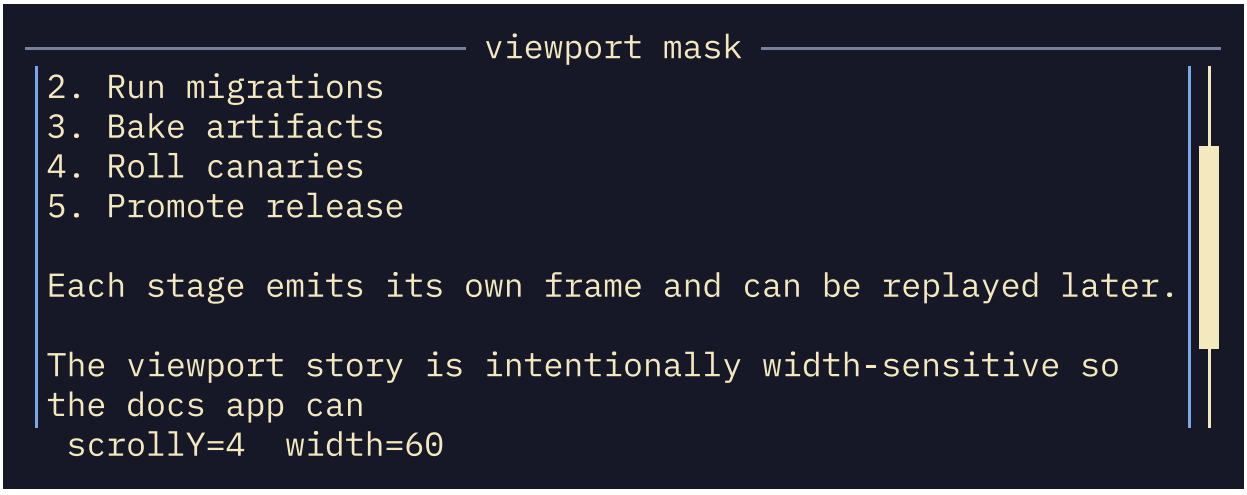
Shell-owned transient messaging system with stacked live notifications, explicit placement, actions, archived review history, and framed runtime routing.



Story: notification-system; variant: live-stack; 60 × 14 cells.

viewportSurface() / pagerSurface() / focusAreaSurface()

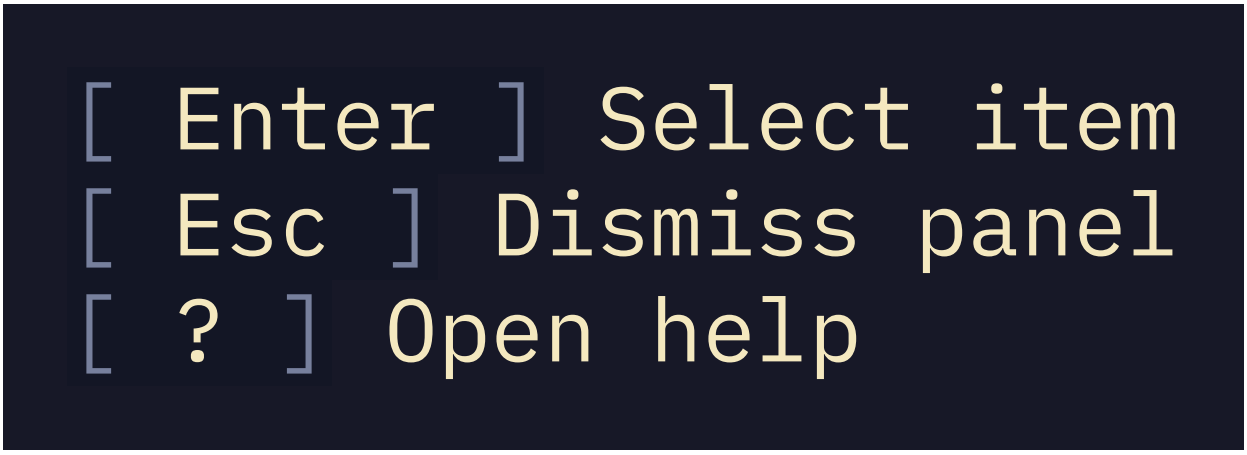
Masking and scrollable-pane family for bounded overflow content, long linear readers, and focused inspection panes without flattening structured surfaces first.



Story: viewport-surface; variant: document; 60 × 11 cells.

kbd()

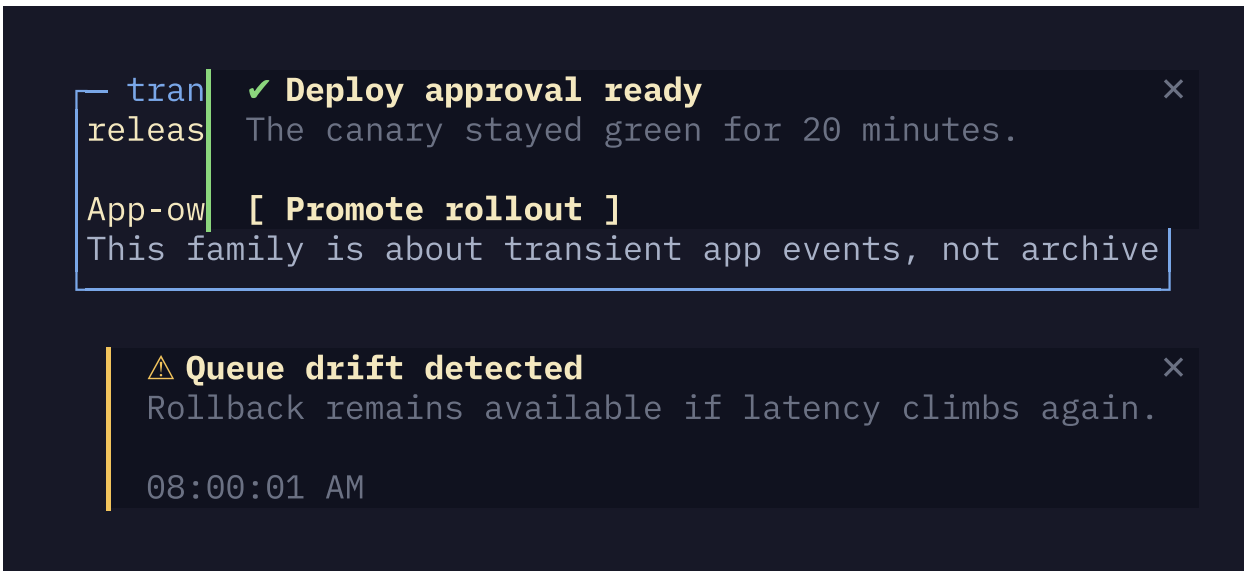
Inline shortcut cue for local actions that should stay adjacent to the thing they affect.



Story: kbd; variant: local-actions; 21 × 3 cells.

pushNotification() / renderNotificationStack()

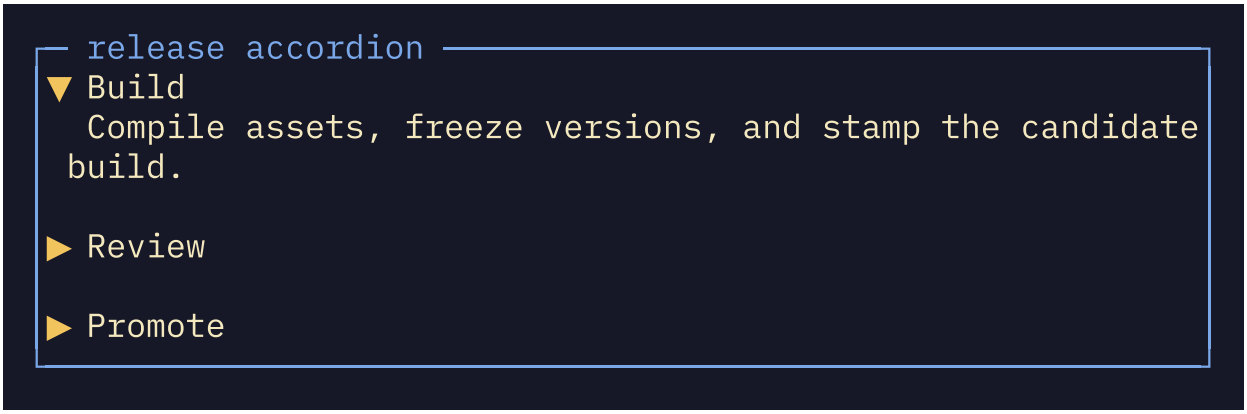
App-owned transient notification family for live stacked events, tones, actions, and placements before archived review or broader shell routing becomes the main concern.



Story: transient-app-notifications; variant: actionable-live; 60 × 13 cells.

accordion() / interactiveAccordion()

Progressive-disclosure family for scanning summaries first and revealing detail only when the user chooses to open a section.



Story: progressive-disclosure; variant: summary-first; 60 × 9 cells.

breadcrumb() / stepper() / paginator()

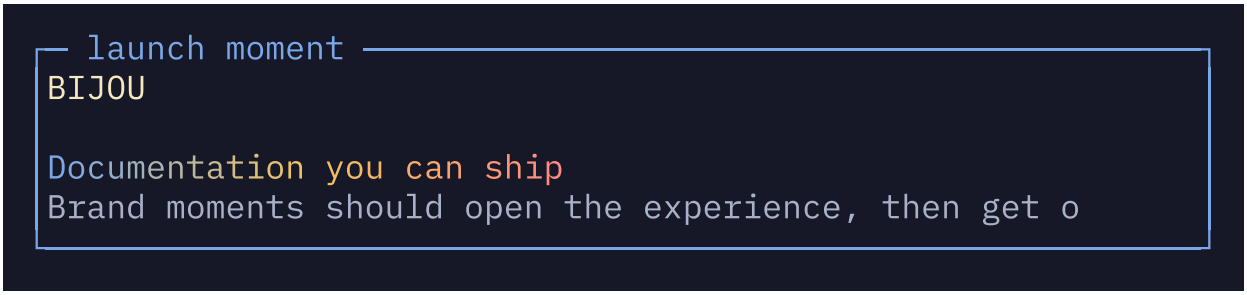
Wayfinding and progress family for showing where the user is, what stage they are in, or how far through a sequence they have moved.



Story: path-and-progress; variant: wayfinding; 60 × 5 cells.

loadRandomLogo() / gradientText()

Expressive branding family for rare logo and gradient emphasis moments that add atmosphere without becoming routine application chrome.



Story: expressive-branding; variant: launch-moment; 60 × 6 cells.

renderByMode()

Authoring seam for app-defined primitives that keep one semantic meaning while lowering honestly across interactive, pipe, and accessible modes.



Story: mode-aware-custom-primitives; variant: deployment-spark; 56 × 4 cells.

table() / navigableTableSurface()

Dense comparison family for row-and-column inspection, responsive table variants, explicit pipe serializations, and keyboard-owned row focus.

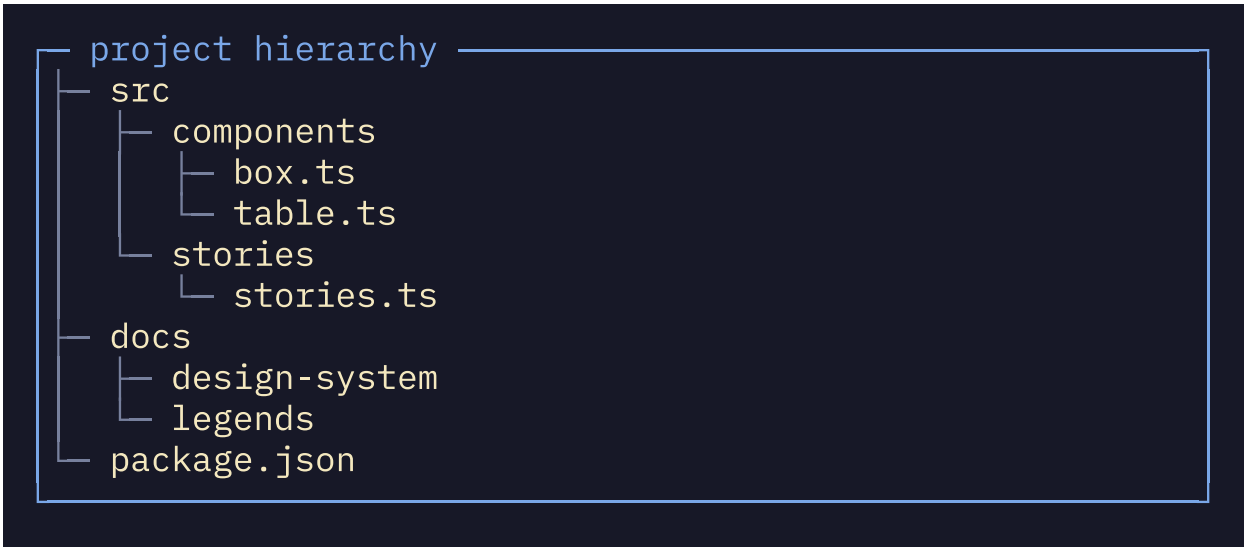
variant: box

Component	Behavior	Owner
table()	Fits terminal width and wraps cells at word boundaries.	core
variants	Switches box, ruled, plain, Markdown, definition, and expanded output.	API
pipeFormat	Exports rows to TSV, CSV, Markdown, or ASCII grid.	pipe
navigableTableSurface()	Adds row focus when keyboard inspection matters.	tui

Story: dense-comparison; variant: box; 60 × 15 cells.

tree() / filePickerSurface()

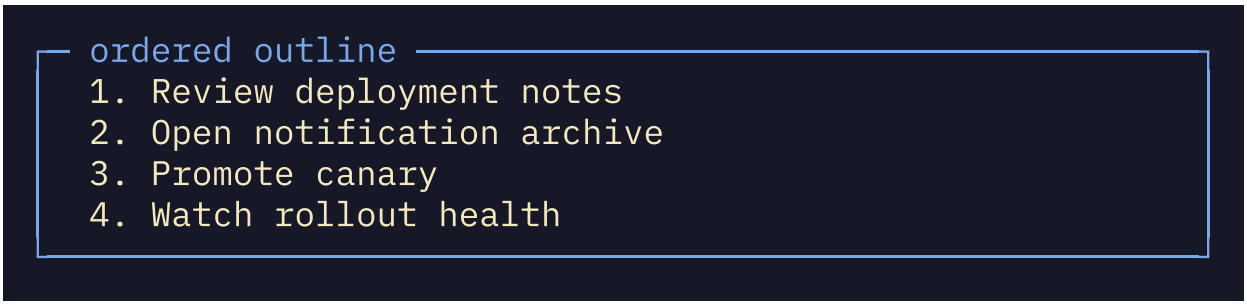
Hierarchy family for parent-child structure and filesystem-style browsing where nesting, path context, and directory boundaries matter more than tabular comparison.



Story: hierarchy; variant: project-tree; 58 × 12 cells.

enumeratedList() / browsableListSurface()

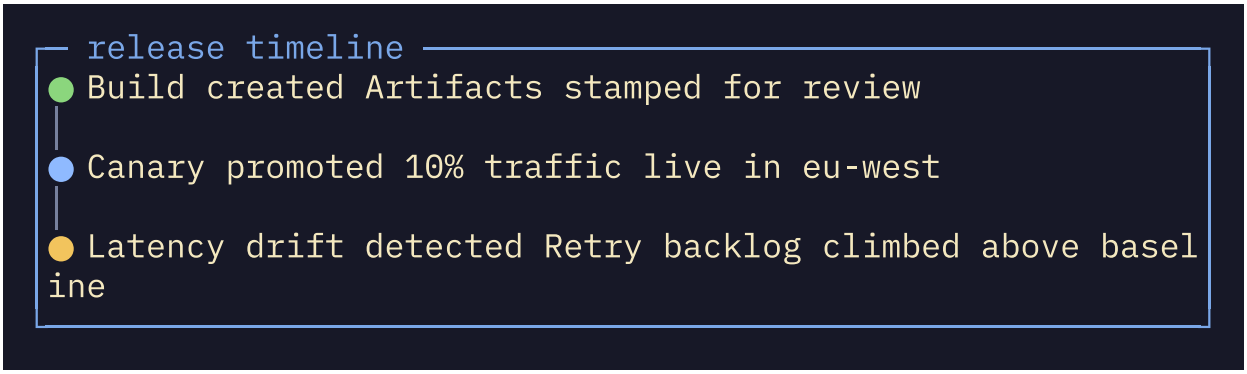
Exploration-list family for one-dimensional scanning where item order and focused inspection matter more than columns or hierarchy.



Story: lists-for-exploration; variant: ordered-outline; 58 × 6 cells.

timeline() / dag()

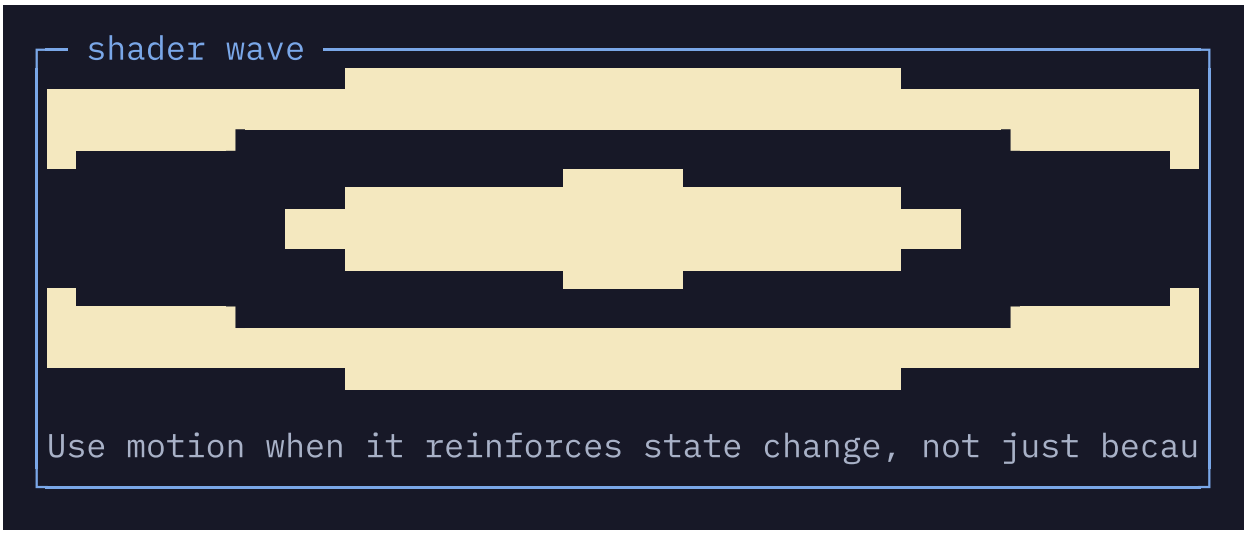
Temporal and dependency family for explaining what happened next or what depends on what when order or causality is the real structure.



Story: temporal-or-dependency-views; variant: release-timeline; 60 × 8 cells.

canvas()

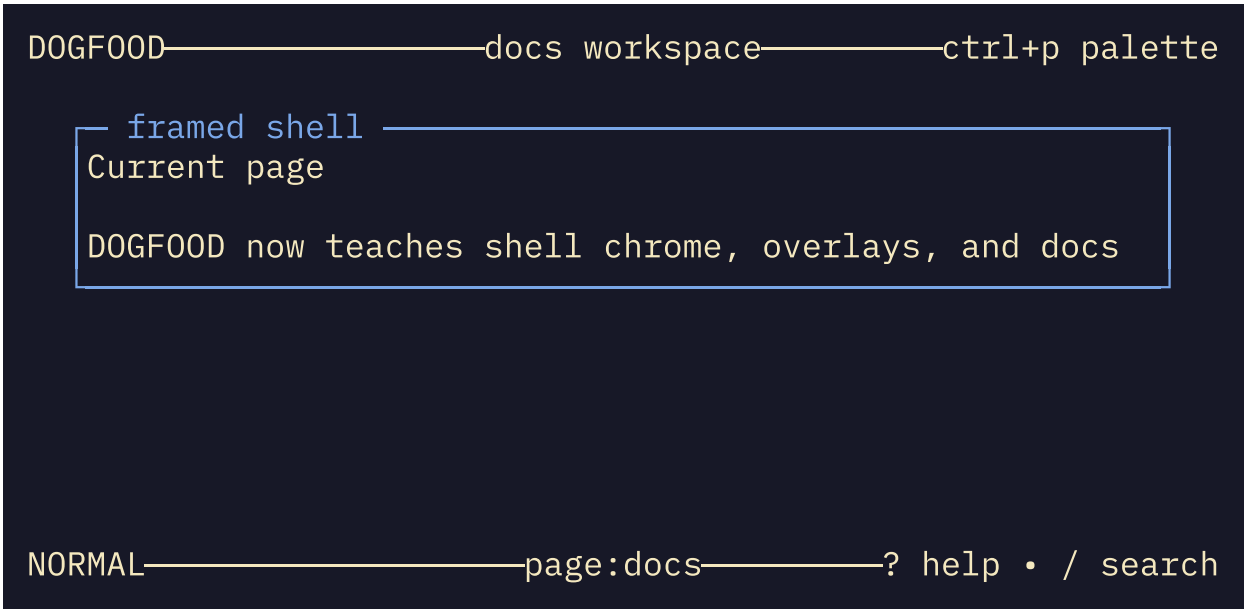
Motion and shader family for deliberate visual emphasis, transitions, and animated atmosphere when the effect reinforces state change or product voice instead of competing with the task.



Story: motion-and-shader-effects; variant: shader-wave; 60 × 12 cells.

createFramedApp() / statusBarSurface() / commandPaletteSurface()

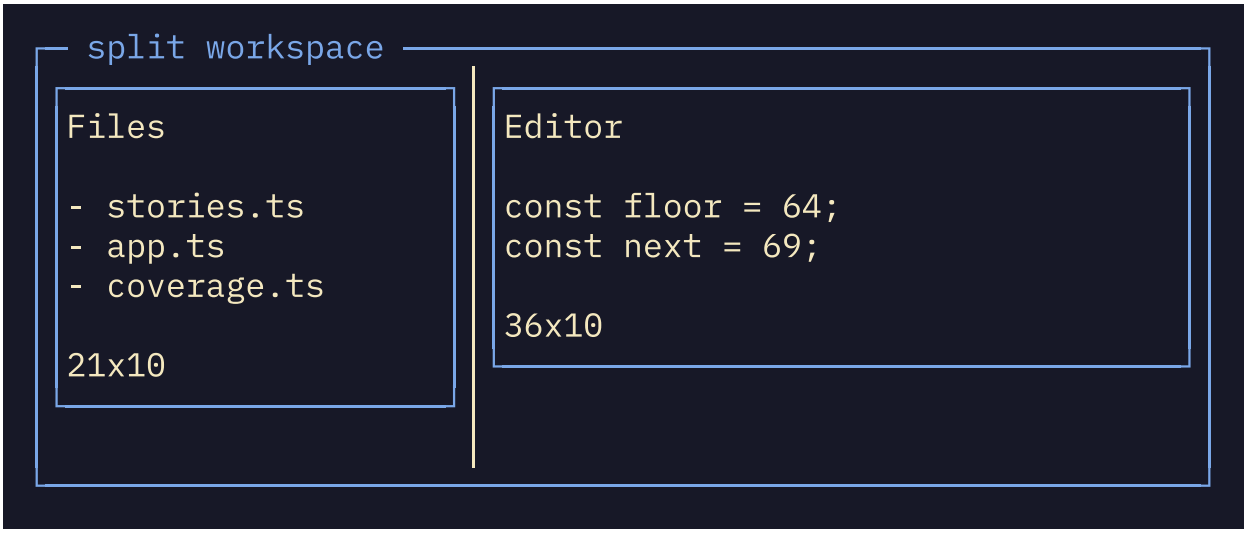
App-shell family for multi-view framed applications with global status, command discovery, overlays, and consistent shell-owned chrome.



Story: app-shell; variant: framed-page; 60 × 14 cells.

splitPaneSurface() / gridSurface()

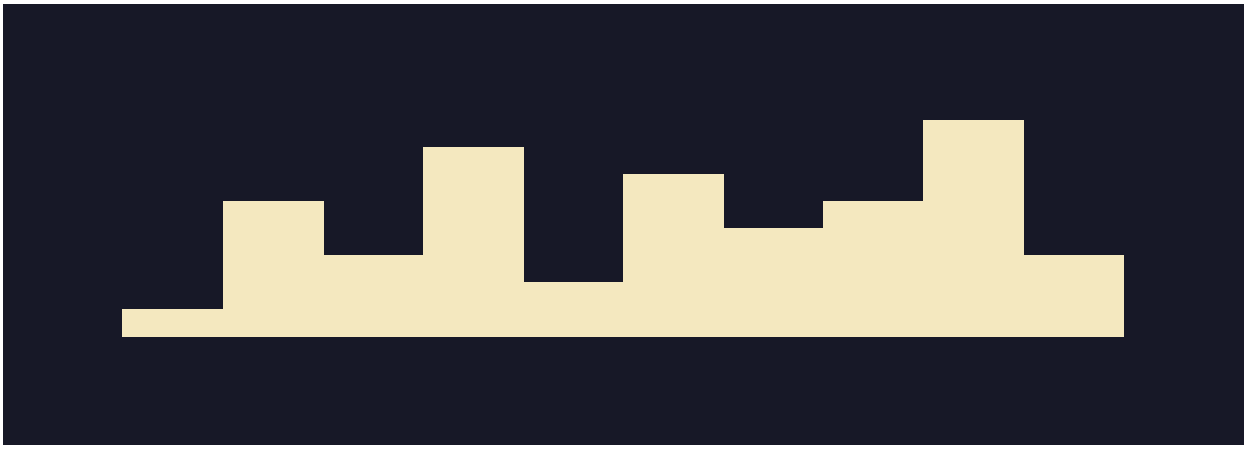
Workspace-layout family for honest spatial composition when simultaneous context materially helps the task and sequential flow would hide important relationships.



Story: workspace-layout; variant: split-context; 60 × 12 cells.

sparkline()

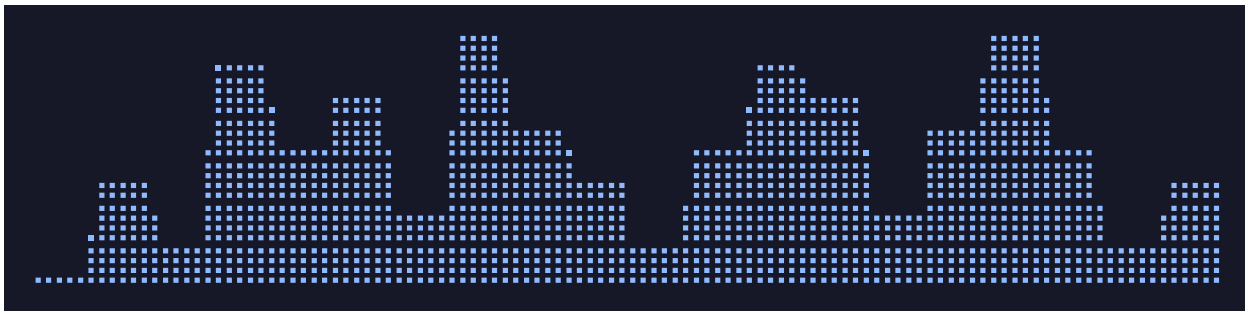
Compact inline trend graph using Unicode block characters — a glanceable shape next to a label instead of a number alone.



Story: sparkline; variant: basic; 10 × 1 cells.

brailleChartSurface()

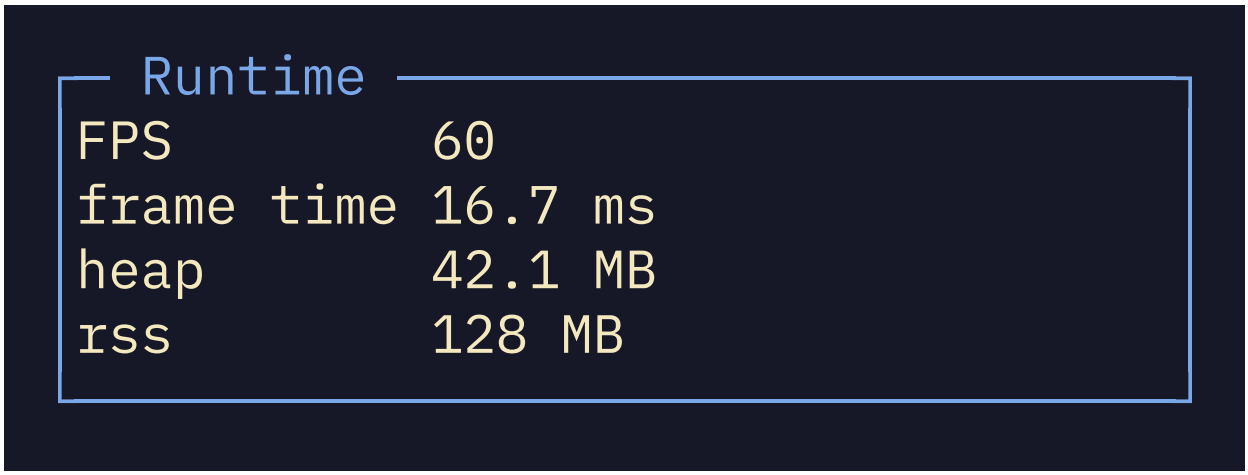
High-density filled area chart using Unicode Braille characters — 2×4 sub-pixel resolution per terminal cell for smooth curves in tight space.



Story: braille-chart; variant: basic; 56 × 6 cells.

statsPanelSurface()

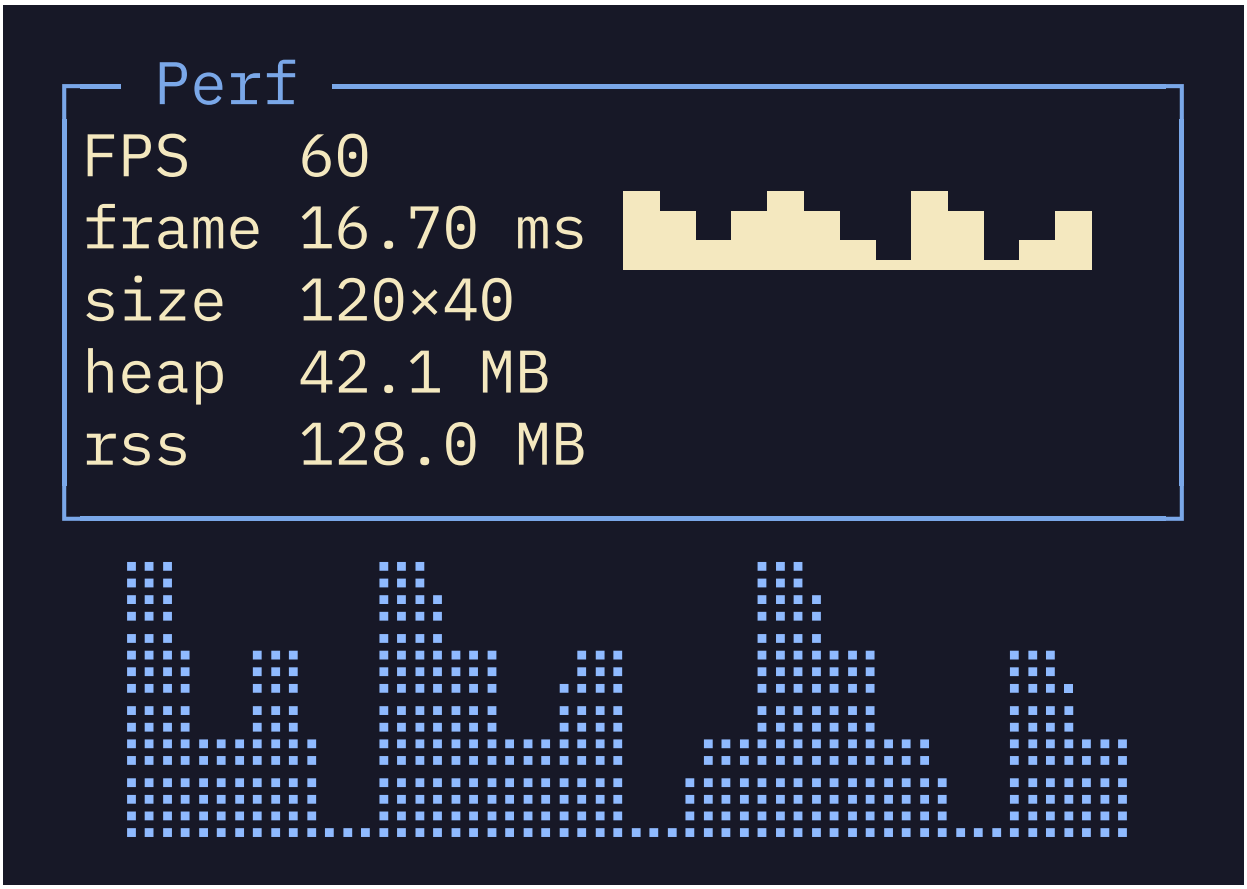
Titled bordered panel with aligned key-value metric rows and optional inline sparklines — the go-to component for labeled metric groups.



Story: stats-panel; variant: basic; 36 × 6 cells.

perfOverlaySurface()

Prebuilt FPS + memory dashboard composing statsPanelSurface and brailleChartSurface — drop-in performance overlay for any app.



Story: perf-overlay; variant: basic; 32 × 11 cells.

Source inventory

All 426 archived source documents in this edition are included. The inventory records the exact UTF-8 source SHA-256 for each chapter. Repository planning prose is historical design evidence; GitHub issues and milestones own current scope. Retired-document links resolve to the archived base revision.

Chapter	Source and SHA-256
1	docs/VISION.md f8f2e091b545be6c9cad24c1f466936ab13e2d1c4358922a19f18781738ee23b
2	docs/strategy/bijou-vision.md 6a865b5c2f8cb94a8272be2b810586c6203aa708318c6c67883c2e9c3a20e1b1
3	docs/strategy/bijou-ux-doctrine.md 230f6826dfcb5cce7bb425d107d8557f4d05a6fd5bad5747a080c1c52a695239
4	ARCHITECTURE.md 442ef44a1b4d6521d38027c2005c3a412058672af84e541d6cb480fa73ef0211
5	docs/ARCHITECTURE.md 0a7857e8ec64796ae51bc94f2f6132471850d5aa950cf5d4a8fa5453a0444c9f
6	docs/system-style-javascript.md 214aab3e2c6b963846d137f59404dfacf8a2d2e27fbe519e3484cdee90907633
7	docs/invariants/README.md 7db124ca5d29972e429e5a7c9d9e6b166a59301a9cb4d11d1bf99ac3aa7d1fc4
8	docs/strategy/README.md 07e851d5a81d2c9ca7967db430f42b7bbe2bcb05867cef7e414b7d3842b43031
9	docs/strategy/accessibility-and-assistive-modes.md 427619bcd84c29205da549d1a8ea03d40107410e53a311987f67a559d67ad83e
10	docs/strategy/ai-explainability-standard.md dcf5425e92087b25949f76382bb4697e3ed46b0d80e5183cff6c5e98b221438c
11	docs/invariants/buffer-holds-facts.md dc43a372f4229fa5c0c6686d2159c5f4a1772d866a7ab87ea1a3ddc561dbcf9d
12	docs/invariants/codecs-are-not-domain-models.md db96a4d07397350853bafd5b9b9458148e5bcab6f424e646cd5a141287dc8c5e
13	docs/invariants/commands-change-state-effects-do-not.md 725555c1aeb300c3ad272a4afc0f6916cc062e90ef75b12c0353d23cfaf68a7b
14	docs/strategy/component-story-v0.md 3df41192ca9217610be3023299ed8f41af2f21a4b79749aa0afac2d434c4bc03
15	docs/strategy/content-guide.md f50a3b8354e0cafdebabc951aa4f00689381cfb19bcd351b6edb643548e6ab25
16	docs/invariants/docs-are-the-demo.md 8bac1bf7ae48b176410d7d1dc2866b73f5c077aef3096faa8a49028b5a24b11d
17	docs/strategy/dogfood.md d5c6e6118e893e5e56b8143a624690e418019596e4ab520e8af3393bdbba3543
18	docs/invariants/focus-owns-input.md bd082054c05e164a2c9a610aae222a3b71dd8df50d7e09bd0b426b26c9811073
19	docs/strategy/focus-owns-input.md 28c55458a5720166e705cc801ddacfc0c8a65c2c86bdcc47a262ad04d4d26e94
20	docs/invariants/graceful-lowering-preserves-meaning.md ad2aeae01d09e2191595ad68bfd74a93fbe50b3d2a867f7a3957890a98dbfcae

Chapter	Source and SHA-256
21	docs/invariants/host-apis-stay-behind-adapters.md 25445acd743e55fa0c7a315ebe079fbcc7baedec8f6e9d9b2d0808ce81c69dac
22	docs/strategy/humane-shell.md 8645ceb1a480e16629db6e108d0f2e86717ef020e79bc2fbfe136e1df096446f
23	docs/strategy/layout-and-viewport-rules.md 748365680f130cc53c801ad0d69bf8958800e7d4dd9579404c00ae22b200d05e
24	docs/strategy/layout-localization-pipeline.md 294a2a1ba2732ba382951acb4370f21b8be1dcf9a41a5fd9b7417c557fcdc19d
25	docs/invariants/layout-owns-interaction-geometry.md d11bbe8713e1017fd99c407fbe799f062b3c240584f12d085bc526175bc909e9
26	docs/strategy/localization-and-bidirectionality.md 63fc6e612a670e4dbe9db89c709ee3c653b626160e4764c8afae90b83bda8990
27	docs/strategy/low-allocation-renderer.md 662619efe3b13cca086b55613a5f79271bd99b4c93e079738ff0524b36962074
28	docs/strategy/notification-history-belongs-to-the-shell.md 221b15ab01a7eda74fea980277a1cc86775b3cdb4d9c60425d6851e79544061c
29	docs/strategy/preference-lists-belong-to-bijou.md af6759131453ee809ff04e37604920cdebe95c61aa0e05958d48d7c9b0bc8c38
30	docs/invariants/runtime-truth-wins.md 5cb451d0326e43666ae8f7d927dfb95d75ffa623b698808ec5192435e80e4bd1
31	docs/invariants/schemas-live-at-boundaries.md 9c71039812bb9a9a2789de1ec880775d941946130caa71086733c33c97a48319
32	docs/strategy/settings-belong-to-the-shell.md d42374960a3565c852c1b4e283121bf8936f1ce89619e866321de6255cd44c7b
33	docs/invariants/shell-owns-shell-concerns.md ce285999e0cfd3f74d9e517a22f4106cbdf93a1793a7e6005e8215a5ac9873de
34	docs/invariants/state-machine-and-view-stack-are-distinct.md 1a48f2017ef95bfa776bc45324d5c0aca8c93cb62e87a56153af5d131cb1be73
35	docs/invariants/tests-are-the-spec.md dbf1f4682760cfdab6fc9714f752145fd23cdb5b48c41e058ae5388d11fd6f4b
36	docs/invariants/topmost-layer-dismisses-first.md 8184d99bc983112af235d7d93cdb7d58fc1d548174cea5c1c4a204e09d1822fc
37	docs/invariants/visible-controls-are-a-promise.md 0871c959012606505dd762c6d217062e605b2e0d7c0d53815c027a65a2407101
38	GUIDE.md 4872895cc8f093f43ce3f52708f8b72cdf7ae8834536d1a6ec66656d32a2eadc
39	docs/guides/terminal-scenarios.md 7eee8276a4ee3eb4920a3e4668b8d4bd5750127a47006695feeb2e502bdb9bee
40	docs/DOGFOOD.md 3529306d790fc95a6b6444894ecb35d1588a75bbd6af5e96dc0e591aa827499b
41	PACKAGE_DEVELOPMENT.md 0e9c3106fbebe762a663f6a94ca5b11651f1ff79eeec775471b93668d73d1ef4
42	docs/CLI.md 984b1375fda2a0329421cb3452720f5801f73eda8455384553042ba6ef087415
43	docs/MCP.md 494957acf95c177e00f52e87d4f555d9a1639f11314f15193ed325dc3d83066f
44	ADVANCED_GUIDE.md 14ac9eab0a673e520c39371b14b89721ab252c6a0eabfaf5b8e860e7f07f90fb
45	docs/DOCUMENTATION_STANDARDS.md 89c80d5d6848af2a2600b953f590f5259d9399f830aadbb9d5a1e808513f4065
46	docs/EXAMPLES.md c85506f97b68e32fc0d1e20a0cbcdbda318aae1fe04ce1f22af4896bbded362a
47	docs/MIGRATING_TO_V4.md f40e7e07719093e6f7b22438b2e49aabc19c5a671020602e07b076124cb5ca15
48	docs/specs/README.md 65a9b58d2e1ea2bd55dee5886d2e5daebbb898638af7eeba6fddf592f90ef62d
49	docs/guides/render-pipeline.md ab87dcac8c7b70ab561d44222fe7bf918bb462c929ee097382c1fc7af67d3fbf
50	docs/design-system/README.md 7cb5f686bca5c2a4a68ce895d717564702950fc97a8a88d9663b7c2e719eae1f
51	docs/design-system/blocks.md c68d6bc6ce4d28ce1f275b468f81da37227e547907366b66cab676b79789db08

Chapter	Source and SHA-256
52	docs/design-system/component-families.md b120de8b83203987714aabcdc151f88c17eee0ad272a30f1ebc105d7fd3535e5
53	docs/design-system/data-visualization.md dea6e1677eb96b13c532100910af91c37ef594f5f8cae840bd4a732628fe4fca
54	docs/design-system/foundations.md a2568f893f7d1c8d062336186b190e2b8998cf16709d43f31f69d621b0bc0a17
55	docs/reference/packed-bijou-cells.md c770ff0a28a11ae225d0a1c362669a908e0022e642aee285d9ce7e86ecb22c6a
56	docs/design-system/patterns.md 5165ed2572e0ad2ca9cd09ee58197ab9061b9a31f96a585706abee61f350498f
57	docs/design-system/pointer.md 3a389aa67374c8d0f5dd488259755c7a69951bcfae882208583f8f22dfa8cca7
58	docs/reference/profunctor-page-inspection.md de103281f22678dac5c1215c9598fd0d3d8effef8d86511ea79650fa23d8fe6d
59	docs/design-system/theme-authoring.md 3b9107b01fc6af173b044d9dcf0653ccb528bcba67afff7cf2a9741af5ab6244
60	docs/design-system/theme-rule-recipes.md b584a18b152c656bfc7175bb361e0afc487711f5c69e52b7ed51a68003a75bde
61	docs/design-system/theme-tokens.md 8ecf378c8ac6929408b8d3f7d0c06a9e5989084b30e6a425ea1e831f56ed8eb7
62	docs/design/DF-001-show-dogfood-coverage-progress.md 61cd9659a916704c7890910b5ecb36ac1d954418ba7f018d8c94388cba49acc8
63	docs/design/DF-003-enforce-dogfood-coverage-floor.md 1517aa52da330ddc93c109654fcaa13f32184ae8f30e696591ea4b7a317e2d01
64	docs/design/DF-004-raise-dogfood-coverage-floor-to-next-target.md eb3614ee5e51f39ecaf1fd65e2c4636eeb3d91c6cc24903db940ab8246f45386
65	docs/design/DF-005-raise-dogfood-coverage-floor-to-19-percent.md eee2b24e0d469c086c4eb04fce1777d717dc03324ac7ad8ccd29f5b2b1bb449c
66	docs/design/DF-006-raise-dogfood-coverage-floor-to-24-percent.md 57eda4cdd7d86aafbbe00110d3c6a1c59c7d566737b5e8c050079cf78c34a327
67	docs/design/DF-007-raise-dogfood-coverage-floor-to-29-percent.md 52cb97b55c8d8ae0e43b20ae4d687ba1c65408cabb06bb3011f0b7d50f34f779
68	docs/design/DF-008-raise-dogfood-coverage-floor-to-34-percent.md 81bc86d24eb70a5505718a163d7c9c6cb374e4ff133e878c2bc80add26161ed9
69	docs/design/DF-009-raise-dogfood-coverage-floor-to-39-percent.md 4c399cf99a96acc37f9bda86962a8c9d72e98c6f642656db24e3c3fe8d19b8c5
70	docs/design/DF-010-raise-dogfood-coverage-floor-to-44-percent.md 0613b574e4c0aa62f21bb81086bb149bb7dc1d7c9c71d9e8278e0e99eec35f0b
71	docs/design/DF-011-raise-dogfood-coverage-floor-to-49-percent.md 5102f4689df11201c552cb3afa3abe483dde387c30d246050552fd2fc75f4404
72	docs/design/DF-012-raise-dogfood-coverage-floor-to-54-percent.md 6bb126dd8c28f14638ca1b2605e74a18ac8cb3e21f0e04a2eed1c23eac7ca979
73	docs/design/DF-013-raise-dogfood-coverage-floor-to-59-percent.md 04729d0116771eba77daae6a8c507ec92426bc8f300b698815ff1c7077d83234
74	docs/design/DF-014-raise-dogfood-coverage-floor-to-64-percent.md 844d11af787f87542f910104236fd09621e6252f01f0904dd855b82a9a5d7370
75	docs/design/DF-015-raise-dogfood-coverage-floor-to-69-percent.md 01d99e1b1b73929ac5f81c53f8f59bfda87e923acc9e2451038ab98871db62fe
76	docs/design/DF-016-raise-dogfood-coverage-floor-to-74-percent.md 86612f99ca41ce3151a3d441fa6bfe6392d749e1a43c42818f8520843ef88834
77	docs/design/DF-017-raise-dogfood-coverage-floor-to-79-percent.md 6fdc177257c82b9fc5f50667c127c2f41ba1b5a3b71ef05b9daf35edaa55fe59
78	docs/design/DF-018-raise-dogfood-coverage-floor-to-84-percent.md 853731a0a187f95e1908d2dd5ee0d67823676fabad1ce63e6e51f30bcfa70215
79	docs/design/DF-019-raise-dogfood-coverage-floor-to-100-percent.md f38208c59ef961c8777c89a15dd7e34f9fd92f0d18be796d8bf59be556b0f53c
80	docs/design/DF-021-shape-dogfood-as-terminal-docs-system.md 83e93b082ba567c3288faa52db83e6bc9372f33fc96c96b58c5375fd63694bbd
81	docs/design/DF-022-build-prose-docs-reader-and-top-level-dogfood-nav.md f825c554f9635e2ca8d957630118733246d08071f23d5cca45f6eb36b4cde968
82	docs/design/DF-023-publish-repo-package-and-release-guides-in-dogfood.md 8b7a4f0fff951298583272beda95577e7dc063f4eb535fcabcda87991358ad4e

Chapter	Source and SHA-256
83	docs/design/DF-024-publish-philosophy-architecture-and-doctrine-guides-in-dogfood.md d52d212a23e3a9c1b8061dfc5706c9fff4be24373148b32cd16903040a93b8fd
84	docs/design/DF-025-make-dogfood-the-only-human-facing-docs-surface.md d31799de49ad53c2128a2ad2a144809dac74ef68e03a14141dd99a17f989f9a4
85	docs/design/DF-026-demote-examples-to-secondary-reference-status.md ed94bee69358ffba730a435da447f69a71eb54013e8a0b754704d44bf618c9e8
86	docs/design/DF-027-storybook-style-tool-for-bijou.md edd4c3ee2ed1b611e2d8adbabdfa560b4f4b93bbcd97f0038bb5f37464efc32d
87	docs/design/DF-028-story-capture-matrix.md 60f385389aaac9eb255a3f8efa356d3b3e2b40ff4ee46c846cfa5dbc705243d7
88	docs/design/DF-029-fixture-to-docs-promotion-path.md a3b9d9aed8d17391d770d05de0751e07861a03fc2ca590cf9a39e07d42ab1857
89	docs/design/DF-030-dogfood-docs-surface-block.md 8c4ca3282bcf95f7d76e139f84b57532d409dbf950ccbcb016cbd2b33ca04806
90	docs/design/DF-031-status-feedback-standard-blocks.md 2208d73c810b6ccc31e8c142b88eb71bde508d04947f4f053fb38cabff3c9c43
91	docs/design/DF-036-audit-loading-placeholders-family-across-real-surfaces.md e253691e10c9c6fa1938d1ec08a3d1037976ddd1b5fa25dc8d3a982d788eda33
92	docs/design/DF-039-component-family-standard-blocks.md 7c037924e05360af8ea5da18fbba4b9f513f04f041c6234fc411a958557183a1
93	docs/design/DF-041-audit-inspector-panels-family-across-real-surfaces.md 1c7d242735e594ea8cecd34bdc9ffe54fd9026eaadd75c87837c3fca8e052c0c
94	docs/design/DF-046-choice-navigation-standard-blocks.md 1890c6d2b7d1b6a1d5e749bc837bd50d699df1fc20e7a42561f19fede2f4a685
95	docs/design/DF-049-audit-multi-field-and-staged-forms-family-across-real-surfaces.md 169da48075fca574a3ce13732cd400a1fd5e72af0571dd898e95cff55a46b900
96	docs/design/DF-053-audit-motion-and-shader-effects-family-across-real-surfaces.md 2e015eee341020fa9687f5f6935f2e07281d6e69d3b1d60dee6eda95f43e37b0
97	docs/design/DF-054-late-family-standard-blocks.md 613924de563a5e8e34843e86116ea9770a206d5c2a8412a80b7ccfad9044b45f
98	docs/design/DF-060-audit-viewport-masking-and-scrollable-inspection-panes-family-across-real-surfaces.md e3aabb4b2f3dcd79e5e2be7490a19b1186b5942d39aa43e4e2a673fc57759fd8
99	docs/design/DF-060-v7-dogfood-release-title-screen.md 33315ff6fcca244f5b09f80643cb6a8f5bafcc1fe58b3413c770326885cc5164
100	docs/design/DF-061-audit-overlay-primitives-family-across-real-surfaces.md 99daa0f9517d2a8210f49f4b5f8af3144a1ebef2263b1c2cc2c9340e99a39c1a
101	docs/design/DF-062-audit-notification-system-family-across-real-surfaces.md f66eb2198f03e120b7c8eb06a9df82176afda40ddc073c7cc84f8e8e1db2013d
102	docs/design/DF-063-audit-app-shell-family-across-real-surfaces.md c20fcab14a7d1db376cd17d0c83b01f82e689e5835c6e4a8ff7a47f1e3f4e608
103	docs/design/DF-064-audit-keybinding-help-and-shell-hints-family-across-real-surfaces.md 5f81e7a6cb62b8cf5f8acec813e1e43f46736c88a52c4d623f2910efc2865c06
104	docs/design/DF-065-audit-workspace-layout-family-across-real-surfaces.md 3e31048ee3a73adfcf7087cecd74a0350bbd3b200faf251b78a44b20bdae0fbd
105	docs/design/DF-066-audit-data-visualization-family-across-real-surfaces.md 8e9bc3002922a8453d86ca478b3b5c2d3c2bcffe1e82e1ed8985addeac4cda01
106	docs/design/DF-067-prove-responsive-dogfood-layout-variants.md 7424af47e3a86c3262b6ee06b0e8ec10d4676f44dee49bc4be947f607ef2ce93
107	docs/design/DF-069-block-authored-dogfood.md 8b37312ac00ed529d5611285b4c542d6b447af5bf9ba65f174e4646c35c241b7
108	docs/design/DF-070-dogfood-block-product-polish.md f8f8dc0a54744179d981befd97d82ec9e23af67fd12732e5bfb4ac659cfc67d0
109	docs/design/DF-071-dogfood-block-authored-surfaces.md 48aea9270294b6e29371fecc13031433b99821aeb850505c84b71ebd2c39e2e4
110	docs/design/DF-072-dogfood-release-title-artifact-gallery.md 0a0ff711cfe9df9a00e5074609a3203eb9b297ae09b70f513eb040e10d374ed1
111	docs/design/DF-073-raster-to-glyph-title-art.md 5b33f30598a21bd0be7b6e012d3f0de8183c27639c1a0283e656bc27580dd350

Chapter	Source and SHA-256
112	docs/design/DF-074-fluid-triangle-title-screen.md d9dbc9c0386aa0b922b22919e00712474b4d5e1827a5bf4e40b4f122749e3d47
113	docs/design/DF-075-dogfood-post-release-polish.md d274b5a309403fcf1eb8d9e7966d72984c2f710dde976e6f327c51cbac62e321
114	docs/design/DF-076-blocklab-demo-readiness.md 1a30d5ee1268129b2cfbe33b6055ef025e3f4fc58d7a80c84fb6c019b3f9f86a
115	docs/design/DF-077-dogfood-app-landing-extraction.md 0148e3c2b2417471d29d0728e72a50e0466b39dfa2e0658e1f16e357c20dd8e6
116	docs/design/DF-078-dogfood-release-story-surfaces.md ddf6519092c218ec85e065bfa2721f2cbf3d9890c4ad3c9bdcbfb9ba0964a6d7
117	docs/design/DF-150-dogfood-terminal-guardrails.md cf4e85be42bf6c3b3b84731fb68fa3a82d1a8982ff9857a05c20e2abdc771c0d
118	docs/design/DX-001-type-framed-app-messages-and-updates-end-to-end.md d2790f3d62d8996ca444ae18dc2c37cf2666c3a4bf82c77e76a9222b5e8783f9
119	docs/design/DX-002-reconcile-cmd-typing-with-cleanup-and-effect-patterns.md 3d5c54faa47d99a9b8e0ee65de503e9e938a2a976145437f7e8fde8d1382800e
120	docs/design/DX-022-layout-inspector-overlay.md 600eab9e22bfa121475bffc3c5438e12c0b0abb51a35f07168b12dc050d19ed4
121	docs/design/DX-023-input-routing-inspector.md 22d81459043c5af85b71db44c781df473bc4fbb542c8f893e1eb7d97e0ddfbba
122	docs/design/DX-024-surface-diff-viewer.md a10888da7f39ac704536e4203b2aa25c02bf8637806f52c4baeae9b58edd74c2
123	docs/design/DX-025-component-metadata-contract.md 551123bef08aaeb14a9fa65a18f0f501ee8cd180530b50bcf7b5c5dc6e66debb
124	docs/design/DX-026-mode-lowering-linter.md 27174dae6a5eeeb44742c92d033d4f8bb6bcb0654ec0bc2f7aa1a919d5d51043
125	docs/design/DX-030-add-boundary-aware-pointer-selection-and-copy.md 068762f2309fac8ded4174f241054590ea28e7fc3905c42ffe609164a8f64987
126	docs/design/DX-031-standard-bijou-blocks.md 965b2ce53b21c31d32a23c31e19c4bb0ce1326fcff7bb38ea46de4101bc728a7
127	docs/design/DX-031F-block-tree-rendering.md 47aa53de080d5a92d99a17ed64131b55c0a91bf1e58455cdbdaa5ba4cf46a11f
128	docs/design/DX-032-create-tui-app-skeleton-renders-consumer-pages.md dc881679f1df72ce14508a166176cba4c932a95cbbec21fbaee7d16ceb1789cc
129	docs/design/DX-033-remove-showcase-layout-sludge.md 506135e7512138ff8f953e81005550e23023f733f774dced9db11155e5969f4a
130	docs/design/DX-034-declarative-view-data-binding.md 94b0d0ad19a0590c6dc35e7d6a86633c97458d52d4f6a9754b516b1c03b3fd41
131	docs/design/DX-036-responsive-table-layout-and-variants.md 27bd4848d05715c01ac233048e7690edc76ad267a9870381a5f242979f706c69
132	docs/design/DX-038-blocklab-rename-and-migration.md a44cbf680ed11ec37fde107884191b3171d4f6bf1bd247d0ac8fbc83ffddcaa6
133	docs/design/DX-039-blocklab-product-parity.md eea297ae6816b24324ba5a6adc0d08fcf0cd17b63a16786d4befb6c72b73b3fe
134	docs/design/DX-040-v7-review-regression-fixes.md 148d2747fd10fdc2e7ae5334e0a8bb68b8c21b3cc97a24438751bff2f0802b53
135	docs/design/DX-041-image-to-glyph-side-app.md 351a631993b4e5c6cb0c2a1c9fb6100b5e8a53f7a5ee305a7d6c598c58f31c94
136	docs/design/DX-042-shared-ui-scene-ir-and-bijou-render-target.md bb9f2f1cf68bff4b8426e83f700ffa956a273592355501ff4eea55003cc05970
137	docs/design/DX-043-portable-bijou-blocks-and-multi-endpoint-ir.md 3e8daa12545583370d1219e6ff97d327c2b0f5d8358715dfd8c03b01eb8e5d36
138	docs/design/DX-044-graphql-authored-bijou-block-ir-proof.md ce905c02be553b475bb336f0db42faa073698eb17ee685436021903ff38201a7
139	docs/design/DX-045-graphql-block-groups-and-debug-facts.md aabd2d6f4d6908a40cfe5af2f2707b4ea12c787768ed96c8399c00115b94bccd
140	docs/design/DX-046-graphql-authored-dogfood-block-fixture.md 34ece72bb060027e7bc9971be67294e2b0dfd0a754a87295eeb5bbab6aa37f12
141	docs/design/DX-047-blocks-app-binding-snippets.md c09b51157054fb1ef9ab76aab1555c6a68d1eafb96e2c18506f548556d755f02
142	docs/design/DX-048-v8-runtime-graph-scene-ir-contract.md c4a11ecb9134d6c8e9e3a72f930a218804dbd22ff7b5f917a9dcf4ac0fbf10dc

Chapter	Source and SHA-256
143	docs/design/DX-049-visor-artifact-bundle-proof.md 05200f144f8e1bc2d93c34b18e1d2fe6a56ae1a7daa78941242a03a847c525b4
144	docs/design/DX-050-profunctor-page-inspection.md a57d7b4503520fb2645409f52be441115e8029666e1e1f055c46c21f4ff4b85d
145	docs/design/DX-052-no-color-is-not-a-capability.md ca919671d025139d4303cbeaa6510c345dceb305ebe9df1830b2a14a260dd361
146	docs/design/DX-125-day-zero-onboarding-bridge.md 8080a9d2b189f33b4c11c5f3c5304225d6a4c6ccf31edc6958dbea8bc5a4403a
147	docs/design/RE-001-define-runtime-engine-architecture.md adf8c9badd844de475b2892c96616be8e248e1bf0fb79135c7d9ef507ae4228c
148	docs/design/RE-002-promote-first-class-state-machine-and-view-stack.md 28ad8ed0d1f0bfa3c8cb9b37ddbb939b75d0928f770770efb1c3bd1b784fa6ef
149	docs/design/RE-003-retain-layout-trees-and-layout-invalidation.md 7fe01c3a240a1a9a7e32b41d5acd8be91567a1ee4ed886fbe054850187d7ba6f
150	docs/design/RE-004-route-input-through-layouts-and-layer-bubbling.md 7cb8caeb7ae6a6561880e475d5cd41e739c07e688595616f6fd2152bc5ae3801
151	docs/design/RE-005-buffer-commands-and-effects-separately.md a1b6c52004e9a2c6fb9f9f680d26c0507fbf04e12a6a18957bd920af029d55fd
152	docs/design/RE-006-formalize-component-layout-and-interaction-contracts.md 1089aac53d706bf949ba9dd3012b923447468093375f9696cba9cc2489ecdc34
153	docs/design/RE-007-migrate-framed-shell-onto-runtime-engine-seams.md 5620ab061d0fa7fb30316e169a6532d2bf27f66d05bdbf3e7b1f1aedcf20cb93
154	docs/design/RE-008-perf-findings.md a0d148cb2eeeab89f40fd41ce8398aefef3e645de8bf2bee86c1541024867d2d
155	docs/design/RE-024-surface-budget-warnings.md b096757954bdd73628309c4779d0a51d75dadf5f9946211f9a5b5926c83470df
156	docs/design/RE-030-average-high-resolution-canvas-colors.md e1da38f52150253dec377a5e9836f08f9916d85e29e1c0d3fad456dddc374157
157	docs/design/RE-031-raytrace-shader-kernel.md c93782df0cd3be182875cea2b119e59ddd2b1bd0bb9079834435ae1341049848
158	docs/design/RE-032-fit-cell-glyphs-from-coverage.md 2832fba24091cf839e468eaf1b27393d177f00dea0c6c8f7b9a2f03f1f3c7f74
159	docs/design/RE-033-stable-spring-pulse-integration.md 46050ea887f478d0764bc6d82d2c9cd87f84644c50afefb2438215cf4ac8c3f6
160	docs/design/RE-034-glyph-fit-canvas-resolution.md a210a823f84fbdb6916c982f0900809fb3f1d47a4c0ffd95841956e54cf6955f
161	docs/design/RE-035-mandatory-layout-envelope-and-constraint-negotiation.md 505cd9de2b4a63a5142a4944962be3a5b2ebfe79d8137f69ef17c70a391fbc2f
162	docs/design/RE-036-packed-bijou-cells-surface-adapter.md 0c3a57d326500bb5ca879f2d4c23f2bac6b1fc9fb9b8c693b2e6a3e3ea2797b1
163	docs/design/RE-041-v72-framework-input-stabilization.md 176716f4fe1b4058fb35d46daa1b5ff17106a9739637da0fd306437c787b0d5c
164	docs/design/RE-127-command-and-render-runtime-guardrails.md e9f7081dda8810b54ebcef2b3e3758aeacf0e604422c18f5de600c16cb5ab456
165	docs/design/RE-142-node-bootstrap-failure-contract.md 706deeb11a8c4f033693b9e9ddcd42851993ad2e2973aed76fb416ec41992551
166	docs/design/DL-001-capture-design-language-doctrine.md e7bece8c6aa43683a3c7bcc8655d3d4cc5475628e628c92149f03f9f33c8d70c
167	docs/design/DL-002-canonicalize-patterns-and-blocks.md 198e757525068a24a39d9f12301728bc2223390ec5b9209bb20856a6df6e1dba
168	docs/design/DL-003-prove-canonical-patterns-in-shared-surfaces.md df6b469eb1f405f9dc8af6da1d3f05b1982c3deae382c34b2265b3c64cd0e259
169	docs/design/DL-004-prove-drawer-rhythm-and-notice-rows.md f1b58cd0ea62db84ecac484095bf122cf25bf36f4b73ddc4fff03641530bb52a
170	docs/design/DL-005-prove-inspector-and-guided-flow-rhythm.md 505ea91e4a40a57d4bdbb43adfe749af6e41658357684d3c53e76dcf88d94ffd
171	docs/design/DL-006-prove-inspector-panel-rhythm.md 0631935205e82843aa3f7da3ce85d3e91635f42b752a18635af1ea61bd4fcb0a
172	docs/design/DL-007-promote-inspector-panel-block.md 5b510a26dd32989c2969a5cd3525fa8e686f661d056acabaec914d4de7e3c930
173	docs/design/DL-008-promote-guided-flow-block.md bb27eb6e16289357b2f17b904ae17bbbabb4f3ac37704b531599ecd42b96a6d2

Chapter	Source and SHA-256
174	docs/design/DL-011-theme-contrast-doctor.md 88f51d925fd21608a34da537ed5d53a9401d6404b678b5543f813a4bb14aa460
175	docs/design/DL-012-separate-focus-gutter-from-scrollbar-ui-tokens.md 7c3124a282c3d6c851e8c1daddb92ec930d4cc249747a64acf61678b5543d62b
176	docs/design/DL-013-design-token-theme-builder-api.md caa44cc2fd59000877319963fc784be201d41dd9d5857f81b576962e0cb98981
177	docs/design/DL-014-theme-inspector-and-lab.md 0051512868da8a90af1ea0a326dbb603aede9e0401872a8dba97e9c80e2b302f
178	docs/design/DL-015-theme-safe-pairs-and-contrast-matrices.md c6596b22c81afa350764944cbc7544fdc84f4908d04055455b9fc9407888637e
179	docs/design/DL-016-mode-aware-runtime-theme-adapter.md e1566cd921a9b97267a1944dc7386a813a7e4ec0096ebc7d8b99ee5c96859308
180	docs/design/DL-017-dogfood-light-theme-readiness.md 97cc468d372d6fd55c73413f07e18f759aab30911b3ff3cb0e622e821a21f8f9
181	docs/design/DL-018-first-party-theme-variant-coverage.md 333e714552f3a96b4721b9d38a18157f8962754a3156105fe14f207ff1b51923
182	docs/design/DL-019-theme-rule-selection-and-inspection.md f0525b2fe1835af07a80e536445b6ec2833fca4aa0147a37170cb5edbd3f09ec
183	docs/design/DL-020-dogfood-rule-authored-first-party-themes.md 20a47e4edf6bef8c2621018e75e93a30677f6164016dc9010a4261f36105547a
184	docs/design/DL-021-dogfood-theme-token-usage-proof.md 2e06c0effa59213d46b32c4d563d05cca2dbb23ae4446099ec87c4393f22c35d
185	docs/design/DL-022-theme-lab-editor-graph-ux.md bcac02b43a7d40fb0a1afbb8472f2a4849519549409f979e3a0d450ebfcaefa2
186	docs/design/LX-001-bijou-i18n-runtime-package.md ef67d636b79340830657b9426975a73e8c72b972517432505b17cc73f422ece8
187	docs/design/LX-002-bijou-i18n-tools-package.md bc3ac6f92027a62229620bfda4ce1dc5e10b823764e7193ec595d563036a56e2
188	docs/design/LX-003-spreadsheet-adapters-and-catalog-exchange-workflows.md b5f3643e06470e72a2240131d4b4a176e909377abc3ced4a22b23698a913a780
189	docs/design/LX-004-provider-adapters-for-workbook-and-bundle-exchange.md 46f10f7556650408ff2f3e7db2afee9fe808fca2cd54049edf1c9695251a8a97
190	docs/design/LX-005-rich-spreadsheet-and-filesystem-adapters.md 77dcfdfe7f24a4bbde47df99ac2de9076b95cc0fdd1191d98667f1e815bafd62
191	docs/design/LX-006-xlsx-localization-adapters.md bc98e5ea25f5980dfd4dc497ff42be1111cefc7d964781bb6e2e7236a00a98a5
192	docs/design/LX-008-localize-shell-chrome-and-dogfood.md f9cd542a83629c283b7925d383bebe398ecd75b06a8ff486bec1064182fca790
193	docs/design/LX-019-localization-port-contract.md 960e096c4b583676ba7724472cc0bf3fa84220194669b2f65bd9093022ce8c58
194	docs/design/LX-020-dogfood-locale-demo-readiness.md 01e6c84b3a6f1ceb97dd11f0acc450f8a1ba4bf6188623648c358d9d8c48e891
195	docs/design/LX-021-dogfood-string-debt-ratchet.md cbb3bc4bd2960898f1284dd48f39ddbe41f342ac9ef79d20903b94efe4ee47d2
196	docs/design/LX-022-dogfood-app-main-ratchet.md 6a1dbf1fa0830eaaa4eb72fbe1c8ce122f0daa90629136e8d8c600f3702dd493
197	docs/design/LX-023-dogfood-blocks-capture-ratchet.md 9538263458dba122a006d56e6e961242242e30a5d14a38474788a5bfb7fd416a
198	docs/design/WF-001-adopt-cycle-workflow.md 598f92584bd87ec46af7d059b47a15a7805a62b98be96b57f2131e3fffe1a658
199	docs/design/WF-003-replace-smoke-examples-with-smoke-dogfood.md e0d2a8aa647cc514025d18871eb854c2a6e38547178249635f0c6e005e6c3683
200	docs/design/WF-004-shape-the-next-release.md 23fcc1df5ed36dc47e123d7fcab8297ca74d270f1c3862567a9a61968da375f3
201	docs/design/WF-005-close-4-1-0-i18n-publish-surface-gap.md bc035e080cb5138fae5cecfc718e22c1187a453cca6cadc24bd2b0ac96c150c2
202	docs/design/WF-006-cut-clean-4-1-0-release-boundary.md cadb1e5ca3526270a255257f5142cc0df1d2edf4b0f54c93e12ab4f74137e113
203	docs/design/WF-009-keep-release-prs-under-automated-review-file-limits.md 846598b23d64463730f5fdc8f6db0b94fdb9c1d195fee042194b0fd26e5f9496
204	docs/design/WF-122-cross-platform-contributor-path.md 11e5fa9ae7cd6f3d83a80181a8681ce85526fb37980fb99558dfdce5652b1f43

Chapter	Source and SHA-256
205	docs/design/WF-123-issues-first-method-intake.md 855da918ab4c9930f5d8765ddcc63e325dc4d57d6b8ec0eb3c2629096ee66727
206	docs/design/WF-125-draft-first-cycle-start-workflow.md b5bec8a915be323165d0beaadf4ef73e0229e28e8905c4f7b5047ad08af2b890
207	docs/design/WF-127-v7-issue-complete-tracker-sync.md 2528774cb6357c91c3237d501ad6ac11d2d2ba39b903759b21ab5459db2b880f
208	docs/design/WF-128-safe-gh-comments-and-milestone-items.md da3464c163e1e7a3def66476e2d28043e58889b5bfd7a10917e98e04629eab89
209	docs/design/WF-129-path-gate-dogfood-ci.md 0afb420a28c0d8b8165bec00c110bd6365e2b0ece25089466d81ef9c5873dfaa
210	docs/design/WF-130-roadmap-goalpost-policy.md e786c5100a29aa6f5d07a01287ce7279d1fa3d9e7f250e6dea158b000c8b6359
211	docs/design/WF-131-roadmap-current-reality-refresh.md 5ee5d7ab7a85f98a717b4aed08d34f79fcfc2c2c087724676aa32db18fa81776
212	docs/design/WF-132-release-readiness-and-dogfood-i18n-gates.md 3d4992f83e24547115195d0261effb7ddbd7651d7f6f6155e4ecdf8df8b1f040
213	docs/design/WF-133-esbuild-security-patch.md 21bb6bee3e77abc9ff962eec91e829e695e967354aad16663a57968108abda4f
214	docs/design/WF-134-code-dojo-eslint-ratchet-1.md de650a59345faa6f552271cad31fa485ea29e893cbabe78bad05dbdee2441de6
215	docs/design/WF-135-respecting-dojo-1000.md bac41e51c5ade73f149ac7daea32ebe6389c2e6b48bebff335dc91d226f6f900
216	docs/design/WF-136-respecting-dojo-ratchet-2018.md 7a14c246a1feb949109c438f238204d2f5f94aafb48c0fd3d16fdfa62937090b
217	docs/design/WF-137-respecting-dojo-ratchet-1965.md ca68bd6b767b95d78bc881ea105e50cfe083f82b1174907433ba3877c8a37b5a
218	docs/design/WF-138-respecting-dojo-ratchet-1857.md 7f853ca8092db15fd2d778a30a8a4d53aa8b0a56261a51db8673deb9acc64291
219	docs/design/WF-139-respecting-dojo-ratchet-1796.md 5cbffe6cc86d04dcfd8448e264c969763118d2a9612946725786739e2673d505
220	docs/design/WF-140-respecting-dojo-ratchet-1729.md 401998654c4348fcb70bcac2a810f448200826b18db4475ee153092a9493006b
221	docs/design/WF-141-respecting-dojo-ratchet-1679.md 134e7f1a6f3c14664d3a014dbac1446198fb8ddcd285d83960911e50534956d6
222	docs/design/WF-142-respecting-dojo-ratchet-1621.md 73f48d80ca3965142b6569342eb44ba38829802f3832e64ebdc9dd223e9bf935
223	docs/design/WF-143-respecting-dojo-ratchet-1567.md ae38fbaae0ba38444370e23da29b6b15f61bcc079972ca2aecf2c92ed5c54d72
224	docs/design/WF-144-respecting-dojo-ratchet-1514.md 9e2a49cb17d0b8bd1f724b9606c1cc18e80cde280bd0887889310c738ee51bfd
225	docs/design/WF-145-respecting-dojo-ratchet-1461.md 8ddcd4fb21b1580b285d89c89eb5c63718eb559be409e74516c301ae50934e6c
226	docs/design/WF-146-respecting-dojo-ratchet-1393.md e440dd2ca21ec90e89572f28b099b753b93ef1220842d66d7d70f41279ebd305
227	docs/design/WF-147-respecting-dojo-ratchet-1319.md 7a87001f8821bf2c4822c9a54900641d22837afcc9e2646eec0660dec28b1471
228	docs/design/WF-148-respecting-dojo-ratchet-1268.md c27ba69886d5b1299166cf38820effd6da1bd9e6f41225b3b2d47b90d6c4cb67
229	docs/design/WF-149-respecting-dojo-ratchet-1216.md 76fd5bd2032aa55345132bc46580c3706a8873898e54cdc4471dce21f37de4fc
230	docs/design/WF-150-respecting-dojo-ratchet-1165.md c4006cad8fc9e09ef4304a0aca8fe1d5c8644d1361a59b972c8df562a74b9573
231	docs/design/WF-151-respecting-dojo-ratchet-1110.md 83ba80dae03c32f5a6514dccf83c35e79ae79c071e70f538fae0141a7085dcd5
232	docs/design/WF-152-respecting-dojo-ratchet-1055.md 858a64b63888310fa32804aceca3c6744679209b448b539154a91830b9226083
233	docs/design/WF-153-respecting-dojo-ratchet-1004.md 28e8d3c186911c00e60888065c30c0601367c28e24a2b49471f28b419298d9ad
234	docs/design/WF-154-respecting-dojo-ratchet-953.md f9b52c1de21efecaa3f4fd72e1dbf918fe99399040a6f3bf412bfa0fe0e0c64a
235	docs/design/WF-155-respecting-dojo-ratchet-624.md 47994c2cdd62d87e0f898bd1d34a4f81286cb4e41c674a8063096094be7f889e

Chapter	Source and SHA-256
236	docs/design/WF-156-respecting-dojo-ratchet-573.md d8d7783a646b0d013f77e047f2165e8c4ec49c9db80c39a201611f218034c3de
237	docs/design/WF-157-respecting-dojo-ratchet-520.md de19ad74b1781349fd3b09b64fc43289a3661670f85ad2f5403d7bd22a12e593
238	docs/design/WF-158-respecting-dojo-ratchet-468.md f75d2f117b4748b342e713d9655bc72de50c004cb0eee2f3a295e3e0040ca3e7
239	docs/design/WF-159-respecting-dojo-ratchet-415.md 85f8357b24a02d4c6f0e9c860a7330e270a2ff55ac4947b22e2d7db3d65c790d
240	docs/design/WF-160-respecting-dojo-ratchet-364.md 7ca22aaca1263ffa75e390a9b2b36815c3f8cb3906ecbd24549d4ed370fc4dac
241	docs/design/WF-161-respecting-dojo-ratchet-262.md 20fb9a4069a8583149f8e7034b417cb408ac402392dac3d3f2c8519dd470c4c7
242	docs/design/WF-162-respecting-dojo-ratchet-162.md 5d41822478c3bd825cde7fe4e1baa5e1862ac50a9de35d86720e813085e23b89
243	docs/design/WF-163-respecting-dojo-ratchet-112.md 81096f4923a2ffd179b0b2050a4955dc95836a1411cd94db03af8d84714fce64
244	docs/design/WF-164-respecting-dojo-ratchet-62.md d99fb1105217d7ca0c85eba0510215b06630c1c71478f65fd5cfbd4457788b97
245	docs/design/WF-165-respecting-dojo-ratchet-12.md 08afb281d849f071937d557b60563efe89108209fc5e7a6cb40f8588733e8644
246	docs/design/WF-166-v8-dependency-security-closeout.md 4691f7ff7f6062b1de637feab696a97ea438f32ae47176e6fcebb6de7fb14305
247	docs/design/WF-167-roadmap-milestone-authority.md 1b4ab3088f5e175f67bfa5703270bf72a677073250767dfc262267e2ba6ceaba
248	docs/design/0001-008-byte-packed-surface-representation/008-byte-packed-surface-representation.md d267cd75283d57488e59d4ba9ac94b4d6b89cb1c793a2d47ab645ea91905f271
249	docs/design/ASAP-001-fix-mcp-dag-type-error.md 4e2616458260cb8b1894348d35d654486fb1cfd0acd70029ea9ae53b45745a9b
250	docs/design/AUDIT-001-showcase-audit.md 768c32b25b65782723412fbcb4c46eec1699dd0d5515d562c08b2e377fd2c285
251	docs/design/BE-001-dogfood-blocks-breakout.md be5cc3b3cd0d81bbe99e124f8befa591151784b315b0ed7d5ec5b7d15115d3b2
252	docs/design/HT-002-layered-focus-and-interaction.md aa29b1cd2fe771eb17c53a1d0e154d347dc90da7ac7d374648f781f431e18388
253	docs/design/HT-003-implement-layer-stack-and-input-map-routing.md ee8522ae911693ae0a16612fbc30853aea1d0e3e2f9753e4aa2785a1dd4bbd05
254	docs/design/HT-004-promote-explicit-layer-objects-and-richer-shell-introspection.md 1493a7d5551c23d2fff0f63682b4bac43dd700833f7ab3af516e59b4d29b0831
255	docs/design/HT-007-focus-map-surface.md 5f0325b9a7cd7adda747d72ffc51cddf836bded08ac687b4563d3d8184689731
256	docs/design/HT_HT-008-surface-native-text-bypasses-the-terminal-sanitization-boundary.md f1a4cc8d25ea83d101ca277a420ed69892d150f135e28de802392d4d1cae8774
257	docs/design/README.md 290c5925eb6bed530b7581bb390b3acbb53163b65e4a0d5896cf603c0cf939dd
258	docs/WORKFLOW.md ec84280b9fb6c730ffac3458b32c707b15d28f82accc0751bec888809b8d06b8
259	docs/releases/7.1.0/README.md a06f6f945f27d1945525d962969b7de304ede25e8261ec04cd47be776696ef65
260	docs/releases/7.2.0/README.md 43e31e5a18641ceaecb8965361ce46632e2a787145412960c836381c91fbcfb1
261	docs/releases/8.0.0/README.md b75b613f7dd80c24a755b01d0345be35f670b73c204678a2f92bb6757d8de80d
262	docs/releases/README.md c37235d80e7dffbab470a4a91ea0e7d470ddedef94de59d8ea4716fa6d8a9dd1
263	docs/releases/4.1.0/migration-guide.md 9b00f78b3619429645dbd7503f0d40ea7fb72ee7ad7d535dca9ead9280f7e050
264	docs/releases/4.2.0/migration-guide.md 24c5f057a025a30066ba2383a15ca2cf623d24dfa8bdeaa2dd46efb1e6215d02
265	docs/releases/4.3.0/migration-guide.md 7a5413bb92aa7716345f8acf1865a5e781857b3ba565d462329a302bf90b1fc9

Chapter	Source and SHA-256
266	docs/releases/4.4.0/migration-guide.md 92da5c04437699d5e3c538f0b31a59c0d44543bd971d9bd11d5878c79833e947
267	docs/releases/4.4.1/migration-guide.md f1ad77c917f616b1ba71270eaa035241a7f60113ca1ba9d0951177f674e05baa
268	docs/releases/5.0.0/migration-guide.md 25b09120645878ed1c57996170271abbba2b12129fb74e7b198449a8ce7fb971
269	docs/releases/7.0.0/migration-guide.md 58368f0b91eee3bcb3dd050e0ad5ca722430d5fe4637812d3b819b77017ab30d
270	docs/releases/7.1.0/migration-guide.md 46d9c4609eed7a4808b439e0b4419331e4d180e668f38a868f359633dfc6674a
271	docs/releases/7.2.0/migration-guide.md 60a7508b0d2451e48b3147dbdbd934ea11dfc3c9970161e71ea8278a6d9e748a
272	docs/releases/8.0.0/migration-guide.md 396ef8dbc87c7cd35b7f1f0013c311b26d24be667d3ffec11b1918f7b9e6cdfa
273	docs/release.md 9a47871733fd05728f5f68d6afbdfa6ff917f34b4ff6180d6bdd3852768713cd
274	docs/releases/4.1.0/whats-new.md f6fe4b7c6ec2f939eed1578ce2067d7c2e66eadb94fe7b216ebf02dc776f53b8
275	docs/releases/4.2.0/whats-new.md 4772f6859fff0dcc33be3f4d4cc446e19e803400c92260f85f9f35e0e9751890
276	docs/releases/4.3.0/whats-new.md 44893d31e13e06dbed48815b670fc019e2c81066fbf6f5f8d3eebc1072e6ef95
277	docs/releases/4.4.0/whats-new.md f8863047ac88cba5a84bbc96484d92e0a9d4c309f2e8696e5d605bb15201e6bc
278	docs/releases/4.4.1/whats-new.md 5d82ea6e77baacd16fdf8b68382123bc2e2cef722f186c5759672c7b35a569c6
279	docs/releases/5.0.0/whats-new.md 5cd3c456a808582660f30bf502b6372c42008cfd3361f80437010625f31c1769
280	docs/releases/7.0.0/whats-new.md 757d004bc0e6b55c485bd98e4d0bc03feec298173879ec52992b8a21f0a35316
281	docs/releases/7.1.0/whats-new.md 03591582ae3ece3d890b47ad6226624180ce4385feb5cf7b088e628f5f2bc243
282	docs/releases/7.2.0/whats-new.md 7255d83c64bba9459e2d16cc505cb6975e9e7f561d0815ad756b8de2c59ed4ee
283	docs/releases/8.0.0/whats-new.md 3f34b0622f828528cb2806fa33b34337185046ae2f642758f63051b2e83d079e
284	docs/method/graveyard/legacy-backlog/cool-ideas/CI-001-mermaid-surface-component.md f857fd606d23f2b3a73ea5abf8aab7029265e6997323c435f88ac5ba68a9fe03
285	docs/method/graveyard/legacy-backlog/cool-ideas/CI-002-deterministic-time-travel-debugger.md 91c073bfc82b2bb7f087d1a146fec53155fc001a0cb64831335b0ab9c92045b1
286	docs/method/graveyard/legacy-backlog/cool-ideas/CI-003-surface-shaders.md 1321d46098709710ef3aacf481edd381234fc50261dc338fbe5c34d638752032
287	docs/method/retro/CI-003-surface-shaders.md 2c16885a7cdab74ebaa1390f5709b9cc138bedc86c97b50bcb8e4ea3b7717391
288	docs/method/retro/DAG-layer-skipping-edges-occluded.md 1b99c2293c794fada62a707423bbf90280883ffa08abeab649e166cad8b43be6
289	docs/method/graveyard/legacy-backlog/DF-002-expand-dogfood-component-family-coverage.md ba7388d201140bcdda63b4ef9162941aa0460ecb99dd98ad600e3e7be614724c
290	docs/method/graveyard/legacy-backlog/v4.5.0/DF-020-deepen-dogfood-story-depth-and-variant-quality.md d9be719af08029f273601becd33d162034615da34da0c9b56a1aa1dea6b3797c
291	docs/method/retro/DF-020-deepen-dogfood-story-depth-and-variant-quality.md 9b70a4778a3833ed30cbc5aace18aeddbbdcbfc187dded73c2d2cc104bdfdd3f
292	docs/method/retro/DF_002-expand-dogfood-component-family-coverage.md 6759d6857a635941a6b93b90156e47ab6e742ce788fd751fa437a34138495b63
293	docs/method/retro/DF_mcp-interactive-documentation-for-all-bijou-components.md b08f6b4ff157ba763c1a62335ba1d431db6e23056770154c89e968793176960f
294	docs/method/graveyard/legacy-backlog/DL-008-promote-guided-flow-block.md 48ec1f9ac21a79906bc3ef8cbacd3e6642ffd0cf7ca97bfc07cba61bfd458793

Chapter	Source and SHA-256
295	docs/method/graveyard/legacy-backlog/v5.0.0/DL-009-formalize-layout-and-viewport-rules.md 4cb4e6dd100a36bf60485dedf4fbfa19673289cf126497e91cd0411f9bc5d06b
296	docs/method/retro/DL-009-formalize-layout-and-viewport-rules.md 3688536d2584ec76828a5f73c93968c215e9b73d04dea359fcbeab7d984943b1
297	docs/method/graveyard/DL-009-strictly-typed-app-examples.md 2e02b3adbe6a54b1e9d55e613e75331deb23553afe7fb81c8f7f24f37d70f2a4
298	docs/method/graveyard/legacy-backlog/cool-ideas/DL-009-strictly-typed-app-examples.md d24b1ccf643935e120a8736bfe0968a0d663f998c5d9eef8024da755af795b06
299	docs/method/retro/DL-010-strictly-typed-app-examples.md 981b9a2e3ce2feb7dc630cf342f4d1e8b0cef7d66b1bc6f7c955603a12a4688c
300	docs/method/retro/DL_008-promote-guided-flow-block.md 597461f39323ea182944a7030aa10b7483e9557016a23f355cd3492343768388
301	docs/method/graveyard/legacy-backlog/DX-003-rationalize-table-apis-and-public-table-types.md 84566f4988502265aae92ff211bb6a264ac8832e2f44ef53feed1d741572d18f
302	docs/method/graveyard/legacy-backlog/DX-004-smooth-surface-and-string-composition-seams.md be238ede52fe9c5d6507f29950f40c70ca669443ba4132cd04300954151509f8
303	docs/method/graveyard/legacy-backlog/DX-005-polish-small-component-and-import-ergonomics.md cbed6d1d9dd7e3861066ea3191bade434110a4d6b983b56e7039feccf3378ec5
304	docs/method/graveyard/legacy-backlog/cool-ideas/DX-006-debug-performance-overlay-component.md 3ce6dcfb2322866ecc4c2b76c57facd26fd008da0cbb7a979bbbe5e0cbb9f1f5
305	docs/method/graveyard/legacy-backlog/bad-code/DX-007-decompose-app-frame-mega-closure.md 6faf74176e08150b87c483766bb6ba7b55f268fa9821629ab3959113b153d976
306	docs/method/retro/DX-007-decompose-app-frame-mega-closure.md fbb7e8bbb983b41fe0b3ec02536808a2f255305625536dc495ed3e972eccc768
307	docs/method/graveyard/legacy-backlog/v4.5.0/DX-007-project-wide-numeric-input-sanitization.md 346162db95138bfae118c3f0307965cb6ef549c2a7b315781dc86727e733a3f1
308	docs/method/retro/DX-007-project-wide-numeric-input-sanitization.md 27d739cc09bf48803feba84e62b49cbb06b35fa88cdcacc052e589cb0be3bb7a
309	docs/method/retro/DX-008-performance-observer-gc-unreliable.md 0ef356a3978ae696d9aea90227d413b660d925c2a6c23c8dfc52518842f51fbb
310	docs/method/graveyard/legacy-backlog/inbox/DX-008-render-pipeline-guide.md 854dfa6f9a194ac4f3b3df5a2602c37494d6264f31afc918dcabbbf73f284f89
311	docs/method/graveyard/legacy-backlog/v4.5.0/DX-008-sanitize-wrapper-for-user-facing-numerics.md 755d658fd5af97054622199e506c56f6459c4487b42883ec5c5089ef17c52524
312	docs/method/retro/DX-008-sanitize-wrapper-for-user-facing-numerics.md 205d00d16330066a6a15c83d6b0973f368fead941c6f06e2a87f19f6b67265df
313	docs/method/graveyard/legacy-backlog/inbox/DX-009-context-auto-init-and-error-messages.md b3ee81de4baf94ac7d7a0dbb99047242238651c87b8048a6599c38d696add5ca
314	docs/method/graveyard/legacy-backlog/cool-ideas/DX-010-typed-subapp-adapter.md 6a690677fc4421589b57b93db6b3ae787f16a4db7a1163b183374f2069773906
315	docs/method/graveyard/legacy-backlog/cool-ideas/DX-011-key-collision-warnings.md 90c4f2bfbdf766f21f2aaea7945dee4d6dcef5cebe407b0bda29a32fd6407d6a
316	docs/method/graveyard/legacy-backlog/cool-ideas/DX-012-render-path-naming-convention.md 2c3b29dba803d1f2893473c1ca4bb34d4ec12f8c813379f43cec71ebd0bed464
317	docs/method/graveyard/legacy-backlog/cool-ideas/DX-013-motion-key-warning.md 31e97816fb03873134f9044c3439a07ca07aebf400b66621c4933ad2d9e20006
318	docs/method/graveyard/legacy-backlog/cool-ideas/DX-014-readme-interactive-example-fix.md 81fc101f2882f533b5650eb4971b033312ddf8e75c92ba63e4280b7044ec57ca

Chapter	Source and SHA-256
319	docs/method/graveyard/legacy-backlog/cool-ideas/DX-015-perf-gradient-as-ci-benchmark.md febc54961901cfaf4adac056d53317dd59b7938ac28cc337383fb536cba8f2c5
320	docs/method/graveyard/DX-016-error-surface-post-mortem.md cc87f04db2c9d461dedf477cdf2749413e129bb3d226e5815e60c2783435d1b6
321	docs/method/graveyard/legacy-backlog/cool-ideas/DX-016-error-surface-post-mortem.md 8b15a26331f6ea2e42f35c3aa6b2c43f788b7c1612fa83ae22b937cb05c8016b
322	docs/method/retro/DX-016-scenario-provenance-tags.md ef5b7c4f6ce3739ef631bd737ac9ce973a8c8d65343658a3719a4910ae964f27
323	docs/method/retro/DX-017-agent-first-bench-output-format.md ef26de2586db318c55bcd3e211031180b09e2a815e415225f1cd13dbeb6d2653
324	docs/method/graveyard/legacy-backlog/up-next/DX-017-one-call-node-host-entry-point.md 5868ad3daee504c12c148403e279ca280e839961412c964df47feee70133efaf
325	docs/method/retro/DX-017-one-call-node-host-entry-point.md 23693ba4e93f9cb203a62a1e4b2822ee8d7be2fc3099b34d204994a51c900da4
326	docs/method/graveyard/legacy-backlog/up-next/DX-018-runtime-test-harness.md 5a38b98051ccefa34484c04541119853d9d372298da1eb1cf84a8678d2f3b72f
327	docs/method/retro/DX-018-runtime-test-harness.md def0c37183a8dfa25bded9f07953eab725faf92c4f539ccf4233a6a657e437e5
328	docs/method/graveyard/legacy-backlog/cool-ideas/DX-019-third-party-theme-reactivity-hook.md a6d7d2857ec0c15419351128d8a113d90ef00edda0a532261280a4b9b0f30306
329	docs/method/retro/DX-019-third-party-theme-reactivity-hook.md 066a45ed622f1c54e58310f837bed4af6850fcc45b3a5c4fa2155d9562757e04
330	docs/method/graveyard/legacy-backlog/up-next/DX-020-layout-localization-pipeline-guide.md f5f2c5a0c0c6fe0ebc8cb49d11bf311084f98876282ac7b969059917b3c73576
331	docs/method/retro/DX-020-layout-localization-pipeline-guide.md cbaba769bf70d0b6664eff778a87b668915f62133b0314614aa3f852b9e4975a
332	docs/method/retro/DX-021-error-surface-post-mortem.md cb517b2349b27858c98e1807e7bc0d6cfc427607b4e0c927148e0129e9087c81
333	docs/method/retro/DX-028-align-front-door-docs-with-repo-truth.md 4f39aaeac2834f58ab899cd4f596dfb6944b0f4855ab254281a8daa5a4148050
334	docs/method/retro/DX_003-rationalize-table-apis-and-public-table-types.md 901aa17922615bf37b2953eb49b815addb6f0248898f5f2569d3ac710f48592f
335	docs/method/retro/DX_004-smooth-surface-and-string-composition-seams.md 223398aa890b7c6dc322967217721d0e84aaa2a28bb9ee1f649658d42228cb44
336	docs/method/retro/DX_005-polish-small-component-and-import-ergonomics.md 7b489fcd99efa1c023a1d6aed127417711c7d20f12ba2a8c5766c91ddf91d4dd
337	docs/method/retro/DX_006-debug-performance-overlay-component.md 7365bd84ee5a35bd5e48cac6a21078bb6e0d1e75d522eaafdac929013edf9c14
338	docs/method/graveyard/legacy-backlog/v5.0.0/DX_007-decompose-app-frame-mega-closure.md 6faf74176e08150b87c483766bb6ba7b55f268fa9821629ab3959113b153d976
339	docs/method/retro/DX_008-render-pipeline-guide.md 0f07463e37ff6f057ebd506e5f2be5bba6d3a0260b4301bcc9ad7f2b5182ef0f
340	docs/method/retro/DX_009-context-auto-init-and-error-messages.md fe38aa992d5801bc203b09f16d55a333152bc796568b754b66a950cc34fa7b0a
341	docs/method/retro/DX_010-typed-subapp-adapter.md 30275a83f0cae15eef31cfaccd33d57fa2d9b0e82789ce9a270339d4696dfe1d
342	docs/method/retro/DX_011-key-collision-warnings.md 706dddec3d29bc7777ba2994448f997ba862b5bcd542b83b3cc4459ea435dfbb
343	docs/method/retro/DX_012-render-path-naming-convention.md 028c243ff5efe5a982b02ca351daa49c942b8b8d8aeb0b15ba22fb1fe3d46352
344	docs/method/retro/DX_013-motion-key-warning.md 86989f9516b406205791c667a595316769edb69d0566dd6edfd472289d4ee67b
345	docs/method/retro/DX_014-readme-interactive-example-fix.md da873637fbb41d31653204dfb72ae069b5cf21ef1bf625a73678e922d641b0e8
346	docs/method/retro/DX_015-perf-gradient-as-ci-benchmark.md 234bfb383425d0cbd0f8268f8c454a1434f625a3c6e3f4d96b559e550cd2a993

Chapter	Source and SHA-256
347	docs/method/graveyard/legacy-backlog/HT-001-notification-center-polish-and-discoverability.md c49311192c1101edea0a5ae461cd89927b72cdc437338b766d3ef0c3a6a40d92
348	docs/method/graveyard/legacy-backlog/HT-005-promote-page-provided-layer-registry-and-shell-control-projection.md 868e9cb2bdccd05a029486a2f58c4dbb80b44f02ea8ef58efd9822f00c27cdee
349	docs/method/retro/HT-006-page-level-notification-push.md 4b86008a9644f7f6c21fe52e6827630bcc09bdb557e7e52720eb66cd67ed63c4
350	docs/method/retro/HT_001-notification-center-polish-and-discoverability.md 867e5fd05a31042e85fe0c9dff3d5dbffb07eded3335a2b169d8b50b80c3f190
351	docs/method/retro/HT_005-promote-page-provided-layer-registry-and-shell-control-projection.md 1ffaa0b078c71e63e96dfd9172d7e7562ad4e87db49a44fd41630a2c11810861
352	docs/method/graveyard/legacy-backlog/v5.0.0/HT_upstream-edge-overlay-scrollbars-and-bcss-overflow-handling.md 69735c6e2e465e73e2906f7ee6a710f644f79d58ff2d801ef6d232e3fc41e60e
353	docs/method/retro/HT_upstream-edge-overlay-scrollbars-and-bcss-overflow-handling.md 63f13ba08c4e82577e966d45d894eeb833affb82c081901201a6f18b2cd3401c
354	docs/method/graveyard/legacy-backlog/LX-007-service-oriented-localization-adapters.md be9f87d7f5e1cf183b0f4097efc808dc0e8b1f1bcfc3810ce3fdc5030cfba512
355	docs/method/graveyard/legacy-backlog/LX-009-localize-shell-help-notification-and-directional-surfaces.md 7807b9ce86937627f0b881a584bf44b93438f1a6ea0e30c13b2f4313c2c935c1
356	docs/method/graveyard/legacy-backlog/v4.5.0/LX-010-built-in-i18n-catalog-loader.md 027d0980353eb27fb8080269d94bcfd7d7146a4fe03a557ea1535052d675a551
357	docs/method/retro/LX-010-built-in-i18n-catalog-loader.md 85464ec77d4b3a8189aa47bab6f719d3b837a024c11b898b18c9adbd4d797a2e
358	docs/method/retro/LX-012-dogfood-i18n-debt-inventory.md 3d41802e3e728a55c22ac6560437980ec02898e642595f9e4c3e587354f41c7d
359	docs/method/retro/LX_007-service-oriented-localization-adapters.md b8c596ff8833272a93037b153e43d1f503936aa88d7ec62f56c9ee7182963aff
360	docs/method/retro/LX_009-localize-shell-help-notification-and-directional-surfaces.md 6eeb5b3cd6e37ca881b6a13c3ea6edeb0cc2db941c76d7b53f6620f2cd7043ae
361	docs/method/graveyard/legacy-backlog/inbox/RE-011-pipeline-per-frame-allocation.md bf28e0a2f396024f8f57294c1b3f83c9e7567b3751377133cff177f780d30486
362	docs/method/graveyard/legacy-backlog/cool-ideas/RE-012-pipeline-observability-hooks.md f7b84deac941d9a3bfb01227cf6a346fa723dc746cc73fcb9a8107b06491d6f4
363	docs/method/graveyard/legacy-backlog/cool-ideas/RE-013-memoize-workspace-layout-tree.md 423f23f7694929f7984c29b5baf27cf8985ec2d575027251ba1c651ad40ca9a4
364	docs/method/graveyard/legacy-backlog/cool-ideas/RE-014-buffer-holds-facts-doctrine.md 4af8eebc16d0890fc34ca2ada42b5cec406a64af6bdc3ec3aced605c9e38cade
365	docs/method/retro/RE-017-frame-does-not-fill-surface-background.md a3553e7dab3ce33655155da0502883054eb17fc39423aebc9fc64b568abebed8
366	docs/method/graveyard/RE-018-hex-color-string-as-canonical-fg-bg.md d28fa274e42163fa523b260a4c15d15ce7d701c3df8cf2d74bce7d553c421b20
367	docs/method/retro/RE-019-dirtyWords-serves-lazy-cache-not-render-dirty.md 3446e6a60bb0319635affa9dee8bada157c8c19026e78cc3909d4c91d70bb594
368	docs/method/retro/RE-019-theme-token-color-cache.md 126f87bbb5461f9dfbd5625689c839b6110e32bce2253c9eb3667737ec4c9a23
369	docs/method/graveyard/legacy-backlog/v5.0.0/RE-020-typed-color-representation.md a3c00c26ea1c3ed1b0183ff80072cbd67ec41ede541b29a2f8a8d773c5fd0a04
370	docs/method/retro/RE-020-typed-color-representation.md b122d93a73ecf9b43177c277731a20ca22fc8008ad25f78ec0b26852d0c25ebb

Chapter	Source and SHA-256
371	docs/method/graveyard/legacy-backlog/v5.0.0/RE-021-frame-owns-the-pump.md e582095ba551d708c1a49b672db773c1bdff5c1590ca5a105cfcc601be69c2bc
372	docs/method/retro/RE-021-frame-owns-the-pump.md 54815d17a6455304cf511db68035a712d0999d7eb4fc994d53049b31e68e56e0
373	docs/method/graveyard/legacy-backlog/v4.5.0/RE-022-public-packed-surface-type-guard.md 21f0438e15aa91c218423b521cb82a170347b8d1c6339b36393931df78dbc2dd
374	docs/method/retro/RE-022-public-packed-surface-type-guard.md e2c2a8894b624cee9312ad280149949fac83cfaaee1a944609aea3f6141f44e7
375	docs/method/graveyard/legacy-backlog/bad-code/RE-022-unified-layout-calculation-engine.md 0b57d32e211919db1a2a0f82deac8b39bd719091e71f5de971dfd18636e03416
376	docs/method/retro/RE-022-unified-layout-calculation-engine.md c94c93f7d85080f5a9e21b4609532c5f79aa05b4a02b777f1ba125199ad876f5
377	docs/method/graveyard/legacy-backlog/bad-code/RE-023-stabilize-render-scheduling.md d5c45546493053206853 4dad73d623884261761149df242ecc5d0e4e7985b7c8
378	docs/method/retro/RE-023-stabilize-render-scheduling.md 3c4426f82173680b84b806e2e68251d1ae2a499d5c34a3c9bad0890c61c34bf1
379	docs/method/graveyard/legacy-backlog/asap/RE-024-terminal-content-sanitization-boundary.md 187cf87443bb61894283eeed3e56cb27ee30efc651d72a1958a8392adf77e380
380	docs/method/retro/RE-024-terminal-content-sanitization-boundary.md e86b46310276b067e4b1ad3164c40b63dde437fb0f626f10f7ff6308614e40a0
381	docs/method/graveyard/legacy-backlog/asap/RE-025-scoped-node-io-adapter.md f88a1a96594e031b24335b17b7fc4b20bad6770ba5b2dab89713dc04ebc02342
382	docs/method/retro/RE-025-scoped-node-io-adapter.md efed58673b0542268916ee83f93fbad67a9113341f317f3796e404ed0a5c38fa
383	docs/method/retro/RE-026-drain-runtime-commands-before-shutdown.md cef0c0b87e770afa828b000abe840a4e3117c15a46471fa040ad80f762c64c21
384	docs/method/graveyard/README.md 83a80207d7639e1a667c2b940e079bf76e4103dcc6db4001c32cc72b92d0a0c1
385	docs/method/graveyard/legacy-backlog/README.md f399231410cd424a8b8d12a52c8a23a8fb381522379492a80d17d8fea5e9d4d9
386	docs/method/graveyard/legacy-backlog/asap/README.md 55021148e5d9e54710dfbb2478fff77908bab42fc8808aa261d1b3f1b261b1fc
387	docs/method/graveyard/legacy-backlog/bad-code/README.md c5ac23b0314ef27a153fbef2871e47fc60eec156135ea4779cfc96fdef3fd25c
388	docs/method/graveyard/legacy-backlog/cool-ideas/README.md 8e9fbc4492576179fd42e8e936fa98dfecc47880b92a0b6d680962922c9c50a8
389	docs/method/graveyard/legacy-backlog/inbox/README.md 93b820e21f395f747e2a51c2f0d4c3b31330c6aa61de646513c2737a04d549ce
390	docs/method/graveyard/legacy-backlog/up-next/README.md 151a123399d3bb66f2e1dbd7d080d898276a70e5dc1ada10fabda2f26d6add57
391	docs/method/graveyard/legacy-backlog/v4.5.0/README.md 89d7490f131a9f8332d09c2bf15cfefc000a944d959ada6a4db56bc972d64b81
392	docs/method/graveyard/legacy-backlog/v5.0.0/README.md 183255f21c0bf96848c7377a799d8f40b3312138d9e935ba51aac9a075ff40c8
393	docs/method/retro/README.md a0e8978205f371462d404de521b39ff8723899931897292b51eab27255cbfbf9
394	docs/retro/README.md 73559dc061671a5cc6b5a1a7045ed6fcb65fbb180afc454e176c0f4c2ab28e4b
395	docs/method/retro/RE_007-migrate-framed-shell-onto-runtime-engine-seams.md 0fa3aba163d5051d515dfc893f82deea3c30b577b5cb307468aedd0f808bab69
396	docs/method/retro/RE_011-pipeline-per-frame-allocation.md fab02b760fdd4d256f5b5ccd150dd9571bdd84ec0568db80cdf9c636bbd5dc23
397	docs/method/retro/RE_012-pipeline-observability-hooks.md 3e9954bbc314f534d8200a1de83a45559925f6c06fa929e541e02a33b1e6fc04
398	docs/method/retro/RE_013-memoize-workspace-layout-tree.md cf2b72b086c6bbbba0dd3a82fcba6a6bdeaf7f1c2be9f083ade40fb636b856ea
399	docs/method/retro/RE_014-buffer-holds-facts-doctrine.md 8b4ce6638ac561b563f1fa0c52dc798c12cbaadb33ef5f97c4e89d18f2d4b590

Chapter	Source and SHA-256
400	docs/method/retro/RE_016-dag-cycle-graceful-handling.md 5d23fa4684b8a43e24957de40fb356e1e9d585aff74c901b93723952ab6fe70c
401	docs/method/retro/RE_016-grapheme-width-miscounts-emoji-like-icons.md e089ced10b0cadea2a70f6ad09f074ba03090a83df6cab839a8dea7274cf68c8
402	docs/method/retro/RE_writeport-byte-level-api-for-zero-alloc-ansi-output.md eca403c003608053057c14c3f4a9a69d1b0618702aa2b61c651b3d5fd4338038
403	docs/method/graveyard/legacy-backlog/WF-002-migrate-legacy-planning-artifacts.md 92e48b2299193296569eead1016e06b780d6d442ab727f8ac77b5bc2a317cb4b
404	docs/method/graveyard/legacy-backlog/bad-code/WF-007-align-local-validation-with-ci-typecheck-test-gate.md 41469470f6e5293a39a04da8236eacffdbf004e8749f33788ebfbad2d6a4be12
405	docs/method/graveyard/WF-008-method-backlog-helper-generates-malformed-note-filenames.md dd245056884f2e5669cf87cf9fbfb982d37c694e219550ba3bed13a22317bb87
406	docs/method/retro/WF-122-day-zero-audit-fixes.md 5a922450532ad0a91d88959a7c93ad293a6c3bda1d1e4c3944aa6e482bef94ba
407	docs/method/retro/WF_002-migrate-legacy-planning-artifacts.md 4315997840a4771f85ec6b0aac724a028afc7a667ed250f32e05a9bfbe86b64a
408	docs/method/retro/WF_007-align-local-validation-with-ci-typecheck-test-gate.md ab01d95760ea7c38b14817743f3204c5ec14a8d07e6f031460e3c035b36f5d0a
409	docs/method/feedback/think-bg-fill-regression-v4.4.0.md a095f6815028d1534a8820480f118f78cf68347c9886692e649ed1783e47c37e
410	docs/audits/2026-04-05-codebase-health-assessment.md 3126f1618df06003f6221fceb4d3b73c2a48cefbcfc066bd1b5bfe2a58d6d008
411	docs/audits/2026-04-05-documentation-audit.md 01dbca558cc4ec33a64894c09d5278163ca819a55e6e6e79a0ae24a41f8e43be
412	docs/audit/2026-04-11_code-quality.md 20a9407c718ea98f79737a1aebfabbc03df15fb2f77c9d2eb6c23e709fb872c3
413	docs/audit/2026-04-11_documentation-quality.md f9151b74dda0c7b23d1f96ad77a566942b080cd1302c08084255a53818855aa3
414	docs/audit/2026-04-11_ship-readiness.md e1e986b029a74495ee22a0a84cf15341bc437c4aacb84f5a72744f6177f08f4e
415	docs/audit/2026-04-14_code-quality.md 9492cd25b5fb28e2df0a92912733470426605528140 66e8d14e30f80b6452ac7
416	docs/audit/2026-04-14_documentation-quality.md 59fa29d649f6f9c2a1f548b915ccf7b61cd8ef0cd7151c5ce527f6a478e1a844
417	docs/audit/2026-04-14_ship-readiness.md 65c9148eb8ca9dc42669551ef23f5639daf136fc02eb4faa639f85a1f47f85ce
418	docs/GRAVEYARD.md cf583e60ccca396b69428286021ea44f8b7ab152c726e0885f60f81dd6d48158
419	docs/graveyard/I-0d-render-pipeline-instrumentation.md 85d044e04f62815e57441925292cc7d40829ce97f45f45a3347a2e84b9a635de
420	docs/graveyard/I-3-custom-baseline-format.md 9375b6f41dc3d4a964617ec6e61a9661fb311c5f0530dfb9193d78e6bedb1235
421	docs/perf/RE-017-byte-pipeline.md deb789a076eeca44ab048a44053ddd06479f69d8090874b6239107308dc7ba9c
422	docs/archive/README.md da4a14da42be1b13cf61ad65268ae5e215122f1fd12acc5f09e4ff023af8db01
423	docs/graveyard/README.md 6db029f2e97b8618ca8560771fddf34b23fb95aee3094acd19874dce8b5db765
424	docs/archive/component-system-audit.md 7634ef8e9e9dbd61943845970f653575d82b13799f0d0887acc42e62f8ba958b
425	docs/archive/dag-exploration.md b6121c7dbfcec04fa394e04b41ea9554ddb607edf3b8d6f6c2df7582182eaba1
426	docs/requests/xyph-title.md e6b2ee035f474e29a8abcf361d71ed85b6a5f9aaf7ab2e7e39e5eab70910f49f